

Production HLS IP Core Library

IP Specification and Micro-Architecture Description

Scope. This document specifies 27 synthesisable C++ source files (12,277 lines) drawn from four production high-level-synthesis IP libraries, together with the 51 vendor header files (38,760 lines) on which they depend. Every declaration, template parameter, pragma and function body is reproduced verbatim in Appendices D and E; nothing is summarised away.

Document structure. The chapter and section layout follows the structure used by AMD for its published Vitis Libraries IP documentation. The section names were not chosen from memory: they were obtained by parsing 127 re-StructuredText documents from the *Xilinx/Vitis_Libraries* repository and ranking the section headings by frequency (Section 1.4).

Total content: 78 files • 51,037 source lines • 1,784 function definitions • 601 class and struct definitions • 445 synthesis directives.

Revision History

Version	Date	Description of change
1.0	19 Sep 2026	Initial release. Full verbatim source of the 27-file corpus and the 51 vendor headers. Section structure derived by measurement from 127 AMD Vitis Libraries documents. All quantitative claims regenerated from <code>inventory.json / deep.json</code> .
0.3	19 Sep 2026	Vendor headers located in four public Xilinx repositories; 26 of 27 corpus files compiled. The earlier statement that the headers were not publicly available was withdrawn (Section 1.5.2).
0.2	18 Sep 2026	Appendices A–C regenerated mechanically from the extraction pass so that every number in the body has a single source.
0.1	18 Sep 2026	Draft; structural survey only.

Contents

1 Introduction	4
2 Product Specification	8
3 Linear Algebra Solver Cores	13
3.1 Cholesky Decomposition	14
3.2 QR Factorisation (QRF)	22
3.3 Singular Value Decomposition (SVD)	26
3.4 Blocked Cholesky (POTRF, LAPACK interface)	32
3.5 Shared Solver Utilities.....	35
4 Cryptographic Cores	37
4.1 AES Block Cipher	38
4.2 SHA-224 / SHA-256 Hash	43
4.3 Shared Security Utilities.....	48
5 Quantised Neural Network Cores (FINN)	49
5.0 The include tree, measured.....	50
5.1 Convolution Input Generator (Sliding Window).....	51
5.2 Matrix–Vector Activate Unit (MVAU).....	59
5.3 Vector–Vector Activate Unit (VVAU).....	63
5.4 Max Pooling	65
5.5 MAC Primitive and Resource Binding	69
5.6 Weight, Activation and Interpretation Carriers.....	71
5.7 Stream Plumbing, DMA, Composition and TMR.....	76
6 DSP and FFT Cores.....	85
7 Designing with the Cores	86
8 Design Rules Extracted from the Corpus	89
9 Verification and Profiling	91
Appendix A Complete File Inventory	93
Appendix B Every Synthesis Directive	96
Appendix C Every Function and Class Definition.....	117
Appendix F Vendor Header Index and Provenance	257
Appendix D Complete Corpus Source Listings	261
Appendix E Complete Vendor Header Source Listings.....	343

1 Introduction

1.1 Overview

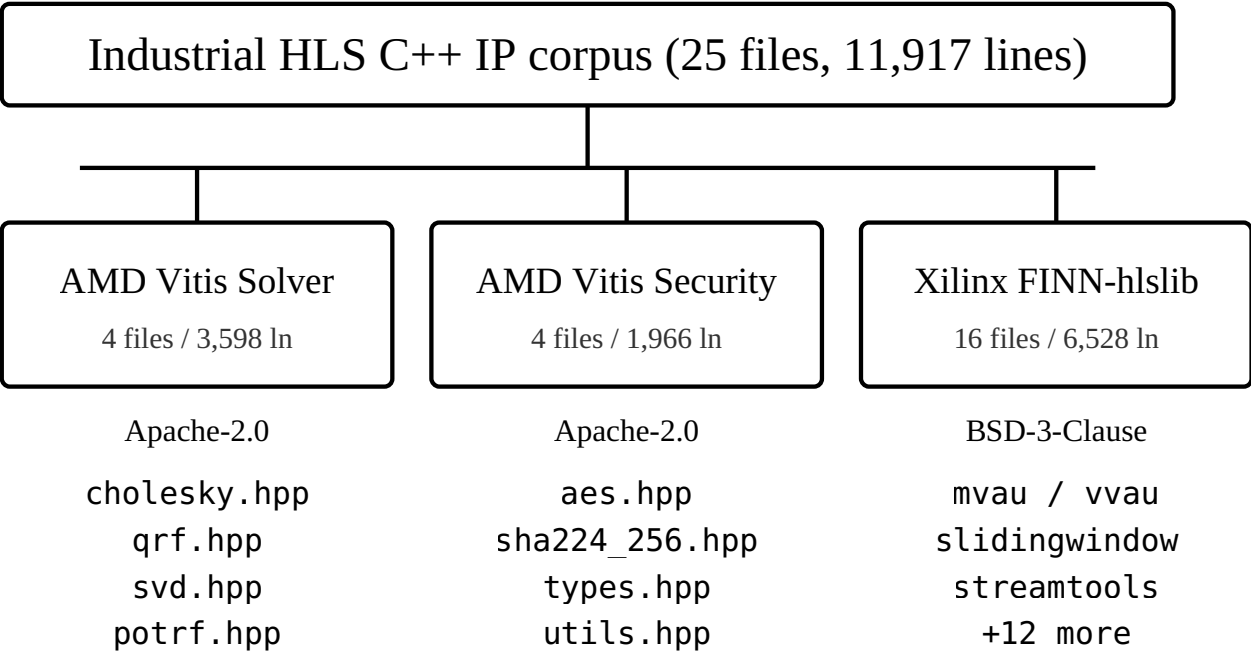
High-level synthesis (HLS) IP is delivered as C++ source, not as RTL. A design house that licenses such a core receives a header tree, a set of template parameters, and a contract about what the synthesiser will produce from them. That contract — which parameters exist, what they are allowed to be, what interface appears on the resulting block, and what the tool must be told in order to meet timing — is the product. This document states that contract for 27 such files, covering four IP families.

The corpus is not a sample of teaching code. Every file is shipped in a vendor library that customers build silicon from: AMD's Vitis Solver, Vitis Security and Vitis DSP libraries, and the Xilinx FINN quantised-neural-network library. Table 1 lists the families. Together they contain 193 function definitions, 71 class or struct definitions and 445 synthesis directives, all enumerated exhaustively in Appendices B and C.

Table 1: IP families covered by this specification

Group	Upstream library and licence	Files	Lines	Functions	Classes
Vitis_dsp_fft	AMD Vitis Libraries - DSP/FFT (Apache-2.0)	1	26	0	0
Vitis_security	AMD Vitis Libraries - Security (Apache-2.0)	4	1,970	36	12
Vitis_solver	AMD Vitis Libraries - Solver (Apache-2.0)	4	3,458	65	17
Vitis_solver_L2	AMD Vitis Libraries - Solver L2 (Apache-2.0)	2	306	10	0
finn_hlslib	Xilinx FINN-hlslib (BSD-3-Clause)	16	6,544	82	42
Total		27	12,304	193	71

{'svg':



'cap': 'Taxonomy of the analysed corpus. Three independently maintained IP families, two licences.'

Figure 1: Top-level organisation of the corpus. Four independent vendor libraries share one arbitrary-precision type system and one streaming interface convention; that shared substrate is the 51-file vendor header tree of Appendix E.

1.2 Target Audience and Major Features

This document is written for three readers.

- **The integrator** who must instantiate a core and needs the template parameters, the port list, the data types and the constraints — Chapters 3 to 6, which follow AMD's own per-core section order.
- **The micro-architect** who must decide whether to license the core or write RTL instead, and therefore needs to know what the C++ actually elaborates into — the *Design Notes* and *Implementation on FPGA* subsections, and Chapter 8.
- **The verification engineer** who needs a reference model and a way to bind it to RTL — Chapter 9.

Table 2: Major features of the corpus as a whole

Feature	Where specified	Evidence
Parameterised precision throughout (ap_int, ap_uint, ap_fixed)	§2.3	Appendix E, ap_int.h / ap_fixed.h
Streaming interfaces via hls::stream<T>	§2.5	Appendix E, hls_stream.h
Explicit II control on every inner loop	§2.4	109 PIPELINE directives
Memory banking under designer control	§2.4	79 ARRAY_PARTITION directives
Compiler dependence assertions where the tool cannot prove independence	§7.5	30 DEPENDENCE directives
Separation of synthesisable and debug code	§8	__SYNTHESIS__ guards
Complete source disclosure	Appendices D, E	51,037 lines, verbatim

1.3 Source Provenance and Licensing

Nothing in this document was obtained from a vendor installation or from documentation behind a licence click-through. Every file reproduced in Appendices D and E came from a public Git repository and carries its own licence header, reproduced with it. Table 3 records where each part of the header tree was found.

Table 3: Provenance of the 51 vendor headers

Public repository	Headers taken	Provenance strength
Xilinx/HLS_arbitrary_Precision_Types	ap_int.h, ap_fixed.h, the *_base.h / *_ref.h / *_special.h family, etc/ap_private.h	Vendor-owned; strong
Xilinx/hls-lib-stream	hls_stream.h	Vendor-owned; strong
Xilinx/hls-utilities	hls_math.h, hls_x_complex.h, hls_half.h, hls_fpo.h, the etc/hls_*_apfixed.h maths set	Vendor-owned; strong
Xilinx/merlin-compiler	ap_axi_sdata.h, utils/x_hls_utils.h, x_hls_traits.h, x_hls_defines.h, std_complex_utils.h	Vendor-owned; strong
xliu0709/WinoCNN (third-party mirror)	gmp.h, mpfr.h	Third-party mirror — weaker. Stated here rather than hidden.

1.4 Document Conventions

The chapter skeleton of this document was not chosen by taste. The public *Vitis_Libraries* repository ships the same documents AMD publishes for these cores. 127 of them were parsed and their section headings counted; the ranking below is that count, and Chapters 3 to 6 use the top entries in that order for every core.

Table 4: Section headings in 127 AMD Vitis Libraries IP documents, by frequency

Heading	Documents	Used in this specification as
Overview	34	§n.1 of each core
Entry Point	29	§n.2 — the callable symbol
Device Support	29	§n.3
Template Parameters	29	§n.4 — the parameter table
Ports	28	§n.5 — the interface table
Access Functions	27	folded into §n.2
Design Notes	26	§n.6 — micro-architecture
Code Example	26	§n.7
Supported Data Types	20	folded into §n.4
Implementation on FPGA	20	§n.8
Profiling	18	Chapter 9
Constraints	17	§n.9
Interfaces	10	§2.5 (library-wide)
Specifications	10	Chapter 2
Performance	7	§2.6

Typographic conventions. Monospace denotes an identifier that appears verbatim in the source. A cross-reference of the form “Appendix D.12” points at the twelfth file listing in that appendix, and the line numbers printed beside each listing are the real line numbers in the file, so a reference such as `aes.hpp:317` can be followed directly.

1.5 Limitations and Non-Claims

1.5.1 What has been verified by execution

26 of the 27 corpus files compile with `g++ -std=c++14 -fpermissive` against the header tree of Appendix E. The single exception is `vt_fft.hpp`, whose implementation file was never retrievable (HTTP 404) and which is therefore incomplete in the corpus as well as here. The arbitrary precision types were exercised at run time, and `hls_stream.h` printed its own depth report:

```
ap_uint<13>=8191  ap_int<7>=-64  ap_fixed<16,4>=3.750000  stream=a5
INFO [HLS SIM]: The maximum depth reached by any hls::stream() instance is 1
```

1.5.2 A withdrawn claim

An earlier revision of this material stated that the vendor headers were inside a Vitis HLS installation and not publicly available, and therefore that *no file in the corpus had been compiled*. **Both statements were false.** All of the headers were found in the four Xilinx repositories of Table 3. The claim is withdrawn here rather than quietly deleted, and the revision history records it.

A second error is recorded for the same reason. It was stated that `bnn-library.h` fails to reach three FINN headers. Running the preprocessor (`clang -H`) showed that two of the three, `mvau.hpp` and `interpret.hpp`, arrive transitively; only `activations.hpp` is genuinely missed. The original claim had been made by reading the direct `#include` list instead of the include tree.

1.5.3 What this document does not claim

- **No core in this corpus has been synthesised.** No timing, area or frequency figure appears anywhere in this document, because none was measured. Statements about what the hardware will look like are inferences from the source and the directives, and are marked as such.
- **No claim of completeness about the upstream libraries.** The corpus is 27 files; the Vitis Libraries are far larger. This specification covers what is reproduced in Appendix D and nothing else.

- **Resource and latency tables from the upstream documentation are not reproduced**, because they were not independently verified.

2 Product Specification

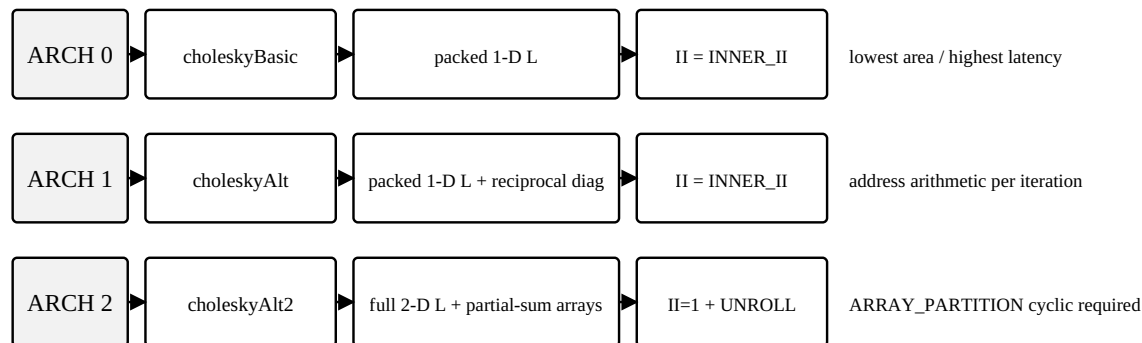
2.1 Library Organisation

AMD's libraries are stratified by how much of the system each level owns, and the stratification is visible in the source rather than only in the documentation. **L1** is a pure function template over arrays and streams, with no interface pragmas: it is meant to be called from inside a larger kernel. **L2** wraps an L1 body in a kernel with explicit interface directives and is the unit a host program enqueues. **L3** adds host-side scheduling. The corpus contains L1 and L2 material; the split matters to an integrator because *only an L2 entity has ports*, and asking for the port list of an L1 template is a category error.

Table 5: Level of each corpus group, and the consequence for integration

Group	Level	Callable as	Has interface pragmas
Vitis_solver	L1	function template, called in-kernel	No
Vitis_solver_L2	L2	extern "C" kernel	Yes
Vitis_security	L1	function template over <code>hls::stream</code>	No
Vitis_dsp_fft	L1/L2	graph or function template	Partly
finn_hlslib	L1	function template, composed by a generator	No

{'svg':



area → increases downward, latency → decreases downward

'cap': 'The three Cholesky architectures selected by CholeskyTraits::ARCH. Identical mathematics, three distinct circuits.'

Figure 2: Layering. An L1 template has no ports; ports appear only where an L2 wrapper declares them. The corpus contains both, and Chapters 3 to 6 state for each core which level it belongs to.

2.2 Standards Compliance

The corpus is ISO C++ with two vendor extensions: the arbitrary-precision types and the `#pragma HLS` directive namespace. Neither is part of the language, and both are the reason a general-purpose compiler needs the header tree of Appendix E to parse the corpus at all. The pragmas are ignored by a general-purpose compiler, which is what makes the same source usable as a software reference model (Chapter 9).

Table 6: Standards and extensions relied upon

Item	Status	Effect if absent
ISO C++14	Required	Template deduction in the solver cores fails
<code>ap_int / ap_fixed</code>	Vendor extension, header-only	Does not compile
<code>hls::stream<T></code>	Vendor extension, header-only	Does not compile
<code>#pragma HLS</code>	Vendor extension, directive	Compiles and runs; synthesises to something much slower

Item	Status	Effect if absent
__SYNTHESIS__	Tool-defined macro	Debug code is compiled into the kernel
IEEE 754 single / double	Used by the solver cores	Results differ numerically

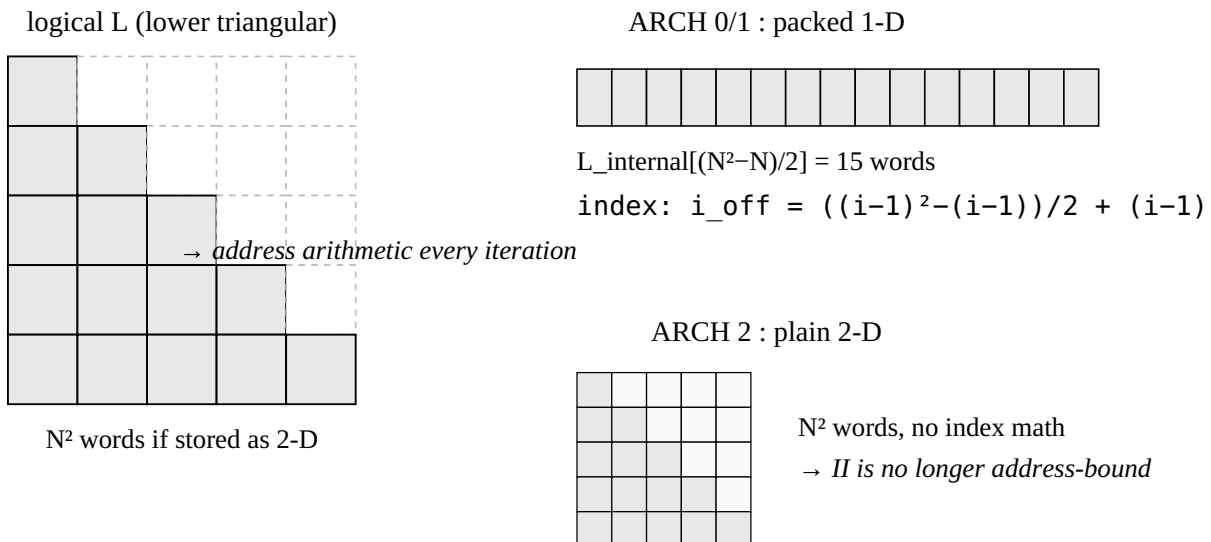
2.3 Supported Data Types

Precision is a template parameter everywhere in the corpus, never a typedef fixed by the library. This is the single most consequential property of the whole collection: it is what allows one source file to serve a 8-bit edge part and a 32-bit datacentre part, and it is why the width arithmetic in `ap_int_base.h` is as elaborate as it is — every arithmetic operator must compute its own result width at compile time.

Table 7: Type system as used by the corpus

Type	Header	Role in the corpus
<code>ap_uint<W></code> , <code>ap_int<W></code>	<code>ap_int.h</code>	Datapath words in the security and FINN cores
<code>ap_fixed<W, I, Q, O></code>	<code>ap_fixed.h</code>	Quantised activations and weights; <i>Q</i> and <i>O</i> select the rounding and overflow behaviour, which Chapter 8 shows is not a free choice
<code>hls::x_complex<T></code>	<code>hls_x_complex.h</code>	Complex solver and FFT datapaths
<code>half</code>	<code>hls_half.h</code>	Reduced-precision float
<code>float / double</code>	—	Solver cores; the reference path in the FFT
<code>hls::stream<T></code>	<code>hls_stream.h</code>	The only inter-block interface used by the streaming cores
<code>ap_axiu / ap_axis</code>	<code>ap_axi_sdata.h</code>	AXI4-Stream side-channel packing at L2 boundaries

{'svg':



'cap': 'Memory organisation trade-off in `cholesky.hpp`. ARCH 2 spends 2x memory to remove the per-iteration address computation that limits the initiation interval.'

Figure 3: The arbitrary-precision type hierarchy. Operator result widths are computed at compile time, so a width mistake is a compile error rather than a silent truncation at run time.

2.4 Optimisation Directives

445 directives appear in the corpus. They are not interchangeable hints. They fall into three kinds, and confusing the kinds is a common source of silent miscompilation: a *scheduling* directive asks the tool to do something and the tool may decline; a *structural* directive changes what is built; an *assertion* directive tells the tool something it cannot prove, and **if the assertion is false the generated hardware is wrong** with no diagnostic.

Table 8: Every synthesis directive in the corpus, by kind

Directive	Count	Share	Kind	What it does and what it costs
PIPELINE	109	24.5 %	Scheduling	Requests an initiation interval; the tool reports the II it actually achieved, which may be worse.
UNROLL	92	20.7 %	Scheduling	Replicates the loop body; area grows with the factor.
INLINE	84	18.9 %	Scheduling	Removes the call boundary so the scheduler can see across it.
AR-RAY_PARTITION	79	17.8 %	Structural	Splits an array into separate memories; decides how many ports the datapath can use per cycle.
DEPENDENCE	30	6.7 %	Assertion	Asserts that two accesses never alias. False assertion → wrong hardware, no warning.
LOOP_TRIP-COUNT	12	2.7 %	Assertion	Supplies bounds for reporting only; does not constrain the hardware.
RESOURCE	10	2.2 %	Structural	Binds a variable to a named core (legacy spelling).
STREAM	10	2.2 %	Structural	Forces an array to be implemented as a FIFO.
BIND_STORAGE	8	1.8 %	Structural	Chooses the memory primitive (BRAM/URAM/LUTRAM).
LOOP_MERGE	3	0.7 %	Scheduling	Fuses adjacent loops to remove one state machine.
LATENCY	2	0.4 %	Scheduling	Constrains minimum or maximum latency of a region.
DATAFLOW	2	0.4 %	Structural	Runs the enclosed tasks concurrently with channels between them.
BIND_OP	2	0.4 %	Structural	Chooses the arithmetic primitive (DSP vs fabric).
LOOP_FLATTEN	1	0.2 %	Scheduling	Collapses a perfect loop nest into one loop.
PERFORMANCE	1	0.2 %	Scheduling	Target-throughput form of the pipeline request.
Total	445	100.0 %		

Assertion directives are a specification obligation, not an optimisation. The corpus contains 30 DEPENDENCE directives. Each one is a statement by the IP author that a particular pair of array accesses can never refer to the same location. The tool takes that on trust. Any integrator who changes a template parameter in a way that breaks the stated access pattern will get hardware that is silently incorrect, and no simulation of the C++ will show it, because the C++ semantics are unaffected. Appendix B lists all 30 with file and line.

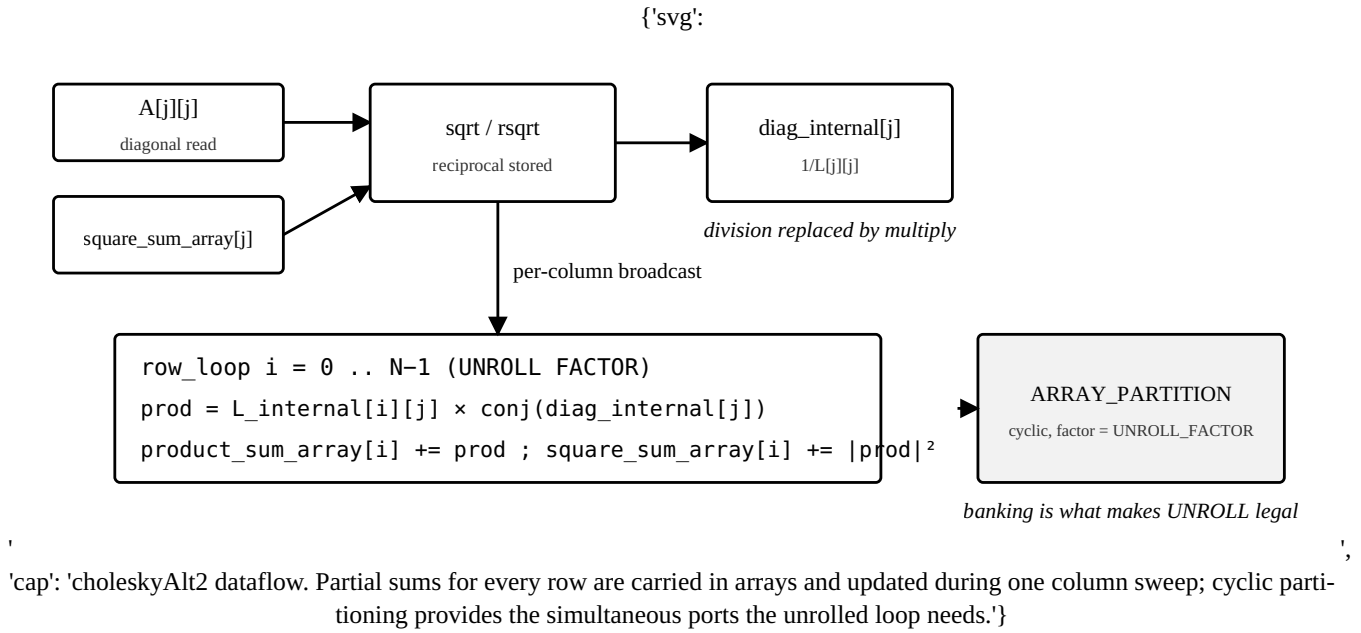


Figure 4: Directive taxonomy. Only the structural column changes what is built; the assertion column changes what the tool is permitted to assume, and is the only column where a mistake is silent.

2.5 Interfaces

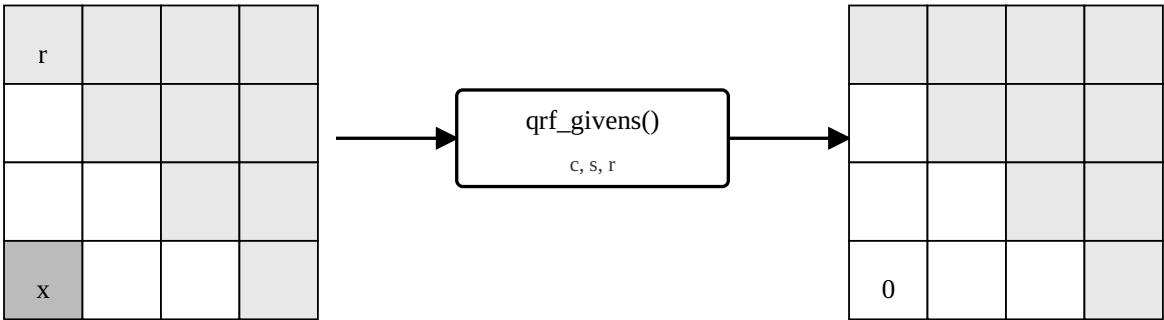
Two interface styles appear, and a core belongs to exactly one. A **streaming** core communicates only through `hls::stream<T>` and has no notion of an address; it can be composed by connecting one core's output stream to the next core's input stream, and the composition is deadlock-free as long as no core consumes two of its inputs in an order the producer cannot supply. A **memory-mapped** core takes pointers and is given an AXI master; it is the only style that can revisit data, which is what the solver cores need.

Table 9: Interface styles and their integration consequences

Style	Declared as	Used by	Composition rule
Streaming	<code>hls::stream<T>&</code>	Security cores, FINN layers	Connect output to input; depth must cover the consumer's burst or the pipeline stalls
Memory-mapped	<code>T*</code> plus <code>#pragma HLS INTERFACE m_axi</code>	Solver L2 kernels	Host allocates; the kernel may re-read, which streaming cannot
Scalar control	<code>s_axilite</code>	Solver L2 kernels	Sizes and strides that must be known before the datapath starts
AXI4-Stream with side channels	<code>ap_axiu<W,U,T,D></code>	L2 boundaries	TLAST carries frame boundaries; a core that ignores it cannot be used in a multi-frame system

{'svg':

Givens rotation annihilates one sub-diagonal element



$G^T \cdot A$ applied to two rows at a time; `qrf_mm()` is the 2x2 product
extra_pass / use_mag arguments select the magnitude-only variant

'cap': 'Givens-rotation QR in qrf.hpp. Each rotation zeroes one element; qrf_mm applies the 2x2 rotation to a row pair.'

Figure 5: Interface styles. The two are not mixable within one core: a streaming core cannot re-read, and a memory-mapped core cannot be chained without a buffer.

2.6 Performance

No performance figures are given in this document. Throughput, latency, clock frequency and resource usage all depend on the target part, the tool version and the parameter set, and none of the 27 cores here was synthesised. What *can* be stated from the source, and is stated in each core's §n.8, is the *shape* of the cost: which loop bounds multiply, which parameters replicate hardware linearly, and which replicate it exponentially. Those relations are properties of the source and do not depend on a tool run.

3 Linear Algebra Solver Cores

Four decomposition cores are specified in this chapter. They divide along a line that matters more to an integrator than the mathematics does: whether the core **streams** or whether it **revisits memory**. Cholesky and QRF consume a matrix from an `hls::stream` and emit the factor to another, and therefore never need the whole matrix resident. SVD and POTRF hold an array and sweep it repeatedly, and therefore cannot be expressed as a stream at all. This is not a style preference: it follows from whether the algorithm touches each element once or many times.

Table 10: The four solver cores at a glance

Core	Factorisation	Algorithm	Data access	Level
Cholesky	$A = LL^*$	Right-looking, square-root form	Single pass, streaming	L1
QRF	$A = QR$	Givens rotations	Single pass, streaming	L1
SVD	$A = U\Sigma V^*$	Two-sided Jacobi, cyclic sweeps	Repeated sweeps over a resident array	L1
POTRF	$A = LL^T$	Blocked, LAPACK interface	Memory-mapped, blocked	L2

3.1 Cholesky Decomposition

Source file `Vitis_solver/cholesky.hpp` • **733 lines**, 31,537 B • **Level L1** (function template, no interface pragmas)
• **13 functions**, **6 classes**, **17 directives** • **Full source** Appendix D

3.1.1 Overview

Computes the Cholesky factor of a Hermitian positive-definite matrix, $A = LL^*$. The core is written in the right-looking square-root form: at step j the diagonal element is produced by a square root, its reciprocal is formed once, and the remainder of column j is scaled by that reciprocal before the trailing submatrix is updated. Forming the reciprocal once and multiplying is the whole reason the inner loop can be pipelined; a divider in the inner loop would set the initiation interval to the divider's latency.

The interface is a pair of streams, so the core never needs the input matrix resident. That is possible because the algorithm reads each element of A exactly once.

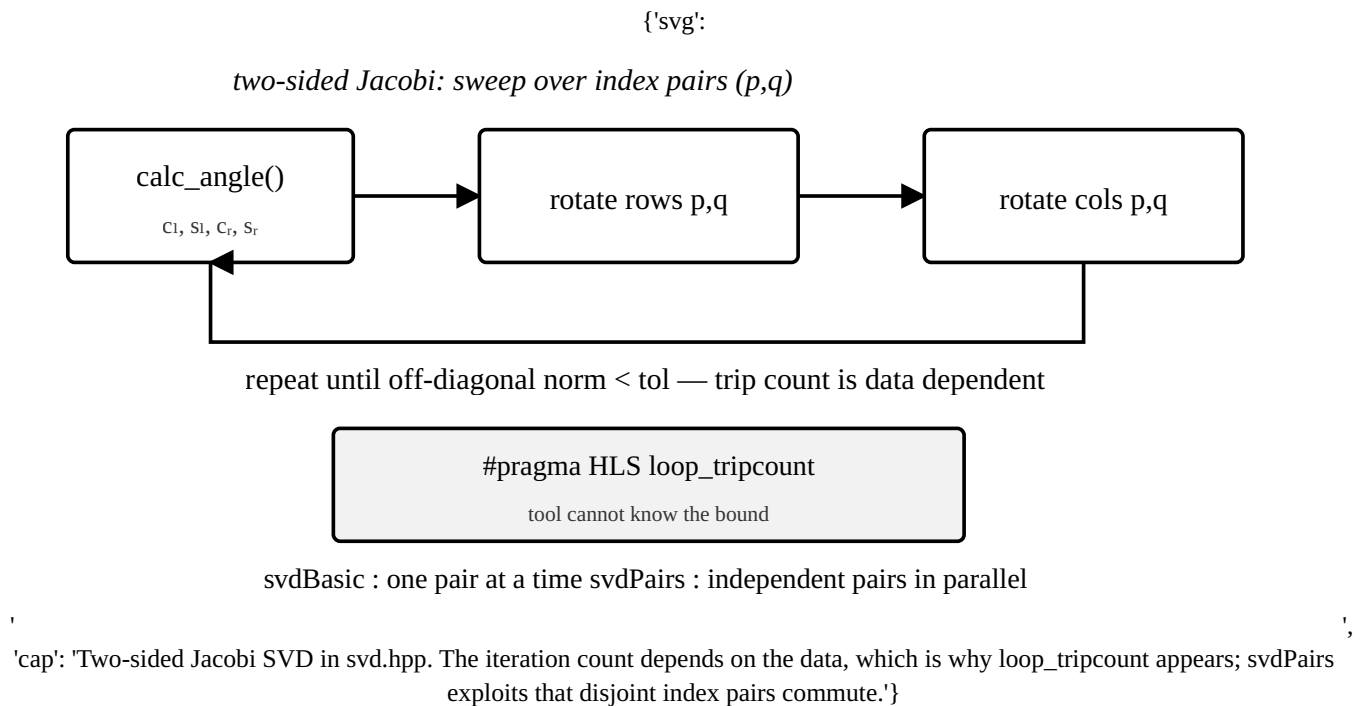


Figure 6: Cholesky datapath. The reciprocal is formed once per column, outside the inner loop; this placement is what makes the inner loop's initiation interval independent of divider latency.

3.1.2 Entry Point and Access Functions

The synthesisable entry point is the two-stream form at line 707:

```
template <bool LowerTriangularL, int RowsColsA, typename InputType,
          typename OutputType, typename TRAITS>
int cholesky(hls::stream<InputType>& matrixAStrm,
             hls::stream<OutputType>& matrixLStrm);
```

The integer return value is the failure indication: a non-zero return means the input was not positive definite, detected when the argument of the square root goes non-positive. **The core reports failure rather than producing a quietly wrong factor**, which is a property an integrator must propagate — ignoring the return value converts a detected numerical failure into corrupt downstream data. The remaining function definitions below are the arithmetic primitives the entry point is built from, including the three overloads of `cholesky_sqrt_op` that give the real, `hls::x_complex` and `std::complex` cases.

Table 11: Every function definition in `Vitis_solver/cholesky.hpp` (13 definitions)

Line	Template signature	Returns	Name	Parameters
213	template <typename T_IN, typename T_OUT>	int	cholesky_sqrt_op	T_IN a, T_OUT& b
224	template <typename T_IN, typename T_OUT>	int	cholesky_sqrt_op	hls::x_complex<T_IN> din, hls::x_complex<T_OUT>& dout
239	template <typename T_IN, typename T_OUT>	int	cholesky_sqrt_op	std::complex<T_IN> din, std::complex<T_OUT>& dout
256	template <typename InputType, typename OutputType>	void	cholesky_rsqrt	InputType x, OutputType& res
261	—	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2>	cholesky_rsqrt	ap_fixed<W1, I1, Q1, O1, N1> x, ap_fixed<W2, I2, Q2, O2, N2>& res
275	template <typename AType, typename BType, typename CType>	void	cholesky_prod_sum_mult	AType A, BType B, CType& C
280	template <typename AType, typename BType, typename CType>	void	cholesky_prod_sum_mult	hls::x_complex<AType> A, BType B, hls::x_complex<CType>& C
286	template <typename AType, typename BType, typename CType>	void	cholesky_prod_sum_mult	std::complex<AType> A, BType B, std::complex<CType>& C
295	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyBasic	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
412	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyAlt	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
515	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyAlt2	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]

Line	Template signature	Returns	Name	Parameters
673	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyTop	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
707	template <bool LowerTriangularL, int RowsColsA, class InputType, class OutputType, typename TRAITS = choleskyTraits<LowerTriangularL, RowsColsA, InputType, OutputType> >	int	cholesky	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixLStrm

3.1.3 Device Support

Device-independent. The core contains no primitive instantiation and no device-specific pragma; the only device dependence enters through BIND_STORAGE-style choices made by the caller and through whether the chosen `InputType` maps onto a DSP primitive on the target part.

3.1.4 Template Parameters and Supported Data Types

Parameterisation is done through a *traits* struct rather than through a long parameter list. `choleskyTraits` is a template whose member typedefs name the type of every intermediate — the product, the accumulator, the adder, the diagonal, and the reciprocal of the diagonal. An integrator changes the internal precision by specialising this struct, not by editing the algorithm. The specialisations shipped in the file fix the traits for `float`, `double`, and the complex forms.

Why a traits struct and not parameters. The intermediate types are not independent: the accumulator must be wide enough for the product type, and the reciprocal type must match the diagonal type. Exposing them as separate template parameters would let a caller choose an inconsistent set. A traits struct makes the consistent set the default and the inconsistent set an explicit, visible specialisation.

Table 12: Every class and struct in `Vitis_solver/cholesky.hpp` (6 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
45	struct	choleskyTraits	template <bool LowerTriangularL, int RowsColsA, typename InputType, typename OutputType>	—	typedef InputType PROD_T; typedef InputType ACCUM_T; typedef InputType ADD_T; typedef InputType DIAG_T; typedef InputType RECIP_DIAG_T; typedef InputType OFF_DIAG_T; typedef OutputType L_OUTPUT_T;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
65	struct	choleskyTraits <LowerTriangularL, RowsColsA, hls::x_com- plex<InputBaseType>, hls::x_com- plex<OutputBaseType> >	template <bool LowerTri- angularL, int RowsColsA, typename InputBaseType, typename OutputBaseType>	—	typedef hls::x_com- plex<InputBaseType> PROD_T; typedef hls::x_com- plex<InputBaseType> AC- CUM_T; typedef hls::x_com- plex<InputBaseType> ADD_T; typedef hls::x_com- plex<InputBaseType> DIAG_T; typedef InputBaseType RE- CIP_DIAG_T; typedef hls::x_com- plex<InputBaseType> OFF_DIAG_T; typedef hls::x_com- plex<OutputBaseType> L_OUTPUT_T;
82	struct	choleskyTraits <LowerTriangularL, RowsColsA, std::com- plex<InputBaseType>, std::com- plex<OutputBaseType> >	template <bool LowerTri- angularL, int RowsColsA, typename InputBaseType, typename OutputBaseType>	—	typedef std::com- plex<InputBaseType> PROD_T; typedef std::com- plex<InputBaseType> AC- CUM_T; typedef std::com- plex<InputBaseType> ADD_T; typedef std::com- plex<InputBaseType> DIAG_T; typedef InputBaseType RE- CIP_DIAG_T; typedef std::com- plex<InputBaseType> OFF_DIAG_T; typedef std::com- plex<OutputBaseType> L_OUTPUT_T;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
110	struct	choleskyTraits <LowerTriangularL, RowsColsA, ap_fixed<W1, I1, Q1, O1, N1>, ap_fixed<W2, I2, Q2, O2, N2>>	template <bool LowerTriangularL, int RowsColsA, int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>	—	typedef ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT, 0> PROD_T; typedef ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0> ADD_T; typedef ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> DIAG_T; typedef ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> OFF_DIAG_T; typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2), AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T; typedef ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0> L_OUTPUT_T;
144	struct	choleskyTraits <LowerTriangularL, RowsColsA, hls::x_complex<ap_fixed<W1, I1, Q1, O1, N1>, hls::x_complex<ap_fixed<W2, I2, Q	template <bool LowerTriangularL, int RowsColsA, int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>	—	typedef hls::x_complex<ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT, 0> > PROD_T; typedef hls::x_complex<ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0> > ADD_T; typedef hls::x_complex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > DIAG_T; typedef hls::x_complex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > OFF_DIAG_T; typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2), AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T; typedef hls::x_complex<ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0> > L_OUTPUT_T;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
181	struct	choleskyTraits <LowerTriangularL, RowsColsA, std::com- plex<ap_fixed<W1, I1, Q1, O1, N1>, std::com- plex<ap_fixed<W2, I2, Q2, O	template <bool LowerTri- angularL, int RowsColsA, int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>	—	typedef std::com- plex<ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT, 0> > PROD_T; typedef std::com- plex<ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0> > ADD_T; typedef std::com- plex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > DIAG_T; typedef std::com- plex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > OFF_DIAG_T; typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2), AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T; typedef std::com- plex<ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0> > L_OUTPUT_T;

3.1.5 Ports

As an L1 template the core declares no ports. It has two stream arguments and one scalar return. When wrapped at L2, `matrixAStrm` and `matrixLStrm` become AXI4-Stream interfaces and the return value must be carried out through a scalar `s_axi_lite` register; there is no other way for the caller to observe the positive-definiteness failure.

Table 13: Include dependencies of `Vitis_solver/cholesky.hpp`

Included file	Resolved to
<code>ap_fixed.h</code>	Vendor (Appendix E)
<code>complex</code>	Vendor (Appendix E)
<code>hls_stream.h</code>	Vendor (Appendix E)
<code>hls_x_complex.h</code>	Vendor (Appendix E)
<code>utils/std_complex_utils.h</code>	Vendor (Appendix E)
<code>utils/x_matrix_utils.hpp</code>	Corpus (Appendix D)

3.1.6 Design Notes

Reciprocal instead of division

The diagonal reciprocal is computed once per column and reused across the column. This is the reason the column loop pipelines.

Loop flattening

The file contains one `LOOP_FLATTEN`. The triangular loop nest is imperfect — the inner trip count depends on the outer index — so flattening removes one level of state machine and the pipeline drain that would otherwise occur at every column boundary. On a triangular nest this is worth roughly one drain per column, which for a large matrix is a significant fraction of the total.

Array partitioning

The five `ARRAY_PARTITION` directives bank the working storage so that the update loop can read more than one element per cycle. The partition factor and the pipeline II are a matched pair: raising the II target without raising the banking gains nothing, and raising the banking without the II target only costs memories.

3.1.7 Code Example

```
#include "cholesky.hpp"

// 16x16 single-precision, lower-triangular result.
typedef xf::solver::choleskyTraits<true, 16, float, float> TR;

hls::stream<float> a_strm, l_strm;
// ... push 16*16 elements of A, row-major, into a_strm ...

int info = xf::solver::cholesky<true, 16, float, float, TR>(a_strm, l_strm);
if (info != 0) {
    // A was not positive definite. l_strm content is undefined.
    // This MUST be propagated; it is the core's only failure channel.
}
```

3.1.8 Implementation on FPGA

The cost shape, read from the source rather than from a synthesis run: the outer loop runs `RowsColsA` times and the inner update is triangular, so the operation count grows as $n^3/6$ while the storage grows as n^2 . One square root and one reciprocal are instantiated per *column*, not per element, so their area is amortised. Raising `ARRAY_PARTITION` multiplies the number of memories by the partition factor and is the dominant area term once n is large.

Table 14: Every synthesis directive in `Vitis_solver/cholesky.hpp` (17)

Line	Directive as written
368	<code>#pragma HLS PIPELINE II = CholeskyTraits::INNER_II</code>
449	<code>#pragma HLS loop_tripcount max = 1 + RowsColsA / 2</code>
461	<code>#pragma HLS loop_tripcount max = 1 + RowsColsA / 2</code>
462	<code>#pragma HLS PIPELINE II = CholeskyTraits::INNER_II</code>
537	<code>#pragma HLS ARRAY_PARTITION variable = A cyclic dim = CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR</code>
538	<code>#pragma HLS ARRAY_PARTITION variable = L cyclic dim = CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR</code>
539	<code>#pragma HLS ARRAY_PARTITION variable = L_internal cyclic dim = CholeskyTraits::UNROLL_DIM factor = \</code>
541	<code>#pragma HLS ARRAY_PARTITION variable = square_sum_array cyclic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR</code>
542	<code>#pragma HLS ARRAY_PARTITION variable = product_sum_array cyclic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR</code>
575	<code>#pragma HLS loop_tripcount max = 1 + RowsColsA / 2</code>
590	<code>#pragma HLS LOOP_FLATTEN off</code>
591	<code>#pragma HLS PIPELINE II = CholeskyTraits::INNER_II</code>
592	<code>#pragma HLS UNROLL FACTOR = CholeskyTraits::UNROLL_FACTOR</code>
656	<code>#pragma HLS loop_tripcount max = 1 + RowsColsA / 2</code>
657	<code>#pragma HLS PIPELINE</code>
712	<code>#pragma HLS PIPELINE</code>

Line	Directive as written
722	<code>#pragma HLS PIPELINE</code>

3.1.9 Constraints

- A must be Hermitian positive definite. The core detects violation and returns non-zero; it does not check symmetry.
- `RowsColsA` is a compile-time constant. A run-time matrix size is not supported by this core; POTRF (§3.4) is the variable-size path.
- Elements must be pushed into `matrixAStrm` in the order the core consumes them. There is no addressing, so an out-of-order producer silently produces a wrong factor.
- The return value must be checked.

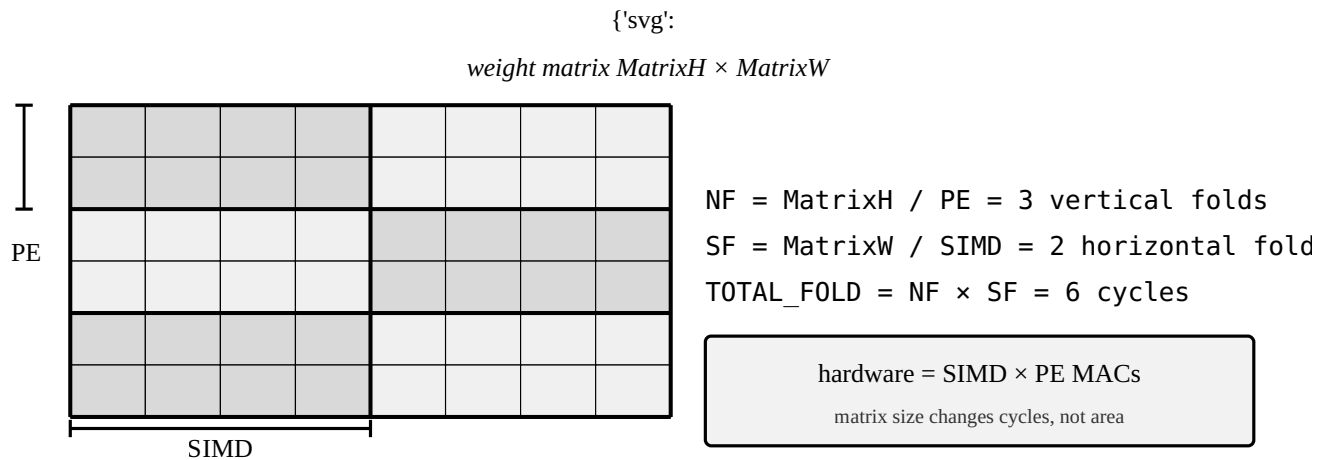
3.2 QR Factorisation (QRF)

Source file `Vitis_solver/qrf.hpp` • **890 lines**, 29,856 B • **Level L1** (function template, no interface pragmas) • **16 functions**, **5 classes**, **22 directives** • **Full source** Appendix D

3.2.1 Overview

Factorises $A = QR$ by Givens rotations. A Givens rotation annihilates one subdiagonal element at a time using a plane rotation built from a magnitude and two ratios. The file contains four separate magnitude routines — real and complex, each in a guarded and an unguarded form — and the comments at lines 159, 184, 241 and 299 state the reason: the naive magnitude $\sqrt{a^2+b^2}$ overflows for large operands and underflows to zero for small ones, and the complex case must avoid squaring a value that has just been square-rooted.

This is the part of the core that is hardest to reproduce correctly in RTL. The rotation arithmetic is four lines; the numerical guards around it are several hundred. An RTL reimplementation that copies the four lines and omits the guards will pass a random-matrix test-bench and fail on badly scaled input.



'cap': 'Folding in mvau.hpp. Physical MAC resources are fixed at SIMD x PE; a larger matrix costs more cycles, never more area.'

Figure 7: QRF rotation pipeline. The guarded magnitude path (shaded) is the larger part of the logic and exists only to keep the factorisation numerically valid at the extremes of the exponent range.

3.2.2 Entry Point and Access Functions

The core exposes the factorisation as a function template over the input and output matrix types. The definitions below include the rotation primitives, which are separately callable and are the reusable part of the file.

Table 15: Every function definition in `Vitis_solver/qrf.hpp` (16 definitions)

Line	Template signature	Returns	Name	Parameters
70	template <type-name T>	template <type-name T> int	qrf_require_extra_pass	const int rows, const int cols, T x
76	template <type-name T>	int	qrf_require_extra_pass	const int rows, const int cols, hls::x_complex<T> x
87	template <type-name T>	int	qrf_require_extra_pass	const int rows, const int cols, std::complex<T> x
98	template <class T>	bool	is_zero	T a

Line	Template signature	Returns	Name	Parameters
124	template <type-name T>	T	qrf_vm	T a, T b, T c, T d
130	template <type-name T>	void	qrf_mm	const T G[2][2], T& a, T& b
140	template <type-name T>	void	qrf_mm_or_mag	const T G[2][2], T& a, T& b, const T mag, const int use_mag, const int extra_pass
161	template <type-name T>	T	qrf_magnitude	T a, T b
187	template <type-name T>	T	qrf_magnitude	hls::x_complex<T> a, hls::x_complex<T> b
244	template <type-name T>	T	qrf_magnitude	std::complex<T> a, std::complex<T> b
306	template <type-name T>	void	qrf_givens	int extra_pass, T a, T b, T& c, T& s, T& ss, T& cc, T& r
353	template <type-name T>	void	qrf_givens	int extra_pass, hls::x_complex<T> a, hls::x_complex<T> b, hls::x_complex<T>& c, hls::x_complex<T>& s, hls::x_complex<T>& ss, hls::x_complex<T>& cc, hls::x_complex<T>& r
407	template <type-name T>	void	qrf_givens	int extra_pass, std::complex<T> a, std::complex<T> b, std::complex<T>& c, std::complex<T>& s, std::complex<T>& ss, std::complex<T>& cc, std::complex<T>& r
587	—	template <bool TransposedQ, int RowsA, int ColsA, typename QRF_TRAITS, typename InputType,	qrf_basic	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixQStrm, hls::stream<OutputType>& matrixRStrm
702	—	template <bool TransposedQ, int RowsA, int ColsA, typename QRF_TRAITS, typename InputType,	qrf_alt	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixQStrm, hls::stream<OutputType>& matrixRStrm
868	template <bool TransposedQ, int RowsA, int ColsA, typename InputType, typename OutputType, typename QRF_TRAITS = qrfTraits>	void	qrf	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixQStrm, hls::stream<OutputType>& matrixRStrm

3.2.3 Device Support

Device-independent. The only device-sensitive choice is whether the chosen element type maps to a DSP primitive; the rotation itself is expressed as multiplies and adds with no primitive instantiation.

3.2.4 Template Parameters and Supported Data Types

As with Cholesky, precision is carried by a traits struct. The architecture selector QRF_ALT is visible in the traits as CALC_ROT_II (line 47), which names the target initiation interval of the rotation-calculation loop. Setting it to 1 asks for a new rotation every cycle and costs the operators needed to sustain that; a larger value lets the scheduler share operators across cycles.

Table 16: Every class and struct in Vitis_solver/qrf.hpp (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
45	struct	qrftTraits	—	—	—
56	struct	is_cmplx	template <typename T>	—	—
61	struct	is_cmplx <hls::x_complex<T> >	template <typename T>	—	—
66	struct	is_cmplx <std::complex<T> >	template <typename T>	—	—
468	struct	qrf_alt_config	template <int ROWS, int COLS, typename InputType>	—	—

3.2.5 Ports

L1; no ports. The two STREAM directives in the file force internal arrays to be implemented as FIFOs rather than as memories, which is what lets the rotation stage and the update stage overlap.

Table 17: Include dependencies of Vitis_solver/qrf.hpp

Included file	Resolved to
complex	Vendor (Appendix E)
hls_math.h	Vendor (Appendix E)
hls_stream.h	Vendor (Appendix E)
hls_x_complex.h	Vendor (Appendix E)
stdio.h	C++ standard library
stdlib.h	C++ standard library
utils/std_complex_utils.h	Vendor (Appendix E)
utils/x_matrix_utils.hpp	Corpus (Appendix D)

3.2.6 Design Notes

Loop merging

Three LOOP_MERGE directives fuse adjacent loops. Each merge removes one state machine and one pipeline drain. This is the single most common structural transformation in the file and is the reason it carries eleven PIPELINE directives rather than a smaller number of larger loops.

Guarded magnitude

The guarded forms scale the operands before squaring and unscale afterwards. The cost is two extra multiplies and a comparison per rotation; the benefit is that the factorisation does not produce NaN for inputs whose squares are outside the exponent range.

3.2.7 Code Example

```
#include "qrf.hpp"

// 32x32 single precision. Q accumulated explicitly.
hls::stream<float> a_strm, q_strm, r_strm;
xf::solver::qrf<true, 32, 32, float, float>(a_strm, q_strm, r_strm);
```

3.2.8 Implementation on FPGA

Cost shape: the number of rotations is $mn - n^2/2$, and each rotation is a fixed-size arithmetic block, so the operation count grows as mn and the area is set by how many rotations are in flight — that is, by CALC_ROT_II and the UNROLL factors, not by the matrix size.

Table 18: Every synthesis directive in Vitis_solver/qrf.hpp (22)

Line	Directive as written
618	#pragma HLS PIPELINE
627	#pragma HLS PIPELINE
661	#pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
670	#pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
686	#pragma HLS PIPELINE
721	#pragma HLS ARRAY_PARTITION variable = q_i cyclic dim = 2 factor = QRF_TRAITS::UNROLL_FACTOR
722	#pragma HLS ARRAY_PARTITION variable = r_i cyclic dim = 2 factor = QRF_TRAITS::UNROLL_FACTOR
725	#pragma HLS STREAM variable = to_rot depth = RowsA / 2
734	#pragma HLS STREAM variable = rotations depth = RowsA / 2
742	#pragma HLS LOOP_MERGE force
745	#pragma HLS PIPELINE
754	#pragma HLS PIPELINE
764	#pragma HLS LOOP_TRIPCOUNT min = 1 max = RowsA / 2
765	#pragma HLS PIPELINE II = QRF_TRAITS::CALC_ROT_II
788	#pragma HLS LOOP_TRIPCOUNT min = 1 max = RowsA / 2
805	#pragma HLS LOOP_MERGE force
808	#pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
809	#pragma HLS UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR
818	#pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
819	#pragma HLS UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR
829	#pragma HLS LOOP_MERGE force
832	#pragma HLS PIPELINE

3.2.9 Constraints

- Matrix dimensions are compile-time constants.
- The guarded magnitude routines must be used for badly scaled input; selecting the unguarded form is a decision about the input range, not about area.
- The FIFO depths implied by the two STREAM directives are fixed in the source. A consumer that stalls longer than the depth will stall the producer.

3.3 Singular Value Decomposition (SVD)

Source file `Vitis_solver/svd.hpp` • **1,578 lines**, 72,409 B • **Level L1** (function template, no interface pragmas) • **15 functions**, **1 classes**, **44 directives** • **Full source** Appendix D

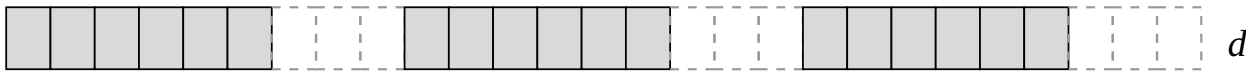
3.3.1 Overview

Computes $A = U\Sigma V^*$ by two-sided Jacobi. The matrix is held resident and swept repeatedly; each sweep visits every off-diagonal pair (p, q) and applies a rotation that zeroes that pair. The traits struct fixes `NUM_SWEEPS = 10` with the comment that the literature suggests 6 to 10 iterations to converge (line 51).

The sweep count is fixed, not adaptive. The core does not test for convergence and stop; it performs exactly `NUM_SWEEPS` sweeps. That is a deliberate hardware decision — a data-dependent trip count would make the latency unbounded and the pipeline unschedulable — but it means **the accuracy of the result is a compile-time constant, not a run-time property**. An integrator whose matrices converge more slowly than the literature case gets a silently inaccurate decomposition, with no error flag. This is the single most important constraint on this core.

{'svg':

(a) nested loops — pipeline drains at every outer boundary



(b) merged iteration space — $II = 1$ throughout



```
for (i = 0; i < reps*NF*SF; i++) { #pragma HLS pipeline
... ; if (++sf == SF) { sf = 0; ++nf; } // counters kep
```

“to get the pipelining the way we want” — source comment

```
'cap': 'Loop flattening. FINN merges the nested fold loops into one iteration space and maintains sf/nf/tile manually so the
pipeline never drains.'
```

Figure 8: Jacobi sweep structure. Pairs within a sweep are disjoint, which is the fact the six `DEPENDENCE` assertions encode; the assertions are properties of the index generator, not of the array.

3.3.2 Entry Point and Access Functions

Two architectures are provided and named in the file header: `svdBASIC`, the default, and `svdPairs`, an alternative that processes rotation pairs concurrently. They are selected through the traits parameter, not through separate entry points.

Table 19: Every function definition in `Vitis_solver/svd.hpp` (15 definitions)

Line	Template signature	Returns	Name	Parameters
69	template <int RowsA, int ColsA, typename InputType, typename OutputType>	template <typename T> bool	with-in_precision	T a, T b
85	template <typename AType, typename BType, typename CType>	void	vm2x1	AType a1, BType b1, AType a2, BType b2, CType& c
95	template <typename AType, typename BType, typename CType>	void	vm2x1_base	AType a1, BType b1, AType a2, BType b2, CType& c
101	template <typename T>	void	vm2x1	T a1, hls::x_complex<T> b1, T a2, hls::x_complex<T> b2, hls::x_complex<T>& c
108	template <typename T>	void	vm2x1	hls::x_complex<T> a1, T b1, hls::x_complex<T> a2, T b2, hls::x_complex<T>& c
115	template <typename T>	void	vm2x1	T a1, std::complex<T> b1, T a2, std::complex<T> b2, std::complex<T>& c
122	template <typename T>	void	vm2x1	std::complex<T> a1, T b1, std::complex<T> a2, T b2, std::complex<T>& c
129	template <typename T>	void	vm2x1	T a1, T b1, T a2, T b2, T& c
136	template <typename AType, typename BType, typename CType>	void	mm2x2	AType a1, AType a2, AType a3, AType a4, BType b1, BType b2, BType b3, BType b4, CType& c1, CType& c2, CType& c3, CType& c4
156	template <typename InType, typename CType>	void	calc_angle	hls::x_complex<InType> A, CType& cosThetaA, CType& sinThetaA, CType& cosThetaAdiv2, CType& sinThetaAdiv2, bool& is_pos_real, bool& is_imag
234	template <typename InType, typename CType>	void	calc_angle	std::complex<InType> A, CType& cosThetaA, CType& sinThetaA, CType& cosThetaAdiv2, CType& sinThetaAdiv2, bool& is_pos_real, bool& is_imag
821	template <int RowsA, int ColsA, class SVDTraits, typename InputType, typename OutputType>	—	svdBasic	const InputType A[RowsA][ColsA], OutputType S[RowsA][ColsA], OutputType U[RowsA][RowsA], OutputType V[ColsA][ColsA]
1037	template <int RowsA, int ColsA, class SVDTraits, typename InputType, typename OutputType>	—	svdPairs	const InputType A[RowsA][ColsA], OutputType S[RowsA][ColsA], OutputType U[RowsA][RowsA], OutputType V[ColsA][ColsA]

Line	Template signature	Returns	Name	Parameters
1503	template <int RowsA, int ColsA, class SVDTraits, typename InputType, typename OutputType>	—	svdTop	const InputType A[RowsA][ColsA], OutputType S[RowsA][ColsA], OutputType U[RowsA][RowsA], OutputType V[ColsA][ColsA]
1539	template <int RowsA, int ColsA, typename InputType, typename OutputType, typename SVDTraits = svdTraits<RowsA, ColsA, InputType, OutputType> >	void	svd	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixSStrm, hls::stream<OutputType>& matrixUSStrm, hls::stream<OutputType>& matrixVStrm

3.3.3 Device Support

Device-independent in source. In practice the twenty `ARRAY_PARTITION` directives make this the most memory-hungry core in the corpus, so the achievable matrix size is set by the target part's block-RAM count.

3.3.4 Template Parameters and Supported Data Types

`svdTraits` carries the sweep count and two separate initiation-interval targets: `OFF_DIAG_II` and `DIAG_II`, both defaulting to 8 (lines 54 and 56). The comment on line 54 explains the number: the off-diagonal loop needs up to four memory accesses per iteration, and an II of 8 lets those accesses be sequenced through shared ports. The commented-out `UNROLL_FACTOR` lines (58–59) record an unrolling strategy that was tried and left disabled — useful evidence for an integrator considering the same change.

Table 20: Every class and struct in `Vitis_solver/svd.hpp` (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
46	struct	<code>svdTraits</code>	template <int RowsA, int ColsA, typename InputType, typename OutputType>	—	typedef OutputType SIntType; typedef OutputType UIntType; typedef OutputType VIntType; typedef OutputType CSIntType;

3.3.5 Ports

L1; no ports. Unlike Cholesky and QRF this core cannot be given a streaming interface, because Jacobi revisits every element on every sweep.

Table 21: Include dependencies of `Vitis_solver/svd.hpp`

Included file	Resolved to
<code>ap_fixed.h</code>	Vendor (Appendix E)
<code>assert.h</code>	C++ standard library
<code>complex</code>	Vendor (Appendix E)
<code>hls_stream.h</code>	Vendor (Appendix E)
<code>hls_x_complex.h</code>	Vendor (Appendix E)
<code>utils/std_complex_utils.h</code>	Vendor (Appendix E)
<code>utils/x_matrix_utils.hpp</code>	Corpus (Appendix D)

3.3.6 Design Notes

Six dependence assertions

The file contains six DEPENDENCE directives — the second highest count in the corpus. Each asserts that a pair of accesses to the working array cannot alias. In a Jacobi sweep that is true *because of how the pair indices are generated*: (p, q) pairs within one sweep are disjoint. **The assertion is a property of the index generation, not of the array.** Any modification to the pair-ordering logic invalidates all six, and the tool will not notice.

Banking dominates

Twenty ARRAY_PARTITION directives against ten PIPELINE directives is an unusual ratio and says that this core is memory-port-bound rather than operator-bound.

3.3.7 Code Example

```
#include "svd.hpp"

// 8x8 double precision, default (svdBasic) architecture, 10 sweeps.
double A[8][8], U[8][8], S[8][8], V[8][8];
xf::solver::svd<8, 8, double, double>(A, S, U, V);
// NOTE: no convergence flag is returned. Accuracy is fixed by NUM_SWEEPS.
```

3.3.8 Implementation on FPGA

Cost shape: work grows as $\text{NUM_SWEEPS} \times n^2/2$ rotations, each a fixed block; storage grows as n^2 multiplied by the partition factor. Latency is deterministic and equal to the sweep count times the sweep length — which is the compensation for the fixed sweep count.

Table 22: Every synthesis directive in Vitis_solver/svd.hpp (44)

Line	Directive as written
98	#pragma HLS inline off
336	#pragma HLS inline
449	#pragma HLS inline
646	#pragma HLS inline
946	#pragma HLS PIPELINE II = SVDTraits::OFF_DIAG_II
975	#pragma HLS PIPELINE II = SVDTraits::OFF_DIAG_II
1106	#pragma HLS ARRAY_PARTITION variable=diag1_i cyclic dim=2 factor=SVDTraits::UNROLL_FACTOR
1107	#pragma HLS ARRAY_PARTITION variable=diag2_i cyclic dim=2 factor=SVDTraits::UNROLL_FACTOR
1108	#pragma HLS ARRAY_PARTITION variable=s_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1109	#pragma HLS ARRAY_PARTITION variable=s_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1110	#pragma HLS ARRAY_PARTITION variable=u_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1111	#pragma HLS ARRAY_PARTITION variable=u_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1112	#pragma HLS ARRAY_PARTITION variable=v_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1113	#pragma HLS ARRAY_PARTITION variable=v_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR

Line	Directive as written
1114	#pragma HLS ARRAY_PARTITION variable=diag_w_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1115	#pragma HLS ARRAY_PARTITION variable=diag_x_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1116	#pragma HLS ARRAY_PARTITION variable=diag_y_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1117	#pragma HLS ARRAY_PARTITION variable=diag_z_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1118	#pragma HLS ARRAY_PARTITION variable=uw_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1119	#pragma HLS ARRAY_PARTITION variable=ux_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1120	#pragma HLS ARRAY_PARTITION variable=uy_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1121	#pragma HLS ARRAY_PARTITION variable=uz_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1122	#pragma HLS ARRAY_PARTITION variable=vw_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1123	#pragma HLS ARRAY_PARTITION variable=vx_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1124	#pragma HLS ARRAY_PARTITION variable=vy_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1125	#pragma HLS ARRAY_PARTITION variable=vz_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1135	#pragma HLS PIPELINE
1166	#pragma HLS UNROLL FACTOR = SVDTraits::UNROLL_FACTOR
1185	#pragma HLS UNROLL FACTOR = SVDTraits::DIAG_UNROLL_FACTOR
1186	#pragma HLS PIPELINE II = SVDTraits::DIAG_II
1234	#pragma HLS UNROLL FACTOR = SVDTraits::UNROLL_FACTOR
1251	#pragma HLS PIPELINE II = SVDTraits::OFF_DIAG_II
1253	#pragma HLS INLINE recursive
1260	#pragma HLS dependence variable=s_col1 inter false
1261	#pragma HLS dependence variable=s_col2 inter false
1262	#pragma HLS dependence variable=u_col1 inter false
1263	#pragma HLS dependence variable=u_col2 inter false
1264	#pragma HLS dependence variable=v_col1 inter false
1265	#pragma HLS dependence variable=v_col2 inter false
1477	#pragma HLS PIPELINE
1549	#pragma HLS PIPELINE
1557	#pragma HLS PIPELINE
1563	#pragma HLS PIPELINE
1569	#pragma HLS PIPELINE

3.3.9 Constraints

- **Convergence is assumed, not checked.** Validate NUM_SWEEPS against the actual matrix population before deployment.
- The six dependence assertions constrain any change to the pair ordering.
- The whole matrix must be resident; this core does not stream.

3.4 Blocked Cholesky (POTRF, LAPACK interface)

Source file `Vitis_solver_L2/potrf.hpp` • 201 lines, 5,967 B • Level L2 (kernel with interface directives) • 1 functions, 0 classes, 24 directives • Full source Appendix D

3.4.1 Overview

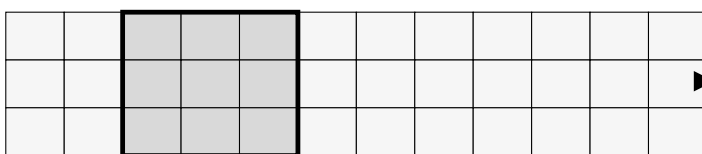
The same factorisation as §3.1, presented at L2 with the LAPACK calling convention:

```
template <typename T>
void potrf(int m, T* A, int lda, int& info);
```

Three things change relative to the L1 Cholesky and all three matter to an integrator. The matrix size `m` is a **run-time** argument, not a template parameter. The matrix is a pointer with a leading dimension `lda`, so it can be a submatrix of a larger array. And `info` follows the LAPACK convention, which makes this core a drop-in for software that already calls `dpotrf`.

{'svg':

line buffer feeding a K×K convolution window



IFMDim × IFMChannels input stream

$K^2 \cdot \text{SIMD values presented per cycle}$

buffer depth = $(K-1) \cdot \text{IFMDim} + K$

K×K window

#pragma HLS RESOURCE / BIND_STORAGE

BRAM vs LUTRAM vs URAM selected here

'cap': 'slidingwindow.h (2,095 lines, the largest file). A line buffer converts a raster stream into overlapping windows; the storage pragma chooses the memory primitive.'

Figure 9: Blocked POTRF under DATAFLOW. Panel factorisation and trailing update run concurrently; the four DEPENDENCE assertions are the sole justification for the overlap.

3.4.2 Entry Point and Access Functions

One function definition, at line 170. The entire blocked algorithm is inside it; the DATAFLOW directive is what turns that single function into concurrently running stages.

Table 23: Every function definition in `Vitis_solver_L2/potrf.hpp` (1 definitions)

Line	Template signature	Returns	Name	Parameters
169	template <typename T, int NMAX, int NCU>	void	potrf	int m, T* A, int lda, int& info

3.4.3 Device Support

Three BIND_STORAGE directives pin working arrays to specific memory primitives. This is the one core in the corpus that is explicitly device-aware: the chosen primitive must exist on the target part.

3.4.4 Template Parameters and Supported Data Types

A single type parameter `T`. There is no traits struct, because the blocked formulation does not need separate accumulator and product types — it calls the same type throughout. The block size is fixed in the source rather than exposed.

3.4.5 Ports

At L2 the pointer argument becomes an AXI master and the scalars become `s_axilite` registers. `info` is an `int&`, so it is an output register: the host reads it after the kernel completes.

Table 24: Include dependencies of `Vitis_solver_L2/potrf.hpp`

Included file	Resolved to
float.h	C++ standard library
hw/math_helper.hpp	Corpus (Appendix D)

3.4.6 Design Notes

DATAFLOW plus four dependence assertions

This is the only core in the corpus that combines DATAFLOW with DEPENDENCE. DATAFLOW runs the panel factorisation and the trailing update concurrently; the four DEPENDENCE assertions are what permit that overlap, by telling the tool that the update never reads a block the panel is still writing. **The correctness of the overlap rests entirely on those four assertions being true for every value of the run-time size m .**

Run-time size and LOOP_TRIPCOUNT

Because m is a run-time argument the tool cannot bound the loops, so two LOOP_TRIPCOUNT directives supply bounds. These affect the latency *report* only; they do not constrain the hardware, and a value of m outside the stated range runs correctly but was not what the reported latency described.

3.4.7 Code Example

```
#include "potrf.hpp"

int info = 0;
// A is m x m stored in an lda-strided buffer; may be a submatrix.
xf::solver::potrf<double>(m, A, lda, info);
if (info > 0) {
    // leading minor of order `info` is not positive definite (LAPACK convention)
}
```

3.4.8 Implementation on FPGA

Cost shape: identical $n^3/6$ operation count to §3.1, but the blocking converts it into a sequence of fixed-size block operations whose area does not grow with m . That is the point of the L2 form: **area is constant in the problem size and only latency grows**, whereas the L1 core's area grows with the compile-time dimension.

Table 25: Every synthesis directive in Vitis_solver_L2/potrf.hpp (24)

Line	Directive as written
37	#pragma HLS loop_tripcount min = 1 max = 8
39	#pragma HLS BIND_STORAGE variable = tmp_i type = ram_2p impl = lutram
43	#pragma HLS loop_tripcount min = 1 max = N
44	#pragma HLS pipeline
45	#pragma HLS dependence variable = tmp_i inter false
46	#pragma HLS dependence variable = dataA inter false
52	#pragma HLS pipeline
58	#pragma HLS pipeline
64	#pragma HLS pipeline
77	#pragma HLS BIND_STORAGE variable = tmp type = ram_2p impl = lutram
81	#pragma HLS pipeline
82	#pragma HLS dependence variable = tmp inter false
83	#pragma HLS dependence variable = dataA inter false
87	#pragma HLS unroll

Line	Directive as written
94	<code>#pragma HLS pipeline</code>
100	<code>#pragma HLS pipeline</code>
106	<code>#pragma HLS pipeline</code>
131	<code>#pragma HLS DATAFLOW</code>
135	<code>#pragma HLS unroll factor = NCU</code>
143	<code>#pragma HLS array_partition variable = dataj cyclic factor = NCU</code>
175	<code>#pragma HLS array_partition variable = matA cyclic factor = NCU</code>
176	<code>#pragma HLS BIND_STORAGE variable = matA type = xpm_memory impl = uram</code>
181	<code>#pragma HLS pipeline</code>
191	<code>#pragma HLS pipeline</code>

3.4.9 Constraints

- The four dependence assertions must hold for every run-time m .
- `LOOP_TRIPCOUNT` bounds are for reporting only.
- The three `BIND_STORAGE` choices must be available on the target device.
- LAPACK info convention: positive means the leading minor of that order is not positive definite.

3.5 Shared Solver Utilities

Two files carry material used by all four cores. They contain no directives, because they are not where the scheduling happens; they are where the types and the scalar maths live.

Vitis_solver/utils/x_matrix_utils.hpp

Matrix traversal helpers and the type plumbing that lets the same core body serve real, `hls::x_complex` and `std::complex` element types. 257 lines, 21 function definitions, 5 class definitions; full source in Appendix D.

Table 26: Every class and struct in Vitis_solver/utils/x_matrix_utils.hpp (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
40	struct	NoTranspose	—	—	—
49	struct	Transpose	template <int RowsA, int ColsA, typename InputType>	—	—
58	struct	ConjugateTranspose	template <int RowsA, int ColsA, typename InputType>	—	—
156	struct	xil_printer	template <typename T>	—	—
170	struct	xil_printer <hls::x_complex<T>>	template <typename T>	—	typedef xil_printer<T> printer;

Table 27: Every function definition in Vitis_solver/utils/x_matrix_utils.hpp (21 definitions)

Line	Template signature	Returns	Name	Parameters
42	template <int RowsA, int ColsA, typename InputType>	static InputType	GetElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
51	template <int RowsA, int ColsA, typename InputType>	template <int RowsA, int ColsA, typename InputType> static InputType	GetElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
60	template <int RowsA, int ColsA, typename InputType>	template <int RowsA, int ColsA, typename InputType> static InputType	GetElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
68	template <int RowsA, int ColsA, typename InputType>	template <class TransposeForm, int RowsA, int ColsA, typename InputType> InputType	GetMatrixElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
78	—	static half	x_sqrt	half x
81	—	static float	x_sqrt	float x
84	—	static double	x_sqrt	double x
87	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1> ap_fixed<W1, I1, Q1, O1, N1>	—	x_sqrt	ap_fixed<W1, I1, Q1, O1, N1> x
93	—	static float	x_copysign	float a, float b

Line	Template signature	Returns	Name	Parameters
96	—	static double	x_copysign	double a, double b
101	—	static float	x_sign	float x
108	—	static double	x_sign	double x
115	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1> ap_fixed<W1, I1, Q1, O1, N1>	—	x_sign	ap_fixed<W1, I1, Q1, O1, N1> x
127	—	static half	x_rsqrtr	half x
130	—	static float	x_rsqrtr	float x
133	—	static double	x_rsqrtr	double x
138	—	static int	x_isneg	float x
142	—	static int	x_isneg	double x
157	template <typename T>	static std::string	to_s	T x, unsigned prec = 10
171	template <typename T>	static std::string	to_s	hls::x_complex<T> x, unsigned prec = 10
187	template <typename T>	template <unsigned ROWS, unsigned COLS, typename T, class TransposeForm> void	print_matrix	T a[ROWS][COLS], std::string prefix = "", unsigned prec = 10, unsigned matlab_format = 0

Vitis_solver_L2/hw/math_helper.hpp

Scalar maths wrappers that select between the standard-library function and the HLS fixed-point equivalent depending on the instantiating type. 105 lines, 9 function definitions, 0 class definitions; full source in Appendix D.

Table 28: Every function definition in Vitis_solver_L2/hw/math_helper.hpp (9 definitions)

Line	Template signature	Returns	Name	Parameters
34	template <typename T>	T	abs	T a
39	—	double	abs	double a
47	—	float	abs	float a
56	template <typename T>	T	fabs	T a
61	—	double	fabs	double a
69	—	float	fabs	float a
78	template <typename T>	T	sqrt	T a
83	—	double	sqrt	double a
91	—	float	sqrt	float a

4 Cryptographic Cores

Two cores are specified here. They are the clearest example in the corpus of a property that distinguishes production HLS IP from a reference implementation: **the algorithm is the easy part, and almost all of the source is about where the constants live and how the rounds are scheduled.** AES is a hundred lines of algorithm surrounded by nine hundred lines of key schedule, table placement and round unrolling; SHA-256 is a sixty-four-step compression function surrounded by a dataflow pipeline that keeps it fed.

4.1 AES Block Cipher

Source file `Vitis_security/aes.hpp` • **1,011 lines**, 45,200 B • **Level L1** (class templates over stream and block types)
 • **30 functions**, **9 classes**, **45 directives** • **Full source** Appendix D

4.1.1 Overview

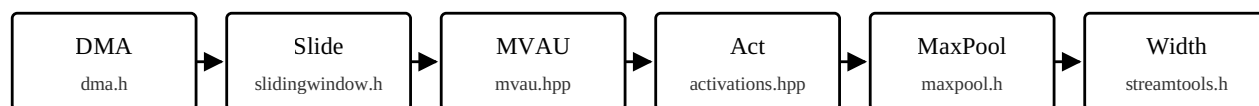
AES encryption and decryption for all three standard key lengths. The structure is a base class holding the substitution tables and four template specialisations holding the key schedules:

```
class aesTable;                                // line 12  -- S-box and inverse S-box
class aesEnc { ... };                          // line 55  -- primary template
class aesEnc<256> : public aesTable;           // line 62
class aesEnc<192> : public aesTable;           // line 241
class aesEnc<128> : public aesTable;           // line 383
class aesDec { ... };                          // line 516  -- and its specialisations
```

The key length is therefore a **type**, not a value. There is no run-time branch on key length anywhere in the generated hardware: instantiating `aesEnc<128>` builds a ten-round datapath and nothing else. An integrator who needs to support more than one key length at run time must instantiate more than one core and multiplex, and must budget the area for all of them.

{'svg':

every stage communicates only through `hls::stream<>` (250 uses)



DATAFLOW composition: stages run concurrently, back-pressure is implicit

no shared arrays → no false dependences → II=1 per stage

'cap': 'FINN composes an accelerator as a chain of stream-connected stages; the stream abstraction is what makes concurrent scheduling and back-pressure automatic.'

Figure 10: AES round datapath. The S-box binding to LUT RAM (shaded) is what allows the table to be replicated; with a block-RAM binding the unrolled round is limited to two lookups per cycle.

4.1.2 Entry Point and Access Functions

The cores are used by constructing the object, calling the key expansion once, and then calling the block operation per block. The key schedule is deliberately separated from the block operation so that it is not re-executed per block; every function definition in the file is listed below.

Table 29: Every function definition in `Vitis_security/aes.hpp` (30 definitions)

Line	Template signature	Returns	Name	Parameters
13	—	public:	<code>aesTable</code>	<code>()</code>
56	<code>template <int W></code>	public: void	<code>updateKey</code>	<code>ap_uint<W> cipherkey</code>
58	<code>template <int W></code>	void	<code>process</code>	<code>ap_uint<128> plaintext, ap_uint<W> cipherkey, ap_uint<128>& ciphertext</code>
71	<code>template <></code>	void	<code>updateKey</code>	<code>ap_uint<256> cipherkey</code>
114	—	static <code>ap_uint<8></code>	<code>GFMul2</code>	<code>ap_uint<8> a</code>

Line	Template signature	Returns	Name	Parameters
126	—	void	process	ap_uint<128> plaintext, ap_uint<256> cipherkey, ap_uint<128>& ciphertext
204	—	void	updateKey_2	ap_uint<256> cipherkey
250	template <>	void	updateKey	ap_uint<192> cipherkey
291	—	static ap_uint<8>	GFMul2	ap_uint<8> a
303	—	void	process	ap_uint<128> plaintext, ap_uint<192> cipherkey, ap_uint<128>& ciphertext
391	template <>	void	updateKey	ap_uint<128> cipherkey
418	template <>	static ap_uint<8>	GFMul2	ap_uint<8> a
429	—	void	process	ap_uint<128> plaintext, ap_uint<128> cipherkey, ap_uint<128>& ciphertext
517	template <int W>	public: void	updateKey	ap_uint<W> cipherkey
531	template <int W>	void	process	ap_uint<128> ciphertext, ap_uint<W> cipherkey, ap_uint<128>& plaintext
544	template <>	void	updateKey	ap_uint<256> cipherkey
584	—	static ap_uint<8>	GFMul2	ap_uint<8> a
596	—	static ap_uint<8>	GFMul4	ap_uint<8> a
610	—	static ap_uint<8>	GFMul8	ap_uint<8> a
624	—	void	process	ap_uint<128> ciphertext, ap_uint<256> cipherkey, ap_uint<128>& plaintext
707	template <>	void	updateKey	ap_uint<192> cipherkey
746	—	static ap_uint<8>	GFMul2	ap_uint<8> a
758	—	static ap_uint<8>	GFMul4	ap_uint<8> a
772	—	static ap_uint<8>	GFMul8	ap_uint<8> a
786	—	void	process	ap_uint<128> ciphertext, ap_uint<192> cipherkey, ap_uint<128>& plaintext
868	template <>	void	updateKey	ap_uint<128> cipherkey
895	template <>	static ap_uint<8>	GFMul2	ap_uint<8> a
907	—	static ap_uint<8>	GFMul4	ap_uint<8> a
921	—	static ap_uint<8>	GFMul8	ap_uint<8> a

Line	Template signature	Returns	Name	Parameters
935	—	void	process	ap_uint<128> ciphertext, ap_uint<128> cipher-key, ap_uint<128>& plaintext

4.1.3 Device Support

Device-sensitive. Two RESOURCE directives at lines 16 and 17 bind the substitution tables explicitly:

```
#pragma HLS resource variable = ssbox core = ROM_nP_LUTRAM
#pragma HLS resource variable = iibox core = ROM_nP_LUTRAM
```

This is the most consequential single decision in the file. The S-box is 256 bytes. Left to itself the tool would place it in block RAM, which has two ports; the round function wants many lookups per cycle. Forcing it into distributed LUT RAM makes the table replicable, so the unrolled round can read as many entries per cycle as it has copies. **The core will synthesise on a part without LUTRAM, but the round pipeline will not reach the intended throughput.**

4.1.4 Template Parameters and Supported Data Types

The key length is the template argument of the specialisations. The data types come from `xf_security/types.hpp`, which is a thin set of fixed-width definitions rather than a traits mechanism — the cipher has no intermediate precision to negotiate, because every value in AES is exactly eight or thirty-two bits by definition.

Table 30: Every class and struct in `Vitis_security/aes.hpp` (9 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
12	class	aesTable	—	—	—
55	class	aesEnc	template <int W>	—	—
62	class	aesEnc <256>	template <>	: public aesTable	—
241	class	aesEnc <192>	template <>	: public aesTable	—
383	class	aesEnc <128>	template <>	: public aesTable	—
516	class	aesDec	template <int W>	—	—
535	class	aesDec <256>	template <>	: public aesTable	—
698	class	aesDec <192>	template <>	: public aesTable	—
859	class	aesDec <128>	template <>	: public aesTable	—

4.1.5 Ports

L1; no ports. The block interface is by value and reference on `ap_uint` words.

Table 31: Include dependencies of `Vitis_security/aes.hpp`

Included file	Resolved to
<code>ap_int.h</code>	Vendor (Appendix E)
<code>hls_stream.h</code>	Vendor (Appendix E)
<code>iostream</code>	C++ standard library

4.1.6 Design Notes

Fifteen UNROLL against eight PIPELINE

More unrolling than pipelining is the signature of a round-based cipher: the byte-level operations inside one round are independent and are unrolled flat, while the rounds themselves are pipelined. The fourteen `INLINE` directives remove the call boundaries between the round steps so that the scheduler sees one flat region instead of four nested calls.

Six ARRAY_PARTITION

The state array and the round-key array are banked so that a whole column can be read per cycle. The partition factor and the unroll factor are a matched pair in exactly the same way as in §3.1.

Key schedule separated from the datapath

The key expansion is a distinct method. In hardware this becomes a separate block that runs once and writes a round-key memory, rather than logic sitting in the per-block critical path. This is the structural decision that makes the per-block latency independent of key length.

4.1.7 Code Example

```
#include "aes.hpp"

xf::security::aesEnc<128> cipher;      // key length fixed at elaboration
cipher.updateKey(key);                 // run once; writes the round-key memory
for (int i = 0; i < n_blocks; ++i)
    cipher.process(plain[i], key, cipher_out[i]);
```

4.1.8 Implementation on FPGA

Cost shape: area is set by the unroll factor of the round, not by the number of blocks; throughput is one block per (rounds / pipeline depth) cycles. Supporting a second key length costs a second full datapath. The S-box copies are the dominant LUT term once the round is unrolled.

Table 32: Every synthesis directive in `Vitis_security/aes.hpp` (45)

Line	Directive as written
15	<code>#pragma HLS inline</code>
16	<code>#pragma HLS resource variable = ssbox core = ROM_nP_LUTRAM</code>
17	<code>#pragma HLS resource variable = iibox core = ROM_nP_LUTRAM</code>
67	<code>#pragma HLS inline</code>
68	<code>#pragma HLS ARRAY_PARTITION variable = key_list complete dim = 1</code>
72	<code>#pragma HLS inline off</code>
78	<code>#pragma HLS pipeline II = 1</code>
79	<code>#pragma HLS pipeline II = 1</code>
138	<code>#pragma HLS unroll</code>
164	<code>#pragma HLS unroll</code>
172	<code>#pragma HLS unroll</code>
205	<code>#pragma HLS inline off</code>
213	<code>#pragma HLS pipeline II = 1</code>
246	<code>#pragma HLS inline</code>
247	<code>#pragma HLS ARRAY_PARTITION variable = key_list complete dim = 1</code>
251	<code>#pragma HLS inline off</code>
259	<code>#pragma HLS pipeline II = 1</code>

Line	Directive as written
315	<code>#pragma HLS unroll</code>
341	<code>#pragma HLS unroll</code>
349	<code>#pragma HLS unroll</code>
387	<code>#pragma HLS inline</code>
388	<code>#pragma HLS ARRAY_PARTITION variable = key_list complete dim = 1</code>
392	<code>#pragma HLS inline off</code>
397	<code>#pragma HLS pipeline II = 1</code>
442	<code>#pragma HLS unroll</code>
470	<code>#pragma HLS unroll</code>
477	<code>#pragma HLS unroll</code>
540	<code>#pragma HLS inline</code>
541	<code>#pragma HLS ARRAY_PARTITION variable = key_list complete dim = 1</code>
545	<code>#pragma HLS inline off</code>
551	<code>#pragma HLS pipeline II = 1</code>
658	<code>#pragma HLS unroll</code>
672	<code>#pragma HLS unroll</code>
703	<code>#pragma HLS inline</code>
704	<code>#pragma HLS ARRAY_PARTITION variable = key_list complete dim = 1</code>
708	<code>#pragma HLS inline off</code>
716	<code>#pragma HLS pipeline II = 1</code>
819	<code>#pragma HLS unroll</code>
833	<code>#pragma HLS unroll</code>
864	<code>#pragma HLS inline</code>
865	<code>#pragma HLS ARRAY_PARTITION variable = key_list complete dim = 1</code>
869	<code>#pragma HLS inline off</code>
874	<code>#pragma HLS pipeline II = 1</code>
968	<code>#pragma HLS unroll</code>
983	<code>#pragma HLS unroll</code>

4.1.9 Constraints

- Key length is fixed at elaboration.
- The ROM_nP_LUTRAM binding must be honoured by the target part for the intended throughput.
- `updateKey` must be called before the first block and after any key change; the core does not detect a stale schedule.
- The core implements the block cipher only. Modes of operation, padding and IV handling are the integrator's responsibility.

4.2 SHA-224 / SHA-256 Hash

Source file `Vitis_security/sha224_256.hpp` • **855 lines**, 35,288 B • **Level L1** (function templates over `hls::stream`)
 • **6 functions**, **3 classes**, **53 directives** • **Full source** Appendix D

4.2.1 Overview

Both digests from one body. The variant is selected by a specialised configuration struct rather than by a flag:

```
template <bool do_sha224> struct sha256_digest_config; // line 60
struct sha256_digest_config<true> { ... };           // line 64 -- SHA-224
struct sha256_digest_config<false> { ... };          // line 69 -- SHA-256
```

SHA-224 and SHA-256 differ only in their initial hash values and in how much of the final state is emitted. Putting that difference in a specialised struct means the compression function exists once in the source *and* once in the hardware, with no run-time selection.

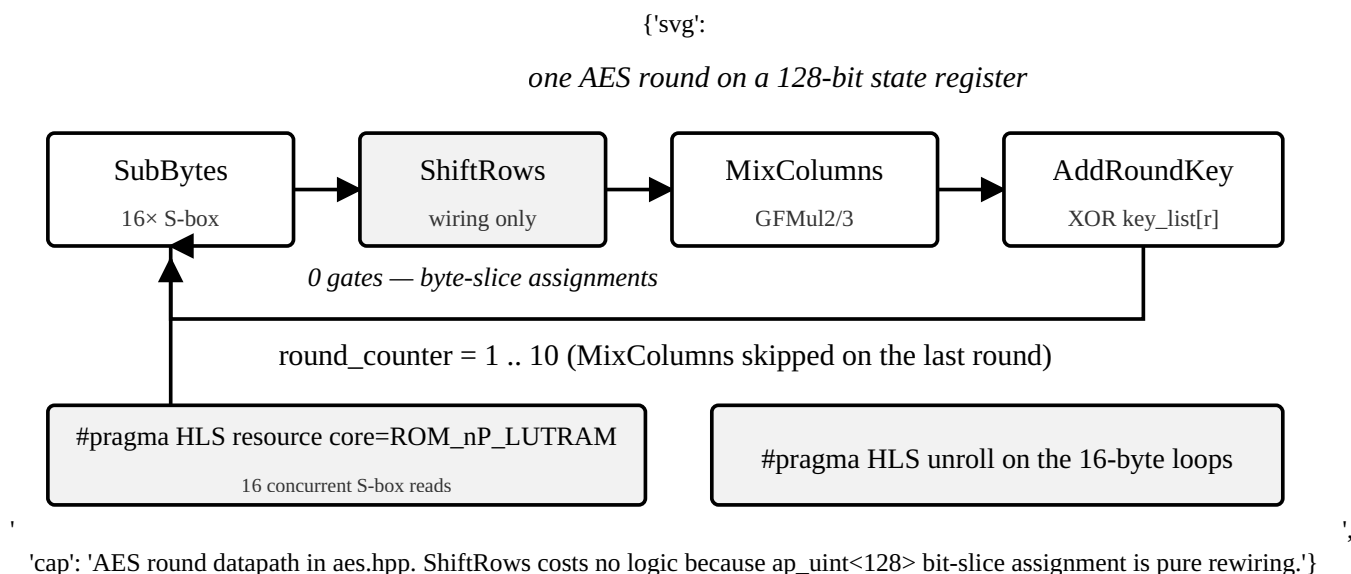


Figure 11: SHA-256 dataflow region. Padding, schedule expansion and compression run on different blocks concurrently; the eight STREAM directives create the channels that make this legal.

4.2.2 Entry Point and Access Functions

Two public entry points, both streaming:

```
template <int m_width>
void sha224(hls::stream<ap_uint<m_width> >& msg_strm, ...); // line 810
template <int m_width>
void sha256(hls::stream<ap_uint<m_width> >& msg_strm, ...); // line 827
```

Both delegate to `sha256Digest` (line 610), which is the dataflow region.

Table 33: Every function definition in `Vitis_security/sha224_256.hpp` (6 definitions)

Line	Template signature	Returns	Name	Parameters
498	—	in-line void	<code>dup_strm</code>	<code>hls::stream<uint64_t>& in_strm, hls::stream<bool>& in_e_strm, hls::stream<uint64_t>& out1_strm, hls::stream<bool>& out1_e_strm, hls::stream<uint64_t>& out2_strm, hls::stream<bool>& out2_e_strm</code>

Line	Template signature	Returns	Name	Parameters
523	—	in-line void	generateMsgSchedule	hls::stream<SHA256Block>& blk_strm, hls::stream<uint64_t>& nblk_strm, hls::stream<bool>& end_nblk_strm, hls::stream<uint32_t>& w_strm
609	template <int h_width>	void	sha256Digest	hls::stream<uint64_t>& nblk_strm, hls::stream<bool>& end_nblk_strm, hls::stream<uint32_t>& w_strm, hls::stream<ap_uint<h_width>>& hash_strm, hls::stream<bool>& end_hash_strm
746	template <int m_width, int h_width>	in-line void	sha256_top	hls::stream<ap_uint<m_width>>& msg_strm, hls::stream<ap_uint<64>>& len_strm, hls::stream<bool>& end_len_strm, hls::stream<ap_uint<h_width>>& hash_strm, hls::stream<bool>& end_hash_strm
809	template <int m_width>	void	sha224	hls::stream<ap_uint<m_width>>& msg_strm, hls::stream<ap_uint<64>>& len_strm, hls::stream<bool>& end_len_strm, hls::stream<ap_uint<224>>& hash_strm, hls::stream<bool>& end_hash_strm
826	template <int m_width>	void	sha256	hls::stream<ap_uint<m_width>>& msg_strm, hls::stream<ap_uint<64>>& len_strm, hls::stream<bool>& end_len_strm, hls::stream<ap_uint<256>>& hash_strm, hls::stream<bool>& end_hash_strm

4.2.3 Device Support

Eight RESOURCE directives bind the round-constant table and the message-schedule storage. As with AES, the core synthesises without the intended primitives but not at the intended rate.

4.2.4 Template Parameters and Supported Data Types

m_width parameterises the width of the input stream word, so the same core serves an 8-bit byte stream and a 512-bit bus without changing the compression function. The config struct carries the variant.

Table 34: Every class and struct in Vitis_security/sha224_256.hpp (3 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
55	struct	SHA256Block	—	—	—
64	struct	sha256_digest_config <true>	template <>	—	—
69	struct	sha256_digest_config <false>	template <>	—	—

4.2.5 Ports

L1; the interface is entirely hls::stream. The eight STREAM directives force the internal arrays between the dataflow stages to be FIFOs; those FIFOs are the channels the DATAFLOW directive at line 752 needs in order to run the stages concurrently.

Table 35: Include dependencies of Vitis_security/sha224_256.hpp

Included file	Resolved to
ap_int.h	Vendor (Appendix E)
cstdio	C++ standard library

Included file	Resolved to
hls_stream.h	Vendor (Appendix E)
xf_security/types.hpp	Corpus (Appendix D)
xf_security/utils.hpp	Corpus (Appendix D)

4.2.6 Design Notes

One DATAFLOW directive is the whole architecture

Line 752 is the single most structurally significant line in the file. Without it, message padding, message-schedule expansion and compression run in sequence and the core processes one block at a time. With it, the three run concurrently on different blocks, and the throughput becomes that of the slowest stage rather than the sum of all three. Everything else in the file — the eight STREAM directives, the thirteen ARRAY_PARTITION directives, the two LATENCY constraints — exists to make that one directive legal and balanced.

Thirteen ARRAY_PARTITION: the highest density in the corpus

The message schedule is sixty-four 32-bit words and the compression function reads several of them per round. Banking is what makes the round pipeline's II achievable.

Four LOOP_TRIPCOUNT

The message length is a run-time property, so the block loop is unbounded at compile time. The four directives supply bounds for the latency report only.

4.2.7 Code Example

```
#include "sha224_256.hpp"

hls::stream<ap_uint<32> > msg;      // 32-bit words
hls::stream<ap_uint<64> > len;     // message length in bits
hls::stream<ap_uint<256> > digest;
hls::stream<bool> e_digest;

xf::security::sha256<32>(msg, len, nstrm, digest, e_digest);
```

4.2.8 Implementation on FPGA

Cost shape: area is constant in message length — it is one compression datapath plus the schedule memory. Latency grows linearly in the number of 512-bit blocks. Throughput after the dataflow split is one block per compression-pipeline interval, independent of the padding and expansion cost.

Table 36: Every synthesis directive in Vitis_security/sha224_256.hpp (53)

Line	Directive as written
102	#pragma HLS pipeline II = 16
103	#pragma HLS loop_tripcount min = 0 max = 1
106	#pragma HLS array_partition variable = b0.M complete
110	#pragma HLS unroll
134	#pragma HLS array_partition variable = b.M complete
141	#pragma HLS unroll
157	#pragma HLS array_partition variable = b.M complete
161	#pragma HLS pipeline
210	#pragma HLS array_partition variable = b.M complete
214	#pragma HLS unroll
255	#pragma HLS array_partition variable = b1.M complete

Line	Directive as written
258	#pragma HLS unroll
306	#pragma HLS pipeline II = 16
307	#pragma HLS loop_tripcount min = 0 max = 1
310	#pragma HLS array_partition variable = b0.M complete
315	#pragma HLS unroll
348	#pragma HLS array_partition variable = b.M complete
355	#pragma HLS unroll
372	#pragma HLS array_partition variable = b.M complete
376	#pragma HLS pipeline
476	#pragma HLS array_partition variable = b1.M complete
479	#pragma HLS unroll
507	#pragma HLS loop_tripcount min = 1 max = 1 avg = 1
508	#pragma HLS pipeline II = 1
531	#pragma HLS latency max = 65
534	#pragma HLS array_partition variable = blk.M complete
538	#pragma HLS array_partition variable = W complete
542	#pragma HLS pipeline II = 1
550	#pragma HLS pipeline II = 1
629	#pragma HLS array_partition variable = K complete
639	#pragma HLS array_partition variable = H complete
664	#pragma HLS loop_tripcount min = 1 max = 1
665	#pragma HLS latency max = 65
683	#pragma HLS pipeline II = 1
703	#pragma HLS unroll
719	#pragma HLS unroll
752	#pragma HLS DATAFLOW
755	#pragma HLS STREAM variable = blk_strm depth = 32
756	#pragma HLS RESOURCE variable = blk_strm core = FIFO_LUTRAM
760	#pragma HLS STREAM variable = nblk_strm depth = 32
761	#pragma HLS RESOURCE variable = nblk_strm core = FIFO_LUTRAM
763	#pragma HLS STREAM variable = nblk_strm1 depth = 32
764	#pragma HLS RESOURCE variable = nblk_strm1 core = FIFO_LUTRAM
766	#pragma HLS STREAM variable = nblk_strm2 depth = 32
767	#pragma HLS RESOURCE variable = nblk_strm2 core = FIFO_LUTRAM
771	#pragma HLS STREAM variable = end_nblk_strm depth = 32
772	#pragma HLS RESOURCE variable = end_nblk_strm core = FIFO_LUTRAM
774	#pragma HLS STREAM variable = end_nblk_strm1 depth = 32
775	#pragma HLS RESOURCE variable = end_nblk_strm1 core = FIFO_LUTRAM

Line	Directive as written
777	<code>#pragma HLS STREAM variable = end_nblk_strm2 depth = 32</code>
778	<code>#pragma HLS RESOURCE variable = end_nblk_strm2 core = FIFO_LUTRAM</code>
782	<code>#pragma HLS STREAM variable = w_strm depth = 32</code>
783	<code>#pragma HLS RESOURCE variable = w_strm core = FIFO_LUTRAM</code>

4.2.9 Constraints

- The message length must be supplied on its own stream; the core does not infer the end of the message from the data stream.
- The DATAFLOW region requires that each internal stream be written by exactly one stage and read by exactly one stage. Any modification that introduces a second reader breaks the region, and the tool reports it as a dataflow violation rather than as a functional error.
- LOOP_TRIPCOUNT bounds are for reporting only.

4.3 Shared Security Utilities

Vitis_security/xf_security/types.hpp

Fixed-width integer definitions used by both cores. No directives: this file defines names, not hardware. 59 lines; full source in Appendix D.

Table 37: Include dependencies of Vitis_security/xf_security/types.hpp

Included file	Resolved to
cstdint	C++ standard library
stdint.h	C++ standard library

Vitis_security/xf_security/utls.hpp

Small helpers shared across the security library. 45 lines; full source in Appendix D.

Table 38: Include dependencies of Vitis_security/xf_security/utls.hpp

Included file	Resolved to
cassert	C++ standard library

5 Quantised Neural Network Cores (FINN)

16 files, 6,544 lines. This library is structured differently from the other three and the difference is the point of the chapter. The solver and security cores are *cores*: an integrator calls one. FINN is a **construction kit**: a generator emits a top-level function that instantiates these templates in a particular arrangement, and the library's job is to make every arrangement schedulable. That is why the file that dominates the library is not a compute kernel but the sliding-window generator, and why the library's highest directive count is DEPENDENCE rather than PIPELINE.

Table 39: The FINN files by role

Role	Files	Lines
Data movement (window generation, stream re-shaping, DMA)	slidingwindow.h, streamtools.h, dma.h	3,344
Compute (matrix–vector, vector–vector, MAC, pooling, upsample)	mvau.hpp, vvau.hpp, mac.hpp, maxpool.h, upsample.hpp	1,588
Type and parameter carriers	weights.hpp, activations.hpp, interpret.hpp, utils.hpp, mmv.hpp	1,034
Composition and reliability	bnn-library.h, convlayer.h, tmrcheck.hpp	578

5.0 The include tree, measured

bnn-library.h is the single header a generated top level includes. It is 61 lines and contains no code. What matters to an integrator is which of the other fifteen headers it actually reaches, and that question cannot be answered by reading its #include list, because most of the tree arrives transitively.

An earlier draft answered it by reading the list, and was wrong. It stated that three headers were unreachable. Running the preprocessor with clang -H showed that mvau.hpp arrives through convlayer.h and interpret.hpp arrives through maxpool.h. Only activations.hpp is genuinely not reached, so a top level that uses a ThresholdsActivation must include it explicitly:

```
#include "bnn-library.h"    // reaches 15 of the 16 headers transitively
#include "activations.hpp"  // the one it does not reach -- verified with clang -H
```

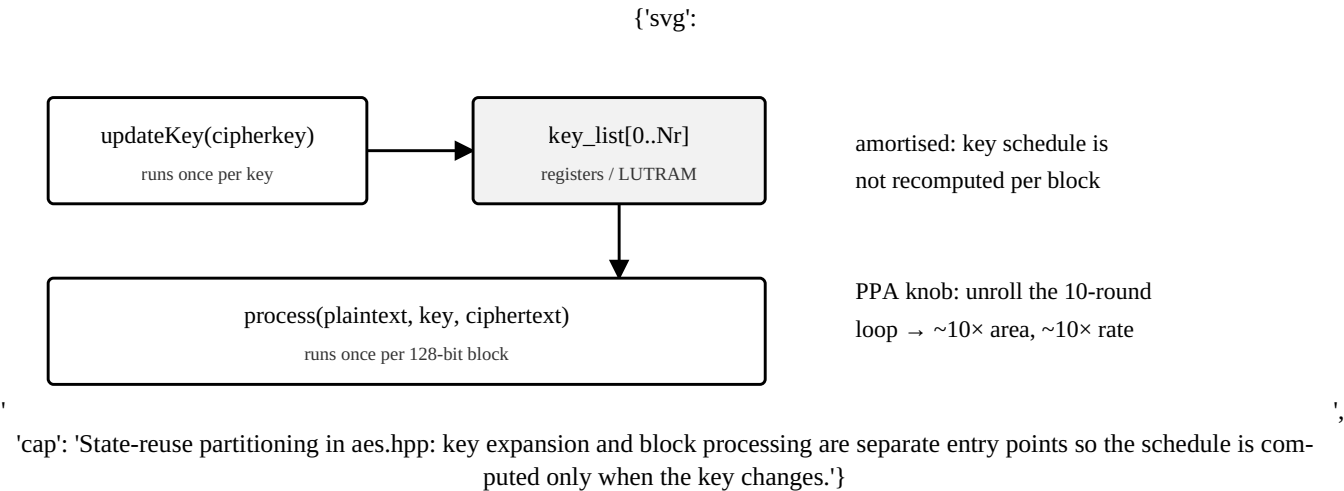


Figure 12: The FINN include tree as reported by the preprocessor, not as read from the include lists. Solid edges are direct includes; dashed edges are transitive arrivals.

5.1 Convolution Input Generator (Sliding Window)

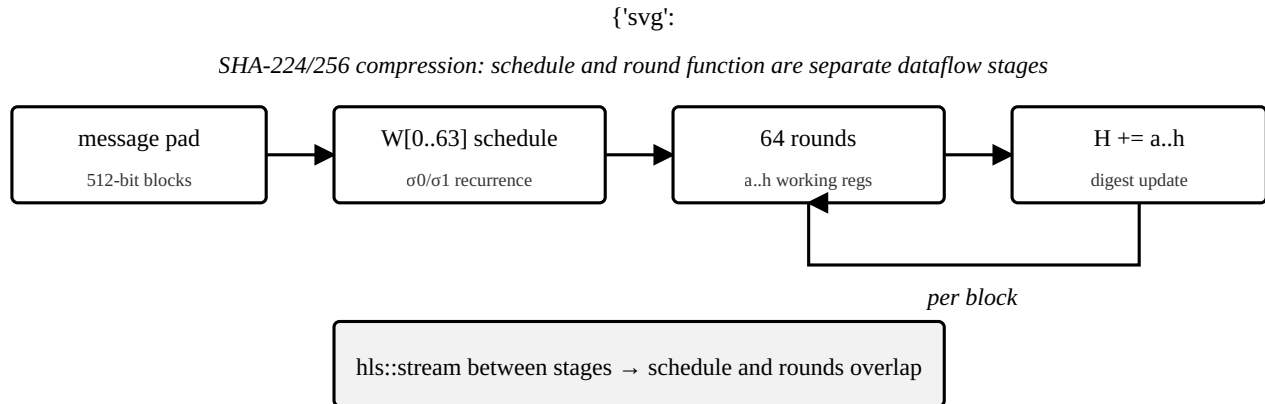
Source file `finn_hlslib/slidingwindow.h` • **2,096 lines**, 87,820 B • **Level L1** (function templates over `hls::stream`)
 • **20 functions**, **0 classes**, **66 directives** • **Full source** Appendix D

5.1.1 Overview

At 2,096 lines this is the largest file in the corpus, and it computes nothing. It turns a raster stream of pixels into a stream of convolution windows. It is large because it is **ten separate implementations of that one job**, each specialised for a case where a general implementation would cost too much:

<code>ConvolutionInputGenerator</code>	line 167	general case
<code>ConvolutionInputGenerator_MMV</code>	line 296	multiple pixels per cycle
<code>ConvolutionInputGenerator_kernel_stride</code>	line 449	stride != 1
<code>ConvolutionInputGenerator_kernel_stride_MMV</code>	line 597	
<code>ConvolutionInputGenerator_dws</code>	line 757	depthwise separable
<code>ConvolutionInputGenerator_kernel_stride_dws</code>	line 886	
<code>ConvolutionInputGenerator_dws_MMV</code>	line 1036	
<code>ConvolutionInputGenerator_2D_kernel1</code>	line 1182	1x1 kernel, 2D
<code>ConvolutionInputGenerator_1D_kernel1</code>	line 1232	1x1 kernel, 1D
<code>ConvolutionInputGenerator_NonSquare</code>	line 1290	non-square kernel

This is the central lesson of the FINN library and the strongest single argument for HLS over hand-written RTL in this domain. Each of these ten is a different line buffer with a different address generator. In RTL they would be ten modules, ten testbenches and ten maintenance burdens. Here they are ten function templates over a shared type system, and the generator picks one. The cost of that convenience is visible in the next paragraph.



53 HLS pragmas in 854 lines — the densest pragma ratio in the corpus

'cap': 'Structure of sha224_256.hpp. Message expansion and the round loop are separate stream-connected stages so that they execute concurrently.'

Figure 13: Line-buffer structure of the sliding-window generator. The write pointer and the window read pointers are disjoint only because the buffer is at least $K-1$ rows deep -- the fact the twenty DEPENDENCE assertions encode.

5.1.2 Entry Point and Access Functions

Ten entry points, listed above and enumerated in full below. The generator selects one by name; there is no dispatch at run time.

Table 40: Every function definition in `finn_hlslib/slidingwindow.h` (20 definitions)

Line	Template signature	Returns	Name	Parameters
72	template<typename T>	void	memory_resource	T inputBuf, ap_resource_dflt const&
93	template<typename T>	void	memory_resource	T inputBuf, ap_resource_bram const&
114	template<typename T>	void	memory_resource	T inputBuf, ap_resource_uram const&
135	template<typename T>	void	memory_resource	T inputBuf, ap_resource_lutram const&
159	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
287	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_MMV	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
441	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_or_kernel_stride	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r

Line	Template signature	Returns	Name	Parameters
588	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_kernel_stride_MMV	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
749	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_dws	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
878	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_kernel_stride_dws	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1027	—	template<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_dws_MMV	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1177	—	template< unsigned int IFMChannels, unsigned int Input_precision, unsigned int IFMDim, unsigned int	ConvolutionInputGenerator_2D_kernel1	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps
1227	—	template< unsigned int IFMChannels, unsigned int Input_precision, unsigned int IFMDim, unsigned int	ConvolutionInputGenerator_1D_kernel1	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps

Line	Template signature	Returns	Name	Parameters
1278	—	template<unsigned int ConvKernelDim_x, unsigned int ConvKernelDim_y, unsigned int IFMChann	ConvolutionInputGenerator_NonSquare	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1412	—	template<unsigned int ConvKernelDim_x, unsigned int ConvKernelDim_y, unsigned int IFMChann	ConvolutionInputGenerator_NonSquare_dws	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1549	—	template<unsigned int ConvKernelDim_x, unsigned int ConvKernelDim_y, unsigned int IFMChann	ConvolutionInputGenerator_NonSquare_Dilated	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1796	—	template<unsigned int ConvKernelDim_x, unsigned int IFMChannels, unsigned int Input_precision	ConvolutionInputGenerator_1D_dws_naive	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1872	—	template< unsigned ConvKernelDim_x, unsigned IFMChannels, unsigned Input_precision, unsign	ConvolutionInputGenerator_1D	hls::stream<ap_uint<SIMD*Input_precision>> &in, hls::stream<ap_uint<SIMD*Input_precision>> &out, unsigned const numReps, R const &r
1950	—	template< unsigned ConvKernelDim_x, unsigned IFMChannels, unsigned Input_precision, unsign	ConvolutionInputGenerator_1D_dws	hls::stream<ap_uint<SIMD*Input_precision>> &in, hls::stream<ap_uint<SIMD*Input_precision>> &out, unsigned const numReps, R const &r

Line	Template signature	Returns	Name	Parameters
2027	—	template< unsigned ConvKernelDim_x, unsigned IFMChannels, unsigned Input_precision, unsigned	ConvolutionInputGenerator_1D_dws_stride	hls::stream<ap_uint<SIMD*Input_precision>> &in, hls::stream<ap_uint<SIMD*Input_precision>> &out, unsigned const numReps, R const &r

5.1.3 Device Support

Four BIND_STORAGE directives place the line buffers. The line buffer is the core's entire state, and whether it lands in block RAM or distributed RAM decides both the area and the achievable II.

5.1.4 Template Parameters and Supported Data Types

Kernel dimension, stride, input feature-map dimension, number of channels, SIMD width and the MMV factor are all template parameters. Every one is a compile-time constant, which is what allows the address generation to collapse into constants and counters rather than multipliers.

5.1.5 Ports

L1; input and output are `hls::stream`. The output stream carries windows in the order the downstream matrix-vector unit consumes them, so the two are tightly coupled by convention rather than by interface.

Table 41: Include dependencies of `finn_hlslib/slidingwindow.h`

Included file	Resolved to
algorithm	C++ standard library
utils.hpp	Corpus (Appendix D)

5.1.6 Design Notes

*Twenty **DEPENDENCE** assertions — the highest count in the corpus*

This file contains 20 of the corpus's 30 **DEPENDENCE** directives: two thirds of all the assertions in the entire corpus are in this one file.

The reason is structural. A line buffer is written by the input side and read by the window side in the same iteration, at addresses that a compiler cannot prove disjoint because they are computed from modular counters. The author knows they are disjoint — it follows from the buffer being at least as deep as the kernel height — and asserts it.

What this means for an integrator. These twenty assertions are the load-bearing correctness argument of the largest file in the corpus, and they are **conditional on the template parameters**. A parameter combination in which the line buffer is not deep enough for the kernel makes the assertions false, the tool still believes them, and the generated hardware reads stale pixels. There is no error message, no simulation failure in C++, and no assertion in the RTL. Any change to the parameter ranges must be checked against the buffer-depth argument in the source, which is Appendix D.

*One **PERFORMANCE** directive*

The only **PERFORMANCE** directive in the corpus is here. It states a target throughput rather than an initiation interval, letting the tool choose the II that achieves it.

5.1.7 Code Example

```
#include "bnn-library.h"
```

```
// 3x3 kernel, stride 1, 32x32 input, 64 channels, SIMD 8
hls::stream<ap_uint<8*8> > in, windows;
ConvolutionInputGenerator<3, 64, 8, 32, 8, 1>(in, windows, 1, ap_resource_dsp());
```

5.1.8 Implementation on FPGA

Cost shape: the line buffer holds $(K-1) \times W \times C$ elements and that term dominates the area. Compute is address generation only. The MMV variants multiply the read ports by the MMV factor, which is why they are separate functions rather than a parameter on the general one.

Table 42: Every synthesis directive in `finn_hlslib/slidingwindow.h` (66)

Line	Directive as written
74	<code>#pragma HLS BIND_STORAGE variable=inputBuf type=RAM_2P</code>
95	<code>#pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P impl=BRAM</code>
116	<code>#pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P impl=URAM</code>
137	<code>#pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P impl=LUTRAM</code>
177	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1</code>
192	<code>#pragma HLS pipeline style=flp II=1</code>
308	<code>#pragma HLS DEPENDENCE variable=inputBuf inter false</code>
309	<code>#pragma HLS DEPENDENCE variable=inputBuf intra false</code>
310	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1</code>
311	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=2</code>
326	<code>#pragma HLS pipeline style=flp II=1</code>
333	<code>#pragma HLS UNROLL</code>
359	<code>#pragma HLS UNROLL</code>
392	<code>#pragma HLS UNROLL</code>
466	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1</code>
467	<code>#pragma HLS DEPENDENCE variable=inputBuf inter false</code>
468	<code>#pragma HLS DEPENDENCE variable=inputBuf intra false</code>
476	<code>#pragma HLS DEPENDENCE variable=current_block_write intra false</code>
479	<code>#pragma HLS pipeline style=flp II=1</code>
544	<code>#pragma HLS DEPENDENCE variable=inputBuf inter false</code>
545	<code>#pragma HLS DEPENDENCE variable=inputBuf intra false</code>
554	<code>#pragma HLS DEPENDENCE variable=current_block_write intra false</code>
610	<code>#pragma HLS DEPENDENCE variable=inputBuf inter false</code>
611	<code>#pragma HLS DEPENDENCE variable=inputBuf intra false</code>
612	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1</code>
613	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=2</code>
628	<code>#pragma HLS DEPENDENCE variable=current_block_write intra false</code>
631	<code>#pragma HLS pipeline style=flp II=1</code>
638	<code>#pragma HLS UNROLL</code>
672	<code>#pragma HLS UNROLL</code>
705	<code>#pragma HLS UNROLL</code>

Line	Directive as written
716	#pragma HLS DEPENDENCE variable=current_block_write intra false
767	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
782	#pragma HLS pipeline style=flp II=1
903	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
904	#pragma HLS DEPENDENCE variable=inputBuf inter false
905	#pragma HLS DEPENDENCE variable=inputBuf intra false
915	#pragma HLS pipeline style=flp II=1
917	#pragma HLS DEPENDENCE variable=current_block_write intra false
983	#pragma HLS DEPENDENCE variable=inputBuf inter false
984	#pragma HLS DEPENDENCE variable=inputBuf intra false
993	#pragma HLS DEPENDENCE variable=current_block_write intra false
1048	#pragma HLS DEPENDENCE variable=inputBuf inter false
1049	#pragma HLS DEPENDENCE variable=inputBuf intra false
1050	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1051	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=2
1066	#pragma HLS pipeline style=flp II=1
1073	#pragma HLS UNROLL
1099	#pragma HLS UNROLL
1132	#pragma HLS UNROLL
1191	#pragma HLS performance target_ti=IFMDim*IFMDim*IFMChannels/SIMD
1198	#pragma HLS pipeline style=flp II=IFMChannels/SIMD
1245	#pragma HLS pipeline style=flp II=1
1300	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1315	#pragma HLS pipeline style=flp II=1
1434	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1449	#pragma HLS pipeline style=flp II=1
1575	#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1590	#pragma HLS pipeline style=flp II=1
1710	#pragma HLS ARRAY_PARTITION variable=inputBuf complete
1718	#pragma HLS pipeline style=flp II=1
1739	#pragma HLS UNROLL
1824	#pragma HLS pipeline style=flp II=1
1908	#pragma HLS pipeline style=flp II=1
1984	#pragma HLS pipeline style=flp II=1
2063	#pragma HLS pipeline style=flp II=1

5.1.9 Constraints

- **The twenty dependence assertions must hold for the chosen parameters.** This is the core's principal constraint and it is not machine-checked.

- The correct variant must be selected for the stride and the depthwise case; using the general variant where a stride variant is intended is functionally correct but much larger.
- The output window order is a convention shared with the consumer, not a declared interface.

5.2 Matrix–Vector Activate Unit (MVAU)

Source file `finn_hlslib/mvau.hpp` • 311 lines, 11,892 B • Level L1 (function templates over `hls::stream`) • 2 functions, 0 classes, 16 directives • Full source Appendix D

5.2.1 Overview

The compute core of a FINN fully-connected or convolutional layer. Two entry points:

```
template<...> void Matrix_Vector_Activate_Batch(...); // line 93
template<...> void Matrix_Vector_Activate_Stream_Batch(...); // line 215
```

The difference is where the weights live. The first takes a weight object held in on-chip memory, instantiated once at elaboration. The second takes weights from a stream, so they can be refreshed at run time. **An integrator choosing between them is choosing between a layer whose weights are fixed in the bitstream and one whose weights cost bandwidth on every inference,** and that decision dominates the system architecture far more than anything inside the layer.

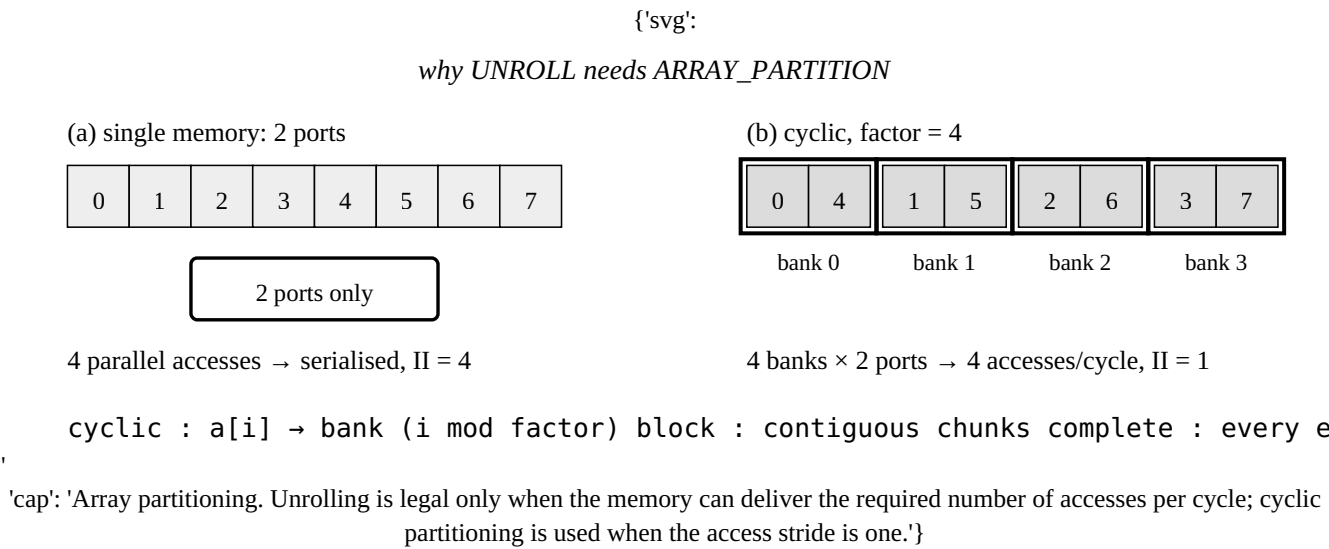


Figure 14: MVAU folding. PE and SIMD select a tile of the weight matrix per cycle; the weight storage is unchanged by the choice, so folding trades cycles against multipliers only.

5.2.2 Entry Point and Access Functions

Two, as above. Both are batch forms: they process many input vectors per call, which is what amortises the pipeline fill.

Table 43: Every function definition in `finn_hlslib/mvau.hpp` (2 definitions)

Line	Tem-plate signa-ture	Re-turms	Name	Parameters
91	—	type-name TI, type-name TO, type-name TW, type-name TA, type-name R > void	Matrix_Vector_Activate_Batch	hls::stream<TI> &in, hls::stream<TO> &out, TW const &weights, TA const &activation, int const reps, R const &r
213	—	type-name TI, type-name TO, type-name TA, type-name R > void	Mat-rix_Vector_Activate_Stream_Batch	hls::stream<TI> &in, hls::stream<TO> &out, hls::stream<ap_uint<PE*SIMD*TW::width>> &weight, TA const &activation, int const reps, R const &r

5.2.3 Device Support

Device choice enters through `mac.hpp` (§5.5), which binds the multiply to fabric or to a DSP primitive.

5.2.4 Template Parameters and Supported Data Types

The folding factors PE and SIMD are the two parameters that matter. PE is how many output channels are computed in parallel and SIMD is how many input channels are consumed per cycle. The layer's matrix is $MW \times MH$, and the number of cycles per vector is $(MW/SIMD) \times (MH/PE)$. **Both must divide exactly**; the library does not pad.

This is the throughput knob for the whole network. The FINN generator sets PE and SIMD per layer so that every layer takes the same number of cycles, because a chain of streaming layers runs at the rate of its slowest member. Changing one layer's folding in isolation therefore buys nothing.

5.2.5 Ports

L1; streams in and out, plus the weight object or weight stream.

Table 44: Include dependencies of `finn_hlslib/mvau.hpp`

Included file	Resolved to
<code>hls_stream.h</code>	Vendor (Appendix E)
<code>interpret.hpp</code>	Corpus (Appendix D)

Included file	Resolved to
mac.hpp	Corpus (Appendix D)
weights.hpp	Corpus (Appendix D)

5.2.6 Design Notes

Nine UNROLL, five ARRAY_PARTITION, two PIPELINE

The unrolls flatten the PE and SIMD loops so that $PE \times SIMD$ multiply-accumulate operations are issued per cycle; the partitions bank the weight memory so those operations can actually get their operands. The two are a matched pair: an unroll without the matching partition produces a design the tool cannot schedule at the requested II, and the failure appears as a missed II in the report rather than as an error.

5.2.7 Code Example

```
#include "bnn-library.h"
#include "activations.hpp"

static FixedPointWeights<SIMD_, ap_int<WBITS>, PE_, (MW/SIMD_)*(MH/PE_)> weights;
static ThresholdsActivation<(MH/PE_), PE_, 1, ap_int<16>, ap_uint<1> > thresh;

Matrix_Vector_Activate_Batch<MW, MH, SIMD_, PE_, 1,
    Slice<ap_uint<IBITS> >, Slice<ap_uint<1> >, Identity>
    (in, out, weights, thresh, reps, ap_resource_dsp());
```

5.2.8 Implementation on FPGA

Cost shape: $PE \times SIMD$ multipliers, and weight storage of $MW \times MH \times W$ bits regardless of folding. Folding trades cycles for multipliers and does not change the weight storage at all — which is why the memory, not the arithmetic, is usually what limits how large a network fits.

Table 45: Every synthesis directive in `finn_hlslib/mvau.hpp` (16)

Line	Directive as written
110	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=0</code>
114	<code>#pragma HLS ARRAY_PARTITION variable=accu complete dim=0</code>
124	<code>#pragma HLS pipeline style=flp II=1</code>
140	<code>#pragma HLS UNROLL</code>
142	<code>#pragma HLS UNROLL</code>
151	<code>#pragma HLS UNROLL</code>
165	<code>#pragma HLS UNROLL</code>
167	<code>#pragma HLS UNROLL</code>
232	<code>#pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1</code>
235	<code>#pragma HLS ARRAY_PARTITION variable=accu complete dim=0</code>
238	<code>#pragma HLS ARRAY_PARTITION variable=w.m_weights complete dim=0</code>
250	<code>#pragma HLS pipeline style=flp II=1</code>
267	<code>#pragma HLS UNROLL</code>
274	<code>#pragma HLS UNROLL</code>
281	<code>#pragma HLS UNROLL</code>
294	<code>#pragma HLS UNROLL</code>

5.2.9 Constraints

- SIMD must divide MW and PE must divide MH , exactly.
- The weight object's fourth template argument must equal $(MW/SIMD) \times (MH/PE)$; an inconsistent value compiles and addresses out of range.
- Folding must be set consistently across the whole layer chain.

5.3 Vector–Vector Activate Unit (VVAU)

Source file `finn_hlslib/vvau.hpp` • 277 lines, 11,276 B • **Level L1** • 2 functions, 0 classes, 14 directives • **Full source** Appendix D

5.3.1 Overview

The depthwise counterpart of §5.2. Where MVAU computes a full matrix–vector product, VVAU computes one multiply per channel, which is what a depthwise-separable convolution needs. It carries the same two entry-point forms, `Vector_Vector_Activate_Batch` (line 85) and the stream-weight form (line 194), and the same PE folding parameter — but no SIMD, because there is no reduction across input channels to parallelise.

5.3.2 Entry Point and Access Functions

Two, mirroring §5.2.

Table 46: Every function definition in `finn_hlslib/vvau.hpp` (2 definitions)

Line	Tem-plate signa-ture	Re-turms	Name	Parameters
83	—	type-name TI, type-name TO, type-name TW, type-name TA, type-name R > void	<code>Vector_Vector_Activate_Batch</code>	<code>hls::stream<TI> &in, hls::stream<TO> &out, TW const &weights, TA const &activation, int const reps, R const &r</code>
192	—	type-name TI, type-name TO, type-name TA, type-name R > void	<code>Vec-tor_Vector_Activate_Stream_Batch</code>	<code>hls::stream<TI> &in, hls::stream<TO> &out, hls::stream<ap_uint<PE*SIMD*TW::width>> &weights, TA const &activation, int const reps, R const &r</code>

5.3.3 Device Support

As §5.2; the multiply binding comes from `mac.hpp`.

5.3.4 Template Parameters and Supported Data Types

PE only. The absence of SIMD is the structural difference from MVAU and is why a depthwise layer is usually the throughput bottleneck of a network that mixes the two.

5.3.5 Ports

L1; streams.

Table 47: Include dependencies of `finn_hlslib/vvau.hpp`

Included file	Resolved to
<code>hls_stream.h</code>	Vendor (Appendix E)
<code>interpret.hpp</code>	Corpus (Appendix D)
<code>mac.hpp</code>	Corpus (Appendix D)

5.3.6 Design Notes

Nine UNROLL and three ARRAY_PARTITION, the same matched pattern as §5.2 at smaller scale.

5.3.7 Code Example

```
Vector_Vector_Activate_Batch<Channels, Kernel, PE, 1,
    Slice<ap_uint<IBITS> >, Slice<ap_uint<OBITS> >, Identity>
(in, out, weights, thresh, reps, ap_resource_dsp());
```

5.3.8 Implementation on FPGA

Cost shape: PE multipliers; weight storage $C \times K^2 \times W$ bits. Far smaller than MVAU for the same feature-map size, which is the entire reason depthwise layers exist.

Table 48: Every synthesis directive in `finn_hlslib/vvau.hpp` (14)

Line	Directive as written
103	<code>#pragma HLS ARRAY_PARTITION variable=accu complete dim=0</code>
112	<code>#pragma HLS pipeline style=flp II=1</code>
119	<code>#pragma HLS UNROLL</code>
128	<code>#pragma HLS UNROLL</code>
142	<code>#pragma HLS UNROLL</code>
144	<code>#pragma HLS UNROLL</code>
213	<code>#pragma HLS ARRAY_PARTITION variable=accu complete dim=0</code>
223	<code>#pragma HLS pipeline style=flp II=1</code>
230	<code>#pragma HLS UNROLL</code>
239	<code>#pragma HLS ARRAY_PARTITION variable=w.m_weights complete dim=0</code>
241	<code>#pragma HLS UNROLL</code>
246	<code>#pragma HLS UNROLL</code>
260	<code>#pragma HLS UNROLL</code>
262	<code>#pragma HLS UNROLL</code>

5.3.9 Constraints

PE must divide the channel count exactly.

5.4 Max Pooling

Source file `finn_hlslib/maxpool.h` • 610 lines, 23,874 B • Level L1 • 10 functions, 0 classes, 35 directives • Full source Appendix D

5.4.1 Overview

Pooling in several forms, including the binary case where a max reduces to an OR. Ten function definitions cover the 2D, 1D, streaming and binary variants.

5.4.2 Entry Point and Access Functions

Ten definitions, enumerated below.

Table 49: Every function definition in `finn_hlslib/maxpool.h` (10 definitions)

Line	Template signature	Returns	Name	Parameters
67	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels>	void	StreamingMax-Pool	hls::stream<ap_uint<NumChannels> > & in, hls::stream<ap_uint<NumChannels> > & out
113	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels>	void	StreamingMax-Pool_Batch	hls::stream<ap_uint<NumChannels> > & in, hls::stream<ap_uint<NumChannels> > & out, unsigned int numReps
138	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, typename Act	StreamingMax-Pool_Precision	hls::stream<ap_uint<StreamW> > & in, hls::stream<ap_uint<StreamW> > & out
205	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, typename Act	StreamingMax-Pool_Precision_Batch	hls::stream<ap_uint<InStreamW> > & in, hls::stream<ap_uint<OutStreamW> > & out, unsigned int numReps
244	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, unsigned int	StreamingMax-Pool_Precision_1d	hls::stream<ap_uint<PE*ActType::width> > & in, hls::stream<ap_uint<PE*ActType::width> > & out

Line	Template signature	Returns	Name	Parameters
326	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, unsigned int	StreamingMax-Pool_Precision_Batch_1d	hls::stream<ap_uint<PE*ActType::width>> & in, hls::stream<ap_uint<PE*ActType::width>> & out, unsigned int numReps
360	template< unsigned int ImgDim, unsigned int NumChannels, typename ActType, unsigned int PEOunt, int offset = 0>	void	ReLU_Batch	hls::stream<ap_uint<PECount * ActType::width>> & in, hls::stream<ap_uint<PECount * ActType::width>> & out, const unsigned int numReps
405	—	template< unsigned int ImgDim, unsigned int NumChannels, typename ActType, unsigned int PE	AccPool_Batch	hls::stream<ap_uint<PECount * ActType::width>> & in, hls::stream<ap_uint<PECount * AccType::width>> & out, const unsigned int numReps
459	—	template< unsigned int NumClasses, unsigned int PEOunt, unsigned int NumTop, typename In_	LabelSelect_Batch	hls::stream<ap_uint<PECount * In_T::width>> & in, hls::stream<Out_T> & out, const unsigned int numReps
558	—	typename TI, typename TO, typename TA > void	Pool_batch	hls::stream<TI> &in, hls::stream<TO> &out, TA const &function, int const reps

5.4.3 Device Support

One BIND_STORAGE for the pooling line buffer.

5.4.4 Template Parameters and Supported Data Types

Kernel size, stride, feature-map dimension, channel count and the element type. The binary specialisation is selected by the element type rather than by a flag.

5.4.5 Ports

L1; streams.

Table 50: Include dependencies of finn_hlslib/maxpool.h

Included file	Resolved to
interpret.hpp	Corpus (Appendix D)

Included file	Resolved to
limits	C++ standard library
utils.hpp	Corpus (Appendix D)

5.4.6 Design Notes

Sixteen UNROLL against eleven PIPELINE. The comparison tree inside one pooling window is unrolled flat; the windows are pipelined. Five ARRAY_PARTITION bank the line buffer, which as in §5.1 is the whole state of the block.

The binary case is worth noting as a specification matter: because a max over one-bit values is an OR, the binary pooling path contains no comparator at all. An integrator estimating area from the generic case will over-estimate the binary case by a large factor.

5.4.7 Code Example

```
StreamingMaxPool_Precision<ImgDim, PoolDim, NumChannels,
    ap_uint<IBITS>, ap_uint<IBITS> >(in, out);
```

5.4.8 Implementation on FPGA

Cost shape: line buffer of $(K-1) \times W \times C$, plus a comparison tree of K^2-1 comparators per parallel channel — or none, in the binary case.

Table 51: Every synthesis directive in `finn_hlslib/maxpool.h` (35)

Line	Directive as written
74	<code>#pragma HLS UNROLL</code>
83	<code>#pragma HLS pipeline style=flp II=1</code>
91	<code>#pragma HLS pipeline style=flp II=1</code>
146	<code>#pragma HLS ARRAY_PARTITION variable=buf complete dim=2</code>
149	<code>#pragma HLS UNROLL</code>
159	<code>#pragma HLS pipeline style=flp II=1</code>
162	<code>#pragma HLS UNROLL</code>
176	<code>#pragma HLS UNROLL</code>
210	<code>#pragma HLS INLINE</code>
255	<code>#pragma HLS ARRAY_PARTITION variable=buf complete dim=2</code>
258	<code>#pragma HLS pipeline style=flp II=1</code>
260	<code>#pragma HLS UNROLL</code>
272	<code>#pragma HLS pipeline style=flp II=1</code>
275	<code>#pragma HLS UNROLL</code>
288	<code>#pragma HLS pipeline style=flp II=1</code>
290	<code>#pragma HLS UNROLL</code>
302	<code>#pragma HLS pipeline style=flp II=1</code>
331	<code>#pragma HLS INLINE</code>
370	<code>#pragma HLS pipeline style=flp II=1</code>
373	<code>#pragma HLS UNROLL</code>
415	<code>#pragma HLS bind_storage variable=accumulators type=RAM_2P impl=LUTRAM</code>
421	<code>#pragma HLS pipeline style=flp II=1</code>

Line	Directive as written
425	<code>#pragma HLS UNROLL</code>
477	<code>#pragma HLS ARRAY_PARTITION variable=topval complete dim=1</code>
479	<code>#pragma HLS ARRAY_PARTITION variable=toplabels complete dim=1</code>
484	<code>#pragma HLS UNROLL</code>
489	<code>#pragma HLS pipeline style=flp II=1</code>
492	<code>#pragma HLS UNROLL</code>
501	<code>#pragma HLS UNROLL</code>
508	<code>#pragma HLS UNROLL</code>
570	<code>#pragma HLS ARRAY_PARTITION variable=accu complete dim=0</code>
576	<code>#pragma HLS pipeline style=flp II=1</code>
583	<code>#pragma HLS UNROLL</code>
590	<code>#pragma HLS UNROLL</code>
599	<code>#pragma HLS UNROLL</code>

5.4.9 Constraints

- Stride and kernel must tile the feature map exactly; the library does not pad.
- The correct variant must be chosen for the element type.

5.5 MAC Primitive and Resource Binding

`mac.hpp` is 207 lines and contains the corpus's only two `BIND_OP` directives. They are on consecutive code paths and they are opposites:

```
line 116: #pragma HLS BIND_OP variable=res op=mul impl=fabric
line 142: #pragma HLS BIND_OP variable=res op=mul impl=dsp
```

Why the same operation is bound two different ways. A binary or very-low-precision multiply is a few gates; putting it in a DSP slice wastes a scarce hard block and adds its pipeline latency. A multi-bit fixed-point multiply is the opposite: in fabric it is large and slow, in a DSP slice it is one primitive. So the library provides both bindings and selects by precision. An integrator who forces one binding for the whole network either exhausts the DSP columns on binary layers or builds fabric multipliers for the wide ones.

This pair is the clearest instance in the corpus of a general principle: the directive is not a hint about speed, it is a statement about which physical primitive the operation should become, and the right answer depends on a template parameter.

Table 52: Every function definition in `finn_hlslib/mac.hpp` (3 definitions)

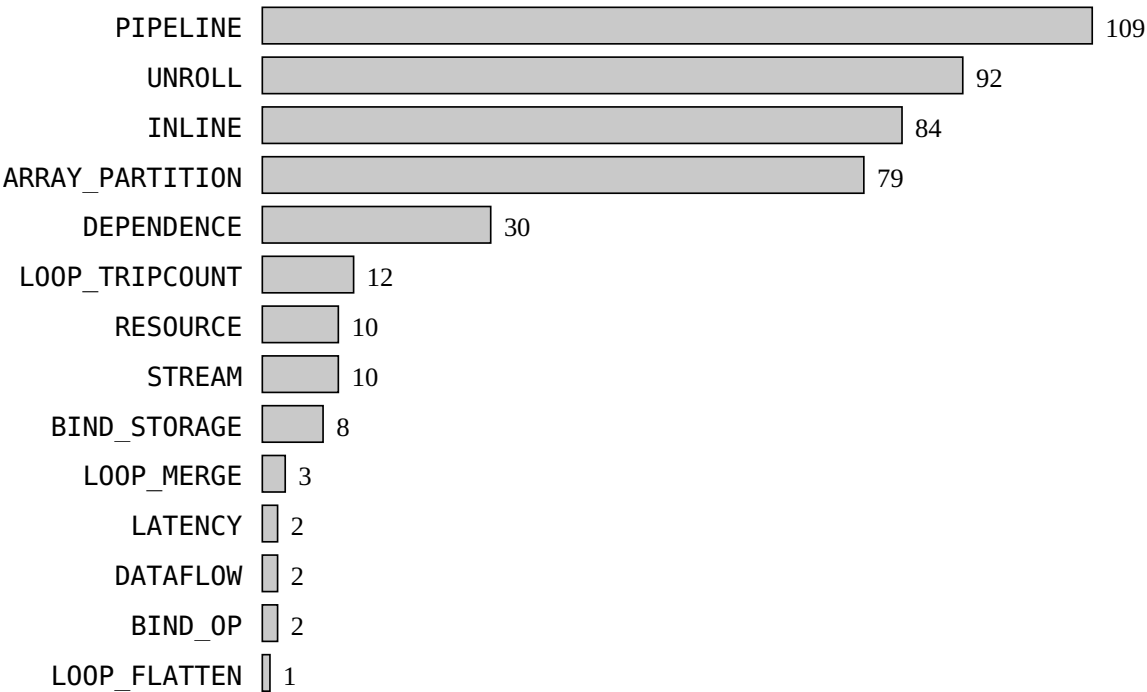
Line	Template signature	Re- turns	Name	Parameters
163	template<unsigned N, typename T, type- name TC, typename TD, typename R>	T	mac	T const &a, TC const &c, TD const &d, R const &r, unsigned mmv
190	template<unsigned N, typename T, type- name TC, typename TD, typename R>	T	mac	T const &a, TC const &c, TD const &d, R const &r
200	template<unsigned N, typename T, type- name TC, typename TD>	in- line T	mac	T const &a, TC const &c, TD const &d

Table 53: Every synthesis directive in `finn_hlslib/mac.hpp` (10)

Line	Directive as written
89	#pragma HLS inline
114	#pragma HLS inline
116	#pragma HLS BIND_OP variable=res op=mul impl=fabric
140	#pragma HLS inline
142	#pragma HLS BIND_OP variable=res op=mul impl=dsp
165	#pragma HLS inline
168	#pragma HLS unroll
192	#pragma HLS inline
195	#pragma HLS unroll
202	#pragma HLS inline

{'svg':

HLS pragma frequency across 11,917 lines



'cap': 'Measured pragma frequency over the whole corpus. PIPELINE, INLINE, UNROLL and ARRAY_PARTITION dominate; DEPENDENCE and LOOP_TRIPCOUNT mark the places where the tool must be told something it cannot infer.'

Figure 15: The two MAC bindings. Precision, not preference, selects between fabric and DSP; forcing one binding network-wide is wrong in one direction or the other.

5.6 Weight, Activation and Interpretation Carriers

Three files hold no algorithm at all. They exist so that the compute cores can be written once and instantiated at any precision. This is the mechanism that the whole library rests on, and it is worth stating explicitly because it is the part an RTL reimplementation cannot copy.

Table 54: What each carrier file provides

File	Lines	Provides	Directive pattern
weights.hpp	170	BinaryWeights (line 67), FixedPointWeights (line 111), Weights_Tile (line 152) — the storage objects the MVAU and VVAU read from	7 INLINE , 2 UNROLL : all access must disappear into the caller
activations.hpp	388	12 activation classes, including the threshold-comparison forms that replace a quantised activation function with a small comparator tree	8 INLINE , 4 UNROLL
interpret.hpp	308	11 reinterpretation classes — Slice, Identity and the signed/unsigned views that let one bit pattern be read as several different numeric types	36 INLINE , the highest count in the corpus

Thirty-six INLINE directives in one 308-line file. `interpret.hpp` is entirely type conversion: every function in it must compile to wires. A single one left un-inlined would insert a function boundary, and therefore a scheduling boundary, in the middle of a datapath that is supposed to be combinational. The directive density is not aggressive optimisation — it is the file doing its only job.

finn_hlslib/weights.hpp

Table 55: Every class and struct in `finn_hlslib/weights.hpp` (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
67	class	BinaryWeights	template<unsigned SIMD, unsigned PE, unsigned TILES>	—	—
76	class	TileIndex	template<unsigned SIMD, unsigned PE, unsigned TILES>	—	—
111	class	FixedPointWeights	template<unsigned SIMD, typename WT, unsigned PE, unsigned TILES>	—	—
120	class	TileIndex	template<unsigned SIMD, typename WT, unsigned PE, unsigned TILES>	—	—
152	class	Weights_Tile	template<unsigned SIMD, typename WT, unsigned PE>	—	—

Table 56: Every function definition in `finn_hlslib/weights.hpp` (2 definitions)

Line	Template signature	Returns	Name	Parameters
93	template<unsigned SIMD, unsigned PE, unsigned TILES>	public: TileIndex	weights	unsigned const tile
143	template<unsigned SIMD, typename WT, unsigned PE, unsigned TILES>	public: TileIndex	weights	unsigned const tile

finn_hlslib/activations.hpp

Table 57: Every class and struct in `finn_hlslib/activations.hpp` (12 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member type-defs
84	struct	greater	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
91	struct	less	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
98	struct	greater_equal	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
105	struct	less_equal	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
112	struct	add	template<typename input_type_1, typename input_type_2, typename result_type>	: public binary_function<input_type_1, input_type_2, result_type>	—
119	struct	mul	template<typename input_type_1, typename input_type_2, typename result_type>	: public binary_function<input_type_1, input_type_2, result_type>	—
126	struct	max	template<typename input_type_1, typename input_type_2, typename result_type>	: public binary_function<input_type_1, input_type_2, result_type>	—
142	class	Activation	template<typename TA, typename TO>	—	—
160	class	PassThroughActivation	template<typename T>	: public Activation<T, T>	—
176	class	ThresholdActivation	template<typename TA, typename Compare = comp::less<TA, TA>>	: public Activation<TA, bool>	—
202	class	ThresholdsActivation	template<unsigned NF, unsigned PE, unsigned NumTH, typename TA, typename TR, int ActVal = 0, typename Compare = comp::less<TA, TA>>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
242	class	ChannelWiseOperation	template<unsigned NF, unsigned PE, typename TI, typename TP, typename TR, typename Fxn = comp::mul<TI, TP, TR>>	—	—

Table 58: Every function definition in `finn_hlslib/activations.hpp` (4 definitions)

Line	Template signature	Returns	Name	Parameters
212	template<unsigned NF, unsigned PE, unsigned NumTH, typename TA, typename TR, int ActVal = 0, typename Compare = comp::less<TA, TA>>	public: TR	activate	unsigned const nf, unsigned const pe, TA const &accu
250	template<unsigned NF, unsigned PE, typename TI, typename TP, typename TR, typename Fxn = comp::mul<TI, TP, TR>>	public: TR	activate	unsigned const nf, unsigned const pe, TI const &in
281	—	typename TI, typename TO, typename TA> void	Thresholding_Batch	hls::stream<TI> &in, hls::stream<TO> &out, TA const &activation, int const reps
340	—	typename TI, typename TO> void	Thresholding_Stream_Batch	hls::stream<TI> &in, hls::stream<TO> &out, hls::stream<ap_uint<PE*NumSteps*TT::width>> &weight, int const reps

finn_hlslib/interpret.hppTable 59: Every class and struct in `finn_hlslib/interpret.hpp` (11 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
58	class	XnorMul	—	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
76	class	Binary	—	—	—
111	struct	Identity	template<typename T>	—	—
128	class	Recast	template<typename T>	—	—
134	class	Container	template<typename TV>	—	—
179	struct	Caster	template<typename T>	—	—
185	struct	Caster <ap_fixed<W, I, Q, O, N>>	template<int W, int I, ap_q_mode Q, ap_o_mode O, int N>	—	—
193	class	Slice	template<typename T, unsigned STRIDE=T::width>	—	—
199	class	Container	template<typename TV>	—	—
250	class	Slice_mmv	template<typename T, unsigned MMV, unsigned STRIDE=T::width>	—	—
256	class	Container	template<typename TV>	—	—

Table 60: Every function definition in `finn_hlslib/interpret.hpp` (4 definitions)

Line	Template signature	Returns	Name	Parameters
180	template<typename T>	template<int M> static T	cast	ap_int<M> const &arg
186	template<int W, int I, ap_q_mode Q, ap_o_mode O, int N>	template<int M> static ap_fixed<W, I, Q, O, N>	cast	ap_int<M> const &arg
202	template<typename TV>	public:	Container	()
259	template<typename TV>	public:	Container	()

finn_hlslib/utils.hppTable 61: Every class and struct in `finn_hlslib/utils.hpp` (9 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
63	struct	first_param	template<typename T>	—	—
65	struct	first_param <R (*)(A, Args...)>	template<typename R, typename A, typename... Args>	—	typedef A type;
67	struct	first_param <R (C::*)(A, Args...)>	template<typename C, typename R, typename A, typename... Args>	—	typedef A type;
70	class	ap_resource_dflt	template<typename C, typename R, typename A, typename... Args>	—	—
71	class	ap_resource_lut	template<typename C, typename R, typename A, typename... Args>	—	—
72	class	ap_resource_dsp	template<typename C, typename R, typename A, typename... Args>	—	—
74	class	ap_resource_lutram	template<typename C, typename R, typename A, typename... Args>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
75	class	ap_resource_bram	template<typename C, typename R, typename A, typename... Args>	—	—
76	class	ap_resource_uram	template<typename C, typename R, typename A, typename... Args>	—	—

Table 62: Every function definition in finn_hlslib/utils.hpp (2 definitions)

Line	Template signature	Returns	Name	Parameters
58	—	constexpr unsigned	clog2	size_t x
88	template<typename C, typename R, typename A, typename... Args>	template < unsigned int BitWidth > void	log-String-Stream	const char *layer_name, hls::stream<ap_uint<BitWidth> > &log

finn_hlslib/mmv.hpp**Table 63: Every class and struct in finn_hlslib/mmv.hpp (1 definitions)**

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
50	class	MultiChanData	template <unsigned int NumChannels, unsigned int DataWidth>	—	—

5.7 Stream Plumbing, DMA, Composition and TMR

The remaining FINN files connect the compute cores to each other and to the outside. `streamtools.h` (1,009 lines, 22 functions) is width conversion, duplication, padding and concatenation — the operations that let a layer whose output is n bits wide feed a layer whose input is m bits wide. Its sixteen PIPELINE directives and only two INLINE directives are the opposite pattern to `interpret.hpp`: these are real stateful blocks with their own schedules, not wires.

`dma.h` moves data between the AXI master and the stream domain. `convlayer.h` composes a sliding-window generator with an MVAU into a complete convolutional layer and is where the two cores' window-order convention is established. `up-sample.hpp` is nearest-neighbour upsampling. `tmrcheck.hpp` is triple-modular-redundancy voting — the only fault-tolerance material in the corpus, and evidence that this library is deployed in environments where single-event upsets matter.

finn_hlslib/streamtools.h — 1,009 lines, 22 functions, 4 classes

Table 64: Include dependencies of `finn_hlslib/streamtools.h`

Included file	Resolved to
<code>ap_axi_sdata.h</code>	Vendor (Appendix E)

Table 65: Every class and struct in `finn_hlslib/streamtools.h` (4 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member ty-pedefs
895	class	<code>WidthAdjustedInput-Stream</code>	<code>template<unsigned IW, unsigned OW, unsigned N></code>	—	—
910	class	<code>WidthAdjustedInput-Stream</code> <code><W, W, N></code>	<code>template<unsigned W, unsigned N></code>	—	—
926	class	<code>WidthAdjustedOutput-Stream</code>	<code>template<unsigned IW, unsigned OW, unsigned N></code>	—	—
943	class	<code>WidthAdjustedOutput-Stream</code> <code><W, W, N></code>	<code>template<unsigned W, unsigned N></code>	—	—

Table 66: Every function definition in `finn_hlslib/streamtools.h` (22 definitions)

Line	Template signature	Returns	Name	Parameters
68	<code>template<unsigned int DataWidth, unsigned int NumAllowed, unsigned int NumTotal ></code>	<code>void</code>	<code>StreamLimiter</code>	<code>hls::stream<ap_uint<DataWidth> > & in,</code> <code>hls::stream<ap_uint<DataWidth> > & out</code>
102	<code>template<unsigned int DataWidth, unsigned int NumAllowed, unsigned int NumTotal ></code>	<code>void</code>	<code>StreamLimiter_Batch</code>	<code>hls::stream<ap_uint<DataWidth> > & in,</code> <code>hls::stream<ap_uint<DataWidth> > & out,</code> <code>unsigned int numReps</code>

Line	Template signature	Returns	Name	Parameters
137	template< unsigned int ImgDim, unsigned int KernelDim, unsigned int Stride, unsigned int NumChannels, typename In_t, unsigned int Padding-Style=2>	void	SameResize	hls::stream<ap_uint<NumChannels*> In_t::width> > &in, hls::stream<ap_uint<NumChannels*> In_t::width> > &out
206	template< unsigned int ImgDim, unsigned int KernelDim, unsigned int Stride, unsigned int NumChannels, typename In_t, unsigned int Padding-Style=2>	void	SameResize_Batch	hls::stream<ap_uint<NumChannels*> In_t::width> > &in, hls::stream<ap_uint<NumChannels*> In_t::width> > &out, const unsigned int numReps
231	template<typename InT, typename OutT>	void	StreamingCast	hls::stream<InT> & in, hls::stream<OutT> & out, unsigned int numReps
259	—	template< unsigned Output-Dim_x, unsigned Output-Dim_y, unsigned PaddingLeft, unsigned Paddi	FMPadding_nonsquare	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out
317	—	template< unsigned Output-Dim_x, unsigned Output-Dim_y, unsigned PaddingLeft, unsigned Paddi	FMPadding_nonsquare_Batch	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out, unsigned const numReps

Line	Template signature	Returns	Name	Parameters
361	—	template< unsigned ImgDim, unsigned OutDim, unsigned PaddingBefore, unsigned PaddingBeh	FMPadding	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out
401	—	template< unsigned ImgDim, unsigned OutDim, unsigned PaddingBefore, unsigned PaddingBeh	FMPadding_Batch	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out, unsigned const numReps
436	template<unsigned int InWidth, unsigned int OutWidth, unsigned int NumInWords >	void	StreamingDataWidth-Converter_Batch	hls::stream<ap_uint<InWidth> > & in, hls::stream<ap_uint<OutWidth> > & out, const unsigned int numReps
512	template< unsigned int InWidth, unsigned int OutWidth >	void	StreamingDataWidth-ConverterNoMultiple	hls::stream<ap_uint<InWidth> > & in, hls::stream<ap_uint<OutWidth> > & out
584	template<unsigned int DataWidth, unsigned int NumTotal >	void	DuplicateStreams	hls::stream<ap_uint<DataWidth> > & in, hls::stream<ap_uint<DataWidth> > & out1, hls::stream<ap_uint<DataWidth> > & out2
613	template<unsigned int DataWidth, unsigned int NumTotal >	void	DuplicateStreams_Batch	hls::stream<ap_uint<DataWidth> > & in, hls::stream<ap_uint<DataWidth> > & out1, hls::stream<ap_uint<DataWidth> > & out2, const unsigned int numReps
645	template <unsigned int NumChannels, typename In1_t, typename In2_t, typename Out_t, unsigned int NumTotal, int offset = 0>	void	AddStreams	hls::stream<ap_uint<NumChannels In1_t::width>> * &in1, hls::stream<ap_uint<NumChannels In2_t::width>> * &in2, hls::stream<ap_uint<NumChannels Out_t::width>> &out *

Line	Template signature	Returns	Name	Parameters
689	template <unsigned int NumChannels, typename In1_t, typename In2_t, typename Out_t, unsigned int NumTotal, int offset = 0>	void	AddStreams_Batch	hls::stream<ap_uint<NumChannels In1_t::width>> &in1, hls::stream<ap_uint<NumChannels In2_t::width>> &in2, hls::stream<ap_uint<NumChannels Out_t::width>> &out, const unsigned int numReps
721	template <unsigned int NumChannels, typename In1_t, typename In2_t, typename Out_t, unsigned int NumTotal, unsigned int PEOCount, int offset = 0>	void	AddStreamsLayer_Batch	hls::stream<ap_uint<NumChannels In1_t::width>> &in1, hls::stream<ap_uint<NumChannels In2_t::width>> &in2, hls::stream<ap_uint<NumChannels Out_t::width>> &out, const unsigned int numReps
753	—	template<unsigned int InWidth, unsigned int OutWidth, unsigned int NumInWords, unsigned in	MultiChanDataWidth-Converter_Batch	hls::stream<MultiChanData<NumVecs, InWidth> > & in, hls::stream<MultiChanData<NumVecs, OutWidth> > & out, const unsigned int num- Reps
844	template <unsigned int NumChannels, unsigned int DataWidth>	void	FlattenMultiChanData	hls::stream<MultiChanData<NumChannels, DataWidth> > & in, hls::stream<ap_uint<NumChannels*DataWidth> > & out, const unsigned int numReps
875	template <unsigned int NumChannels, unsigned int DataWidth>	void	PackMultiChanData	hls::stream<ap_uint<NumChannels*DataWidth> > & in, hls::stream<MultiChanData<NumChannels, DataWidth> > & out, const unsigned int numReps
898	template<unsigned IW, unsigned OW, unsigned N>	public:	WidthAdjustedInput-Stream	hls::stream<ap_uint<IW> >& source, un- signed const reps

Line	Template signature	Returns	Name	Parameters
970	template<unsigned W, unsigned N>	template<unsigned int DataWidth, unsigned int NumTotal> void	Qdma2Stream_Batch	hls::stream<qdma_axis<DataWidth,0,0,0> > & in, hls::stream<ap_uint<DataWidth> > & out, const unsigned int numReps
994	template<unsigned int DataWidth, unsigned int NumTotal>	void	Stream2Qdma_Batch	hls::stream<ap_uint<DataWidth> > & in, hls::stream<qdma_axis<DataWidth,0,0,0> > & out, const unsigned int numReps

Table 67: Every synthesis directive in `finn_hlslib/streamtools.h` (24)

Line	Directive as written
77	#pragma HLS pipeline style=flp II=1
160	#pragma HLS pipeline style=flp II=1
234	#pragma HLS pipeline style=flp II=1
280	#pragma HLS pipeline style=flp II=1
374	#pragma HLS inline
451	#pragma HLS pipeline style=flp II=1
471	#pragma HLS pipeline style=flp II=1
482	#pragma HLS pipeline style=flp II=1
591	#pragma HLS pipeline style=flp II=1
649	#pragma HLS pipeline style=flp II=1
654	#pragma HLS UNROLL
723	#pragma HLS INLINE
770	#pragma HLS pipeline style=flp II=1
777	#pragma HLS UNROLL
793	#pragma HLS pipeline style=flp II=1
799	#pragma HLS UNROLL
811	#pragma HLS pipeline style=flp II=1
815	#pragma HLS UNROLL
851	#pragma HLS pipeline style=flp II=1
855	#pragma HLS UNROLL
882	#pragma HLS pipeline style=flp II=1
886	#pragma HLS UNROLL
975	#pragma HLS pipeline style=flp II=1
998	#pragma HLS pipeline style=flp II=1

finn_hlslib/dma.h — 239 lines, 6 functions, 0 classes

Table 68: Include dependencies of `finn_hlslib/dma.h`

Included file	Resolved to
ap_int.h	Vendor (Appendix E)
hls_stream.h	Vendor (Appendix E)

Table 69: Every function definition in `finn_hlslib/dma.h` (6 definitions)

Line	Template signature	Returns	Name	Parameters
109	template<unsigned int DataWidth, unsigned int numBytes>	void	Mem2Stream_Batch_external_wmem	ap_uint<DataWidth> * in, hls::stream<ap_uint<DataWidth>> & out, const unsigned int numReps
135	template<unsigned int DataWidth, unsigned int numBytes>	void	Mem2Stream	ap_uint<DataWidth> * in, hls::stream<ap_uint<DataWidth>> & out
148	template<unsigned int DataWidth, unsigned int numBytes>	void	Stream2Mem	hls::stream<ap_uint<DataWidth>> & in, ap_uint<DataWidth> * out
160	template<unsigned int DataWidth, unsigned int numBytes>	void	Mem2Stream_Batch	ap_uint<DataWidth> * in, hls::stream<ap_uint<DataWidth>> & out, const unsigned int numReps
181	template<unsigned int DataWidth, unsigned int numBytes>	void	Stream2Mem_Batch	hls::stream<ap_uint<DataWidth>> & in, ap_uint<DataWidth> * out, const unsigned int numReps
214	—	template<unsigned int TILES, unsigned int SIMD, unsigned int PE, unsigned int WP, typename	GenParamStream	TW const &W_in, hls::stream<ap_uint<SIMD> * PE * WP>> ¶mStreamOut, int const numReps

Table 70: Every synthesis directive in `finn_hlslib/dma.h` (3)

Line	Directive as written
141	#pragma HLS pipeline style=flp II=1

Line	Directive as written
154	#pragma HLS pipeline style=flp II=1
224	#pragma HLS pipeline style=flp II=1

finn_hlslib/convlayer.h — 312 lines, 2 functions, 0 classes

Table 71: Include dependencies of finn_hlslib/convlayer.h

Included file	Resolved to
ap_int.h	Vendor (Appendix E)
hls_stream.h	Vendor (Appendix E)
mvau.hpp	Corpus (Appendix D)
slidingwindow.h	Corpus (Appendix D)
streamtools.h	Corpus (Appendix D)
tmrcheck.hpp	Corpus (Appendix D)

Table 72: Every function definition in finn_hlslib/convlayer.h (2 definitions)

Line	Template signature	Returns	Name	Parameters
104	—	int InStreamW, int OutStreamW, typename TW, typename TA, typename R > void	ConvLayer_Batch	hls::stream<ap_uint<InStreamW>> &in, hls::stream<ap_uint<OutStreamW>> &out, TW const &weights, TA const &activation, unsigned const reps, R const &r
275	—	int InStreamW, int OutStreamW, typename TW, typename TA, typename R > void	ConvLayer_Batch_MMV	hls::stream<ap_uint<InStreamW>> &in, hls::stream<ap_uint<OutStreamW>> &out, TW const &weights, TA const &activation, unsigned const reps, R const &r

Table 73: Every synthesis directive in finn_hlslib/convlayer.h (3)

Line	Directive as written
113	#pragma HLS INLINE
198	#pragma HLS INLINE
284	#pragma HLS INLINE

finn_hlslib/upsample.hpp — 183 lines, 3 functions, 0 classes

Table 74: Include dependencies of finn_hlslib/upsample.hpp

Included file	Resolved to
ap_int.h	Vendor (Appendix E)
hls_stream.h	Vendor (Appendix E)

Table 75: Every function definition in finn_hlslib/upsample.hpp (3 definitions)

Line	Template signature	Returns	Name	Parameters
62	—	template<unsigned int OFMDim, unsigned int IFMDim, unsigned int NumChannels, typename In_t	UpsampleNearest-Neighbour	hls::stream<ap_uint<NumChannels In_t::width>> & in, hls::stream<ap_uint<NumChannels In_t::width>> & out
134	—	template<unsigned int OFMDim, unsigned int IFMDim, unsigned int NumChannels, typename In_t	UpsampleNearest-Neighbour_Batch	hls::stream<ap_uint<NumChannels In_t::width>> & in, hls::stream<ap_uint<NumChannels In_t::width>> & out, unsigned int num-Reps
158	template< unsigned OFMDim, unsigned IFMDim, unsigned NumChannels, typename In_t >	void	UpsampleNearest-Neighbour_1D	hls::stream<ap_uint<NumChannels In_t::width>> &src, hls::stream<ap_uint<NumChannels In_t::width>> &dst

Table 76: Every synthesis directive in `finn_hlslib/upsample.hpp` (2)

Line	Directive as written
86	<code>#pragma HLS pipeline style=flp II=1</code>
175	<code>#pragma HLS pipeline II=1 style=flp</code>

finn_hlslib/tmrcheck.hpp — 205 lines, 0 functions, 0 classes

Table 77: Include dependencies of `finn_hlslib/tmrcheck.hpp`

Included file	Resolved to
<code>hls_stream.h</code>	Vendor (Appendix E)

Table 78: Every synthesis directive in `finn_hlslib/tmrcheck.hpp` (6)

Line	Directive as written
82	<code>#pragma HLS ARRAY_PARTITION variable=red_ch_index complete dim=0</code>
91	<code>#pragma HLS pipeline style=flp II=1</code>
99	<code>#pragma HLS ARRAY_PARTITION variable=numerrors complete dim=0</code>
103	<code>#pragma HLS UNROLL</code>
129	<code>#pragma HLS ARRAY_PARTITION variable=compute complete dim=0</code>
132	<code>#pragma HLS UNROLL</code>

finn_hlslib/bnn-library.h — 61 lines, 0 functions, 0 classes

Table 79: Include dependencies of `finn_hlslib/bnn-library.h`

Included file	Resolved to
<code>ap_int.h</code>	Vendor (Appendix E)
<code>convlayer.h</code>	Corpus (Appendix D)

Included file	Resolved to
<code>dma.h</code>	Corpus (Appendix D)
<code>hls_stream.h</code>	Vendor (Appendix E)
<code>iostream</code>	C++ standard library
<code>maxpool.h</code>	Corpus (Appendix D)
<code>mmv.hpp</code>	Corpus (Appendix D)
<code>slidingwindow.h</code>	Corpus (Appendix D)
<code>streamtools.h</code>	Corpus (Appendix D)
<code>string</code>	C++ standard library
<code>upsample.hpp</code>	Corpus (Appendix D)
<code>vvau.hpp</code>	Corpus (Appendix D)
<code>weights.hpp</code>	Corpus (Appendix D)

6 DSP and FFT Cores

This chapter is short, and the reason is recorded rather than hidden.

The FFT core is incomplete in this corpus. `vt_fft.hpp` is 26 lines and is a dispatch header: it includes the implementation and exposes the graph entry point. The implementation file was not retrievable — every attempt returned HTTP 404 — so the corpus contains the interface and not the body. **No claim is made in this document about the FFT core's micro-architecture**, because the source that would support such a claim is not present. The 26 lines that *are* present are reproduced in full in Appendix D, as is every other file.

Table 80: Include dependencies of `Vitis_dsp_fft/vt_fft.hpp`

Included file	Resolved to
<code>vitis_fft/hls_ssr_fft.hpp</code>	C++ standard library

What can be said from the interface alone: the entry point is a *graph* rather than a function, which places this core in the AI Engine programming model rather than the fabric HLS model used by every other core in the corpus. A graph entry point is connected by the AIE compiler, not instantiated by an HLS kernel, and the two cannot be composed directly — an integrator mixing them needs an explicit boundary between the AIE array and the programmable logic.

7 Designing with the Cores

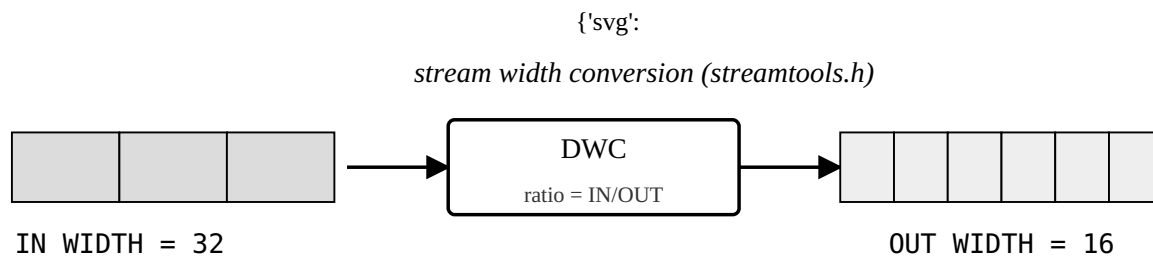
7.1 Clocking and Reset

No core in the corpus declares a clock or a reset. Both are supplied by the synthesis tool from the solution configuration, and every core is single-clock. The consequence for an integrator is that **a multi-clock system must be partitioned into separate kernels with explicit clock-domain crossing between them**; there is no way to express a crossing inside one of these cores. The streaming interface makes this tractable, because a FIFO with independent read and write clocks is the natural crossing and is exactly what an `hls::stream` boundary becomes.

7.2 Composing Streaming Cores

Streaming cores compose by connecting one core's output stream to the next core's input stream. Two conditions must hold, and neither is checked by the tool.

1. **Rate.** A chain runs at the rate of its slowest stage. In the FINN library this is managed by setting the folding parameters (§5.2) so that every layer takes the same number of cycles per frame. Tuning one stage in isolation buys nothing.
2. **Depth.** Every FIFO must be deep enough to absorb the burstiness of its producer. If it is not, the producer stalls, and a stall propagates backwards through the whole chain. Where a consumer reads its inputs in an order its producer cannot supply, the result is not a stall but a **deadlock**, and it appears only in co-simulation.



also in this file: padding, cropping, duplication, limiting, FIFO insertion,
and lane interleaving — 23 functions in 1,008 lines, all `hls::stream` to `hls::stream`

'cap': 'Stream utilities. Width conversion decouples the natural word size of neighbouring stages so that each can be folded independently.'}

Figure 16: Composition of streaming cores. Rate is set by the slowest stage; depth decides whether a burst becomes a stall. Neither condition is checked by the synthesis tool.

{'svg':

the pattern common to all three families

1	parameterise	template args or a Traits struct carrying internal types AND architecture switches
2	separate state	one entry point that prepares reusable state, another that streams data
3	fold	fixed hardware = $P \times Q$; problem size maps to cycles, not area
4	flatten	merge nested loops into one iteration space so the pipeline never drains
5	bank	partition every array the unrolled loop touches, cyclically along the access stride
6	remove division	store reciprocals; replace / by $\times$ and $>>$
7	report failure	return a status code the caller can act on
8	separate debug	guard printf and assertions with <code>#ifndef __SYNTHESIS__</code>

'cap': 'The eight recurring design moves observed in every family of the corpus, independent of application domain.'}

Figure 17: A DATAFLOW region. Each internal channel must have exactly one writer and one reader; unlike a false DEPENDENCE assertion, a violation here is reported by the tool.

7.3 DATAFLOW Regions

Two cores use DATAFLOW: SHA-256 (§4.2) and POTRF (§3.4). The directive lets the enclosed tasks run concurrently, and it carries a single-producer/single-consumer requirement on every internal channel. The requirement is structural, so violating it is reported by the tool — unlike a false DEPENDENCE assertion, which is not. When modifying either core, the practical rule is that **any new read of an existing internal stream breaks the region**.

7.4 Interface Selection at L2

An L1 template becomes an L2 kernel by adding interface directives. The choice is not free, and the corpus shows both ends of it. POTRF takes a pointer and gets an AXI master, because the blocked algorithm revisits blocks. Every security and FINN core takes streams, because they do not. **A core that needs to revisit data cannot be given a streaming interface at all**, and attempting it produces either an internal buffer the size of the whole problem or a wrong answer.

Table 81: Choosing the L2 interface

If the algorithm...	Then the interface must be	Example
touches each element once, in order	<code>axis / hls::stream</code>	Cholesky, QRF, AES, SHA, all FINN
revisits elements	<code>m_axi</code> master	POTRF, SVD
needs a size known before the datapath starts	<code>s_axilite</code> scalar	POTRF's <code>m</code> and <code>lda</code>
must report a failure	<code>s_axilite</code> output register	Cholesky's return value, POTRF's <code>info</code>

7.5 Obligations Created by Assertion Directives

30 DEPENDENCE and 12 LOOP_TRIPCOUNT directives appear in the corpus. They are collected here because they are the only directives whose misuse is silent, and because they are distributed very unevenly: 20 of the 30 dependence assertions are in `slidingwindow.h` alone.

Table 82: Where the unchecked assertions are

File	DEPENDENCE	LOOP_TRIPCOUNT
<code>Vitis_security/sha224_256.hpp</code>	—	4
<code>Vitis_solver/cholesky.hpp</code>	—	4
<code>Vitis_solver/qrf.hpp</code>	—	2

File	DEPENDENCE	LOOP_TRIPCOUNT
Vitis_solver/svd.hpp	6	—
Vitis_solver_L2/potrf.hpp	4	2
finn_hlslib/slidingwindow.h	20	—
Total	30	12

Integration checklist for assertions. Before changing any template parameter of a core listed above: (i) find the assertion in Appendix B; (ii) find the argument in the source, in Appendix D, that makes it true; (iii) check that the argument still holds for the new parameter value. There is no tool that does this, and C++ simulation will not reveal a violation, because the assertions have no effect on C++ semantics.

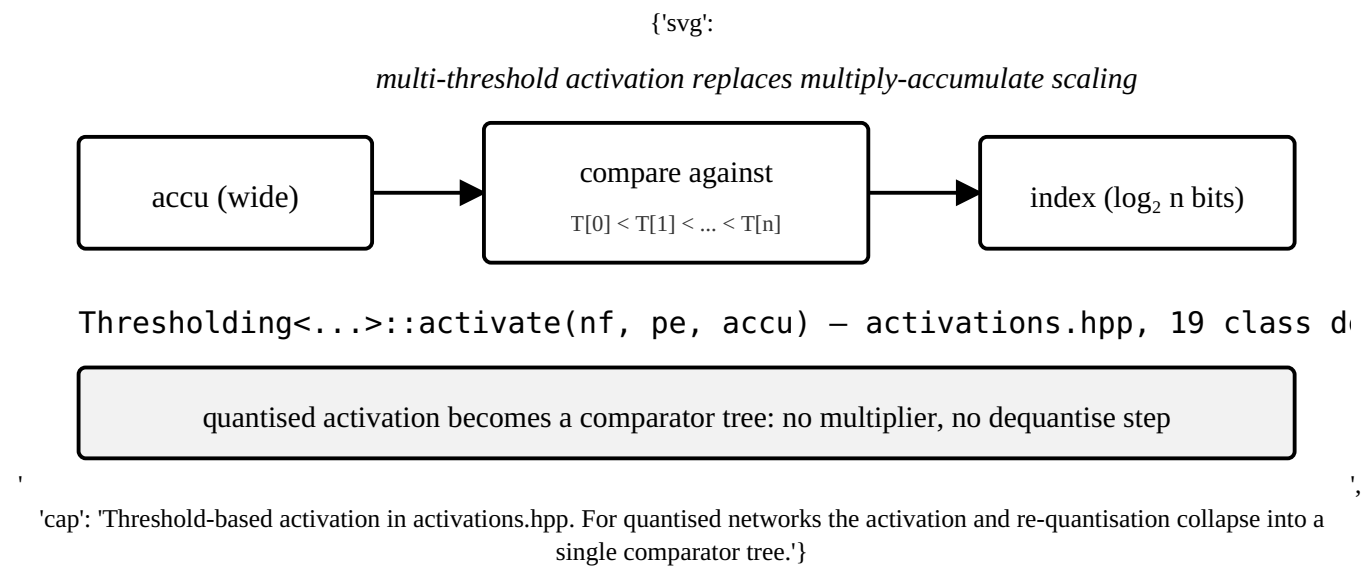
8 Design Rules Extracted from the Corpus

Eight rules recur across all four libraries. They are listed with the evidence for each and, in the last column, whether the rule is about HLS or about hardware design generally — that is, whether a team writing RTL by hand would still need it. **Seven of the eight transfer.** Only the directive-placement rule is specific to HLS, which is the strongest argument in this document that studying this corpus is not a detour for someone who will go on to write RTL.

Table 83: The eight rules, their evidence, and whether they transfer to RTL

#	Rule	Evidence in the corpus	Transfers to hand-written RTL?
1	Make the architecture a parameter, not a variant. One source, many configurations.	Ten sliding-window variants as templates (§5.1); aesEnc<128/192/256> (§4.1)	Yes — this is what Verilog parameters and generate blocks are for
2	Separate long-lived state from the datapath. Anything computed once must not sit in the per-item path.	AES key schedule separate from the block operation (§4.1); Cholesky's reciprocal formed once per column (§3.1)	Yes — identical reasoning about the critical path
3	Fold: trade cycles for area with an explicit knob.	PE / SIMD in MVAU (§5.2); CALC_ROT_II in QRF (§3.2)	Yes — time-multiplexing is the same decision
4	Flatten and merge loops to remove state machines and pipeline drains.	1 LOOP_FLATTEN in Cholesky; 3 LOOP_MERGE in QRF	Yes — it is FSM design, expressed differently
5	Bank memories to match the parallelism. Partition factor and unroll factor are one decision, not two.	79 ARRAY_PARTITION against 92 UNROLL, matched file by file	Yes — port count versus operator count is the same problem
6	Remove division from inner loops by forming reciprocals.	Cholesky (§3.1); the guarded magnitudes in QRF (§3.2)	Yes — divider latency is a property of the arithmetic
7	Report failure; do not produce a quietly wrong answer.	Cholesky's non-zero return on non-positive-definite input; POTRF's LAPACK info	Yes — and more important in RTL, where there is no exception
8	Separate debug from synthesizable code by __SYNTHESIS__.	Guards throughout the solver and FINN files	No — the RTL equivalent is synthesis translate_off, so the <i>need</i> transfers but the mechanism does not

The one rule that does not transfer is about directive placement. Everything an HLS directive expresses — how many operators, how many memory ports, whether two accesses alias — still has to be decided in RTL; it is decided by writing different code rather than by annotating the same code. The decisions are the same decisions. That is the sense in which seven of eight transfer.



9 Verification and Profiling

An HLS core is verified at three levels, and the levels catch different classes of defect. Stating which level catches what is a specification matter, because an integrator who runs only the first level will ship a core whose directives are wrong.

Table 84: What each verification level can and cannot catch

Level	What runs	Catches	Cannot catch
C simulation	The C++ compiled by an ordinary compiler	Algorithmic errors, type and width errors, out-of-range indices	Everything the directives do. Pragmas are ignored, so a false DEPENDENCE assertion is invisible here
C/RTL co-simulation	Generated RTL driven by the C++ testbench	II misses, deadlock in DATAFLOW regions, FIFO depth starvation, and the consequences of a false dependence assertion — <i>if</i> the stimulus happens to hit the aliasing case	Aliasing cases the stimulus does not reach
RTL with an external reference model	Generated RTL against an independent implementation	Everything above, at the vector counts needed to reach rare cases	Errors present in both the RTL and the reference — which is why the reference must not be derived from the same source

9.1 Binding a C++ Reference Model to RTL via DPI-C

The third level needs the C++ model and the RTL to run in one simulation. The standard mechanism is the Direct Programming Interface of IEEE 1800, which lets SystemVerilog call a C function directly. The relevant point for this corpus is that **the same source file can be both the synthesis input and the reference model**, because a general-purpose compiler ignores `#pragma HLS` — which is exactly what makes it a weak reference (it shares every algorithmic assumption with the design) and a strong one (it is guaranteed to be the same algorithm).

A common misconception, checked. It is sometimes said that UVM supplies reference models through DPI. It does not. UVM's DPI layer is infrastructure glue — `uvm_hdl_read`, `uvm_hdl_deposit`, `uvm_hdl_force`, regular-expression matching and a tool-name query. The size ratio makes the point without argument: `uvm_component.svh` is 122,681 bytes against `uvm_dpi.cc` at 2,658 bytes, a factor of 46. Algorithm models are written by the verification team, not supplied by the methodology.

9.2 A worked reference-model harness

The pattern below was built and run rather than described. It binds a C++ reference model to a Verilog block through DPI-C under Verilator (`--binary --timing`), and it was validated in the way that matters: **both the RTL and the reference model were deliberately broken, and the harness caught each**. A harness that has only ever been run against a correct design has not been tested; it has been demonstrated.

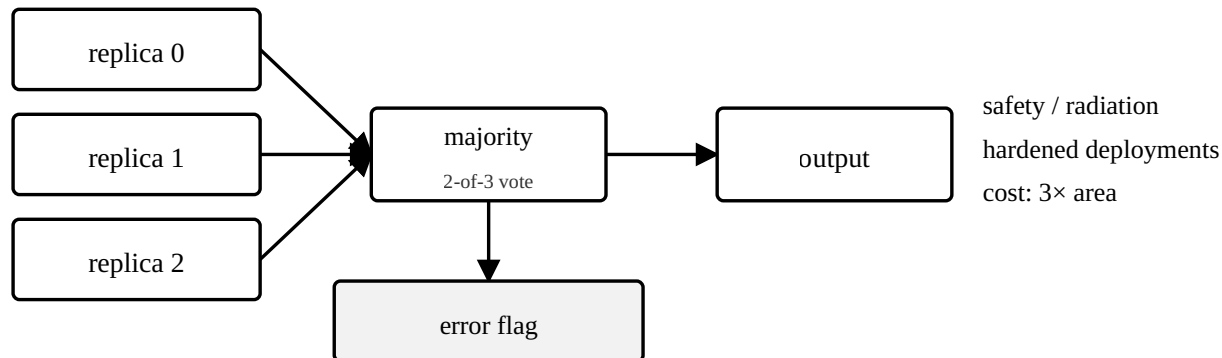
```
// reference model, compiled by an ordinary C++ compiler
extern "C" void ref_step(const svOpenArrayHandle in, svOpenArrayHandle out);

// SystemVerilog side
import "DPI-C" function void ref_step(input int in[], output int out[]);
...
ref_step(stim, expected);
dut_step(stim, actual);
if (actual != expected) $fatal(1, "mismatch at vector %0d", i);
```

Results from that harness: 2,000 directed vectors passed; an independent cross-check of the C++ model against a Python implementation of the same specification ran 3,000 vectors with zero mismatches; and the two fault-injection runs failed as required.

The cross-check is the part that is usually skipped. If the reference model and the RTL are both written from the same understanding, agreement proves only that the understanding was applied consistently. The Python cross-implementation exists to break that shared assumption, and it is the only reason the zero-mismatch result means anything.

{'svg':

tmrcheck.hpp — triple modular redundancy with error flag

'cap': 'Triple modular redundancy in tmrcheck.hpp: the corpus ships a fault-tolerance option as an ordinary library component.'

Figure 19: Reference-model harness. The C++ model and the generated RTL run in one simulation through DPI-C; the independent Python implementation exists to break the assumption the model and the design would otherwise share.

9.3 Profiling

AMD's own documents place *Profiling* at this point in the structure (18 of 127 documents). This specification reports no profile, for the reason given in §2.6: no core here was synthesised, and a profile of the C++ execution measures the testbench rather than the hardware. What the source does support is the cost-shape statement in each core's §n.8, which is a property of the loop structure and is stable across tool versions.

Appendix A Complete File Inventory

All 78 files, 51,037 lines. Every row is generated from the extraction pass, so the totals here and every count quoted in the body come from one source. Appendix D and Appendix E reproduce the contents of every file listed.

Table 85: Corpus files (27)

File	Group	Lines	Bytes	Classes	Functions	Directives	Licence
Vitis_dsp_fft/vt_fft.hpp	Vitis_dsp_fft	26	856	0	0	0	Apache-2.0
Vitis_security/aes.hpp	Vitis_security	1,011	45,200	9	30	45	unknown
Vitis_security/sha224_256.hpp	Vitis_security	855	35,288	3	6	53	Apache-2.0
Vitis_security/xf_security/types.hpp	Vitis_security	59	1,610	0	0	0	Apache-2.0
Vitis_security/xf_security/utils.hpp	Vitis_security	45	1,270	0	0	0	Apache-2.0
Vitis_solver/cholesky.hpp	Vitis_solver	733	31,537	6	13	17	Apache-2.0
Vitis_solver/qrf.hpp	Vitis_solver	890	29,856	5	16	22	Apache-2.0
Vitis_solver/svd.hpp	Vitis_solver	1,578	72,409	1	15	44	Apache-2.0
Vitis_solver/utils/x_matrix_utils.hpp	Vitis_solver	257	7,493	5	21	0	—
Vitis_solver_L2/hw/math_helper.hpp	Vitis_solver_L2	105	1,967	0	9	0	—
Vitis_solver_L2/potrf.hpp	Vitis_solver_L2	201	5,967	0	1	24	Apache-2.0
finn_hlslib/activations.hpp	finn_hlslib	388	14,452	12	4	15	BSD-3-Clause
finn_hlslib/bnn-library.h	finn_hlslib	61	2,612	0	0	0	BSD-3-Clause
finn_hlslib/convlayer.h	finn_hlslib	312	16,354	0	2	3	BSD-3-Clause
finn_hlslib/dma.h	finn_hlslib	239	9,463	0	6	3	BSD-3-Clause
finn_hlslib/interpret.hpp	finn_hlslib	308	8,529	11	4	36	BSD-3-Clause
finn_hlslib/mac.hpp	finn_hlslib	207	8,379	0	3	10	BSD-3-Clause
finn_hlslib/maxpool.h	finn_hlslib	610	23,874	0	10	35	BSD-3-Clause
finn_hlslib/mmv.hpp	finn_hlslib	60	2,449	1	0	1	BSD-3-Clause
finn_hlslib/mvau.hpp	finn_hlslib	311	11,892	0	2	16	BSD-3-Clause
finn_hlslib/slidingwindow.h	finn_hlslib	2,096	87,820	0	20	66	BSD-3-Clause
finn_hlslib/streamtools.h	finn_hlslib	1,009	35,734	4	22	24	BSD-3-Clause

File	Group	Lines	Bytes	Classes	Functions	Directives	Licence
finn_hlslib/tmrcheck.hpp	finn_hlslib	205	9,581	0	0	6	BSD-3-Clause
finn_hlslib/upsample.hpp	finn_hlslib	183	6,383	0	3	2	BSD-3-Clause
finn_hlslib/utils.hpp	finn_hlslib	108	4,173	9	2	0	BSD-3-Clause
finn_hlslib/vvau.hpp	finn_hlslib	277	11,276	0	2	14	BSD-3-Clause
finn_hlslib/weights.hpp	finn_hlslib	170	5,627	5	2	9	BSD-3-Clause

Table 86: Vendor header files (51)

File	Lines	Bytes	Classes	Functions
ap_axi_sdata.h	344	8,240	10	0
ap_common.h	377	10,180	8	10
ap_decl.h	213	6,473	0	0
ap_fixed.h	361	12,185	2	4
ap_fixed_base.h	2,355	85,182	4	107
ap_fixed_ref.h	719	27,302	2	27
ap_fixed_special.h	231	6,985	1	8
ap_int.h	331	10,012	2	16
ap_int_base.h	1,886	71,735	14	114
ap_int_ref.h	1,347	55,218	3	35
ap_int_special.h	224	6,301	1	8
etc/ap_decl.h	202	6,119	0	0
etc/ap_private.h	7,200	261,465	19	335
etc/hls_all_any_apfixed.h	103	1,837	0	8
etc/hls_cbrt_apfixed.h	142	5,575	0	4
etc/hls_comparison_apfixed.h	208	3,783	0	20
etc/hls_cordic_apfixed.h	1,605	36,667	0	64
etc/hls_diff_apfixed.h	283	7,036	0	20
etc/hls_erf_apfixed.h	661	23,172	9	8
etc/hls_exp_apfixed.h	3,447	243,307	0	17
etc/hls_fmod_rem_quo_remainder_divide_apfixed.h	205	6,592	0	16
etc/hls_fract_apfixed.h	43	1,028	0	4
etc/hls_frexp_apfixed.h	104	3,002	0	4
etc/hls_half_fpo.h	610	20,888	24	63
etc/hls_half_x_utils.h	626	18,339	3	87
etc/hls_hotbm_apfixed.h	483	16,508	4	6
etc/hls_hypot_apfixed.h	207	6,908	0	4

File	Lines	Bytes	Classes	Functions
etc/hls_isequal_isnotequal_apfixed.h	146	2,915	0	16
etc/hls_ldexp_apfixed.h	102	2,061	0	12
etc/hls_lgamma_apfixed.h	67	1,878	0	8
etc/hls_log_apfixed.h	1,449	70,493	48	24
etc/hls_mad_apfixed.h	65	1,944	0	4
etc/hls_modf_apfixed.h	60	1,533	0	4
etc/hls_pow_apfixed.h	544	18,358	0	16
etc/hls_round_copysign_apfixed.h	541	11,796	0	60
etc/hls_select_bitselect_apfixed.h	80	2,142	0	4
etc/hls_sqrt_apfixed.h	441	20,674	0	12
etc/hls_tgamma_apfixed.h	47	1,286	0	4
floating_point_v7_1_bitacc_cmodel.h	404	23,001	0	0
gmp.h	2,392	91,885	0	8
hls_compat.h	10	248	0	0
hls_fpo.h	620	18,568	0	44
hls_half.h	156	3,956	1	0
hls_math.h	3,210	78,904	0	339
hls_stream.h	531	13,032	6	30
hls_x_complex.h	375	9,874	1	14
mpfr.h	946	47,985	0	0
utils/std_complex_utils.h	152	5,699	5	0
utils/x_hls_defines.h	138	4,143	2	0
utils/x_hls_traits.h	1,133	38,531	300	0
utils/x_hls_utils.h	685	17,745	61	3
Total (51 files)	38,811	1,450,690	530	1591

Appendix B Every Synthesis Directive

All 445 directives, with file and line, in file order. The DEPENDENCE and LOOP_TRIPCOUNT rows are the assertions discussed in §7.5; they are marked so that the integration checklist there can be worked through mechanically.

Table 87: Every #pragma HLS in the corpus (445)

File	Line	Directive	Kind
Vitis_security/ aes.hpp	15	inline	
Vitis_security/ aes.hpp	16	resource variable = ssbox core = ROM_nP_LUTRAM	
Vitis_security/ aes.hpp	17	resource variable = iibox core = ROM_nP_LUTRAM	
Vitis_security/ aes.hpp	67	inline	
Vitis_security/ aes.hpp	68	ARRAY_PARTITION variable = key_list complete dim = 1	
Vitis_security/ aes.hpp	72	inline off	
Vitis_security/ aes.hpp	78	pipeline II = 1	
Vitis_security/ aes.hpp	79	pipeline II = 1	
Vitis_security/ aes.hpp	138	unroll	
Vitis_security/ aes.hpp	164	unroll	
Vitis_security/ aes.hpp	172	unroll	
Vitis_security/ aes.hpp	205	inline off	
Vitis_security/ aes.hpp	213	pipeline II = 1	
Vitis_security/ aes.hpp	246	inline	
Vitis_security/ aes.hpp	247	ARRAY_PARTITION variable = key_list complete dim = 1	
Vitis_security/ aes.hpp	251	inline off	
Vitis_security/ aes.hpp	259	pipeline II = 1	
Vitis_security/ aes.hpp	315	unroll	
Vitis_security/ aes.hpp	341	unroll	

File	Line	Directive	Kind
Vitis_security/ aes.hpp	349	unroll	
Vitis_security/ aes.hpp	387	inline	
Vitis_security/ aes.hpp	388	ARRAY_PARTITION variable = key_list complete dim = 1	
Vitis_security/ aes.hpp	392	inline off	
Vitis_security/ aes.hpp	397	pipeline II = 1	
Vitis_security/ aes.hpp	442	unroll	
Vitis_security/ aes.hpp	470	unroll	
Vitis_security/ aes.hpp	477	unroll	
Vitis_security/ aes.hpp	540	inline	
Vitis_security/ aes.hpp	541	ARRAY_PARTITION variable = key_list complete dim = 1	
Vitis_security/ aes.hpp	545	inline off	
Vitis_security/ aes.hpp	551	pipeline II = 1	
Vitis_security/ aes.hpp	658	unroll	
Vitis_security/ aes.hpp	672	unroll	
Vitis_security/ aes.hpp	703	inline	
Vitis_security/ aes.hpp	704	ARRAY_PARTITION variable = key_list complete dim = 1	
Vitis_security/ aes.hpp	708	inline off	
Vitis_security/ aes.hpp	716	pipeline II = 1	
Vitis_security/ aes.hpp	819	unroll	
Vitis_security/ aes.hpp	833	unroll	
Vitis_security/ aes.hpp	864	inline	
Vitis_security/ aes.hpp	865	ARRAY_PARTITION variable = key_list complete dim = 1	

File	Line	Directive	Kind
Vitis_security/ aes.hpp	869	inline off	
Vitis_security/ aes.hpp	874	pipeline II = 1	
Vitis_security/ aes.hpp	968	unroll	
Vitis_security/ aes.hpp	983	unroll	
Vitis_security/ sha224_256.hpp	102	pipeline II = 16	
Vitis_security/ sha224_256.hpp	103	loop_tripcount min = 0 max = 1	as- sert
Vitis_security/ sha224_256.hpp	106	array_partition variable = b0.M complete	
Vitis_security/ sha224_256.hpp	110	unroll	
Vitis_security/ sha224_256.hpp	134	array_partition variable = b.M complete	
Vitis_security/ sha224_256.hpp	141	unroll	
Vitis_security/ sha224_256.hpp	157	array_partition variable = b.M complete	
Vitis_security/ sha224_256.hpp	161	pipeline	
Vitis_security/ sha224_256.hpp	210	array_partition variable = b.M complete	
Vitis_security/ sha224_256.hpp	214	unroll	
Vitis_security/ sha224_256.hpp	255	array_partition variable = b1.M complete	
Vitis_security/ sha224_256.hpp	258	unroll	
Vitis_security/ sha224_256.hpp	306	pipeline II = 16	
Vitis_security/ sha224_256.hpp	307	loop_tripcount min = 0 max = 1	as- sert
Vitis_security/ sha224_256.hpp	310	array_partition variable = b0.M complete	
Vitis_security/ sha224_256.hpp	315	unroll	
Vitis_security/ sha224_256.hpp	348	array_partition variable = b.M complete	
Vitis_security/ sha224_256.hpp	355	unroll	

File	Line	Directive	Kind
Vitis_security/ sha224_256.hpp	372	array_partition variable = b.M complete	
Vitis_security/ sha224_256.hpp	376	pipeline	
Vitis_security/ sha224_256.hpp	476	array_partition variable = b1.M complete	
Vitis_security/ sha224_256.hpp	479	unroll	
Vitis_security/ sha224_256.hpp	507	loop_tripcount min = 1 max = 1 avg = 1	as- sert
Vitis_security/ sha224_256.hpp	508	pipeline II = 1	
Vitis_security/ sha224_256.hpp	531	latency max = 65	
Vitis_security/ sha224_256.hpp	534	array_partition variable = blk.M complete	
Vitis_security/ sha224_256.hpp	538	array_partition variable = W complete	
Vitis_security/ sha224_256.hpp	542	pipeline II = 1	
Vitis_security/ sha224_256.hpp	550	pipeline II = 1	
Vitis_security/ sha224_256.hpp	629	array_partition variable = K complete	
Vitis_security/ sha224_256.hpp	639	array_partition variable = H complete	
Vitis_security/ sha224_256.hpp	664	loop_tripcount min = 1 max = 1	as- sert
Vitis_security/ sha224_256.hpp	665	latency max = 65	
Vitis_security/ sha224_256.hpp	683	pipeline II = 1	
Vitis_security/ sha224_256.hpp	703	unroll	
Vitis_security/ sha224_256.hpp	719	unroll	
Vitis_security/ sha224_256.hpp	752	DATAFLOW	
Vitis_security/ sha224_256.hpp	755	STREAM variable = blk_strm depth = 32	
Vitis_security/ sha224_256.hpp	756	RESOURCE variable = blk_strm core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	760	STREAM variable = nblk_strm depth = 32	

File	Line	Directive	Kind
Vitis_security/ sha224_256.hpp	761	RESOURCE variable = nblk_strm core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	763	STREAM variable = nblk_strm1 depth = 32	
Vitis_security/ sha224_256.hpp	764	RESOURCE variable = nblk_strm1 core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	766	STREAM variable = nblk_strm2 depth = 32	
Vitis_security/ sha224_256.hpp	767	RESOURCE variable = nblk_strm2 core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	771	STREAM variable = end_nblk_strm depth = 32	
Vitis_security/ sha224_256.hpp	772	RESOURCE variable = end_nblk_strm core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	774	STREAM variable = end_nblk_strm1 depth = 32	
Vitis_security/ sha224_256.hpp	775	RESOURCE variable = end_nblk_strm1 core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	777	STREAM variable = end_nblk_strm2 depth = 32	
Vitis_security/ sha224_256.hpp	778	RESOURCE variable = end_nblk_strm2 core = FIFO_LUTRAM	
Vitis_security/ sha224_256.hpp	782	STREAM variable = w_strm depth = 32	
Vitis_security/ sha224_256.hpp	783	RESOURCE variable = w_strm core = FIFO_LUTRAM	
Vitis_solver/ cholesky.hpp	368	PIPELINE II = CholeskyTraits::INNER_II	
Vitis_solver/ cholesky.hpp	449	loop_tripcount max = 1 + RowsColsA / 2	as- sert
Vitis_solver/ cholesky.hpp	461	loop_tripcount max = 1 + RowsColsA / 2	as- sert
Vitis_solver/ cholesky.hpp	462	PIPELINE II = CholeskyTraits::INNER_II	
Vitis_solver/ cholesky.hpp	537	ARRAY_PARTITION variable = A cyclic dim = CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR	
Vitis_solver/ cholesky.hpp	538	ARRAY_PARTITION variable = L cyclic dim = CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR	
Vitis_solver/ cholesky.hpp	539	ARRAY_PARTITION variable = L_internal cyclic dim = CholeskyTraits::UNROLL_DIM factor = \	
Vitis_solver/ cholesky.hpp	541	ARRAY_PARTITION variable = square_sum_array cyclic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR	

File	Line	Directive	Kind
Vitis_solver/ cholesky.hpp	542	ARRAY_PARTITION variable = product_sum_array cyclic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR	
Vitis_solver/ cholesky.hpp	575	loop_tripcount max = 1 + RowsColsA / 2	as- sert
Vitis_solver/ cholesky.hpp	590	LOOP_FLATTEN off	
Vitis_solver/ cholesky.hpp	591	PIPELINE II = CholeskyTraits::INNER_II	
Vitis_solver/ cholesky.hpp	592	UNROLL_FACTOR = CholeskyTraits::UNROLL_FACTOR	
Vitis_solver/ cholesky.hpp	656	loop_tripcount max = 1 + RowsColsA / 2	as- sert
Vitis_solver/ cholesky.hpp	657	PIPELINE	
Vitis_solver/ cholesky.hpp	712	PIPELINE	
Vitis_solver/ cholesky.hpp	722	PIPELINE	
Vitis_solver/ qrf.hpp	618	PIPELINE	
Vitis_solver/ qrf.hpp	627	PIPELINE	
Vitis_solver/ qrf.hpp	661	PIPELINE II = QRF_TRAITS::UPDATE_II	
Vitis_solver/ qrf.hpp	670	PIPELINE II = QRF_TRAITS::UPDATE_II	
Vitis_solver/ qrf.hpp	686	PIPELINE	
Vitis_solver/ qrf.hpp	721	ARRAY_PARTITION variable = q_i cyclic dim = 2 factor = QRF_TRAITS::UNROLL_FACTOR	
Vitis_solver/ qrf.hpp	722	ARRAY_PARTITION variable = r_i cyclic dim = 2 factor = QRF_TRAITS::UNROLL_FACTOR	
Vitis_solver/ qrf.hpp	725	STREAM variable = to_rot depth = RowsA / 2	
Vitis_solver/ qrf.hpp	734	STREAM variable = rotations depth = RowsA / 2	
Vitis_solver/ qrf.hpp	742	LOOP_MERGE force	
Vitis_solver/ qrf.hpp	745	PIPELINE	
Vitis_solver/ qrf.hpp	754	PIPELINE	
Vitis_solver/ qrf.hpp	764	LOOP_TRIPCOUNT min = 1 max = RowsA / 2	as- sert

File	Line	Directive	Kind
Vitis_solver/ qrf.hpp	765	PIPELINE II = QRF_TRAITS::CALC_ROT_II	
Vitis_solver/ qrf.hpp	788	LOOP_TRIPCOUNT min = 1 max = RowsA / 2	as- sert
Vitis_solver/ qrf.hpp	805	LOOP_MERGE force	
Vitis_solver/ qrf.hpp	808	PIPELINE II = QRF_TRAITS::UPDATE_II	
Vitis_solver/ qrf.hpp	809	UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR	
Vitis_solver/ qrf.hpp	818	PIPELINE II = QRF_TRAITS::UPDATE_II	
Vitis_solver/ qrf.hpp	819	UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR	
Vitis_solver/ qrf.hpp	829	LOOP_MERGE force	
Vitis_solver/ qrf.hpp	832	PIPELINE	
Vitis_solver/ svd.hpp	98	inline off	
Vitis_solver/ svd.hpp	336	inline	
Vitis_solver/ svd.hpp	449	inline	
Vitis_solver/ svd.hpp	646	inline	
Vitis_solver/ svd.hpp	946	PIPELINE II = SVDTraits::OFF_DIAG_II	
Vitis_solver/ svd.hpp	975	PIPELINE II = SVDTraits::OFF_DIAG_II	
Vitis_solver/ svd.hpp	1106	ARRAY_PARTITION variable=diag1_i cyclic dim=2 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1107	ARRAY_PARTITION variable=diag2_i cyclic dim=2 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1108	ARRAY_PARTITION variable=s_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1109	ARRAY_PARTITION variable=s_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1110	ARRAY_PARTITION variable=u_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1111	ARRAY_PARTITION variable=u_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1112	ARRAY_PARTITION variable=v_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	

File	Line	Directive	Kind
Vitis_solver/ svd.hpp	1113	ARRAY_PARTITION variable=v_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1114	ARRAY_PARTITION variable=diag_w_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1115	ARRAY_PARTITION variable=diag_x_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1116	ARRAY_PARTITION variable=diag_y_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1117	ARRAY_PARTITION variable=diag_z_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1118	ARRAY_PARTITION variable=uw_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1119	ARRAY_PARTITION variable=ux_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1120	ARRAY_PARTITION variable=uy_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1121	ARRAY_PARTITION variable=uz_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1122	ARRAY_PARTITION variable=vw_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1123	ARRAY_PARTITION variable=vx_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1124	ARRAY_PARTITION variable=vy_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1125	ARRAY_PARTITION variable=vz_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1135	PIPELINE	
Vitis_solver/ svd.hpp	1166	UNROLL FACTOR = SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1185	UNROLL FACTOR = SVDTraits::DIAG_UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1186	PIPELINE II = SVDTraits::DIAG_II	
Vitis_solver/ svd.hpp	1234	UNROLL FACTOR = SVDTraits::UNROLL_FACTOR	
Vitis_solver/ svd.hpp	1251	PIPELINE II = SVDTraits::OFF_DIAG_II	
Vitis_solver/ svd.hpp	1253	INLINE recursive	
Vitis_solver/ svd.hpp	1260	dependence variable=s_col1 inter false	as- sert
Vitis_solver/ svd.hpp	1261	dependence variable=s_col2 inter false	as- sert

File	Line	Directive	Kind
Vitis_solver/ svd.hpp	1262	dependence variable=u_col1 inter false	as- sert
Vitis_solver/ svd.hpp	1263	dependence variable=u_col2 inter false	as- sert
Vitis_solver/ svd.hpp	1264	dependence variable=v_col1 inter false	as- sert
Vitis_solver/ svd.hpp	1265	dependence variable=v_col2 inter false	as- sert
Vitis_solver/ svd.hpp	1477	PIPELINE	
Vitis_solver/ svd.hpp	1549	PIPELINE	
Vitis_solver/ svd.hpp	1557	PIPELINE	
Vitis_solver/ svd.hpp	1563	PIPELINE	
Vitis_solver/ svd.hpp	1569	PIPELINE	
Vitis_solver_L2/po- trf.hpp	37	loop_tripcount min = 1 max = 8	as- sert
Vitis_solver_L2/po- trf.hpp	39	BIND_STORAGE variable = tmp_i type = ram_2p impl = lutram	
Vitis_solver_L2/po- trf.hpp	43	loop_tripcount min = 1 max = N	as- sert
Vitis_solver_L2/po- trf.hpp	44	pipeline	
Vitis_solver_L2/po- trf.hpp	45	dependence variable = tmp_i inter false	as- sert
Vitis_solver_L2/po- trf.hpp	46	dependence variable = dataA inter false	as- sert
Vitis_solver_L2/po- trf.hpp	52	pipeline	
Vitis_solver_L2/po- trf.hpp	58	pipeline	
Vitis_solver_L2/po- trf.hpp	64	pipeline	
Vitis_solver_L2/po- trf.hpp	77	BIND_STORAGE variable = tmp type = ram_2p impl = lutram	
Vitis_solver_L2/po- trf.hpp	81	pipeline	
Vitis_solver_L2/po- trf.hpp	82	dependence variable = tmp inter false	as- sert
Vitis_solver_L2/po- trf.hpp	83	dependence variable = dataA inter false	as- sert

File	Line	Directive	Kind
Vitis_solver_L2/po-trf.hpp	87	unroll	
Vitis_solver_L2/po-trf.hpp	94	pipeline	
Vitis_solver_L2/po-trf.hpp	100	pipeline	
Vitis_solver_L2/po-trf.hpp	106	pipeline	
Vitis_solver_L2/po-trf.hpp	131	DATAFLOW	
Vitis_solver_L2/po-trf.hpp	135	unroll factor = NCU	
Vitis_solver_L2/po-trf.hpp	143	array_partition variable = dataj cyclic factor = NCU	
Vitis_solver_L2/po-trf.hpp	175	array_partition variable = matA cyclic factor = NCU	
Vitis_solver_L2/po-trf.hpp	176	BIND_STORAGE variable = matA type = xpm_memory impl = uram	
Vitis_solver_L2/po-trf.hpp	181	pipeline	
Vitis_solver_L2/po-trf.hpp	191	pipeline	
finn_hlslib/activations.hpp	145	inline	
finn_hlslib/activations.hpp	163	inline	
finn_hlslib/activations.hpp	180	inline	
finn_hlslib/activations.hpp	185	inline	
finn_hlslib/activations.hpp	208	inline	
finn_hlslib/activations.hpp	214	inline	
finn_hlslib/activations.hpp	217	unroll	
finn_hlslib/activations.hpp	247	inline	
finn_hlslib/activations.hpp	252	inline	
finn_hlslib/activations.hpp	296	pipeline style=flp II=1	
finn_hlslib/activations.hpp	302	UNROLL	

File	Line	Directive	Kind
finn_hlslib/activations.hpp	352	ARRAY_PARTITION variable=internal_thr.m_thresholds complete dim=0	
finn_hlslib/activations.hpp	358	pipeline style=flp II=1	
finn_hlslib/activations.hpp	371	UNROLL	
finn_hlslib/activations.hpp	376	UNROLL	
finn_hlslib/convlayer.h	113	INLINE	
finn_hlslib/convlayer.h	198	INLINE	
finn_hlslib/convlayer.h	284	INLINE	
finn_hlslib/dma.h	141	pipeline style=flp II=1	
finn_hlslib/dma.h	154	pipeline style=flp II=1	
finn_hlslib/dma.h	224	pipeline style=flp II=1	
finn_hlslib/interpret.hpp	62	inline	
finn_hlslib/interpret.hpp	67	inline	
finn_hlslib/interpret.hpp	72	inline	
finn_hlslib/interpret.hpp	80	inline	
finn_hlslib/interpret.hpp	89	inline	
finn_hlslib/interpret.hpp	97	inline	
finn_hlslib/interpret.hpp	102	inline	
finn_hlslib/interpret.hpp	116	inline	
finn_hlslib/interpret.hpp	122	inline	
finn_hlslib/interpret.hpp	138	inline	
finn_hlslib/interpret.hpp	143	inline	
finn_hlslib/interpret.hpp	147	inline	
finn_hlslib/interpret.hpp	151	inline	
finn_hlslib/interpret.hpp	155	inline	

File	Line	Directive	Kind
finn_hlslib/inter-pret.hpp	163	inline	
finn_hlslib/inter-pret.hpp	168	inline	
finn_hlslib/inter-pret.hpp	173	inline	
finn_hlslib/inter-pret.hpp	204	inline	
finn_hlslib/inter-pret.hpp	207	inline	
finn_hlslib/inter-pret.hpp	211	inline	
finn_hlslib/inter-pret.hpp	215	inline	
finn_hlslib/inter-pret.hpp	220	inline	
finn_hlslib/inter-pret.hpp	224	inline	
finn_hlslib/inter-pret.hpp	232	inline	
finn_hlslib/inter-pret.hpp	237	inline	
finn_hlslib/inter-pret.hpp	242	inline	
finn_hlslib/inter-pret.hpp	261	inline	
finn_hlslib/inter-pret.hpp	264	inline	
finn_hlslib/inter-pret.hpp	267	inline	
finn_hlslib/inter-pret.hpp	271	inline	
finn_hlslib/inter-pret.hpp	275	inline	
finn_hlslib/inter-pret.hpp	279	inline	
finn_hlslib/inter-pret.hpp	284	inline	
finn_hlslib/inter-pret.hpp	292	inline	
finn_hlslib/inter-pret.hpp	297	inline	
finn_hlslib/inter-pret.hpp	302	inline	
finn_hlslib/mac.hpp	89	inline	

File	Line	Directive	Kind
finn_hlslib/mac.hpp	114	inline	
finn_hlslib/mac.hpp	116	BIND_OP variable=res op=mul impl=fabric	
finn_hlslib/mac.hpp	140	inline	
finn_hlslib/mac.hpp	142	BIND_OP variable=res op=mul impl=dsp	
finn_hlslib/mac.hpp	165	inline	
finn_hlslib/mac.hpp	168	unroll	
finn_hlslib/mac.hpp	192	inline	
finn_hlslib/mac.hpp	195	unroll	
finn_hlslib/mac.hpp	202	inline	
finn_hlslib/max-pool.h	74	UNROLL	
finn_hlslib/max-pool.h	83	pipeline style=flp II=1	
finn_hlslib/max-pool.h	91	pipeline style=flp II=1	
finn_hlslib/max-pool.h	146	ARRAY_PARTITION variable=buf complete dim=2	
finn_hlslib/max-pool.h	149	UNROLL	
finn_hlslib/max-pool.h	159	pipeline style=flp II=1	
finn_hlslib/max-pool.h	162	UNROLL	
finn_hlslib/max-pool.h	176	UNROLL	
finn_hlslib/max-pool.h	210	INLINE	
finn_hlslib/max-pool.h	255	ARRAY_PARTITION variable=buf complete dim=2	
finn_hlslib/max-pool.h	258	pipeline style=flp II=1	
finn_hlslib/max-pool.h	260	UNROLL	
finn_hlslib/max-pool.h	272	pipeline style=flp II=1	
finn_hlslib/max-pool.h	275	UNROLL	
finn_hlslib/max-pool.h	288	pipeline style=flp II=1	
finn_hlslib/max-pool.h	290	UNROLL	
finn_hlslib/max-pool.h	302	pipeline style=flp II=1	

File	Line	Directive	Kind
finn_hlslib/max-pool.h	331	INLINE	
finn_hlslib/max-pool.h	370	pipeline style=flp II=1	
finn_hlslib/max-pool.h	373	UNROLL	
finn_hlslib/max-pool.h	415	bind_storage variable=accumulators type=RAM_2P impl=LUTRAM	
finn_hlslib/max-pool.h	421	pipeline style=flp II=1	
finn_hlslib/max-pool.h	425	UNROLL	
finn_hlslib/max-pool.h	477	ARRAY_PARTITION variable=topval complete dim=1	
finn_hlslib/max-pool.h	479	ARRAY_PARTITION variable=toplabels complete dim=1	
finn_hlslib/max-pool.h	484	UNROLL	
finn_hlslib/max-pool.h	489	pipeline style=flp II=1	
finn_hlslib/max-pool.h	492	UNROLL	
finn_hlslib/max-pool.h	501	UNROLL	
finn_hlslib/max-pool.h	508	UNROLL	
finn_hlslib/max-pool.h	570	ARRAY_PARTITION variable=accu complete dim=0	
finn_hlslib/max-pool.h	576	pipeline style=flp II=1	
finn_hlslib/max-pool.h	583	UNROLL	
finn_hlslib/max-pool.h	590	UNROLL	
finn_hlslib/max-pool.h	599	UNROLL	
finn_hlslib/mmv.hpp	53	inline	
finn_hlslib/mvau.hpp	110	ARRAY_PARTITION variable=inputBuf complete dim=0	
finn_hlslib/mvau.hpp	114	ARRAY_PARTITION variable=accu complete dim=0	
finn_hlslib/mvau.hpp	124	pipeline style=flp II=1	
finn_hlslib/mvau.hpp	140	UNROLL	

File	Line	Directive	Kind
finn_hlslib/ mvau.hpp	142	UNROLL	
finn_hlslib/ mvau.hpp	151	UNROLL	
finn_hlslib/ mvau.hpp	165	UNROLL	
finn_hlslib/ mvau.hpp	167	UNROLL	
finn_hlslib/ mvau.hpp	232	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/ mvau.hpp	235	ARRAY_PARTITION variable=accu complete dim=0	
finn_hlslib/ mvau.hpp	238	ARRAY_PARTITION variable=w.m_weights complete dim=0	
finn_hlslib/ mvau.hpp	250	pipeline style=flp II=1	
finn_hlslib/ mvau.hpp	267	UNROLL	
finn_hlslib/ mvau.hpp	274	UNROLL	
finn_hlslib/ mvau.hpp	281	UNROLL	
finn_hlslib/ mvau.hpp	294	UNROLL	
finn_hlslib/slidingwindow.h	74	BIND_STORAGE variable=inputBuf type=RAM_2P	
finn_hlslib/slidingwindow.h	95	BIND_STORAGE variable=inputBuf type=RAM_S2P impl=BRAM	
finn_hlslib/slidingwindow.h	116	BIND_STORAGE variable=inputBuf type=RAM_S2P impl=URAM	
finn_hlslib/slidingwindow.h	137	BIND_STORAGE variable=inputBuf type=RAM_S2P impl=LUTRAM	
finn_hlslib/slidingwindow.h	177	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	192	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	308	DEPENDENCE variable=inputBuf inter false	as- sert
finn_hlslib/slidingwindow.h	309	DEPENDENCE variable=inputBuf intra false	as- sert
finn_hlslib/slidingwindow.h	310	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	311	ARRAY_PARTITION variable=inputBuf complete dim=2	

File	Line	Directive	Kind
finn_hlslib/slidingwindow.h	326	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	333	UNROLL	
finn_hlslib/slidingwindow.h	359	UNROLL	
finn_hlslib/slidingwindow.h	392	UNROLL	
finn_hlslib/slidingwindow.h	466	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	467	DEPENDENCE variable=inputBuf inter false	assert
finn_hlslib/slidingwindow.h	468	DEPENDENCE variable=inputBuf intra false	assert
finn_hlslib/slidingwindow.h	476	DEPENDENCE variable=current_block_write intra false	assert
finn_hlslib/slidingwindow.h	479	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	544	DEPENDENCE variable=inputBuf inter false	assert
finn_hlslib/slidingwindow.h	545	DEPENDENCE variable=inputBuf intra false	assert
finn_hlslib/slidingwindow.h	554	DEPENDENCE variable=current_block_write intra false	assert
finn_hlslib/slidingwindow.h	610	DEPENDENCE variable=inputBuf inter false	assert
finn_hlslib/slidingwindow.h	611	DEPENDENCE variable=inputBuf intra false	assert
finn_hlslib/slidingwindow.h	612	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	613	ARRAY_PARTITION variable=inputBuf complete dim=2	
finn_hlslib/slidingwindow.h	628	DEPENDENCE variable=current_block_write intra false	assert
finn_hlslib/slidingwindow.h	631	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	638	UNROLL	
finn_hlslib/slidingwindow.h	672	UNROLL	
finn_hlslib/slidingwindow.h	705	UNROLL	
finn_hlslib/slidingwindow.h	716	DEPENDENCE variable=current_block_write intra false	assert

File	Line	Directive	Kind
finn_hlslib/slidingwindow.h	767	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	782	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	903	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	904	DEPENDENCE variable=inputBuf inter false	assert
finn_hlslib/slidingwindow.h	905	DEPENDENCE variable=inputBuf intra false	assert
finn_hlslib/slidingwindow.h	915	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	917	DEPENDENCE variable=current_block_write intra false	assert
finn_hlslib/slidingwindow.h	983	DEPENDENCE variable=inputBuf inter false	assert
finn_hlslib/slidingwindow.h	984	DEPENDENCE variable=inputBuf intra false	assert
finn_hlslib/slidingwindow.h	993	DEPENDENCE variable=current_block_write intra false	assert
finn_hlslib/slidingwindow.h	1048	DEPENDENCE variable=inputBuf inter false	assert
finn_hlslib/slidingwindow.h	1049	DEPENDENCE variable=inputBuf intra false	assert
finn_hlslib/slidingwindow.h	1050	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	1051	ARRAY_PARTITION variable=inputBuf complete dim=2	
finn_hlslib/slidingwindow.h	1066	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1073	UNROLL	
finn_hlslib/slidingwindow.h	1099	UNROLL	
finn_hlslib/slidingwindow.h	1132	UNROLL	
finn_hlslib/slidingwindow.h	1191	performance target_ti=IFMDim*IFMDim*IFMChannels/SIMD	
finn_hlslib/slidingwindow.h	1198	pipeline style=flp II=IFMChannels/SIMD	
finn_hlslib/slidingwindow.h	1245	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1300	ARRAY_PARTITION variable=inputBuf complete dim=1	

File	Line	Directive	Kind
finn_hlslib/slidingwindow.h	1315	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1434	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	1449	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1575	ARRAY_PARTITION variable=inputBuf complete dim=1	
finn_hlslib/slidingwindow.h	1590	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1710	ARRAY_PARTITION variable=inputBuf complete	
finn_hlslib/slidingwindow.h	1718	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1739	UNROLL	
finn_hlslib/slidingwindow.h	1824	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1908	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	1984	pipeline style=flp II=1	
finn_hlslib/slidingwindow.h	2063	pipeline style=flp II=1	
finn_hlslib/streamtools.h	77	pipeline style=flp II=1	
finn_hlslib/streamtools.h	160	pipeline style=flp II=1	
finn_hlslib/streamtools.h	234	pipeline style=flp II=1	
finn_hlslib/streamtools.h	280	pipeline style=flp II=1	
finn_hlslib/streamtools.h	374	inline	
finn_hlslib/streamtools.h	451	pipeline style=flp II=1	
finn_hlslib/streamtools.h	471	pipeline style=flp II=1	
finn_hlslib/streamtools.h	482	pipeline style=flp II=1	
finn_hlslib/streamtools.h	591	pipeline style=flp II=1	
finn_hlslib/streamtools.h	649	pipeline style=flp II=1	

File	Line	Directive	Kind
finn_hlslib/streamtools.h	654	UNROLL	
finn_hlslib/streamtools.h	723	INLINE	
finn_hlslib/streamtools.h	770	pipeline style=flp II=1	
finn_hlslib/streamtools.h	777	UNROLL	
finn_hlslib/streamtools.h	793	pipeline style=flp II=1	
finn_hlslib/streamtools.h	799	UNROLL	
finn_hlslib/streamtools.h	811	pipeline style=flp II=1	
finn_hlslib/streamtools.h	815	UNROLL	
finn_hlslib/streamtools.h	851	pipeline style=flp II=1	
finn_hlslib/streamtools.h	855	UNROLL	
finn_hlslib/streamtools.h	882	pipeline style=flp II=1	
finn_hlslib/streamtools.h	886	UNROLL	
finn_hlslib/streamtools.h	975	pipeline style=flp II=1	
finn_hlslib/streamtools.h	998	pipeline style=flp II=1	
finn_hlslib/tm-rcheck.hpp	82	ARRAY_PARTITION variable=red_ch_index complete dim=0	
finn_hlslib/tm-rcheck.hpp	91	pipeline style=flp II=1	
finn_hlslib/tm-rcheck.hpp	99	ARRAY_PARTITION variable=numerrors complete dim=0	
finn_hlslib/tm-rcheck.hpp	103	UNROLL	
finn_hlslib/tm-rcheck.hpp	129	ARRAY_PARTITION variable=compute complete dim=0	
finn_hlslib/tm-rcheck.hpp	132	UNROLL	
finn_hlslib/up-sample.hpp	86	pipeline style=flp II=1	
finn_hlslib/up-sample.hpp	175	pipeline II=1 style=flp	

File	Line	Directive	Kind
finn_hlslib/ vvau.hpp	103	ARRAY_PARTITION variable=accu complete dim=0	
finn_hlslib/ vvau.hpp	112	pipeline style=flp II=1	
finn_hlslib/ vvau.hpp	119	UNROLL	
finn_hlslib/ vvau.hpp	128	UNROLL	
finn_hlslib/ vvau.hpp	142	UNROLL	
finn_hlslib/ vvau.hpp	144	UNROLL	
finn_hlslib/ vvau.hpp	213	ARRAY_PARTITION variable=accu complete dim=0	
finn_hlslib/ vvau.hpp	223	pipeline style=flp II=1	
finn_hlslib/ vvau.hpp	230	UNROLL	
finn_hlslib/ vvau.hpp	239	ARRAY_PARTITION variable=w.m_weights complete dim=0	
finn_hlslib/ vvau.hpp	241	UNROLL	
finn_hlslib/ vvau.hpp	246	UNROLL	
finn_hlslib/ vvau.hpp	260	UNROLL	
finn_hlslib/ vvau.hpp	262	UNROLL	
finn_hlslib/ weights.hpp	83	inline	
finn_hlslib/ weights.hpp	88	inline	
finn_hlslib/ weights.hpp	95	inline	
finn_hlslib/ weights.hpp	127	inline	
finn_hlslib/ weights.hpp	132	inline	
finn_hlslib/ weights.hpp	135	unroll	
finn_hlslib/ weights.hpp	145	inline	
finn_hlslib/ weights.hpp	157	inline	

File	Line	Directive	Kind
finn_hlslib/ weights.hpp	160	unroll	

Appendix C Every Function and Class Definition

1,784 function definitions and 601 class or struct definitions, with the template signature of each. The corpus tables are grouped by file and repeat the tables shown in Chapters 3 to 6; the vendor tables appear here only, because the vendor headers are the dependency rather than the subject.

C.1 Corpus

Vitis_security/aes.hpp

Table 88: Every class and struct in Vitis_security/aes.hpp (9 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
12	class	aesTable	—	—	—
55	class	aesEnc	template <int W>	—	—
62	class	aesEnc <256>	template <>	: public aesTable	—
241	class	aesEnc <192>	template <>	: public aesTable	—
383	class	aesEnc <128>	template <>	: public aesTable	—
516	class	aesDec	template <int W>	—	—
535	class	aesDec <256>	template <>	: public aesTable	—
698	class	aesDec <192>	template <>	: public aesTable	—
859	class	aesDec <128>	template <>	: public aesTable	—

Table 89: Every function definition in Vitis_security/aes.hpp (30 definitions)

Line	Template signature	Returns	Name	Parameters
13	—	public:	aesTable	()
56	template <int W>	public: void	updateKey	ap_uint<W> cipherkey
58	template <int W>	void	process	ap_uint<128> plaintext, ap_uint<W> cipherkey, ap_uint<128>& ciphertext
71	template <>	void	updateKey	ap_uint<256> cipherkey
114	—	static ap_uint<8>	GFMul2	ap_uint<8> a
126	—	void	process	ap_uint<128> plaintext, ap_uint<256> cipherkey, ap_uint<128>& ciphertext
204	—	void	updateKey_2	ap_uint<256> cipherkey
250	template <>	void	updateKey	ap_uint<192> cipherkey
291	—	static ap_uint<8>	GFMul2	ap_uint<8> a

Line	Template signature	Returns	Name	Parameters
303	—	void	process	ap_uint<128> plaintext, ap_uint<192> cipherkey, ap_uint<128>& ciphertext
391	template <>	void	updateKey	ap_uint<128> cipherkey
418	template <>	static ap_uint<8>	GFMul2	ap_uint<8> a
429	—	void	process	ap_uint<128> plaintext, ap_uint<128> cipherkey, ap_uint<128>& ciphertext
517	template <int W>	public: void	updateKey	ap_uint<W> cipherkey
531	template <int W>	void	process	ap_uint<128> ciphertext, ap_uint<W> cipherkey, ap_uint<128>& plaintext
544	template <>	void	updateKey	ap_uint<256> cipherkey
584	—	static ap_uint<8>	GFMul2	ap_uint<8> a
596	—	static ap_uint<8>	GFMul4	ap_uint<8> a
610	—	static ap_uint<8>	GFMul8	ap_uint<8> a
624	—	void	process	ap_uint<128> ciphertext, ap_uint<256> cipherkey, ap_uint<128>& plaintext
707	template <>	void	updateKey	ap_uint<192> cipherkey
746	—	static ap_uint<8>	GFMul2	ap_uint<8> a
758	—	static ap_uint<8>	GFMul4	ap_uint<8> a
772	—	static ap_uint<8>	GFMul8	ap_uint<8> a
786	—	void	process	ap_uint<128> ciphertext, ap_uint<192> cipherkey, ap_uint<128>& plaintext
868	template <>	void	updateKey	ap_uint<128> cipherkey
895	template <>	static ap_uint<8>	GFMul2	ap_uint<8> a
907	—	static ap_uint<8>	GFMul4	ap_uint<8> a
921	—	static ap_uint<8>	GFMul8	ap_uint<8> a
935	—	void	process	ap_uint<128> ciphertext, ap_uint<128> cipherkey, ap_uint<128>& plaintext

Vitis_security/sha224_256.hpp

Table 90: Every class and struct in Vitis_security/sha224_256.hpp (3 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
55	struct	SHA256Block	—	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
64	struct	sha256_digest_config <true>	template <>	—	—
69	struct	sha256_digest_config <false>	template <>	—	—

Table 91: Every function definition in Vitis_security/sha224_256.hpp (6 definitions)

Line	Template signature	Re-returns	Name	Parameters
498	—	in-line void	dup_strm	hls::stream<uint64_t>& in_strm, hls::stream<bool>& in_e_strm, hls::stream<uint64_t>& out1_strm, hls::stream<bool>& out1_e_strm, hls::stream<uint64_t>& out2_strm, hls::stream<bool>& out2_e_strm
523	—	in-line void	generateMsgSchedule	hls::stream<SHA256Block>& blk_strm, hls::stream<uint64_t>& nblk_strm, hls::stream<bool>& end_nblk_strm, hls::stream<uint32_t>& w_strm
609	template<int h_width>	void	sha256Digest	hls::stream<uint64_t>& nblk_strm, hls::stream<bool>& end_nblk_strm, hls::stream<uint32_t>& w_strm, hls::stream<ap_uint<h_width>>& hash_strm, hls::stream<bool>& end_hash_strm
746	template<int m_width, int h_width>	in-line void	sha256_top	hls::stream<ap_uint<m_width>>& msg_strm, hls::stream<ap_uint<64>>& len_strm, hls::stream<bool>& end_len_strm, hls::stream<ap_uint<h_width>>& hash_strm, hls::stream<bool>& end_hash_strm
809	template<int m_width>	void	sha224	hls::stream<ap_uint<m_width>>& msg_strm, hls::stream<ap_uint<64>>& len_strm, hls::stream<bool>& end_len_strm, hls::stream<ap_uint<224>>& hash_strm, hls::stream<bool>& end_hash_strm
826	template<int m_width>	void	sha256	hls::stream<ap_uint<m_width>>& msg_strm, hls::stream<ap_uint<64>>& len_strm, hls::stream<bool>& end_len_strm, hls::stream<ap_uint<256>>& hash_strm, hls::stream<bool>& end_hash_strm

Vitis_solver/cholesky.hpp

Table 92: Every class and struct in Vitis_solver/cholesky.hpp (6 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
45	struct	choleskyTraits	template <bool LowerTriangularL, int RowsColsA, typename InputType, typename OutputType>	—	typedef InputType PROD_T; typedef InputType ACCUM_T; typedef InputType ADD_T; typedef InputType DIAG_T; typedef InputType RE-CIP_DIAG_T; typedef InputType OFF_DIAG_T; typedef OutputType L_OUTPUT_T;
65	struct	choleskyTraits <LowerTriangularL, RowsColsA, hls::x_complex<InputBaseType>, hls::x_complex<OutputBaseType> >	template <bool LowerTriangularL, int RowsColsA, typename InputBaseType, typename OutputBaseType>	—	typedef hls::x_complex<InputBaseType> PROD_T; typedef hls::x_complex<InputBaseType> ACCUM_T; typedef hls::x_complex<InputBaseType> ADD_T; typedef hls::x_complex<InputBaseType> DIAG_T; typedef InputBaseType RE-CIP_DIAG_T; typedef hls::x_complex<InputBaseType> OFF_DIAG_T; typedef hls::x_complex<OutputBaseType> L_OUTPUT_T;
82	struct	choleskyTraits <LowerTriangularL, RowsColsA, std::complex<InputBaseType>, std::complex<OutputBaseType> >	template <bool LowerTriangularL, int RowsColsA, typename InputBaseType, typename OutputBaseType>	—	typedef std::complex<InputBaseType> PROD_T; typedef std::complex<InputBaseType> ACCUM_T; typedef std::complex<InputBaseType> ADD_T; typedef std::complex<InputBaseType> DIAG_T; typedef InputBaseType RE-CIP_DIAG_T; typedef std::complex<InputBaseType> OFF_DIAG_T; typedef std::complex<OutputBaseType> L_OUTPUT_T;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
110	struct	choleskyTraits <LowerTriangularL, RowsColsA, ap_fixed<W1, I1, Q1, O1, N1>, ap_fixed<W2, I2, Q2, O2, N2>>	template <bool LowerTriangularL, int RowsColsA, int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>	—	typedef ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT, 0> PROD_T; typedef ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0> ADD_T; typedef ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> DIAG_T; typedef ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> OFF_DIAG_T; typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2), AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T; typedef ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0> L_OUTPUT_T;
144	struct	choleskyTraits <LowerTriangularL, RowsColsA, hls::x_complex<ap_fixed<W1, I1, Q1, O1, N1>, hls::x_complex<ap_fixed<W2, I2, Q	template <bool LowerTriangularL, int RowsColsA, int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>	—	typedef hls::x_complex<ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT, 0> > PROD_T; typedef hls::x_complex<ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0> > ADD_T; typedef hls::x_complex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > DIAG_T; typedef hls::x_complex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > OFF_DIAG_T; typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2), AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T; typedef hls::x_complex<ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0> > L_OUTPUT_T;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
181	struct	choleskyTraits <LowerTriangularL, RowsColsA, std::com- plex<ap_fixed<W1, I1, Q1, O1, N1> >, std::com- plex<ap_fixed<W2, I2, Q2, O	template <bool LowerTri- angularL, int RowsColsA, int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>	—	typedef std::com- plex<ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT, 0> > PROD_T; typedef std::com- plex<ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0> > ADD_T; typedef std::com- plex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > DIAG_T; typedef std::com- plex<ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV, AP_SAT, 0> > OFF_DIAG_T; typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2), AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T; typedef std::com- plex<ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0> > L_OUTPUT_T;

Table 93: Every function definition in Vitis_solver/cholesky.hpp (13 definitions)

Line	Template signature	Returns	Name	Parameters
213	template <typename T_IN, type- name T_OUT>	int	cholesky_sqrt_op	T_IN a, T_OUT& b
224	template <typename T_IN, type- name T_OUT>	int	cholesky_sqrt_op	hls::x_complex<T_IN> din, hls::x_com- plex<T_OUT>& dout
239	template <typename T_IN, type- name T_OUT>	int	cholesky_sqrt_op	std::complex<T_IN> din, std::complex<T_OUT>& dout
256	template <typename InputType, typename OutputType>	void	cholesky_rsqr	InputType x, OutputType& res
261	—	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1, int W2, int I2, ap_q_mode Q2	cholesky_rsqr	ap_fixed<W1, I1, Q1, O1, N1> x, ap_fixed<W2, I2, Q2, O2, N2>& res
275	template <typename AType, type- name BType, typename CType>	void	cholesky_prod_sum_mult	AType A, BType B, CType& C
280	template <typename AType, type- name BType, typename CType>	void	cholesky_prod_sum_mult	hls::x_complex<AType> A, BType B, hls::x_com- plex<CType>& C

Line	Template signature	Returns	Name	Parameters
286	template <typename AType, typename BType, typename CType>	void	cholesky_prod_sum_mult	std::complex<AType> A, BType B, std::complex<CType>& C
295	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyBasic	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
412	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyAlt	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
515	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyAlt2	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
673	—	template <bool LowerTriangularL, int RowsColsA, typename CholeskyTraits, class InputType,	choleskyTop	const InputType A[RowsColsA][RowsColsA], OutputType L[RowsColsA][RowsColsA]
707	template <bool LowerTriangularL, int RowsColsA, class InputType, class OutputType, typename TRAITS = choleskyTraits<LowerTriangularL, RowsColsA, InputType, OutputType> >	int	cholesky	hls::stream<InputType>& matrixAstrm, hls::stream<OutputType>& matrixLstrm

Vitis_solver/qrf.hpp

Table 94: Every class and struct in Vitis_solver/qrf.hpp (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
45	struct	qrfTraits	—	—	—
56	struct	is_cmplx	template <typename T>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
61	struct	is_cmplx <hls::x_complex<T> >	template <typename T>	—	—
66	struct	is_cmplx <std::complex<T> >	template <typename T>	—	—
468	struct	qrf_alt_config	template <int ROWS, int COLS, typename InputType>	—	—

Table 95: Every function definition in Vitis_solver/qrf.hpp (16 definitions)

Line	Template signature	Returns	Name	Parameters
70	template <type-name T>	template <type-name T> int	qrf_require_extra_pass	const int rows, const int cols, T x
76	template <type-name T>	int	qrf_require_extra_pass	const int rows, const int cols, hls::x_complex<T> x
87	template <type-name T>	int	qrf_require_extra_pass	const int rows, const int cols, std::complex<T> x
98	template <class T>	bool	is_zero	T a
124	template <type-name T>	T	qrf_vm	T a, T b, T c, T d
130	template <type-name T>	void	qrf_mm	const T G[2][2], T& a, T& b
140	template <type-name T>	void	qrf_mm_or_mag	const T G[2][2], T& a, T& b, const T mag, const int use_mag, const int extra_pass
161	template <type-name T>	T	qrf_magnitude	T a, T b
187	template <type-name T>	T	qrf_magnitude	hls::x_complex<T> a, hls::x_complex<T> b
244	template <type-name T>	T	qrf_magnitude	std::complex<T> a, std::complex<T> b
306	template <type-name T>	void	qrf_givens	int extra_pass, T a, T b, T& c, T& s, T& ss, T& cc, T& r
353	template <type-name T>	void	qrf_givens	int extra_pass, hls::x_complex<T> a, hls::x_complex<T> b, hls::x_complex<T>& c, hls::x_complex<T>& s, hls::x_complex<T>& ss, hls::x_complex<T>& cc, hls::x_complex<T>& r
407	template <type-name T>	void	qrf_givens	int extra_pass, std::complex<T> a, std::complex<T> b, std::complex<T>& c, std::complex<T>& s, std::complex<T>& ss, std::complex<T>& cc, std::complex<T>& r

Line	Template signature	Returns	Name	Parameters
587	—	template <bool TransposedQ, int RowsA, int ColsA, typename QRF_TRAITS, typename InputType,	qrf_basic	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixQStrm, hls::stream<OutputType>& matrixRStrm
702	—	template <bool TransposedQ, int RowsA, int ColsA, typename QRF_TRAITS, typename InputType,	qrf_alt	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixQStrm, hls::stream<OutputType>& matrixRStrm
868	template <bool TransposedQ, int RowsA, int ColsA, typename InputType, typename OutputType, typename QRF_TRAITS = qrfTraits>	void	qrf	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixQStrm, hls::stream<OutputType>& matrixRStrm

Vitis_solver/svd.hpp

Table 96: Every class and struct in Vitis_solver/svd.hpp (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
46	struct	svdTraits	template <int RowsA, int ColsA, typename InputType, typename OutputType>	—	typedef OutputType SIntType; typedef OutputType UIntType; typedef OutputType VIntType; typedef OutputType CSIntType;

Table 97: Every function definition in Vitis_solver/svd.hpp (15 definitions)

Line	Template signature	Returns	Name	Parameters
69	template <int RowsA, int ColsA, typename InputType, typename OutputType>	template <typename T> bool	within_precision	T a, T b
85	template <typename AType, typename BType, typename CType>	void	vm2x1	AType a1, BType b1, AType a2, BType b2, CType& c
95	template <typename AType, typename BType, typename CType>	void	vm2x1_base	AType a1, BType b1, AType a2, BType b2, CType& c

Line	Template signature	Returns	Name	Parameters
101	template <typename T>	void	vm2x1	T a1, hls::x_complex<T> b1, T a2, hls::x_complex<T> b2, hls::x_complex<T>& c
108	template <typename T>	void	vm2x1	hls::x_complex<T> a1, T b1, hls::x_complex<T> a2, T b2, hls::x_complex<T>& c
115	template <typename T>	void	vm2x1	T a1, std::complex<T> b1, T a2, std::complex<T> b2, std::complex<T>& c
122	template <typename T>	void	vm2x1	std::complex<T> a1, T b1, std::complex<T> a2, T b2, std::complex<T>& c
129	template <typename T>	void	vm2x1	T a1, T b1, T a2, T b2, T& c
136	template <typename AType, typename BType, typename CType>	void	mm2x2	AType a1, AType a2, AType a3, AType a4, BType b1, BType b2, BType b3, BType b4, CType& c1, CType& c2, CType& c3, CType& c4
156	template <typename InType, typename CType>	void	calc_angle	hls::x_complex<InType> A, CType& cosThetaA, CType& sinThetaA, CType& cosThetaAdiv2, CType& sinThetaAdiv2, bool& is_pos_real, bool& is_imag
234	template <typename InType, typename CType>	void	calc_angle	std::complex<InType> A, CType& cosThetaA, CType& sinThetaA, CType& cosThetaAdiv2, CType& sinThetaAdiv2, bool& is_pos_real, bool& is_imag
821	template <int RowsA, int ColsA, class SVDTraits, typename InputType, typename OutputType>	—	svdBasic	const InputType A[RowsA][ColsA], OutputType S[RowsA][ColsA], OutputType U[RowsA][RowsA], OutputType V[ColsA][ColsA]
1037	template <int RowsA, int ColsA, class SVDTraits, typename InputType, typename OutputType>	—	svdPairs	const InputType A[RowsA][ColsA], OutputType S[RowsA][ColsA], OutputType U[RowsA][RowsA], OutputType V[ColsA][ColsA]
1503	template <int RowsA, int ColsA, class SVDTraits, typename InputType, typename OutputType>	—	svdTop	const InputType A[RowsA][ColsA], OutputType S[RowsA][ColsA], OutputType U[RowsA][RowsA], OutputType V[ColsA][ColsA]
1539	template <int RowsA, int ColsA, typename InputType, typename OutputType, typename SVDTraits = svdTraits<RowsA, ColsA, InputType, OutputType> >	void	svd	hls::stream<InputType>& matrixAStrm, hls::stream<OutputType>& matrixSStrm, hls::stream<OutputType>& matrixUSStrm, hls::stream<OutputType>& matrixVStrm

Vitis_solver/utils/x_matrix_utils.hpp

Table 98: Every class and struct in Vitis_solver/utils/x_matrix_utils.hpp (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
40	struct	NoTranspose	—	—	—
49	struct	Transpose	template <int RowsA, int ColsA, typename InputType>	—	—
58	struct	ConjugateTranspose	template <int RowsA, int ColsA, typename InputType>	—	—
156	struct	xil_printer	template <typename T>	—	—
170	struct	xil_printer <hls::x_complex<T>>	template <typename T>	—	typedef xil_printer<T> printer;

Table 99: Every function definition in Vitis_solver/utils/x_matrix_utils.hpp (21 definitions)

Line	Template signature	Returns	Name	Parameters
42	template <int RowsA, int ColsA, typename InputType>	static InputType	GetElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
51	template <int RowsA, int ColsA, typename InputType>	template <int RowsA, int ColsA, typename InputType> static InputType	GetElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
60	template <int RowsA, int ColsA, typename InputType>	template <int RowsA, int ColsA, typename InputType> static InputType	GetElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
68	template <int RowsA, int ColsA, typename InputType>	template <class TransposeForm, int RowsA, int ColsA, typename InputType> InputType	GetMatrixElement	const InputType A[RowsA][ColsA], unsigned Row, unsigned Col
78	—	static half	x_sqrt	half x
81	—	static float	x_sqrt	float x
84	—	static double	x_sqrt	double x
87	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1> ap_fixed<W1, I1, Q1, O1, N1>	—	x_sqrt	ap_fixed<W1, I1, Q1, O1, N1> x
93	—	static float	x_copysign	float a, float b
96	—	static double	x_copysign	double a, double b
101	—	static float	x_sign	float x
108	—	static double	x_sign	double x
115	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1> ap_fixed<W1, I1, Q1, O1, N1>	—	x_sign	ap_fixed<W1, I1, Q1, O1, N1> x
127	—	static half	x_rsqrt	half x

Line	Template signature	Returns	Name	Parameters
130	—	static float	x_rsqr	float x
133	—	static double	x_rsqr	double x
138	—	static int	x_isneg	float x
142	—	static int	x_isneg	double x
157	template <typename T>	static std::string	to_s	T x, unsigned prec = 10
171	template <typename T>	static std::string	to_s	hls::x_complex<T> x, unsigned prec = 10
187	template <typename T>	template <unsigned ROWS, unsigned COLS, typename T, class TransposeForm> void	print_matrix	T a[ROWS][COLS], std::string prefix = "", unsigned prec = 10, unsigned matlab_format = 0

Vitis_solver_L2/hw/math_helper.hpp

Table 100: Every function definition in Vitis_solver_L2/hw/math_helper.hpp (9 definitions)

Line	Template signature	Returns	Name	Parameters
34	template <typename T>	T	abs	T a
39	—	double	abs	double a
47	—	float	abs	float a
56	template <typename T>	T	fabs	T a
61	—	double	fabs	double a
69	—	float	fabs	float a
78	template <typename T>	T	sqrt	T a
83	—	double	sqrt	double a
91	—	float	sqrt	float a

Vitis_solver_L2/potrf.hpp

Table 101: Every function definition in Vitis_solver_L2/potrf.hpp (1 definitions)

Line	Template signature	Returns	Name	Parameters
169	template <typename T, int NMAX, int NCU>	void	potrf	int m, T* A, int lda, int& info

finn_hlslib/activations.hpp

Table 102: Every class and struct in finn_hlslib/activations.hpp (12 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
84	struct	greater	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member type-defs
91	struct	less	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
98	struct	greater_equal	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
105	struct	less_equal	template<typename input_type_1, typename input_type_2>	: public binary_function<input_type_1, input_type_2, ap_uint<1>>	—
112	struct	add	template<typename input_type_1, typename input_type_2, typename result_type>	: public binary_function<input_type_1, input_type_2, result_type>	—
119	struct	mul	template<typename input_type_1, typename input_type_2, typename result_type>	: public binary_function<input_type_1, input_type_2, result_type>	—
126	struct	max	template<typename input_type_1, typename input_type_2, typename result_type>	: public binary_function<input_type_1, input_type_2, result_type>	—
142	class	Activation	template<typename TA, typename TO>	—	—
160	class	PassThroughActivation	template<typename T>	: public Activation<T, T>	—
176	class	ThresholdActivation	template<typename TA, typename Compare = comp::less<TA, TA>>	: public Activation<TA, bool>	—
202	class	ThresholdsActivation	template<unsigned NF, unsigned PE, unsigned NumTH, typename TA, typename TR, int ActVal = 0, typename Compare = comp::less<TA, TA>>	—	—
242	class	ChannelWiseOperation	template<unsigned NF, unsigned PE, typename TI, typename TP, typename TR, typename Fxn = comp::mul<TI, TP, TR>>	—	—

Table 103: Every function definition in `finn_hlslib/activations.hpp` (4 definitions)

Line	Template signature	Returns	Name	Parameters
212	template<unsigned NF, unsigned PE, unsigned NumTH, typename TA, typename TR, int ActVal = 0, typename Compare = comp::less<TA, TA>>	public: TR	activate	unsigned const nf, unsigned const pe, TA const &accu
250	template<unsigned NF, unsigned PE, typename TI, typename TP, typename TR, typename Fxn = comp::mul<TI, TP, TR>>	public: TR	activate	unsigned const nf, unsigned const pe, TI const &in
281	—	typename TI, typename TO, typename TA> void	Thresholding_Batch	hls::stream<TI> &in, hls::stream<TO> &out, TA const &activation, int const reps
340	—	typename TI, typename TO> void	Thresholding_Stream_Batch	hls::stream<TI> &in, hls::stream<TO> &out, hls::stream<ap_uint<PE*NumSteps*TT::width>> &weight, int const reps

finn_hlslib/convlayer.h

Table 104: Every function definition in finn_hlslib/convlayer.h (2 definitions)

Line	Template signature	Returns	Name	Parameters
104	—	int InStreamW, int OutStreamW, typename TW, typename TA, typename R > void	ConvLayer_Batch	hls::stream<ap_uint<InStreamW>> &in, hls::stream<ap_uint<OutStreamW>> &out, TW const &weights, TA const &activation, unsigned const reps, R const &r

Line	Template signature	Returns	Name	Parameters
275	—	int InStreamW, int OutStreamW, typename TW, typename TA, typename R > void	ConvLay- er_Batch_MMV	hls::stream<ap_uint<InStreamW>> &in, hls::stream<ap_uint<OutStreamW>> &out, TW const &weights, TA const &activation, unsigned const reps, R const &r

finn_hlslib/dma.h

Table 105: Every function definition in finn_hlslib/dma.h (6 definitions)

Line	Template signature	Returns	Name	Parameters
109	template<unsigned int DataWidth, unsigned int numBytes>	void	Mem2Stream_Batch_external_wmem	ap_uint<DataWidth> * in, hls::stream<ap_uint<DataWidth> > & out, const unsigned int numReps
135	template<unsigned int DataWidth, unsigned int numBytes>	void	Mem2Stream	ap_uint<DataWidth> * in, hls::stream<ap_uint<DataWidth> > & out
148	template<unsigned int DataWidth, unsigned int numBytes>	void	Stream2Mem	hls::stream<ap_uint<DataWidth> > & in, ap_uint<DataWidth> * out
160	template<unsigned int DataWidth, unsigned int numBytes>	void	Mem2Stream_Batch	ap_uint<DataWidth> * in, hls::stream<ap_uint<DataWidth> > & out, const unsigned int numReps
181	template<unsigned int DataWidth, unsigned int numBytes>	void	Stream2Mem_Batch	hls::stream<ap_uint<DataWidth> > & in, ap_uint<DataWidth> * out, const unsigned int num- Reps

Line	Template signature	Returns	Name	Parameters
214	—	template< unsigned int TILES, unsigned int SIMD, unsigned int PE, unsigned int WP, typename	GenParamStream	TW const &W_in, hls::stream<ap_uint<SIMD * PE * WP>> ¶mStreamOut, int const numReps

finn_hlslib/interpret.hpp

Table 106: Every class and struct in finn_hlslib/interpret .hpp (11 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
58	class	XnorMul	—	—	—
76	class	Binary	—	—	—
111	struct	Identity	template<typename T>	—	—
128	class	Recast	template<typename T>	—	—
134	class	Container	template<typename TV>	—	—
179	struct	Caster	template<typename T>	—	—
185	struct	Caster <ap_fixed<W, I, Q, O, N>>	template<int W, int I, ap_q_mode Q, ap_o_mode O, int N>	—	—
193	class	Slice	template<typename T, unsigned STRIDE=T::width>	—	—
199	class	Container	template<typename TV>	—	—
250	class	Slice_mmv	template<typename T, unsigned MMV, unsigned STRIDE=T::width>	—	—
256	class	Container	template<typename TV>	—	—

Table 107: Every function definition in finn_hlslib/interpret .hpp (4 definitions)

Line	Template signature	Returns	Name	Parameters
180	template<typename T>	template<int M> static T	cast	ap_int<M> const &arg
186	template<int W, int I, ap_q_mode Q, ap_o_mode O, int N>	template<int M> static ap_fixed<W, I, Q, O, N>	cast	ap_int<M> const &arg
202	template<typename TV>	public:	Container	()
259	template<typename TV>	public:	Container	()

finn_hlslib/mac.hpp**Table 108: Every function definition in finn_hlslib/mac.hpp (3 definitions)**

Line	Template signature	Re- turns	Name	Parameters
163	template<unsigned N, typename T, typename TC, typename TD, typename R>	T	mac	T const &a, TC const &c, TD const &d, R const &r, unsigned mmv
190	template<unsigned N, typename T, typename TC, typename TD, typename R>	T	mac	T const &a, TC const &c, TD const &d, R const &r
200	template<unsigned N, typename T, typename TC, typename TD>	in- line T	mac	T const &a, TC const &c, TD const &d

finn_hlslib/maxpool.h**Table 109: Every function definition in finn_hlslib/maxpool.h (10 definitions)**

Line	Template signature	Returns	Name	Parameters
67	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels>	void	StreamingMax-Pool	hls::stream<ap_uint<NumChannels> > & in, hls::stream<ap_uint<NumChannels> > & out
113	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels>	void	StreamingMax-Pool_Batch	hls::stream<ap_uint<NumChannels> > & in, hls::stream<ap_uint<NumChannels> > & out, unsigned int numReps
138	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, typename Act	StreamingMax-Pool_Precision	hls::stream<ap_uint<StreamW> > & in, hls::stream<ap_uint<StreamW> > & out
205	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, typename Act	StreamingMax-Pool_Precision_Batch	hls::stream<ap_uint<InStreamW> > & in, hls::stream<ap_uint<OutStreamW> > & out, unsigned int numReps

Line	Template signature	Returns	Name	Parameters
244	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, unsigned int	StreamingMax-Pool_Precision_1d	hls::stream<ap_uint<PE*ActType::width> & in, hls::stream<ap_uint<PE*ActType::width> > & out
326	—	template<unsigned int ImgDim, unsigned int PoolDim, unsigned int NumChannels, unsigned int	StreamingMax-Pool_Precision_Batch_1d	hls::stream<ap_uint<PE*ActType::width> & in, hls::stream<ap_uint<PE*ActType::width> > & out, unsigned int numReps
360	template< unsigned int ImgDim, unsigned int NumChannels, typename ActType, unsigned int PEOcount, int offset = 0>	void	ReLU_Batch	hls::stream<ap_uint<PECount * ActType::width> > & in, hls::stream<ap_uint<PECount * ActType::width> > & out, const unsigned int numReps
405	—	template< unsigned int ImgDim, unsigned int NumChannels, typename ActType, unsigned int PE	AccPool_Batch	hls::stream<ap_uint<PECount * ActType::width> > & in, hls::stream<ap_uint<PECount * ActType::width> > & out, const unsigned int numReps
459	—	template< unsigned int NumClasses, unsigned int PEOcount, unsigned int NumTop, typename In_	LabelSelect_Batch	hls::stream<ap_uint<PECount * In_T::width> > & in, hls::stream<Out_T> & out, const unsigned int numReps
558	—	typename TI, typename TO, typename TA > void	Pool_batch	hls::stream<TI> &in, hls::stream<TO> &out, TA const &function, int const reps

finn_hlslib/mmv.hpp

Table 110: Every class and struct in finn_hlslib/mmv.hpp (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
50	class	MultiChanData	template <unsigned int NumChannels, unsigned int DataWidth>	—	—

finn_hlslib/mvau.hpp

Table 111: Every function definition in finn_hlslib/mvau.hpp (2 definitions)

Line	Template signature	Returns	Name	Parameters
91	—	type-name TI, type-name TO, type-name TW, type-name TA, type-name R > void	Matrix_Vector_Activate_Batch	hls::stream<TI> &in, hls::stream<TO> &out, TW const &weights, TA const &activation, int const reps, R const &r
213	—	type-name TI, type-name TO, type-name TA, type-name R > void	Matrix_Vector_Activate_Stream_Batch	hls::stream<TI> &in, hls::stream<TO> &out, hls::stream<ap_uint<PE*SIMD*TW::width>> &weight, TA const &activation, int const reps, R const &r

finn_hlslib/slidingwindow.h

Table 112: Every function definition in finn_hlslib/slidingwindow.h (20 definitions)

Line	Template signature	Returns	Name	Parameters
72	template <type-name T>	void	memory_resource	T inputBuf, ap_resource_dflt const&

Line	Tem-plate signa-ture	Returns	Name	Parameters
93	tem-plate <type-name T>	void	memory_resource	T inputBuf, ap_resource_bram const&
114	tem-plate <type-name T>	void	memory_resource	T inputBuf, ap_resource_uram const&
135	tem-plate <type-name T>	void	memory_resource	T inputBuf, ap_resource_lutram const&
159	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
287	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_MMV	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
441	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_or_kernel_stride	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
588	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_or_kernel_stride_MMV	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out, const unsigned int numReps, R const &r

Line	Tem-plate signa-ture	Returns	Name	Parameters
749	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_dws	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
878	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_or_kernel_stride_dws	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1027	—	tem-plate<unsigned int ConvKernelDim, unsigned int IFMChannels, unsigned int Input_precision>	ConvolutionInputGenerator_or_dws_MMV	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1177	—	template< unsigned int IFMChannels, unsigned int Input_precision, unsigned int IFMDim, uns	ConvolutionInputGenerator_or_2D_kernel1	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps
1227	—	template< unsigned int IFMChannels, unsigned int Input_precision, unsigned int IFMDim, uns	ConvolutionInputGenerator_or_1D_kernel1	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps
1278	—	tem-plate<unsigned int ConvKernelDim_x, unsigned int ConvKernelDim_y, unsigned int IFMChann	ConvolutionInputGenerator_or_NonSquare	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r

Line	Tem-plate signa-ture	Returns	Name	Parameters
1412	—	tem-plate<unsigned int ConvKernelDim_x, unsigned int ConvKernelDim_y, unsigned int IFMChann	ConvolutionInputGenerator_NonSquare_dws	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1549	—	tem-plate<unsigned int ConvKernelDim_x, unsigned int ConvKernelDim_y, unsigned int IFMChann	ConvolutionInputGenerator_Non-Square_Dilated	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1796	—	tem-plate<unsigned int ConvKernelDim_x, unsigned int IFMChannels, unsigned int Input_precis	ConvolutionInputGenerator_1D_dws_naive	hls::stream<ap_uint<SIMD*Input_precision> > & in, hls::stream<ap_uint<SIMD*Input_precision> > & out, const unsigned int numReps, R const &r
1872	—	template< unsigned ConvKernelDim_x, unsigned IFMChannels, unsigned Input_precision, unsign	ConvolutionInputGenerator_1D	hls::stream<ap_uint<SIMD*Input_precision>> &in, hls::stream<ap_uint<SIMD*Input_precision>> &out, unsigned const numReps, R const &r
1950	—	template< unsigned ConvKernelDim_x, unsigned IFMChannels, unsigned Input_precision, unsign	ConvolutionInputGenerator_1D_dws	hls::stream<ap_uint<SIMD*Input_precision>> &in, hls::stream<ap_uint<SIMD*Input_precision>> &out, unsigned const numReps, R const &r
2027	—	template< unsigned ConvKernelDim_x, unsigned IFMChannels, unsigned Input_precision, unsign	ConvolutionInputGenerator_1D_dws_stride	hls::stream<ap_uint<SIMD*Input_precision>> &in, hls::stream<ap_uint<SIMD*Input_precision>> &out, unsigned const numReps, R const &r

finn_hlslib/streamtools.h**Table 113: Every class and struct in finn_hlslib/streamtools.h (4 definitions)**

Line	Kind	Name / specialisation	Template parameters	Bases	Member ty- pedefs
895	class	WidthAdjustedInput-Stream	template<unsigned IW, unsigned OW, unsigned N>	—	—
910	class	WidthAdjustedInput-Stream <W, W, N>	template<unsigned W, unsigned N>	—	—
926	class	WidthAdjustedOutput-Stream	template<unsigned IW, unsigned OW, unsigned N>	—	—
943	class	WidthAdjustedOutput-Stream <W, W, N>	template<unsigned W, unsigned N>	—	—

Table 114: Every function definition in finn_hlslib/streamtools.h (22 definitions)

Line	Template signature	Returns	Name	Parameters
68	template<unsigned int DataWidth, unsigned int NumAllowed, unsigned int NumTotal >	void	StreamLimiter	hls::stream<ap_uint<DataWidth> > & in, hls::stream<ap_uint<DataWidth> > & out
102	template<unsigned int DataWidth, unsigned int NumAllowed, unsigned int NumTotal >	void	StreamLimiter_Batch	hls::stream<ap_uint<DataWidth> > & in, hls::stream<ap_uint<DataWidth> > & out, unsigned int numReps
137	template< unsigned int ImgDim, unsigned int KernelDim, unsigned int Stride, unsigned int NumChannels, typename In_t, unsigned int Padding-Style=2>	void	SameResize	hls::stream<ap_uint<NumChannels* In_t::width> > &in, hls::stream<ap_uint<NumChannels* In_t::width> > &out

Line	Template signature	Returns	Name	Parameters
206	template< unsigned int ImgDim, unsigned int KernelDim, unsigned int Stride, unsigned int NumChannels, typename In_t, unsigned int PaddingStyle=2>	void	SameResize_Batch	hls::stream<ap_uint<NumChannels*> In_t::width> > &in, hls::stream<ap_uint<NumChannels*> In_t::width> > &out, const unsigned int numReps
231	template<typename InT, typename OutT>	void	StreamingCast	hls::stream<InT> & in, hls::stream<OutT> & out, unsigned int numReps
259	—	template< unsigned OutputDim_x, unsigned OutputDim_y, unsigned PaddingLeft, unsigned Paddi	FMPadding_nonsquare	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out
317	—	template< unsigned OutputDim_x, unsigned OutputDim_y, unsigned PaddingLeft, unsigned Paddi	FMPadding_nonsquare_Batch	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out, unsigned const numReps
361	—	template< unsigned ImgDim, unsigned OutputDim, unsigned PaddingBefore, unsigned PaddingBeh	FMPadding	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out
401	—	template< unsigned ImgDim, unsigned OutputDim, unsigned PaddingBefore, unsigned PaddingBeh	FMPadding_Batch	hls::stream<ap_uint<SIMD*In_t::width>> &in, hls::stream<ap_uint<SIMD*In_t::width>> &out, unsigned const numReps

Line	Template signature	Returns	Name	Parameters
436	template<unsigned int InWidth, unsigned int OutWidth, unsigned int NumInWords >	void	StreamingDataWidth-Converter_Batch	hls::stream<ap_uint<InWidth> > & in, hls::stream<ap_uint<OutWidth> > & out, const unsigned int numReps
512	template< unsigned int InWidth, unsigned int OutWidth >	void	StreamingDataWidth-ConverterNoMultiple	hls::stream<ap_uint<InWidth> > & in, hls::stream<ap_uint<OutWidth> > & out
584	template<unsigned int DataWidth, unsigned int NumTotal >	void	DuplicateStreams	hls::stream<ap_uint<DataWidth> > & in, hls::stream<ap_uint<DataWidth> > & out1, hls::stream<ap_uint<DataWidth> > & out2
613	template<unsigned int DataWidth, unsigned int NumTotal >	void	DuplicateStreams_Batch	hls::stream<ap_uint<DataWidth> > & in, hls::stream<ap_uint<DataWidth> > & out1, hls::stream<ap_uint<DataWidth> > & out2, const unsigned int numReps
645	template <unsigned int NumChannels, typename In1_t, typename In2_t, typename Out_t, unsigned int NumTotal, int offset = 0>	void	AddStreams	hls::stream<ap_uint<NumChannels In1_t::width>> * &in1, hls::stream<ap_uint<NumChannels In2_t::width>> * &in2, hls::stream<ap_uint<NumChannels Out_t::width>> * &out
689	template <unsigned int NumChannels, typename In1_t, typename In2_t, typename Out_t, unsigned int NumTotal, int offset = 0>	void	AddStreams_Batch	hls::stream<ap_uint<NumChannels In1_t::width>> * &in1, hls::stream<ap_uint<NumChannels In2_t::width>> * &in2, hls::stream<ap_uint<NumChannels Out_t::width>> * &out, const unsigned int numReps

Line	Template signature	Returns	Name	Parameters
721	template <unsigned int NumChannels, typename In1_t, typename In2_t, typename Out_t, unsigned int NumTotal, unsigned int PEOCount, int offset = 0>	void	AddStreamsLayer_Batch	hls::stream<ap_uint<NumChannels> > * In1_t::width>> &in1, hls::stream<ap_uint<NumChannels> > * In2_t::width>> &in2, hls::stream<ap_uint<NumChannels> > * Out_t::width>> &out, const unsigned int numReps
753	—	template<unsigned int InWidth, unsigned int OutWidth, unsigned int NumInWords, unsigned int	MultiChanDataWidthConverter_Batch	hls::stream<MultiChanData<NumVecs, InWidth> > & in, hls::stream<MultiChanData<NumVecs, OutWidth> > & out, const unsigned int numReps
844	template <unsigned int NumChannels, unsigned int DataWidth>	void	FlattenMultiChanData	hls::stream<MultiChanData<NumChannels, DataWidth> > & in, hls::stream<ap_uint<NumChannels*DataWidth> > & out, const unsigned int numReps
875	template <unsigned int NumChannels, unsigned int DataWidth>	void	PackMultiChanData	hls::stream<ap_uint<NumChannels*DataWidth> > & in, hls::stream<MultiChanData<NumChannels, DataWidth> > & out, const unsigned int numReps
898	template<unsigned IW, unsigned OW, unsigned N>	public:	WidthAdjustedInputStream	hls::stream<ap_uint<IW> > & source, unsigned const reps
970	template<unsigned W, unsigned N>	template<unsigned int DataWidth, unsigned int NumTotal> void	Qdma2Stream_Batch	hls::stream<qdma_axis<DataWidth,0,0,0> > & in, hls::stream<ap_uint<DataWidth> > & out, const unsigned int numReps
994	template<unsigned int DataWidth, unsigned int NumTotal>	void	Stream2Qdma_Batch	hls::stream<ap_uint<DataWidth> > & in, hls::stream<qdma_axis<DataWidth,0,0,0> > & out, const unsigned int numReps

finn_hlslib/upsample.hpp

Table 115: Every function definition in finn_hlslib/upsample.hpp (3 definitions)

Line	Template signature	Returns	Name	Parameters
62	—	template<unsigned int OFMDim, unsigned int IFMDim, unsigned int NumChannels, typename In_t	UpsampleNearest-Neighbour	hls::stream<ap_uint<NumChannels In_t::width>> & in, hls::stream<ap_uint<NumChannels In_t::width>> & out
134	—	template<unsigned int OFMDim, unsigned int IFMDim, unsigned int NumChannels, typename In_t	UpsampleNearest-Neighbour_Batch	hls::stream<ap_uint<NumChannels In_t::width>> & in, hls::stream<ap_uint<NumChannels In_t::width>> & out, unsigned int num-Reps
158	template< unsigned OFMDim, unsigned IFMDim, unsigned NumChannels, typename In_t >	void	UpsampleNearest-Neighbour_1D	hls::stream<ap_uint<NumChannels In_t::width>> &src, hls::stream<ap_uint<NumChannels In_t::width>> &dst

finn_hlslib/utils.hpp

Table 116: Every class and struct in `finn_hlslib/utils.hpp` (9 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
63	struct	first_param	template<typename T>	—	—
65	struct	first_param<R (*)(A, Args...)>	template<typename R, typename A, typename... Args>	—	typedef A type;
67	struct	first_param<R (C::*)(A, Args...)>	template<typename C, typename R, typename A, typename... Args>	—	typedef A type;
70	class	ap_resource_dflt	template<typename C, typename R, typename A, typename... Args>	—	—
71	class	ap_resource_lut	template<typename C, typename R, typename A, typename... Args>	—	—
72	class	ap_resource_dsp	template<typename C, typename R, typename A, typename... Args>	—	—
74	class	ap_resource_lutram	template<typename C, typename R, typename A, typename... Args>	—	—
75	class	ap_resource_bram	template<typename C, typename R, typename A, typename... Args>	—	—
76	class	ap_resource_uram	template<typename C, typename R, typename A, typename... Args>	—	—

Table 117: Every function definition in `finn_hlslib/utils.hpp` (2 definitions)

Line	Template signature	Returns	Name	Parameters
58	—	constexpr unsigned	clog2	size_t x

Line	Template signature	Returns	Name	Parameters
88	template<typename C, typename R, typename A, typename... Args>	template < unsigned int BitWidth > void	log-String-Stream	const char *layer_name, hls::stream<ap_uint<BitWidth>> &log

finn_hlslib/vvau.hpp

Table 118: Every function definition in finn_hlslib/vvau.hpp (2 definitions)

Line	Template signature	Returns	Name	Parameters
83	—	typename TI, typename TO, typename TW, typename TA, typename R > void	Vector_Vector_Activate_Batch	hls::stream<TI> &in, hls::stream<TO> &out, TW const &weights, TA const &activation, int const reps, R const &r
192	—	typename TI, typename TO, typename TA, typename R > void	Vec-tor_Vector_Activate_Stream_Batch	hls::stream<TI> &in, hls::stream<TO> &out, hls::stream<ap_uint<PE*SIMD*TW::width>> &weights, TA const &activation, int const reps, R const &r

finn_hlslib/weights.hpp

Table 119: Every class and struct in finn_hlslib/weights.hpp (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
67	class	BinaryWeights	template<unsigned SIMD, unsigned PE, unsigned TILES>	—	—
76	class	TileIndex	template<unsigned SIMD, unsigned PE, unsigned TILES>	—	—
111	class	FixedPoint-Weights	template<unsigned SIMD, typename WT ,unsigned PE, unsigned TILES>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
120	class	TileIndex	template<unsigned SIMD, typename WT ,unsigned PE, unsigned TILES>	—	—
152	class	Weights_Tile	template<unsigned SIMD, typename WT, unsigned PE>	—	—

Table 120: Every function definition in `finn_hlslib/weights.hpp` (2 definitions)

Line	Template signature	Returns	Name	Parameters
93	template<unsigned SIMD, unsigned PE, unsigned TILES>	public: TileIndex	weights	unsigned const tile
143	template<unsigned SIMD, typename WT ,unsigned PE, unsigned TILES>	public: TileIndex	weights	unsigned const tile

C.2 Vendor headers

ap_axi_sdata.h

Table 121: Every class and struct in `ap_axi_sdata.h` (10 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
55	struct	ap_axis	template<int D,int U,int TI,int TD>	—	—
66	struct	ap_axis<D, 0, 0, 0>	template<int D>	—	—
74	struct	ap_axiu	template<int D,int U,int TI,int TD>	—	—
85	struct	ap_axiu<D, 0, 0, 0>	template<int D>	—	—
96	struct	qdma_axis<D, 0, 0, 0>	template<int D>	—	—
118	class	stream<ap_axis<D, U, TI, TD>>	template <int D,int U,int TI,int TD>	—	typedef ap_axis<D, U, TI, TD> __STREAM_T__;
165	class	stream<ap_axis<D, 0, 0, 0>>	template <int D>	—	typedef ap_axis<D, 0, 0, 0> __STREAM_T__ ;
209	class	stream<ap_axiu<D, U, TI, TD>>	template <int D,int U,int TI,int TD>	—	typedef ap_axiu<D, U, TI, TD> __STREAM_T__;
255	class	stream<ap_axiu<D, 0, 0, 0>>	template <int D>	—	typedef ap_axiu<D, 0, 0, 0> __STREAM_T__;
298	class	stream<qdma_axis<D, 0, 0, 0>>	template <int D>	—	typedef qdma_axis<D, 0, 0, 0> __STREAM_T__;

ap_common.h

Table 122: Every class and struct in `ap_common.h` (8 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
147	struct	is_signed	template <typename _Tp>	—	—
152	struct	is_integral	template <typename _Tp>	—	—
157	struct	is_integral <CTYPE>	—	—	—
175	struct	enable_if	template <bool, typename _Tp = void>	—	—
178	struct	enable_if <true, _Tp>	template <typename _Tp>	—	typedef _Tp type;
183	struct	remove_const	template <typename _Tp>	—	typedef _Tp type;
188	struct	remove_const <_Tp const>	template <typename _Tp>	—	typedef _Tp type;
245	struct	ssdm_int_sim	template <int _AP_W, bool _AP_S>	—	—

Table 123: Every function definition in ap_common.h (10 definitions)

Line	Template signature	Returns	Name	Parameters
204	template <typename _Tp>	static inline unsigned char	guess_radix	const char* s
255	template <int _AP_W, bool _AP_S>	template <typename _Tp1, typename _Tp2, typename _Tp3> inline _Tp1	_AP_ROOT_op_concat	const _Tp1& Ret, const _Tp2& X, const _Tp3& Y
261	template <typename _Tp1, typename _Tp2, typename _Tp3>	template <typename _Tp1, typename _Tp2, typename _Tp3> inline _Tp1&	_AP_ROOT_op_set_bit	_Tp1& Val, const _Tp2& Bit, const _Tp3& Repl
269	template <typename _Tp1, typename _Tp2, typename _Tp3>	template <typename _Tp1, typename _Tp2, typename _Tp3, typename _Tp4> inline _Tp1&	_AP_ROOT_op_set_range	_Tp1& Val, const _Tp2& Lo, const _Tp3& Hi, const _Tp4& Repl
302	template <typename _Tp1, typename _Tp2, typename _Tp3, typename _Tp4>	INLINE ap_ulong	doubleToRawBits	double pf
311	template <typename _Tp1, typename _Tp2, typename _Tp3, typename _Tp4>	INLINE unsigned int	floatToRawBits	float pf
321	template <typename _Tp1, typename _Tp2, typename _Tp3, typename _Tp4>	INLINE unsigned short	halfToRawBits	half pf
336	—	INLINE double	rawBitsToDouble	ap_ulong pi
346	—	INLINE float	rawBitsToFloat	unsigned long pi
357	—	INLINE half	rawBitsToHalf	unsigned short pi

ap_fixed.h

Table 124: Every class and struct in ap_fixed.h (2 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
29	struct	ap_fixed	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	: ap_fixed_base<_AP_W, _AP_I, true, _AP_Q, _AP_O, _AP_N>	typedef ap_fixed_base<_AP_W, _AP_I, true, _AP_Q, _AP_O, _AP_N> Base;
191	struct	ap_ufixed	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	: ap_fixed_base<_AP_W, _AP_I, false, _AP_Q, _AP_O, _AP_N>	typedef ap_fixed_base<_AP_W, _AP_I, false, _AP_Q, _AP_O, _AP_N> Base;

Table 125: Every function definition in ap_fixed.h (4 definitions)

Line	Template signature	Re-returns	Name	Parameters
36	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	IN-LINE	ap_fixed	const ap_fixed& op
198	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	IN-LINE	ap_ufixed	const ap_ufixed& op
339	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	IN-LINE void	sc_trace	sc_core::sc_trace_file* tf, const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>& op, const std::string& name
346	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	IN-LINE void	sc_trace	sc_core::sc_trace_file* tf, const ap_ufixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>& op, const std::string& name

ap_fixed_base.h

Table 126: Every class and struct in ap_fixed_base.h (4 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
101	struct	_ap_fixed_factory<_AP_W2, _AP_I2, true>	template <int _AP_W2, int _AP_I2>	—	typedef ap_fixed<_AP_W2, _AP_I2> type;
105	struct	_ap_fixed_factory<_AP_W2, _AP_I2, false>	template <int _AP_W2, int _AP_I2>	—	typedef ap_ufixed<_AP_W2, _AP_I2> type;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
121	struct	ap_fixed_base	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	: _AP_ROOT_TYPE<_AP_W, _AP_S>	typedef _AP_ROOT_TYPE<_AP_W, _AP_S> Base; typedef ap_fixed_base<_AP_W, _AP_I, _AP_S> lhs; typedef ap_fixed_base<_AP_W2, _AP_I2, _AP_S2> rhs; typedef ap_fixed_base<mult_w, mult_i, mult_s> mult_base; typedef ap_fixed_base<plus_w, plus_i, plus_s> plus_base; typedef ap_fixed_base<minus_w, minus_i, minus_s> minus_base; typedef ap_fixed_base<logic_w, lo- gic_i, logic_s> logic_base; typedef ap_fixed_base<div_w, div_i, div_s> div_base; typedef ap_fixed_base<_AP_W, _AP_I, _AP_S> arg1_base; typedef typename _ap_fixed_factory<mult_w, mult_i, mult_s>::type mult; ... and 5 more (Appendix D)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
131	struct	RType	template <int _AP_W2, int _AP_I2, bool _AP_S2>	—	typedef ap_fixed_base<_AP_W, _AP_I, _AP_S> lhs; typedef ap_fixed_base<_AP_W2, _AP_I2, _AP_S2> rhs; typedef ap_fixed_base<mult_w, mult_i, mult_s> mult_base; typedef ap_fixed_base<plus_w, plus_i, plus_s> plus_base; typedef ap_fixed_base<minus_w, minus_i, minus_s> minus_base; typedef ap_fixed_base<logic_w, lo- gic_i, logic_s> logic_base; typedef ap_fixed_base<div_w, div_i, div_s> div_base; typedef ap_fixed_base<_AP_W, _AP_I, _AP_S> arg1_base; typedef typename _ap_fixed_factory<mult_w, mult_i, mult_s>::type mult; typedef typename _ap_fixed_factory<plus_w, plus_i, plus_s>::type plus; ... and 4 more (Appendix D)

Table 127: Every function definition in ap_fixed_base.h (107 definitions)

Line	Template signature	Returns	Name	Parameters
64	template <typename _Tp1, typename _Tp2>	inline bool	_AP_c- type_op_get_bit	_Tp1& var, const _Tp2& index
68	template <typename _Tp1, typename _Tp2>	template <typename _Tp1, typename _Tp2, typename _Tp3> inline _Tp1	_AP_c- type_op_set_bit	_Tp1& var, const _Tp2& index, const _Tp3& x
73	template <typename _Tp1, typename _Tp2, typename _Tp3>	template <typename _Tp1, typename _Tp2, typename _Tp3> inline _Tp1	_AP_c- type_op_get_range	_Tp1& var, const _Tp2& low, const _Tp3& high
83	template <typename _Tp1, typename _Tp2, typename _Tp3>	template <typename _Tp1, typename _Tp2, typename _Tp3, type- name _Tp4> inline _Tp1	_AP_c- type_op_set_range	_Tp1& var, const _Tp2& low, const _Tp3& high, const _Tp4& x

Line	Template signature		Returns	Name	Parameters
184	—		void	fromString	const std::string& val, unsigned char radix
283	—		INLINE void	report	()
298	—		INLINE void	report	()
303	—		INLINE void	overflow_adjust	bool underflow, bool overflow, bool ld, bool sign
367	—		INLINE bool	quantization_adjust	bool qb, bool r, bool s
385	—		public: INLINE	ap_fixed_base	()
392	—		template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_fixed_base	const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op
400	—		template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_fixed_base	const volatile ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op
408	template _AP_W2, _AP_S2>	<int bool	INLINE	ap_fixed_base	const ap_int_base<_AP_W2, _AP_S2>& op
416	template _AP_W2, _AP_S2>	<int bool	INLINE	ap_fixed_base	const volatile ap_int_base<_AP_W2, _AP_S2>& op
426	—		INLINE	ap_fixed_base	const char* s, signed char rd = 0
434	—		INLINE	ap_fixed_base	const char* s, signed char rd = 10
442	—		INLINE	ap_fixed_base	const char* s
448	—		INLINE	ap_fixed_base	const char* s, signed char rd
456	template _AP_W2, _AP_S2>	<int bool	INLINE	ap_fixed_base	const ap_bit_ref<_AP_W2, _AP_S2>& op
462	template _AP_W2, _AP_S2>	<int bool	INLINE	ap_fixed_base	const ap_range_ref<_AP_W2, _AP_S2>& op
468	template _AP_W2, _AP_T2, _AP_W3, _AP_T3>	<int typename int typename	INLINE	ap_fixed_base	const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>& op

Line	Template signature	Returns	Name	Parameters
475	—	template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_fixed_base	const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op
483	—	template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_fixed_base	const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op
494	—	INLINE	ap_fixed_base	const C_TYPE x
519	—	AP_WEAK	ap_fixed_base	double d
637	—	INLINE	ap_fixed_base	float d
641	—	INLINE	ap_fixed_base	half d
774	—	INLINE ap_fixed_base&	setBits	ap_ulong bv
781	—	static INLINE ap_fixed_base	bitsToFixed	ap_ulong bv
834	—	INLINE char	to_char	0
836	—	INLINE int	to_int	0
838	—	INLINE unsigned	to_uint	0
840	—	INLINE ap_slong	to_int64	0
842	—	INLINE ap_ulong	to_uint64	0
846	—	INLINE double	to_double	0
896	—	INLINE float	to_float	0
944	—	INLINE half	to_half	0
991	—	INLINE operator long	double	0
993	—	INLINE operator	double	0
995	—	INLINE operator	float	0
998	—	INLINE operator	half	0
1001	—	INLINE operator	bool	0
1003	—	INLINE operator	char	0
1005	—	INLINE operator signed	char	0
1007	—	INLINE operator un- signed	char	0
1009	—	INLINE operator	short	0
1011	—	INLINE operator un- signed	short	0
1013	—	INLINE operator	int	0
1015	—	INLINE operator un- signed	int	0
1019	—	INLINE operator	long	0

Line	Template signature	Returns	Name	Parameters
1021	—	INLINE operator unsigned	long	()
1023	—	INLINE operator	long	()
1025	—	INLINE operator unsigned	long	()
1028	—	INLINE operator	ap_ulong	()
1030	—	INLINE operator	ap_slong	()
1032	—	INLINE int	length	()
1037	—	INLINE ap_ulong	bits_to_uint64	()
1045	—	INLINE int	countLeadingZeros	()
1154	—	INLINE typename RType<_AP_W2, _AP_I2, _AP_S2>::Rty operator	Sym	\ const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& \ op2
1228	—	INLINE ap_fixed_base<_AP_W, _AP_I, true, _AP_Q, _AP_O, _AP_N>	getNeg	()
1451	—	INLINE bool operator	Sym	const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, \ _AP_O2, _AP_N2>& op2
1474	—	INLINE bool operator	Sym	double d
1504	—	INLINE af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	bit	unsigned index
1510	template <int _AP_W2, bool _AP_S2>	INLINE af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _A	bit	const ap_int_base<_AP_W2, _AP_S2>& index
1519	—	INLINE bool	bit	unsigned index
1524	template <int _AP_W2> INLINE af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	—	get_bit	const ap_int_base<_AP_W2, true>& index
1534	—	INLINE bool	get_bit	int index
1541	—	INLINE af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	get_bit	int index
1551	template <int _AP_W2>	INLINE bool	get_bit	const ap_int_base<_AP_W2, true>& index

Line	Template signature		Returns	Name	Parameters
1559	—		INLINE af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	range	int Hi, int Lo
1566	—		INLINE af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	range	int Hi, int Lo
1573	template _AP_W2, _AP_S2, _AP_W3, _AP_S3>	<int bool int bool	INLINE af_range_ref<_AP_W, _AP	range	const ap_int_base<_AP_W2, _AP_S2>& HiIdx, const ap_int_base<_AP_W3, _AP_S3>& LoIdx
1582	template _AP_W2, _AP_S2, _AP_W3, _AP_S3>	<int bool int bool	INLINE af_range_ref<_AP_W, _AP	range	const ap_int_base<_AP_W2, _AP_S2>& HiIdx, const ap_int_base<_AP_W3, _AP_S3>& LoIdx
1591	—		INLINE af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	range	()
1595	—		INLINE af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	range	()
1627	—		INLINE bool	is_zero	()
1629	—		INLINE bool	is_neg	()
1634	—		INLINE int	wl	()
1636	—		INLINE int	iwl	()
1638	—		INLINE ap_q_mode	q_mode	()
1640	—		INLINE ap_o_mode	o_mode	()
1642	—		INLINE int	n_bits	()
1651	—		std::string	to_string	unsigned char radix = 2, bool sign = _AP_S
1734	—		INLINE char*	to_string	unsigned char radix = 2, bool sign = _AP_S
1740	template _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	<int _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>	I	b_not	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op

Line	Template signature	Returns	Name	Parameters
1748	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	I	b_and	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op1, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op2
1757	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	I	b_or	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op1, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op2
1766	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	I	b_xor	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op1, const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op2
1775	—	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N, i	neg	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op
1788	—	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N, i	lshift	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op, int i
1808	—	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N, i	rshift	ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& ret, const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op, int i
1840	—	INLINE std::string	scientificFormat	std::string& input
1894	—	INLINE std::string	reduceToPrecision	std::string& input, int precision

Line	Template signature	Returns	Name	Parameters
2022	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	I	print	const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& x
2094	—	operator	BIN_OP	\ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ C_TYPE i_op
2105	—	operator	BIN_OP	\ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ C_TYPE i_op
2114	—	operator	BIN_OP	\ C_TYPE i_op, \ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op
2123	—	INLINE bool operator	REL_OP	\ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ C_TYPE i_op
2130	—	INLINE bool operator	REL_OP	\ C_TYPE i_op, \ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op
2140	—	operator	ASSIGN_OP	\ ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ C_TYPE i_op
2150	—	operator	ASSIGN_OP	\ ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ C_TYPE i_op
2221	—	operator	BIN_OP	\ const ap_int_base<_AP_W2, _AP_S2>& i_op, \ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op
2231	—	operator	BIN_OP	\ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ const ap_int_base<_AP_W2, _AP_S2>& i_op

Line	Template signature	Returns	Name	Parameters
2240	—	INLINE bool operator	REL_OP	\ ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op, \ const ap_int_base<_AP_W2, _AP_S2>& i_op
2248	—	INLINE bool operator	REL_OP	\ ap_int_base<_AP_W2, _AP_S2>& i_op, \ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op
2258	—	operator	ASSIGN_OP	\ ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op, \ const ap_int_base<_AP_W2, _AP_S2>& i_op
2266	—	INLINE ap_int_base<_AP_W2, _AP_S2>& operator	ASSIGN_OP	\ ap_int_base<_AP_W2, _AP_S2>& i_op, \ const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op

ap_fixed_ref.h

Table 128: Every class and struct in ap_fixed_ref.h (2 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
35	struct	af_bit_ref	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_0, int _AP_N>	—	typedef ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N> ref_type;
222	struct	af_range_ref	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_0, int _AP_N>	—	typedef ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N> ref_type;

Table 129: Every function definition in ap_fixed_ref.h (27 definitions)

Line	Template signature	Returns	Name	Parameters
61	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_0, int _AP_N>	INLINE operator	bool	()
193	—	INLINE bool	get	()
195	—	INLINE int	length	()
198	—	std::string	to_string	()
201	—	INLINE char*	to_string	()
528	—	INLINE operator	ap_ulong	()

Line	Template signature	Returns	Name	Parameters
540	—	INLINE ap_int_base<_AP_W, false>	to_ap_int_base	()
547	—	INLINE char	to_char	()
551	—	INLINE int	to_int	()
555	—	INLINE unsigned	to_uint	()
559	—	INLINE long	to_long	()
563	—	INLINE unsigned long	to_ulong	()
567	—	INLINE ap_slong	to_int64	()
571	—	INLINE ap_ulong	to_uint64	()
575	—	INLINE ap_int_base<_AP_W, false>	get	()
581	template <int _AP_W2>	INLINE void	set	const ap_int_base<_AP_W2, false>& val
586	—	INLINE int	length	()
591	—	std::string	to_string	signed char rd = 2
598	—	INLINE char*	to_string	signed char rd = 2
621	—	INLINE bool operat- or	REL_OP	\ const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op, \ C_TYPE op2
630	—	INLINE bool operat- or	REL_OP	\ C_TYPE op2, \ const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op
639	—	INLINE bool operat- or	REL_OP	\ const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op, \ C_TYPE op2
647	—	INLINE bool operat- or	REL_OP	\ C_TYPE op2, \ const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op
680	—	INLINE bool operat- or	REL_OP	\ const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op, \ const ap_int_base<_AP_W2, _AP_S>& op2
687	—	INLINE bool operat- or	REL_OP	\ const ap_int_base<_AP_W2, _AP_S2>& op2, \ const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0, _AP_N>& op

Line	Template signature	Returns	Name	Parameters
694	—	INLINE bool operator	REL_OP	\ const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op, \ const ap_int_base<_AP_W2, _AP_S2>& op2
701	—	INLINE bool operator	REL_OP	\ const ap_int_base<_AP_W2, _AP_S2>& op2, \ const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& op

ap_fixed_special.h

Table 130: Every class and struct in ap_fixed_special.h (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
60	class	complex <ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>>	template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	—	typedef ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> _Tp; typedef _Tp value_type;

Table 131: Every function definition in ap_fixed_special.h (8 definitions)

Line	Template signature	Returns	Name	Parameters
79	—	const _Tp&	real	()
80	—	const _Tp&	imag	()
82	—	_Tp&	real	()
83	—	const _Tp&	real	()
84	—	_Tp&	imag	()
85	—	const _Tp&	imag	()
88	—	void	real	_Tp __val
90	—	void	imag	_Tp __val

ap_int.h

Table 132: Every class and struct in ap_int.h (2 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
28	struct	ap_int	template <int _AP_W>	: ap_int_base<_AP_W, true>	typedef ap_int_base<_AP_W, true> Base;
166	struct	ap_uint	template <int _AP_W>	: ap_int_base<_AP_W, false>	typedef ap_int_base<_AP_W, false> Base;

Table 133: Every function definition in ap_int.h (16 definitions)

Line	Template signature	Returns	Name	Parameters
34	template <int _AP_W>	INLINE	ap_int	const ap_int& op
36	template <int _AP_W>	template <int _AP_W2> INLINE	ap_int	const ap_int<_AP_W2>& op

Line	Template signature	Returns	Name	Parameters
41	template <int _AP_W2>	INLINE	ap_int	const volatile ap_int<_AP_W2>& op
46	template <int _AP_W2>	INLINE	ap_int	const ap_uint<_AP_W2>& op
51	template <int _AP_W2>	INLINE	ap_int	const volatile ap_uint<_AP_W2>& op
90	template <int _AP_W2, _AP_S2>	INLINE bool	ap_int	const ap_int_base<_AP_W2, _AP_S2>& op
114	—	INLINE	ap_int	TYPE val
172	template <int _AP_W>	INLINE	ap_uint	const ap_uint& op
174	template <int _AP_W>	template <int _AP_W2> INLINE	ap_uint	const ap_uint<_AP_W2>& op
179	template <int _AP_W2>	INLINE	ap_uint	const ap_int<_AP_W2>& op
184	template <int _AP_W2>	INLINE	ap_uint	const volatile ap_uint<_AP_W2>& op
189	template <int _AP_W2>	INLINE	ap_uint	const volatile ap_int<_AP_W2>& op
228	template <int _AP_W2, _AP_S2>	INLINE bool	ap_uint	const ap_int_base<_AP_W2, _AP_S2>& op
252	—	INLINE	ap_uint	TYPE val
306	template <int _AP_W>	INLINE void	sc_trace	sc_core::sc_trace_file* tf, const ap_int<_AP_W>& op, const std::string& name
312	template <int _AP_W>	INLINE void	sc_trace	sc_core::sc_trace_file* tf, const ap_uint<_AP_W>& op, const std::string& name

ap_int_base.h

Table 134: Every class and struct in ap_int_base.h (14 definitions)

Line	Kind	Name / special- isation	Template para- meters	Bases	Member typedefs
50	struct	retval <_AP_N, true>	template <int _AP_N>	—	typedef ap_slong Type;
55	struct	retval <_AP_N, false>	template <int _AP_N>	—	typedef ap_ulong Type;
61	struct	retval <1, true>	template <>	—	typedef signed char Type;
66	struct	retval <1, false>	template <>	—	typedef unsigned char Type;
72	struct	retval <2, true>	template <>	—	typedef short Type;
77	struct	retval <2, false>	template <>	—	typedef unsigned short Type;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
83	struct	retval <3, true>	template <>	—	typedef long Type;
88	struct	retval <3, false>	template <>	—	typedef unsigned long Type;
93	struct	retval <4, true>	template <>	—	typedef long Type;
98	struct	retval <4, false>	template <>	—	typedef unsigned long Type;
108	struct	_ap_int_factory <_AP_W2,true>	template <int _AP_W2>	—	typedef ap_int<_AP_W2> type;
110	struct	_ap_int_factory <_AP_W2,false>	template <int _AP_W2>	—	typedef ap_uint<_AP_W2> type;
113	struct	ap_int_base	template <int _AP_W, bool _AP_S>	: _AP_ROOT_TYPE<_AP_W, _AP_S> public	typedef _AP_ROOT_TYPE<_AP_W, _AP_S> Base; typedef typename retval<AP_MAX((_AP_W + 7) / 8, 8), _AP_S>::Type RetType; typedef ap_int_base<mult_w, mult_s> mult_base; typedef ap_int_base<plus_w, plus_s> plus_base; typedef ap_int_base<minus_w, minus_s> minus_base; typedef ap_int_base<logic_w, logic_s> logic_base; typedef ap_int_base<div_w, div_s> div_base; typedef ap_int_base<mod_w, mod_s> mod_base; typedef ap_int_base<_AP_W, _AP_S> arg1_base; typedef typename _ap_int_factory<mult_w, mult_s>::type mult; ... and 7 more (Appendix D)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
128	struct	RType	template <int _AP_W2, bool _AP_S2>	—	typedef ap_int_base<mult_w, mult_s> mult_base; typedef ap_int_base<plus_w, plus_s> plus_base; typedef ap_int_base<minus_w, minus_s> minus_base; typedef ap_int_base<logic_w, logic_s> logic_base; typedef ap_int_base<div_w, div_s> div_base; typedef ap_int_base<mod_w, mod_s> mod_base; typedef ap_int_base<_AP_W, _AP_S> arg1_base; typedef typename _ap_int_factory<mult_w, mult_s>::type mult; typedef typename _ap_int_factory<plus_w, plus_s>::type plus; typedef typename _ap_int_factory<minus_w, minus_s>::type minus; ... and 5 more (Appendix D)

Table 135: Every function definition in ap_int_base.h (114 definitions)

Line	Template signature	Returns	Name	Parameters
169	—	INLINE	ap_int_base	()
178	template <int _AP_W2, bool _AP_S2>	INLINE	ap_int_base	const ap_int_base<_AP_W2, _AP_S2>& op
184	template <int _AP_W2, bool _AP_S2>	INLINE	ap_int_base	const volatile ap_int_base<_AP_W2, _AP_S2>& op
200	—	INLINE	ap_int_base	const Type op
219	—	INLINE	ap_int_base	half op
226	—	INLINE	ap_int_base	float op
274	—	INLINE	ap_int_base	double op
322	—	template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_int_base	const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& op
329	template <int _AP_W2, bool _AP_S2>	INLINE	ap_int_base	const ap_range_ref<_AP_W2, _AP_S2>& ref
334	template <int _AP_W2, bool _AP_S2>	INLINE	ap_int_base	const ap_bit_ref<_AP_W2, _AP_S2>& ref

Line	Template signature	Returns	Name	Parameters
339	template <int _AP_W2, typename _AP_T2, int _AP_W3, typename _AP_T3>	INLINE	ap_int_base	const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>& ref
350	—	INLINE	ap_int_base	const char* s, signed char rd = 0
358	—	INLINE	ap_int_base	const char* s
364	—	INLINE	ap_int_base	const char* s, signed char rd
372	—	template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_int_base	const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& val
379	—	template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode _AP_Q2, ap_o_mode _AP_O2, int _AP	ap_int_base	const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2, _AP_N2>& val
511	—	INLINE operator	RetType	()
516	—	INLINE bool	to_bool	()
517	—	INLINE char	to_char	()
518	—	INLINE signed char	to_schar	()
519	—	INLINE unsigned char	to_uchar	()
520	—	INLINE short	to_short	()
521	—	INLINE unsigned short	to_ushort	()
522	—	INLINE int	to_int	()
523	—	INLINE unsigned	to_uint	()
524	—	INLINE long	to_long	()
525	—	INLINE unsigned long	to_ulong	()
526	—	INLINE ap_slong	to_int64	()
527	—	INLINE ap_ulong	to_uint64	()
528	—	INLINE float	to_float	()
529	—	INLINE double	to_double	()
533	—	INLINE operator	char	()
534	—	INLINE operator signed	char	()
535	—	INLINE operator unsigned	char	()
536	—	INLINE operator	short	()
537	—	INLINE operator unsigned	short	()
538	—	INLINE operator	int	()
539	—	INLINE operator unsigned	int	()
540	—	INLINE operator	long	()

Line	Template signature	Returns	Name	Parameters
541	—	INLINE operator unsigned	long	()
542	—	INLINE operator	ap_slong	()
543	—	INLINE operator	ap_ulong	()
556	—	INLINE bool	iszero	()
559	—	INLINE bool	is_zero	()
562	—	INLINE bool	sign	()
571	—	INLINE void	clear	int i
577	—	INLINE void	invert	int i
586	—	INLINE bool	test	int i
592	—	INLINE ap_int_base&	get	()
595	—	INLINE void	set	int i
601	—	INLINE void	set	int i, bool v
608	—	INLINE ap_int_base&	lrotate	int n
623	—	INLINE ap_int_base&	rrotate	int n
638	—	INLINE ap_int_base&	reverse	()
644	—	INLINE void	set_bit	int i, bool v
649	—	INLINE bool	get_bit	int i
654	—	INLINE void	b_not	()
658	—	INLINE ap_int_base& op- erator	Sym	const ap_int_base<_AP_W2, _AP_S2>& op2
678	—	INLINE ap_int_base& op- erator	Sym	const ap_int_base<_AP_W2, _AP_S2>& op2
872	—	INLINE ap_range_ref<_AP_W, _AP_S>	range	int Hi, int Lo
879	—	INLINE ap_range_ref<_AP_W, _AP_S>	range	int Hi, int Lo
885	template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>	INLINE ap_range_ref<_AP_W, _AP	range	const ap_int_base<_AP_W2, _AP_S2>& HiIdx, const ap_int_base<_AP_W3, _AP_S3>& LoIdx
894	template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>	INLINE ap_range_ref<_AP_W, _AP	range	const ap_int_base<_AP_W2, _AP_S2>& HiIdx, const ap_int_base<_AP_W3, _AP_S3>& LoIdx
903	—	INLINE ap_range_ref<_AP_W, _AP_S>	range	()
907	—	INLINE ap_range_ref<_AP_W, _AP_S>	range	()

Line	Template signature	Returns	Name	Parameters
982	—	INLINE ap_bit_ref<_AP_W, _AP_S>	bit	int index
988	template <int _AP_W2, bool _AP_S2> INLINE ap_bit_ref<_AP_W, _AP_S>	—	bit	const ap_int_base<_AP_W2, _AP_S2>& index
997	—	INLINE bool	bit	int index
1004	template <int _AP_W2, bool _AP_S2>	INLINE bool	bit	const ap_int_base<_AP_W2, _AP_S2>& index
1020	—	INLINE int	countLeadingZeros	()
1059	template <int _AP_W2, bool _AP_S2>	INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2, ap_int	concat	const ap_int_base<_AP_W2, _AP_S2>& a2
1068	template <int _AP_W2, bool _AP_S2>	INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2, ap_int	concat	ap_int_base<_AP_W2, _AP_S2>& a2
1231	template <int _AP_W3>	INLINE void	set	const ap_int_base<_AP_W3, false>& val
1240	—	INLINE bool	and_reduce	()
1241	—	INLINE bool	nand_reduce	()
1242	—	INLINE bool	or_reduce	()
1243	—	INLINE bool	nor_reduce	()
1244	—	INLINE bool	xor_reduce	()
1245	—	INLINE bool	xnor_reduce	()
1253	—	std::string	to_string	signed char rd = 2, bool sign = _AP_S
1260	—	INLINE char*	to_string	signed char rd = 2, bool sign = _AP_S
1307	—	operator	Sym	const ap_int_base<_AP_W, _AP_S>& op, \ const ap_int_base<_AP_W2, _AP_S2>& op2
1330	—	operator	Sym	const ap_int_base<_AP_W, _AP_S>& op, \ const ap_int_base<_AP_W2, _AP_S2>& op2
1354	—	INLINE PTR_TYPE* operator	BIN_OP	PTR_TYPE* i_op, \ const ap_int_base<_AP_W, _AP_S>& op
1360	—	INLINE PTR_TYPE* operator	BIN_OP	const ap_int_base<_AP_W, _AP_S>& op, \ PTR_TYPE* i_op

Line	Template signature	Returns	Name	Parameters
1376	—	INLINE C_TYPE operator	BIN_OP	C_TYPE i_op, \ const ap_int_base<_AP_W, _AP_S>& op
1382	—	INLINE C_TYPE operator	BIN_OP	const ap_int_base<_AP_W, _AP_S>& op, \ C_TYPE i_op
1410	—	operator	BIN_OP	C_TYPE i_op, const ap_int_base<_AP_W, _AP_S>& op
1416	—	operator	BIN_OP	const ap_int_base<_AP_W, _AP_S>& op, C_TYPE i_op
1505	—	INLINE ap_int_base<_AP_W, _AP_S>& operator	ASSIGN_OP	\ ap_int_base<_AP_W, _AP_S>& op, C_TYPE op2
1543	—	INLINE bool operator	REL_OP	C_TYPE i_op, \ const ap_int_base<_AP_W, _AP_S>& op
1548	—	INLINE bool operator	REL_OP	const ap_int_base<_AP_W, _AP_S>& op, \ C_TYPE op2
1579	—	INLINE bool operator	Sym	const ap_int_base<_AP_W, _AP_S>& op1, \ double op2
1584	—	INLINE bool operator	Sym	double op1, \ const ap_int_base<_AP_W, _AP_S>& op2
1589	—	INLINE bool operator	Sym	const ap_int_base<_AP_W, _AP_S>& op1, \ float op2
1594	—	INLINE bool operator	Sym	float op1, \ const ap_int_base<_AP_W, _AP_S>& op2
1616	—	operator	BIN_OP	const ap_range_ref<_AP_W1, _AP_S1>& op1, \ const ap_int_base<_AP_W2, _AP_S2>& op2
1623	—	operator	BIN_OP	const ap_int_base<_AP_W1, _AP_S1>& op1, \ const ap_range_ref<_AP_W2, _AP_S2>& op2
1644	—	INLINE ap_int_base<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ ap_int_base<_AP_W1, _AP_S1>& op1, ap_range_ref<_AP_W2, _AP_S2>& op2
1649	—	INLINE ap_range_ref<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ ap_range_ref<_AP_W1, _AP_S1>& op1, ap_int_base<_AP_W2, _AP_S2>& op2
1673	—	INLINE bool operator	REL_OP	const ap_range_ref<_AP_W1, _AP_S1>& op1, \ const ap_int_base<_AP_W2, _AP_S2>& op2

Line	Template signature	Returns	Name	Parameters
1678	—	INLINE bool operator	REL_OP	const ap_int_base<_AP_W1, _AP_S1>& op1, \ const ap_range_ref<_AP_W2, _AP_S2>& op2
1699	—	operator	BIN_OP	const ap_int_base<_AP_W1, _AP_S1>& op1, \ const ap_bit_ref<_AP_W2, _AP_S2>& op2
1705	—	operator	BIN_OP	const ap_bit_ref<_AP_W1, _AP_S1>& op1, \ const ap_int_base<_AP_W2, _AP_S2>& op2
1726	—	INLINE ap_int_base<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ ap_int_base<_AP_W1, _AP_S1>& op1, ap_bit_ref<_AP_W2, _AP_S2>& op2
1731	—	INLINE ap_bit_ref<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ ap_bit_ref<_AP_W1, _AP_S1>& op1, ap_int_base<_AP_W2, _AP_S2>& op2
1755	—	INLINE bool operator	REL_OP	const ap_int_base<_AP_W1, _AP_S1>& op1, \ const ap_bit_ref<_AP_W2, _AP_S2>& op2
1760	—	INLINE bool operator	REL_OP	const ap_bit_ref<_AP_W1, _AP_S1>& op1, \ const ap_int_base<_AP_W2, _AP_S2>& op2
1787	—	operator	BIN_OP	const ap_int_base<_AP_W3, _AP_S3>& op1, \ const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>& op2
1796	—	operator	BIN_OP	const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>& op1, \ const ap_int_base<_AP_W3, _AP_S3>& op2
1821	—	operator	ASSIGN_OP	\ const ap_int_base<_AP_W3, _AP_S3>& op1, \ const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>& op2
1831	—	operator	ASSIGN_OP	const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>& op1, \ const ap_int_base<_AP_W3, _AP_S3>& op2

Line	Template signature	Returns	Name	Parameters
1858	—	INLINE bool operator	REL_OP	\ const ap_int_base<_AP_W3, _AP_S3>& op1, \ const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>& op2
1866	—	INLINE bool operator	REL_OP	\ const ap_con- cat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>& op1, \ const ap_int_base<_AP_W3, _AP_S3>& op2

ap_int_ref.h

Table 136: Every class and struct in ap_int_ref.h (3 definitions)

Line	Kind	Name / special- isation	Template parameters	Bases	Member typedefs
37	struct	ap_concat_ref	template <int _AP_W1, typename _AP_T1, int _AP_W2, typename _AP_T2>	—	—
301	struct	ap_range_ref	template <int _AP_W, bool _AP_S>	—	typedef ap_int_base<_AP_W, _AP_S> ref_type;
740	struct	ap_bit_ref	template <int _AP_W, bool _AP_S>	—	typedef ap_int_base<_AP_W, _AP_S> ref_type;

Table 137: Every function definition in ap_int_ref.h (35 definitions)

Line	Template signature	Returns	Name	Parameters
149	—	INLINE operator	ap_ulong	()
269	—	INLINE ap_int_base<_AP_W, false>	get	()
281	template <int _AP_W3>	INLINE void	set	const ap_int_base<_AP_W3, false>& val
294	—	INLINE int	length	()
326	template <int _AP_W, bool _AP_S>	INLINE operator	ap_ulong	()
612	—	INLINE ap_int_base<_AP_W, false>	get	()
618	template <int _AP_W2>	INLINE void	set	const ap_int_base<_AP_W2, false>& val
623	—	INLINE int	length	()
627	—	INLINE int	to_int	()
631	—	INLINE unsigned	to_uint	()

Line	Template signature	Returns	Name	Parameters
635	—	INLINE long	to_long	()
639	—	INLINE unsigned long	to_ulong	()
643	—	INLINE ap_slong	to_int64	()
647	—	INLINE ap_ulong	to_uint64	()
651	—	INLINE bool	and_reduce	()
665	—	INLINE bool	or_reduce	()
679	—	INLINE bool	xor_reduce	()
693	—	std::string	to_string	signed char radix = 2
700	—	INLINE char*	to_string	signed char radix = 2
758	template <int _AP_W, bool _AP_S>	INLINE operator	bool	()
759	template <int _AP_W, bool _AP_S>	INLINE bool	to_bool	()
940	—	INLINE bool	get	()
942	—	INLINE bool	get	()
944	template <int _AP_W3>	INLINE void	set	const ap_int_base<_AP_W3, false>& val
954	—	INLINE int	length	()
957	—	std::string	to_string	()
960	—	INLINE char*	to_string	()
970	—	INLINE bool operator	REL_OP	const ap_range_ref<_AP_W, _AP_S>& op, \ C_TYPE op2
976	—	INLINE bool operator	REL_OP	const ap_bit_ref<_AP_W, _AP_S>& op, \ C_TYPE op2
981	—	INLINE bool operator	REL_OP	C_TYPE op2, \ const ap_bit_ref<_AP_W, _AP_S>& op
986	—	INLINE bool operator	REL_OP	\ const ap_concat_ref<_AP_W, _AP_T, _AP_W1, _AP_T1>& op, C_TYPE op2
1038	—	operator	BIN_OP	const ap_range_ref<_AP_W, _AP_S>& op, C_TYPE op2
1045	—	operator	BIN_OP	C_TYPE op2, const ap_range_ref<_AP_W, _AP_S>& op
1103	—	operator	BIN_OP	const ap_range_ref<_AP_W, _AP_S>& lhs, \ const ap_range_ref<_AP_W2, _AP_S2>& rhs

Line	Template signature	Returns	Name	Parameters
1152	—	operator	BIN_OP	\ const ap_concat_ref<_AP_LW1, _AP_LT1, _AP_LW2, _AP_LT2>& lhs, \ const ap_concat_ref<_AP_RW1, _AP_RT1, _AP_RW2, _AP_RT2>& rhs

ap_int_special.h

Table 138: Every class and struct in ap_int_special.h (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
60	class	complex <ap_int<_AP_W> >	template <int _AP_W>	—	typedef ap_int<_AP_W> _Tp; typedef _Tp value_type;

Table 139: Every function definition in ap_int_special.h (8 definitions)

Line	Template signature	Returns	Name	Parameters
79	—	const _Tp&	real	()
80	—	const _Tp&	imag	()
82	—	_Tp&	real	()
83	—	const _Tp&	real	()
84	—	_Tp&	imag	()
85	—	const _Tp&	imag	()
88	—	void	real	_Tp __val
90	—	void	imag	_Tp __val

etc/ap_private.h

Table 140: Every class and struct in etc/ap_private.h (19 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
1250	struct	valtype <_AP_N8, false>	template <int _AP_N8>	—	typedef uint64_t Type;
1255	struct	valtype <_AP_N8, true>	template <int _AP_N8>	—	typedef int64_t Type;
1260	struct	valtype <1, false>	template <>	—	typedef unsigned char Type;
1264	struct	valtype <2, false>	template <>	—	typedef unsigned short Type;
1268	struct	valtype <3, false>	template <>	—	typedef unsigned int Type;
1272	struct	valtype <4, false>	template <>	—	typedef unsigned int Type;
1276	struct	valtype <1, true>	template <>	—	typedef signed char Type;
1280	struct	valtype <2, true>	template <>	—	typedef short Type;
1284	struct	valtype <3, true>	template <>	—	typedef int Type;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
1288	struct	valtype <4, true>	template <>	—	typedef int Type;
1293	struct	ap_private_enable_if	template <bool enable>	—	—
1295	struct	ap_private_enable_if <true>	template <>	—	—
1301	class	ap_private <_AP_W, _AP_S, true>	template <int _AP_W, bool _AP_S>	—	typedef typename valtype<(_AP_W + 7) / 8, _AP_S>::Type ValType; typedef ap_private<_AP_W, _AP_S> Type; typedef ap_private<mult_w, mult_s> mult; typedef ap_private<plus_w, plus_s> plus; typedef ap_private<minus_w, minus_s> minus; typedef ap_private<logic_w, logic_s> logic; typedef ap_private<div_w, div_s> div; typedef ap_private<mod_w, mod_s> mod; typedef ap_private<_AP_W, _AP_S> arg1; typedef bool reduce;
1312	struct	RType	template <int _AP_W2, bool _AP_S2>	—	typedef ap_private<mult_w, mult_s> mult; typedef ap_private<plus_w, plus_s> plus; typedef ap_private<minus_w, minus_s> minus; typedef ap_private<logic_w, logic_s> logic; typedef ap_private<div_w, div_s> div; typedef ap_private<mod_w, mod_s> mod; typedef ap_private<_AP_W, _AP_S> arg1; typedef bool reduce;
1348	struct	sign_ext_mask	template <int _AP_W1>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
3017	class	ap_private <_AP_W, _AP_S, false>	template <int _AP_W, bool _AP_S>	—	typedef typename valtype<8, _AP_S>::Type ValType; typedef ap_private<_AP_W, _AP_S> Type; typedef ap_private<mult_w, mult_s> mult; typedef ap_private<plus_w, plus_s> plus; typedef ap_private<minus_w, minus_s> minus; typedef ap_private<logic_w, lo- gic_s> logic; typedef ap_private<div_w, div_s> div; typedef ap_private<mod_w, mod_s> mod; typedef ap_private<_AP_W, _AP_S> arg1; typedef bool reduce;
3120	struct	RType	template <int _AP_W2, bool _AP_S2>	—	typedef ap_private<mult_w, mult_s> mult; typedef ap_private<plus_w, plus_s> plus; typedef ap_private<minus_w, minus_s> minus; typedef ap_private<logic_w, lo- gic_s> logic; typedef ap_private<div_w, div_s> div; typedef ap_private<mod_w, mod_s> mod; typedef ap_private<_AP_W, _AP_S> arg1; typedef bool reduce;
6105	struct	_private_range_ref	template <int _AP_W, bool _AP_S>	—	—
6587	struct	_private_bit_ref	template <int _AP_W, bool _AP_S>	—	—

Table 141: Every function definition in etc/ap_private.h (335 definitions)

Line	Template signature	Returns	Name	Parameters
65	template <int _AP_W, bool _AP_S>	template <class DataType> DataType INLINE	min	DataType a, DataType b
70	template <class Data- Type>	DataType INLINE	max	DataType a, DataType b
89	—	static INLINE uint32_t	Hi_32	uint64_t Value
94	—	static INLINE uint32_t	Lo_32	uint64_t Value
98	template <int _AP_W>	INLINE bool	isNegative	const ap_private<_AP_W, false>& a

Line	Template signature	Returns	Name	Parameters
103	template <int _AP_W>	INLINE bool	isNegative	const ap_private<_AP_W, true>& a
117	—	static INLINE unsigned	CountLeadingZeros_32	uint32_t Value
145	—	static INLINE unsigned	CountLeadingZeros_64	uint64_t Value
189	—	static INLINE unsigned	CountTrailingZeros_64	uint64_t Value
204	—	static INLINE unsigned	CountPopulation_64	uint64_t Value
215	—	static INLINE uint32_t	countLeadingOnes_64	uint64_t __V, uint32_t skip
225	—	static INLINE std::string	oct2Bin	char oct
262	—	static INLINE std::string	hex2Bin	char hex
329	—	static INLINE uint32_t	decode_digit	char cdigit, int radix
356	—	static INLINE std::string	parseString	const std::string& input, unsigned char& radix
501	—	static INLINE bool	sub_1	uint64_t x[], uint32_t len, uint64_t y
519	—	static INLINE bool	add_1	uint64_t dest[], uint64_t x[], uint32_t len, uint64_t y
537	—	static INLINE bool	add	uint64_t* dest, const uint64_t* x, const uint64_t* y, uint32_t destlen, uint32_t xlen, uint32_t ylen, bool xsigned, bool ysigned
569	—	static INLINE bool	sub	uint64_t* dest, const uint64_t* x, const uint64_t* y, uint32_t destlen, uint32_t xlen, uint32_t ylen, bool xsigned, bool ysigned
606	—	static INLINE uint64_t	mul_1	uint64_t dest[], const uint64_t x[], uint32_t len, uint64_t y
644	—	static INLINE void	mul	uint64_t dest[], const uint64_t x[], uint32_t xlen, const uint64_t y[], uint32_t ylen, uint32_t destlen
681	—	static INLINE void	KnuthDiv	uint32_t* u, uint32_t* v, uint32_t* q, uint32_t* r, uint32_t m, uint32_t n

Line	Template signature	Returns	Name	Parameters
857	template <int _AP_W, bool _AP_S>	void	divide	const ap_private<_AP_W, _AP_S>& LHS, uint32_t lhs- Words, const ap_private<_AP_W, _AP_S>& RHS, uint32_t rhsWords, ap_private<_AP_W, _AP_S>* Quotient, ap_private<_AP_W, _AP_S>* Remainder
1017	template <int _AP_W, bool _AP_S>	void	divide	const ap_private<_AP_W, _AP_S>& LHS, uint32_t lhs- Words, uint64_t RHS, ap_private<_AP_W, _AP_S>* Quotient, ap_private<_AP_W, _AP_S>* Remainder
1175	template <int _AP_W, bool _AP_S, bool _AP_C> INLINE ap_private<_AP_W, _AP_S, _AP_C>	—	lshr	const ap_private<_AP_W, _AP_S, _AP_C>& LHS, uint32_t shiftAmt
1183	template <int _AP_W, bool _AP_S, bool _AP_C> INLINE ap_private<_AP_W, _AP_S, _AP_C>	—	shl	const ap_private<_AP_W, _AP_S, _AP_C>& LHS, uint32_t shiftAmt
1360	template <int _AP_W1>	void	check_canary	()
1361	template <int _AP_W1>	void	set_canary	()
1363	template <int _AP_W1>	void	check_canary	()
1364	template <int _AP_W1>	void	set_canary	()
1367	template <int _AP_W1>	INLINE ValType&	get_VAL	void
1368	template <int _AP_W1>	INLINE ValType	get_VAL	void
1370	template <int _AP_W1>	INLINE void	set_VAL	uint64_t value
1371	template <int _AP_W1>	INLINE ValType&	get_pVal	int i
1372	template <int _AP_W1>	INLINE ValType	get_pVal	int i
1373	template <int _AP_W1>	INLINE const uint64_t*	get_pVal	()
1382	template <int _AP_W1>	INLINE void	set_pVal	int i, uint64_t value
1384	template <int _AP_W1>	INLINE uint32_t	getBitWidth	()
1475	—	INLINE bool	isSingleWord	()
1477	—	public: INLINE void	fromString	const char* strStart, uint32_t slen, uint8_t radix
1592	—	public: INLINE	ap_private	()
1640	—	explicit INLINE	ap_private	const char* val
1653	—	INLINE	ap_private	const char* val, signed char rd

Line	Template signature	Returns	Name	Parameters
1668	—	INLINE bool	isNegative	()
1673	—	INLINE bool	isPositive	()
1675	—	INLINE bool	isStrictly-Positive	()
1677	—	INLINE bool	isAllOnes-Value	()
1741	—	INLINE std::string	toStringUnsigned	uint8_t radix = 10
1744	—	INLINE std::string	toStringSigned	uint8_t radix = 10
1747	—	INLINE void	clear	()
1748	—	INLINE ap_private&	clear	uint32_t bitPosition
1754	—	INLINE ap_private	ashr	uint32_t shiftAmt
1763	—	INLINE ap_private	lshr	uint32_t shiftAmt
1789	—	INLINE int64_t	getSExtValue	()
1792	—	INLINE uint64_t	getZExtValue	()
1794	template <int _AP_W2, bool _AP_S2>	INLINE	ap_private	const _private_range_ref<_AP_W2, _AP_S2>& ref
1801	template <int _AP_W2, bool _AP_S2>	INLINE	ap_private	const _private_bit_ref<_AP_W2, _AP_S2>& ref
1839	—	INLINE operator	ValType	()
1841	—	INLINE int	to_uchar	()
1843	—	INLINE int	to_char	()
1845	—	INLINE int	to_ushort	()
1847	—	INLINE int	to_short	()
1849	—	INLINE int	to_int	()
1854	—	INLINE unsigned	to_uint	()
1856	—	INLINE long	to_long	()
1858	—	INLINE unsigned long	to_ulong	()
1860	—	INLINE ap_slong	to_int64	()
1862	—	INLINE ap_ulong	to_uint64	()
1864	—	INLINE double	to_double	()
1871	—	INLINE unsigned	length	()
1873	—	INLINE bool	isMinValue	()
1955	—	INLINE ap_private	And	const ap_private& RHS
1959	—	INLINE ap_private	Or	const ap_private& RHS
1963	—	INLINE ap_private	Xor	const ap_private& RHS
1981	—	INLINE ap_private	Mul	const ap_private& RHS

Line	Template signature	Returns	Name	Parameters
1985	—	INLINE ap_private	Add	const ap_private& RHS
1989	—	INLINE ap_private	Sub	const ap_private& RHS
2024	—	INLINE bool	isMinSigned-Value	()
2055	—	INLINE uint32_t	countPopula- tion	()
2058	—	INLINE uint32_t	countLead- ingZeros	()
2067	—	INLINE ap_private<_AP_W, _AP_S>	getHiBits	uint32_t numBits
2074	—	INLINE ap_private<_AP_W, _AP_S>	getLoBits	uint32_t numBits
2082	—	INLINE ap_private<_AP_W, _AP_S>&	set	uint32_t bitPosition
2088	—	INLINE void	set	()
2093	template <int _AP_W3>	INLINE void	set	const ap_private<_AP_W3, false>& val
2098	—	INLINE void	set	const ap_private& val
2114	—	INLINE void	clearUnused- BitsToZero	void
2120	—	INLINE ap_private	udiv	const ap_private& RHS
2126	—	INLINE ap_private	sdiv	const ap_private& RHS
2137	template <bool _AP_S2>	INLINE ap_private	urem	const ap_private<_AP_W, _AP_S2>& RHS
2145	template <bool _AP_S2>	INLINE ap_private	srem	const ap_private<_AP_W, _AP_S2>& RHS
2161	template <int _AP_W1, bool _AP_S1>	INLINE bool	eq	const ap_private<_AP_W1, _AP_S1>& RHS
2166	template <int _AP_W1, bool _AP_S1>	INLINE bool	ne	const ap_private<_AP_W1, _AP_S1>& RHS
2175	template <int _AP_W1, bool _AP_S1>	INLINE bool	ult	const ap_private<_AP_W1, _AP_S1>& RHS
2210	template <int _AP_W1, bool _AP_S1>	INLINE bool	ule	const ap_private<_AP_W1, _AP_S1>& RHS
2219	template <int _AP_W1, bool _AP_S1>	INLINE bool	sle	const ap_private<_AP_W1, _AP_S1>& RHS
2228	template <int _AP_W1, bool _AP_S1>	INLINE bool	ugt	const ap_private<_AP_W1, _AP_S1>& RHS
2237	template <int _AP_W1, bool _AP_S1>	INLINE bool	sgt	const ap_private<_AP_W1, _AP_S1>& RHS
2246	template <int _AP_W1, bool _AP_S1>	INLINE bool	uge	const ap_private<_AP_W1, _AP_S1>& RHS
2255	template <int _AP_W1, bool _AP_S1>	INLINE bool	sge	const ap_private<_AP_W1, _AP_S1>& RHS

Line	Template signature	Returns	Name	Parameters
2260	—	INLINE ap_private	abs	()
2265	—	INLINE ap_private<_AP_W, false>	get	()
2270	—	INLINE static uint32_t	getBits- Needed	const char* str, uint32_t slen, uint8_t radix
2275	—	INLINE uint32_t	getActive- Bits	()
2280	—	INLINE double	roundTo- Double	bool isSigned = false
2286	—	INLINE ap_private&	reverse	()
2303	—	INLINE bool	iszero	()
2305	—	INLINE bool	to_bool	()
2308	—	INLINE bool	sign	()
2314	—	INLINE void	invert	int i
2321	—	INLINE bool	test	int i
2329	—	INLINE void	lrotate	int n
2337	—	INLINE void	rrotate	int n
2344	—	INLINE void	set	int i, bool v
2351	—	INLINE void	set_bit	int i, bool v
2358	—	INLINE bool	get_bit	int i
2365	—	INLINE ap_private&	flip	()
2372	—	INLINE ap_private&	flip	uint32_t bitPosition
2379	—	INLINE void	b_not	()
2385	—	INLINE typename RType<_AP_W2, _AP_S2>::Rty operator	Sym	\ const ap_private<_AP_W2, _AP_S2>& op
2654	—	INLINE _private_range_ref<_AP_W, _AP_S>	range	int Hi, int Lo
2659	—	INLINE _private_range_ref<_AP_W, _AP_S>	range	int Hi, int Lo
2685	—	INLINE _private_bit_ref<_AP_W, _AP_S>	bit	int index
2689	template <int _AP_W2, bool _AP_S2> INLINE _private_bit_ref<_AP_W, _AP_S>	—	bit	const ap_private<_AP_W2, _AP_S2>& index
2694	—	INLINE const _private_bit_ref<_AP_W, _AP_S>	bit	int index

Line	Template signature	Returns	Name	Parameters
2699	template <int _AP_W2, bool _AP_S2> INLINE const _private_bit_ref<_AP_W, _AP_S>	—	bit	const ap_private<_AP_W2, _AP_S2>& index
2885	—	INLINE bool	and_reduce	()
2887	—	INLINE bool	nand_reduce	()
2889	—	INLINE bool	or_reduce	()
2891	—	INLINE bool	nor_reduce	()
2893	—	INLINE bool	xor_reduce	()
2898	—	INLINE bool	xnor_reduce	()
2903	—	INLINE std::string	to_string	uint8_t radix = 2, bool sign = false
3028	template <int _AP_W, bool _AP_S>	private: INLINE	ap_private	uint32_t numWords, const uint64_t bigVal[]
3064	template <int _AP_W, bool _AP_S>	INLINE	ap_private	const std::string& val, uint8_t radix = 2
3082	template <int _AP_W, bool _AP_S>	INLINE	ap_private	const char strStart[], uint32_t slen, uint8_t radix
3088	template <int _AP_W, bool _AP_S>	INLINE void	report	()
3103	—	INLINE void	check_canary	()
3104	—	INLINE void	set_canary	()
3106	—	INLINE void	check_canary	()
3107	—	INLINE void	set_canary	()
3147	template <int _AP_W2, bool _AP_S2>	INLINE uint64_t&	get_VAL	void
3148	template <int _AP_W2, bool _AP_S2>	INLINE uint64_t	get_VAL	void
3150	template <int _AP_W2, bool _AP_S2>	INLINE void	set_VAL	uint64_t value
3151	template <int _AP_W2, bool _AP_S2>	INLINE uint64_t&	get_pVal	int index
3152	template <int _AP_W2, bool _AP_S2>	INLINE uint64_t*	get_pVal	()
3153	template <int _AP_W2, bool _AP_S2>	INLINE const uint64_t*	get_pVal	()
3154	template <int _AP_W2, bool _AP_S2>	INLINE uint64_t	get_pVal	int index
3157	—	INLINE void	set_pVal	int i, uint64_t value
3172	—	public: explicit INLINE	ap_private	const char* val
3186	—	INLINE	ap_private	const char* val, unsigned char rd

Line	Template signature	Returns	Name	Parameters
3200	template <int _AP_W2, bool _AP_S2>	INLINE	ap_private	const _private_range_ref<_AP_W2, _AP_S2>& ref
3208	template <int _AP_W2, bool _AP_S2>	INLINE	ap_private	const _private_bit_ref<_AP_W2, _AP_S2>& ref
3246	—	INLINE	ap_private	const ap_private& that
3253	template <int _AP_W1, bool _AP_S1>	INLINE	ap_private	const ap_private<_AP_W1, _AP_S1, false>& that
3260	template <int _AP_W1, bool _AP_S1>	INLINE	ap_private	const volatile ap_private<_AP_W1, _AP_S1, false>& that
3267	template <int _AP_W1, bool _AP_S1>	INLINE	ap_private	const ap_private<_AP_W1, _AP_S1, true>& that
3286	template <int _AP_W1, bool _AP_S1>	INLINE	ap_private	const volatile ap_private<_AP_W1, _AP_S1, true>& that
3303	—	INLINE	ap_private	()
3309	—	INLINE	ap_private	uint64_t* val, uint32_t bits = _AP_W
3310	—	INLINE	ap_private	const uint64_t* const val, uint32_t bits
3321	—	INLINE	ap_private	TYPE val, bool isSigned = SIGNED
3354	—	INLINE bool	isSingleWord	()
3358	—	static INLINE uint32_t	whichWord	uint32_t bitPosition
3366	—	static INLINE uint32_t	whichBit	uint32_t bitPosition
3375	—	static INLINE uint64_t	maskBit	uint32_t bitPosition
3381	—	INLINE uint64_t	getWord	uint32_t bitPosition
3403	—	INLINE void	clearUnused- BitsToZero	void
3405	—	INLINE void	clearUnused- BitsToOne	void
3409	—	INLINE void	fromString	const char* str, uint32_t slen, uint8_t radix
3520	—	INLINE operator	ValType	()
3522	—	INLINE int	to_uchar	()
3524	—	INLINE int	to_char	()
3526	—	INLINE int	to_ushort	()
3528	—	INLINE int	to_short	()
3530	—	INLINE int	to_int	()
3532	—	INLINE unsigned	to_uint	()
3534	—	INLINE long	to_long	()

Line	Template signature	Returns	Name	Parameters
3536	—	INLINE unsigned long	to_ulong	()
3538	—	INLINE ap_slong	to_int64	()
3540	—	INLINE ap_ulong	to_uint64	()
3542	—	INLINE double	to_double	()
3549	—	INLINE unsigned	length	()
3553	—	INLINE ap_private&	reverse	()
3570	—	INLINE bool	iszero	()
3572	—	INLINE bool	to_bool	()
3575	—	INLINE bool	sign	()
3581	—	INLINE void	invert	int i
3588	—	INLINE bool	test	int i
3595	—	INLINE void	set	int i, bool v
3602	—	INLINE void	set_bit	int i, bool v
3609	—	INLINE ap_private&	set	uint32_t bitPosition
3615	—	INLINE void	set	()
3621	—	INLINE bool	get	int i
3628	—	INLINE bool	get_bit	int i
3636	—	INLINE void	lrotate	int n
3644	—	INLINE void	rrotate	int n
3652	—	INLINE ap_private&	clear	uint32_t bitPosition
3659	—	INLINE void	clear	()
3662	—	ap_private&	flip	()
3669	—	INLINE ap_private&	flip	uint32_t bitPosition
3676	—	INLINE void	b_not	()
3678	—	INLINE ap_private	getLoBits	uint32_t numBits
3683	—	INLINE ap_private	getHiBits	uint32_t numBits
3713	—	INLINE ap_private& operator	Sym	const ap_private<_AP_W1, _AP_S1>& RHS
3814	—	INLINE typename RType<_AP_W1, _AP_S1>::logic operator	Sym	\ const ap_private<_AP_W1, _AP_S1>& RHS
4138	—	INLINE _private_range_ref<_AP_W, _AP_S>	range	int Hi, int Lo
4143	—	INLINE _private_range_ref<_AP_W, _AP_S>	range	int Hi, int Lo
4147	template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>	INLINE _private_range_ref<_AP_	range	const ap_private<_AP_W2, _AP_S2>& HiIdx, const ap_private<_AP_W3, _AP_S3>& LoIdx

Line	Template signature	Returns	Name	Parameters
4165	template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>	INLINE _private_range_ref<_AP_	range	const ap_private<_AP_W2, _AP_S2>& HiIdx, const ap_private<_AP_W3, _AP_S3>& LoIdx
4205	—	INLINE _private_bit_ref<_AP_W, _AP_S>	bit	int index
4209	template <int _AP_W2, bool _AP_S2> INLINE _private_bit_ref<_AP_W, _AP_S>	—	bit	const ap_private<_AP_W2, _AP_S2>& index
4214	—	INLINE const _private_bit_ref<_AP_W, _AP_S>	bit	int index
4219	template <int _AP_W2, bool _AP_S2> INLINE const _private_bit_ref<_AP_W, _AP_S>	—	bit	const ap_private<_AP_W2, _AP_S2>& index
4385	—	INLINE ap_private<_AP_W, false>	get	()
4390	template <int _AP_W3>	INLINE void	set	const ap_private<_AP_W3, false>& val
4401	—	INLINE bool	isNegative	()
4410	—	INLINE bool	isPositive	()
4415	—	INLINE bool	isStrictly- Positive	()
4422	—	INLINE bool	isAllOnes- Value	()
4427	—	INLINE bool	isMaxValue	()
4432	—	INLINE bool	isMaxSigned- Value	()
4439	—	INLINE bool	isMinValue	()
4444	—	INLINE bool	isMinSigned- Value	()
4451	—	INLINE const uint64_t*	getRawData	()
4460	—	INLINE ap_private	sqrt	()
4721	template <bool _AP_S1>	INLINE ap_private	Or	const ap_private<_AP_W, _AP_S1>& RHS
4725	template <bool _AP_S1>	INLINE ap_private	Xor	const ap_private<_AP_W, _AP_S1>& RHS
4730	—	INLINE ap_private	Mul	const ap_private& RHS
4736	—	INLINE ap_private	Add	const ap_private& RHS
4744	—	INLINE ap_private	Sub	const ap_private& RHS
4754	—	INLINE ap_private	ashr	uint32_t shiftAmt

Line	Template signature	Returns	Name	Parameters
4828	—	INLINE ap_private	lshr	uint32_t shiftAmt
4883	—	INLINE ap_private	shl	uint32_t shiftAmt
4933	—	INLINE ap_private	rotr	uint32_t rotateAmt
4943	—	INLINE ap_private	rotr	uint32_t rotateAmt
4958	—	INLINE ap_private	udiv	const ap_private& RHS
4990	—	INLINE ap_private	sdiv	const ap_private& RHS
5009	—	INLINE ap_private	urem	const ap_private& RHS
5041	—	INLINE ap_private	urem	uint64_t RHS
5072	—	INLINE ap_private	srem	const ap_private& RHS
5089	—	INLINE ap_private	srem	int64_t RHS
5104	template <bool _AP_S1>	INLINE bool	eq	const ap_private<_AP_W, _AP_S1>& RHS
5113	template <bool _AP_S1>	INLINE bool	ne	const ap_private<_AP_W, _AP_S1>& RHS
5122	template <bool _AP_S1>	INLINE bool	ult	const ap_private<_AP_W, _AP_S1>& RHS
5147	—	INLINE bool	ult	uint64_t RHS
5165	template <bool _AP_S1>	INLINE bool	slt	const ap_private<_AP_W, _AP_S1>& RHS
5199	template <bool _AP_S1>	INLINE bool	ule	const ap_private<_AP_W, _AP_S1>& RHS
5208	template <bool _AP_S1>	INLINE bool	sle	const ap_private<_AP_W, _AP_S1>& RHS
5217	template <bool _AP_S1>	INLINE bool	ugt	const ap_private<_AP_W, _AP_S1>& RHS
5226	template <bool _AP_S1>	INLINE bool	sgt	const ap_private<_AP_W, _AP_S1>& RHS
5235	template <bool _AP_S1>	INLINE bool	uge	const ap_private<_AP_W, _AP_S1>& RHS
5244	template <bool _AP_S1>	INLINE bool	sge	const ap_private<_AP_W, _AP_S1>& RHS
5250	template <int _AP_W1, bool _AP_S1>	INLINE void	cpSext	const ap_private<_AP_W1, _AP_S1>& that
5286	template <int _AP_W1, bool _AP_S1>	INLINE void	cpZext	const ap_private<_AP_W1, _AP_S1>& that
5297	template <int _AP_W1, bool _AP_S1>	INLINE void	cpZex- tOrTrunc	const ap_private<_AP_W1, _AP_S1>& that
5307	template <int _AP_W1, bool _AP_S1>	INLINE void	cpSex- tOrTrunc	const ap_private<_AP_W1, _AP_S1>& that
5322	—	INLINE uint32_t	getBitWidth	()
5327	—	INLINE uint32_t	getNumWords	()
5335	—	INLINE uint32_t	getActive- Bits	()

Line	Template signature	Returns	Name	Parameters
5345	—	INLINE uint64_t	getZExtValue	()
5355	—	INLINE int64_t	getSExtValue	()
5363	—	INLINE static uint32_t	getBits-Needed	const char* str, uint32_t slen, uint8_t radix
5397	—	INLINE uint32_t	countLeadingZeros	()
5424	—	INLINE uint32_t	countLeadingOnes	()
5453	—	INLINE uint32_t	countTrailingZeros	()
5466	—	INLINE uint32_t	countPopulation	()
5487	—	INLINE std::string	toStringUnsigned	uint8_t radix = 10
5496	—	INLINE std::string	toStringSigned	uint8_t radix = 10
5501	—	INLINE double	roundToDouble	bool isSigned
5563	—	INLINE double	roundToDouble	()
5566	—	INLINE double	signed-RoundToDouble	()
5572	—	INLINE double	bitsToDouble	()
5585	—	INLINE float	bitsToFloat	()
5598	—	INLINE ap_private&	doubleToBits	double __V
5612	—	INLINE ap_private&	floatToBits	float __V
5623	—	INLINE bool	and_reduce	()
5625	—	INLINE bool	nand_reduce	()
5627	—	INLINE bool	or_reduce	()
5629	—	INLINE bool	nor_reduce	()
5631	—	INLINE bool	xor_reduce	()
5636	—	INLINE bool	xnor_reduce	()
5640	—	INLINE std::string	to_string	uint8_t radix = 16, bool sign = false
5658	template <int _AP_W, bool _AP_S, int index>	INLINE bool	get	const ap_private<_AP_W, _AP_S>& a
6215	—	INLINE operator unsigned long	long	()
6423	template <int _AP_W2>	INLINE void	set	const ap_private<_AP_W2, false>& val
6471	—	INLINE ap_private<_AP_W, false>	get	()

Line	Template signature	Returns	Name	Parameters
6494	—	INLINE ap_private<_AP_W, false>	get	()
6516	—	INLINE int	length	()
6520	—	INLINE int	to_int	()
6525	—	INLINE unsigned int	to_uint	()
6530	—	INLINE long	to_long	()
6535	—	INLINE unsigned long	to_ulong	()
6540	—	INLINE ap_slong	to_int64	()
6545	—	INLINE ap_ulong	to_uint64	()
6550	—	INLINE std::string	to_string	uint8_t radix = 2
6554	—	INLINE bool	and_reduce	()
6563	—	INLINE bool	or_reduce	()
6572	—	INLINE bool	xor_reduce	()
6608	template <int _AP_W, bool _AP_S>	INLINE operator	bool	()
6610	template <int _AP_W, bool _AP_S>	INLINE bool	to_bool	()
6716	—	INLINE bool	get	()
6728	—	INLINE int	length	()
6744	—	INLINE PTR_TYPE* operator	BIN_OP	PTR_TYPE* i_op, \ const ap_private<_AP_W, _AP_S>& op
6750	—	INLINE PTR_TYPE* operator	BIN_OP	const ap_private<_AP_W, _AP_S>& op, \ PTR_TYPE* i_op
6764	—	INLINE C_TYPE operator	BIN_OP	C_TYPE i_op, \ const ap_private<_AP_W, _AP_S>& op
6770	—	INLINE C_TYPE operator	BIN_OP	const ap_private<_AP_W, _AP_S>& op, \ C_TYPE i_op
6799	—	operator	BIN_OP	C_TYPE i_op, const ap_private<_AP_W, _AP_S>& op
6805	—	operator	BIN_OP	const ap_private<_AP_W, _AP_S>& op, C_TYPE i_op
6811	—	INLINE bool operator	REL_OP	const ap_private<_AP_W, _AP_S>& op, \ C_TYPE op2
6816	—	INLINE bool operator	REL_OP	C_TYPE op2, \ const ap_private<_AP_W, _AP_S, false>& op
6823	—	INLINE ap_private<_AP_W, _AP_S>& operator	ASSIGN_OP	\ ap_private<_AP_W, _AP_S>& op, C_TYPE op2

Line	Template signature	Returns	Name	Parameters
6830	—	C_TYPE operator	BIN_OP	C_TYPE i_op, \ const ap_private<_AP_W, _AP_S, false>& op
6837	—	operator	BIN_OP	const ap_private<_AP_W, _AP_S>& op, C_TYPE i_op
6843	—	INLINE ap_private<_AP_W, _AP_S>& operator	ASSIGN_OP	\ ap_private<_AP_W, _AP_S>& op, C_TYPE op2
6851	—	INLINE ap_private<_AP_W, _AP_S>& operator	ASSIGN_OP	\ ap_private<_AP_W, _AP_S>& op, C_TYPE op2
6912	—	operator	BIN_OP	const _private_range_ref<_AP_W1, _AP_S1>& op1, \ const ap_private<_AP_W2, _AP_S2>& op2
6919	—	operator	BIN_OP	const ap_private<_AP_W1, _AP_S1>& op1, \ const _private_range_ref<_AP_W2, _AP_S2>& op2
6926	—	INLINE ap_private<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ ap_private<_AP_W1, _AP_S1>& op1, \ const _private_range_ref<_AP_W2, _AP_S2>& op2
6932	—	INLINE _private_range_ref<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ _private_range_ref<_AP_W1, _AP_S1>& op1, \ ap_private<_AP_W2, _AP_S2>& op2
6943	—	INLINE bool operator	REL_OP	const _private_range_ref<_AP_W1, _AP_S1>& op1, \ const ap_private<_AP_W2, _AP_S2>& op2
6948	—	INLINE bool operator	REL_OP	const ap_private<_AP_W1, _AP_S1>& op1, \ const _private_range_ref<_AP_W2, _AP_S2>& op2
6988	—	operator	BIN_OP	const _private_bit_ref<_AP_W1, _AP_S1>& op1, \ const ap_private<_AP_W2, _AP_S2>& op2
6994	—	operator	BIN_OP	const ap_private<_AP_W1, _AP_S1>& op1, \ const _private_bit_ref<_AP_W2, _AP_S2>& op2
7001	—	INLINE ap_private<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ ap_private<_AP_W1, _AP_S1>& op1, \ _private_bit_ref<_AP_W2, _AP_S2>& op2

Line	Template signature	Returns	Name	Parameters
7007	—	INLINE _private_bit_ref<_AP_W1, _AP_S1>& operator	ASSIGN_OP	\ _private_bit_ref<_AP_W1, _AP_S1>& op1, \ ap_private<_AP_W2, _AP_S2>& op2
7018	—	INLINE bool operator	REL_OP	const _private_bit_ref<_AP_W1, _AP_S1>& op1, \ const ap_private<_AP_W2, _AP_S2>& op2
7023	—	INLINE bool operator	REL_OP	const ap_private<_AP_W1, _AP_S1>& op1, \ const _private_bit_ref<_AP_W2, _AP_S2>& op2
7062	—	INLINE bool operator	REL_OP	const _private_range_ref<_AP_W, _AP_S>& op, \ C_TYPE op2
7069	—	INLINE bool operator	REL_OP	C_TYPE op2, \ const _private_range_ref<_AP_W, _AP_S>& op
7075	—	INLINE bool operator	REL_OP	const _private_bit_ref<_AP_W, _AP_S>& op, \ C_TYPE op2
7080	—	INLINE bool operator	REL_OP	C_TYPE op2, \ const _private_bit_ref<_AP_W, _AP_S>& op
7112	—	operator	BIN_OP	const _private_range_ref<_AP_W, _AP_S>& op, \ C_TYPE op2
7121	—	operator	BIN_OP	C_TYPE op2, \ const _private_range_ref<_AP_W, _AP_S>& op
7158	—	operator	BIN_OP	const _private_range_ref<_AP_W, _AP_S>& lhs, \ const _private_range_ref<_AP_W2, _AP_S2>& rhs

etc/hls_all_any_apfixed.h

Table 142: Every function definition in etc/hls_all_any_apfixed.h (8 definitions)

Line	Template signature	Returns	Name	Parameters
12	template <int W_, int I_>	bool	generic_all	ap_fixed<W_, I_> x
23	template <int W_, int I_>	bool	generic_all	ap_ufixed<W_, I_> x
34	template <int I_>	bool	generic_all	ap_int<I_> x
45	template <int I_>	bool	generic_all	ap_uint<I_> x
56	template <int W_, int I_>	bool	generic_any	ap_fixed<W_, I_> x
67	template <int W_, int I_>	bool	generic_any	ap_ufixed<W_, I_> x
78	template <int I_>	bool	generic_any	ap_int<I_> x

Line	Template signature	Returns	Name	Parameters
89	template <int I>	bool	generic_any	ap_uint<I> x

etc/hls_cbrt_apfixed.h

Table 143: Every function definition in etc/hls_cbrt_apfixed.h (4 definitions)

Line	Template signature	Returns	Name	Parameters
15	template <int W, int I> ap_fixed<W, I>	—	cbrt_fixed	ap_fixed<W, I> x
122	template<int W, int I> ap_ufixed<W, I>	—	cbrt_fixed	ap_ufixed<W, I> x
127	template<int I> ap_int<I>	—	cbrt_fixed	ap_int<I> x
132	template<int I> ap_uint<I>	—	cbrt_fixed	ap_uint<I> x

etc/hls_comparison_apfixed.h

Table 144: Every function definition in etc/hls_comparison_apfixed.h (20 definitions)

Line	Template signature	Returns	Name	Parameters
13	template<int W, int I>	bool	isgreater_fixed	ap_fixed<W, I> x1, ap_fixed<W, I> x2
22	template<int W, int I>	bool	isgreater_fixed	ap_ufixed<W, I> x1, ap_ufixed<W, I> x2
31	template<int I>	bool	isgreater_fixed	ap_int<I> x1, ap_int<I> x2
40	template<int I>	bool	isgreater_fixed	ap_uint<I> x1, ap_uint<I> x2
51	template<int W, int I>	bool	isgreate-requal_fixed	ap_fixed<W, I> x1, ap_fixed<W, I> x2
60	template<int W, int I>	bool	isgreate-requal_fixed	ap_ufixed<W, I> x1, ap_ufixed<W, I> x2
69	template<int I>	bool	isgreate-requal_fixed	ap_int<I> x1, ap_int<I> x2
78	template<int I>	bool	isgreate-requal_fixed	ap_uint<I> x1, ap_uint<I> x2
89	template<int W, int I>	bool	isless_fixed	ap_fixed<W, I> x1, ap_fixed<W, I> x2
98	template<int W, int I>	bool	isless_fixed	ap_ufixed<W, I> x1, ap_ufixed<W, I> x2
107	template<int I>	bool	isless_fixed	ap_int<I> x1, ap_int<I> x2
116	template<int I>	bool	isless_fixed	ap_uint<I> x1, ap_uint<I> x2
127	template<int W, int I>	bool	islessequal_fixed	ap_fixed<W, I> x1, ap_fixed<W, I> x2
136	template<int W, int I>	bool	islessequal_fixed	ap_ufixed<W, I> x1, ap_ufixed<W, I> x2
145	template<int I>	bool	islessequal_fixed	ap_int<I> x1, ap_int<I> x2
154	template<int I>	bool	islessequal_fixed	ap_uint<I> x1, ap_uint<I> x2
165	template<int W, int I>	bool	islessgreater_fixed	ap_fixed<W, I> x1, ap_fixed<W, I> x2

Line	Template signature	Re- turns	Name	Parameters
174	template<int W, int I>	bool	islessgreater_fixed	ap_ufixed<W,I> x1, ap_ufixed<W,I> x2
183	template<int I>	bool	islessgreater_fixed	ap_int<I> x1, ap_int<I> x2
192	template<int I>	bool	islessgreater_fixed	ap_uint<I> x1, ap_uint<I> x2

etc/hls_cordic_apfixed.h

Table 145: Every function definition in etc/hls_cordic_apfixed.h (64 definitions)

Line	Template signature	Re- turns	Name	Parameters
31	template<int W, int I, int Wo>	void	circ_range_redux	ap_ufixed<W,I> x, ap_uint<2> &k, ap_ufixed<Wo,1> &r
50	template<int W, int I, int MODE>	void	cordic_circ_apfixed	ap_fixed<W,I> &x, ap_fixed<W,I> &y, ap_fixed<W,I> &z
109	template<int W, int I, int MODE, int IN- TYPE>	void	doublecordic_apfixed	ap_fixed<W,I> &x, ap_fixed<W,I> &y, ap_fixed<W,I> &z, ap_fixed<W,I> &t
200	template<int W, int I>	void	generic_sincos	ap_fixed<W,I> in, ap_fixed<W-I+2,2> & outsin, ap_fixed<W-I+2,2> & outcos
270	template<int W, int I> ap_fixed<W,I>	—	generic_tan	ap_fixed<W,I> x
300	template<int W, int I> ap_ufixed<W,I>	—	generic_tan	ap_ufixed<W,I> in
312	template<int I> ap_int<I>	—	generic_tan	ap_int<I> in
321	template<int I> ap_uint<I>	—	generic_tan	ap_uint<I> in
340	template<int W, int I, int Wo>	void	circ_range_redux_pi	ap_ufixed<W,I> x, ap_uint<2> &k, ap_ufixed<Wo,1> &r
353	template<int W, int I>	void	generic_sincospi	ap_fixed<W,I> in, ap_fixed<W-I+2,2> & outsin, ap_fixed<W-I+2,2> & outcos
422	template<int W, int I> ap_fixed<W,I>	—	generic_tanpi	ap_fixed<W,I> x
460	template<int W, int I> ap_ufixed<W,I>	—	generic_tanpi	ap_ufixed<W,I> in
472	template<int I> ap_int<I>	—	generic_tanpi	ap_int<I> in
479	template<int I> ap_uint<I>	—	generic_tanpi	ap_uint<I> in
486	template<int W, int I> ap_fixed<W,2>	—	generic_atan	ap_fixed<W,I> in

Line	Template signature	Re- turns	Name	Parameters
534	template<int W, int I> ap_ufixed<W, 2>	—	generic_atan	ap_ufixed<W, I> in
546	template<int I> ap_int<2>	—	generic_atan	ap_int<I> in
557	template<int I> ap_uint<1>	—	generic_atan	ap_uint<I> in
566	template<int W, int I> ap_fixed<W, 3>	—	generic_atan2	ap_fixed<W, I> in1, ap_fixed<W, I> in2
672	template<int W, int I> ap_ufixed<W, 3>	—	generic_atan2	ap_ufixed<W, I> in1, ap_ufixed<W, I> in2
686	template<int I> ap_int<3>	—	generic_atan2	ap_int<I> in1, ap_int<I> in2
697	template<int I> ap_uint<2>	—	generic_atan2	ap_uint<I> in1, ap_uint<I> in2
714	template<int W, int I, int Wo>	void	hyperb_range_redux	ap_ufixed<W, I> x, ap_uint<I+1> &k, ap_ufixed<Wo, 0> &r
732	template<int W, int I, int MODE>	void	cordic_hyperb_apfixed	ap_fixed<W, I> &x, ap_fixed<W, I> &y, ap_fixed<W, I> &z
784	template <int W, int I>	void	sinh_cosh_range_redux_cordic	ap_fixed<W, I> in, ap_fixed<W, I> &s_out, ap_fixed<W, I> &c_out
869	template<int W, int I>	void	tanh_range_redux_cordic	ap_fixed<W, I> in, ap_fixed<W, I> &tan_out
904	template<int W, int I> ap_fixed<W, I>	—	generic_tanh	ap_fixed<W, I> in
911	template<int W, int I> ap_ufixed<W, I>	—	generic_tanh	ap_ufixed<W, I> in
919	template<int I> ap_int<I>	—	generic_tanh	ap_int<I> in
927	template<int I> ap_uint<I>	—	generic_tanh	ap_uint<I> in
935	template<int W, int I> ap_fixed<W, I>	—	generic_sinh	ap_fixed<W, I> in
943	template<int W, int I> ap_ufixed<W, I>	—	generic_sinh	ap_ufixed<W, I> in
955	template<int I> ap_int<I>	—	generic_sinh	ap_int<I> in
964	template<int I> ap_uint<I>	—	generic_sinh	ap_uint<I> in
976	template<int W, int I> ap_fixed<W, I>	—	generic_cosh	ap_fixed<W, I> in
984	template<int W, int I> ap_ufixed<W, I>	—	generic_cosh	ap_ufixed<W, I> in

Line	Template signature	Re- turns	Name	Parameters
996	template<int I> ap_int<I>	—	generic_cosh	ap_int<I> in
1005	template<int I> ap_uint<I>	—	generic_cosh	ap_uint<I> in
1126	template<int W, int I>	void	cordic_semi_recovery_asinh	ap_fixed<W,I> in_top, ap_fixed<W,I>& out
1214	template<int W, int I>	void	cordic_semi_recovery_acosh	ap_fixed<W,I> in, ap_fixed<W,I>& out
1293	template<int W, int I> ap_fixed<W,I>	—	generic_asinh	ap_fixed<W,I> in
1301	template<int W, int I> ap_ufixed<W,I>	—	generic_asinh	ap_ufixed<W,I> in
1310	template<int I> ap_int<I>	—	generic_asinh	ap_int<I> in
1319	template<int I> ap_uint<I>	—	generic_asinh	ap_uint<I> in
1328	template<int W, int I> ap_fixed<W,I>	—	generic_acosh	ap_fixed<W,I> in
1337	template<int W, int I> ap_ufixed<W,I>	—	generic_acosh	ap_ufixed<W,I> in
1346	template<int I> ap_int<I>	—	generic_acosh	ap_int<I> in
1355	template<int I> ap_uint<I>	—	generic_acosh	ap_uint<I> in
1364	template<int W, int I> ap_fixed<W,I>	—	generic_asinpi	ap_fixed<W,I> x
1396	template<int W, int I> ap_ufixed<W,I>	—	generic_asinpi	ap_ufixed<W,I> x
1401	template<int I> ap_int<I>	—	generic_asinpi	ap_int<I> x
1420	template<int I> ap_uint<I>	—	generic_asinpi	ap_uint<I> x
1437	template<int W, int I> ap_fixed<W,I>	—	generic_acospi	ap_fixed<W,I> x
1473	template<int W, int I> ap_ufixed<W,I>	—	generic_acospi	ap_ufixed<W,I> x
1478	template<int I> ap_int<I>	—	generic_acospi	ap_int<I> x
1496	template<int I> ap_uint<I>	—	generic_acospi	ap_uint<I> x
1512	template<int W, int I> ap_fixed<W,I>	—	generic_atanpi	ap_fixed<W,I> x
1537	template<int W, int I> ap_ufixed<W,I>	—	generic_atanpi	ap_ufixed<W,I> x

Line	Template signature	Re- turns	Name	Parameters
1542	template<int I> ap_int<I>	—	generic_atanpi	ap_int<I> x
1548	template<int I> ap_uint<I>	—	generic_atanpi	ap_uint<I> x
1555	template<int W, int I> ap_fixed<W,I>	—	generic_atan2pi	ap_fixed<W,I> y, ap_fixed<W,I> x
1569	template<int W, int I> ap_ufixed<W,I>	—	generic_atan2pi	ap_ufixed<W,I> y, ap_ufixed<W,I> x
1575	template<int I> ap_int<I>	—	generic_atan2pi	ap_int<I> y, ap_int<I> x
1592	template<int I> ap_uint<I>	—	generic_atan2pi	ap_uint<I> y, ap_uint<I> x

etc/hls_diff_apfixed.h

Table 146: Every function definition in etc/hls_diff_apfixed.h (20 definitions)

Line	Template signature	Re- turns	Name	Parameters
20	template <int W_, int I_> ap_fixed<W_,I_>	—	fdim_fixed	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y
46	template <int W_, int I_> ap_ufixed<W_,I_>	—	fdim_fixed	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y
53	template <int I_> ap_int<I_>	—	fdim_fixed	ap_int<I_> x, ap_int<I_> y
60	template <int I_> ap_uint<I_>	—	fdim_fixed	ap_uint<I_> x, ap_uint<I_> y
76	template <int W_, int I_> ap_fixed<W_,I_>	—	fmax_fixed	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y
84	template <int W_, int I_> ap_ufixed<W_,I_>	—	fmax_fixed	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y
92	template <int I_> ap_int<I_>	—	fmax_fixed	ap_int<I_> x, ap_int<I_> y
100	template <int I_> ap_uint<I_>	—	fmax_fixed	ap_uint<I_> x, ap_uint<I_> y
117	template <int W_, int I_> ap_fixed<W_,I_>	—	fmin_fixed	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y
125	template <int W_, int I_> ap_ufixed<W_,I_>	—	fmin_fixed	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y
133	template <int I_> ap_int<I_>	—	fmin_fixed	ap_int<I_> x, ap_int<I_> y
141	template <int I_> ap_uint<I_>	—	fmin_fixed	ap_uint<I_> x, ap_uint<I_> y
160	template <int W_, int I_> ap_fixed<W_,I_>	—	maxmag_fixed	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y
192	template <int W_, int I_> ap_ufixed<W_,I_>	—	maxmag_fixed	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y
200	template <int I_> ap_int<I_>	—	maxmag_fixed	ap_int<I_> x, ap_int<I_> y
207	template <int I_> ap_uint<I_>	—	maxmag_fixed	ap_uint<I_> x, ap_uint<I_> y
226	template <int W_, int I_> ap_fixed<W_,I_>	—	minmag_fixed	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y

Line	Template signature	Re- turns	Name	Parameters
257	template <int W_, int I_> ap_ufixed<W_, I_>	—	minmag_fixed	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
265	template <int I_> ap_int<I_>	—	minmag_fixed	ap_int<I_> x, ap_int<I_> y
272	template <int I_> ap_uint<I_>	—	minmag_fixed	ap_uint<I_> x, ap_uint<I_> y

etc/hls_erf_apfixed.h

Table 147: Every class and struct in etc/hls_erf_apfixed.h (9 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
16	class	coeff	template <typename T>	—	—
18	class	coeff <double>	template <>	—	—
119	class	coeff <float>	template <>	—	—
219	class	erf_traits	template<typename T>	—	—
221	class	erf_traits <float>	template<>	—	—
241	class	erf_traits <double>	template<>	—	—
261	class	erfc_traits	template<typename T>	—	—
263	class	erfc_traits <float>	template<>	—	—
282	class	erfc_traits <double>	template<>	—	—

Table 148: Every function definition in etc/hls_erf_apfixed.h (8 definitions)

Line	Template signature	Returns	Name	Parameters
320	template<>	template <int W_, int I_> ap_fixed<W_, I_>	erf	ap_fixed<W_, I_> x_fixed
462	template <int W_, int I_> ap_ufixed<W_, I_>	—	erf	ap_ufixed<W_, I_> x
468	template <int I_> ap_int<I_>	—	erf	ap_int<I_> x
474	template <int I_> ap_uint<I_>	—	erf	ap_uint<I_> x
487	template <int W_, int I_> ap_fixed<W_, I_>	—	erfc	ap_fixed<W_, I_> x_fixed
638	template <int W_, int I_> ap_ufixed<W_, I_>	—	erfc	ap_ufixed<W_, I_> x
644	template <int I_> ap_int<I_>	—	erfc	ap_int<I_> x
650	template <int I_> ap_uint<I_>	—	erfc	ap_uint<I_> x

etc/hls_exp_apfixed.h

Table 149: Every function definition in etc/hls_exp_apfixed.h (17 definitions)

Line	Template signature	Re- turns	Name	Parameters
12	template<int W_, int I_> ap_fixed<W_,I_>	—	exp	ap_fixed<W_,I_> x
930	template<int W0_, int I_, int WI_> ap_fixed<W0_,I_>	—	exp_core	ap_fixed<WI_,I_> x
2074	template<int W_, int I_> ap_ufixed<W_,I_>	—	exp	ap_ufixed<W_,I_> x
2080	template<int I_> ap_int<I_>	—	exp	ap_int<I_> x
2086	template<int I_> ap_uint<I_>	—	exp	ap_uint<I_> x
2092	template<int W, int I> ap_fixed<W,I>	—	expm1	ap_fixed<W,I> x
2105	template<int W, int I> ap_ufixed<W,I>	—	expm1	ap_ufixed<W,I> x
2111	template<int I> ap_int<I>	—	expm1	ap_int<I> x
2117	template<int I> ap_uint<I>	—	expm1	ap_uint<I> x
2123	template<int W, int I> ap_fixed<W,I>	—	exp2	ap_fixed<W,I> x
2843	template<int W, int I> ap_ufixed<W,I>	—	exp2	ap_ufixed<W,I> x
2849	template<int I> ap_int<I>	—	exp2	ap_int<I> x
2855	template<int I> ap_uint<I>	—	exp2	ap_uint<I> x
2862	template<int W_, int I_> ap_fixed<W_,I_>	—	exp10	ap_fixed<W_,I_> x
3425	template<int W, int I> ap_ufixed<W,I>	—	exp10	ap_ufixed<W,I> x
3431	template<int I> ap_int<I>	—	exp10	ap_int<I> x
3437	template<int I> ap_uint<I>	—	exp10	ap_uint<I> x

etc/hls_fmod_rem_quo_remainder_divide_apfixed.h

Table 150: Every function definition in etc/hls_fmod_rem_quo_remainder_divide_apfixed.h (16 definitions)

Line	Template signature	Re- turns	Name	Parameters
20	template <int W_, int I_> ap_fixed<W_,I_>	—	gener- ic_divide	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y
34	template <int W_, int I_> ap_ufixed<W_,I_>	—	gener- ic_divide	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y
41	template <int I_> ap_int<I_>	—	gener- ic_divide	ap_int<I_> x, ap_int<I_> y
48	template <int I_> ap_uint<I_>	—	gener- ic_divide	ap_uint<I_> x, ap_uint<I_> y
55	template <int W_, int I_> ap_fixed<W_,I_>	—	generic_fmod	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y
87	template <int W_, int I_> ap_ufixed<W_,I_>	—	generic_fmod	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y
94	template <int I_> ap_int<I_>	—	generic_fmod	ap_int<I_> x, ap_int<I_> y
101	template <int I_> ap_uint<I_>	—	generic_fmod	ap_uint<I_> x, ap_uint<I_> y
109	template <int W_, int I_> ap_fixed<W_,I_>	—	gener- ic_remquo	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y, int* quo

Line	Template signature	Re- turns	Name	Parameters
154	template <int W_, int I_> ap_ufixed<W_, I_>	—	gener- ic_remquo	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y, int* quo
161	template <int I_> ap_int<I_>	—	gener- ic_remquo	ap_int<I_> x, ap_int<I_> y, int* quo
168	template <int I_> ap_uint<I_>	—	gener- ic_remquo	ap_uint<I_> x, ap_uint<I_> y, int* quo
176	template <int W_, int I_> ap_fixed<W_, I_>	—	gener- ic_remainder	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y
182	template <int W_, int I_> ap_ufixed<W_, I_>	—	gener- ic_remainder	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
189	template <int I_> ap_int<I_>	—	gener- ic_remainder	ap_int<I_> x, ap_int<I_> y
196	template <int I_> ap_uint<I_>	—	gener- ic_remainder	ap_uint<I_> x, ap_uint<I_> y

etc/hls_fract_apfixed.h

Table 151: Every function definition in etc/hls_fract_apfixed.h (4 definitions)

Line	Template signature	Returns	Name	Parameters
14	template <int W_, int I_> ap_fixed<W_, I_>	—	generic_fract	ap_fixed<W_, I_> x
22	template <int W_, int I_> ap_ufixed<W_, I_>	—	generic_fract	ap_ufixed<W_, I_> x
28	template <int I_> ap_int<I_>	—	generic_fract	ap_int<I_> x
34	template <int I_> ap_uint<I_>	—	generic_fract	ap_uint<I_> x

etc/hls_frexp_apfixed.h

Table 152: Every function definition in etc/hls_frexp_apfixed.h (4 definitions)

Line	Template signature	Re- turns	Name	Parameters
17	template<int W, int I> ap_fixed<W, I>	—	frexp	ap_fixed<W, I> x, ap_fixed<W, I>* exp
73	template<int W, int I> ap_ufixed<W, I>	—	frexp	ap_ufixed<W, I> x, ap_ufixed<W, I>* exp
82	template<int I> ap_uint<I>	—	frexp	ap_uint<I> x, ap_uint<I>* exp
91	template<int I> ap_int<I>	—	frexp	ap_int<I> x, ap_int<I>* exp

etc/hls_half_fpo.h

Table 153: Every class and struct in etc/hls_half_fpo.h (24 definitions)

Line	Kind	Name / specialisation	Template para- meters	Bases	Member typedefs
33	class	half	—	—	typedef unsigned short uint16;
398	struct	half_axu_func- tions	—	—	—
495	struct	half	template <>	::half_traits<char, half>	typedef half rtvl_t;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
500	struct	half	template <>	::half_traits<signed char, half>	typedef half rtvl_t;
505	struct	half	template <>	::half_traits<unsigned char, half>	typedef half rtvl_t;
510	struct	half	template <>	::half_traits<short, half>	typedef half rtvl_t;
515	struct	half	template <>	::half_traits<unsigned short, half>	typedef half rtvl_t;
520	struct	half	template <>	::half_traits<int, half>	typedef half rtvl_t;
525	struct	half	template <>	::half_traits<unsigned int, half>	typedef half rtvl_t;
530	struct	half	template <>	::half_traits<float, half>	typedef float rtvl_t;
535	struct	half	template <>	::half_traits<double, half>	typedef double rtvl_t;
540	struct	half	template <>	::half_traits<long, half>	typedef half rtvl_t;
546	struct	half	template <>	::half_traits<unsigned long, half>	typedef half rtvl_t;
552	struct	half	template <>	::half_traits<half, char>	typedef half rtvl_t;
557	struct	half	template <>	::half_traits<half, signed char>	typedef half rtvl_t;
562	struct	half	template <>	::half_traits<half, unsigned char>	typedef half rtvl_t;
567	struct	half	template <>	::half_traits<half, short>	typedef half rtvl_t;
572	struct	half	template <>	::half_traits<half, unsigned short>	typedef half rtvl_t;
577	struct	half	template <>	::half_traits<half, int>	typedef half rtvl_t;
582	struct	half	template <>	::half_traits<half, unsigned int>	typedef half rtvl_t;
587	struct	half	template <>	::half_traits<half, float>	typedef float rtvl_t;
592	struct	half	template <>	::half_traits<half, double>	typedef double rtvl_t;
597	struct	half	template <>	::half_traits<half, long>	typedef half rtvl_t;
603	struct	half	template <>	::half_traits<half, unsigned long>	typedef half rtvl_t;

Table 154: Every function definition in etc/hls_half_fpo.h (63 definitions)

Line	Template signature	Returns	Name	Parameters
56	template <typename T1, typename T2>	template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q, ap_o_mode _AP_O, int _AP_N>	half	const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>& rhs

Line	Template signature	Returns	Name	Parameters
63	template <int _AP_W, bool _AP_S>	—	half	const ap_int_base<_AP_W, _AP_S>& rhs
77	—	static bool	half_isnan	half hf
81	—	operator	float	()
135	—	void	set_bits	uint16 d
137	—	uint16	get_bits	()
290	template <typename F>	static half	math_function_1arg	F f, half x
302	template <typename F>	template <typename F> static half	binary_operator	F f, half x, half y
315	template <typename F>	template <typename F> static bool	binary_operator_compare	F f, half x, half y
327	template <typename F>	private: xip_fpo_exc_t	xip_fpo_get_data	xip_fpo_ptr op
349	template <typename F>	float	xip_fpo_half_get_flt	()
370	—	void	xip_fpo_set_data	xip_fpo_ptr op
388	—	void	xip_fpo_half_set_flt	float rhs
400	template <typename T1, type- name T2>	static half	half_plus	T1 x, T2 y
404	template <typename T1, type- name T2>	static float	half_plus	float x, half y
405	template <typename T1, type- name T2>	static float	half_plus	half x, float y
406	template <typename T1, type- name T2>	static double	half_plus	double x, half y
407	template <typename T1, type- name T2>	static double	half_plus	half x, double y

Line	Template signature	Returns	Name	Parameters
409	template <typename T1, typename T2>	template <typename T1, typename T2> static half	half_minus	T1 x, T2 y
413	template <typename T1, typename T2>	static float	half_minus	float x, half y
414	template <typename T1, typename T2>	static float	half_minus	half x, float y
415	template <typename T1, typename T2>	static double	half_minus	double x, half y
416	template <typename T1, typename T2>	static double	half_minus	half x, double y
418	template <typename T1, typename T2>	template <typename T1, typename T2> static half	half_multiplies	T1 x, T2 y
422	template <typename T1, typename T2>	static float	half_multiplies	float x, half y
423	template <typename T1, typename T2>	static float	half_multiplies	half x, float y
424	template <typename T1, typename T2>	static double	half_multiplies	double x, half y
425	template <typename T1, typename T2>	static double	half_multiplies	half x, double y
427	template <typename T1, typename T2>	template <typename T1, typename T2> static half	half_divides	T1 x, T2 y
431	template <typename T1, typename T2>	static float	half_divides	float x, half y

Line	Template signature	Returns	Name	Parameters
432	template <typename T1, type- name T2>	static float	half_divides	half x, float y
433	template <typename T1, type- name T2>	static double	half_divides	double x, half y
434	template <typename T1, type- name T2>	static double	half_divides	half x, double y
436	template <typename T1, type- name T2>	template <typename T1, typename T2> static bool	half_isequal	T1 x, T2 y
440	template <typename T1, type- name T2>	static bool	half_isequal	float x, half y
441	template <typename T1, type- name T2>	static bool	half_isequal	half x, float y
442	template <typename T1, type- name T2>	static bool	half_isequal	double x, half y
443	template <typename T1, type- name T2>	static bool	half_isequal	half x, double y
445	template <typename T1, type- name T2>	template <typename T1, typename T2> static bool	half_isnotequal	T1 x, T2 y
449	template <typename T1, type- name T2>	static bool	half_isnotequal	float x, half y
450	template <typename T1, type- name T2>	static bool	half_isnotequal	half x, float y
451	template <typename T1, type- name T2>	static bool	half_isnotequal	double x, half y

Line	Template signature	Returns	Name	Parameters
452	template <typename T1, type- name T2>	static bool	half_isnotequal	half x, double y
454	template <typename T1, type- name T2>	template <typename T1, typename T2> static bool	half_isgreater	T1 x, T2 y
458	template <typename T1, type- name T2>	static bool	half_isgreater	float x, half y
459	template <typename T1, type- name T2>	static bool	half_isgreater	half x, float y
460	template <typename T1, type- name T2>	static bool	half_isgreater	double x, half y
461	template <typename T1, type- name T2>	static bool	half_isgreater	half x, double y
463	template <typename T1, type- name T2>	template <typename T1, typename T2> static bool	half_isgreaterequal	T1 x, T2 y
467	template <typename T1, type- name T2>	static bool	half_isgreaterequal	float x, half y
468	template <typename T1, type- name T2>	static bool	half_isgreaterequal	half x, float y
469	template <typename T1, type- name T2>	static bool	half_isgreaterequal	double x, half y
470	template <typename T1, type- name T2>	static bool	half_isgreaterequal	half x, double y
472	template <typename T1, type- name T2>	template <typename T1, typename T2> static bool	half_isless	T1 x, T2 y

Line	Template signature	Returns	Name	Parameters
476	template <typename T1, type- name T2>	static bool	half_isless	float x, half y
477	template <typename T1, type- name T2>	static bool	half_isless	half x, float y
478	template <typename T1, type- name T2>	static bool	half_isless	double x, half y
479	template <typename T1, type- name T2>	static bool	half_isless	half x, double y
481	template <typename T1, type- name T2>	template <typename T1, typename T2> static bool	half_islessequal	T1 x, T2 y
485	template <typename T1, type- name T2>	static bool	half_islessequal	float x, half y
486	template <typename T1, type- name T2>	static bool	half_islessequal	half x, float y
487	template <typename T1, type- name T2>	static bool	half_islessequal	double x, half y
488	template <typename T1, type- name T2>	static bool	half_islessequal	half x, double y

etc/hls_half_x_utils.h

Table 155: Every class and struct in etc/hls_half_x_utils.h (3 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
185	struct	binary_specialized	template <typename T, typename U>	—	—
199	struct	binary_specialized <half, half>	template <>	—	—
419	struct	half_round_style	—	—	—

Table 156: Every function definition in etc/hls_half_x_utils.h (87 definitions)

Line	Template signature	Returns	Name	Parameters
84	—	inline bool	isfinite	half arg
88	—	inline bool	isinf	half arg
92	—	inline bool	isnan	half arg
96	—	inline bool	isnormal	half arg
100	—	inline bool	signbit	half arg
104	—	inline int	fpclassify	half arg
111	—	inline half	fmod	half x, half y
115	—	inline half	sinh	half arg
119	—	inline half	cosh	half arg
123	—	inline half	sin	half arg
127	—	inline half	cos	half arg
130	—	inline half	frexp	half arg, int* exp
144	—	inline half	modf	half arg, half* iptr
166	—	inline int	ilogb	half arg
178	—	inline float	fdim	float x, float y
186	template <typename T, typename U>	static float	fmin	float x, float y
192	template <typename T, typename U>	static float	fmax	float x, float y
200	template <>	static half	fmin	half x, half y
208	template <>	static half	fmax	half x, half y
218	template <>	inline half	fmax	half x, half y
221	template <>	inline float	fmax	half x, float y
224	template <>	inline float	fmax	float x, half y
227	template <>	inline float	fmax	float x, float y
231	template <>	inline half	fmin	half x, half y
234	template <>	inline float	fmin	half x, float y
237	template <>	inline float	fmin	float x, half y
240	—	inline float	fmin	float x, float y
244	—	inline half	fabs	half x
249	—	inline half	abs	half x
255	—	inline half	scalbln	half arg, long exp

Line	Template signature	Returns	Name	Parameters
287	—	inline half	ldexp	half arg, int exp
290	—	inline half	ldexp	float arg, int exp
301	template <std::float_round_style R, bool E, typename T>	T	half2int_impl	half::uint16 value
338	template <std::float_round_style R, typename T>	T	half2int	half::uint16 value
346	template <typename T>	T	half2int_up	half::uint16 value
356	template <std::float_round_style R, bool E>	half::uint16	round_half_impl	half::uint16 value
382	template <std::float_round_style R>	half::uint16	round_half	half::uint16 value
387	—	inline half::uint16	round_half_up	half::uint16 value
391	—	inline half	floor	half arg
397	—	inline half	ceil	half arg
403	—	inline half	trunc	half arg
409	—	inline half	round	half arg
415	—	inline long	lround	half arg
423	—	inline half	rint	half arg
429	—	inline long	lrint	half arg
434	—	inline long long	llround	half arg
438	—	inline long long	llrint	half arg
444	—	inline float	sqrt	half arg
448	—	inline half	sqrt	half arg
453	—	inline bool	isunordered	half x, half y
458	—	inline bool	isequal	half x, half y
462	template <typename T1, typename T2>	inline bool	isequal	T1 x, T2 y
466	template <typename T1, typename T2>	inline bool	isequal	float x, half y
469	template <typename T1, typename T2>	inline bool	isequal	half x, float y
472	template <typename T1, typename T2>	inline bool	isequal	double x, half y
475	template <typename T1, typename T2>	inline bool	isequal	half x, double y

Line	Template signature	Returns	Name	Parameters
481	template <typename T1, typename T2>	inline bool	isnotequal	half x, half y
485	template <typename T1, typename T2>	template <typename T1, typename T2> inline bool	isnotequal	T1 x, T2 y
489	template <typename T1, typename T2>	inline bool	isnotequal	float x, half y
492	template <typename T1, typename T2>	inline bool	isnotequal	half x, float y
495	template <typename T1, typename T2>	inline bool	isnotequal	double x, half y
498	template <typename T1, typename T2>	inline bool	isnotequal	half x, double y
504	template <typename T1, typename T2>	inline bool	isgreater	half x, half y
510	template <typename T1, typename T2>	template <typename T1, typename T2> inline bool	isgreater	T1 x, T2 y
514	template <typename T1, typename T2>	inline bool	isgreater	float x, half y
517	template <typename T1, typename T2>	inline bool	isgreater	half x, float y
520	template <typename T1, typename T2>	inline bool	isgreater	double x, half y
523	template <typename T1, typename T2>	inline bool	isgreater	half x, double y
529	template <typename T1, typename T2>	inline bool	isgreaterequal	half x, half y
535	template <typename T1, typename T2>	template <typename T1, typename T2> inline bool	isgreaterequal	T1 x, T2 y
539	template <typename T1, typename T2>	inline bool	isgreaterequal	float x, half y
542	template <typename T1, typename T2>	inline bool	isgreaterequal	half x, float y
545	template <typename T1, typename T2>	inline bool	isgreaterequal	double x, half y
548	template <typename T1, typename T2>	inline bool	isgreaterequal	half x, double y
554	template <typename T1, typename T2>	inline bool	isless	half x, half y
560	template <typename T1, typename T2>	template <typename T1, typename T2> inline bool	isless	T1 x, T2 y
564	template <typename T1, typename T2>	inline bool	isless	float x, half y

Line	Template signature	Returns	Name	Parameters
567	template <typename T1, typename T2>	inline bool	isless	half x, float y
570	template <typename T1, typename T2>	inline bool	isless	double x, half y
573	template <typename T1, typename T2>	inline bool	isless	half x, double y
579	template <typename T1, typename T2>	static bool	islessequal	half x, half y
585	template <typename T1, typename T2>	template <typename T1, typename T2> inline bool	islessequal	T1 x, T2 y
589	template <typename T1, typename T2>	inline bool	islessequal	float x, half y
592	template <typename T1, typename T2>	inline bool	islessequal	half x, float y
595	template <typename T1, typename T2>	inline bool	islessequal	double x, half y
598	template <typename T1, typename T2>	inline bool	islessequal	half x, double y
603	template <typename T1, typename T2>	inline bool	islessgreater	half x, half y

etc/hls_hotbm_apfixed.h

Table 157: Every class and struct in etc/hls_hotbm_apfixed.h (4 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
65	struct	value_list	template<class T>	—	—
69	struct	value_list <double>	template<>	—	typedef double T;
163	struct	value_list <float>	template<>	—	typedef float T;
241	struct	sin_cos_fixed_16	—	—	typedef half T;

Table 158: Every function definition in etc/hls_hotbm_apfixed.h (6 definitions)

Line	Template signature	Returns	Name	Parameters
82	template<>	template <int Wx, int Wr> static ap_ufixed<Wx, 1>	sin_or_cos_approximation	bool do_cos, ap_uint<3> &k, ap_ufixed<Wx, 0> &x, ap_ufixed<Wr, 0> &Mx, ap_int<fp_struct<double>::EXP_BITS> &Ex
117	template <int Wx, int Wr>	static void	sin-cos_approximation	ap_uint<3> &k, ap_ufixed<Wx, 0> &x, ap_ufixed<Wr, 0> Mx, ap_int<fp_struct<double>::EXP_BITS> Ex, ap_ufixed<Wr, 1> &sin_result, ap_ufixed<Wr, 1> &cos_result

Line	Template signature	Returns	Name	Parameters
174	template<>	template <int Wx, int Wr> static ap_ufixed<Wx, 1>	sin_or_cos_ap- proximation	bool do_cos, ap_uint<3> &k, ap_ufixed<Wx, 0> &x, ap_ufixed<Wr, 0> &Mx, ap_int<fp_struct<float>::EXP_BITS> &Ex
205	template <int Wx, int Wr>	template <int Wx, int Wr> static void	sin- cos_approxima- tion	ap_uint<3> &k, ap_ufixed<Wx, 0> &x, ap_ufixed<Wr, 0> &Mx, ap_int<fp_struct<float>::EXP_BITS> &Ex, ap_ufixed<Wr, 1> &sin_result, ap_ufixed<Wr, 1> &cos_result
249	template <int W_dout> static ap_ufixed<W_dout, 1>	—	sin_or_cos_ap- proximation	bool do_cos, ap_uint<2> &k, ap_ufixed<W_dout, 0> &x
417	template<int W, int I> ap_fixed<W, 2>	—	sinf_or_cosf	ap_fixed<W, I> t_in, bool do_cos, bool do_pi, typename hls::enable_if<W <= 16, bool>::type dummy = true

etc/hls_hypot_apfixed.h

Table 159: Every function definition in etc/hls_hypot_apfixed.h (4 definitions)

Line	Template signature	Re- turns	Name	Parameters
16	template<int W, int I> ap_fixed<W, I>	—	hy- pot_fixed	ap_fixed<W, I> x, ap_fixed<W, I> y
183	template<int W, int I> ap_ufixed<W, I>	—	hy- pot_fixed	ap_ufixed<W, I> x, ap_fixed<W, I> y
189	template<int I> ap_int<I>	—	hy- pot_fixed	ap_int<I> x, ap_int<I> y
195	template<int I> ap_uint<I>	—	hy- pot_fixed	ap_uint<I> x, ap_uint<I> y

etc/hls_isequal_isnotequal_apfixed.h

Table 160: Every function definition in etc/hls_isequal_isnotequal_apfixed.h (16 definitions)

Line	Template signature	Re- turns	Name	Parameters
14	template <int W_, int I_>	bool	generic_isequal	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y
22	template <int W_, int I_>	bool	gener- ic_isnotequal	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y
30	template <int W_, int I_>	bool	generic_isequal	ap_ufixed<W_, I_> x, ap_fixed<W_, I_> y
38	template <int W_, int I_>	bool	gener- ic_isnotequal	ap_ufixed<W_, I_> x, ap_fixed<W_, I_> y
46	template <int W_, int I_>	bool	generic_isequal	ap_fixed<W_, I_> x, ap_ufixed<W_, I_> y
54	template <int W_, int I_>	bool	gener- ic_isnotequal	ap_fixed<W_, I_> x, ap_ufixed<W_, I_> y

Line	Template signature	Re- turns	Name	Parameters
62	template <int W_, int I_>	bool	generic_isequal	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
70	template <int W_, int I_>	bool	gener- ic_isnotequal	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
79	template <int I_>	bool	generic_isequal	ap_int<I_> x, ap_int<I_> y
87	template <int I_>	bool	gener- ic_isnotequal	ap_int<I_> x, ap_int<I_> y
95	template <int I_>	bool	generic_isequal	ap_uint<I_> x, ap_int<I_> y
103	template <int I_>	bool	gener- ic_isnotequal	ap_uint<I_> x, ap_int<I_> y
111	template <int I_>	bool	generic_isequal	ap_int<I_> x, ap_uint<I_> y
119	template <int I_>	bool	gener- ic_isnotequal	ap_int<I_> x, ap_uint<I_> y
127	template <int I_>	bool	generic_isequal	ap_uint<I_> x, ap_uint<I_> y
135	template <int I_>	bool	gener- ic_isnotequal	ap_uint<I_> x, ap_uint<I_> y

etc/hls_ldexp_apfixed.h

Table 161: Every function definition in etc/hls_ldexp_apfixed.h (12 definitions)

Line	Template signature	Returns	Name	Parameters
12	template<int W, int I> ap_fixed<W,I>	—	ldexp	ap_fixed<W,I> x, int exp
23	template<int W, int I> ap_ufixed<W,I>	—	ldexp	ap_ufixed<W,I> x, int exp
29	template<int I> ap_int<I>	—	ldexp	ap_int<I> x, int exp
35	template<int I> ap_uint<I>	—	ldexp	ap_uint<I> x, int exp
41	template<int W, int I> ap_fixed<W,I>	—	scalbn	ap_fixed<W,I> x, int n
52	template<int W, int I> ap_ufixed<W,I>	—	scalbn	ap_ufixed<W,I> x, int n
58	template<int I> ap_int<I>	—	scalbn	ap_int<I> x, int n
64	template<int I> ap_uint<I>	—	scalbn	ap_uint<I> x, int n
70	template<int W, int I> ap_fixed<W,I>	—	scalbln	ap_fixed<W,I> x, long int n
81	template<int W, int I> ap_ufixed<W,I>	—	scalbln	ap_ufixed<W,I> x, long int n
87	template<int I> ap_int<I>	—	scalbln	ap_int<I> x, long int n
93	template<int I> ap_uint<I>	—	scalbln	ap_uint<I> x, long int n

etc/hls_lgamma_apfixed.h

Table 162: Every function definition in etc/hls_lgamma_apfixed.h (8 definitions)

Line	Template signature	Re- turns	Name	Parameters
16	template <int W, int I> ap_fixed<W, I>	—	lgamma_generic	ap_fixed<W, I> x_fixed
30	template <int W, int I> ap_ufixed<W, I>	—	lgamma_generic	ap_ufixed<W, I> x

Line	Template signature	Re- turns	Name	Parameters
34	template <int I_> ap_int<I_>	—	lgamma_generic	ap_int<I_> x
38	template <int I_> ap_uint<I_>	—	lgamma_generic	ap_uint<I_> x
44	template <int W, int I> ap_fixed<W, I>	—	lgamma_r_generic	ap_fixed<W, I> x, int *signgamp
49	template <int W, int I> ap_ufixed<W, I>	—	lgamma_r_generic	ap_ufixed<W, I> x, int *signgamp
54	template <int I_> ap_int<I_>	—	lgamma_r_generic	ap_int<I_> x, int *signgamp
59	template <int I_> ap_uint<I_>	—	lgamma_r_generic	ap_uint<I_> x, int *signgamp

etc/hls_log_apfixed.h

Table 163: Every class and struct in etc/hls_log_apfixed.h (48 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
16	class	log_lut_table	template <typename T, int p, int alpha, int size>	—	—
19	class	log_lut_table <ap_fixed<29, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
22	class	log_lut_table <ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
25	class	log_lut_table <ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
28	class	log_lut_table <ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
31	class	log_lut_table <ap_fixed<89, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
34	class	log_lut_table <ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
37	class	log_lut_table <ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16>	template <>	—	—
41	class	log_lut_table <ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,7,6,64>	template <>	—	—
44	class	log_lut_table <ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,7,6,64>	template <>	—	—
47	class	log_lut_table <ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,7,6,64>	template <>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
50	class	<code>log_lut_table</code> <code><ap_fixed<89, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,7,6,64></code>	<code>template <></code>	—	—
53	class	<code>log_lut_table</code> <code><ap_fixed<104, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,7,6,64></code>	<code>template <></code>	—	—
56	class	<code>log_lut_table</code> <code><ap_fixed<119, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,7,6,64></code>	<code>template <></code>	—	—
59	class	<code>log_lut_table</code> <code><ap_fixed<59, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,12,6,64></code>	<code>template <></code>	—	—
62	class	<code>log_lut_table</code> <code><ap_fixed<74, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,12,6,64></code>	<code>template <></code>	—	—
65	class	<code>log_lut_table</code> <code><ap_fixed<89, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,12,6,64></code>	<code>template <></code>	—	—
68	class	<code>log_lut_table</code> <code><ap_fixed<104, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,12,6,64></code>	<code>template <></code>	—	—
71	class	<code>log_lut_table</code> <code><ap_fixed<119, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,12,6,64></code>	<code>template <></code>	—	—
75	class	<code>log_lut_table</code> <code><ap_fixed<74, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,17,6,64></code>	<code>template <></code>	—	—
78	class	<code>log_lut_table</code> <code><ap_fixed<89, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,17,6,64></code>	<code>template <></code>	—	—
81	class	<code>log_lut_table</code> <code><ap_fixed<104, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,17,6,64></code>	<code>template <></code>	—	—
84	class	<code>log_lut_table</code> <code><ap_fixed<119, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,17,6,64></code>	<code>template <></code>	—	—
88	class	<code>log_lut_table</code> <code><ap_fixed<89, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,22,6,64></code>	<code>template <></code>	—	—
91	class	<code>log_lut_table</code> <code><ap_fixed<104, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,22,6,64></code>	<code>template <></code>	—	—
94	class	<code>log_lut_table</code> <code><ap_fixed<119, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,22,6,64></code>	<code>template <></code>	—	—
98	class	<code>log_lut_table</code> <code><ap_fixed<104, 7, (ap_q_mode)5,</code> <code>(ap_o_mode)3, 0>,27,6,64></code>	<code>template <></code>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
101	class	log_lut_table <ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,27,6,64>	template <>	—	—
105	class	log_lut_table <ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,32,6,64>	template <>	—	—
109	class	log0_lut_table	template <typename T, int p, int alpha, int size>	—	—
112	class	log0_lut_table <ap_fixed<20, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
116	class	log0_lut_table <ap_fixed<29, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
119	class	log0_lut_table <ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
122	class	log0_lut_table <ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
125	class	log0_lut_table <ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
128	class	log0_lut_table <ap_fixed<89, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
131	class	log0_lut_table <ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
134	class	log0_lut_table <ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64>	template <>	—	—
138	class	log_inverse_lut_table	template <>	—	—
186	class	log_traits	template <int W_s_>	—	—
189	class	log_traits <1>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;
208	class	log_traits <2>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;
234	class	log_traits <3>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;
264	class	log_traits <4>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
298	class	log_traits <5>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;
336	class	log_traits <6>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;
378	class	log_traits <7>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;
424	class	log_traits <8>	template <>	—	typedef ap_fixed<1 + we + MaxPrecision, 1 + we > LOG_TYPE;

Table 164: Every function definition in etc/hls_log_apfixed.h (24 definitions)

Line	Template signature	Returns	Name	Parameters
472	template <int size>	template<int W_, int I_> ap_fixed<W_, I_>	log	ap_fixed<W_, I_> x
770	template<int W_, int I_> ap_ufixed<W_, I_>	—	log	ap_ufixed<W_, I_> x
776	template<int I_> ap_int<I_>	—	log	ap_int<I_> x
782	template<int I_> ap_uint<I_>	—	log	ap_uint<I_> x
789	template<int W_, int I_> ap_fixed<W_, I_>	—	log10	ap_fixed<W_, I_> x
987	template<int W_, int I_> ap_ufixed<W_, I_>	—	log10	ap_ufixed<W_, I_> x
993	template<int I_> ap_int<I_>	—	log10	ap_int<I_> x
999	template<int I_> ap_uint<I_>	—	log10	ap_uint<I_> x
1005	template<int W, int I> ap_fixed<W, I>	—	log1p	ap_fixed<W, I> x
1012	template<int W, int I> ap_ufixed<W, I>	—	log1p	ap_ufixed<W, I> x
1018	template<int I> ap_int<I>	—	log1p	ap_int<I> x
1024	template<int I> ap_uint<I>	—	log1p	ap_uint<I> x
1030	template<int W, int I> ap_fixed<W, I>	—	ilogb	ap_fixed<W, I> x
1046	template<int W, int I> ap_ufixed<W, I>	—	ilogb	ap_ufixed<W, I> x
1052	template<int I> ap_int<I>	—	ilogb	ap_int<I> x
1058	template<int I> ap_uint<I>	—	ilogb	ap_uint<I> x

Line	Template signature	Returns	Name	Parameters
1065	template<int W_, int I_> ap_fixed<W_, I_>	—	log2	ap_fixed<W_, I_> x
1404	template<int W_, int I_> ap_ufixed<W_, I_>	—	log2	ap_ufixed<W_, I_> x
1410	template<int I_> ap_int<I_>	—	log2	ap_int<I_> x
1416	template<int I_> ap_uint<I_>	—	log2	ap_uint<I_> x
1422	template<int W_, int I_> ap_fixed<W_, I_>	—	logb	ap_fixed<W_, I_> x
1427	template<int W_, int I_> ap_ufixed<W_, I_>	—	logb	ap_ufixed<W_, I_> x
1433	template<int I_> ap_int<I_>	—	logb	ap_int<I_> x
1439	template<int I_> ap_uint<I_>	—	logb	ap_uint<I_> x

etc/hls_mad_apfixed.h

Table 165: Every function definition in etc/hls_mad_apfixed.h (4 definitions)

Line	Template signature	Re- turns	Name	Parameters
10	template <int W_, int I_> ap_fixed<W_, I_>	—	gener- ic_mad	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y, ap_fixed<W_, I_> z
37	template <int W_, int I_> ap_ufixed<W_, I_>	—	gener- ic_mad	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y, ap_ufixed<W_, I_> z
45	template <int I_> ap_int<I_>	—	gener- ic_mad	ap_int<I_> x, ap_int<I_> y, ap_int<I_> z
53	template <int I_> ap_uint<I_>	—	gener- ic_mad	ap_uint<I_> x, ap_uint<I_> y, ap_uint<I_> z

etc/hls_modf_apfixed.h

Table 166: Every function definition in etc/hls_modf_apfixed.h (4 definitions)

Line	Template signature	Re- turns	Name	Parameters
15	template<int W, int I> ap_fixed<W, I>	—	modf	ap_fixed<W, I> x, ap_fixed<W, I>* int_part
35	template<int W, int I> ap_ufixed<W, I>	—	modf	ap_ufixed<W, I> x, ap_ufixed<W, I>* int_part
44	template<int I> ap_int<I>	—	modf	ap_int<I> x, ap_int<I>* int_part
50	template<int I> ap_uint<I>	—	modf	ap_uint<I> x, ap_uint<I>* int_part

etc/hls_pow_apfixed.h

Table 167: Every function definition in etc/hls_pow_apfixed.h (16 definitions)

Line	Template signature	Re- turns	Name	Parameters
26	template<int W_, int I_> ap_fixed<W_, I_>	—	pow	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y

Line	Template signature	Re- turns	Name	Parameters
184	template<int W_, int I_> ap_fixed<W_, I_>	—	pown	ap_fixed<W_, I_> x, int n
354	template<int W_, int I_> ap_fixed<W_, I_>	—	rootn	ap_fixed<W_, I_> x, int n
458	template<int W_, int I_> ap_ufixed<W_, I_>	—	pow	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
465	template<int I_> ap_int<I_>	—	pow	ap_int<I_> x, ap_int<I_> y
472	template<int I_> ap_uint<I_>	—	pow	ap_uint<I_> x, ap_uint<I_> y
480	template<int W_, int I_> ap_fixed<W_, I_>	—	powr	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y
485	template<int W_, int I_> ap_ufixed<W_, I_>	—	powr	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
492	template<int I_> ap_int<I_>	—	powr	ap_int<I_> x, ap_int<I_> y
499	template<int I_> ap_uint<I_>	—	powr	ap_uint<I_> x, ap_uint<I_> y
506	template<int W_, int I_> ap_ufixed<W_, I_>	—	pown	ap_ufixed<W_, I_> x, int n
512	template<int I_> ap_int<I_>	—	pown	ap_int<I_> x, int n
518	template<int I_> ap_uint<I_>	—	pown	ap_uint<I_> x, int n
523	template<int W_, int I_> ap_ufixed<W_, I_>	—	rootn	ap_ufixed<W_, I_> x, int n
529	template<int I_> ap_int<I_>	—	rootn	ap_int<I_> x, int n
535	template<int I_> ap_uint<I_>	—	rootn	ap_uint<I_> x, int n

etc/hls_round_copysign_apfixed.h

Table 168: Every function definition in etc/hls_round_copysign_apfixed.h (60 definitions)

Line	Template signature	Returns	Name	Parameters
11	template <int W_, int I_> ap_fixed<W_, I_>	—	ceil_fixed	ap_fixed<W_, I_> x
28	template <int W_, int I_> ap_ufixed<W_, I_>	—	ceil_fixed	ap_ufixed<W_, I_> x
46	template <int I_> ap_int<I_>	—	ceil_fixed	ap_int<I_> x
51	template <int I_> ap_uint<I_>	—	ceil_fixed	ap_uint<I_> x
57	template <int W_, int I_> ap_fixed<W_, I_>	—	floor_fixed	ap_fixed<W_, I_> x
71	template <int W_, int I_> ap_ufixed<W_, I_>	—	floor_fixed	ap_ufixed<W_, I_> x
77	template <int I_> ap_int<I_>	—	floor_fixed	ap_int<I_> x
82	template <int I_> ap_uint<I_>	—	floor_fixed	ap_uint<I_> x
88	template <int W_, int I_> ap_fixed<W_, I_>	—	trunc_fixed	ap_fixed<W_, I_> x
107	template <int W_, int I_> ap_ufixed<W_, I_>	—	trunc_fixed	ap_ufixed<W_, I_> x

Line	Template signature	Returns	Name	Parameters
113	template <int I_> ap_int<I_>	—	trunc_fixed	ap_int<I_> x
118	template <int I_> ap_uint<I_>	—	trunc_fixed	ap_uint<I_> x
124	template <int W_, int I_> ap_fixed<W_, I_>	—	copysign_fixed	ap_fixed<W_, I_> x, ap_fixed<W_, I_> y
130	template <int W_, int I_> ap_ufixed<W_, I_>	—	copysign_fixed	ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y
136	template <int I_> ap_int<I_>	—	copysign_fixed	ap_int<I_> x, ap_int<I_> y
142	template <int I_> ap_uint<I_>	—	copysign_fixed	ap_uint<I_> x, ap_uint<I_> y
149	template <int W_, int I_> ap_fixed<W_, I_>	—	fabs_fixed	ap_fixed<W_, I_> x
156	template <int W_, int I_> ap_ufixed<W_, I_>	—	fabs_fixed	ap_ufixed<W_, I_> x
161	template <int I_> ap_int<I_>	—	fabs_fixed	ap_int<I_> x
168	template <int I_> ap_uint<I_>	—	fabs_fixed	ap_uint<I_> x
174	template <int W_, int I_> ap_fixed<W_, I_>	—	round_fixed	ap_fixed<W_, I_> x
193	template <int W_, int I_> ap_ufixed<W_, I_>	—	round_fixed	ap_ufixed<W_, I_> x
199	template <int I_> ap_int<I_>	—	round_fixed	ap_int<I_> x
204	template <int I_> ap_uint<I_>	—	round_fixed	ap_uint<I_> x
210	template <int W_, int I_> ap_fixed<W_, I_>	—	rint_fixed	ap_fixed<W_, I_> x
265	template <int W_, int I_> ap_ufixed<W_, I_>	—	rint_fixed	ap_ufixed<W_, I_> x
271	template <int I_> ap_int<I_>	—	rint_fixed	ap_int<I_> x
276	template <int I_> ap_uint<I_>	—	rint_fixed	ap_uint<I_> x
282	template <int W_, int I_> ap_fixed<W_, I_>	—	nearby- int_fixed	ap_fixed<W_, I_> x
288	template <int W_, int I_> ap_ufixed<W_, I_>	—	nearby- int_fixed	ap_ufixed<W_, I_> x
294	template <int I_> ap_int<I_>	—	nearby- int_fixed	ap_int<I_> x
299	template <int I_> ap_uint<I_>	—	nearby- int_fixed	ap_uint<I_> x
305	template <int W_, int I_>	bool	signbit_fixed	ap_fixed<W_, I_> x
310	template <int W_, int I_>	bool	signbit_fixed	ap_ufixed<W_, I_> x
316	template <int I_>	bool	signbit_fixed	ap_int<I_> x
322	template <int I_>	bool	signbit_fixed	ap_uint<I_> x
328	template<int W, int I> ap_fixed<W, I>	—	nextafter	ap_fixed<W, I> x, ap_fixed<W, I> y
339	template<int W, int I> ap_ufixed<W, I>	—	nextafter	ap_ufixed<W, I> x, ap_ufixed<W, I> y

Line	Template signature	Returns	Name	Parameters
346	template<int I> ap_int<I>	—	nextafter	ap_int<I> x, ap_int<I> y
352	template<int I> ap_uint<I>	—	nextafter	ap_uint<I> x, ap_uint<I> y
358	template<int W, int I> ap_fixed<W, I>	—	nexttoward	ap_fixed<W, I> x, ap_fixed<W, I> y
362	template<int W, int I> ap_ufixed<W, I>	—	nexttoward	ap_ufixed<W, I> x, ap_ufixed<W, I> y
366	template<int I> ap_int<I>	—	nexttoward	ap_int<I> x, ap_int<I> y
370	template<int I> ap_uint<I>	—	nexttoward	ap_uint<I> x, ap_uint<I> y
377	template <int W, int I>	long long int	llround_fixed	ap_fixed<W,I> x
383	template <int W, int I>	long long int	llround_fixed	ap_ufixed<W,I> x
389	template <int I>	long long int	llround_fixed	ap_int<I> x
395	template <int I>	long long int	llround_fixed	ap_uint<I> x
401	template <int W, int I>	long int	lround_fixed	ap_fixed<W,I> x
416	template <int W, int I>	long int	lround_fixed	ap_ufixed<W,I> x
429	template <int I>	long int	lround_fixed	ap_int<I> x
444	template <int I>	long int	lround_fixed	ap_uint<I> x
457	template <int W, int I>	long long int	llrint_fixed	ap_fixed<W,I> x
463	template <int W, int I>	long long int	llrint_fixed	ap_ufixed<W,I> x
469	template <int I>	long long int	llrint_fixed	ap_int<I> x
475	template <int I>	long long int	llrint_fixed	ap_uint<I> x
481	template <int W, int I>	long int	lrint_fixed	ap_fixed<W,I> x
497	template <int W, int I>	long int	lrint_fixed	ap_ufixed<W,I> x
510	template <int I>	long int	lrint_fixed	ap_int<I> x
525	template <int I>	long int	lrint_fixed	ap_uint<I> x

etc/hls_select_bitselect_apfixed.h

Table 169: Every function definition in etc/hls_select_bitselect_apfixed.h (4 definitions)

Line	Template signature	Re- turns	Name	Parameters
36	template <int W_, int I_> ap_fixed<W_,I_>	—	gener- ic_bitselect	ap_fixed<W_,I_> x, ap_fixed<W_,I_> y, ap_fixed<W_,I_> z
44	template <int W_, int I_> ap_ufixed<W_,I_>	—	gener- ic_bitselect	ap_ufixed<W_,I_> x, ap_ufixed<W_,I_> y, ap_ufixed<W_,I_> z
52	template <int I_> ap_int<I_>	—	gener- ic_bitselect	ap_int<I_> x, ap_int<I_> y, ap_int<I_> z

Line	Template signature	Re- turns	Name	Parameters
59	template <int I> ap_uint<I>	—	generic_bitselect	ap_uint<I> x, ap_uint<I> y, ap_uint<I> z

etc/hls_sqrt_apfixed.h

Table 170: Every function definition in etc/hls_sqrt_apfixed.h (12 definitions)

Line	Template signature	Returns	Name	Parameters
18	template <int W_, int I> ap_fixed<W_, I>	—	sqrt_fixed	ap_fixed<W_, I> x
350	template<int W_, int I> ap_ufixed<W_, I>	—	sqrt_fixed	ap_ufixed<W_, I> x
355	template<int I> ap_int<I>	—	sqrt_fixed	ap_int<I> x
360	template<int I> ap_uint<I>	—	sqrt_fixed	ap_uint<I> x
367	template <int W_, int I> ap_fixed<W_, I>	—	rsqrt_fixed	ap_fixed<W_, I> x
397	template<int W_, int I> ap_ufixed<W_, I>	—	rsqrt_fixed	ap_ufixed<W_, I> x
402	template<int I> ap_int<I>	—	rsqrt_fixed	ap_int<I> x
407	template<int I> ap_uint<I>	—	rsqrt_fixed	ap_uint<I> x
414	template<int W_, int I> ap_fixed<W_, I>	—	recip_fixed	ap_fixed<W_, I> x
422	template<int W_, int I> ap_ufixed<W_, I>	—	recip_fixed	ap_ufixed<W_, I> x
427	template<int I> ap_int<I>	—	recip_fixed	ap_int<I> x
432	template<int I> ap_uint<I>	—	recip_fixed	ap_uint<I> x

etc/hls_tgamma_apfixed.h

Table 171: Every function definition in etc/hls_tgamma_apfixed.h (4 definitions)

Line	Template signature	Returns	Name	Parameters
17	template <int W, int I> ap_fixed<W, I>	—	tgamma_generic	ap_fixed<W, I> x_fixed
31	template <int W, int I> ap_ufixed<W, I>	—	tgamma_generic	ap_ufixed<W, I> x
35	template <int I> ap_int<I>	—	tgamma_generic	ap_int<I> x
39	template <int I> ap_uint<I>	—	tgamma_generic	ap_uint<I> x

gmp.h

Table 172: Every function definition in gmp.h (8 definitions)

Line	Template signature	Returns	Name	Parameters
1799	—	__GMP_EX- TERN_INLINE void	mpz_abs	mpz_ptr __gmp_w, mpz_srcptr __gmp_u
1897	—	__GMP_EX- TERN_INLINE void	mpz_neg	mpz_ptr __gmp_w, mpz_srcptr __gmp_u
1910	—	int	mpz_perfect_square_p	mpz_srcptr __gmp_a
1946	—	void	mpz_set_q	mpz_ptr __gmp_w, mpq_srcptr __gmp_u

Line	Template signature	Returns	Name	Parameters
1968	—	__G- MP_EX- TERN_INLINE void	mpq_abs	mpq_ptr __gmp_w, mpq_srcptr __gmp_u
1978	—	__G- MP_EX- TERN_INLINE void	mpq_neg	mpq_ptr __gmp_w, mpq_srcptr __gmp_u
2222	—	mp_limb_t	mpn_add	mp_ptr __gmp_wp, mp_srcptr __gmp_xp, mp_size_t __gmp_xsize, mp_srcptr __gmp_yp, mp_size_t __gmp_ysize
2261	—	mp_limb_t	mpn_sub	mp_ptr __gmp_wp, mp_srcptr __gmp_xp, mp_size_t __gmp_xsize, mp_srcptr __gmp_yp, mp_size_t __gmp_ysize

hls_fpo.h

Table 173: Every function definition in hls_fpo.h (44 definitions)

Line	Template signature	Returns	Name	Parameters
148	—	inline float	xil_fpo_add_flt	float a, float b
157	—	inline double	xil_fpo_add_d	double a, double b
170	—	inline float	xil_fpo_sub_flt	float a, float b
179	—	inline double	xil_fpo_sub_d	double a, double b
192	—	inline float	xil_fpo_mul_flt	float a, float b
201	—	inline double	xil_fpo_mul_d	double a, double b
214	—	inline float	xil_fpo_div_flt	float a, float b
223	—	inline double	xil_fpo_div_d	double a, double b
236	—	inline float	xil_fpo_rec_flt	float a
245	—	inline double	xil_fpo_rec_d	double a
258	—	inline float	xil_fpo_sqrt_flt	float a
269	—	inline double	xil_fpo_sqrt_d	double a
283	—	inline float	xil_fpo_recsqrt_flt	float a
292	—	inline double	xil_fpo_recsqrt_d	double a
305	—	inline float	xil_fpo_abs_flt	float a
313	—	inline double	xil_fpo_abs_d	double a
325	—	inline float	xil_fpo_log_flt	float a
334	—	inline double	xil_fpo_log_d	double a
347	—	inline float	xil_fpo_exp_flt	float a
356	—	inline double	xil_fpo_exp_d	double a
369	—	inline int	xil_fpo_unordered_flt	float a, float b
378	—	inline int	xil_fpo_unordered_d	double a, double b
392	—	inline int	xil_fpo_equal_flt	float a, float b

Line	Template signature	Returns	Name	Parameters
401	—	inline int	xil_fpo_equal_d	double a, double b
414	—	inline int	xil_fpo_less_flt	float a, float b
423	—	inline int	xil_fpo_less_d	double a, double b
436	—	inline int	xil_fpo_lessequal_flt	float a, float b
445	—	inline int	xil_fpo_lessequal_d	double a, double b
458	—	inline int	xil_fpo_greater_flt	float a, float b
467	—	inline int	xil_fpo_greater_d	double a, double b
480	—	inline int	xil_fpo_greaterequal_flt	float a, float b
489	—	inline int	xil_fpo_greaterequal_d	double a, double b
502	—	inline int	xil_fpo_notequal_flt	float a, float b
511	—	inline int	xil_fpo_notequal_d	double a, double b
523	—	inline int	xil_fpo_condcode_flt	float a, float b
532	—	inline int	xil_fpo_condcode_d	double a, double b
544	—	inline int	xil_fpo_flttofix_int_flt	float a
553	—	inline int	xil_fpo_flttofix_int_d	double a
562	—	inline float	xil_fpo_fixtoflt_flt_int	int a
571	—	inline double	xil_fpo_fixtoflt_d_int	int a
580	—	inline float	xil_fpo_flttoflt_flt_flt	float a
589	—	inline float	xil_fpo_flttoflt_flt_d	double a
598	—	inline double	xil_fpo_flttoflt_d_flt	float a
607	—	inline double	xil_fpo_flttoflt_d_d	double a

hls_half.h

Table 174: Every class and struct in hls_half.h (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
40	struct	numeric_limits <half>	template <>	: public numeric_limits<float>	—

hls_math.h

Table 175: Every function definition in hls_math.h (339 definitions)

Line	Template signature	Returns	Name	Parameters
239	template <class T>	T	max	T x, T y
245	template <class T>	T	min	T x, T y
1100	template<int W, int I> ap_fixed<W, 2>	—	sinpi	ap_fixed<W, I> x
1108	template<int W, int I> ap_ufixed<W, 2>	—	sinpi	ap_ufixed<W, I> x
1119	template<int W, int I> ap_fixed<W, 2>	—	cospi	ap_fixed<W, I> x

Line	Template signature	Returns	Name	Parameters
1123	template<int W, int I> ap_ufixed<W, 2>	—	cospi	ap_ufixed<W, I> x
1138	template<int I> ap_int<I>	—	sinpi	ap_int<I> in
1144	template<int I> ap_int<I>	—	cospi	ap_int<I> in
1150	template<int I> ap_uint<I>	—	sinpi	ap_uint<I> in
1156	template<int I> ap_uint<I>	—	cospi	ap_uint<I> in
1176	template<int W, int I> ap_fixed<W, I>	—	exp	ap_fixed<W, I> x
1180	template<int W, int I> ap_ufixed<W, I>	—	exp	ap_ufixed<W, I> x
1184	template<int I> ap_int<I>	—	exp	ap_int<I> x
1188	template<int I> ap_uint<I>	—	exp	ap_uint<I> x
1194	template<int W, int I> ap_fixed<W, I>	—	log	ap_fixed<W, I> x
1198	template<int W, int I> ap_ufixed<W, I>	—	log	ap_ufixed<W, I> x
1202	template<int I> ap_int<I>	—	log	ap_int<I> x
1206	template<int I> ap_uint<I>	—	log	ap_uint<I> x
1212	template<int W, int I> ap_fixed<W, I>	—	log10	ap_fixed<W, I> x
1216	template<int W, int I> ap_ufixed<W, I>	—	log10	ap_ufixed<W, I> x
1220	template<int I> ap_int<I>	—	log10	ap_int<I> x
1224	template<int I> ap_uint<I>	—	log10	ap_uint<I> x
1230	template<int W, int I> ap_fixed<W, I>	—	log2	ap_fixed<W, I> x
1234	template<int W, int I> ap_ufixed<W, I>	—	log2	ap_ufixed<W, I> x
1238	template<int I> ap_int<I>	—	log2	ap_int<I> x
1242	template<int I> ap_uint<I>	—	log2	ap_uint<I> x
1248	template<int W, int I> ap_fixed<W, I>	—	logb	ap_fixed<W, I> x

Line	Template signature	Returns	Name	Parameters
1252	template<int W, int I> ap_ufixed<W,I>	—	logb	ap_ufixed<W,I> x
1256	template<int I> ap_int<I>	—	logb	ap_int<I> x
1260	template<int I> ap_uint<I>	—	logb	ap_uint<I> x
1266	template<int W, int I> ap_fixed<W,I>	—	sqrt	ap_fixed<W,I> x
1270	template<int W, int I> ap_ufixed<W,I>	—	sqrt	ap_ufixed<W,I> x
1274	template<int I> ap_int<I>	—	sqrt	ap_int<I> x
1278	template<int I> ap_uint<I>	—	sqrt	ap_uint<I> x
1284	template<int W, int I> ap_fixed<W,I>	—	rsqrt	ap_fixed<W,I> x
1288	template<int W, int I> ap_ufixed<W,I>	—	rsqrt	ap_ufixed<W,I> x
1292	template<int I> ap_int<I>	—	rsqrt	ap_int<I> x
1296	template<int I> ap_uint<I>	—	rsqrt	ap_uint<I> x
1302	template<int W, int I> ap_fixed<W,I>	—	recip	ap_fixed<W,I> x
1306	template<int W, int I> ap_ufixed<W,I>	—	recip	ap_ufixed<W,I> x
1310	template<int I> ap_int<I>	—	recip	ap_int<I> x
1314	template<int I> ap_uint<I>	—	recip	ap_uint<I> x
1320	template<int W, int I> ap_fixed<W,I>	—	ceil	ap_fixed<W,I> x
1324	template<int W, int I> ap_ufixed<W,I>	—	ceil	ap_ufixed<W,I> x
1328	template<int I> ap_int<I>	—	ceil	ap_int<I> x
1332	template<int I> ap_uint<I>	—	ceil	ap_uint<I> x
1338	template<int W, int I> ap_fixed<W,I>	—	floor	ap_fixed<W,I> x
1342	template<int W, int I> ap_ufixed<W,I>	—	floor	ap_ufixed<W,I> x
1346	template<int I> ap_int<I>	—	floor	ap_int<I> x

Line	Template signature	Returns	Name	Parameters
1350	template<int I> ap_uint<I>	—	floor	ap_uint<I> x
1355	template<int W, int I> ap_fixed<W,I>	—	trunc	ap_fixed<W,I> x
1359	template<int W, int I> ap_ufixed<W,I>	—	trunc	ap_ufixed<W,I> x
1363	template<int I> ap_int<I>	—	trunc	ap_int<I> x
1367	template<int I> ap_uint<I>	—	trunc	ap_uint<I> x
1373	template<int W, int I> ap_fixed<W,I>	—	copysign	ap_fixed<W,I> x, ap_fixed<W,I> y
1377	template<int W, int I> ap_ufixed<W,I>	—	copysign	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1381	template<int I> ap_int<I>	—	copysign	ap_int<I> x, ap_int<I> y
1385	template<int I> ap_uint<I>	—	copysign	ap_uint<I> x, ap_uint<I> y
1391	template<int W, int I> ap_fixed<W,I>	—	fabs	ap_fixed<W,I> x
1395	template<int W, int I> ap_ufixed<W,I>	—	fabs	ap_ufixed<W,I> x
1399	template<int I> ap_int<I>	—	fabs	ap_int<I> x
1403	template<int I> ap_uint<I>	—	fabs	ap_uint<I> x
1408	template<int W, int I> ap_fixed<W,I>	—	abs	ap_fixed<W,I> x
1412	template<int W, int I> ap_ufixed<W,I>	—	abs	ap_ufixed<W,I> x
1416	template<int I> ap_int<I>	—	abs	ap_int<I> x
1420	template<int I> ap_uint<I>	—	abs	ap_uint<I> x
1426	template<int W, int I> ap_fixed<W,I>	—	fdim	ap_fixed<W,I> x, ap_fixed<W,I> y
1430	template<int W, int I> ap_ufixed<W,I>	—	fdim	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1434	template<int I> ap_int<I>	—	fdim	ap_int<I> x, ap_int<I> y
1438	template<int I> ap_uint<I>	—	fdim	ap_uint<I> x, ap_uint<I> y
1443	template<int W, int I> ap_fixed<W,I>	—	fmax	ap_fixed<W,I> x, ap_fixed<W,I> y

Line	Template signature	Returns	Name	Parameters
1447	template<int W, int I> ap_ufixed<W,I>	—	fmax	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1451	template<int I> ap_int<I>	—	fmax	ap_int<I> x, ap_int<I> y
1455	template<int I> ap_uint<I>	—	fmax	ap_uint<I> x, ap_uint<I> y
1460	template<int W, int I> ap_fixed<W,I>	—	fmin	ap_fixed<W,I> x, ap_fixed<W,I> y
1464	template<int W, int I> ap_ufixed<W,I>	—	fmin	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1468	template<int I> ap_int<I>	—	fmin	ap_int<I> x, ap_int<I> y
1472	template<int I> ap_uint<I>	—	fmin	ap_uint<I> x, ap_uint<I> y
1477	template<int W, int I> ap_fixed<W,I>	—	maxmag	ap_fixed<W,I> x, ap_fixed<W,I> y
1481	template<int W, int I> ap_ufixed<W,I>	—	maxmag	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1485	template<int I> ap_int<I>	—	maxmag	ap_int<I> x, ap_int<I> y
1489	template<int I> ap_uint<I>	—	maxmag	ap_uint<I> x, ap_uint<I> y
1494	template<int W, int I> ap_fixed<W,I>	—	minmag	ap_fixed<W,I> x, ap_fixed<W,I> y
1498	template<int W, int I> ap_ufixed<W,I>	—	minmag	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1502	template<int I> ap_int<I>	—	minmag	ap_int<I> x, ap_int<I> y
1506	template<int I> ap_uint<I>	—	minmag	ap_uint<I> x, ap_uint<I> y
1513	template<int W, int I> ap_fixed<W,I>	—	sinh	ap_fixed<W,I> x
1519	template<int W, int I> ap_ufixed<W,I>	—	sinh	ap_ufixed<W,I> x
1525	template<int I> ap_int<I>	—	sinh	ap_int<I> x
1531	template<int I> ap_uint<I>	—	sinh	ap_uint<I> x
1537	template<int W, int I> ap_fixed<W,I>	—	cosh	ap_fixed<W,I> x
1543	template<int W, int I> ap_ufixed<W,I>	—	cosh	ap_ufixed<W,I> x
1549	template<int I> ap_int<I>	—	cosh	ap_int<I> x

Line	Template signature	Returns	Name	Parameters
1555	template<int I> ap_uint<I>	—	cosh	ap_uint<I> x
1561	template<int W, int I> ap_fixed<W,I>	—	tanh	ap_fixed<W,I> x
1567	template<int W, int I> ap_ufixed<W,I>	—	tanh	ap_ufixed<W,I> x
1573	template<int I> ap_int<I>	—	tanh	ap_int<I> x
1579	template<int I> ap_uint<I>	—	tanh	ap_uint<I> x
1585	template<int W, int I> ap_fixed<W,I>	—	atanh	ap_fixed<W,I> x
1596	template<int W, int I> ap_ufixed<W,I>	—	atanh	ap_ufixed<W,I> x
1604	template<int I> ap_int<I>	—	atanh	ap_int<I> x
1612	template<int I> ap_uint<I>	—	atanh	ap_uint<I> x
1620	template<int W, int I> ap_fixed<W,I>	—	asinh	ap_fixed<W,I> x
1626	template<int W, int I> ap_ufixed<W,I>	—	asinh	ap_ufixed<W,I> x
1632	template<int I> ap_int<I>	—	asinh	ap_int<I> x
1638	template<int I> ap_uint<I>	—	asinh	ap_uint<I> x
1644	template<int W, int I> ap_fixed<W,I>	—	acosh	ap_fixed<W,I> x
1650	template<int W, int I> ap_ufixed<W,I>	—	acosh	ap_ufixed<W,I> x
1656	template<int I> ap_int<I>	—	acosh	ap_int<I> x
1662	template<int I> ap_uint<I>	—	acosh	ap_uint<I> x
1722	template<int W, int I> ap_fixed<W,2>	—	atan	ap_fixed<W,I> x
1728	template<int W, int I> ap_ufixed<W,2>	—	atan	ap_ufixed<W,I> x
1734	template<int I> ap_int<2>	—	atan	ap_int<I> x
1740	template<int I> ap_uint<1>	—	atan	ap_uint<I> x
1767	template<int W, int I> ap_fixed<W,I>	—	asinpi	ap_fixed<W,I> x

Line	Template signature	Returns	Name	Parameters
1772	template<int W, int I> ap_ufixed<W,I>	—	asinpi	ap_ufixed<W,I> x
1777	template<int I> ap_int<I>	—	asinpi	ap_int<I> x
1782	template<int I> ap_uint<I>	—	asinpi	ap_uint<I> x
1796	template<int W, int I> ap_fixed<W,I>	—	acospi	ap_fixed<W,I> x
1801	template<int W, int I> ap_ufixed<W,I>	—	acospi	ap_ufixed<W,I> x
1806	template<int I> ap_int<I>	—	acospi	ap_int<I> x
1811	template<int I> ap_uint<I>	—	acospi	ap_uint<I> x
1824	template<int W, int I> ap_fixed<W,I>	—	atanpi	ap_fixed<W,I> x
1830	template<int W, int I> ap_ufixed<W,I>	—	atanpi	ap_ufixed<W,I> x
1836	template<int I> ap_int<2>	—	atanpi	ap_int<I> x
1842	template<int I> ap_uint<1>	—	atanpi	ap_uint<I> x
1855	template<int W, int I> ap_fixed<W,3>	—	atan2	ap_fixed<W,I> x, ap_fixed<W,I> y
1861	template<int W, int I> ap_ufixed<W,3>	—	atan2	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1867	template<int I> ap_int<3>	—	atan2	ap_int<I> x, ap_int<I> y
1873	template<int I> ap_uint<2>	—	atan2	ap_uint<I> x, ap_uint<I> y
1886	template<int W, int I> ap_fixed<W,I>	—	atan2pi	ap_fixed<W,I> x, ap_fixed<W,I> y
1892	template<int W, int I> ap_ufixed<W,I>	—	atan2pi	ap_ufixed<W,I> x, ap_ufixed<W,I> y
1898	template<int I> ap_int<I>	—	atan2pi	ap_int<I> x, ap_int<I> y
1904	template<int I> ap_uint<I>	—	atan2pi	ap_uint<I> x, ap_uint<I> y
1915	template<int W, int I>	void	sincos	ap_fixed<W,I> in, ap_fixed<W-I+2,2>* outsin, ap_fixed<W-I+2,2>* outcos
1976	template<int I> ap_int<I>	—	sin	ap_int<I> in
1981	template<int I> ap_int<I>	—	cos	ap_int<I> in

Line	Template signature	Returns	Name	Parameters
1988	template<int I> ap_uint<I>	—	sin	ap_uint<I> in
1993	template<int I> ap_uint<I>	—	cos	ap_uint<I> in
2021	template<int W, int I> ap_fixed<W,I>	—	tan	ap_fixed<W,I> x
2025	template<int W, int I> ap_ufixed<W,I>	—	tan	ap_ufixed<W,I> x
2029	template<int I> ap_int<I>	—	tan	ap_int<I> x
2033	template<int I> ap_uint<I>	—	tan	ap_uint<I> x
2046	template<int W, int I> ap_fixed<W,I>	—	tanpi	ap_fixed<W,I> x
2050	template<int W, int I> ap_ufixed<W,I>	—	tanpi	ap_ufixed<W,I> x
2054	template<int I> ap_int<I>	—	tanpi	ap_int<I> x
2058	template<int I> ap_uint<I>	—	tanpi	ap_uint<I> x
2071	template<int W, int I> ap_fixed<W,I>	—	pow	ap_fixed<W,I> x, ap_fixed<W,I> y
2076	template<int W, int I> ap_ufixed<W,I>	—	pow	ap_ufixed<W,I> x, ap_ufixed<W,I> y
2080	template<int I> ap_int<I>	—	pow	ap_int<I> x, ap_int<I> y
2084	template<int I> ap_uint<I>	—	pow	ap_uint<I> x, ap_uint<I> y
2089	template<int W, int I> ap_fixed<W,I>	—	pown	ap_fixed<W,I> x, int n
2094	template<int W, int I> ap_ufixed<W,I>	—	pown	ap_ufixed<W,I> x, int n
2098	template<int I> ap_int<I>	—	pown	ap_int<I> x, int n
2102	template<int I> ap_uint<I>	—	pown	ap_uint<I> x, int n
2113	template<int W, int I> ap_fixed<W,I>	—	rootn	ap_fixed<W,I> x, int n
2118	template<int W, int I> ap_ufixed<W,I>	—	rootn	ap_ufixed<W,I> x, int n
2122	template<int I> ap_int<I>	—	rootn	ap_int<I> x, int n
2126	template<int I> ap_uint<I>	—	rootn	ap_uint<I> x, int n

Line	Template signature	Returns	Name	Parameters
2137	template<int W, int I> ap_fixed<W,I>	—	powr	ap_fixed<W,I> x, ap_fixed<W,I> y
2142	template<int W, int I> ap_ufixed<W,I>	—	powr	ap_ufixed<W,I> x, ap_ufixed<W,I> y
2146	template<int I> ap_int<I>	—	powr	ap_int<I> x, ap_int<I> y
2150	template<int I> ap_uint<I>	—	powr	ap_uint<I> x, ap_uint<I> y
2157	template<int W, int I> ap_fixed<W,I>	—	round	ap_fixed<W,I> x
2161	template<int W, int I> ap_ufixed<W,I>	—	round	ap_ufixed<W,I> x
2165	template<int I> ap_int<I>	—	round	ap_int<I> x
2169	template<int I> ap_uint<I>	—	round	ap_uint<I> x
2181	template<int W, int I> ap_fixed<W,I>	—	rint	ap_fixed<W,I> x
2185	template<int W, int I> ap_ufixed<W,I>	—	rint	ap_ufixed<W,I> x
2189	template<int I> ap_int<I>	—	rint	ap_int<I> x
2193	template<int I> ap_uint<I>	—	rint	ap_uint<I> x
2205	template<int W, int I> ap_fixed<W,I>	—	nearbyint	ap_fixed<W,I> x
2209	template<int W, int I> ap_ufixed<W,I>	—	nearbyint	ap_ufixed<W,I> x
2213	template<int I> ap_int<I>	—	nearbyint	ap_int<I> x
2217	template<int I> ap_uint<I>	—	nearbyint	ap_uint<I> x
2229	template<int W, int I>	long long int	llround	ap_fixed<W,I> x
2233	template<int W, int I>	long long int	llround	ap_ufixed<W,I> x
2237	template<int I>	long long int	llround	ap_int<I> x
2241	template<int I>	long long int	llround	ap_uint<I> x
2255	template<int W, int I>	long int	lround	ap_fixed<W,I> x

Line	Template signature	Returns	Name	Parameters
2259	template<int W, int I>	long int	lround	ap_ufixed<W,I> x
2263	template<int I>	long int	lround	ap_int<I> x
2267	template<int I>	long int	lround	ap_uint<I> x
2281	template<int W, int I>	long long int	llrint	ap_fixed<W,I> x
2285	template<int W, int I>	long long int	llrint	ap_ufixed<W,I> x
2289	template<int I>	long long int	llrint	ap_int<I> x
2293	template<int I>	long long int	llrint	ap_uint<I> x
2307	template<int W, int I>	long int	lrint	ap_fixed<W,I> x
2311	template<int W, int I>	long int	lrint	ap_ufixed<W,I> x
2315	template<int I>	long int	lrint	ap_int<I> x
2319	template<int I>	long int	lrint	ap_uint<I> x
2333	template<int W, int I>	bool	isgreater	ap_fixed<W,I> x1, ap_fixed<W,I> x2
2337	template<int W, int I>	bool	isgreater	ap_ufixed<W,I> x1, ap_ufixed<W,I> x2
2341	template<int I>	bool	isgreater	ap_int<I> x1, ap_int<I> x2
2345	template<int I>	bool	isgreater	ap_uint<I> x1, ap_uint<I> x2
2357	template<int W, int I>	bool	isgreate- requal	ap_fixed<W,I> x1, ap_fixed<W,I> x2
2361	template<int W, int I>	bool	isgreate- requal	ap_ufixed<W,I> x1, ap_ufixed<W,I> x2
2365	template<int I>	bool	isgreate- requal	ap_int<I> x1, ap_int<I> x2
2369	template<int I>	bool	isgreate- requal	ap_uint<I> x1, ap_uint<I> x2
2381	template<int W, int I>	bool	isless	ap_fixed<W,I> x1, ap_fixed<W,I> x2
2385	template<int W, int I>	bool	isless	ap_ufixed<W,I> x1, ap_ufixed<W,I> x2
2389	template<int I>	bool	isless	ap_int<I> x1, ap_int<I> x2
2393	template<int I>	bool	isless	ap_uint<I> x1, ap_uint<I> x2
2405	template<int W, int I>	bool	islessequal	ap_fixed<W,I> x1, ap_fixed<W,I> x2
2409	template<int W, int I>	bool	islessequal	ap_ufixed<W,I> x1, ap_ufixed<W,I> x2

Line	Template signature	Returns	Name	Parameters
2413	template<int I>	bool	islessequal	ap_int<I> x1, ap_int<I> x2
2417	template<int I>	bool	islessequal	ap_uint<I> x1, ap_uint<I> x2
2429	template<int W, int I>	bool	isless-greater	ap_fixed<W,I> x1, ap_fixed<W,I> x2
2433	template<int W, int I>	bool	isless-greater	ap_ufixed<W,I> x1, ap_ufixed<W,I> x2
2437	template<int I>	bool	isless-greater	ap_int<I> x1, ap_int<I> x2
2441	template<int I>	bool	isless-greater	ap_uint<I> x1, ap_uint<I> x2
2460	template<int W, int I> ap_fixed<W,I>	—	frexp	ap_fixed<W,I> x, ap_fixed<W,I>* exp
2465	template<int W, int I> ap_ufixed<W,I>	—	frexp	ap_ufixed<W,I> x, ap_ufixed<W,I>* exp
2469	template<int I> ap_int<I>	—	frexp	ap_int<I> x, ap_int<I>* exp
2473	template<int I> ap_uint<I>	—	frexp	ap_uint<I> x, ap_uint<I>* exp
2485	template<int W, int I> ap_fixed<W,I>	—	modf	ap_fixed<W,I> x, ap_fixed<W,I>* intpart
2490	template<int W, int I> ap_ufixed<W,I>	—	modf	ap_ufixed<W,I> x, ap_ufixed<W,I>* intpart
2494	template<int I> ap_int<I>	—	modf	ap_int<I> x, ap_int<I>* intpart
2498	template<int I> ap_uint<I>	—	modf	ap_uint<I> x, ap_uint<I>* intpart
2518	template<int W, int I>	bool	isequal	ap_fixed<W,I> x, ap_fixed<W,I> y
2522	template<int W, int I>	bool	isequal	ap_ufixed<W,I> x, ap_ufixed<W,I> y
2526	template<int I>	bool	isequal	ap_int<I> x, ap_int<I> y
2530	template<int I>	bool	isequal	ap_uint<I> x, ap_uint<I> y
2535	template<int W, int I>	bool	isnotequal	ap_fixed<W,I> x, ap_fixed<W,I> y
2539	template<int W, int I>	bool	isnotequal	ap_ufixed<W,I> x, ap_ufixed<W,I> y
2543	template<int I>	bool	isnotequal	ap_int<I> x, ap_int<I> y
2547	template<int I>	bool	isnotequal	ap_uint<I> x, ap_uint<I> y
2552	template<int W, int I>	bool	any	ap_fixed<W,I> x
2556	template<int W, int I>	bool	any	ap_ufixed<W,I> x
2560	template<int I>	bool	any	ap_int<I> x
2564	template<int I>	bool	any	ap_uint<I> x
2569	template<int W, int I>	bool	all	ap_fixed<W,I> x
2573	template<int W, int I>	bool	all	ap_ufixed<W,I> x
2577	template<int I>	bool	all	ap_int<I> x
2581	template<int I>	bool	all	ap_uint<I> x

Line	Template signature	Returns	Name	Parameters
2586	template<int W, int I> ap_fixed<W, I>	—	bitselect	ap_fixed<W, I> x, ap_fixed<W, I> y, ap_fixed<W, I> z
2590	template<int W, int I> ap_ufixed<W, I>	—	bitselect	ap_ufixed<W, I> x, ap_ufixed<W, I> y, ap_ufixed<W, I> z
2594	template<int I> ap_int<I>	—	bitselect	ap_int<I> x, ap_int<I> y, ap_int<I> z
2598	template<int I> ap_uint<I>	—	bitselect	ap_uint<I> x, ap_uint<I> y, ap_uint<I> z
2603	template<int W, int I> ap_fixed<W, I>	—	erf	ap_fixed<W, I> x
2607	template<int W, int I> ap_ufixed<W, I>	—	erf	ap_ufixed<W, I> x
2611	template<int I> ap_int<I>	—	erf	ap_int<I> x
2615	template<int I> ap_uint<I>	—	erf	ap_uint<I> x
2620	template<int W, int I> ap_fixed<W, I>	—	erfc	ap_fixed<W, I> x
2624	template<int W, int I> ap_ufixed<W, I>	—	erfc	ap_ufixed<W, I> x
2628	template<int I> ap_int<I>	—	erfc	ap_int<I> x
2632	template<int I> ap_uint<I>	—	erfc	ap_uint<I> x
2644	template<int W, int I> ap_fixed<W, I>	—	divide	ap_fixed<W, I> x, ap_fixed<W, I> y
2648	template<int W, int I> ap_ufixed<W, I>	—	divide	ap_ufixed<W, I> x, ap_ufixed<W, I> y
2652	template<int I> ap_int<I>	—	divide	ap_int<I> x, ap_int<I> y
2656	template<int I> ap_uint<I>	—	divide	ap_uint<I> x, ap_uint<I> y
2668	template<int W, int I> ap_fixed<W, I>	—	fmod	ap_fixed<W, I> x, ap_fixed<W, I> y
2672	template<int W, int I> ap_ufixed<W, I>	—	fmod	ap_ufixed<W, I> x, ap_ufixed<W, I> y
2676	template<int I> ap_int<I>	—	fmod	ap_int<I> x, ap_int<I> y
2680	template<int I> ap_uint<I>	—	fmod	ap_uint<I> x, ap_uint<I> y
2692	template<int W, int I> ap_fixed<W, I>	—	remainder	ap_fixed<W, I> x, ap_fixed<W, I> y
2696	template<int W, int I> ap_ufixed<W, I>	—	remainder	ap_ufixed<W, I> x, ap_ufixed<W, I> y

Line	Template signature	Returns	Name	Parameters
2700	template<int I> ap_int<I>	—	remainder	ap_int<I> x, ap_int<I> y
2704	template<int I> ap_uint<I>	—	remainder	ap_uint<I> x, ap_uint<I> y
2716	template<int W, int I> ap_fixed<W,I>	—	remquo	ap_fixed<W,I> x, ap_fixed<W,I> y, int* quo
2720	template<int W, int I> ap_ufixed<W,I>	—	remquo	ap_ufixed<W,I> x, ap_ufixed<W, I> y, int* quo
2724	template<int I> ap_int<I>	—	remquo	ap_int<I> x, ap_int<I> y, int* quo
2728	template<int I> ap_uint<I>	—	remquo	ap_uint<I> x, ap_uint<I> y, int* quo
2733	template<int W, int I>	bool	signbit	ap_fixed<W, I> x
2737	template<int W, int I>	bool	signbit	ap_ufixed<W, I> x
2741	template<int I>	bool	signbit	ap_int<I> x
2745	template<int I>	bool	signbit	ap_uint<I> x
2757	template<int W, int I> ap_fixed<W,I>	—	lgamma_r	ap_fixed<W,I> x, int *signgamp
2761	template<int W, int I> ap_ufixed<W,I>	—	lgamma_r	ap_ufixed<W,I> x, int *signgamp
2765	template<int I> ap_int<I>	—	lgamma_r	ap_int<I> x, int *signgamp
2769	template<int I> ap_uint<I>	—	lgamma_r	ap_uint<I> x, int *signgamp
2774	template<int W, int I> ap_fixed<W,I>	—	lgamma	ap_fixed<W,I> x
2778	template<int W, int I> ap_ufixed<W,I>	—	lgamma	ap_ufixed<W,I> x
2782	template<int I> ap_int<I>	—	lgamma	ap_int<I> x
2786	template<int I> ap_uint<I>	—	lgamma	ap_uint<I> x
2798	template<int W, int I> ap_fixed<W,I>	—	mad	ap_fixed<W,I> x, ap_fixed<W,I> y, ap_fixed<W,I> z
2802	template<int W, int I> ap_ufixed<W,I>	—	mad	ap_ufixed<W,I> x, ap_ufixed<W,I> y, ap_ufixed<W,I> z
2806	template<int I> ap_int<I>	—	mad	ap_int<I> x, ap_int<I> y, ap_int<I> z
2810	template<int I> ap_uint<I>	—	mad	ap_uint<I> x, ap_uint<I> y, ap_uint<I> z
2822	template<int W, int I> ap_fixed<W,I>	—	fma	ap_fixed<W,I> x, ap_fixed<W,I> y, ap_fixed<W,I> z
2826	template<int W, int I> ap_ufixed<W,I>	—	fma	ap_ufixed<W,I> x, ap_ufixed<W,I> y, ap_ufixed<W,I> z

Line	Template signature	Returns	Name	Parameters
2830	template<int I> ap_int<I>	—	fma	ap_int<I> x, ap_int<I> y, ap_int<I> z
2834	template<int I> ap_uint<I>	—	fma	ap_uint<I> x, ap_uint<I> y, ap_uint<I> z
2846	template<int W, int I> ap_fixed<W, I>	—	fract	ap_fixed<W, I> x
2850	template<int W, int I> ap_ufixed<W, I>	—	fract	ap_ufixed<W, I> x
2854	template<int I> ap_int<I>	—	fract	ap_int<I> x
2858	template<int I> ap_uint<I>	—	fract	ap_uint<I> x
2863	template<int W, int I> ap_fixed<W, I>	—	cbirt	ap_fixed<W, I> x
2867	template<int W, int I> ap_ufixed<W, I>	—	cbirt	ap_ufixed<W, I> x
2871	template<int I> ap_int<I>	—	cbirt	ap_int<I> x
2875	template<int I> ap_uint<I>	—	cbirt	ap_uint<I> x
2887	template<int W, int I> ap_fixed<W, I>	—	tgamma	ap_fixed<W, I> x
2891	template<int W, int I> ap_ufixed<W, I>	—	tgamma	ap_ufixed<W, I> x
2895	template<int I> ap_int<I>	—	tgamma	ap_int<I> x
2899	template<int I> ap_uint<I>	—	tgamma	ap_uint<I> x
2905	template<int W, int I> ap_fixed<W, I>	—	nextafter	ap_fixed<W, I> x, ap_fixed<W, I> y
2909	template<int W, int I> ap_ufixed<W, I>	—	nextafter	ap_ufixed<W, I> x, ap_ufixed<W, I> y
2913	template<int I> ap_int<I>	—	nextafter	ap_int<I> x, ap_int<I> y
2917	template<int I> ap_uint<I>	—	nextafter	ap_uint<I> x, ap_uint<I> y
2929	template<int W, int I> ap_fixed<W, I>	—	nexttoward	ap_fixed<W, I> x, ap_fixed<W, I> y
2933	template<int W, int I> ap_ufixed<W, I>	—	nexttoward	ap_ufixed<W, I> x, ap_ufixed<W, I> y
2937	template<int I> ap_int<I>	—	nexttoward	ap_int<I> x, ap_int<I> y
2941	template<int I> ap_uint<I>	—	nexttoward	ap_uint<I> x, ap_uint<I> y

Line	Template signature	Returns	Name	Parameters
2953	template<int W, int I> ap_fixed<W,I>	—	hypot	ap_fixed<W,I> x, ap_fixed<W,I> y
2957	template<int W, int I> ap_ufixed<W,I>	—	hypot	ap_ufixed<W,I> x, ap_ufixed<W,I> y
2961	template<int I> ap_int<I>	—	hypot	ap_int<I> x, ap_int<I> y
2965	template<int I> ap_uint<I>	—	hypot	ap_uint<I> x, ap_uint<I> y
2976	template<int W, int I> ap_fixed<W,I>	—	exp2	ap_fixed<W,I> x
2981	template<int W, int I> ap_ufixed<W,I>	—	exp2	ap_ufixed<W,I> x
2986	template<int I> ap_int<I>	—	exp2	ap_int<I> x
2991	template<int I> ap_uint<I>	—	exp2	ap_uint<I> x
2996	template<int W, int I> ap_fixed<W,I>	—	exp10	ap_fixed<W,I> x
3001	template<int W, int I> ap_ufixed<W,I>	—	exp10	ap_ufixed<W,I> x
3006	template<int I> ap_int<I>	—	exp10	ap_int<I> x
3011	template<int I> ap_uint<I>	—	exp10	ap_uint<I> x
3016	template<int W, int I> ap_fixed<W,I>	—	expm1	ap_fixed<W,I> x
3021	template<int W, int I> ap_ufixed<W,I>	—	expm1	ap_ufixed<W,I> x
3026	template<int I> ap_int<I>	—	expm1	ap_int<I> x
3031	template<int I> ap_uint<I>	—	expm1	ap_uint<I> x
3036	template<int W, int I> ap_fixed<W,I>	—	log1p	ap_fixed<W,I> x
3041	template<int W, int I> ap_ufixed<W,I>	—	log1p	ap_ufixed<W,I> x
3046	template<int I> ap_int<I>	—	log1p	ap_int<I> x
3051	template<int I> ap_uint<I>	—	log1p	ap_uint<I> x
3056	template<int W, int I> ap_fixed<W,I>	—	ilogb	ap_fixed<W,I> x
3061	template<int W, int I> ap_ufixed<W,I>	—	ilogb	ap_ufixed<W,I> x

Line	Template signature	Returns	Name	Parameters
3066	template<int I> ap_int<I>	—	ilogb	ap_int<I> x
3071	template<int I> ap_uint<I>	—	ilogb	ap_uint<I> x
3076	template<int W, int I> ap_fixed<W,I>	—	ldexp	ap_fixed<W,I> x, int exp
3081	template<int W, int I> ap_ufixed<W,I>	—	ldexp	ap_ufixed<W,I> x, int exp
3086	template<int I> ap_int<I>	—	ldexp	ap_int<I> x, int exp
3091	template<int I> ap_uint<I>	—	ldexp	ap_uint<I> x, int exp
3096	template<int W, int I> ap_fixed<W,I>	—	scalbn	ap_fixed<W,I> x, int n
3101	template<int W, int I> ap_ufixed<W,I>	—	scalbn	ap_ufixed<W,I> x, int n
3106	template<int I> ap_int<I>	—	scalbn	ap_int<I> x, int n
3111	template<int I> ap_uint<I>	—	scalbn	ap_uint<I> x, int n
3116	template<int W, int I> ap_fixed<W,I>	—	scalbln	ap_fixed<W,I> x, long int n
3121	template<int W, int I> ap_ufixed<W,I>	—	scalbln	ap_ufixed<W,I> x, long int n
3126	template<int I> ap_int<I>	—	scalbln	ap_int<I> x, long int n
3131	template<int I> ap_uint<I>	—	scalbln	ap_uint<I> x, long int n

hls_stream.h

Table 176: Every class and struct in hls_stream.h (6 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
68	class	stream_delegate	template<size_t SIZE>	—	—
76	class	stream_globals	template<size_t SIZE>	—	—
176	class	stream_entity	template<size_t SIZE>	—	—
295	class	stream_map	template<size_t SIZE>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
337	class	stream <__STREAM_T__, 0>	template<typename __STREAM_T__>	—	typedef stream_map<sizeof(__STREAM_T__)> map_t;
519	class	stream	template<typename __STREAM_T__, int DEPTH>	: public stream<__STREAM_T__, 0>	—

Table 177: Every function definition in `hls_stream.h` (30 definitions)

Line	Template signature	Returns	Name	Parameters
77	template<size_t SIZE>	public: static void	print_max_size	()
82	template<size_t SIZE>	static void	incr_b- locked_counter	()
86	template<size_t SIZE>	static void	decr_b- locked_counter	()
90	template<size_t SIZE>	static void	incr_task_counter	()
94	template<size_t SIZE>	static void	decr_task_counter	()
98	template<size_t SIZE>	static void	start_threads	()
122	template<size_t SIZE>	private: static bool	check_deadlock	()
136	template<size_t SIZE>	static void	deadlock_thread	()
189	template<size_t SIZE>	bool	read	void *elem
224	template<size_t SIZE>	void	write	const void *elem
249	—	bool	read_nb	void *elem
266	—	size_t	size	()
277	—	void	set_name	const char *n
296	template<size_t SIZE>	public: static size_t	count	void *p
304	template<size_t SIZE>	static void	insert	void *p
352	template<typename __STREAM_T__>	protected: pub- lic:	stream	()
438	—	bool	exist	()
444	—	bool	empty	()
448	—	bool	full	()
451	—	void	read	__STREAM_T__& head
456	—	bool	read_dep	__STREAM_T__& head, volatile bool flag
461	—	__STREAM_T__	read	()
470	—	void	write	const __STREAM_T__& tail
475	—	bool	write_dep	const __STREAM_T__& tail, volatile bool flag

Line	Template signature	Returns	Name	Parameters
481	—	bool	read_nb	__STREAM_T__ & head
491	—	bool	write_nb	const __STREAM_T__ & tail
498	—	size_t	size	()
503	—	size_t	capacity	()
509	—	void	set_name	const char *name
520	template<typename __STREAM_T__, int DEPTH>	public:	stream	()

hls_x_complex.h

Table 178: Every class and struct in `hls_x_complex.h` (1 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
33	class	x_complex	template <typename T>	—	typedef x_complex<T> MULT_RT;

Table 179: Every function definition in `hls_x_complex.h` (14 definitions)

Line	Template signature	Returns	Name	Parameters
52	template <typename T>	inline	x_complex	const std::complex<T> &z
56	template <typename T>	template <typename T2> inline	x_complex	const x_complex<T2> &z
63	template <typename T2>	inline T	real	()
66	template <typename T2>	inline void	real	const T &r
68	template <typename T2>	inline T	imag	()
71	template <typename T2>	inline void	imag	const T &i
277	template <typename T2>	std::complex<T>	to_std_complex	()
330	template <typename T>	template <typename T> x_complex<T>	x_neg	x_complex<T> &din
337	template <typename T> x_complex<T>	—	x_conj_sq	x_complex<T> &din
342	template <typename T> x_complex<T>	—	x_conj	const x_complex<T> &din
350	template <typename T> x_complex<T>	—	x_conj	x_complex<T> &din
355	template <int W, int I>	void	set_to_one	x_complex<ap_fixed<W, I>> &a
362	template <typename T>	T	x_real	const x_complex<T> &din
364	template <typename T>	T	x_imag	const x_complex<T> &din

utils/std_complex_utils.h

Table 180: Every class and struct in `utils/std_complex_utils.h` (5 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
68	struct	<code>x_traits</code> <std::complex<T1>,std::complex<T2>>	template<typename T1, typename T2>	—	typedef std::complex<T1> ADD_T; typedef std::complex<T1> SAT_T; typedef std::complex<T1> MULT_T; typedef std::complex<T1> MADD_T; typedef std::complex<int> INT_T; typedef std::complex<float_struct<24,8>> FLOAT_STRUCT_T;
80	struct	<code>x_traits</code> <std::complex<ap_fixed<W1,I1>>,std::complex<ap_fixed<W2,I2>>>	template<int W1, int I1, int W2, int I2>	—	typedef std::complex<ap_fixed<X_COMPLEX_MAX(I1,I2)+1+X_COMPLEX_MAX(W1-I1,I2),X_COMPLEX_MAX(I1,I2)+1>> ADD_T; typedef std::complex<ap_fixed<W1,I1,AP_RND,AP_SAT>> SAT_T; typedef std::complex<ap_fixed<W1+W2+1,I1+I2+1>> MULT_T; typedef std::complex<ap_fixed<W1+W2+2,I1+I2+2>> MADD_T; typedef std::complex<ap_fixed<(2*W1)+1,(2*I1)+1>> CONJ_SQ_T; typedef std::complex<int> INT_T; typedef std::complex<float_struct<24,8>> FLOAT_STRUCT_T;
91	struct	<code>x_traits</code> <std::complex<ap_int<W1>>,std::complex<ap_int<W2>>>	template<int W1, int W2>	—	typedef std::complex<ap_int<X_COMPLEX_MAX(W1,W2)+1>> ADD_T; typedef std::complex<ap_int<W1>> SAT_T; typedef std::complex<ap_int<W1+W2+1>> MULT_T; typedef std::complex<ap_int<W1+W2+2>> MADD_T; typedef std::complex<ap_int<(2*W1)+1>> CONJ_SQ_T; typedef std::complex<int> INT_T; typedef std::complex<float_struct<24,8>> FLOAT_STRUCT_T;
104	struct	<code>x_traits_d</code> <std::complex<T>,DIM>	template<int DIM, typename T>	—	typedef std::complex<T> ACCUM_T; typedef std::complex<T> ACCUM2_T; typedef std::complex<ap_int<25+DIM>> ADD_TREE_T;
111	struct	<code>x_traits_d</code> <std::complex<ap_fixed<W,I>>,DIM>	template<int DIM, int W, int I>	—	typedef std::complex<ap_fixed<W+BitWidth<DIM>::Value,I+BitWidth<DIM>::Value>> ACCUM_T; typedef std::complex<ap_fixed<(2*W)+BitWidth<DIM>::Value,(2*I)+BitWidth<DIM>::Value>> ACCUM2_T;

utils/x_hls_defines.h

Table 181: Every class and struct in `utils/x_hls_defines.h` (2 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
118	struct	<code>float_struct</code>	template<int M, int E>	—	—
128	struct	<code>float_struct2</code>	template<int M, int E>	—	—

utils/x_hls_traits.h

Table 182: Every class and struct in `utils/x_hls_traits.h` (300 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
57	struct	<code>x_traits_default</code>	template<typename T>	—	typedef T ADD_T; typedef T SAT_T; typedef T MULT_T; typedef T MADD_T; typedef T INT_T; typedef T FLOAT_STRUCT_T;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
66	struct	x_traits	template<typename T1, typename T2>	—	—
80	struct	x_traits<double, T2>	template<typename T2>	: public x_traits_default<double>	—
83	struct	x_traits<T1, double>	template<typename T1>	: public x_traits_default<double>	—
86	struct	x_traits<float, T2>	template<typename T2>	: public x_traits_default<float>	—
89	struct	x_traits<T1, float>	template<typename T1>	: public x_traits_default<float>	—
92	struct	x_traits<half, T2>	template<typename T2>	: public x_traits_default<float>	—
95	struct	x_traits<T1, half>	template<typename T1>	: public x_traits_default<float>	—
112	struct	x_traits<char, char>	template<>	: public x_traits_default<char>	—
115	struct	x_traits<char, signed char>	template<>	: public x_traits_default<signed char>	—
118	struct	x_traits<char, unsigned char>	template<>	: public x_traits_default<unsigned char>	—
121	struct	x_traits<char, short>	template<>	: public x_traits_default<short>	—
124	struct	x_traits<char, unsigned short>	template<>	: public x_traits_default<unsigned short>	—
127	struct	x_traits<char, int>	template<>	: public x_traits_default<int>	—
130	struct	x_traits<char, unsigned int>	template<>	: public x_traits_default<unsigned int>	—
133	struct	x_traits<char, long>	template<>	: public x_traits_default<long>	—
136	struct	x_traits<char, unsigned long>	template<>	: public x_traits_default<unsigned long>	—
139	struct	x_traits<char, ap_slong>	template<>	: public x_traits_default<ap_slong>	—
142	struct	x_traits<char, ap_ulong>	template<>	: public x_traits_default<ap_ulong>	—
145	struct	x_traits<char, half>	template<>	: public x_traits_default<float>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
148	struct	<code>x_traits</code> < char , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
151	struct	<code>x_traits</code> < char , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
154	struct	<code>x_traits</code> < signed char , char >	<code>template<></code>	<code>: public x_traits_default< signed char ></code>	—
157	struct	<code>x_traits</code> < signed char , signed char >	<code>template<></code>	<code>: public x_traits_default< signed char ></code>	—
160	struct	<code>x_traits</code> < signed char , unsigned char >	<code>template<></code>	<code>: public x_traits_default< unsigned char ></code>	—
163	struct	<code>x_traits</code> < signed char , short >	<code>template<></code>	<code>: public x_traits_default< short ></code>	—
166	struct	<code>x_traits</code> < signed char , unsigned short >	<code>template<></code>	<code>: public x_traits_default< unsigned short ></code>	—
169	struct	<code>x_traits</code> < signed char , int >	<code>template<></code>	<code>: public x_traits_default< int ></code>	—
172	struct	<code>x_traits</code> < signed char , unsigned int >	<code>template<></code>	<code>: public x_traits_default< unsigned int ></code>	—
175	struct	<code>x_traits</code> < signed char , long >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
178	struct	<code>x_traits</code> < signed char , unsigned long >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
181	struct	<code>x_traits</code> < signed char , ap_slong >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
184	struct	<code>x_traits</code> < signed char , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
187	struct	<code>x_traits</code> < signed char , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
190	struct	<code>x_traits</code> < signed char , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
193	struct	<code>x_traits</code> < signed char , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
196	struct	<code>x_traits</code> < unsigned char , char >	<code>template<></code>	<code>: public x_traits_default< unsigned char ></code>	—
199	struct	<code>x_traits</code> < unsigned char , signed char >	<code>template<></code>	<code>: public x_traits_default< unsigned char ></code>	—
202	struct	<code>x_traits</code> < unsigned char , unsigned char >	<code>template<></code>	<code>: public x_traits_default< unsigned char ></code>	—
205	struct	<code>x_traits</code> < unsigned char , short >	<code>template<></code>	<code>: public x_traits_default< short ></code>	—
208	struct	<code>x_traits</code> < unsigned char , unsigned short >	<code>template<></code>	<code>: public x_traits_default< unsigned short ></code>	—
211	struct	<code>x_traits</code> < unsigned char , int >	<code>template<></code>	<code>: public x_traits_default< int ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
214	struct	<code>x_traits</code> < unsigned char , unsigned int >	<code>template<></code>	<code>: public x_traits_default< unsigned int ></code>	—
217	struct	<code>x_traits</code> < unsigned char , long >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
220	struct	<code>x_traits</code> < unsigned char , unsigned long >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
223	struct	<code>x_traits</code> < unsigned char , ap_slong >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
226	struct	<code>x_traits</code> < unsigned char , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
229	struct	<code>x_traits</code> < unsigned char , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
232	struct	<code>x_traits</code> < unsigned char , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
235	struct	<code>x_traits</code> < unsigned char , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
238	struct	<code>x_traits</code> < short , char >	<code>template<></code>	<code>: public x_traits_default< short ></code>	—
241	struct	<code>x_traits</code> < short , signed char >	<code>template<></code>	<code>: public x_traits_default< short ></code>	—
244	struct	<code>x_traits</code> < short , unsigned char >	<code>template<></code>	<code>: public x_traits_default< short ></code>	—
247	struct	<code>x_traits</code> < short , short >	<code>template<></code>	<code>: public x_traits_default< short ></code>	—
250	struct	<code>x_traits</code> < short , unsigned short >	<code>template<></code>	<code>: public x_traits_default< unsigned short ></code>	—
253	struct	<code>x_traits</code> < short , int >	<code>template<></code>	<code>: public x_traits_default< int ></code>	—
256	struct	<code>x_traits</code> < short , unsigned int >	<code>template<></code>	<code>: public x_traits_default< unsigned int ></code>	—
259	struct	<code>x_traits</code> < short , long >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
262	struct	<code>x_traits</code> < short , unsigned long >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
265	struct	<code>x_traits</code> < short , ap_slong >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
268	struct	<code>x_traits</code> < short , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
271	struct	<code>x_traits</code> < short , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
274	struct	<code>x_traits</code> < short , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
277	struct	<code>x_traits</code> < short , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
280	struct	<code>x_traits</code> < unsigned short , char >	template<>	: public <code>x_traits_default< unsigned short ></code>	—
283	struct	<code>x_traits</code> < unsigned short , signed char >	template<>	: public <code>x_traits_default< unsigned short ></code>	—
286	struct	<code>x_traits</code> < unsigned short , unsigned char >	template<>	: public <code>x_traits_default< unsigned short ></code>	—
289	struct	<code>x_traits</code> < unsigned short , short >	template<>	: public <code>x_traits_default< unsigned short ></code>	—
292	struct	<code>x_traits</code> < unsigned short , unsigned short >	template<>	: public <code>x_traits_default< unsigned short ></code>	—
295	struct	<code>x_traits</code> < unsigned short , int >	template<>	: public <code>x_traits_default< int ></code>	—
298	struct	<code>x_traits</code> < unsigned short , unsigned int >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
301	struct	<code>x_traits</code> < unsigned short , long >	template<>	: public <code>x_traits_default< long ></code>	—
304	struct	<code>x_traits</code> < unsigned short , unsigned long >	template<>	: public <code>x_traits_default< unsigned long ></code>	—
307	struct	<code>x_traits</code> < unsigned short , ap_slong >	template<>	: public <code>x_traits_default< ap_slong ></code>	—
310	struct	<code>x_traits</code> < unsigned short , ap_ulong >	template<>	: public <code>x_traits_default< ap_ulong ></code>	—
313	struct	<code>x_traits</code> < unsigned short , half >	template<>	: public <code>x_traits_default< float ></code>	—
316	struct	<code>x_traits</code> < unsigned short , float >	template<>	: public <code>x_traits_default< float ></code>	—
319	struct	<code>x_traits</code> < unsigned short , double >	template<>	: public <code>x_traits_default< double ></code>	—
322	struct	<code>x_traits</code> < int , char >	template<>	: public <code>x_traits_default< int ></code>	—
325	struct	<code>x_traits</code> < int , signed char >	template<>	: public <code>x_traits_default< int ></code>	—
328	struct	<code>x_traits</code> < int , unsigned char >	template<>	: public <code>x_traits_default< int ></code>	—
331	struct	<code>x_traits</code> < int , short >	template<>	: public <code>x_traits_default< int ></code>	—
334	struct	<code>x_traits</code> < int , unsigned short >	template<>	: public <code>x_traits_default< int ></code>	—
337	struct	<code>x_traits</code> < int , int >	template<>	: public <code>x_traits_default< int ></code>	—
340	struct	<code>x_traits</code> < int , unsigned int >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
343	struct	<code>x_traits</code> < int , long >	template<>	: public <code>x_traits_default< long ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
346	struct	<code>x_traits</code> < int , unsigned long >	template<>	: public <code>x_traits_default< unsigned long ></code>	—
349	struct	<code>x_traits</code> < int , ap_slong >	template<>	: public <code>x_traits_default< ap_slong ></code>	—
352	struct	<code>x_traits</code> < int , ap_ulong >	template<>	: public <code>x_traits_default< ap_ulong ></code>	—
355	struct	<code>x_traits</code> < int , half >	template<>	: public <code>x_traits_default< float ></code>	—
358	struct	<code>x_traits</code> < int , float >	template<>	: public <code>x_traits_default< float ></code>	—
361	struct	<code>x_traits</code> < int , double >	template<>	: public <code>x_traits_default< double ></code>	—
364	struct	<code>x_traits</code> < unsigned int , char >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
367	struct	<code>x_traits</code> < unsigned int , signed char >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
370	struct	<code>x_traits</code> < unsigned int , unsigned char >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
373	struct	<code>x_traits</code> < unsigned int , short >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
376	struct	<code>x_traits</code> < unsigned int , unsigned short >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
379	struct	<code>x_traits</code> < unsigned int , int >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
382	struct	<code>x_traits</code> < unsigned int , unsigned int >	template<>	: public <code>x_traits_default< unsigned int ></code>	—
385	struct	<code>x_traits</code> < unsigned int , long >	template<>	: public <code>x_traits_default< long ></code>	—
388	struct	<code>x_traits</code> < unsigned int , unsigned long >	template<>	: public <code>x_traits_default< unsigned long ></code>	—
391	struct	<code>x_traits</code> < unsigned int , ap_slong >	template<>	: public <code>x_traits_default< ap_slong ></code>	—
394	struct	<code>x_traits</code> < unsigned int , ap_ulong >	template<>	: public <code>x_traits_default< ap_ulong ></code>	—
397	struct	<code>x_traits</code> < unsigned int , half >	template<>	: public <code>x_traits_default< float ></code>	—
400	struct	<code>x_traits</code> < unsigned int , float >	template<>	: public <code>x_traits_default< float ></code>	—
403	struct	<code>x_traits</code> < unsigned int , double >	template<>	: public <code>x_traits_default< double ></code>	—
406	struct	<code>x_traits</code> < long , char >	template<>	: public <code>x_traits_default< long ></code>	—
409	struct	<code>x_traits</code> < long , signed char >	template<>	: public <code>x_traits_default< long ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
412	struct	<code>x_traits</code> < long , unsigned char >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
415	struct	<code>x_traits</code> < long , short >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
418	struct	<code>x_traits</code> < long , unsigned short >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
421	struct	<code>x_traits</code> < long , int >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
424	struct	<code>x_traits</code> < long , unsigned int >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
427	struct	<code>x_traits</code> < long , long >	<code>template<></code>	<code>: public x_traits_default< long ></code>	—
430	struct	<code>x_traits</code> < long , unsigned long >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
433	struct	<code>x_traits</code> < long , ap_slong >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
436	struct	<code>x_traits</code> < long , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
439	struct	<code>x_traits</code> < long , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
442	struct	<code>x_traits</code> < long , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
445	struct	<code>x_traits</code> < long , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
448	struct	<code>x_traits</code> < unsigned long , char >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
451	struct	<code>x_traits</code> < unsigned long , signed char >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
454	struct	<code>x_traits</code> < unsigned long , unsigned char >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
457	struct	<code>x_traits</code> < unsigned long , short >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
460	struct	<code>x_traits</code> < unsigned long , unsigned short >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
463	struct	<code>x_traits</code> < unsigned long , int >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
466	struct	<code>x_traits</code> < unsigned long , unsigned int >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
469	struct	<code>x_traits</code> < unsigned long , long >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
472	struct	<code>x_traits</code> < unsigned long , unsigned long >	<code>template<></code>	<code>: public x_traits_default< unsigned long ></code>	—
475	struct	<code>x_traits</code> < unsigned long , ap_slong >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
478	struct	<code>x_traits</code> < unsigned long , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
481	struct	<code>x_traits</code> < unsigned long , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
484	struct	<code>x_traits</code> < unsigned long , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
487	struct	<code>x_traits</code> < unsigned long , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
490	struct	<code>x_traits</code> < ap_slong , char >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
493	struct	<code>x_traits</code> < ap_slong , signed char >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
496	struct	<code>x_traits</code> < ap_slong , unsigned char >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
499	struct	<code>x_traits</code> < ap_slong , short >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
502	struct	<code>x_traits</code> < ap_slong , unsigned short >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
505	struct	<code>x_traits</code> < ap_slong , int >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
508	struct	<code>x_traits</code> < ap_slong , unsigned int >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
511	struct	<code>x_traits</code> < ap_slong , long >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
514	struct	<code>x_traits</code> < ap_slong , unsigned long >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
517	struct	<code>x_traits</code> < ap_slong , ap_slong >	<code>template<></code>	<code>: public x_traits_default< ap_slong ></code>	—
520	struct	<code>x_traits</code> < ap_slong , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
523	struct	<code>x_traits</code> < ap_slong , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
526	struct	<code>x_traits</code> < ap_slong , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
529	struct	<code>x_traits</code> < ap_slong , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
532	struct	<code>x_traits</code> < ap_ulong , char >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
535	struct	<code>x_traits</code> < ap_ulong , signed char >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
538	struct	<code>x_traits</code> < ap_ulong , unsigned char >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—
541	struct	<code>x_traits</code> < ap_ulong , short >	<code>template<></code>	<code>: public x_traits_default< ap_ulong ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
544	struct	<code>x_traits</code> < <code>ap_ulong</code> , unsigned short >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
547	struct	<code>x_traits</code> < <code>ap_ulong</code> , int >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
550	struct	<code>x_traits</code> < <code>ap_ulong</code> , unsigned int >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
553	struct	<code>x_traits</code> < <code>ap_ulong</code> , long >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
556	struct	<code>x_traits</code> < <code>ap_ulong</code> , unsigned long >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
559	struct	<code>x_traits</code> < <code>ap_ulong</code> , <code>ap_slong</code> >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
562	struct	<code>x_traits</code> < <code>ap_ulong</code> , <code>ap_ulong</code> >	<code>template<></code>	: public <code>x_traits_default<ap_ulong ></code>	—
565	struct	<code>x_traits</code> < <code>ap_ulong</code> , half >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
568	struct	<code>x_traits</code> < <code>ap_ulong</code> , float >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
571	struct	<code>x_traits</code> < <code>ap_ulong</code> , double >	<code>template<></code>	: public <code>x_traits_default<double ></code>	—
574	struct	<code>x_traits</code> < half , char >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
577	struct	<code>x_traits</code> < half , signed char >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
580	struct	<code>x_traits</code> < half , unsigned char >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
583	struct	<code>x_traits</code> < half , short >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
586	struct	<code>x_traits</code> < half , unsigned short >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
589	struct	<code>x_traits</code> < half , int >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
592	struct	<code>x_traits</code> < half , unsigned int >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
595	struct	<code>x_traits</code> < half , long >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
598	struct	<code>x_traits</code> < half , unsigned long >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
601	struct	<code>x_traits</code> < half , <code>ap_slong</code> >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
604	struct	<code>x_traits</code> < half , <code>ap_ulong</code> >	<code>template<></code>	: public <code>x_traits_default<float ></code>	—
607	struct	<code>x_traits</code> < half , half >	<code>template<></code>	: public <code>x_traits_default<half ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
610	struct	<code>x_traits</code> < half , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
613	struct	<code>x_traits</code> < half , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
616	struct	<code>x_traits</code> < float , char >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
619	struct	<code>x_traits</code> < float , signed char >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
622	struct	<code>x_traits</code> < float , unsigned char >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
625	struct	<code>x_traits</code> < float , short >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
628	struct	<code>x_traits</code> < float , unsigned short >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
631	struct	<code>x_traits</code> < float , int >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
634	struct	<code>x_traits</code> < float , unsigned int >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
637	struct	<code>x_traits</code> < float , long >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
640	struct	<code>x_traits</code> < float , unsigned long >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
643	struct	<code>x_traits</code> < float , ap_slong >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
646	struct	<code>x_traits</code> < float , ap_ulong >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
649	struct	<code>x_traits</code> < float , half >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
652	struct	<code>x_traits</code> < float , float >	<code>template<></code>	<code>: public x_traits_default< float ></code>	—
655	struct	<code>x_traits</code> < float , double >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
658	struct	<code>x_traits</code> < double , char >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
661	struct	<code>x_traits</code> < double , signed char >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
664	struct	<code>x_traits</code> < double , unsigned char >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
667	struct	<code>x_traits</code> < double , short >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
670	struct	<code>x_traits</code> < double , unsigned short >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
673	struct	<code>x_traits</code> < double , int >	<code>template<></code>	<code>: public x_traits_default< double ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
676	struct	<code>x_traits</code> <code>< double , unsigned int ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
679	struct	<code>x_traits</code> <code>< double , long ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
682	struct	<code>x_traits</code> <code>< double , unsigned long ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
685	struct	<code>x_traits</code> <code>< double , ap_slong ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
688	struct	<code>x_traits</code> <code>< double , ap_ulong ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
691	struct	<code>x_traits</code> <code>< double , half ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
694	struct	<code>x_traits</code> <code>< double , float ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
697	struct	<code>x_traits</code> <code>< double , double ></code>	<code>template<></code>	<code>: public x_traits_default< double ></code>	—
703	class	<code>MAX</code>	<code>template<int A, int B></code>	—	—
709	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2></code>	—	<code>typedef type</code> <code>RType;</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef</code> <code>I1>::value+1</code> <code>... and 2 more (A</code>
725	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_ufixed<W2,I2,Q2,O2> ></code>	<code>template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2></code>	—	<code>typedef</code> <code>RType<W2, I2,</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code>
733	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2></code>	—	<code>typedef</code> <code>RType<W2, I2,</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code>

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
741	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_ufixed<W2,I2,Q2,O2></code> <code>></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2, int</code> <code>I2, ap_q_mode</code> <code>Q2, ap_o_mode</code> <code>O2></code>	—	<code>typedef</code> <code>RType<W2, I2,</code> <code>typedef ap_u</code> <code>typedef ap_f</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef</code> <code>I1>::value+1</code> <code>... and 2 more (A</code>
757	struct	<code>x_traits</code> <code><ap_int<W1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>
765	struct	<code>x_traits</code> <code><ap_int<W1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>
772	struct	<code>x_traits</code> <code><ap_uint<W1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>
779	struct	<code>x_traits</code> <code><ap_uint<W1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_u</code> <code>typedef ap_i</code> <code>typedef ap_u</code>
786	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,I1>,</code> <code>ap_fixed<W2,W2> ></code>	—
789	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,I1>,</code> <code>ap_ufixed<W2,W2> ></code>	—
792	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,I1>,</code> <code>ap_fixed<W2,W2> ></code>	—
795	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,I1>,</code> <code>ap_ufixed<W2,W2> ></code>	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
798	struct	<code>x_traits</code> <code><ap_int<W1>, ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,W1>,</code> <code>ap_fixed<W2,I2> ></code>	—
801	struct	<code>x_traits</code> <code><ap_int<W1>, ap_ufixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,W1>,</code> <code>ap_ufixed<W2,I2> ></code>	—
804	struct	<code>x_traits</code> <code><ap_uint<W1>, ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,W1>,</code> <code>ap_fixed<W2,I2> ></code>	—
807	struct	<code>x_traits</code> <code><ap_uint<W1>, ap_ufixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,W1>,</code> <code>ap_ufixed<W2,I2> ></code>	—
812	struct	<code>fixed_type</code>	<code>tem-</code> <code>plate<typename</code> <code>T></code>	—	—
813	struct	<code>fixed_type</code> <code><char></code>	<code>template<></code>	—	<code>typedef ap_f</code>
814	struct	<code>fixed_type</code> <code><unsigned char></code>	<code>template<></code>	—	<code>typedef ap_u</code>
815	struct	<code>fixed_type</code> <code><short></code>	<code>template<></code>	—	<code>typedef ap_f</code>
816	struct	<code>fixed_type</code> <code><unsigned short></code>	<code>template<></code>	—	<code>typedef ap_u</code>
817	struct	<code>fixed_type</code> <code><int></code>	<code>template<></code>	—	<code>typedef ap_f</code>
818	struct	<code>fixed_type</code> <code><unsigned int></code>	<code>template<></code>	—	<code>typedef ap_u</code>
819	struct	<code>fixed_type</code> <code><float></code>	<code>template<></code>	—	<code>typedef ap_u</code>
820	struct	<code>fixed_type</code> <code><ap_int<W> ></code>	<code>template<int</code> <code>W></code>	—	<code>typedef ap_f</code>
821	struct	<code>fixed_type</code> <code><ap_uint<W> ></code>	<code>template<int</code> <code>W></code>	—	<code>typedef ap_u</code>
822	struct	<code>fixed_type</code> <code><ap_fixed<W,I, _AP_Q, _AP_O> ></code>	<code>template<int</code> <code>W, int I,</code> <code>ap_q_mode</code> <code>_AP_Q,</code> <code>ap_o_mode</code> <code>_AP_O></code>	—	<code>typedef ap_f</code>

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
823	struct	<code>fixed_type</code> <code><ap_ufixed<W,I,_AP_Q,_AP_O> ></code>	<code>template<int</code> <code>W, int I,</code> <code>ap_q_mode</code> <code>_AP_Q,</code> <code>ap_o_mode</code> <code>_AP_O></code>	—	<code>typedef ap_u</code>
826	struct	<code>x_traits_d_default</code>	<code>tem-</code> <code>plate<typename</code> <code>T1, int DIM></code>	—	<code>typedef T1 A</code> <code>typedef T1 A</code> <code>typedef T1 A</code>
833	struct	<code>x_traits_d</code>	<code>tem-</code> <code>plate<typename</code> <code>T1, int DIM></code>	—	—
836	struct	<code>x_traits_d</code> <code><int,DIM></code>	<code>template<int</code> <code>DIM></code>	<code>:x_traits_d_default<int,</code> <code>DIM></code>	—
839	struct	<code>x_traits_d</code> <code><float,DIM></code>	<code>template<int</code> <code>DIM></code>	<code>:x_traits_d_default<float,</code> <code>DIM></code>	—
842	struct	<code>x_traits_d</code> <code><double,DIM></code>	<code>template<int</code> <code>DIM></code>	<code>:x_traits_d_default<double,</code> <code>DIM></code>	—
845	struct	<code>x_traits_d</code> <code><half,DIM></code>	<code>template<int</code> <code>DIM></code>	<code>:x_traits_d_default<half,</code> <code>DIM></code>	—
848	struct	<code>x_traits_d</code> <code><ap_int<W>,DIM></code>	<code>template<int</code> <code>W, int DIM></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>
855	struct	<code>x_traits_d</code> <code><ap_uint<W>,DIM></code>	<code>template<int</code> <code>W, int DIM></code>	—	<code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code>
862	struct	<code>x_traits_d</code> <code><ap_fixed<W,I,Q,O>,DIM></code>	<code>template<int</code> <code>W, int I,</code> <code>ap_q_mode Q,</code> <code>ap_o_mode O,</code> <code>int DIM></code>	—	<code>typedef</code> <code>ap_fixed<W+B</code> <code>ACCUM_T;</code> <code>typedef</code> <code>+BitWidth<DI</code> <code>typedef</code> <code>25+I+BitWidt</code>
869	struct	<code>x_traits_d</code> <code><ap_ufixed<W,I,Q,O>,DIM></code>	<code>template<int</code> <code>W, int I,</code> <code>ap_q_mode Q,</code> <code>ap_o_mode O,</code> <code>int DIM></code>	—	<code>typedef</code> <code>ap_ufixed<W+</code> <code>ACCUM_T;</code> <code>typedef</code> <code>+BitWidth<DI</code> <code>typedef</code> <code>25+I+BitWidt</code>
883	struct	<code>x_traits_default</code>	<code>tem-</code> <code>plate<typename</code> <code>T></code>	—	<code>typedef T AD</code> <code>typedef T SU</code> <code>typedef T SA</code> <code>typedef T MU</code> <code>typedef T MA</code>
892	struct	<code>x_traits</code>	<code>tem-</code> <code>plate<typename</code> <code>T1, typename</code> <code>T2></code>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedef
907	struct	<code>x_traits</code> <double, double>	<code>template<></code>	<code>: public</code> <code>x_traits_default<double></code>	—
910	struct	<code>x_traits</code> <double, T2>	<code>template<typename</code> <code>T2></code>	<code>: public</code> <code>x_traits_default<double></code>	—
913	struct	<code>x_traits</code> <T1, double>	<code>template<typename</code> <code>T1></code>	<code>: public</code> <code>x_traits_default<double></code>	—
916	struct	<code>x_traits</code> <float, float>	<code>template<></code>	<code>: public</code> <code>x_traits_default<float></code>	—
919	struct	<code>x_traits</code> <float, T2>	<code>template<typename</code> <code>T2></code>	<code>: public</code> <code>x_traits_default<float></code>	—
922	struct	<code>x_traits</code> <T1, float>	<code>template<typename</code> <code>T1></code>	<code>: public</code> <code>x_traits_default<float></code>	—
925	struct	<code>x_traits</code> <half, half>	<code>template<></code>	<code>: public</code> <code>x_traits_default<half></code>	—
928	struct	<code>x_traits</code> <half, T2>	<code>template<typename</code> <code>T2></code>	<code>: public</code> <code>x_traits_default<float></code>	—
931	struct	<code>x_traits</code> <T1, half>	<code>template<typename</code> <code>T1></code>	<code>: public</code> <code>x_traits_default<float></code>	—
934	struct	<code>x_traits</code> <double, float>	<code>template<></code>	<code>: public</code> <code>x_traits_default<double></code>	—
937	struct	<code>x_traits</code> <float, double>	<code>template<></code>	<code>: public</code> <code>x_traits_default<double></code>	—
940	struct	<code>x_traits</code> <double, half>	<code>template<></code>	<code>: public</code> <code>x_traits_default<double></code>	—
943	struct	<code>x_traits</code> <half, double>	<code>template<></code>	<code>: public</code> <code>x_traits_default<double></code>	—
946	struct	<code>x_traits</code> <int, int>	<code>template<></code>	<code>: public</code> <code>x_traits_default<int></code>	—
949	struct	<code>x_traits</code> <unsigned int, unsigned int>	<code>template<></code>	<code>: public</code> <code>x_traits_default<unsigned</code> <code>int></code>	—
952	struct	<code>x_traits</code> <int64_t, int64_t>	<code>template<></code>	<code>: public</code> <code>x_traits_default<int64_t></code>	—
954	class	<code>MAX</code>	<code>template<int</code> <code>A, int B></code>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
960	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2, int</code> <code>I2, ap_q_mode</code> <code>Q2, ap_o_mode</code> <code>O2></code>	—	<code>typedef type</code> <code>RType;</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef</code> <code>I1>::value+1</code> <code>... and 2 more (A</code>
976	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_ufixed<W2,I2,Q2,O2></code> <code>></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2, int</code> <code>I2, ap_q_mode</code> <code>Q2, ap_o_mode</code> <code>O2></code>	—	<code>typedef</code> <code>RType<W2, I2,</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code>
984	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_fixed<W2,I2,Q2,O2></code> <code>></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2, int</code> <code>I2, ap_q_mode</code> <code>Q2, ap_o_mode</code> <code>O2></code>	—	<code>typedef</code> <code>RType<W2, I2,</code> <code>typedef ap_f</code> <code>typedef ap_f</code> <code>typedef ap_f</code>
992	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_ufixed<W2,I2,Q2,O2></code> <code>></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2, int</code> <code>I2, ap_q_mode</code> <code>Q2, ap_o_mode</code> <code>O2></code>	—	<code>typedef</code> <code>RType<W2, I2,</code> <code>typedef ap_u</code> <code>typedef ap_f</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef ap_u</code> <code>typedef</code> <code>I1>::value+1</code> <code>... and 2 more (A</code>
1008	struct	<code>x_traits</code> <code><ap_int<W1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>
1016	struct	<code>x_traits</code> <code><ap_int<W1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>
1023	struct	<code>x_traits</code> <code><ap_uint<W1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
1030	struct	<code>x_traits</code> <code><ap_uint<W1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int W2></code>	—	<code>typedef ap_u</code> <code>typedef ap_i</code> <code>typedef ap_u</code>
1037	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,I1>,</code> <code>ap_fixed<W2,W2> ></code>	—
1040	struct	<code>x_traits</code> <code><ap_fixed<W1,I1,Q1,O1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,I1>,</code> <code>ap_ufixed<W2,W2> ></code>	—
1043	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_int<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,I1>,</code> <code>ap_fixed<W2,W2> ></code>	—
1046	struct	<code>x_traits</code> <code><ap_ufixed<W1,I1,Q1,O1>,ap_uint<W2> ></code>	<code>template<int</code> <code>W1, int I1,</code> <code>ap_q_mode Q1,</code> <code>ap_o_mode O1,</code> <code>int W2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,I1>,</code> <code>ap_ufixed<W2,W2> ></code>	—
1049	struct	<code>x_traits</code> <code><ap_int<W1>, ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,W1>,</code> <code>ap_fixed<W2,I2> ></code>	—
1052	struct	<code>x_traits</code> <code><ap_int<W1>, ap_ufixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_fixed<W1,W1>,</code> <code>ap_ufixed<W2,I2> ></code>	—
1055	struct	<code>x_traits</code> <code><ap_uint<W1>, ap_fixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,W1>,</code> <code>ap_fixed<W2,I2> ></code>	—
1058	struct	<code>x_traits</code> <code><ap_uint<W1>, ap_ufixed<W2,I2,Q2,O2> ></code>	<code>template<int</code> <code>W1, int W2,</code> <code>int I2,</code> <code>ap_q_mode Q2,</code> <code>ap_o_mode O2></code>	<code>:</code> <code>public</code> <code>x_traits<ap_ufixed<W1,W1>,</code> <code>ap_ufixed<W2,I2> ></code>	—
1063	struct	<code>fixed_type</code>	<code>tem-</code> <code>plate<typename</code> <code>T></code>	—	—
1064	struct	<code>fixed_type</code> <code><char></code>	<code>template<></code>	—	<code>typedef ap_f</code>

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
1065	struct	<code>fixed_type</code> <unsigned char>	<code>template<></code>	—	<code>typedef ap_u</code>
1066	struct	<code>fixed_type</code> <short>	<code>template<></code>	—	<code>typedef ap_f</code>
1067	struct	<code>fixed_type</code> <unsigned short>	<code>template<></code>	—	<code>typedef ap_u</code>
1068	struct	<code>fixed_type</code> <int>	<code>template<></code>	—	<code>typedef ap_f</code>
1069	struct	<code>fixed_type</code> <unsigned int>	<code>template<></code>	—	<code>typedef ap_u</code>
1070	struct	<code>fixed_type</code> <float>	<code>template<></code>	—	<code>typedef ap_u</code>
1071	struct	<code>fixed_type</code> <ap_int<W> >	<code>template<int W></code>	—	<code>typedef ap_f</code>
1072	struct	<code>fixed_type</code> <ap_uint<W> >	<code>template<int W></code>	—	<code>typedef ap_u</code>
1073	struct	<code>fixed_type</code> <ap_fixed<W,I, _AP_Q, _AP_O> >	<code>template<int W, int I, ap_q_mode _AP_Q, ap_o_mode _AP_O></code>	—	<code>typedef ap_f</code>
1074	struct	<code>fixed_type</code> <ap_ufixed<W,I, _AP_Q, _AP_O> >	<code>template<int W, int I, ap_q_mode _AP_Q, ap_o_mode _AP_O></code>	—	<code>typedef ap_u</code>
1077	struct	<code>x_traits_d_default</code>	<code>template<typename T1, int DIM></code>	—	<code>typedef T1 A</code> <code>typedef T1 A</code> <code>typedef T1 A</code>
1084	struct	<code>x_traits_d</code>	<code>template<typename T1, int DIM></code>	—	—
1087	struct	<code>x_traits_d</code> <int,DIM>	<code>template<int DIM></code>	<code>:x_traits_d_default<int, DIM></code>	—
1090	struct	<code>x_traits_d</code> <float,DIM>	<code>template<int DIM></code>	<code>:x_traits_d_default<float, DIM></code>	—
1093	struct	<code>x_traits_d</code> <double,DIM>	<code>template<int DIM></code>	<code>:x_traits_d_default<double, DIM></code>	—
1096	struct	<code>x_traits_d</code> <half,DIM>	<code>template<int DIM></code>	<code>:x_traits_d_default<half, DIM></code>	—
1099	struct	<code>x_traits_d</code> <ap_int<W>,DIM>	<code>template<int W, int DIM></code>	—	<code>typedef ap_i</code> <code>typedef ap_i</code> <code>typedef ap_i</code>

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
1106	struct	x_traits_d <ap_uint<W>,DIM>	template<int W, int DIM>	—	typedef ap_u typedef ap_u typedef ap_u
1113	struct	x_traits_d <ap_fixed<W,I,Q,O>,DIM>	template<int W, int I, ap_q_mode Q, ap_o_mode 0, int DIM>	—	typedef ap_fixed<W+B ACCUM_T; typedef +BitWidth<DI typedef 25+I+BitWidt
1120	struct	x_traits_d <ap_ufixed<W,I,Q,O>,DIM>	template<int W, int I, ap_q_mode Q, ap_o_mode 0, int DIM>	—	typedef ap_ufixed<W+ ACCUM_T; typedef +BitWidth<DI typedef 25+I+BitWidt

utils/x_hls_utils.h

Table 183: Every class and struct in utils/x_hls_utils.h (61 definitions)

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
81	class	numeric_limits	template<typename T>	—	—
89	class	numeric_limits <ap_fixed<W,I,Q,O> >	template <int W, int I, ap_q_mode Q, ap_o_mode 0>	—	—
97	class	numeric_limits <ap_ufixed<W,I,Q,O> >	template <int W, int I, ap_q_mode Q, ap_o_mode 0>	—	—
105	class	numeric_limits <ap_int<W> >	template <int W>	—	—
113	class	numeric_limits <ap_uint<W> >	template <int W>	—	—
125	class	numeric_limits	template<typename T>	—	—
133	class	numeric_limits <ap_fixed<W,I,Q,O> >	template <int W, int I, ap_q_mode Q, ap_o_mode 0>	—	—
141	class	numeric_limits <ap_ufixed<W,I,Q,O> >	template <int W, int I, ap_q_mode Q, ap_o_mode 0>	—	—
149	class	numeric_limits <ap_int<W> >	template <int W>	—	—
157	class	numeric_limits <ap_uint<W> >	template <int W>	—	—
208	class	fp_struct	template <typename T>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
218	class	fp_struct <float>	template <>	—	typedef uint32_t int- type; typedef ap_uint<32> data_type;
290	class	fp_struct <double>	template <>	—	typedef uint64_t int- type; typedef ap_uint<64> data_type;
367	class	fp_struct <half>	template <>	—	typedef uint16_t int- type; typedef ap_uint<16> data_type;
440	class	Power	template < unsigned int _Base, unsigned int _Num >	—	—
447	class	Power < _Base, 0 >	template < unsigned int _Base >	—	—
462	class	BitWidth	template < unsigned int _Num, unsigned int _I=_Num/2>	—	—
469	class	BitWidth <_Num, 0>	template <unsigned int _Num>	—	—
484	class	Unsigned- BitWidth	template < unsigned int _Num, unsigned int _I=_Num/2>	—	—
491	class	Unsigned- BitWidth <_Num, 0>	template <unsigned int _Num>	—	—
505	class	Type_BitWidth	template < typename T >	—	—
512	class	Type_BitWidth < ap_uint<W> >	template <int W >	—	—
519	class	Type_BitWidth < ap_int<W> >	template < int W >	—	—
526	class	Type_BitWidth < ap_ufixed<W, I> >	template < int W, int I >	—	—
533	class	Type_BitWidth < ap_fixed<W, I> >	template < int W, int I >	—	—
553	class	Table	template <typename _T, int _Num, int _I=_Num-1>	: public Table<_T, _Num, _I-1>	typedef type- name _T::Table- Type TableType;

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
563	class	Table <_T, _Num, 0>	template <typename _T, int _Num>	—	—
596	struct	is_fptype	template <class T>	—	—
597	struct	is_fptype <float>	template <>	—	—
598	struct	is_fptype <double>	template <>	—	—
599	struct	is_fptype <half>	template <>	—	—
602	struct	is_integral- type	template <class T>	—	—
603	struct	is_integral- type <int>	template <>	—	—
604	struct	is_integral- type <unsigned int>	template <>	—	—
605	struct	is_integral- type <char>	template <>	—	—
606	struct	is_integral- type <signed char>	template <>	—	—
607	struct	is_integral- type <unsigned char>	template <>	—	—
608	struct	is_integral- type <short>	template <>	—	—
609	struct	is_integral- type <unsigned short>	template <>	—	—
610	struct	is_integral- type <long>	template <>	—	—
611	struct	is_integral- type <unsigned long>	template <>	—	—
612	struct	is_integral- type <long long>	template <>	—	—
613	struct	is_integral- type <unsigned long long>	template <>	—	—
614	struct	is_integral- type <ap_int<W> >	template <int W>	—	—

Line	Kind	Name / specialisation	Template parameters	Bases	Member typedefs
615	struct	is_integral_type <ap_uint<W> >	template <int W>	—	—
618	struct	is_fixedtype	template <class T>	—	—
619	struct	is_fixedtype <ap_fixed<W,I,Q,O> >	template <int W, int I, ap_q_mode Q, ap_o_mode O>	—	—
620	struct	is_fixedtype <ap_ufixed<W,I,Q,O> >	template <int W, int I, ap_q_mode Q, ap_o_mode O>	—	—
624	struct	enable_if	template<bool B, class T = void>	—	—
627	struct	enable_if <true, T>	template<class T>	—	typedef T type;
629	struct	integral_constant	template<typename T, T _v>	—	typedef T value_type; typedef integral_constant<T, _v> type;
644	struct	is_same	template<typename T1, typename T2>	: public false_type	—
647	struct	is_same <T1,T1>	template<typename T1>	: public true_type	—
650	struct	is_arithmetic	template<typename T>	: public integral_constant<bool, (is_integral_type<T>::value is_fptype<T>::value)>	—
654	struct	enable_or	template<typename T1, typename T2>	—	—
657	struct	enable_and	template<typename T1, typename T2>	—	—
662	struct	__promote	template<typename T, bool = is_integral_type<T>::value>	—	typedef double type;
665	struct	__promote <T, false>	template<typename T>	—	—
668	struct	__promote <double>	template<>	—	typedef double type;
671	struct	__promote <float>	template<>	—	typedef float type;
674	struct	__promote <half>	template<>	—	typedef half type;

Table 184: Every function definition in utils/x_hls_utils.h (3 definitions)

Line	Template signature	Returns	Name	Parameters
188	—	static inline float	to_float	float v
194	template<int _W, int _I>	float	to_float	ap_fixed<_W, _I> v
634	template<typename T, T _v>	operator	value_type	()

Appendix F Vendor Header Index and Provenance

Each vendor header, where it came from, and what depends on it. The fifth row of Table 3 is repeated here as a caution: two of these files came from a third-party mirror rather than from a vendor repository, and that is a weaker provenance than the rest.

Table 185: Vendor headers, provenance, and direct dependents (51)

Header	Lines	Provenance	Included directly by
ap_axi_sdata.h	344	Xilinx public repository	streamtools.h
ap_common.h	377	Xilinx public repository	<i>transitive only</i>
ap_decl.h	213	Xilinx public repository	<i>transitive only</i>
ap_fixed.h	361	Xilinx public repository	cholesky.hpp, svd.hpp, x_matrix_utils.hpp
ap_fixed_base.h	2,355	Xilinx public repository	<i>transitive only</i>
ap_fixed_ref.h	719	Xilinx public repository	<i>transitive only</i>
ap_fixed_special.h	231	Xilinx public repository	<i>transitive only</i>
ap_int.h	331	Xilinx public repository	aes.hpp, bnn-library.h, convlayer.h, dma.h, interpret.hpp, sha224_256.hpp, up-sample.hpp, utils.hpp, weights.hpp
ap_int_base.h	1,886	Xilinx public repository	<i>transitive only</i>
ap_int_ref.h	1,347	Xilinx public repository	<i>transitive only</i>
ap_int_special.h	224	Xilinx public repository	<i>transitive only</i>
etc/ap_decl.h	202	Xilinx public repository	<i>transitive only</i>
etc/ap_private.h	7,200	Xilinx public repository	<i>transitive only</i>
etc/hls_all_any_apfixed.h	103	Xilinx public repository	<i>transitive only</i>

Header	Lines	Provenance	Included directly by
etc/hls_cbrt_apfixed.h	142	Xilinx public repository	<i>transitive only</i>
etc/hls_comparison_apfixed.h	208	Xilinx public repository	<i>transitive only</i>
etc/hls_cordic_apfixed.h	1,605	Xilinx public repository	<i>transitive only</i>
etc/hls_diff_apfixed.h	283	Xilinx public repository	<i>transitive only</i>
etc/hls_erf_apfixed.h	661	Xilinx public repository	<i>transitive only</i>
etc/hls_exp_apfixed.h	3,447	Xilinx public repository	<i>transitive only</i>
etc/hls_fmod_rem_quo_remainder_divide_apfixed.h	205	Xilinx public repository	<i>transitive only</i>
etc/hls_fract_apfixed.h	43	Xilinx public repository	<i>transitive only</i>
etc/hls_frexp_apfixed.h	104	Xilinx public repository	<i>transitive only</i>
etc/hls_half_fpo.h	610	Xilinx public repository	<i>transitive only</i>
etc/hls_half_x_utils.h	626	Xilinx public repository	<i>transitive only</i>
etc/hls_hotbm_apfixed.h	483	Xilinx public repository	<i>transitive only</i>
etc/hls_hypot_apfixed.h	207	Xilinx public repository	<i>transitive only</i>
etc/hls_isequal_isnotequal_apfixed.h	146	Xilinx public repository	<i>transitive only</i>
etc/hls_ldexp_apfixed.h	102	Xilinx public repository	<i>transitive only</i>
etc/hls_lgamma_apfixed.h	67	Xilinx public repository	<i>transitive only</i>

Header	Lines	Provenance	Included directly by
etc/hls_log_apfixed.h	1,449	Xilinx public repository	<i>transitive only</i>
etc/hls_mad_apfixed.h	65	Xilinx public repository	<i>transitive only</i>
etc/hls_modf_apfixed.h	60	Xilinx public repository	<i>transitive only</i>
etc/hls_pow_apfixed.h	544	Xilinx public repository	<i>transitive only</i>
etc/hls_round_copysign_apfixed.h	541	Xilinx public repository	<i>transitive only</i>
etc/hls_select_bitselect_apfixed.h	80	Xilinx public repository	<i>transitive only</i>
etc/hls_sqrt_apfixed.h	441	Xilinx public repository	<i>transitive only</i>
etc/hls_tgamma_apfixed.h	47	Xilinx public repository	<i>transitive only</i>
floating_point_v7_1_bitacc_cmodel.h	404	Xilinx public repository	<i>transitive only</i>
gmp.h	2,392	third-party mirror	<i>transitive only</i>
hls_compat.h	10	Xilinx public repository	<i>transitive only</i>
hls_fpo.h	620	Xilinx public repository	<i>transitive only</i>
hls_half.h	156	Xilinx public repository	<i>transitive only</i>
hls_math.h	3,210	Xilinx public repository	math_helper.hpp, qrf.hpp, x_matrix_utils.hpp
hls_stream.h	531	Xilinx public repository	activations.hpp, aes.hpp, bnn-library.h, cholesky.hpp, convlayer.h, dma.h, mmv.hpp, mvau.hpp, qrf.hpp, sha224_256.hpp, svd.hpp, tmrcheck.hpp, upsample.hpp, vvau.hpp
hls_x_complex.h	375	Xilinx public repository	cholesky.hpp, qrf.hpp, svd.hpp, x_matrix_utils.hpp

Header	Lines	Provenance	Included directly by
mpfr.h	946	third-party mirror	<i>transitive only</i>
utils/std_complex_utils.h	152	Xilinx public repository	cholesky.hpp, qrf.hpp, svd.hpp, x_matrix_utils.hpp
utils/x_hls_defines.h	138	Xilinx public repository	<i>transitive only</i>
utils/x_hls_traits.h	1,133	Xilinx public repository	<i>transitive only</i>
utils/x_hls_utils.h	685	Xilinx public repository	x_matrix_utils.hpp

Appendix D Complete Corpus Source Listings

Every line of every corpus file, reproduced verbatim from disk. Line numbers are the real line numbers in the file, so a reference of the form `aes.hpp:317` given anywhere in this document can be followed directly to the line below. Nothing has been elided, reformatted or summarised; licence headers are reproduced with the code they govern.

D.1 Vitis_dsp_fft/vt_fft.hpp (25 lines, 856 B)

```

1  /*
2  * Copyright (C) 2019-2022, Xilinx, Inc.
3  * Copyright (C) 2022-2025, Advanced Micro Devices, Inc.
4  *
5  * Licensed under the Apache License, Version 2.0 (the "Li-
6  * cense");
7  * you may not use this file except in compliance with the
8  * license.
9  * You may obtain a copy of the License at
10 *
11 * http://www.apache.org/licenses/LICENSE-2.0
12 *
13 * Unless required by applicable law or agreed to in writing,
14 * software
15 * distributed under the License is distributed on an "AS IS"
16 * BASIS,
17 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18 * press or implied.
19 * See the License for the specific language governing per-
20 * missions and
21 * limitations under the License.
22 */
23 //===== End Lic
24 =====
25 #ifndef _VT_FFT_H_
26 #define _VT_FFT_H_
27 #include "vitis_fft/hls_ssr_fft.hpp"
28 #endif // _VT_FFT_H_

```

D.2 Vitis_security/aes.hpp (1,010 lines, 45,200 B)

```

1  #ifndef _XF_SECURITY_AES_HPP_
2  #define _XF_SECURITY_AES_HPP_
3
4  #include <ap_int.h>
5  #include <hls_stream.h>
6  #ifndef __SYNTHESIS__
7  #include <iostream>
8  #endif
9  namespace xf {
10 namespace security {
11
12 class aesTable {
13 public:
14     aesTable() {
15 #pragma HLS inline
16 #pragma HLS resource variable = ssbox core = ROM_nP_LUTRAM
17 #pragma HLS resource variable = iibox core = ROM_nP_LUTRAM
18     }
19
20     const ap_uint<8> ssbox[256] = {
21
22         0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30,
23         0x1, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76, 0xca, 0x82,
24
25         0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2,
26         0xaf, 0x9c, 0xa4, 0x72, 0xc0, 0xb7, 0xfd, 0x93, 0x26,
27
28         0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71,
29         0xd8, 0x31, 0x15, 0x4, 0xc7, 0x23, 0xc3, 0x18, 0x96,
30
31         0x5, 0x9a, 0x7, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2,
32         0x75, 0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0,
33
34         0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84, 0x53,
35         0xd1, 0x0, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb,
36
37         0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf, 0xd0, 0xef, 0xaa,
38         0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x2, 0x7f,
39
40         0x50, 0x3c, 0x9f, 0xa8, 0x51, 0xa3, 0x40, 0x8f, 0x92,
41         0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff,
42
43         0xf3, 0xd2, 0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44,
44         0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
45
46         0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46,
47         0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb, 0xe0, 0x32,

```

```

30         0x3a, 0x0a, 0x49, 0x6, 0x24, 0x5c, 0xc2, 0xd3, 0xac,
31         0x62, 0x91, 0x95, 0xe4, 0x79, 0xe7, 0xc8, 0x37, 0x6d,
32
33         0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65,
34         0x7a, 0xae, 0x08, 0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6,
35
36         0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
37         0x8a, 0x70, 0x3e, 0xb5, 0x66, 0x48, 0x3, 0xf6, 0x0e,
38
39         0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e, 0xe1,
40         0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e,
41
42         0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf, 0x8c, 0xa1, 0x89,
43         0xd, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0xf,
44         0xb0, 0x54, 0xbb, 0x16;
45     const ap_uint<8> iibox[256] = {
46
47         0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf,
48         0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb, 0x7c, 0xe3,
49
50         0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43,
51         0x44, 0xc4, 0xde, 0xe9, 0xcb, 0x54, 0x7b, 0x94, 0x32,
52
53         0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42,
54         0xfa, 0xc3, 0x4e, 0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9,
55
56         0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1,
57         0x25, 0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16,
58
59         0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65, 0xb6, 0x92, 0x6c,
60         0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15,
61
62         0x46, 0x57, 0xa7, 0x8d, 0x9d, 0x84, 0x90, 0xd8, 0xab,
63         0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58, 0x05,
64
65         0xb8, 0xb3, 0x45, 0x06, 0xd0, 0x2c, 0x1e, 0x8f, 0xca,
66         0x3f, 0xf, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13,
67
68         0x8a, 0x6b, 0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc,
69         0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6, 0x73,
70
71         0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2,
72         0xf9, 0x37, 0xe8, 0x1c, 0x75, 0xdf, 0x6e, 0x47, 0xf1,
73
74         0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62,
75         0x0e, 0xaa, 0x18, 0xbe, 0x1b, 0xfc, 0x56, 0x03e, 0x4b,
76
77         0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78,
78         0xcd, 0x5a, 0xf4, 0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07,
79
80         0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec,
81         0x5f, 0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0xd,
82
83         0x2d, 0xe5, 0x7a, 0x9f, 0x93, 0xc9, 0x9c, 0xef, 0xa0,
84         0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb,
85
86         0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61, 0x17, 0x2b, 0x04,
87         0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14, 0x63,
88         0x55, 0x21, 0x0c, 0x7d;
89     };
90
91     template <int W>
92     class aesEnc {
93     public:
94         void updateKey(ap_uint<W> cipherkey) {}
95         void process(ap_uint<128> plaintext, ap_uint<W> cipher-
96         key, ap_uint<128> ciphertext) {}
97     };
98
99     template <>
100     class aesEnc<256> : public aesTable {
101     public:
102         ap_uint<128> key_list[16];
103
104         aesEnc() {
105 #pragma HLS inline
106 #pragma HLS ARRAY_PARTITION variable = key_list complete dim
107 = 1
108         }
109
110         void updateKey(ap_uint<256> cipherkey) {
111 #pragma HLS inline off
112             const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
113             0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
114             ap_uint<256> lastRound = cipherkey;
115             key_list[0] = lastRound.range(127, 0);
116             key_list[1] = lastRound.range(255, 128);

```

```

77     for (ap_uint<5> iter = 2; iter < 15; iter += 2) {
78 #pragma HLS pipeline II = 1
79     // #pragma HLS pipeline II = 1
80     ap_uint<128> currRound_0, currRound_1;
81
82     ap_uint<32> round_tmp = lastRound.range(255,
224);
83     round_tmp = (round_tmp >> 8) | (round_tmp << 24);
84
85     round_tmp.range(7, 0) = ssbox[round_tmp.range(7,
0)] ^ Rcon[(iter >> 1) - 1];
86     round_tmp.range(15, 8) = ss-
box[round_tmp.range(15, 8)];
87     round_tmp.range(23, 16) = ss-
box[round_tmp.range(23, 16)];
88     round_tmp.range(31, 24) = ss-
box[round_tmp.range(31, 24)];
89
90     currRound_0.range(31, 0) = lastRound.range(31, 0) ^
round_tmp;
91     currRound_0.range(63, 32) = lastRound.range(63,
32) ^ currRound_0.range(31, 0);
92     currRound_0.range(95, 64) = lastRound.range(95,
64) ^ currRound_0.range(63, 32);
93     currRound_0.range(127, 96) = lastRound.range(127,
96) ^ currRound_0.range(95, 64);
94
95     ap_uint<32> round_tmp2 = currRound_0.range(127,
96);
96
97     round_tmp2.range(7, 0) = ss-
box[round_tmp2.range(7, 0)];
98     round_tmp2.range(15, 8) = ss-
box[round_tmp2.range(15, 8)];
99     round_tmp2.range(23, 16) = ss-
box[round_tmp2.range(23, 16)];
100    round_tmp2.range(31, 24) = ss-
box[round_tmp2.range(31, 24)];
101
102    currRound_1.range(31, 0) = lastRound.range(159,
128) ^ round_tmp2;
103    currRound_1.range(63, 32) = lastRound.range(191,
160) ^ currRound_1.range(31, 0);
104    currRound_1.range(95, 64) = lastRound.range(223,
192) ^ currRound_1.range(63, 32);
105
106    currRound_1.range(127, 96) = lastRound.range(255,
224) ^ currRound_1.range(95, 64);
107    lastRound.range(127, 0) = currRound_0;
108    lastRound.range(255, 128) = currRound_1;
109    key_list[iter] = currRound_0;
110    key_list[iter + 1] = currRound_1;
111    }
112
113    // the function of Finite field multiplication *2
114    static ap_uint<8> GFMul2(ap_uint<8> a) {
115    ap_uint<8> kModulus = 0b00011011;
116
117    if (a[7] == 1) {
118    a = (a << 1) ^ kModulus;
119    } else {
120    a <<= 1;
121    }
122
123    return a;
124    }
125
126    void process(ap_uint<128> plaintext, ap_uint<256> cipher-
key, ap_uint<128> & ciphertext) {
127    ap_uint<128> state, state_1;
128    ap_uint<8> tmp_1, tmp_2_1, tmp_2_2, tmp_3;
129    ap_uint<4> round_counter;
130
131    // state init and add roundkey[0]
132    state = plaintext ^ key_list[0];
133
134    // Start 14 rounds of process
135    for (round_counter = 1; round_counter <= 14;
round_counter++) {
136    // SubByte
137    for (int i = 0; i < 16; i++) {
138 #pragma HLS unroll
139
state(i * 8 + 7, i * 8) = ssbox[state(i * 8 +
7, i * 8)];
140    }
141    // ShiftRow
142    tmp_1 = state(15, 8);
143    state(15, 8) = state(47, 40);
144    state(47, 40) = state(79, 72);
145    state(79, 72) = state(111, 104);
146    state(111, 104) = tmp_1;
147
148    tmp_2_1 = state(23, 16);
149    state(23, 16) = state(87, 80);
150    state(87, 80) = tmp_2_1;
151

```

```

152    tmp_2_2 = state(55, 48);
153    state(55, 48) = state(119, 112);
154    state(119, 112) = tmp_2_2;
155
156    tmp_3 = state(127, 120);
157    state(127, 120) = state(95, 88);
158    state(95, 88) = state(63, 56);
159    state(63, 56) = state(31, 24);
160    state(31, 24) = tmp_3;
161
162    // MixColumn
163    for (int i = 0; i < 16; i++) {
164 #pragma HLS unroll
165
ap_uint<8> tmp = state(i * 8 + 7, i * 8);
166    state_1(i * 8 + 7, i * 8) = state(i * 8 + 7,
i * 8);
167    }
168
169    if (round_counter < 14) {
170    // clang-format off
171    for (int i = 0; i < 4; i++) {
172 #pragma HLS unroll
173
state(i * 32 + 7, i * 32)
=GFMul2(state_1(i * 32 + 7, i * 32)) ^
174
GFMul2(state_1(i * 32 + 15, i * 32 + 8))^
175
state_1(i * 32 +
15, i * 32 + 8)^
176
state_1(i * 32 +
23, i * 32 + 16) ^
177
state_1(i * 32 +
31, i * 32 + 24);
178
state(i * 32 + 15, i * 32 + 8) =
state_1(i * 32 + 7, i * 32) ^
179
GFMul2(state_1(i * 32 + 15, i * 32 + 8)) ^
180
GFMul2(state_1(i * 32 + 23, i * 32 + 16)) ^
181
state_1(i * 32 + 23, i * 32 + 16)^
182
state_1(i * 32 + 31, i * 32 + 24);
183
state(i * 32 + 23, i * 32 + 16) =
state_1(i * 32 + 7, i * 32) ^
184
state_1(i * 32 + 15, i * 32 + 8) ^
185
GFMul2(state_1(i * 32 + 23, i * 32 + 16)) ^
186
GFMul2(state_1(i * 32 + 31, i * 32 + 24)) ^
187
state_1(i * 32 + 31, i * 32 + 24);
188
state(i * 32 + 31, i * 32 + 24) =
GFMul2(state_1(i * 32 + 7, i * 32))^
189
state_1(i * 32 + 7, i * 32) ^
190
state_1(i * 32 + 15, i * 32 + 8) ^
191
state_1(i * 32 + 23, i * 32 + 16) ^
192
GFMul2(state_1(i * 32 + 31, i * 32 + 24));
193
}
194    // clang-format on
195    } else {
196    state = state_1;
197    }
198
199    state ^= key_list[round_counter];
200    }
201    ciphertext = state;
202    }
203
204    void updateKey_2(ap_uint<256> cipherkey) {
205 #pragma HLS inline off
206    const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
207    ap_uint<128> lastRound[2];
208    key_list[0] = cipherkey.range(127, 0);
209    key_list[1] = cipherkey.range(255, 128);
210    lastRound[0] = cipherkey.range(127, 0);
211    lastRound[1] = cipherkey.range(255, 128);
212    for (ap_uint<5> iter = 2; iter < 15; iter++) {
213 #pragma HLS pipeline II = 1
214
ap_uint<32> round_tmp = lastRound[ap_uint<1>(1) -
iter[0]].range(127, 96);
215
if (iter[0] == ap_uint<1>(0)) {
216
round_tmp = (round_tmp >> 8) | (round_tmp <<
24);
217
}
218
round_tmp.range(7, 0) = ssbox[round_tmp.range(7,
0)];
219
round_tmp.range(15, 8) = ss-
box[round_tmp.range(15, 8)];
220
round_tmp.range(23, 16) = ss-
box[round_tmp.range(23, 16)];
221

```

```

box[round_tmp.range(23, 16)];
222     round_tmp.range(31, 24) = ss-
box[round_tmp.range(31, 24)];
223
224     if (iter[0] == ap_uint<1>(0)) {
225         round_tmp.range(7, 0) ^= Rcon[(iter >> 1) -
1];
226     }
227
228     ap_uint<128> tmp_key;
229     tmp_key.range(31, 0) =
lastRound[iter[0]].range(31, 0) ^ round_tmp;
230     tmp_key.range(63, 32) =
lastRound[iter[0]].range(63, 32) ^ tmp_key.range(31, 0);
231     tmp_key.range(95, 64) =
lastRound[iter[0]].range(95, 64) ^ tmp_key.range(63, 32);
232     tmp_key.range(127, 96) =
lastRound[iter[0]].range(127, 96) ^ tmp_key.range(95, 64);
233
234     key_list[iter] = tmp_key;
235     lastRound[iter[0]] = tmp_key;
236 }
237 }
238 };
239
240 template <>
241 class aesEnc<192> : public aesTable {
242 public:
243     ap_uint<128> key_list[14];
244
245     aesEnc() {
246 #pragma HLS inline
247 #pragma HLS ARRAY_PARTITION variable = key_list complete dim
= 1
248     }
249
250     void updateKey(ap_uint<192> cipherkey) {
251 #pragma HLS inline off
252         const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
253         ap_uint<192> lastRound = cipherkey;
254         key_list[0] = lastRound.range(127, 0);
255         key_list[1].range(63, 0) = lastRound.range(191, 128);
256         ap_uint<4> rIter = 0;
257
258         for (ap_uint<5> iter = 3; iter < 26; iter += 3) {
259 #pragma HLS pipeline II = 1
260
261             ap_uint<192> thisRound = 0;
262
263             ap_uint<32> round_tmp = lastRound.range(191,
160);
264             round_tmp = (round_tmp >> 8) | (round_tmp << 24);
265
266             round_tmp.range(7, 0) = ssbox[round_tmp.range(7,
0)] ^ Rcon[rIter++];
267             round_tmp.range(15, 8) = ss-
box[round_tmp.range(15, 8)];
268             round_tmp.range(23, 16) = ss-
box[round_tmp.range(23, 16)];
269             round_tmp.range(31, 24) = ss-
box[round_tmp.range(31, 24)];
270
271             thisRound.range(31, 0) = lastRound.range(31, 0) ^
round_tmp;
272
273             thisRound.range(63, 32) = lastRound.range(63, 32) ^
thisRound.range(31, 0);
274
275             thisRound.range(95, 64) = lastRound.range(95, 64) ^
thisRound.range(63, 32);
276             thisRound.range(127, 96) = lastRound.range(127,
96) ^ thisRound.range(95, 64);
277             thisRound.range(159, 128) = lastRound.range(159,
128) ^ thisRound.range(127, 96);
278             thisRound.range(191, 160) = lastRound.range(191,
160) ^ thisRound.range(159, 128);
279
280             if (iter[0] == 1) {
281                 key_list[iter.range(4, 1)].range(127, 64) =
thisRound.range(63, 0);
282                 key_list[iter.range(4, 1) + 1].range(127, 0)
= thisRound.range(191, 64);
283             } else {
284                 key_list[iter.range(4, 1)].range(127, 0) =
thisRound.range(127, 0);
285
286                 key_list[iter.range(4, 1) + 1].range(63, 0) =
thisRound.range(191, 128);
287             }
288             lastRound = thisRound;
289         }
290     }
291
292     // the function of Finite field multiplication *2
293     static ap_uint<8> GFmul2(ap_uint<8> a) {
294         ap_uint<8> kModulus = 0b00011011;
295     }

```

```

294         if (a[7] == 1) {
295             a = (a << 1) ^ kModulus;
296         } else {
297             a <<= 1;
298         }
299     }
300     return a;
301 }
302
303 void process(ap_uint<128> plaintext, ap_uint<192> cipher-
key, ap_uint<128>& ciphertext) {
304     ap_uint<128> state, state_1;
305     ap_uint<8> tmp_1, tmp_2_1, tmp_2_2, tmp_3;
306     ap_uint<4> round_counter;
307
308     // state init and add roundkey[0]
309     state = plaintext ^ key_list[0];
310
311     // Start 14 rounds of process
312     for (round_counter = 1; round_counter <= 12;
round_counter++) {
313         // SubByte
314         for (int i = 0; i < 16; i++) {
315 #pragma HLS unroll
316
317             state(i * 8 + 7, i * 8) = ssbox[state(i * 8 +
7, i * 8)];
318         }
319         // ShiftRow
320         tmp_1 = state(15, 8);
321         state(15, 8) = state(47, 40);
322         state(47, 40) = state(79, 72);
323         state(79, 72) = state(111, 104);
324         state(111, 104) = tmp_1;
325
326         tmp_2_1 = state(23, 16);
327         state(23, 16) = state(87, 80);
328         state(87, 80) = tmp_2_1;
329
330         tmp_2_2 = state(55, 48);
331         state(55, 48) = state(119, 112);
332         state(119, 112) = tmp_2_2;
333
334         tmp_3 = state(127, 120);
335         state(127, 120) = state(95, 88);
336         state(95, 88) = state(63, 56);
337         state(63, 56) = state(31, 24);
338         state(31, 24) = tmp_3;
339
340         // MixColumn
341         for (int i = 0; i < 16; i++) {
342 #pragma HLS unroll
343             ap_uint<8> tmp = state(i * 8 + 7, i * 8);
344             state_1(i * 8 + 7, i * 8) = state(i * 8 + 7,
i * 8);
345         }
346
347         if (round_counter < 12) {
348             // clang-format off
349             for (int i = 0; i < 4; i++) {
350 #pragma HLS unroll
351                 state(i * 32 + 7, i * 32)
=GFmul2(state_1(i * 32 + 7, i * 32)) ^
GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
state_1(i * 32 +
15, i * 32 + 8) ^
state_1(i * 32 +
23, i * 32 + 16) ^
state_1(i * 32 +
31, i * 32 + 24);
352                 state(i * 32 + 15, i * 32 + 8) =
state_1(i * 32 + 7, i * 32) ^
GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
state_1(i * 32 + 23, i * 32 + 16) ^
state_1(i * 32 + 31, i * 32 + 24);
353                 state(i * 32 + 23, i * 32 + 16) =
state_1(i * 32 + 7, i * 32) ^
state_1(i * 32 + 15, i * 32 + 8) ^
GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
GFmul2(state_1(i * 32 + 31, i * 32 + 24)) ^
state_1(i * 32 + 31, i * 32 + 24);
354                 state(i * 32 + 31, i * 32 + 24) =
GFmul2(state_1(i * 32 + 7, i * 32)) ^
state_1(i * 32 + 7, i * 32) ^
state_1(i * 32 + 15, i * 32 + 8) ^

```

```

368 state_1(i * 32 + 23, i * 32 + 16) ^
369 GFMul2(state_1(i * 32 + 31, i * 32 + 24));
370     }
371     // clang-format on
372     } else {
373         state = state_1;
374     }
375
376     state ^= key_list[round_counter];
377 }
378 ciphertext = state;
379 }
380 };
381
382 template <>
383 class aesEnc<128> : public aesTable {
384 public:
385     ap_uint<128> key_list[11];
386     aesEnc() {
387 #pragma HLS inline
388 #pragma HLS ARRAY_PARTITION variable = key_list complete dim
= 1
389     }
390
391     void updateKey(ap_uint<128> cipherkey) {
392 #pragma HLS inline off
393         const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
394         ap_uint<128> lastRound = cipherkey;
395         key_list[0] = lastRound;
396         for (ap_uint<5> iter = 1; iter < 11; iter++) {
397 #pragma HLS pipeline II = 1
398
399             ap_uint<32> round_tmp = lastRound.range(127, 96);
400             round_tmp = (round_tmp >> 8) | (round_tmp << 24);
401
402             round_tmp.range(7, 0) = ssbox[round_tmp.range(7,
0)] ^ Rcon[iter - 1];
403             round_tmp.range(15, 8) = ss-
box[round_tmp.range(15, 8)];
404             round_tmp.range(23, 16) = ss-
box[round_tmp.range(23, 16)];
405             round_tmp.range(31, 24) = ss-
box[round_tmp.range(31, 24)];
406
407             key_list[iter].range(31, 0) = lastRound.range(31,
0) ^ round_tmp;
408             key_list[iter].range(63, 32) =
lastRound.range(63, 32) ^ key_list[iter].range(31, 0);
409             key_list[iter].range(95, 64) =
lastRound.range(95, 64) ^ key_list[iter].range(63, 32);
410             key_list[iter].range(127, 96) =
lastRound.range(127, 96) ^ key_list[iter].range(95, 64);
411
412             lastRound.range(127, 0) = key_list[iter];
413         }
414     }
415
416     // the function of Finite field multiplication *2
417
418     static ap_uint<8> GFMul2(ap_uint<8> a) {
419         ap_uint<8> kModulus = 0b00011011;
420         if (a[7] == 1) {
421             a = (a << 1) ^ kModulus;
422         } else {
423             a <<= 1;
424         }
425
426         return a;
427     }
428
429     void process(ap_uint<128> plaintext, ap_uint<128> cipher-
key, ap_uint<128>& ciphertext) {
430         ap_uint<128> state, state_1;
431         ap_uint<8> tmp_1, tmp_2_1, tmp_2_2, tmp_3;
432         ap_uint<4> round_counter;
433
434         // state init and add roundkey[0]
435         state = plaintext ^ key_list[0];
436         // std::cout << "the state is " <<state<< std::endl;
437
438         // Start 10 rounds of process
439         for (round_counter = 1; round_counter <= 10;
round_counter++) {
440             // SubByte
441             for (int i = 0; i < 16; i++) {
442 #pragma HLS unroll
443
444                 state(i * 8 + 7, i * 8) = ssbox[state(i * 8 +
7, i * 8)];
445             }
446
447             // ShiftRow
448             tmp_1 = state(15, 8);
449             state(15, 8) = state(47, 40);
450             state(47, 40) = state(79, 72);
451             state(79, 72) = state(111, 104);

```

```

451         state(111, 104) = tmp_1;
452
453         tmp_2_1 = state(23, 16);
454         state(23, 16) = state(87, 80);
455         state(87, 80) = tmp_2_1;
456
457         tmp_2_2 = state(55, 48);
458         state(55, 48) = state(119, 112);
459         state(119, 112) = tmp_2_2;
460
461         tmp_3 = state(127, 120);
462         state(127, 120) = state(95, 88);
463         state(95, 88) = state(63, 56);
464         state(63, 56) = state(31, 24);
465         state(31, 24) = tmp_3;
466
467         // MixColumn
468
469         for (int i = 0; i < 16; i++) {
470 #pragma HLS unroll
471             ap_uint<8> tmp = state(i * 8 + 7, i * 8);
472             state_1(i * 8 + 7, i * 8) = state(i * 8 + 7,
i * 8);
473         }
474         if (round_counter < 10) {
475             // clang-format off
476             for (int i = 0; i < 4; i++) {
477 #pragma HLS unroll
478
479                 state(i * 32 + 7, i * 32)
=GFMul2(state_1(i * 32 + 7, i * 32)) ^
480                 GFMul2(state_1(i * 32 + 15, i * 32 + 8))^
481                 state_1(i * 32 +
15, i * 32 + 8)^
482                 state_1(i * 32 +
23, i * 32 + 16) ^
483                 state_1(i * 32 +
31, i * 32 + 24);
484                 state(i * 32 + 15, i * 32 + 8) =
state_1(i * 32 + 7, i * 32) ^
485                 GFMul2(state_1(i * 32 + 15, i * 32 + 8)) ^
486                 GFMul2(state_1(i * 32 + 23, i * 32 + 16)) ^
487                 state_1(i * 32 + 23, i * 32 + 16)^
488                 state_1(i * 32 + 31, i * 32 + 24);
489                 state(i * 32 + 23, i * 32 + 16) =
state_1(i * 32 + 7, i * 32) ^
490                 state_1(i * 32 + 15, i * 32 + 8) ^
491                 GFMul2(state_1(i * 32 + 23, i * 32 + 16)) ^
492                 GFMul2(state_1(i * 32 + 31, i * 32 + 24)) ^
493                 state_1(i * 32 + 31, i * 32 + 24);
494                 state(i * 32 + 31, i * 32 + 24) =
GFMul2(state_1(i * 32 + 7, i * 32))^
495                 state_1(i * 32 + 7, i * 32) ^
496                 state_1(i * 32 + 15, i * 32 + 8) ^
497                 state_1(i * 32 + 23, i * 32 + 16) ^
498                 GFMul2(state_1(i * 32 + 31, i * 32 + 24));
499             }
500             // clang-format on
501         } else {
502             state = state_1;
503         }
504         state ^= key_list[round_counter];
505     }
506     ciphertext = state;
507 }
508 };
509
510 /**
511  * @brief AES decryption
512  *
513  * @tparam W Bit width of AES key, which is 128, 192 or 256
514  */
515 template <int W>
516 class aesDec {
517 public:
518     /**
519     * @brief Update key before using it to decrypt.
520     *
521     * @param cipherkey Key to be used in decryption.
522     */
523     void updateKey(ap_uint<W> cipherkey) {}
524     /**
525     * @brief Decrypt message using AES algorithm
526     */

```



```

527     * @param ciphertext Cipher text to be decrypted.
528     * @param cipherkey Key to be used in decryption.
529     * @param plaintext Decryption result.
530     */
531     void process(ap_uint<128> ciphertext, ap_uint<W> cipher-
key, ap_uint<128>& plaintext) {}
532 };
533
534 template <>
535 class aesDec<256> : public aesTable {
536     public:
537         ap_uint<128> key_list[16];
538
539         aesDec() {
540             #pragma HLS inline
541             #pragma HLS ARRAY_PARTITION variable = key_list complete dim
= 1
542         }
543
544         void updateKey(ap_uint<256> cipherkey) {
545             #pragma HLS inline off
546             const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
547             ap_uint<256> lastRound = cipherkey;
548             key_list[0] = lastRound.range(127, 0);
549             key_list[1] = lastRound.range(255, 128);
550             for (ap_uint<5> iter = 2; iter < 15; iter += 2) {
551                 #pragma HLS pipeline II = 1
552
553                 ap_uint<32> round_tmp = lastRound.range(255,
224);
554                 round_tmp = (round_tmp >> 8) | (round_tmp << 24);
555
556                 round_tmp.range(7, 0) = ssbox[round_tmp.range(7,
0)] ^ Rcon[(iter >> 1) - 1];
557                 round_tmp.range(15, 8) = ss-
box[round_tmp.range(15, 8)];
558                 round_tmp.range(23, 16) = ss-
box[round_tmp.range(23, 16)];
559                 round_tmp.range(31, 24) = ss-
box[round_tmp.range(31, 24)];
560
561                 key_list[iter].range(31, 0) = lastRound.range(31,
0) ^ round_tmp;
562                 key_list[iter].range(63, 32) =
lastRound.range(63, 32) ^ key_list[iter].range(31, 0);
563                 key_list[iter].range(95, 64) =
lastRound.range(95, 64) ^ key_list[iter].range(63, 32);
564                 key_list[iter].range(127, 96) =
lastRound.range(127, 96) ^ key_list[iter].range(95, 64);
565
566                 ap_uint<32> round_tmp2 = key_l-
ist[iter].range(127, 96);
567
568                 round_tmp2.range(7, 0) = ss-
box[round_tmp2.range(7, 0)];
569                 round_tmp2.range(15, 8) = ss-
box[round_tmp2.range(15, 8)];
570                 round_tmp2.range(23, 16) = ss-
box[round_tmp2.range(23, 16)];
571                 round_tmp2.range(31, 24) = ss-
box[round_tmp2.range(31, 24)];
572
573                 key_list[iter + 1].range(31, 0) =
lastRound.range(159, 128) ^ round_tmp2;
574                 key_list[iter + 1].range(63, 32) =
lastRound.range(191, 160) ^ key_list[iter + 1].range(31, 0);
575                 key_list[iter + 1].range(95, 64) =
lastRound.range(223, 192) ^ key_list[iter + 1].range(63, 32);
576                 key_list[iter + 1].range(127, 96) =
lastRound.range(255, 224) ^ key_list[iter + 1].range(95, 64);
577                 lastRound.range(127, 0) = key_list[iter];
578                 lastRound.range(255, 128) = key_list[iter + 1];
579             }
580         }
581
582         // the function of Finite field multiplication *2 *4 *8
583
584         static ap_uint<8> GFmul2(ap_uint<8> a) {
585             ap_uint<8> kModulus = 0b00011011;
586
587             if (a[7] == 1) {
588                 a = (a << 1) ^ kModulus;
589             } else {
590                 a <<= 1;
591             }
592
593             return a;
594         }
595
596         static ap_uint<8> GFmul4(ap_uint<8> a) {
597             ap_uint<8> kModulus = 0b00011011;
598
599             for (int i = 0; i < 2; ++i) {
600                 if (a[7] == 1) {
601                     a = (a << 1) ^ kModulus;
602                 } else {
603                     a <<= 1;
604                 }

```

```

605     }
606
607     return a;
608 }
609
610 static ap_uint<8> GFmul8(ap_uint<8> a) {
611     ap_uint<8> kModulus = 0b00011011;
612
613     for (int i = 0; i < 3; ++i) {
614         if (a[7] == 1) {
615             a = (a << 1) ^ kModulus;
616         } else {
617             a <<= 1;
618         }
619     }
620
621     return a;
622 }
623
624 void process(ap_uint<128> ciphertext, ap_uint<256>
cipherkey, ap_uint<128>& plaintext) {
625     ap_uint<128> state;
626     ap_uint<4> round_counter;
627
628     // state init and add roundkey[0]
629     state = ciphertext ^ key_list[14];
630
631     // Start 14 rounds of process
632     for (round_counter = 1; round_counter <= 14;
round_counter++) {
633         ap_uint<8> tmp_1, tmp_2_1, tmp_2_2, tmp_3;
634         ap_uint<128> state_1;
635         ap_uint<8> tmp16_s[4] = {0x0, 0x0, 0x0, 0x0};
636         // Inv ShiftRow
637         tmp_1 = state(111, 104);
638         state(111, 104) = state(79, 72);
639         state(79, 72) = state(47, 40);
640         state(47, 40) = state(15, 8);
641         state(15, 8) = tmp_1;
642
643         tmp_2_1 = state(87, 80);
644         state(87, 80) = state(23, 16);
645         state(23, 16) = tmp_2_1;
646
647         tmp_2_2 = state(119, 112);
648         state(119, 112) = state(55, 48);
649         state(55, 48) = tmp_2_2;
650
651         tmp_3 = state(31, 24);
652         state(31, 24) = state(63, 56);
653         state(63, 56) = state(95, 88);
654         state(95, 88) = state(127, 120);
655         state(127, 120) = tmp_3;
656         // Inv SubByte
657         for (int i = 0; i < 16; i++) {
658             #pragma HLS unroll
659
660             state(i * 8 + 7, i * 8) = iibox[state(i * 8 +
7, i * 8)];
661         }
662
663         // Add Round Key
664         state ^= key_list[14 - round_counter];
665
666         // Inverse Mix Column
667         for (int i = 0; i < 16; i++) {
668             state_1(i * 8 + 7, i * 8) = state(i * 8 + 7,
i * 8);
669         }
670         if (round_counter < 14) {
671             // clang-format off
672             for (int i = 0; i < 4; i++) {
673                 #pragma HLS unroll
674
675                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ GFmul4(state_1(i * 32 + 7, i *
32)) ^ GFmul2(state_1(i * 32 + 7, i * 32));
676
677                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 +
15, i * 32 + 8)) ^ state_1(i * 32 + 15, i * 32 + 8) ^
678
679                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ state_1(i * 32 + 23, i * 32 + 16) ^
680
681                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ state_1(i * 32 + 31, i *
32 + 24) ^
682
683                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
684
685                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
686
687                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
688
689                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
690
691                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
692
693                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
694
695                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
696
697                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
698
699                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
700
701                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
702
703                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
704
705                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
706
707                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
708
709                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
710
711                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
712
713                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
714
715                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
716
717                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
718
719                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
720
721                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
722
723                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
724
725                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
726
727                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
728
729                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
730
731                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
732
733                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
734
735                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
736
737                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
738
739                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
740
741                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
742
743                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
744
745                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
746
747                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
748
749                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
750
751                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
752
753                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
754
755                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
756
757                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
758
759                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
760
761                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
762
763                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
764
765                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
766
767                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
768
769                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
770
771                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
772
773                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
774
775                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
776
777                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
778
779                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
780
781                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
782
783                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
784
785                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
786
787                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
788
789                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
790
791                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
792
793                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
794
795                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
796
797                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
798
799                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
800
801                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
802
803                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
804
805                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
806
807                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
808
809                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
810
811                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
812
813                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
814
815                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
816
817                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
818
819                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
820
821                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
822
823                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
824
825                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
826
827                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
828
829                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
830
831                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
832
833                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
834
835                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
836
837                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
838
839                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
840
841                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
842
843                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
844
845                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
846
847                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
848
849                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
850
851                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
852
853                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
854
855                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
856
857                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32);
858
859                 state(i * 32 + 23, i * 32 + 16) =
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ GFmul2(state_1(i * 32 + 23, i * 32 + 16)) ^
860
861                 state(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
862
863                 state(i * 32 + 15, i * 32 + 8) =
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 + 15, i * 32 + 8)) ^
864
865                 state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ state_1(i * 32 + 7, i * 32
```

```

GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul2(state_1(i * 32 +
31, i * 32 + 24)) ^ state_1(i * 32 + 31, i * 32 + 24) ^
683
GFmul8(state_1(i * 32 + 7, i * 32)) ^ GFmul4(state_1(i * 32 + 7, i *
32)) ^ state_1(i * 32 + 7, i * 32) ^
684
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ state_1(i * 32 + 15, i *
32 + 8);
685
state_1(i * 32 + 31, i * 32 + 24) =
GFmul8(state_1(i * 32 + 31, i * 32 + 24)) ^ GFmul4(state_1(i * 32 +
31, i * 32 + 24)) ^ GFmul2(state_1(i * 32 + 31, i * 32 + 24)) ^
686
GFmul8(state_1(i * 32 + 7, i * 32)) ^ GFmul2(state_1(i * 32 + 7, i *
32)) ^ state_1(i * 32 + 7, i * 32) ^
687
GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul4(state_1(i * 32 +
15, i * 32 + 8)) ^ state_1(i * 32 + 15, i * 32 + 8) ^
688
GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ state_1(i * 32 + 23, i *
32 + 16);
689
} // clang-format on
690
}
691
}
692
plaintext = state;
693
}
694
}
695
};
696
697 template <>
698 class aesDec<192> : public aesTable {
699 public:
700     ap_uint<128> key_list[14];
701
702     aesDec() {
703 #pragma HLS inline
704 #pragma HLS ARRAY_PARTITION variable = key_list complete dim
= 1
705     }
706
707     void updateKey(ap_uint<192> cipherkey) {
708 #pragma HLS inline off
709         const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
710         ap_uint<192> lastRound = cipherkey;
711         key_list[0] = lastRound.range(127, 0);
712         key_list[1].range(63, 0) = lastRound.range(191, 128);
713         ap_uint<4> rIter = 0;
714
715         for (ap_uint<5> iter = 3; iter < 26; iter += 3) {
716 #pragma HLS pipeline II = 1
717
718             ap_uint<192> thisRound = 0;
719
720             ap_uint<32> round_tmp = lastRound.range(191,
160);
721             round_tmp = (round_tmp >> 8) | (round_tmp << 24);
722
723             round_tmp.range(7, 0) = ssbox[round_tmp.range(7,
0)] ^ Rcon[rIter++];
724             round_tmp.range(15, 8) = ss-
box[round_tmp.range(15, 8)];
725             round_tmp.range(23, 16) = ss-
box[round_tmp.range(23, 16)];
726             round_tmp.range(31, 24) = ss-
box[round_tmp.range(31, 24)];
727
728             thisRound.range(31, 0) = lastRound.range(31, 0) ^
round_tmp;
729
730             thisRound.range(63, 32) = lastRound.range(63, 32) ^
thisRound.range(31, 0);
731
732             thisRound.range(95, 64) = lastRound.range(95, 64) ^
thisRound.range(63, 32);
733             thisRound.range(127, 96) = lastRound.range(127,
96) ^ thisRound.range(95, 64);
734             thisRound.range(159, 128) = lastRound.range(159,
128) ^ thisRound.range(127, 96);
735             thisRound.range(191, 160) = lastRound.range(191,
160) ^ thisRound.range(159, 128);
736
737             if (iter[0] == 1) {
738                 key_list[iter.range(4, 1)].range(127, 64) =
thisRound.range(63, 0);
739                 key_list[iter.range(4, 1) + 1].range(127, 0) =
thisRound.range(191, 64);
740             } else {
741                 key_list[iter.range(4, 1)].range(127, 0) =
thisRound.range(127, 0);
742
743                 key_list[iter.range(4, 1) + 1].range(63, 0) =
thisRound.range(191, 128);
744             }
745             lastRound = thisRound;
746         }
747     }
748
749     // the function of Finite field multiplication *2 *4 *8
750     static ap_uint<8> GFmul2(ap_uint<8> a) {
751         ap_uint<8> kModulus = 0b00011011;

```

```

748
749         if (a[7] == 1) {
750             a = (a << 1) ^ kModulus;
751         } else {
752             a <<= 1;
753         }
754
755         return a;
756     }
757
758     static ap_uint<8> GFmul4(ap_uint<8> a) {
759         ap_uint<8> kModulus = 0b00011011;
760
761         for (int i = 0; i < 2; ++i) {
762             if (a[7] == 1) {
763                 a = (a << 1) ^ kModulus;
764             } else {
765                 a <<= 1;
766             }
767         }
768
769         return a;
770     }
771
772     static ap_uint<8> GFmul8(ap_uint<8> a) {
773         ap_uint<8> kModulus = 0b00011011;
774
775         for (int i = 0; i < 3; ++i) {
776             if (a[7] == 1) {
777                 a = (a << 1) ^ kModulus;
778             } else {
779                 a <<= 1;
780             }
781         }
782
783         return a;
784     }
785
786     void process(ap_uint<128> ciphertext, ap_uint<192>
cipherkey, ap_uint<128> & plaintext) {
787         ap_uint<128> state;
788         ap_uint<4> round_counter;
789
790         // state init and add roundkey[0]
791         state = ciphertext ^ key_list[12];
792
793         // Start 14 rounds of process
794         for (round_counter = 1; round_counter <= 12;
round_counter++) {
795             ap_uint<8> tmp_1, tmp_2_1, tmp_2_2, tmp_3;
796             ap_uint<128> state_1;
797             // Inv ShiftRow
798             tmp_1 = state(111, 104);
799             state(111, 104) = state(79, 72);
800             state(79, 72) = state(47, 40);
801             state(47, 40) = state(15, 8);
802             state(15, 8) = tmp_1;
803
804             tmp_2_1 = state(87, 80);
805             state(87, 80) = state(23, 16);
806             state(23, 16) = tmp_2_1;
807
808             tmp_2_2 = state(119, 112);
809             state(119, 112) = state(55, 48);
810             state(55, 48) = tmp_2_2;
811
812             tmp_3 = state(31, 24);
813             state(31, 24) = state(63, 56);
814             state(63, 56) = state(95, 88);
815             state(95, 88) = state(127, 120);
816             state(127, 120) = tmp_3;
817             // Inv SubByte
818             for (int i = 0; i < 16; i++) {
819 #pragma HLS unroll
820
821                 state(i * 8 + 7, i * 8) = iibox[state(i * 8 +
7, i * 8)];
822             }
823
824             // Add Round Key
825             state ^= key_list[12 - round_counter];
826
827             // Inverse Mix Column
828             for (int i = 0; i < 16; i++) {
829                 state_1(i * 8 + 7, i * 8) = state(i * 8 + 7,
i * 8);
830             }
831             if (round_counter < 12) {
832                 // clang-format off
833                 for (int i = 0; i < 4; i++) {
834 #pragma HLS unroll
835
836                     state(i * 32 + 7, i * 32) =
GFmul8(state_1(i * 32 + 7, i * 32)) ^ GFmul4(state_1(i * 32 + 7, i *
32)) ^ GFmul2(state_1(i * 32 + 7, i * 32)) ^
837
838                     GFmul8(state_1(i * 32 + 15, i * 32 + 8)) ^ GFmul2(state_1(i * 32 +
15, i * 32 + 8)) ^ state_1(i * 32 + 15, i * 32 + 8) ^
839
840                     GFmul8(state_1(i * 32 + 23, i * 32 + 16)) ^ GFmul4(state_1(i * 32 +
23, i * 32 + 16)) ^ state_1(i * 32 + 23, i * 32 + 16) ^

```

```

837 GFMul8(state_1(i * 32 + 31, i * 32 + 24))^state_1(i * 32 + 31, i *
838 32 + 24);
838 state(i * 32 + 15, i * 32 + 8) =
838 GFMul8(state_1(i * 32 + 15, i * 32 + 8))^GFMul4(state_1(i * 32 +
839 15, i * 32 + 8))^GFMul2(state_1(i * 32 + 15, i * 32 + 8))^
839 GFMul8(state_1(i * 32 + 23, i * 32 + 16)) ^GFMul2(state_1(i * 32 +
840 23, i * 32 + 16))^state_1(i * 32 + 23, i * 32 + 16)^
840 GFMul8(state_1(i * 32 + 31, i * 32 + 24))^GFMul4(state_1(i * 32 +
841 31, i * 32 + 24))^state_1(i * 32 + 31, i * 32 + 24)^
841 GFMul8(state_1(i * 32 + 7, i * 32))^state_1(i * 32 + 7, i * 32);
842 state(i * 32 + 23, i * 32 + 16) =
842 GFMul8(state_1(i * 32 + 23, i * 32 + 16))^GFMul4(state_1(i * 32 +
843 23, i * 32 + 16))^GFMul2(state_1(i * 32 + 23, i * 32 + 16))^
843 GFMul8(state_1(i * 32 + 31, i * 32 + 24)) ^GFMul2(state_1(i * 32 +
844 31, i * 32 + 24))^state_1(i * 32 + 31, i * 32 + 24)^
844 GFMul8(state_1(i * 32 + 7, i * 32))^GFMul4(state_1(i * 32 + 7, i *
845 32))^state_1(i * 32 + 7, i * 32)^
845 GFMul8(state_1(i * 32 + 15, i * 32 + 8))^state_1(i * 32 + 15, i *
846 32 + 8);
846 state(i * 32 + 31, i * 32 + 24) =
846 GFMul8(state_1(i * 32 + 31, i * 32 + 24))^GFMul4(state_1(i * 32 +
847 31, i * 32 + 24))^GFMul2(state_1(i * 32 + 31, i * 32 + 24))^
847 GFMul8(state_1(i * 32 + 7, i * 32))^GFMul2(state_1(i * 32 + 7, i *
848 32))^state_1(i * 32 + 7, i * 32)^
848 GFMul8(state_1(i * 32 + 15, i * 32 + 8))^GFMul4(state_1(i * 32 +
849 15, i * 32 + 8))^state_1(i * 32 + 15, i * 32 + 8)^
849 GFMul8(state_1(i * 32 + 23, i * 32 + 16))^state_1(i * 32 + 23, i *
850 32 + 16);
850 } // clang-format on
851 }
852 }
853 }
854 plaintext = state;
855 }
856 };
857
858 template <>
859 class aesDec<128> : public aesTable {
860 public:
861     ap_uint<128> key_list[11];
862
863     aesDec() {
864 #pragma HLS inline
865 #pragma HLS ARRAY_PARTITION variable = key_list complete dim
866 = 1
867 }
868
869     void updateKey(ap_uint<128> cipherkey) {
870 #pragma HLS inline off
871         const ap_uint<8> Rcon[10] = {0x01, 0x02, 0x04, 0x08,
872 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36};
873         ap_uint<128> lastRound = cipherkey;
874         key_list[0] = lastRound;
875         for (ap_uint<5> iter = 1; iter < 11; iter++) {
876 #pragma HLS pipeline II = 1
877
878             ap_uint<32> round_tmp = lastRound.range(127, 96);
879             round_tmp = (round_tmp >> 8) | (round_tmp << 24);
880
881             round_tmp.range(7, 0) = sbox[round_tmp.range(7,
882 0)] ^ Rcon[iter - 1];
883             round_tmp.range(15, 8) = ss-
884 box[round_tmp.range(15, 8)];
885             round_tmp.range(23, 16) = ss-
886 box[round_tmp.range(23, 16)];
887             round_tmp.range(31, 24) = ss-
888 box[round_tmp.range(31, 24)];
889
890             key_list[iter].range(31, 0) = lastRound.range(31,
891 0) ^ round_tmp;
892             key_list[iter].range(63, 32) =
893 lastRound.range(63, 32) ^ key_list[iter].range(31, 0);
894             key_list[iter].range(95, 64) =
895 lastRound.range(95, 64) ^ key_list[iter].range(63, 32);
896             key_list[iter].range(127, 96) =
897 lastRound.range(127, 96) ^ key_list[iter].range(95, 64);
898
899             lastRound.range(127, 0) = key_list[iter];
900         }
901     }
902
903     // the function of Finite field multiplication *2 *4 *8
904
905     static ap_uint<8> GFMul2(ap_uint<8> a) {
906         ap_uint<8> kModulus = 0b00011011;
907
908         if (a[7] == 1) {
909             a = (a << 1) ^ kModulus;
910         } else {

```

```

901         a <<= 1;
902     }
903
904     return a;
905 }
906
907 static ap_uint<8> GFMul4(ap_uint<8> a) {
908     ap_uint<8> kModulus = 0b00011011;
909
910     for (int i = 0; i < 2; ++i) {
911         if (a[7] == 1) {
912             a = (a << 1) ^ kModulus;
913         } else {
914             a <<= 1;
915         }
916     }
917
918     return a;
919 }
920
921 static ap_uint<8> GFMul8(ap_uint<8> a) {
922     ap_uint<8> kModulus = 0b00011011;
923
924     for (int i = 0; i < 3; ++i) {
925         if (a[7] == 1) {
926             a = (a << 1) ^ kModulus;
927         } else {
928             a <<= 1;
929         }
930     }
931
932     return a;
933 }
934
935 void process(ap_uint<128> ciphertext, ap_uint<128>
936 cipherkey, ap_uint<128>& plaintext) {
937     ap_uint<128> state;
938     ap_uint<4> round_counter;
939
940     // state init and add roundkey[0]
941     state = ciphertext ^ key_list[10];
942
943     // Start 10 rounds of process
944     for (round_counter = 1; round_counter <= 10;
945 round_counter++) {
946         ap_uint<8> tmp_1, tmp_2_1, tmp_2_2, tmp_3;
947         ap_uint<128> state_1;
948         // Inv ShiftRow
949         tmp_1 = state(111, 104);
950         state(111, 104) = state(79, 72);
951         state(79, 72) = state(47, 40);
952         state(47, 40) = state(15, 8);
953         state(15, 8) = tmp_1;
954
955         tmp_2_1 = state(87, 80);
956         state(87, 80) = state(23, 16);
957         state(23, 16) = tmp_2_1;
958
959         tmp_2_2 = state(119, 112);
960         state(119, 112) = state(55, 48);
961         state(55, 48) = tmp_2_2;
962
963         tmp_3 = state(31, 24);
964         state(31, 24) = state(63, 56);
965         state(63, 56) = state(95, 88);
966         state(95, 88) = state(127, 120);
967         state(127, 120) = tmp_3;
968         // Inv SubByte
969         for (int i = 0; i < 16; i++) {
970             state(i * 8 + 7, i * 8) = iibox[state(i * 8 +
971 7, i * 8)];
972         }
973
974         // Add Round Key
975         state ^= key_list[10 - round_counter];
976
977         // Inverse Mix Column
978         for (int i = 0; i < 16; i++) {
979             ap_uint<8> tmp = state(i * 8 + 7, i * 8);
980             state_1(i * 8 + 7, i * 8) = state(i * 8 + 7,
981 i * 8);
982             if (round_counter < 10) {
983                 // clang-format off
984                 for (int i = 0; i < 4; i++) {
985                     #pragma HLS unroll
986                     state(i * 32 + 7, i * 32) =
987 GFMul8(state_1(i * 32 + 7, i * 32))^GFMul4(state_1(i * 32 + 7, i *
988 32))^GFMul2(state_1(i * 32 + 7, i * 32))^
989 GFMul8(state_1(i * 32 + 15, i * 32 + 8)) ^GFMul2(state_1(i * 32 +
990 15, i * 32 + 8))^state_1(i * 32 + 15, i * 32 + 8)^
991 GFMul8(state_1(i * 32 + 23, i * 32 + 16))^GFMul4(state_1(i * 32 +
992 23, i * 32 + 16))^state_1(i * 32 + 23, i * 32 + 16)^
993 GFMul8(state_1(i * 32 + 31, i * 32 + 24))^state_1(i * 32 + 31, i *
994 32 + 24);

```

```

988         state(i * 32 + 15, i * 32 + 8) =
GFMul8(state_1(i * 32 + 15, i * 32 + 8))^GFMul4(state_1(i * 32 +
15, i * 32 + 8))^GFMul2(state_1(i * 32 + 15, i * 32 + 8))^
989
GFMul8(state_1(i * 32 + 23, i * 32 + 16))^GFMul2(state_1(i * 32 +
23, i * 32 + 16))^state_1(i * 32 + 23, i * 32 + 16)^
990
GFMul8(state_1(i * 32 + 31, i * 32 + 24))^GFMul4(state_1(i * 32 +
31, i * 32 + 24))^state_1(i * 32 + 31, i * 32 + 24)^
991
GFMul8(state_1(i * 32 + 7, i * 32))^state_1(i * 32 + 7, i * 32);
992         state(i * 32 + 23, i * 32 + 16) =
GFMul8(state_1(i * 32 + 23, i * 32 + 16))^GFMul4(state_1(i * 32 +
23, i * 32 + 16))^GFMul2(state_1(i * 32 + 23, i * 32 + 16))^
993
GFMul8(state_1(i * 32 + 31, i * 32 + 24))^GFMul2(state_1(i * 32 +
31, i * 32 + 24))^state_1(i * 32 + 31, i * 32 + 24)^
994
GFMul8(state_1(i * 32 + 7, i * 32))^GFMul4(state_1(i * 32 + 7, i *
32))^state_1(i * 32 + 7, i * 32)^
995
GFMul8(state_1(i * 32 + 15, i * 32 + 8))^state_1(i * 32 + 15, i *
32 + 8);
996         state(i * 32 + 31, i * 32 + 24) =
GFMul8(state_1(i * 32 + 31, i * 32 + 24))^GFMul4(state_1(i * 32 +
31, i * 32 + 24))^GFMul2(state_1(i * 32 + 31, i * 32 + 24))^
997
GFMul8(state_1(i * 32 + 7, i * 32))^GFMul2(state_1(i * 32 + 7, i *
32))^state_1(i * 32 + 7, i * 32)^
998
GFMul8(state_1(i * 32 + 15, i * 32 + 8))^GFMul4(state_1(i * 32 +
15, i * 32 + 8))^state_1(i * 32 + 15, i * 32 + 8)^
999
GFMul8(state_1(i * 32 + 23, i * 32 + 16))^state_1(i * 32 + 23, i *
32 + 16);
1000         }
1001         // clang-format on
1002     }
1003     }
1004     plaintext = state;
1005 }
1006 };
1007
1008 } // namespace security
1009 } // namespace xf
1010 #endif

```

D.3 Vitis_security/sha224_256.hpp (854 lines, 35,288 B)

```

1 /*
2  * Copyright 2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  * cense");
6  * you may not use this file except in compliance with the
7  * License.
8  * You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing,
13  * software
14  * distributed under the License is distributed on an "AS IS"
15  * BASIS,
16  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17  * press or implied.
18  * See the License for the specific language governing per-
19  * missions and
20  * limitations under the License.
21  */
22
23 #ifndef _XF_SECURITY_SHA224_256_HPP_
24 #define _XF_SECURITY_SHA224_256_HPP_
25
26 #include <ap_int.h>
27 #include <hls_stream.h>
28
29 #include "xf_security/types.hpp"
30 #include "xf_security/utils.hpp"
31
32 // For debug
33 #ifndef __SYNTHESIS__
34 #include <cstdio>
35 #endif
36 #ifdef _DEBUG
37 #define _DEBUG (0)
38 #endif
39 #define _XF_SECURITY_VOID_CAST static_cast<void>
40 // XXX toggle here to debug this file
41 #define _XF_SECURITY_PRINT(msg...) \
42     do { \
43         if (_DEBUG) printf(msg); \
44     } while (0)
45
46 #define ROTR(n, x) ((x >> n) | (x << (32 - n)))
47 #define ROTL(n, x) ((x << n) | (x >> (32 - n)))
48 #define SHR(n, x) (x >> n)
49 #define CH(x, y, z) ((x & y) ^ ((~x) & z))

```

```

44 #define MAJ(x, y, z) ((x & y) ^ (x & z) ^ (y & z))
45 #define BSI60(x) (ROTR(2, x) ^ ROTR(13, x) ^ ROTR(22, x))
46 #define BSI61(x) (ROTR(6, x) ^ ROTR(11, x) ^ ROTR(25, x))
47 #define SSI60(x) (ROTR(7, x) ^ ROTR(18, x) ^ SHR(3, x))
48 #define SSI61(x) (ROTR(17, x) ^ ROTR(19, x) ^ SHR(10, x))
49
50 namespace xf {
51 namespace security {
52 namespace internal {
53
54 /// Processing block
55 struct SHA256Block {
56     uint32_t M[16];
57 };
58
59 /// @brief Static config for SHA224 and SHA256.
60 template <bool do_sha224>
61 struct sha256_digest_config;
62
63 template <>
64 struct sha256_digest_config<true> {
65     static const short numH = 7;
66 };
67
68 template <>
69 struct sha256_digest_config<false> {
70     static const short numH = 8;
71 };
72
73 /// @brief Generate 512bit processing blocks for SHA224/
74 SHA256 (pipeline)
75 /// with const width.
76 /// The performance goal of this function is to yield a 512b
77 block per cycle.
78 /// @param msg_strm the message being hashed.
79 /// @param len_strm the message length in byte.
80 /// @param end_len_strm that flag to signal end of input.
81 /// @param blk_strm the 512-bit hash block.
82 /// @param nblk_strm the number of hash block for this mes-
83 sage.
84 /// @param end_nblk_strm end flag for number of hash block.
85 inline void preProcessing(hls::stream<ap_uint<32>> &
86 msg_strm,
87 hls::stream<ap_uint<64>> &
88 len_strm,
89 hls::stream<bool> & end_len_strm,
90 hls::stream<SHA256Block> & blk_strm,
91 hls::stream<uint64_t> & nblk_strm,
92 hls::stream<bool> & end_nblk_strm) {
93
94     LOOP_SHA256_GENENERATE_MAIN:
95     for (bool end_flag = end_len_strm.read(); !end_flag;
96 end_flag = end_nblk_strm.read()) {
97         /// message length in byte.
98         uint64_t len = len_strm.read();
99         /// message length in bit.
100         uint64_t L = 8 * len;
101         /// total number blocks to digest.
102         uint64_t blk_num = (len >> 6) + 1 + ((len & 0x3f) >
103 55);
104
105         // inform digest function.
106         nblk_strm.write(blk_num);
107         end_nblk_strm.write(false);
108
109         LOOP_SHA256_GEN_FULL_BLKs:
110         for (uint64_t j = 0; j < uint64_t(len >> 6); ++j) {
111             #pragma HLS pipeline II = 16
112             #pragma HLS loop_tripcount min = 0 max = 1
113             /// message block.
114             SHA256Block b0;
115             #pragma HLS array_partition variable = b0.M complete
116             // this block will hold 64 byte of message.
117             LOOP_SHA256_GEN_ONE_FULL_BLK:
118             for (int i = 0; i < 16; ++i) {
119                 #pragma HLS unroll
120                 uint32_t l = msg_strm.read();
121                 // XXX algorithm assumes big-endian.
122                 l = ((0x000000ffUL & l) << 24) |
123                 ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8) |
124                 ((0xff000000UL & l) >> 24);
125                 b0.M[i] = l;
126                 _XF_SECURITY_PRINT("DEBUG: M[%d] = %t08x
127 (32bx16)\n", i, b0.M[i]);
128             }
129             // send block
130             blk_strm.write(b0);
131             _XF_SECURITY_PRINT("DEBUG: block sent\n");
132             // shift the buffer. high will be zero.
133         }
134
135         /// number of bytes not in blocks yet.
136         char left = (char)(len & 0x3f); // < 64
137
138         _XF_SECURITY_PRINT("DEBUG: sent = %d, left = %d\n",
139 int(len & (~1ULL ^ 0x3f)), (int)left);
140
141         if (left == 0) {
142             // end at block boundary, start with pad 1.
143
144             /// last block
145             SHA256Block b;

```

```

134 #pragma HLS array_partition variable = b.M complete
135 // pad 1
136 b.M[0] = 0x80000000UL;
137 _XF_SECURITY_PRINT("DEBUG: M[0] =\t%08x (pad
1)\n", b.M[0]);
138 // zero
139 LOOP_SHA256_GEN_PAD_13_ZEROS:
140 for (int i = 1; i < 14; ++i) {
141 #pragma HLS unroll
142     b.M[i] = 0;
143     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(zero)\n", i, b.M[i]);
144 }
145 // append L
146 b.M[14] = (uint32_t)(0xffffffffUL & (L >> 32));
147 b.M[15] = (uint32_t)(0xffffffffUL & (L));
148 _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
L)\n", 14, b.M[14]);
149 _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
L)\n", 15, b.M[15]);
150 // emit
151 blk_strm.write(b);
152 } else if (left < 56) {
153     // can pad 1 and append L.
154
155     // last message block.
156     SHA256Block b;
157 #pragma HLS array_partition variable = b.M complete
158
159     LOOP_SHA256_GEN_COPY_TAIL_AND_ONE:
160     for (int i = 0; i < 14; ++i) {
161 #pragma HLS pipeline
162         if (i < (left >> 2)) {
163             uint32_t l = msg_strm.read();
164             // pad 1 byte not in this word
165             // XXX algorithm assumes big-endian.
166             l = ((0x000000ffUL & l) << 24) |
167                 ((0x00ff0000UL & l) >> 8) |
168                 ((0xff000000UL & l) >> 24);
169             b.M[i] = l;
170             _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(32b)\n", i, b.M[i]);
171         } else if (i > (left >> 2)) {
172             // pad 1 not in this word, and no word to
173             read.
174             b.M[i] = 0UL;
175         } else {
176             // pad 1 byte in this word
177             uint32_t e = left & 3L;
178             if (e == 0) {
179                 b.M[i] = 0x80000000UL;
180             } else if (e == 1) {
181                 uint32_t l = msg_strm.read();
182                 // XXX algorithm assumes big-endian.
183                 l = ((0x000000ffUL & l) << 24);
184                 b.M[i] = l | 0x00800000UL;
185             } else if (e == 2) {
186                 uint32_t l = msg_strm.read();
187                 // XXX algorithm assumes big-endian.
188                 l = ((0x000000ffUL & l) << 24) |
189                     ((0x00ff0000UL & l) >> 8);
190                 b.M[i] = l | 0x00000080UL;
191             }
192             _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(pad 1)\n", i, b.M[i]);
193         }
194     }
195     // append L
196     b.M[14] = (uint32_t)(0xffffffffUL & (L >> 32));
197     b.M[15] = (uint32_t)(0xffffffffUL & (L));
198     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
L)\n", 14, b.M[14]);
199     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
L)\n", 15, b.M[15]);
200
201     blk_strm.write(b);
202     _XF_SECURITY_PRINT("DEBUG: block sent\n");
203 } else {
204     // cannot append L.
205
206     // last but 1 block.
207     SHA256Block b;
208 #pragma HLS array_partition variable = b.M complete
209 // copy and pad 1
210 LOOP_SHA256_GEN_COPY_TAIL_ONLY:
211 for (int i = 0; i < 16; ++i) {
212 #pragma HLS unroll
213     if (i < (left >> 2)) {
214         // pad 1 byte not in this word
215         uint32_t l = msg_strm.read();
216         // XXX algorithm assumes big-endian.
217         l = ((0x000000ffUL & l) << 24) |
218             ((0x00ff0000UL & l) >> 8) |

```

```

219             ((0xff000000UL & l) >> 24);
220         b.M[i] = l;
221         _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(32b)\n", i, b.M[i]);
222     } else if (i > (left >> 2)) {
223         // pad 1 byte not in this word, and no
224         msg word to read
225         b.M[i] = 0UL;
226     } else {
227         // last in this word
228         uint32_t e = left & 3L;
229         if (e == 0) {
230             b.M[i] = 0x80000000UL;
231         } else if (e == 1) {
232             uint32_t l = msg_strm.read();
233             // XXX algorithm assumes big-endian.
234             l = ((0x000000ffUL & l) << 24);
235             b.M[i] = l | 0x00800000UL;
236         } else if (e == 2) {
237             uint32_t l = msg_strm.read();
238             // XXX algorithm assumes big-endian.
239             l = ((0x000000ffUL & l) << 24) |
240                 ((0x00ff0000UL & l) >> 8);
241             b.M[i] = l | 0x00000080UL;
242         } else {
243             uint32_t l = msg_strm.read();
244             // XXX algorithm assumes big-endian.
245             l = ((0x000000ffUL & l) << 24) |
246                 ((0x00ff0000UL & l) >> 8);
247             b.M[i] = l | 0x00000080UL;
248         }
249         _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(pad 1)\n", i, b.M[i]);
250     }
251     blk_strm.write(b);
252     _XF_SECURITY_PRINT("DEBUG: block sent\n");
253
254     // last block.
255     SHA256Block b1;
256 #pragma HLS array_partition variable = b1.M complete
257 LOOP_SHA256_GEN_L_ONLY_BLK:
258 for (int i = 0; i < 14; ++i) {
259 #pragma HLS unroll
260     b1.M[i] = 0;
261     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(zero)\n", i, b1.M[i]);
262 }
263 // append L
264 b1.M[14] = (uint32_t)(0xffffffffUL & (L >> 32));
265 b1.M[15] = (uint32_t)(0xffffffffUL & (L));
266 _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
L)\n", 14, b1.M[14]);
267 _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
L)\n", 15, b1.M[15]);
268
269     blk_strm.write(b1);
270     _XF_SECURITY_PRINT("DEBUG: block sent\n");
271 } // main loop
272 end_nblk_strm.write(true);
273
274 } // preProcessing (32-bit ver)
275
276 /// @brief Generate 512bit processing blocks for SHA224/
277 SHA256 (pipeline)
278 /// with const width.
279 /// The performance goal of this function is to yield a 512b
280 block per cycle.
281 /// @param msg_strm the message being hashed.
282 /// @param len_strm the message length in byte.
283 /// @param end_len_strm that flag to signal end of input.
284 /// @param blk_strm the 512-bit hash block.
285 /// @param nblk_strm the number of hash block for this mes-
286 sage.
287 /// @param end_nblk_strm end flag for number of hash block.
288 inline void preProcessing(hls::stream<ap_uint<64> >&
msg_strm,
289 hls::stream<ap_uint<64> >&
len_strm,
290 hls::stream<bool>& end_len_strm,
291 hls::stream<SHA256Block>& blk_strm,
292 hls::stream<uint64_t>& nblk_strm,
293 hls::stream<bool>& end_nblk_strm) {
294     LOOP_SHA256_GENENERATE_MAIN:
295     for (bool end_flag = end_len_strm.read(); !end_flag;
296         end_flag = end_len_strm.read()) {
297         // message length in byte.
298         uint64_t len = len_strm.read();
299         _XF_SECURITY_PRINT("DEBUG: working on a new message
of %ld bytes\n", len);
300         // message length in bit.
301         uint64_t L = 8 * len;
302         // total number blocks to digest.
303         uint64_t blk_num = (len >> 6) + 1 + ((len & 0x3f) >
55);
304         // inform digest function.
305         nblk_strm.write(blk_num);
306         end_nblk_strm.write(false);
307     }

```



```

304     LOOP_SHA256_GEN_FULL_BLK:
305     for (uint64_t j = 0; j < uint64_t(len >> 6); ++j) {
306     #pragma HLS pipeline II = 16
307     #pragma HLS loop_tripcount min = 0 max = 1
308     /// message block.
309     SHA256Block b0;
310     #pragma HLS array_partition variable = b0.M complete
311
312     // this block will hold 64 byte of message.
313     LOOP_SHA256_GEN_ONE_FULL_BLK:
314     for (int i = 0; i < 16; i += 2) {
315     #pragma HLS unroll
316     uint64_t ll = msg_strm.read().to_uint64();
317     // low
318     uint32_t l = ll & 0xffffffffUL;
319     // XXX algorithm assumes big-endian.
320     l = ((0x000000ffUL & l) << 24) |
321     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8) |
322     ((0xff000000UL & l) >> 24);
323     b0.M[i] = l;
324     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
325     (64bx8 low)\n", i, b0.M[i]);
326     // high
327     l = (ll >> 32) & 0xffffffffUL;
328     // XXX algorithm assumes big-endian.
329     l = ((0x000000ffUL & l) << 24) |
330     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8) |
331     ((0xff000000UL & l) >> 24);
332     b0.M[i+1] = l;
333     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
334     (64bx8 high)\n", i, b0.M[i]);
335     }
336     // send block
337     blk_strm.write(b0);
338     _XF_SECURITY_PRINT("DEBUG: block sent\n");
339     // shift the buffer. high will be zero.
340
341     /// number of bytes not in blocks yet.
342     char left = (char)(len & 0x3FULL); // < 64
343     _XF_SECURITY_PRINT("DEBUG: sent = %d, left = %d\n",
344     int(len & (-1ULL ^ 0x3FULL)), (int)left);
345
346     if (left == 0) {
347     // end at block boundary, start with pad 1.
348
349     /// last block
350     SHA256Block b;
351     #pragma HLS array_partition variable = b.M complete
352     // pad 1
353     b.M[0] = 0x80000000UL;
354     _XF_SECURITY_PRINT("DEBUG: M[0] =\t%08x (pad
355     1)\n", b.M[0]);
356     // zero
357     LOOP_SHA256_GEN_PAD_13_ZEROS:
358     for (int i = 1; i < 14; ++i) {
359     #pragma HLS unroll
360     b.M[i] = 0;
361     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
362     (zero)\n", i, b.M[i]);
363     }
364     // append L
365     b.M[14] = (uint32_t)(0xffffffffUL & (L >> 32));
366     b.M[15] = (uint32_t)(0xffffffffUL & (L));
367     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
368     L)\n", 14, b.M[14]);
369     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (append
370     L)\n", 15, b.M[15]);
371     // emit
372     blk_strm.write(b);
373     _XF_SECURITY_PRINT("DEBUG: block sent\n");
374     } else {
375     // can pad 1 and append L.
376
377     // last message block.
378     SHA256Block b;
379     #pragma HLS array_partition variable = b.M complete
380
381     LOOP_SHA256_GEN_COPY_TAIL_PAD_ONE:
382     for (int i = 0; i < ((left < 56) ? 7 : 8); ++i) {
383     #pragma HLS pipeline
384     if (i < (left >> 3)) {
385     // pad 1 not in this 64b word, and need
386     to copy
387     uint64_t ll =
388     msg_strm.read().to_uint64();
389     // low
390     uint32_t l = ll & 0xffffffffUL;
391     // XXX algorithm assumes big-endian.
392     l = ((0x000000ffUL & l) << 24) |
393     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8) |
394     ((0xff000000UL & l) >> 24);
395     b.M[i * 2] = l;
396     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
397     (64b low)\n", i * 2, b.M[i * 2]);
398     // high
399     l = (ll >> 32) & 0xffffffffUL;
400     // XXX algorithm assumes big-endian.
401     l = ((0x000000ffUL & l) << 24) |

```

```

402     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8) |
403     ((0xff000000UL & l) >> 24);
404     b.M[i * 2 + 1] = l;
405     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
406     (64b high)\n", i * 2 + 1, b.M[i * 2 + 1]);
407     } else if (i > (left >> 3)) {
408     // pad 1 not in this 64b word, and no
409     word to read.
410     b.M[i * 2] = 0UL;
411     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
412     (zero)\n", i * 2, b.M[i * 2]);
413     b.M[i * 2 + 1] = 0UL;
414     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
415     (zero)\n", i * 2 + 1, b.M[i * 2 + 1]);
416     } else {
417     // pad 1 byte in this 64b word
418     if ((left & 4) == 0) {
419     // left in low 32b
420     uint32_t e = left & 3L;
421     if (e == 0) {
422     b.M[i * 2] = 0x80000000UL;
423     } else if (e == 1) {
424     uint32_t l =
425     msg_strm.read().to_uint64() & 0xffffffffUL;
426     // XXX algorithm assumes big-en-
427     dian.
428     l = ((0x000000ffUL & l) << 24);
429     b.M[i * 2] = l | 0x00800000UL;
430     } else if (e == 2) {
431     uint32_t l =
432     msg_strm.read().to_uint64() & 0xffffffffUL;
433     // XXX algorithm assumes big-en-
434     dian.
435     l = ((0x000000ffUL & l) << 24) |
436     ((0x0000ff00UL & l) << 8);
437     b.M[i * 2] = l | 0x00008000UL;
438     } else {
439     uint32_t l =
440     msg_strm.read().to_uint64() & 0xffffffffUL;
441     // XXX algorithm assumes big-en-
442     dian.
443     l = ((0x000000ffUL & l) << 24) |
444     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8);
445     b.M[i * 2] = l | 0x00000080UL;
446     }
447     _XF_SECURITY_PRINT("DEBUG: M[%d]
448     =\t%08x (pad 1)\n", i * 2, b.M[i * 2]);
449     // high
450     b.M[i * 2 + 1] = 0UL;
451     _XF_SECURITY_PRINT("DEBUG: M[%d]
452     =\t%08x (zero)\n", i * 2 + 1, b.M[i * 2 + 1]);
453     } else {
454     // left in high 32b
455     uint64_t ll =
456     msg_strm.read().to_uint64();
457     // low 32b
458     uint32_t l = ll & 0xffffffffUL;
459     // XXX algorithm assumes big-endian.
460     l = ((0x000000ffUL & l) << 24) |
461     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8) |
462     ((0xff000000UL & l) >> 24);
463     b.M[i * 2] = l;
464     _XF_SECURITY_PRINT("DEBUG: M[%d]
465     =\t%08x (64b low)\n", i, b.M[i * 2]);
466     // high 32b
467     l = (ll >> 32) & 0xffffffffUL;
468     uint32_t e = left & 3L;
469     if (e == 0) {
470     b.M[i * 2 + 1] = 0x80000000UL;
471     } else if (e == 1) {
472     // XXX algorithm assumes big-en-
473     dian.
474     l = ((0x000000ffUL & l) << 24);
475     b.M[i * 2 + 1] = l |
476     0x00800000UL;
477     } else if (e == 2) {
478     // XXX algorithm assumes big-en-
479     dian.
480     l = ((0x000000ffUL & l) << 24) |
481     ((0x0000ff00UL & l) << 8);
482     b.M[i * 2 + 1] = l |
483     0x00008000UL;
484     } else {
485     // XXX algorithm assumes big-en-
486     dian.
487     l = ((0x000000ffUL & l) << 24) |
488     ((0x0000ff00UL & l) << 8) | ((0x00ff0000UL & l) >> 8);
489     b.M[i * 2 + 1] = l |
490     0x00000080UL;
491     }
492     _XF_SECURITY_PRINT("DEBUG: M[%d]
493     =\t%08x (pad 1)\n", i * 2 + 1, b.M[i * 2 + 1]);
494     }
495     }
496     if (left < 56) {
497     // append L
498     b.M[14] = (uint32_t)(0xffffffffUL & (L >>
499     32));

```

```

463         b.M[15] = (uint32_t)(0xffffffffUL & (L));
464         _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (ap-
pend L)\n", 14, b.M[14]);
465         _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (ap-
pend L)\n", 15, b.M[15]);
466         blk_strm.write(b);
467         _XF_SECURITY_PRINT("DEBUG: block sent\n");
468     } else {
469         // send block without L
470         blk_strm.write(b);
471         _XF_SECURITY_PRINT("DEBUG: block sent\n");
472     }
473     /// last block.
474     SHA256BBlock b1;
475     #pragma HLS array_partition variable = b1.M complete
476     LOOP_SHA256_GEN_L_ONLY_BLK:
477     for (int i = 0; i < 14; ++i) {
478     #pragma HLS unroll
479         b1.M[i] = 0;
480         _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x
(zero)\n", i, b1.M[i]);
481     }
482     // append L
483     b1.M[14] = (uint32_t)(0xffffffffUL & (L >>
32));
484     b1.M[15] = (uint32_t)(0xffffffffUL & (L));
485     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (ap-
pend L)\n", 14, b1.M[14]);
486     _XF_SECURITY_PRINT("DEBUG: M[%d] =\t%08x (ap-
pend L)\n", 15, b1.M[15]);
487     blk_strm.write(b1);
488     _XF_SECURITY_PRINT("DEBUG: block sent\n");
489     } // left < 56
490 } // main loop
491 end_nblk_strm.write(true);
492 } // preProcessing (64bit ver)
493
494 inline void dup_strm(hls::stream<uint64_t>& in_strm,
495 hls::stream<bool>& in_e_strm,
496 hls::stream<uint64_t>& out1_strm,
497 hls::stream<bool>& out1_e_strm,
498 hls::stream<uint64_t>& out2_strm,
499 hls::stream<bool>& out2_e_strm) {
500     bool e = in_e_strm.read();
501     while (!e) {
502     #pragma HLS loop_tripcount min = 1 max = 1 avg = 1
503     #pragma HLS pipeline II = 1
504         uint64_t in_r = in_strm.read();
505         out1_strm.write(in_r);
506         out1_e_strm.write(false);
507         out2_strm.write(in_r);
508         out2_e_strm.write(false);
509     }
510     e = in_e_strm.read();
511     out1_e_strm.write(true);
512     out2_e_strm.write(true);
513 }
514
515 inline void generateMsgSchedule(hls::stream<SHA256BBlock>&
blk_strm,
516 hls::stream<uint64_t>&
nblk_strm,
517 hls::stream<bool>& end_n-
blk_strm,
518 hls::stream<uint32_t>&
w_strm) {
519     bool e = end_nblk_strm.read();
520     while (!e) {
521         uint64_t n = nblk_strm.read();
522         for (uint64_t i = 0; i < n; ++i) {
523         #pragma HLS latency max = 65
524             SHA256BBlock blk = blk_strm.read();
525             #pragma HLS array_partition variable = blk.M complete
526             /// message schedule W, from message or
527             uint32_t W[16];
528             #pragma HLS array_partition variable = W complete
529             LOOP_SHA256_PREPARE_WT16:
530             for (short t = 0; t < 16; ++t) {
531             #pragma HLS pipeline II = 1
532                 uint32_t Wt = blk.M[t];
533                 W[t] = Wt;
534                 w_strm.write(Wt);
535             }
536             LOOP_SHA256_PREPARE_WT64:
537             for (short t = 16; t < 64; ++t) {
538             #pragma HLS pipeline II = 1
539                 // uint32_t Wt = SSIg1(W[t - 2]) + W[t - 7] +

```

```

SSIG0(W[t - 15]) + W[t - 16];
552         // W[t] = Wt;
553         uint32_t Wt = SSIg1(W[14]) + W[9] +
SSIG0(W[1]) + W[0];
554         for (unsigned char j = 0; j < 15; ++j) {
555             W[j] = W[j + 1];
556         }
557         W[15] = Wt;
558         w_strm.write(Wt);
559     }
560     e = end_nblk_strm.read();
561 }
562 }
563 }
564
565 inline void sha256_iter(uint32_t& a,
566 uint32_t& b,
567 uint32_t& c,
568 uint32_t& d,
569 uint32_t& e,
570 uint32_t& f,
571 uint32_t& g,
572 uint32_t& h,
573 hls::stream<uint32_t>& w_strm,
574 uint32_t& Kt,
575 const uint32_t K[],
576 short t) {
577     uint32_t Wt = w_strm.read();
578     /// temporal variables
579     uint32_t T1, T2;
580     T1 = h + BSIg1(e) + CH(e, f, g) + Kt + Wt;
581     T2 = BSIg0(a) + MAJ(a, b, c);
582     // update working variables.
583     h = g;
584     g = f;
585     f = e;
586     e = d + T1;
587     d = c;
588     c = b;
589     b = a;
590     a = T1 + T2;
591
592     _XF_SECURITY_PRINT(
593         "DEBUG: Kt=%08x, Wt=%08x\n"
594         "\ta=%08x, b=%08x, c=%08x, d=%08x\n"
595         "\te=%08x, f=%08x, g=%08x, h=%08x\n",
596         Kt, Wt, a, b, c, d, e, f, g, h);
597     // for next cycle
598     Kt = K[(t + 1) & 63];
599 }
600
601 // @brief Digest message blocks and emit final hash.
602 // @tparam h_width the hash width(type).
603 // @param nblk_strm number of message block.
604 // @param end_nblk_strm end flag for number of message
605 // block.
606 // @param hash_strm the hash result stream.
607 // @param end_hash_strm end flag for hash result.
608 template <int h_width>
609 void sha256Digest(hls::stream<uint64_t>& nblk_strm,
610 hls::stream<bool>& end_nblk_strm,
611 hls::stream<uint32_t>& w_strm,
612 hls::stream<ap_uint<h_width>>& hash_strm,
613 hls::stream<bool>& end_hash_strm) {
614     // h_width determine the hash type.
615
616     XF_SECURITY_STATIC_ASSERT((h_width == 256) || (h_width ==
224),
617 "Unsupported hash stream width,
must be 224 or 256");
618
619     /// constant K
620     static const uint32_t K[64] = {
621         0x428a2f98UL, 0x71374491UL, 0xb5c0fbcfUL, 0xe9b5d-
ba5UL, 0x3956c25bUL, 0x59f111f1UL, 0x923f82a4UL, 0xab1c5ed5UL,
622         0xd807aa98UL, 0x12835b01UL, 0x243185beUL, 0x550c7d-
c3UL, 0x72be5d74UL, 0x80deb1feUL, 0x9bdc06a7UL, 0xc19bf174UL,
623         0xe49b69c1UL, 0xefb4786UL, 0xf0fc19dc6UL, 0x240ca1c-
cUL, 0x2de92c6fUL, 0x4a7484aaUL, 0x5cbb0a9dUL, 0x76f988daUL,
624         0x983e5152UL, 0xa831c66dUL, 0xb00327c8UL, 0xb-
f597fc7UL, 0xc6e00bf3UL, 0xd5a79147UL, 0x06ca6351UL, 0x14292967UL,
625         0x27b70a85UL, 0x2e1b2138UL, 0x4d2c6dfcUL,
0x53380d13UL, 0x650a7354UL, 0x766a0abbUL, 0x81c2c92eUL,
0x92722c85UL,
626         0xa2bfe8a1UL, 0xa81a664bUL, 0xc24b8b70UL, 0x-
c76c51a3UL, 0xd192e819UL, 0xd6990624UL, 0xf40e3585UL, 0x106aa070UL,
627         0x19a4c116UL, 0x1e377c08UL, 0x2748774cUL, 0x34b0b-
cb5UL, 0x391c0cb3UL, 0x4ed8aa4aUL, 0x5b9cca4fUL, 0x682e6ff3UL,
628         0x748f82eeUL, 0x78a5636fUL, 0x84c87814UL, 0x8c-
c70208UL, 0x90bafffaUL, 0xa4506cebUL, 0xbef9a3f7UL, 0xc67178f2UL};
629     #pragma HLS array_partition variable = K complete
630
631     LOOP_SHA256_DIGEST_MAIN:
632     for (bool end_flag = end_nblk_strm.read(); !end_flag;
end_flag = end_nblk_strm.read()) {
633         /// total number blocks to digest.
634         uint64_t blk_num = nblk_strm.read();

```

```

635 // _XF_SECURITY_PRINT("expect %ld blocks.\n",
blk_num);
636
637 /// internal states
638 uint32_t H[8];
639 #pragma HLS array_partition variable = H complete
640
641 // initialize
642 if (h_width == 224) {
643     H[0] = 0xc1059ed8UL;
644     H[1] = 0x367cd507UL;
645     H[2] = 0x3070dd17UL;
646     H[3] = 0xf70e5939UL;
647     H[4] = 0xffc00b31UL;
648     H[5] = 0x68581511UL;
649     H[6] = 0x64f98fa7UL;
650     H[7] = 0xbef44fa4UL;
651 } else {
652     H[0] = 0x6a09e667UL;
653     H[1] = 0xbb67ae85UL;
654     H[2] = 0x3c6ef372UL;
655     H[3] = 0xa54ff53aUL;
656     H[4] = 0x510e527fUL;
657     H[5] = 0x9b05688cUL;
658     H[6] = 0x1f83d9abUL;
659     H[7] = 0x5be0cd19UL;
660 }
661
662 LOOP_SHA256_DIGEST_NBLK:
663 for (uint64_t n = 0; n < blk_num; ++n) {
664 #pragma HLS loop_tripcount min = 1 max = 1
665 #pragma HLS latency max = 65
666
667     /// working variables.
668     uint32_t a, b, c, d, e, f, g, h;
669
670     // loading working variables.
671     a = H[0];
672     b = H[1];
673     c = H[2];
674     d = H[3];
675     e = H[4];
676     f = H[5];
677     g = H[6];
678     h = H[7];
679
680     uint32_t Kt = K[0];
681     LOOP_SHA256_UPDATE_64_ROUNDS:
682     for (short t = 0; t < 64; ++t) {
683 #pragma HLS pipeline II = 1
684         sha256_iter(a, b, c, d, e, f, g, h, w_strm,
Kt, K, t);
685     } // 64 round loop
686
687     // store working variables to internal states.
688     H[0] = a + H[0];
689     H[1] = b + H[1];
690     H[2] = c + H[2];
691     H[3] = d + H[3];
692     H[4] = e + H[4];
693     H[5] = f + H[5];
694     H[6] = g + H[6];
695     H[7] = h + H[7];
696 } // block loop
697
698 // Emit digest
699 if (h_width == 224) {
700     ap_uint<224> w224;
701     LOOP_SHA256_EMIT_H224:
702     for (short i = 0; i <
sha256_digest_config::numH; ++i) {
703 #pragma HLS unroll
704         uint32_t l = H[i];
705         // XXX shift algorithm's big endian to HLS's
little endian.
706         uint8_t t0 = (((l) >> 24) & 0xff);
707         uint8_t t1 = (((l) >> 16) & 0xff);
708         uint8_t t2 = (((l) >> 8) & 0xff);
709         uint8_t t3 = (((l) & 0xff));
710         uint32_t l_little =
(((uint32_t)t0) | (((uint32_t)t1) << 8) |
(((uint32_t)t2) << 16) | (((uint32_t)t3) << 24);
711         w224.range(32 * i + 31, 32 * i) = l_little;
712     }
713     hash_strm.write(w224);
714 } else {
715     ap_uint<256> w256;
716     LOOP_SHA256_EMIT_H256:
717     for (short i = 0; i <
sha256_digest_config::numH; ++i) {
718 #pragma HLS unroll
719         uint32_t l = H[i];
720         // XXX shift algorithm's big endian to HLS's
little endian.
721         uint8_t t0 = (((l) >> 24) & 0xff);
722         uint8_t t1 = (((l) >> 16) & 0xff);
723         uint8_t t2 = (((l) >> 8) & 0xff);
724         uint8_t t3 = (((l) & 0xff));
725         uint32_t l_little =
(((uint32_t)t0) | (((uint32_t)t1) << 8) |

```

```

(((uint32_t)t2) << 16) | (((uint32_t)t3) << 24);
726         w256.range(32 * i + 31, 32 * i) = l_little;
727     }
728     hash_strm.write(w256);
729 }
730 end_hash_strm.write(false);
731 } // main loop
732 end_hash_strm.write(true);
733
734 // sha256Digest (pipelined override)
735
736 /// @brief SHA-256/224 implementation top overload for
ap_uint input.
737
738 /// @tparam m_width the input message stream width.
739 /// @tparam h_width the output hash stream width.
740 /// @param msg_strm the message being hashed.
741 /// @param len_strm the length message being hashed in byte.
742 /// @param end_len_strm end flag stream of input, one per
message.
743 /// @param hash_strm the result.
744 /// @param end_hash_strm end flag stream of output, one per
hash.
745
746 template <int m_width, int h_width>
747 inline void sha256_top(hls::stream<ap_uint<m_width>> &
msg_strm,
748 hls::stream<ap_uint<64>> & len_strm,
749 hls::stream<bool> & end_len_strm,
750 hls::stream<ap_uint<h_width>> &
hash_strm,
751 hls::stream<bool> & end_hash_strm) {
752 #pragma HLS DATAFLOW
753     /// 512-bit Block stream
754     hls::stream<SHA256Block> blk_strm("blk_strm");
755     #pragma HLS STREAM variable = blk_strm depth = 32
756     #pragma HLS RESOURCE variable = blk_strm core = FIFO_LUTRAM
757
758     /// number of Blocks, send per msg
759     hls::stream<uint64_t> nblk_strm("nblk_strm");
760     #pragma HLS STREAM variable = nblk_strm depth = 32
761     #pragma HLS RESOURCE variable = nblk_strm core = FIFO_LUTRAM
762     hls::stream<uint64_t> nblk_strm1("nblk_strm1");
763     #pragma HLS STREAM variable = nblk_strm1 depth = 32
764     #pragma HLS RESOURCE variable = nblk_strm1 core = FIFO_LUTRAM
765     hls::stream<uint64_t> nblk_strm2("nblk_strm2");
766     #pragma HLS STREAM variable = nblk_strm2 depth = 32
767     #pragma HLS RESOURCE variable = nblk_strm2 core = FIFO_LUTRAM
768
769     /// end flag, send per msg.
770     hls::stream<bool> end_nblk_strm("end_nblk_strm");
771     #pragma HLS STREAM variable = end_nblk_strm depth = 32
772     #pragma HLS RESOURCE variable = end_nblk_strm core =
FIFO_LUTRAM
773     hls::stream<bool> end_nblk_strm1("end_nblk_strm1");
774     #pragma HLS STREAM variable = end_nblk_strm1 depth = 32
775     #pragma HLS RESOURCE variable = end_nblk_strm1 core =
FIFO_LUTRAM
776     hls::stream<bool> end_nblk_strm2("end_nblk_strm2");
777     #pragma HLS STREAM variable = end_nblk_strm2 depth = 32
778     #pragma HLS RESOURCE variable = end_nblk_strm2 core =
FIFO_LUTRAM
779
780     /// W, 64 items for each block
781     hls::stream<uint32_t> w_strm("w_strm");
782     #pragma HLS STREAM variable = w_strm depth = 32
783     #pragma HLS RESOURCE variable = w_strm core = FIFO_LUTRAM
784
785     // Generate block stream
786     preProcessing(msg_strm, len_strm, end_len_strm, //
blk_strm, nblk_strm, end_nblk_strm);
787
788     // Duplicate number of block stream and its end flag
789     stream
790     dup_strm(nblk_strm, end_nblk_strm, nblk_strm1, end_n-
blk_strm1, nblk_strm2, end_nblk_strm2);
791
792     // Generate the message schedule in stream
793
794     generateMsgSchedule(blk_strm, nblk_strm1, end_nblk_strm1,
w_strm);
795
796     // Digest block stream, and write hash stream.
797     // fully pipelined version will calculate SHA-224 if
hash_strm width is 224.
798     sha256Digest(nblk_strm2, end_nblk_strm2, w_strm, //
hash_strm, end_hash_strm);
799 } // sha256_top
800 } // namespace internal
801
802 /// @brief SHA-224 algorithm with ap_uint stream input and
output.
803
804 /// @tparam m_width the input message stream width, currently
only 32 allowed.
805 /// @param msg_strm the message being hashed.
806 /// @param len_strm the length message being hashed.
807 /// @param end_len_strm the flag for end of message length
input.
808 /// @param hash_strm the result.
809 /// @param end_hash_strm the flag for end of hash output.
810 template <int m_width>

```



```

810 void sha224(hls::stream<ap_uint<m_width> >& msg_strm,    //
in
811             hls::stream<ap_uint<64> >& len_strm,          //
in
812             hls::stream<bool>& end_len_strm,              //
in
813             hls::stream<ap_uint<224> >& hash_strm,        //
out
814             hls::stream<bool>& end_hash_strm) {           //
out
815     internal::sha256_top(msg_strm, len_strm, end_len_strm, //
in
816                          hash_strm, end_hash_strm);      //
out
817 }
818
819 /// @brief SHA-256 algorithm with ap_uint stream input and
output.

820 /// @tparam m_width the input message stream width, currently
only 32 allowed.
821 /// @param msg_strm the message being hashed.
822 /// @param len_strm the length message being hashed.
823 /// @param end_len_strm the flag for end of message length
input.
824 /// @param hash_strm the result.
825 /// @param end_hash_strm the flag for end of hash output.
826 template <int m_width>

827 void sha256(hls::stream<ap_uint<m_width> >& msg_strm,    //
in
828             hls::stream<ap_uint<64> >& len_strm,          //
in
829             hls::stream<bool>& end_len_strm,              //
in
830             hls::stream<ap_uint<256> >& hash_strm,        //
out
831             hls::stream<bool>& end_hash_strm) {           //
out
832     internal::sha256_top(msg_strm, len_strm, end_len_strm, //
in
833                          hash_strm, end_hash_strm);      //
out
834 }
835 } // namespace security
836 } // namespace xf
837
838 // Clean up macros.
839 #undef ROTR
840 #undef ROTL
841 #undef SHR
842 #undef CH
843 #undef MAJ
844 #undef BSI0
845 #undef BSI1
846 #undef SSI0
847 #undef SSI1
848
849 #undef _XF_SECURITY_PRINT
850 #undef _XF_SECURITY_VOID_CAST
851
852 #endif // XF_SECURITY_SHA2_H
853 // -*- cpp -*-
854 // vim: ts=8:sw=2:sts=2:ft=cpp

```

D.4 Vitis_security/xf_security/types.hpp (58 lines, 1,610 B)

```

1 /*
2  * Copyright 2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
5  * you may not use this file except in compliance with the
License.
6  * You may obtain a copy of the License at
7  *
8  * http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-

```

```

missions and
14 * Limitations under the License.
15 */
16
17 /**
18  * @file types.h
19  * @brief This file is part of Vitis Security Library, con-
tains type definitions.
20 */
21
22 #ifndef _XF_SECURITY_TYPES_HPP_
23 #define _XF_SECURITY_TYPES_HPP_
24
25 // Fixed width integers
26 #if defined(__cplusplus) && __cplusplus >= 201103L
27 #include <stdint>
28 #elif defined(__STDC_VERSION__) && __STDC_VERSION__ >=
199901L
29 #include <stdint.h>
30 #else
31 // Signed.
32 // avoid conflict with <sys/types.h>
33 #ifndef __int8_t_defined
34 #define __int8_t_defined
35 typedef signed char int8_t;
36 typedef short int int16_t;
37 typedef int int32_t;
38 #if defined(__LP64__) && __LP64__
39 typedef long int int64_t;
40 #else
41 typedef long long int int64_t;
42 #endif // __LP64__
43 #endif // __int8_t_defined
44 // Unsigned.
45 typedef unsigned char uint8_t;
46 typedef unsigned short int uint16_t;
47 typedef unsigned int uint32_t;
48 #if defined(__LP64__) && __LP64__
49 typedef unsigned long int uint64_t;
50 #else
51 typedef unsigned long long int uint64_t;
52 #endif // __LP64__
53 #endif
54
55 #endif // ifndef XF_SECURITY_TYPES_H
56
57 // -*- cpp -*-
58 // vim: ts=8:sw=2:sts=2:ft=cpp

```

D.5 Vitis_security/xf_security/utls.hpp (44 lines, 1,270 B)

```

1 /*
2  * Copyright 2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
5  * you may not use this file except in compliance with the
License.
6  * You may obtain a copy of the License at
7  *
8  * http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-
missions and
14 * Limitations under the License.
15 */
16
17 /**
18  * @file utls.h
19  * @brief This file is part of XF Database Library, contains
utilities.
20 */
21
22 #ifndef _XF_SECURITY_UTILS_HPP_
23 #define _XF_SECURITY_UTILS_HPP_
24
25 #ifndef __SYNTHESIS__
26 // for assert function.
27 #include <cassert>
28 #define XF_SECURITY_ASSERT(b) assert((b))
29 #else
30 #define XF_SECURITY_ASSERT(b) ((void)0)
31 #endif
32
33 #if __cplusplus >= 201103L
34 #define XF_SECURITY_STATIC_ASSERT(b, m) static_assert((b), m)
35 #else
36 #define XF_SECURITY_STATIC_ASSERT(b, m) XF_SECU-
RITY_ASSERT((b) && (m))
37 #endif
38
39 #define XF_SECURITY_MACRO_QUOTE(s) #s

```

```

40 #define XF_SECURITY_MACRO_STR(s) XF_SECURITY_MACRO_QUOTE(s)
41
42 #endif // XF_SECURITY_UTILS_H
43 // -*- cpp -*-
44 // vim: ts=8:sw=2:sts=2:ft=cpp

```

D.6 Vitis_solver/cholesky.hpp (732 lines, 31,537 B)

```

1 /*
2  * Copyright (C) 2019-2022, Xilinx, Inc.
3  * Copyright (C) 2022-2026, Advanced Micro Devices, Inc.
4  *
5  * Licensed under the Apache License, Version 2.0 (the "Li-
6  * cense");
7  * you may not use this file except in compliance with the
8  * License.
9  * You may obtain a copy of the License at
10  *
11  * http://www.apache.org/licenses/LICENSE-2.0
12  *
13  * Unless required by applicable law or agreed to in writing,
14  * software
15  *
16  * distributed under the License is distributed on an "AS IS"
17  * BASIS,
18  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
19  * press or implied.
20  * See the License for the specific language governing per-
21  * missions and
22  * limitations under the License.
23  */
24 /**
25  * @file cholesky.hpp
26  * @brief This file contains cholesky functions
27  * - cholesky : Entry point function
28  * - choleskyTop : Top level function that se-
29  * lects implementation architecture and internal types based
30  * on a traits class.
31  * - choleskyBasic : Basic implementation requir-
32  * ing lower resource
33  * - choleskyAlt : Lower latency architecture
34  * requiring more resources
35  * - choleskyAlt2 : Further improved latency ar-
36  * chitecture requiring higher resource
37  */
38
39 #ifndef _XF_SOLVER_CHOLESKY_HPP_
40 #define _XF_SOLVER_CHOLESKY_HPP_
41
42 #include "ap_fixed.h"
43 #include "hls_x_complex.h"
44 #include <complex>
45 #include "utils/std_complex_utils.h"
46 #include "utils/x_matrix_utils.hpp"
47 #include "hls_stream.h"
48
49 namespace xf {
50 namespace solver {
51
52 // Default traits struct defining the internal variable types
53 // for the cholesky function
54 template <bool LowerTriangularL, int RowsColsA, typename In-
55 putType, typename OutputType>
56 struct choleskyTraits {
57     typedef InputType PROD_T;
58     typedef InputType ACCUM_T;
59     typedef InputType ADD_T;
60     typedef InputType DIAG_T;
61     typedef InputType RECIP_DIAG_T;
62     typedef InputType OFF_DIAG_T;
63     typedef OutputType L_OUTPUT_T;
64     static const int ARCH =
65
66     1; // Select implementation: 0=Basic, 1=Lower latency
67     architecture, 2=Further improved latency architecture
68     static const int INNER_II = 1; // Specify the pipelining
69     target for the inner loop
70     static const int UNROLL_FACTOR =
71
72     1; // Specify the inner loop unrolling factor for the
73     choleskyAlt2 architecture(2) to increase throughput
74
75     static const int UNROLL_DIM = (LowerTriangularL == true ?
76     1 : 2); // Dimension to unroll matrix
77     static const int ARCH2_ZERO_LOOP =
78
79     true; // Additional implementation "switch" for the
80     choleskyAlt2 architecture (2).
81 };
82
83 // Specialization for complex
84 template <bool LowerTriangularL, int RowsColsA, typename In-
85 putBaseType, typename OutputBaseType>
86 struct choleskyTraits<LowerTriangularL, RowsColsA,
87 hls::x_complex<InputBaseType>, hls::x_complex<OutputBaseType> > {

```

```

66     typedef hls::x_complex<InputBaseType> PROD_T;
67     typedef hls::x_complex<InputBaseType> ACCUM_T;
68     typedef hls::x_complex<InputBaseType> ADD_T;
69     typedef hls::x_complex<InputBaseType> DIAG_T;
70     typedef InputBaseType RECIP_DIAG_T;
71     typedef hls::x_complex<InputBaseType> OFF_DIAG_T;
72     typedef hls::x_complex<OutputBaseType> L_OUTPUT_T;
73     static const int ARCH = 1;
74     static const int INNER_II = 1;
75     static const int UNROLL_FACTOR = 1;
76
77     static const int UNROLL_DIM = (LowerTriangularL == true ?
78     1 : 2);
79     static const int ARCH2_ZERO_LOOP = true;
80 };
81
82 // Specialization for std complex
83 template <bool LowerTriangularL, int RowsColsA, typename In-
84 putBaseType, typename OutputBaseType>
85 struct choleskyTraits<LowerTriangularL, RowsColsA, std::com-
86 plex<InputBaseType>, std::complex<OutputBaseType> > {
87     typedef std::complex<InputBaseType> PROD_T;
88     typedef std::complex<InputBaseType> ACCUM_T;
89     typedef std::complex<InputBaseType> ADD_T;
90     typedef std::complex<InputBaseType> DIAG_T;
91     typedef InputBaseType RECIP_DIAG_T;
92     typedef std::complex<InputBaseType> OFF_DIAG_T;
93     typedef std::complex<OutputBaseType> L_OUTPUT_T;
94     static const int ARCH = 1;
95     static const int INNER_II = 1;
96     static const int UNROLL_FACTOR = 1;
97
98     static const int UNROLL_DIM = (LowerTriangularL == true ?
99     1 : 2);
100     static const int ARCH2_ZERO_LOOP = true;
101 };
102
103 // Specialization for ap_fixed
104 template <bool LowerTriangularL,
105 int RowsColsA,
106 int W1,
107 int I1,
108 ap_q_mode Q1,
109 ap_o_mode O1,
110 int N1,
111 int W2,
112 int I2,
113 ap_q_mode Q2,
114 ap_o_mode O2,
115 int N2>
116 struct choleskyTraits<LowerTriangularL, RowsColsA,
117 ap_fixed<W1, I1, Q1, O1, N1>, ap_fixed<W2, I2, Q2, O2, N2> > {
118     typedef ap_fixed<W1 + W1, I1 + I1, AP_RND_CONV, AP_SAT,
119 0> PROD_T;
120     typedef ap_fixed<(W1 + W1) + BitWidth<RowsColsA>::Value,
121 (I1 + I1) + BitWidth<RowsColsA>::Value,
122 AP_RND_CONV,
123 AP_SAT,
124 0>
125 ACCUM_T;
126     typedef ap_fixed<W1 + 1, I1 + 1, AP_RND_CONV, AP_SAT, 0>
127 ADD_T;
128     typedef ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV,
129 AP_SAT, 0> DIAG_T; // Takes result of sqrt
130     typedef ap_fixed<(W1 + 1) * 2, I1 + 1, AP_RND_CONV,
131 AP_SAT, 0> OFF_DIAG_T; // Takes result of /
132     typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2),
133 AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T;
134     typedef ap_fixed<W2, I2, AP_RND_CONV, AP_SAT, 0>
135 L_OUTPUT_T; // Takes new L value. Same as L output
136 but saturation set
137     static const int ARCH = 1;
138     static const int INNER_II = 1;
139     static const int UNROLL_FACTOR = 1;
140
141     static const int UNROLL_DIM = (LowerTriangularL == true ?
142     1 : 2);
143     static const int ARCH2_ZERO_LOOP = true;
144 };
145
146 // Further specialization for hls::complex<ap_fixed>
147 template <bool LowerTriangularL,
148 int RowsColsA,
149 int W1,
150 int I1,
151 ap_q_mode Q1,
152 ap_o_mode O1,
153 int N1,
154 int W2,
155 int I2,
156 ap_q_mode Q2,
157 ap_o_mode O2,
158 int N2>
159 struct choleskyTraits<LowerTriangularL,
160 RowsColsA,
161 hls::x_complex<ap_fixed<W1, I1, Q1, O1,
162 N1>,
163 hls::x_complex<ap_fixed<W2, I2, Q2, O2,

```

```

N2> > {
148     typedef hls::x_complex<ap_fixed<W1 + W1, I1 + I1,
AP_RND_CONV, AP_SAT, 0> > PROD_T;
149     typedef hls::x_complex<ap_fixed<W1 + W1> +
BitWidth<RowsColsA>::Value,
150                                     (I1 + I1) +
BitWidth<RowsColsA>::Value,
151                                     AP_RND_CONV,
152                                     AP_SAT,
153                                     0> >
154     ACCUM_T;
155     typedef hls::x_complex<ap_fixed<W1 + 1, I1 + 1,
AP_RND_CONV, AP_SAT, 0> > ADD_T;
156     typedef hls::x_complex<ap_fixed<(W1 + 1) * 2, I1 + 1,
AP_RND_CONV, AP_SAT, 0> > DIAG_T; // Takes result of sqrt
157     typedef hls::x_complex<ap_fixed<(W1 + 1) * 2, I1 + 1,
AP_RND_CONV, AP_SAT, 0> > OFF_DIAG_T; // Takes result of /
158     typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2),
AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T;
159     typedef hls::x_complex<ap_fixed<W2, I2, AP_RND_CONV,
AP_SAT, 0> >
160     L_OUTPUT_T; // Takes new L value. Same as L output
but saturation set
161     static const int ARCH = 1;
162     static const int INNER_II = 1;
163     static const int UNROLL_FACTOR = 1;

164     static const int UNROLL_DIM = (LowerTriangularL == true ?
1 : 2);
165     static const int ARCH2_ZERO_LOOP = true;
166 };
167
168 // Further specialization for std::complex<ap_fixed>
169 template <bool LowerTriangularL,
170          int RowsColsA,
171          int W1,
172          int I1,
173          ap_q_mode Q1,
174          ap_o_mode O1,
175          int N1,
176          int W2,
177          int I2,
178          ap_q_mode Q2,
179          ap_o_mode O2,
180          int N2>
181 struct choleskyTraits<LowerTriangularL,
182                      RowsColsA,
183                      std::complex<ap_fixed<W1, I1, Q1, O1,
N1>,
184                      std::complex<ap_fixed<W2, I2, Q2, O2,
N2>> > {
185     typedef std::complex<ap_fixed<W1 + W1, I1 + I1,
AP_RND_CONV, AP_SAT, 0> > PROD_T;
186     typedef std::complex<ap_fixed<(W1 + W1) +
BitWidth<RowsColsA>::Value,
187                                     (I1 + I1) +
BitWidth<RowsColsA>::Value,
188                                     AP_RND_CONV,
189                                     AP_SAT,
190                                     0> >
191     ACCUM_T;
192     typedef std::complex<ap_fixed<W1 + 1, I1 + 1,
AP_RND_CONV, AP_SAT, 0> > ADD_T;
193     typedef std::complex<ap_fixed<(W1 + 1) * 2, I1 + 1,
AP_RND_CONV, AP_SAT, 0> > DIAG_T; // Takes result of sqrt
194     typedef std::complex<ap_fixed<(W1 + 1) * 2, I1 + 1,
AP_RND_CONV, AP_SAT, 0> > OFF_DIAG_T; // Takes result of /
195     typedef ap_fixed<2 + (W2 - I2) + W2, 2 + (W2 - I2),
AP_RND_CONV, AP_SAT, 0> RECIP_DIAG_T;
196     typedef std::complex<ap_fixed<W2, I2, AP_RND_CONV,
AP_SAT, 0> >
197     L_OUTPUT_T; // Takes new L value. Same as L output
but saturation set
198     static const int ARCH = 1;
199     static const int INNER_II = 1;
200     static const int UNROLL_FACTOR = 1;

201     static const int UNROLL_DIM = (LowerTriangularL == true ?
1 : 2);
202     static const int ARCH2_ZERO_LOOP = true;
203 };
204
205 //
=====
206 // Helper functions
207
208 // Square root
209 // o Overloaded versions of the sqrt function
210 // o The square root of a complex number is expensive. How-
ever, the diagonal values of a Cholesky decomposition are
211 // always
212 // real so we don't need a full complex square root.
213 template <typename T_IN, typename T_OUT>
214 int cholesky_sqrt_op(T_IN a, T_OUT& b) {
215     Function_cholesky_sqrt_op_real::
216     const T_IN ZERO = 0;
217     if (a < ZERO) {
218         b = ZERO;
219         return (1);

```

```

220     }
221     b = x_sqrt(a);
222     return (0);
223 }
224 template <typename T_IN, typename T_OUT>
225 int cholesky_sqrt_op(hls::x_complex<T_IN> din, hls::x_com-
plex<T_OUT>& dout) {
226     Function_cholesky_sqrt_op_complex::
227     const T_IN ZERO = 0;
228     T_IN a = din.real();
229     dout.imag(ZERO);
230
231     if (a < ZERO) {
232         dout.real(ZERO);
233         return (1);
234     }
235
236     dout.real(x_sqrt(a));
237     return (0);
238 }
239 template <typename T_IN, typename T_OUT>
240 int cholesky_sqrt_op(std::complex<T_IN> din, std::com-
plex<T_OUT>& dout) {
241     Function_cholesky_sqrt_op_complex::
242     const T_IN ZERO = 0;
243     T_IN a = din.real();
244     dout.imag(ZERO);
245
246     if (a < ZERO) {
247         dout.real(ZERO);
248         return (1);
249     }
250
251     dout.real(x_sqrt(a));
252     return (0);
253 }
254
255 // Reciprocal square root.
256 template <typename InputType, typename OutputType>
257 void cholesky_rsqrt(InputType x, OutputType& res) {
258     Function_cholesky_rsqrt_default::
259     res = x_rsqrt(x);
260 }

261 template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1,
int W2, int I2, ap_q_mode Q2, ap_o_mode O2, int N2>
262 void cholesky_rsqrt(ap_fixed<W1, I1, Q1, O1, N1> x,
ap_fixed<W2, I2, Q2, O2, N2>& res) {
263     Function_cholesky_rsqrt_fixed::
264     ap_fixed<W2, I2, Q2, O2, N2> one = 1;
265     ap_fixed<W1, I1, Q1, O1, N1> sqrt_res;
266     ap_fixed<W2, I2, Q2, O2, N2> sqrt_res_cast;
267     sqrt_res = x_sqrt(x);
268     sqrt_res_cast = sqrt_res;
269     res = one / sqrt_res_cast;
270 }
271
272 // Local multiplier to handle a complex case currently not
supported by the hls::x_complex class
273 // - Complex multiplied by a real of a different type
274 // - Required for complex fixed point implementations
275 template <typename AType, typename BType, typename CType>
276 void cholesky_prod_sum_mult(AType A, BType B, CType& C) {
277     Function_cholesky_prod_sum_mult_real::
278     C = A * B;
279 }
280 template <typename AType, typename BType, typename CType>

281 void cholesky_prod_sum_mult(hls::x_complex<AType> A, BType B,
hls::x_complex<CType>& C) {
282     Function_cholesky_prod_sum_mult_complex::
283     C.real(A.real() * B);
284     C.imag(A.imag() * B);
285 }
286 template <typename AType, typename BType, typename CType>
287 void cholesky_prod_sum_mult(std::complex<AType> A, BType B,
std::complex<CType>& C) {
288     Function_cholesky_prod_sum_mult_complex::
289     C.real(A.real() * B);
290     C.imag(A.imag() * B);
291 }
292
293 //
=====
294 // choleskyBasic
295 template <bool LowerTriangularL, int RowsColsA, typename
CholeskyTraits, class InputType, class OutputType>
296 int choleskyBasic(const InputType A[RowsColsA][RowsColsA],
OutputType L[RowsColsA][RowsColsA]) {
297     int return_code = 0;
298
299     // Use the traits struct to specify the correct type for
the intermediate variables. This is really only needed for
300     // fixed point.
301     typename CholeskyTraits::PROD_T prod;
302     typename CholeskyTraits::ACCUM_T sum[RowsColsA];
303     typename CholeskyTraits::ACCUM_T A_cast_to_sum; // A
with the same dimensions as sum.
304     typename CholeskyTraits::ACCUM_T prod_cast_to_sum; //

```

```

prod with the same dimensions as sum.
305
306     typename CholeskyTraits::ADD_T A_minus_sum;
307     typename CholeskyTraits::DIAG_T new_L_diag; //
sqrt(A_minus_sum)
308     typename CholeskyTraits::OFF_DIAG_T new_L_off_diag; //
sum/L
309     typename CholeskyTraits::OFF_DIAG_T
L_cast_to_new_L_off_diag;
310
311     typename CholeskyTraits::L_OUTPUT_T new_L;
312     OutputType retrieved_L;
313     // Internal memory used to avoid read access from func-
tion output argument L.
314     // NOTE: The internal matrix only needs to be triangular
but optimization using a 1-D array it will require addition
315     // logic to generate the indexes. Refer to the cholesky-
Alt function.
316     OutputType L_internal[RowsColsA][RowsColsA];
317
318 col_loop:
319     for (int j = 0; j < RowsColsA; j++) {
320         sum[j] = 0;
321
322         // Calculate the diagonal value for this column
diag_loop:
323         for (int k = 0; k < RowsColsA; k++) {
324             if (k <= (j - 1)) {
325                 if (LowerTriangularL == true) {
326                     retrieved_L = L_internal[j][k];
327                 } else {
328                     retrieved_L = L_internal[k][j];
329                 }
330                 sum[j] = hls::x_conj(retrieved_L) * re-
trieved_L;
331             }
332         }
333         A_cast_to_sum = A[j][j];
334         A_minus_sum = A_cast_to_sum - sum[j];
335
336         if (cholesky_sqrt_op(A_minus_sum, new_L_diag)) {
337             #ifndef SYNTHESIS
338                 printf("ERROR: Trying to find the square root of
a negative number\n");
339             #endif
340             return_code = 1;
341         }
342
343         // Round to target format using method specified by
traits defined types.
344         new_L = new_L_diag;
345
346         if (LowerTriangularL == true) {
347             L_internal[j][j] = new_L;
348         } else {
349             L_internal[j][j] = hls::x_conj(new_L);
350         }
351
352         // Calculate the off diagonal values for this column
off_diag_loop:
353         for (int i = 0; i < RowsColsA; i++) {
354             if (i > j) {
355                 if (LowerTriangularL == true) {
356                     sum[j] = A[i][j];
357                 } else {
358                     sum[j] = hls::x_conj(A[j][i]);
359                 }
360
361                 sum_loop:
362                 for (int k = 0; k < RowsColsA; k++) {
363                     #pragma HLS PIPELINE II = CholeskyTraits::INNER_II
364                     if (k <= (j - 1)) {
365                         if (LowerTriangularL == true) {
366                             prod = -L_internal[i][k] *
hls::x_conj(L_internal[j][k]);
367                         } else {
368                             prod = -hls::x_conj(L_internal[k]
[i]) * (L_internal[k][j]);
369                         }
370                         prod_cast_to_sum = prod;
371                         sum[j] += prod_cast_to_sum;
372                     }
373                 }
374
375                 new_L_off_diag = sum[j];
376                 L_cast_to_new_L_off_diag = L_internal[j][j];
377
378                 // Diagonal is always real, avoid complex di-
vision
379                 new_L_off_diag = new_L_off_diag /
hls::x_real(L_cast_to_new_L_off_diag);
380
381                 // Round to target format using method spe-
cified by traits defined types.
382                 new_L = new_L_off_diag;
383

```

```

390         if (LowerTriangularL == true) {
391             L[i][j] = new_L;
392             L_internal[i][j] = new_L;
393         } else {
394             L[j][i] = hls::x_conj(new_L);
395             L_internal[j][i] = hls::x_conj(new_L);
396         }
397     }
398     } else if (i < j) {
399         if (LowerTriangularL == true) {
400             L[i][j] = 0;
401         } else {
402             L[j][i] = 0;
403         }
404     }
405 }
406 }
407 return (return_code);
408 }
409
410 //
=====
411 // choleskyAlt: Alternative architecture with improved
latency at the expense of higher resource
412 template <bool LowerTriangularL, int RowsColsA, typename
CholeskyTraits, class InputType, class OutputType>
413 int choleskyAlt(const InputType A[RowsColsA][RowsColsA], Out-
putType L[RowsColsA][RowsColsA]) {
414     int return_code = 0;
415
416     // Optimize internal memories
417     // - For complex data types the diagonal will be real
only, plus for fixed point it must be stored to a
418     // higher precision.
419     // - Requires additional logic to generate the memory in-
dexes
420     // - For smaller matrix sizes there maybe be an increase
in memory usage
421     OutputType L_internal[(RowsColsA * RowsColsA -
RowsColsA) / 2];
422     typename CholeskyTraits::RECIP_DIAG_T diag_intern-
al[RowsColsA];
423
424     typename CholeskyTraits::ACCUM_T square_sum;
425     typename CholeskyTraits::ACCUM_T A_cast_to_sum;
426     typename CholeskyTraits::ADD_T A_minus_sum;
427     typename CholeskyTraits::DIAG_T A_minus_sum_cast_diag;
428     typename CholeskyTraits::DIAG_T new_L_diag;
429     typename CholeskyTraits::RECIP_DIAG_T new_L_diag_recip;
430     typename CholeskyTraits::PROD_T prod;
431     typename CholeskyTraits::ACCUM_T prod_cast_to_sum;
432     typename CholeskyTraits::ACCUM_T product_sum;
433     typename CholeskyTraits::OFF_DIAG_T
prod_cast_to_off_diag;
434     typename CholeskyTraits::RECIP_DIAG_T L_diag_recip;
435     typename CholeskyTraits::OFF_DIAG_T new_L_off_diag;
436     typename CholeskyTraits::L_OUTPUT_T new_L;
437     typename CholeskyTraits::L_OUTPUT_T new_L_recip;
438
439 row_loop:
440     for (int i = 0; i < RowsColsA; i++) {
441         // Index generation for optimized/packed L_internal
memory
442         int i_sub1 = i - 1;
443         int i_off = ((i_sub1 * i_sub1 - i_sub1) / 2) +
i_sub1;
444
445         // Off diagonal calculation
446         square_sum = 0;
447         col_loop:
448         for (int j = 0; j < i; j++) {
449             #pragma HLS loop_tripcount max = 1 + RowsColsA / 2
450             // Index generation
451             int j_sub1 = j - 1;
452             int j_off = ((j_sub1 * j_sub1 - j_sub1) / 2) +
j_sub1;
453             // Prime the off-diagonal sum with target ele-
ments A value.
454             if (LowerTriangularL == true) {
455                 product_sum = A[i][j];
456             } else {
457                 product_sum = hls::x_conj(A[j][i]);
458             }
459             sum_loop:
460             for (int k = 0; k < j; k++) {
461                 #pragma HLS loop_tripcount max = 1 + RowsColsA / 2
462                 #pragma HLS PIPELINE II = CholeskyTraits::INNER_II
463                 prod = -L_internal[i_off + k] *
hls::x_conj(L_internal[j_off + k]);
464                 prod_cast_to_sum = prod;
465                 product_sum += prod_cast_to_sum;
466             }
467             prod_cast_to_off_diag = product_sum;
468             // Fetch diagonal value
469             L_diag_recip = diag_intern[j];
470             // Diagonal is stored in its reciprocal form so
only need to multiply the product sum
471             cholesky_prod_sum_mult(prod_cast_to_off_diag,
L_diag_recip, new_L_off_diag);

```

```

472 // Round to target format using method specified
by traits defined types.
473 new_L = new_L_off_diag;
474 // Build sum for use in diagonal calculation for
this row.
475 square_sum += hls::x_conj(new_L) * new_L;
476 // Store result
477 L_internal[i_off + j] = new_L;
478 if (LowerTriangularL == true) {
479     L[i][j] = new_L; // store in lower triangle
480     L[j][i] = 0; // Zero upper
481 } else {
482     L[j][i] = hls::x_conj(new_L); // store in up-
per triangle
483     L[i][j] = 0; // Zero lower
484 }
485 }
486 // Diagonal calculation
487 A_cast_to_sum = A[i][i];
488 A_minus_sum = A_cast_to_sum - square_sum;
489 if (cholesky_sqrt_op(A_minus_sum, new_L_diag)) {
490 #ifndef __SYNTHESIS__
491     printf("ERROR: Trying to find the square root of
a negative number\n");
492 #endif
493     return_code = 1;
494 }
495 // Round to target format using method specified by
traits defined types.
496 new_L = new_L_diag;
497 // Generate the reciprocal of the diagonal for in-
ternal use to avoid the latency of a divide in every
498 // off-diagonal calculation
499 A_minus_sum_cast_diag = A_minus_sum;
500 cholesky_rsqrtsqrt(hls::x_real(A_minus_sum_cast_diag),
new_L_diag_recip);
501 // Store diagonal value
502 diag_internal[i] = new_L_diag_recip;
503 if (LowerTriangularL == true) {
504     L[i][i] = new_L;
505 } else {
506     L[i][i] = hls::x_conj(new_L);
507 }
508 }
509 return (return_code);
510 }
511 }
512 //
513 //
=====
514 // choleskyAlt2: Further improved latency architecture re-
quiring higher resource
515 template <bool LowerTriangularL, int RowsColsA, typename
CholeskyTraits, class InputType, class OutputType>
516 int choleskyAlt2(const InputType A[RowsColsA][RowsColsA],
OutputType L[RowsColsA][RowsColsA]) {
517     int return_code = 0;
518 }
519 // To avoid array index calculations every iteration this
architecture uses a simple 2D array rather than a
520 // optimized/packed triangular matrix.
521 OutputType L_internal[RowsColsA][RowsColsA];
522 OutputType prod_column_top;
523 typename CholeskyTraits::ACCUM_T
square_sum_array[RowsColsA];
524 typename CholeskyTraits::ACCUM_T A_cast_to_sum;
525 typename CholeskyTraits::ADD_T A_minus_sum;
526 typename CholeskyTraits::DIAG_T A_minus_sum_cast_diag;
527 typename CholeskyTraits::DIAG_T new_L_diag;
528 typename CholeskyTraits::RECIP_DIAG_T new_L_diag_recip;
529 typename CholeskyTraits::PROD_T prod;
530 typename CholeskyTraits::ACCUM_T prod_cast_to_sum;
531 typename CholeskyTraits::ACCUM_T product_sum;
532 typename CholeskyTraits::ACCUM_T
product_sum_array[RowsColsA];
533 typename CholeskyTraits::OFF_DIAG_T
prod_cast_to_off_diag;
534 typename CholeskyTraits::OFF_DIAG_T new_L_off_diag;
535 typename CholeskyTraits::L_OUTPUT_T new_L;
536
537 #pragma HLS ARRAY_PARTITION variable = A cyclic dim =
CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR
538 #pragma HLS ARRAY_PARTITION variable = L cyclic dim =
CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR
539 #pragma HLS ARRAY_PARTITION variable = L_internal cyclic dim
= CholeskyTraits::UNROLL_DIM factor = \
CholeskyTraits::UNROLL_FACTOR
540 #pragma HLS ARRAY_PARTITION variable = square_sum_array cyc-
lic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR
541 #pragma HLS ARRAY_PARTITION variable = product_sum_array cyc-
lic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR
542
543 col_loop:
544 for (int j = 0; j < RowsColsA; j++) {
545     // Diagonal calculation
546     A_cast_to_sum = A[j][j];
547     if (j == 0) {
548         A_minus_sum = A_cast_to_sum;
549         } else {
550             A_minus_sum = A_cast_to_sum -
square_sum_array[j];
551         }
552         if (cholesky_sqrt_op(A_minus_sum, new_L_diag)) {
553             #ifndef __SYNTHESIS__
554                 printf("ERROR: Trying to find the square root of
a negative number\n");
555             #endif
556             return_code = 1;
557         }
558         // Round to target format using method specified by
traits defined types.
559         new_L = new_L_diag;
560         // Generate the reciprocal of the diagonal for in-
ternal use to avoid the latency of a divide in every
561         // off-diagonal calculation
562         A_minus_sum_cast_diag = A_minus_sum;
563         cholesky_rsqrtsqrt(hls::x_real(A_minus_sum_cast_diag),
new_L_diag_recip);
564         // Store diagonal value
565         if (LowerTriangularL == true) {
566             L[j][j] = new_L;
567         } else {
568             L[j][j] = hls::x_conj(new_L);
569         }
570     }
571     sum_loop:
572     for (int k = 0; k <= j; k++) {
573         // Define average trip count for reporting, loop reduces in
length for every iteration of col_loop
574         #pragma HLS loop_tripcount max = 1 + RowsColsA / 2
575         // Same value used in all calcs
576         // o Implement -i* here
577         prod_column_top = -hls::x_conj(L_internal[j][k]);
578         // NOTE: Using a fixed loop length combined with a
579         // "if" to implement reducing loop length
580         // o Ensures the inner loop can achieve the maximum
II (1)
581         // o May introduce a small overhead resolving the
582         // "if" statement but HLS struggled to schedule when the variable
583         // loop bound expression was used.
584         // o Will report inaccurate trip count as it will re-
duce by one with the col_loop
585         // o Variable loop bound code: row_loop: for(int i =
j+1; i < RowsColsA; i++) {
586             row_loop:
587             for (int i = 0; i < RowsColsA; i++) {
588                 // IMPORTANT: row_loop must not merge with sum_loop as the
merged loop becomes variable length and HLS will struggle
589                 // with scheduling
590                 #pragma HLS LOOP_FLATTEN off
591                 #pragma HLS PIPELINE II = CholeskyTraits::INNER_II
592                 #pragma HLS UNROLL FACTOR = CholeskyTraits::UNROLL_FACTOR
593                 if (i > j) {
594                     prod = L_internal[i][k] *
prod_column_top;
595                     prod_cast_to_sum = prod;
596                     if (k == 0) {
597                         // Prime first sum
598                         if (LowerTriangularL == true) {
599                             A_cast_to_sum = A[i][j];
600                         } else {
601                             A_cast_to_sum = hls::x_conj(A[j]
[i]);
602                         }
603                         product_sum = A_cast_to_sum;
604                     } else {
605                         product_sum = product_sum_array[i];
606                     }
607                     if (k < j) {
608                         // Accumulate row sum of columns
609                         product_sum_array[i] = product_sum +
prod_cast_to_sum;
610                     } else {
611                         // Final calculation for off diagonal
value
612                         prod_cast_to_off_diag = product_sum;
613                         // Diagonal is stored in its recip-
rocal form so only need to multiply the product sum
614                         cholesky_prod_sum_mult(prod_cast_to_off_diag, new_L_diag_recip,
new_L_off_diag);
615                         // Round to target format using meth-
od specified by traits defined types.
616                         new_L = new_L_off_diag;
617                         // Build sum for use in diagonal cal-
culation for this row.
618                         if (k == 0) {
619                             square_sum_array[j] =
hls::x_conj(new_L) * new_L;
620                         } else {
621                             square_sum_array[j] =
hls::x_conj(new_L) * new_L;
622                         }
623                     }
624                 }
625             }

```



```

626         // Store result
627         L_internal[i][j] = new_L;

628         // NOTE: Use the upper/lower triangle
        zeroing in the subsequent loop so the double memory access
629         // does not
630         // become a bottleneck

631         // o Results in a further increase of
        DSP resources due to the higher II of this loop.
632         // o Retaining the zeroing operation
        here can give this a loop a max II of 2 and HLS will
633         // resource share.
634         if (LowerTriangularL == true) {
635             L[i][j] =
new_L;
636             if (!
CholeskyTraits::ARCH2_ZERO_LOOP) L[j][i] = 0; // Zero upper
637             } else {
638                 L[j][i] =
hls::x_conj(new_L);
639                 if (!
CholeskyTraits::ARCH2_ZERO_LOOP) L[i][j] = 0; // Zero lower
640             }
641         }
642     }
643 }
644 }
645 }
646 // Zero upper/lower triangle
647 // o Use separate loop to ensure main calculation can
        achieve an II of 1
648 // o As noted above this may increase the DSP resources.
649 // o Required when unrolling the inner loop due to array
        dimension access
650 if (CholeskyTraits::ARCH2_ZERO_LOOP) {
651     zero_rows_loop:
        for (int i = 0; i < RowsColsA - 1; i++) {
652         zero_cols_loop:
        for (int j = i + 1; j < RowsColsA; j++) {
653             // Define average trip count for reporting, loop reduces in
        length for every iteration of zero_rows_loop
654 #pragma HLS loop_tripcount max = 1 + RowsColsA / 2
655 #pragma HLS PIPELINE
656             if (LowerTriangularL == true) {
657                 L[i][j] = 0; // Zero upper
658             } else {
659                 L[j][i] = 0; // Zero lower
660             }
661         }
662     }
663 }
664 }
665 }
666 return (return_code);
667 }
668 //
669 //
=====
670 // choleskyTop: Top level function that selects implementa-
        tion architecture and internal types based on the
671 // traits class provided via the CholeskyTraits template
        parameter.
672 // o Call this function directly if you wish to override the
        default architecture choice or internal types
673 template <bool LowerTriangularL, int RowsColsA, typename
        CholeskyTraits, class InputType, class OutputType>
674 int choleskyTop(const InputType A[RowsColsA][RowsColsA], Out-
        putType L[RowsColsA][RowsColsA]) {
675     switch (CholeskyTraits::ARCH) {
676     case 0:
677         return choleskyBasic<LowerTriangularL, RowsColsA,
        CholeskyTraits, InputType, OutputType>(A, L);
678     case 1:
679         return choleskyAlt<LowerTriangularL, RowsColsA,
        CholeskyTraits, InputType, OutputType>(A, L);
680     case 2:
681         return choleskyAlt2<LowerTriangularL, RowsColsA,
        CholeskyTraits, InputType, OutputType>(A, L);
682     default:
683         return choleskyBasic<LowerTriangularL, RowsColsA,
        CholeskyTraits, InputType, OutputType>(A, L);
684     }
685 }
686 /**
687 * @brief cholesky
688 *
689 * @tparam LowerTriangularL When false generates the result
        in the upper triangle
690 * @tparam RowsColsA Defines the matrix dimensions
691 * @tparam InputType Input data type
692 * @tparam OutputType Output data type
693 * @tparam TRAITS choleskyTraits class
694 *
695 * @param matrixAStm Stream of Hermitian/symmetric
        positive definite input matrix
696 * @param matrixLStm Stream of Lower or upper trian-
        gular output matrix

```

```

698 *
699 * @return An integer type. 0=Success.
        1=Failure. The function attempted to find the square root of
        700 * a negative number i.e. the input matrix A was not Her-
        mitian/symmetric positive definite.
701 */
702 template <bool LowerTriangularL,
703         int RowsColsA,
704         class InputType,
705         class OutputType,
706         typename TRAITS = choleskyTraits<LowerTriangularL,
        RowsColsA, InputType, OutputType> >
707 int cholesky(hls::stream<InputType>& matrixAStm,
        hls::stream<OutputType>& matrixLStm) {
708     InputType A[RowsColsA][RowsColsA];
709     OutputType L[RowsColsA][RowsColsA];
710     for (int r = 0; r < RowsColsA; r++) {
711         #pragma HLS PIPELINE
712         for (int c = 0; c < RowsColsA; c++) {
713             matrixAStm.read(A[r][c]);
714         }
715     }
716     int ret = 0;
717     ret = choleskyTop<LowerTriangularL, RowsColsA, TRAITS,
        InputType, OutputType>(A, L);
718     for (int r = 0; r < RowsColsA; r++) {
719         #pragma HLS PIPELINE
720         for (int c = 0; c < RowsColsA; c++) {
721             matrixLStm.write(L[r][c]);
722         }
723     }
724     return ret;
725 }
726 }
727 }
728 }
729 }
730 } // end namespace solver
731 } // end namespace xf
732 #endif

```

D.7 Vitis_solver/qrf.hpp (889 lines, 29,856 B)

```

1 /*
2  * Copyright (C) 2019-2022, Xilinx, Inc.
3  * Copyright (C) 2022-2025, Advanced Micro Devices, Inc.
4  *
5  * Licensed under the Apache License, Version 2.0 (the "Li-
        cense");
6  * you may not use this file except in compliance with the
        License.
7  * You may obtain a copy of the License at
8  *
9  *     http://www.apache.org/licenses/LICENSE-2.0
10 *
11 * Unless required by applicable law or agreed to in writing,
        software
12 * distributed under the License is distributed on an "AS IS"
        BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
        press or implied.
14 * See the License for the specific language governing per-
        missions and
15 * limitations under the License.
16 */
17
18 /**
19  * @file qrf.hpp
20  * @brief This file contains QRF functions
21  * - QRF : Entry point function
22  * - QRF_BASIC : Basic implementation requiring
        lower resource
23  * - QRF_ALT : Optimized for throughput requir-
        ing more resources
24  */
25
26 #ifndef _XF_SOLVER_QRF_HPP_
27 #define _XF_SOLVER_QRF_HPP_
28
29 #include "utils/x_matrix_utils.hpp"
30 #include "hls_x_complex.h"
31 #include <complex>
32 #include "utils/std_complex_utils.h"
33 #include "hls_stream.h"
34 #ifndef __SYNTHESIS__
35 #include <stdio.h>
36 #endif
37 #include <stdlib.h>
38 #include "hls_math.h"
39
40 namespace xf {
41 namespace solver {
42
43 //
=====
44 // Traits struct defining architecture selection
45 struct qrfTraits {

```

```

46     static const int ARCH = 1;           // Select implementa-
tion. 0=Basic. 1=Lower latency/throughput architecture.

47     static const int CALC_ROT_II = 1; // Specify the rotation
calculation loop target II of the QRF_ALT architecture(1)
48     static const int UPDATE_II = 4;    // Specify the pipelin-
ing target for the Q & R update loops
49     static const int UNROLL_FACTOR =
50     1; // Specify the unrolling factor for Q & R update
loops of the QRF_ALT architecture(1)
51 };
52
53 //
=====
54 // Helper functions
55 template <typename T>
56 struct is_cplx {
57     static const bool value = 0;
58 };
59
60 template <typename T>
61 struct is_cplx<hls::x_complex<T> > {
62     static const bool value = 1;
63 };
64
65 template <typename T>
66 struct is_cplx<std::complex<T> > {
67     static const bool value = 1;
68 };
69
70 template <typename T>
71 int qrf_require_extra_pass(const int rows, const int cols, T
x) {
72     qrf_require_extra_pass_real;
73     return 0;
74 }
75
76 template <typename T>
77 int qrf_require_extra_pass(const int rows, const int cols,
hls::x_complex<T> x) {
78     qrf_require_extra_pass_complex;
79     if (rows == cols) {
80         // Unitary transformation only required for square
matrices
81         return 1;
82     } else {
83         return 0;
84     }
85 }
86
87 template <typename T>
88 int qrf_require_extra_pass(const int rows, const int cols,
std::complex<T> x) {
89     qrf_require_extra_pass_complex;
90     if (rows == cols) {
91         // Unitary transformation only required for square
matrices
92         return 1;
93     } else {
94         return 0;
95     }
96 }
97
98 template <class T>
99 bool is_zero(T a) {
100     return a == 0;
101 }
102
103 template <>
104 bool is_zero<float>(float a) {
105     union single_cast v;
106     union single_cast zero_p;
107     union single_cast zero_n;
108     v.f = a;
109     zero_p.f = +0.0f;
110     zero_n.f = -0.0f;
111     return (v.i == zero_p.i || v.i == zero_n.i);
112 }
113
114 template <>
115 bool is_zero<double>(double a) {
116     union double_cast v;
117     union double_cast zero_p;
118     union double_cast zero_n;
119     v.d = a;
120     zero_p.d = +0.0f;
121     zero_n.d = -0.0f;
122     return (v.i == zero_p.i || v.i == zero_n.i);
123 }
124
125 // Vector multiply
126 template <typename T>
127 T qrf_vm(T a, T b, T c, T d) {
128     return a * c + b * d;
129 }
130
131 // Matrix-vector multiply
132 template <typename T>
133 void qrf_mm(const T G[2][2], T& a, T& b) {
134     T x, y;
135     x = qrf_vm(G[0][0], G[0][1], a, b);
136     y = qrf_vm(G[1][0], G[1][1], a, b);
137 }
138
139 //
140 template <typename T>
141 void qrf_mm_or_mag(const T G[2][2], T& a, T& b, const T mag,
const int use_mag, const int extra_pass) {
142     T x, y;
143     x = qrf_vm(G[0][0], G[0][1], a, b);
144     y = qrf_vm(G[1][0], G[1][1], a, b);
145     if (use_mag == 0) {
146         a = x;
147         b = y;
148     } else {
149         if (extra_pass == 1) {
150             // a = a;
151             b = hls::x_real(mag);
152         } else {
153             a = hls::x_real(mag);
154             b = 0;
155         }
156     }
157 }
158
159 // Magnitude computation for real Givens rotation, guarding
against under-/overflow
160 // - Returns real-valued magnitude
161 template <typename T>
162 T qrf_magnitude(T a, T b) {
163     Function_qrf_magnitude_real;
164     const T ONE = 1.0;
165     const T ZERO = 0.0;
166     T abs_a, abs_b;
167     T y, mag;
168
169     abs_a = hls::abs(a); // declared in x_hls_utils.h
170     abs_b = hls::abs(b);
171     if (abs_a > abs_b) {
172         y = b / a;
173         mag = a * x_sqrt(ONE + y * y);
174     } else if (abs_b == ZERO) {
175         mag = ZERO;
176     } else {
177         y = a / b;
178         mag = b * x_sqrt(ONE + y * y);
179     }
180     return mag;
181 }
182
183 // Magnitude computation for complex Givens rotation, avoid-
ing squaring values which have just been square-rooted
184 // and guarding against under-/overflow
185 // - Returns real-valued magnitude
186 template <typename T>
187 T qrf_magnitude(hls::x_complex<T> a, hls::x_complex<T> b) {
188     Function_qrf_magnitude_complex;
189     const T ONE = 1.0;
190     const T ZERO = 0.0;
191     T ar, ai, br, bi, d1, d2, d3, d3a, d3b, x1, x2, x3;
192     T a_largest, b_largest, largest;
193     ar = hls::abs(a.real());
194     ai = hls::abs(a.imag());
195     br = hls::abs(b.real());
196     bi = hls::abs(b.imag());
197
198     // Lower latency, builds tree structures.
199     if (ar > ai && ar > br && ar > bi) {
200         largest = ar;
201         d1 = ai;
202         d2 = br;
203         d3 = bi;
204     } else if (ai > ar && ai > br && ai > bi) {
205         largest = ai;
206         d1 = ar;
207         d2 = br;
208         d3 = bi;
209     } else if (br > ar && br > ai && br > bi) {
210         largest = br;
211         d1 = ar;
212         d2 = ai;
213         d3 = bi;
214     } else {
215         largest = bi;
216         d1 = ar;
217         d2 = ai;
218         d3 = br;
219     }
220
221 if (largest == ZERO) { // Avoid divide-by-zero
222     return ZERO;
223 } else {
224     x1 = d1 / largest;
225     x2 = d2 / largest;
226     x3 = d3 / largest;
227     T x1_sqd = x1 * x1;
228     T x2_sqd = x2 * x2;
229     T x3_sqd = x3 * x3;
230 }

```

```

231     T x3_sqd = x3 * x3;
232
233     T s1 = ONE + x1_sqd;
234     T s2 = x2_sqd + x3_sqd;
235     T s3 = s1 + s2;
236
237     return largest * x_sqrt(s3);
238 }
239 }
240
241 // Magnitude computation for complex Givens rotation, avoid-
ing squaring values which have just been square-rooted
242 // and guarding against under-/overflow
243 // - Returns real-valued magnitude
244 template <typename T>
245 T qrf_magnitude(std::complex<T> a, std::complex<T> b) {
246     Function_qrf_magnitude_complex;
247
248     const T ONE = 1.0;
249     const T ZERO = 0.0;
250     T ar, ai, br, bi, d1, d2, d3, d3a, d3b, x1, x2, x3;
251     T a_largest, b_largest, largest;
252     ar = hls::abs(a.real());
253     ai = hls::abs(a.imag());
254     br = hls::abs(b.real());
255     bi = hls::abs(b.imag());
256
257     // Lower latency, builds tree structures.
258     if (ar > ai && ar > br && ar > bi) {
259         largest = ar;
260         d1 = ai;
261         d2 = br;
262         d3 = bi;
263     } else if (ai > ar && ai > br && ai > bi) {
264         largest = ai;
265         d1 = ar;
266         d2 = br;
267         d3 = bi;
268     } else if (br > ar && br > ai && br > bi) {
269         largest = br;
270         d1 = ar;
271         d2 = ai;
272         d3 = bi;
273     } else {
274         largest = bi;
275         d1 = ar;
276         d2 = ai;
277         d3 = br;
278     }
279
280     if (largest == ZERO) { // Avoid divide-by-zero
281         return ZERO;
282     } else {
283         x1 = d1 / largest;
284         x2 = d2 / largest;
285         x3 = d3 / largest;
286         T x1_sqd = x1 * x1;
287         T x2_sqd = x2 * x2;
288         T x3_sqd = x3 * x3;
289
290         T s1 = ONE + x1_sqd;
291         T s2 = x2_sqd + x3_sqd;
292         T s3 = s1 + s2;
293
294         return largest * x_sqrt(s3);
295     }
296 }
297
298 //
=====
299 // Real Givens rotation guarding against under-/overflow
situations.
300 //
301 // Returns matrix G = | c s |
302 //                    | ss cc |
303 //
304 // Note: argument 'extra_pass' is not used for the real case,
but is present only to permit function overloading
305 //
306 template <typename T>
307 void qrf_givens(int extra_pass, T a, T b, T& c, T& s, T& ss,
T& cc, T& r) {
308     Function_qrf_givens_real;
309     const T ONE = 1.0;
310     const T ZERO = 0.0;
311     T mag;
312
313     mag = qrf_magnitude(a, b);
314
315     if (hls::abs(a) == ZERO && hls::abs(b) == ZERO) { // more
efficient than "if (mag == ZERO)"
316         c = x_copysign(ONE, a);
317         s = ZERO;
318     } else {
319         c = a / mag;
320         s = b / mag;
321     }
322
323     cc = c;
324     ss = -s;
325     r = mag;
326 }
327
328 //
=====
329 // Complex Givens rotation
330 //
331 // This implements a modified Givens rotation of the form:
332 //
333 // G = | c* s* |
334 //     | -s c |
335 //
336 // to produce real diagonal elements suitable for subsequent
computation of the inverse of input matrix A.
337 //
338 // Returns matrix G = | c s |
339 //                    | ss cc |
340 //
341 // This implementation does not use the same approach as the
version of qrf_givens() for real data, as that
342 // would require that a divider for complex data be implemen-
ted, which is expensive.
343 //
344 // When argument 'extra_pass' is set to 1, the function com-
putes a unitary transformation rather than a standard Givens
345 // matrix.
346 // This is required to ensure that the bottom-rightmost ele-
ment of the R matrix is real. This transformation matrix has
347 // the form:
348 //
349 // G(trans) = | 1 0 |
350 //             | 0 e^-j*T |
351 // where T = Theta for the bottom-rightmost element
352 //
353 template <typename T>
354 void qrf_givens(int extra_pass,
hls::x_complex<T> a,
hls::x_complex<T> b,
hls::x_complex<T>& c,
hls::x_complex<T>& s,
hls::x_complex<T>& ss,
hls::x_complex<T>& cc,
hls::x_complex<T>& r) {
355     Function_qrf_givens_complex;
356     const T ONE = 1.0;
357     const T ZERO = 0.0;
358     const hls::x_complex<T> CZERO = ZERO;
359     T sqrt_mag_a_mag_b;
360     hls::x_complex<T> c_tmp, s_tmp;
361
362     if (extra_pass == 0) {
363         // Standard modified Givens matrix, guarding against
over-/underflow
364         sqrt_mag_a_mag_b = qrf_magnitude(a, b);
365         if (is_zero(hls::abs(a.real())) &&
is_zero(hls::abs(a.imag())) && is_zero(hls::abs(b.real())) &&
is_zero(hls::abs(b.imag()))) { // more efficient
than "if (sqrt_mag_a_mag_b == ZERO)"
366             c_tmp = x_copysign(ONE, a.real());
367             s_tmp = ZERO;
368         } else {
369             c_tmp = a / sqrt_mag_a_mag_b;
370             s_tmp = b / sqrt_mag_a_mag_b;
371         }
372         c = hls::x_conj(c_tmp);
373         cc = c_tmp;
374         s = hls::x_conj(s_tmp);
375         ss = -s_tmp;
376
377         r.real() = sqrt_mag_a_mag_b;
378     } else {
379         // Transformation matrix to ensure real diagonal in
R, guarding against over-/underflow
380         sqrt_mag_a_mag_b = qrf_magnitude(CZERO, b);
381         c_tmp = ONE;
382
383         if (hls::abs(b.real()) == ZERO &&
hls::abs(b.imag()) == ZERO) { // more efficient
than "if (sqrt_mag_a_mag_b == ZERO)"
384             s_tmp = ONE;
385         } else {
386             s_tmp = b / sqrt_mag_a_mag_b;
387         }
388         c = c_tmp;
389         cc = hls::x_conj(s_tmp);
390         s = ZERO;
391         ss = ZERO;
392         r.real() = sqrt_mag_a_mag_b;
393     }
394 }
395
396 template <typename T>
397 void qrf_givens(int extra_pass,
std::complex<T> a,

```



```

410         std::complex<T> b,
411         std::complex<T>& c,
412         std::complex<T>& s,
413         std::complex<T>& ss,
414         std::complex<T>& cc,
415         std::complex<T>& r) {
416     Function_qrf_givens_complex;
417     const T ONE = 1.0;
418     const T ZERO = 0.0;
419     const std::complex<T> CZERO = ZERO;
420     T sqrt_mag_a_mag_b;
421     std::complex<T> c_tmp, s_tmp;
422
423     if (extra_pass == 0) {
424         // Standard modified Givens matrix, guarding against
over-/underflow
425         sqrt_mag_a_mag_b = qrf_magnitude(a, b);
426         if (is_zero(hls::abs(a.real())) &&
is_zero(hls::abs(a.imag())) && is_zero(hls::abs(b.real())) &&
427         is_zero(hls::abs(b.imag())) { // more efficient
than "if (sqrt_mag_a_mag_b == ZERO)"
428             c_tmp = x_copysign(ONE, a.real());
429             s_tmp = ZERO;
430         } else {
431             c_tmp = a / sqrt_mag_a_mag_b;
432             s_tmp = b / sqrt_mag_a_mag_b;
433         }
434         c = hls::x_conj(c_tmp);
435         cc = c_tmp;
436         s = hls::x_conj(s_tmp);
437         ss = -s_tmp;
438
439         r.real(sqrt_mag_a_mag_b);
440     } else {
441         // Transformation matrix to ensure real diagonal in
R, guarding against over-/underflow
442         sqrt_mag_a_mag_b = qrf_magnitude(CZERO, b);
443
444         c_tmp = ONE;
445
446         if (hls::abs(b.real()) == ZERO &&
447         hls::abs(b.imag()) == ZERO) { // more efficient
than "if (sqrt_mag_a_mag_b == ZERO)"
448             s_tmp = ONE;
449         } else {
450             s_tmp = b / sqrt_mag_a_mag_b;
451         }
452
453         c = c_tmp;
454         cc = hls::x_conj(s_tmp);
455         s = ZERO;
456         ss = ZERO;
457         r.real(sqrt_mag_a_mag_b);
458     }
459 }
460
461 //
=====
462 // Configuration class for QRF_ALT implementation
=====
463 // o Determines the ROM content for the address sequence used
to zero the array elements plus the number and size of
464 // the batches pushed through the inner loop. The batches
consist of independent address accesses.
465 // o Initially calculates a static constant batch estimate
which is used to size the look up tables.
466 // o Then an accurate batch count is generated by calculating
the actually processing sequence.
467 template <int ROWS, int COLS, typename InputType>
468 struct qrf_alt_config {
469     // Catch for div by 0 and ensure we infer a ROM for the
lookup arrays
470     static const int ROWS_INT = (ROWS < 5 ? 5 : ROWS);
471     static const int COLS_INT = (COLS < 5 ? 5 : COLS);
472
473     // Intermediate values used to calculate NUM_BATCHES_EST
for rectangular cases
474     static const int SEQ_LEN_FULL_SQ = (ROWS_INT * (ROWS_INT
- 1) / 2);
475     static const int NUM_BATCHES_FULL_SQ_EST =
SEQ_LEN_FULL_SQ * 4 / ROWS_INT;
476     static const int NUM_BATCHES_RECT_EST =
NUM_BATCHES_FULL_SQ_EST - (ROWS_INT - COLS_INT) - (ROWS_INT /
COLS_INT);
477
478     // Triangle + square
479     static const int SEQUENCE_LENGTH = (COLS_INT * (COLS_INT
- 1) / 2) + ((ROWS - COLS) * COLS) +
480     (is_cm-
plx<InputType>::value && ROWS_INT == COLS_INT ? 1 : 0);
481     static const int NUM_BATCHES_EST = (ROWS_INT ==
COLS_INT ? SEQUENCE_LENGTH * 4 / COLS_INT : NUM_BATCHES_RECT_EST);
482
483     // Actual number of batches calculated in the constructor
484     int NUM_BATCHES;
485     // SEQUENCE & BATCH_CNTS should implement as roms.
486     int BATCH_CNTS[NUM_BATCHES_EST];
487     int SEQUENCE[SEQUENCE_LENGTH][3];

```

```

488
489     qrf_alt_config() {
490         int available[SEQUENCE_LENGTH][COLS];
491         int available_cnt[COLS];
492         int available_first[COLS];
493         int available_last[COLS];
494         int zeroed[COLS];
495         int a, b, num_avail, tmp;
496
497         int cnt = 0; // Counts how many pairs we process in a
batch
498         int seq_cnt = 0;
499         int actual_num_batches;
500
501         // Initialize first column and counters
init_indices:
502         for (int row = 0; row < ROWS; row++) {
503             available[row][0] = row;
504         }
505         init_counts:
506         for (int col = 0; col < COLS; col++) {
507             if (col == 0) {
508                 available_cnt[col] = ROWS;
509                 available_last[col] = ROWS;
510             } else {
511                 available_cnt[col] = 0;
512                 available_last[col] = 0;
513             }
514             available_first[col] = 0;
515             zeroed[col] = 0;
516         }
517         // Increment through the processing sequence
518         NUM_BATCHES = 0; // Set to zero so we can test if
we've completed within the estimate
519         px:
520         for (int batch_num = 0; batch_num < NUM_BATCHES_EST;
batch_num++) {
521             cnt = 0;
522             check_col_indices:
523             for (int col = COLS - 1; col >= 0; col--) {
524                 num_avail = available_cnt[col];
525                 if (num_avail > 1) {
526                     read_indices:
527                     for (int rows = 0; rows < num_avail / 2;
rows++) {
528                         if (rows < num_avail / 2) {
529                             a = avail-
530                             available_first[col]++;
531                             b = avail-
532                             available_first[col]++;
533                             available_cnt[col] = avail-
534                             available_last[col] - 2;
535                             if (b < a) {
536                                 tmp = a;
537                                 a = b;
538                                 b = tmp;
539                             }
540                             // a & b are the row indexes we
read from the memory for this rotation
541                             SEQUENCE[seq_cnt][0] = a;
542                             SEQUENCE[seq_cnt][1] = b;
543                             SEQUENCE[seq_cnt][2] = col;
544                             seq_cnt++;
545                             cnt++;
546                             available[available_last[col]]
[col] = a; // Non-zeroed element so store again
547                             available_cnt[col]++;
548                             available_last[col]++;
549                             zeroed[col]++;
550                             if (col < COLS - 1) {
551                                 available[available_last[col]
+ 1][col + 1] =
552                                 b; // Zeroed, row avail-
able to the next column
553                                 available_cnt[col + 1]++;
554                                 available_last[col + 1]++;
555                             }
556                         }
557                     }
558                 }
559                 BATCH_CNTS[batch_num] = cnt;
560                 // Check for end condition
561                 if ((ROWS == COLS && zeroed[COLS - 2] == 1) ||
(ROWS > COLS && zeroed[COLS - 1] == ROWS - COLS)) {
562                     NUM_BATCHES = batch_num + 1;
563                     if (is_cmplx<InputType>::value && ROWS ==
COLS) {
564                         // Add an extra rotation to ensure last
element on the diagonal is real
565                         NUM_BATCHES = batch_num + 2;
566                         BATCH_CNTS[batch_num + 1] = 1;
567                         SEQUENCE[seq_cnt][0] = ROWS - 2;
568                         SEQUENCE[seq_cnt][1] = ROWS - 1;
569                         SEQUENCE[seq_cnt][2] = ROWS - 1;
570                     }
571                     break;
572                 }

```

```

573     }
574     if (NUM_BATCHES == 0) {
575     #ifndef __SYNTHESIS__
576         printf(
577             "ERROR: hls_qrf.h: qrf_alt_config: ERROR:
NUM_BATCHES_EST count reached without completing the "
578             "processing sequence. Increase the
NUM_BATCHES_EST value.\n");
579         exit(1);
580     #endif
581     };
582 };
583 };
584 };
585 //
=====
586 // QRF_BASIC
587 template <bool TransposedQ, int RowsA, int ColsA, typename
QRF_TRAITS, typename InputType, typename OutputType>
588 void qrf_basic(hls::stream<InputType>& matrixAStm,
589               hls::stream<OutputType>& matrixQStm,
590               hls::stream<OutputType>& matrixRStm) {
591     // Verify that template parameters are correct in simula-
tion
592     if (RowsA < ColsA) {
593     #ifndef __SYNTHESIS__
594         printf(
595             "ERROR: hls_qrf.h: Template parameter error -
RowsA must be greater than ColsA; currently RowsA = %d ColsA = "
596             "%d\n",
597             RowsA, ColsA);
598     #endif
599         exit(1);
600     }
601     // Buffers
602     OutputType Qi[RowsA][RowsA];
603     OutputType Ri[RowsA][ColsA];
604     OutputType G[2][2];
605
606     // Magnitude from Givens computation
607     OutputType mag = 0;
608
609     // Flags for complex-valued case
610     const int DO_UNITARY_TF = qrf_require_extra_pass(RowsA,
ColsA, mag);
611     int extra_pass = 0;
612
613     // Initialize Qi and initialize/load Ri
614     qrf_in_row_assign:
615     for (int r = 0; r < RowsA; r++) {
616         qrf_in_col_assign_Qi:
617         for (int c = 0; c < RowsA; c++) {
618         #pragma HLS PIPELINE
619             if (r == c) {
620                 Qi[r][c] = 1.0;
621             } else {
622                 Qi[r][c] = 0.0;
623             }
624         }
625         qrf_in_col_assign_Ri:
626         for (int c = 0; c < ColsA; c++) {
627         #pragma HLS PIPELINE
628             Ri[r][c] = matrixAStm.read();
629         }
630     }
631
632     qrf_col_loop:
633     for (int j = 0; j < ColsA; j++) {
634         // For complex data and square matrices, we perform
an additional pass to ensure that the diagonal of R is real
635         // For non-square matrices, the modified Givens rota-
tion ensures that the diagonal will be real-valued
636         if (DO_UNITARY_TF == 1) {
637             if (j == ColsA - 1) {
638                 extra_pass = 1;
639             } else {
640                 extra_pass = 0;
641             }
642         } else {
643             extra_pass = 0;
644         }
645         qrf_row_loop:
646         for (int i = RowsA - 1; i > 0; i--) {
647             if (i <= j - extra_pass) {
648                 continue;
649             } else {
650                 // Compute Givens values
651                 qrf_givens(extra_pass, Ri[i - 1][j], Ri[i]
[j], G[0][0], G[0][1], G[1][0], G[1][1], mag);
652
653                 if (!extra_pass) {
654                     Ri[i - 1][j] = hls::x_real(mag);
655                 } else {
656                     Ri[i][j] = hls::x_real(mag);
657                 }
658
659                 qrf_r_update:
660                 for (int k = 0; k < ColsA; k++) {
661         #pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II

```

```

662             if (k < j + 1) {
663                 continue;
664             } else {
665                 qrf_mm(G, Ri[i - 1][k], Ri[i][k]);
666             }
667         }
668         qrf_q_update:
669         for (int k = 0; k < RowsA; k++) {
670         #pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
671             if (k < (i - (1 + j) + extra_pass)) {
672                 continue;
673             } else {
674                 qrf_mm(G, Qi[i - 1][k], Qi[i][k]);
675             }
676         }
677     } // end if i<=j
678 } // end qrf_row_loop
679 } // end qrf_col_loop
680
681 // Assign final outputs
682 qrf_out_row_assign:
683 for (int r = 0; r < RowsA; r++) {
684     qrf_out_col_assign:
685     for (int c = 0; c < RowsA; c++) {
686     #pragma HLS PIPELINE
687         if (TransposedQ == true) {
688             matrixQStm.write(Qi[r][c]);
689         } else {
690             matrixQStm.write(hls::x_conj(Qi[c][r]));
691         }
692         if (c < ColsA) {
693             matrixRStm.write(Ri[r][c]);
694         }
695     }
696 } // end template qrf_basic
697
698 } // end template qrf_basic
699
700 //
=====
701 // QRF_ALT: Optimized for throughput.
702 template <bool TransposedQ, int RowsA, int ColsA, typename
QRF_TRAITS, typename InputType, typename OutputType>
703 void qrf_alt(hls::stream<InputType>& matrixAStm,
704             hls::stream<OutputType>& matrixQStm,
705             hls::stream<OutputType>& matrixRStm) {
706     // Verify that template parameters are correct in simula-
tion
707     if (RowsA < ColsA) {
708         exit(1);
709     }
710
711     // Declare the ROMs defining the processing sequence
712     static const qrf_alt_config<RowsA, ColsA, InputType> CON-
FIG;
713
714     // Internal array memories
715
716     // IMPLEMENTATION TIP: To further increase the throughput
of the function partition the q_i and r_i arrays on the
717     // column
718     // dimension and unroll the update_r/q loops by the same
amount.
719     OutputType q_i[RowsA][RowsA];
720     OutputType r_i[RowsA][ColsA];
721
722     #pragma HLS ARRAY_PARTITION variable = q_i cyclic dim = 2
factor = QRF_TRAITS::UNROLL_FACTOR
723     #pragma HLS ARRAY_PARTITION variable = r_i cyclic dim = 2
factor = QRF_TRAITS::UNROLL_FACTOR
724
725     hls::stream<int> to_rot[3];
726     #pragma HLS STREAM variable = to_rot depth = RowsA / 2
727     int seq_cnt = 0;
728     int extra_pass = 0;
729     int extra_pass2 = 0;
730     int use_mag = 0;
731     int px_row1, px_row2, px_col, rot_row1, rot_row2,
rot_col;
732     OutputType G[2][2];
733     OutputType mag = 0;
734     hls::stream<OutputType> rotations[5];
735     #pragma HLS STREAM variable = rotations depth = RowsA / 2
736     OutputType G_delay[2][2];
737     OutputType mag_delay;
738
739     // Copy input data to local R memory and initialize Q
740     row_copy:
741     for (int r = 0; r < RowsA; r++) {
742         // Merge loops to parallelize the A input read and the Q mat-
rix prime.
743         #pragma HLS LOOP_MERGE force
744         col_copy_q_i:
745         for (int c = 0; c < RowsA; c++) {
746             if (r == c) {
747                 q_i[r][c] = 1.0;
748             } else {
749                 q_i[r][c] = 0.0;

```

```

750     }
751   }
752   col_copy_r_i:
753   for (int c = 0; c < ColsA; c++) {
754     #pragma HLS PIPELINE
755     r_i[r][c] = matrixAstrm.read();
756   }
757 }
758
759 // Process R in batches of non-dependent array elements
760 px:
761 for (int batch_num = 0; batch_num < CONFIG.NUM_BATCHES;
batch_num++) {
762   calc_rotations:
763   for (int px_cnt = 0; px_cnt < CON-
FIG.BATCH_CNTS[batch_num]; px_cnt++) {
764     #pragma HLS LOOP_TRIPCOUNT min = 1 max = RowsA / 2
765     #pragma HLS PIPELINE II = QRF_TRAITS::CALC_ROT_II
766     px_row1 = CONFIG.SEQUENCE[seq_cnt][0];
767     px_row2 = CONFIG.SEQUENCE[seq_cnt][1];
768     px_col = CONFIG.SEQUENCE[seq_cnt][2];
769     seq_cnt++;
770     extra_pass = 0;
771     if (is_cplx<InputType>::value && RowsA == ColsA
&& batch_num == CONFIG.NUM_BATCHES - 1) {
772       extra_pass = 1;
773     }
774     qrf_givens(extra_pass, r_i[px_row1][px_col],
r_i[px_row2][px_col], G[0][0], G[0][1], G[1][0], G[1][1], mag);
775     // Pass on rotation to next block to apply rota-
tions
776     rotations[0].write(G[0][0]);
777     rotations[1].write(G[0][1]);
778     rotations[2].write(G[1][0]);
779     rotations[3].write(G[1][1]);
780     rotations[4].write(mag);
781     to_rot[0].write(px_row1);
782     to_rot[1].write(px_row2);
783     to_rot[2].write(px_col);
784   }
785
786   rotate:
787   for (int px_cnt = 0; px_cnt < CON-
FIG.BATCH_CNTS[batch_num]; px_cnt++) {
788     #pragma HLS LOOP_TRIPCOUNT min = 1 max = RowsA / 2
789     G_delay[0][0] = rotations[0].read();
790     G_delay[0][1] = rotations[1].read();
791     G_delay[1][0] = rotations[2].read();
792     G_delay[1][1] = rotations[3].read();
793     mag_delay = rotations[4].read();
794     rot_row1 = to_rot[0].read();
795     rot_row2 = to_rot[1].read();
796     rot_col = to_rot[2].read();
797
798     extra_pass2 = 0;
799     if (is_cplx<InputType>::value && RowsA == ColsA
&& batch_num == CONFIG.NUM_BATCHES - 1) {
800       extra_pass2 = 1;
801     }
802
803 // Merge the loops to maximize throughput, otherwise HLS will
execute them sequentially and
804 // share hardware.
805 #pragma HLS LOOP_MERGE force
806   update_r:
807   for (int k = 0; k < ColsA; k++) {
808     #pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
809     #pragma HLS UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR
810     use_mag = 0;
811     if (k == rot_col) {
812       use_mag = 1;
813     }
814     qrf_mm_or_mag(G_delay, r_i[rot_row1][k],
r_i[rot_row2][k], mag_delay, use_mag, extra_pass2);
815   }
816   update_q:
817   for (int k = 0; k < RowsA; k++) {
818     #pragma HLS PIPELINE II = QRF_TRAITS::UPDATE_II
819     #pragma HLS UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR
820     qrf_mm(G_delay, q_i[rot_row1][k],
q_i[rot_row2][k]);
821   }
822 }
823 }
824
825 // Assign final outputs
826 row_assign_loop:
827 for (int r = 0; r < RowsA; r++) {
828 // Merge loops to parallelize the Q and R writes
829 #pragma HLS LOOP_MERGE force
830   col_assign_loop:
831   for (int c = 0; c < RowsA; c++) {
832     #pragma HLS PIPELINE
833     if (TransposedQ == true) {
834       matrixQStrm.write(q_i[r][c]);
835     } else {
836       matrixQStrm.write(hls::x_conj(q_i[c][r]));
837     }
838   }

```

```

839   if (c < ColsA) {
840     matrixRStrm.write(r_i[r][c]);
841   }
842 }
843 }
844 }
845 } // end qrf_alt
846
847 /**
848 * @brief QRF, to computes the full QR factorization (QR de-
composition) of input matrix A, A=QR, producing orthogonal
849 * output matrix Q and upper-triangular matrix R.
850 *
851 * @tparam TransposedQ      Selects whether Q is output in
transposed form
852 * @tparam RowsA            Number of rows in input matrix A
853 * @tparam ColsA            Number of columns in input matrix
A
854 * @tparam InputType        Input data type
855 * @tparam OutputType        Output data type
856 * @tparam QRF_TRAITS        qrfTraits type with specified val-
ues
857 *
858 * @param matrixAstrm        Stream of Input matrix
859 * @param matrixQStrm        Stream of Orthogonal output matrix
860 * @param matrixRStrm        Stream of Upper triangular output
matrix
861 */
862 template <bool TransposedQ,
863           int RowsA,
864           int ColsA,
865           typename InputType,
866           typename OutputType,
867           typename QRF_TRAITS = qrfTraits>
868 void qrf(hls::stream<InputType>& matrixAstrm,
869         hls::stream<OutputType>& matrixQStrm,
870         hls::stream<OutputType>& matrixRStrm) {
871   switch (QRF_TRAITS::ARCH) {
872     case 0:
873       qrf_basic<TransposedQ, RowsA, ColsA, QRF_TRAITS,
InputType, OutputType>(matrixAstrm, matrixQStrm,
874                       matrixRStrm);
875       break;
876     case 1:
877       qrf_alt<TransposedQ, RowsA, ColsA, QRF_TRAITS,
InputType, OutputType>(matrixAstrm, matrixQStrm,
878                       matrixRStrm);
879       break;
880     default:
881       qrf_basic<TransposedQ, RowsA, ColsA, QRF_TRAITS,
InputType, OutputType>(matrixAstrm, matrixQStrm,
882                       matrixRStrm);
883       break;
884   }
885 }
886 } // namespace solver
887 } // namespace xf
888
889 #endif

```

D.8 Vitis_solver/svd.hpp (1,577 lines, 72,409 B)

```

1 /*
2 * Copyright (C) 2019-2022, Xilinx, Inc.
3 * Copyright (C) 2022-2025, Advanced Micro Devices, Inc.
4 *
5 * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
6 * you may not use this file except in compliance with the
License.
7 * You may obtain a copy of the License at
8 *
9 * http://www.apache.org/licenses/LICENSE-2.0
10 *
11 * Unless required by applicable law or agreed to in writing,
software
12 * distributed under the License is distributed on an "AS IS"
BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 * See the License for the specific language governing per-
missions and
15 * limitations under the License.
16 */
17
18 /**
19 * @file svd.hpp
20 * @brief This file contains implementation of SVD functions
21 * -svd : Entry point function
22 * -svdTop : Top level function that selects
implementation architecture and internal types based on
23 * traits class

```

```

24 * -svdBasic : Default architecture
25 * -svdPairs : Alternative architecture
26 */
27
28 #ifndef _XF_SOLVER_SVD_HPP_
29 #define _XF_SOLVER_SVD_HPP_
30
31 #include "ap_fixed.h"
32 #include "hls_x_complex.h"
33 #include "utils/x_matrix_utils.hpp"
34 #include "hls_stream.h"
35 #include "utils/std_complex_utils.h"
36 #include <complex>
37
38 #include <assert.h>
39
40 namespace xf {
41 namespace solver {
42
43 //
44 // =====
45 // Default traits struct
46 template <int RowsA, int ColsA, typename InputType, typename
OutputType>
47 struct svdTraits {
48     typedef OutputType SIntType;
49     typedef OutputType UIntType;
50     typedef OutputType VIntType;
51     typedef OutputType CSIntType;
52
53     static const int NUM_SWEEPS = 10; // Literature typically
suggestions 6 to 10 iterations to successfully converge
54     static const int MIN_DIM = (RowsA < ColsA ? RowsA :
ColsA);
55     static const int ARCH = 1; // Select implementa-
tion. 0 = Basic loop engine. 1 = Pairs based engine
56     static const int OFF_DIAG_II = 8; // Specify the pipelin-
ing target for the off diagonal loop. Upto 4 memory accesses
57     // on single array in
one iteration, use multiple
58     static const int DIAG_II = 8; // Specify the pipelining
target for the diagonal loop. >1 enables reuse of operators.
59
60 // IMPLEMENTATION TIP: Potential additional configuration
parameters to fully unroll the "Pairs" based engine
61 // static const int UNROLL_FACTOR = (MIN_DIM + 1) /
2; // Specify off-diagonal loop unrolling factor
62 // static const int DIAG_UNROLL_FACTOR = (MIN_DIM + 1) /
2; // Specify diagonal loop unrolling factor
63 };
64 //
65 // =====
66 // Helper functions
67 //
68 // Compare 2 values relative magnitude
69 // - Replaces a test using EPS as a scaling factor:
70 // abs(b) <= (e*abs(a)) where e = hls::numer-
ic_limits<CSType>::epsilon()/2;
71 // - b is within/just outside the representable precision of
a
72 template <typename T>
73 bool within_precision(T a, T b) {
74     fp_struct<T> fs_a = a;
75     fp_struct<T> fs_b = b;
76
77     if ((fs_b.exp + fs_a.SIG_BITS + 1) < fs_a.exp || fs_b.exp ==
0) {
78         return false;
79     } else {
80         return true;
81     }
82 }
83
84 // 2x1 vector dot product
85 // o Used throughout the code for non-vector operations to
ease the application of resource sharing directives
86 // o When complex data type is used multiple version of this
function will be created with a mix of complex and real
87 // argument types.
88 #ifdef HLS_SVD_SKIP_COMMON_VM2X1
89 template <typename AType, typename BType, typename CType>
90 void vm2x1(AType a1, BType b1, AType a2, BType b2, CType& c)
{
91     c = a1 * b1 + a2 * b2;
92 }
93 #endif
94
95 // Define additional overloaded forms of vm2x1
96 // o Simplifies controlling resource utilization for complex
data types implementations
97 // o Maps all forms of complex vm2x1 to a single complex only
input
98 #ifndef HLS_SVD_SKIP_COMMON_VM2X1
99 template <typename AType, typename BType, typename CType>
100 void vm2x1_base(AType a1, BType b1, AType a2, BType b2,
CType& c) {
101
102 // Disable the inlining of the base vm2x1 function and
limit instances using the ALLOCATION directive
103 // #pragma HLS inline off
104 c = a1 * b1 + a2 * b2;
105 }
106
107 template <typename T>
108 void vm2x1(T a1, hls::x_complex<T> b1, T a2, hls::x_com-
plex<T> b2, hls::x_complex<T>& c) {
109     hls::x_complex<T> c_a1, c_a2;
110     c_a1 = a1;
111     c_a2 = a2;
112     vm2x1_base(c_a1, b1, c_a2, b2, c);
113 }
114
115 template <typename T>
116 void vm2x1(hls::x_complex<T> a1, T b1, hls::x_complex<T> a2,
T b2, hls::x_complex<T>& c) {
117     hls::x_complex<T> c_b1, c_b2;
118     c_b1 = b1;
119     c_b2 = b2;
120     vm2x1_base(a1, c_b1, a2, c_b2, c);
121 }
122
123 template <typename T>
124 void vm2x1(std::complex<T> a1, T b1, std::complex<T> a2, T
b2, std::complex<T>& c) {
125     std::complex<T> c_a1, c_a2;
126     c_a1 = a1;
127     c_a2 = a2;
128     vm2x1_base(c_a1, b1, c_a2, b2, c);
129 }
130
131 template <typename T>
132 void vm2x1(std::complex<T> a1, T b1, std::complex<T> a2, T
b2, std::complex<T>& c) {
133     std::complex<T> c_b1, c_b2;
134     c_b1 = b1;
135     c_b2 = b2;
136     vm2x1_base(a1, c_b1, a2, c_b2, c);
137 }
138
139 template <typename T>
140 void vm2x1(T a1, T b1, T a2, T b2, T& c) {
141     vm2x1_base(a1, b1, a2, b2, c);
142 }
143 #endif
144
145 // 2x2 matrix multiply
146 template <typename AType, typename BType, typename CType>
147 void mm2x2(AType a1,
148           AType a2,
149           AType a3,
150           AType a4,
151           BType b1,
152           BType b2,
153           BType b3,
154           BType b4,
155           CType& c1,
156           CType& c2,
157           CType& c3,
158           CType& c4) {
159     vm2x1(a1, b1, a2, b3, c1);
160     vm2x1(a1, b2, a2, b4, c2);
161     vm2x1(a3, b1, a4, b3, c3);
162     vm2x1(a3, b2, a4, b4, c4);
163 }
164
165 // Calculate the sin and cos of the complex input's phase
angle and phase angle divided by 2
166 template <typename InType, typename CSType>
167 void calc_angle(hls::x_complex<InType> A,
168               CSType& cosThetaA,
169               CSType& sinThetaA,
170               CSType& cosThetaAdiv2,
171               CSType& sinThetaAdiv2,
172               bool& is_pos_real,
173               bool& is_imag) {
174     const InType inZERO = 0;
175     const CSType csZERO = 0;
176     const CSType csONE = 1;
177     // NOTE: Hard single precision floating point value
178     const float ONE_OVER_ROOT2 = 1.0f / sqrtf(2.0f);
179
180     CSType tanThetaA, cosThetaA_int, sinThetaA_int, tanThet-
aAdiv2, cosThetaAdiv2_int;
181
182     InType re = A.real();
183     InType im = A.imag();
184
185     // Helpers to avoid testing the sin and cos outputs for
particular characteristics.
186     is_pos_real = false;
187     is_imag = false;
188
189     // Check for when effectively real only or imag only
190     if (!within_precision(re, im)) {
191         if (x_isneg(re)) {
192             // 180 degs (half is 90!)
193             cosThetaA = -csONE;
194             sinThetaA = csZERO;
195             cosThetaAdiv2 = csZERO;
196             sinThetaAdiv2 = csONE;
197         } else {
198             // 0 degs

```

```

189         cosThetaA = csONE;
190         sinThetaA = csZERO;
191         cosThetaAdiv2 = csONE;
192         sinThetaAdiv2 = csZERO;
193         is_pos_real = true;
194     }
195 } else if (!within_precision(im, re)) {
196     is_imag = true;
197     if (x_isneg(im)) {
198         // 270 deg
199         cosThetaA = csZERO;
200         sinThetaA = -csONE;
201         cosThetaAdiv2 = -ONE_OVER_ROOT2;
202         sinThetaAdiv2 = ONE_OVER_ROOT2;
203     } else {
204         // 90 deg
205         cosThetaA = csZERO;
206         sinThetaA = csONE;
207         cosThetaAdiv2 = ONE_OVER_ROOT2;
208         sinThetaAdiv2 = -ONE_OVER_ROOT2;
209     }
210 } else {
211     // Basic
212     // Full angle values
213     tanThetaA = im / re;
214     cosThetaA_int = x_copysign(csONE, re) * x_rsqrt(csONE +
tanThetaA * tanThetaA);
215     cosThetaA = cosThetaA_int;
216     sinThetaA_int = cosThetaA_int * tanThetaA;
217     sinThetaA = sinThetaA_int;
218
219     // Half angle values
220     // o Select the correct expression to minimize error
221     in tan(thetaA/2)
222     // - Avoid creating near eps values
223     if (x_isneg(cosThetaA_int)) {
224         tanThetaAdiv2 = (csONE - cosThetaA_int) / sin-
ThetaA_int;
225     } else {
226         tanThetaAdiv2 = sinThetaA_int / (csONE + cosThet-
aA_int);
227     }
228     cosThetaAdiv2_int = x_rsqrt(csONE + tanThetaAdiv2 *
tanThetaAdiv2);
229     cosThetaAdiv2 = cosThetaAdiv2_int;
230     sinThetaAdiv2 = cosThetaAdiv2_int * tanThetaAdiv2;
231 }
232 }
233 // Calculate the sin and cos of the complex input's phase
angle and phase angle divided by 2
234 template <typename InType, typename CStype>
235 void calc_angle(std::complex<InType> A,
236                 CStype& cosThetaA,
237                 CStype& sinThetaA,
238                 CStype& cosThetaAdiv2,
239                 CStype& sinThetaAdiv2,
240                 bool& is_pos_real,
241                 bool& is_imag) {
242     const InType inZERO = 0;
243     const CStype csZERO = 0;
244     const CStype csONE = 1;
245     // NOTE: Hard single precision floating point value
246     const float ONE_OVER_ROOT2 = 1.0f / sqrtf(2.0f);
247
248     CStype tanThetaA, cosThetaA_int, sinThetaA_int, tanThet-
aAdiv2, cosThetaAdiv2_int;
249
250     InType re = A.real();
251     InType im = A.imag();
252
253     // Helpers to avoid testing the sin and cos outputs for
particular characteristics.
254     is_pos_real = false;
255     is_imag = false;
256
257     // Check for when effectively real only or imag only
258     if (!within_precision(re, im)) {
259         if (x_isneg(re)) {
260             // 180 degs (half is 90!)
261             cosThetaA = -csONE;
262             sinThetaA = csZERO;
263             cosThetaAdiv2 = csZERO;
264             sinThetaAdiv2 = csONE;
265         } else {
266             // 0 degs
267             cosThetaA = csONE;
268             sinThetaA = csZERO;
269             cosThetaAdiv2 = csONE;
270             sinThetaAdiv2 = csZERO;
271             is_pos_real = true;
272         }
273     } else if (!within_precision(im, re)) {
274         is_imag = true;
275         if (x_isneg(im)) {
276             // 270 deg
277             cosThetaA = csZERO;
278             sinThetaA = -csONE;
279             cosThetaAdiv2 = -ONE_OVER_ROOT2;
280             sinThetaAdiv2 = ONE_OVER_ROOT2;
281         } else {
282             // 90 deg
283             cosThetaA = csZERO;
284             sinThetaA = csONE;
285             cosThetaAdiv2 = ONE_OVER_ROOT2;
286             sinThetaAdiv2 = -ONE_OVER_ROOT2;
287         }
288     } else {
289         // Basic
290         // Full angle values
291         tanThetaA = im / re;
292         cosThetaA_int = x_copysign(csONE, re) * x_rsqrt(csONE +
tanThetaA * tanThetaA);
293         cosThetaA = cosThetaA_int;
294         sinThetaA_int = cosThetaA_int * tanThetaA;
295         sinThetaA = sinThetaA_int;
296
297         // Half angle values
298         // o Select the correct expression to minimize error
299         in tan(thetaA/2)
300         // - Avoid creating near eps values
301         if (x_isneg(cosThetaA_int)) {
302             tanThetaAdiv2 = (csONE - cosThetaA_int) / sin-
ThetaA_int;
303         } else {
304             tanThetaAdiv2 = sinThetaA_int / (csONE + cosThet-
aA_int);
305         }
306         cosThetaAdiv2_int = x_rsqrt(csONE + tanThetaAdiv2 *
tanThetaAdiv2);
307         cosThetaAdiv2 = cosThetaAdiv2_int;
308         sinThetaAdiv2 = cosThetaAdiv2_int * tanThetaAdiv2;
309     }
310 }
311
312 //
=====
313 // Diagonal processing functions
314
315 // Real 2x2 SVD function
316 template <typename AInType, typename CStype, typename AOut-
Type>
317 void svd2x2(AInType w_in,
318             AInType x_in,
319             AInType y_in,
320             AInType z_in,
321             CStype& uw_out,
322             CStype& ux_out,
323             CStype& uy_out,
324             CStype& uz_out,
325             CStype& vw_out,
326             CStype& vx_out,
327             CStype& vy_out,
328             CStype& vz_out,
329             AOutType& w_out,
330             AOutType& x_out,
331             AOutType& y_out,
332             AOutType& z_out) {
333     Function_svd2x2_real::;
334
335 // Inline to bring common lower level functions to this level
of hierarchy to simplify the application
336 // of resource sharing directives.
337 #pragma HLS inline
338
339     const AOutType outZERO = 0;
340     CStype s1, c1, s2, c2;
341     AInType u1, u2;
342     // hls::x_complex<AInType> A, B;
343     std::complex<AInType> A, B;
344     CStype cosA_full, sinA_full, cosA_half, sinA_half;
345     CStype cosB_full, sinB_full, cosB_half, sinB_half;
346     bool A_is_pos_real, A_is_imag;
347     bool B_is_pos_real, B_is_imag;
348     CStype uw_int, ux_int, uy_int, uz_int;
349     CStype vw_int, vx_int, vy_int, vz_int;
350     AOutType w_out1, w_out2, z_out1, z_out2, w_out_int,
z_out_int;
351
352     // Determine first half angle required to zero off-diag-
onal values
353     u1 = z_in - w_in;
354     u2 = y_in + x_in;
355     A.imag(u2);
356     A.real(u1);
357
358     calc_angle(A, cosA_full, sinA_full, cosA_half, sinA_half,
A_is_pos_real, A_is_imag);
359
360     // Determine second half angle
361     u1 = z_in + w_in;
362     u2 = y_in - x_in;
363     B.imag(u2);
364     B.real(u1);
365
366     calc_angle(B, cosB_full, sinB_full, cosB_half, sinB_half,

```



```

B_is_pos_real, B_is_imag);
364
365 // Combine half angles to produce left and right rota-
tions
366 // IMPLEMENTATION TIP: There are common products in the
following calculations. For parallel implementations these
367 // should be shared.
368 // Consider in-lining these function calls.
369 vm2x1(cosA_half, cosB_half, sinA_half, sinB_half, c1);
370 vm2x1(sinA_half, cosB_half, -cosA_half, sinB_half, s1);
371 vm2x1(cosA_half, cosB_half, -sinA_half, sinB_half, c2);
372 vm2x1(sinA_half, cosB_half, cosA_half, sinB_half, s2);
373
374 // Build full U and V matrix
375 uw_int = c1;
376 ux_int = s1;
377 uy_int = -s1;
378 uz_int = c1;
379
380 vw_int = c2;
381 vx_int = s2;
382 vy_int = -s2;
383 vz_int = c2;
384
385 // Apply rotation
386 // - Uses the transpose version of U
387 // w_out
388 vm2x1(w_in, vw_int, x_in, vy_int, w_out1);
389 vm2x1(y_in, vw_int, z_in, vy_int, w_out2);
390 vm2x1(uw_int, w_out1, uy_int, w_out2, w_out_int);
391 // z_out
392 vm2x1(w_in, vx_int, x_in, vz_int, z_out1);
393 vm2x1(y_in, vx_int, z_in, vz_int, z_out2);
394 vm2x1(ux_int, z_out1, uz_int, z_out2, z_out_int);
395 x_out = outZERO;
396 y_out = outZERO;
397
398 // Ensure singular values are positive
399 if (x_isneg(w_out_int)) {
400     w_out = -w_out_int;
401     vw_int = -c2;
402     vy_int = s2;
403 } else {
404     w_out = w_out_int;
405 }
406 if (x_isneg(z_out_int)) {
407     z_out = -z_out_int;
408     vx_int = -s2;
409     vz_int = -c2;
410 } else {
411     z_out = z_out_int;
412 }
413
414 // Assign outputs
415 uw_out = uw_int;
416 ux_out = ux_int;
417 uy_out = uy_int;
418 uz_out = uz_int;
419 vw_out = vw_int;
420 vx_out = vx_int;
421 vy_out = vy_int;
422 vz_out = vz_int;
423 }
424
425 // Complex 2x2 SVD function
426 // o Calculates several additional rotations to convert the
w,x,y & z values to real only before calling the real 2x2
427 // svd
428 // function
429 template <typename AInType, typename CStype, typename AOut-
Type>
430 void svd2x2(hls::x_complex<AInType> w_in,
431 hls::x_complex<AInType> x_in,
432 hls::x_complex<AInType> y_in,
433 hls::x_complex<AInType> z_in,
434 hls::x_complex<CStype>& uw_out,
435 hls::x_complex<CStype>& ux_out,
436 hls::x_complex<CStype>& uy_out,
437 hls::x_complex<CStype>& uz_out,
438 hls::x_complex<CStype>& vw_out,
439 hls::x_complex<CStype>& vx_out,
440 hls::x_complex<CStype>& vy_out,
441 hls::x_complex<CStype>& vz_out,
442 hls::x_complex<AOutType>& w_out,
443 hls::x_complex<AOutType>& x_out,
444 hls::x_complex<AOutType>& y_out,
445 hls::x_complex<AOutType>& z_out) {
446 Function_svd2x2_complex;;
447
448 // Inline to bring common lower level functions to this level
of hierarchy to simplify the application
449 // of resource sharing directives.
450 #pragma HLS inline
451
452 const hls::x_complex<AInType> CMPLX_ZERO = 0;
453 const AInType REAL_ZERO = 0;
454 const CStype csZERO = 0;
455
456 AInType wMag, xMag, yMag, zMag;
457 CStype cosThetaW, sinThetaW, cosThetaWdiv2, sinThetaWdi-

```

```

v2, cosThetaX, sinThetaX, cosThetaXdiv2, sinThetaXdiv2,
457 cosThetaY, sinThetaY, cosThetaYdiv2, sinThetaYdiv2,
cosThetaZ, sinThetaZ, cosThetaZdiv2, sinThetaZdiv2,
458 RotL1_w_re, RotL1_w_im, RotR1_w_re, RotR1_w_im,
Rot2_C, Rot2_S, Rot2_Cdiv2, Rot2_Sdiv2, RotL2_w, RotL2_x,
459 RotL2_y, RotL2_z, RotL3_w_re, RotL3_w_im, RotR3_w_im,
RotL4_w, RotL4_x, RotL4_y, RotL4_z, RotR4_w, RotR4_x,
460 RotR4_y, RotR4_z;
461
462 hls::x_complex<CStype> RotL1_w, RotL1_x, RotL1_y,
RotL1_z, RotR1_w, RotR1_x, RotR1_y, RotR1_z, RotL12_w, RotL12_x,
463 RotL12_y, RotL12_z, RotL3_w, RotL3_x, RotL3_y,
RotL3_z, RotR3_w, RotR3_x, RotR3_y, RotR3_z, Rot2_cmplx,
464 RotL123_w, RotL123_x, RotL123_y, RotL123_z, RotR13_w,
RotR13_x, RotR13_y, RotR13_z, uw_int, ux_int, uy_int,
465 uz_int;
466
467 hls::x_complex<AInType> w_int, x_int, x_int1, y_int,
z_int, z_int1;
468 AInType w_out_re, x_out_re, y_out_re, z_out_re, w_int_re;
469
470 bool w_is_pos_real, w_is_imag, x_is_pos_real, x_is_imag,
y_is_pos_real, y_is_imag, z_is_pos_real, z_is_imag,
471 tmp_is_pos_real, tmp_is_imag;
472
473 // Determine sin and cos values of W and Y phase angles,
ThetaW and ThetaY
474 calc_angle(w_in, cosThetaW, sinThetaW, cosThetaWdiv2,
sinThetaWdiv2, w_is_pos_real, w_is_imag);
475 calc_angle(y_in, cosThetaY, sinThetaY, cosThetaYdiv2,
sinThetaYdiv2, y_is_pos_real, y_is_imag);
476
477 // Rotation 1
478 // o 2-sided Unitary Complex rotation to make W and Y
real
479 // RotL1 = | exp(j*(ThetaY-ThetaW)/2)
0 | 0 exp(-
480 // j*(ThetaY-ThetaW)/2) |
481 // RotR1 = | exp(-j*(ThetaY+ThetaW)/2)
0 | 0 exp(-
482 // j*(ThetaY+ThetaW)/2) |
483 // o So
484 // exp(j*(ThetaY-ThetaW)/2) = cos((ThetaY-ThetaW)/2) + j
sin((ThetaY-ThetaW)/2)
485 // = cos(ThetaY/2)cos(ThetaW/
2) + sin(ThetaY/2)*sin(ThetaW/2) + j (
486 // sin(ThetaY/2)cos(ThetaW/2) -
cos(ThetaY/2)sin(ThetaW/2) )
487 // exp(-j*(ThetaY+ThetaW)/2) = cos((ThetaY+ThetaW)/2) - j
sin((ThetaY+ThetaW)/2)
488 // = cos(ThetaY/2)cos(ThetaW/
2) - sin(ThetaY/2)*sin(ThetaW/2) - j (
489 // sin(ThetaY/2)cos(ThetaW/2) +
cos(ThetaY/2)sin(ThetaW/2) )
490 vm2x1(cosThetaYdiv2, cosThetaWdiv2, sinThetaYdiv2, sin-
ThetaWdiv2, RotL1_w_re);
491 vm2x1(sinThetaYdiv2, cosThetaWdiv2, -cosThetaYdiv2, sin-
ThetaWdiv2, RotL1_w_im);
492 RotL1_w_real(RotL1_w_re);
493 RotL1_w_imag(RotL1_w_im);
494 RotL1_x = 0; // Unused
495 RotL1_y = 0; // Unused
496 RotL1_z = hls::x_conj(RotL1_w);
497
498 // IMPLEMENTATION TIP: The following calls duplicate the
multiplies also implemented above. For parallel
499 // implementations
500 // these should be optimized/inlined.
501 vm2x1(cosThetaYdiv2, cosThetaWdiv2, -sinThetaYdiv2, sin-
ThetaWdiv2, RotR1_w_re);
502 vm2x1(-sinThetaYdiv2, cosThetaWdiv2, -cosThetaYdiv2, sin-
ThetaWdiv2, RotR1_w_im);
503 RotR1_w_real(RotR1_w_re);
504 RotR1_w_imag(RotR1_w_im);
505 RotR1_x = 0; // Unused
506 RotR1_y = 0; // Unused
507 RotR1_z = RotR1_w;
508
509 // Rotation 2
510 // o 1-sided real Givens rotation to zero Y
511 // o Use the magnitudes of W and Y and calculate the sin
and the cos of the rotation required to zero Y
512 vm2x1(w_in.real(), cosThetaW, w_in.imag(), sinThetaW,
wMag);
513 vm2x1(y_in.real(), cosThetaY, y_in.imag(), sinThetaY,
yMag);
514
515 Rot2_cmplx.real(wMag);
516 Rot2_cmplx.imag(yMag);
517 calc_angle(Rot2_cmplx, Rot2_C, Rot2_S, Rot2_Cdiv2,
Rot2_Sdiv2, tmp_is_pos_real, tmp_is_imag);
518 RotL2_w = Rot2_C;

```

```

519     RotL2_x = Rot2_S;
520     RotL2_y = -Rot2_S;
521     RotL2_z = Rot2_C;
522
523     // Combine left hand rotations 1 & 2
524     // o Using the constant value CMPLX_ZERO to obtain some
optimization when the implementation allows.
525     // o Note that rotation 2 contains real only values
526     mm2x2(RotL2_w, RotL2_x, RotL2_y, RotL2_z, RotL1_w, CM-
PLX_ZERO, CMPLX_ZERO, RotL1_z, RotL12_w, RotL12_x, RotL12_y,
527         RotL12_z);
528
529     // Update w,x,y & z values to reflect rotations
530     w_int.imag(0);
531     vm2x1(wMag, Rot2_C, yMag, Rot2_S, w_int_re);
532     w_int.real(w_int_re);
533     y_int = 0;
534     vm2x1(x_in, RotL12_w, z_in, RotL12_x, x_int1);
535     x_int = x_int1 * RotR1_z;
536     vm2x1(x_in, RotL12_y, z_in, RotL12_z, z_int1);
537     z_int = z_int1 * RotR1_z;
538
539     // Determine sin and cos values of the updated X and Z
phase angles, ThetaX and ThetaZ
540     calc_angle(x_int, cosThetaX, sinThetaX, cosThetaZdiv2,
sinThetaZdiv2, x_is_pos_real, x_is_imag);
541     calc_angle(z_int, cosThetaZ, sinThetaZ, cosThetaZdiv2,
sinThetaZdiv2, z_is_pos_real, z_is_imag);
542
543     // Rotation 3
544     // o 2 additional 2-sided complex rotations to turn the
updated X and Z into real only values.
545     // o The 2 rotations are combined into a single 2-sided
complex rotation.
546     // o The first rotation rotates W and X - Rotation 3a:
547     //      RotL3a = | exp(-j*(ThetaX+ThetaW)/2)
0
548     //      | 0
549     //      RotR3a = | exp(j*(ThetaX-ThetaW)/2)
0
550     //      | 0
551     //      exp(-
j*(ThetaX+ThetaW)/2) |
552     // - Note ThetaW already equals 0 so the above simpli-
fies to only use ThetaX
553     // o The second rotation rotates X and Z - Rotation 3b:
554     //      RotL3b = | exp(j*(ThetaZ-ThetaX)/2)
0
555     //      | 0
556     //      exp(-
j*(ThetaZ-ThetaX)/2) |
557     // - Note Following rotation 3a ThetaX equals 0 so this
rotation simplifies to use only ThetaZ
558     // o Finally we can then combine these 2 rotations into
single left and right unitary matrix giving the final
559     // rotation we'll use:
560     //      RotL3 = | exp(j*(ThetaZ/2 - ThetaX))
0
561     //      | 0
562     //      RotR3 = | exp(j*(ThetaX - ThetaZ/2))
0
563     //      | 0
564     //      exp(j*-
(ThetaZ/2)) |
565     // o So
566     //      exp(j*(ThetaZ/2 - ThetaX)) = cos(ThetaZ/2 - ThetaX)
+ j sin(ThetaZ/2 - ThetaX)
567     //      = cos(ThetaZ/
2)cos(ThetaX) + sin(ThetaZ/2)*sin(ThetaX) + j (
sin(ThetaZ/2)
568     //      exp(j*(ThetaX - ThetaZ/2)) = cos(ThetaX - ThetaZ/2)
+ j sin(ThetaX - ThetaZ/2)
569     //      = cos(ThetaX)cos(ThetaZ/
2) + sin(ThetaX)*sin(ThetaZ/2) + j (
570     //      sin(ThetaX)cos(ThetaZ/2)
- cos(ThetaX)sin(ThetaZ/2) )
571
572     vm2x1(cosThetaZdiv2, cosThetaX, sinThetaZdiv2, sinThetaX,
RotL3_w_re);
573     vm2x1(sinThetaZdiv2, cosThetaX, -cosThetaZdiv2, sin-
ThetaX, RotL3_w_im);
574     RotL3_w.real(RotL3_w_re);
575     RotL3_w.imag(RotL3_w_im);
576     RotL3_x = 0; // Unused
577     RotL3_y = 0; // Unused
578     RotL3_z.real(cosThetaZdiv2);
579     RotL3_z.imag(-sinThetaZdiv2);
580
581     RotR3_w.real(RotL3_w.real());
582     // IMPLEMENTATION TIP: Below duplicates the multiplies
implemented above. For parallel implementations these should
583     // be inlined
584     vm2x1(cosThetaZdiv2, sinThetaX, -sinThetaZdiv2, cos-

```

```

ThetaX, RotR3_w_im);
585     RotR3_w.imag(RotR3_w_im);
586     RotR3_x = 0; // Unused
587     RotR3_y = 0; // Unused
588     RotR3_z = RotL3_z;
589
590     // Combine rotation 3 with 2 & 1
591     mm2x2(RotL3_w, CMPLX_ZERO, CMPLX_ZERO, RotL3_z, RotL12_w,
RotL12_x, RotL12_y, RotL12_z, RotL123_w, RotL123_x,
592         RotL123_y, RotL123_z);
593     mm2x2(RotR1_w, CMPLX_ZERO, CMPLX_ZERO, RotR1_z, RotR3_w,
CMPLX_ZERO, CMPLX_ZERO, RotR3_z, RotR13_w, RotR13_x,
594         RotR13_y, RotR13_z);
595
596     // Calculate the magnitudes of X and Z for use in real
SVD calculation
597     vm2x1(x_int.real(), cosThetaX, x_int.imag(), sinThetaX,
xMag);
598     vm2x1(z_int.real(), cosThetaZ, z_int.imag(), sinThetaZ,
zMag);
599
600     // Call real SVD function
601     svd2x2(w_int.real(), xMag, REAL_ZERO, zMag, RotL4_w,
RotL4_x, RotL4_y, RotL4_z, RotR4_w, RotR4_x, RotR4_y, RotR4_z,
602         w_out_re, x_out_re, y_out_re, z_out_re);
603
604     // Generate and assign outputs
605     // o The combined U values (Left rotations) must be Her-
mitian Transposed.
606     // o Note the rotation values output by the SVD function
are real only
607     w_out = w_out_re;
608     x_out = x_out_re; // Zero'ed by the SVD function
609     y_out = y_out_re; // Zero'ed by the SVD function
610     z_out = z_out_re;
611
612     mm2x2(RotL4_w, RotL4_y, RotL4_x, RotL4_z, RotL123_w,
RotL123_x, RotL123_y, RotL123_z, uw_int, ux_int, uy_int,
613         uz_int);
614     uw_out = hls::x_conj(uw_int);
615     ux_out = hls::x_conj(ux_int); // Transposed
616     uy_out = hls::x_conj(uy_int); // Transposed
617     uz_out = hls::x_conj(uz_int);
618
619     mm2x2(RotR13_w, RotR13_x, RotR13_y, RotR13_z, RotR4_w,
RotR4_x, RotR4_y, RotR4_z, vw_out, vx_out, vy_out, vz_out);
620 }
621
622 // Complex 2x2 SVD function
623 // o Calculates several additional rotations to convert the
w,x,y & z values to real only before calling the real 2x2
624 // svd
625 // function
626 template <typename AInType, typename CStype, typename AOut-
Type>
627 void svd2x2(std::complex<AInType> w_in,
628             std::complex<AInType> x_in,
629             std::complex<AInType> y_in,
630             std::complex<AInType> z_in,
631             std::complex<CStype>& uw_out,
632             std::complex<CStype>& ux_out,
633             std::complex<CStype>& uy_out,
634             std::complex<CStype>& uz_out,
635             std::complex<CStype>& vw_out,
636             std::complex<CStype>& vx_out,
637             std::complex<CStype>& vy_out,
638             std::complex<CStype>& vz_out,
639             std::complex<AOutType>& w_out,
640             std::complex<AOutType>& x_out,
641             std::complex<AOutType>& y_out,
642             std::complex<AOutType>& z_out) {
643     Function_svd2x2_complex;
644
645     // Inline to bring common lower level functions to this level
of hierarchy to simplify the application
646     // of resource sharing directives.
647     #pragma HLS inline
648
649     const std::complex<AInType> CMPLX_ZERO = 0;
650     const AInType REAL_ZERO = 0;
651     const CStype cZERO = 0;
652
653     AInType wMag, xMag, yMag, zMag;
654     CStype cosThetaW, sinThetaW, cosThetaWdiv2, sinThetaWdi-
v2, cosThetaX, sinThetaX, cosThetaXdiv2, sinThetaXdiv2,
655         cosThetaY, sinThetaY, cosThetaYdiv2, sinThetaYdiv2,
cosThetaZ, sinThetaZ, cosThetaZdiv2, sinThetaZdiv2,
656         RotL1_w_re, RotL1_w_im, RotR1_w_re, RotR1_w_im,
Rot2_C, Rot2_S, Rot2_Cdiv2, Rot2_Sdiv2, RotL2_w, RotL2_x,
657         RotL2_y, RotL2_z, RotL3_w_re, RotL3_w_im, RotR3_w_im,
RotL4_w, RotL4_x, RotL4_y, RotL4_z, RotR4_w, RotR4_x,
658         RotR4_y, RotR4_z;
659
660     std::complex<CStype> RotL1_w, RotL1_x, RotL1_y, RotL1_z,
RotR1_w, RotR1_x, RotR1_y, RotR1_z, RotL12_w, RotL12_x,
661         RotL12_y, RotL12_z, RotL3_w, RotL3_x, RotL3_y,
RotL3_z, RotR3_w, RotR3_x, RotR3_y, RotR3_z, Rot2_cmplx,

```

```

661 RotL123_w, RotL123_x, RotL123_y, RotL123_z, RotR13_w,
RotR13_x, RotR13_y, RotR13_z, uw_int, ux_int, uy_int,
662 uz_int;
663
664 std::complex<AInType> w_int, x_int, x_int1, y_int, z_int,
z_int1;
665 AInType w_out_re, x_out_re, y_out_re, z_out_re, w_int_re;
666
667 bool w_is_pos_real, w_is_imag, x_is_pos_real, x_is_imag,
y_is_pos_real, y_is_imag, z_is_pos_real, z_is_imag,
668 tmp_is_pos_real, tmp_is_imag;
669
670 // Determine sin and cos values of W and Y phase angles,
ThetaW and ThetaY
671 calc_angle(w_in, cosThetaW, sinThetaW, cosThetaWdiv2,
sinThetaWdiv2, w_is_pos_real, w_is_imag);
672 calc_angle(y_in, cosThetaY, sinThetaY, cosThetaYdiv2,
sinThetaYdiv2, y_is_pos_real, y_is_imag);
673
674 // Rotation 1
675 // o 2-sided Unitary Complex rotation to make W and Y
real
676 // RotL1 = | exp(j*(ThetaY-ThetaW)/2)
0 // | 0 exp(-
677 // j*(ThetaY-ThetaW)/2) |
678 // RotR1 = | exp(-j*(ThetaY+ThetaW)/2)
0 // | 0 exp(-
679 // j*(ThetaY+ThetaW)/2) |
680 // o So
681 // exp(j*(ThetaY-ThetaW)/2) = cos((ThetaY-ThetaW)/2) + j
sin((ThetaY-ThetaW)/2)
682 // = cos(ThetaY/2)cos(ThetaW/
2) + sin(ThetaY/2)*sin(ThetaW/2) + j (
683 // sin(ThetaY/2)cos(ThetaW/2) -
cos(ThetaY/2)sin(ThetaW/2) )
684 // exp(-j*(ThetaY+ThetaW)/2) = cos((ThetaY+ThetaW)/2) - j
sin((ThetaY+ThetaW)/2)
685 // = cos(ThetaY/2)cos(ThetaW/
2) - sin(ThetaY/2)*sin(ThetaW/2) - j (
686 // sin(ThetaY/2)cos(ThetaW/2) +
cos(ThetaY/2)sin(ThetaW/2) )
687 vm2x1(cosThetaYdiv2, cosThetaWdiv2, sinThetaYdiv2, sin-
ThetaWdiv2, RotL1_w_re);
688 vm2x1(sinThetaYdiv2, cosThetaWdiv2, -cosThetaYdiv2, sin-
ThetaWdiv2, RotL1_w_im);
689 RotL1_w.real(RotL1_w_re);
690 RotL1_w.imag(RotL1_w_im);
691 RotL1_x = 0; // Unused
692 RotL1_y = 0; // Unused
693 RotL1_z = hls::x_conj(RotL1_w);
694
695 // IMPLEMENTATION TIP: The following calls duplicate the
multiplies also implemented above. For parallel
696 // implementations
697 // these should be optimized/inlined.
698 vm2x1(cosThetaYdiv2, cosThetaWdiv2, -sinThetaYdiv2, sin-
ThetaWdiv2, RotR1_w_re);
699 vm2x1(-sinThetaYdiv2, cosThetaWdiv2, -cosThetaYdiv2, sin-
ThetaWdiv2, RotR1_w_im);
700 RotR1_w.real(RotR1_w_re);
701 RotR1_w.imag(RotR1_w_im);
702 RotR1_x = 0; // Unused
703 RotR1_y = 0; // Unused
704 RotR1_z = RotR1_w;
705
706 // Rotation 2
707 // o 1-sided real Givens rotation to zero Y
708 // o Use the magnitudes of W and Y and calculate the sin
and the cos of the rotation required to zero Y
709 vm2x1(w_in.real(), cosThetaW, w_in.imag(), sinThetaW,
wMag);
710 vm2x1(y_in.real(), cosThetaY, y_in.imag(), sinThetaY,
yMag);
711
712 Rot2_cplx.real(wMag);
713 Rot2_cplx.imag(yMag);
714 calc_angle(Rot2_cplx, Rot2_C, Rot2_S, Rot2_Cdiv2,
Rot2_Sdiv2, tmp_is_pos_real, tmp_is_imag);
715 RotL2_w = Rot2_C;
716 RotL2_x = Rot2_S;
717 RotL2_y = -Rot2_S;
718 RotL2_z = Rot2_C;
719
720 // Combine left hand rotations 1 & 2
721 // o Using the constant value Cmplx_ZERO to obtain some
optimization when the implementation allows.
722 // o Note that rotation 2 contains real only values
723 mm2x2(RotL2_w, RotL2_x, RotL2_y, RotL2_z, RotL1_w, CM-
PLX_ZERO, Cmplx_ZERO, RotL1_z, RotL12_w, RotL12_x, RotL12_y,
724 RotL12_z);
725
726 // Update w,x,y & z values to reflect rotations
727 w_int.imag(0);

```

```

728 vm2x1(wMag, Rot2_C, yMag, Rot2_S, w_int_re);
729 w_int.real(w_int_re);
730 y_int = 0;
731 vm2x1(x_in, RotL12_w, z_in, RotL12_x, x_int1);
732 x_int = x_int1 * RotR1_z;
733 vm2x1(x_in, RotL12_y, z_in, RotL12_z, z_int1);
734 z_int = z_int1 * RotR1_z;
735
736 // Determine sin and cos values of the updated X and Z
phase angles, ThetaX and ThetaZ
737 calc_angle(x_int, cosThetaX, sinThetaX, cosThetaXdiv2,
sinThetaXdiv2, x_is_pos_real, x_is_imag);
738 calc_angle(z_int, cosThetaZ, sinThetaZ, cosThetaZdiv2,
sinThetaZdiv2, z_is_pos_real, z_is_imag);
739
740 // Rotation 3
741 // o 2 additional 2-sided complex rotations to turn the
updated X and Z into real only values.
742 // o The 2 rotations are combined into a single 2-sided
complex rotation.
743 // o The first rotation rotates W and X - Rotation 3a:
744 // RotL3a = | exp(-j*(ThetaX+ThetaW)/2)
0 // | 0 exp(-
745 // j*(ThetaX+ThetaW)/2) |
746 // RotR3a = | exp(j*(ThetaX-ThetaW)/2)
0 // | 0 exp(-
747 // j*(ThetaX-ThetaW)/2) |
748 // - Note ThetaW already equals 0 so the above simpli-
fies to only use ThetaX
749 // o The second rotation rotates X and Z - Rotation 3b:
750 // RotL3b = | exp(j*(ThetaZ-ThetaX)/2)
0 // | 0 exp(-
751 // j*(ThetaZ-ThetaX)/2) |
752 // RotR3b = | exp(-j*(ThetaZ+ThetaX)/2)
0 // | 0 exp(-
753 // j*(ThetaZ+ThetaX)/2) |
754 // - Note Following rotation 3a ThetaX equals 0 so this
rotation simplifies to use only ThetaZ
755 // o Finally we can then combine these 2 rotations into
single left and right unitary matrix giving the final
756 // rotation we'll use:
757 // RotL3 = | exp(j*(ThetaZ/2 - ThetaX))
0 // | 0 exp(j*-
758 // (ThetaZ/2)) |
759 // RotR3 = | exp(j*(ThetaX - ThetaZ/2))
0 // | 0 exp(j*-
760 // (ThetaZ/2)) |
761 // o So
762 // exp(j*(ThetaZ/2 - ThetaX)) = cos(ThetaZ/2 - ThetaX)
+ j sin(ThetaZ/2 - ThetaX)
763 // = cos(ThetaZ/
2)cos(ThetaX) + sin(ThetaZ/2)*sin(ThetaX) + j (
764 // sin(ThetaZ/2)cos(ThetaX)
- cos(ThetaZ/2)sin(ThetaX) )
765 // exp(j*-(ThetaZ/2)) = cos(ThetaZ/2) - j
sin(ThetaZ/2)
766 // exp(j*(ThetaX - ThetaZ/2)) = cos(ThetaX - ThetaZ/2)
+ j sin(ThetaX - ThetaZ/2)
767 // = cos(ThetaX)cos(ThetaZ/
2) + sin(ThetaX)*sin(ThetaZ/2) + j (
768 // sin(ThetaX)cos(ThetaZ/2)
- cos(ThetaX)sin(ThetaZ/2) )
769 vm2x1(cosThetaZdiv2, cosThetaX, sinThetaZdiv2, sinThetaX,
RotL3_w_re);
770 vm2x1(sinThetaZdiv2, cosThetaX, -cosThetaZdiv2, sin-
ThetaX, RotL3_w_im);
771 RotL3_w.real(RotL3_w_re);
772 RotL3_w.imag(RotL3_w_im);
773 RotL3_x = 0; // Unused
774 RotL3_y = 0; // Unused
775 RotL3_z.real(cosThetaZdiv2);
776 RotL3_z.imag(-sinThetaZdiv2);
777
778 RotR3_w.real(RotL3_w.real());
779 // IMPLEMENTATION TIP: Below duplicates the multiplies
implemented above. For parallel implementations these should
780 // be inlined
781 vm2x1(cosThetaZdiv2, sinThetaX, -sinThetaZdiv2, cos-
ThetaX, RotR3_w_im);
782 RotR3_w.imag(RotR3_w_im);
783 RotR3_x = 0; // Unused
784 RotR3_y = 0; // Unused
785 RotR3_z = RotL3_z;
786
787 // Combine rotation 3 with 2 & 1
788 mm2x2(RotL3_w, Cmplx_ZERO, Cmplx_ZERO, RotL3_z, RotL12_w,
RotL12_x, RotL12_y, RotL12_z, RotL123_w, RotL123_x,
789 RotL123_y, RotL123_z);
790 mm2x2(RotR1_w, Cmplx_ZERO, Cmplx_ZERO, RotR1_z, RotR3_w,
Cmplx_ZERO, Cmplx_ZERO, RotR3_z, RotR13_w, RotR13_x,
791 RotR13_y, RotR13_z);

```



```

792
793 // Calculate the magnitudes of X and Z for use in real
SVD calculation
794 vm2x1(x_int.real(), cosThetaX, x_int.imag(), sinThetaX,
xMag);
795 vm2x1(z_int.real(), cosThetaZ, z_int.imag(), sinThetaZ,
zMag);
796
797 // Call real SVD function
798 svd2x2(w_int.real(), xMag, REAL_ZERO, zMag, RotL4_w,
RotL4_x, RotL4_y, RotL4_z, RotR4_w, RotR4_x, RotR4_y, RotR4_z,
w_out_re, x_out_re, y_out_re, z_out_re);
799
800
801 // Generate and assign outputs
802 // o The combined U values (Left rotations) must be Her-
mitian Transposed.
803 // o Note the rotation values output by the SVD function
are real only
804 w_out = w_out_re;
805 x_out = x_out_re; // Zero'ed by the SVD function
806 y_out = y_out_re; // Zero'ed by the SVD function
807 z_out = z_out_re;
808
809 mm2x2(RotL4_w, RotL4_y, RotL4_x, RotL4_z, RotL123_w,
RotL123_x, RotL123_y, RotL123_z, uw_int, ux_int, uy_int,
uz_int);
810
811 uw_out = hls::x_conj(uw_int);
812 ux_out = hls::x_conj(ux_int); // Transposed
813 uy_out = hls::x_conj(uy_int); // Transposed
814 uz_out = hls::x_conj(uz_int);
815
816 mm2x2(RotR13_w, RotR13_x, RotR13_y, RotR13_z, RotR4_w,
RotR4_x, RotR4_y, RotR4_z, vw_out, vx_out, vy_out, vz_out);
817 }
818
819 //
=====
820 // SVD_BASIC: Top level function taking a SVDTraits template
parameter to defines internal types
821 template <int RowsA, int ColsA, class SVDTraits, typename In-
putType, typename OutputType>
822 void svdBasic(const InputType A[RowsA][ColsA],
OutputType S[RowsA][ColsA],
OutputType U[RowsA][RowsA],
OutputType V[ColsA][ColsA]) {
823 // Initially only supporting square matrix
assert(RowsA == ColsA);
824
825 // Internal memories for partial results
826 typename SVDTraits::SIntType s_in[RowsA][ColsA];
827 typename SVDTraits::UIntType u_in[RowsA][ColsA];
828 typename SVDTraits::VIntType v_in[RowsA][ColsA];
829
830 // Current S,U,V values being worked on
831 typename SVDTraits::SIntType w_in, x_in, y_in, z_in;
832 typename SVDTraits::SIntType w_out, x_out, y_out, z_out;
833 typename SVDTraits::UIntType uw_in, ux_in, uy_in, uz_in;
834 typename SVDTraits::UIntType uw_out, ux_out, uy_out,
uz_out;
835
836 typename SVDTraits::VIntType vw_in, vx_in, vy_in, vz_in;
837 typename SVDTraits::VIntType vw_out, vx_out, vy_out,
vz_out;
838
839 // 2x2 Rotation values
840 typename SVDTraits::CSIntType uw_new, ux_new, uy_new,
uz_new;
841
842 typename SVDTraits::CSIntType vw_new, vx_new, vy_new,
vz_new;
843
844 sweep_loop:
845 for (int sweepnum = 0; sweepnum < SVDTraits::NUM_SWEEPS;
sweepnum++) {
846 // NOTE: Using the minimum dimension. i.e. will process a
square matrix
847 row_loop:
848 for (int top_left = 0; top_left < SVDTraits::MIN_DIM;
top_left++) {
849 col_loop:
850 for (int bottom_right = top_left + 1; bot-
tom_right < SVDTraits::MIN_DIM; bottom_right++) {
851 // Fetch w,x,y,z values
852 if (sweepnum == 0 && top_left == 0) {
853 if (bottom_right == 1) {
854 w_in = A[top_left][top_left];
855 x_in = A[top_left][bottom_right];
856 y_in = A[bottom_right][top_left];
857 } else {
858 // Now revist values already updated
in first diagonal pass
859 w_in = s_in[top_left][top_left];
860 x_in = s_in[top_left][bottom_right];
861 y_in = s_in[bottom_right][top_left];
862 }
863 z_in = A[bottom_right][bottom_right];
864 } else {
865 w_in = s_in[top_left][top_left];
866 x_in = s_in[top_left][bottom_right];
867 }

```

```

868 y_in = s_in[bottom_right][top_left];
869 z_in = s_in[bottom_right][bottom_right];
870 }
871 // Diagonal
872 svd2x2(w_in, x_in, y_in, z_in, uw_new,
ux_new, uy_new, uz_new, vw_new, vx_new, vy_new, vz_new, w_out,
x_out, y_out, z_out);
873
874 // Update S on diagonal
875 s_in[top_left][top_left] = w_out;
876 s_in[top_left][bottom_right] = x_out;
877 s_in[bottom_right][top_left] = y_out;
878 s_in[bottom_right][bottom_right] = z_out;
879 if (sweepnum == SVDTraits::NUM_SWEEPS - 1) {
880 S[top_left][top_left] = w_out;
881 S[top_left][bottom_right] = x_out;
882 S[bottom_right][top_left] = y_out;
883 S[bottom_right][bottom_right] = z_out;
884 }
885
886 // Update U & V
887 // o On the diagonal use a 2x2 as per the
sigma
888
889 // o Need to create the identity in U & V at
the start
890 if (sweepnum == 0 && top_left == 0) {
891 if (bottom_right == 1) {
892 uw_in = 1;
893 vx_in = 1;
894 } else {
895 // Now re-visiting diagonal values
where I has been set
896 uw_in = u_in[top_left][top_left];
897 vx_in = v_in[top_left][top_left];
898 }
899
900 ux_in = 0;
901 uy_in = 0;
902 uz_in = 1;
903
904 vx_in = 0;
905 vy_in = 0;
906 vz_in = 1;
907 } else {
908 uw_in = u_in[top_left][top_left];
909 ux_in = u_in[top_left][bottom_right];
910 uy_in = u_in[bottom_right][top_left];
911 uz_in = u_in[bottom_right][bottom_right];
912 vw_in = v_in[top_left][top_left];
913 vx_in = v_in[top_left][bottom_right];
914 vy_in = v_in[bottom_right][top_left];
915 vz_in = v_in[bottom_right][bottom_right];
916 }
917
918 mm2x2(uw_in, ux_in, uy_in, uz_in, uw_new,
ux_new, uy_new, uz_new, uw_out, ux_out, uy_out, uz_out);
919 mm2x2(vw_in, vx_in, vy_in, vz_in, vw_new,
vx_new, vy_new, vz_new, vw_out, vx_out, vy_out, vz_out);
920
921 u_in[top_left][top_left] = uw_out;
922 u_in[top_left][bottom_right] = ux_out;
923 u_in[bottom_right][top_left] = uy_out;
924 u_in[bottom_right][bottom_right] = uz_out;
925 v_in[top_left][top_left] = vw_out;
926 v_in[top_left][bottom_right] = vx_out;
927 v_in[bottom_right][top_left] = vy_out;
928 v_in[bottom_right][bottom_right] = vz_out;
929 if (sweepnum == SVDTraits::NUM_SWEEPS - 1) {
930 U[top_left][top_left] = uw_out;
931 U[top_left][bottom_right] = ux_out;
932 U[bottom_right][top_left] = uy_out;
933 U[bottom_right][bottom_right] = uz_out;
934 V[top_left][top_left] = vw_out;
935 V[top_left][bottom_right] = vx_out;
936 V[bottom_right][top_left] = vy_out;
937 V[bottom_right][bottom_right] = vz_out;
938 }
939
940 // Off-diagonal
941 // Col updates
942 off_col:
943 for (int off_col = 0; off_col <
SVDTraits::MIN_DIM; off_col++) {
944 #pragma HLS PIPELINE II = SVDTraits::OFF_DIAG_II
945
946 if (off_col != bottom_right && off_col !=
top_left) {
947 if (sweepnum == 0 && top_left == 0 &&
bottom_right == 1) {
948 w_in = A[top_left][off_col];
949 } else {
950 w_in = s_in[top_left][off_col];
951 }
952
953 if (sweepnum == 0 && top_left == 0 &&
off_col > bottom_right) {
954 y_in = A[bottom_right][off_col];

```

```

955         } else {
956             y_in = s_in[bottom_right]
[off_col];
957         }
958     }
959     // U must be Hermitian transposed before it is applied to A
960     vm2x1(hls::x_conj(uw_new), w_in,
hls::x_conj(uy_new), y_in, w_out);
961     vm2x1(hls::x_conj(ux_new), w_in,
hls::x_conj(uz_new), y_in, y_out);
962     // Store off-diagonal updates
963     s_in[top_left][off_col] = w_out;
964     s_in[bottom_right][off_col] = y_out;
965
966     if (sweepnum == SVDTraits::NUM_SWEEPS -
1) {
967         S[top_left][off_col] = w_out;
968         S[bottom_right][off_col] = y_out;
969     }
970     }
971     }
972     // Row update
973     off_row:
974     for (int off_row = 0; off_row <
SVDTraits::MIN_DIM; off_row++) {
975 #pragma HLS PIPELINE II = SVDTraits::OFF_DIAG_II
976
977         if (off_row != bottom_right && off_row !=
top_left) {
978             if (sweepnum == 0 && top_left == 0 &&
bottom_right == 1) {
979                 w_in = A[off_row][top_left];
980                 vw_in = 0;
981                 uw_in = 0;
982             } else {
983                 w_in = s_in[off_row][top_left];
984                 vw_in = v_in[off_row][top_left];
985                 uw_in = u_in[off_row][top_left];
986             }
987         }
988         if (sweepnum == 0 && top_left == 0 &&
off_row > bottom_right) {
989             x_in = A[off_row][bottom_right];
990             } else {
991                 x_in = s_in[off_row][bot-
tom_right];
992             }
993         if (sweepnum == 0 && top_left == 0) {
994             vx_in = 0;
995             ux_in = 0;
996         } else {
997             vx_in = v_in[off_row][bot-
tom_right];
998             ux_in = u_in[off_row][bot-
tom_right];
999         }
1000         vm2x1(w_in, vw_new, x_in, vy_new,
w_out);
1001         vm2x1(w_in, vx_new, x_in, vz_new,
x_out);
1002         vm2x1(vw_in, vw_new, vx_in, vy_new,
vw_out);
1003         vm2x1(vw_in, vx_new, vx_in, vz_new,
vx_out);
1004         vm2x1(uw_in, uw_new, ux_in, uy_new,
uw_out);
1005         vm2x1(uw_in, ux_new, ux_in, uz_new,
ux_out);
1006         // Store off-diagonal updates
1007         s_in[off_row][top_left] = w_out;
1008         s_in[off_row][bottom_right] = x_out;
1009         v_in[off_row][top_left] = vw_out;
1010         v_in[off_row][bottom_right] = vx_out;
1011         u_in[off_row][top_left] = uw_out;
1012         u_in[off_row][bottom_right] = ux_out;
1013
1014         if (sweepnum == SVDTraits::NUM_SWEEPS
- 1) {
1015             S[off_row][top_left] = w_out;
1016             S[off_row][bottom_right] = x_out;
1017             V[off_row][top_left] = vw_out;
1018             V[off_row][bottom_right] =
vx_out;
1019             U[off_row][top_left] = uw_out;
1020             U[off_row][bottom_right] =
ux_out;
1021         }
1022     }
1023     }
1024     }
1025     }
1026     }
1027     }
1028 }

1029
1030 //
=====
1031 // SVD_PAIRS: Alternative architecture with improved paral-
lization/pipelining at the expense of higher resource
1032 // requirements.
1033 // - Processes the input matrix by pairs of columns.
1034 // - IMPLEMENTATION TIP: If a fully unrolled implementation
is required partition the function's input/output argument
1035 // arrays on the column
1036 // dimension.
1037 template <int RowsA, int ColsA, class SVDTraits, typename In-
putType, typename OutputType>
1038 void svdPairs(const InputType A[RowsA][ColsA],
1039               OutputType S[RowsA][ColsA],
1040               OutputType U[RowsA][RowsA],
1041               OutputType V[ColsA][ColsA]) {
1042     // Initially only supporting square matrix
1043     assert(RowsA == ColsA);
1044
1045     const int ODD_DIM = (SVDTraits::MIN_DIM % 2 == 1 ? 1 :
0);
1046
1047     // The number of diagonal/column processors used in one
step. In a single step all rows/columns of the input array
1048     // will be processed.
1049     const int NUM_PROCESSORS = (SVDTraits::MIN_DIM + 1) / 2;
1050
1051     // The number of steps required to process the whole in-
put matrix for one sweep iteration.
1052     const int NUM_STEPS = SVDTraits::MIN_DIM + ODD_DIM - 1;
1053
1054     // Defines the number of banks (or pages) of the internal
memory.
1055     // - The step_loop reads from one page and writes to the
other, ping pongs between two pages for each
1056     // iteration/step.
1057     const int BANKS = 2;
1058
1059     // Stores the current indexes being process by each
columns/diagonal processor. The "pairs".
1060     int diag1_i[BANKS][NUM_PROCESSORS];
1061     int diag2_i[BANKS][NUM_PROCESSORS];
1062
1063     int d1_i, d2_i, c1_i, c2_i;
1064     bool col_swap = false;
1065
1066     // Internal memories for partial results
1067     typename SVDTraits::SIntType s_col1[NUM_PROCESSORS]
[RowsA + ODD_DIM];
1068     typename SVDTraits::SIntType s_col2[NUM_PROCESSORS]
[RowsA + ODD_DIM];
1069     typename SVDTraits::UIntType u_col1[NUM_PROCESSORS]
[RowsA + ODD_DIM];
1070     typename SVDTraits::UIntType u_col2[NUM_PROCESSORS]
[RowsA + ODD_DIM];
1071     typename SVDTraits::UIntType v_col1[NUM_PROCESSORS]
[RowsA + ODD_DIM];
1072     typename SVDTraits::UIntType v_col2[NUM_PROCESSORS]
[RowsA + ODD_DIM];
1073
1074     // Current S,U,V values being worked on, used in each
column processor
1075     typename SVDTraits::SIntType w_in, x_in, y_in, z_in;
1076     typename SVDTraits::SIntType w_int1, x_int1, y_int1,
z_int1;
1077     typename SVDTraits::SIntType w_int2, x_int2, y_int2,
z_int2;
1078     typename SVDTraits::SIntType w_out, x_out, y_out, z_out;
1079     typename SVDTraits::UIntType uw_in, ux_in, uy_in, uz_in;
1080     typename SVDTraits::UIntType uw_out, ux_out, uy_out,
uz_out;
1081     typename SVDTraits::VIntType vw_in, vx_in, vy_in, vz_in;
1082     typename SVDTraits::VIntType vw_out, vx_out, vy_out,
vz_out;
1083
1084     typename SVDTraits::SIntType w_out_sel, x_out_sel,
y_out_sel, z_out_sel;
1085     typename SVDTraits::UIntType uw_out_sel, ux_out_sel,
uy_out_sel, uz_out_sel;
1086     typename SVDTraits::VIntType vw_out_sel, vx_out_sel,
vy_out_sel, vz_out_sel;
1087
1088     // Diagonal processor results - 2x2 Rotation values (U
&V) and S
1089     typename SVDTraits::SIntType diag_w_out[NUM_PROCESSORS];
1090     typename SVDTraits::SIntType diag_x_out[NUM_PROCESSORS];
1091     typename SVDTraits::SIntType diag_y_out[NUM_PROCESSORS];
1092     typename SVDTraits::SIntType diag_z_out[NUM_PROCESSORS];
1093     typename SVDTraits::CSIntType uw_new[NUM_PROCESSORS];
1094     typename SVDTraits::CSIntType ux_new[NUM_PROCESSORS];
1095     typename SVDTraits::CSIntType uy_new[NUM_PROCESSORS];
1096     typename SVDTraits::CSIntType uz_new[NUM_PROCESSORS];
1097     typename SVDTraits::CSIntType vw_new[NUM_PROCESSORS];
1098     typename SVDTraits::CSIntType vx_new[NUM_PROCESSORS];
1099     typename SVDTraits::CSIntType vy_new[NUM_PROCESSORS];
1100     typename SVDTraits::CSIntType vz_new[NUM_PROCESSORS];
1101     // Local copies to avoid multiple reads

```

```

1102     typename SVDTraits::CSIntType uw_new_px, ux_new_px,
uy_new_px, uz_new_px;
1103     typename SVDTraits::CSIntType vw_new_px, vx_new_px,
vy_new_px, vz_new_px;
1104
1105 // IMPLEMENTATION TIP: Additional directives for fully un-
rolling
1106 // #pragma HLS ARRAY_PARTITION variable=diag1_i cyclic dim=2
factor=SVDTraits::UNROLL_FACTOR
1107 // #pragma HLS ARRAY_PARTITION variable=diag2_i cyclic dim=2
factor=SVDTraits::UNROLL_FACTOR
1108 // #pragma HLS ARRAY_PARTITION variable=s_col1 cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1109 // #pragma HLS ARRAY_PARTITION variable=s_col2 cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1110 // #pragma HLS ARRAY_PARTITION variable=u_col1 cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1111 // #pragma HLS ARRAY_PARTITION variable=u_col2 cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1112 // #pragma HLS ARRAY_PARTITION variable=v_col1 cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1113 // #pragma HLS ARRAY_PARTITION variable=v_col2 cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1114 // #pragma HLS ARRAY_PARTITION variable=diag_w_out cyclic
dim=1 factor=SVDTraits::UNROLL_FACTOR
1115 // #pragma HLS ARRAY_PARTITION variable=diag_x_out cyclic
dim=1 factor=SVDTraits::UNROLL_FACTOR
1116 // #pragma HLS ARRAY_PARTITION variable=diag_y_out cyclic
dim=1 factor=SVDTraits::UNROLL_FACTOR
1117 // #pragma HLS ARRAY_PARTITION variable=diag_z_out cyclic
dim=1 factor=SVDTraits::UNROLL_FACTOR
1118 // #pragma HLS ARRAY_PARTITION variable=uw_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1119 // #pragma HLS ARRAY_PARTITION variable=ux_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1120 // #pragma HLS ARRAY_PARTITION variable=uy_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1121 // #pragma HLS ARRAY_PARTITION variable=uz_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1122 // #pragma HLS ARRAY_PARTITION variable=vw_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1123 // #pragma HLS ARRAY_PARTITION variable=vx_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1124 // #pragma HLS ARRAY_PARTITION variable=vy_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1125 // #pragma HLS ARRAY_PARTITION variable=vz_new cyclic dim=1
factor=SVDTraits::UNROLL_FACTOR
1126
1127 // Load column memories
1128 // - Up-front transfer but can be pipelined/data flow
1129 col_load:
1130     for (int col = 0; col < NUM_PROCESSORS; col++) {
1131         diag1_i[0][col] = 2 * col;
1132         diag2_i[0][col] = 2 * col + 1;
1133         row_load:
1134             for (int row = 0; row < RowsA + ODD_DIM; row++) {
1135                 #pragma HLS PIPELINE
1136                 if (ODD_DIM) {
1137                     // When odd dimensions the first column of
the first processor is padded with 0's
1138                     if (row == 0) {
1139                         s_col1[col][0][row] = 0;
1140                         s_col2[col][0][row] = 0;
1141                     } else {
1142                         if (col == 0) {
1143                             s_col1[col][0][row] = 0;
1144                         } else {
1145                             s_col1[col][0][row] = A[row - 1][2 *
col - 1];
1146                             s_col2[col][0][row] = A[row - 1][2 *
col];
1147                         }
1148                     }
1149                 } else {
1150                     s_col1[col][0][row] = A[row][2 * col];
1151                     s_col2[col][0][row] = A[row][2 * col + 1];
1152                 }
1153             }
1154         }
1155     }
1156 sweep_loop:
1157     for (int sweepnum = 0; sweepnum < SVDTraits::NUM_SWEEPS;
sweepnum++) {
1158         step_loop:
1159         for (int step = 0; step < NUM_STEPS; step++) {
1160             const int INPUT_BANK = ((sweepnum * NUM_STEPS) +
step) % BANKS;
1161             const int OUTPUT_BANK = ((sweepnum * NUM_STEPS) +
step + 1) % BANKS;
1162
1163             // Independent loop to remove any dependency on the pro-
cessor loops
1164             diag_index_update:
1165             for (int px = 0; px < NUM_PROCESSORS; px++) {
1166                 // #pragma HLS UNROLL FACTOR =
SVDTraits::UNROLL_FACTOR
1167                 if (px == 0) {
1168                     diag1_i[OUTPUT_BANK][px] =
diag1_i[INPUT_BANK][px]; // Unchanged

```

```

1169                     diag1_i[OUTPUT_BANK][px + 1] =
diag2_i[INPUT_BANK][px];
1170                 } else if (px == NUM_PROCESSORS - 1) {
1171                     diag2_i[OUTPUT_BANK][px] =
diag1_i[INPUT_BANK][px];
1172                     diag2_i[OUTPUT_BANK][px - 1] =
diag2_i[INPUT_BANK][px];
1173                 } else {
1174                     diag1_i[OUTPUT_BANK][px + 1] =
diag1_i[INPUT_BANK][px];
1175                     diag2_i[OUTPUT_BANK][px - 1] =
diag2_i[INPUT_BANK][px];
1176                 }
1177             }
1178             // Diagonal processor loop
1179             // - In a separate loop as we need the rotations
from each diagonal processor before updating the
1181             // off diagonal values
1182             diag_px:
1183             for (int px = 0; px < NUM_PROCESSORS; px++) {
1184                 // IMPLEMENTATION TIP: Additional directives for fully un-
rolling
1185                 // #pragma HLS UNROLL FACTOR = SVDTraits::DIAG_UNROLL_FACTOR
1186                 #pragma HLS PIPELINE II = SVDTraits::DIAG_II
1187                 if (diag1_i[INPUT_BANK][px] <
diag2_i[INPUT_BANK][px]) {
1188                     d1_i = diag1_i[INPUT_BANK][px];
1189                     d2_i = diag2_i[INPUT_BANK][px];
1190                     col_swap = false;
1191                 } else {
1192                     d2_i = diag1_i[INPUT_BANK][px];
1193                     d1_i = diag2_i[INPUT_BANK][px];
1194                     col_swap = true;
1195                 }
1196                 // Fetch diagonal inputs; w, x, y, z
1197                 if (!col_swap) {
1198                     w_in = s_col1[px][INPUT_BANK][d1_i];
1199                     x_in = s_col2[px][INPUT_BANK][d1_i];
1200                     y_in = s_col1[px][INPUT_BANK][d2_i];
1201                     z_in = s_col2[px][INPUT_BANK][d2_i];
1202                 } else {
1203                     w_in = s_col2[px][INPUT_BANK][d1_i];
1204                     x_in = s_col1[px][INPUT_BANK][d1_i];
1205                     y_in = s_col2[px][INPUT_BANK][d2_i];
1206                     z_in = s_col1[px][INPUT_BANK][d2_i];
1207                 }
1208                 // Diagonal processor
1209                 if (ODD_DIM && px == 0) {
1210                     // Column 0 is padded with zeros. w and y
will be 0 so pass through
1211                     diag_w_out[px] = w_in;
1212                     diag_x_out[px] = x_in;
1213                     diag_y_out[px] = y_in;
1214                     diag_z_out[px] = z_in;
1215                     uw_new[px] = 1;
1216                     ux_new[px] = 0;
1217                     uy_new[px] = 0;
1218                     uz_new[px] = 1;
1219                     vw_new[px] = 1;
1220                     vx_new[px] = 0;
1221                     vy_new[px] = 0;
1222                     vz_new[px] = 1;
1223                 } else {
1224                     svd2x2(w_in, x_in, y_in, z_in,
uw_new[px], ux_new[px], uy_new[px], uz_new[px], vw_new[px],
1225                     vx_new[px], vy_new[px],
vz_new[px], diag_w_out[px], diag_x_out[px], diag_y_out[px],
1226                     diag_z_out[px]);
1227                 }
1228             }
1229             // Off-diagonal processors
1230             off_diag_px:
1231             for (int px = 0; px < NUM_PROCESSORS; px++) {
1232                 // IMPLEMENTATION TIP: Additional directives
for fully unrolling
1233                 // #pragma HLS UNROLL FACTOR =
SVDTraits::UNROLL_FACTOR
1234                 if (diag1_i[INPUT_BANK][px] <
diag2_i[INPUT_BANK][px]) {
1235                     c1_i = diag1_i[INPUT_BANK][px];
1236                     c2_i = diag2_i[INPUT_BANK][px];
1237                     col_swap = false;
1238                 } else {
1239                     c2_i = diag1_i[INPUT_BANK][px];
1240                     c1_i = diag2_i[INPUT_BANK][px];
1241                     col_swap = true;
1242                 }
1243                 // Fetch new U rotations for this column to
avoid multiple access to the ux_new arrays
1244                 uw_new_px = uw_new[px];
1245                 ux_new_px = ux_new[px];
1246                 uy_new_px = uy_new[px];
1247                 uz_new_px = uz_new[px];
1248                 off_diag_loop:
1249                 for (int off_px = 0; off_px < NUM_PROCESSORS;
off_px++) {

```

```

1251 #pragma HLS PIPELINE II = SVDTraits::OFF_DIAG_II
1252 // Inline mm2x2 function to enable resource sharing
1253 #pragma HLS INLINE recursive
1254 // Additional pragmas sometime required
when HLS is unable to identify that there are no inter loop
1255 // dependencies.
1256 // o HLS can identify false dependencies
on the column memories between loops. All the outputs in
1257 // the
1258 // loop are written to a different
memory "bank"/region using the INPUT_BANK and OUTPUT_BANK
1259 // indices

1260 // #pragma HLS dependence variable=s_col1
inter false

1261 // #pragma HLS dependence variable=s_col2
inter false

1262 // #pragma HLS dependence variable=u_col1
inter false

1263 // #pragma HLS dependence variable=u_col2
inter false

1264 // #pragma HLS dependence variable=v_col1
inter false

1265 // #pragma HLS dependence variable=v_col2
inter false

1266 // Determine index to use
1267 if (diag1_i[INPUT_BANK][off_px] <
1268     diag2_i[INPUT_BANK][off_px]) {
1269     d1_i = diag1_i[INPUT_BANK][off_px];
1270     d2_i = diag2_i[INPUT_BANK][off_px];
1271 } else {
1272     d2_i = diag1_i[INPUT_BANK][off_px];
1273     d1_i = diag2_i[INPUT_BANK][off_px];
1274 }
1275 // Fetch stored U and V values
1276 if (sweepnum == 0 && step == 0) {
1277     // First sweep construct identity
1278     if (px == off_px) {
1279         uw_in = 1;
1280         ux_in = 0;
1281         uy_in = 0;
1282         uz_in = 1;
1283         vw_in = 1;
1284         vx_in = 0;
1285         vy_in = 0;
1286         vz_in = 1;
1287     } else {
1288         uw_in = 0;
1289         ux_in = 0;
1290         uy_in = 0;
1291         uz_in = 0;
1292         vw_in = 0;
1293         vx_in = 0;
1294         vy_in = 0;
1295         vz_in = 0;
1296     }
1297 } else {
1298     // Using the diagonal indexes of the
other column processors
1299     if (!col_swap) {
1300         uw_in = u_col1[px][INPUT_BANK]
1301         [d1_i];
1302         ux_in = u_col2[px][INPUT_BANK]
1303         [d1_i];
1304         uy_in = u_col1[px][INPUT_BANK]
1305         [d2_i];
1306         uz_in = u_col2[px][INPUT_BANK]
1307         [d2_i];
1308         vw_in = v_col1[px][INPUT_BANK]
1309         [d1_i];
1310         vx_in = v_col2[px][INPUT_BANK]
1311         [d1_i];
1312         vy_in = v_col1[px][INPUT_BANK]
1313         [d2_i];
1314         vz_in = v_col2[px][INPUT_BANK]
1315         [d2_i];
1316     }
1317 }

```

```

1318 }
1319 // Fetch S values when not overlapping
with this columns diagonal
1320 if (off_px != px) {
1321     if (!col_swap) {
1322         w_in = s_col1[px][INPUT_BANK]
1323         [d1_i];
1324         x_in = s_col2[px][INPUT_BANK]
1325         [d1_i];
1326         y_in = s_col1[px][INPUT_BANK]
1327         [d2_i];
1328         z_in = s_col2[px][INPUT_BANK]
1329         [d2_i];
1330     } else {
1331         w_in = s_col2[px][INPUT_BANK]
1332         [d1_i];
1333         x_in = s_col1[px][INPUT_BANK]
1334         [d1_i];
1335         y_in = s_col2[px][INPUT_BANK]
1336         [d2_i];
1337         z_in = s_col1[px][INPUT_BANK]
1338         [d2_i];
1339     }
1340 } else {
1341     // Otherwise use values output by di-
agonal processor
1342     w_in = diag_w_out[px];
1343     x_in = diag_x_out[px];
1344     y_in = diag_y_out[px];
1345     z_in = diag_z_out[px];
1346 }
1347 // Update S values
1348 if (off_px < px) {
1349     // S values must be pre-multiplied
with earlier columns new U rotation values before
1350     // the V rotations due to this
columns processor are applied
1351     // U must be Hermitian transposed be-
fore it is applied to S
1352     mm2x2(hls::x_conj(uw_new[off_px]),
1353         hls::x_conj(ux_new[off_px]),
1354         hls::x_conj(uy_new[off_px]),
1355         hls::x_conj(uz_new[off_px]),
1356         w_in, x_in, y_in, z_in, w_int1, x_int1, y_int1, z_int1);
1357     } else {
1358         w_int1 = w_in;
1359         x_int1 = x_in;
1360         y_int1 = y_in;
1361         z_int1 = z_in;
1362     }
1363     if (off_px != px) {
1364         // Non-diagonal S values must be
post-multiplied with V rotations
1365         mm2x2(w_int1, x_int1, y_int1, z_int1,
1366             vw_new[px], vx_new[px], vy_new[px], vz_new[px], w_int2,
1367             x_int2, y_int2, z_int2);
1368     } else {
1369         w_int2 = w_int1;
1370         x_int2 = x_int1;
1371         y_int2 = y_int1;
1372         z_int2 = z_int1;
1373     }
1374     if (off_px > px) {
1375         // S values must be post-multiplied
with subsequent columns new U rotation values after
1376         // the V rotations due to this
columns processor are applied
1377         // U must be Hermitian transposed be-
fore it is applied to S
1378         mm2x2(hls::x_conj(uw_new[off_px]),
1379             hls::x_conj(ux_new[off_px]),
1380             hls::x_conj(uy_new[off_px]),
1381             hls::x_conj(uz_new[off_px]),
1382             w_int2, x_int2, y_int2, z_int2, w_out, x_out, y_out, z_out);
1383     } else {
1384         w_out = w_int2;
1385         x_out = x_int2;
1386         y_out = y_int2;
1387         z_out = z_int2;
1388     }
1389     // Update U and V values
1390     mm2x2(uw_in, ux_in, uy_in, uz_in,
1391         uw_new_px, ux_new_px, uy_new_px, uz_new_px, uw_out, ux_out,
1392         uy_out, uz_out);
1393     mm2x2(vw_in, vx_in, vy_in, vz_in,
1394         vw_new[px], vx_new[px], vy_new[px], vz_new[px], vw_out, vx_out,
1395         vy_out, vz_out);
1396 }
1397 // If the current indices for this column
have required the column sources to be swapped we need to
1398 // do the
1399 // same on the output
1400 if (!col_swap) {
1401     w_out_sel = w_out;

```

```

1391         x_out_sel = x_out;
1392         y_out_sel = y_out;
1393         z_out_sel = z_out;
1394
1395         uw_out_sel = uw_out;
1396         ux_out_sel = ux_out;
1397         uy_out_sel = uy_out;
1398         uz_out_sel = uz_out;
1399
1400         vw_out_sel = vw_out;
1401         vx_out_sel = vx_out;
1402         vy_out_sel = vy_out;
1403         vz_out_sel = vz_out;
1404     } else {
1405         w_out_sel = x_out;
1406         x_out_sel = w_out;
1407         y_out_sel = z_out;
1408         z_out_sel = y_out;
1409
1410         uw_out_sel = ux_out;
1411         ux_out_sel = uw_out;
1412         uy_out_sel = uz_out;
1413         uz_out_sel = uy_out;
1414
1415         vw_out_sel = vx_out;
1416         vx_out_sel = vw_out;
1417         vy_out_sel = vz_out;
1418         vz_out_sel = vy_out;
1419     }
1420
1421     // Store updated values in the correct
column memory
1422     if (px == 0) {
1423         s_col1[px][OUTPUT_BANK][d1_i] =
w_out_sel;
1424         s_col1[px + 1][OUTPUT_BANK][d1_i] =
x_out_sel;
1425         s_col1[px][OUTPUT_BANK][d2_i] =
y_out_sel;
1426         s_col1[px + 1][OUTPUT_BANK][d2_i] =
z_out_sel;
1427
1428         u_col1[px][OUTPUT_BANK][d1_i] =
uw_out_sel;
1429         u_col1[px + 1][OUTPUT_BANK][d1_i] =
ux_out_sel;
1430         u_col1[px][OUTPUT_BANK][d2_i] =
uy_out_sel;
1431         u_col1[px + 1][OUTPUT_BANK][d2_i] =
uz_out_sel;
1432
1433         v_col1[px][OUTPUT_BANK][d1_i] =
vw_out_sel;
1434         v_col1[px + 1][OUTPUT_BANK][d1_i] =
vx_out_sel;
1435         v_col1[px][OUTPUT_BANK][d2_i] =
vy_out_sel;
1436         v_col1[px + 1][OUTPUT_BANK][d2_i] =
vz_out_sel;
1437     } else if (px == NUM_PROCESSORS - 1) {
1438         s_col2[px][OUTPUT_BANK][d1_i] =
w_out_sel;
1439         s_col2[px - 1][OUTPUT_BANK][d1_i] =
x_out_sel;
1440         s_col2[px][OUTPUT_BANK][d2_i] =
y_out_sel;
1441         s_col2[px - 1][OUTPUT_BANK][d2_i] =
z_out_sel;
1442
1443         u_col2[px][OUTPUT_BANK][d1_i] =
uw_out_sel;
1444         u_col2[px - 1][OUTPUT_BANK][d1_i] =
ux_out_sel;
1445         u_col2[px][OUTPUT_BANK][d2_i] =
uy_out_sel;
1446         u_col2[px - 1][OUTPUT_BANK][d2_i] =
uz_out_sel;
1447
1448         v_col2[px][OUTPUT_BANK][d1_i] =
vw_out_sel;
1449         v_col2[px - 1][OUTPUT_BANK][d1_i] =
vx_out_sel;
1450         v_col2[px][OUTPUT_BANK][d2_i] =
vy_out_sel;
1451         v_col2[px - 1][OUTPUT_BANK][d2_i] =
vz_out_sel;
1452     } else {
1453         s_col1[px + 1][OUTPUT_BANK][d1_i] =
w_out_sel;
1454         s_col2[px - 1][OUTPUT_BANK][d1_i] =
x_out_sel;
1455         s_col1[px + 1][OUTPUT_BANK][d2_i] =
y_out_sel;
1456         s_col2[px - 1][OUTPUT_BANK][d2_i] =
z_out_sel;
1457
1458         u_col1[px + 1][OUTPUT_BANK][d1_i] =
uw_out_sel;
1459         u_col2[px - 1][OUTPUT_BANK][d1_i] =
ux_out_sel;

```

```

1460         u_col1[px + 1][OUTPUT_BANK][d2_i] =
uy_out_sel;
1461         u_col2[px - 1][OUTPUT_BANK][d2_i] =
uz_out_sel;
1462
1463         v_col1[px + 1][OUTPUT_BANK][d1_i] =
vw_out_sel;
1464         v_col2[px - 1][OUTPUT_BANK][d1_i] =
vx_out_sel;
1465         v_col1[px + 1][OUTPUT_BANK][d2_i] =
vy_out_sel;
1466         v_col2[px - 1][OUTPUT_BANK][d2_i] =
vz_out_sel;
1467     }
1468 }
1469 }
1470 }
1471 }
1472 // Transfer results
1473 col_store:
1474 for (int col = 0; col < NUM_PROCESSORS; col++) {
1475     row_store:
1476     for (int row = 0; row < RowsA; row++) {
1477 #pragma HLS PIPELINE
1478         if (ODD_DIM) {
1479             if (col != 0) {
1480                 S[row][2 * col - 1] = s_col1[col]
1481                 U[row][2 * col - 1] = u_col1[col]
1482                 V[row][2 * col - 1] = v_col1[col]
1483             }
1484             S[row][2 * col] = s_col2[col]
1485             U[row][2 * col] = u_col2[col]
1486             V[row][2 * col] = v_col2[col]
1487         } else {
1488             S[row][2 * col] = s_col1[col]
1489             U[row][2 * col + 1] = s_col2[col]
1490             V[row][2 * col + 1] = v_col2[col]
1491             U[row][2 * col + 1] = u_col2[col]
1492             V[row][2 * col] = v_col1[col]
1493             V[row][2 * col + 1] = v_col2[col]
1494         }
1495     }
1496 }
1497 }
1498 //
1499 //
=====
1500 // SVD_TOP: Top level function that selects implementation
architecture and internal types based on the
1501 // traits class provided via the SVDTraits template paramet-
er.
1502 // o Call this function directly if you wish to override the
default architecture choice or internal types
1503 template <int RowsA, int ColsA, class SVDTraits, typename In-
putType, typename OutputType>
1504 void svdTop(const InputType A[RowsA][ColsA],
1505             OutputType S[RowsA][ColsA],
1506             OutputType U[RowsA][RowsA],
1507             OutputType V[ColsA][ColsA]) {
1508     switch (SVDTraits::ARCH) {
1509     case 0:
1510         svdBasic<RowsA, ColsA, SVDTraits, InputType, Out-
putType>(A, S, U, V);
1511         break;
1512     case 1:
1513         svdPairs<RowsA, ColsA, SVDTraits, InputType, Out-
putType>(A, S, U, V);
1514         break;
1515     default:
1516         svdBasic<RowsA, ColsA, SVDTraits, InputType, Out-
putType>(A, S, U, V);
1517         break;
1518     }
1519 }
1520 /**
1521  * @brief SVD the entry point function
1522  *
1523  * @tparam RowsA      Row dimension
1524  * @tparam ColsA      Column dimension
1525  * @tparam InputType  Input data type
1526  * @tparam OutputType Output data type
1527  *
1528  * @param matrixAStrm Stream of input matrix
1529  * @param matrixSStrm Stream of singular values
1530  * @param matrixUStrm Stream of left singular vec-

```



```

tors output matrix
1532 * @param matrixVStrm          Stream of right singular
vectors output matrix
1533 */
1534 template <int RowsA,
1535           int ColsA,
1536           typename InputType,
1537           typename OutputType,
1538           typename SVDTraits = svdTraits<RowsA, ColsA, Input-
Type, OutputType> >
1539 void svd(hls::stream<InputType>& matrixAStrm,
1540         hls::stream<OutputType>& matrixSStrm,
1541         hls::stream<OutputType>& matrixUStrm,
1542         hls::stream<OutputType>& matrixVStrm) {
1543     InputType A[RowsA][ColsA];
1544     OutputType S[RowsA][ColsA];
1545     OutputType U[RowsA][ColsA];
1546     OutputType V[ColsA][ColsA];
1547
1548     for (int r = 0; r < RowsA; r++) {
1549 #pragma HLS PIPELINE
1550         for (int c = 0; c < ColsA; c++) {
1551             matrixAStrm.read(A[r][c]);
1552         }
1553     }
1554
1555     svdTop<RowsA, ColsA, SVDTraits, InputType, OutputType>(A,
S, U, V);
1556     for (int r = 0; r < RowsA; r++) {
1557 #pragma HLS PIPELINE
1558         for (int c = 0; c < ColsA; c++) {
1559             matrixSStrm.write(S[r][c]);
1560         }
1561     }
1562     for (int r = 0; r < RowsA; r++) {
1563 #pragma HLS PIPELINE
1564         for (int c = 0; c < RowsA; c++) {
1565             matrixUStrm.write(U[r][c]);
1566         }
1567     }
1568     for (int r = 0; r < ColsA; r++) {
1569 #pragma HLS PIPELINE
1570         for (int c = 0; c < ColsA; c++) {
1571             matrixVStrm.write(V[r][c]);
1572         }
1573     }
1574 }
1575 } // end namespace solver
1576 } // end namespace xf
1577 #endif

```

D.9 Vitis_solver/utlils/x_matrix_utils.hpp (256 lines, 7,493 B)

```

1 /*
2  * Copyright (C) 2019-2022, Xilinx, Inc.
3  * Copyright (C) 2022-2026, Advanced Micro Devices, Inc.
4  *
5  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
6  * you may not use this file except in compliance with the
License.
7  * You may obtain a copy of the License at
8  *
9  * http://www.apache.org/licenses/LICENSE-2.0
10  *
11  * Unless required by applicable law or agreed to in writing,
software
12  * distributed under the License is distributed on an "AS IS"
BASIS,
13  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14  * See the License for the specific language governing per-
missions and
15  * limitations under the License.
16  */
17
18 #ifndef _XF_SOLVER_MATRIX_UTILS_HPP_
19 #define _XF_SOLVER_MATRIX_UTILS_HPP_
20
21 #include "ap_fixed.h"
22 #include "utils/x_hls_utils.h"
23 #include "hls_x_complex.h"
24 #include "utils/std_complex_utils.h"
25 #include "hls_math.h"
26
27 #ifndef __SYNTHESIS__
28 #include <iostream>
29 #include <iomanip>
30 #include <string>
31 #include <sstream>
32 #include <stdio.h>
33 #endif
34
35 namespace xf {
36 namespace solver {
37
38 //

```

```

=====
39 // Transpose Types & Operation
40 struct NoTranspose {
41     const static int TransposeType = 0;
42     template <int RowsA, int ColsA, typename InputType>
43     static InputType GetElement(const InputType A[RowsA]
[ColsA], unsigned Row, unsigned Col) {
44         // No transpose, default
45         return A[Row][Col];
46     }
47 };
48
49 struct Transpose {
50     const static int TransposeType = 1;
51     template <int RowsA, int ColsA, typename InputType>
52     static InputType GetElement(const InputType A[RowsA]
[ColsA], unsigned Row, unsigned Col) {
53         // Transpose, no conjugate
54         return A[Col][Row];
55     }
56 };
57
58 struct ConjugateTranspose {
59     const static int TransposeType = 2;
60     template <int RowsA, int ColsA, typename InputType>
61     static InputType GetElement(const InputType A[RowsA]
[ColsA], unsigned Row, unsigned Col) {
62         // Complex conjugate transpose.
63         // o For non x_complex types this function will re-
turn an unaltered value.
64         return hls::x_conj(A[Col][Row]);
65     }
66 };
67
68 template <class TransposeForm, int RowsA, int ColsA, typename
InputType>
69 InputType GetMatrixElement(const InputType A[RowsA][ColsA],
unsigned Row, unsigned Col) {
70     // Need to help the compiler identify that the GetElement
member function is a template
71     return TransposeForm::template GetElement<RowsA, ColsA,
InputType>(A, Row, Col);
72 }
73
74 //
=====
75 // Common math operations and constants. Wrappers/templates
to select the correct function based on type.
76
77 // sqrt
78 static half x_sqrt(half x) {
79     return hls::half_sqrt(x);
80 }
81 static float x_sqrt(float x) {
82     return sqrtf(x);
83 }
84 static double x_sqrt(double x) {
85     return sqrt(x);
86 }
87 template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1>
88 ap_fixed<W1, I1, Q1, O1, N1> x_sqrt(ap_fixed<W1, I1, Q1, O1,
N1> x) {
89     return hls::sqrt((double)x);
90 }
91
92 // copysign
93 static float x_copysign(float a, float b) {
94     return copysignf(a, b);
95 }
96 static double x_copysign(double a, double b) {
97     return copysign(a, b);
98 }
99
100 // sign
101 static float x_sign(float x) {
102     if (x == 0.0f) {
103         return 0.0f;
104     } else {
105         return copysignf(1.0f, x);
106     }
107 }
108 static double x_sign(double x) {
109     if (x == 0.0) {
110         return 0.0;
111     } else {
112         return copysign(1.0, x);
113     }
114 }
115 template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int N1>
116 ap_fixed<W1, I1, Q1, O1, N1> x_sign(ap_fixed<W1, I1, Q1, O1,
N1> x) {
117     ap_fixed<W1, I1, Q1, O1, N1> tmp = 0;
118     if (x > 0) {
119         tmp = 1;
120     } else {
121         tmp = -1;

```

```

122     }
123     return tmp;
124 }
125
126 // reciprocal sqrt
127 static half x_rsqrt(half x) {
128     return hls::half_rsqrt(x);
129 }
130 static float x_rsqrt(float x) {
131     return hls::rsqrtf(x);
132 }
133 static double x_rsqrt(double x) {
134     return hls::rsqrt(x);
135 }
136
137 // isneg
138 static int x_isneg(float x) {
139     fp_struct<float> fs = x;
140     return fs.__signbit();
141 }
142 static int x_isneg(double x) {
143     fp_struct<double> fs = x;
144     return fs.__signbit();
145 }
146
147 #ifndef __SYNTHESIS__
148 // Matrix Display
149 //
=====
150 // Non-specific data
151 // -----
152 // This can be used by PoD or any class supporting "<<"
153 // setprecision may fail on non floating point types
154 template <typename T>
155 struct xil_printer {
156     static std::string to_s(T x, unsigned prec = 10) {
157         std::stringstream ss;
158         ss << std::setiosflags(std::ios::fixed) << std::set-
precision(prec) << x;
159
160         return ss.str();
161     }
162 };
163
164 // Complex data
165 // -----
166 // This is used by complex data of any type. A printer is
called to print the real and imaginary
167 // parts, so this just handles the formatting of x+jy
168 template <typename T>
169 struct xil_printer<hls::x_complex<T>> {
170     static std::string to_s(hls::x_complex<T> x, unsigned
prec = 10) {
171         // Use the basic type printer to print the real and
imaginary parts
172         typedef xil_printer<T> printer;
173
174         std::stringstream ss;
175         // Remember to deal with -0
176         bool neg_imag = x.imag() <= -0 ? true : false;
177
178         // Casting to "T" avoids "error: operands to ?: have
different types." when using ap.fixed.
179         T imag = neg_imag ? (T)-x.imag() : (T)x.imag();
180
181         ss << printer::to_s(x.real(), prec) << (neg_imag ? "
- j*" : " + j*") << printer::to_s(imag, prec);
182         return ss.str();
183     }
184 };
185
186 template <unsigned ROWS, unsigned COLS, typename T, class
TransposeForm>
187 void print_matrix(T a[ROWS][COLS], std::string prefix = "",
unsigned prec = 10, unsigned matlab_format = 0) {
188     typedef xil_printer<T> printer;
189
190     std::string res[ROWS][COLS];
191     unsigned widest_entry = 0;
192
193     // Get the individual fields
194     for (int r = 0; r < ROWS; r++) {
195         for (int c = 0; c < COLS; c++) {
196             T tmp;
197
198             tmp = GetMatrixElement<TransposeForm, ROWS, COLS,
T>(a, r, c);
199
200             res[r][c] = printer::to_s(tmp, prec);
201             if (res[r][c].length() > widest_entry) {
202                 widest_entry = res[r][c].length();
203             }
204         }
205     }
206
207     // Print fields. Each column should be "widest_entry"
chars wide

```

```

209     char col_gap_str[5] = " ";
210     unsigned col_width = widest_entry;
211
212     for (int r = 0; r < ROWS; r++) {
213         if (!matlab_format) {
214             printf("%s|", prefix.c_str());
215         } else {
216             if (r == 0) {
217                 printf("%s[", prefix.c_str());
218             } else {
219                 printf("%s ", prefix.c_str());
220             }
221         }
222         for (int c = 0; c < COLS; c++) {
223             unsigned num_spaces_needed = col_width - res[r]
[c].length();
224             for (int x = 0; x < num_spaces_needed; x++) {
225                 printf(" ");
226             }
227             if (!matlab_format) {
228                 printf("(%s)", res[r][c].c_str());
229             } else {
230                 printf("%s", res[r][c].c_str());
231             }
232
233             if (c != COLS - 1) {
234                 printf("%s", col_gap_str);
235                 if (matlab_format) {
236                     printf(",");
237                 }
238             }
239         }
240         if (!matlab_format) {
241             printf(" |\n");
242         } else {
243             if (r == ROWS - 1) {
244                 printf(" ];\n");
245             } else {
246                 printf(" ;\n");
247             }
248         }
249     }
250 }
251 #endif
252
253 } // namespace solver
254 } // namespace xf
255
256 #endif

```

D.10 Vitis_solver_L2/hw/math_helper.hpp (104 lines, 1,967 B)

```

1 /*
2  * Copyright (C) 2019-2022, Xilinx, Inc.
3  * Copyright (C) 2022-2025, Advanced Micro Devices, Inc.
4  *
5  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
6  * you may not use this file except in compliance with the
License.
7  * You may obtain a copy of the License at
8  *
9  * http://www.apache.org/licenses/LICENSE-2.0
10 *
11 * Unless required by applicable law or agreed to in writing,
software
12 * distributed under the License is distributed on an "AS IS"
BASIS,
13 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 * See the License for the specific language governing per-
missions and
15 * limitations under the License.
16 */
17
18 /**
19  * @file math_helper.hpp
20  * @brief enable sw-emu on embedded platforms.
21  */
22
23 #ifndef XF_SOLVER_MATH_HELPER_HPP
24 #define XF_SOLVER_MATH_HELPER_HPP
25
26 #include "hls_math.h"
27
28 namespace xf {
29 namespace solver {
30 namespace internal {
31 namespace m {
32
33 // abs
34 template <typename T>
35 T abs(T a) {
36     return hls::abs(a);
37 }
38
39 double abs(double a) {
40     #if defined(__aarch64__) || defined(__arm__)

```

```

41     return std::abs(a);
42 #else
43     return hls::abs(a);
44 #endif
45 }
46
47 float abs(float a) {
48 #if defined(__aarch64__) || defined(__arm__)
49     return std::abs(a);
50 #else
51     return hls::abs(a);
52 #endif
53 }
54
55 // fabs
56 template <typename T>
57 T fabs(T a) {
58     return hls::fabs(a);
59 }
60
61 double fabs(double a) {
62 #if defined(__aarch64__) || defined(__arm__)
63     return std::fabs(a);
64 #else
65     return hls::fabs(a);
66 #endif
67 }
68
69 float fabs(float a) {
70 #if defined(__aarch64__) || defined(__arm__)
71     return std::fabs(a);
72 #else
73     return hls::fabs(a);
74 #endif
75 }
76
77 // sqrt
78 template <typename T>
79 T sqrt(T a) {
80     return hls::sqrt(a);
81 }
82
83 double sqrt(double a) {
84 #if defined(__aarch64__) || defined(__arm__)
85     return std::sqrt(a);
86 #else
87     return hls::sqrt(a);
88 #endif
89 }
90
91 float sqrt(float a) {
92 #if defined(__aarch64__) || defined(__arm__)
93     return std::sqrt(a);
94 #else
95     return hls::sqrt(a);
96 #endif
97 }
98
99 } // m
100 } // internal
101 } // solver
102 } // xf
103
104 #endif

```

D.11 Vitis_solver_L2/potrf.hpp (200 lines, 5,967 B)

```

1  /*
2  * Copyright (C) 2019-2022, Xilinx, Inc.
3  * Copyright (C) 2022-2025, Advanced Micro Devices, Inc.
4  *
5  * Licensed under the Apache License, Version 2.0 (the "Li-
6  * cense");
7  * you may not use this file except in compliance with the
8  * License.
9  * You may obtain a copy of the License at
10 *
11 * http://www.apache.org/licenses/LICENSE-2.0
12 *
13 * Unless required by applicable law or agreed to in writing,
14 * software
15 * distributed under the License is distributed on an "AS IS"
16 * BASIS,
17 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either
18 * express or
19 * implied.
20 * See the License for the specific language governing per-
21 * missions and
22 * limitations under the License.
23 */
24
25 * @file potrf.hpp
26 * @brief This file contains implementation of Cholesky De-
27 * composition.
28 */
29
30 #ifndef _XF_SOLVER_POTRF_HPP_

```

```

24 #define _XF_SOLVER_POTRF_HPP_
25
26 #include <hw/math_helper.hpp>
27 #include <float.h>
28
29 namespace xf {
30 namespace solver {
31 namespace internal {
32
33 template <typename T, int N, int NCU>
34 void chol_col(int n, T dataA[(N + NCU - 1) / NCU][N], T
35 dataj[N], T tmp1_i, int num, int j) {
36     Loop_per_Unit:
37     for (int p = (j + 1) / NCU; p < (n + NCU - 1) / NCU; p++) {
38         #pragma HLS loop_tripcount min = 1 max = 8
39         T tmp_i[16] = {0}; tmp3_i, tmp1[8], tmp2[4], tmp3[2];
40         #pragma HLS BIND_STORAGE variable = tmp_i type = ram_2p impl
41         = lutram
42         Loop_vec_mul:
43         for (int k = 0; k < j; k++) {
44             #pragma HLS loop_tripcount min = 1 max = N
45             #pragma HLS pipeline
46             #pragma HLS dependence variable = tmp_i inter false
47             #pragma HLS dependence variable = dataA inter false
48             tmp_i[k % 16] += dataA[p][k] * dataj[k];
49         }
50         Loop_add_1:
51         for (int j = 0; j < 8; j++) {
52             #pragma HLS pipeline
53             tmp1[j] = tmp_i[j] + tmp_i[j + 8];
54         }
55         Loop_add_2:
56         for (int j = 0; j < 4; j++) {
57             #pragma HLS pipeline
58             tmp2[j] = tmp1[j] + tmp1[j + 4];
59         }
60         Loop_add_3:
61         for (int j = 0; j < 2; j++) {
62             #pragma HLS pipeline
63             tmp3[j] = tmp2[j] + tmp2[j + 2];
64         }
65         tmp3_i = tmp3[0] + tmp3[1];
66         if (p * NCU + num > j) dataA[p][j] = (dataA[p][j] -
67 tmp3_i) / tmp1_i;
68     }
69 }
70
71 template <typename T, int N, int NCU>
72 void chol_jj(T dataA[NCU][(N + NCU - 1) / NCU][N], T
73 dataj[NCU][N], T& tmp1_j, int& j) {
74     T tmp1[16] = {0}; tmp3_j, tmp1[8], tmp2[4], tmp3[2];
75     #pragma HLS BIND_STORAGE variable = tmp type = ram_2p impl =
76     lutram
77     Loop_vec_mul_jj:
78     for (int k = 0; k < j; k++) {
79         #pragma HLS pipeline
80         #pragma HLS dependence variable = tmp inter false
81         #pragma HLS dependence variable = dataA inter false
82         T tmp2_j = dataA[j % NCU][j / NCU][k];
83         tmp[k % 16] += tmp2_j * tmp2_j;
84         for (int p = 0; p < NCU; p++) {
85             #pragma HLS unroll
86             dataj[p][k] = tmp2_j;
87         }
88     }
89     Loop_add_1:
90     for (int j = 0; j < 8; j++) {
91         #pragma HLS pipeline
92         tmp1[j] = tmp[j] + tmp[j + 8];
93     }
94     Loop_add_2:
95     for (int j = 0; j < 4; j++) {
96         #pragma HLS pipeline
97         tmp2[j] = tmp1[j] + tmp1[j + 4];
98     }
99     Loop_add_3:
100    for (int j = 0; j < 2; j++) {
101        #pragma HLS pipeline
102        tmp3[j] = tmp2[j] + tmp2[j + 2];
103    }
104    tmp3_j = tmp3[0] + tmp3[1];
105    const T limit = (sizeof(T) == 4) ? 1e-1 : 1e-6;
106    T sub = dataA[j % NCU][j / NCU][j] - tmp3_j;
107    if ((sub < 0) && (sub > -limit)) {
108        tmp1_j = 0;
109    } else {
110        tmp1_j = xf::solver::internal::m::sqrt(sub);
111    }
112 }

```



```

118 #ifndef __SYNTHESIS__
119 #ifdef _DEBUG_SOLVER_
120     if (sub <= -limit) {
121         std::cout << "sub = " << sub << "\t " << dataA[j %
NCU][j / NCU][j] << "\t tmp = " << tmp3_j << std::endl;
122     }
123 #endif
124 #endif
125
126     dataA[j % NCU][j / NCU][j] = tmp1_j;
127 }
128
129 template <typename T, int N, int NCU>
130 void chol_col_wrapper(int n, T dataA[NCU][(N + NCU - 1) /
NCU][N], T dataj[NCU][N], T tmp1, int j) {
131 #pragma HLS DATAFLOW
132
133 Loop_row:
134     for (int num = 0; num < NCU; num++) {
135 #pragma HLS unroll factor = NCU
136         chol_col<T, N, NCU>(n, dataA[num], dataj[num], tmp1,
num, j);
137     }
138 }
139
140 template <typename T, int N, int NCU>
141 void cholesky_core(int n, T dataA[NCU][(N + NCU - 1) / NCU
[N]] {
142     T tmp1, dataj[NCU][N];
143 #pragma HLS array_partition variable = dataj cyclic factor =
NCU
144
145 Loop_col:
146     for (int j = 0; j < n; ++j) {
147         chol_jj<T, N, NCU>(dataA, dataj, tmp1, j);
148         chol_col_wrapper<T, N, NCU>(n, dataA, dataj, tmp1,
j);
149     }
150 }
151
152 } // namespace internal
153 /**
154  * @brief This function computes the Cholesky decomposition
of matrix \f$A\f$ \n
155  * \f{equation*} \{A = L \{L\}^T\}\f{
156  * where \f$A\f$ is a dense symmetric
positive-definite matrix of size \f$m \times m\f$, \f$A\f$ is
157  * a lower triangular matrix, and \f$L\}^T\f$ is the trans-
posed matrix of \f$L\}\f$. \n
158  * The maximum matrix size supported in FPGA is templated by
NMAX.
159  *
160  * @tparam T data type (support float and double)
161  * @tparam NMAX maximum number of rows/columns of input mat-
rix
162  * @tparam NCU number of computation unit
163  * @param[in] m number of rows/cols of matrix A
164  * @param[in,out] A input matrix of size \f$m \times m\f$
165  * @param[in] lda leading dimension of input matrix A
166  * @param[out] info output info (unused)
167  */
168
169 template <typename T, int NMAX, int NCU>
170 void potrf(int m, T* A, int lda, int& info) {
171     if (NMAX == 1)
172         A[0] = xfst::solver::internal::m::sqrt(A[0]);
173     else {
174         static T matA[NCU][(NMAX + NCU - 1) / NCU][NMAX];
175 #pragma HLS array_partition variable = matA cyclic factor =
NCU
176 #pragma HLS BIND_STORAGE variable = matA type = xpm_memory
impl = uram
177
178         Loop_read:
179             for (int r = 0; r < m; r++) {
180                 for (int c = 0; c < m; c++) {
181 #pragma HLS pipeline
182                     matA[r % NCU][r / NCU][c] = A[r * lda + c];
183                 }
184             }
185
186             internal::cholesky_core<T, NMAX, NCU>(m, matA);
187
188         Loop_write:
189             for (int r = 0; r < m; r++) {
190                 for (int c = 0; c < m; c++) {
191 #pragma HLS pipeline
192                     A[r * lda + c] = matA[r % NCU][r / NCU][c];
193                 }
194             }
195         }
196 }
197
198 } // namespace solver
199 } // namespace xfst
200 #endif

```

D.12 finn_hlslib/activations.hpp (387 lines, 14,453 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31
*****
32 /
*****
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 *          Thomas B. Preusser <thomas.preusser@utexas.edu>
37 *          Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 *          Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * @file activations.hpp
41 *
42 * Library of templated HLS classes for BNN deployment.
43 * This file lists a set of classes used to implement
threshold memory in neural network.
44 *
45 *
46 * This project has received funding from the European Uni-
on's Framework
47 * Programme for Research and Innovation Horizon 2020
(2014-2020) under
48 * the Marie Skłodowska-Curie Grant Agreement No. 751339.
49 *
50
*****
*****/
51
52 #ifndef ACTIVATIONS_HPP
53 #define ACTIVATIONS_HPP
54
55 #include "interpret.hpp"
56 #include <hls_stream.h>
57 #include <functional>
58
59 namespace comp{
60     using std::binary_function;
61
62     template<typename input_type_1 = void, typename in-
put_type_2 = void>

```

```

63     struct greater;
64
65     template<typename input_type_1 = void, typename in-
put_type_2 = void>
66     struct less;
67
68     template<typename input_type_1 = void, typename in-
put_type_2 = void>
69     struct greater_equal;
70
71     template<typename input_type_1 = void, typename in-
put_type_2 = void>
72     struct less_equal;
73
74     template<typename input_type_1 = void, typename in-
put_type_2 = void, typename result_type = void>
75     struct add;
76
77     template<typename input_type_1 = void, typename in-
put_type_2 = void, typename result_type = void>
78     struct mul;
79
80     template<typename input_type_1 = void, typename in-
put_type_2 = void, typename result_type = void>
81     struct max;
82
83     template<typename input_type_1, typename input_type_2>
84     struct greater : public binary_function<input_type_1, in-
put_type_2, ap_uint<1>> {
85         ap_uint<1>
86         operator()(const input_type_1& a, const input_type_2&
b) const
87         { return a > b; }
88     };
89
90     template<typename input_type_1, typename input_type_2>
91     struct less : public binary_function<input_type_1, in-
put_type_2, ap_uint<1>> {
92         ap_uint<1>
93         operator()(const input_type_1& a, const input_type_2&
b) const
94         { return a < b; }
95     };
96
97     template<typename input_type_1, typename input_type_2>
98     struct greater_equal : public bin-
ary_function<input_type_1, input_type_2, ap_uint<1>> {
99         ap_uint<1>
100         operator()(const input_type_1& a, const input_type_2&
b) const
101         { return a >= b; }
102     };
103
104     template<typename input_type_1, typename input_type_2>
105     struct less_equal : public binary_function<input_type_1,
input_type_2, ap_uint<1>> {
106         ap_uint<1>
107         operator()(const input_type_1& a, const input_type_2&
b) const
108         { return a <= b; }
109     };
110
111     template<typename input_type_1, typename input_type_2,
typename result_type>
112     struct add : public binary_function<input_type_1, in-
put_type_2, result_type> {
113         result_type
114         operator()(const input_type_1& a, const input_type_2&
b) const
115         { return a + b; }
116     };
117
118     template<typename input_type_1, typename input_type_2,
typename result_type>
119     struct mul : public binary_function<input_type_1, in-
put_type_2, result_type> {
120         result_type
121         operator()(const input_type_1& a, const input_type_2&
b) const
122         { return a * b; }
123     };
124
125     template<typename input_type_1, typename input_type_2,
typename result_type>
126     struct max : public binary_function<input_type_1, in-
put_type_2, result_type> {
127         result_type
128         operator()(const input_type_1& a, const input_type_2&
b) const
129         { return a > b ? a : b; }
130     };
131 }
132
133 /**
134  * General contract for activation functions.
135  *
136  * This class itself has no formal significance for the im-
plementation
137  * of the MVAU. Implementations of activation functions are
encouraged

```

```

138  * to implement it nonetheless to guarantee appropriate func-
tion
139  * signatures.
140  */
141     template<typename TA, typename TO>
142     class Activation {
143     public:
144         TA init(__attribute__((unused)) unsigned const nf, __at-
tribute__((unused)) unsigned const pe) const {
145             #pragma HLS inline
146             return TA(0);
147         }
148     };
149     /**
150     * Compute the activation of the passed accumulator value
accu in row idx.
151     */
152     TO activate(unsigned const nf, unsigned const pe, TA
const &accu) const;
153 };
154
155 /**
156  * A no-op activation that simply outputs the computed accu-
mulator
157  * output as the final result.
158  */
159     template<typename T>
160     class PassThroughActivation : public Activation<T, T> {
161     public:
162         T activate(__attribute__((unused)) unsigned const nf,
__attribute__((unused)) unsigned const pe, T const &accu) const {
163             #pragma HLS inline
164             return accu;
165         }
166     };
167
168     /**
169     * Use a simple global threshold comparison as activation
function.
170     *
171     * The constant threshold is initialized at construction.
172
173     * The default comparison returns true if the threshold value
is
174     * smaller than the passed accumulator value.
175     */
176     template<typename TA, typename Compare = comp::less<TA, TA>>
177     class ThresholdActivation : public Activation<TA, bool> {
178     public:
179         ThresholdActivation(TA const &threshold) :
m_threshold(threshold) {
180             #pragma HLS inline
181         }
182     };
183     public:
184         bool activate(__attribute__((unused)) unsigned const nf,
__attribute__((unused)) unsigned const pe, TA const &accu) const {
185             #pragma HLS inline
186             return Compare()(m_threshold, accu);
187         }
188     };
189
190     /**
191     * Use a simple per-row threshold comparison as activation
function.
192     *
193     * The thresholds are taken from an array indexed by output
row.
194     * It is currently public to allow direct initialization and
195     * to make its name accessible for top-level HLS pragmas.
196     */
197     * The default comparison returns true if the threshold value
defined for
198     * the indexed row is smaller than the passed accumulator
value.
199     */
200     template<unsigned NF, unsigned PE, unsigned NumTH,
typename TA, typename TR, int ActVal = 0, typename
Compare = comp::less<TA, TA>>
201     class ThresholdsActivation {
202     public:
203         TA m_thresholds[PE][NF][NumTH];
204     };
205     public:
206         TA init(__attribute__((unused)) unsigned const nf, __at-
tribute__((unused)) unsigned const pe) const {
207             #pragma HLS inline
208             return TA(0);
209         }
210     };
211     public:
212         TR activate(unsigned const nf, unsigned const pe, TA
const &accu) const {
213             #pragma HLS inline
214             TR result=ActVal;
215             for(unsigned int i=0; i< NumTH; i++){
216                 #pragma HLS unroll
217                 result+=Compare()(m_thresholds[pe][nf][i], accu);

```

```

219     }
220     return result;
221 }
222 };
223
224
225 /*!
226 * \brief Use a simple activation function with per-row parameters.
227 *
228 * The parameters are taken from an array indexed by output row.
229 * It is currently public to allow direct initialization and
230 * to make its name accessible for top-level HLS pragmas.
231 *
232 * \tparam NF      First dimension of the parameter matrix
233 * \tparam PE      Second dimension of the parameter matrix
234 * \tparam TI      DataType of input layer values
235 * \tparam TP      DataType of parameters
236 * \tparam TR      DataType of return values
237 * \tparam Fxn     Function to be applied on the channel input value
238 */
239
240 template<unsigned NF, unsigned PE,
241         typename TI, typename TP, typename TR, typename Fxn =
242         comp::mul<TI, TP, TR>>
243 class ChannelWiseOperation {
244 public:
245     TP parameters[PE][NF];
246     TI init(__attribute__((unused)) unsigned const nf, __attribute__((unused)) unsigned const pe) const {
247         #pragma HLS inline
248         return TI(0);
249     }
250 public:
251     TR activate(unsigned const nf, unsigned const pe, TI const &in) const {
252         #pragma HLS inline
253         TR result = Fxn()(parameters[pe][nf], in);
254         return result;
255     }
256 };
257
258 /*!
259 * \brief Thresholding function for multiple images
260 *
261 * The function performs thresholds comparison with input activation vector,
262 * and generating output based on the comparison results
263 */
264
265 * \tparam ImgDim      Total spatial size of input feature map
266
267 * \tparam NumChannels  Number of channels in input feature map
268
269 * \tparam PE          Number of output rows computed in parallel
270 * \tparam TSrcI       DataType of the input activation (as used in the MAC)
271 * \tparam TDstI       DataType of the output activation (as generated by the activation)
272 * \tparam TI          DataType of the input stream - safely deducible from the parameters
273 * \tparam TO          DataType of the output stream - safely deducible from the parameters
274 * \tparam TA          DataType of the activation class (e.g. thresholds) - safely deducible from the parameters
275 *
276 * \param in           Input stream
277 * \param out          Output stream
278 * \param activation   Activation class
279 * \param reps         Number of time the function has to be repeatedly executed (e.g. number of images)
280 */
281
282 template <
283     unsigned ImgDim, unsigned NumChannels, unsigned PE,
284     typename TSrcI = Identity, typename TDstI = Identity,
285     typename TI, typename TO, typename TA>
286 void Thresholding_Batch(hls::stream<TI> &in,
287                        hls::stream<TO> &out,
288                        TA const &activation,
289                        int const reps)
290 {
291     // how many different rows each neuron will compute
292     // alternatively: number of vertical matrix chunks
293     constexpr unsigned NF = NumChannels / PE;
294
295     // everything merged into a common iteration space (one "big" loop instead
296
297     // of smaller nested loops) to get the pipelining the way we want
298     unsigned nf = 0;
299     for (unsigned i = 0; i < reps * ImgDim * NF; i++) {
300         #pragma HLS pipeline style=fllp II=1

```

```

301         TI const inElem = in.read();
302         auto outElem = TDstI().template operator()<TO>();
303         for (unsigned pe = 0; pe < PE; pe++)
304         {
305             #pragma HLS UNROLL
306             auto const act = TSrcI()(inElem);
307             outElem(pe,0,1) = activation.activate(nf, pe, act(pe, 0));
308         }
309         out.write(outElem);
310         if (++nf == NF)
311         {
312             nf = 0;
313         }
314     }
315 }
316
317 /*!
318 * \brief Thresholding function for multiple images, with streaming thresholds
319 *
320 * The function performs thresholds comparison with input activation vector,
321 * and generating output based on the comparison results
322 */
323
324 * \tparam ImgDim      Total spatial size of input feature map
325
326 * \tparam NumChannels  Number of channels in input feature map
327
328 * \tparam PE          Number of output rows computed in parallel
329 * \tparam TSrcI       DataType of the input activation (as used in the MAC)
330 * \tparam TDstI       DataType of the output activation (as generated by the activation)
331 * \tparam ActVal      Initial value of activation at start of thresholding procedure
332 * \tparam TT          DataType of the thresholds stream
333 * \tparam NumSteps    Number of thresholds per activation
334 * \tparam TI          DataType of the input stream - safely deducible from the parameters
335 * \tparam TO          DataType of the output stream - safely deducible from the parameters
336 *
337 * \param in           Input stream
338 * \param out          Output stream
339 * \param weight       Weight stream
340 * \param reps         Number of time the function has to be repeatedly executed (e.g. number of images)
341 */
342
343 template <
344     unsigned ImgDim, unsigned NumChannels, unsigned PE,
345     typename TSrcI = Identity, typename TDstI = Identity,
346     int ActVal=0, typename TT, unsigned int NumSteps,
347     typename TI, typename TO>
348 void Thresholding_Stream_Batch(hls::stream<TI> &in,
349                               hls::stream<TO> &out,
350                               hls::stream<ap_uint<PE*NumSteps*TT::width>> &weight,
351                               int const reps)
352 {
353     // how many different rows each neuron will compute
354     // alternatively: number of vertical matrix chunks
355     unsigned const NF = NumChannels / PE;
356
357     ThresholdsActivation<1, PE, NumSteps, TT, TO, ActVal,
358     comp::less_equal<TT, TT>> internal_thr;
359     #pragma HLS ARRAY_PARTITION variable=internal_thr.m_thresholds complete dim=0
360
361     // everything merged into a common iteration space (one "big" loop instead
362
363     // of smaller nested loops) to get the pipelining the way we want
364     for (unsigned i = 0; i < reps * ImgDim * NF; i++)
365     {
366         #pragma HLS pipeline style=fllp II=1
367         ap_uint<PE*NumSteps*TT::width> packed_thr;
368         packed_thr = weight.read();
369         // slicer to get 1 PE's worth of thresholds
370         auto const pe_slicer =
371         Slice<ap_uint<NumSteps*TT::width>>()>(packed_thr);
372
373         TI inElem;
374         inElem = in.read();
375         auto outElem = TDstI().template operator()<TO>();
376
377         for (unsigned pe = 0; pe < PE; pe++)
378         {
379             #pragma HLS UNROLL
380             // slicer to get individual thresholds
381             auto const thr_slicer = Slice<TT>()(pe_slicer(pe, 0));
382             for (unsigned nt = 0; nt < NumSteps; nt++)
383             {
384                 #pragma HLS UNROLL

```

```

377     internal_thr.m_thresholds[pe][0][nt] = thr_slicer(nt,
0);
378     }
379
380     auto const act = TSrcI()(inElem);
381     outElem(pe,0,1) = internal_thr.activate(0, pe, act(pe,
0));
382     }
383     out.write(outElem);
384     }
385 }
386
387 #endif

```

D.13 finn_hlslib/bnn-library.h (60 lines, 2,612 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 *
21 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
22 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
23 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
24 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
25 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
26 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
27 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
28 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
29 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
30 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31
*****/
32
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giuliog@xilinx.com>
36 * Thomas B. Preusser <thomas.preusser@utexas.edu>
37 * Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 * Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * @file bnn-library.h
41 *
42 * Library of templated HLS functions for BNN deployment.
43 * Include this file in the network top funtion.
44 *
45
*****
46
47 #include <hls_stream.h>
48 #include <ap_int.h>
49 #include <iostream>
50 #include <string>
51

```

```

52 #include "weights.hpp"
53 #include "mmv.hpp"
54 #include "streamtools.h"
55 #include "dma.h"
56 #include "slidingwindow.h"
57 #include "maxpool.h"
58 #include "convlayer.h"
59 #include "vvau.hpp"
60 #include "upsample.hpp"

```

D.14 finn_hlslib/convlayer.h (311 lines, 16,354 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 *
21 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
22 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
23 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
24 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
25 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
26 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
27 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
28 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
29 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
30 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31
*****/
32
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giuliog@xilinx.com>
36 * Thomas B. Preusser <thomas.preusser@utexas.edu>
37 * Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 * Christoph Doebling <cdoehrin@xilinx.com>
39 * Timoteo Garcia Bertoa <timoteog@xilinx.com>
40 *
41 * \file convlayer.h
42 *
43 * Library of templated HLS functions for BNN deployment.
44 * This file lists a set of convenience functions used to
implement
45 * convolutional layers.
46 *
47
*****/
48
49 #ifndef CONV_LAYER_H
50 #define CONV_LAYER_H
51
52 #include <ap_int.h>
53 #include <hls_stream.h>
54
55 #include "streamtools.h"

```

```

56 #include "slidingwindow.h"
57 #include "mvau.hpp"
58 #include "tmccheck.hpp"
59
60 /**
61  * \brief      Convolutional layer implementation
62  *
63  * The function implements a generic convolutional layer, and
64  * it's basically composed of the sliding window generator
65  * implementing the im2col algorithm and the Mat-
66  * rix_Vector_Activate_Batch function to perform computation.
67  *
68  * \tparam ConvKernelDim      Dimension of the convolutional
69  * kernel (assumed square)
70  * \tparam IFMChannels        Number of Input Feature Maps
71  * \tparam IFMDim             Width and Height of
72  * the Input Feature Map (assumed square)
73  * \tparam OFMChannels        Number of Output Feature Maps
74  * \tparam OFMDim            Width and Height of
75  * the Output Feature Map (assumed square)
76  * \tparam SIMD              Number of input
77  * columns computed in parallel
78  * \tparam PE                Number of output rows
79  * computed in parallel
80  * \tparam TSrcI             DataType of the input
81  * activation (as used in the MAC)
82  *
83  * \tparam TDstI             DataType of the output
84  * activation (as generated by the activation)
85  * \tparam TWeightI          DataType of the weights (as
86  * used in the MAC)
87  * \tparam InStreamW         Width of the input stream
88  * \tparam OutStreamW        Width of the output stream
89  * \tparam TW                DataType of the
90  * weights matrix - safely deducible from the parameters
91  * \tparam TA                DataType of the activ-
92  * ation class (e.g. thresholds) - safely deducible from the parama-
93  * ters
94  * \tparam R                 DataType for the re-
95  * source used for FPGA implementation of the MAC - safely deducible
96  * from the parameters
97  * \param in                 Input stream
98  * \param out                Output stream
99  * \param weights            Weights matrix (cur-
100  * rently supports BinaryWeights or FixedPointWeights)
101  * \param activation         Activation class
102  * \param reps              Number of time the
103  * function has to be repeatedly executed (e.g. number of images)
104  * \param r                 Resource type for the
105  * hardware implementation of the MAC block
106  */
107 template<
108     unsigned int ConvKernelDim,
109     unsigned int IFMChannels,
110     unsigned int IFMDim,
111     unsigned int OFMChannels,
112     unsigned int OFMDim,
113     unsigned int SIMD, // number of SIMD lanes
114     unsigned int PE, // number of PEs
115     typename TSrcI = Identity, // redefine I/
116     // 0 interpretation as needed for input activations
117     typename TDstI = Identity, // re-
118     // define I/O interpretation as needed for output activations
119     typename TWeightI = Identity, // redefine I/
120     // 0 interpretation as needed for weights
121     int InStreamW, int OutStreamW, // safely de-
122     // ducible (stream width must be int though!)
123     typename TW, typename TA, typename R
124 >
125 void ConvLayer_Batch(hls::stream<ap_uint<InStreamW>> &in,
126                     hls::stream<ap_uint<OutStreamW>>
127                     &out,
128                     TW const &weights,
129                     TA const &activation,
130                     unsigned const reps,
131                     R const &r) {
132 #pragma HLS INLINE
133     unsigned const MatrixW = ConvKernelDim * ConvKernelDim *
134     IFMChannels;
135     unsigned const MatrixH = OFMChannels;
136     unsigned const InpPerImage = IFMDim * IFMDim * IFMChannels
137     * TSrcI::width / InStreamW;
138     hls::stream<ap_uint<SIMD*TSrcI::width>> wa_in("Streaming-
139     ConvLayer_Batch.wa_in");
140     hls::stream<ap_uint<SIMD*TSrcI::width>> con-
141     vInp("StreamingConvLayer_Batch.convInp");
142     hls::stream<ap_uint<PE*TDstI::width>>
143     mvOut("StreamingConvLayer_Batch.mvOut");
144     StreamingDataWidthConverter_Batch<InStreamW, SIMD*TS-
145     rcI::width, InpPerImage>(in, wa_in, reps);
146     ConvolutionInputGenerator<ConvKernelDim, IFMChannels, TS-

```

```

rcI::width, IFMDim,
122     OFMDim, SIMD,1>(wa_in, convInp, reps,
123     ap_resource_dflt());
124
125 Matrix_Vector_Activate_Batch<MatrixW, MatrixH, SIMD, PE, 1,
126 TSrcI, TDstI, TWeightI>
127 (stat-
128 ic_cast<hls::stream<ap_uint<SIMD*TSrcI::width>>&>(convInp),
129 static_cast<hls::stream<ap_uint<PE*TDstI::width>>&>
130 (mvOut),
131 weights, activation, reps* OFMDim * OFMDim, r);
132 StreamingDataWidthConverter_Batch<PE*TDstI::width, Out-
133 StreamW, OFMDim * OFMDim * (OFMChannels / PE)>(mvOut, out, reps);
134 }
135
136 /**
137  * \brief      Convolutional layer implementation with STMR
138  *
139  * The function implements a generic convolutional layer, and
140  * it's basically composed of the sliding window generator
141  * implementing the im2col algorithm and the Mat-
142  * rix_Vector_Activate_Batch function to perform computation. Addi-
143  * tionally,
144  * a TMR checker function performs error checks and outputs
145  * valid data.
146  *
147  * \tparam ConvKernelDim      Dimension of the convolutional
148  * kernel (assumed square)
149  * \tparam IFMChannels        Number of Input Feature Maps
150  * \tparam IFMDim             Width and Height of the Input
151  * Feature Map (assumed square)
152  * \tparam OFMChannels        Number of Output Feature Maps
153  * \tparam OFMDim            Width and Height of the Output
154  * Feature Map (assumed square)
155  * \tparam SIMD              Number of input columns computed
156  * in parallel
157  * \tparam PE                Number of output rows computed in
158  * parallel
159  * \tparam NUM_RED           Number of redundancies (or trip-
160  * licated channels)
161  * \tparam REDF              Redundancy factor (3 to triplic-
162  * ate)
163  * \tparam MAX_CH_WIDTH      Value to determine the precision
164  * of channel indexes
165  * \tparam TSrcI             DataType of the input activation
166  * (as used in the MAC)
167  *
168  * \tparam TDstI             DataType of the output activation
169  * (as generated by the activation)
170  * \tparam TWeightI          DataType of the weights (as used
171  * in the MAC)
172  * \tparam InStreamW         Width of the input stream
173  * \tparam OutStreamW        Width of the output stream
174  * \tparam TW                DataType of the weights matrix -
175  * safely deducible from the parameters
176  * \tparam TA                DataType of the activation class
177  * (e.g. thresholds) - safely deducible from the parameters
178  * \tparam R                 DataType for the resource used
179  * for FPGA implementation of the MAC - safely deducible from the
180  * parameters
181  * \param in                 Input stream
182  * \param out                Output stream
183  * \param weights            Weights matrix (currently sup-
184  * ports BinaryWeights or FixedPointWeights)
185  * \param activation         Activation class
186  * \param reps              Number of time the function has
187  * to be repeatedly executed (e.g. number of images)
188  * \param r                 Resource type for the hardware
189  * implementation of the MAC block
190  * \param errortype          Flag to inform redundancy check
191  * results. 0 if no faults, 1 if one PE is faulty, 2 if all differ
192  * \param channel_mask       Value with binary channel masks
193  * (1 if channel is triplicated, 0 otherwise)
194  * \param red_ch_index       Array of redundant triplets' in-
195  * dexes. Each position stores the first triplicated channel index of
196  * a triplet
197  */
198 template<
199     unsigned int ConvKernelDim,
200     unsigned int IFMChannels,
201     unsigned int IFMDim,
202     unsigned int OFMChannels,
203     unsigned int OFMDim,
204     unsigned int SIMD, // number of SIMD
205     unsigned int PE, // number of PEs
206     unsigned int NUM_RED, // number of redund-
207     // ant channels
208     unsigned int REDF, // redundancy factor
209     // (3 to triplicate)
210     unsigned int MAX_CH_WIDTH, // width to represent
211     channel indexes

```



```

181
182     typename TSrcI = Identity,      // redefine I/O in-
terpretation as needed for input activations
183     typename TDstI = Identity,      // redefine I/O in-
terpretation as needed for output activations
184     typename TweightI = Identity,   // redefine I/O in-
terpretation as needed for weights
185
186     int InStreamW, int OutStreamW,   // safely deducible
(stream width must be int though!)
187     typename TW,   typename TA,   typename R
188 >
189 void ConvLayer_Batch_TMR(hls::stream<ap_uint<InStreamW>>
&in,
190                          hls::stream<ap_uint<OutStreamW>>
&out,
191                          TW const &weights,
192                          TA const &activation,
193                          unsigned const reps,
194                          R const &r,
195                          ap_uint<2> &errortype,
196                          ap_uint<OFMChannels> channel_mask,
197                          ap_uint<MAX_CH_WIDTH> red_cha_in-
dex[NUM_RED]) {
198     #pragma HLS INLINE
199     unsigned const MatrixW = ConvKernelDim * ConvKernelDim *
IFMChannels;
200     unsigned const MatrixH = OFMChannels;
201     unsigned const InpPerImage = IFMDim*IFMDim;
202
203     hls::stream<ap_uint<SIMD*TSrcI::width> > wa_in("Streaming-
ConvLayer_Batch.wa_in");
204     hls::stream<ap_uint<SIMD*TSrcI::width> > con-
vInp("StreamingConvLayer_Batch.convInp");
205     hls::stream<ap_uint<PE*TDstI::width> >
mvOut("StreamingConvLayer_Batch.mvOut");
206     hls::stream<ap_uint<OFMChannels*TDstI::width> >
tmr_in("StreamingConvLayer_Batch.tmr_in");
207
208     StreamingDataWidthConverter_Batch<InStreamW, SIMD*TS-
rcI::width, InpPerImage>(in, wa_in, reps);
209
210     //Sliding window unit
211     ConvolutionInputGenerator<ConvKernelDim, IFMChannels, TS-
rcI::width, IFMDim,
212                             OFMDim, SIMD,1>(wa_in, convInp, reps, ap_re-
source_dflt());
213
214     //MVTU
215     Matrix_Vector_Activate_Batch<MatrixW, MatrixH, SIMD, PE, 1,
TSrcI, TDstI, TweightI>
216     (stat-
ic_cast<hls::stream<ap_uint<SIMD*TSrcI::width>>&>(convInp),
217      static_cast<hls::stream<ap_uint<PE*TDstI::width>>&>
(mvOut),
218       weights, activation, reps* OFMDim * OFMDim, r);
219
220     StreamingDataWidthConverter_Batch<PE*TDstI::width, OFMChan-
nels*TDstI::width, OFMDim * OFMDim * (OFMChannels / PE)>(mvOut,
tmr_in, reps);
221
222     //Error check
223     TMRCheck_Batch<TDstI::width, OFMChannels, NUM_RED, REDF,
OFMDim, MAX_CH_WIDTH>(tmr_in, out, errortype, channel_mask,
red_cha_index, reps);
224 }
225
226 /**
227  * brief      Convolutional layer implementation
228  *
229  * The function implements a generic convolutional layer, and
it's basically composed of the sliding window generator
230  * implementng the im2col algorithm and the Mat-
rix_Vector_Activate_Batch function to perform computation.
231  *
232  * \tparam ConvKernelDim      Dimension of the convolutional
kernel (assumed square)
233  * \tparam IFMChannels        Number of Input Feature Maps
234  * \tparam IFMDim            Width and Height of
the Input Feature Map (assumed square)
235  * \tparam OFMChannels        Number of Output Feature Maps
236  * \tparam OFMDim            Width and Height of
the Output Feature Map (assumed square)
237  * \tparam STRIDE            Stride of the convolu-
tional kernel
238  *
239  * \tparam SIMD              Number of input
columns computed in parallel
240  * \tparam PE                Number of output rows
computed in parallel
241  * \tparam MMV              Number of output
pixels computed in parallel
242  *
243  * \tparam TSrcI             DataType of the input
activation (as used in the MAC)
244  * \tparam TDstI            DataType of the output

```

```

activation (as generated by the activation)
245  * \tparam TweightI        DataType of the weights (as
used in the MAC)
246  * \tparam InStreamW       Width of the input stream
247  * \tparam OutStreamW      Width of the output stream
248  * \tparam TW              DataType of the
weights matrix - safely deducible from the paramaters
249  * \tparam TA              DataType of the activ-
ation class (e.g. thresholds) - safely deducible from the parama-
ters
250  * \tparam R               DataType for the re-
source used for FPGA implementation of the MAC - safely deducible
from the paramaters
251  *
252  * \param in               Input stream
253  * \param out              Output stream
254  * \param weights          Weights matrix (cur-
rently supports BinaryWeights or FixedPointWeights)
255  * \param activation       Activation class
256  * \param reps             Number of time the
function has to be repeatedly executed (e.g. number of images)
257  * \param r               Resource type for the
hardware implementation of the MAC block
258  */
259 template<
260     unsigned int ConvKernelDim,
261     unsigned int IFMChannels,
262     unsigned int IFMDim,
263     unsigned int OFMChannels,
264     unsigned int OFMDim,
265     unsigned int STRIDE,
266
267     unsigned int
SIMD, // number of SIMD lanes
268     unsigned int PE, //
number of PEs
269     unsigned int MMV,
270
271     typename TSrcI = Identity, // redefine I/
O interpretation as needed for input activations
272     typename TDstI = Identity, // re-
define I/O interpretation as needed for output activations
273     typename TweightI = Identity, // redefine I/
O interpretation as needed for weights
274
275     int InStreamW, int OutStreamW, // safely de-
ducible (stream width must be int though!)
276     typename TW,   typename TA,   typename R
277 >
278 void ConvLayer_Batch_MMV(hls::stream<ap_uint<InStreamW>>
&in,
279                          hls::stream<ap_uint<OutStreamW>>
&out,
280                          TW const &weights,
281                          TA const &activation,
282                          unsigned const reps,
283                          R const &r) {
284     #pragma HLS INLINE
285     unsigned const MatrixW = ConvKernelDim * ConvKernelDim *
IFMChannels;
286     unsigned const MatrixH = OFMChannels;
287     unsigned const InpPerImage = IFMDim*IFMDim*IFMChannels *
TSrcI::width/InStreamW;
288     const unsigned int mmvReps = (reps * OFMDim * OFMDim) /
MMV;
289
290     hls::stream<ap_uint<SIMD * TSrcI::width> > wa_in("Stream-
ingConvLayerMMV_Batch.wa_in");
291     hls::stream<MultiChanData<MMV, SIMD * TSrcI::width> > con-
vInp("StreamingConvLayerMMV_Batch.convInp");
292     hls::stream<MultiChanData<MMV, PE * TDstI::width> > mmv2d-
wc("StreamingConvLayerMMV_Batch.mmv2dwc");
293     hls::stream<MultiChanData<MMV, OFMChannels * TDstI::width>>
dwc2flat("dwc2flat");
294     hls::stream<ap_uint<MMV * OFMChannels * TDstI::width> >
mvOut("StreamingConvLayerMMV_Batch.mvOut");
295
296     StreamingDataWidthConverter_Batch<InStreamW, SIMD * TS-
rcI::width, InpPerImage>(in, wa_in, reps);
297
298     ConvolutionInputGenerator_MMV<ConvKernelDim, IFMChannels,
TSrcI::width, IFMDim,
299                                OFMDim, SIMD, STRIDE, MMV>(wa_in, con-
vInp, reps, ap_resource_dflt());
300     Matrix_Vector_Activate_Batch<MatrixW, MatrixH, SIMD, PE,
MMV, TSrcI, TDstI, TweightI>
301     (stat-
ic_cast<hls::stream<MultiChanData<MMV, SIMD*TSrcI::width>>&>(convInp
),
302      stat-
ic_cast<hls::stream<MultiChanData<MMV, PE*TDstI::width>>&>(mmv2dwc),
303       weights, activation, mmvReps, r);
304
305     MultiChanDataWidthConverter_Batch<PE * TDstI::width,
OFMChannels * TDstI::width, OFMDim * OFMDim * (OFMChannels / PE),
MMV>(mmv2dwc, dwc2flat, reps);
306     FlattenMultiChanData<MMV, OFMChannels *
TDstI::width>(dwc2flat, mvOut, mmvReps);
307     StreamingDataWidthConverter_Batch<MMV * OFMChannels *

```

```
TDstI::width, OutStreamW, OFMDim * OFMDim / MMV>(mvOut, out, reps);
308
309 }
310
311 #endif
```

D.15 finn_hlslib/dma.h (238 lines, 9,463 B)

```
1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 *
21 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
22 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
23 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
24 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
25 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
26 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
27 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
28 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
29 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
30 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31
*****/
32
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 * Thomas B. Preusser <thomas.preusser@utexas.edu>
37 * Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 * Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * \file dma.h
41 *
42 * Library of templated HLS functions for BNN deployment.
43 * This file lists a set of functions to access memory
mapped values into
44 * streams.
45 *
46
*****/
47 #ifndef DMA_HPP
48 #define DMA_HPP
49
50
51 #include <ap_int.h>
52 #include <hls_stream.h>
53
54 /*!
55 * \brief DMA block accessing AXI4 memory and output HLS
streams
56 *
57 *
58 * \tparam DataWidth Width, in number of bits, of the AXI4
```

```
memory pointer and the output HLS stream
59 * \tparam numBytes Number of bytes to be read from the
memory
60 *
61 * \param in Input memory pointer
62 * \param out Output HLS stream
63 */
64 template<unsigned int DataWidth, unsigned int numBytes>
65 void Mem2Stream(ap_uint<DataWidth> * in,
hls::stream<ap_uint<DataWidth> > & out);
66
67 /*!
68 * \brief DMA block accessing AXI4 memory and output HLS
streams multiple times
69 *
70 * It basically calls Mem2Stream function multiple times,
possibly with bigger sizes so to increase
71 * the burst size
72 *
73 * \tparam DataWidth Width, in number of bits, of the AXI4
memory pointer and the output HLS stream
74 * \tparam numBytes Number of bytes to be read from the
memory
75 *
76 * \param in Input memory pointer
77 * \param out Output HLS stream
78 *
79 * \param numReps Number of times the Stream2Mem function has to
be called
80 */
81 template<unsigned int DataWidth, unsigned int numBytes>
82 void Mem2Stream_Batch(ap_uint<DataWidth> * in,
hls::stream<ap_uint<DataWidth> > & out, const unsigned int num-
Reps);
83
84 /*!
85 * \brief DMA block writing HLS streams content in AXI4 poin-
ted memory
86 *
87 * \tparam DataWidth Width, in number of bits, of the AXI4
memory pointer and the output HLS stream
88 * \tparam numBytes Number of bytes to be read from the
memory
89 *
90 * \param in Input HLS stream
91 * \param out Output memory pointer
92 */
93 template<unsigned int DataWidth, unsigned int numBytes>
94 void Stream2Mem(hls::stream<ap_uint<DataWidth> > & in,
ap_uint<DataWidth> * out);
95
96 /*!
97 * \brief DMA block that accesses the external memory and
outputs HLS streams multiple times
98 *
99 * It basically calls Mem2Stream function multiple times,
possibly with bigger sizes so to increase
100 * the burst size
101 *
102 * \tparam DataWidth Width, in number of bits, of the AXI4
memory pointer and the output HLS stream
103 * \tparam numBytes Number of bytes to be read from the
memory
104 *
105 * \param in Input pointer to external memory
106 * \param out Output the generated HLS stream
107 *
108 * \param numReps Number of times the Mem2Stream function has
to be called
109 */
110 template<unsigned int DataWidth, unsigned int numBytes>
111 void Mem2Stream_Batch_external_wmem(ap_uint<DataWidth> * in,
hls::stream<ap_uint<DataWidth> > & out, const un-
signed int numReps) {
112     unsigned int rep = 0;
113     while (rep != numReps) {
114         Mem2Stream<DataWidth, numBytes>(&in[0], out);
115         rep += 1;
116     }
117 }
118
119 /*!
120 * \brief DMA block writing HLS streams content in AXI4 poin-
ted memory multiple times
121 *
122 * It basically calls Stream2Mem function multiple times,
possibly with bigger sizes so to increase
123 * the burst size
124 *
125 * \tparam DataWidth Width, in number of bits, of the AXI4
memory pointer and the output HLS stream
126 * \tparam numBytes Number of bytes to be read from the
memory
127 *
128 * \param in Input HLS stream
129 * \param out Output memory pointer
130 *
131 * \param numReps Number of times the Stream2Mem function has
to be called
```

```

131 */
132 template<unsigned int DataWidth, unsigned int numBytes>
133 void Stream2Mem_Batch(hls::stream<ap_uint<DataWidth> > & in,
ap_uint<DataWidth> * out, const unsigned int numReps);
134
135 template<unsigned int DataWidth, unsigned int numBytes>
136 void Mem2Stream(ap_uint<DataWidth> * in,
hls::stream<ap_uint<DataWidth> > & out) {
137     static_assert(DataWidth % 8 == 0, "");
138     const unsigned int numWords = numBytes / (DataWidth / 8);
139     static_assert(numWords != 0, "");
140     for (unsigned int i = 0; i < numWords; i++) {
141         #pragma HLS pipeline style=flp II=1
142         ap_uint<DataWidth> e = in[i];
143         out.write(e);
144     }
145 }
146
147
148 template<unsigned int DataWidth, unsigned int numBytes>
149 void Stream2Mem(hls::stream<ap_uint<DataWidth> > & in,
ap_uint<DataWidth> * out) {
150     static_assert(DataWidth % 8 == 0, "");
151     const unsigned int numWords = numBytes / (DataWidth / 8);
152     static_assert(numWords != 0, "");
153     for (unsigned int i = 0; i < numWords; i++) {
154         #pragma HLS pipeline style=flp II=1
155         ap_uint<DataWidth> e = in.read();
156         out[i] = e;
157     }
158 }
159
160 template<unsigned int DataWidth, unsigned int numBytes>
161 void Mem2Stream_Batch(ap_uint<DataWidth> * in,
hls::stream<ap_uint<DataWidth> > & out, const unsigned int numReps)
{
162     const unsigned int indsPerRep = numBytes / (DataWidth / 8);
163     unsigned int rep = 0;
164     // make sure Mem2Stream does not get inlined here
165     // we lose burst inference otherwise
166     while (rep != numReps) {
167         unsigned int repsLeft = numReps - rep;
168         if ((repsLeft & 0xF) == 0) {
169             // repsLeft divisible by 16, read 16 images
170             Mem2Stream<DataWidth, numBytes * 16>(&in[rep * ind-
sPerRep], out);
171             rep += 16;
172         } else {
173             // fallback, read single image
174             Mem2Stream<DataWidth, numBytes>(&in[rep * indsPerRep],
out);
175             rep += 1;
176         }
177     }
178 }
179
180
181 template<unsigned int DataWidth, unsigned int numBytes>
182 void Stream2Mem_Batch(hls::stream<ap_uint<DataWidth> > & in,
ap_uint<DataWidth> * out, const unsigned int numReps) {
183     const unsigned int indsPerRep = numBytes / (DataWidth / 8);
184     unsigned int rep = 0;
185     // make sure Stream2Mem does not get inlined here
186     // we lose burst inference otherwise
187     while (rep != numReps) {
188         unsigned int repsLeft = numReps - rep;
189         if ((repsLeft & 0xF) == 0) {
190             // repsLeft divisible by 16, write 16 images
191             Stream2Mem<DataWidth, numBytes * 16>(&in[rep *
indsPerRep]);
192             rep += 16;
193         } else {
194             // fallback, write single image
195             Stream2Mem<DataWidth, numBytes>(&in[rep * ind-
sPerRep]);
196             rep += 1;
197         }
198     }
199 }
200
201 /*!
202 * \brief Streaming block that fetches parameters from in-
ternal memory and presents them to the MVAU
203 *
204 * \tparam TILES          Total folding factor of the neural
network mapped on the fabric (Neuron Fold * Synapse Fold)
205 * \tparam SIMD           Number of input columns computed in
parallel
206 * \tparam PE             Number of output rows computed in
parallel
207 * \tparam WP             Precision of the weights in the
network
208 * \tparam TW             DataType of the weights matrix -
safely deducible from the paramaters
209 *
210 * \param W_in            Pointer to the weight memory
211 * \param paramStreamOut  Parameter stream that contains SIMD *
PE * WP long words to digest by the MVAU

```

```

212 * \param numReps        Number of time the function has to
be repeatedly executed (e.g. number of images)
213 */
214 template<
215     unsigned int TILES,
216     unsigned int SIMD,
217     unsigned int PE,
218     unsigned int WP,
219     typename TW
220 >
221 void GenParamStream(TW const &W_in, hls::stream<ap_uint<SIMD
* PE * WP>> &paramStreamOut, int const numReps) {
222     for (unsigned rep = 0; rep < numReps; rep++) {
223         for (unsigned tile = 0; tile < TILES; tile++) {
224             #pragma HLS pipeline style=flp II=1
225
226             ap_uint<SIMD * PE * WP> strMem;
227             for (unsigned pe = 0; pe < PE; pe++) {
228                 // Concatenate the weights within the tile into a
large SIMD * PE * WP wide word
229                 // Using Little Endian PE order
230                 strMem((SIMD * WP)*(pe+1)-1, (SIMD * WP)*pe) =
W_in.m_weights[pe][tile];
231             }
232
233             paramStreamOut.write(strMem);
234         }
235     }
236 }
237
238 #endif

```

D.16 finn_hlslib/interpret.hpp (307 lines, 8,530 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 *
21 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
22 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
23 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
24 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
25 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
26 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
27 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
28 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
29 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE,
30 * EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31
*****/
32
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio.g@xilinx.com>
36 *          Thomas B. Preusser <thomas.preusser@utexas.edu>
37 *          Marie-Curie Fellow, Xilinx Ireland, Grant

```



```

Agreement No. 751339
38 *      Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * \file interpret.hpp
41 *
42 * This project has received funding from the European Uni-
on's Framework
43 * Programme for Research and Innovation Horizon 2020
(2014-2020) under
44 * the Marie Skłodowska-Curie Grant Agreement No. 751339.
45 *
46
*****
47
48 #ifndef INTERPRET_HPP
49 #define INTERPRET_HPP
50
51 #include <ap_int.h>
52 #include <ostream>
53
54 /**
55  * Thin wrapper around an ap_uint<1> redefining multiplica-
tion with
56  * another ap_uint<1> as XNOR operation for use in XNOR net-
works.
57  */
58 class XnorMul {
59     ap_uint<1> const m_val;
60 public:
61     XnorMul(ap_uint<1> const val) : m_val(val) {
62 #pragma HLS inline
63     }
64
65 public:
66     int operator*(ap_uint<1> const &b) const {
67 #pragma HLS inline
68         return m_val == b? 1 : 0;
69     }
70 };
71 inline int operator*(ap_uint<1> const &a, XnorMul const &b) {
72 #pragma HLS inline
73     return b*a;
74 }
75
76 class Binary {
77 public:
78     ap_uint<1> const m_val;
79     explicit Binary(ap_uint<1> const val) : m_val(val) {
80 #pragma HLS inline
81     }
82
83 public:
84     operator ap_int<2>() const {
85         return ap_int<2>(m_val? 1 : -1);
86     }
87     template<typename T>
88     auto operator*(T const &b) const -> decl-
type(ap_int<2>(1)*b) {
89 #pragma HLS inline
90         return m_val? static_cast<decltype(-b)>(b) : -b;
91     }
92     friend std::ostream& operator<<(std::ostream&, Binary
const&);
93 };
94
95 template<typename T>
96 inline int operator*(T const &a, Binary const &b) {
97 #pragma HLS inline
98     return b*a;
99 }
100
101 inline int operator*(Binary const &a, Binary const &b) {
102 #pragma HLS inline
103     return (ap_int<2>) b* (ap_int<2>)a;
104 }
105
106 inline std::ostream& operator<<(std::ostream &out, Binary
const &b) {
107     out << (b.m_val? "1" : "-1");
108     return out;
109 }
110
111 struct Identity {
112     static unsigned const width = 1;
113
114     template<typename T>
115     T const &operator()(T const &v) const {
116 #pragma HLS inline
117         return v;
118     }
119
120     template<typename T>
121     T operator()() const {
122 #pragma HLS inline
123         return T();
124     }
125 };
126
127 template<typename T>

```

```

128 class Recast {
129 public:
130     static unsigned const width = 1;
131
132 private:
133     template<typename TV>
134     class Container {
135         TV m_val;
136     public:
137         Container(TV const &val) : m_val(val) {
138 #pragma HLS inline
139         }
140
141     public:
142         T operator[](unsigned const idx) const {
143 #pragma HLS inline
144             return T(m_val[idx]);
145         }
146         auto operator[](unsigned const idx) -> decl-
type(m_val[idx]) {
147 #pragma HLS inline
148             return m_val[idx];
149         }
150         T operator()(unsigned const idx, __attribute__((unused))
unsigned const mmv) const {
151 #pragma HLS inline
152             return T(m_val[idx]);
153         }
154         operator TV const&() const {
155 #pragma HLS inline
156             return m_val;
157         }
158     };
159
160 public:
161     template<typename TV>
162     Container<TV> operator()(TV const &val) const {
163 #pragma HLS inline
164         return Container<TV>(val);
165     }
166     template<typename TV>
167     Container<TV> operator()(TV const &val, __attrib-
ute__((unused)) unsigned const mmv) const {
168 #pragma HLS inline
169         return Container<TV>(val);
170     }
171     template<typename TV>
172     Container<TV> operator()() const {
173 #pragma HLS inline
174         return Container<TV>();
175     }
176 };
177
178 template<typename T>
179 struct Caster {
180     template<int M>
181     static T cast(ap_int<M> const &arg) { return
T(arg); }
182 };
183
184 template<int W, int I, ap_q_mode Q, ap_o_mode O, int N>
185 struct Caster<ap_fixed<W, I, Q, O, N>> {
186     template<int M>
187     static ap_fixed<W, I, Q, O, N> cast(ap_int<M> const &arg) {
188         return ap_fixed<W, I, Q, O, N>(arg);
189     }
190 };
191
192 template<typename T, unsigned STRIDE=T::width>
193 class Slice {
194 public:
195     static unsigned const width = STRIDE;
196
197 private:
198     template<typename TV>
199     class Container {
200         TV m_val;
201
202     public:
203         Container() {
204 #pragma HLS inline
205         }
206         Container(TV const &val) : m_val(val) {
207 #pragma HLS inline
208         }
209     public:
210         auto operator()(unsigned const idx, __attrib-
ute__((unused)) unsigned const mmv, __attribute__((unused)) bool
const flag) const -> decltype(m_val(STRIDE, 0)) {
211 #pragma HLS inline
212             return m_val((idx+1)*STRIDE-1, idx*STRIDE);
213         }
214         T operator()(unsigned const idx, __attribute__((unused))
unsigned const mmv) const {
215 #pragma HLS inline
216             ap_uint<STRIDE> const r = m_val((idx+1)*STRIDE-1,
idx*STRIDE);
217             return Caster<T>::cast(ap_int<STRIDE>(r));
218         }
219         auto operator[](__attribute__((unused)) unsigned mmv)

```

```

const -> decltype(m_val) {
220 #pragma HLS inline
221     return m_val;
222 }
223 operator TV const&() const {
224 #pragma HLS inline
225     return m_val;
226 }
227 };
228
229 public:
230 template<typename TV>
231 Container<TV> operator()(TV const &val) const {
232 #pragma HLS inline
233     return Container<TV>(val);
234 }
235 template<typename TV>
236 Container<TV> operator()() const {
237 #pragma HLS inline
238     return Container<TV>();
239 }
240 template<typename TV>
241 Container<TV> operator() (TV const &val, __attrib-
ute__((unused)) unsigned mmv) const {
242 #pragma HLS inline
243     return Container<TV>(val);
244 }
245 };
246
247
248 // This class is done for Slicing an MMV container (vector of
ap_uint)
249 template<typename T, unsigned MMV, unsigned STRIDE=T::width>
250 class Slice_mmv {
251 public:
252     static unsigned const width = STRIDE;
253
254 private:
255     template<typename TV>
256     class Container {
257     TV m_val;
258
259     public:
260     Container() {
261 #pragma HLS inline
262     }
263
264     Container(TV const &val, __attribute__((unused)) unsigned
mmv) : m_val(val){
265 #pragma HLS inline
266     }
267     Container(TV const &val) : m_val(val){
268 #pragma HLS inline
269     }
270     public:
271     operator TV const&() const {
272 #pragma HLS inline
273     return m_val;
274     };
275     auto operator()(unsigned const idx, unsigned const mmv,
__attribute__((unused)) bool const flag) const -> decl-
type(m_val.data[mmv](STRIDE, 0)) {
276 #pragma HLS inline
277     return m_val.data[mmv]((idx+1)*STRIDE-1, idx*STRIDE);
278     };
279     T operator()(unsigned const idx, unsigned const mmv)
const {
280 #pragma HLS inline
281     ap_uint<STRIDE> const r = m_val.data[mmv]
((idx+1)*STRIDE-1, idx*STRIDE);
282     return Caster<T>::cast(ap_int<STRIDE>(r));
283     };
284     auto operator[](unsigned const mmv) const -> decl-
type(m_val.data[mmv]) {
285 #pragma HLS inline
286     return m_val.data[mmv];
287     };
288
289 public:
290 template<typename TV>
291 Container<TV> operator()(TV const &val) const {
292 #pragma HLS inline
293     return Container<TV>(val);
294 }
295 template<typename TV>
296 Container<TV> operator()() const {
297 #pragma HLS inline
298     return Container<TV>();
299 }
300 template<typename TV>
301 Container<TV> operator()(TV const &val, __attrib-
ute__((unused)) unsigned mmv) {
302 #pragma HLS inline
303     return Container<TV>(val);
304 }
305 };
306
307 #endif

```

D.17 finn_hlslib/mac.hpp (206 lines, 8,380 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31
*****/
32
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 *          Thomas B. Preusser <thomas.preusser@utexas.edu>
37 *          Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 *          Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * \file mac.hpp
41 *
42 * Library of templated HLS functions for BNN deployment.
43 * This file lists a set of convenience functions used to im-
plement
44 * multipliers with selectable implementation resource
45 *
46 * This project has received funding from the European Uni-
on's Framework
47 * Programme for Research and Innovation Horizon 2020
(2014-2020) under
48 * the Marie Skłodowska-Curie Grant Agreement No. 751339.
49 *
50
*****/
51
52 /
*****
53 * MAC operation template:
54 *
55 * mac<N, T, TC, TD>(T a, TC c[N], TD d[N])
56 *     = a + SUM_{i=0}^{N-1} c(i)*d(i)
57 *
58 * All template arguments but N can typically be inferred.
59 *
60 * mac<ap_uint<14>>(0, c, d)

```

```

61
*****
*****/
62
63 #ifndef MAC_HPP
64 #define MAC_HPP
65
66 #include "utils.hpp"
67
68
69 /**
70  * \brief      Multiplier operation between 2 operands, HLS
choose the best resource
71  *
72  * The same multiply operation can be implemented using mul-
multiple Vivado HLS pragmas to select the
73  * hardware resource to be used:
74  * ap_resource_dflt will let HLS choose the best one
75
76 * ap_resource_lut will force HLS to implement the multiplier in
LUTs
77
78 * ap_resource_dsp will force HLS to implement the multiplier in
DSP48
79
80 *
81 * \tparam      TC      First operand datatype (weights)
82 * \tparam      TD      Second operand datatype (input)
83 *
84 * \param      c      First operand (array of weights)
85 * \param      d      Second operand (array of input activa-
tion)
86 * \param      r      Resource type for the hardware imple-
mentation of the MAC block
87 *
88 * \return      Result of the multiply operation
89 */
90 template<typename TC, typename TD>
91 auto mul(TC const &c, TD const &d, ap_resource_dflt const&) -
> decltype(c*d) {
92     #pragma HLS inline
93     auto r = c*d;
94     return r;
95 }
96
97 /**
98  * \brief      Multiplier operation between 2 operands, imple-
mented in LUT
99  *
100 * The same multiply operation can be implemented using mul-
multiple Vivado HLS pragmas to select the
101 * hardware resource to be used:
102 * ap_resource_dflt will let HLS choose the best one
103
104 * ap_resource_lut will force HLS to implement the multiplier
in LUTs
105
106 * ap_resource_dsp will force HLS to implement the multiplier
in DSP48
107
108 *
109 * \tparam      TC      First operand datatype (weights)
110 * \tparam      TD      Second operand datatype (input)
111 *
112 * \param      c      First operand (array of weights)
113 * \param      d      Second operand (array of input activa-
tion)
114 * \param      r      Resource type for the hardware imple-
mentation of the MAC block
115 *
116 * \return      Result of the multiply operation
117 */
118 template<typename TC, typename TD>
119 auto mul(TC const &c, TD const &d, ap_resource_lut const&) -
> decltype(c*d) {
120     #pragma HLS inline
121     decltype(c*d) const res = c*d;
122     #pragma HLS BIND_OP variable=res op=mul impl=fabric
123     return res;
124 }
125
126 /**
127  * \brief      Multiplier operation between 2 operands, imple-
mented in a DSP48
128  *
129 * The same multiply operation can be implemented using mul-
multiple Vivado HLS pragmas to select the
130 * hardware resource to be used:
131 * ap_resource_dflt will let HLS choose the best one
132
133 * ap_resource_lut will force HLS to implement the multiplier
in LUTs
134
135 * ap_resource_dsp will force HLS to implement the multiplier
in DSP48
136
137 *
138 * \tparam      TC      First operand datatype (weights)
139 * \tparam      TD      Second operand datatype (input)
140 *
141 * \param      c      First operand (array of weights)
142 * \param      d      Second operand (array of input activa-

```

```

tion)
143 * \param      r      Resource type for the hardware imple-
mentation of the MAC block
144 *
145 * \return      Result of the multiply operation
146 */
147 template<typename TC, typename TD>
148
149 auto mul(TC const &c, TD const &d, ap_resource_dsp const&) -
> decltype(c*d) {
150     #pragma HLS inline
151     decltype(c*d) const res = c*d;
152     #pragma HLS BIND_OP variable=res op=mul impl=dsp
153     return res;
154 }
155
156 /**
157  * \brief      MAC with selectable implementation resource,
used by Matrix_Vector_Activate_Batch
158  *
159 *
160 * \tparam      N      Number of MAC to be performed (equals to
SIMD in mvau)
161 * \tparam      T      Accumulator datatype
162 * \tparam      TC      First operand datatype (weights)
163 * \tparam      TD      Second operand datatype (input)
164 * \tparam      R      Datatype for the resource used for FPGA
implementation of the MAC - safely deducible from the parameters
165 *
166 * \param      a      Initialization value of the accumulation
167 * \param      c      First operand (array of weights)
168 * \param      d      Second operand (array of input activa-
tion)
169 * \param      r      Resource type for the hardware imple-
mentation of the MAC block
170 * \param      mmv      MMV value to address accumulator and ac-
tivation
171 *
172 * \return      Result of the MAC operation
173 */
174 template<unsigned N, typename T, typename TC, typename TD,
typename R>
175 T mac(T const &a, TC const &c, TD const &d, R const &r, un-
signed mmv) {
176     #pragma HLS inline
177     T res = a;
178     for(unsigned i = 0; i < N; i++) {
179         #pragma HLS unroll
180         res += mul(c[i], d[i], mmv, r);
181     }
182     return res;
183 }
184
185 /**
186  * \brief      MAC with selectable implementation resource,
used by Matrix_Vector_Activate_Batch
187  *
188 *
189 * \tparam      N      Number of MAC to be performed (equals to
SIMD in mvau)
190 * \tparam      T      Accumulator datatype
191 * \tparam      TC      First operand datatype (weights)
192 * \tparam      TD      Second operand datatype (input)
193 * \tparam      R      Datatype for the resource used for FPGA
implementation of the MAC - safely deducible from the parameters
194 *
195 * \param      a      Initialization value of the accumulation
196 * \param      c      First operand (array of weights)
197 * \param      d      Second operand (array of input activa-
tion)
198 * \param      r      Resource type for the hardware imple-
mentation of the MAC block
199 *
200 * \return      Result of the MAC operation
201 */
202 template<unsigned N, typename T, typename TC, typename TD,
typename R>
203 T mac(T const &a, TC const &c, TD const &d, R const &r) {
204     #pragma HLS inline
205     T res = a;
206     for(unsigned i = 0; i < N; i++) {
207         #pragma HLS unroll
208         res += mul(c[i], d[i], r);
209     }
210     return res;
211 }
212
213 template<unsigned N, typename T, typename TC, typename TD>
214 inline T mac(T const &a, TC const &c, TD const &d) {
215     #pragma HLS inline
216     return mac<N>(a, c, d, ap_resource_dflt());
217 }
218
219 #endif

```

D.18 finn_hlslib/maxpool.h (609 lines, 23,874 B)

```

1 /
*****
*****
2 * Copyright (c) 2019, Xilinx, Inc.

```

```

3  * All rights reserved.
4  *
5  * Redistribution and use in source and binary forms, with
or without
6  * modification, are permitted provided that the following
conditions are met:
7  *
8  * 1. Redistributions of source code must retain the above
copyright notice,
9  * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 *
21 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
22 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
23 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
24 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
25 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
26 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
27 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
28 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
29 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
30 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31 *
32 *
33 *
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 * Thomas B. Preusser <thomas.preusser@utexas.edu>
37 * Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 * Christoph Doebling <cdoehrin@xilinx.com>
39 * Felix Jentsch <felixj@xilinx.com>
40 *
41 *
42 * Library of templated HLS functions for QNN deployment.
43 *
44 *
45 *
46 *
47 *
48 *
49 *
50 *
51 *
52 *
53 *
54 *
55 * \brief Max Pool implementation for Binarized values
56 *
57 * This function performs the maxpool for binary inputs, and
works with kernel and stride being equal
58 *
59 * \tparam ImgDim Width and Height of the Input
Feature Map (assumed square)
60 * \tparam PoolDim Dimension of the Max Pool kernel
(assumed square)
61 * \tparam NumChannels Number of Input Feature Maps
62 *
63 * \param in Input stream
64 * \param out Output stream
65 *
66 *
67 template<unsigned int ImgDim, unsigned int PoolDim, unsigned
int NumChannels>

```

```

68 void StreamingMaxPool(hls::stream<ap_uint<NumChannels>> &
in,
69 hls::stream<ap_uint<NumChannels>> & out) {
70 static_assert(ImgDim % PoolDim == 0, "");
71 // need buffer space for a single maxpooled row of the im-
age
72 ap_uint<NumChannels> buf[ImgDim / PoolDim];
73 for(unsigned int i = 0; i < ImgDim / PoolDim; i++) {
74 #pragma HLS UNROLL
75 buf[i] = 0;
76 }
77
78 for (unsigned int yp = 0; yp < ImgDim / PoolDim; yp++) {
79 for (unsigned int ky = 0; ky < PoolDim; ky++) {
80 for (unsigned int xp = 0; xp < ImgDim / PoolDim; xp++)
{
81 ap_uint<NumChannels> acc = 0;
82 for (unsigned int kx = 0; kx < PoolDim; kx++) {
83 #pragma HLS pipeline style=fpl II=1
84 acc = acc | in.read();
85 }
86 // pool with old value in row buffer
87 buf[xp] |= acc;
88 }
89 }
90 for (unsigned int outpix = 0; outpix < ImgDim / PoolDim;
outpix++) {
91 #pragma HLS pipeline style=fpl II=1
92 out.write(buf[outpix]);
93 // get buffer ready for next use
94 buf[outpix] = 0;
95 }
96 }
97 }
98 /**
99 * \brief Max Pool implementation for Binarized values on
multiple images
100 *
101 *
102 * This function performs the maxpool for binary inputs, and
works with kernel and stride being equal
103 *
104 * \tparam ImgDim Width and Height of the Input Feature
Map (assumed square)
105 * \tparam PoolDim Dimension of the Max Pool kernel (as-
sumed square)
106 * \tparam NumChannels Number of Input Feature Maps
107 *
108 * \param in Input stream
109 * \param out Output stream
110 *
111 * \param numReps Number of time the function has to be
repeatedly executed (e.g. number of images)
112 */
113 template<unsigned int ImgDim, unsigned int PoolDim, unsigned
int NumChannels>
114 void StreamingMaxPool_Batch(hls::stream<ap_uint<NumChannels>>
& in,
115 hls::stream<ap_uint<NumChannels>> & out, unsigned
int numReps) {
116 for (unsigned int rep = 0; rep < numReps; rep++) {
117 StreamingMaxPool<ImgDim, PoolDim, NumChannels>(in, out);
118 }
119 }
120
121 /**
122 * \brief Max Pool implementation for non Binarized values
123 *
124 *
125 * This function performs the maxpool for non-binary inputs,
and works with kernel and stride being equal
126 *
127 * \tparam ImgDim Width and Height of the Input Feature
Map (assumed square)
128 * \tparam PoolDim Dimension of the Max Pool kernel (as-
sumed square)
129 * \tparam NumChannels Number of Input Feature Maps
130 * \tparam ActType DataType of the input activation (as
used in the comparison)
131 *
132 * \tparam min_value Minimum value possible with the given
ActType, used to initialize the value before the comparison
133 * \tparam StreamW Width of the input and output stream
134 *
135 * \param in Input stream
136 * \param out Output stream
137 */
138 template<unsigned int ImgDim, unsigned int PoolDim, unsigned
int NumChannels, typename ActType, int min_value,
139 int StreamW>
140 void StreamingMaxPool_Precision(hls::stream<ap_uint<StreamW>>
& in,
141 hls::stream<ap_uint<StreamW>> & out) {

```

```

143 static_assert(ImgDim % PoolDim == 0, "");
144 // need buffer space for a single maxpooled row of the im-
age
145 ActType buf[ImgDim / PoolDim][NumChannels];
146 #pragma HLS ARRAY_PARTITION variable=buf complete dim=2
147 for(unsigned int i = 0; i < ImgDim / PoolDim; i++) {
148     for(unsigned int ch = 0; ch < NumChannels; ch++){
149         #pragma HLS UNROLL
150         buf[i][ch] = min_value; //std::nume-
ic_limits<ActType>::min();
151     }
152 }
153 ap_uint<StreamW> inputData,outputData;
154 for (unsigned int yp = 0; yp < ImgDim / PoolDim; yp++) {
155     for (unsigned int ky = 0; ky < PoolDim; ky++) {
156         for (unsigned int xp = 0; xp < ImgDim / PoolDim; xp++)
        {
157             // Change to comparator
158             for (unsigned int kx = 0; kx < PoolDim; kx++) {
159                 #pragma HLS pipeline style=fllp II=1
160                 inputData = in.read();
161                 for(unsigned int ch = 0; ch<NumChannels; ch++){
162                     #pragma HLS UNROLL
163                     unsigned int lowBit = ch * ActType::width;
164                     unsigned int highBit = (ch+1) * ActType::width
-1;
165                     ActType channeldata = inputData(highBit, low-
Bit);
166                     ActType oldMax = buf[xp][ch];
167                     if(channeldata > oldMax){
168                         buf[xp][ch] = channeldata;
169                     }
170                 }
171             }
172         }
173     }
174     for (unsigned int outpix = 0; outpix < ImgDim / PoolDim;
outpix++) {
175         for(unsigned int ch = 0; ch < NumChannels; ch++){
176             #pragma HLS UNROLL
177             unsigned int lowBit = ch * ActType::width;
178             unsigned int highBit = (ch+1) * ActType::width -1;
179             outputData(highBit, lowBit) = buf[outpix][ch];
180             // get buffer ready for next use
181             buf[outpix][ch] = min_value;
182         }
183         out.write(outputData);
184     }
185 }
186 }
187 /**
188 * \brief Max Pool implementation for non binarized values
on multiple images
190 *
191 * This function performs the maxpool for non binary inputs,
and works with kernel and stride being equal
192 *
193 * \tparam ImgDim Width and Height of the Input Feature
Map (assumed square)
194 * \tparam PoolDim Dimension of the Max Pool kernel (as-
sumed square)
195 * \tparam NumChannels Number of Input Feature Maps
196 * \tparam ActType DataType of the input activation (as
used in the comparison)
197 * \tparam min_value Minimum value possible with the given
ActType, used to initialize the value before the comparison
198 * \tparam StreamW Width of the input and output stream
199 * \param in Input stream
200 * \param out Output stream
201
202 * \param numReps Number of time the function has to be
repeatedly executed (e.g. number of images)
203 *
204 */
205 template<unsigned int ImgDim, unsigned int PoolDim, unsigned
int NumChannels, typename ActType, int min_value,
206 int InStreamW, int OutStreamW // safely deducible
(stream width must be int though!)
207 >
208 void StreamingMax-
Pool_Precision_Batch(hls::stream<ap_uint<InStreamW>> & in,
209 numReps) {
210     #pragma HLS INLINE
211     unsigned const InpPerImage = Img-
Dim*ImgDim*NumChannels*ActType::width/InStreamW ;
212     unsigned const OutPerImage = ImgDim*ImgDim /
(PoolDim*PoolDim);
213     hls::stream<ap_uint<NumChannels*ActType::width> >
wa_in("StreamingMaxPool_Precision_Batch.wa_in");
214     hls::stream<ap_uint<NumChannels*ActType::width> >
mvOut("StreamingMaxPool_Precision_Batch.mvOut");
215     StreamingDataWidthConverter_Batch<InStreamW, NumChan-
nels*ActType::width, InpPerImage>(in, wa_in, numReps);

```

```

216     for (unsigned int rep = 0; rep < numReps; rep++) {
217         StreamingMaxPool_Precision<ImgDim, PoolDim, NumChannels,
ActType, min_value>
218         (stat-
ic_cast<hls::stream<ap_uint<NumChannels*ActType::width>>>(wa_in),
219         stat-
ic_cast<hls::stream<ap_uint<NumChannels*ActType::width>>>(mvOut));
220     }
221     StreamingDataWidthConvert-
er_Batch<NumChannels*ActType::width, OutStreamW, OutPerIm-
age>(mvOut, out, numReps);
222 }
223 }
224
225 /**
226 * \brief 1D Max Pool implementation for non Binarized val-
ues
227 *
228 *
229 * This function performs the maxpool for non-binary inputs,
and works with kernel and stride being equal
230 *
231 * \tparam ImgDim Length of the Input Feature Map
232 * \tparam PoolDim Dimension of the Max Pool kernel
233 * \tparam NumChannels Number of Input Feature Maps
234 * \tparam PE Number of input rows (channels) com-
puted in parallel
235 * \tparam OutputSize Length of the Output Feature Map
236 * \tparam ActType DataType of the input activation (as
used in the comparison)
237 * \tparam min_value Minimum value possible with the giv-
en ActType, used to initialize the value before the comparison
238 *
239 * \param in Input stream
240 * \param out Output stream
241 *
242 */
243
244 template<unsigned int ImgDim, unsigned int PoolDim, unsigned
int NumChannels, unsigned int PE,
245 unsigned int OutputSize, typename ActType, int
min_value
246 >
247 void StreamingMax-
Pool_Precision_1d(hls::stream<ap_uint<PE*ActType::width> > & in,
248 hls::stream<ap_uint<PE*ActType::width> > & out) {
249     static_assert(NumChannels % PE == 0, "");
250     constexpr unsigned NF = NumChannels / PE;
251     constexpr unsigned REMAINDER_PIXELS = ImgDim > PoolDim *
OutputSize ? ImgDim - OutputSize * PoolDim : 0;
252
253     // need buffer space for a single maxpooled pixel of the
image
254     ActType buf[NF][PE];
255     #pragma HLS ARRAY_PARTITION variable=buf complete dim=2
256
257     for(unsigned int ch = 0; ch < NF; ch++){
258         #pragma HLS pipeline style=fllp II=1
259         for(unsigned int p = 0; p < PE; p++){
260             #pragma HLS UNROLL
261             buf[ch][p] = min_value;
262         }
263     }
264
265     ap_uint<PE*ActType::width> inputData,outputData;
266     unsigned input_count = 0;
267     for (unsigned int xp = 0; xp < OutputSize; xp++) {
268         // Change to comparator
269         for (unsigned int kx = 0; kx < PoolDim; kx++) {
270             if (input_count++ < ImgDim){
271                 for (unsigned int ch = 0; ch < NF; ch++){
272                     #pragma HLS pipeline style=fllp II=1
273                     inputData = in.read();
274                     for(unsigned int p = 0; p < PE; p++){
275                         #pragma HLS UNROLL
276                         unsigned const lowBit = p * ActType::width;
277                         unsigned const highBit = (p+1) * ActType::width
-1;
278                         ActType const channeldata = inputData(highBit,
lowBit);
279                         ActType const oldMax = buf[ch][p];
280                         if(channeldata > oldMax){
281                             buf[ch][p] = channeldata;
282                         }
283                     }
284                 }
285             }
286         }
287         for(unsigned int ch = 0; ch < NF; ch++){
288             #pragma HLS pipeline style=fllp II=1
289             for(unsigned int p = 0; p < PE; p++){
290                 #pragma HLS UNROLL
291                 unsigned const lowBit = p * ActType::width;
292                 unsigned const highBit = (p+1) * ActType::width -1;
293                 outputData(highBit, lowBit) = buf[ch][p];
294                 // get buffer ready for next use
295                 buf[ch][p] = min_value;
296             }
297         }
298     }

```



```

297     out.write(outputData);
298   }
299 }
300
301 for (unsigned int r = 0; r < REMAINDER_PIXELS*NF; r++){
302 #pragma HLS pipeline style=fllp II=1
303   inputData = in.read();
304 }
305
306 }
307
308 /**
309  * \brief 1D Max Pool implementation for non binarized val-
310  * ues on multiple images
311  *
312  * This function performs the maxpool for non binary inputs,
313  * and works with kernel and stride being equal
314  * \tparam ImgDim      Length of the Input Feature Map
315  * \tparam PoolDim      Dimension of the Max Pool kernel
316  * \tparam NumChannels  Number of Input Feature Maps
317  * \tparam PE           Number of input rows (channels) com-
318  * puted in parallel
319  * \tparam ActType      DataType of the input activation (as
320  * used in the comparison)
321  *
322  * \param min_value    Minimum value possible with the given
323  * ActType, used to initialize the value before the comparison
324  * \param in            Input stream
325  * \param out           Output stream
326  *
327  * \param numReps      Number of time the function has to be
328  * repeatedly executed (e.g. number of images)
329  */
330 template<unsigned int ImgDim, unsigned int PoolDim, unsigned
331 int NumChannels, unsigned int PE,
332 unsigned int OutputSize, typename ActType, int
333 min_value
334 >
335 void StreamingMax-
336 Pool_Precision_Batch_Id(hls::stream<ap_uint<PE*ActType::width> > &
337 in,
338 hls::stream<ap_uint<PE*ActType::width> > & out, un-
339 signed int numReps) {
340 #pragma HLS INLINE
341   for (unsigned int rep = 0; rep < numReps; rep++) {
342     StreamingMaxPool_Precision_1d<ImgDim, PoolDim, NumChan-
343 nels, PE, OutputSize,
344 ActType, min_value>
345 (in, out);
346 }
347 }
348
349 /**
350  * \brief ReLU for fixed-point or integer; can accept a
351  * bias at input, which it removes
352  *
353  * \tparam ImgDim      Width and Height of the Input Feature
354  * Map (assumed square)
355  * \tparam NumChannels  Number of Input Feature Maps
356  * \tparam ActType      DataType of the input activation (as
357  * used in the comparison)
358  * \tparam PEOcount     PE parallelism to apply ReLU
359  * \tparam offset       Offset to be subtracted before apply-
360  * ing ReLU
361  * \param in            Input stream
362  * \param out           Output stream
363  *
364  * \param numReps      Number of time the function has to be
365  * repeatedly executed (e.g. number of images)
366  */
367 template<
368   unsigned int ImgDim,
369   unsigned int NumChannels,
370   typename ActType,
371   unsigned int PEOcount,
372   int offset = 0>
373 void ReLU_Batch(hls::stream<ap_uint<PEOcount * ActType::width> >
374 & in,
375 hls::stream<ap_uint<PEOcount * ActType::width> > &
376 out, const unsigned int numReps) {
377   ap_uint<PEOcount * ActType::width> thin;
378   ap_uint<PEOcount * ActType::width> thout;
379
380   //call to thresholding library function
381   for(unsigned int reps=0; reps<numReps; reps++){
382     for(unsigned int pixel=0; pixel<ImgDim*ImgDim; pixel+
383 )){
384       for(unsigned int fold=0; fold<NumChannels/PEOcount;
385 fold++){

```

```

370 #pragma HLS pipeline style=fllp II=1
371   thin = in.read();
372   for(unsigned int pe=0; pe<PEOcount; pe++){
373 #pragma HLS UNROLL
374   // Threshold and assign to right bits of output
375   buffers
376     unsigned int lowBit = pe * ActType::width;
377     unsigned int highBit = (pe+1) * ActType::width - 1;
378     ActType val = thin(highBit, lowBit);
379     ActType result;
380     if(val < offset)
381       result = 0;
382     else
383       result = val - offset;
384     thout(highBit, lowBit) = result;
385   }
386   out.write(thout);
387 }
388 }
389 }
390
391 /**
392  * \brief Accumulate-pool - like average pooling over the
393  * whole frame, but without the division at end
394  *
395  * \tparam ImgDim      Width and Height of the Input Feature
396  * Map (assumed square)
397  * \tparam NumChannels  Number of Input Feature Maps
398  * \tparam ActType      DataType of the input activation (as
399  * used in the comparison)
400  * \tparam PEOcount     PE parallelism to apply ReLU
401  * \tparam AccType      Datatype of the accumulation (e.g.
402  * output)
403  * \param in            Input stream
404  * \param out           Output stream
405  *
406  * \param numReps      Number of time the function has to be
407  * repeatedly executed (e.g. number of images)
408  */
409 template<
410   unsigned int ImgDim,
411   unsigned int NumChannels,
412   typename ActType,
413   unsigned int PEOcount,
414   typename AccType>
415 void AccPool_Batch(hls::stream<ap_uint<PEOcount * Act-
416 Type::width> > & in,
417 hls::stream<ap_uint<PEOcount * AccType::width> > &
418 out, const unsigned int numReps) {
419   ap_uint<PEOcount * ActType::width> thin;
420   ap_uint<PEOcount * AccType::width> accumulators[NumChannels/
421 PEOcount];
422 #pragma HLS bind_storage variable=accumulators type=RAM_2P
423 impl=LUTRAM
424
425   //call to thresholding library function
426   for(unsigned int reps=0; reps<numReps; reps++){
427     for(unsigned int pixel=0; pixel<ImgDim*ImgDim; pixel+
428 )){
429       for(unsigned int fold=0; fold<NumChannels/PEOcount;
430 fold++){
431 #pragma HLS pipeline style=fllp II=1
432       thin = in.read();
433       ap_uint<PEOcount * AccType::width> accbank = accumu-
434 lators[fold];
435       for(unsigned int pe=0; pe<PEOcount; pe++){
436 #pragma HLS UNROLL
437       // Threshold and assign to right bits of output
438       buffers
439         ActType const val = thin((pe+1) * ActType::width -
440 1, pe * ActType::width);
441         AccType const acc = accbank((pe+1) *
442 ActType::width - 1, pe * ActType::width);
443         AccType const result = val + (pixel == 0?
444 AccType(0) : acc);
445         accbank((pe+1) * ActType::width - 1, pe *
446 ActType::width) = result;
447       }
448       accumulators[fold] = accbank;
449     }
450   }
451
452   for (unsigned int fold = 0; fold < NumChannels / PEOcount;
453 fold++){
454     {
455       out.write(accumulators[fold]);
456     }
457   }
458 }
459
460 /**
461  * \brief LabelSelect_Batch - returns labels of top-NumTop
462  * in stream

```

```

446 *
447 * \tparam NumClasses    Number of classes of the dataset
448 * \tparam PEOunt        Number of inputs to be processed in
parallel
449 * \tparam NumTop        Number of top classes to be selected
in output
450 * \tparam In_T          Datatype of the input
451 * \tparam Out_T         Datatype of the output
452 *
453 * \param in              Input stream
454 * \param out             Output stream
455 * \param numReps         Number of times the function has to
be repeatedly executed (e.g. number of images)
456 *
457 */
458
459 template<
460 // tensor size parameters
461 unsigned int NumClasses,
462 unsigned int PEOunt,
463 unsigned int NumTop,
464 typename In_T,
465 typename Out_T>
466 void LabelSelect_Batch(hls::stream<ap_uint<PEOunt *
In_T::width> > & in,
467
468 hls::stream<Out_T> & out, const unsigned int numReps) {
469 // Check that classes, aka. labels / indeces, can be en-
coded as non-negative outputs
470 static_assert(clog2(NumClasses) <= Out_T::width -
Out_T::sign_flag, "");
471 static In_T const In_T_MIN_VAL = (In_T(-1)<0)?
1<<(In_T::width-1) : 0;
472
473 // Array of encountered top values
474 // - maintains topval[i] <= topval[i+1]
475 // - keeps in alignment with toplabels
476 In_T topval[NumTop];
477 #pragma HLS ARRAY_PARTITION variable=topval complete dim=1
478 Out_T toplabels[NumTop];
479 #pragma HLS ARRAY_PARTITION variable=toplabels complete dim=1
480
481 for(unsigned int reps=0; reps<numReps; reps++){
482 unsigned int idx = 0;
483 for(unsigned int topx=0; topx<NumTop; topx++){
484 #pragma HLS UNROLL
485 topval [topx] = In_T_MIN_VAL;
486 toplabels[topx] = 0;
487 }
488 for(unsigned int block=0; block<(NumClasses/PEOunt);
block++){
489 #pragma HLS pipeline style=flp II=1
490 ap_uint<PEOunt * In_T::width> const inval =
in.read();
491 for(unsigned int elem=0; elem<PEOunt; elem++){
492 #pragma HLS UNROLL
493 // Extract individual input
494 unsigned const lowBit = elem * In_T::width;
495 unsigned const highBit = (elem+1) * In_T::width - 1;
496 In_T const val = inval(highBit,lowBit);
497
498 // Compare input against all current tops
499 bool cmp[NumTop+1];
500 for(unsigned i = 0; i < NumTop; i++) {
501 #pragma HLS UNROLL
502 cmp[i] = val > topval[i];
503 }
504 cmp[NumTop] = false;
505
506 // Shift input into top array at the highest index
where it is greater
507 for(unsigned i = 0; i < NumTop; i++) {
508 #pragma HLS UNROLL
509 if(cmp[i]) {
510 if(cmp[i+1]) {
511 // Shift
512 topval [i] = topval [i+1];
513 toplabels[i] = toplabels[i+1];
514 }
515 else {
516 // Insert
517 topval [i] = val;
518 toplabels[i] = idx;
519 }
520 }
521 }
522 idx++;
523 }
524 }
525
526 // Output - index of highest value first
527 for(unsigned int topx = 0; topx < NumTop; topx++){
528 out.write(toplabels[NumTop - topx - 1]);
529 }
530 }
531 }
532
533
534 /**

```

```

535 * \brief Pool_batch function
536 *
537 * The function performs a generic pool function (defined in
pool.hpp) and works in conjunction
538 * with a sliding window unit performing im2col on the input
data, allowing
539 * generic kernel and stride values
540 *
541 * \tparam Channels      Number of channels in the pool layer
542 * \tparam PE            Number of channels in the pool layer
computed in parallel
543 * \tparam TotalK        Total kernel size of pooling (e.g.
3x3=9)
544 * \tparam TSrcI         DataType of the input value (Slice)
545 * \tparam TDstI         DataType of the output value (Slice)
546 * \tparam TI            DataType of the input stream - safely
deducible from the paramaters
547 * \tparam TO            DataType of the output stream - safely
deducible from the paramaters
548 * \tparam TA            DataType of the function class (e.g.
Max, Avg, Sum) - safely deducible from the paramaters
549 *
550 * \param in              Input stream
551 * \param out             Output stream
552 * \param function        Function class in the pool (Max, Avg,
Sum)
553 * \param reps            Number of time the function has to be
repeatedly executed (e.g. number of images)
554 */
555 template<
556 unsigned Channels, unsigned PE, unsigned TotalK,
557 typename TSrcI = Identity, typename TDstI = Identity,
558 typename TI, typename TO, typename TA
559 >
560 void Pool_batch(hls::stream<TI> &in,
561 hls::stream<TO> &out,
562 TA const &function,
563 int const reps) {
564
565 constexpr unsigned NF = Channels / PE;
566 constexpr unsigned SF = TotalK;
567 constexpr unsigned TOTAL_FOLD = NF * SF ;
568
569 decltpe(function.init()) accu[PE];
570 #pragma HLS ARRAY_PARTITION variable=accu complete dim=0
571
572 unsigned sf = 0;
573 // everything merged into a common iteration space (one
"big" loop instead
574 // of smaller nested loops) to get the pipelining the way
we want
575 for(unsigned i = 0; i < reps * TOTAL_FOLD; i++) {
576 #pragma HLS pipeline style=flp II=1
577 TI pixel_slice;
578 pixel_slice = in.read();
579
580 // Threshold Initialisation
581 if(sf == 0) {
582 for(unsigned pe = 0; pe < PE; pe++) {
583 #pragma HLS UNROLL
584 accu[pe] = function.init();
585 }
586 }
587
588 auto const slice_channels = TSrcI()(pixel_slice,0);
589 for(unsigned pe = 0; pe < PE; pe++) {
590 #pragma HLS UNROLL
591 accu[pe] = function.pool(slice_channels(pe,0),
accu[pe]);
592 }
593
594 // keep track of which folded synapse/neuron we are pro-
cessing
595 if(++sf == SF) {
596 // produce output and clear accumulators
597 auto outElem = TDstI().template operator()(<TO>());
598 for(unsigned pe = 0; pe < PE; pe++) {
599 #pragma HLS UNROLL
600 outElem(pe,0,1) = function.activate(accu[pe]); //
601 }
602 out.write(outElem);
603 // next folded neuron or image
604 sf = 0;
605 }
606 }
607 }
608
609 #endif

```

D.19 finn_hlslib/mmv.hpp (59 lines, 2,508 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following

```

```

conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"

20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,

21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,

25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,

27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31 *
*****
*****/
32 *
33 /
*****
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 *
37 * \file
38 *

39 * This file described the MultiChanData class used for MMV,
whenever we exploit
40 * the pixel level of parallelism.
41 *
42 *
*****
*****/
43 *
44 #ifndef MMVCLASS_H
45 #define MMVCLASS_H
46 *
47 #include "hls_stream.h"
48 *
49 template <unsigned int NumChannels, unsigned int DataWidth>
50 class MultiChanData {
51 public: ap_uint<DataWidth> data[NumChannels];

52     auto operator[](unsigned const mm) -> decltype(data[mm]) {
53 #pragma HLS inline
54         return data[mm];
55     }
56 *
57 };
58 *
59 #endif

```

D.20 finn_hlslib/mvau.hpp (310 lines, 11.893 B)

```

1 /
*****
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,

```

```

9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"

20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,

21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,

25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,

27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31 *
*****
*****/
32 *
33 /
*****
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 * Thomas B. Preusser <thomas.preusser@utexas.edu>
37 * Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 * Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * \file mvau.hpp
41 *
42 * This file lists a templated function used to implement
43 * Matrix-Vector-Activation Unit
44 *
45 * This project has received funding from the European Uni-
on's Framework
46 * Programme for Research and Innovation Horizon 2020
(2014-2020) under
47 * the Marie Skłodowska-Curie Grant Agreement No. 751339.
48 *
49 *
*****
*****/
50 *
51 #ifndef MVAU_HPP
52 #define MVAU_HPP
53 *
54 #include "hls_stream.h"
55 *
56 #include "mac.hpp"
57 #include "interpret.hpp"
58 #include "weights.hpp"
59 *
60 /**
61 * \brief Matrix vector activate function
62 *
63 * The function performs the multiplication between a weight
matrix and the input activation vector,
64 * accumulating the results and then applying an activation
function on the accumulated result.
65 *
66 *
67 * \tparam MatrixW Width of the input matrix
68 * \tparam MatrixH Height of the input matrix
69 * \tparam SIMD Number of input columns computed in
parallel
70 * \tparam PE Number of output rows computed in par-
allel
71 * \tparam MMV Number of output pixels computed in
parallel
72 * \tparam TSrcI DataType of the input activation (as
used in the MAC)
73 * \tparam TDstI DataType of the output activation (as

```



```

generated by the activation)
74 * \tparam TweightI    DataType of the weights and how to access them in the array
75 * \tparam TI          DataType of the input stream - safely deducible from the parameters
76 * \tparam TO          DataType of the output stream - safely deducible from the parameters

77 * \tparam TW          DataType of the weights matrix - safely deducible from the parameters
78 * \tparam TA          DataType of the activation class (e.g. thresholds) - safely deducible from the parameters

79 * \tparam R           Datatype for the resource used for FPGA implementation of the MAC - safely deducible from the parameters
80 *
81 * \param in           Input stream
82 * \param out          Output stream
83 * \param weights      Weights matrix (currently supports BinaryWeights or FixedPointWeights)
84 * \param activation    Activation class
85 * \param reps         Number of time the function has to be repeatedly executed (e.g. number of images)
86 * \param r           Resource type for the hardware implementation of the MAC block
87 */
88 template<
89   unsigned MatrixW, unsigned MatrixH, unsigned SIMD, unsigned PE, unsigned MMV,
90   typename TSrcI = Identity, typename TDstI = Identity, typename TweightI = Identity,
91   typename TI, typename TO, typename TW, typename TA, typename R
92 >
93 void Matrix_Vector_Activate_Batch(hls::stream<TI> &in,
94                                   hls::stream<TO> &out,
95                                   TW const &weights,
96                                   TA const &activation,
97                                   int const reps,
98                                   R const &r) {
99
100 // how many different rows each neuron will compute
101 // alternatively: number of vertical matrix chunks
102 unsigned const NF = MatrixH / PE;
103
104 // how many synapse groups each row is split into
105 // alternatively: number of horizontal matrix chunks
106 unsigned const SF = MatrixW / SIMD;
107
108 // input vector buffers
109 TI inputBuf[SF];
110 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=0
111
112 decltype(activation.init(0,0)) accu[MMV][PE];
113 #pragma HLS ARRAY_PARTITION variable=accu complete dim=0
114
115 unsigned nf = 0;
116 unsigned sf = 0;
117 unsigned tile = 0; // invariant: tile = nf*SF + sf
118
119 // everything merged into a common iteration space (one "big" loop instead
120 // of smaller nested loops) to get the pipelining the way we want
121
122 unsigned const TOTAL_FOLD = NF * SF;
123 for(unsigned i = 0; i < reps * TOTAL_FOLD; i++) {
124 #pragma HLS pipeline style=flp II=1
125   TI inElem;
126   if(nf == 0) {
127     // read input from stream
128     inElem = in.read();
129     // store in appropriate buffer for reuse
130     inputBuf[sf] = inElem;
131   }
132   else {
133     // reuse buffered input
134     inElem = inputBuf[sf];
135   }
136
137 // Threshold Initialisation
138 if(sf == 0) {
139   for(unsigned pe = 0; pe < PE; pe++) {
140 #pragma HLS UNROLL
141     for(unsigned mmv = 0; mmv < MMV; mmv++) {
142 #pragma HLS UNROLL
143       accu[mmv][pe] = activation.init(nf, pe);
144     }
145   }
146 }
147
148 // compute matrix-vector product for each processing element
149 auto const &w = weights.weights(tile);
150 for(unsigned pe = 0; pe < PE; pe++) {
151 #pragma HLS UNROLL
152   auto const wgt = TweightI()(w[pe]);
153   for (unsigned mmv = 0; mmv < MMV; mmv++){

```

```

154     auto const act = TSrcI()(inElem, mmv);
155     accu[mmv][pe] = mac<SIMD>(accu[mmv][pe], wgt, act, r, mmv);
156   }
157 }
158
159 // keep track of which folded synapse/neuron we are processing
160 ++tile;
161 if(++sf == SF) {
162   // produce output and clear accumulators
163   auto outElem = TDstI().template operator()(<TO>());
164   for (unsigned pe = 0; pe < PE; pe++) {
165 #pragma HLS UNROLL
166     for (unsigned mmv = 0; mmv < MMV; mmv++){
167 #pragma HLS UNROLL
168       outElem(pe,mmv,1) = activation.activate(nf, pe, accu[mmv][pe]);
169     }
170   }
171   out.write(outElem);
172   // next folded neuron or image
173   sf = 0;
174   if(++nf == NF) {
175     nf = 0;
176     tile = 0;
177   }
178 }
179 }
180 }
181
182 /**
183 * \brief Matrix vector activate function with streaming weights
184 *
185 * The function performs the multiplication between a weight matrix, presented as an input stream, and the input activation vector,
186 * accumulating the results and then applying an activation function on the accumulated result. Does not support MMV.
187 *
188 * \tparam MatrixW      Width of the input matrix
189 * \tparam MatrixH      Height of the input matrix
190 * \tparam SIMD         Number of input columns computed in parallel
191 * \tparam PE           Number of output rows computed in parallel
192 * \tparam TSrcI        DataType of the input activation (as used in the MAC)
193 * \tparam TDstI        DataType of the output activation (as generated by the activation)
194 * \tparam TweightI     DataType of the weights and how to access them in the array
195 * \tparam TW           DataType of the weights (as used in the MAC) - not deducible from the parameters
196 * \tparam TI           DataType of the input stream - safely deducible from the parameters
197 * \tparam TO           DataType of the output stream - safely deducible from the parameters
198 * \tparam TA           DataType of the activation class (e.g. thresholds) - safely deducible from the parameters
199 * \tparam R            Datatype for the resource used for FPGA implementation of the MAC - safely deducible from the parameters
200 *
201 * \param in           Input stream
202 * \param out          Output stream
203 * \param weight       Weight stream (currently supports BinaryWeights or FixedPointWeights)
204 * \param activation    Activation class
205 * \param reps         Number of time the function has to be repeatedly executed (e.g. number of images)
206 * \param r           Resource type for the hardware implementation of the MAC block
207 */
208 template<
209   unsigned MatrixW, unsigned MatrixH, unsigned SIMD, unsigned PE,
210   typename TSrcI = Identity, typename TDstI = Identity, typename TweightI = Identity,
211   typename TI, typename TO, typename TA, typename R
212 >
213 void Matrix_Vector_Activate_Stream_Batch(hls::stream<TI> &in,
214                                           hls::stream<TO> &out,
215                                           hls::stream<ap_uint<PE*SIMD*TW::width>> &weight,
216                                           TA const &activation,
217                                           int const reps,
218                                           R const &r) {
219
220 // how many different rows each neuron will compute
221 // alternatively: number of vertical matrix chunks
222 unsigned const NF = MatrixH / PE;
223
224 // how many synapse groups each row is split into
225 // alternatively: number of horizontal matrix chunks

```

```

228 unsigned const SF = MatrixW / SIMD;
229
230 // input vector buffers
231 TI inputBuf[SF];
232 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
233 // accumulators
234 decltype(activation.init(0,0)) accu[1][PE];
235 #pragma HLS ARRAY_PARTITION variable=accu complete dim=0
236 // unpacked and packed buffers for weight stream
237 Weights_Tile<SIMD, TW, PE > w;
238 #pragma HLS ARRAY_PARTITION variable=w.m_weights complete
dim=0
239 ap_uint<PE * SIMD * TW::width> W_packed;
240
241
242 unsigned nf = 0;
243 unsigned sf = 0;
244 unsigned tile = 0; // invariant: tile = nf*SF + sf
245
246 // everything merged into a common iteration space (one
"big" loop instead
247 // of smaller nested loops) to get the pipelining the way
we want
248 unsigned const TOTAL_FOLD = NF * SF;
249 for(unsigned i = 0; i < reps * TOTAL_FOLD; i++) {
250 #pragma HLS pipeline style=flp II=1
251 TI inElem;
252
253 if(nf == 0) {
254 // read input from stream
255 inElem = in.read();
256 // store in appropriate buffer for reuse
257 inputBuf[sf] = inElem;
258 }
259 else {
260 // reuse buffered input
261 inElem = inputBuf[sf];
262 }
263
264 // read from the parameter stream
265 W_packed = weight.read();
266 for (unsigned pe = 0; pe < PE; pe++) {
267 #pragma HLS UNROLL
268 w.m_weights[pe] =
W_packed((pe+1)*SIMD*TW::width-1,pe*SIMD*TW::width);
269 }
270
271 // Threshold Initialisation
272 if(sf == 0) {
273 for(unsigned pe = 0; pe < PE; pe++) {
274 #pragma HLS UNROLL
275 accu[0][pe] = activation.init(nf, pe);
276 }
277 }
278
279 // compute matrix-vector product for each processing ele-
ment
280 for(unsigned pe = 0; pe < PE; pe++) {
281 #pragma HLS UNROLL
282 auto const act = TSrcI()(inElem, 0);
283 auto const wgt = TWeightI()(w[pe]);
284 //auto const wgt = w[pe];
285 accu[0][pe] = mac<SIMD>(accu[0][pe], wgt, act, r, 0);
286 }
287
288 // keep track of which folded synapse/neuron we are pro-
cessing
289 ++tile;
290 if(++sf == SF) {
291 // produce output and clear accumulators
292 auto outElem = TDstI().template operator()<TO>();
293 for (unsigned pe = 0; pe < PE; pe++) {
294 #pragma HLS UNROLL
295 outElem(pe,0,1) = activation.activate(nf, pe, accu[0]
[pe]);
296 }
297
298 out.write(outElem);
299
300 // next folded neuron or image
301 sf = 0;
302 if(++nf == NF) {
303 nf = 0;
304 tile = 0;
305 }
306 }
307 }
308 }
309
310 #endif

```

D.21 finn_hlslib/slidingwindow.h (2,095 lines, 87,820 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *

```

```

5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31 *****/
32
33 /
34 *****
35 *
36 * Authors: Giulio Gambardella <giulio@xilinx.com>
37 * Thomas B. Preusser <thomas.preusser@utexas.edu>
38 * Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
39 * Christoph Doebling <cdoehrin@xilinx.com>
40 * Felix Jentzsch <felix.jentzsch@upb.de>
41 * \file slidingwindow.h
42 *
43 * Library of templated HLS functions for BNN deployment.
44 * This file lists a set of convenience functions used to im-
plement
45 * Sliding window generator for convolutions
46 *
47 *****/
48
49 #ifndef SLIDINGWINDOW_H
50 #define SLIDINGWINDOW_H
51
52 #include <algorithm>
53 #include "utils.hpp"
54
55 /**
56 * \brief Memory resource pragma instantiation for the
sliding window generator, default resource
57 *
58 * The buffer in the sliding window generator can be imple-
mented in multiple hardware resources.
59 *
60 * ap_resource_dflt will let HLS choose the best one
61 *
62 * ap_resource_bram will force HLS to implement the buffer in
BRAMS
63 *
64 * ap_resource_uram will force HLS to implement the buffer in
URAMS
65 *
66 * ap_resource_lutram will force HLS to implement the buffer
in LUTRAMS
67 *
68 * \tparam T Datatype of the buffer instan-
tiated in the sliding window generator

```

```

66 *
67 * \param      inputBufBuffer used in the SWG
68 * \param      r      Resource type for the hardware imple-
mentation
69 *
70 * \return      Result of the multiply operation
71 */
72 template <typename T>
73 void memory_resource(T inputBuf, ap_resource_dflt const&){
74 #pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P
75 }
76 /**
77 * \brief      Memory resource pragma instantiation for the
sliding window generator, BRAM resource
78 *
79 * The buffer in the sliding window generator can be imple-
mented in multiple hardware resources.
80 *
81 * ap_resource_dflt will let HLS choose the best one

82 * ap_resource_bram will force HLS to implement the buffer in
BRAMS

83 * ap_resource_uram will force HLS to implement the buffer in
URAMS
84 * ap_resource_lutram will force HLS to implement the buffer
in LUTRAMs
85 *
86 * \tparam      T      Datatype of the buffer instan-
tiated in the sliding window generator
87 *
88 * \param      inputBufBuffer used in the SWG
89 * \param      r      Resource type for the hardware imple-
mentation
90 *
91 * \return      Result of the multiply operation
92 */
93 template <typename T>
94 void memory_resource(T inputBuf, ap_resource_bram const&){
95 #pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P
impl=BRAM
96 }
97 /**
98 * \brief      Memory resource pragma instantiation for the
sliding window generator, URAM resource
99 *
100 * The buffer in the sliding window generator can be imple-
mented in multiple hardware resources.
101 *
102 * ap_resource_dflt will let HLS choose the best one

103 * ap_resource_bram will force HLS to implement the buffer in
BRAMS

104 * ap_resource_uram will force HLS to implement the buffer in
URAMS
105 * ap_resource_lutram will force HLS to implement the buffer
in LUTRAMs
106 *
107 * \tparam      T      Datatype of the buffer instan-
tiated in the sliding window generator
108 *
109 * \param      inputBufBuffer used in the SWG
110 * \param      r      Resource type for the hardware imple-
mentation
111 *
112 * \return      Result of the multiply operation
113 */
114 template <typename T>
115 void memory_resource(T inputBuf, ap_resource_uram const&){
116 #pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P
impl=URAM
117 }
118 /**
119 * \brief      Memory resource pragma instantiation for the
sliding window generator, LUTRAM resource
120 *
121 * The buffer in the sliding window generator can be imple-
mented in multiple hardware resources.
122 *
123 * ap_resource_dflt will let HLS choose the best one

124 * ap_resource_bram will force HLS to implement the buffer in
BRAMS

125 * ap_resource_uram will force HLS to implement the buffer in
URAMS
126 * ap_resource_lutram will force HLS to implement the buffer
in LUTRAMs
127 *
128 * \tparam      T      Datatype of the buffer instan-
tiated in the sliding window generator
129 *
130 * \param      inputBufBuffer used in the SWG
131 * \param      r      Resource type for the hardware imple-
mentation
132 *
133 * \return      Result of the multiply operation
134 */
135 template <typename T>

```

```

136 void memory_resource(T inputBuf, ap_resource_lutram const&){
137 #pragma HLS BIND_STORAGE variable=inputBuf type=RAM_S2P
impl=LUTRAM
138 }
139
140 /**
141 * \brief Sliding Window unit that produces output vectors
for feeding
142 * a Matrix_Vector_Activate_Batch, implementing the im2col
algorithm. To be used only if
143 * ConvKernelDim%Stride = 0
144 *
145 * \tparam ConvKernelDim      Dimension of the convolutional
kernel (assumed square)
146 * \tparam IFMChannels      Number of Input Feature Maps
147 * \tparam Input_precision      Number bits per pixel
148 * \tparam IFMDim      Width and Height of the Input
Feature Map (assumed square)
149 * \tparam OFMDim      Width and Height of the Output
Feature Map (assumed square)
150 * \tparam SIMD      Number of input columns computed
in parallel
151 * \tparam Stride      Stride of the convolutional ker-
nel
152 * \tparam R      Datatype for the resource used for
FPGA implementation of the SWG - safely deducible from the parama-
ters
153 *
154 * \param in      Input stream
155 * \param out      Output stream
156 * \param numReps      Number of time the function has
to be repeatedly executed (e.g. number of images)
157 * \param r      Resource type for the hard-
ware implementation of the memory block
158 */
159 template<unsigned int ConvKernelDim,
160         unsigned int IFMChannels,
161         unsigned int Input_precision,
162         unsigned int IFMDim,
163         unsigned int OFMDim,
164         unsigned int SIMD,
165         unsigned int Stride,
166         typename R>
167 void ConvolutionInputGenerator(
168     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
169     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
170     const unsigned int numReps,
171     R const &r) {
172     static_assert(IFMChannels % SIMD == 0, "");
173     static_assert(ConvKernelDim % Stride == 0, "");
174     const unsigned int multiplying_factor = IFMChannels/SIMD;
175     const unsigned int number_blocks = ConvKernelDim/Stride +
1;
176     ap_uint<SIMD*Input_precision> inputBuf[number_blocks]
[Stride * IFMDim * multiplying_factor];
177 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
178     memory_resource(inputBuf, r);
179     const unsigned int cycles_write_block = (OFMDim * ConvKer-
nelDim * ConvKernelDim * multiplying_factor);
180     const unsigned int cycles_read_block = Stride * IFMDim *
multiplying_factor;
181     const unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);
182     const unsigned int baseIter = IFMDim * ConvKernelDim * mul-
tiplying_factor// Initial buffer
183                                     + OFMDim *
std::max(cycles_write_block, cycles_read_block);
184     unsigned int counter_internal_block = 0;
185     unsigned int current_block_write = 0;
186     unsigned int current_line = 0;
187     unsigned int read_block = 0;
188     unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0, k_x =
0, count_simd=0;
189
190     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
191         for (unsigned int i = 0; i < baseIter; i++) {
192             #pragma HLS pipeline style=flp II=1
193             if (inp < IFMDim * ConvKernelDim*multiplying_factor)
{// Initial buffer of ConvKernelDim lines
194                 ap_uint<SIMD*Input_precision> inElem;
195                 inElem = in.read();
196                 inputBuf[current_block_write][current_line] = inElem;
197                 current_line++;
198                 inp++;
199                 if (current_line == Stride * IFMDim * multiply-
ing_factor ) {
200                     current_line = 0;
201                     current_block_write++;
202                     if (current_block_write == number_blocks) {
203                         current_block_write=0;
204                     }
205                     read_block++;
206                     counter_internal_block = 0;
207                 }
208             } else {
209                 if (counter_internal_block < cycles_write_block-1)
{ // We are writing output, MMV IFMchan per cycle

```

```

210     unsigned int current_block_read = (current_block_write + 1 + k_y / Stride);
211     if (current_block_read >= number_blocks) {
212         current_block_read = number_blocks;
213     }
214     unsigned int current_line_in_block = ((k_y % Stride)
* IFMDim + ofm_x * Stride + k_x) * multiplying_factor + count_simd;
215     ap_uint<SIMD*Input_precision> outElem = inputBuf[current_block_read][current_line_in_block];
216     out.write(outElem);
217     count_simd++;
218     if (count_simd == multiplying_factor) {
219         count_simd = 0;
220         k_x++;
221         if (k_x == ConvKernelDim) {
222             k_x = 0;
223             k_y++;
224             if (k_y == ConvKernelDim) {
225                 k_y = 0;
226                 ofm_x++;
227                 if (ofm_x == OFMDim) {
228                     ofm_x = 0;
229                     ofm_y++;
230                     if (ofm_y == OFMDim) {
231                         ofm_y = 0;
232                         inp = 0;
233                     }
234                 }
235             }
236         }
237     }
238 }

239 if ((counter_internal_block < cycles_read_block-1) &&
(read_block < IFMDim / Stride)) { // In parallel we write in the buffer,
in the current block write if we still need to
240     ap_uint<SIMD*Input_precision> inElem;
241     inElem = in.read();
242     inputBuf[current_block_write][current_line] = inElem;
243 #pragma AP dependence variable=inputBuf intra false
244 #pragma AP dependence variable=inputBuf inter false
245     current_line++;
246     if (current_line == Stride * IFMDim * multiplying_factor) { // We read the whole block, we change the next block
in which we want to we
247         // We filled up a block, let's not read until
248         current_line = 0;
249         read_block++;
250         current_block_write++;
251         if (current_block_write == number_blocks) {
252             current_block_write = 0;
253         }
254 #pragma AP dependence variable=current_block_write intra
false
255     }
256 }
257 counter_internal_block++; // =
(counter_internal_block + 1) % max_cycles;
258 if (counter_internal_block == (max_cycles-1)) {
259     counter_internal_block = 0;
260 }
261 } // End base_iter
262 } // End read_block = 0;
263 } // End count_image
264 } // End generator
265 } // End generator
266
267 /**
268  * Brief Sliding Window unit that produces output vectors
for feeding
269  * a Matrix_Vector_Activate_Batch, implementing the im2col
algorithm with support to multiple output pixels
270  *
271  *
272  * \tparam ConvKernelDim    Dimension of the convolutional
kernel (assumed square)
273  * \tparam IFMChannels      Number of Input Feature Maps
274  * \tparam Input_precision  Number bits per pixel
275  * \tparam IFMDim          Width and Height of the Input
Feature Map (assumed square)
276  * \tparam OFMDim          Width and Height of the Output
Feature Map (assumed square)
277  * \tparam SIMD            Number of input columns computed
in parallel
278  * \tparam Stride          Stride of the convolutional kernel
279  * \tparam MMV             Number of pixels that have to be
produced in parallel
280  * \tparam R               Datatype for the resource used for
FPGA implementation of the SWG - safely deducible from the parameters
281  *
282  * \param in               Input stream
283  * \param out              Output stream
284  * \param numReps          Number of time the function has
to be repeatedly executed (e.g. number of images)
285  * \param r               Resource type for the hardware
implementation of the memory block

```

```

286  */
287  template<unsigned int ConvKernelDim,
288          unsigned int IFMChannels,
289          unsigned int Input_precision,
290          unsigned int IFMDim,
291          unsigned int OFMDim,
292          unsigned int SIMD,
293          unsigned int Stride,
294          unsigned int MMV,
295          typename R>
296  void ConvolutionInputGenerator_MMV(
297      hls::stream<ap_uint<SIMD*Input_precision> > &
in,
298      hls::stream<MultiChanData<MMV, SIMD*Input_precision> > & out,
299      const unsigned int numReps,
300      R const &r) {
301      static_assert(IFMChannels % SIMD == 0, "");
302      static_assert(OFMDim % MMV == 0, "");
303      static_assert(ConvKernelDim % Stride == 0, "");
304      static_assert(MMV <= OFMDim, "");
305      constexpr unsigned int multiplying_factor = IFMChannels /
SIMD;
306      constexpr unsigned int number_blocks = ConvKernelDim /
Stride + 1;
307      ap_uint<SIMD*Input_precision> inputBuf[MMV][number_blocks]
[Stride * IFMDim * multiplying_factor];
308      #pragma HLS DEPENDENCE variable=inputBuf intra false
309      #pragma HLS DEPENDENCE variable=inputBuf intra false
310      #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
311      #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=2
312      memory_resource(inputBuf, r);
313      constexpr unsigned int cycles_write_block = (OFMDim *
ConvKernelDim * ConvKernelDim * multiplying_factor) / MMV;
314      constexpr unsigned int cycles_read_block = Stride *
IFMDim * multiplying_factor;
315      constexpr unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);
316      const unsigned int baseIter = IFMDim * ConvKernelDim *
multiplying_factor // Initial buffer
317      + OFMDim *
std::max(cycles_write_block, cycles_read_block);
318      unsigned int counter_internal_block = 0;
319      unsigned int current_block_write = 0;
320      unsigned int current_line = 0;
321      unsigned int read_block = 0;
322      unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0,
k_x = 0, count_simd = 0;
323
324      for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
325          for (unsigned int i = 0; i < baseIter; i++) {
326              #pragma HLS pipeline style=fpl II=1
327              if (inp < IFMDim * ConvKernelDim * multiplying_factor) // Initial buffer of ConvKernelDim lines
{
328                  ap_uint<SIMD*Input_precision>
inElem;
329                  inElem = in.read();
330                  for (unsigned int v = 0; v <
MMV; v++)
331                      inputBuf[v][current_block_write][current_line] = inElem;
332              }
333              #pragma HLS UNROLL
334              current_line++;
335              inp++;
336              if (current_line == Stride *
IFMDim * multiplying_factor) {
337                  current_line = 0;
338                  current_block_write++;
339                  if (current_block_write == number_blocks) {
340                      current_block_write = 0;
341                      read_block++;
342                      counter_internal_block = 0;
343                  }
344              }
345              else {
346                  if (counter_internal_block <
cycles_write_block-1) // We are writing output, MMV IFMChan per
cycle
347                  {
348                      unsigned int current_block_read = (current_block_write + 1 + k_y / Stride);
349                      if (current_block_read
>= number_blocks)
350                      {
351                          current_block_read = number_blocks;
352                      }
353                      unsigned int current_line_in_block = ((k_y % Stride) * IFMDim + ofm_x * Stride +
k_x) * multiplying_factor + count_simd;

```

```

356                                     MultiChanData<MMV,
SIMD*Input_precision> outElem;                                     // parallel read from
357                                     // parallel read from
all input buffers                                     for(unsigned int v =
358                                     for(unsigned int v =
0; v < MMV; v++) {                                     // each buf-
359 #pragma HLS UNROLL                                     fer's read addr is offset by its buffer index
360                                     // each buf-
361                                     ap_uint<SIMD*Input_precision> temp_value = inputBuf[v][cur-
rent_block_read][(current_line_in_block +
v*Stride*multiplying_factor)];
362 outElem.data[v] = temp_value;
363                                     }
364                                     out.write(outElem);
365                                     count_simd++;
366                                     if (count_simd == mul-
tiplying_factor) {
367                                     count_simd=0;
368                                     k_x++;
369                                     if (k_x ==
ConvKernelDim) {
370                                     k_x =
0;                                     k_y++;
371                                     k_y++;
372                                     if
(k_y == ConvKernelDim) {
373                                     k_y = 0;
374                                     ofm_x += MMV;
375                                     if (ofm_x == OFMDim) {
376                                     ofm_x = 0;
377                                     ofm_y++;
378                                     if (ofm_y == OFMDim) {
379                                     ofm_y = 0;
380                                     inp = 0;
381                                     }
382                                     }
383                                     }
384                                     }
385                                     }
386                                     }
387                                     if ((counter_internal_block <
cycles_read_block-1) && (read_block<IFMDim/Stride)) // In parallel
we write in the buffer, in the current block write if we still need
to
388                                     {
389                                     {
ap_uint<SIMD*Input_precision> inElem;
390                                     inElem = in.read();
391                                     for(unsigned int v =
0; v < MMV; v++) {
392 #pragma HLS UNROLL
393                                     inputBuf[v]
[current_block_write][current_line] = inElem;
394 #pragma AP dependence variable=inputBuf intra false
395 #pragma AP dependence variable=inputBuf inter false
396                                     }
397                                     current_line++;
398                                     if (current_line ==
Stride * IFMDim * multiplying_factor) // We read the whole block,
we change the next block in which we want to we
400                                     { // We filled up a
block, let's not read until
401                                     current_line =
0;
402                                     read_block++;
403                                     cur-
rent_block_write++;
404                                     if (cur-
rent_block_write == number_blocks)
405                                     cur-
rent_block_write=0;
406 #pragma AP dependence variable=current_block_write intra
false
407                                     }

```

```

408                                     }
409                                     counter_internal_block++; // =
(counter_internal_block +1) % max_cycles;
410                                     if (counter_internal_block ==
(max_cycles-1))
411                                     {
412                                     counter_internal_block =
0;
413                                     }
414                                     } // End base_iter
415                                     } // End count_image
416                                     read_block = 0;
417                                     } // End generator
418                                     } // End generator
419                                     }
420                                     }
421                                     /**
422                                     * \brief Sliding Window unit that produces output vectors
for feeding
423                                     * a Matrix_Vector_Activate_Batch, implementing the im2col
algorithm. To be used when
424                                     * ConvKernelDim%Stride != 0 (e.g., Kernel=3, Stride=2)
425                                     *
426                                     * \tparam ConvKernelDim      Dimension of the convolutional
kernel (assumed square)
427                                     * \tparam IFMChannels        Number of Input Feature Maps
428                                     * \tparam Input_precision    Number bits per pixel
429                                     * \tparam IFMDim             Width and Height of the Input
Feature Map (assumed square)
430                                     * \tparam OFMDim             Width and Height of the Output
Feature Map (assumed square)
431                                     * \tparam SIMD               Number of input columns computed
in parallel
432                                     * \tparam Stride              Stride of the convolutional ker-
nel
433                                     * \tparam R                  Datatype for the resource used for
FPGA implementation of the SWG - safely deducible from the parama-
ters
434                                     *
435                                     * \param in                  Input stream
436                                     * \param out                 Output stream
437                                     * \param numReps             Number of time the function has
to be repeatedly executed (e.g. number of images)
438                                     * \param r                   Resource type for the hard-
ware implementation of the memory block
439                                     */
440                                     template<unsigned int ConvKernelDim,
441                                     unsigned int IFMChannels,
442                                     unsigned int Input_precision,
443                                     unsigned int IFMDim,
444                                     unsigned int OFMDim,
445                                     unsigned int SIMD,
446                                     unsigned int Stride,
447                                     typename R>
448                                     void ConvolutionInputGenerator_kernel_stride(
449                                     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
450                                     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
451                                     const unsigned int numReps,
452                                     R const &r) {
453                                     static_assert(IFMChannels % SIMD == 0, "");
454                                     static_assert(ConvKernelDim % Stride != 0, "");
455                                     constexpr unsigned multiplying_factor = IFMChannels/
SIMD;
456                                     constexpr unsigned number_blocks = ConvKernelDim +
Stride ;
457                                     constexpr unsigned cycles_write_block = OFMDim * Con-
vKernelDim * ConvKernelDim * multiplying_factor;
458                                     constexpr unsigned cycles_read_block = IFMDim *
Stride * multiplying_factor;
459                                     constexpr unsigned max_cycles =
std::max(cycles_write_block, cycles_read_block);
460                                     constexpr unsigned baseIter = (IFMDim * ConvKernelDim *
multiplying_factor) + (OFMDim-1) *
max_cycles+std::max(cycles_write_block,OFMDim);
461                                     constexpr unsigned initial_buffer_cycles = (IFMDim *
ConvKernelDim * multiplying_factor) ;
462                                     ap_uint<SIMD*Input_precision> inputBuf[number_blocks]
[IFMDim * multiplying_factor];
463                                     memory_resource(inputBuf, r);
464                                     #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
465                                     #pragma HLS DEPENDENCE variable=inputBuf inter false
466                                     #pragma HLS DEPENDENCE variable=inputBuf intra false
467                                     unsigned int counter_internal_block = 0;
468                                     unsigned int current_line = 0;
469                                     unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0,
k_x = 0, count_simd = 0;
470                                     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
471                                     unsigned int floor_block_read = 0, ceil_block_read = num-
ber_blocks;
472                                     unsigned int current_block_write = 0;
473                                     #pragma HLS DEPENDENCE variable=current_block_write intra

```



```

false
477   unsigned int read_block = 0;
478   for (unsigned int i = 0; i < baseIter; i++) {
479   #pragma HLS pipeline style=flp II=1
480   if (inp < initial_buffer_cycles) //
Initial buffer of PoolDim lines
481   {
482       ap_uint<SIMD*Input_precision>
inElem;
483       inElem = in.read();
484       inputBuf[current_block_write]
[current_line] = inElem;
485       current_line++;
486       inp++;
487       if (current_line == IFMDim *
multiplying_factor)
488       {
489           current_line = 0;
490           current_block_write++;
491           if (cur-
rent_block_write == number_blocks)
492           {
493               cur-
read_block++;
494               counter_internal_block =
0;
495           }
496       }
497       else
498       {
499           if (counter_internal_block <
cycles_write_block-1 || read_block==IFMDim) // We are writing out-
put, MMV IFMChan per cycle
500           {
501               //following code im-
plements: current_block_read = (ofm_y*Stride + k_y)%number_blocks;
502               unsigned int current_block_read = (ofm_y*Stride +
k_y);
503               //reminder computation
504               if (current_block_read >= ceil_block_read)
505               {
506                   floor_block_read += number_blocks;
507                   ceil_block_read += number_blocks;
508               }else if(current_block_read < floor_block_read){
509                   ceil_block_read -= number_blocks;
510                   floor_block_read -= number_blocks;
511               }
512               current_block_read -= floor_block_read;
513
514               unsigned int cur-
rent_line_in_block = (ofm_x * Stride + k_x)*multiplying_factor +
count_simd;
515       ap_uint<SIMD*Input_precision> outElem = input-
Buf[current_block_read][(current_line_in_block)];
516       out.write(outElem);
517       count_simd++;
518       if (count_simd == mul-
tipling_factor) {
519           count_simd=0;
520           k_x++;
521           if (k_x ==
ConvKernelDim) {
522               k_x =
0;
523               k_y++;
524               if
(k_y == ConvKernelDim) {
525                   k_y = 0;
526                   ofm_x++;
527                   if (ofm_x == OFMDim) {
528                       ofm_x = 0;
529                       ofm_y++;
530                       if (ofm_y == OFMDim) {
531                           ofm_y = 0;
532                           inp = 0;
533                       }
534                   }

```

```

535   }
536   }
537   }
538   }
539   if ((counter_internal_block <
cycles_read_block - 1) && (read_block<IFMDim)) // In parallel we
write in the buffer, in the current block write if we still need to
540   {
541       ap_uint<SIMD*Input_precision> inElem;
542       inElem = in.read();
543       input-
Buf[current_block_write][current_line] = inElem;
544       #pragma HLS DEPENDENCE variable=inputBuf inter false
545       #pragma HLS DEPENDENCE variable=inputBuf intra false
546       current_line++;
547       if (current_line ==
IFMDim * multiplying_factor) // We read the whole block, we change
the next block in which we want to we
548       { // We filled up a
block, let's not read until
549           current_line =
0;
550           read_block++;
551           cur-
rent_block_write++;
552           if (cur-
rent_block_write == number_blocks)
553           {
554               cur-
rent_block_write = 0;
555               #pragma HLS DEPENDENCE variable=current_block_write intra
false
556           }
557           counter_internal_block++; // =
(counter_internal_block +1) % max_cycles;
558           if (counter_internal_block == (max_cycles-1))
559           {
560               counter_internal_block = 0;
561           }
562       }
563       } // End base_iter
564   }
565 }
566
567 /**
568  * \brief Sliding Window unit that produces output vectors
for feeding
569  * a Matrix_Vector_Activate_Batch, implementing the im2col
algorithm. To be used when
570  * ConvKernelDim%Stride != 0 (e.g., Kernel=3, Stride=2)
571  *
572  * \tparam ConvKernelDim      Dimension of the convolutional
kernel (assumed square)
573  * \tparam IFMChannels        Number of Input Feature Maps
574  * \tparam Input_precision    Number bits per pixel
575  * \tparam IFMDim             Width and Height of the Input
Feature Map (assumed square)
576  * \tparam OFMDim             Width and Height of the Output
Feature Map (assumed square)
577  * \tparam SIMD               Number of input columns computed
in parallel
578  * \tparam Stride             Stride of the convolutional ker-
nel
579  * \tparam MMV                Number of pixels that have to be
produced in parallel
580  * \tparam R                  Datatype for the resource used for
FPGA implementation of the SWG - safely deducible from the parama-
ters
581  *
582  * \param in                  Input stream
583  * \param out                  Output stream
584  * \param numReps              Number of time the function has
to be repeatedly executed (e.g. number of images)
585  * \param r                    Resource type for the hard-
ware implementation of the memory block
586  */
587
588 template<unsigned int ConvKernelDim,
589          unsigned int IFMChannels,
590          unsigned int Input_precision,
591          unsigned int IFMDim,
592          unsigned int OFMDim,
593          unsigned int SIMD,
594          unsigned int Stride,
595          unsigned int MMV,
596          typename R>
597 void ConvolutionInputGenerator_kernel_stride_MMV(
598     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
599     hls::stream<MultiChanData<MMV, SIMD*In-
put_precision> > & out,
600     const unsigned int numReps,
601     R const &r) {
602     static_assert(IFMChannels % SIMD == 0, "");
603     static_assert(ConvKernelDim % Stride != 0, "");
604     static_assert(OFMDim % MMV == 0, "");
605     static_assert(MMV <= OFMDim, "");

```

```

606
607     const unsigned int multiplying_factor = IFMChannels/
SIMD;
608     const unsigned int number_blocks = ConvKernelDim +
Stride ;
609     ap_uint<SIMD*Input_precision> inputBuf[MMV][num-
ber_blocks][IFMDim * multiplying_factor];
610 #pragma HLS DEPENDENCE variable=inputBuf inter false
611 #pragma HLS DEPENDENCE variable=inputBuf intra false
612 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
613 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=2
614     memory_resource(inputBuf, r);
615     const unsigned int cycles_write_block = (OFMDim * Con-
vKernelDim * ConvKernelDim * multiplying_factor)/MMV;
616     const unsigned int cycles_read_block = IFMDim * Stride *
multiplying_factor;
617     const unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);
618     const unsigned int baseIter = (IFMDim * ConvKernelDim
* multiplying_factor) + (OFMDim-1) *
max_cycles+std::max(cycles_write_block,OFMDim);
619     const unsigned int initial_buffer_cycles = (IFMDim *
ConvKernelDim * multiplying_factor) ;
620     unsigned int counter_internal_block = 0;
621     unsigned int current_line = 0;
622
623     unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0,
k_x = 0, count_simd = 0;
624
625     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
626         unsigned int floor_block_read = 0, ceil_block_read = num-
ber_blocks;
627         unsigned int current_block_write = 0;
628         #pragma HLS DEPENDENCE variable=current_block_write intra
false
629         unsigned int read_block = 0;
630         for (unsigned int i = 0; i < baseIter; i++) {
631             #pragma HLS pipeline style=flp II=1
632             if (inp < initial_buffer_cycles) //
Initial buffer of PoolDim lines
633             {
634                 ap_uint<SIMD*Input_precision>
inElem;
635                 inElem = in.read();
636                 for(unsigned int v = 0; v <
MMV; v++)
637                 {
638                     #pragma HLS UNROLL
639                     inputBuf[v][cur-
rent_block_write][current_line] = inElem;
640                 }
641                 current_line++;
642                 inp++;
643                 if (current_line == IFMDim *
multiplying_factor)
644                 {
645                     current_line = 0;
646                     current_block_write++;
647                     if (cur-
rent_block_write == number_blocks)
648                     cur-
rent_block_write = 0;
649                     read_block++;
650                     counter_internal_block =
0;
651                 }
652             }
653             else
654             {
655                 if (counter_internal_block <
cycles_write_block-1 || read_block==IFMDim) // We are writing out-
put, MMV IFMChan per cycle
656                 {
657                     //following code im-
plements: current_block_read = (ofm_y*Stride + k_y)%number_blocks;
658                     unsigned int current_block_read = (ofm_y*Stride +
k_y);
659                     //reminder computation
660                     if (current_block_read >= ceil_block_read)
661                     {
662                         floor_block_read += number_blocks;
663                         ceil_block_read += number_blocks;
664                     }else if(current_block_read < floor_block_read){
665                         ceil_block_read -= number_blocks;
666                         floor_block_read -= number_blocks;
667                     }
668                     current_block_read -= floor_block_read;
669                     unsigned int current_line_in_block =
(ofm_x * Stride + k_x)*multiplying_factor + count_simd;
670                     MultiChanData<MMV, SIMD*In-
put_precision> outElem;
671                     for(unsigned int v = 0; v < MMV; v++)
672                     {
673                         #pragma HLS UNROLL
674                         // each buffer's read addr is
offset by its buffer index

```

```

674         ap_uint<SIMD*Input_precision>
temp_value = inputBuf[v][current_block_read][(current_line_in_block
+ v*Stride*multiplying_factor)];
675         outElem.data[v] = temp_value;
676     }
677     out.write(outElem);
678     count_simd++;
679     if (count_simd == multiplying_factor)
680     {
681         count_simd=0;
682         k_x++;
683         if (k_x == ConvKernelDim) {
684             k_x = 0;
685             k_y++;
686             if (k_y == ConvKer-
nelDim) {
687                 k_y = 0;
688                 ofm_x += MMV;
689                 if (ofm_x ==
OFMDim) {
690                     ofm_x
= 0;
691                     ofm_y +=
OFMDim) {
692                         if
693                         {
694                             if ((counter_internal_block <
cycles_read_block - 1) && (read_block<IFMDim)) // In parallel we
write in the buffer, in the current block write if we still need to
695                             {
696                                 ap_uint<SIMD*Input_precision>
inElem;
697                                 inElem = in.read();
698                                 for(unsigned int v = 0; v <
MMV; v++) {
699                                     #pragma HLS UNROLL
700                                     inputBuf[v][cur-
rent_block_write][current_line] = inElem;
701                                     current_line++;
702                                     if (current_line == IFMDim *
multiplying_factor) // We read the whole block, we change the next
block in which we want to we
703                                     { // We filled up a block,
let's not read until
704                                     current_line = 0;
705                                     read_block++;
706                                     current_block_write++;
707                                     if (cur-
rent_block_write == number_blocks)
708                                     cur-
rent_block_write = 0;
709                                     #pragma HLS DEPENDENCE variable=current_block_write intra
false
710                                     counter_internal_block++; // =
(counter_internal_block +1) % max_cycles;
711                                     if (counter_internal_block ==
(max_cycles-1))
712                                     {
713                                         counter_internal_block = 0;
714                                     }
715                                     } // End base_iter
716                                 }
717                             }
718                             //**
719                             * \brief Sliding Window unit that produces output vectors
for feeding
720                             * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm for depthwise separable convolutions. To be used only if
721                             * ConvKernelDim%Stride = 0 and square kernel
722                             * \tparam ConvKernelDim Dimension of the convolutional
kernel (assumed square)
723                             * \tparam IFMChannels Number of Input Feature Maps
724                             * \tparam Input_precision Number bits per pixel
725                             * \tparam IFMDim Width and Height of the Input
Feature Map (assumed square)
726                             * \tparam OFMDim Width and Height of the Output
Feature Map (assumed square)
727                             * \tparam SIMD Number of input columns computed
in parallel

```

```

741 * \tparam Stride          Stride of the convolutional kernel
742 * \tparam R              Datatype for the resource used for
FPGA implementation of the SWG - safely deducible from the parameters
743 *
744 * \param in              Input stream
745 * \param out             Output stream
746 * \param numReps         Number of time the function has
to be repeatedly executed (e.g. number of images)
747 * \param r              Resource type for the hardware
ware implementation of the memory block
748 */
749 template<unsigned int ConvKernelDim,
750          unsigned int IFMChannels,
751          unsigned int Input_precision,
752          unsigned int IFMDim,
753          unsigned int OFMDim,
754          unsigned int SIMD,
755          unsigned int Stride,
756          typename R>
757 void ConvolutionInputGenerator_dws(
758     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
759     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
760     const unsigned int numReps,
761     R const &r) {
762     static_assert(IFMChannels % SIMD == 0, "");
763     static_assert(ConvKernelDim % Stride == 0, "");
764     const unsigned int multiplying_factor = IFMChannels/SIMD;
765     const unsigned int number_blocks = ConvKernelDim/Stride +
1;
766     ap_uint<SIMD*Input_precision> inputBuf[number_blocks]
[Stride * IFMDim * multiplying_factor];
767     #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
768     memory_resource(inputBuf, r);
769     const unsigned int cycles_write_block = (OFMDim * ConvKernelDim * multiplying_factor);
770     const unsigned int cycles_read_block = Stride * IFMDim * multiplying_factor;
771     const unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);
772     const unsigned int baseIter = IFMDim * ConvKernelDim * multiplying_factor // Initial buffer
+ OFMDim *
std::max(cycles_write_block, cycles_read_block);
774     unsigned int counter_internal_block = 0;
775     unsigned int current_block_write = 0;
776     unsigned int current_line = 0;
777     unsigned int read_block = 0;
778     unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0, k_x =
0, count_simd = 0;
779
780     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
781         for (unsigned int i = 0; i < baseIter; i++) {
782             #pragma HLS pipeline style=fpl II=1
783             if (inp < IFMDim * ConvKernelDim * multiplying_factor)
{ // Initial buffer of ConvKernelDim lines
784                 ap_uint<SIMD*Input_precision> inElem;
785                 inElem = in.read();
786                 inputBuf[current_block_write][current_line] = inElem;
787                 current_line++;
788                 inp++;
789                 if (current_line == Stride * IFMDim * multiplying_factor) {
790                     current_line = 0;
791                     current_block_write++;
792                     if (current_block_write == number_blocks) {
793                         current_block_write = 0;
794                     }
795                     read_block++;
796                     counter_internal_block = 0;
797                 }
798             } else {
799                 if (counter_internal_block < cycles_write_block-1)
{ // We are writing output, MMV IFMChan per cycle
800                     unsigned int current_block_read = (current_block_write + 1 + k_y / Stride);
801                     if (current_block_read >= number_blocks) {
802                         current_block_read = number_blocks;
803                     }
804                     unsigned int current_line_in_block = ((k_y%Stride)
* IFMDim + ofm_x*Stride + k_x)*multiplying_factor + count_simd;
805                     ap_uint<SIMD*Input_precision> outElem = inputBuf[current_block_read][current_line_in_block];
806                     out.write(outElem);
807                     k_x++;
808                     if (k_x == ConvKernelDim) {
809                         k_x = 0;
810                         k_y++;
811                         if (k_y == ConvKernelDim) {
812                             k_y = 0;
813                             count_simd++;
814                             if (count_simd == multiplying_factor) {
815                                 count_simd = 0;
816                                 ofm_x++;
817                                 if (ofm_x == OFMDim) {

```

```

818                 ofm_x = 0;
819                 ofm_y++;
820                 if (ofm_y == OFMDim) {
821                     ofm_y = 0;
822                     inp = 0;
823                 }
824             }
825         }
826     }
827 }
828 }

829     if ((counter_internal_block < cycles_read_block-1) &&
(read_block < IFMDim/Stride)) { // In parallel we write in the buffer, in the current block write if we still need to
830         ap_uint<SIMD*Input_precision> inElem;
831         inElem = in.read();
832         inputBuf[current_block_write][current_line] = inElem;
833     #pragma AP dependence variable=inputBuf intra false
834     #pragma AP dependence variable=inputBuf inter false
835         current_line++;
836         if (current_line == Stride * IFMDim * multiplying_factor) { // We read the whole block, we change the next block in which we want to we
837             // We filled up a block, let's not read until
838             current_line = 0;
839             read_block++;
840             current_block_write++;
841             if (current_block_write == number_blocks) {
842                 current_block_write = 0;
843             }
844     #pragma AP dependence variable=current_block_write intra false
845     }
846     }
847     counter_internal_block++; // =
(counter_internal_block + 1) % max_cycles;
848     if (counter_internal_block == (max_cycles-1)) {
849         counter_internal_block = 0;
850     }
851     } // End base_iter
852     read_block = 0;
853     } // End count_image
854     } // End generator
855 }

856 /**
857 * \brief Sliding Window unit that produces output vectors for feeding
858 * a Vector_Vector_Activate_Batch, implementing the im2col algorithm for depthwise separable convolutions. To be used when
859 * ConvKernelDim%Stride != 0 (e.g., Kernel=3, Stride=2)
860 */
861 * \tparam ConvKernelDim    Dimension of the convolutional kernel (assumed square)
862 * \tparam IFMChannels      Number of Input Feature Maps
863 * \tparam Input_precision  Number bits per pixel
864 * \tparam IFMDim           Width and Height of the Input Feature Map (assumed square)
865 * \tparam OFMDim           Width and Height of the Output Feature Map (assumed square)
866 * \tparam SIMD             Number of input columns computed in parallel
867 * \tparam Stride           Stride of the convolutional kernel
868 * \tparam R               Datatype for the resource used for FPGA implementation of the SWG - safely deducible from the parameters
869 *
870 * \param in              Input stream
871 * \param out             Output stream
872 * \param numReps         Number of time the function has to be repeatedly executed (e.g. number of images)
873 * \param r              Resource type for the hardware implementation of the memory block
874 */
875
876 template<unsigned int ConvKernelDim,
877          unsigned int IFMChannels,
878          unsigned int Input_precision,
879          unsigned int IFMDim,
880          unsigned int OFMDim,
881          unsigned int SIMD,
882          unsigned int Stride,
883          typename R>
884 void ConvolutionInputGenerator_kernel_stride_dws(
885     hls::stream<ap_uint<SIMD*Input_precision> > & in,
886     hls::stream<ap_uint<SIMD*Input_precision> > & out,
887     const unsigned int numReps,
888     R const &r) {
889     static_assert(IFMChannels % SIMD == 0, "");
890     static_assert(ConvKernelDim % Stride != 0, "");
891     constexpr unsigned multiplying_factor = IFMChannels/SIMD;
892     constexpr unsigned number_blocks = ConvKernelDim + Stride;
893     constexpr unsigned cycles_write_block = OFMDim * ConvKernelDim * multiplying_factor;

```



```

896     constexpr unsigned cycles_read_block = IFMDim * Stride *
multiplying_factor;
897     constexpr unsigned max_cycles =
std::max(cycles_write_block, cycles_read_block);
898     constexpr unsigned baseIter = (IFMDim * ConvKernelDim *
multiplying_factor) + (OFMDim-1) *
max_cycles+std::max(cycles_write_block,OFMDim);
899     constexpr unsigned initial_buffer_cycles = (IFMDim *
ConvKernelDim * multiplying_factor) ;
900
901     ap_uint<SIMD*Input_precision> inputBuf[number_blocks]
[IFMDim * multiplying_factor];
902     memory_resource(inputBuf, r);
903     #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
904     #pragma HLS DEPENDENCE variable=inputBuf inter false
905     #pragma HLS DEPENDENCE variable=inputBuf intra false
906     unsigned int counter_internal_block = 0;
907     unsigned int current_line = 0;
908     unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0, k_x
= 0, count_simd=0;
909
910     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
911         unsigned int floor_block_read = 0, ceil_block_read =
number_blocks;
912         unsigned int read_block = 0;
913         unsigned int current_block_write = 0;
914         for (unsigned int i = 0; i < baseIter; i++) {
915             #pragma HLS pipeline style=flp II=1
916
917             #pragma HLS DEPENDENCE variable=current_block_write intra
false
918
919             if (inp < initial_buffer_cycles) // Initial buf-
fer of PoolDim lines
920             {
921                 ap_uint<SIMD*Input_precision> inElem;
922                 inElem = in.read();
923
924                 inputBuf[current_block_write][current_line] =
inElem;
925                 current_line++;
926                 inp++;
927                 if (current_line == IFMDim * multiply-
ing_factor)
928                 {
929                     current_line = 0;
930                     current_block_write++;
931                     if (current_block_write == number_blocks)
932                         current_block_write = 0;
933                     read_block++;
934                     counter_internal_block = 0;
935                 }
936             }
937             else
938             {
939                 if (counter_internal_block <
cycles_write_block-1 || read_block==IFMDim) // We are writing out-
put, MMV IFMChan per cycle
940                 {
941                     //following code implements: current_block_read =
(ofm_y*Stride + k_y)%number_blocks;
942
943                     unsigned int current_block_read = (ofm_y*Stride +
k_y);
944                     //remainder computation
945                     if (current_block_read >= ceil_block_read)
946                     {
947                         floor_block_read += number_blocks;
948                         ceil_block_read += number_blocks;
949                     }
950                     else if (current_block_read < floor_block_read){
951                         ceil_block_read -= number_blocks;
952                         floor_block_read -= number_blocks;
953                     }
954                     current_block_read -= floor_block_read;
955
956                     unsigned int current_line_in_block =
(ofm_x * Stride + k_x)*multiplying_factor + count_simd;
957                     ap_uint<SIMD*Input_precision> outElem =
inputBuf[current_block_read][(current_line_in_block)];
958                     out.write(outElem);
959                     k_x++;
960                     if (k_x == ConvKernelDim) {
961                         k_x = 0;
962                         k_y++;
963                         if (k_y == ConvKernelDim) {
964                             k_y = 0;
965                             count_simd++;
966                             if (count_simd == multiply-
ing_factor) {
967                                 count_simd=0;
968                                 ofm_x++;
969                                 if (ofm_x == OFMDim) {
970                                     ofm_x = 0;
971                                     ofm_y++;
972                                     if (ofm_y == OFMDim) {
973                                         ofm_y = 0;
974                                         inp = 0;

```

```

973                                     }
974                                     }
975                                     }
976                                     }
977                                     }
978                                     if ((counter_internal_block <
cycles_read_block - 1) && (read_block<IFMDim)) // In parallel we
write in the buffer, in the current block write if we still need to
979                                     {
980                                         ap_uint<SIMD*Input_precision> inElem;
981                                         inElem = in.read();
982                                         inputBuf[current_block_write][cur-
rent_line] = inElem;
983                                         #pragma HLS DEPENDENCE variable=inputBuf inter false
984                                         #pragma HLS DEPENDENCE variable=inputBuf intra false
985                                         current_line++;
986                                         if (current_line == IFMDim * multiply-
ing_factor) // We read the whole block, we change the next block in
which we want to we
987                                         { // We filled up a block, let's not read
until
988                                             current_line = 0;
989                                             read_block++;
990                                             current_block_write++;
991                                             if (current_block_write == num-
ber_blocks)
992                                                 current_block_write = 0;
993                                         #pragma HLS DEPENDENCE variable=current_block_write intra
false
994                                         }
995                                     }
996                                     counter_internal_block++; // =
(counter_internal_block +1) % max_cycles;
997                                     if (counter_internal_block == (max_cycles-1))
998                                     {
999                                         counter_internal_block = 0;
1000                                     }
1001                                     } // End base_iter
1002                                     }
1003                                     }
1004                                     }
1005                                     }
1006                                     }
1007                                     /**
1008                                     * \brief Sliding Window unit that produces output vectors
for feeding
1009                                     * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm for depthwise separable convolutions with support to
multiple output pixels
1010                                     *
1011                                     * \tparam ConvKernelDim      Dimension of the convolutional
kernel (assumed square)
1012                                     * \tparam IFMChannels        Number of Input Feature Maps
1013                                     * \tparam Input_precision    Number bits per pixel
1014                                     * \tparam IFMDim            Width and Height of the Input
Feature Map (assumed square)
1015                                     * \tparam OFMDim            Width and Height of the Output
Feature Map (assumed square)
1016                                     * \tparam SIMD              Number of input columns computed
in parallel
1017                                     * \tparam Stride              Stride of the convolutional ker-
nel
1018                                     * \tparam MMV                Number of pixels that have to be
produced in parallel
1019                                     * \tparam R                  Datatype for the resource used for
FPGA implementation of the SWG - safely deducible from the parama-
ters
1020                                     *
1021                                     * \param in                  Input stream
1022                                     * \param out                 Output stream
1023                                     * \param numReps            Number of time the function has
to be repeatedly executed (e.g. number of images)
1024                                     * \param r                  Resource type for the hard-
ware implementation of the memory block
1025                                     */
1026                                     template<unsigned int ConvKernelDim,
unsigned int IFMChannels,
unsigned int Input_precision,
unsigned int IFMDim,
unsigned int OFMDim,
unsigned int SIMD,
unsigned int Stride,
typename R>
1027                                     void ConvolutionInputGenerator_dws_MMV(
1028                                     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
1029                                     hls::stream<MultiChanData<MMV, SIMD*In-
put_precision> > & out,
1030                                     const unsigned int numReps,
1031                                     R const &r) {
1032                                     static_assert(IFMChannels % SIMD == 0, "");
1033                                     static_assert(OFMDim % MMV == 0, "");
1034                                     static_assert(ConvKernelDim % Stride == 0, "");
1035                                     static_assert(MMV <= OFMDim, "");
1036                                     constexpr unsigned int multiplying_factor = IFMChan-
nels/SIMD;
1037                                     constexpr unsigned int number_blocks = ConvKernelDim/

```

```

Stride + 1 ;
1047 ap_uint<SIMD*Input_precision> inputBuf[MMV][number_blocks]
[Stride * IFMDim * multiplying_factor];
1048 #pragma HLS DEPENDENCE variable=inputBuf inter false
1049 #pragma HLS DEPENDENCE variable=inputBuf intra false
1050 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1051 #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=2
1052 memory_resource(inputBuf, r);
1053 constexpr unsigned int cycles_write_block = (OFMDim *
ConvKernelDim * ConvKernelDim * multiplying_factor)/MMV;
1054 constexpr unsigned int cycles_read_block = Stride *
IFMDim * multiplying_factor;
1055 constexpr unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);

1056 const unsigned int baseIter = IFMDim * ConvKernelDim *
multiplying_factor// Initial buffer
1057 + OFMDim *
std::max(cycles_write_block, cycles_read_block);
1058 unsigned int counter_internal_block = 0;
1059 unsigned int current_block_write = 0;
1060 unsigned int current_line = 0;
1061 unsigned int read_block = 0;
1062 unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0,
k_x = 0, count_simd = 0;
1063
1064 for (unsigned int count_image = 0; count_image < num-
Reps; count_image++) {
1065     for (unsigned int i = 0; i < baseIter; i++) {
1066         #pragma HLS pipeline style=flp II=1
1067         if (inp < IFMDim * ConvKer-
nelDim*multiplying_factor) // Initial
buffer of ConvKernelDim lines
1068         {
1069             ap_uint<SIMD*Input_precision>
inElem;
1070             inElem = in.read();
1071             for(unsigned int v = 0; v <
MMV; v++)
1072             {
1073                 #pragma HLS UNROLL
1074                 inputBuf[v][cur-
rent_block_write][current_line] = inElem;
1075             }
1076             current_line++;
1077             inp++;
1078             if (current_line == Stride *
IFMDim * multiplying_factor )
1079             {
1080                 current_line = 0;
1081                 current_block_write++;
1082                 if (cur-
rent_block_write == number_blocks)
1083                     cur-
rent_block_write=0;
1084                 read_block++;
1085                 counter_internal_block
= 0;
1086             }
1087         }
1088         else
1089         {
1090             if (counter_internal_block <
cycles_write_block-1) // We are writing output, MMV IFMChan per
cycle
1091             {
1092                 unsigned int cur-
rent_block_read = (current_block_write + 1 + k_y / Stride);
1093                 if (current_block_read
>= number_blocks)
1094                     cur-
rent_block_read -= number_blocks;
1095                 unsigned int cur-
rent_line_in_block = ((k_y*Stride) * IFMDim + ofm_x*Stride +
k_x)*multiplying_factor + count_simd;
1096                 MultiChanData<MMV,
SIMD*Input_precision> outElem;
1097                 // parallel read from
all input buffers
1098                 for(unsigned int v =
0; v < MMV; v++) {
1099                     #pragma HLS UNROLL
1100                     // each buf-
fer's read addr is offset by its buffer index
1101                     ap_uint<SIMD*Input_precision> temp_value = inputBuf[v][cur-
rent_block_read][(current_line_in_block +
v*Stride*multiplying_factor)];
1102                     outElem.data[v] = temp_value;
1103                 }
1104                 out.write(outElem);
1105                 k_x++;
1106                 if (k_x == ConvKer-
nelDim) {
1107                     k_x = 0;
1108                     k_y++;
1109                     if (k_y ==

```

```

ConvKernelDim) {
1110                                     k_y =
0;
1111                                     count_simd++;
1112                                     if
(count_simd == multiplying_factor) {
1113                                     count_simd=0;
1114                                     ofm_x += MMV;
1115                                     if (ofm_x == OFMDim) {
1116                                     ofm_x = 0;
1117                                     ofm_y++;
1118                                     if (ofm_y == OFMDim) {
1119                                     ofm_y = 0;
1120                                     inp = 0;
1121                                     }
1122                                     }
1123                                     }
1124                                     }
1125                                     }
1126                                     }
1127                                     if ((counter_internal_block <
cycles_read_block-1) && (read_block<IFMDim/Stride)) // In parallel
we write in the buffer, in the current block write if we still need
to
1128                                     {
1129                                     ap_uint<SIMD*Input_precision> inElem;
1130                                     inElem = in.read();
1131                                     for(unsigned int v =
0; v < MMV; v++) {
1132                     #pragma HLS UNROLL
1133                     inputBuf[v]
[current_block_write][current_line] = inElem;
1134                     #pragma AP dependence variable=inputBuf intra false
1135                     #pragma AP dependence variable=inputBuf inter false
1136                     }
1137                     current_line++;
1138                     if (current_line ==
Stride * IFMDim * multiplying_factor) // We read the whole block,
we change the next block in which we want to we
1139                     { // We filled up a
block, let's not read until
1140                     current_line =
0;
1141                     read_block++;
1142                     cur-
rent_block_write++;
1143                     if (cur-
rent_block_write == number_blocks)
1144                     cur-
rent_block_write=0;
1145                     #pragma AP dependence variable=current_block_write intra
false
1146                     }
1147                     }
1148                     }
1149                     counter_internal_block++; // =
(counter_internal_block +1) % max_cycles;
1150                     if (counter_internal_block ==
(max_cycles-1))
1151                     {
1152                     counter_internal_block
= 0;
1153                     }
1154                     } // base_iter
1155                     read_block = 0;
1156                     } // End count_image
1157                     } // End generator
1158                     }
1159                     }
1160                     }
1161                     /**
1162                     * \brief Sliding Window for 1x1 kernel with stride!=1
1163                     *
1164                     *
1165                     * Performs a downsampling of a 2D square image, removing

```

```

rows and columns
1166 *
1167 * \tparam IFMChannels      Number of Input Feature Maps
1168 * \tparam Input_precision Number bits per pixel
1169 * \tparam IFMDim          Width and Height of the Input
Feature Map (assumed square)
1170 * \tparam SIMD            Number of input columns computed
in parallel
1171 * \tparam Stride          Stride of the convolutional kernel
1172 *
1173 * \param in                Input stream
1174 * \param out               Output stream
1175 * \param numReps           Number of time the function has
to be repeatedly executed (e.g. number of images)
1176 */
1177 template< unsigned int IFMChannels,
1178           unsigned int Input_precision,
1179           unsigned int IFMDim,
1180           unsigned int SIMD,
1181           unsigned int Stride>
1182 void ConvolutionInputGenerator_2D_kernel1(
1183     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
1184     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
1185     const unsigned int numReps) {
1186     static_assert(IFMChannels % SIMD == 0, "");
1187     constexpr unsigned COUNTER_WIDTH = clog2(Stride-1) + 1;
1188     constexpr unsigned COUNTER_RESET = Stride - 2;
1189     constexpr unsigned MULTIPLYING_FACTOR = IFMChannels/SIMD;
1190     for (unsigned int im=0; im<numReps; im++) {
1191     #pragma HLS performance target_ti=IFMDim*IFMDim*IFMChannels/
SIMD
1192         ap_int<COUNTER_WIDTH> counter_y = -1;
1193         for (unsigned int y = 0; y < IFMDim; y++) {
1194             const bool keep_y = counter_y < 0;
1195             counter_y = keep_y ?
ap_int<COUNTER_WIDTH>(COUNTER_RESET) :
ap_int<COUNTER_WIDTH>(counter_y - 1);
1196             ap_int<COUNTER_WIDTH> counter_x = -1;
1197             for (unsigned int x = 0; x < IFMDim;
x++) {
1198                 #pragma HLS pipeline style=flp II=IFMChannels/SIMD
1199                 const bool keep_x = counter_x
< 0;
1200                 counter_x = keep_x ?
ap_int<COUNTER_WIDTH>(COUNTER_RESET) :
ap_int<COUNTER_WIDTH>(counter_x - 1);
1201                 for (unsigned int count_simd =
0; count_simd < MULTIPLYING_FACTOR; count_simd++) {
1202                     ap_uint<SIMD*Input_precision> inElem = in.read();
1203                     if (keep_y && keep_x)
{
1204                         out.write(inElem);
1205                     }
1206                 }
1207             }
1208         }
1209     }
1210 }
1211 /**
1212 * \brief Sliding Window for 1x1 kernel with stride!=1
1213 *
1214 * \brief Performs a downsampling of 1D data, removing elements at
stride
1215 *
1216 *
1217 * \tparam IFMChannels      Number of Input Feature Maps
1218 * \tparam Input_precision Number bits per pixel
1219 * \tparam IFMDim          Width of the Input Feature Map
(assumed 1D)
1220 * \tparam SIMD            Number of input columns computed
in parallel
1221 * \tparam Stride          Stride of the convolutional kernel
1222 *
1223 * \param in                Input stream
1224 * \param out               Output stream
1225 * \param numReps           Number of time the function has
to be repeatedly executed (e.g. number of images)
1226 */
1227 template< unsigned int IFMChannels,
1228           unsigned int Input_precision,
1229           unsigned int IFMDim,
1230           unsigned int SIMD,
1231           unsigned int Stride>
1232 void ConvolutionInputGenerator_1D_kernel1(
1233     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
1234     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
1235     const unsigned int numReps) {
1236     static_assert(IFMChannels % SIMD == 0, "");
1237     constexpr unsigned COUNTER_WIDTH = clog2(Stride-1) + 1;
1238     constexpr unsigned COUNTER_RESET = Stride - 2;
1239     for (unsigned int im=0; im<numReps; im++) {

```

```

1240         ap_int<COUNTER_WIDTH> counter_x = -1;
1241         for (unsigned int x = 0; x < IFMDim; x++) {
1242             const bool keep_x = counter_x < 0;
1243             counter_x = keep_x ?
ap_int<COUNTER_WIDTH>(COUNTER_RESET) :
ap_int<COUNTER_WIDTH>(counter_x - 1);
1244             for (unsigned int count_simd = 0;
count_simd < IFMChannels/SIMD; count_simd++) {
1245                 #pragma HLS pipeline style=flp II=1
1246                 ap_uint<SIMD*Input_precision>
inElem = in.read();
1247                 if (keep_x) {
1248                     out.write(inElem);
1249                 }
1250             }
1251         }
1252     }
1253 }
1254 *
1255 *
1256 /**
1257 * \brief Sliding Window unit that produces output vectors
for feeding
1258 * a Matrix_Vector_Activate_Batch, implementing the im2col
algorithm. To be used when kernel is not square
1259 *
1260 * \tparam ConvKernelDim_x   Dimension of the convolutional
kernel - x axis
1261 * \tparam ConvKernelDim_y   Dimension of the convolutional
kernel - y axis
1262 * \tparam IFMChannels      Number of Input Feature Maps
1263 * \tparam Input_precision Number bits per pixel
1264 * \tparam IFMDim_x         Width of the Input Feature Map
1265 * \tparam IFMDim_y         Height of the Input Feature
Map
1266 * \tparam OFMDim_x         Width of the Output Feature
Map
1267 * \tparam OFMDim_y         Height of the Output Feature
Map
1268 * \tparam SIMD            Number of input columns com-
puted in parallel
1269 * \tparam Stride_x         Stride of the convolutional
kernel - x axis
1270 * \tparam Stride_y         Stride of the convolutional
kernel - y axis
1271 * \tparam R                Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
1272 *
1273 * \param in                Input stream
1274 * \param out               Output stream
1275 * \param numReps           Number of time the function
has to be repeatedly executed (e.g. number of images)
1276 * \param r                 Re-
source type for the hardware implementation of the memory block
1277 */
1278 template<unsigned int ConvKernelDim_x,
1279           unsigned int ConvKernelDim_y,
1280           unsigned int IFMChannels,
1281           unsigned int Input_precision,
1282           unsigned int IFMDim_x,
1283           unsigned int IFMDim_y,
1284           unsigned int OFMDim_x,
1285           unsigned int OFMDim_y,
1286           unsigned int SIMD,
1287           unsigned int Stride_x,
1288           unsigned int Stride_y,
1289           typename R>
1290 void ConvolutionInputGenerator_NonSquare(
1291     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
1292     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
1293     const unsigned int numReps,
1294     R const &r) {
1295     static_assert(IFMChannels % SIMD == 0, "");
1296     const unsigned int multiplying_factor = IFMChannels/SIMD;
1297     const unsigned int number_blocks = ConvKernelDim_y/Stride_y
+ 1;
1298     ap_uint<SIMD*Input_precision> inputBuf[number_blocks]
[Stride_x * IFMDim_x * multiplying_factor];
1299     #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1300     memory_resource(inputBuf, r);
1301     const unsigned int cycles_write_block = (OFMDim_x * Con-
vKernelDim_x * ConvKernelDim_y * multiplying_factor);
1302     const unsigned int cycles_read_block = Stride_x * IFMDim_x
* multiplying_factor;
1303     const unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);
1304     const unsigned int baseIter = IFMDim_x * ConvKernelDim_y *
multiplying_factor// Initial buffer
+ OFMDim_y *
std::max(cycles_write_block, cycles_read_block);
1305     unsigned int counter_internal_block = 0;
1306     unsigned int current_block_write = 0;
1307     unsigned int current_line = 0;

```

```

1310 unsigned int read_block = 0;
1311 unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0, k_x =
0, count_simd = 0;
1312
1313 for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
1314     for (unsigned int i = 0; i < baseIter; i++) {
1315         #pragma HLS pipeline style=fllp II=1
1316         if (inp < IFMDim_x * ConvKernelDim_y * multiply-
ing_factor) { // Initial buffer of ConvKernelDim lines
1317             ap_uint<SIMD*Input_precision> inElem;
1318             inElem = in.read();
1319             inputBuf[current_block_write][current_line] = inElem;
1320             current_line++;
1321             inp++;
1322             if (current_line == Stride_x * IFMDim_x * multiply-
ing_factor) {
1323                 current_line = 0;
1324                 current_block_write++;
1325                 if (current_block_write == number_blocks) {
1326                     current_block_write=0;
1327                 }
1328                 read_block++;
1329                 counter_internal_block = 0;
1330             }
1331         } else {
1332             if (counter_internal_block < cycles_write_block-1)
{ // We are writing output, MMV IFMchan per cycle
1333                 unsigned int current_block_read = (cur-
rent_block_write + 1 + k_y / Stride_y);
1334                 if (current_block_read >= number_blocks) {
1335                     current_block_read = number_blocks;
1336                 }
1337                 unsigned int current_line_in_block =
((k_y*Stride_y) * IFMDim_y + ofm_x*Stride_x + k_x)*multiply-
ing_factor + count_simd;
1338                 ap_uint<SIMD*Input_precision> outElem = input-
Buf[current_block_read][current_line_in_block];
1339                 out.write(outElem);
1340                 count_simd++;
1341                 if (count_simd == multiplying_factor) {
1342                     count_simd=0;
1343                     k_x++;
1344                     if (k_x == ConvKernelDim_x) {
1345                         k_x = 0;
1346                         k_y++;
1347                         if (k_y == ConvKernelDim_y) {
1348                             k_y = 0;
1349                             ofm_x ++;
1350                             if (ofm_x == OFMDim_x) {
1351                                 ofm_x = 0;
1352                                 ofm_y++;
1353                                 if (ofm_y == OFMDim_y) {
1354                                     ofm_y = 0;
1355                                     inp = 0;
1356                                 }
1357                             }
1358                         }
1359                     }
1360                 }
1361             }
1362             if ((counter_internal_block < cycles_read_block-1) &&
(read_block<IFMDim_y/Stride_y)) { // In parallel we write in the
buffer, in the current block write if we still need to
1363                 ap_uint<SIMD*Input_precision> inElem;
1364                 inElem = in.read();
1365                 inputBuf[current_block_write][current_line] = in-
Elem;
1366                 #pragma AP dependence variable=inputBuf intra false
1367                 #pragma AP dependence variable=inputBuf inter false
1368                 current_line++;
1369                 if (current_line == Stride_x * IFMDim_x * multiply-
ing_factor) { // We read the whole block, we change the next block
in which we want to we
1370                     // We filled up a block, let's not read until
1371                     current_line = 0;
1372                     read_block++;
1373                     current_block_write++;
1374                     if (current_block_write == number_blocks) {
1375                         current_block_write=0;
1376                     }
1377                 }
1378                 #pragma AP dependence variable=current_block_write intra
false
1379             }
1380             counter_internal_block++; // =
(counter_internal_block + 1) % max_cycles;
1381             if (counter_internal_block == (max_cycles-1)) {
1382                 counter_internal_block = 0;
1383             }
1384         }
1385     } // End base_iter
1386     read_block = 0;
1387 } // End count_image
1388 } // End generator
1389
1390 /**
1391  * \brief Sliding Window unit that produces output vectors
for feeding

```

```

1392  * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm for depthwise separable convolutions. To be used when
kernel is not square
1393  *
1394  * \tparam ConvKernelDim_x      Dimension of the convolutional
kernel - x axis
1395  * \tparam ConvKernelDim_y      Dimension of the convolutional
kernel - y axis
1396  * \tparam IFMChannels          Number of Input Feature Maps
1397  * \tparam Input_precision      Number bits per pixel
1398  * \tparam IFMDim_x             Width of the Input Feature Map
1399  * \tparam IFMDim_y            Height of the Input Feature
Map
1400  * \tparam OFMDim_x            Width of the Output Feature
Map
1401  * \tparam OFMDim_y            Height of the Output Feature
Map
1402  * \tparam SIMD                Number of input columns com-
puted in parallel
1403  * \tparam Stride_x             Stride of the convolutional
kernel - x axis
1404  * \tparam Stride_y             Stride of the convolutional
kernel - y axis
1405  * \tparam R                    Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
1406  *
1407  * \param in                    Input stream
1408  * \param out                    Output stream
1409  * \param numReps                Number of time the function
has to be repeatedly executed (e.g. number of images)
1410  * \param r                      Re-
source type for the hardware implementation of the memory block
1411  */
1412 template<unsigned int ConvKernelDim_x,
1413          unsigned int ConvKernelDim_y,
1414          unsigned int IFMChannels,
1415          unsigned int Input_precision,
1416          unsigned int IFMDim_x,
1417          unsigned int IFMDim_y,
1418          unsigned int OFMDim_x,
1419          unsigned int OFMDim_y,
1420          unsigned int SIMD,
1421          unsigned int Stride_x,
1422          unsigned int Stride_y,
1423          typename R>
1424 void ConvolutionInputGenerator_NonSquare_dws(
1425     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
1426     hls::stream<ap_uint<SIMD*Input_precision> > &
out,
1427     const unsigned int numReps,
1428     R const &r) {
1429     static_assert(IFMChannels % SIMD == 0, "");
1430     const unsigned int multiplying_factor = IFMChannels/SIMD;
1431
1432     const unsigned int number_blocks = ConvKernelDim_y/Stride_y
+ 1;
1433     ap_uint<SIMD*Input_precision> inputBuf[number_blocks]
[Stride_x * IFMDim_x * multiplying_factor];
1434     #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1435     memory_resource(inputBuf, r);
1436     const unsigned int cycles_write_block = (OFMDim_x * Con-
vKernelDim_x * ConvKernelDim_y * multiplying_factor);
1437     const unsigned int cycles_read_block = Stride_x * IFMDim_x
* multiplying_factor;
1438     const unsigned int max_cycles =
std::max(cycles_write_block, cycles_read_block);
1439     const unsigned int baseIter = IFMDim_x * ConvKernelDim_y *
multiplying_factor // Initial buffer
+ OFMDim_y *
std::max(cycles_write_block, cycles_read_block);
1440     unsigned int counter_internal_block = 0;
1441     unsigned int current_block_write = 0;
1442     unsigned int current_line = 0;
1443     unsigned int read_block = 0;
1444     unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0, k_x =
0, count_simd = 0;
1445     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
1446         for (unsigned int i = 0; i < baseIter; i++) {
1447             #pragma HLS pipeline style=fllp II=1
1448             if (inp < IFMDim_x * ConvKernelDim_y * multiply-
ing_factor) { // Initial buffer of ConvKernelDim lines
1449                 ap_uint<SIMD*Input_precision> inElem;
1450                 inElem = in.read();
1451                 inputBuf[current_block_write][current_line] = inElem;
1452                 current_line++;
1453                 inp++;
1454                 if (current_line == Stride_x * IFMDim_x * multiply-
ing_factor) {
1455                     current_line = 0;
1456                     current_block_write++;
1457                     if (current_block_write == number_blocks) {
1458                         current_block_write=0;
1459                     }
1460                 }
1461             }

```

```

1462         read_block++;
1463         counter_internal_block = 0;
1464     }
1465     } else {
1466         if (counter_internal_block < cycles_write_block-1)
1467     { // We are writing output, MMV IFMChan per cycle
1468         unsigned int current_block_read = (current_block_write + 1 + k_y / Stride_y);
1469         if (current_block_read >= number_blocks) {
1470             current_block_read = number_blocks;
1471         }
1472         unsigned int current_line_in_block =
1473         ((k_y*Stride_y) * IFMDim_y + ofm_x*Stride_x + k_x)*multiplying_factor + count_simd;
1474         ap_uint<SIMD*Input_precision> outElem = inputBuf[current_block_read][(current_line_in_block)];
1475         out.write(outElem);
1476         k_x++;
1477         if (k_x == ConvKernelDim_x) {
1478             k_x = 0;
1479             k_y++;
1480             if (k_y == ConvKernelDim_y) {
1481                 k_y = 0;
1482                 count_simd++;
1483                 if (count_simd == multiplying_factor) {
1484                     count_simd=0;
1485                     ofm_x ++;
1486                     if (ofm_x == OFMDim_x) {
1487                         ofm_x = 0;
1488                         ofm_y++;
1489                         if (ofm_y == OFMDim_y) {
1490                             ofm_y = 0;
1491                             inp = 0;
1492                         }
1493                     }
1494                 }
1495             }
1496         }
1497         if ((counter_internal_block < cycles_read_block-1) && (read_block<IFMDim_y/Stride_y)) { // In parallel we write in the buffer, in the current block write if we still need to
1498             ap_uint<SIMD*Input_precision> inElem;
1499             inElem = in.read();
1500             inputBuf[current_block_write][current_line] = inElem;
1501         }
1502         #pragma AP dependence variable=inputBuf intra false
1503         #pragma AP dependence variable=inputBuf inter false
1504         current_line++;
1505         if (current_line == Stride_x * IFMDim_x * multiplying_factor) { // We read the whole block, we change the next block in which we want to we
1506             // We filled up a block, let's not read until
1507             current_line = 0;
1508             read_block++;
1509             current_block_write++;
1510             if (current_block_write == number_blocks) {
1511                 current_block_write=0;
1512             }
1513         }
1514         #pragma AP dependence variable=current_block_write intra false
1515     }
1516     }
1517     counter_internal_block++; // =
1518     (counter_internal_block + 1) % max_cycles;
1519     if (counter_internal_block == (max_cycles-1)) {
1520         counter_internal_block = 0;
1521     }
1522     } // End base_iter
1523     read_block = 0;
1524     } // End count_image
1525     } // End generator
1526     /**
1527     * \brief Sliding Window unit that produces output vectors for feeding
1528     * a Matrix_Vector_Activate_Batch, implementing the im2col algorithm. To be used when kernel is not square and with dilation
1529     * NOTE: Dilation over the Y axis not yet supported
1530     */
1531     * \tparam ConvKernelDim_x Dimension of the convolutional kernel - x axis
1532     * \tparam ConvKernelDim_y Dimension of the convolutional kernel - y axis
1533     * \tparam IFMChannels Number of Input Feature Maps
1534     * \tparam Input_precision Number bits per pixel
1535     * \tparam IFMDim_x Width of the Input Feature Map
1536     * \tparam IFMDim_y Height of the Input Feature Map
1537     * \tparam OFMDim_x Width of the Output Feature Map
1538     * \tparam OFMDim_y Height of the Output Feature Map
1539     * \tparam SIMD Number of input columns computed in parallel

```

```

1538     * \tparam Stride_x Stride of the convolutional kernel - x axis
1539     * \tparam Stride_y Stride of the convolutional kernel - y axis
1540     * \tparam Dilation_x Dilation the convolutional kernel - x axis
1541     * \tparam Dilation_y Dilation the convolutional kernel - y axis
1542     * \tparam R Datatype for the resource used for FPGA implementation of the SWG - safely deducible from the parameters
1543     *
1544     * \param in Input stream
1545     * \param out Output stream
1546     * \param numReps Number of time the function has to be repeatedly executed (e.g. number of images)
1547     * \param r Re-source type for the hardware implementation of the memory block
1548     */
1549     template<unsigned int ConvKernelDim_x,
1550             unsigned int ConvKernelDim_y,
1551             unsigned int IFMChannels,
1552             unsigned int Input_precision,
1553             unsigned int IFMDim_x,
1554             unsigned int IFMDim_y,
1555             unsigned int OFMDim_x,
1556             unsigned int OFMDim_y,
1557             unsigned int SIMD,
1558             unsigned int Stride_x,
1559             unsigned int Stride_y,
1560             unsigned int Dilation_x,
1561             unsigned int Dilation_y,
1562             typename R>
1563     void ConvolutionInputGenerator_NonSquare_Dilated(
1564         hls::stream<ap_uint<SIMD*Input_precision> > & in,
1565         hls::stream<ap_uint<SIMD*Input_precision> > & out,
1566         const unsigned int numReps,
1567         R const &r) {
1568         static_assert(IFMChannels % SIMD == 0, "");
1569         static_assert(Dilation_y == 1, ""); // Dilation on the Y axes not yet supported, available only for API definition
1570         const unsigned int multiplying_factor = IFMChannels/SIMD;
1571         const unsigned int number_blocks = (ConvKernelDim_y*Dilation_y)/Stride_y + 1;
1572         ap_uint<SIMD*Input_precision> inputBuf[number_blocks][Stride_x * IFMDim_x * multiplying_factor];
1573         #pragma HLS ARRAY_PARTITION variable=inputBuf complete dim=1
1574         memory_resource(inputBuf, r);
1575         const unsigned int cycles_write_block = (OFMDim_x * ConvKernelDim_x * ConvKernelDim_y * multiplying_factor);
1576         const unsigned int cycles_read_block = Stride_x * IFMDim_x * multiplying_factor;
1577         const unsigned int max_cycles = std::max(cycles_write_block, cycles_read_block);
1578         const unsigned int baseIter = IFMDim_x * ConvKernelDim_y * Dilation_y * multiplying_factor // Initial buffer
1579         + OFMDim_y * std::max(cycles_write_block, cycles_read_block);
1580         unsigned int counter_internal_block = 0;
1581         unsigned int current_block_write = 0;
1582         unsigned int current_line = 0;
1583         unsigned int read_block = 0;
1584         unsigned int inp = 0, ofm_y = 0, ofm_x = 0, k_y = 0, k_x = 0, count_simd = 0;
1585         for (unsigned int count_image = 0; count_image < numReps; count_image++) {
1586             for (unsigned int i = 0; i < baseIter; i++) {
1587                 #pragma HLS pipeline style=flp II=1
1588                 if (inp < IFMDim_x * ConvKernelDim_y * Dilation_y * multiplying_factor) { // Initial buffer of ConvKernelDim lines
1589                     ap_uint<SIMD*Input_precision> inElem;
1590                     inElem = in.read();
1591                     inputBuf[current_block_write][current_line] = inElem;
1592                     current_line++;
1593                     inp++;
1594                     if (current_line == Stride_x * IFMDim_x * multiplying_factor) {
1595                         current_line = 0;
1596                         current_block_write++;
1597                         if (current_block_write == number_blocks) {
1598                             current_block_write=0;
1599                         }
1600                     }
1601                     read_block++;
1602                     counter_internal_block = 0;
1603                 } else {
1604                     if (counter_internal_block < cycles_write_block-1)
1605                 { // We are writing output, IFMChan per cycle
1606                     unsigned int current_block_read = (current_block_write + 1 + (k_y*Dilation_y) / Stride_y);
1607                     if (current_block_read >= number_blocks) {
1608                         current_block_read = number_blocks;
1609                     }
1610                     unsigned int current_line_in_block =
1611                     ((ofm_x*Stride_x + k_x*Dilation_x)*multiplying_factor +

```



```

count_simd);
1613     ap_uint<SIMD*Input_precision> outElem;
1614     outElem = inputBuf[current_block_read][(cur-
rent_line_in_block)];
1615     out.write(outElem);
1616     count_simd++;
1617     if (count_simd == multiplying_factor) {
1618         count_simd=0;
1619         k_x++;
1620         if (k_x == ConvKernelDim_x) {
1621             k_x = 0;
1622             k_y++;
1623             if (k_y == ConvKernelDim_y) {
1624                 k_y = 0;
1625                 ofm_x++;
1626                 if (ofm_x == OFMDim_x) {
1627                     ofm_x = 0;
1628                     ofm_y++;
1629                     if (ofm_y == OFMDim_y) {
1630                         ofm_y = 0;
1631                         inp = 0;
1632                     }
1633                 }
1634             }
1635         }
1636     }
1637 }

1638 if ((counter_internal_block < cycles_read_block-1) &&
(read_block<IFMDim_y/Stride_y)) { // In parallel we write in the
buffer, in the current block write if we still need to
1639     ap_uint<SIMD*Input_precision> inElem;
1640     inElem = in.read();
1641     inputBuf[current_block_write][current_line] = in-
Elem;
1642 #pragma AP dependence variable=inputBuf intra false
1643 #pragma AP dependence variable=inputBuf inter false
1644     current_line++;
1645     if (current_line == Stride_x * IFMDim_x * multiply-
ing_factor) { // We read the whole block, we change the next block
in which we want to we
1646         // We filled up a block, let's not read until
1647         current_line = 0;
1648         read_block++;
1649         current_block_write++;
1650         if (current_block_write == number_blocks) {
1651             current_block_write=0;
1652         }
1653 #pragma AP dependence variable=current_block_write intra
false
1654     }
1655 }
1656 counter_internal_block++; // =
(counter_internal_block +1) % max_cycles;
1657 if (counter_internal_block == (max_cycles-1)) {
1658     counter_internal_block = 0;
1659 }
1660 } // End base_iter
1661 read_block = 0;
1662 } // End count_image
1663 } // End generator
1664 } // End generator
1665
1666 /**
1667  * \brief Sliding Window unit that produces output vectors
for feeding
1668  * a Matrix_Vector_Activate_Batch, implementing the im2col
algorithm.
1669  * To be used only for 1D feature maps.
1670  * Feeds all pixels of the window in parallel for full SIMD
unfolding of following layer.
1671  * NOTE: Currently restricted to: Stride = 1 and SIMD =
IFMChannels
1672  */

1673 * \tparam ConvKernelDim      Dimension of the convolutional
kernel
1674 * \tparam IFMChannels        Number of Input Feature Maps
1675 * \tparam Input_precision    Number bits per pixel
1676 * \tparam IFMDim            Height of the Input Feature
Map
1677 * \tparam OFMDim            Height of the Output Feature
Map
1678 * \tparam Stride            Stride of the convolution-
al kernel
1679 * \tparam SIMD              Number of input columns com-
puted in parallel
1680 * \tparam R                 Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
1681 *
1682 * \param in                  Input stream
1683 * \param out                 Output stream
1684 * \param numReps             Number of time the function
has to be repeatedly executed (e.g. number of images)
1685 * \param r
Resource type for the hardware implementation of the memory block
1686 */
1687 template<unsigned int ConvKernelDim,
unsigned int IFMChannels,
1688
1689     unsigned int Input_precision,
1690     unsigned int IFMDim,
1691     unsigned int OFMDim,
1692     unsigned int Stride,
1693     unsigned int SIMD,
1694     typename R>
1695 void ConvolutionInputGenerator_1D_parallel(
1696     hls::stream<ap_uint<SIMD*Input_precision> > &
in,
1697     hls::stream<ap_uint<ConvKernelDim*SIMD*Input_precision> > & out,
1698     const unsigned int numReps,
1699     __attribute__((unused)) R const &r) {
1700
1701     static_assert(Stride == 1, "");
1702     static_assert(IFMChannels % SIMD == 0, "");
1703     constexpr unsigned number_blocks = ConvKernelDim + 1 ;
1704     constexpr unsigned cycles_write_block = 1;
1705     constexpr unsigned cycles_read_block = 1;
1706     constexpr unsigned baseIter = ConvKernelDim // Initial
buffer
1707     + OFMDim *
std::max(cycles_write_block,cycles_read_block);
1708
1709     ap_uint<SIMD*Input_precision> inputBuf[number_blocks];
1710 #pragma HLS ARRAY_PARTITION variable=inputBuf complete
1711 //memory_resource(inputBuf, r); use reg regardless of set-
ting
1712
1713     unsigned int current_block_write = 0;
1714     unsigned int read_block = 0;
1715     unsigned int inp = 0, ofm_y = 0;
1716     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
1717         for (unsigned int i = 0; i < baseIter; i++) {
1718             #pragma HLS pipeline style=fllp II=1
1719             if (inp < ConvKernelDim) { // Initial buffer of ConvKer-
nelDim lines
1720                 ap_uint<SIMD*Input_precision> inElem;
1721                 inElem = in.read();
1722                 inputBuf[current_block_write] = inElem;
1723
1724                 inp++;
1725
1726                 current_block_write++;
1727                 if (current_block_write == number_blocks) {
1728                     current_block_write=0;
1729                 }
1730                 read_block++;
1731             }
1732         } else {
1733             // READING from buffer
1734
1735             ap_uint<ConvKernelDim*SIMD*Input_precision> outElem;
1736             for(int k_y=0; k_y<ConvKernelDim; k_y++)
1737             {
1738                 #pragma HLS UNROLL
1739                 unsigned int current_block_read = (cur-
rent_block_write + 1 + k_y);
1740                 if (current_block_read >= number_blocks) {
1741                     current_block_read= number_blocks;
1742                 }
1743                 outElem((k_y+1)*SIMD*Input_precision-1,
k_y*SIMD*Input_precision) = inputBuf[current_block_read];
1744             }
1745
1746             out.write(outElem);
1747
1748             ofm_y++;
1749             if (ofm_y == OFMDim) {
1750                 ofm_y = 0;
1751                 inp = 0;
1752             }
1753         }
1754
1755         if (read_block<IFMDim) {
1756             // WRITING to buffer
1757             ap_uint<SIMD*Input_precision> inElem;
1758             inElem = in.read();
1759             inputBuf[current_block_write] = inElem;
1760 #pragma AP dependence variable=inputBuf intra false
1761 #pragma AP dependence variable=inputBuf inter false
1762
1763             read_block++;
1764             current_block_write++;
1765             if (current_block_write == number_blocks) {
1766                 current_block_write=0;
1767             }
1768 #pragma AP dependence variable=current_block_write intra
false
1769         }
1770     }
1771 } // End base_iter
1772 read_block = 0;
1773 } // End count_image
1774 } // End generator
1775 } // End generator
1776
1777 /**
1778  * \brief Sliding Window unit that produces output vectors

```

```

for feeding
1779 * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm. To be used with 1D kernels
1780 *

1781 * \tparam ConvKernelDim_x      Dimension of the convolutional
kernel - x axis
1782 * \tparam IFMChannels          Number of Input Feature Maps
1783 * \tparam Input_precision      Number bits per pixel
1784 * \tparam IFMDim_x            Width of the Input Feature Map
1785 * \tparam OFMDim_x            Width of the Output Feature
Map
1786 * \tparam Stride_x             Stride of the convolutional
kernel - x axis

1787 * \tparam Dilation_x           Dilation of the convolutional
kernel - x axis
1788 * \tparam SIMD                 Number of input columns com-
puted in parallel
1789 * \tparam R                     Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
1790 *
1791 * \param in                     Input stream
1792 * \param out                     Output stream
1793 * \param numReps                 Number of time the function
has to be repeatedly executed (e.g. number of images)
1794 * \param r                       Re-
source type for the hardware implementation of the memory block
1795 */
1796 template<unsigned int ConvKernelDim_x,
1797          unsigned int IFMChannels,
1798          unsigned int Input_precision,
1799          unsigned int IFMDim_x,
1800          unsigned int OFMDim_x,
1801          unsigned int Stride_x,
1802          unsigned int Dilation_x,
1803          unsigned int SIMD,
1804          typename R>
1805 void ConvolutionInputGenerator_1D_dws_naive(
1806     hls::stream<ap_uint<SIMD*>Input_precision> > &
in,
1807     hls::stream<ap_uint<SIMD*>Input_precision> > &
out,
1808     const unsigned int numReps,
1809     R const &r) {
1810     static_assert(IFMChannels % SIMD == 0, "");
1811
1812     constexpr unsigned multiplying_factor = IFMChannels / SIMD;
1813     constexpr unsigned cycles_write_block = OFMDim_x * ConvKer-
nelDim_x * multiplying_factor;
1814     constexpr unsigned cycles_read_block = IFMDim_x * multiply-
ing_factor;
1815     ap_uint<SIMD*>Input_precision> inputBuf[cycles_read_block];
1816     memory_resource(inputBuf, r);
1817     constexpr unsigned baseIter = cycles_read_block // Initial
buffer
1818     +
cycles_write_block;
1819     unsigned int current_line = 0;
1820     unsigned int inp = 0, ofm_x = 0, k_x = 0, count_simd = 0;
1821
1822     for (unsigned int count_image = 0; count_image < numReps;
count_image++) {
1823         for (unsigned int i = 0; i < baseIter; i++) {
1824             #pragma HLS pipeline style=fllp II=1
1825             if (inp < cycles_read_block) { // Initial buffer of Con-
vKernelDim lines
1826                 ap_uint<SIMD*>Input_precision> inElem;
1827                 inElem = in.read();
1828                 inputBuf[current_line] = inElem;
1829                 current_line++;
1830                 inp++;
1831             }
1832             else {
1833                 unsigned int current_line_in_block = (ofm_x *
Stride_x + k_x * Dilation_x) * multiplying_factor + count_simd;
1834                 ap_uint<SIMD*>Input_precision> outElem = input-
Buf[current_line_in_block];
1835                 out.write(outElem);
1836                 k_x++;
1837                 if (k_x == ConvKernelDim_x) {
1838                     k_x = 0;
1839                     count_simd++;
1840                     if (count_simd == multiplying_factor) {
1841                         count_simd = 0;
1842                         ofm_x++;
1843                         if (ofm_x == OFMDim_x) {
1844                             ofm_x = 0;
1845                             inp = 0;
1846                         }
1847                     }
1848                 } // End else
1849             } // End base_iter
1850         } // End image
1851     } // End image
1852 }
1853
1854 /**
1855  * \brief Sliding Window unit that produces output vectors

```

```

for feeding
1856 * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm. To be used with 1D kernels
1857 *

1858 * \tparam ConvKernelDim_x      Dimension of the convolutional
kernel - x axis
1859 * \tparam IFMChannels          Number of Input Feature Maps
1860 * \tparam Input_precision      Number bits per pixel
1861 * \tparam IFMDim_x            Width of the Input Feature Map
1862 * \tparam OFMDim_x            Width of the Output Feature
Map
1863 * \tparam Stride_x             Stride of the convolutional
kernel - x axis
1864 * \tparam SIMD                 Number of input columns com-
puted in parallel
1865 * \tparam R                     Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
1866 *
1867 * \param in                     Input stream
1868 * \param out                     Output stream
1869 * \param numReps                 Number of time the function
has to be repeatedly executed (e.g. number of images)
1870 * \param r                       Re-
source type for the hardware implementation of the memory block
1871 */
1872 template<
1873     unsigned ConvKernelDim_x,
1874     unsigned IFMChannels,
1875     unsigned Input_precision,
1876     unsigned IFMDim_x,
1877     unsigned OFMDim_x,
1878     unsigned Stride_x,
1879     unsigned SIMD,
1880     typename R // Memory resource selector
1881 >
1882 void ConvolutionInputGenerator_1D(
1883     hls::stream<ap_uint<SIMD*>Input_precision> > &in,
1884     hls::stream<ap_uint<SIMD*>Input_precision> > &out,
1885     unsigned const numReps,
1886     R const &r) {
1887     static_assert((IFMChannels % SIMD) == 0, "SIMD paral-
lelism must divide the number of IFM channels");
1888     static_assert(OFMDim_x == ((IFMDim_x - ConvKer-
nelDim_x) / Stride_x + 1), "Unexpected OFM dimension");
1889
1890     constexpr unsigned SIMD_COUNT = IFMChannels / SIMD;
1891     constexpr unsigned BUFFER_SIZE = (ConvKernelDim_x -
1) * SIMD_COUNT;
1892     constexpr unsigned OUTPUT_SIZE = OFMDim_x * ConvKer-
nelDim_x * SIMD_COUNT;
1893     constexpr unsigned INPUT_SIZE = IFMDim_x *
SIMD_COUNT;
1894     constexpr unsigned WINDOW_SIZE = ConvKernelDim_x *
SIMD_COUNT;
1895     constexpr unsigned OCNT_INITIAL = BUFFER_SIZE +
(Stride_x - 1);
1896     ap_uint<SIMD*>Input_precision> buffer[BUFFER_SIZE];
1897     memory_resource(buffer, r);
1898     for (unsigned count_image = 0; count_image < numReps;
count_image++) {
1899         signed ocnt = OCNT_INITIAL < WINDOW_SIZE ?
OCNT_INITIAL : -1;
1900         unsigned wp = 0;
1901         unsigned rp = 0;
1902         unsigned offset = 0;
1903         unsigned inp_count = 0;
1904         for (unsigned i = 0; i < 1+OUTPUT_SIZE; i++) {
1905             #pragma HLS pipeline style=fllp II=1
1906             bool const re = i > 0;
1907             bool const we = (i < WINDOW_SIZE) ||
(ocnt < SIMD_COUNT * Stride_x);
1908             if (re) {
1909                 out.write(buffer[wp]);
1910                 if (++offset == WINDOW_SIZE) {
1911                     offset = 0;
1912                 }
1913                 rp += 1 + SIMD_COUNT *
(Stride_x - 1);
1914                 if (rp >= BUFFER_SIZE)
1915                     rp -= BUFFER_SIZE;
1916                 if (rp >= BUFFER_SIZE)
1917                     }
1918                 else { // Explicit else-block
1919                     required to work around bug in RTL simulation
1920                     if (++rp >= BUF-
FER_SIZE) rp -= BUFFER_SIZE;
1921                 }
1922                 if (++ocnt == WINDOW_SIZE)
1923                     if (we) {
1924                         if (++inp_count <= INPUT_SIZE)
1925                             buffer[wp] =
in.read();

```

```

1926                                     if(++wp == BUF-
FER_SIZE) wp = 0;
1927                                     }
1928                                     }
1929                                     }
1930                                     }
1931 }
1932
1933 /**
1934  * \brief Sliding Window unit that produces output vectors
for feeding
1935  * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm. To be used with 1D kernels
1936  */

1937 * \tparam ConvKernelDim_x      Dimension of the convolutional
kernel - x axis
1938 * \tparam IFMChannels          Number of Input Feature Maps
1939 * \tparam Input_precision      Number bits per pixel
1940 * \tparam IFMDim_x            Width of the Input Feature Map
1941 * \tparam OFMDim_x            Width of the Output Feature
Map
1942 * \tparam SIMD                Number of input columns com-
puted in parallel
1943 * \tparam R                    Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
1944 *
1945 * \param in                    Input stream
1946 * \param out                    Output stream
1947 * \param numReps                Number of time the function
has to be repeatedly executed (e.g. number of images)
1948 * \param r                      Re-
source type for the hardware implementation of the memory block
1949 */
1950 template<
1951     unsigned ConvKernelDim_x,
1952     unsigned IFMChannels,
1953     unsigned Input_precision,
1954     unsigned IFMDim_x,
1955     unsigned OFMDim_x,
1956     unsigned SIMD,
1957     typename R                // Memory resource selector
1958 >
1959 void ConvolutionInputGenerator_1D_dws(
1960     hls::stream<ap_uint<SIMD*Input_precision>> &in,
1961     hls::stream<ap_uint<SIMD*Input_precision>> &out,
1962     unsigned const numReps,
1963     R const &r
1964 ) {
1965     static_assert((IFMChannels % SIMD) == 0, "SIMD paral-
lelism must divide the number of IFM channels");
1966     static_assert(OFMDim_x == (IFMDim_x-ConvKer-
nelDim_x+1), "Unexpected OFM dimension");
1967
1968     constexpr unsigned SIMD_COUNT = IFMChannels / SIMD;
1969     constexpr unsigned BUFFER_SIZE = ConvKernelDim_x *
SIMD_COUNT;
1970     constexpr unsigned OUTPUT_SIZE = OFMDim_x * ConvKer-
nelDim_x * SIMD_COUNT;
1971     constexpr unsigned INPUT_SIZE = IFMDim_x *
SIMD_COUNT;
1972     constexpr unsigned READ_CYCLES = SIMD_COUNT * (Con-
vKernelDim_x- 1) - (ConvKernelDim_x - 1);
1973
1974     ap_uint<SIMD*Input_precision> buffer[BUFFER_SIZE];
1975     memory_resource(buffer, r);
1976
1977     for(unsigned count_image = 0; count_image < numReps;
count_image++) {
1978         unsigned ocnt = SIMD_COUNT;
1979         unsigned wp = 0;
1980         unsigned rp = 0;
1981         unsigned inp_count = 0;
1982         unsigned wcnt = 0;
1983         for(unsigned i = 0; i < 1 + READ_CYCLES +
OUTPUT_SIZE; i++) {
1984             #pragma HLS pipeline style=flp II=1
1985             bool const re = i > READ_CYCLES;
1986             bool const we = (i < BUFFER_SIZE) ||
(ocnt < SIMD_COUNT);
1987             if(re) {
1988                 out.write(buffer[rp]);
1989                 if(++wcnt == ConvKernelDim_x){
1990                     rp += SIMD_COUNT + 1;
1991                     wcnt = 0;
1992                 }
1993                 else{
1994                     rp += SIMD_COUNT;
1995                 }
1996                 if(rp >= BUFFER_SIZE) rp -=
BUFFER_SIZE;
1997                 if(++ocnt == BUFFER_SIZE)
ocnt = 0;
1998             }
1999             if(we) {
2000                 if(++inp_count <= INPUT_SIZE){
2001                     buffer[wp] =
in.read();
2002                     if(++wp == BUF-

```

```

FER_SIZE) wp = 0;
2003                                     }
2004                                     }
2005                                     }
2006                                     }
2007 }
2008
2009 /**
2010  * \brief Sliding Window unit that produces output vectors
for feeding
2011  * a Vector_Vector_Activate_Batch, implementing the im2col
algorithm. To be used with 1D kernels
2012  */

2013 * \tparam ConvKernelDim_x      Dimension of the convolutional
kernel - x axis
2014 * \tparam IFMChannels          Number of Input Feature Maps
2015 * \tparam Input_precision      Number bits per pixel
2016 * \tparam IFMDim_x            Width of the Input Feature Map
2017 * \tparam OFMDim_x            Width of the Output Feature
Map
2018 * \tparam Stride_x            Stride of the convolutional
kernel - x axis
2019 * \tparam SIMD                Number of input columns com-
puted in parallel
2020 * \tparam R                    Datatype for the re-
source used for FPGA implementation of the SWG - safely deducible
from the parameters
2021 *
2022 * \param in                    Input stream
2023 * \param out                    Output stream
2024 * \param numReps                Number of time the function
has to be repeatedly executed (e.g. number of images)
2025 * \param r                      Re-
source type for the hardware implementation of the memory block
2026 */
2027 template<
2028     unsigned ConvKernelDim_x,
2029     unsigned IFMChannels,
2030     unsigned Input_precision,
2031     unsigned IFMDim_x,
2032     unsigned OFMDim_x,
2033     unsigned Stride_x,
2034     unsigned SIMD,
2035     typename R                // Memory resource selector
2036 >
2037 void ConvolutionInputGenerator_1D_dws_stride(
2038     hls::stream<ap_uint<SIMD*Input_precision>> &in,
2039     hls::stream<ap_uint<SIMD*Input_precision>> &out,
2040     unsigned const numReps,
2041     R const &r
2042 ) {
2043     static_assert((IFMChannels % SIMD) == 0, "SIMD paral-
lelism must divide the number of IFM channels");
2044     static_assert(OFMDim_x == ((IFMDim_x - ConvKer-
nelDim_x) / Stride_x + 1), "Unexpected OFM dimension");
2045
2046     constexpr unsigned SIMD_COUNT = IFMChannels / SIMD;
2047     constexpr unsigned BUFFER_SIZE = ConvKernelDim_x *
SIMD_COUNT;
2048     constexpr unsigned OUTPUT_SIZE = OFMDim_x * ConvKer-
nelDim_x * SIMD_COUNT;
2049     constexpr unsigned INPUT_SIZE = IFMDim_x *
SIMD_COUNT;
2050     constexpr unsigned READ_CYCLES = SIMD_COUNT * (Con-
vKernelDim_x- 1) - (ConvKernelDim_x - 1);
2051
2052     ap_uint<SIMD*Input_precision> buffer[BUFFER_SIZE];
2053     memory_resource(buffer, r);
2054
2055     for(unsigned count_image = 0; count_image < numReps;
count_image++) {
2056         unsigned ocnt = SIMD_COUNT;
2057         unsigned wp = 0;
2058         unsigned rp = 0;
2059         unsigned offset = 0;
2060         unsigned inp_count = 0;
2061         unsigned wcnt = 0;
2062         for(unsigned i = 0; i < 1 + READ_CYCLES +
OUTPUT_SIZE; i++) {
2063             #pragma HLS pipeline style=flp II=1
2064             bool const re = i > READ_CYCLES;
2065             bool const we = (i < BUFFER_SIZE) ||
(ocnt < SIMD_COUNT * Stride_x);
2066             if(re) {
2067                 out.write(buffer[rp]);
2068                 if(++wcnt == ConvKernelDim_x){
2069                     if(++offset ==
SIMD_COUNT){
2070                         rp += Stride_x
* SIMD_COUNT + 1;
2071                         offset = 0;
2072                     }
2073                     else{
2074                         rp +=
SIMD_COUNT + 1;
2075                     }
2076                     wcnt = 0;
2077                 }

```



```

2078         else{
2079             rp += SIMD_COUNT;
2080         }
2081         if(rp >= BUFFER_SIZE) rp -=
BUFFER_SIZE;
2082         if(++ocnt == BUFFER_SIZE)
ocnt = 0;
2083     }
2084     if(we) {
2085         if(++inp_count <= INPUT_SIZE){
2086             buffer[wp] =
in.read();
2087             if(++wp == BUF-
FER_SIZE) wp = 0;
2088         }
2089     }
2090 }
2091 }
2092 }
2093 }
2094 }
2095 #endif

```

D.22 finn_hlslib/streamtools.h (1,008 lines, 35,734 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31 *****
32 /
*****
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 *          Thomas B. Preusser <thomas.preusser@utexas.edu>
37 *          Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 *          Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * @file stream-tools.h
41 *
42 * Library of templated HLS functions for BNN deployment.
43 * This file lists a set of convenience funtions used to ad-
apt stream size,
44 * remove unnecessary streams (padding) and casting

```

```

45 *
46 *****
*****/
47
48 #ifndef STREAMTOOLS_H
49 #define STREAMTOOLS_H
50
51 #include "ap_axi_sdata.h"
52
53 /**
54 * \brief Stream limiter - limits the number of stream
packets
55 *
56 * The block only let the first NumAllowed elements of a
stream to pass through, the remainder
57 * (NumTotal-NumAllowed) are consumed from input but not re-
mitted from the output.
58 * Useful to remove padding
59 *
60 * \tparam DataWidth Width, in number of bits, of the
input and output stream
61 * \tparam NumAllowed Number of words to pass through
62 * \tparam NumTotal Total number of words (NumAl-
lowed+NumDropped)
63 *
64 * \param in Input stream
65 * \param out Output stream
66 *
67 */
68 template<unsigned int DataWidth,
69          unsigned int NumAllowed,
70          unsigned int NumTotal
71 >
72 void StreamLimiter(hls::stream<ap_uint<DataWidth> > & in,
73                  hls::stream<ap_uint<DataWidth> > & out) {
74     static_assert(NumTotal >= NumAllowed, "");
75     unsigned int numLeft = NumAllowed;
76     for (unsigned int i = 0; i < NumTotal; i++) {
77         #pragma HLS pipeline style=fpl II=1
78         ap_uint<DataWidth> e = in.read();
79         if (numLeft > 0) {
80             out.write(e);
81             numLeft--;
82         }
83     }
84 }
85
86 /**
87 * \brief Stream limiter batch - limits the number of
stream packets multiple times
88 *
89 * The block only let the first NumAllowed elements of a
stream to pass through, the remainder
90 * (NumTotal-NumAllowed) are consumed from input but not re-
mitted from the output.
91 * Useful to remove padding on multiple images (numReps)
92 *
93 * \tparam DataWidth Width, in number of bits, of the
input and output stream
94 * \tparam NumAllowed Number of words to pass through
95 * \tparam NumTotal Total number of words (NumAl-
lowed+NumDropped)
96 *
97 * \param in Input stream
98 * \param out Output stream
99 *
100 * \param numReps Number of times the StreamLimiter
function has to be called
101 *
102 */
103 template<unsigned int DataWidth,
104          unsigned int NumAllowed,
105          unsigned int NumTotal
106 >
107 void StreamLimiter_Batch(hls::stream<ap_uint<DataWidth> > &
in,
108                          hls::stream<ap_uint<DataWidth> > & out, un-
signed int numReps) {
109     for (unsigned int rep = 0; rep < numReps; rep++) {
110         StreamLimiter<DataWidth, NumAllowed, NumTotal>(in, out);
111     }
112 }
113
114 /**
115 * \brief Stream Padding - Padds the input with zeroes for
when the sliding window is
116 * centered on border pixels
117 *
118 * Used to add padding to the input with zeroes in case the
sliding window is
119 * centered on border pixels
120 *
121 * \tparam ImgDim Size of the input feature map
122 * \tparam KernelDim Size of the sliding window
123 * \tparam Stride Stride of the sliding window
124 * \tparam NumChannels Amount of channels of the in-
put feature map
125 * \tparam In_t Input datatype
126 * \tparam PaddingStyle Type of padding that will be

```

```

applied
126 *
127 * \param      in      Input stream
128 * \param      out     Output stream
129 *
130 */
131 template<      unsigned int ImgDim,
132               unsigned int KernelDim,
133               unsigned int Stride,
134               unsigned int NumChannels,
135               typename In_t,
136               unsigned int PaddingStyle=2>

137 void SameResize(hls::stream<ap_uint<NumChannels> In_t::width> >
&in,
138               hls::stream<ap_uint<NumChannels> In_t::width>
> &out){
139
140     // Number of "same" windows over the input data

141     constexpr unsigned int SameWindows = (ImgDim) / Stride +
((ImgDim % Stride) > 0);
142
143     // Number of elements to generate as output per dimension
144     constexpr unsigned int OutputDim = KernelDim + Stride
* (SameWindows - 1);
145
146     // Padding
147     constexpr unsigned int Padding = OutputDim - ImgDim;
148
149     // Padding Up and Left
150     constexpr unsigned int PaddingUp = Padding/2 + ((Padding-
Style == 2) ? ((Padding % 2) > 0) : 0);
151     constexpr unsigned int PaddingLeft = Padding/2 + ((Padding-
Style == 2) ? ((Padding % 2) > 0) : 0);
152
153     // Padding Down and Right (might be 1 element more
than up and left in case of odd padding)
154     constexpr unsigned int PaddingDown = Padding - Paddin-
gUp;
155     constexpr unsigned int PaddingRight = Padding - Pad-
dingLeft;
156     ap_uint<NumChannels> In_t::width> outData, inData;
157
158     for(unsigned int y = 0; y<OutputDim; y++){
159         for(unsigned int x=0; x < OutputDim; x++){
160             #pragma HLS pipeline style=f1p II=1
161
162             // Padding Rows
163             if(y < PaddingUp || y >= (OutputDim -
PaddingDown)){
164                 outData = 0;
165             }
166             // Padding Cols
167             else if(x < PaddingLeft || x >= (Out-
putDim - PaddingRight)){
168                 outData = 0;
169             }
170             // No Padding
171             else{
172                 inData = in.read();
173                 outData = inData;
174             }
175
176             out.write(outData);
177         }
178     }
179 }
180
181 /**
182 * \brief Stream Padding - Padds the input of multiple
frames with zeroes

183 *      for when the sliding window is centered on border
pixels
184 *

185 * Used to add padding with zeroes to multiple inputs in case
the sliding window is
186 * centered on border pixels
187 *
188 * \tparam      ImgDim      Size of the input feature map
189 * \tparam      KernelDim   Size of the sliding window
190 * \tparam      Stride      Stride of the sliding window
191 * \tparam      NumChannels  Amount of channels of the in-
put feature map
192 * \tparam      In_t        Input datatype
193 * \tparam      PaddingStyle Type of padding that will be
applied
194 *
195 * \param      in      Input stream
196 * \param      out     Output stream
197 * \param      numReps   Amount of frames / images
198 *
199 */
200 template<      unsigned int ImgDim,
201               unsigned int KernelDim,
202               unsigned int Stride,
203               unsigned int NumChannels,

```

```

204               typename In_t,
205               unsigned int PaddingStyle=2>
206 void SameResize_Batch(hls::stream<ap_uint<NumChannels>
In_t::width> > &in,
207               hls::stream<ap_uint<NumChannels> In_t::width>
> &out,
208               const unsigned int numReps) {
209     for (unsigned int rep = 0; rep < numReps; rep++) {
210         SameResize<ImgDim, KernelDim, Stride, NumChan-
nels, In_t, PaddingStyle>(in, out);
211     }
212 }
213 }
214
215
216
217 /**
218 * \brief Stream cast - Casts the input stream to a differ-
ent datatype (OutT)
219 *
220 * Used to upscale or downscale a stream, enabling loss of
information for downscaling or
221 * 0 padding for upscaling
222 *
223 * \tparam      InT          Width, in number of bits, of the
input and output stream
224 * \tparam      OutT         Number of words to pass through
225 *
226 * \param      in      Input stream
227 * \param      out     Output stream
228 *
229 * \param      numReps   Number of times the StreamLimiter
function has to be called
230 *
231 */
232 template<typename InT, typename OutT>

233 void StreamingCast(hls::stream<InT> &in, hls::stream<OutT> &
out, unsigned int numReps) {
234     for(unsigned int i = 0; i < numReps; i++) {
235         #pragma HLS pipeline style=f1p II=1
236         out.write((OutT) in.read());
237     }
238 }
239 /**
240 * \brief FM Padding - Padds the input with zeroes for when
the sliding window is
241 * centered on border pixels
242 *
243 * Used to add padding with zeroes to multiple inputs in case
the sliding window is
244 * centered on border pixels - working on non-square images
and padding
245 *
246 * \tparam      OutputDim_x   Padded width of the output
feature map
247 * \tparam      OutputDim_y   Padded height of the output
feature map
248 * \tparam      PaddingLeft   Left image padding on
x-axis
249 * \tparam      PaddingRight  Right image padding on x-axis
250 * \tparam      PaddingTop    Top image padding on
y-axis
251 * \tparam      PaddingBottom Bottom image padding on y-axis
252 * \tparam      NumChannels   Number of channels of
the input feature map
253 * \tparam      SIMD          Input parallelism
254 * \tparam      In_t          Input datatype
255 *
256 * \param      in      Input stream
257 * \param      out     Output stream
258 *
259 */
260 template<
261     unsigned int OutputDim_x,
262     unsigned int OutputDim_y,
263     unsigned int PaddingLeft,
264     unsigned int PaddingRight,
265     unsigned int PaddingTop,
266     unsigned int PaddingBottom,
267     unsigned int NumChannels,
268     unsigned int SIMD,
269     typename In_t
270 >
271 void FMPadding_nonsquare(
272     hls::stream<ap_uint<SIMD*In_t::width>> &in,
273     hls::stream<ap_uint<SIMD*In_t::width>> &out
274 ){
275     static_assert(NumChannels%SIMD == 0, "Channel count
must be a SIMD multiple.");
276     constexpr unsigned int Folding = NumChannels/SIMD;
277
278     for(unsigned int y = 0; y < OutputDim_y; y++) {
279         for(unsigned int x = 0; x < OutputDim_x; x++) {
280             for(unsigned int sf = 0; sf < Folding;
sf++) {
281                 #pragma HLS pipeline style=f1p II=1
282                 ap_uint<SIMD*In_t::width>

```

```

outData = 0;
282
283 // Read & forward real data
only for non-padding image kernel
284 if(
285     /* rows */ (Padding-
Top <= y) && (y < OutputDim_y - PaddingBottom) &&
286     /* cols */ (Pad-
dingLeft <= x) && (x < OutputDim_x - PaddingRight)
287 ) {
288     outData = in.read();
289 }
290 out.write(outData);
291 }
292 }
293 }
294 }
295
296 /**
297  * \brief FM Padding Non Square - Padds the input of mul-
multiple frames with zeroes
298  *
299  * for when the sliding window is centered on border
pixels
300  *
301  * Used to add padding with zeroes to multiple inputs in case
the sliding window is
302  * centered on border pixels - working on non-square images
and padding
303  *
304  * \tparam OutputDim_x Padded width of the output
feature map
305  * \tparam OutputDim_y Padded height of the output
feature map
306  * \tparam PaddingLeft Left image padding on
x-axis
307  * \tparam PaddingRight Right image padding on x-axis
308  * \tparam PaddingTop Top image padding on
y-axis
309  * \tparam PaddingBottom Bottom image padding on y-axis
310  * \tparam NumChannels Number of channels of
the input feature map
311  * \tparam SIMD Input parallelism
312  * \tparam In_t Input datatype
313  * \param in Input stream
314  * \param out Output stream
315  * \param numReps Number of frames / images
316  */
317 template<
318     unsigned OutputDim_x,
319     unsigned OutputDim_y,
320     unsigned PaddingLeft,
321     unsigned PaddingRight,
322     unsigned PaddingTop,
323     unsigned PaddingBottom,
324     unsigned NumChannels,
325     unsigned SIMD,
326     typename In_t
327 >
328 void FMPadding_nonsquare_Batch(
329     hls::stream<ap_uint<SIMD*In_t::width>> &in,
330     hls::stream<ap_uint<SIMD*In_t::width>> &out,
331     unsigned const numReps
332 ) {
333     for (unsigned int rep = 0; rep < numReps; rep++) {
334         FMPadding_nonsquare<
335             OutputDim_x, OutputDim_y,
336             PaddingLeft, PaddingRight, PaddingTop,
337             PaddingBottom,
338             NumChannels, SIMD, In_t
339             >(in, out);
340     }
341 }
342 /**
343  * \brief FM Padding - Padds the input with zeroes for when
the sliding window is
344  * centered on border pixels
345  *
346  * Used to add padding to the input with zeroes in case the
sliding window is
347  * centered on border pixels
348  *
349  * \tparam ImgDim <ignored>
350  * \tparam OutputDim Size of the output
feature map
351  * \tparam PaddingBefore Top / left padding
352  * \tparam PaddingBehind Bottom / right padding
353  * \tparam NumChannels Number of channels of
the input feature map
354  * \tparam SIMD Input parallelism
355  * \tparam In_t Input datatype
356  *
357  * \param in Input stream
358  * \param out Output stream
359  *

```

```

360  */
361 template<
362     unsigned ImgDim,
363     unsigned OutputDim,
364     unsigned PaddingBefore,
365     unsigned PaddingBehind,
366     unsigned NumChannels,
367     unsigned SIMD,
368     typename In_t
369 >
370 void FMPadding(
371     hls::stream<ap_uint<SIMD*In_t::width>> &in,
372     hls::stream<ap_uint<SIMD*In_t::width>> &out
373 ){
374     #pragma HLS inline
375     FMPadding_nonsquare<
376         OutputDim, OutputDim,
377         PaddingBefore, PaddingBehind, PaddingBefore,
378         PaddingBehind,
379         NumChannels, SIMD, In_t
380         >(in, out);
381 }
382 /**
383  * \brief FM Padding - Padds the input of multiple frames
with zeroes
384  *
385  * for when the sliding window is centered on border
pixels
386  *
387  * Used to add padding with zeroes to multiple inputs in case
the sliding window is
388  * centered on border pixels
389  *
390  * \tparam ImgDim <ignored>
391  * \tparam OutputDim Size of the output
feature map
392  * \tparam PaddingBefore Top / left padding
393  * \tparam PaddingBehind Bottom / right padding
394  * \tparam NumChannels Number of channels of
the input feature map
395  * \tparam SIMD Input parallelism
396  * \tparam In_t Input datatype
397  *
398  * \param in Input stream
399  * \param out Output stream
400  * \param numReps Number of frames / images
401  */
402 template<
403     unsigned ImgDim,
404     unsigned OutputDim,
405     unsigned PaddingBefore,
406     unsigned PaddingBehind,
407     unsigned NumChannels,
408     unsigned SIMD,
409     typename In_t
410 >
411 void FMPadding_Batch(
412     hls::stream<ap_uint<SIMD*In_t::width>> &in,
413     hls::stream<ap_uint<SIMD*In_t::width>> &out,
414     unsigned const numReps
415 ) {
416     for (unsigned int rep = 0; rep < numReps; rep++) {
417         FMPadding<ImgDim, OutputDim, PaddingBefore,
418             PaddingBehind, NumChannels, SIMD, In_t>(in, out);
419     }
420 }
421 /**
422  * \brief Stream Data Width Converter - Converts the width
of the input stream in the output stream
423  *
424  * Used to upscale or downscale a stream, without any loss of
data in the procedure.
425  * For downscaling (InWidth > OutWidth), InWidth has to be a
multiple of OutWidth.
426  * For upscaling (InWidth < OutWidth), OutWidth has to be a
multiple of InWidth.
427  *
428  * \tparam InWidth Width, in number of bits, of the
input stream
429  * \tparam OutWidth Width, in number of bits, of the
output stream
430  * \tparam NumInWords Number of input words to process
431  *
432  * \param in Input stream
433  * \param out Output stream
434  * \param numReps Number of times the function has
to be called
435  */
436 template<unsigned int InWidth,
437     unsigned int OutWidth,
438     unsigned int NumInWords
439 >
440 void StreamingDataWidthConvert-
er_Batch(hls::stream<ap_uint<InWidth>> & in,
441     hls::stream<ap_uint<OutWidth>> & out, const

```

```

unsigned int numReps) {
442     static_assert((InWidth % OutWidth == 0) || (OutWidth % In-
Width == 0), "");
443
444     if (InWidth > OutWidth) {
445         // emit multiple output words per input word read
446         const unsigned int outPerIn = InWidth / OutWidth;
447         const unsigned int totalIters = NumInWords * outPerIn *
numReps;
448         unsigned int o = 0;
449         ap_uint<InWidth> ei = 0;
450         for (unsigned int t = 0; t < totalIters; t++) {
451             #pragma HLS pipeline style=fllp II=1
452             // read new input word if current out count is zero
453             if (o == 0) {
454                 ei = in.read();
455             }
456             // pick output word from the rightmost position
457             ap_uint<OutWidth> eo = ei(OutWidth - 1, 0);
458             out.write(eo);
459             // shift input to get new output word for next itera-
tion
460             ei = ei >> OutWidth;
461             // increment written output count
462             o++;
463             // wraparound indices to recreate the nested loop
structure
464             if (o == outPerIn) {
465                 o = 0;
466             }
467         } else if (InWidth == OutWidth) {
468             // straight-through copy
469             for (unsigned int i = 0; i < NumInWords * numReps; i++) {
470                 #pragma HLS pipeline style=fllp II=1
471                 ap_uint<InWidth> e = in.read();
472                 out.write(e);
473             }
474         } else { // InWidth < OutWidth
475             // read multiple input words per output word emitted
476             const unsigned int inPerOut = OutWidth / InWidth;
477             const unsigned int totalIters = NumInWords * numReps;
478             unsigned int i = 0;
479             ap_uint<OutWidth> eo = 0;
480             for (unsigned int t = 0; t < totalIters; t++) {
481                 #pragma HLS pipeline style=fllp II=1
482                 // read input and shift into output buffer
483                 ap_uint<InWidth> ei = in.read();
484                 eo = eo >> InWidth;
485                 eo(OutWidth - 1, OutWidth - InWidth) = ei;
486                 // increment read input count
487                 i++;
488                 // wraparound logic to recreate nested loop functional-
ity
489                 if (i == inPerOut) {
490                     i = 0;
491                     out.write(eo);
492                 }
493             }
494         }
495     }
496 }
497
498 /**
499  * \brief Stream Data Width Converter No Multiple -
500  * Converts the width of the input stream in the
output stream for no multiple dimensions
501  *
502  * Used to downscale a stream, without any loss of data in
the procedure.
503  * For downscaling (InWidth > OutWidth), InWidth has to be a
multiple of OutWidth.
504  *
505  * \tparam InWidth Width, in number of bits, of the
input stream
506  * \tparam OutWidth Width, in number of bits, of the
output stream
507  *
508  * \param in Input stream
509  * \param out Output stream
510  *
511  */
512 template<
513     unsigned int InWidth,
514     unsigned int OutWidth
515 >
516 void StreamingDataWidthConverterNoMultiple(
hls::stream<ap_uint<InWidth>> & in,
hls::stream<ap_uint<OutWidth>> & out) {
517     static_assert((InWidth % 2) == 0, "");
518     static_assert((OutWidth % 2) == 0, "");
519     static_assert(InWidth != OutWidth, "");
520     static unsigned int offset = 0;
521
522     if (InWidth > OutWidth) {
523         static ap_uint<OutWidth> remainder = 0;
524         ap_uint<InWidth> valueIn = in.read();
525
526         if (offset != 0) {
527             ap_uint<OutWidth> valueOut = 0;

```

```

531         valueOut = (valueIn(offset-1,0), remainder(OutWidth-
offset-1,0));
532         valueIn = valueIn(InWidth-1,offset); // leave the
next part prepared
533         out.write(valueOut);
534     }
535     for (; offset <= (InWidth-OutWidth) ; offset+=OutWidth)
{
536         ap_uint<OutWidth> valueOut = valueIn(OutWidth-1,0);
537         valueIn = valueIn(InWidth-1,OutWidth); // leave the
next part prepared
538         out.write(valueOut);
539     }
540     remainder = valueIn;
541     if (offset == InWidth)
542         offset = 0;
543     else
544         offset = offset + OutWidth - InWidth;
545 }
546 else {
547     /*OutWidth > InWidth*/
548     static ap_uint<InWidth> remainder = 0;
549     ap_uint<OutWidth> value = 0;
550     if (offset != 0) {
551         value(offset-1,0) = remainder(InWidth-1,InWidth-off-
set);
552     }
553     for (; offset <= (OutWidth-InWidth); offset+=InWidth){
554         ap_uint<InWidth> aux = in.read();
555         value(offset+InWidth-1,offset) = aux;
556     }
557     if (offset != OutWidth){
558         ap_uint<InWidth> aux = in.read();
559         value(OutWidth-1,offset) = aux(OutWidth-offset-1,0);
560         remainder = aux;
561         offset = offset + InWidth - OutWidth;
562     }
563     else
564         offset = 0;
565     out.write(value);
566 }
567 }
568 }
569
570 /**
571  * \brief Stream Duplicator - Reads in a stream and writes
the data into two identical streams
572  *
573  * Used to generate the inputs to the bypass and convolution-
al branches in Resnet-50
574  *
575  * \tparam DataWidth Width, in number of bits, of the
streams
576  * \tparam NumTotal Total number of words in the in-
put stream
577  *
578  * \param in Input stream
579  * \param out1 Output stream I
580  * \param out2 Output stream II
581  *
582  */
583 template<unsigned int DataWidth,
584     unsigned int NumTotal
585 >
586 void DuplicateStreams(hls::stream<ap_uint<DataWidth>> & in,
hls::stream<ap_uint<DataWidth>> & out1,
hls::stream<ap_uint<DataWidth>> & out2) {
587     for (unsigned int i = 0; i < NumTotal; i++) {
588         #pragma HLS pipeline style=fllp II=1
589         ap_uint<DataWidth> e = in.read();
590
591         out1.write(e);
592         out2.write(e);
593     }
594 }
595
596 /**
597  * \brief Batch Stream Duplicator - Reads in a stream mul-
tiple times and writes the data into two identical streams
598  *
599  * Used to generate the inputs to the bypass and convolution-
al branches in Resnet-50 when dealing with multiple 'frames'
600  *
601  * \tparam DataWidth Width, in number of bits, of the
streams
602  * \tparam NumTotal Total number of words in the in-
put stream
603  *
604  * \param in Input stream
605  * \param out1 Output stream I
606  * \param out2 Output stream II
607  * \param numReps Number of frames / images
608  *
609  */
610 template<unsigned int DataWidth,
611     unsigned int NumTotal
612 >
613 void DuplicateStreams_Batch(hls::stream<ap_uint<DataWidth>> &

```

```

& in, hls::stream<ap_uint<DataWidth>> & out1,

617     hls::stream<ap_uint<DataWidth>> & out2, const
unsigned int numReps) {
618     for (unsigned int image = 0; image < numReps; image++) {
619         DuplicateStreams<DataWidth, NumTotal>(in,
out1, out2);
620     }
621 }
622
623 /**
624  * \brief Element-Wise Addition - Reads in data elements
from two streams and writes the sum of these elements to an output
625  *
626  * \tparam NumChannels Amount of channels of the streams
627  * \tparam In1_t First operand datatype
628  * \tparam In2_t Second operand datatype
629  * \tparam Out_t Datatype of the accumulation out-
put
630  * \tparam NumTotal Total number of words in the in-
put streams
631  * \tparam offset Offset value for the accumulation
632  *
633  * \param in1 Input stream I
634  * \param in2 Input stream II
635  * \param out Output stream
636  *
637  */
638
639 template <unsigned int NumChannels,
640          typename In1_t,
641          typename In2_t,
642          typename Out_t,
643          unsigned int NumTotal,
644          int offset = 0>
645 void AddStreams(hls::stream<ap_uint<NumChannels> *
In1_t::width>> &in1, hls::stream<ap_uint<NumChannels> *
In2_t::width>> &in2,
646                hls::stream<ap_uint<NumChannels> *
Out_t::width>> &out) {
647     for (unsigned int i = 0; i < NumTotal; i++) {
648         #pragma HLS pipeline style=flp II=1
649         ap_uint<NumChannels> * In1_t::width> e1 = in1.read();
650         ap_uint<NumChannels> * In2_t::width> e2 = in2.read();
651         ap_uint<NumChannels> * Out_t::width> e;
652         for (unsigned int j = 0; j < NumChannels; j++) {
653             #pragma HLS UNROLL
654             In1_t op1 = e1((j + 1) * In1_t::width - 1, j *
In1_t::width);
655             In2_t op2 = e2((j + 1) * In2_t::width - 1, j *
In2_t::width);
656             Out_t sum = op1 + op2 + offset;
657             e((j + 1) * Out_t::width - 1, j * Out_t::width) = sum;
658         }
659         out.write(e);
660     }
661 }
662 }
663
664
665 /**
666  * \brief
667  *
668  * Used to implement point-wise addition in Resnet-50 for
multiple images
669  *
670  * \tparam NumChannels Amount of channels of the streams
671  * \tparam In1_t First operand datatype
672  * \tparam In2_t Second operand datatype
673  * \tparam Out_t Datatype of the accumulation out-
put
674  * \tparam NumTotal Total number of words in the in-
put streams
675  * \tparam offset Offset value for the accumulation
676  *
677  * \param in1 Input stream I
678  * \param in2 Input stream II
679  * \param out Output stream
680  * \param numReps Number of frames / images
681  *
682  */
683
684 template <unsigned int NumChannels,
685          typename In1_t,
686          typename In2_t,
687          typename Out_t,
688          unsigned int NumTotal,
689          int offset = 0>
690 void AddStreams_Batch(hls::stream<ap_uint<NumChannels> *
In1_t::width>> &in1, hls::stream<ap_uint<NumChannels> *
In2_t::width>> &in2,
691                      hls::stream<ap_uint<NumChannels> *
Out_t::width>> &out, const unsigned int numReps) {
692     for (unsigned int image = 0; image < numReps; image++) {
693         AddStreams<NumChannels, In1_t, In2_t, Out_t, NumTotal,
offset>(in1, in2, out);
694     }
695 }
696 /**

```

```

697  * \brief Addition Layer - Reads in two streams and writes
the sum of these streams to an output
698  *
699  * Used to merge the outputs of the bypass and convolutional
branches in Resnet-50
700  *
701  * \tparam NumChannels Amount of channels of the streams
702  * \tparam In1_t First operand datatype
703  * \tparam In2_t Second operand datatype
704  * \tparam Out_t Datatype of the accumulation out-
put
705  * \tparam NumTotal Total number of words in the input
streams
706  * \tparam PEOCount Amount of processing elements
working in parallel
707  * \tparam offset Offset value for the accumula-
tion
708  *
709  * \param in1 Input stream I
710  * \param in2 Input stream II
711  * \param out Output stream
712  * \param numReps Number of frames / images
713  */
714
715 template <unsigned int NumChannels,
716          typename In1_t,
717          typename In2_t,
718          typename Out_t,
719          unsigned int NumTotal,
720          unsigned int PEOCount,
721          int offset = 0>
722 void AddStreamsLayer_Batch(hls::stream<ap_uint<NumChannels> *
In1_t::width>> &in1, hls::stream<ap_uint<NumChannels> *
In2_t::width>> &in2,
723                           hls::stream<ap_uint<NumChannels> *
Out_t::width>> &out, const unsigned int numReps) {
724     #pragma HLS INLINE
725     static_assert(NumChannels % PEOCount == 0, "");
726     hls::stream<ap_uint<PEOCount> * In1_t::width>> in_folded1;
727     hls::stream<ap_uint<PEOCount> * In2_t::width>> in_folded2;
728     StreamingDataWidthConverter_Batch<NumChannels> *
In1_t::width, PEOCount * In1_t::width, NumTotal>(in1, in_folded1,
numReps);
729     StreamingDataWidthConverter_Batch<NumChannels> *
In2_t::width, PEOCount * In2_t::width, NumTotal>(in2, in_folded2,
numReps);
730     AddStreams_Batch<PEOCount, In1_t, In2_t, Out_t, NumTotal>
*(NumChannels / PEOCount), offset>(in_folded1, in_folded2, out_fol-
ded, numReps);
731     StreamingDataWidthConverter_Batch<PEOCount * Out_t::width,
NumChannels * Out_t::width, NumTotal>*(NumChannels /
PEOCount)>(out_folded, out, numReps);
732 }
733
734
735 /**
736  * \brief Stream Multi Chan Data Width Converter - Converts
the width of the input stream in the output stream, working on mul-
tiple parallel streams
737  *
738  * Used to upscale or downscale a stream, without any loss of
data in the procedure.
739  * For downscaling (InWidth > OutWidth), InWidth has to be a
multiple of OutWidth.
740  * For upscaling (InWidth < OutWidth), OutWidth has to be a
multiple of InWidth.
741  * This version works on the MMV structure, with multiple
parallel streams
742  *
743  * \tparam InWidth Width, in number of bits, of the
input stream
744  * \tparam OutWidth Width, in number of bits, of the
output stream
745  * \tparam NumInWords Number of input words to process
746  * \tparam NumVecs Number of parallel vectors MMV
747  *
748  * \param in Input stream
749  * \param out Output stream
750  * \param numReps Number of times the function has
to be called
751  *
752  */
753
754 template <unsigned int InWidth, // width of input
stream
755          unsigned int OutWidth, // width of
output stream
756          unsigned int NumInWords, // number of
input words to process
757          unsigned int NumVecs
758 >
759 void MultiChanDataWidthConverter_Batch(
760     hls::stream<MultiChanData<NumVecs, InWidth>> & in,
761     hls::stream<MultiChanData<NumVecs, OutWidth>> & out,
762     const unsigned int numReps) {
763     static_assert((InWidth % OutWidth == 0) || (OutWidth %
InWidth == 0), "");
764     if (InWidth > OutWidth) {

```



```

764          // emit multiple output words per input word
read
765          const unsigned int outPerIn = InWidth /
OutWidth;
766          const unsigned int totalIters = NumInWords *
outPerIn * numReps;
767          unsigned int o = 0;
768          MultiChanData<NumVecs, InWidth> ei;
769          for (unsigned int t = 0; t < totalIters; t++)
{
770      #pragma HLS pipeline style=flp II=1
771          // read new input word if current out
count is zero
772          if (o == 0)
773              ei = in.read();

774          // pick output word from the rightmost
position
775          MultiChanData<NumVecs, OutWidth> eo;
776          for(unsigned int v = 0; v < NumVecs;
v++) {
777      #pragma HLS UNROLL
778          eo.data[v] = (ei.data[v])
(OutWidth - 1, 0);
779          // shift input to get new out-
put word for next iteration
780          ei.data[v] = ei.data[v] >>
OutWidth;
781          }
782          out.write(eo);
783          // increment written output count
784          o++;
785          // wraparound indices to recreate the
nested loop structure
786          if (o == outPerIn) {
787              o = 0;
788          }
789          } else if (InWidth == OutWidth) {
790              // straight-through copy
791              for (unsigned int i = 0; i < NumInWords * num-
Reps; i++) {
792      #pragma HLS pipeline style=flp II=1
793          MultiChanData<NumVecs, InWidth> e =
in.read();
794          MultiChanData<NumVecs, OutWidth> eo;
795          // we don't support typecasting
796          between templated types, so explicitly
797          // transfer vector-by-vector here
798          for(unsigned int v=0; v < NumVecs; v+
+) {
799      #pragma HLS UNROLL
800          eo.data[v] = e.data[v];
801          out.write(eo);
802          }
803          } else { // InWidth < OutWidth
804              // read multiple input words per output word
emitted
805          const unsigned int inPerOut = OutWidth / In-
Width;
806          const unsigned int totalIters = NumInWords *
numReps;
807          unsigned int i = 0;
808          MultiChanData<NumVecs, OutWidth> eo;
809          for (unsigned int t = 0; t < totalIters; t++)
{
810      #pragma HLS pipeline style=flp II=1
811          // read input and shift into output
buffer
812          MultiChanData<NumVecs, InWidth> ei =
in.read();
813          for(unsigned int v = 0; v < NumVecs;
v++) {
814      #pragma HLS UNROLL
815          eo.data[v] = eo.data[v] >> In-
Width;
816          (eo.data[v])(OutWidth - 1,
OutWidth - InWidth) = ei.data[v];
817          // increment read input count
818          i++;
819          // wraparound logic to recreate nested
loop functionality
820          if (i == inPerOut) {
821              i = 0;
822              out.write(eo);
823          }
824          }
825          }
826          }
827          }
828          }
829          }
830          /**
831          * \brief Flatten Multi Chan Data - Converts the parallel
input stream in a flatten output stream
832          *
833          * Used to pack a flattened stream into a structure with mul-
tiple parallel streams

```

```

835          *
836          * \tparam NumChannels Number of channels flattened in
the input stream

837          * \tparam DataWidth Width, in number of bits, of each
stream
838          *
839          * \param in Input parallel stream
840          * \param out Output stream
841          * \param numReps Number of times the function has
to be called
842          *
843          */
844          template<unsigned int NumChannels, unsigned int DataWidth>
845          void FlattenMultiChanData(
846              hls::stream<MultiChanData<NumChannels, DataWidth> > &
in,
847              hls::stream<ap_uint<NumChannels*DataWidth> > & out,
848              const unsigned int numReps
849          ) {
850              for(unsigned int r = 0; r < numReps; r++) {
851      #pragma HLS pipeline style=flp II=1
852          MultiChanData<NumChannels, DataWidth> e =
in.read();
853          ap_uint<NumChannels*DataWidth> o = 0;
854          for(unsigned int v = 0; v < NumChannels; v++)
{
855      #pragma HLS UNROLL
856          o(DataWidth*(v+1)-1, DataWidth*v) =
e.data[v];
857          }
858          out.write(o);
859          }
860          }
861          /**
862          * \brief Pack Multi Chan Data - Converts the flatten input
stream into a parallel output stream
863          *
864          * Used to pack a flattened stream into a structure with mul-
tiple parallel streams
865          *
866          * \tparam NumChannels Number of channels flattened in
the input stream

867          * \tparam DataWidth Width, in number of bits, of each
stream
868          *
869          * \param in Input stream
870          * \param out Output parallel stream
871          * \param numReps Number of times the function has
to be called
872          *
873          */
874          template<unsigned int NumChannels, unsigned int DataWidth>
875          void PackMultiChanData(
876              hls::stream<ap_uint<NumChannels*DataWidth> > & in,
877              hls::stream<MultiChanData<NumChannels, DataWidth> > &
out,
878              const unsigned int numReps
879          ) {
880              for(unsigned int r = 0; r < numReps; r++) {
881      #pragma HLS pipeline style=flp II=1
882          ap_uint<NumChannels*DataWidth> e = in.read();
883          MultiChanData<NumChannels, DataWidth> o;
884          for(unsigned int v = 0; v < NumChannels; v++)
{
885      #pragma HLS UNROLL
886          o.data[v] = e(DataWidth*(v+1)-1,
DataWidth*v);
887          }
888          out.write(o);
889          }
890          }
891          }
892          }
893          }
894          template<unsigned IW, unsigned OW, unsigned N>
895          class WidthAdjustedInputStream {
896              hls::stream<ap_uint<OW>> m_target;
897          public:
898              WidthAdjustedInputStream(hls::stream<ap_uint<IW> >&
source, unsigned const reps) {
899                  StreamingDataWidthConverter_Batch<IW, OW, N>(source,
m_target, reps);
900              }
901              ~WidthAdjustedInputStream() {}
902          public:
903              operator hls::stream<ap_uint<OW> >&() {
904                  return m_target;
905              }
906          };
907          template<unsigned W, unsigned N>
908          class WidthAdjustedInputStream<W, W, N> {
909              hls::stream<ap_uint<W>> &m_source;
910          public:

```

```

915 WidthAdjustedInputStream(hls::stream<ap_uint<W> >& source,
    __attribute__((unused)) unsigned const reps) : m_source(source) {}
916 ~WidthAdjustedInputStream() {}
917
918 public:
919 operator hls::stream<ap_uint<W> >&() {
920     return m_source;
921 }
922 };
923
924
925 template<unsigned IW, unsigned OW, unsigned N>
926 class WidthAdjustedOutputStream {
927     hls::stream<ap_uint<IW> > m_buffer;
928     hls::stream<ap_uint<OW> > &m_target;
929     unsigned const m_reps;
930
931 public:
932     WidthAdjustedOutputStream(hls::stream<ap_uint<OW> >& target,
    unsigned const reps) : m_target(target), m_reps(reps) {}
933     ~WidthAdjustedOutputStream() {
934         StreamingDataWidthConverter_Batch<IW, OW, N>(m_buffer,
    m_target, m_reps);
935     }
936
937 public:
938 operator hls::stream<ap_uint<IW> >&() {
939     return m_buffer;
940 }
941 };
942
943 template<unsigned W, unsigned N>
944 class WidthAdjustedOutputStream<W, W, N> {
945     hls::stream<ap_uint<W> > &m_target;
946
947 public:
948     WidthAdjustedOutputStream(hls::stream<ap_uint<W> >& target,
    __attribute__((unused)) unsigned const reps) : m_target(target) {}
949     ~WidthAdjustedOutputStream() {}
950
951 public:
952 operator hls::stream<ap_uint<W> >&() {
953     return m_target;
954 }
955 };
956
957 /**
958  * \brief QDMA stream to normal stream conversion - Reads
    in a QDMA stream and strips metadata (TLAST, TKEEP)
959  *
960  * Used as an adapter when connecting blocks through top-
    level Vitis streams (kernel to kernel or host to platform streaming)
961  *
962  * \tparam DataWidth Width, in number of bits, of the
    data on streams
963  * \tparam NumTotal Total number of words in the in-
    put stream
964  *
965  * \param in Input stream
966  * \param out Output stream
967  * \param numReps Number of frames / images
968  */
969
970 template<unsigned int DataWidth, unsigned int NumTotal>
971 void Qdma2Stream_Batch(hls::stream<qdma_axis<DataWidth,0,0,0> >
    & in, hls::stream<ap_uint<DataWidth> > & out, const unsigned int
    numReps){
972     //TODO: static_assert to ensure DataWidth is power of
    2 between 8 and 512
973
974     for (unsigned int image = 0; image < numReps; image++) {
975         for (unsigned int word = 0; word < NumTotal;
    word++) {
976             #pragma HLS pipeline style=flp II=1
977             out.write(in.read().get_data());
978         }
979     }
980
981 /**
982  * \brief Normal stream to QDMA stream conversion - Reads
    in a stream and outputs a QDMA stream including metadata (TLAST,
    TKEEP)
983  *
984  * Used as an adapter when connecting blocks through top-
    level Vitis streams (kernel to kernel or host to platform streaming)
985  *
986  * \tparam DataWidth Width, in number of bits, of the
    data on streams
987  * \tparam NumTotal Total number of words in the in-
    put stream
988  *
989  * \param in Input stream
990  * \param out Output stream
991  * \param numReps Number of frames / images
992  */
993
994 template<unsigned int DataWidth, unsigned int NumTotal>

```

```

995 void Stream2Qdma_Batch(hls::stream<ap_uint<DataWidth> > & in,
    hls::stream<qdma_axis<DataWidth,0,0,0> > & out, const unsigned int
    numReps){
996     for (unsigned int image = 0; image < numReps; image++) {
997         for (unsigned int word = 0; word < NumTotal;
    word++) {
998             #pragma HLS pipeline style=flp II=1
999             qdma_axis<DataWidth,0,0,0> temp;
1000             temp.set_data(in.read());
1001             temp.set_keep(-1);
1002             temp.set_last(word == NumTotal-1);
1003             out.write(temp);
1004         }
1005     }
1006 }
1007
1008 #endif

```

D.23 finn_hlslib/tmrcheck.hpp (204 lines, 9,581 B)

```

1 /
*****
2 * Copyright (c) 2021, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
    or without
6 * modification, are permitted provided that the following
    conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
    copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
    above copyright
12 * notice, this list of conditions and the following
    disclaimer in the
13 * documentation and/or other materials provided with
    the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
    names of its
16 * contributors may be used to endorse or promote
    products derived from
17 * this software without specific prior written permis-
    sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
    CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
    LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
    PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
    HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
    ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
    LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
    DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
    ORY OF LIABILITY,
27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
    NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
    SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31
32
33 /
*****
34 *
35 * Authors: Timoteo Garcia Bertoa <timoteog@xilinx.com>
36 *
37 * \file tmrcheck.hpp
38 *
39 * Library of templated HLS functions for BNN deployment.
40 * This file performs error checks to triplicated channels
    of an OFM
41 *
42
43
44 #ifndef TMR_HPP
45 #define TMR_HPP
46

```

```

47 #include "hls_stream.h"
48
49 /**
50  * \brief Smart TMR block
51  *
52  * The function receives an OFM from the MVTU, with triplic-
53  * ated channels. It outputs a single channel for each triplication,
54  * being this channel one which contains valid data. A flag
55  * is also available to watch the character of the error detected,
56  * either all channels in a triplication are different, or
57  * just one.
58  *
59  * \tparam InW          Input data width, activation pre-
60  * cision
61  * \tparam OFMChannels  Number of Output Feature Map
62  * channels, including triplications
63  * \tparam NUM_RED      Number of redundancies (or trip-
64  * licated channels)
65  * \tparam REDF         Redundancy factor (3 to triplic-
66  * ate)
67  * \tparam OFMDim       Width and Height of the Output
68  * Feature Map (assumed square)
69  * \tparam MAX_CH_WIDTH Value to determine the precision
70  * of channel indexes
71  *
72  * \param in            Input stream
73  * \param out           Output stream
74  * \param errortype     Flag to inform redundancy check
75  * results. 0 if no faults, LSB set if one PE is faulty, MSB set if
76  * all differ
77  * \param channel_mask  Value with binary channel masks
78  * (1 if channel is triplicated, 0 otherwise)
79  * \param red_ch_index  Array of redundant triplets' in-
80  * dexes. Each position stores the first triplicated channel index of
81  * a triplet
82  */
83 template<unsigned int InW,
84         unsigned int OFMChannels,
85         unsigned int NUM_RED,
86         unsigned int REDF,
87         unsigned int OFMDim,
88         unsigned int MAX_CH_WIDTH>
89 void TMRCheck(hls::stream<ap_uint<InW*OFMChannels>> &in,
90              hls::stream<ap_uint<InW*(OFMChannels-
91 NUM_RED*(REDF-1))>> &out,
92              ap_uint<2> &errortype,
93              ap_uint<OFMChannels> channel_mask,
94              ap_uint<MAX_CH_WIDTH> red_ch_index[NUM_RED]) {
95
96 #pragma HLS ARRAY_PARTITION variable=red_ch_index complete
97 dim=0
98
99     ap_uint<InW*OFMChannels> input;
100
101     // Number of channels without triplications
102     constexpr unsigned int OFMChannelsTMR = (OFMChannels-
103 NUM_RED*(REDF-1));
104     errortype = 0;
105
106     // CheckLoop: iterates over all OFM positions
107     for(unsigned int pos = 0; pos < (OFMDim * OFMDim); pos++)
108     {
109 #pragma HLS pipeline style=flp II=1
110
111         // Read input stream
112         input = in.read();
113
114         ap_uint<InW*NUM_RED> tmr_out = {0};
115         ap_uint<InW*OFMChannelsTMR> out_aux = {0};
116         ap_uint<2> numerrors[NUM_RED];
117 #pragma HLS ARRAY_PARTITION variable=numerrors complete dim=0
118
119         // Check triplicated channel indexes and store the
120         // corresponding data to perform TMR check
121         for(unsigned int i = 0; i < NUM_RED; i++){
122 #pragma HLS UNROLL
123
124             numerrors[i] = 0;
125             // TMR CHECK: start
126             // Store index of triplicated channel
127             unsigned int idx = red_ch_index[i];
128
129             // CompareLoop: performs comparisons between PE0,
130             // PE1, PE2
131             for(unsigned int y = 0; y < REDF; y++){
132                 for(unsigned int x = y+1; x < REDF; x++){
133
134                     if( (input((idx+y+1)*InW-1, (idx+y)*InW))
135 == (input((idx+x+1)*InW-1, (idx+x)*InW)) ){
136                         tmr_out((i+1)*InW-1, i*InW) = in-
137 put((idx+x+1)*InW-1, (idx+x)*InW);
138                     } else {
139                         numerrors[i]++;
140                         if(numerrors[i] == REDF){
141                             errortype |= (ap_uint<2>)0b10;
142                         } else {
143                             errortype |= (ap_uint<2>)0b1;
144                         }
145                     }
146                     tmr_out((i+1)*InW-1, i*InW) = in-
147 put((idx+x+1)*InW-1, idx*InW);

```

```

122         }
123     } // end CompareLoop
124 } // end Check triplicated channel indexes
125
126     ap_uint<OFMChannels> unitl = 1;
127     ap_uint<1> compute[OFMChannels];
128 #pragma HLS ARRAY_PARTITION variable=compute complete dim=0
129 // ChannelLoop: iterates over all OFM channels (in-
130 cluding triplications), and outputs either: TMR check output/input/
131 nothing
132 for(unsigned int k = 0; k < OFMChannels; k++){
133 #pragma HLS UNROLL
134     compute[k] = 0;
135     // Check if current channel is any of the FIRST
136     triplicated
137     for(unsigned int i = 0; i < NUM_RED; i++){
138         // Store index of triplicated channel
139         unsigned int idx = red_ch_index[i];
140         if(k == idx){
141             compute[k] = 1;
142         }
143     }
144
145     // If it is one of the first triplicated, forward
146     the TMR check output, which contains valid data
147     if(compute[k]){
148         out_aux = out_aux >> InW;
149         out_aux(OFMChannelsTMR*InW-1, (OFMChannelsT-
150 MR-1)*InW) = tmr_out(InW-1, 0);
151         tmr_out = tmr_out >> InW;
152         // If it is not a first triplicated channel,
153         check if it is a triplicated or not using mask
154         } else if((channel_mask & (unitl << k)) != 0){
155             // Nothing to do, skip triplicated channel
156         } // If it is a not triplicated channel, forward
157         the input data
158         } else {
159             out_aux = out_aux >> InW;
160             out_aux(OFMChannelsTMR*InW-1, (OFMChannelsT-
161 MR-1)*InW) = input((k+1)*InW-1, k*InW);
162         }
163     } // end ChannelLoop
164
165     out.write(out_aux);
166 } // end CheckLoop
167 } // end TMRCheck
168
169 /**
170  * \brief Smart TMR block (batch)
171  *
172  * The function receives an OFM from the MVTU, with triplic-
173  * ated channels. It outputs a single channel for each triplication,
174  * being this channel one which contains valid data. A flag
175  * is also available to watch the character of the error detected,
176  * either all channels in a triplication are different, or
177  * just one.
178  *
179  * \tparam InW          Input data width, activation pre-
180  * cision
181  * \tparam OFMChannels  Number of Output Feature Map
182  * channels, including triplications
183  * \tparam NUM_RED      Number of redundancies (or trip-
184  * licated channels)
185  * \tparam REDF         Redundancy factor (3 to triplic-
186  * ate)
187  * \tparam OFMDim       Width and Height of the Output
188  * Feature Map (assumed square)
189  * \tparam MAX_CH_WIDTH Value to determine the precision
190  * of channel indexes
191  *
192  * \param in            Input stream
193  * \param out           Output stream
194  * \param errortype     Flag to inform redundancy check
195  * results. 0 if no faults, LSB set if one PE is faulty, MSB set if
196  * all differ
197  * \param channel_mask  Value with binary channel masks
198  * (1 if channel is triplicated, 0 otherwise)
199  * \param red_ch_index  Array of redundant triplets' in-
200  * dexes. Each position stores the first triplicated channel index of
201  * a triplet
202  * \param numReps       Number of time the function has
203  * to be repeatedly executed (e.g. number of images)
204  */
205
206 template<unsigned int InW,
207         unsigned int OFMChannels,
208         unsigned int NUM_RED,
209         unsigned int REDF,
210         unsigned int OFMDim,
211         unsigned int MAX_CH_WIDTH>
212 void TMRCheck_Batch(hls::stream<ap_uint<InW*OFMChannels>>
213 &in,
214                    hls::stream<ap_uint<InW*(OFMChannels-
215 NUM_RED*(REDF-1))>> &out,
216                    ap_uint<2> &errortype,
217                    ap_uint<OFMChannels> channel_mask,

```



```

196         ap_uint<MAX_CH_WIDTH> red_ch_in-
dex[NUM_RED],
197         unsigned int numReps) {
198
199     for (unsigned int rep = 0; rep < numReps; rep++) {
200         TMRCheck<InW, OFMChannels, NUM_RED, REDF, OFMDim,
MAX_CH_WIDTH>(in, out, errortype, channel_mask, red_ch_index);
201     }
202 }
203
204 #endif

```

D.24 finn_hlslib/upsample.hpp (182 lines, 6,383 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,
27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31 *****
32 /
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio@xilinx.com>
36 *          erling on 5/10/21.
37 *
38 *
39 * Library of templated HLS functions for QNN deployment.
40 * Targeting upsampling layers
41 *
42 *****
43
44 #ifndef UPSAMPLE_HPP
45 #define UPSAMPLE_HPP
46
47 #include <ap_int.h>
48 #include <hls_stream.h>
49
50 /**
51  * \brief Upsampling with the Nearest Neighbour algorithm.
Works with square feature maps
52  *
53  * \tparam OFMDim Size of the output feature map
54

```

```

55  * \tparam IFMDim Size of the input feature map
56  * \tparam NumChannels Amount of channels of the in-
put feature map
57  * \tparam In_t Input datatype
58  *
59  * \param in Input stream
60  * \param out Output stream
61  */
62 template<unsigned int OFMDim,
63          unsigned int IFMDim,
64          unsigned int NumChannels,
65          typename In_t>
66 void UpsampleNearestNeighbour(
67     hls::stream<ap_uint<NumChannels> * In_t::width>> & in,
68     hls::stream<ap_uint<NumChannels> * In_t::width>> & out
69 ) {
70     static_assert(OFMDim > IFMDim, "");
71
72     constexpr unsigned int scale_factor = OFMDim/IFMDim;
73     constexpr unsigned int Padding = OFMDim % IFMDim;
74     // Padding might be asymmetrical
75     constexpr unsigned int PaddingDown = Padding/2;
76     constexpr unsigned int PaddingUp = Padding - PaddingDown;
77     // Padding might be asymmetrical
78     constexpr unsigned int PaddingRight = Padding/2;
79     constexpr unsigned int PaddingLeft = Padding - Paddin-
gRight;
80
81     ap_uint<NumChannels * In_t::width> outData, inData;
82     ap_uint<NumChannels * In_t::width> RowBuf[IFMDim];
83     int count_row = -PaddingUp; // Counter used to understand
whether reading (and buffering) a row or not - Made in order to
avoid modulo operations
84     for (unsigned int y = 0; y < OFMDim; y++) {
85         for (unsigned int x = 0; x < OFMDim; x++) {
86             #pragma HLS pipeline style=flp II=1
87             bool read_row = (y == 0) ||
count_row==scale_factor;
88             if ((x < IFMDim) && read_row)
89             {
90                 inData = in.read();
91                 RowBuf[x] = inData;
92             }
93             // Padding Cols
94             if(x < PaddingLeft){
95                 outData = RowBuf[0];
96             }
97             else if (x >= (OFMDim - PaddingRight)){
98                 outData = RowBuf[IFMDim-1];
99             }
100             // Padding Rows
101             else if(y < PaddingUp || y >= (OFMDim - Pad-
dingDown)){
102                 outData = RowBuf[(x-PaddingLeft)/
scale_factor];
103             }
104             // No Padding
105             else{
106                 outData = RowBuf[(x-PaddingLeft)/
scale_factor];
107             }
108             //std::cout << outData << " ";
109             out.write(outData);
110             // end for y
111             //std::cout << std::endl;
112             count_row++;
113             if (count_row > scale_factor)
114                 count_row = 1;
115         } // end for x
116     }
117 }
118
119 /**
120  * \brief Upsampling with the Nearest Neighbour algorithm.
Works with square feature maps on multiple images
121  *
122  * \tparam OFMDim Size of the output feature map
123  * \tparam IFMDim Size of the input feature map
124  * \tparam NumChannels Amount of channels of the in-
put feature map
125  * \tparam In_t Input datatype
126  *
127  * \param in Input stream
128  * \param out Output stream
129  *
130  * \param numReps Number of time the function has to
be repeatedly executed (e.g. number of images)
131  */
132 template<unsigned int OFMDim,
133          unsigned int IFMDim,
134          unsigned int NumChannels,
135          typename In_t>
136 void UpsampleNearestNeighbour_Batch(
137     hls::stream<ap_uint<NumChannels> * In_t::width>> & in,
138     hls::stream<ap_uint<NumChannels> * In_t::width>> &
out,
139     unsigned int numReps) {
140

```

```

142   for (unsigned int rep = 0; rep < numReps; rep++) {
143       UpsampleNearestNeighbour<OFMDim, IFMDim, NumChannels,
In_t>(in, out);
144   }
145 }
146
147 /**
148  * \brief Upsampling a vector with the Nearest Neighbour al-
gorithm.
149  *
150  * \tparam   OFMDim           Output vector length - must be a
whole
multiple of IFMDim
151  * \tparam   IFMDim           Input vector length
152  * \tparam   NumChannels      Channels per element
153  * \tparam   In_t             Per-channel input type
154  *
155  * \param    src              Input stream
156  * \param    dst              Output stream
157  */
158 template<
159     unsigned OFMDim,           // Output vector
length - must be a whole multiple of IFMDim
160     unsigned IFMDim,           // Input vector length
161     unsigned NumChannels,      // Channels per element
162     typename In_t              // Per-channel input
type
163 >
164 void UpsampleNearestNeighbour_1D(
165     hls::stream<ap_uint<NumChannels> * In_t::width>> &src,
166     hls::stream<ap_uint<NumChannels> * In_t::width>> &dst
167 ) {
168     static_assert(OFMDim % IFMDim == 0, "OFMDim must be a
whole
multiple of IFMDim.");
169     constexpr unsigned REPS = OFMDim / IFMDim;
170
171     using buf_t = ap_uint<NumChannels * In_t::width>;
172     buf_t buf;
173     unsigned rep = 0;
174     for(unsigned i = 0; i < OFMDim; i++) {
175 #pragma HLS pipeline II=1 style=flp
176         if(rep == 0) buf = src.read();
177         dst.write(buf);
178         if(++rep == REPS) rep = 0;
179     }
180 }
181
182 #endif

```

D.25 finn_hlslib/utils.hpp (107 lines, 4,174 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,
9 * this list of conditions and the following disclaimer.
10 *
11 * 2. Redistributions in binary form must reproduce the
above copyright
12 * notice, this list of conditions and the following
disclaimer in the
13 * documentation and/or other materials provided with
the distribution.
14 *
15 * 3. Neither the name of the copyright holder nor the
names of its
16 * contributors may be used to endorse or promote
products derived from
17 * this software without specific prior written permis-
sion.
18 *
19 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,
21 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,
25 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,

```

```

27 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30 *
31 *****
32
33 /
*****
34 *
35 * Authors: Giulio Gambardella <giulio.g@xilinx.com>
36 *          Thomas B. Preusser <thomas.preusser@utexas.edu>
37 *          Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
38 *          Christoph Doebling <cdoehrin@xilinx.com>
39 *
40 * @file utils.hpp
41 *
42 * This project has received funding from the European Uni-
on's Framework
43 * Programme for Research and Innovation Horizon 2020
(2014-2020) under
44 * the Marie Skłodowska-Curie Grant Agreement No. 751339.
45 *
46 *****
47
48 #ifndef UTILS_HPP
49 #define UTILS_HPP
50
51 #include <ap_int.h>
52
53 #include <iostream>
54 #include <fstream>
55 #include <cstdlib>
56
57 // - Static Evaluation of ceil(log2(x))
-----
58 constexpr unsigned clog2(size_t x) {
59     return x<2? 0 : 1+clog2((x+1)/2);
60 }
61
62 // - Helpers to get hold of types
-----
63 template<typename T> struct first_param {};
64 template<typename R, typename A, typename... Args>
65 struct first_param<R (*) (A, Args...)> { typedef A type; };
66 template<typename C, typename R, typename A, typename...
Args>
67 struct first_param<R (C::*) (A, Args...)> { typedef A
type; };
68
69 // - Resource Representatives
-----
70 class ap_resource_dflt {};
71 class ap_resource_lut {};
72 class ap_resource_dsp {};
73 // - Resource Representatives for sliding win-
dow-----
74 class ap_resource_lutram {};
75 class ap_resource_bram {};
76 class ap_resource_uram {};
77
78 /**
79  * \brief Stream logger - Logging call to dump on file -
not synthesizable
80  *
81  *
82  * \tparam   BitWidth         Width, in number of bits, of the
input (and output) stream
83  *
84  * \param    layer_name       File name of the dump
85  * \param    log              Input (and output) stream
86  *
87  */
88 template < unsigned int BitWidth >
89 void logStringStream(const char *layer_name,
hls::stream<ap_uint<BitWidth> > &log){
90     std::ofstream ofs(layer_name);
91     hls::stream<ap_uint<BitWidth> > tmp_stream;
92
93     while(!log.empty()){
94         ap_uint<BitWidth> tmp = (ap_uint<BitWidth>) log.read();
95         ofs << std::hex << tmp << std::endl;
96         tmp_stream.write(tmp);
97     }
98
99     while(!tmp_stream.empty()){
100         ap_uint<BitWidth> tmp = tmp_stream.read();
101         log.write((ap_uint<BitWidth>) tmp);
102     }
103
104     ofs.close();
105 }

```

```
106
107 #endif
```

D.26 finn_hislib/vvau.hpp (276 lines, 11.276 B)

```
1 /
2 *****
3 * Copyright (c) 2019, Xilinx, Inc.
4 * All rights reserved.
5 * Redistribution and use in source and binary forms, with
6 * or without modification, are permitted provided that the following
7 * conditions are met:
8 * 1. Redistributions of source code must retain the above
9 * copyright notice, this list of conditions and the following disclaimer.
10 * 2. Redistributions in binary form must reproduce the
11 * above copyright notice, this list of conditions and the following
12 * disclaimer in the documentation and/or other materials provided with
13 * the distribution.
14 * 3. Neither the name of the copyright holder nor the
15 * names of its contributors may be used to endorse or promote
16 * products derived from this software without specific prior written permis-
17 * sion.
18 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
19 * CONTRIBUTORS "AS IS"
20 * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
21 * LIMITED TO,
22 * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
23 * PARTICULAR
24 * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
25 * HOLDER OR
26 * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
27 * ENTAL, SPECIAL,
28 * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
29 * LIMITED TO,
30 * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
31 * DATA, OR PROFITS;
32 * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
33 * ORY OF LIABILITY,
34 * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
35 * NEGLIGENCE OR
36 * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
37 * SOFTWARE, EVEN IF
38 * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
39 *****/
40 /
41 *****
42 *
43 * Authors: Giulio Gambardella <giulio.g@xilinx.com>
44 *
45 * \file vvau.hpp
46 *
47 * This file lists a templated function used to implement
48 * Vector-Vector-Activation Unit (used for depthwise separ-
49 * able convolutions)
50 *****/
51 /
52 *****
53 *
54 * #ifndef VVAU_HPP
55 * #define VVAU_HPP
56 *
57 * #include "hls_stream.h"
58 *
59 * #include "mac.hpp"
60 * #include "interpret.hpp"
61 *
62 /**
63 * \brief Vector vector activate function
64 *
65 * The function performs the multiplication between a weight
66 * vector and the input activation vector,
67 * accumulating the results and then applying an activation
68 * function on the accumulated result.
69 * It is used to implement depth-wise separable convolution
70 *
71 * \tparam Channels Number of channels
72 * \tparam Kernel_2 Kernel * Kernel dimension (Kernel ^ 2
73 * if square)
```

```
61 * \tparam SIMD Number of input columns computed in
62 * parallel, must be set to 1
63 * \tparam PE Number of output rows computed in par-
64 * allel
65 * \tparam MMV Number of output pixels computed in
66 * parallel
67 * \tparam TSrcI DataType of the input activation (as
68 * used in the MAC)
69 * \tparam TDstI DataType of the output activation (as
70 * generated by the activation)
71 * \tparam TWeightI DataType of the weights (as used in the
72 * MAC)
73 * \tparam TI DataType of the input stream - safely
74 * deducible from the parameters
75 * \tparam TO DataType of the output stream - safely
76 * deducible from the parameters
77 * \tparam TW DataType of the weights matrix - safely
78 * deducible from the parameters
79 * \tparam TA DataType of the activation class (e.g.
80 * thresholds) - safely deducible from the parameters
81 * \tparam R Datatype for the resource used for FPGA
82 * implementation of the MAC - safely deducible from the parameters
83 *
84 * \param in Input stream
85 * \param out Output stream
86 * \param weights Weights matrix (currently supports Bin-
87 * aryWeights or FixedPointWeights)
88 * \param activation Activation class
89 * \param reps Number of time the function has to be
90 * repeatedly executed (e.g. number of images)
91 * \param r Resource type for the hardware imple-
92 * mentation of the MAC block
93 */
94 template<
95 unsigned Channels, unsigned Kernel_2, unsigned SIMD, un-
96 signed PE, unsigned MMV,
97 typename TSrcI = Identity, typename TDstI = Identity, type-
98 name TWeightI = Identity,
99 typename TI, typename TO, typename TW, typename TA, type-
100 name R
101 >
102 void Vector_Vector_Activate_Batch(hls::stream<TI> &in,
103 hls::stream<TO> &out,
104 TW const &weights,
105 TA const &activation,
106 int const reps,
107 R const &r) {
108 static_assert(SIMD == 1, "SIMD parallelism not yet suppor-
109 ted.");
110 // how many different rows each neuron will compute
111 // alternatively: number of vertical matrix chunks
112 unsigned const NF = Channels / PE;
113 // how many synapse groups each row is split into
114 // alternatively: number of horizontal matrix chunks
115 // always equal to # kernel pixels since no SIMD
116 unsigned const SF = Kernel_2;
117 decltype(activation.init(0,0)) accu[MMV][PE];
118 #pragma HLS ARRAY_PARTITION variable=accu complete dim=0
119
120 unsigned nf = 0;
121 unsigned sf = 0;
122 unsigned tile = 0; // invariant: tile = nf*SF + sf
123 // everything merged into a common iteration space (one
124 "big" loop instead
125
126 // of smaller nested loops) to get the pipelining the way
127 we want
128 unsigned const TOTAL_FOLD = NF * SF; // Channels/SIMD;
129 for(unsigned i = 0; i < reps * TOTAL_FOLD; i++) {
130 #pragma HLS pipeline style=fpl II=1
131 TI inElem;
132 inElem = in.read();
133 // Threshold Initialisation
134 if(sf == 0) {
135 for(unsigned pe = 0; pe < PE; pe++) {
136 for(unsigned mmv = 0; mmv < MMV; mmv++) {
137 #pragma HLS UNROLL
138 accu[mmv][pe] = activation.init(nf, pe);
139 }
140 }
141 }
142 // compute matrix-vector product for each processing ele-
143 ment
144 auto const &w = weights.weights(tile);
145 for(unsigned pe = 0; pe < PE; pe++) {
146 #pragma HLS UNROLL
147 auto const wgt = TWeightI()(w[pe]);
148 for (unsigned mmv = 0; mmv < MMV; mmv++){
149 auto const act = TSrcI()(inElem, mmv);
150 accu[mmv][pe] += mul(wgt[0], act(pe,mmv), r);
151 }
152 }
153 }
```

```

136 // keep track of which folded synapse/neuron we are pro-
cessing
137 ++tile;
138 if(++sf == SF) {
139 // produce output and clear accumulators
140 auto outElem = TDstI().template operator()<TO>();
141 for (unsigned pe = 0; pe < PE; pe++) {
142 #pragma HLS UNROLL
143 for (unsigned mmv = 0; mmv < MMV; mmv++){
144 #pragma HLS UNROLL
145 outElem(pe,mmv,1) = activation.activate(nf, pe,
accu[mmv][pe]);
146 }
147 }
148 out.write(outElem);
149 // next folded neuron or image
150 sf = 0;
151 if(++nf == NF) {
152 nf = 0;
153 tile = 0;
154 }
155 }
156 }
157 }
158
159 /**
160 * \brief Vector vector activate with streaming weights
161 * The function performs the multiplication between a weight
vector and the input activation vector,
162 * accumulating the results and then applying an activation
function on the accumulated result.
163 * It is used to implement depth-wise separable convolution.
The weights are supplied from a stream
164 * input to facilitate memory-compute decoupling.
165 *
166 * \tparam Channels Number of channels
167 * \tparam Kernel_2 Kernel * Kernel dimension (Kernel ^ 2
if square)
168 * \tparam SIMD Number of input columns computed in
parallel, must be set to 1
169 * \tparam PE Number of output rows computed in par-
allel
170 * \tparam MMV Number of output pixels computed in
parallel
171 * \tparam TSrcI DataType of the input activation (as
used in the MAC)
172 * \tparam TDstI DataType of the output activation (as
generated by the activation)
173 * \tparam TweightI DataType of the weights (as used in the
MAC)
174 * \tparam TI DataType of the input stream - safely
deducible from the parameters
175 * \tparam TO DataType of the output stream - safely
deducible from the parameters
176 * \tparam TW DataType of the weights matrix - safely
deducible from the parameters
177 * \tparam TA DataType of the activation class (e.g.
thresholds) - safely deducible from the parameters
178 * \tparam R Datatype for the resource used for FPGA
implementation of the MAC - safely deducible from the parameters
179 *
180 * \param in Input stream
181 * \param out Output stream
182 * \param weights Weights matrix (currently supports Bin-
aryWeights or FixedPointWeights)
183 * \param activation Activation class
184 * \param reps Number of time the function has to be
repeatedly executed (e.g. number of images)
185 * \param r Resource type for the hardware imple-
mentation of the MAC block
186 */
187 template<
188 unsigned Channels, unsigned Kernel_2, unsigned SIMD,
unsigned PE, unsigned MMV,
189 typename TSrcI = Identity, typename TDstI = Identity,
typename TweightI = Identity, typename TW,
190 typename TI, typename TO, typename TA, typename R
191 >
192 void Vector_Vector_Activate_Stream_Batch(
193 hls::stream<TI> &in,
194 hls::stream<TO> &out,
195 hls::stream<ap_uint<PE*SIMD*TW::width>> &weights,
196 int const activation,
197 int const reps,
198 R const &r
199 ) {
200 static_assert(SIMD == 1, "SIMD parallelism not yet
supported.");
201
202 // how many different rows each neuron will compute
203 // alternatively: number of vertical matrix chunks
204 constexpr unsigned NF = Channels / PE;
205
206 // how many synapse groups each row is split into
207 // alternatively: number of horizontal matrix chunks

```

```

210 // always equal to # kernel pixels since no SIMD
211 constexpr unsigned SF = Kernel_2;
212 decltype(activation.init(0,0)) accu[MMV][PE];
213 #pragma HLS ARRAY_PARTITION variable=accu complete dim=0
214
215 // unpacked and packed buffers for weight stream
216 unsigned nf = 0;
217 unsigned sf = 0;
218 unsigned tile = 0; // invariant: tile = nf*SF + sf
219 // everything merged into a common iteration space
220 (one "big" loop instead
221 // of smaller nested loops) to get the pipelining
the way we want
222 constexpr unsigned TOTAL_FOLD = NF * SF; /** Chan-
nels/SIMD;
223 for(unsigned i = 0; i < reps * TOTAL_FOLD; i++) {
224 #pragma HLS pipeline style=fpl II=1
225 TI inElem;
226 inElem = in.read();
227 // Threshold Initialisation
228 if(sf == 0) {
229 for(unsigned pe = 0; pe < PE; pe++) {
230 for(unsigned mmv = 0; mmv <
MMV; mmv++) {
231 #pragma HLS UNROLL
232 accu[mmv][pe] = activ-
ation.init(nf, pe);
233 }
234 }
235 }
236 // Packed and unpacked weight representations
237 ap_uint<PE * SIMD * TW::width> const W_packed =
weights.read();
238 Weights_Tile<SIMD, TW, PE> w;
239 #pragma HLS ARRAY_PARTITION variable=w.m_weights complete
dim=0
240 for(unsigned pe = 0; pe < PE; pe++) {
241 #pragma HLS UNROLL
242 w.m_weights[pe] =
W_packed((pe+1)*SIMD*TW::width-1, pe*SIMD*TW::width);
243 }
244 for(unsigned pe = 0; pe < PE; pe++) {
245 #pragma HLS UNROLL
246 auto const wgt = TweightI()(w[pe]);
247 for(unsigned mmv = 0; mmv < MMV; mmv+
+){
248 auto const act = TSrcI()(in-
Elem, mmv);
249 accu[mmv][pe] += mul(wgt[0],
act(pe,mmv), r);
250 }
251 }
252 }
253 // keep track of which folded synapse/neuron
we are processing
254 ++tile;
255 if(++sf == SF) {
256 // produce output and clear accumulat-
ors
257 auto outElem = TDstI().template oper-
ator()<TO>();
258 for(unsigned pe = 0; pe < PE; pe++) {
259 #pragma HLS UNROLL
260 for(unsigned mmv = 0; mmv <
MMV; mmv++) {
261 #pragma HLS UNROLL
262 outElem(pe,mmv,1) =
activation.activate(nf, pe, accu[mmv][pe]);
263 }
264 }
265 out.write(outElem);
266 // next folded neuron or image
267 sf = 0;
268 if(++nf == NF) {
269 nf = 0;
270 tile = 0;
271 }
272 }
273 }
274 }
275 }
276 #endif

```

D.27 finn_hlslib/weights.hpp (169 lines, 5,628 B)

```

1 /
*****
2 * Copyright (c) 2019, Xilinx, Inc.
3 * All rights reserved.
4 *
5 * Redistribution and use in source and binary forms, with
or without
6 * modification, are permitted provided that the following
conditions are met:
7 *
8 * 1. Redistributions of source code must retain the above
copyright notice,

```

```

9      *      this list of conditions and the following disclaimer.
10     *
11     * 2. Redistributions in binary form must reproduce the
above copyright
12     *      notice, this list of conditions and the following
disclaimer in the
13     *      documentation and/or other materials provided with
the distribution.
14     *
15     * 3. Neither the name of the copyright holder nor the
names of its
16     *      contributors may be used to endorse or promote
products derived from
17     *      this software without specific prior written permis-
sion.
18     *
19     * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS"

20     * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO,

21     * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
PARTICULAR
22     * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
HOLDER OR
23     * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCID-
ENTAL, SPECIAL,
24     * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO,

25     * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS;
26     * OR BUSINESS INTERRUPTION). HOWEVER CAUSED AND ON ANY THE-
ORY OF LIABILITY,

27     * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR
28     * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF
29     * ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
30
*****
31
32 /
*****
33 *
34 * Authors: Giulio Gambardella <giulio@xilinx.com>
35 *          Thomas B. Preusser <thomas.preusser@utexas.edu>
36 *          Marie-Curie Fellow, Xilinx Ireland, Grant
Agreement No. 751339
37 *          Christoph Doebling <cdoehrin@xilinx.com>
38 *
39 * @file weights.hpp
40 *
41 * Library of templated HLS classes for BNN deployment.
42 * This file lists a set of classes used to implement
43 * weights in neural network.
44 *
45 * This project has received funding from the European Uni-
on's Framework
46 * Programme for Research and Innovation Horizon 2020
(2014-2020) under
47 * the Marie Skłodowska-Curie Grant Agreement No. 751339.
48 *
*****
49
50
51 #ifndef WEIGHTS_HPP
52 #define WEIGHTS_HPP
53
54 #include <ap_int.h>
55 #include <array>
56
57 /**
58  * \brief      A binary weight storage adapter that trans-
lates the internal
59  * organization optimized for storage to the generalized ac-
cess by the MVAU.
60  *
61  *
62  * \tparam      SIMD      Number of input columns (channels) com-
puted in parallel
63  * \tparam      PE        Number of output rows (channels) com-
puted in parallel
64  * \tparam      TILES     3rd dimension of the weights matrix
65  */
66 template<unsigned SIMD, unsigned PE, unsigned TILES>
67 class BinaryWeights {
68 public:
69     ap_uint<SIMD> m_weights[PE][TILES];
70
71 private:
72     /**
73      * Temporary container for the tile index to implement the
74      * memory access in pe -> tile order.
75      */

```

```

76     class TileIndex {
77     public:
78         BinaryWeights const &m_par;
79         unsigned const m_idx;
80
81     public:
82         TileIndex(BinaryWeights const &par, unsigned const idx)
83             : m_par(par), m_idx(idx) {
84         }
85
86     public:
87         ap_uint<SIMD> operator[](unsigned const pe) const {
88 #pragma HLS inline
89             return m_par.m_weights[pe][m_idx];
90         }
91     };
92
93     public:
94         TileIndex weights(unsigned const tile) const {
95 #pragma HLS inline
96             return TileIndex(*this, tile);
97         }
98     };
99
100
101 /**
102  * \brief      A fixed point weight storage adapter that
translates the internal
103  * organization optimized for storage to the generalized ac-
cess by the MVAU.
104  *
105  * \tparam      SIMD      Number of input columns (channels) com-
puted in parallel
106  * \tparam      WT         Datatype of the weights
107  * \tparam      PE        Number of output rows (channels) com-
puted in parallel
108  * \tparam      TILES     3rd dimension of the weights matrix
109  */
110 template<unsigned SIMD, typename WT, unsigned PE, unsigned
TILES>
111 class FixedPointWeights {
112 public:
113     ap_uint<SIMD*WT::width> m_weights[PE][TILES];
114
115 private:
116     /**
117      * Temporary container for the tile index to implement the
118      * memory access in pe -> tile order.
119      */
120     class TileIndex {
121     public:
122         FixedPointWeights const &m_par;
123         unsigned const m_idx;
124
125     public:
126         TileIndex(FixedPointWeights const &par, unsigned const
idx)
127             : m_par(par), m_idx(idx) {
128 #pragma HLS inline
129         }
130
131     public:
132         std::array<WT, SIMD> operator[](unsigned const pe) const
{
133 #pragma HLS inline
134             std::array<WT, SIMD> ret;
135             for(unsigned int i=0; i<SIMD; i++) {
136 #pragma HLS unroll
137                 ap_int<WT::width> const local_temp =
m_par.m_weights[pe][m_idx]((i+1)*WT::width-1, i*WT::width);
138                 ret[i] = WT(local_temp);
139             }
140             return ret;
141         }
142     };
143
144     public:
145         TileIndex weights(unsigned const tile) const {
146 #pragma HLS inline
147             return TileIndex(*this, tile);
148         }
149
150
151     template<unsigned SIMD, typename WT, unsigned PE>
152     class Weights_Tile {
153     public:
154         ap_uint<SIMD*WT::width> m_weights[PE];
155
156         std::array<WT, SIMD> operator[](unsigned const pe) const {
157 #pragma HLS inline
158             std::array<WT, SIMD> ret;
159             for(unsigned int i=0; i<SIMD; i++) {
160 #pragma HLS unroll
161                 ap_int<WT::width> const local_temp = m_weights[pe]
((i+1)*WT::width-1, i*WT::width);
162                 ret[i] = WT(local_temp);
163             }
164             return ret;
165         }
166     };

```

```
167  
168  
169 #endif
```


Appendix E Complete Vendor Header Source Listings

The header tree the corpus depends on, reproduced on the same terms. These files were obtained from the public repositories listed in Table 3 and are included because no statement in Chapters 3 to 6 about what the corpus elaborates into can be checked without them.

E.1 vendor/ap_axi_sdata.h (343 lines, 8,240 B)

```

1 // (C) Copyright 2016-2021 Xilinx, Inc.
2 // All Rights Reserved.
3 //
4 // Licensed to the Apache Software Foundation (ASF) under one
5 // or more contributor license agreements. See the NOTICE
file
6 // distributed with this work for additional information
7 // regarding copyright ownership. The ASF licenses this file
8 // to you under the Apache License, Version 2.0 (the
9 // "License"); you may not use this file except in compliance
10 // with the License. You may obtain a copy of the License at
11 //
12 // http://www.apache.org/licenses/LICENSE-2.0
13 //
14 // Unless required by applicable law or agreed to in writing,
15 // software distributed under the License is distributed on
an
16 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 // KIND, either express or implied. See the License for the
18 // specific language governing permissions and limitations
19 // under the License.
20
21 // (C) Copyright 2016-2021 Xilinx, Inc.
22 // All Rights Reserved.
23 //
24 // Licensed to the Apache Software Foundation (ASF) under one
25 // or more contributor license agreements. See the NOTICE
file
26 // distributed with this work for additional information
27 // regarding copyright ownership. The ASF licenses this file
28 // to you under the Apache License, Version 2.0 (the
29 // "License"); you may not use this file except in compliance
30 // with the License. You may obtain a copy of the License at
31 //
32 // http://www.apache.org/licenses/LICENSE-2.0
33 //
34 // Unless required by applicable law or agreed to in writing,
35 // software distributed under the License is distributed on
an
36 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
37 // KIND, either express or implied. See the License for the
38 // specific language governing permissions and limitations
39 // under the License.
40
41
42
43 /*
44 * This file contains the definition of the data types for
AXI streaming.
45 * ap_axi_s is a signed interpretation of the AXI stream
46 * ap_axi_u is an unsigned interpretation of the AXI stream
47 */
48
49 #ifndef __AP_AXI_SDATA__
50 #define __AP_AXI_SDATA__
51
52 #include "ap_int.h"
53
54 template<int D,int U,int TI,int TD>
55 struct ap_axis{
56     ap_int<D> data;
57     ap_uint<(D+7)/8> keep;
58     ap_uint<(D+7)/8> strb;
59     ap_uint<U> user;
60     ap_uint<1> last;
61     ap_uint<TI> id;
62     ap_uint<TD> dest;
63 };
64
65 template<int D>
66 struct ap_axis<D, 0, 0, 0>{
67     ap_int<D> data;
68     ap_uint<(D+7)/8> keep;
69     ap_uint<(D+7)/8> strb;
70     ap_uint<1> last;
71 };
72
73 template<int D,int U,int TI,int TD>
74 struct ap_axiu{
75     ap_uint<D> data;
76     ap_uint<(D+7)/8> keep;
77     ap_uint<(D+7)/8> strb;
78     ap_uint<U> user;
79     ap_uint<1> last;
80     ap_uint<TI> id;
81     ap_uint<TD> dest;
82 };
83
84 template<int D>
85 struct ap_axiu<D, 0, 0, 0>{
86     ap_uint<D> data;
87     ap_uint<(D+7)/8> keep;
88     ap_uint<(D+7)/8> strb;
89     ap_uint<1> last;
90 };
91
92
93 template<int D,int U,int TI,int TD> struct qdma_axis;
94
95 template<int D>
96 struct qdma_axis<D, 0, 0, 0>{
97 // private:
98     ap_uint<D> data;
99     ap_uint<(D+7)/8> keep;
100     ap_uint<1> last;
101 // public:
102     ap_uint<D> get_data() const;
103     ap_uint<(D+7)/8> get_keep() const;
104     ap_uint<1> get_last() const;
105
106     void set_data(const ap_uint<D> &d);
107     void set_keep(const ap_uint<(D+7)/8> &k);
108     void set_last(const ap_uint<1> &l);
109     void keep_all();
110
111     qdma_axis(ap_uint<D> d = ap_uint<D>(), ap_uint<(D+7)/8> k =
ap_uint<(D+7)/8>(), ap_uint<1> l = ap_uint<1>());
112     qdma_axis(const qdma_axis<D, 0, 0, 0> &d);
113 };
114 #include "hls_stream.h"
115 namespace hls {
116
117 template <int D,int U,int TI,int TD>
118 class stream<ap_axis<D, U, TI, TD> > {
119     typedef ap_axis<D, U, TI, TD> __STREAM_T__;
120 public:
121     /// Constructors
122     inline stream();
123
124     inline stream(const char* name);
125
126     /// Make copy constructor and assignment operator private
127 private:
128     inline stream(const stream< __STREAM_T__ >& chn);
129
130     inline stream& operator= (const stream< __STREAM_T__ >&
chn);
131
132 public:
133     /// Overload >> and << operators to implement read() and
write()
134     inline void operator >> (__STREAM_T__ &rdata);
135
136     inline void operator << (const __STREAM_T__ &wdata);
137
138     /// empty & full
139     bool empty() const;
140
141     bool full() const;
142
143     /// Blocking read
144     void read(__STREAM_T__ &dout);
145
146     __STREAM_T__ read();
147
148     /// Blocking write
149     void write(const __STREAM_T__ &din);
150
151     /// Non-Blocking read
152     bool read_nb(__STREAM_T__ &dout);
153
154     /// Non-Blocking write
155     bool write_nb(const __STREAM_T__ &in);
156
157     __STREAM_T__ V;
158
159 };
160
161 // Specialization
162 template <int D>

```

```

165 class stream<ap_axis<D, 0, 0, 0> > {
166     typedef ap_axis<D, 0, 0, 0> __STREAM_T__ ;
167 public:
168     /// Constructors
169     inline stream();
170
171     inline stream(const char* name);
172
173     /// Make copy constructor and assignment operator private
174 private:
175     inline stream(const stream< __STREAM_T__ >& chn);
176
177     inline stream& operator= (const stream< __STREAM_T__ >&
chn);
178
179 public:
180     /// Overload >> and << operators to implement read() and
write()
181     inline void operator >> (__STREAM_T__ &rdata);
182
183     inline void operator << (const __STREAM_T__ &wdata);
184
185     /// Empty & Full
186     bool empty() const;
187
188     bool full() const;
189
190     /// Blocking read
191     void read(__STREAM_T__ &dout);
192
193     __STREAM_T__ read();
194
195     /// Nonblocking read
196     bool read_nb(__STREAM_T__ &dout);
197
198     /// Blocking write
199     void write(const __STREAM_T__ &din);
200
201     /// Nonblocking write
202     bool write_nb(const __STREAM_T__ &din);
203
204     __STREAM_T__ V;
205 };
206
207 // ap_axiu
208 template <int D,int U,int TI,int TD>
209 class stream<ap_axiu<D, U, TI, TD> > {
210     typedef ap_axiu<D, U, TI, TD> __STREAM_T__;
211 public:
212     /// Constructors
213     inline stream();
214
215     inline stream(const char* name);
216
217     /// Make copy constructor and assignment operator private
218 private:
219     inline stream(const stream< __STREAM_T__ >& chn);
220
221     inline stream& operator= (const stream< __STREAM_T__ >&
chn);
222
223 public:
224     /// Overload >> and << operators to implement read() and
write()
225     inline void operator >> (__STREAM_T__ &rdata);
226
227     inline void operator << (const __STREAM_T__ &wdata);
228
229     /// empty & full
230     bool empty() const;
231
232     bool full() const;
233
234     /// Blocking read
235     void read(__STREAM_T__ &dout);
236
237     __STREAM_T__ read();
238
239     /// Blocking write
240     void write(const __STREAM_T__ &din);
241
242     /// Non-Blocking read
243     bool read_nb(__STREAM_T__ & dout);
244
245     /// Non-Blocking write
246     bool write_nb(const __STREAM_T__ & in);
247
248     __STREAM_T__ V;
249
250 };
251
252 // Specialization
253 template <int D>
254 class stream<ap_axiu<D, 0, 0, 0> > {
255     typedef ap_axiu<D, 0, 0, 0> __STREAM_T__;
256 public:
257     /// Constructors
258     inline stream();
259
260

```

```

261     inline stream(const char* name);
262
263     /// Make copy constructor and assignment operator private
264 private:
265     inline stream(const stream< __STREAM_T__ >& chn);
266
267     inline stream& operator= (const stream< __STREAM_T__ >&
chn);
268
269 public:
270     /// Overload >> and << operators to implement read() and
write()
271     inline void operator >> (__STREAM_T__ &rdata);
272
273     inline void operator << (const __STREAM_T__ &wdata);
274
275     /// Empty & Full
276     bool empty() const;
277
278     bool full() const;
279
280     /// Blocking read
281     void read(__STREAM_T__ &dout);
282
283     __STREAM_T__ read();
284
285     /// Nonblocking read
286     bool read_nb(__STREAM_T__ &dout);
287
288     /// Blocking write
289     void write(const __STREAM_T__ &din);
290
291     /// Nonblocking write
292     bool write_nb(const __STREAM_T__ &din);
293
294     __STREAM_T__ V;
295 };
296
297 // Specialization for qdma
298 template <int D>
299 class stream<qdma_axis<D, 0, 0, 0> > {
300     typedef qdma_axis<D, 0, 0, 0> __STREAM_T__;
301 public:
302     /// Constructors
303     inline stream();
304
305     inline stream(const char* name);
306
307     /// Make copy constructor and assignment operator private
308 private:
309     inline stream(const stream< __STREAM_T__ >& chn);
310
311     inline stream& operator= (const stream< __STREAM_T__ >&
chn);
312
313 public:
314     /// Overload >> and << operators to implement read() and
write()
315     inline void operator >> (__STREAM_T__ &rdata);
316
317     inline void operator << (const __STREAM_T__ &wdata);
318
319     /// Empty & Full
320     bool empty() const;
321
322     bool full() const;
323
324     /// Blocking read
325     void read(__STREAM_T__ &dout);
326
327     __STREAM_T__ read();
328
329     /// Nonblocking read
330     bool read_nb(__STREAM_T__ &dout);
331
332     /// Blocking write
333     void write(const __STREAM_T__ &din);
334
335     /// Nonblocking write
336     bool write_nb(const __STREAM_T__ &din);
337
338     /// Fifo size
339     //inline unsigned size();
340
341     __STREAM_T__ V;
342 };
343
344 } // end of hls namespace
345 #endif

```

E.2 vendor/ap_common.h (376 lines, 10,180 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  * cense");
6  * you may not use this file except in compliance with the
7  * License.
8  * You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0

```



```

9  *
10 * Unless required by applicable law or agreed to in writing,
    software
11 * distributed under the License is distributed on an "AS IS"
    BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
    press or implied.
13 * See the License for the specific language governing per-
    missions and
14 * limitations under the License.
15 */
16
17 #ifndef __AP_COMMON_H__
18 #define __AP_COMMON_H__
19
20 //
-----
21
22 // Forward declaration of all AP types.
23 #include <ap_decl.h>
24
25
26 #ifdef __SYNTHESIS__
27 #error "The open-source version of AP types does not support
    synthesis."
28 #endif // ifdef __SYNTHESIS__
29 #define _AP_ENABLE_HALF_ 0
30
31
32 #if _AP_ENABLE_HALF_ == 1
33 // Before ap_private definition.
34 #ifdef __SYNTHESIS__
35 #define HLS_HALF_DEFINED_
36 typedef __fp16 half;
37 #else
38 class half;
39 #endif // __SYNTHESIS__
40 #endif // _AP_ENABLE_HALF_
41
42 //
-----
43
44 // Macro functions
45 #define AP_MAX(a, b) ((a) > (b) ? (a) : (b))
46 #define AP_MIN(a, b) ((a) < (b) ? (a) : (b))
47 #define AP_ABS(a) ((a) >= 0 ? (a) : -(a))
48
49 #ifndef AP_ASSERT
50 #ifndef __SYNTHESIS__
51 #include <assert.h>
52 #define AP_ASSERT(cond, msg) assert((cond) && (msg))
53 #else
54 #define AP_ASSERT(cond, msg)
55 #endif // ifdef __SYNTHESIS__
56 #endif // ifndef AP_ASSERT
57
58 #ifndef __SYNTHESIS__
59 // for fprintf messages.
60 #include <stdio.h>
61 // for exit on error.
62 #include <stdlib.h>
63 #endif
64
65 // same disable condition as assert.
66 #if !defined(__SYNTHESIS__) && !defined(NDEBUG)
67
68 #define _AP_DEBUG(cond, ...) \
69     do { \
70         if ((cond)) { \
71             fprintf(stderr, "DEBUG: " __VA_ARGS__); \
72             fprintf(stderr, "\n"); \
73         } \
74     } while (0)
75 #define _AP_WARNING(cond, ...) \
76     do { \
77         if ((cond)) { \
78             fprintf(stderr, "WARNING: " __VA_ARGS__); \
79             fprintf(stderr, "\n"); \
80         } \
81     } while (0)
82 #define _AP_ERROR(cond, ...) \
83     do { \
84         if ((cond)) { \
85             fprintf(stderr, "ERROR: " __VA_ARGS__); \
86             fprintf(stderr, "\n"); \
87             abort(); \
88         } \
89     } while (0)
90
91 #else // if !defined(__SYNTHESIS__) && !defined(NDEBUG)
92
93 #define __AP_VOID_CAST static_cast<void>
94 #define _AP_DEBUG(cond, ...) (__AP_VOID_CAST(0))
95 #define _AP_WARNING(cond, ...) (__AP_VOID_CAST(0))
96 #define _AP_ERROR(cond, ...) (__AP_VOID_CAST(0))
97

```

```

98 #endif // if !defined(__SYNTHESIS__) && !defined(NDEBUG) else
99
100 //
-----
101
102 // Attribute only for synthesis
103 #ifndef __SYNTHESIS__
104 #define INLINE inline __attribute__((always_inline))
105 // #define INLINE inline __attribute__((noinline))
106 #else
107 #define INLINE inline
108 #endif
109
110 #define AP_WEAK
111 // __attribute__((weak))
112
113 #ifndef AP_INT_MAX_W
114 #define AP_INT_MAX_W 1024
115 #endif
116
117 #define BIT_WIDTH_UPPER_LIMIT (1 << 15)
118 #if AP_INT_MAX_W > BIT_WIDTH_UPPER_LIMIT
119 #error "Bitwidth exceeds 32768 (1 << 15), the maximum allowed
    value"
120 #endif
121
122 #define MAX_MODE(BITS) ((BITS + 1023) / 1024)
123
124 //
-----
125
126 // XXX apcc cannot handle global std::ios_base::Init()
    brought in by <iostream>
127 #ifndef AP_AUTOCC
128 #ifndef __SYNTHESIS__
129 // for overload operator<<
130 #include <iostream>
131 #endif
132 #endif // ifndef AP_AUTOCC
133
134 #ifndef __SYNTHESIS__
135 // for string format.
136 #include <sstream>
137 // for string.
138 #include <string>
139 #endif
140
141 // for detecting if char is signed.
142 enum { CHAR_IS_SIGNED = (char)-1 < 0 };
143
144 // TODO we have similar traits in x_hls_utils.h, should con-
    sider unify.
145 namespace _ap_type {
146 template <typename _Tp>
147 struct is_signed {
148     static const bool value = _Tp(-1) < _Tp(1);
149 };
150
151 template <typename _Tp>
152 struct is_integral {
153     static const bool value = false;
154 };
155 #define DEF_IS_INTEGRAL(CTYPE) \
156     template <> \
157     struct is_integral<CTYPE> { \
158         static const bool value = true; \
159     };
160 DEF_IS_INTEGRAL(bool)
161 DEF_IS_INTEGRAL(char)
162 DEF_IS_INTEGRAL(signed char)
163 DEF_IS_INTEGRAL(unsigned char)
164 DEF_IS_INTEGRAL(short)
165 DEF_IS_INTEGRAL(unsigned short)
166 DEF_IS_INTEGRAL(int)
167 DEF_IS_INTEGRAL(unsigned int)
168 DEF_IS_INTEGRAL(long)
169 DEF_IS_INTEGRAL(unsigned long)
170 DEF_IS_INTEGRAL(ap_slong)
171 DEF_IS_INTEGRAL(ap_ulong)
172 #undef DEF_IS_INTEGRAL
173
174 template <bool, typename _Tp = void>
175 struct enable_if {};
176 // partial specialization for true
177 template <typename _Tp>
178 struct enable_if<true, _Tp> {
179     typedef _Tp type;
180 };
181
182 template <typename _Tp>
183 struct remove_const {
184     typedef _Tp type;
185 };
186
187 template <typename _Tp>
188 struct remove_const<_Tp const> {
189     typedef _Tp type;
190 };

```

```

190 };
191 } // namespace _ap_type
192
193 //
-----
194
195 // Define ssdm_int and _ssdm_op.
196 // XXX deleted in open-source version
197
198 #ifndef NON_C99STRING
199 #define _AP_C99 true
200 #else
201 #define _AP_C99 false
202 #endif
203
204 static inline unsigned char guess_radix(const char* s) {
205     unsigned char rd = 10; ///< default radix
206     const char* p = s;
207     // skip neg sign if it exists
208     if (p[0] == '-' || p[0] == '+') ++p;
209     // guess based on following two bits.
210     if (p[0] == '0') {
211         if (p[1] == 'b' || p[1] == 'B') {
212             rd = 2;
213         } else if (p[1] == 'o' || p[1] == 'O') {
214             rd = 8;
215         } else if (p[1] == 'x' || p[1] == 'X') {
216             rd = 16;
217         } else if (p[1] == 'd' || p[1] == 'D') {
218             rd = 10;
219         }
220     }
221     return rd;
222 }
223
224 //
-----
225
226 // Basic integral struct upon which ap_int and ap_fixed are
defined.
227 #ifdef __SYNTHESIS__
228 // Use ssdm_int, a compiler dependent, attribute constrained
integral type as
229 // basic data type.
230 #define _AP_ROOT_TYPE ssdm_int
231 // Basic ops.
232
233 #define _AP_ROOT_op_concat(Ret, X, Y) _ssdm_op_concat(Ret, X,
Y)
234 #define _AP_ROOT_op_get_bit(Val, Bit) _ssdm_op_get_bit(Val,
Bit)
235 #define _AP_ROOT_op_set_bit(Val, Bit, Repl)
_ssdm_op_set_bit(Val, Bit, Repl)
236 #define _AP_ROOT_op_get_range(Val, Lo, Hi)
_ssdm_op_get_range(Val, Lo, Hi)
237 #define _AP_ROOT_op_set_range(Val, Lo, Hi, Repl) \
_ssdm_op_set_range(Val, Lo, Hi, Repl)
238 #define _AP_ROOT_op_reduce(Op, Val) _ssdm_op_reduce(Op, Val)
239 #else // ifdef __SYNTHESIS__
240 // Use ap_private for compiler-independent basic data type
241 template <int _AP_W, bool _AP_S, bool _AP_C = _AP_W <= 64>
class ap_private;
242 // model ssdm_int in standard C++ for simulation.
243 template <int _AP_W, bool _AP_S>
244 struct ssdm_int_sim {
245     /// integral type with template-specified width and signed-
ness.
246     ap_private<_AP_W, _AP_S> V;
247     ssdm_int_sim() {}
248 };
249 #define _AP_ROOT_TYPE ssdm_int_sim
250 // private's ref uses _AP_ROOT_TYPE.
251 #include <etc/ap_private.h>
252 // XXX The C-sim model cannot use GCC-extension
253 // Basic ops. Ret and Val are ap_private.
254 template <typename _Tp1, typename _Tp2, typename _Tp3>
255 inline _Tp1 _AP_ROOT_op_concat(const _Tp1& Ret, const _Tp2&
X, const _Tp3& Y) {
256     _Tp1 r = (X).operator,(Y);
257     return r;
258 }
259 #define _AP_ROOT_op_get_bit(Val, Bit) (Val).get_bit((Bit))
260 template <typename _Tp1, typename _Tp2, typename _Tp3>
261 inline _Tp1& _AP_ROOT_op_set_bit(_Tp1& Val, const _Tp2& Bit,
const _Tp3& Repl) {
262     (Val).set_bit((Bit), (Repl));
263     return Val;
264 }
265 // notice the order of high and low index is different in
ssdm call and
266 // ap_private.range()...
267 #define _AP_ROOT_op_get_range(Val, Lo, Hi) (Val).range((Hi),
(Lo))
268 template <typename _Tp1, typename _Tp2, typename _Tp3, type-
name _Tp4>
269 inline _Tp1& _AP_ROOT_op_set_range(_Tp1& Val, const _Tp2& Lo,
const _Tp3& Hi,

```

```

270 const _Tp4& Repl) {
271     (Val).range((Hi), (Lo)) = Repl;
272     return (Val);
273 }
274 #define _AP_ROOT_op_and_reduce(Val) (Val).and_reduce()
275 #define _AP_ROOT_op_nand_reduce(Val) (Val).nand_reduce()
276 #define _AP_ROOT_op_or_reduce(Val) (Val).or_reduce()
277 #define _AP_ROOT_op_xor_reduce(Val) (Val).xor_reduce()
278 // ## is the concatenation in preprocessor:
279 #define _AP_ROOT_op_reduce(Op, Val)
_AP_ROOT_op_##Op##_reduce(Val)
280 #endif // ifdef __SYNTHESIS__ else
281
282 //
-----
283
284
285 // Constants for half, single, double precision floating
points
286 #define HALF_MAN 10
287 #define FLOAT_MAN 23
288 #define DOUBLE_MAN 52
289
290 #define HALF_EXP 5
291 #define FLOAT_EXP 8
292 #define DOUBLE_EXP 11
293
294 #define BIAS(e) ((1L << (e - 1L)) - 1L)
295 #define HALF_BIAS BIAS(HALF_EXP)
296 #define FLOAT_BIAS BIAS(FLOAT_EXP)
297 #define DOUBLE_BIAS BIAS(DOUBLE_EXP)
298
299 #define APFX_IEEE_DOUBLE_E_MAX DOUBLE_BIAS
300 #define APFX_IEEE_DOUBLE_E_MIN (-DOUBLE_BIAS + 1)
301
302 INLINE ap_ulong doubleToRawBits(double pf) {
303     union {
304         ap_ulong __L;
305         double __D;
306     } LD;
307     LD.__D = pf;
308     return LD.__L;
309 }
310
311 INLINE unsigned int floatToRawBits(float pf) {
312     union {
313         unsigned int __L;
314         float __D;
315     } LD;
316     LD.__D = pf;
317     return LD.__L;
318 }
319
320 #if _AP_ENABLE_HALF_ == 1
321 INLINE unsigned short halfToRawBits(half pf) {
322     #ifdef __SYNTHESIS__
323     union {
324         unsigned short __L;
325         half __D;
326     } LD;
327     LD.__D = pf;
328     return LD.__L;
329     #else
330     return pf.get_bits();
331     #endif
332 }
333 #endif
334
335 // unsigned long long is at least 64-bit
336 INLINE double rawBitsToDouble(ap_ulong pi) {
337     union {
338         ap_ulong __L;
339         double __D;
340     } LD;
341     LD.__L = pi;
342     return LD.__D;
343 }
344
345 // long is at least 32-bit
346 INLINE float rawBitsToFloat(unsigned long pi) {
347     union {
348         unsigned int __L;
349         float __D;
350     } LD;
351     LD.__L = pi;
352     return LD.__D;
353 }
354
355 #if _AP_ENABLE_HALF_ == 1
356 // short is at least 16-bit
357 INLINE half rawBitsToHalf(unsigned short pi) {
358     #ifdef __SYNTHESIS__
359     union {
360         unsigned short __L;
361         half __D;
362     } LD;
363     LD.__L = pi;
364     return LD.__D;
365     #else
366     // sim model of half has a non-trivial constructor

```

```

367     half __D;
368     __D.set_bits(pi);
369     return __D;
370 #endif
371 }
372 #endif
373
374 #endif // ifndef __AP_COMMON_H__
375
376 // -*- cpp -*-

```

E.3 vendor/ap_decl.h (212 lines, 6,473 B)

```

1  /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  * cense");
6  * you may not use this file except in compliance with the
7  * License.
8  * You may obtain a copy of the License at
9  *
10 * http://www.apache.org/licenses/LICENSE-2.0
11 *
12 * Unless required by applicable law or agreed to in writing,
13 * software
14 * distributed under the License is distributed on an "AS IS"
15 * BASIS,
16 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 * press or implied.
18 * See the License for the specific language governing per-
19 * missions and
20 * limitations under the License.
21 */
22 #ifndef __AP_DECL_H__
23 #define __AP_DECL_H__
24
25 -----
26
27 #if !defined(__AP_FIXED_H__) && !defined(__AP_INT_H__) && !
28 defined(__AUTOPILOT_CBE_H__) && !defined(__HLS_HALF_H__)
29
30 #error "Only ap_fixed.h and ap_int.h can be included directly in
31 user code."
32 #endif
33
34 // Test __SYNTHESIS__ only for mode
35 #if !defined(__SYNTHESIS__) && (defined(AESL_SYN) ||
36 defined(__HLS_SYN__))
37 //pragma message "AESL_SYN and __HLS_SYN__ should be re-
38 placed by __SYNTHESIS__"
39 #define __SYNTHESIS__
40 #endif
41
42 /* for safety*/
43 #if (defined(_AP_N) || defined(_AP_C))
44
45 #error One or more of the following is defined: _AP_N, _AP_C.
46 Definition conflicts with their usage as template parameters.
47 #endif
48
49 /* for safety*/
50 #if (defined(_AP_W) || defined(_AP_I) || defined(_AP_S) ||
51 defined(_AP_Q) || \
52 defined(_AP_O) || defined(_AP_W2) || defined(_AP_I2)
53 ||
54 defined(_AP_S2) || defined(_AP_Q2) || defined(_AP_O2)
55 ||
56 defined(_AP_N) || defined(_AP_N2))
57 #error \
58 "One or more of the following is defined: _AP_W, _AP_I,
59 _AP_S, _AP_Q, _AP_O, _AP_N, _AP_W2, _AP_I2, _AP_S2, _AP_Q2,
60 _AP_O2, _AP_N2. Definition conflicts with their usage as template
61 parameters."
62 #endif
63
64 /*for safety*/
65
66 #if (defined(_AP_W3) || defined(_AP_S3) || defined(_AP_W4) ||
67 defined(_AP_S4))
68 #error \
69 "One or more of the following is defined: _AP_W3, _AP_S3,
70 _AP_W4, _AP_S4. Definition conflicts with their usage as template
71 parameters."
72 #endif
73
74 #if (defined(_AP_W1) || defined(_AP_S1) || defined(_AP_T) ||
75 \
76 defined(_AP_T1) || defined(_AP_T2) || defined(_AP_T3) ||
77 defined(_AP_T4))
78 #error \
79 "One or more of the following is defined: _AP_W1, _AP_S1,

```

_AP_T, _AP_T1, _AP_T2, _AP_T3, _AP_T4. Definition conflicts with their usage as template parameters."

```

56 #endif
57
58 #ifndef __cplusplus
59 #error "AP data type can only be used in C++"
60 #endif
61
62 //
63
64 -----
65
66 #ifndef __SC_COMPATIBLE__
67 /// ap_fixed quantification mode
68 enum ap_q_mode {
69     AP_RND,          ///< rounding to plus infinity
70     AP_RND_ZERO,     ///< rounding to zero
71     AP_RND_MIN_INF,  ///< rounding to minus infinity
72     AP_RND_INF,      ///< rounding to infinity
73     AP_RND_CONV,     ///< convergent rounding
74     AP_TRN,          ///< truncation
75     AP_TRN_ZERO,     ///< truncation to zero
76 };
77
78 // FIXME for legacy code
79 #ifndef SYSTEMC_INCLUDED
80 #define SC_RND AP_RND
81 #define SC_RND_ZERO AP_RND_ZERO
82 #define SC_RND_MIN_INF AP_RND_MIN_INF
83 #define SC_RND_INF AP_RND_INF
84 #define SC_RND_CONV AP_RND_CONV
85 #define SC_TRN AP_TRN
86 #define SC_TRN_ZERO AP_TRN_ZERO
87 #endif // !defined(SYSTEMC_INCLUDED)
88
89 /// ap_fixed saturation mode
90 enum ap_o_mode {
91     AP_SAT,          ///< saturation
92     AP_SAT_ZERO,     ///< saturation to zero
93     AP_SAT_SYM,      ///< symmetrical saturation
94     AP_WRAP,         ///< wrap-around (*)
95     AP_WRAP_SM,      ///< sign magnitude wrap-around (*)
96 };
97
98 // FIXME for legacy code
99 #ifndef SYSTEMC_INCLUDED
100 #define SC_SAT AP_SAT
101 #define SC_SAT_ZERO AP_SAT_ZERO
102 #define SC_SAT_SYM AP_SAT_SYM
103 #define SC_WRAP AP_WRAP
104 #define SC_WRAP_SM AP_WRAP_SM
105 #endif // !defined(SYSTEMC_INCLUDED)
106
107 #else // defined(__SC_COMPATIBLE__)
108 // There will not be sc_fxdefs.h, and the emu should be
109 defined by ap_fixed.
110
111 /// ap_fixed quantification mode
112 enum ap_q_mode {
113     SC_RND,          ///< rounding to plus infinity
114     SC_RND_ZERO,     ///< rounding to zero
115     SC_RND_MIN_INF,  ///< rounding to minus infinity
116     SC_RND_INF,      ///< rounding to infinity
117     SC_RND_CONV,     ///< convergent rounding
118     SC_TRN,          ///< truncation
119     SC_TRN_ZERO,     ///< truncation to zero
120 };
121
122 #define AP_RND SC_RND
123 #define AP_RND_ZERO SC_RND_ZERO
124 #define AP_RND_MIN_INF SC_RND_MIN_INF
125 #define AP_RND_INF SC_RND_INF
126 #define AP_RND_CONV SC_RND_CONV
127 #define AP_TRN SC_TRN
128 #define AP_TRN_ZERO SC_TRN_ZERO
129
130 /// ap_fixed saturation mode
131 enum ap_o_mode {
132     SC_SAT,          ///< saturation
133     SC_SAT_ZERO,     ///< saturation to zero
134     SC_SAT_SYM,      ///< symmetrical saturation
135     SC_WRAP,         ///< wrap-around (*)
136     SC_WRAP_SM,      ///< sign magnitude wrap-around (*)
137 };
138
139 #define AP_SAT SC_SAT
140 #define AP_SAT_ZERO SC_SAT_ZERO
141 #define AP_SAT_SYM SC_SAT_SYM
142 #define AP_WRAP SC_WRAP
143 #define AP_WRAP_SM SC_WRAP_SM
144
145 #endif // defined(__SC_COMPATIBLE__)
146
147 template <int _AP_W, bool _AP_S>
148 struct ap_int_base;
149
150 template <int _AP_W>
151 struct ap_int;
152

```

```

151 template <int _AP_W>
152 struct ap_uint;
153
154 template <int _AP_W, bool _AP_S>
155 struct ap_range_ref;
156
157 template <int _AP_W, bool _AP_S>
158 struct ap_bit_ref;
159
160 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
161 _AP_T2>
162 struct ap_concat_ref;
163
164 template <int _AP_W, int _AP_I, bool _AP_S = true, ap_q_mode
165 _AP_Q = AP_TRN,
166         ap_o_mode _AP_O = AP_WRAP, int _AP_N = 0>
167 struct ap_fixed_base;
168
169 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q = AP_TRN,
170         ap_o_mode _AP_O = AP_WRAP, int _AP_N = 0>
171 struct ap_fixed;
172
173 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q = AP_TRN,
174         ap_o_mode _AP_O = AP_WRAP, int _AP_N = 0>
175 struct ap_ufixed;
176
177 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
178         ap_o_mode _AP_O, int _AP_N>
179 struct af_range_ref;
180
181 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
182         ap_o_mode _AP_O, int _AP_N>
183 struct af_bit_ref;
184
185 /// string base mode
186
187 enum BaseMode { AP_BIN = 2, AP_OCT = 8, AP_DEC = 10, AP_HEX =
188 16 };
189
190 #ifndef SYSTEMC_INCLUDED
191 #define SC_BIN 2
192 #define SC_OCT 8
193 #define SC_DEC 10
194 #define SC_HEX 16
195 #endif // !defined(SYSTEMC_INCLUDED)
196
197 // Alias C data types
198 #ifdef _MSC_VER
199 typedef signed __int64 ap_slong;
200 typedef unsigned __int64 ap_ulong;
201 #else // !defined(_MSC_VER)
202 typedef signed long long ap_slong;
203 typedef unsigned long long ap_ulong;
204 #endif // !defined(_MSC_VER)
205
206 enum {
207     _AP_SIZE_char = 8,
208     _AP_SIZE_short = sizeof(short) * 8,
209     _AP_SIZE_int = sizeof(int) * 8,
210     _AP_SIZE_long = sizeof(long) * 8,
211     _AP_SIZE_ap_slong = sizeof(ap_slong) * 8
212 };
213
214 #endif // !defined(__AP_DECL_H__)
215
216 // -*- cpp -*-

```

E.4 vendor/ap_fixed.h (360 lines, 12.185 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  * cense");
6  * you may not use this file except in compliance with the
7  * License.
8  * You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing,
13  * software
14  * distributed under the License is distributed on an "AS IS"
15  * BASIS,
16  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17  * press or implied.
18  * See the License for the specific language governing per-
19  * missions and
20  * limitations under the License.
21  */
22
23 #ifndef __AP_FIXED_H__
24 #define __AP_FIXED_H__
25
26 #include <ap_common.h>
27 #include <ap_fixed_base.h>

```

```

22 #include <ap_fixed_ref.h>
23
24 //-----
25
26 /// Signed Arbitrary Precision Fixed-Point Type.
27 // default for _AP_Q, _AP_O and _AP_N set in ap_decl.h
28 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
29 _AP_O, int _AP_N>
30 struct ap_fixed : ap_fixed_base<_AP_W, _AP_I, true, _AP_Q,
31 _AP_O, _AP_N> {
32     typedef ap_fixed_base<_AP_W, _AP_I, true, _AP_Q, _AP_O,
33 _AP_N> Base;
34     // Constructor
35     // default ctor
36     INLINE ap_fixed() : Base() {}
37
38     /// default copy ctor
39     INLINE ap_fixed(const ap_fixed& op) { Base::V = op.V; }
40
41     /// copy ctor from ap_fixed_base.
42     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
43 _AP_Q2,
44         ap_o_mode _AP_O2, int _AP_N2>
45     INLINE ap_fixed(const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2,
46 _AP_Q2,
47         _AP_O2, _AP_N2>& op)
48         : Base(op) {}
49
50     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
51 _AP_Q2,
52         ap_o_mode _AP_O2, int _AP_N2>
53     INLINE ap_fixed(const volatile ap_fixed_base<_AP_W2,
54 _AP_I2, _AP_S2, _AP_Q2,
55         _AP_O2, _AP_N2>& op)
56         : Base(op) {}
57
58     // from ap_fixed
59     //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
60 ap_o_mode _AP_O2,
61         int _AP_N2>
62     //INLINE ap_fixed(
63         const ap_fixed<_AP_W2, _AP_I2, _AP_Q2, _AP_O2,
64 _AP_N2>& op)
65         : Base(ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
66 _AP_O2, _AP_N2>(op)) {}
67
68     //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
69 ap_o_mode _AP_O2,
70         int _AP_N2>
71     //INLINE ap_fixed(
72         const ap_ufixed<_AP_W2, _AP_I2, _AP_Q2, _AP_O2,
73 _AP_N2>& op)
74         : Base(ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
75 _AP_O2, _AP_N2>(op)) {}
76
77     // from ap_ufixed.
78     //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
79 ap_o_mode _AP_O2,
80         int _AP_N2>
81     //INLINE ap_fixed(
82         const volatile ap_ufixed<_AP_W2, _AP_I2, _AP_Q2,
83 _AP_O2, _AP_N2>& op)
84         : Base(ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
85 _AP_O2, _AP_N2>(op)) {}
86
87     // copy ctor from ap_int_base.
88     template <int _AP_W2, bool _AP_S2>
89     INLINE ap_fixed(const ap_int_base<_AP_W2, _AP_S2>& op) :
90 Base(op) {}
91
92     template <int _AP_W2, bool _AP_S2>
93     INLINE ap_fixed(const volatile ap_int_base<_AP_W2, _AP_S2>&
94 op) : Base(op) {}
95
96     // from ap_int.
97     //template <int _AP_W2>
98     //INLINE ap_fixed(const volatile ap_int<_AP_W2>& op)
99     // : Base(ap_int_base<_AP_W2, true>(op)) {}
100
101     //template <int _AP_W2>
102     //INLINE ap_fixed(const volatile ap_int<_AP_W2>& op)
103     // : Base(ap_int_base<_AP_W2, true>(op)) {}
104
105     // from ap_uint.
106     //template <int _AP_W2>

```

```

97 //INLINE ap_fixed(const ap_uint<_AP_W2>& op)
98 //    : Base(ap_int_base<_AP_W2, false>(op)) {}
99
100 //template <int _AP_W2>
101 //INLINE ap_fixed(const volatile ap_uint<_AP_W2>& op)
102 //    : Base(ap_int_base<_AP_W2, false>(op)) {}
103
104 // from ap_bit_ref.
105 template <int _AP_W2, bool _AP_S2>
106 INLINE ap_fixed(const ap_bit_ref<_AP_W2, _AP_S2>& op) :
Base(op) {}
107
108 // from ap_range_ref.
109 template <int _AP_W2, bool _AP_S2>
110 INLINE ap_fixed(const ap_range_ref<_AP_W2, _AP_S2>& op) :
Base(op) {}
111
112 // from ap_concat_ref.
113
114 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
115
116 INLINE ap_fixed(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>& op)
117     : Base(op) {}
118
119 // from af_bit_ref.
120 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
121         ap_o_mode _AP_O2, int _AP_N2>
122 INLINE ap_fixed(
123     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2,
124     _AP_N2>& op)
125     : Base(op) {}
126
127 // from af_range_ref.
128 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
129         ap_o_mode _AP_O2, int _AP_N2>
130 INLINE ap_fixed(
131     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2,
132     _AP_N2>& op)
133     : Base(op) {}
134
135 // from c types.
136 #define CTOR(TYPE) \
137     INLINE ap_fixed(TYPE v) : Base(v) {}
138
139 CTOR(bool)
140 CTOR(char)
141 CTOR(signed char)
142 CTOR(unsigned char)
143 CTOR(short)
144 CTOR(unsigned short)
145 CTOR(int)
146 CTOR(unsigned int)
147 CTOR(long)
148 CTOR(unsigned long)
149 CTOR(ap_slong)
150 CTOR(ap_ulong)
151
152 #if _AP_ENABLE_HALF == 1
153 CTOR(half)
154 #endif
155
156 CTOR(float)
157 CTOR(double)
158 #undef CTOR
159
160 INLINE ap_fixed(const char* s) : Base(s) {}
161
162 INLINE ap_fixed(const char* s, signed char rd) : Base(s,
rd) {}
163
164 // Assignment
165 // The assignment operator is technically inherited; how-
ever, it is always
166
167 // hidden by an explicitly or implicitly defined assignment
operator for the
168 // derived class.
169 /* XXX ctor will be used when right is not of proper type.
*/
170
171 INLINE ap_fixed& operator=(
172     const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>& op)
173 {
174     Base::V = op.V;
175     return *this;
176 }
177
178 INLINE void operator=(
179     const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>& op)
180 volatile {
181     Base::V = op.V;
182 }
183
184
185 INLINE ap_fixed& operator=(
186     const volatile ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O,
_AP_N>& op) {
187     Base::V = op.V;
188     return *this;
189 }

```

```

179
180     INLINE void operator=(
181         const volatile ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_0,
182         _AP_N>& op) volatile {
183         Base::V = op.V;
184     }; // struct ap_fixed.
185
186 //-----
187
188 // Unsigned Arbitrary Precision Fixed-Point Type.
189 // default for _AP_Q, _AP_0 and _AP_N set in ap_decl.h
190 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
191 _AP_0, int _AP_N>
192 struct ap_ufixed : ap_fixed_base<_AP_W, _AP_I, false, _AP_Q,
193 _AP_0, _AP_N> {
194     typedef ap_fixed_base<_AP_W, _AP_I, false, _AP_Q, _AP_0,
195 _AP_N> Base;
196     // Constructor
197     /// default ctor
198     INLINE ap_ufixed() : Base() {}
199
200     /// default copy ctor
201     INLINE ap_ufixed(const ap_ufixed& op) { Base::V = op.V; }
202
203     /// copy ctor from ap_fixed_base
204     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
205 _AP_Q2,
206         ap_o_mode _AP_O2, int _AP_N2>
207     INLINE ap_ufixed(const ap_fixed_base<_AP_W2, _AP_I2,
208 _AP_S2, _AP_Q2,
209         _AP_O2, _AP_N2>& op)
210         : Base(op) {}
211
212     /// copy ctor from ap_fixed_base
213     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
214 _AP_Q2,
215         ap_o_mode _AP_O2, int _AP_N2>
216     INLINE ap_ufixed(const volatile ap_fixed_base<_AP_W2,
217 _AP_I2, _AP_S2, _AP_Q2,
218         _AP_O2, _AP_N2>& op)
219         : Base(op) {}
220
221 //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
222 ap_o_mode _AP_O2,
223         int _AP_N2>
224 //INLINE ap_ufixed(
225 //    const ap_fixed<_AP_W2, _AP_I2, _AP_Q2, _AP_O2,
226 _AP_N2>& op)
227 //    : Base(ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
228 _AP_O2, _AP_N2>(op)) {}
229
230 //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
231 ap_o_mode _AP_O2,
232         int _AP_N2>
233 //INLINE ap_ufixed(
234 //    const volatile ap_fixed<_AP_W2, _AP_I2, _AP_Q2,
235 _AP_O2, _AP_N2>& op)
236 //    : Base(ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
237 _AP_O2, _AP_N2>(op)) {}
238
239 //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
240 ap_o_mode _AP_O2,
241         int _AP_N2>
242 //INLINE ap_ufixed(
243 //    const ap_ufixed<_AP_W2, _AP_I2, _AP_Q2, _AP_O2,
244 _AP_N2>& op)
245 //    : Base(ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
246 _AP_O2, _AP_N2>(op)) {}
247
248 //template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
249 ap_o_mode _AP_O2,
250         int _AP_N2>
251 //INLINE ap_ufixed(
252 //    const volatile ap_ufixed<_AP_W2, _AP_I2, _AP_Q2,
253 _AP_O2, _AP_N2>& op)
254 //    : Base(ap_ufixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
255 _AP_O2, _AP_N2>(op)) {}
256
257 /// copy ctor from ap_int_base.
258 template <int _AP_W2, bool _AP_S2>
259 INLINE ap_ufixed(const ap_int_base<_AP_W2, _AP_S2>& op) :
260 Base(op) {}
261
262 template <int _AP_W2, bool _AP_S2>
263 INLINE ap_ufixed(const volatile ap_int_base<_AP_W2,
264 _AP_S2>& op) : Base(op) {}
265
266 //template <int _AP_W2>
267 //INLINE ap_ufixed(const ap_int<_AP_W2>& op)
268 //    : Base(ap_int_base<_AP_W2, true>(op)) {}
269
270 //template <int _AP_W2>
271 //INLINE ap_ufixed(const volatile ap_int<_AP_W2>& op)
272 //    : Base(ap_int_base<_AP_W2, true>(op)) {}

```



```

255 //template <int _AP_W2>
256 //INLINE ap_ufixed(const ap_uint<_AP_W2>& op)
257 // : Base(ap_int_base<_AP_W2, false>(op)) {}
258
259 //template <int _AP_W2>
260 //INLINE ap_ufixed(const volatile ap_uint<_AP_W2>& op)
261 // : Base(ap_int_base<_AP_W2, false>(op)) {}
262
263 template <int _AP_W2, bool _AP_S2>
264 INLINE ap_ufixed(const ap_bit_ref<_AP_W2, _AP_S2>& op) :
Base(op) {}
265
266 template <int _AP_W2, bool _AP_S2>
267 INLINE ap_ufixed(const ap_range_ref<_AP_W2, _AP_S2>& op) :
Base(op) {}
268
269 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
270 INLINE ap_ufixed(const ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>& op)
271 : Base(op) {}
272
273 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
274 ap_o_mode _AP_O2, int _AP_N2>
275 INLINE ap_ufixed(
276 const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op)
277 : Base(op) {}
278
279 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
280 ap_o_mode _AP_O2, int _AP_N2>
281 INLINE ap_ufixed(
282 const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op)
283 : Base(op) {}
284
285 #define CTOR(TYPE) \
286 INLINE ap_ufixed(TYPE v) : Base(v) {}
287
288 CTOR(bool)
289 CTOR(char)
290 CTOR(signed char)
291 CTOR(unsigned char)
292 CTOR(short)
293 CTOR(unsigned short)
294 CTOR(int)
295 CTOR(unsigned int)
296 CTOR(long)
297 CTOR(unsigned long)
298 CTOR(ap_slong)
299 CTOR(ap_ulong)
300 #if _AP_ENABLE_HALF_ == 1
301 CTOR(half)
302 #endif
303 CTOR(float)
304 CTOR(double)
305 #undef CTOR
306
307 INLINE ap_ufixed(const char* s) : Base(s) {}
308
309 INLINE ap_ufixed(const char* s, signed char rd) : Base(s,
rd) {}
310
311 // Assignment
312 INLINE ap_ufixed& operator=(
313 const ap_ufixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>& op) {
314 Base::V = op.V;
315 return *this;
316 }
317
318 INLINE void operator=(
319 const ap_ufixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N>& op)
volatile {
320 Base::V = op.V;
321 }
322
323 INLINE ap_ufixed& operator=(
324 const volatile ap_ufixed<_AP_W, _AP_I, _AP_Q, _AP_O,
_AP_N>& op) {
325 Base::V = op.V;
326 return *this;
327 }
328
329 INLINE void operator=(const volatile ap_ufixed<_AP_W,
_AP_I, _AP_Q, _AP_O,
330 _AP_N>& op)
volatile {
331 Base::V = op.V;
332 }
333 }; // struct ap_ufixed
334
335
336 #if !defined(__SYNTHESIS__) && (defined(SYSTEMC_H) ||
defined(SYSTEMC_INCLUDED))
337 // XXX sc_trace overload for ap_fixed is already included in

```

```

338 // "ap_sysc/ap_sc_extras.h", so do not define in synthesis.
339 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
_AP_O, int _AP_N>
340 INLINE void sc_trace(sc_core::sc_trace_file* tf,
341 const ap_fixed<_AP_W, _AP_I, _AP_Q,
_AP_O, _AP_N>& op,
342 const std::string& name) {
343 tf->trace(sc_dt::sc_lv<_AP_W>(op.to_string(2).c_str()),
name);
344 }
345
346 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
_AP_O, int _AP_N>
347 INLINE void sc_trace(sc_core::sc_trace_file* tf,
348 const ap_ufixed<_AP_W, _AP_I, _AP_Q,
_AP_O, _AP_N>& op,
349 const std::string& name) {
350 tf->trace(sc_dt::sc_lv<_AP_W>(op.to_string(2).c_str()),
name);
351 }
352 #endif // System C sim
353
354 // Specialization of std containers, so that std::com-
plex<ap_fixed> can have its
355 // image part automatically zero-initialized when only real
part is provided.
356 #include <ap_fixed_special.h>
357
358 #endif // ifndef __AP_FIXED_H__
359
360 // -*- cpp -*-

```

E.5 vendor/ap_fixed_base.h (2,354 lines, 85,182 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
5  * you may not use this file except in compliance with the
License.
6  * You may obtain a copy of the License at
7  *
8  * http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-
missions and
14 * limitations under the License.
15 */
16
17 #ifndef __AP_FIXED_BASE_H__
18 #define __AP_FIXED_BASE_H__
19
20 #ifndef __AP_FIXED_H__
21 #error "Only ap_fixed.h and ap_int.h can be included directly in
user code."
22 #endif
23
24 // for ap_int_base and its reference types.
25 #include <ap_int.h>
26 #ifndef __SYNTHESIS__
27 #if _AP_ENABLE_HALF_ == 1
28 // for half type
29 #include <hls_half.h>
30 #endif
31 // for std io
32 #include <iostream>
33 #endif
34
35 #ifndef __cplusplus
36 #error "C++ is required to include this header file"
37 #else // __cplusplus
38
39 // for warning on unsupported rounding mode in conversion to
float/double.
40 #if !defined(__SYNTHESIS__) && __cplusplus >= 201103L && \
(defined(__gnu_linux__) || defined(_WIN32))
41 #define AP_FIXED_ENABLE_CPP_FENV 1
42 #include <cfenv>
43 #endif
44
45 //
-----
47
48 /* Major TODO
49 long double support: constructor, assign and other operat-
ors.
50 binary operators with ap_fixed_base and const char*.
51 return ap_fixed/ap_ufixed when result signedness is known.
52 */

```

```

53
54 // Helper function in conversion to floating point types.
55
56 #ifndef __SYNTHESIS__
57 #define _AP_ctype_op_get_bit(var, index)
58 #define _AP_ctype_op_set_bit(var, index, x)
59 #define _AP_ctype_op_get_range(var, low, high) \
60 _AP_ROOT_op_get_range(var, low, high)
61 #define _AP_ctype_op_set_range(var, low, high, x) \
62 _AP_ROOT_op_set_range(var, low, high, x)
63 #else // ifdef __SYNTHESIS__
64 template <typename _Tp1, typename _Tp2>
65 inline bool _AP_ctype_op_get_bit(_Tp1& var, const _Tp2& in-
dex) {
66     return !!(var & (1ull << (index)));
67 }
68 template <typename _Tp1, typename _Tp2, typename _Tp3>
69 inline _Tp1 _AP_ctype_op_set_bit(_Tp1& var, const _Tp2& in-
dex, const _Tp3& x) {
70     var |= (((x) ? 1ull : 0ull) << (index));
71     return var;
72 }
73 template <typename _Tp1, typename _Tp2, typename _Tp3>
74 inline _Tp1 _AP_ctype_op_get_range(_Tp1& var, const _Tp2&
low,
75                                     const _Tp3& high) {
76     _Tp1 r = var;
77     ap_ulong mask = -1ll;
78     mask >>= (sizeof(_Tp1) * 8 - ((high) - (low) + 1));
79     r >>= (low);
80     r &= mask;
81     return r;
82 }
83 template <typename _Tp1, typename _Tp2, typename _Tp3, type-
name _Tp4>
84 inline _Tp1 _AP_ctype_op_set_range(_Tp1& var, const _Tp2&
low, const _Tp3& high,
85                                     const _Tp4& x) {
86     ap_ulong mask = -1ll;
87     mask >>= (_AP_SIZE_ap_slong - ((high) - (low) + 1));
88     var &= ~(mask << (low));
89     var |= ((mask & x) << (low));
90     return var;
91 }
92 #endif // ifdef __SYNTHESIS__
93
94
95 // trait for letting base class to return derived class.
96 // Notice that derived class template is incomplete, and we
cannot use
97 // the member of the derived class.
98 template <int _AP_W2, int _AP_I2, bool _AP_S2>
99 struct _ap_fixed_factory;
100 template <int _AP_W2, int _AP_I2>
101 struct _ap_fixed_factory<_AP_W2, _AP_I2, true> {
102     typedef ap_fixed<_AP_W2, _AP_I2> type;
103 };
104 template <int _AP_W2, int _AP_I2>
105 struct _ap_fixed_factory<_AP_W2, _AP_I2, false> {
106     typedef ap_ufixed<_AP_W2, _AP_I2> type;
107 };
108
109 /// ap_fixed_base: AutoPilot fixed point.
110 /** partial specialization of signed.
111 @tparam _AP_W width.
112 @tparam _AP_I integral part width.
113 @tparam _AP_S signed.
114 @tparam _AP_Q quantization mode. Default is AP_TRN.
115 @tparam _AP_O saturation mode. Default is AP_WRAP.
116 @tparam _AP_N saturation wrap value. Default is 0.
117 */
118 // default for _AP_Q, _AP_O and _AP_N set in ap_decl.h
119 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
120             int _AP_N>
121 struct ap_fixed_base : _AP_ROOT_TYPE<_AP_W, _AP_S> {
122 public:
123     typedef _AP_ROOT_TYPE<_AP_W, _AP_S> Base;
124     static const int width = _AP_W;
125     static const int iwidth = _AP_I;
126     static const ap_q_mode qmode = _AP_Q;
127     static const ap_o_mode omode = _AP_O;
128
129     /// Return type trait.
130     template <int _AP_W2, int _AP_I2, bool _AP_S2>
131     struct RType {
132     enum {
133         _AP_F = _AP_W - _AP_I,
134         F2 = _AP_W2 - _AP_I2,
135         mult_w = _AP_W + _AP_W2,
136         mult_i = _AP_I + _AP_I2,
137         mult_s = _AP_S || _AP_S2,
138         plus_w = AP_MAX(_AP_I + (_AP_S2 && !_AP_S), _AP_I2 +
(_AP_S && !_AP_S2)) +
139             1 + AP_MAX(_AP_F, F2),
140         plus_i =
141             AP_MAX(_AP_I + (_AP_S2 && !_AP_S), _AP_I2 + (_AP_S
&& !_AP_S2)) + 1,

```

```

142         plus_s = _AP_S || _AP_S2,
143         minus_w =
144             AP_MAX(_AP_I + (_AP_S2 && !_AP_S), _AP_I2 + (_AP_S
&& !_AP_S2)) + 1 +
145             AP_MAX(_AP_F, F2),
146         minus_i =
147             AP_MAX(_AP_I + (_AP_S2 && !_AP_S), _AP_I2 + (_AP_S
&& !_AP_S2)) + 1,
148         minus_s = true,
149         #ifndef __SC_COMPATIBLE__
150         div_w = _AP_S2 + _AP_W + AP_MAX(F2, 0),
151         #else
152         div_w = _AP_S2 + _AP_W + AP_MAX(F2, 0) + AP_MAX(_AP_I2,
0),
153         #endif
154         div_i = _AP_S2 + _AP_I + F2,
155         div_s = _AP_S || _AP_S2,
156         logic_w =
157             AP_MAX(_AP_I + (_AP_S2 && !_AP_S), _AP_I2 + (_AP_S
&& !_AP_S2)) +
158             AP_MAX(_AP_F, F2),
159         logic_i = AP_MAX(_AP_I + (_AP_S2 && !_AP_S), _AP_I2 +
(_AP_S && !_AP_S2)),
160         logic_s = _AP_S || _AP_S2
161     };
162     typedef ap_fixed_base<_AP_W, _AP_I, _AP_S> lhs;
163     typedef ap_fixed_base<_AP_W2, _AP_I2, _AP_S2> rhs;
164     typedef ap_fixed_base<mult_w, mult_i, mult_s> mult_base;
165     typedef ap_fixed_base<plus_w, plus_i, plus_s> plus_base;
166     typedef ap_fixed_base<minus_w, minus_i, minus_s>
minus_base;
167     typedef ap_fixed_base<logic_w, logic_i, logic_s> lo-
gic_base;
168     typedef ap_fixed_base<div_w, div_i, div_s> div_base;
169     typedef ap_fixed_base<_AP_W, _AP_I, _AP_S> arg1_base;
170     typedef typename _ap_fixed_factory<mult_w, mult_i,
mult_s>::type mult;
171     typedef typename _ap_fixed_factory<plus_w, plus_i,
plus_s>::type plus;
172     typedef typename _ap_fixed_factory<minus_w, minus_i,
minus_s>::type minus;
173     typedef typename _ap_fixed_factory<logic_w, logic_i, lo-
gic_s>::type logic;
174     typedef typename _ap_fixed_factory<div_w, div_i,
div_s>::type div;
175     typedef typename _ap_fixed_factory<_AP_W, _AP_I,
_AP_S>::type arg1;
176     };
177 private:
178     #ifndef __SYNTHESIS__
179     // This cannot handle hex float format string.
180     void fromString(const std::string& val, unsigned char
radix) {
181         _AP_ERROR(!(radix == 2 || radix == 8 || radix == 10 ||
radix == 16),
182                 "ap_fixed_base::fromString(%s, %d)",
val.c_str(), radix);
183     }
184     Base::V = 0;
185     int startPos = 0;
186     int endPos = val.length();
187     int decPos = val.find(".");
188     if (decPos == -1) decPos = endPos;
189
190     // handle sign
191     bool isNegative = false;
192     if (val[0] == '-') {
193         isNegative = true;
194         ++startPos;
195     } else if (val[0] == '+')
196         ++startPos;
197
198     // If there are no integer bits, e.g.:
199     // .0000XXXX, then keep at least one bit.
200     // If the width is greater than the number of integer
bits, e.g.:
201     // XXXX.XXXX, then we keep the integer bits
202     // if the number of integer bits is greater than the
width, e.g.:
203     // XXX000 then we keep the integer bits.
204     // Always keep one bit.
205
206     ap_fixed_base<AP_MAX(_AP_I, 4) + 4, AP_MAX(_AP_I, 4) + 4,
false>
207         integer_bits = 0;
208
209     // Figure out if we can shift instead of multiply
210     unsigned shift = (radix == 16 ? 4 : radix == 8 ? 3 :
radix == 2 ? 1 : 0);
211
212     //std::cout << "\n\n" << val << "\n";
213
214     //std::cout << startPos << " " << decPos << " " << endPos
<< "\n";
215

```

```

218     bool sticky_int = false;
219
220     // Traverse the integer digits from the MSD, multiplying
    by radix as we go.
221     for (int i = startPos; i < decPos; i++) {
222         // Get a digit
223         char cdigit = val[i];
224         if (cdigit == '\0') continue;
225         unsigned digit = ap_private_ops::decode_digit(cdigit,
    radix);
226
227         sticky_int |= integer_bits[AP_MAX(_AP_I, 4) + 4 - 1] |
228                     integer_bits[AP_MAX(_AP_I, 4) + 4 - 2] |
229                     integer_bits[AP_MAX(_AP_I, 4) + 4 - 3] |
230                     integer_bits[AP_MAX(_AP_I, 4) + 4 - 4];
231         // Shift or multiply the value by the radix
232         if (shift)
233             integer_bits <<= shift;
234         else
235             integer_bits *= radix;
236
237         // Add in the digit we just interpreted
238         integer_bits += digit;
239         //std::cout << "idigit = " << digit << " " << in-
    teger_bits.to_string()
240         // << " " << sticky_int << "\n";
241     }
242     integer_bits[AP_MAX(_AP_I, 4) + 4 - 3] =
243         integer_bits[AP_MAX(_AP_I, 4) + 4 - 3] | sticky_int;
244
245     ap_fixed_base<AP_MAX(_AP_W - _AP_I, 0) + 4 + 4, 4, false>
    fractional_bits = 0;
246     bool sticky = false;
247
248     // Traverse the fractional digits from the LSD, dividing
    by radix as we go.
249     for (int i = endPos - 1; i >= decPos + 1; i--) {
250         // Get a digit
251         char cdigit = val[i];
252         if (cdigit == '\0') continue;
253         unsigned digit = ap_private_ops::decode_digit(cdigit,
    radix);
254         // Add in the digit we just interpreted
255         fractional_bits += digit;
256
257         sticky |= fractional_bits[0] | fractional_bits[1] |
    fractional_bits[2] |
258                 fractional_bits[3];
259         // Shift or divide the value by the radix
260         if (shift)
261             fractional_bits >>= shift;
262         else
263             fractional_bits /= radix;
264
265         //std::cout << "fdigit = " << digit << " " << fraction-
    al_bits.to_string()
266         // << " " << sticky << "\n";
267     }
268
269     //std::cout << "Int = " << integer_bits.to_string() << " "
    <<
270     // fractional_bits.to_string() << "\n";
271
272     fractional_bits[0] = fractional_bits[0] | sticky;
273
274     if (isNegative)
275         *this = -(integer_bits + fractional_bits);
276     else
277         *this = integer_bits + fractional_bits;
278
279     //std::cout << "end = " << this->to_string(16) << "\n";
280 }
281
282 /// report invalid construction of ap_fixed_base
283 inline void report() {
284     if (!_AP_S && _AP_O == AP_WRAP_SM) {
285         fprintf(stderr, "ap_ufixed<...> cannot support
    AP_WRAP_SM.\n");
286         exit(1);
287     }
288     if (_AP_W > MAX_MODE(AP_INT_MAX_W) * 1024) {
289         fprintf(stderr,
290             "[E] ap_%sfixed<%d, ...>: Bitwidth exceeds the
291             "default max value %d. Please use macro "
292             "AP_INT_MAX_W to set a larger max value.\n",
293             _AP_S ? "" : "u", _AP_W, MAX_MODE(AP_INT_MAX_W) *
    1024);
294         exit(1);
295     }
296 }
297 #else
298 inline void report() {}
299 #endif // ifdef __SYNTHESIS__
300
301 /// @name helper functions.
302 // @{

```

```

303 inline void overflow_adjust(bool underflow, bool overflow,
    bool ld,
304                             bool sign) {
305     if (!underflow && !overflow) return;
306     if (_AP_O == AP_WRAP) {
307         if (_AP_N == 0) return;
308         if (_AP_S) {
309             // signed AP_WRAP
310             // n_bits == 1
311             Base::V = _AP_ROOT_op_set_bit(Base::V, _AP_W - 1,
    sign);
312             if (_AP_N > 1) {
313                 // n_bits > 1
314                 ap_int_base<_AP_W, false> mask(-1);
315                 if (sign) mask.V = 0;
316                 Base::V =
317                     _AP_ROOT_op_set_range(Base::V, _AP_W - _AP_N,
    _AP_W - 2, mask.V);
318             }
319             } else {
320                 // unsigned AP_WRAP
321                 ap_int_base<_AP_W, false> mask(-1);
322                 Base::V =
323                     _AP_ROOT_op_set_range(Base::V, _AP_W - _AP_N,
    _AP_W - 1, mask.V);
324             }
325             } else if (_AP_O == AP_SAT_ZERO) {
326                 Base::V = 0;
327             } else if (_AP_O == AP_WRAP_SM && _AP_S) {
328                 bool Ro = _AP_ROOT_op_get_bit(Base::V, _AP_W - 1);
329                 if (_AP_N == 0) {
330                     if (ld != Ro) {
331                         Base::V = ~Base::V;
332                         Base::V = _AP_ROOT_op_set_bit(Base::V, _AP_W - 1,
    ld);
333                     }
334                 } else {
335                     if (_AP_N == 1 && sign != Ro) {
336                         Base::V = ~Base::V;
337                     } else if (_AP_N > 1) {
338                         bool lNo = _AP_ROOT_op_get_bit(Base::V, _AP_W -
    _AP_N);
339                         if (lNo == sign) Base::V = ~Base::V;
340                         ap_int_base<_AP_W, false> mask(-1);
341                         if (sign) mask.V = 0;
342                         Base::V =
343                             _AP_ROOT_op_set_range(Base::V, _AP_W - _AP_N,
    _AP_W - 2, mask.V);
344                     }
345                     Base::V = _AP_ROOT_op_set_bit(Base::V, _AP_W - 1,
    sign);
346                 }
347             } else {
348                 if (_AP_S) {
349                     if (overflow) {
350                         Base::V = 1;
351                         Base::V <= _AP_W - 1;
352                         Base::V = ~Base::V;
353                     } else if (underflow) {
354                         Base::V = 1;
355                         Base::V <= _AP_W - 1;
356                         if (_AP_O == AP_SAT_SYM) Base::V |= 1;
357                     }
358                 } else {
359                     if (overflow)
360                         Base::V = ~(ap_int_base<_AP_W, false>(0).V);
361                     else if (underflow)
362                         Base::V = 0;
363                 }
364             }
365         }
366     }
367
368     inline bool quantization_adjust(bool qb, bool r, bool s) {
369         bool carry = (bool)_AP_ROOT_op_get_bit(Base::V, _AP_W -
    1);
370         if (_AP_Q == AP_TRN) return false;
371         if (_AP_Q == AP_RND_ZERO)
372             qb &= s || r;
373         else if (_AP_Q == AP_RND_MIN_INF)
374             qb &= r;
375         else if (_AP_Q == AP_RND_INF)
376             qb &= !s || r;
377         else if (_AP_Q == AP_RND_CONV)
378             qb &= _AP_ROOT_op_get_bit(Base::V, 0) || r;
379         else if (_AP_Q == AP_TRN_ZERO)
380             qb = s && (qb || r);
381         Base::V += qb;
382         return carry && (!(bool)_AP_ROOT_op_get_bit(Base::V,
    _AP_W - 1));
383     }
384     // @}
385
386 public:
387     /// @name constructors.
388     // @{
389     /// default ctor.
390     inline ap_fixed_base() {}
391
392     /// copy ctor.
393     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode

```



```

_AP_Q2,
393     ap_o_mode _AP_Q2, int _AP_N2>
394     INLINE ap_fixed_base(
395         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op) {
396         operator=(op);
397         report();
398     }
399
400     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
401         ap_o_mode _AP_Q2, int _AP_N2>
402     INLINE ap_fixed_base(
403         const volatile ap_fixed_base<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_Q2, _AP_N2>& op) {
404         operator=(op);
405         report();
406     }
407
408     template <int _AP_W2, bool _AP_S2>
409     INLINE ap_fixed_base(const ap_int_base<_AP_W2, _AP_S2>& op) {
410         ap_fixed_base<_AP_W2, _AP_W2, _AP_S2> tmp;
411         tmp.V = op.V;
412         operator=(tmp);
413         report();
414     }
415
416     template <int _AP_W2, bool _AP_S2>
417     INLINE ap_fixed_base(const volatile ap_int_base<_AP_W2,
_AP_S2>& op) {
418         ap_fixed_base<_AP_W2, _AP_W2, _AP_S2> tmp;
419         tmp.V = op.V;
420         operator=(tmp);
421         report();
422     }
423
424     #ifndef __SYNTHESIS__
425     #ifndef NON_C99STRING
426     INLINE ap_fixed_base(const char* s, signed char rd = 0) {
427         unsigned char radix = rd;
428         std::string str = ap_private_ops::parseString(s,
radix); // will guess rd, default 10
429         _AP_ERROR(radix == 0, "ap_fixed_base(const char* \"%s\",
%d), str=%s, radix = %d",
430             s, rd, str.c_str(), radix); // TODO remove this
check
431         fromString(str, radix);
432     }
433     #else
434     INLINE ap_fixed_base(const char* s, signed char rd = 10) {
435         ap_int_base<_AP_W, _AP_S> t(s, rd);
436         Base::V = t.V;
437     }
438     #endif // ifndef NON_C99STRING
439     #else // ifndef __SYNTHESIS__
440     // XXX _ssdm_string2bits only takes const string and const
radix.
441     // It seems XFORM will do compile time processing of the
string.
442     INLINE ap_fixed_base(const char* s) {
443         typedef(Base::V) t;
444         _ssdm_string2bits((void*)&t, (const char*)(s), 10,
_AP_I, _AP_S, _AP_Q,
445             _AP_O, _AP_N, _AP_C99);
446         Base::V = t;
447     }
448     INLINE ap_fixed_base(const char* s, signed char rd) {
449         typedef(Base::V) t;
450         _ssdm_string2bits((void*)&t, (const char*)(s), rd,
_AP_I, _AP_S, _AP_Q,
451             _AP_O, _AP_N, _AP_C99);
452         Base::V = t;
453     }
454     #endif // ifndef __SYNTHESIS__ else
455
456     template <int _AP_W2, bool _AP_S2>
457     INLINE ap_fixed_base(const ap_bit_ref<_AP_W2, _AP_S2>& op)
{
458         *this = ((bool)op);
459         report();
460     }
461
462     template <int _AP_W2, bool _AP_S2>
463     INLINE ap_fixed_base(const ap_range_ref<_AP_W2, _AP_S2>&
op) {
464         *this = (ap_int_base<_AP_W2, false>(op));
465         report();
466     }
467
468     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
469     INLINE ap_fixed_base(
470         const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
op) {
471         *this = (ap_int_base<_AP_W2 + _AP_W3, false>(op));
472         report();
473     }
474
475     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
476         ap_o_mode _AP_Q2, int _AP_N2>
477     INLINE ap_fixed_base(
478         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op) {
479         *this = (bool(op));
480         report();
481     }
482
483     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
484         ap_o_mode _AP_Q2, int _AP_N2>
485     INLINE ap_fixed_base(
486         const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op) {
487         *this = (ap_int_base<_AP_W2, false>(op));
488         report();
489     }
490
491     // ctors from c types.
492     // make a temp ap_fixed_base first, and use
ap_fixed_base.operator=
493     #define CTOR_FROM_INT(C_TYPE, _AP_W2, _AP_S2) \
494     INLINE ap_fixed_base(const C_TYPE x) { \
495         ap_fixed_base<(_AP_W2), (_AP_W2), (_AP_S2)> tmp; \
496         tmp.V = x; \
497         *this = tmp; \
498     }
499
500     CTOR_FROM_INT(bool, 1, false)
501     CTOR_FROM_INT(char, 8, CHAR_IS_SIGNED)
502     CTOR_FROM_INT(signed char, 8, true)
503     CTOR_FROM_INT(unsigned char, 8, false)
504     CTOR_FROM_INT(short, _AP_SIZE_short, true)
505     CTOR_FROM_INT(unsigned short, _AP_SIZE_short, false)
506     CTOR_FROM_INT(int, _AP_SIZE_int, true)
507     CTOR_FROM_INT(unsigned int, _AP_SIZE_int, false)
508     CTOR_FROM_INT(long, _AP_SIZE_long, true)
509     CTOR_FROM_INT(unsigned long, _AP_SIZE_long, false)
510     CTOR_FROM_INT(ap_slong, _AP_SIZE_ap_slong, true)
511     CTOR_FROM_INT(ap_ulong, _AP_SIZE_ap_slong, false)
512     #undef CTOR_FROM_INT
513     /*
514     * TODO:
515     * There used to be several funtions which were AP_WEAK.
516     * Now they're all INLINE expect ap_fixed_base(double d)
517     * Maybe we can use '#pragma HLS inline' instead of INLINE.
518     */
519     AP_WEAK ap_fixed_base(double d) {
520         ap_int_base<64, false> ireg;
521         ireg.V = doubleToRawBits(d);
522         bool isneg = _AP_ROOT_op_get_bit(ireg.V, 63);
523
524         ap_int_base<DOUBLE_EXP + 1, true> exp;
525         ap_int_base<DOUBLE_EXP, false> exp_tmp;
526         exp_tmp.V =
527             _AP_ROOT_op_get_range(ireg.V, DOUBLE_MAN, DOUBLE_MAN
+ DOUBLE_EXP - 1);
528         exp = exp_tmp - DOUBLE_BIAS;
529         ap_int_base<DOUBLE_MAN + 2, true> man;
530         man.V = _AP_ROOT_op_get_range(ireg.V, 0, DOUBLE_MAN - 1);
531         // do not support NaN
532
533         _AP_WARNING(exp == APFX_IEEE_DOUBLE_E_MAX + 1 && man.V !=
0,
534             "assign NaN to fixed point value");
535         man.V = _AP_ROOT_op_set_bit(man.V, DOUBLE_MAN, 1);
536         if (isneg) man = -man;
537         if ((ireg.V & 0x7fffffffLL) == 0) {
538             Base::V = 0;
539         } else {
540             int _AP_W2 = DOUBLE_MAN + 2, _AP_I2 = exp.V + 2, _AP_F
= _AP_W - _AP_I,
541                 F2 = _AP_W2 - _AP_I2;
542             bool _AP_S2 = true,
543                 QUAN_INC = F2 > _AP_F &&
544                     !(_AP_Q == AP_TRN || (_AP_Q ==
AP_TRN_ZERO && !_AP_S2));
545             bool carry = false;
546             // handle quantization
547             unsigned sh_amt = (F2 > _AP_F) ? F2 - _AP_F : _AP_F -
F2;
548             if (F2 == _AP_F)
549                 Base::V = man.V;
550             else if (F2 > _AP_F) {
551                 if (sh_amt < DOUBLE_MAN + 2)
552                     Base::V = man.V >> sh_amt;
553                 else {
554                     Base::V = isneg ? -1 : 0;
555                 }
556                 if ((_AP_Q != AP_TRN) && !((_AP_Q == AP_TRN_ZERO)
&& !_AP_S2)) {
557                     bool qb = (F2 - _AP_F > _AP_W2) ? isneg :
558                         (bool)_AP_ROOT_op_get_bit(
559                             man.V, F2 - _AP_F - 1);
559                     bool r =
560                         (F2 > _AP_F + 1)

```

```

560      ? _AP_ROOT_op_get_range(man.V, 0, (F2 -
_AP_F - 2 < _AP_W2)
561      ? (F2 -
_AP_F - 2)
562      :
563      : false;
564      carry = quantization_adjust(qb, r, isneg);
565      }
566      } else { // no quantization
567      Base::V = man.V;
568      if (sh_amt < _AP_W)
569      Base::V = Base::V << sh_amt;
570      else
571      Base::V = 0;
572      }
573      // handle overflow/underflow
574      if ((_AP_0 != AP_WRAP || _AP_N != 0) &&
575      (!_AP_S && _AP_S2) ||
576      !_AP_I - _AP_S <
577      _AP_I2 - _AP_S2 +
578      (QUAN_INC ||
579      (_AP_S2 && (_AP_0 == AP_SAT_SYM)))) { //
saturation
580      bool deleted_zeros = _AP_S2 ? true : !carry, de-
leted_ones = true;
581      bool neg_src = isneg;
582      bool ld = false;
583      int pos1 = F2 - _AP_F + _AP_W;
584      int pos2 = F2 - _AP_F + _AP_W + 1;
585      bool newsignbit = _AP_ROOT_op_get_bit(Base::V, _AP_W
- 1);
586      if (pos1 < _AP_W2 && pos1 >= 0)
587      // ld = _AP_ROOT_op_get_bit(man.V, pos1);
588      ld = (man.V >> pos1) & 1;
589      if (pos1 < _AP_W2) {
590      bool Range1_all_ones = true;
591      bool Range1_all_zeros = true;
592      bool Range2_all_ones = true;
593      ap_int_base<DOUBLE_MAN + 2, false> Range2;
594      ap_int_base<DOUBLE_MAN + 2, false> all_ones(-1);
595
596      if (pos2 >= 0 && pos2 < _AP_W2) {
597      // Range2.V = _AP_ROOT_op_get_range(man.V,
598      // pos2, _AP_W2 - 1);
599      Range2.V = man.V;
600      Range2.V >>= pos2;
601      Range2_all_ones = Range2 == (all_ones >> pos2);
602      } else if (pos2 < 0)
603      Range2_all_ones = false;
604      if (pos1 >= 0 && pos2 < _AP_W2) {
605      Range1_all_ones = Range2_all_ones && ld;
606      Range1_all_zeros = !Range2.V && !ld;
607      } else if (pos2 == _AP_W2) {
608      Range1_all_ones = ld;
609      Range1_all_zeros = !ld;
610      } else if (pos1 < 0) {
611      Range1_all_zeros = !man.V;
612      Range1_all_ones = false;
613      }
614
615      deleted_zeros =
616      deleted_zeros && (carry ? Range1_all_ones :
Range1_all_zeros);
617      deleted_ones =
618      carry ? Range2_all_ones && (pos1 < 0 || !ld) :
Range1_all_ones;
619      neg_src = isneg && !(carry && Range1_all_ones);
620      } else
621      neg_src = isneg && newsignbit;
622      bool neg_trg = _AP_S && newsignbit;
623      bool overflow = (neg_trg || !deleted_zeros) && !is-
neg;
624      bool underflow = (!neg_trg || !deleted_ones) &&
neg_src;
625      if ((_AP_0 == AP_SAT_SYM) && _AP_S2 && _AP_S)
626      underflow |=
627      neg_src &&
628      (_AP_W > 1 ? _AP_ROOT_op_get_range(Base::V, 0,
_AP_W - 2) == 0
629      : true);
630      overflow_adjust(underflow, overflow, ld, neg_src);
631      }
632      }
633      report();
634      }
635      // TODO more optimized implementation.
636      INLINE ap_fixed_base(float d) { *this =
ap_fixed_base(double(d)); }
637
638      #if _AP_ENABLE_HALF == 1
639      // TODO more optimized implementation.
640      INLINE ap_fixed_base(half d) { *this =
ap_fixed_base(double(d)); }
641      #endif
642      // @}
643
644

```

```

645      /// @name assign operator
646      /// assign, using another ap_fixed_base of same template
parameters.
647      /*
648      INLINE ap_fixed_base& operator=(
649      const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N>& op) {
650      Base::V = op.V;
651      return *this;
652      }
653      */
654
655      template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
656      ap_o_mode _AP_O2, int _AP_N2>
657      INLINE ap_fixed_base& operator=(
658      const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op) {
659
660      const int _AP_F = _AP_W - _AP_I;
661      const int F2 = _AP_W2 - _AP_I2;
662      const int QUAN_INC =
663      F2 > _AP_F && !(_AP_Q == AP_TRN || (_AP_Q ==
AP_TRN_ZERO && !_AP_S2));
664
665      if (!op) Base::V = 0;
666      bool carry = false;
667      bool signbit = _AP_ROOT_op_get_bit(op.V, _AP_W2 - 1);
668      bool isneg = signbit && _AP_S2;
669      if (F2 == _AP_F)
670      Base::V = op.V;
671      else if (F2 > _AP_F) {
672      unsigned int sh_amt = F2 - _AP_F;
673      // moves bits right, handle quantization.
674      if (sh_amt < _AP_W2) {
675      Base::V = op.V >> sh_amt;
676      } else {
677      Base::V = isneg ? -1 : 0;
678      }
679      if (_AP_Q != AP_TRN && !(_AP_Q == AP_TRN_ZERO && !
_AP_S2)) {
680      bool qbit = _AP_ROOT_op_get_bit(op.V, F2 - _AP_F -
1);
681      // bit after LSB.
682
683      bool qb = (F2 - _AP_F > _AP_W2) ? _AP_S2 && signbit :
qbit;
684
685      enum { hi = ((F2 - _AP_F - 2) < _AP_W2) ? (F2 - _AP_F -
2) : (_AP_W2 - 1) };
686      // bits after qb.
687      bool r = (F2 > _AP_F + 1) ?
(_AP_ROOT_op_get_range(op.V, 0, hi) != 0) : false;
688      carry = quantization_adjust(qb, r, isneg);
689      } else {
690      unsigned sh_amt = _AP_F - F2;
691      // moves bits left, no quantization
692      if (sh_amt < _AP_W) {
693      if (_AP_W > _AP_W2) {
694      // extend and then shift, avoid losing bits.
695      Base::V = op.V;
696      Base::V <= sh_amt;
697      } else {
698      // shift and truncate.
699      Base::V = op.V << sh_amt;
700      }
701      } else {
702      Base::V = 0;
703      }
704      // handle overflow/underflow
705      if ((_AP_0 != AP_WRAP || _AP_N != 0) &&
706      (!_AP_S && _AP_S2) ||
707      !_AP_I - _AP_S <
708      _AP_I2 - _AP_S2 +
709      (QUAN_INC || (_AP_S2 && _AP_0 ==
AP_SAT_SYM)))) { // saturation
710      bool deleted_zeros = _AP_S2 ? true : !carry;
711      bool deleted_ones = true;
712      bool neg_src = isneg;
713      bool newsignbit = _AP_ROOT_op_get_bit(Base::V, _AP_W -
1);
714      enum { pos1 = F2 - _AP_F + _AP_W, pos2 = F2 - _AP_F +
_AP_W + 1 };
715      bool ld = (pos1 < _AP_W2 && pos1 >= 0) ?
_AP_ROOT_op_get_bit(op.V, pos1)
716      : false;
717      if (pos1 < _AP_W2) {
718      bool Range1_all_ones = true;
719      bool Range1_all_zeros = true;
720      bool Range2_all_ones = true;
721      ap_int_base<_AP_W2, false> all_ones(-1);
722
723      if (pos2 < _AP_W2 && pos2 >= 0) {
724      ap_int_base<_AP_W2, false> Range2;
725
726      Range2.V = _AP_ROOT_op_get_range(op.V, pos2, _AP_W2 -
1);
727      Range2_all_ones = Range2 == (all_ones >> pos2);

```

```

727     } else if (pos2 < 0) {
728         Range2_all_ones = false;
729     }
730
731     if (pos1 >= 0 && pos2 < _AP_W2) {
732         ap_int_base<_AP_W2, false> Range1;
733
734         Range1.V = _AP_ROOT_op_get_range(op.V, pos1, _AP_W2 -
1);
735         Range1_all_ones = Range1 == (all_ones >> pos1);
736         Range1_all_zeros = !Range1.V;
737     } else if (pos2 == _AP_W2) {
738         Range1_all_ones = !D;
739         Range1_all_zeros = !D;
740     } else if (pos1 < 0) {
741         Range1_all_zeros = !op.V;
742         Range1_all_ones = false;
743     }
744
745     deleted_zeros =
746     deleted_zeros && (carry ? Range1_all_ones :
Range1_all_zeros);
747     deleted_ones =
748     carry ? Range2_all_ones && (pos1 < 0 || !D) :
Range1_all_ones;
749     neg_src = isneg && !(carry && Range1_all_ones);
750     } else
751     neg_src = isneg && newsignbit;
752     bool neg_trg = _AP_S && newsignbit;
753     bool overflow = (neg_trg || !deleted_zeros) && !isneg;
754     bool underflow = (!neg_trg || !deleted_ones) &&
neg_src;
755     if ((_AP_O == AP_SAT_SYM) && _AP_S2 && _AP_S)
756     underflow |=
757     neg_src &&
758     (_AP_W > 1 ? _AP_ROOT_op_get_range(Base::V, 0,
_AP_W - 2) == 0
759     : true);
760     overflow_adjust(underflow, overflow, LD, neg_src);
761     }
762     return *this;
763     } // operator=
764
765     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
766             ap_o_mode _AP_O2, int _AP_N2>
767     INLINE ap_fixed_base& operator=(
768     const volatile ap_fixed_base<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>& op) {
769     operator=(const_cast<const ap_fixed_base<_AP_W2, _AP_I2,
_AP_S2,
770     _AP_Q2, _AP_O2, _AP_N2>&>(op));
771     return *this;
772     }
773     /// Set this ap_fixed_base with ULL.
774     INLINE ap_fixed_base& setBits(ap_ulong bv) {
775     // TODO when ull is not be long enough...
776     Base::V = bv;
777     return *this;
778     }
779
780     /// Return a ap_fixed_base object whose this->V is assigned
by bv.
781     static INLINE ap_fixed_base bitsToFixed(ap_ulong bv) {
782     // TODO fix when ull is not be long enough...
783     ap_fixed_base t;
784     #ifndef __SYNTHESIS__
785     t.V = bv;
786     #else
787     t.V.set_bits(bv);
788     #endif
789     return t;
790     }
791
792     // Explicit conversion functions to ap_int_base.
793     /** Captures all integer bits, in truncate mode.
794     * @param[in] Cnative follow conversion from double to
int.
795     */
796     INLINE ap_int_base<AP_MAX(_AP_I, 1), _AP_S> to_ap_int_base(
797     bool Cnative = true) const {
798     ap_int_base<AP_MAX(_AP_I, 1), _AP_S> ret;
799     if (_AP_I == 0) {
800     ret.V = 0;
801     } else if (_AP_I > 0 && _AP_I <= _AP_W) {
802     ret.V = _AP_ROOT_op_get_range(Base::V, _AP_W - _AP_I,
_AP_W - 1);
803     } else if (_AP_I > _AP_W) {
804     ret.V = _AP_ROOT_op_get_range(Base::V, 0, _AP_W - 1);
805     ret.V <= (_AP_I - _AP_W);
806     }
807     /* Consider the following case
808     * float f = -7.5f;
809     * ap_fixed<8,4> t = f; // -8 0 0 0 . 0.5
810     * int i = t.to_int();
811     * the result should be -7 instead of -8.
812     * Therefore, after truncation, the value should be in-
created by 1.

```

```

813     * For (-1, 0), carry to MSB will happen, but result 0 is
still correct.
814     */
815     if (Cnative && _AP_I < _AP_W) {
816     // Follow C native data type, conversion from double to
int
817     if (_AP_S && _AP_ROOT_op_get_bit(Base::V, _AP_W - 1) &&
(_AP_I < _AP_W) &&
818     (_AP_ROOT_op_get_range(
819     Base::V, 0, _AP_I < 0 ? _AP_W - 1 : _AP_W -
_AP_I - 1) != 0))
820     ++ret;
821     } else {
822     // Follow OSCI library, conversion from sc_fixed to
sc_int
823     }
824     return ret;
825     };
826
827     public:
828     template <int _AP_W2, bool _AP_S2>
829     INLINE operator ap_int_base<_AP_W2, _AP_S2>() const {
830     return ap_int_base<_AP_W2, _AP_S2>(to_ap_int_base());
831     }
832
833     // Explicit conversion function to C built-in integral
type.
834     INLINE char to_char() const { return
to_ap_int_base().to_char(); }
835
836     INLINE int to_int() const { return
to_ap_int_base().to_int(); }
837
838     INLINE unsigned to_uint() const { return
to_ap_int_base().to_uint(); }
839
840     INLINE ap_slong to_int64() const { return
to_ap_int_base().to_int64(); }
841
842     INLINE ap_ulong to_uint64() const { return
to_ap_int_base().to_uint64(); }
843
844     /// covert function to double.
845     /** only round-half-to-even mode supported, does not obey
FE env. */
846     INLINE double to_double() const {
847     #if defined(AP_FIXED_ENABLE_CPP_FENV)
848     _AP_WARNING(std::fegetround() != FE_TONEAREST,
849     "Only FE_TONEAREST is supported");
850     #endif
851     enum { BITS = DOUBLE_MAN + DOUBLE_EXP + 1 };
852     if (!Base::V) return 0.0f;
853     bool s = _AP_S && _AP_ROOT_op_get_bit(Base::V, _AP_W -
1); // sign.
854     ap_int_base<_AP_W, false> tmp;
855     if (s)
856     tmp.V = -Base::V; // may truncate one bit extra from
neg in sim.
857     else
858     tmp.V = Base::V;
859     int l = tmp.countLeadingZeros(); //< number of leading
zeros.
860     int e = _AP_I - l - 1 + DOUBLE_BIAS; //< exponent
861     int lsb_index = _AP_W - l - 1 - DOUBLE_MAN;
862     // more than 0.5?
863     bool a = (lsb_index >= 2) ?
864     (_AP_ROOT_op_get_range(tmp.V, 0, lsb_index - 2) !=
0) : 0;
865     // round to even
866     a |= (lsb_index >= 0) ? _AP_ROOT_op_get_bit(tmp.V, lsb_in-
dex) : 0;
867     // ull is at least 64-bit
868     ap_ulong m;
869     // may actually left shift, ensure buffer is wide enough.
870     if (_AP_W > BITS) {
871     m = (lsb_index >= 1) ? (ap_ulong)(tmp.V >> (lsb_index -
1))
872     : (ap_ulong)(tmp.V << (1 - lsb_in-
dex));
873     } else {
874     m = (ap_ulong)tmp.V;
875     m = (lsb_index >= 1) ? (m >> (lsb_index - 1))
876     : (m << (1 - lsb_index));
877     }
878     m += a;
879     m >>= 1;
880     //std::cout << '\n' << std::hex << m << '\n'; // TODO de-
lete this
881     // carry to MSB, increase exponent
882     if (_AP_ctype_op_get_bit(m, DOUBLE_MAN + 1)) {
883     e += 1;
884     }
885     // set sign and exponent
886     m = _AP_ctype_op_set_bit(m, BITS - 1, s);
887     //std::cout << m << '\n'; // TODO delete this
888     m = _AP_ctype_op_set_range(m, DOUBLE_MAN, DOUBLE_MAN +

```

```

DOUBLE_EXP - 1, e);
889 //std::cout << std::hex << m << std::dec << std::endl; //
TODO delete this
890 // cast to fp
891 return rawBitsToDouble(m);
892 }
893
894 /// convert function to float.
895 /** only round-half-to-even mode supported, does not obey
FE env. */
896 INLINE float to_float() const {
897 #if defined(AP_FIXED_ENABLE_CPP_FENV)
898 _AP_WARNING(std::fegetround() != FE_TONEAREST,
899 "Only FE_TONEAREST is supported");
900 #endif
901 enum { BITS = FLOAT_MAN + FLOAT_EXP + 1 };
902 if (!Base::V) return 0.0f;
903 bool s = _AP_S && _AP_ROOT_op_get_bit(Base::V, _AP_W -
1); //< sign.
904 ap_int_base<_AP_W, false> tmp;
905 if (s)
906 tmp.V = -Base::V; // may truncate one bit extra from
neg in sim.
907 else
908 tmp.V = Base::V;
909 int l = tmp.countLeadingZeros(); //< number of leading
zeros.
910 int e = _AP_I - l - 1 + FLOAT_BIAS; //< exponent
911 int lsb_index = _AP_W - l - 1 - FLOAT_MAN;
912 // more than 0.5?
913 bool a = (lsb_index >= 2) ?
914 (_AP_ROOT_op_get_range(tmp.V, 0, lsb_index - 2) !=
0) : 0;
915 // round to even
916 a |= (lsb_index >= 0) ? _AP_ROOT_op_get_bit(tmp.V, lsb_in-
dex) : 0;
917 // ul is at least 32-bit
918 unsigned long m;
919 // may actually left shift, ensure buffer is wide enough.
920 if (_AP_W > BITS) {
921 m = (lsb_index >= 1) ? (unsigned long)(tmp.V >>
(lsb_index - 1))
922 : (unsigned long)(tmp.V << (1 -
lsb_index));
923 } else {
924 m = (unsigned long)tmp.V;
925 m = (lsb_index >= 1) ? (m >> (lsb_index - 1))
926 : (m << (1 - lsb_index));
927 }
928 m += a;
929 m >>= 1;
930 // carry to MSB, increase exponent
931 if (_AP_ctype_op_get_bit(m, FLOAT_MAN + 1)) {
932 e += 1;
933 }
934 // set sign and exponent
935 m = _AP_ctype_op_set_bit(m, BITS - 1, s);
936 m = _AP_ctype_op_set_range(m, FLOAT_MAN, FLOAT_MAN +
- 1, e);
937 // cast to fp
938 return rawBitsToFloat(m);
939 }
940
941 #if _AP_ENABLE_HALF_ == 1
942 /// convert function to half.
943 /** only round-half-to-even mode supported, does not obey
FE env. */
944 INLINE half to_half() const {
945 #if defined(AP_FIXED_ENABLE_CPP_FENV)
946 _AP_WARNING(std::fegetround() != FE_TONEAREST,
947 "Only FE_TONEAREST is supported");
948 #endif
949 enum { BITS = HALF_MAN + HALF_EXP + 1 };
950 if (!Base::V) return 0.0f;
951 bool s = _AP_S && _AP_ROOT_op_get_bit(Base::V, _AP_W -
1); //< sign.
952 ap_int_base<_AP_W, false> tmp;
953 if (s)
954 tmp.V = -Base::V; // may truncate one bit extra from
neg in sim.
955 else
956 tmp.V = Base::V;
957 int l = tmp.countLeadingZeros(); //< number of leading
zeros.
958 int e = _AP_I - l - 1 + HALF_BIAS; //< exponent
959 int lsb_index = _AP_W - l - 1 - HALF_MAN;
960 // more than 0.5?
961 bool a = (lsb_index >= 2) ?
962 (_AP_ROOT_op_get_range(tmp.V, 0, lsb_index - 2) !=
0) : 0;
963 // round to even
964 a |= (lsb_index >= 0) ? _AP_ROOT_op_get_bit(tmp.V, lsb_in-
dex) : 0;
965 // short is at least 16-bit
966 unsigned short m;
967 // may actually left shift, ensure buffer is wide enough.
968 if (_AP_W > BITS) {
969 m = (lsb_index >= 1) ? (unsigned short)(tmp.V >>
(lsb_index - 1))
970 : (unsigned short)(tmp.V << (1 -
lsb_index));
971 } else {
972 m = (unsigned short)tmp.V;
973 m = (lsb_index >= 1) ? (m >> (lsb_index - 1))
974 : (m << (1 - lsb_index));
975 }
976 m += a;
977 m >>= 1;
978 // carry to MSB, increase exponent
979 if (_AP_ctype_op_get_bit(m, HALF_MAN + 1)) {
980 e += 1;
981 }
982 // set sign and exponent
983 m = _AP_ctype_op_set_bit(m, BITS - 1, s);
984 m = _AP_ctype_op_set_range(m, HALF_MAN, HALF_MAN +
HALF_EXP - 1, e);
985 // cast to fp
986 return rawBitsToHalf(m);
987 }
988 #endif
989 // FIXME inherited from old code, this may loose precision!
990 INLINE operator long double() const { return (long
double)to_double(); }
991
992 INLINE operator double() const { return to_double(); }
993
994 INLINE operator float() const { return to_float(); }
995
996 #if _AP_ENABLE_HALF_ == 1
997 #if _AP_ENABLE_HALF_ == 1
998 INLINE operator half() const { return to_half(); }
999 #endif
1000
1001 INLINE operator bool() const { return (bool)Base::V != 0; }
1002
1003 INLINE operator char() const { return (char)to_int(); }
1004
1005 INLINE operator signed char() const { return (signed
char)to_int(); }
1006
1007 INLINE operator unsigned char() const { return (unsigned
char)to_int(); }
1008
1009 INLINE operator short() const { return (short)to_int(); }
1010
1011 INLINE operator unsigned short() const { return (unsigned
short)to_int(); }
1012
1013 INLINE operator int() const { return to_int(); }
1014
1015 INLINE operator unsigned int() const { return to_uint(); }
1016
1017 // FIXME don't assume data width...
1018 #ifdef __x86_64__
1019 INLINE operator long() const { return (long)to_int64(); }
1020
1021 INLINE operator unsigned long() const { return (unsigned
long)to_int64(); }
1022 #else
1023 INLINE operator long() const { return (long)to_int(); }
1024
1025 INLINE operator unsigned long() const { return (unsigned
long)to_int(); }
1026 #endif // ifdef __x86_64__ else
1027
1028 INLINE operator ap_ulong() const { return to_uint64(); }
1029
1030 INLINE operator ap_slong() const { return to_int64(); }
1031
1032 INLINE int length() const { return _AP_W; };
1033
1034 // bits_to_int64 deleted.
1035 #ifndef __SYNTHESIS__
1036 // Used in autowrap, when _AP_W < 64.
1037 INLINE ap_ulong bits_to_uint64() const {
1038 return (Base::V).to_uint64();
1039 }
1040 #endif
1041
1042 // Count the number of zeros from the most significant bit
1043 // to the first one bit. Note this is only for
ap_fixed_base whose
1044 // _AP_W <= 64, otherwise will incur assertion.
1045 INLINE int countLeadingZeros() {
1046 #if defined __SYNTHESIS__
1047 // TODO: used llvm.ctzll intrinsic ?
1048 if (_AP_W <= 32) {
1049 ap_int_base<32, false> t(-1ULL);
1050 t.range(_AP_W - 1, 0) = this->range(0, _AP_W - 1);
1051 return __builtin_ctzll(t.V);
1052 } else if (_AP_W <= 64) {
1053 ap_int_base<64, false> t(-1ULL);
1054 t.range(_AP_W - 1, 0) = this->range(0, _AP_W - 1);
1055 return __builtin_ctzll(t.V);
1056 } else {
1057 enum {__N = (_AP_W + 63) / 64};
1058 int NZeros = 0;
1059 int i = 0;
1060 bool hitNonZero = false;

```

```

1061     for (i = 0; i < _N - 1; ++i) {
1062         ap_int_base<64, false> t;
1063         t.range(0, 63) = this->range(_AP_W - i * 64 - 64,
1064             _AP_W - i * 64 - 1);
1065         NZeros += hitNonZero ? 0 : __builtin_clzll(t.V);
1066         hitNonZero |= (t != 0);
1067     }
1068     if (!hitNonZero) {
1069         ap_int_base<64, false> t(-1ULL);
1070         t.range(63 - (_AP_W - 1) % 64, 63) = this->range(0,
1071             (_AP_W - 1) % 64);
1072         NZeros += __builtin_clzll(t.V);
1073     }
1074     return NZeros;
1075 }
1076 #else
1077     return Base::V.countLeadingZeros();
1078 #endif
1079 // Arithmetic : Binary
1080 //
-----
1081 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1082     _AP_Q2,
1083     ap_o_mode _AP_O2, int _AP_N2>
1084     inline typename RType<_AP_W2, _AP_I2, _AP_S2>::mult operat-
1085     or*(
1086         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1087             _AP_O2, _AP_N2>& op2)
1088     const {
1089         typename RType<_AP_W2, _AP_I2, _AP_S2>::mult_base r, t;
1090         r.V = Base::V;
1091         t.V = op2.V;
1092         r.V *= op2.V;
1093         return r;
1094     }
1095 // multiply function deleted.
1096
1097 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1098     _AP_Q2,
1099     ap_o_mode _AP_O2, int _AP_N2>
1100     inline typename RType<_AP_W2, _AP_I2, _AP_S2>::div operat-
1101     or*(
1102         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1103             _AP_O2, _AP_N2>& op2)
1104     const {
1105         typename RType<_AP_W2, _AP_I2, _AP_S2>::div_base r;
1106         #ifndef __SYNTHESIS__
1107         enum {F2 = _AP_W2 - _AP_I2,
1108             W1 = AP_MAX(_AP_W + AP_MAX(F2, 0) + ((_AP_S2
1109                 && !_AP_S) ? 1 : 0), _AP_W2 + ((_AP_S && !_AP_S2) ? 1 : 0))};
1110         ap_int_base<W1, _AP_S> dividend, divisor;
1111         ap_int_base<W1, _AP_S> tmp1;
1112         ap_int_base<W1, _AP_S> tmp2;
1113         tmp1.V = Base::V;
1114         tmp2.V = op2.V;
1115         dividend.V = tmp1;
1116         divisor.V = tmp2;
1117         r.V = ((_AP_S2 || _AP_S2) ? dividend.V.sdiv(divisor.V): di-
1118             vidend.V.udiv(divisor.V));
1119         #else
1120         #ifndef __SC_COMPATIBLE__
1121         ap_fixed_base<_AP_W + AP_MAX(_AP_W2 - _AP_I2,
1122             0), _AP_I, _AP_S> t(*this);
1123         #else
1124         ap_fixed_base<_AP_W + AP_MAX(_AP_W2 - _AP_I2, 0) +
1125             AP_MAX(_AP_I2, 0), _AP_I, _AP_S> t(*this);
1126         #endif
1127         r.V = t.V / op2.V;
1128     }
1129 #endif
1130 /*
1131     enum {
1132         F2 = _AP_W2 - _AP_I2,
1133         shl = AP_MAX(F2, 0) + AP_MAX(_AP_I2, 0),
1134         #ifndef __SC_COMPATIBLE__
1135         shr = AP_MAX(_AP_I2, 0),
1136         #else
1137         shr = 0,
1138         #endif
1139         W3 = _AP_S2 + _AP_W + shl,
1140         S3 = _AP_S || _AP_S2,
1141     };
1142     ap_int_base<W3, S3> dividend, t;
1143     dividend.V = Base::V;
1144     // multiply both by (1 << F2), and then do integer divi-
1145     sion.
1146     dividend.V <<= (int) shl;
1147     #ifndef __SYNTHESIS__
1148     // .V's have right signedness, and will have right ex-
1149     tending.
1150     t.V = dividend.V / op2.V;
1151     #else
1152     // XXX op2 may be wider than dividend, and sdiv and udiv
1153     takes the same with
1154     // as left hand operand, so data might be truncated by
1155     mistake if not

```

```

1143     // handled here.
1144     t.V = S3 ? dividend.V.sdiv(op2.V) : di-
1145     vidend.V.udiv(op2.V);
1146     #endif
1147     r.V = t.V >> (int) shr;
1148     return r;
1149 }
1150
1151 #define OP_BIN_AF(Sym,
1152     Rty)
1153     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1154         _AP_Q2,
1155         ap_o_mode _AP_O2, int
1156         _AP_N2>
1157     inline typename RType<_AP_W2, _AP_I2, _AP_S2>::Rty operator
1158     Sym(
1159         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1160             _AP_O2, _AP_N2>& \
1161         op2) const
1162     {
1163         typename RType<_AP_W2, _AP_I2, _AP_S2>::Rty##_base ret,
1164         lhs(*this), \
1165         rhs(op2);
1166         \
1167         ret.V = lhs.V Sym
1168         rhs.V;
1169         return
1170         ret;
1171     }
1172
1173     OP_BIN_AF(+, plus)
1174     OP_BIN_AF(-, minus)
1175     OP_BIN_AF(&, logic)
1176     OP_BIN_AF(|, logic)
1177     OP_BIN_AF(^, logic)
1178
1179 // Arithmetic : assign
1180 //
-----
1181 #define
1182 OP_ASSIGN_AF(Sym)
1183 \
1184     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1185         _AP_Q2,
1186         ap_o_mode _AP_O2, int
1187         _AP_N2>
1188     inline ap_fixed_base& operator
1189     Sym##_=(
1190         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1191             _AP_O2, _AP_N2>& \
1192         op2)
1193     {
1194         *this = operator
1195         Sym(op2);
1196         return
1197         *this;
1198     }
1199
1200     OP_ASSIGN_AF(*)
1201     OP_ASSIGN_AF(/)
1202     OP_ASSIGN_AF(+)
1203     OP_ASSIGN_AF(-)
1204     OP_ASSIGN_AF(&)
1205     OP_ASSIGN_AF(|)
1206     OP_ASSIGN_AF(^)
1207
1208 // Prefix and postfix increment and decrement.
1209 //
-----
1210
1211 // Prefix increment
1212     inline ap_fixed_base& operator++() {
1213         operator+=(ap_fixed_base<_AP_W - _AP_I + 1, 1,
1214             false>(1));
1215         return *this;
1216     }
1217
1218 // Prefix decrement.
1219     inline ap_fixed_base& operator--() {
1220         operator--(ap_fixed_base<_AP_W - _AP_I + 1, 1,
1221             false>(1));
1222         return *this;
1223     }
1224
1225 // Postfix increment
1226     inline const ap_fixed_base operator++(int) {
1227         ap_fixed_base r(*this);
1228         operator++();
1229         return r;
1230     }
1231
1232 // Postfix decrement
1233     inline const ap_fixed_base operator--(int) {
1234         ap_fixed_base r(*this);
1235         operator--();

```



```

1215     return r;
1216 }
1217 // Unary arithmetic.
1218 //
1219 //-----
1220 inline ap_fixed_base operator+() { return *this; }
1221
1222 inline ap_fixed_base<_AP_W + 1, _AP_I + 1, true> operator-
1223 () const {
1224     ap_fixed_base<_AP_W + 1, _AP_I + 1, true> r(*this);
1225     r.V = -r.V;
1226     return r;
1227 }
1228 inline ap_fixed_base<_AP_W, _AP_I, true, _AP_Q, _AP_O,
1229 _AP_N> getNeg() {
1230     ap_fixed_base<_AP_W, _AP_I, true, _AP_Q, _AP_O, _AP_N>
1231     r(*this);
1232     r.V = -r.V;
1233     return r;
1234 }
1235 // Not (!)
1236 //-----
1237 inline bool operator!() const { return Base::V == 0; }
1238 // Bitwise complement
1239 //-----
1240 // XXX different from Mentor's ac_fixed.
1241
1242 inline ap_fixed_base<_AP_W, _AP_I, _AP_S> operator~() const
1243 {
1244     ap_fixed_base<_AP_W, _AP_I, _AP_S> r;
1245     r.V = ~Base::V;
1246     return r;
1247 }
1248 // Shift
1249 //-----
1250 // left shift is the same as moving point right, i.e. in-
1251 create I.
1252 template <int _AP_SHIFT>
1253 inline ap_fixed_base<_AP_W, _AP_I + _AP_SHIFT, _AP_S>
1254 lshift() const {
1255     ap_fixed_base<_AP_W, _AP_I + _AP_SHIFT, _AP_S> r;
1256     r.V = Base::V;
1257     return r;
1258 }
1259 template <int _AP_SHIFT>
1260 inline ap_fixed_base<_AP_W, _AP_I - _AP_SHIFT, _AP_S>
1261 rshift() const {
1262     ap_fixed_base<_AP_W, _AP_I - _AP_SHIFT, _AP_S> r;
1263     r.V = Base::V;
1264     return r;
1265 }
1266 // Because the return type is the type of the the first op-
1267 erand, shift assign
1268 // operators do not carry out any quantization or overflow
1269
1270 // While systemc, shift assigns for sc_fixed/sc_ufixed will
1271 result in
1272 // quantization or overflow (depending on the mode of the
1273 first operand)
1274 inline ap_fixed_base operator<<(unsigned int sh) const {
1275     ap_fixed_base r;
1276     r.V = Base::V << sh;
1277     // TODO check shift overflow?
1278 #ifdef __SC_COMPATIBLE__
1279     if (sh == 0) return r;
1280     if (_AP_O != AP_WRAP || _AP_N != 0) {
1281         bool neg_src = _AP_S && _AP_ROOT_op_get_bit(Base::V,
1282 _AP_W - 1);
1283         bool allones, allzeros;
1284         ap_int_base<_AP_W, false> ones(-1);
1285         if (sh <= _AP_W) {
1286             ap_int_base<_AP_W, false> range1;
1287             range1.V = _AP_ROOT_op_get_range(
1288 const_cast<ap_fixed_base*>(this)->Base::V, _AP_W
1289 - sh, _AP_W - 1);
1290             allones = range1 == (ones >> (_AP_W - sh));
1291             allzeros = range1 == 0;
1292         } else {
1293             allones = false;
1294             allzeros = Base::V == 0;
1295         }
1296         bool overflow = !allzeros && !neg_src;
1297         bool underflow = !allones && neg_src;
1298         if ((!_AP_O == AP_SAT_SYM) && !_AP_S)
1299             underflow |=
1300 neg_src &&

```

```

1293         (_AP_W > 1 ? _AP_ROOT_op_get_range(r.V, 0, _AP_W
1294 - 2) == 0 : true);
1295         bool ld = false;
1296     }
1297     if (sh < _AP_W) ld = _AP_ROOT_op_get_bit(Base::V, _AP_W
1298 - sh - 1);
1299     r.overflow_adjust(underflow, overflow, ld, neg_src);
1300 }
1301 #endif
1302 return r;
1303 }
1304 inline ap_fixed_base operator>>(unsigned int sh) const {
1305     ap_fixed_base r;
1306     r.V = Base::V >> sh;
1307     // TODO check shift overflow?
1308 #ifdef __SC_COMPATIBLE__
1309     if (sh == 0) return r;
1310     if (_AP_Q != AP_TRN) {
1311         bool qb = false;
1312         if (sh <= _AP_W) qb = _AP_ROOT_op_get_bit(Base::V, sh -
1313 1);
1314         bool rb = false;
1315         if (sh > 1 && sh <= _AP_W)
1316             rb =
1317 _AP_ROOT_op_get_range(const_cast<ap_fixed_base*>(this)->Base::V, 0,
1318 sh - 2) != 0;
1319         else if (sh > _AP_W)
1320             rb = Base::V != 0;
1321         r.quantization_adjust(qb, rb,
1322 _AP_S &&
1323 _AP_ROOT_op_get_bit(Base::V, _AP_W - 1));
1324     }
1325 #endif
1326 return r;
1327 }
1328 // left and right shift for int
1329 inline ap_fixed_base operator<<(int sh) const {
1330     ap_fixed_base r;
1331     bool isNeg = sh < 0;
1332     unsigned int ush = isNeg ? -sh : sh;
1333     if (isNeg) {
1334         return operator>>(ush);
1335     } else {
1336         return operator<<(ush);
1337     }
1338 }
1339 inline ap_fixed_base operator>>(int sh) const {
1340     ap_fixed_base r;
1341     bool isNeg = sh < 0;
1342     unsigned int ush = isNeg ? -sh : sh;
1343     if (isNeg) {
1344         return operator<<(ush);
1345     } else {
1346         return operator>>(ush);
1347     }
1348 }
1349 // left and right shift for ap_int.
1350 template <int _AP_W2>
1351 inline ap_fixed_base operator<<(const ap_int_base<_AP_W2,
1352 true>& op2) const {
1353     // TODO the code seems not optimal. ap_fixed<8,8> <<
1354 ap_int<2>
1355 needs only a
1356 // small mux, but integer need a big one!
1357 int sh = op2.to_int();
1358 return operator<<(sh);
1359 }
1360
1361 template <int _AP_W2>
1362 inline ap_fixed_base operator>>(const ap_int_base<_AP_W2,
1363 true>& op2) const {
1364     int sh = op2.to_int();
1365     return operator>>(sh);
1366 }
1367
1368 // left and right shift for ap_uint.
1369 template <int _AP_W2>
1370 inline ap_fixed_base operator<<(const ap_int_base<_AP_W2,
1371 false>& op2) const {
1372     unsigned int sh = op2.to_uint();
1373     return operator<<(sh);
1374 }
1375
1376 template <int _AP_W2>
1377 inline ap_fixed_base operator>>(const ap_int_base<_AP_W2,
1378 false>& op2) const {
1379     unsigned int sh = op2.to_uint();
1380     return operator>>(sh);
1381 }
1382
1383 // left and right shift for ap_fixed
1384 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1385 _AP_Q2,
1386 ap_o_mode _AP_O2, int _AP_N2>
1387 inline ap_fixed_base operator<<(
1388 const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1389 _AP_O2, _AP_N2>&

```

```

1379         op2) {
1380             return operator<<(op2.to_ap_int_base());
1381         }
1382     }
1383     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1384     _AP_Q2,
1385             ap_o_mode _AP_O2, int _AP_N2>
1386     inline ap_fixed_base operator>>({
1387         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1388         _AP_O2, _AP_N2>&
1389         op2) {
1390             return operator>>(op2.to_ap_int_base());
1391         }
1392     // Shift assign.
1393     //
-----
1393
1394     // left shift assign.
1395     inline ap_fixed_base& operator<<=(const int sh) {
1396         *this = operator<<(sh);
1397         return *this;
1398     }
1399
1400     inline ap_fixed_base& operator<<=(const unsigned int sh) {
1401         *this = operator<<(sh);
1402         return *this;
1403     }
1404
1405     template <int _AP_W2, bool _AP_S2>
1406     inline ap_fixed_base& operator<<=(const ap_int_base<_AP_W2,
1407     _AP_S2>& sh) {
1408         *this = operator<<(sh.to_int());
1409         return *this;
1410     }
1411
1412     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1413     _AP_Q2,
1414             ap_o_mode _AP_O2, int _AP_N2>
1415     inline ap_fixed_base& operator<<=(
1416         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1417         _AP_O2, _AP_N2>&
1418         sh) {
1419         *this = operator<<(sh.to_int());
1420         return *this;
1421     }
1422
1423     // right shift assign.
1424     inline ap_fixed_base& operator>>=(const int sh) {
1425         *this = operator>>(sh);
1426         return *this;
1427     }
1428
1429     inline ap_fixed_base& operator>>=(const unsigned int sh) {
1430         *this = operator>>(sh);
1431         return *this;
1432     }
1433
1434     template <int _AP_W2, bool _AP_S2>
1435     inline ap_fixed_base& operator>>=(const ap_int_base<_AP_W2,
1436     _AP_S2>& sh) {
1437         *this = operator>>(sh.to_int());
1438         return *this;
1439     }
1440
1441     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1442     _AP_Q2,
1443             ap_o_mode _AP_O2, int _AP_N2>
1444     inline ap_fixed_base& operator>>=(
1445         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
1446         _AP_O2, _AP_N2>&
1447         sh) {
1448         *this = operator>>(sh.to_int());
1449         return *this;
1450     }
1451
1452     // Comparisons.
1453     //
-----
1453
1454     #define
1455     OP_CMP_AF(Sym)
1456     \
1457     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
1458     _AP_Q2,
1459             ap_o_mode _AP_O2, int
1460             _AP_N2>
1461     inline bool operator Sym(const ap_fixed_base<_AP_W2,
1462     _AP_I2, _AP_S2, _AP_Q2, \
1463             _AP_O2,
1464     _AP_N2>& op2) const { \
1465         enum { _AP_F = _AP_W - _AP_I, F2 = _AP_W2 -
1466     _AP_I2 };
1467         if (_AP_F ==
1468     F2) \
1469             return Base::V Sym
1470     op2.V; \

```

```

1456         else if (_AP_F >
1457     F2) \
1458             return Base::V Sym ap_fixed_base<AP_MAX(_AP_W2 + _AP_F
1459     - F2, 1), _AP_I2, \
1460             _AP_S2, _AP_Q2,
1461     _AP_O2, _AP_N2>(op2).V; \
1462         else
1463             \
1464
1465         return ap_fixed_base<AP_MAX(_AP_W + F2 - _AP_F + 1, 1),
1466     _AP_I + 1, \
1467             _AP_S, _AP_Q, _AP_O,
1468     _AP_N>(*this).V Sym op2.V; \
1469         return
1470     false;
1471     \
1472     }
1473
1474     OP_CMP_AF(>)
1475     OP_CMP_AF(<)
1476     OP_CMP_AF(>=)
1477     OP_CMP_AF(<=)
1478     OP_CMP_AF(==)
1479     OP_CMP_AF(!=)
1480
1481     // FIXME: Move compare with double out of struct
1482     ap_fixed_base defination
1483     // and combine it with compare operator(double,
1484     ap_fixed_base)
1485     #define DOUBLE_CMP_AF(Sym) \
1486     inline bool operator Sym(double d) const { return
1487     to_double() Sym d; }
1488
1489     DOUBLE_CMP_AF(>)
1490     DOUBLE_CMP_AF(<)
1491     DOUBLE_CMP_AF(>=)
1492     DOUBLE_CMP_AF(<=)
1493     DOUBLE_CMP_AF(==)
1494     DOUBLE_CMP_AF(!=)
1495
1496     // Bit and Slice Select
1497
1498     inline af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>
1499     operator[]({
1500         unsigned index) {
1501         _AP_WARNING(index >= _AP_W, "Attempting to read bit bey-
1502     ond MSB");
1503         return af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1504     _AP_N>(this, index);
1505     }
1506
1507     template <int _AP_W2, bool _AP_S2>
1508
1509     inline af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>
1510     operator[]({
1511         const ap_int_base<_AP_W2, _AP_S2>& index) {
1512         _AP_WARNING(index < 0, "Attempting to read bit with neg-
1513     ative index");
1514         _AP_WARNING(index >= _AP_W, "Attempting to read bit bey-
1515     ond MSB");
1516         return af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1517     _AP_N>(this,
1518     index.to_int());
1519     }
1520
1521     inline bool operator[] (unsigned index) const {
1522         _AP_WARNING(index >= _AP_W, "Attempting to read bit bey-
1523     ond MSB");
1524         return
1525         _AP_ROOT_op_get_bit(const_cast<ap_fixed_base*>(this)->V, index);
1526     }
1527
1528     inline af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>
1529     bit({
1530         unsigned index) {
1531         _AP_WARNING(index >= _AP_W, "Attempting to read bit bey-
1532     ond MSB");
1533         return af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1534     _AP_N>(this, index);
1535     }
1536
1537     template <int _AP_W2, bool _AP_S2>
1538
1539     inline af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>
1540     bit({
1541         const ap_int_base<_AP_W2, _AP_S2>& index) {
1542         _AP_WARNING(index < 0, "Attempting to read bit with neg-
1543     ative index");
1544         _AP_WARNING(index >= _AP_W, "Attempting to read bit bey-
1545     ond MSB");
1546         return af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1547     _AP_N>(this,
1548     index.to_int());
1549     }
1550
1551     }
1552

```

```

1519     inline bool bit(unsigned index) const {
1520         _AP_WARNING(index >= _AP_W, "Attempting to read bit bey-
ond MSB");
1521         return
1522         _AP_ROOT_op_get_bit(const_cast<ap_fixed_base*>(this)->v, index);
1523     }
1524     template <int _AP_W2>

1525     inline af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>
get_bit(
1526         const ap_int_base<_AP_W2, true>& index) {
1527         _AP_WARNING(index < _AP_I - _AP_W,
1528             "Attempting to read bit with negative in-
dex");
1529         _AP_WARNING(index >= _AP_I, "Attempting to read bit bey-
ond MSB");
1530         return af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1531             _AP_N>(
1532             this, index.to_int() + _AP_W - _AP_I);
1533     }
1534     inline bool get_bit(int index) const {
1535         _AP_WARNING(index >= _AP_I, "Attempting to read bit bey-
ond MSB");
1536         _AP_WARNING(index < _AP_I - _AP_W, "Attempting to read
bit beyond MSB");
1537         return
1538         _AP_ROOT_op_get_bit(const_cast<ap_fixed_base*>(this)->v,
1539             index + _AP_W - _AP_I);
1540     }
1541     #if 0

1541     inline af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>
get_bit(
1542         int index) {
1543         _AP_WARNING(index < _AP_I - _AP_W,
1544             "Attempting to read bit with negative index");
1545         _AP_WARNING(index >= _AP_I, "Attempting to read bit bey-
ond MSB");
1546         return af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1547             _AP_N>(
1548             this, index + _AP_W - _AP_I);
1549     }
1550     #endif
1551     template <int _AP_W2>

1552     inline bool get_bit(const ap_int_base<_AP_W2, true>& index)
const {
1553         _AP_WARNING(index >= _AP_I, "Attempting to read bit bey-
ond MSB");
1554         _AP_WARNING(index < _AP_I - _AP_W, "Attempting to read
bit beyond MSB");
1555         return
1556         _AP_ROOT_op_get_bit(const_cast<ap_fixed_base*>(this)->v,
1557             index.to_int() + _AP_W -
1558             _AP_I);
1559     }
1560     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1561         _AP_N> range(int Hi,
1562             int Lo) {
1563         _AP_WARNING((Hi >= _AP_W) || (Lo >= _AP_W), "Out of
bounds in range()");
1564         return af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1565             _AP_N>(this, Hi, Lo);
1566     }
1567     // This is a must to strip constness to produce reference
type.
1568     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1569         _AP_N> range(
1570         int Hi, int Lo) const {
1571         _AP_WARNING((Hi >= _AP_W) || (Lo >= _AP_W), "Out of
bounds in range()");
1572         return af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1573             _AP_N>(
1574             const_cast<ap_fixed_base*>(this), Hi, Lo);
1575     }
1576     template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
1577     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1578         _AP_N> range(
1579         const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
1580         const ap_int_base<_AP_W3, _AP_S3>& LoIdx) {
1581         int Hi = HiIdx.to_int();
1582         int Lo = LoIdx.to_int();
1583         return this->range(Hi, Lo);
1584     }
1585     template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
1586     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1587         _AP_N> range(
1588         const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
1589         const ap_int_base<_AP_W3, _AP_S3>& LoIdx) const {
1590         int Hi = HiIdx.to_int();
1591         int Lo = LoIdx.to_int();
1592         return this->range(Hi, Lo);
1593     }

1588     return this->range(Hi, Lo);
1589 }
1590
1591     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1592         _AP_N> range() {
1593         return this->range(_AP_W - 1, 0);
1594     }
1595
1596     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1597         _AP_N> range() const {
1598         return this->range(_AP_W - 1, 0);
1599     }
1600
1601     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1602         _AP_N> operator()(
1603         int Hi, int Lo) {
1604         return this->range(Hi, Lo);
1605     }
1606
1607     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1608         _AP_N> operator()(
1609         int Hi, int Lo) const {
1610         return this->range(Hi, Lo);
1611     }
1612
1613     template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
1614     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1615         _AP_N> operator()(
1616         const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
1617         const ap_int_base<_AP_W3, _AP_S3>& LoIdx) {
1618         int Hi = HiIdx.to_int();
1619         int Lo = LoIdx.to_int();
1620         return this->range(Hi, Lo);
1621     }
1622
1623     template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
1624     inline af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
1625         _AP_N> operator()(
1626         const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
1627         const ap_int_base<_AP_W3, _AP_S3>& LoIdx) const {
1628         int Hi = HiIdx.to_int();
1629         int Lo = LoIdx.to_int();
1630         return this->range(Hi, Lo);
1631     }
1632
1633     inline bool is_zero() const { return Base::V == 0; }
1634
1635     inline bool is_neg() const {
1636         if (_AP_S && _AP_ROOT_op_get_bit(Base::V, _AP_W - 1)) re-
turn true;
1637     }
1638
1639     inline int wl() const { return _AP_W; }
1640
1641     inline int iwl() const { return _AP_I; }
1642
1643     inline ap_q_mode q_mode() const { return _AP_Q; }
1644
1645     inline ap_o_mode o_mode() const { return _AP_O; }
1646
1647     inline int n_bits() const { return _AP_N; }
1648
1649     // print a string representation of this number in the giv-
en radix.
1650     // Radix support is 2, 8, 10, or 16.
1651     // The result will include a prefix indicating the radix,
except for decimal,
1652     // where no prefix is needed. The default is to output a
signed
representation
1653     // of signed numbers, or an unsigned representation of un-
signed numbers. For
1654     // non-decimal formats, this can be changed by the 'sign'
argument.
1655     #ifndef __SYNTHESIS__
1656     std::string to_string(unsigned char radix = 2, bool sign =
1657         _AP_S) const {
1658         // XXX in autosim/autowrap.tcl "($
1659         {name}).to_string(2).c_str()" is used to
1660         // initialize sc_lv, which seems incapable of handling
format "-0b".
1661         if (radix == 2) sign = false;
1662
1663         std::string str;
1664         str.clear();
1665         char step = 0;
1666         bool isNeg = sign && (Base::V < 0);
1667
1668         // Extend to take care of the -MAX case.
1669         ap_fixed_base<_AP_W + 1, _AP_I + 1> tmp(*this);
1670         if (isNeg) {
1671             tmp = -tmp;
1672             str += '-';
1673         }
1674         std::string prefix;
1675         switch (radix) {
1676             case 2:
1677                 prefix = "0b";
1678                 step = 1;
1679                 break;

```



```

1673     case 8:
1674         prefix = "00";
1675         step = 3;
1676         break;
1677     case 16:
1678         prefix = "0x";
1679         step = 4;
1680         break;
1681     default:
1682         break;
1683 }
1684
1685 if (_AP_I > 0) {
1686     // Note we drop the quantization and rounding flags
1687     // integer part is always in range, and the fractional
1688     // want to drop. Also, the number is always positive,
1689     // of the absolute value above.
1690     ap_int_base<AP_MAX(_AP_I + 1, 1), false> int_part;
1691     // [1] [ I ] d [ W - I ]
1692     // | | |
1693     // | W-I 0
1694     // W
1695     int_part.V = _AP_ROOT_op_get_range(
1696         tmp.V, _AP_W - _AP_I, _AP_W);
1697     str += int_part.to_string(radix, false);
1698 } else {
1699     str += prefix;
1700     str += '0';
1701 }
1702
1703 ap_fixed_base<AP_MAX(_AP_W - _AP_I, 1), 0, false>
frac_part = tmp;
1704
1705 if (radix == 10) {
1706     if (frac_part != 0) {
1707         str += ".";
1708         while (frac_part != 0) {
1709             char digit = (frac_part * radix).to_char();
1710             str += static_cast<char>(digit + '0');
1711             frac_part *= radix;
1712         }
1713     }
1714 } else {
1715     if (frac_part != 0) {
1716         str += ".";
1717
1718         for (signed i = _AP_W - _AP_I - 1; i >= 0; i -= step)
1719         {
1720             char digit = frac_part.range(i, AP_MAX(0, i - step
+ 1)).to_char();
1721             // If we have a partial bit pattern at the end,
1722             // to put it in the high-order bits of 'digit'.
1723             int offset = AP_MIN(0, i - step + 1);
1724             digit <<= -offset;
1725             str += digit < 10 ? static_cast<char>(digit + '0')
: static_cast<char>(digit - 10 +
'a');
1726         }
1727         if (radix == 16)
1728             str += "p0"; // C99 Hex constants are required to
have an exponent.
1729     }
1730     return str;
1731 }
1732 #else
1733 // XXX HLS will delete this in synthesis
1734
1735 inline char* to_string(unsigned char radix = 2, bool sign =
_AP_S) const {
1736     return 0;
1737 }
1738 #endif
1739 }; // struct ap_fixed_base.
1740
1741 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1742         int _AP_N>
1743 inline void b_not(
1744     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1745     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op1,
1746     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
1747     ret.V = op1.V & op2.V;
1748 }
1749
1750 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1751         int _AP_N>
1752 inline void b_or(
1753     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1754     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op1,
1755     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
1756     ret.V = op1.V | op2.V;
1757 }
1758
1759 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1760         int _AP_N>
1761 inline void b_xor(
1762     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1763     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op1,
1764     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
1765     ret.V = op1.V ^ op2.V;
1766 }
1767
1768 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1769         int _AP_N, int _AP_W2, int _AP_I2, bool _AP_S2,
ap_q_mode _AP_Q2,
1770         ap_o_mode _AP_O2, int _AP_N2>
1771 inline void neg(
1772     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1773     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op) {
1774     ap_fixed_base<_AP_W2 + !_AP_S2, _AP_I2 + !_AP_S2, true,
_AP_O2, _AP_N2>
1775     t;
1776     t.V = -op.V;
1777     ret = t;
1778 }
1779
1780 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1781         int _AP_N, int _AP_W2, int _AP_I2, bool _AP_S2,
ap_q_mode _AP_Q2,
1782         ap_o_mode _AP_O2, int _AP_N2>
1783 inline void lshift(
1784     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1785     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op,
1786     int i) {
1787     enum {
1788         F2 = _AP_W2 - _AP_I2,
1789         _AP_I3 = AP_MAX(_AP_I, _AP_I2),
1790         _AP_W3 = _AP_I3 + F2,
1791     };
1792     // wide buffer
1793     ap_fixed_base<_AP_W3, _AP_I3, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> t;
1794     t.V = op.V;
1795     t.V <<= i; // FIXME overflow?
1796     // handle quantization and overflow
1797     ret = t;
1798 }
1799
1800 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1801         int _AP_N, int _AP_W2, int _AP_I2, bool _AP_S2,
ap_q_mode _AP_Q2,
1802         ap_o_mode _AP_O2, int _AP_N2>
1803 inline void rshift(
1804     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1805     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op,
1806     int i) {
1807     enum {
1808         F = _AP_W - _AP_I,
1809         F2 = _AP_W2 - _AP_I2,
1810         F3 = AP_MAX(F, F2),
1811         _AP_W3 = _AP_I2 + F3,
1812         sh = F - F2,
1813     };
1814     // wide buffer
1815     ap_fixed_base<_AP_W3, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> t;
1816     t.V = op.V;
1817     if (sh >= 0)
1818         t.V >>= (int) sh;
1819     // handle quantization and overflow
1820     ret = t;
1821 }

```

```

1754     ret.V = op1.V & op2.V;
1755 }
1756
1757 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1758         int _AP_N>
1759 inline void b_or(
1760     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1761     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op1,
1762     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
1763     ret.V = op1.V | op2.V;
1764 }
1765
1766 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1767         int _AP_N>
1768 inline void b_xor(
1769     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1770     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op1,
1771     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
1772     ret.V = op1.V ^ op2.V;
1773 }
1774
1775 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1776         int _AP_N, int _AP_W2, int _AP_I2, bool _AP_S2,
ap_q_mode _AP_Q2,
1777         ap_o_mode _AP_O2, int _AP_N2>
1778 inline void neg(
1779     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1780     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op) {
1781     ap_fixed_base<_AP_W2 + !_AP_S2, _AP_I2 + !_AP_S2, true,
_AP_O2, _AP_N2>
1782     t;
1783     t.V = -op.V;
1784     ret = t;
1785 }
1786
1787 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1788         int _AP_N, int _AP_W2, int _AP_I2, bool _AP_S2,
ap_q_mode _AP_Q2,
1789         ap_o_mode _AP_O2, int _AP_N2>
1790 inline void lshift(
1791     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1792     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op,
1793     int i) {
1794     enum {
1795         F2 = _AP_W2 - _AP_I2,
1796         _AP_I3 = AP_MAX(_AP_I, _AP_I2),
1797         _AP_W3 = _AP_I3 + F2,
1798     };
1799     // wide buffer
1800     ap_fixed_base<_AP_W3, _AP_I3, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> t;
1801     t.V = op.V;
1802     t.V <<= i; // FIXME overflow?
1803     // handle quantization and overflow
1804     ret = t;
1805 }
1806
1807 template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
1808         int _AP_N, int _AP_W2, int _AP_I2, bool _AP_S2,
ap_q_mode _AP_Q2,
1809         ap_o_mode _AP_O2, int _AP_N2>
1810 inline void rshift(
1811     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
ret,
1812     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op,
1813     int i) {
1814     enum {
1815         F = _AP_W - _AP_I,
1816         F2 = _AP_W2 - _AP_I2,
1817         F3 = AP_MAX(F, F2),
1818         _AP_W3 = _AP_I2 + F3,
1819         sh = F - F2,
1820     };
1821     // wide buffer
1822     ap_fixed_base<_AP_W3, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> t;
1823     t.V = op.V;
1824     if (sh >= 0)
1825         t.V >>= (int) sh;
1826     // handle quantization and overflow
1827     ret = t;
1828 }
1829
1830 }

```

```

1831
1832 //// FIXME
1833 //// These partial specialization ctors allow code like
1834 //// char c = 'a';
1835 //// ap_fixed_base<8, 8, true> x(c);
1836 //// but what bout ap_fixed_base<9, 9, true> y(c) ?
1837 //
1838
1839 #ifndef __SYNTHESIS__
1840 inline std::string scientificFormat(std::string& input) {
1841     if (input.length() == 0) return input;
1842
1843     size_t decPosition = input.find('.');
1844     if (decPosition == std::string::npos) decPosition = in-
put.length();
1845
1846     size_t firstNonZeroPos = 0;
1847     for (; input[firstNonZeroPos] > '9' || in-
put[firstNonZeroPos] < '1';
1848         firstNonZeroPos++)
1849         ;
1850
1851     int exp;
1852     if (firstNonZeroPos > decPosition)
1853         exp = decPosition - firstNonZeroPos;
1854     else
1855         exp = decPosition - firstNonZeroPos - 1;
1856     std::string expString = "";
1857     if (exp == 0)
1858         ;
1859     else if (exp < 0) {
1860         expString += "e-";
1861         exp = -exp;
1862     } else
1863         expString += "e+";
1864
1865     if (exp < 10 && exp > 0) {
1866         expString += '0';
1867         expString += (char)('0' + exp);
1868     } else if (exp != 0) {
1869         std::string tmp;
1870
1871         std::ostringstream oss;
1872         oss << exp;
1873
1874         tmp = oss.str();
1875         expString += tmp;
1876     }
1877
1878     int lastNonZeroPos = (int)(input.length() - 1);
1879     for (; lastNonZeroPos >= 0; --lastNonZeroPos)
1880
1881     if (input[lastNonZeroPos] <= '9' && input[lastNonZeroPos]
> '0') break;
1882
1883     std::string ans = "";
1884     ans += input[firstNonZeroPos];
1885     if (firstNonZeroPos != (size_t)lastNonZeroPos) {
1886         ans += '.';
1887         for (int i = firstNonZeroPos + 1; i <= lastNonZeroPos; i+
+)
1888             if (input[i] != '.') ans += input[i];
1889     }
1890     ans += expString;
1891     return ans;
1892 }
1893
1894 inline std::string reduceToPrecision(std::string& input, int
precision) {
1895     bool isZero = true;
1896     size_t inputLen = input.length();
1897     for (size_t i = 0; i < inputLen && isZero; i++)
1898         if (input[i] != '.' && input[i] != '0') isZero = false;
1899     if (isZero) return "0";
1900
1901     // Find the first valid number, skip '-'
1902     int FirstNonZeroPos = 0;
1903     int LastNonZeroPos = (int)inputLen - 1;
1904     int truncBitPosition = 0;
1905     size_t decPosition = input.find('.');
1906     for (; input[FirstNonZeroPos] < '1' || in-
put[FirstNonZeroPos] > '9';
1907         FirstNonZeroPos++)
1908         ;
1909
1910     for (; input[LastNonZeroPos] < '1' || input[LastNonZeroPos]
> '9';
1911         LastNonZeroPos--)
1912         ;
1913
1914     if (decPosition == std::string::npos) decPosition = input-
Len;
1915     // Count the valid number, to decide whether we need to
truncate
1916     if ((int)decPosition > LastNonZeroPos) {
1917         if (LastNonZeroPos - FirstNonZeroPos + 1 <= precision)
return input;
1918         truncBitPosition = FirstNonZeroPos + precision;

```

```

1919     } else if ((int)decPosition < FirstNonZeroPos) { // This is
pure decimal
1920         if (LastNonZeroPos - FirstNonZeroPos + 1 <= precision) {
1921             if (FirstNonZeroPos - decPosition - 1 < 4) {
1922                 return input;
1923             } else {
1924                 if (input[0] == '-') {
1925                     std::string tmp = input.substr(1, inputLen - 1);
1926                     return std::string("-") + scientificFormat(tmp);
1927                 } else
1928                     return scientificFormat(input);
1929             }
1930         }
1931         truncBitPosition = FirstNonZeroPos + precision;
1932     } else {
1933         if (LastNonZeroPos - FirstNonZeroPos <= precision) return
input;
1934         truncBitPosition = FirstNonZeroPos + precision + 1;
1935     }
1936
1937     // duplicate the input string, we want to add "0" before
the valid numbers
1938     // This is easy for quantization, since we may change 9999
to 10000
1939     std::string ans = "";
1940     std::string dupInput = "0";
1941     if (input[0] == '-') {
1942         ans += '-';
1943         dupInput += input.substr(1, inputLen - 1);
1944     } else {
1945         dupInput += input.substr(0, inputLen);
1946         ++truncBitPosition;
1947     }
1948
1949     // Add 'carry' after truncation, if necessary
1950     bool carry = dupInput[truncBitPosition] > '4';
1951     for (int i = truncBitPosition - 1; i >= 0 && carry; i--) {
1952         if (dupInput[i] == '.') continue;
1953         if (dupInput[i] == '9')
1954             dupInput[i] = '0';
1955         else {
1956             ++dupInput[i];
1957             carry = false;
1958         }
1959     }
1960
1961     // bits outside precision range should be set to 0
1962     if (dupInput[0] == '1')
1963         FirstNonZeroPos = 0;
1964     else {
1965         FirstNonZeroPos = 0;
1966         while (dupInput[FirstNonZeroPos] < '1' || dupIn-
put[FirstNonZeroPos] > '9')
1967             ++FirstNonZeroPos;
1968     }
1969
1970     unsigned it = FirstNonZeroPos;
1971     int NInvalidNumber = 0;
1972     while (it < dupInput.length()) {
1973         if (dupInput[it] == '.') {
1974             ++it;
1975             continue;
1976         }
1977         ++NInvalidNumber;
1978         if (NInvalidNumber > precision) dupInput[it] = '0';
1979         ++it;
1980     }
1981
1982     // Here we wanted to adjust the truncate position and the
value
1983     decPosition = dupInput.find('.');
1984     if (decPosition == std::string::npos) // When this is in-
teger
1985         truncBitPosition = (int)dupInput.length();
1986     else
1987         for (truncBitPosition = (int)(dupInput.length() - 1);
truncBitPosition >= 0;
1988             --truncBitPosition) {
1989             if (dupInput[truncBitPosition] == '.') break;
1990             if (dupInput[truncBitPosition] != '0') {
1991                 truncBitPosition++;
1992                 break;
1993             }
1994         }
1995
1996     if (dupInput[0] == '1')
1997         dupInput = dupInput.substr(0, truncBitPosition);
1998     else
1999         dupInput = dupInput.substr(1, truncBitPosition - 1);
2000
2001     decPosition = dupInput.find('.');
2002     if (decPosition != std::string::npos) {
2003         size_t it = 0;
2004         for (it = decPosition + 1; dupInput[it] == '0'; it++)
2005             ;
2006         if (it - decPosition - 1 < 4) {
2007             ans += dupInput;
2008             return ans;

```

```

2009     } else {
2010         ans += scientificFormat(dupInput);
2011         return ans;
2012     }
2013 } else if ((int)(dupInput.length()) <= precision) {
2014     ans += dupInput;
2015     return ans;
2016 }
2017
2018 ans += scientificFormat(dupInput);
2019 return ans;
2020 }
2021
2022 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2023         int _AP_N>
2024 inline void print(
2025     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& x) {
2026     if (_AP_I > 0) {
2027         ap_int_base<_AP_I, _AP_S> p1;
2028         p1.V = x.V >> (_AP_W - _AP_I);
2029         print(p1.V); // print overload for .V should exit
2030     } else {
2031         printf("0");
2032     }
2033     printf(".");
2034     if (_AP_I < _AP_W) {
2035         ap_int_base<_AP_W - _AP_I, false> p2;
2036         p2.V = _AP_ROOT_op_get_range(x.V, 0, _AP_W - _AP_I);
2037         print(p2.V, false); // print overload for .V should exit
2038     }
2039 }
2040 #endif // ifndef __SYNTHESIS__
2041
2042 // XXX the following two functions have to exist in synthes-
is,
2043 // as some old HLS Video Library code uses the ostream over-
load,
2044 // although HLS will later delete I/O function call.
2045
2046 /// Output streaming
2047 //-----
2048 // XXX apcc cannot handle global std::ios_base::Init()
brought in by <iostream>
2049 #ifndef AP_AUTOCC
2050 #ifndef __SYNTHESIS__
2051 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2052         int _AP_N>
2053 inline std::ostream& operator<<(
2054     std::ostream& out,
2055     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& x) {
2056     // TODO support std::ios_base::fmtflags
2057     unsigned width = out.width();
2058     unsigned precision = out.precision();
2059     char fill = out.fill();
2060     std::string str = x.to_string(10, _AP_S);
2061     str = reduceToPrecision(str, precision);
2062     if (width > str.length()) {
2063         for (unsigned i = 0; i < width - str.length(); ++i)
2064             out << fill;
2065     }
2066     out << str;
2067     return out;
2068 }
2069 #endif // ifndef __SYNTHESIS__
2070
2071 /// Input streaming
2072 //-----
2073 #ifndef __SYNTHESIS__
2074 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2075         int _AP_N>
2076 inline std::istream& operator>>(
2077     std::istream& in,
2078     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O, _AP_N>&
x) {
2079     double d;
2080     in >> d;
2081     x = ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>(d);
2082     return in;
2083 }
2084 #endif
2085 #endif // ifndef AP_AUTOCC
2086
2087 /// Operators mixing Integers with ap_fixed_base
2088 //-----
2089 #define AF_BIN_OP_WITH_INT_SF(BIN_OP, C_TYPE, _AP_W2, _AP_S2,
RTYPE) \
2090     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,

```

```

2091         ap_o_mode _AP_O, int
_AP_N> \
2092     inline typename ap_fixed_base<_AP_W, _AP_I, _AP_S>::tem-
plate RTYPE< \
2093         _AP_W2, _AP_W2, \
2094         _AP_S2>::RTYPE \
2095     operator \
2096     BIN_OP( \
2097         const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op, \
2098         C_TYPE i_op) \
2099     { \
2100         return op.operator BIN_OP(ap_int_base<_AP_W2, \
_AP_S2>(i_op)); \
2101     } \
2102 #define AF_BIN_OP_WITH_INT(BIN_OP, C_TYPE, _AP_W2, _AP_S2,
RTYPE) \
2103     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
2104         ap_o_mode _AP_O, int \
2105         _AP_N> \
2106     inline typename ap_fixed_base<_AP_W, _AP_I, _AP_S>::tem-
plate RTYPE< \
2107         _AP_W2, _AP_W2, \
2108         _AP_S2>::RTYPE \
2109     operator \
2110     BIN_OP( \
2111         const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op, \
2112         C_TYPE i_op) \
2113     { \
2114         return op.operator BIN_OP(ap_fixed_base<_AP_W2, _AP_W2,
_AP_S2>(i_op)); \
2115     } \
2116     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
2117         ap_o_mode _AP_O, int \
2118         _AP_N> \
2119     inline typename ap_fixed_base<_AP_W, _AP_I, _AP_S>::tem-
plate RTYPE< \
2120         _AP_W2, _AP_W2, \
2121         _AP_S2>::RTYPE \
2122     operator \
2123     BIN_OP( \
2124         C_TYPE \
2125         i_op, \
2126         const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op) { \
2127         return ap_fixed_base<_AP_W2, _AP_W2, _AP_S2>(i_op).oper-
ator BIN_OP(op); \
2128     } \
2129 #define AF_REL_OP_WITH_INT(REL_OP, C_TYPE, _AP_W2, \
_AP_S2) \
2130     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
2131         ap_o_mode _AP_O, int \
2132         _AP_N> \
2133     inline bool operator \
2134     REL_OP( \
2135         const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op, \
2136         C_TYPE i_op) \
2137     { \
2138         return op.operator REL_OP(ap_fixed_base<_AP_W2, _AP_W2,
_AP_S2>(i_op)); \
2139     } \
2140     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
2141         ap_o_mode _AP_O, int \
2142         _AP_N> \
2143     inline bool operator \
2144     REL_OP( \
2145         C_TYPE \
2146         i_op, \
2147         const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op) { \
2148         return ap_fixed_base<_AP_W2, _AP_W2, _AP_S2>(i_op).oper-
ator REL_OP(op); \
2149     } \
2150 #define AF_ASSIGN_OP_WITH_INT(ASSIGN_OP, C_TYPE, _AP_W2, \
_AP_S2) \
2151     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
2152         ap_o_mode _AP_O, int \
2153         _AP_N> \
2154     inline ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& \
2155     operator AS- \
2156     SIGN_OP( \
2157         ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op, \
2158         C_TYPE i_op) \
2159     { \

```

```

2143     return op.operator ASSIGN_OP(ap_fixed_base<_AP_W2,
2144     _AP_W2, _AP_S2>(i_op)); \
2145 }
2146 #define AF_ASSIGN_OP_WITH_INT_SF(ASSIGN_OP, C_TYPE, _AP_W2,
2147     _AP_S2) \
2148     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2149     _AP_Q, \
2150     ap_o_mode _AP_0, int
2151     _AP_N> \
2152     INLINE ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2153     _AP_N>& \
2154     operator AS-
2155     SIGN_OP(
2156     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2157     _AP_N>& op, \
2158     C_TYPE i_op) \
2159 {
2160     return op.operator ASSIGN_OP(ap_int_base<_AP_W2,
2161     _AP_S2>(i_op)); \
2162 }
2163 #define ALL_AF_OP_WITH_INT(C_TYPE, BITS, SIGN)
2164 \
2165 AF_BIN_OP_WITH_INT(+, C_TYPE, (BITS), (SIGN), plus) \
2166 AF_BIN_OP_WITH_INT(-, C_TYPE, (BITS), (SIGN), minus) \
2167 AF_BIN_OP_WITH_INT(*, C_TYPE, (BITS), (SIGN), mult) \
2168 AF_BIN_OP_WITH_INT(/, C_TYPE, (BITS), (SIGN), div) \
2169 AF_BIN_OP_WITH_INT(&, C_TYPE, (BITS), (SIGN), logic) \
2170 AF_BIN_OP_WITH_INT(|, C_TYPE, (BITS), (SIGN), logic) \
2171 AF_BIN_OP_WITH_INT(^, C_TYPE, (BITS), (SIGN), logic) \
2172 AF_BIN_OP_WITH_INT(>=, C_TYPE, (BITS), (SIGN), lhs) \
2173 AF_BIN_OP_WITH_INT(<=, C_TYPE, (BITS), (SIGN), lhs) \
2174 AF_ASSIGN_OP_WITH_INT(+=, C_TYPE, (BITS), (SIGN)) \
2175 AF_ASSIGN_OP_WITH_INT(-=, C_TYPE, (BITS), (SIGN)) \
2176 AF_ASSIGN_OP_WITH_INT(*=, C_TYPE, (BITS), (SIGN)) \
2177 AF_ASSIGN_OP_WITH_INT(/=, C_TYPE, (BITS), (SIGN)) \
2178 AF_ASSIGN_OP_WITH_INT(&=, C_TYPE, (BITS), (SIGN)) \
2179 AF_ASSIGN_OP_WITH_INT(|=, C_TYPE, (BITS), (SIGN)) \
2180 AF_ASSIGN_OP_WITH_INT(^=, C_TYPE, (BITS), (SIGN)) \
2181 AF_ASSIGN_OP_WITH_INT_SF(>=, C_TYPE, (BITS), (SIGN)) \
2182 AF_ASSIGN_OP_WITH_INT_SF(<=, C_TYPE, (BITS), (SIGN)) \
2183 \
2184 ALL_AF_OP_WITH_INT(bool, 1, false)
2185 ALL_AF_OP_WITH_INT(char, 8, CHAR_IS_SIGNED)
2186 ALL_AF_OP_WITH_INT(signed char, 8, true)
2187 ALL_AF_OP_WITH_INT(unsigned char, 8, false)
2188 ALL_AF_OP_WITH_INT(short, _AP_SIZE_short, true)
2189 ALL_AF_OP_WITH_INT(unsigned short, _AP_SIZE_short, false)
2190 ALL_AF_OP_WITH_INT(int, _AP_SIZE_int, true)
2191 ALL_AF_OP_WITH_INT(unsigned int, _AP_SIZE_int, false)
2192 ALL_AF_OP_WITH_INT(long, _AP_SIZE_long, true)
2193 ALL_AF_OP_WITH_INT(unsigned long, _AP_SIZE_long, false)
2194 ALL_AF_OP_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
2195 ALL_AF_OP_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
2196 \
2197 #undef ALL_AF_OP_WITH_INT
2198 #undef AF_BIN_OP_WITH_INT
2199 #undef AF_BIN_OP_WITH_INT_SF
2200 #undef AF_ASSIGN_OP_WITH_INT
2201 #undef AF_ASSIGN_OP_WITH_INT_SF
2202 #undef AF_REL_OP_WITH_INT
2203 \
2204 /*
2205 *
2206 * *****
2207 * * TODO
2208 * * There is no operator defined with float/double/long
2209 * * double, so that
2210 * * code like
2211 * * ap_fixed<8,4> a = 1.5f;
2212 * * a += 0.5f;
2213 * * will fail in compilation.
2214 * * Operator with warning about conversion might be wanted.
2215 * *
2216 * *****
2217 * */
2218 \
2219 #define AF_BIN_OP_WITH_AP_INT(BIN_OP,
2220     RTYPE) \
2221     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2222     _AP_Q, \
2223     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
2224     _AP_S2> \
2225     INLINE typename ap_fixed_base<_AP_W2, _AP_W2, _AP_S2>::tem-
2226     plate RTYPE<
2227     _AP_W, _AP_I,
2228     _AP_S>::RTYPE
2229     \
2230     operator
2231     BIN_OP(

```

```

2222     const ap_int_base<_AP_W2, _AP_S2>&
2223     i_op, \
2224     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2225     _AP_N>& op) { \
2226     return ap_fixed_base<_AP_W2, _AP_W2, _AP_S2>(i_op).oper-
2227     ator BIN_OP(op); \
2228 }
2229 \
2230 \
2231 \
2232 \
2233     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2234     _AP_Q, \
2235     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
2236     _AP_S2> \
2237     INLINE typename ap_fixed_base<_AP_W, _AP_I, _AP_S>::tem-
2238     plate RTYPE<
2239     _AP_W2, _AP_W2,
2240     _AP_S2>::RTYPE
2241     \
2242     operator
2243     BIN_OP(
2244     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2245     _AP_N>& op, \
2246     const ap_int_base<_AP_W2, _AP_S2>& i_op) \
2247 {
2248     return op.operator BIN_OP(ap_fixed_base<_AP_W2, _AP_W2,
2249     _AP_S2>(i_op)); \
2250 }
2251 \
2252 \
2253 \
2254 #define
2255 AF_REL_OP_WITH_AP_INT(REL_OP)
2256 \
2257     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2258     _AP_Q, \
2259     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
2260     _AP_S2> \
2261     INLINE bool operator
2262     REL_OP(
2263     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2264     _AP_N>& op, \
2265     const ap_int_base<_AP_W2, _AP_S2>& i_op) \
2266 {
2267     return op.operator REL_OP(ap_fixed_base<_AP_W2, _AP_W2,
2268     _AP_S2>(i_op)); \
2269 }
2270 \
2271 \
2272 \
2273 \
2274     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2275     _AP_Q, \
2276     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
2277     _AP_S2> \
2278     INLINE bool operator
2279     REL_OP(
2280     const ap_int_base<_AP_W2, _AP_S2>&
2281     i_op, \
2282     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2283     _AP_N>& op) { \
2284     return ap_fixed_base<_AP_W2, _AP_W2, _AP_S2>(i_op).oper-
2285     ator REL_OP(op); \
2286 }
2287 \
2288 \
2289 \
2290 #define
2291 AF_ASSIGN_OP_WITH_AP_INT(ASSIGN_OP)
2292 \
2293     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2294     _AP_Q, \
2295     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
2296     _AP_S2> \
2297     INLINE ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2298     _AP_N>& \
2299     operator AS-
2300     SIGN_OP(
2301     ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2302     _AP_N>& op, \
2303     const ap_int_base<_AP_W2, _AP_S2>& i_op) \
2304 {
2305     return op.operator ASSIGN_OP(ap_fixed_base<_AP_W2,
2306     _AP_S2>(i_op)); \
2307 }
2308 \
2309 \
2310 \
2311 \
2312     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
2313     _AP_Q, \
2314     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
2315     _AP_S2> \
2316     INLINE ap_int_base<_AP_W2, _AP_S2>& operator AS-
2317     SIGN_OP(
2318     ap_int_base<_AP_W2, _AP_S2>&
2319     i_op, \
2320     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
2321     _AP_N>& op) { \
2322     return i_op.operator AS-
2323     SIGN_OP(op.to_ap_int_base()); \
2324 }

```

```

2270 }
2271
2272 AF_BIN_OP_WITH_AP_INT(+, plus)
2273 AF_BIN_OP_WITH_AP_INT(-, minus)
2274 AF_BIN_OP_WITH_AP_INT(*, mult)
2275 AF_BIN_OP_WITH_AP_INT(/, div)
2276 AF_BIN_OP_WITH_AP_INT(&, logic)
2277 AF_BIN_OP_WITH_AP_INT(|, logic)
2278 AF_BIN_OP_WITH_AP_INT(^, logic)
2279
2280 #undef AF_BIN_OP_WITH_AP_INT
2281
2282 AF_ASSIGN_OP_WITH_AP_INT(+=)
2283 AF_ASSIGN_OP_WITH_AP_INT(-=)
2284 AF_ASSIGN_OP_WITH_AP_INT(*=)
2285 AF_ASSIGN_OP_WITH_AP_INT(/=)
2286 AF_ASSIGN_OP_WITH_AP_INT(&=)
2287 AF_ASSIGN_OP_WITH_AP_INT(|=)
2288 AF_ASSIGN_OP_WITH_AP_INT(^=)
2289
2290 #undef AF_ASSIGN_OP_WITH_AP_INT
2291
2292 AF_REL_OP_WITH_AP_INT(==)
2293 AF_REL_OP_WITH_AP_INT(!=)
2294 AF_REL_OP_WITH_AP_INT(>)
2295 AF_REL_OP_WITH_AP_INT(>=)
2296 AF_REL_OP_WITH_AP_INT(<)
2297 AF_REL_OP_WITH_AP_INT(<=)
2298
2299 #undef AF_REL_OP_WITH_AP_INT
2300
2301 // Relational Operators with double
2302 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2303         int _AP_N>
2304 inline bool operator==(
2305     double op1,
2306     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
2307     return op2.operator==(op1);
2308 }
2309
2310 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2311         int _AP_N>
2312 inline bool operator!=(
2313     double op1,
2314     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
2315     return op2.operator!=(op1);
2316 }
2317
2318 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2319         int _AP_N>
2320 inline bool operator>(
2321     double op1,
2322     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
2323     return op2.operator>(op1);
2324 }
2325
2326 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2327         int _AP_N>
2328 inline bool operator<=(
2329     double op1,
2330     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
2331     return op2.operator<=(op1);
2332 }
2333
2334 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2335         int _AP_N>
2336 inline bool operator<(
2337     double op1,
2338     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
2339     return op2.operator<(op1);
2340 }
2341
2342 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
2343         int _AP_N>
2344 inline bool operator>=(
2345     double op1,
2346     const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op2) {
2347     return op2.operator>=(op1);
2348 }
2349
2350 #endif // ifndef __cplusplus else
2351
2352 #endif // ifndef __AP_FIXED_BASE_H__ else
2353
2354 // -*- cpp -*-

```

E.6 vendor/ap_fixed_ref.h (718 lines, 27,302 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
5  * you may not use this file except in compliance with the
License.
6  * You may obtain a copy of the License at
7  *
8  * http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-
missions and
14 * limitations under the License.
15 */
16
17 #ifndef __AP_FIXED_REF_H__
18 #define __AP_FIXED_REF_H__
19
20 #ifndef __AP_FIXED_H__
21 #error "Only ap_fixed.h and ap_int.h can be included directly in
user code."
22 #endif
23
24 #ifndef __cplusplus
25 #error "C++ is required to include this header file"
26
27 #else
28 #ifndef __SYNTHESIS__
29 #include <iostream>
30 #endif
31
32 // Proxy class, which allows bit selection to be used as
both rvalue (for
33 // reading) and lvalue (for writing)
34 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
35         int _AP_N>
36 struct af_bit_ref {
37     #ifdef _MSC_VER
38     #pragma warning(disable : 4521 4522)
39     #endif
40     typedef ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>
ref_type;
41     ref_type& d_bv;
42     int d_index;
43
44 public:
45     inline af_bit_ref(
46         const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& ref)
47         : d_bv(ref.d_bv), d_index(ref.d_index) {
48     #ifndef __SYNTHESIS__
49     _AP_WARNING(d_index < 0, "Index of bit vector (%d) can-
not be negative.",
50         d_index);
51     _AP_WARNING(d_index >= _AP_W, "Index of bit vector (%d)
out of range (%d).",
52         d_index, _AP_W);
53     #endif
54 }
55
56 inline af_bit_ref(ref_type* bv, int index = 0) : d_bv(*bv),
d_index(index) {}
57
58 inline af_bit_ref(const ref_type* bv, int index = 0)
59     : d_bv(*const_cast<ref_type*>(bv)), d_index(index) {}
60
61 // convert operators.
62 inline operator bool() const { return
_AP_ROOT_op_get_bit(d_bv.V, d_index); }
63
64 // @name assign operators
65 // @{
66 inline af_bit_ref& operator=(bool val) {
67     d_bv.V = _AP_ROOT_op_set_bit(d_bv.V, d_index, val);
68     return *this;
69 }
70
71 // Be explicit to prevent it from being deleted, as field
d_bv
72 // is of reference type.
73 inline af_bit_ref& operator=(const af_bit_ref& val) {
74     return operator=(bool(val));
75 }
76
77 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,

```



```

77     ap_o_mode _AP_O2, int _AP_N2>
78     INLINE af_bit_ref& operator=(
79         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
80         return operator=(bool(val));
81     }
82
83     template <int _AP_W2, bool _AP_S2>
84     INLINE af_bit_ref& operator=(const ap_bit_ref<_AP_W2,
_AP_S2>& val) {
85         return operator=(bool(val));
86     }
87
88     template <int _AP_W2, bool _AP_S2>
89     INLINE af_bit_ref& operator=(const ap_int_base<_AP_W2,
_AP_S2>& val) {
90         return operator=(val != 0);
91     }
92
93     template <int _AP_W2, bool _AP_S2>
94     INLINE af_bit_ref& operator=(const ap_range_ref<_AP_W2,
_AP_S2>& val) {
95         return operator=(ap_int_base<_AP_W2, false>(val));
96     }
97
98     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
99         ap_o_mode _AP_O2, int _AP_N2>
100     INLINE af_bit_ref& operator=(
101         const ap_concat_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
102         return operator=(ap_int_base<_AP_W2, false>(val));
103     }
104
105     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
106     INLINE af_bit_ref& operator=(
107         const ap_concat_ref<_AP_W2, _AP_T3, _AP_W3, _AP_T3>&
val) {
108         return operator=(ap_int_base<_AP_W2 + _AP_W3,
false>(val));
109     }
110     // @}
111
112     /// @name concatenate operators
113     // @{
114     template <int _AP_W2, int _AP_S2>
115     INLINE ap_concat_ref<1, af_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
116     operator,(ap_int_base<_AP_W2, _AP_S2> &op) {
117         return ap_concat_ref<1, af_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >(&
118             *this, op);
119     }
120
121     template <int _AP_W2, int _AP_S2>
122     INLINE ap_concat_ref<1, af_bit_ref, 1, ap_bit_ref<_AP_W2,
_AP_S2> > operator,(
123         const ap_bit_ref<_AP_W2, _AP_S2> &op) {
124         return ap_concat_ref<1, af_bit_ref, 1, ap_bit_ref<_AP_W2,
_AP_S2> >(&*this,
125             op);
126     }
127
128     template <int _AP_W2, int _AP_S2>
129     INLINE ap_concat_ref<1, af_bit_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >
130     operator,(const ap_range_ref<_AP_W2, _AP_S2> &op) {
131         return ap_concat_ref<1, af_bit_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >(&
132             *this, op);
133     }
134
135     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
136     INLINE ap_concat_ref<1, af_bit_ref, _AP_W2 + _AP_W3,
ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> >
137     operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &op) {
138         return ap_concat_ref<1, af_bit_ref, _AP_W2 + _AP_W3,
ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >(&*this,
139             op);
140     }
141
142     }
143
144     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
145         ap_o_mode _AP_O2, int _AP_N2>
146     INLINE ap_concat_ref<
147         1, af_bit_ref, _AP_W2,
148         af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >

```

```

149     operator, (
150         const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op) {
151         return ap_concat_ref<
152             1, af_bit_ref, _AP_W2,
153             af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_Q2,
_AP_N2> >(*this,
154
155         op);
156     }
157     template<int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
158         ap_o_mode _AP_Q2, int _AP_N2>
159     INLINE ap_concat_ref<1, af_bit_ref, 1, af_bit_ref<_AP_W2,
_AP_I2, _AP_S2,
160         _AP_Q2,
_AP_Q2, _AP_N2> >
161     operator, (
162         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2> &op) {
163         return ap_concat_ref<1, af_bit_ref, 1, af_bit_ref<_AP_W2,
_AP_I2, _AP_S2,
164         _AP_Q2,
_AP_Q2, _AP_N2> > (
165             *this,
166             const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op>());
167     }
168     // @}
169     /// @name comparison
170     // @{
171     template<int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
172         ap_o_mode _AP_Q2, int _AP_N2>
173     INLINE bool operator==(
174         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op) {
175         return get() == op.get();
176     }
177     }
178     template<int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
179         ap_o_mode _AP_Q2, int _AP_N2>
180     INLINE bool operator!=(
181         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op) {
182         return get() != op.get();
183     }
184     // @}
185     INLINE bool operator-() const {
186         bool bit = _AP_ROOT_op_get_bit(d_bv.V, d_index);
187         return bit ? false : true;
188     }
189     INLINE bool get() const { return
_AP_ROOT_op_get_bit(d_bv.V, d_index); }
190
191     INLINE int length() const { return 1; }
192
193     #ifndef __SYNTHESIS__
194     std::string to_string() const { return get() ? "1" : "0"; }
195     #else
196     // XXX HLS will delete this in synthesis
197     INLINE char* to_string() const { return 0; }
198     #endif
199     }; // struct af_bit_ref
200
201     // XXX apcc cannot handle global std::ios_base::Init()
202     brought in by <iostream>
203     #ifndef AP_AUTOCC
204     #ifndef __SYNTHESIS__
205     template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
206         int _AP_N>
207     INLINE std::ostream& operator<<(
208         std::ostream& os,
209         const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& x) {
210         os << x.to_string();
211         return os;
212     }
213     #endif // ifndef __SYNTHESIS__
214     #endif // ifndef AP_AUTOCC
215
216     /// Range (slice) reference.
217     template<int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
218         int _AP_N>
219     struct af_range_ref {
220     #ifdef _MSC_VER
221     #pragma warning(disable : 4521 4522)
222     #endif

```

```

226 typedef ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N> ref_type;
227 ref_type& d_bv;
228 int l_index;
229 int h_index;
230
231 public:
232 /// copy ctor
233 INLINE af_range_ref(
234 const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& ref)
235 : d_bv(ref.d_bv), l_index(ref.l_index), h_in-
dex(ref.h_index) {}
236
237 /// ctor from ap_fixed_base, higher and lower bound.
238
239 /** if h is less than l, the bits selected will be returned
in reverse order.
240 */
241 INLINE af_range_ref(ref_type* bv, int h, int l)
242 : d_bv(*bv), l_index(l), h_index(h) {}
243 #ifndef __SYNTHESIS__
244 _AP_WARNING(h < 0 || l < 0,
245 "Higher bound(%d) and lower(%d) bound cannot
be negative.", h,
246 l);
247 _AP_WARNING(h >= _AP_W || l >= _AP_W,
248 "Higher bound(%d) or lower(%d) bound out of
range.", h, l);
249
250 _AP_WARNING(h < l, "The bits selected will be returned in
reverse order.");
251 #endif
252
253 INLINE af_range_ref(const ref_type* bv, int h, int l)
254 : d_bv(*const_cast<ref_type*>(bv)), l_index(l), h_in-
dex(h) {}
255 #ifndef __SYNTHESIS__
256 _AP_WARNING(h < 0 || l < 0,
257 "Higher bound(%d) and lower(%d) bound cannot
be negative.", h,
258 l);
259 _AP_WARNING(h >= _AP_W || l >= _AP_W,
260 "Higher bound(%d) or lower(%d) bound out of
range.", h, l);
261
262 _AP_WARNING(h < l, "The bits selected will be returned in
reverse order.");
263 #endif
264
265 /// @name assign operators
266 // @{
267 #define AS-
SIGN_CTYPE_TO_AF_RANGE(DATA_TYPE) \
268 INLINE af_range_ref& operator=(const DATA_TYPE val)
{
269 \
270 ap_int_base<_AP_W, false>
loc(val);
271 d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
loc.V); \
272 return
*this; \
273 }
274
275 ASSIGN_CTYPE_TO_AF_RANGE(bool)
276 ASSIGN_CTYPE_TO_AF_RANGE(char)
277 ASSIGN_CTYPE_TO_AF_RANGE(signed char)
278 ASSIGN_CTYPE_TO_AF_RANGE(unsigned char)
279 ASSIGN_CTYPE_TO_AF_RANGE(short)
280 ASSIGN_CTYPE_TO_AF_RANGE(unsigned short)
281 ASSIGN_CTYPE_TO_AF_RANGE(int)
282 ASSIGN_CTYPE_TO_AF_RANGE(unsigned int)
283 ASSIGN_CTYPE_TO_AF_RANGE(long)
284 ASSIGN_CTYPE_TO_AF_RANGE(unsigned long)
285 ASSIGN_CTYPE_TO_AF_RANGE(ap_ulong)
286 #if _AP_ENABLE_HALF_ == 1
287 ASSIGN_CTYPE_TO_AF_RANGE(half)
288 #endif
289 ASSIGN_CTYPE_TO_AF_RANGE(float)
290 ASSIGN_CTYPE_TO_AF_RANGE(double)
291 #undef ASSIGN_CTYPE_TO_AF_RANGE
292
293 /// assign using a string. XXX crucial for cosim.
294 INLINE af_range_ref& operator=(const char* val) {
295 const ap_int_base<_AP_W, false> tmp(val); // XXX figure
out radix
296 d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
tmp.V);
297 return *this;
298 }
299
300 /// assign from ap_int_base.
301 // NOTE Base of other assign operators.
302 template <int _AP_W3, bool _AP_S3>
303 INLINE af_range_ref& operator=(const ap_int_base<_AP_W3,
_AP_S3>& val) {

```

```

304 d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
val.V);
305 return *this;
306 }
307
308 /// assign from range reference to ap_int_base.
309 template <int _AP_W2, bool _AP_S2>
310 INLINE af_range_ref& operator=(const ap_range_ref<_AP_W2,
_AP_S2>& val) {
311 const ap_int_base<_AP_W2, false> tmp(val);
312 return operator=(tmp);
313 }
314
315 /// assign from bit reference to ap_int_base..
316 template <int _AP_W2, bool _AP_S2>
317 INLINE af_range_ref& operator=(const ap_bit_ref<_AP_W2,
_AP_S2>& val) {
318 const ap_int_base<1, false> tmp((bool)val);
319 return operator=(tmp);
320 }
321
322 /// assign from ap_fixed_base.
323 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
324 ap_o_mode _AP_O2, int _AP_N2>
325 INLINE af_range_ref& operator=(
326 const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&
327 val) {
328 d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
val.V);
329 return *this;
330 }
331
332 /// copy assign.
333 // XXX This has to be explicit, otherwise it will be de-
334 leted, as d_bv is
335 // of reference type.
336 INLINE af_range_ref& operator=(const af_range_ref& val) {
337 ap_int_base<_AP_W, false> tmp(val);
338 return operator=(tmp);
339 }
340
341 /// assign from range reference to ap_fixed_base.
342 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
343 ap_o_mode _AP_O2, int _AP_N2>
344 INLINE af_range_ref& operator=(
345 const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
346 ap_int_base<_AP_W2, false> tmp(val);
347 return operator=(tmp);
348 }
349
350 /// assign from bit reference to ap_fixed_base.
351 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
352 ap_o_mode _AP_O2, int _AP_N2>
353 INLINE af_range_ref& operator=(
354 const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
355 ap_int_base<1, false> tmp((bool)val);
356 return operator=(tmp);
357 }
358
359 /// assign from compound reference.
360
361 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
362 INLINE af_range_ref& operator=(
363 const ap_concat_ref<_AP_W2, _AP_T3, _AP_W3, _AP_T3>&
364 val) {
365 const ap_int_base<_AP_W2 + _AP_W3, false> tmp(val);
366 return operator=(tmp);
367 }
368 // @}
369
370 /// @name comparison operators with ap_range_ref.
371 // @{
372 template <int _AP_W2, bool _AP_S2>
373 INLINE bool operator==(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
374 ap_int_base<_AP_W, false> lop(*this);
375 ap_int_base<_AP_W2, false> rop(op2);
376 return lop == rop;
377 }
378
379 template <int _AP_W2, bool _AP_S2>
380 INLINE bool operator!=(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
381 return !(operator==(op2));
382 }
383
384 template <int _AP_W2, bool _AP_S2>
385 INLINE bool operator<(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
386 ap_int_base<_AP_W, false> lop(*this);
387 ap_int_base<_AP_W2, false> rop(op2);
388 return lop < rop;
389 }

```

```

387
388 template <int _AP_W2, bool _AP_S2>
389 inline bool operator>(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
390     ap_int_base<_AP_W, false> lop(*this);
391     ap_int_base<_AP_W2, false> rop(op2);
392     return lop > rop;
393 }
394
395 template <int _AP_W2, bool _AP_S2>
396 inline bool operator<=(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
397     return !(operator>(op2));
398 }
399
400 template <int _AP_W2, bool _AP_S2>
401 inline bool operator>=(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
402     return !(operator<(op2));
403 }
404 // @}
405
406 /// @name comparison operators with af_range_ref.
407 // @{
408 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
409         ap_o_mode _AP_O2, int _AP_N2>
410 inline bool operator==(
411     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2,
412     _AP_N2>& op2) {
413     ap_int_base<_AP_W, false> lop(*this);
414     ap_int_base<_AP_W2, false> rop(op2);
415     return lop == rop;
416 }
417
418 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
419         ap_o_mode _AP_O2, int _AP_N2>
420 inline bool operator!=(
421     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2,
422     _AP_N2>& op2) {
423     return !(operator==(op2));
424 }
425
426 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
427         ap_o_mode _AP_O2, int _AP_N2>
428 inline bool operator<(
429     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2,
430     _AP_N2>& op2) {
431     ap_int_base<_AP_W, false> lop(*this);
432     ap_int_base<_AP_W2, false> rop(op2);
433     return lop < rop;
434 }
435
436 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
437         ap_o_mode _AP_O2, int _AP_N2>
438 inline bool operator>(
439     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2,
440     _AP_N2>& op2) {
441     ap_int_base<_AP_W, false> lop(*this);
442     ap_int_base<_AP_W2, false> rop(op2);
443     return lop > rop;
444 }
445
446 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
447         ap_o_mode _AP_O2, int _AP_N2>
448 inline bool operator<=(
449     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2,
450     _AP_N2>& op2) {
451     return !(operator>(op2));
452 }
453 // @}
454
455 /// @name concatenate operators.
456 // @{
457 /// concatenate with ap_int_base.
458 template <int _AP_W2, int _AP_S2>
459 inline
460     ap_concat_ref<_AP_W, af_range_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
461     operator,(ap_int_base<_AP_W2, _AP_S2> &op) {
462     return ap_concat_ref<_AP_W, af_range_ref, _AP_W2,
463     ap_int_base<_AP_W2, _AP_S2> >(*this,
op);
464 }
465
466 /// concatenate with ap_bit_ref.

```

```

469     template <int _AP_W2, int _AP_S2>
470     inline ap_concat_ref<_AP_W, af_range_ref, 1,
ap_bit_ref<_AP_W2, _AP_S2> >
471     operator,(const ap_bit_ref<_AP_W2, _AP_S2> &op) {
472     return ap_concat_ref<_AP_W, af_range_ref, 1,
ap_bit_ref<_AP_W2, _AP_S2> >(*this, const_cast<ap_bit_ref<_AP_W2, _AP_S2>&>(op));
473 }
474
475 /// concatenate with ap_bit_ref.
476 template <int _AP_W2, int _AP_S2>
477 inline ap_concat_ref<_AP_W, af_range_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >
478     operator,(const ap_range_ref<_AP_W2, _AP_S2> &op) {
479     return ap_concat_ref<_AP_W, af_range_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >(*this, const_cast<ap_range_ref<_AP_W2,
_AP_S2>&>(op));
480 }
481
482 /// concatenate with ap_concat_ref.
483
484 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
485 inline ap_concat_ref<_AP_W, af_range_ref, _AP_W2 + _AP_W3,
ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> >
486     operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &op) {
487     return ap_concat_ref<_AP_W, af_range_ref, _AP_W2 +
_AP_W3,
488     ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>&>(op));
489 }
490
491 /// concatenate with another af_range_ref.
492 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
493         ap_o_mode _AP_O2, int _AP_N2>
494 inline
495     ap_concat_ref<_AP_W, af_range_ref, _AP_W2,
af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
496     operator,(const af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> &op) {
497     return ap_concat_ref<
498     _AP_W, af_range_ref, _AP_W2,
499     af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_Q2, _AP_O2, _AP_N2>&>(
500     *this,
501     const_cast<af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>&>(op));
502 }
503
504 /// concatenate with another af_bit_ref.
505 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
506         ap_o_mode _AP_O2, int _AP_N2>
507 inline
508     ap_concat_ref<_AP_W, af_range_ref, 1,
af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
509     operator,(
510     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_O2, _AP_N2> &op) {
511     return ap_concat_ref<
512     _AP_W, af_range_ref, 1,
513     af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_Q2, _AP_O2, _AP_N2>&>(
514     *this,
515     const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_O2, _AP_N2>&>(op));
516 }
517
518 /// @}
519
520 inline operator ap_ulong() const {
521     ap_int_base<_AP_W, false> ret;
522     ret.V = _AP_ROOT_op_get_range(d.bv.V, l_index, h_index);
523     return ret.to_uint64();
524 }
525
526 inline operator ap_int_base<_AP_W, false>() const {
527     ap_int_base<_AP_W, false> ret;
528     ret.V = _AP_ROOT_op_get_range(d.bv.V, l_index, h_index);
529     return ret;
530 }
531
532 inline ap_int_base<_AP_W, false> to_ap_int_base() const {
533     ap_int_base<_AP_W, false> ret;
534     ret.V = _AP_ROOT_op_get_range(d.bv.V, l_index, h_index);
535     return ret;
536 }
537
538 // used in ap_fixed_base::to_string()

```



```

547     inline char to_char() const {
548         return (char)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
549     }
550 }
551     inline int to_int() const {
552         return (int)(_AP_ROOT_op_get_range(d_bv.V, l_index, h_in-
dex));
553     }
554 }
555     inline unsigned to_uint() const {
556         return (unsigned)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
557     }
558 }
559     inline long to_long() const {
560         return (long)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
561     }
562 }
563     inline unsigned long to_ulong() const {
564         return (unsigned long)(_AP_ROOT_op_get_range(d_bv.V,
l_index, h_index));
565     }
566 }
567     inline ap_slong to_int64() const {
568         return (ap_slong)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
569     }
570 }
571     inline ap_ulong to_uint64() const {
572         return (ap_ulong)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
573     }
574 }
575     inline ap_int_base<_AP_W, false> get() const {
576         ap_int_base<_AP_W, false> ret;
577         ret.V = _AP_ROOT_op_get_range(d_bv.V, l_index, h_index);
578         return ret;
579     }
580 }
581     template <int _AP_W2>
582     inline void set(const ap_int_base<_AP_W2, false>& val) {
583         d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
val.V);
584     }
585 }
586     inline int length() const {
587         return h_index >= l_index ? h_index - l_index + 1 : l_in-
dex - h_index + 1;
588     }
589 }
590 #ifndef __SYNTHESIS__
591     std::string to_string(signed char rd = 2) const {
592         ap_int_base<_AP_W, false> ret;
593         ret.V = _AP_ROOT_op_get_range(d_bv.V, l_index, h_index);
594         return ret.to_string(rd);
595     }
596 #else
597     // XXX HLS will delete this in synthesis
598     inline char* to_string(signed char rd = 2) const {
599         return 0;
600     }
601 #endif
602 }; // struct af_range_ref
603
604 // XXX apcc cannot handle global std::ios_base::Init()
brought in by <iostream>
605 #ifndef AP_AUTOCC
606 #ifndef __SYNTHESIS__
607     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
608             int _AP_N>
609     inline std::ostream& operator<< (
610         std::ostream& os,
611         const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& x) {
612         os << x.to_string();
613         return os;
614     }
615 #endif
616 #endif // ifndef AP_AUTOCC
617
618 #define AF_REF_REL_OP_WITH_INT(REL_OP, C_TYPE, _AP_W2,
_AP_S2)
619     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
620             ap_o_mode _AP_O, int
_AP_N>
621     inline bool operator
REL_OP(
622         const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op,
623         C_TYPE op2)
624     {
625         return ap_int_base<_AP_W,
false>(op)
626         REL_OP ap_int_base<_AP_W2,
_AP_S2>(op2);

```

```

626     }
627
628     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
629             ap_o_mode _AP_O, int
_AP_N>
630     inline bool operator
REL_OP(
631         C_TYPE
op2,
632         const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op) {
633         return ap_int_base<_AP_W2,
_AP_S2>(op2)
634         REL_OP ap_int_base<_AP_W,
false>(op);
635     }
636
637     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
638             ap_o_mode _AP_O, int
_AP_N>
639     inline bool operator
REL_OP(
640         const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op,
641         C_TYPE op2)
642     {
643         return bool(op) REL_OP
op2;
644     }
645
646     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
647             ap_o_mode _AP_O, int
_AP_N>
648     inline bool operator
REL_OP(
649         C_TYPE
op2,
650         const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op) {
651         return op2 REL_OP
bool(op);
652     }
653 #define AF_REF_REL_OPS_WITH_INT(C_TYPE, _AP_W2, _AP_S2) \
654     AF_REF_REL_OP_WITH_INT(>, C_TYPE, (_AP_W2), (_AP_S2)) \
655     AF_REF_REL_OP_WITH_INT(<, C_TYPE, (_AP_W2), (_AP_S2)) \
656     AF_REF_REL_OP_WITH_INT(>=, C_TYPE, (_AP_W2), (_AP_S2)) \
657     AF_REF_REL_OP_WITH_INT(<=, C_TYPE, (_AP_W2), (_AP_S2)) \
658     AF_REF_REL_OP_WITH_INT(==, C_TYPE, (_AP_W2), (_AP_S2)) \
659     AF_REF_REL_OP_WITH_INT(!=, C_TYPE, (_AP_W2), (_AP_S2))
660
661 AF_REF_REL_OPS_WITH_INT(bool, 1, false)
662 AF_REF_REL_OPS_WITH_INT(char, 8, CHAR_IS_SIGNED)
663 AF_REF_REL_OPS_WITH_INT(signed char, 8, true)
664 AF_REF_REL_OPS_WITH_INT(unsigned char, 8, false)
665 AF_REF_REL_OPS_WITH_INT(short, _AP_SIZE_short, true)
666 AF_REF_REL_OPS_WITH_INT(unsigned short, _AP_SIZE_short,
false)
667 AF_REF_REL_OPS_WITH_INT(int, _AP_SIZE_int, true)
668 AF_REF_REL_OPS_WITH_INT(unsigned int, _AP_SIZE_int, false)
669 AF_REF_REL_OPS_WITH_INT(long, _AP_SIZE_long, true)
670 AF_REF_REL_OPS_WITH_INT(unsigned long, _AP_SIZE_long, false)
671 AF_REF_REL_OPS_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
672 AF_REF_REL_OPS_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
673
674 #undef AF_REF_REL_OP_INT
675 #undef AF_REF_REL_OPS_WITH_INT
676
677 #define
AF_REF_REL_OP_WITH_AP_INT(REL_OP)
678     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
679             ap_o_mode _AP_O, int _AP_N, int _AP_W2, bool
_AP_S2>
680     inline bool operator
REL_OP(
681         const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_O,
_AP_N>& op,
682         const ap_int_base<_AP_W2, _AP_S>& op2)
683     {
684         return ap_int_base<_AP_W, false>(op) REL_OP
op2;
685     }
686
687     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode

```

```

_AP_Q, \
686     ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
_AP_S2> \
687     INLINE bool operator
REL_OP( \
688         const ap_int_base<_AP_W2, _AP_S2>&
op2, \
689         const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N>& op) { \
690     return op2 REL_OP ap_int_base<_AP_W,
false>(op);
691 }
\
692     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
693         ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
_AP_S2> \
694     INLINE bool operator
REL_OP( \
695         const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N>& op, \
696         const ap_int_base<_AP_W2, _AP_S2>& op2)
{
697     return ap_int_base<1, false>(op) REL_OP
op2;
698 }
\
699     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q, \
700         ap_o_mode _AP_0, int _AP_N, int _AP_W2, bool
_AP_S2> \
701     INLINE bool operator
REL_OP( \
702         const ap_int_base<_AP_W2, _AP_S2>&
op2, \
703         const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N>& op) { \
704     return op2 REL_OP ap_int_base<1,
false>(op);
705 }
706
707 AF_REF_REL_OP_WITH_AP_INT(>)
708 AF_REF_REL_OP_WITH_AP_INT(<=)
709 AF_REF_REL_OP_WITH_AP_INT(>=)
710 AF_REF_REL_OP_WITH_AP_INT(<=)
711 AF_REF_REL_OP_WITH_AP_INT(==)
712 AF_REF_REL_OP_WITH_AP_INT(!=)
713
714 #endif // ifndef __cplusplus
715
716 #endif // ifndef __AP_FIXED_REF_H__
717
718 // -*- cpp -*-

```

E.7 vendor/ap_fixed_special.h (230 lines, 6,985 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  *  * you may not use this file except in compliance with the
6  *  * You may obtain a copy of the License at
7  *  *
8  *      http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
11 * software
12 * distributed under the License is distributed on an "AS IS"
13 * BASIS,
14 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 * press or implied.
16 * See the License for the specific language governing per-
17 * missions and
18 * limitations under the License.
19 *
20 */
21 #ifndef __AP_FIXED_SPECIAL_H__
22 #define __AP_FIXED_SPECIAL_H__
23 #endif
24 #ifndef __AP_FIXED_H__
25 #error "Only ap_fixed.h and ap_int.h can be included in
26 user code."
27 #endif
28 #ifndef __SYNTHESIS__
29 #include <stdio>
30 #include <stdlib>
31 #endif
32 // FIXME AP_AUTOCC cannot handle many standard headers, so
33 declare instead of
34 // include.
35 // #include <complex>
36 namespace std {

```

```

32 template<typename _Tp> class complex;
33 }
34
35 /*
36  * TODO: Modernize the code using C++11/C++14
37  * 1. constexpr http://www.open-std.org/jtc1/sc22/wg21/docs/
38  *    papers/2016/p0415r0.html
39  * 2. move constructor
40  */
41 namespace std {
42 /*
43  * Specialize std::complex<ap_fixed> to zero initialization
44  * ap_fixed.
45
46  * To reduce the area cost, ap_fixed is not zero initialized,
47  * just like basic
48  * types float or double. However, libstdc++ provides spe-
49  * cialization for float,
50  * double and long double, initializing image part to 0 when
51  * not specified.
52
53  * This has become a difficulty in switching legacy code from
54  * these C types to
55  * ap_fixed. To ease the transform of legacy code, we have to
56  * implement
57  * specialization of std::complex<> for our type.
58
59  * As ap_fixed is a template, it is impossible to specialize
60  * only the methods
61  * that causes default initialization of value type in
62  * std::complex<>. An
63  * explicit full specialization of the template class has to
64  * be done, covering
65  * all the member functions and operators of std::complex<>
66  * as specified
67  * in standard 26.2.4 and 26.2.5.
68  */
69 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
_AP_0, int _AP_N>
70 class complex<ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_0, _AP_N> > {
71 public:
72     typedef ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_0, _AP_N> _Tp;
73     typedef _Tp value_type;
74
75     // 26.2.4/1
76     // Constructor without argument
77
78     // Default initialize, so that in dataflow, the variable is
79     // only written once.
80     complex() : _M_real(_Tp()), _M_imag(_Tp()) {}
81     // Constructor with ap_fixed.
82     // Zero initialize image part when not specified, so that
83     // C(1,0) == C(1,0)
84     complex(const _Tp &r, const _Tp &i = _Tp(0))
85         : _M_real(r), _M_imag(i) {}
86
87     // Constructor with another complex number
88     template <typename _Up>
89     complex(const complex<_Up> &z) : _M_real(z.real()),
90         _M_imag(z.imag()) {}
91
92     #if __cplusplus >= 201103L
93     const _Tp& real() const { return _M_real; }
94     const _Tp& imag() const { return _M_imag; }
95     #else
96     _Tp& real() { return _M_real; }
97     const _Tp& real() const { return _M_real; }
98     _Tp& imag() { return _M_imag; }
99     const _Tp& imag() const { return _M_imag; }
100 #endif
101
102     void real(_Tp __val) { _M_real = __val; }
103     void imag(_Tp __val) { _M_imag = __val; }
104
105     // Assign this complex number with ap_fixed.
106     // Zero initialize image poarrt, so that `C c; c = 1; c ==
107     C(1,0);`
108     complex<_Tp> &operator=(const _Tp __t) {
109         _M_real = __t;
110         _M_imag = _Tp(0);
111         return *this;
112     }
113
114     // 26.2.5/1
115     // Add ap_fixed to this complex number.
116     complex<_Tp> &operator+=(const _Tp __t) {
117         _M_real += __t;
118         return *this;
119     }
120
121     // 26.2.5/3
122     // Subtract ap_fixed from this complex number.
123     complex<_Tp> &operator-=(const _Tp __t) {
124         _M_real -= __t;
125         return *this;
126     }
127 }

```

```

113 // 26.2.5/5
114 // Multiply this complex number by ap_fixed.
115 complex<Tp> &operator*=(const _Tp &__t) {
116     _M_real *= __t;
117     _M_imag *= __t;
118     return *this;
119 }
120
121 // 26.2.5/7
122 // Divide this complex number by ap_fixed.
123 complex<Tp> &operator/=(const _Tp &__t) {
124     _M_real /= __t;
125     _M_imag /= __t;
126     return *this;
127 }
128
129 // Assign complex number to this complex number.
130 template <typename _Up>
131 complex<Tp> &operator=(const complex<Up> &__z) {
132     _M_real = __z.real();
133     _M_imag = __z.imag();
134     return *this;
135 }
136
137 // 26.2.5/9
138 // Add complex number to this.
139 template <typename _Up>
140 complex<Tp> &operator+=(const complex<Up> &__z) {
141     _M_real += __z.real();
142     _M_imag += __z.imag();
143     return *this;
144 }
145
146 // 26.2.5/11
147 // Subtract complex number from this.
148 template <typename _Up>
149 complex<Tp> &operator-=(const complex<Up> &__z) {
150     _M_real -= __z.real();
151     _M_imag -= __z.imag();
152     return *this;
153 }
154
155 // 26.2.5/13
156 // Multiply this by complex number.
157 template <typename _Up>
158 complex<Tp> &operator*=(const complex<Up> &__z) {
159     const _Tp __r = _M_real * __z.real() - _M_imag *
160     __z.imag();
161     _M_imag = _M_real * __z.imag() + _M_imag * __z.real();
162     _M_real = __r;
163     return *this;
164 }
165
166 // 26.2.5/15
167 // Divide this by complex number.
168 template <typename _Up>
169 complex<Tp> &operator/=(const complex<Up> &__z) {
170     complex<Tp> cj (__z.real(), -__z.imag());
171     complex<Tp> a = (*this) * cj;
172     complex<Tp> b = cj * __z;
173     _M_real = a.real() / b.real();
174     _M_imag = a.imag() / b.real();
175     return *this;
176 }
177
178 private:
179     _Tp _M_real;
180     _Tp _M_imag;
181
182 }; // class complex<ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O,
183 _AP_N> >
184
185 /*
186     Non-member operations
187     These operations are not required by standard in 26.2.6,
188     but libstdc++
189     defines them for
190     float, double or long double's specialization.
191 */
192 // Compare complex number with ap_fixed.
193 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
194 _AP_O, int _AP_N>
195 inline bool operator==(
196     const complex<ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> >
197     &__x,
198     const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> &__y) {
199     return __x.real() == __y &&
200     __x.imag() == 0;
201 }
202
203 // Compare ap_fixed with complex number.
204 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
205 _AP_O, int _AP_N>
206 inline bool operator!=(
207     const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> &__x,
208     const complex<ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> >
209     &__y) {

```

```

204     return __x == __y.real() &&
205     0 == __y.imag();
206 }
207
208 // Compare complex number with ap_fixed.
209 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
210 _AP_O, int _AP_N>
211 inline bool operator!=(
212     const complex<ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> >
213     &__x,
214     const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> &__y) {
215     return __x.real() != __y ||
216     __x.imag() != 0;
217 }
218
219 // Compare ap_fixed with complex number.
220 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q, ap_o_mode
221 _AP_O, int _AP_N>
222 inline bool operator==(
223     const ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> &__x,
224     const complex<ap_fixed<_AP_W, _AP_I, _AP_Q, _AP_O, _AP_N> >
225     &__y) {
226     return __x != __y.real() ||
227     0 != __y.imag();
228 }
229
230 // namespace std
231
232 #ifndef __AP_FIXED_SPECIAL_H__
233 #endif
234
235 // *- cpp *-

```

E.8 vendor/ap_int.h (330 lines, 10,012 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  * cense");
6  * you may not use this file except in compliance with the
7  * License.
8  * You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing,
13  * software
14  * distributed under the License is distributed on an "AS IS"
15  * BASIS,
16  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17  * press or implied.
18  * See the License for the specific language governing per-
19  * missions and
20  * limitations under the License.
21  */
22
23 #ifndef __AP_INT_H__
24 #define __AP_INT_H__
25
26 #include <ap_common.h>
27 #include <ap_int_base.h>
28 #include <ap_int_ref.h>
29
30 //-----
31
32 /// Sign Arbitrary Precision Type.
33 template <int _AP_W>
34 struct ap_int : ap_int_base<_AP_W, true> {
35     typedef ap_int_base<_AP_W, true> Base;
36     // Constructor
37     inline ap_int() : Base() {}
38
39     // Copy ctor
40     inline ap_int(const ap_int& op) { Base::V = op.V; }
41
42     template <int _AP_W2>
43     inline ap_int(const ap_int<_AP_W2>& op) {
44         Base::V = op.V;
45     }
46
47     template <int _AP_W2>
48     inline ap_int(const volatile ap_int<_AP_W2>& op) {
49         Base::V = op.V;
50     }
51
52     template <int _AP_W2>
53     inline ap_int(const volatile ap_uint<_AP_W2>& op) {
54         Base::V = op.V;
55     }
56 }

```

```

56     template <int _AP_W2, bool _AP_S2>
57     inline ap_int(const ap_range_ref<_AP_W2, _AP_S2>& ref) :
Base(ref) {}
58
59     template <int _AP_W2, bool _AP_S2>
60     inline ap_int(const ap_bit_ref<_AP_W2, _AP_S2>& ref) :
Base(ref) {}
61
62     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
63     inline ap_int(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>& ref)
64         : Base(ref) {}
65
66     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_O2,
67         int _AP_N2>
68     inline ap_int(const ap_fixed<_AP_W2, _AP_I2, _AP_Q2,
_AP_O2, _AP_N2>& op)
69         : Base((ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
_AP_O2, _AP_N2>)op) {}
70
71     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_O2,
72         int _AP_N2>
73     inline ap_int(const ap_ufixed<_AP_W2, _AP_I2, _AP_Q2,
_AP_O2, _AP_N2>& op)
74         : Base((ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
_AP_O2, _AP_N2>)op) {}
75
76
77     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_O2,
78         int _AP_N2>
79     inline ap_int(
80         const volatile ap_fixed<_AP_W2, _AP_I2, _AP_Q2, _AP_O2,
_AP_N2>& op)
81         : Base((ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
_AP_O2, _AP_N2>)op) {}
82
83     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_O2,
84         int _AP_N2>
85     inline ap_int(
86         const volatile ap_ufixed<_AP_W2, _AP_I2, _AP_Q2,
_AP_O2, _AP_N2>& op)
87         : Base((ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
_AP_O2, _AP_N2>)op) {}
88
89
90     template <int _AP_W2, bool _AP_S2>
91     inline ap_int(const ap_int_base<_AP_W2, _AP_S2>& op) {
92         Base::V = op.V;
93     }
94
95     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
96         ap_o_mode _AP_O2, int _AP_N2>
97     inline ap_int(
98         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op)
99         : Base(op) {}
100
101     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
102         ap_o_mode _AP_O2, int _AP_N2>
103     inline ap_int(
104         const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op)
105         : Base(op) {}
106
107     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
108         ap_o_mode _AP_O2, int _AP_N2>
109     inline ap_int(
110         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op)
111         : Base(op) {}
112
113 #define CTOR(TYPE) \
114     inline ap_int(TYPE val) { Base::V = val; }
115 CTOR(bool)
116 CTOR(char)
117 CTOR(signed char)
118 CTOR(unsigned char)
119 CTOR(short)
120 CTOR(unsigned short)
121 CTOR(int)
122 CTOR(unsigned int)
123 CTOR(long)
124 CTOR(unsigned long)
125 CTOR(ap_slong)
126 CTOR(ap_ulong)
127 #undef CTOR
128 ap_int(double val) : Base(val) {}
129 ap_int(float val) : Base(val) {}
130 #if _AP_ENABLE_HALF == 1
131 ap_int(half val) : Base(val) {}

```

```

132 #endif
133
134 // ap_int_base will guess radix if radix is not provided.
135 inline ap_int(const char* s) : Base(s) {}
136
137 inline ap_int(const char* s, signed char rd) : Base(s, rd)
{}
138
139 // Assignment
140 /* ctor will be used when right is not of proper type. */
141
142 inline ap_int& operator=(const ap_int<_AP_W>& op2) {
143     Base::V = op2.V;
144     return *this;
145 }
146
147 /* cannot bind volatile reference to non-volatile type. */
148 inline ap_int& operator=(const volatile ap_int<_AP_W>& op2) {
149     Base::V = op2.V;
150     return *this;
151 }
152
153 /* cannot return volatile *this. */
154 inline void operator=(const ap_int<_AP_W>& op2) volatile
{ Base::V = op2.V; }
155
156 inline void operator=(const volatile ap_int<_AP_W>& op2)
volatile {
157     Base::V = op2.V;
158 }
159
160 }; // struct ap_int.
161
162 //-----
163
164 /// Unsigned Arbitrary Precision Type.
165 template <int _AP_W>
166 struct ap_uint : ap_int_base<_AP_W, false> {
167     typedef ap_int_base<_AP_W, false> Base;
168     // Constructor
169     inline ap_uint() : Base() {}
170
171     // Copy ctor
172     inline ap_uint(const ap_uint& op) { Base::V = op.V; }
173
174     template <int _AP_W2>
175     inline ap_uint(const ap_uint<_AP_W2>& op) {
176         Base::V = op.V;
177     }
178
179     template <int _AP_W2>
180     inline ap_uint(const ap_int<_AP_W2>& op) {
181         Base::V = op.V;
182     }
183
184     template <int _AP_W2>
185     inline ap_uint(const volatile ap_uint<_AP_W2>& op) {
186         Base::V = op.V;
187     }
188
189     template <int _AP_W2>
190     inline ap_uint(const volatile ap_int<_AP_W2>& op) {
191         Base::V = op.V;
192     }
193
194     template <int _AP_W2, bool _AP_S2>
195     inline ap_uint(const ap_range_ref<_AP_W2, _AP_S2>& ref) :
Base(ref) {}
196
197     template <int _AP_W2, bool _AP_S2>
198     inline ap_uint(const ap_bit_ref<_AP_W2, _AP_S2>& ref) :
Base(ref) {}
199
200     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
201     inline ap_uint(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>& ref)
202         : Base(ref) {}
203
204     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_O2,
205         int _AP_N2>
206     inline ap_uint(const ap_fixed<_AP_W2, _AP_I2, _AP_Q2,
_AP_O2, _AP_N2>& op)
207         : Base((ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
_AP_O2, _AP_N2>)op) {}
208
209     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_O2,
210         int _AP_N2>
211     inline ap_uint(const ap_ufixed<_AP_W2, _AP_I2, _AP_Q2,
_AP_O2, _AP_N2>& op)
212         : Base((ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
_AP_O2, _AP_N2>)op) {}
213
214
215     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,

```

```

ap_o_mode _AP_Q2,
216     int _AP_N2>
217     INLINE ap_uint{

218     const volatile ap_fixed<_AP_W2, _AP_I2, _AP_Q2, _AP_Q2,
_AP_N2>& op)
219     : Base((ap_fixed_base<_AP_W2, _AP_I2, true, _AP_Q2,
_AP_Q2, _AP_N2>)op) {}
220
221     template <int _AP_W2, int _AP_I2, ap_q_mode _AP_Q2,
ap_o_mode _AP_Q2,
222     int _AP_N2>
223     INLINE ap_uint{
224     const volatile ap_ufixed<_AP_W2, _AP_I2, _AP_Q2,
_AP_Q2, _AP_N2>& op)
225     : Base((ap_fixed_base<_AP_W2, _AP_I2, false, _AP_Q2,
_AP_Q2, _AP_N2>)op) {}
226     }
227
228     template <int _AP_W2, bool _AP_S2>
229     INLINE ap_uint(const ap_int_base<_AP_W2, _AP_S2>& op) {
230     Base::V = op.V;
231     }
232
233     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
234     ap_o_mode _AP_Q2, int _AP_N2>
235     INLINE ap_uint{
236     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op)
237     : Base(op) {}
238
239     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
240     ap_o_mode _AP_Q2, int _AP_N2>
241     INLINE ap_uint{
242     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op)
243     : Base(op) {}
244
245     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
246     ap_o_mode _AP_Q2, int _AP_N2>
247     INLINE ap_uint{
248     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_Q2, _AP_N2>& op)
249     : Base(op) {}
250
251     #define CTOR(TYPE) \
252     INLINE ap_uint(TYPE val) { Base::V = val; }
253     CTOR(bool)
254     CTOR(char)
255     CTOR(signed char)
256     CTOR(unsigned char)
257     CTOR(short)
258     CTOR(unsigned short)
259     CTOR(int)
260     CTOR(unsigned int)
261     CTOR(long)
262     CTOR(unsigned long)
263     CTOR(ap_slong)
264     CTOR(ap_ulong)
265     #undef CTOR
266     ap_uint(double val) : Base(val) {}
267     ap_uint(float val) : Base(val) {}
268     #if _AP_ENABLE_HALF_ == 1
269     ap_uint(half val) : Base(val) {}
270     #endif
271
272     // ap_int_base will guess radix if radix is not provided.
273     INLINE ap_uint(const char* s) : Base(s) {}
274
275     INLINE ap_uint(const char* s, signed char rd) : Base(s, rd)
{}
276
277     // Assignment
278     /* XXX ctor will be used when right is not of proper type.
*/
279
280     INLINE ap_uint& operator=(const ap_uint<_AP_W>& op2) {
281     Base::V = op2.V;
282     return *this;
283     }
284
285     /* cannot bind volatile reference to non-volatile type. */
286     INLINE ap_uint& operator=(const volatile ap_uint<_AP_W>&
op2) {
287     Base::V = op2.V;
288     return *this;
289     }
290
291     /* cannot return volatile *this. */
292     INLINE void operator=(const ap_uint<_AP_W>& op2) volatile
{ Base::V = op2.V; }
293
294     INLINE void operator=(const volatile ap_uint<_AP_W>& op2)
volatile {
295     Base::V = op2.V;
296     }

```

```

297
298 }; // struct ap_uint.
299
300 #define ap_bigint ap_int
301 #define ap_biguint ap_uint
302
303 #if !defined(__SYNTHESIS__) && (defined(SYSTEMC_H) ||
defined(SYSTEMC_INCLUDED))
304 // XXX sc_trace overload for ap_fixed is already included in
305 // "ap_sysc/ap_sc_extras.h", so do not define in synthesis.
306 template <int _AP_W>
307 INLINE void sc_trace(sc_core::sc_trace_file* tf, const
ap_int<_AP_W>& op,
308     const std::string& name) {
309     if (tf) tf-
>trace(sc_dt::sc_lv<_AP_W>(op.to_string(2).c_str()), name);
310 }
311
312 template <int _AP_W>
313 INLINE void sc_trace(sc_core::sc_trace_file* tf, const
ap_uint<_AP_W>& op,
314     const std::string& name) {
315     if (tf) tf-
>trace(sc_dt::sc_lv<_AP_W>(op.to_string(2).c_str()), name);
316 }
317 #endif // System C sim
318
319 #include <ap_int_special.h>
320
321 #endif // ifndef __AP_INT_H__ else
322
323 // FIXME user should include ap_fixed.h when using ap_fixed.
324 // to avoid circular inclusion, must check whether this is
required by
325 // ap_fixed.h
326 #ifndef __AP_FIXED_H__
327 #include <ap_fixed.h>
328 #endif
329
330 // -*- cpp -*-

```

E.9 vendor/ap_int_base.h (1,885 lines, 71,735 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
5  * you may not use this file except in compliance with the
License.
6  * You may obtain a copy of the License at
7  *
8  * http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-
missions and
14 * limitations under the License.
15 */
16
17 #ifndef __AP_INT_BASE_H__
18 #define __AP_INT_BASE_H__
19
20 #ifndef __AP_INT_H__
21 #error "Only ap_fixed.h and ap_int.h can be included directly in
user code."
22 #endif
23
24 #ifndef __cplusplus
25 #error "C++ is required to include this header file"
26 #else
27
28 #include <ap_common.h>
29 #ifndef __SYNTHESIS__
30 #if _AP_ENABLE_HALF_ == 1
31 #include <hls_half.h>
32 #endif
33 #include <iostream>
34 #include <string.h>
35 #endif
36
37 /*
38  * ap_int_base: AutoPilot integer/Arbitrary precision in-
teger.
39  *
40  */
41
42 /* helper trait. Selecting the smallest C type that can hold
the value,
43  * return 64 bit C type if not possible.

```



```

44  */
45  template <int _AP_N, bool _AP_S>
46  struct retval;
47
48  // at least 64 bit
49  template <int _AP_N>
50  struct retval<_AP_N, true> {
51      typedef ap_slong Type;
52  };
53
54  template <int _AP_N>
55  struct retval<_AP_N, false> {
56      typedef ap_ulong Type;
57  };
58
59  // at least 8 bit
60  template <>
61  struct retval<1, true> {
62      typedef signed char Type;
63  };
64
65  template <>
66  struct retval<1, false> {
67      typedef unsigned char Type;
68  };
69
70  // at least 16 bit
71  template <>
72  struct retval<2, true> {
73      typedef short Type;
74  };
75
76  template <>
77  struct retval<2, false> {
78      typedef unsigned short Type;
79  };
80
81  // at least 32 bit
82  template <>
83  struct retval<3, true> {
84      typedef long Type;
85  };
86
87  template <>
88  struct retval<3, false> {
89      typedef unsigned long Type;
90  };
91
92  template <>
93  struct retval<4, true> {
94      typedef long Type;
95  };
96
97  template <>
98  struct retval<4, false> {
99      typedef unsigned long Type;
100 };
101
102 // trait for letting base class to return derived class.
103 // Notice that derived class template is incomplete, and we
cannot use
104 // the member of the derived class.
105 template <int _AP_W2, bool _AP_S2>
106 struct _ap_int_factory;
107 template <int _AP_W2>
108 struct _ap_int_factory<_AP_W2, true> { typedef ap_int<_AP_W2>
type; };
109 template <int _AP_W2>
110 struct _ap_int_factory<_AP_W2, false> { typedef
ap_uint<_AP_W2> type; };
111
112 template <int _AP_W, bool _AP_S>
113 struct ap_int_base : public _AP_ROOT_TYPE<_AP_W, _AP_S> {
114     public:
115         typedef _AP_ROOT_TYPE<_AP_W, _AP_S> Base;
116
117         /* ap_int_base<_AP_W, _AP_S, true>
118          * typedef typename retval<(_AP_W + 7) / 8, _AP_S>::Type
RetType;
119          *
120          * ap_int_base<_AP_W, _AP_S, false>
121          * typedef typename retval<8, _AP_S>::Type RetType;
122          */
123         typedef typename retval<AP_MAX((_AP_W + 7) / 8, 8),
_AP_S>::Type RetType;
124
125         static const int width = _AP_W;
126
127         template <int _AP_W2, bool _AP_S2>
128         struct RType {
129             enum {
130                 mult_w = _AP_W + _AP_W2,
131                 mult_s = _AP_S || _AP_S2,
132                 plus_w =
133                     AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 + (_AP_S
&& !_AP_S2)) + 1,
134                 plus_s = _AP_S || _AP_S2,
135                 minus_w =
136                     AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 + (_AP_S
&& !_AP_S2)) + 1,

```

```

137         minus_s = true,
138         div_w = _AP_W + _AP_S2,
139         div_s = _AP_S || _AP_S2,
140         mod_w = AP_MIN(_AP_W, _AP_W2 + (!_AP_S2 && !_AP_S)),
141         mod_s = _AP_S,
142         logic_w = AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 +
(_AP_S && !_AP_S2)),
143         logic_s = _AP_S || _AP_S2
144     };
145
146     typedef ap_int_base<mult_w, mult_s> mult_base;
147     typedef ap_int_base<plus_w, plus_s> plus_base;
148     typedef ap_int_base<minus_w, minus_s> minus_base;
149     typedef ap_int_base<logic_w, logic_s> logic_base;
150     typedef ap_int_base<div_w, div_s> div_base;
151     typedef ap_int_base<mod_w, mod_s> mod_base;
152     typedef ap_int_base<_AP_W, _AP_S> arg1_base;
153
154     typedef typename _ap_int_factory<mult_w, mult_s>::type
mult;
155     typedef typename _ap_int_factory<plus_w, plus_s>::type
plus;
156     typedef typename _ap_int_factory<minus_w, minus_s>::type
minus;
157     typedef typename _ap_int_factory<logic_w, logic_s>::type
logic;
158     typedef typename _ap_int_factory<div_w, div_s>::type div;
159     typedef typename _ap_int_factory<mod_w, mod_s>::type mod;
160     typedef typename _ap_int_factory<_AP_W, _AP_S>::type
arg1;
161     typedef bool reduce;
162 };
163
164 /* Constructors.
165  *
166  */
167
168 // default ctor
169 inline ap_int_base() {
170     /*
171     #ifdef __SC_COMPATIBLE__
172     Base::V = 0;
173     #endif
174     */
175 }
176
177 // copy ctor
178 template <int _AP_W2, bool _AP_S2>
179 inline ap_int_base(const ap_int_base<_AP_W2, _AP_S2>& op) {
180     Base::V = op.V;
181 }
182
183 // volatile copy ctor
184 template <int _AP_W2, bool _AP_S2>
185 inline ap_int_base(const volatile ap_int_base<_AP_W2,
_AP_S2>& op) {
186     Base::V = op.V;
187 }
188
189 // XXX C++11 feature.
190 // The explicit specifier specifies that a constructor or
conversion function
191 // (since C++11) doesn't allow implicit conversions or copy-
initialization.
192 // ap_int_base<W,S> x = 1;
193 // ap_int_base<W,S> foo() { return 1; }
194 // but allows
195 // ap_int_base<W,S> x(1);
196 // ap_int_base<W,S> y {1};
197
198 // from all c types.
199 #define CTOR_FROM_INT(Type, Size, Signed) \
200     inline ap_int_base(const Type op) { Base::V = op; }
201
202     CTOR_FROM_INT(bool, 1, false)
203     CTOR_FROM_INT(char, 8, CHAR_IS_SIGNED)
204     CTOR_FROM_INT(signed char, 8, true)
205     CTOR_FROM_INT(unsigned char, 8, false)
206     CTOR_FROM_INT(short, _AP_SIZE_short, true)
207     CTOR_FROM_INT(unsigned short, _AP_SIZE_short, false)
208     CTOR_FROM_INT(int, _AP_SIZE_int, true)
209     CTOR_FROM_INT(unsigned int, _AP_SIZE_int, false)
210     CTOR_FROM_INT(long, _AP_SIZE_long, true)
211     CTOR_FROM_INT(unsigned long, _AP_SIZE_long, false)
212     CTOR_FROM_INT(ap_slong, _AP_SIZE_ap_slong, true)
213     CTOR_FROM_INT(ap_ulong, _AP_SIZE_ap_slong, false)
214 #undef CTOR_FROM_INT
215
216 #if _AP_ENABLE_HALF_ == 1
217     // ctor from half.
218     // TODO optimize
219     inline ap_int_base(half op) {
220         ap_int_base<_AP_W, _AP_S> t((float)op);
221         Base::V = t.V;
222     }
223 #endif
224
225     // ctor from float.
226     inline ap_int_base(float op) {

```

```

227     const int BITS = FLOAT_MAN + FLOAT_EXP + 1;
228     ap_int_base<BITS, false> reg;
229     reg.V = floatToRawBits(op);
230     bool is_neg = _AP_ROOT_op_get_bit(reg.V, BITS - 1);
231
232     ap_int_base<FLOAT_EXP + 1, true> exp = 0;
233     exp.V = _AP_ROOT_op_get_range(reg.V, FLOAT_MAN, BITS -
234 );
235     exp = exp - FLOAT_BIAS;
236
237     ap_int_base<FLOAT_MAN + 2, true> man;
238     man.V = _AP_ROOT_op_get_range(reg.V, 0, FLOAT_MAN - 1);
239     // check for NaN
240     _AP_WARNING(exp == ((unsigned char)(FLOAT_BIAS + 1)) &&
man.V != 0,
241         "assign NaN to ap integer value");
242     // set leading 1.
243     man.V = _AP_ROOT_op_set_bit(man.V, FLOAT_MAN, 1);
244     //if (is_neg) man = -man;
245
246     if ((reg.V & 0x7fffffff) == 0) {
247         Base::V = 0;
248     } else {
249         int sh_amt = FLOAT_MAN - exp.V;
250         if (sh_amt == 0) {
251             Base::V = man.V;
252         } else if (sh_amt > 0) {
253             if (sh_amt < FLOAT_MAN + 2) {
254                 Base::V = man.V >> sh_amt;
255             } else {
256                 if (is_neg)
257                     Base::V = -1;
258                 else
259                     Base::V = 0;
260             }
261         } else {
262             sh_amt = -sh_amt;
263             if (sh_amt < _AP_W) {
264                 Base::V = man.V;
265                 Base::V <= sh_amt;
266             } else {
267                 Base::V = 0;
268             }
269         }
270         if (is_neg) *this = -(*this);
271     }
272
273     /// ctor from double.
274     INLINE ap_int_base(double op) {
275         const int BITS = DOUBLE_MAN + DOUBLE_EXP + 1;
276         ap_int_base<BITS, false> reg;
277         reg.V = doubleToRawBits(op);
278         bool is_neg = _AP_ROOT_op_get_bit(reg.V, BITS - 1);
279
280         ap_int_base<DOUBLE_EXP + 1, true> exp = 0;
281         exp.V = _AP_ROOT_op_get_range(reg.V, DOUBLE_MAN, BITS -
282 );
283         exp = exp - DOUBLE_BIAS;
284
285         ap_int_base<DOUBLE_MAN + 2, true> man;
286         man.V = _AP_ROOT_op_get_range(reg.V, 0, DOUBLE_MAN - 1);
287         // check for NaN
288         _AP_WARNING(exp == ((unsigned char)(DOUBLE_BIAS + 1)) &&
man.V != 0,
289             "assign NaN to ap integer value");
290         // set leading 1.
291         man.V = _AP_ROOT_op_set_bit(man.V, DOUBLE_MAN, 1);
292         //if (is_neg) man = -man;
293
294         if ((reg.V & 0x7fffffffffffffff) == 0) {
295             Base::V = 0;
296         } else {
297             int sh_amt = DOUBLE_MAN - exp.V;
298             if (sh_amt == 0) {
299                 Base::V = man.V;
300             } else if (sh_amt > 0) {
301                 if (sh_amt < DOUBLE_MAN + 2) {
302                     Base::V = man.V >> sh_amt;
303                 } else {
304                     if (is_neg)
305                         Base::V = -1;
306                     else
307                         Base::V = 0;
308                 }
309             } else {
310                 sh_amt = -sh_amt;
311                 if (sh_amt < _AP_W) {
312                     Base::V = man.V;
313                     Base::V <= sh_amt;
314                 } else {
315                     Base::V = 0;
316                 }
317             }
318             if (is_neg) *this = -(*this);
319         }
320
321         /// from higher rank type.
322         template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode

```

```

_AP_Q2,
323         ap_o_mode _AP_O2, int _AP_N2>
324     INLINE ap_int_base(
325         const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
326         _AP_O2, _AP_N2>& op) {
327         Base::V = op.to_ap_int_base().V;
328     }
329     template <int _AP_W2, bool _AP_S2>
330     INLINE ap_int_base(const ap_range_ref<_AP_W2, _AP_S2>& ref) {
331         Base::V = (ref.get()).V;
332     }
333     template <int _AP_W2, bool _AP_S2>
334     INLINE ap_int_base(const ap_bit_ref<_AP_W2, _AP_S2>& ref) {
335         Base::V = ref.operator bool();
336     }
337 }
338
339     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
340     INLINE ap_int_base(const ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>& ref) {
341         const ap_int_base<ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>::_AP_WR,
342             false>
343             tmp = ref.get();
344         Base::V = tmp.V;
345     }
346     /* radix has default value in set */
347
348     #ifndef __SYNTHESIS__
349     INLINE ap_int_base(const char* s, signed char rd = 0) {
350         if (rd == 0)
351             rd = guess_radix(s);
352         unsigned int length = strlen(s);
353         Base::V.fromString(s, length, rd);
354     }
355     #else
356     // XXX __builtin_bit_from_string(...) requires const C
string and radix.
357     INLINE ap_int_base(const char* s) {
358         typedef(Base::V) t;
359         _ssdm_string2bits((void*)&t, (const char*)(s), 10,
_AP_W, _AP_S,
360             AP_TRN, AP_WRAP, 0, _AP_C99);
361         Base::V = t;
362     }
363     INLINE ap_int_base(const char* s, signed char rd) {
364         typedef(Base::V) t;
365         _ssdm_string2bits((void*)&t, (const char*)(s), rd,
_AP_W, _AP_S,
366             AP_TRN, AP_WRAP, 0, _AP_C99);
367         Base::V = t;
368     }
369     #endif
370
371     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
372         ap_o_mode _AP_O2, int _AP_N2>
373     INLINE ap_int_base(
374         const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
375         _AP_O2, _AP_N2>& val) {
376         Base::V = (val.get()).V;
377     }
378     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
379         ap_o_mode _AP_O2, int _AP_N2>
380     INLINE ap_int_base(
381         const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
382         _AP_O2, _AP_N2>& val) {
383         Base::V = val.operator bool();
384     }
385
386     INLINE ap_int_base read() volatile {
387         /*AP_DEBUG printf("call read %d\n", Base::V);*/
388         ap_int_base ret;
389         ret.V = Base::V;
390         return ret;
391     }
392
393     INLINE void write(const ap_int_base<_AP_W, _AP_S>& op2)
volatile {
394         /*AP_DEBUG printf("call write %d\n", op2.V);*/
395         Base::V = op2.V;
396     }
397
398     /* Another form of "write". */
399     template <int _AP_W2, bool _AP_S2>
400     INLINE void operator=(
401         const volatile ap_int_base<_AP_W2, _AP_S2>& op2) volat-
ile {
402         Base::V = op2.V;
403     }
404
405     INLINE void operator=(

```

```

406     const volatile ap_int_base<_AP_W, _AP_S>& op2) volatile {
407         Base::V = op2.V;
408     }
409 }
410 template <int _AP_W2, bool _AP_S2>
411 inline void operator=(const ap_int_base<_AP_W2, _AP_S2>&
op2) volatile {
412     Base::V = op2.V;
413 }
414
415 inline void operator=(const ap_int_base<_AP_W, _AP_S>& op2)
volatile {
416     Base::V = op2.V;
417 }
418
419 template <int _AP_W2, bool _AP_S2>
420 inline ap_int_base& operator=(
421     const volatile ap_int_base<_AP_W2, _AP_S2>& op2) {
422     Base::V = op2.V;
423     return *this;
424 }
425
426 template <int _AP_W2, bool _AP_S2>
427 inline ap_int_base& operator=(const ap_int_base<_AP_W2,
_AP_S2>& op2) {
428     Base::V = op2.V;
429     return *this;
430 }
431
432 inline ap_int_base& operator=(const volatile
ap_int_base<_AP_W, _AP_S>& op2) {
433     Base::V = op2.V;
434     return *this;
435 }
436
437 inline ap_int_base& operator=(const ap_int_base<_AP_W,
_AP_S>& op2) {
438     Base::V = op2.V;
439     return *this;
440 }
441
442 #define ASSIGN_OP_FROM_INT(Type, Size, Signed) \
443 inline ap_int_base& operator=(Type op) { \
444     Base::V = op; \
445     return *this; \
446 }
447
448 ASSIGN_OP_FROM_INT(bool, 1, false)
449 ASSIGN_OP_FROM_INT(char, 8, CHAR_IS_SIGNED)
450 ASSIGN_OP_FROM_INT(signed char, 8, true)
451 ASSIGN_OP_FROM_INT(unsigned char, 8, false)
452 ASSIGN_OP_FROM_INT(short, _AP_SIZE_short, true)
453 ASSIGN_OP_FROM_INT(unsigned short, _AP_SIZE_short, false)
454 ASSIGN_OP_FROM_INT(int, _AP_SIZE_int, true)
455 ASSIGN_OP_FROM_INT(unsigned int, _AP_SIZE_int, false)
456 ASSIGN_OP_FROM_INT(long, _AP_SIZE_long, true)
457 ASSIGN_OP_FROM_INT(unsigned long, _AP_SIZE_long, false)
458 ASSIGN_OP_FROM_INT(ap_slong, _AP_SIZE_ap_slong, true)
459 ASSIGN_OP_FROM_INT(ap_ulong, _AP_SIZE_ap_slong, false)
460
461 #undef ASSIGN_OP_FROM_INT
462
463 template <int _AP_W2, bool _AP_S2>
464 inline ap_int_base& operator=(const ap_bit_ref<_AP_W2,
_AP_S2>& op2) {
465     Base::V = (bool)op2;
466     return *this;
467 }
468
469 template <int _AP_W2, bool _AP_S2>
470 inline ap_int_base& operator=(const ap_range_ref<_AP_W2,
_AP_S2>& op2) {
471     Base::V = (ap_int_base<_AP_W2, false>(op2)).V;
472     return *this;
473 }
474
475
476 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
477 inline ap_int_base& operator=(
478     const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
op2) {
479     Base::V = op2.get().V;
480     return *this;
481 }
482
483 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
484     ap_o_mode _AP_O2, int _AP_N2>
485 inline ap_int_base& operator=(
486     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op) {
487     Base::V = op.to_ap_int_base().V;
488     return *this;
489 }
490
491 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,

```

```

492     ap_o_mode _AP_O2, int _AP_N2>
493 inline ap_int_base& operator=(
494     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op) {
495     Base::V = (bool)op;
496     return *this;
497 }
498
499 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
500     ap_o_mode _AP_O2, int _AP_N2>
501 inline ap_int_base& operator=(
502     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& op) {
503     Base::V = ((const ap_int_base<_AP_W2, false>)(op)).V;
504     return *this;
505 }
506
507 // FIXME: UG902 has clearly required user to use to_int()
to convert to built-in
508 // types, but this implicit conversion is relied on in
hls_cordic.h and hls_rsr.h.
509 // For example:
510 //     int d_exp = fps_x.exp - fps_y.exp;
511 inline operator RetType() const { return (RetType)
(Base::V); }
512
513 /* Explicit conversions to C types.
514 *
515 */
516 inline bool to_bool() const { return (bool)(Base::V); }
517 inline char to_char() const { return (char)(Base::V); }
518 inline signed char to_schar() const { return (signed char)
(Base::V); }
519 inline unsigned char to_uchar() const { return (unsigned
char)(Base::V); }
520 inline short to_short() const { return (short)(Base::V); }
521 inline unsigned short to_ushort() const { return (unsigned
short)(Base::V); }
522 inline int to_int() const { return (int)(Base::V); }
523 inline unsigned to_uint() const { return (unsigned)
(Base::V); }
524 inline long to_long() const { return (long)(Base::V); }
525 inline unsigned long to_ulong() const { return (unsigned
long)(Base::V); }
526 inline ap_slong to_int64() const { return (ap_slong)
(Base::V); }
527 inline ap_ulong to_uint64() const { return (ap_ulong)
(Base::V); }
528 inline float to_float() const { return (float)(Base::V); }
529 inline double to_double() const { return (double)
(Base::V); }
530
531 // TODO decide if user-defined conversion should be
provided.
532 #if 0
533 inline operator char() const { return (char)(Base::V); }
534 inline operator signed char() const { return (signed char)
(Base::V); }
535 inline operator unsigned char() const { return (unsigned
char)(Base::V); }
536 inline operator short() const { return (short)(Base::V); }
537 inline operator unsigned short() const { return (unsigned
short)(Base::V); }
538 inline operator int() const { return (int)(Base::V); }
539 inline operator unsigned int () const { return (unsigned)
(Base::V); }
540 inline operator long () const { return (long)(Base::V); }
541 inline operator unsigned long () const { return (unsigned
long)(Base::V); }
542 inline operator ap_slong () const { return (ap_slong)(Base::V); }
543 inline operator ap_ulong () const { return (ap_ulong)(Base::V); }
544 #endif
545
546 /* Helper methods.
547
548 */
549 /* we cannot call a non-volatile function on a volatile in-
stance.
550 * but calling a volatile function is ok.
551 * XXX deleted non-volatile version.
552 */
553 inline int length() const volatile { return _AP_W; }
554
555 /*Return true if the value of ap_int_base instance is
zero*/
556 inline bool iszero() const { return Base::V == 0; }
557
558 /*Return true if the value of ap_int_base instance is
zero*/
559 inline bool is_zero() const { return Base::V == 0; }
560
561 /* x < 0 */
562 inline bool sign() const {
563     if (_AP_S &&
_AP_ROOT_op_get_bit(Base::V, _AP_W - 1))
564         return true;
565     else

```



```

567     return false;
568 }
569
570 /* x[i] = 0 */
571 inline void clear(int i) {
572     AP_ASSERT(i >= 0 && i < _AP_W, "position out of range");
573     Base::V = _AP_ROOT_op_set_bit(Base::V, i, 0);
574 }
575
576 /* x[i] = !x[i] */
577 inline void invert(int i) {
578     AP_ASSERT(i >= 0 && i < _AP_W, "position out of range");
579     bool val = _AP_ROOT_op_get_bit(Base::V, i);
580     if (val)
581         Base::V = _AP_ROOT_op_set_bit(Base::V, i, 0);
582     else
583         Base::V = _AP_ROOT_op_set_bit(Base::V, i, 1);
584 }
585
586 inline bool test(int i) const {
587     AP_ASSERT(i >= 0 && i < _AP_W, "position out of range");
588     return _AP_ROOT_op_get_bit(Base::V, i);
589 }
590
591 // Get self. For ap_concat_ref expansion.
592 inline ap_int_base& get() { return *this; }
593
594 // Set the ith bit into 1
595 inline void set(int i) {
596     AP_ASSERT(i >= 0 && i < _AP_W, "position out of range");
597     Base::V = _AP_ROOT_op_set_bit(Base::V, i, 1);
598 }
599
600 // Set the ith bit into v
601 inline void set(int i, bool v) {
602     AP_ASSERT(i >= 0 && i < _AP_W, "position out of range");
603     Base::V = _AP_ROOT_op_set_bit(Base::V, i, v);
604 }
605
606 // This is used for sc_lv and sc_bv, which is implemented
607 by sc_uint
608 // Rotate an ap_int_base object n places to the left
609 inline ap_int_base& lrotate(int n) {
610     AP_ASSERT(n >= 0 && n < _AP_W, "shift value out of
range");
611     // TODO unify this.
612     #ifndef __SYNTHESIS__
613     typedef(Base::V) l_p = Base::V << n;
614     typedef(Base::V) r_p = Base::V >> (_AP_W - n);
615     Base::V = l_p | r_p;
616     #else
617     Base::V = lrotate(n);
618     #endif
619     return *this;
620 }
621
622 // This is used for sc_lv and sc_bv, which is implemented
623 by sc_uint
624 // Rotate an ap_int_base object n places to the right
625 inline ap_int_base& rrotate(int n) {
626     AP_ASSERT(n >= 0 && n < _AP_W, "shift value out of
range");
627     // TODO unify this.
628     #ifndef __SYNTHESIS__
629     typedef(Base::V) l_p = Base::V << (_AP_W - n);
630     typedef(Base::V) r_p = Base::V >> n;
631     Base::V = l_p | r_p;
632     #else
633     Base::V = rrotate(n);
634     #endif
635     return *this;
636 }
637
638 // Reverse the contents of ap_int_base instance.
639 // I.e. LSB becomes MSB and vise versa.
640 inline ap_int_base& reverse() {
641     Base::V = _AP_ROOT_op_get_range(Base::V, _AP_W - 1, 0);
642     return *this;
643 }
644
645 // Set the ith bit into v
646 inline void set_bit(int i, bool v) {
647     Base::V = _AP_ROOT_op_set_bit(Base::V, i, v);
648 }
649
650 // Get the value of ith bit
651 inline bool get_bit(int i) const {
652     return (bool)_AP_ROOT_op_get_bit(Base::V, i);
653 }
654
655 // complements every bit
656 inline void b_not() { Base::V = ~Base::V; }
657
658 #define
OP_ASSIGN_AP(Sym)
659 \
660     template <int _AP_W2, bool
661     _AP_S2>
662     inline ap_int_base& operator Sym(const ap_int_base<_AP_W2,
663     _AP_S2>& op2) { \

```

```

659     Base::V Sym
660 op2.V; \
661     return
662 *this; \
663 }
664
665 /* Arithmetic assign.
666 *
667 -----
668 */
669 OP_ASSIGN_AP(+=)
670 OP_ASSIGN_AP(+)=
671 OP_ASSIGN_AP(-=)
672 OP_ASSIGN_AP(/=)
673 OP_ASSIGN_AP(%=)
674 #undef OP_ASSIGN_AP
675
676 /* Bitwise assign: and, or, xor.
677 *
678 -----
679 */
680 #define
OP_ASSIGN_AP_CHK(Sym)
681 \
682     template <int _AP_W2, bool
683     _AP_S2>
684     inline ap_int_base& operator Sym(const ap_int_base<_AP_W2,
685     _AP_S2>& op2) { \
686     _AP_WARNING((_AP_W !=
687     _AP_W2),
688     "Bitsize mismatch for ap_[u]int" #Sym
689     "ap_[u]int."); \
690     Base::V Sym
691 op2.V; \
692     return
693 *this; \
694 }
695
696 OP_ASSIGN_AP_CHK(&=)
697 OP_ASSIGN_AP_CHK(|=)
698 OP_ASSIGN_AP_CHK(^=)
699 #undef OP_ASSIGN_AP_CHK
700
701 /* Prefix increment, decrement.
702 *
703 -----
704 */
705 inline ap_int_base& operator++() {
706     operator+=(ap_int_base<1, false>1);
707     return *this;
708 }
709
710 inline ap_int_base& operator--() {
711     operator-=(ap_int_base<1, false>1);
712     return *this;
713 }
714
715 /* Postfix increment, decrement
716 *
717 -----
718 */
719 inline const typename RType<_AP_W, _AP_S>::arg1 operator++
(int) {
720     ap_int_base t = *this;
721     operator+=(ap_int_base<1, false>1);
722     return t;
723 }
724
725 inline const typename RType<_AP_W, _AP_S>::arg1 operator--
(int) {
726     ap_int_base t = *this;
727     operator-=(ap_int_base<1, false>1);
728     return t;
729 }
730
731 /* Unary arithmetic.
732 *
733 -----
734 */
735 inline typename RType<_AP_W, _AP_S>::arg1 operator+() const
{ return *this; }
736
737 // TODO used to be W>64 only... need check.
738 inline typename RType<1, false>::minus operator-() const {
739     return ap_int_base<1, false>(0) - *this;
740 }
741
742 /* Not (!)
743 *
744 -----
745 */
746 inline bool operator!() const { return Base::V == 0; }
747
748 /* Bitwise (arithmetic) unary: complement
749 *
750 -----
751 */
752 inline typename RType<_AP_W, _AP_S>::arg1 operator~() const
{
753     ap_int_base<_AP_W, _AP_S> r;
754     r.V = ~Base::V;
755     return r;
756 }

```

```

738 }
739
740 /* Shift (result constrained by left operand).
741 *
-----
742 */
743 template <int _AP_W2>
744 inline typename RType<_AP_W, _AP_S>::arg1 operator<<(const
ap_int_base<_AP_W2, true>& op2) const {
745     bool isNeg = _AP_ROOT_op_get_bit(op2.V, _AP_W2 - 1);
746     ap_int_base<_AP_W2, false> sh = op2;
747     if (isNeg) {
748         sh = -op2;
749         return operator>>(sh);
750     } else
751         return operator<<(sh);
752 }
753
754 template <int _AP_W2>
755 inline typename RType<_AP_W, _AP_S>::arg1 operator<<(const
ap_int_base<_AP_W2, false>& op2) const {
756     ap_int_base r;
757     r.V = Base::V << op2.to_uint();
758     return r;
759 }
760
761 template <int _AP_W2>
762 inline typename RType<_AP_W, _AP_S>::arg1 operator>>(const
ap_int_base<_AP_W2, true>& op2) const {
763     bool isNeg = _AP_ROOT_op_get_bit(op2.V, _AP_W2 - 1);
764     ap_int_base<_AP_W2, false> sh = op2;
765     if (isNeg) {
766         sh = -op2;
767         return operator<<(sh);
768     }
769     return operator>>(sh);
770 }
771
772 template <int _AP_W2>
773 inline typename RType<_AP_W, _AP_S>::arg1 operator>>(const
ap_int_base<_AP_W2, false>& op2) const {
774     ap_int_base r;
775     r.V = Base::V >> op2.to_uint();
776     return r;
777 }
778
779 // FIXME we standalone operator>> for ap_int_base and
ap_range_ref.
780 #if 0
781 template <int _AP_W2, bool _AP_S2>
782 inline ap_int_base operator<<(const ap_range_ref<_AP_W2,
_AP_S2>& op2) const {
783     return *this << (op2.operator ap_int_base<_AP_W2,
false>());
784 }
785
786 template <int _AP_W2, bool _AP_S2>
787 inline ap_int_base operator>>(const ap_range_ref<_AP_W2,
_AP_S2>& op2) const {
788     return *this >> (op2.operator ap_int_base<_AP_W2,
false>());
789 }
790 #endif
791
792 /* Shift assign
793 *
-----
794 */
795 template <int _AP_W2>
796 inline ap_int_base& operator<<=(const ap_int_base<_AP_W2,
true>& op2) {
797     bool isNeg = _AP_ROOT_op_get_bit(op2.V, _AP_W2 - 1);
798     ap_int_base<_AP_W2, false> sh = op2;
799     if (isNeg) {
800         sh = -op2;
801         return operator>>=(sh);
802     } else
803         return operator<<=(sh);
804 }
805
806 template <int _AP_W2>
807 inline ap_int_base& operator<<=(const ap_int_base<_AP_W2,
false>& op2) {
808     Base::V <= op2.to_uint();
809     return *this;
810 }
811
812 template <int _AP_W2>
813 inline ap_int_base& operator>>=(const ap_int_base<_AP_W2,
true>& op2) {
814     bool isNeg = _AP_ROOT_op_get_bit(op2.V, _AP_W2 - 1);
815     ap_int_base<_AP_W2, false> sh = op2;
816     if (isNeg) {
817         sh = -op2;
818         return operator<<=(sh);
819     }
820     return operator>>=(sh);
821 }
822
823 template <int _AP_W2>

```

```

824 inline ap_int_base& operator>>=(const ap_int_base<_AP_W2,
false>& op2) {
825     Base::V >= op2.to_uint();
826     return *this;
827 }
828
829 // FIXME we standalone operator>> for ap_int_base and
ap_range_ref.
830 #if 0
831 template <int _AP_W2, bool _AP_S2>
832 inline ap_int_base& operator<<=(const ap_range_ref<_AP_W2,
_AP_S2>& op2) {
833     return *this <= (op2.operator ap_int_base<_AP_W2,
false>());
834 }
835
836 template <int _AP_W2, bool _AP_S2>
837 inline ap_int_base& operator>>=(const ap_range_ref<_AP_W2,
_AP_S2>& op2) {
838     return *this >= (op2.operator ap_int_base<_AP_W2,
false>());
839 }
840 #endif
841
842 /* Equality and Relational.
843 *
-----
844 */
845 template <int _AP_W2, bool _AP_S2>
846 inline bool operator==(const ap_int_base<_AP_W2, _AP_S2>&
op2) const {
847     return Base::V == op2.V;
848 }
849
850 template <int _AP_W2, bool _AP_S2>
851 inline bool operator!=(const ap_int_base<_AP_W2, _AP_S2>&
op2) const {
852     return !(Base::V == op2.V);
853 }
854
855 template <int _AP_W2, bool _AP_S2>
856 inline bool operator<(const ap_int_base<_AP_W2, _AP_S2>&
op2) const {
857     return Base::V < op2.V;
858 }
859
860 template <int _AP_W2, bool _AP_S2>
861 inline bool operator>(const ap_int_base<_AP_W2, _AP_S2>&
op2) const {
862     return Base::V > op2.V;
863 }
864
865 template <int _AP_W2, bool _AP_S2>
866 inline bool operator<=(const ap_int_base<_AP_W2, _AP_S2>&
op2) const {
867     return Base::V <= op2.V;
868 }
869
870 /* Bit and Part Select
871 *
-----
872 */
873 inline ap_range_ref<_AP_W, _AP_S> range(int Hi, int Lo) {
874     _AP_ERROR(Hi >= _AP_W, "Hi(%d)out of bound(%d) in
range()", Hi, _AP_W);
875     _AP_ERROR(Lo >= _AP_W, "Lo(%d)out of bound(%d) in
range()", Lo, _AP_W);
876     return ap_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
877 }
878
879 // This is a must to strip constness to produce reference
type.
880 inline ap_range_ref<_AP_W, _AP_S> range(int Hi, int Lo)
const {
881     _AP_ERROR(Hi >= _AP_W, "Hi(%d)out of bound(%d) in
range()", Hi, _AP_W);
882     _AP_ERROR(Lo >= _AP_W, "Lo(%d)out of bound(%d) in
range()", Lo, _AP_W);
883     return ap_range_ref<_AP_W,
_AP_S>(const_cast<ap_int_base*>(this), Hi, Lo);
884 }
885
886 template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
887 inline ap_range_ref<_AP_W, _AP_S> range(
const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
const ap_int_base<_AP_W3, _AP_S3>& LoIdx) {
888     int Hi = HiIdx.to_int();
889     int Lo = LoIdx.to_int();
890     return this->range(Hi, Lo);
891 }
892
893 template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
894 inline ap_range_ref<_AP_W, _AP_S> range(
const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
const ap_int_base<_AP_W3, _AP_S3>& LoIdx) const {
895     int Hi = HiIdx.to_int();
896     int Lo = LoIdx.to_int();
897     return this->range(Hi, Lo);
898 }
899
900 }
901
902

```

```

903  inline ap_range_ref<_AP_W, _AP_S> range() {
904      return this->range(_AP_W - 1, 0);
905  }
906
907  inline ap_range_ref<_AP_W, _AP_S> range() const {
908      return this->range(_AP_W - 1, 0);
909  }
910
911  inline ap_range_ref<_AP_W, _AP_S> operator()(int Hi, int
Lo) {
912      return this->range(Hi, Lo);
913  }
914
915  inline ap_range_ref<_AP_W, _AP_S> operator()(int Hi, int
Lo) const {
916      return this->range(Hi, Lo);
917  }
918
919  template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
920  inline ap_range_ref<_AP_W, _AP_S> operator()(
921      const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
922      const ap_int_base<_AP_W3, _AP_S3>& LoIdx) {
923      int Hi = HiIdx.to_int();
924      int Lo = LoIdx.to_int();
925      return this->range(Hi, Lo);
926  }
927
928  template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
929  inline ap_range_ref<_AP_W, _AP_S> operator()(
930      const ap_int_base<_AP_W2, _AP_S2>& HiIdx,
931      const ap_int_base<_AP_W3, _AP_S3>& LoIdx) const {
932      int Hi = HiIdx.to_int();
933      int Lo = LoIdx.to_int();
934      return this->range(Hi, Lo);
935  }
936
937  #if 0
938  template<int Hi, int Lo>
939  inline ap_int_base<Hi-Lo+1, false> slice() const {
940      AP_ASSERT(Hi >= Lo && Hi < _AP_W && Lo < _AP_W, "Out of
bounds in slice()");
941      ap_int_base<Hi-Lo+1, false> tmp;
942      tmp.V = _AP_ROOT_op_get_range(Base::V, Lo, Hi);
943      return tmp;
944  }
945
946  inline ap_bit_ref<_AP_W, _AP_S> operator [] ( unsigned int
uindex) {
947      AP_ASSERT(uindex < _AP_W, "Attempting to read bit beyond
MSB");
948      ap_bit_ref<_AP_W, _AP_S> bvh( this, uindex );
949      return bvh;
950  }
951  #endif
952
953  inline ap_bit_ref<_AP_W, _AP_S> operator [] (int index) {
954      AP_ASSERT(index >= 0, "Attempting to read bit with negat-
ive index");
955      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
956      ap_bit_ref<_AP_W, _AP_S> bvh(this, index);
957      return bvh;
958  }
959
960  template <int _AP_W2, bool _AP_S2>
961  inline ap_bit_ref<_AP_W, _AP_S> operator [] (
962      const ap_int_base<_AP_W2, _AP_S2>& index) {
963      AP_ASSERT(index >= 0, "Attempting to read bit with negat-
ive index");
964      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
965      ap_bit_ref<_AP_W, _AP_S> bvh(this, index.to_int());
966      return bvh;
967  }
968
969  inline bool operator [] (int index) const {
970      AP_ASSERT(index >= 0, "Attempting to read bit with negat-
ive index");
971      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
972      ap_bit_ref<_AP_W, _AP_S> br(this, index);
973      return br.to_bool();
974  }
975
976  template <int _AP_W2, bool _AP_S2>
977  inline bool operator [] (const ap_int_base<_AP_W2, _AP_S2>&
index) const {
978      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
979      ap_bit_ref<_AP_W, _AP_S> br(this, index.to_int());
980      return br.to_bool();
981  }
982
983  inline ap_bit_ref<_AP_W, _AP_S> bit(int index) {
984      AP_ASSERT(index >= 0, "Attempting to read bit with negat-
ive index");
985      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
986      ap_bit_ref<_AP_W, _AP_S> bvh(this, index);
987      return bvh;
988  }
989
990  template <int _AP_W2, bool _AP_S2>
991  inline ap_bit_ref<_AP_W, _AP_S> bit(
992      const ap_int_base<_AP_W2, _AP_S2>& index) {
993      AP_ASSERT(index >= 0, "Attempting to read bit with negat-
ive index");
994      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
995      ap_bit_ref<_AP_W, _AP_S> bvh(this, index.to_int());
996      return bvh;
997  }
998
999  inline bool bit(int index) const {
1000      AP_ASSERT(index >= 0, "Attempting to read bit with negat-
ive index");
1001      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
1002      ap_bit_ref<_AP_W, _AP_S> br(this, index);
1003      return br.to_bool();
1004  }
1005
1006  template <int _AP_W2, bool _AP_S2>
1007  inline bool bit(const ap_int_base<_AP_W2, _AP_S2>& index)
const {
1008      return bit(index.to_int());
1009  }
1010
1011  #if 0
1012  template<typename _AP_T>
1013  inline bool operator [] (_AP_T index) const {
1014      AP_ASSERT(index < _AP_W, "Attempting to read bit beyond
MSB");
1015      ap_bit_ref<_AP_W, _AP_S> br = operator [] (index);
1016      return br.to_bool();
1017  }
1018  #endif
1019
1020  // Count the number of zeros from the most significant bit
1021  // to the first one bit.
1022  inline int countLeadingZeros() {
1023      #ifdef __SYNTHESIS__
1024      if (_AP_W <= 32) {
1025          ap_int_base<32, false> t(-1UL), x;
1026          x.V = _AP_ROOT_op_get_range(this->V, _AP_W - 1, 0); //
reverse
1027          t.V = _AP_ROOT_op_set_range(t.V, 0, _AP_W - 1, x.V);
1028          return __builtin_ctz(t.V); // count trailing zeros.
1029      } else if (_AP_W <= 64) {
1030          ap_int_base<64, false> t(-1ULL);
1031          ap_int_base<64, false> x;
1032          x.V = _AP_ROOT_op_get_range(this->V, _AP_W - 1, 0); //
reverse
1033          t.V = _AP_ROOT_op_set_range(t.V, 0, _AP_W - 1, x.V);
1034          return __builtin_ctzll(t.V); // count trailing zeros.
1035      } else {
1036          enum { _N = (_AP_W + 63) / 64 };
1037          int NZeros = 0;
1038          int i = 0;
1039          bool hitNonZero = false;
1040          for (i = 0; i < _N - 1; ++i) {
1041              ap_int_base<64, false> t;
1042              t.V = _AP_ROOT_op_get_range(this->V, _AP_W - i * 64 -
64, _AP_W - i * 64 - 1);
1043              NZeros += hitNonZero ? 0 : __builtin_clzll(t.V); //
count leading zeros.
1044              hitNonZero |= (t.V != 0);
1045          }
1046          if (!hitNonZero) {
1047              ap_int_base<64, false> t(-1ULL);
1048              enum { REST = (_AP_W - 1) % 64 };
1049              ap_int_base<64, false> x;
1050              x.V = _AP_ROOT_op_get_range(this->V, 0, REST);
1051              t.V = _AP_ROOT_op_set_range(t.V, 63 - REST, 63, x.V);
1052              NZeros += __builtin_clzll(t.V);
1053          }
1054          return NZeros;
1055      }
1056  #endif
1057  } // countLeadingZeros
1058
1059  template <int _AP_W2, bool _AP_S2>
1060  inline ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_int_base<_AP_W2, _AP_S2>>
1061  concat(const ap_int_base<_AP_W2, _AP_S2>& a2) const {
1062      return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1063          ap_int_base<_AP_W2, _AP_S2>> (
1064          const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1065          const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
1066  }
1067
1068  template <int _AP_W2, bool _AP_S2>
1069  inline ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_int_base<_AP_W2, _AP_S2>>
1070  concat(ap_int_base<_AP_W2, _AP_S2>& a2) {
1071      return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1072          ap_int_base<_AP_W2, _AP_S2>> (*this,
a2);

```

```

1073     }
1074
1075     template <int _AP_W2, bool _AP_S2>
1076     INLINE
1077     ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >
1078     operator,(const ap_range_ref<_AP_W2, _AP_S2> &a2) const
{
1079     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1080     ap_range_ref<_AP_W2, _AP_S2> >(
1081     const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1082     const_cast<ap_range_ref<_AP_W2, _AP_S2>&>(a2));
1083 }
1084
1085     template <int _AP_W2, bool _AP_S2>
1086     INLINE
1087     ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >
1088     operator,(ap_range_ref<_AP_W2, _AP_S2> &a2) {
1089     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1090     ap_range_ref<_AP_W2, _AP_S2>
>(*this, a2);
1091 }
1092
1093     template <int _AP_W2, bool _AP_S2>
1094     INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
1095     operator,(const ap_int_base<_AP_W2, _AP_S2> &a2) {
1096     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1097     ap_int_base<_AP_W2, _AP_S2> >(
1098     *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
1099 }
1100
1101     template <int _AP_W2, bool _AP_S2>
1102     INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
1103     operator,(ap_int_base<_AP_W2, _AP_S2> &a2) const {
1104     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1105     ap_int_base<_AP_W2, _AP_S2> >(
1106     const_cast<ap_int_base<_AP_W, _AP_S>&>(*this), a2);
1107 }
1108
1109     template <int _AP_W2, bool _AP_S2>
1110     INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
1111     operator,(const ap_int_base<_AP_W2, _AP_S2> &a2) const {
1112     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1113     ap_int_base<_AP_W2, _AP_S2> >(
1114     const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1115     const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
1116 }
1117
1118     template <int _AP_W2, bool _AP_S2>
1119     INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
1120     operator,(ap_int_base<_AP_W2, _AP_S2> &a2) {
1121     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2,
1122     ap_int_base<_AP_W2, _AP_S2> >(*this,
a2);
1123 }
1124
1125     template <int _AP_W2, bool _AP_S2>
1126     INLINE ap_concat_ref<_AP_W, ap_int_base, 1,
ap_bit_ref<_AP_W2, _AP_S2> >
1127     operator,(const ap_bit_ref<_AP_W2, _AP_S2> &a2) const {
1128     return ap_concat_ref<_AP_W, ap_int_base, 1,
ap_bit_ref<_AP_W2, _AP_S2> >(
1129     const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1130     const_cast<ap_bit_ref<_AP_W2, _AP_S2>&>(a2));
1131 }
1132
1133     template <int _AP_W2, bool _AP_S2>
1134     INLINE ap_concat_ref<_AP_W, ap_int_base, 1,
ap_bit_ref<_AP_W2, _AP_S2> >
1135     operator,(ap_bit_ref<_AP_W2, _AP_S2> &a2) {
1136     return ap_concat_ref<_AP_W, ap_int_base, 1,
ap_bit_ref<_AP_W2, _AP_S2> >(
1137     *this, a2);
1138 }
1139
1140     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
1141     INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2 + _AP_W3,
1142     ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> >
1143     operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &a2) {
1144     return ap_concat_ref<_AP_W, ap_int_base, _AP_W2 + _AP_W3,
1145     ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >(
1146     const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1147     const_cast<ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>&>(a2));
1148 }
1149
1150     template <int AP_W2, typename AP_T2, int AP_W3, typename

```

```

1151 _AP_T3>
1152     INLINE ap_concat_ref<_AP_W, ap_int_base, _AP_W2 + _AP_W3,
1153             ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_W3,
_AP_T3> >
1154     operator, (ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>
&a2) {
1155         return ap_concat_ref<_AP_W, ap_int_base, _AP_W2 + _AP_W3,
1156             ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >(*this,
1157
1158     a2);
1159 }
1160
1161 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
1162             ap_o_mode _AP_O2, int _AP_N2>
1163     INLINE ap_concat_ref<
1164         _AP_W, ap_int_base, _AP_W2,
1165         af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_T3> >
1166     operator, (const af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
&a2) const {
1167         return ap_concat_ref<
1168             _AP_W, ap_int_base, _AP_W2,
1169             af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*
1170             const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1171             const_cast<
1172                 af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&(&a2));
1173     }
1174
1175 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
1176             ap_o_mode _AP_O2, int _AP_N2>
1177     INLINE ap_concat_ref<
1178         _AP_W, ap_int_base, _AP_W2,
1179         af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_T3> >
1180     operator, (af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
1181         return ap_concat_ref<
1182             _AP_W, ap_int_base, _AP_W2,
1183             af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*this,
1184
1185     a2);
1186 }
1187
1188 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
1189             ap_o_mode _AP_O2, int _AP_N2>
1190     INLINE
1191     ap_concat_ref<_AP_W, ap_int_base, 1,
1192     af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
1193     operator, (const af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
&a2) const {
1194         return ap_concat_ref<
1195             _AP_W, ap_int_base, 1,
1196             af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*
1197             const_cast<ap_int_base<_AP_W, _AP_S>&>(*this),
1198             const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&(&a2));
1199     }
1200
1201 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
1202             ap_o_mode _AP_O2, int _AP_N2>
1203     INLINE
1204     ap_concat_ref<_AP_W, ap_int_base, 1,
1205     af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
1206     operator, (
1207         af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> &a2) {
1208         return ap_concat_ref<
1209             _AP_W, ap_int_base, 1,
1210             af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*this, a2);
1211     }
1212 }
1213
1214 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
1215     INLINE ap_int_base<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator, (
1216         const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3> &a2) {
1217         return *this & a2.get();
1218     }
1219 }
1220

```

```

1219 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
1220 INLINE ap_int_base<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator|{
1221     const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
1222     return *this | a2.get();
1223     }
1224
1225 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
1226 INLINE ap_int_base<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator^({
1227     const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
1228     return *this ^ a2.get();
1229     }
1230
1231 template <int _AP_W3>
1232 INLINE void set(const ap_int_base<_AP_W3, false>& val) {
1233     Base::V = val.V;
1234 }
1235
1236 /* Reduce operations.
1237 *
-----
1238 */
1239 // XXX non-const version deleted.
1240 INLINE bool and_reduce() const { return
_AP_ROOT_op_reduce(and, Base::V); }
1241 INLINE bool nand_reduce() const { return
_AP_ROOT_op_reduce(nand, Base::V); }
1242 INLINE bool or_reduce() const { return
_AP_ROOT_op_reduce(or, Base::V); }
1243 INLINE bool nor_reduce() const { return !
(_AP_ROOT_op_reduce(or, Base::V)); }
1244 INLINE bool xor_reduce() const { return _AP_ROOT_op_reduce
(xor, Base::V); }
1245 INLINE bool xnor_reduce() const {
1246     return !(_AP_ROOT_op_reduce(xor, Base::V));
1247 }
1248
1249 /* Output as a string.
1250 *
-----
1251 */
1252 #ifndef __SYNTHESIS__
1253 std::string to_string(signed char rd = 2, bool sign =
_AP_S) const {
1254     // XXX in autosim/autowrap.tcl "($
{name}).to_string(2).c_str()" is used to
1255     // initialize sc_lv, which seems incapable of handling
format "-0b".
1256     if (rd == 2) sign = false;
1257     return (Base::V).to_string(rd, sign);
1258 }
1259 #else
1260 INLINE char* to_string(signed char rd = 2, bool sign =
_AP_S) const {
1261     return 0;
1262 }
1263 #endif
1264 }; // struct ap_int_base
1265
1266 // XXX apcc cannot handle global std::ios_base::Init()
brought in by <iostream>
1267 #ifndef AP_AUTOCC
1268 #ifndef __SYNTHESIS__
1269 template <int _AP_W, bool _AP_S>
1270 INLINE std::ostream& operator<<(std::ostream& os,
const ap_int_base<_AP_W,
_AP_S>& x) {
1271     std::ios_base::fmtflags ff = std::cout.flags();
1272     if (ff & std::cout.hex) {
1273         os << x.to_string(16); // don't print sign
1274     } else if (ff & std::cout.oct) {
1275         os << x.to_string(8); // don't print sign
1276     } else {
1277         os << x.to_string(10);
1278     }
1279     return os;
1280 }
1281 #endif // ifndef __SYNTHESIS__
1282
1283 #ifndef __SYNTHESIS__
1284 template <int _AP_W, bool _AP_S>
1285 INLINE std::istream& operator>>(std::istream& in,
ap_int_base<_AP_W, _AP_S>&
op) {
1286     std::string str;
1287     in >> str;
1288     const std::ios_base::fmtflags basefield = in.flags() &
std::ios_base::basefield;
1289     unsigned radix = (basefield == std::ios_base::dec) ? 0 : (
(basefield == std::ios_base::oct) ? 8 :
1290     (basefield == std::ios_base::hex) ?
16 : 0);
1291     op = ap_int_base<_AP_W, _AP_S>(str.c_str(), radix);

```

```

1295     return in;
1296 }
1297 #endif // ifndef __SYNTHESIS__
1298 #endif // ifndef AP_AUTOCC
1299
1300 /* Operators with another ap_int_base.
1301 *
-----
1302 */
1303 #define OP_BIN_AP(Sym,
Rty)
1304 template <int _AP_W, bool _AP_S, int _AP_W2, bool
_AP_S2>
1305 IN-
LINE
1306     typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W2, _AP_S2>::Rty \
1307     operator Sym(const ap_int_base<_AP_W, _AP_S>&
op,
1308     const ap_int_base<_AP_W2, _AP_S2>& op2)
{
1309     typename ap_int_base<_AP_W, _AP_S>::template
RType<
1310         _AP_W2, _AP_S2>::Rty##_base \
1311     typename ap_int_base<_AP_W, _AP_S>::template
RType<
1312         _AP_W2, _AP_S2>::Rty##_base \
1313     rhs(op2);
1314     typename ap_int_base<_AP_W, _AP_S>::template
RType<
1315         _AP_W2, _AP_S2>::Rty##_base \
1316     ret;
1317     ret.V = lhs.V Sym
1318     return
1319     ret;
1320 }
1321 OP_BIN_AP(*, mult)
1322 OP_BIN_AP(+, plus)
1323 OP_BIN_AP(-, minus)
1324 OP_BIN_AP(&, logic)
1325 OP_BIN_AP(|, logic)
1326 OP_BIN_AP(^, logic)
1327
1328 #define OP_BIN_AP2(Sym,
Rty)
1329 template <int _AP_W, bool _AP_S, int _AP_W2, bool
_AP_S2>
1330 IN-
LINE
1331     typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W2, _AP_S2>::Rty \
1332     operator Sym(const ap_int_base<_AP_W, _AP_S>&
op,
1333     const ap_int_base<_AP_W2, _AP_S2>& op2)
{
1334     typename ap_int_base<_AP_W, _AP_S>::template
RType<
1335         _AP_W2, _AP_S2>::Rty##_base \
1336     ret;
1337     ret.V = op.V Sym
1338     return
1339     ret;
1340 }
1341 OP_BIN_AP2(/, div)
1342 OP_BIN_AP2(%, mod)
1343
1344 // shift operators are defined inside class.
1345 // compound assignment operators are defined inside class.
1346
1347 /* Operators with a pointer type.
1348 *
-----
1349 * char a[100];
1350 * char* ptr = a;
1351 * ap_int<2> n = 3;
1352 * char* ptr2 = ptr + n*2;
1353 * avoid ambiguous errors.
1354 */
1355 #define
OP_BIN_WITH_PTR(BIN_OP)
1356 template <typename PTR_TYPE, int _AP_W, bool
_AP_S>
1357 IN-
LINE PTR_TYPE* operator BIN_OP(PTR_TYPE*
i_op,
1358     const ap_int_base<_AP_W,
_AP_S>& op) { \
1359     ap_slong op2 = op.to_int64(); /* Not all implementation
*/
1360     return i_op BIN_OP
op2;

```



```

1358 }
1359 template <typename PTR_TYPE, int _AP_W, bool
_AP_S>
1360 inline PTR_TYPE* operator BIN_OP(const ap_int_base<_AP_W,
_AP_S>& op, \
1361                                PTR_TYPE* i_op)
{
1362     ap_slong op2 = op.to_int64(); /* Not all implementation
*/
1363     return op2 BIN_OP
i_op;
1364 }
1365 OP_BIN_WITH_PTR(+)
1366 OP_BIN_WITH_PTR(-)
1367
1368 /* Operators with a native floating point types.
1370 *
-----
1371 */
1372 // float OP ap_int
1373 // when ap_int<w>'s width > 64, then trunc ap_int<w> to
ap_int<64>
1374 #define OP_BIN_WITH_FLOAT(BIN_OP,
C_TYPE)
1375 template <int _AP_W, bool
_AP_S>
1376 inline C_TYPE operator BIN_OP(C_TYPE
i_op, \
1377                                const ap_int_base<_AP_W,
_AP_S>& op) { \
1378     typename ap_int_base<_AP_W, _AP_S>::RetType op2 =
op;
1379     return i_op BIN_OP
op2;
1380 }
1381 template <int _AP_W, bool
_AP_S>
1382 inline C_TYPE operator BIN_OP(const ap_int_base<_AP_W,
_AP_S>& op, \
1383                                C_TYPE i_op)
{
1384     typename ap_int_base<_AP_W, _AP_S>::RetType op2 =
op;
1385     return op2 BIN_OP
i_op;
1386 }
1387
1388 #define ALL_OP_WITH_FLOAT(C_TYPE) \
1389 OP_BIN_WITH_FLOAT(*, C_TYPE) \
1390 OP_BIN_WITH_FLOAT(/, C_TYPE) \
1391 OP_BIN_WITH_FLOAT(+, C_TYPE) \
1392 OP_BIN_WITH_FLOAT(-, C_TYPE)
1393
1394 #if _AP_ENABLE_HALF == 1
1395 ALL_OP_WITH_FLOAT(half)
1396 #endif
1397 ALL_OP_WITH_FLOAT(float)
1398 ALL_OP_WITH_FLOAT(double)
1399
1400 // TODO no shift?
1401
1402 /* Operators with a native integral types.
1403 *
-----
1404 */
1405 // arithmetic and bitwise operators.
1406 #define OP_BIN_WITH_INT(BIN_OP, C_TYPE, _AP_W2, _AP_S2,
RType)
1407 template <int _AP_W, bool
_AP_S>
1408 inline typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W2, \
1409 _AP_S2>::RType \
1410 operator BIN_OP(C_TYPE i_op, const ap_int_base<_AP_W,
_AP_S>& op) { \
1411     return ap_int_base<_AP_W2, _AP_S2>(i_op)
BIN_OP(op);
1412 }
1413 template <int _AP_W, bool
_AP_S>
1414 inline typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W2, \
1415 _AP_S2>::RType \
1416 operator BIN_OP(const ap_int_base<_AP_W, _AP_S>& op, C_TYPE
i_op) { \
1417     return op BIN_OP ap_int_base<_AP_W2,
_AP_S2>(i_op);
1418 }
1419
1420 #define ALL_OP_BIN_WITH_INT(C_TYPE, _AP_W2, _AP_S2) \
1421 OP_BIN_WITH_INT(*, C_TYPE, _AP_W2, _AP_S2, mult) \

```

```

1422 OP_BIN_WITH_INT(+, C_TYPE, _AP_W2, _AP_S2, plus) \
1423 OP_BIN_WITH_INT(-, C_TYPE, _AP_W2, _AP_S2, minus) \
1424 OP_BIN_WITH_INT(/, C_TYPE, _AP_W2, _AP_S2, div) \
1425 OP_BIN_WITH_INT(% , C_TYPE, _AP_W2, _AP_S2, mod) \
1426 OP_BIN_WITH_INT(&, C_TYPE, _AP_W2, _AP_S2, logic) \
1427 OP_BIN_WITH_INT(|, C_TYPE, _AP_W2, _AP_S2, logic) \
1428 OP_BIN_WITH_INT(^, C_TYPE, _AP_W2, _AP_S2, logic) \
1429
1430 ALL_OP_BIN_WITH_INT(bool, 1, false)
1431 ALL_OP_BIN_WITH_INT(char, 8, CHAR_IS_SIGNED)
1432 ALL_OP_BIN_WITH_INT(signed char, 8, true)
1433 ALL_OP_BIN_WITH_INT(unsigned char, 8, false)
1434 ALL_OP_BIN_WITH_INT(short, _AP_SIZE_short, true)
1435 ALL_OP_BIN_WITH_INT(unsigned short, _AP_SIZE_short, false)
1436 ALL_OP_BIN_WITH_INT(int, _AP_SIZE_int, true)
1437 ALL_OP_BIN_WITH_INT(unsigned int, _AP_SIZE_int, false)
1438 ALL_OP_BIN_WITH_INT(long, _AP_SIZE_long, true)
1439 ALL_OP_BIN_WITH_INT(unsigned long, _AP_SIZE_long, false)
1440 ALL_OP_BIN_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
1441 ALL_OP_BIN_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
1442
1443 #undef OP_BIN_WITH_INT
1444 #undef ALL_OP_BIN_WITH_INT
1445
1446 // shift operators.
1447 #define ALL_OP_SHIFT_WITH_INT(C_TYPE, _AP_W2, _AP_S2) \
1448 template <int _AP_W, bool _AP_S> \
1449 inline typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W, _AP_S>::arg1 operator<<(\
1450     const ap_int_base<_AP_W, _AP_S>& op, C_TYPE op2) { \
1451     ap_int_base<_AP_W, _AP_S> r; \
1452     if (_AP_S2) \
1453         r.V = op2 >= 0 ? (op.V << op2) : (op.V >> (-op2)); \
1454     else \
1455         r.V = op.V << op2; \
1456     return r; \
1457 }
1458 template <int _AP_W, bool _AP_S> \
1459 inline typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W, _AP_S>::arg1 operator>>(\
1460     const ap_int_base<_AP_W, _AP_S>& op, C_TYPE op2) { \
1461     ap_int_base<_AP_W, _AP_S> r; \
1462     if (_AP_S2) \
1463         r.V = op2 >= 0 ? (op.V >> op2) : (op.V << (-op2)); \
1464     else \
1465         r.V = op.V >> op2; \
1466     return r; \
1467 }
1468
1469 ALL_OP_SHIFT_WITH_INT(char, 8, CHAR_IS_SIGNED)
1470 ALL_OP_SHIFT_WITH_INT(signed char, 8, true)
1471 ALL_OP_SHIFT_WITH_INT(short, _AP_SIZE_short, true)
1472 ALL_OP_SHIFT_WITH_INT(int, _AP_SIZE_int, true)
1473 ALL_OP_SHIFT_WITH_INT(long, _AP_SIZE_long, true)
1474 ALL_OP_SHIFT_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
1475
1476 #undef ALL_OP_SHIFT_WITH_INT
1477
1478 #define ALL_OP_SHIFT_WITH_INT(C_TYPE, _AP_W2, _AP_S2) \
1479 template <int _AP_W, bool _AP_S> \
1480 inline typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W, _AP_S>::arg1 operator<<(\
1481     const ap_int_base<_AP_W, _AP_S>& op, C_TYPE op2) { \
1482     ap_int_base<_AP_W, _AP_S> r; \
1483     r.V = op.V << op2; \
1484     return r; \
1485 }
1486 template <int _AP_W, bool _AP_S> \
1487 inline typename ap_int_base<_AP_W, _AP_S>::template
RType<_AP_W, _AP_S>::arg1 operator>>(\
1488     const ap_int_base<_AP_W, _AP_S>& op, C_TYPE op2) { \
1489     ap_int_base<_AP_W, _AP_S> r; \
1490     r.V = op.V >> op2; \
1491     return r; \
1492 }
1493 ALL_OP_SHIFT_WITH_INT(bool, 1, false)
1494 ALL_OP_SHIFT_WITH_INT(unsigned char, 8, false)
1495 ALL_OP_SHIFT_WITH_INT(unsigned short, _AP_SIZE_short, false)
1496 ALL_OP_SHIFT_WITH_INT(unsigned int, _AP_SIZE_int, false)
1497 ALL_OP_SHIFT_WITH_INT(unsigned long, _AP_SIZE_long, false)
1498 ALL_OP_SHIFT_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
1499
1500 #undef ALL_OP_SHIFT_WITH_INT
1501
1502 // compound assign operators.
1503 #define OP_ASSIGN_WITH_INT(ASSIGN_OP, C_TYPE, _AP_W2,
_AP_S2) \
1504 template <int _AP_W, bool
_AP_S> \
1505 inline ap_int_base<_AP_W, _AP_S>& operator AS-
SIGN_OP(\
1506     ap_int_base<_AP_W, _AP_S>& op, C_TYPE op2) \
1507 { \
1508     return op ASSIGN_OP ap_int_base<_AP_W2,
_AP_S2>(op2); \
1509 }
1510 // TODO int a; ap_int<16> b; a += b;
1511
1512 #define ALL_OP_ASSIGN_WITH_INT(C_TYPE, _AP_W2, _AP_S2) \

```

```

1513 OP_ASSIGN_WITH_INT(+, C_TYPE, _AP_W2, _AP_S2) \
1514 OP_ASSIGN_WITH_INT(-, C_TYPE, _AP_W2, _AP_S2) \
1515 OP_ASSIGN_WITH_INT(*, C_TYPE, _AP_W2, _AP_S2) \
1516 OP_ASSIGN_WITH_INT(/, C_TYPE, _AP_W2, _AP_S2) \
1517 OP_ASSIGN_WITH_INT(%=, C_TYPE, _AP_W2, _AP_S2) \
1518 OP_ASSIGN_WITH_INT(&=, C_TYPE, _AP_W2, _AP_S2) \
1519 OP_ASSIGN_WITH_INT(|=, C_TYPE, _AP_W2, _AP_S2) \
1520 OP_ASSIGN_WITH_INT(^=, C_TYPE, _AP_W2, _AP_S2) \
1521 OP_ASSIGN_WITH_INT(>=, C_TYPE, _AP_W2, _AP_S2) \
1522 OP_ASSIGN_WITH_INT(<=, C_TYPE, _AP_W2, _AP_S2) \
1523
1524 ALL_OP_ASSIGN_WITH_INT(bool, 1, false)
1525 ALL_OP_ASSIGN_WITH_INT(char, 8, CHAR_IS_SIGNED)
1526 ALL_OP_ASSIGN_WITH_INT(signed char, 8, true)
1527 ALL_OP_ASSIGN_WITH_INT(unsigned char, 8, false)
1528 ALL_OP_ASSIGN_WITH_INT(short, _AP_SIZE_short, true)
1529 ALL_OP_ASSIGN_WITH_INT(unsigned short, _AP_SIZE_short, false)
1530 ALL_OP_ASSIGN_WITH_INT(int, _AP_SIZE_int, true)
1531 ALL_OP_ASSIGN_WITH_INT(unsigned int, _AP_SIZE_int, false)
1532 ALL_OP_ASSIGN_WITH_INT(long, _AP_SIZE_long, true)
1533 ALL_OP_ASSIGN_WITH_INT(unsigned long, _AP_SIZE_long, false)
1534 ALL_OP_ASSIGN_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
1535 ALL_OP_ASSIGN_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
1536
1537 #undef OP_ASSIGN_WITH_INT
1538 #undef ALL_OP_ASSIGN_WITH_INT
1539
1540 // equality and relational operators.
1541 #define OP_REL_WITH_INT(REL_OP, C_TYPE, _AP_W2,
_AP_S2) \
1542     template <int _AP_W, bool \
1543     _AP_S> \
1544     inline bool operator REL_OP(C_TYPE \
1545     const ap_int_base<_AP_W, \
1546     _AP_S>& op) { \
1547         return ap_int_base<_AP_W2, _AP_S2>(i_op) REL_OP \
1548         op; \
1549     } \
1550     template <int _AP_W, bool \
1551     _AP_S> \
1552     inline bool operator REL_OP(const ap_int_base<_AP_W, \
1553     _AP_S>& op, \
1554     C_TYPE op2) \
1555     { \
1556         return op REL_OP ap_int_base<_AP_W2, \
1557         _AP_S2>(op2); \
1558     } \
1559     #define ALL_OP_REL_WITH_INT(C_TYPE, _AP_W2, _AP_S2) \
1560     OP_REL_WITH_INT(>, C_TYPE, _AP_W2, _AP_S2) \
1561     OP_REL_WITH_INT(<, C_TYPE, _AP_W2, _AP_S2) \
1562     OP_REL_WITH_INT(>=, C_TYPE, _AP_W2, _AP_S2) \
1563     OP_REL_WITH_INT(<=, C_TYPE, _AP_W2, _AP_S2) \
1564     OP_REL_WITH_INT(==, C_TYPE, _AP_W2, _AP_S2) \
1565     OP_REL_WITH_INT(!=, C_TYPE, _AP_W2, _AP_S2) \
1566
1567 ALL_OP_REL_WITH_INT(bool, 1, false)
1568 ALL_OP_REL_WITH_INT(char, 8, CHAR_IS_SIGNED)
1569 ALL_OP_REL_WITH_INT(signed char, 8, true)
1570 ALL_OP_REL_WITH_INT(unsigned char, 8, false)
1571 ALL_OP_REL_WITH_INT(short, _AP_SIZE_short, true)
1572 ALL_OP_REL_WITH_INT(unsigned short, _AP_SIZE_short, false)
1573 ALL_OP_REL_WITH_INT(int, _AP_SIZE_int, true)
1574 ALL_OP_REL_WITH_INT(unsigned int, _AP_SIZE_int, false)
1575 ALL_OP_REL_WITH_INT(long, _AP_SIZE_long, true)
1576 ALL_OP_REL_WITH_INT(unsigned long, _AP_SIZE_long, false)
1577 ALL_OP_REL_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
1578 ALL_OP_REL_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
1579
1580 #undef OP_REL_WITH_INT
1581 #undef ALL_OP_REL_WITH_INT
1582
1583 #define \
1584 OP_REL_WITH_DOUBLE_OR_FLOAT(Sym) \
1585     template <int _AP_W, bool \
1586     _AP_S> \
1587     inline bool operator Sym(const ap_int_base<_AP_W, _AP_S>& \
1588     op1, \
1589     double op2) \
1590     { \
1591         return op1.to_double() Sym \
1592         op2; \
1593     } \
1594     template <int _AP_W, bool \
1595     _AP_S> \
1596     inline bool operator Sym(double \
1597     op1, \
1598     const ap_int_base<_AP_W, _AP_S>& \
1599     op2) { \
1600         return op1 Sym \
1601         op2.to_double(); \
1602     } \
1603     template <int _AP_W, bool

```

```

_AP_S> \
1589     inline bool operator Sym(const ap_int_base<_AP_W, _AP_S>& \
1590     op1, \
1591     float op2) \
1592     { \
1593         return op1.to_double() Sym \
1594         op2; \
1595     } \
1596     template <int _AP_W, bool \
1597     _AP_S> \
1598     inline bool operator Sym(float \
1599     op1, \
1600     const ap_int_base<_AP_W, _AP_S>& \
1601     op2) { \
1602         return op1 Sym \
1603         op2.to_double(); \
1604     } \
1605     OP_REL_WITH_DOUBLE_OR_FLOAT(>) \
1606     OP_REL_WITH_DOUBLE_OR_FLOAT(<) \
1607     OP_REL_WITH_DOUBLE_OR_FLOAT(>=) \
1608     OP_REL_WITH_DOUBLE_OR_FLOAT(<=) \
1609     OP_REL_WITH_DOUBLE_OR_FLOAT(==) \
1610     OP_REL_WITH_DOUBLE_OR_FLOAT(!=) \
1611
1612 #undef OP_REL_WITH_DOUBLE_OR_FLOAT
1613
1614 /* Operators with ap_bit_ref.
1615 *
1616 -----
1617 */
1618 // arithmetic, bitwise and shift operators.
1619 #define OP_BIN_WITH_RANGE(BIN_OP, \
RTYPE) \
1620     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool \
1621     _AP_S2> \
1622     inline typename ap_int_base<_AP_W1, _AP_S1>::template \
1623     RType<_AP_W2, \
1624     _AP_S2>::RTYPE \
1625     operator BIN_OP(const ap_range_ref<_AP_W1, _AP_S1>& \
1626     op1, \
1627     const ap_int_base<_AP_W2, _AP_S2>& op2) \
1628     { \
1629         return ap_int_base<_AP_W1, false>(op1) BIN_OP \
1630         op2; \
1631     } \
1632     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool \
1633     _AP_S2> \
1634     inline typename ap_int_base<_AP_W1, _AP_S1>::template \
1635     RType<_AP_W2, \
1636     _AP_S2>::RTYPE \
1637     operator BIN_OP(const ap_int_base<_AP_W1, _AP_S1>& \
1638     op1, \
1639     const ap_range_ref<_AP_W2, _AP_S2>& op2) \
1640     { \
1641         return op1 BIN_OP ap_int_base<_AP_W2, \
1642         false>(op2); \
1643     } \
1644     OP_BIN_WITH_RANGE(+, plus) \
1645     OP_BIN_WITH_RANGE(-, minus) \
1646     OP_BIN_WITH_RANGE(*, mult) \
1647     OP_BIN_WITH_RANGE(/, div) \
1648     OP_BIN_WITH_RANGE(%=, mod) \
1649     OP_BIN_WITH_RANGE(&, logic) \
1650     OP_BIN_WITH_RANGE(|, logic) \
1651     OP_BIN_WITH_RANGE(^, logic) \
1652     OP_BIN_WITH_RANGE(>>, arg1) \
1653     OP_BIN_WITH_RANGE(<<, arg1) \
1654
1655 #undef OP_BIN_WITH_RANGE
1656
1657 // compound assignment operators.
1658 #define \
1659 OP_ASSIGN_WITH_RANGE(ASSIGN_OP) \
1660     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool \
1661     _AP_S2> \
1662     inline ap_int_base<_AP_W1, _AP_S1>& operator AS- \
1663     SIGN_OP( \
1664     ap_int_base<_AP_W1, _AP_S1>& op1, ap_range_ref<_AP_W2, \
1665     _AP_S2>& op2) { \
1666         return op1 ASSIGN_OP ap_int_base<_AP_W2, \
1667         false>(op2); \
1668     } \
1669     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool \
1670     _AP_S2> \
1671     inline ap_range_ref<_AP_W1, _AP_S1>& operator AS- \
1672     SIGN_OP( \
1673     ap_range_ref<_AP_W1, _AP_S1>& op1, ap_int_base<_AP_W2,

```

```

_AP_S2>& op2) { \
1651     ap_int_base<_AP_W1, false>
tmp(op1); \
1652     tmp ASSIGN_OP
op2; \
1653     op1 =
tmp; \
1654     return
op1; \
1655 }
1656
1657 OP_ASSIGN_WITH_RANGE(+=)
1658 OP_ASSIGN_WITH_RANGE(=)
1659 OP_ASSIGN_WITH_RANGE(*=)
1660 OP_ASSIGN_WITH_RANGE(/=)
1661 OP_ASSIGN_WITH_RANGE(%=)
1662 OP_ASSIGN_WITH_RANGE(&=)
1663 OP_ASSIGN_WITH_RANGE(|=)
1664 OP_ASSIGN_WITH_RANGE(^=)
1665 OP_ASSIGN_WITH_RANGE(>=)
1666 OP_ASSIGN_WITH_RANGE(<=)
1667
1668 #undef OP_ASSIGN_WITH_RANGE
1669
1670 // equality and relational operators
1671 #define
OP_REL_WITH_RANGE(REL_OP)
\
1672     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1673     inline bool operator REL_OP(const ap_range_ref<_AP_W1,
_AP_S1>& op1, \
1674                                const ap_int_base<_AP_W2,
_AP_S2>& op2) { \
1675         return ap_int_base<_AP_W1, false>(op1).operator
REL_OP(op2);
1676     }
\
1677     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1678     inline bool operator REL_OP(const ap_int_base<_AP_W1,
_AP_S1>& op1, \
1679                                const ap_range_ref<_AP_W2,
_AP_S2>& op2) { \
1680         return op1.operator REL_OP(op2.operator
ap_int_base<_AP_W2, false>()); \
1681     }
1682
1683 OP_REL_WITH_RANGE(==)
1684 OP_REL_WITH_RANGE(!=)
1685 OP_REL_WITH_RANGE(>)
1686 OP_REL_WITH_RANGE(<=)
1687 OP_REL_WITH_RANGE(<)
1688 OP_REL_WITH_RANGE(<=)
1689
1690 #undef OP_REL_WITH_RANGE
1691
1692 /* Operators with ap_bit_ref.
1693 *
-----
1694 */
1695 // arithmetic, bitwise and shift operators.
1696 #define OP_BIN_WITH_BIT(BIN_OP,
RTYPE) \
1697     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1698     inline typename ap_int_base<_AP_W1, _AP_S1>::template
RTYPE<1, false>::RTYPE \
1699     operator BIN_OP(const ap_int_base<_AP_W1, _AP_S1>&
op1, \
1700                    const ap_bit_ref<_AP_W2, _AP_S2>& op2)
{ \
1701         return op1 BIN_OP ap_int_base<1, \
false>(op2);
1702     }
\
1703     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1704     inline typename ap_int_base<1, false>::template
RTYPE<_AP_W2, _AP_S2>::RTYPE \
1705     operator BIN_OP(const ap_bit_ref<_AP_W1, _AP_S1>&
op1, \
1706                    const ap_int_base<_AP_W2, _AP_S2>& op2)
{ \
1707         return ap_int_base<1, false>(op1) BIN_OP
op2;
1708     }
1709
1710 OP_BIN_WITH_BIT(+, plus)
1711 OP_BIN_WITH_BIT(-, minus)
1712 OP_BIN_WITH_BIT(*, mult)
1713 OP_BIN_WITH_BIT(/, div)
1714 OP_BIN_WITH_BIT(% , mod)
1715 OP_BIN_WITH_BIT(&, logic)
1716 OP_BIN_WITH_BIT(|, logic)
1717 OP_BIN_WITH_BIT(^, logic)
1718 OP_BIN_WITH_BIT(>, arg1)

```

```

1719 OP_BIN_WITH_BIT(<=, arg1)
1720
1721 #undef OP_BIN_WITH_BIT
1722
1723 // compound assignment operators.
1724 #define
OP_ASSIGN_WITH_BIT(ASSIGN_OP)
\
1725     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1726     inline ap_int_base<_AP_W1, _AP_S1>& operator AS-
SIGN_OP(
1727         ap_int_base<_AP_W1, _AP_S1>& op1, ap_bit_ref<_AP_W2,
_AP_S2>& op2) { \
1728         return op1 ASSIGN_OP ap_int_base<1,
false>(op2);
1729     }
\
1730     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1731     inline ap_bit_ref<_AP_W1, _AP_S1>& operator AS-
SIGN_OP(
1732         ap_bit_ref<_AP_W1, _AP_S1>& op1, ap_int_base<_AP_W2,
_AP_S2>& op2) { \
1733         ap_int_base<1, false>
tmp(op1); \
1734         tmp ASSIGN_OP
op2; \
1735         op1 =
tmp; \
1736         return
op1; \
1737     }
1738
1739 OP_ASSIGN_WITH_BIT(+=)
1740 OP_ASSIGN_WITH_BIT(=)
1741 OP_ASSIGN_WITH_BIT(*=)
1742 OP_ASSIGN_WITH_BIT(/=)
1743 OP_ASSIGN_WITH_BIT(%=)
1744 OP_ASSIGN_WITH_BIT(&=)
1745 OP_ASSIGN_WITH_BIT(|=)
1746 OP_ASSIGN_WITH_BIT(^=)
1747 OP_ASSIGN_WITH_BIT(>=)
1748 OP_ASSIGN_WITH_BIT(<=)
1749
1750 #undef OP_ASSIGN_WITH_BIT
1751
1752 // equality and relational operators.
1753 #define
OP_REL_WITH_BIT(REL_OP)
\
1754     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1755     inline bool operator REL_OP(const ap_int_base<_AP_W1,
_AP_S1>& op1, \
1756                                const ap_bit_ref<_AP_W2,
_AP_S2>& op2) { \
1757         return op1 REL_OP ap_int_base<1,
false>(op2);
1758     }
\
1759     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
1760     inline bool operator REL_OP(const ap_bit_ref<_AP_W1,
_AP_S1>& op1, \
1761                                const ap_int_base<_AP_W2,
_AP_S2>& op2) { \
1762         return ap_int_base<1, false>(op1) REL_OP
op2;
1763     }
1764
1765 OP_REL_WITH_BIT(==)
1766 OP_REL_WITH_BIT(!=)
1767 OP_REL_WITH_BIT(>)
1768 OP_REL_WITH_BIT(>=)
1769 OP_REL_WITH_BIT(<)
1770 OP_REL_WITH_BIT(<=)
1771
1772 #undef OP_REL_WITH_BIT
1773
1774
1775 /* Operators with ap_concat_ref.
1776 *
-----
1777 */
1778 // arithmetic, bitwise and shift operators.
1779 // bitwise operators are defined in struct.
1780 // TODO specify whether to define arithmetic and bitwise op-
erators.
1781 #if 0
1782 #define OP_BIN_WITH_CONCAT(BIN_OP,
RTYPE) \
\
1783     template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2, \
1784                int _AP_W3, bool
_AP_S3>
1785     inline typename ap_int_base<_AP_W3, _AP_S3>::template
RTYPE<_AP_W1 + _AP_W2, \

```



```

1786 false>::RTYPE \
1787 operator BIN_OP(const ap_int_base<_AP_W3, _AP_S3>&
op1, \
1788 const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2,
_AP_T2>& op2) { \
1789 /* convert ap_concat_ref to ap_int_base
*/
1790 return op1 BIN_OP
op2.get(); \
1791 } \
1792 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2, \
1793 int _AP_W3, bool
_AP_S3> \
1794 INLINE typename ap_int_base<_AP_W1 +
_AP_W2, \
1795 false>::template RType<_AP_W3,
_AP_S3>::RTYPE \
1796 operator BIN_OP(const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2,
_AP_T2>& op1, \
1797 const ap_int_base<_AP_W3, _AP_S3>& op2)
{ \
1798 /* convert ap_concat_ref to ap_int_base
*/
1799 return op1.get() BIN_OP
op2; \
1800 } \
1801 OP_BIN_WITH_CONCAT(+, plus)
1802 OP_BIN_WITH_CONCAT(-, minus)
1803 OP_BIN_WITH_CONCAT(*, mult)
1804 OP_BIN_WITH_CONCAT(/, div)
1805 OP_BIN_WITH_CONCAT(% , mod)
1806 OP_BIN_WITH_CONCAT(&, logic)
1807 OP_BIN_WITH_CONCAT(|, logic)
1808 OP_BIN_WITH_CONCAT(^, logic)
1809 OP_BIN_WITH_CONCAT(>>, arg1)
1810 OP_BIN_WITH_CONCAT(<<, arg1)
1811 OP_BIN_WITH_CONCAT(<=, arg1)
1812
1813 #undef OP_BIN_WITH_CONCAT
1814
1815 // compound assignment operators.
1816 #define
OP_ASSIGN_WITH_CONCAT(ASSIGN_OP) \
1817 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2, \
1818 int _AP_W3, bool
_AP_S3> \
1819 INLINE typename ap_int_base<_AP_W3, _AP_S3>::template
RType<_AP_W1 + _AP_W2, \
1820 false>::RTYPE \
1821 operator AS-
SIGN_OP( \
1822 const ap_int_base<_AP_W3, _AP_S3>&
op1, \
1823 const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>&
op2) { \
1824 /* convert ap_concat_ref to ap_int_base
*/
1825 return op1 ASSIGN_OP
op2.get(); \
1826 } \
1827 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2, \
1828 int _AP_W3, bool
_AP_S3> \
1829 INLINE typename ap_int_base<_AP_W1 +
_AP_W2, \
1830 false>::template RType<_AP_W3,
_AP_S3>::RTYPE \
1831 operator ASSIGN_OP(const ap_concat_ref<_AP_W1, _AP_T1,
_AP_W2, _AP_T2>& op1, \
1832 const ap_int_base<_AP_W3, _AP_S3>& op2)
{ \
1833 /* convert ap_concat_ref to ap_int_base
*/
1834 ap_int_base<_AP_W1 + _AP_W2, false> tmp =
op1.get(); \
1835 tmp ASSIGN_OP
op2; \
1836 op1 =
tmp; \
1837 return
op1; \

```

```

1838 }
1839
1840 OP_ASSIGN_WITH_CONCAT(+=)
1841 OP_ASSIGN_WITH_CONCAT(-=)
1842 OP_ASSIGN_WITH_CONCAT(*=)
1843 OP_ASSIGN_WITH_CONCAT(/=)
1844 OP_ASSIGN_WITH_CONCAT(%=)
1845 OP_ASSIGN_WITH_CONCAT(&=)
1846 OP_ASSIGN_WITH_CONCAT(|=)
1847 OP_ASSIGN_WITH_CONCAT(^=)
1848 OP_ASSIGN_WITH_CONCAT(>>=)
1849 OP_ASSIGN_WITH_CONCAT(<<=)
1850
1851 #undef OP_ASSIGN_WITH_CONCAT
1852 #endif
1853
1854 // equality and relational operators.
1855 #define
OP_REL_WITH_CONCAT(REL_OP) \
1856 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2, \
1857 int _AP_W3, bool
_AP_S3> \
1858 INLINE bool operator
REL_OP( \
1859 const ap_int_base<_AP_W3, _AP_S3>&
op1, \
1860 const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>&
op2) { \
1861 /* convert ap_concat_ref to ap_int_base
*/
1862 return op1 REL_OP
op2.get(); \
1863 } \
1864 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2, \
1865 int _AP_W3, bool
_AP_S3> \
1866 INLINE bool operator
REL_OP( \
1867 const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>&
op1, \
1868 const ap_int_base<_AP_W3, _AP_S3>& op2)
{ \
1869 /* convert ap_concat_ref to ap_int_base
*/
1870 return op1.get() REL_OP
op2; \
1871 } \
1872 OP_REL_WITH_CONCAT(==)
1873 OP_REL_WITH_CONCAT(!=)
1874 OP_REL_WITH_CONCAT(>)
1875 OP_REL_WITH_CONCAT(>=)
1876 OP_REL_WITH_CONCAT(<)
1877 OP_REL_WITH_CONCAT(<=)
1878 OP_REL_WITH_CONCAT(<=)
1879
1880 #undef OP_REL_WITH_CONCAT
1881
1882 #endif // ifndef __cplusplus
1883 #endif // ifndef __AP_INT_BASE_H__
1884
1885 // -*- cpp -*-

```

E.10 vendor/ap_int_ref.h (1,346 lines, 55,218 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  * cense");
6  * you may not use this file except in compliance with the
7  * License.
8  * You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing,
13  * software
14  *
15  * distributed under the License is distributed on an "AS IS"
16  * BASIS,
17  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18  * press or implied.
19  * See the License for the specific language governing per-
20  * missions and
21  * limitations under the License.
22  */
23 #ifndef __AP_INT_REF_H__
24 #define __AP_INT_REF_H__
25
26 #ifndef __AP_INT_H__
27 #error "Only ap_fixed.h and ap_int.h can be included directly in

```

```

user code."
22 #endif
23
24 #ifndef __cplusplus
25 #error "C++ is required to include this header file"
26
27 #else
28
29 #ifndef __SYNTHESIS__
30 #include <iostream>
31 #endif
32
33 /* Concatination reference.
34
-----
35 */
36 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2>
37 struct ap_concat_ref {
38     enum {
39         _AP_WR = _AP_W1 + _AP_W2,
40     };
41     _AP_T1& mbv1;
42     _AP_T2& mbv2;
43
44     INLINE ap_concat_ref(const ap_concat_ref<_AP_W1, _AP_T1,
_AP_W2, _AP_T2>& ref)
45         : mbv1(ref.mbv1), mbv2(ref.mbv2) {}
46
47
48     INLINE ap_concat_ref(_AP_T1& bv1, _AP_T2& bv2) : mbv1(bv1),
mbv2(bv2) {}
49
50     template <int _AP_W3, bool _AP_S3>
51     INLINE ap_concat_ref& operator=(const ap_int_base<_AP_W3,
_AP_S3>& val) {
52         ap_int_base<_AP_W1 + _AP_W2, false> vval(val);
53         int W_ref1 = mbv1.length();
54         int W_ref2 = mbv2.length();
55         ap_int_base<_AP_W1, false> Part1;
56         Part1.V = _AP_ROOT_op_get_range(vval.V, W_ref2, W_ref1 +
W_ref2 - 1);
57         mbv1.set(Part1);
58         ap_int_base<_AP_W2, false> Part2;
59         Part2.V = _AP_ROOT_op_get_range(vval.V, 0, W_ref2 - 1);
60         mbv2.set(Part2);
61         return *this;
62     }
63
64     // assign op from hls supported C integral types.
65     // FIXME disabled to support legacy code directly assign
from sc_signal<T>
66     //template <typename T>
67     //INLINE typename
_ap_type::enable_if<_ap_type::is_integral<T>::value,
68     //                                ap_concat_ref&::type
69     //operator=(T val) {
70     //    ap_int_base<_AP_W1 + _AP_W2, false> tmpVal(val);
71     //    return operator=(tmpVal);
72     //}
73 #define ASSIGN_WITH_CTYPE(_Tp) \
74     INLINE ap_concat_ref& operator=(_Tp val) { \
75         ap_int_base<_AP_W1 + _AP_W2, false> tmpVal(val); \
76         return operator=(tmpVal); \
77     }
78
79     ASSIGN_WITH_CTYPE(bool)
80     ASSIGN_WITH_CTYPE(char)
81     ASSIGN_WITH_CTYPE(signed char)
82     ASSIGN_WITH_CTYPE(unsigned char)
83     ASSIGN_WITH_CTYPE(short)
84     ASSIGN_WITH_CTYPE(unsigned short)
85     ASSIGN_WITH_CTYPE(int)
86     ASSIGN_WITH_CTYPE(unsigned int)
87     ASSIGN_WITH_CTYPE(long)
88     ASSIGN_WITH_CTYPE(unsigned long)
89     ASSIGN_WITH_CTYPE(ap_ulong)
90     ASSIGN_WITH_CTYPE(ap_ulong)
91 #if _AP_ENABLE_HALF == 1
92     ASSIGN_WITH_CTYPE(half)
93 #endif
94     ASSIGN_WITH_CTYPE(float)
95     ASSIGN_WITH_CTYPE(double)
96
97 #undef ASSIGN_WITH_CTYPE
98
99 // Be explicit to prevent it from being deleted, as field
d_bv
100 // is of reference type.
101 INLINE ap_concat_ref& operator=(
102     const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>&
val) {
103     ap_int_base<_AP_W1 + _AP_W2, false> tmpVal(val);
104     return operator=(tmpVal);
105 }
106
107 template <int _AP_W3, typename _AP_T3, int _AP_W4, typename
_AP_T4>
108     INLINE ap_concat_ref& operator=(
109         const ap_concat_ref<_AP_W3, _AP_T3, _AP_W4, _AP_T4>&
val) {
110         ap_int_base<_AP_W1 + _AP_W2, false> tmpVal(val);
111         return operator=(tmpVal);
112     }
113
114     template <int _AP_W3, bool _AP_S3>
115     INLINE ap_concat_ref& operator=(const ap_bit_ref<_AP_W3,
_AP_S3>& val) {
116         ap_int_base<_AP_W1 + _AP_W2, false> tmpVal(val);
117         return operator=(tmpVal);
118     }
119
120     template <int _AP_W3, bool _AP_S3>
121     INLINE ap_concat_ref& operator=(const ap_range_ref<_AP_W3,
_AP_S3>& val) {
122         ap_int_base<_AP_W1 + _AP_W2, false> tmpVal(val);
123         return operator=(tmpVal);
124     }
125
126     template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
127         ap_o_mode _AP_O3, int _AP_N3>
128     INLINE ap_concat_ref& operator=(
129         const af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3>& val) {
130         return operator=((const ap_int_base<_AP_W3, false>)
(val));
131     }
132
133     template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
134         ap_o_mode _AP_O3, int _AP_N3>
135     INLINE ap_concat_ref& operator=(
136         const af_fixed_base<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3>&
137         val) {
138         return operator=(val.to_ap_int_base());
139     }
140
141     template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
142         ap_o_mode _AP_O3, int _AP_N3>
143     INLINE ap_concat_ref& operator=(
144         const af_bit_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3>& val) {
145         return operator=((ap_ulong)(bool)(val));
146     }
147
148     INLINE operator ap_int_base<_AP_WR, false>() const { return
get(); }
149
150     INLINE operator ap_ulong() const { return
get().to_uint64(); }
151
152     template <int _AP_W3, bool _AP_S3>
153     INLINE ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
154         ap_range_ref<_AP_W3, _AP_S3> >
155     operator,(const ap_range_ref<_AP_W3, _AP_S3> &a2) {
156         return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
157             ap_range_ref<_AP_W3, _AP_S3> >(&a2);
158     }
159
160     template <int _AP_W3, bool _AP_S3>
161     INLINE
162     ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
163         ap_int_base<_AP_W3, _AP_S3> >
164     operator,(ap_int_base<_AP_W3, _AP_S3> &a2) {
165         return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
166             ap_int_base<_AP_W3, _AP_S3> >(&a2);
167     }
168
169     template <int _AP_W3, bool _AP_S3>
170     INLINE
171     ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
172         ap_int_base<_AP_W3, _AP_S3> >
173     operator,(volatile ap_int_base<_AP_W3, _AP_S3> &a2) {
174         return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
175             ap_int_base<_AP_W3, _AP_S3> >(&a2);
176     }
177
178     template <int _AP_W3, bool _AP_S3>
179     INLINE
180     ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
181         ap_int_base<_AP_W3, _AP_S3> >
182     operator,(const ap_int_base<_AP_W3, _AP_S3> &a2) {
183         return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
184             ap_int_base<_AP_W3, _AP_S3> >(&a2);
185     }
186
187     template <int _AP_W3, bool _AP_S3>
188     INLINE
189     ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,

```

```

ap_int_base<_AP_W3, _AP_S3> >
189     operator,(const volatile ap_int_base<_AP_W3, _AP_S3>
&a2) {
190         // FIXME op's life does not seem long enough
191         ap_int_base<_AP_W3, _AP_S3> op(a2);
192         return ap_concat_ref<_AP_W3, ap_concat_ref, _AP_W3,
193             ap_int_base<_AP_W3, _AP_S3> >{
194             *this, const_cast<ap_int_base<_AP_W3, _AP_S3>&>(op));
195     }
196
197     template <int _AP_W3, bool _AP_S3>
198     INLINE ap_concat_ref<_AP_W3, ap_concat_ref, 1,
ap_bit_ref<_AP_W3, _AP_S3> >
199     operator,(const ap_bit_ref<_AP_W3, _AP_S3> &a2) {
200         return ap_concat_ref<_AP_W3, ap_concat_ref, 1,
ap_bit_ref<_AP_W3, _AP_S3> >{
201             *this, const_cast<ap_bit_ref<_AP_W3, _AP_S3>&>(a2));
202         }
203
204     template <int _AP_W3, typename _AP_T3, int _AP_W4, typename
_AP_T4>
205     INLINE ap_concat_ref<_AP_W3, ap_concat_ref, _AP_W3 +
_AP_W4,
206         ap_concat_ref<_AP_W3, _AP_T3, _AP_W4,
_AP_T4> >
207     operator,(const ap_concat_ref<_AP_W3, _AP_T3, _AP_W4,
_AP_T4> &a2) {
208         return ap_concat_ref<_AP_W3, ap_concat_ref, _AP_W3 +
_AP_W4,
209             ap_concat_ref<_AP_W3, _AP_T3,
_AP_W4, _AP_T4> >{
210             *this, const_cast<ap_concat_ref<_AP_W3, _AP_T3,
_AP_W4,
211             _AP_T4>&>(a2));
212         }
213     template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
214         ap_o_mode _AP_O3, int _AP_N3>
215     INLINE ap_concat_ref<
216         _AP_W3, ap_concat_ref, _AP_W3,
217         af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3, _AP_O3,
_AP_N3> >
218     operator,(
219         const af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3,
220         _AP_N3> &a2) {
221         return ap_concat_ref<
222             _AP_W3, ap_concat_ref, _AP_W3,
223             af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3, _AP_O3,
_AP_N3> >{
224             *this,
225             const_cast<
226             af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3,
227             _AP_N3>&>(a2));
228         }
229     template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
230         ap_o_mode _AP_O3, int _AP_N3>
231     INLINE
232     ap_concat_ref<_AP_W3, ap_concat_ref, 1,
233     af_bit_ref<_AP_W3, _AP_I3, _AP_S3,
_AP_Q3, _AP_O3, _AP_N3> >
234     operator,(const af_bit_ref<_AP_W3, _AP_I3, _AP_S3,
_AP_Q3,
235     _AP_O3, _AP_N3> &a2) {
236         return ap_concat_ref<
237             _AP_W3, ap_concat_ref, 1,
238             af_bit_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3, _AP_O3,
_AP_N3> >{
239             *this,
240             const_cast<af_bit_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3,
241             _AP_N3>&>(a2));
242         }
243     template <int _AP_W3, bool _AP_S3>
244     INLINE ap_int_base<AP_MAX(_AP_W3, _AP_W3), _AP_S3> operat-
or&
245     const ap_int_base<_AP_W3, _AP_S3>& a2) {
246         return get() & a2;
247     }
248     template <int _AP_W3, bool _AP_S3>
249     INLINE ap_int_base<AP_MAX(_AP_W3, _AP_W3), _AP_S3> operat-
or|
250     const ap_int_base<_AP_W3, _AP_S3>& a2) {
251         return get() | a2;
252     }
253     template <int _AP_W3, bool _AP_S3>
254     INLINE ap_int_base<AP_MAX(_AP_W3, _AP_W3), _AP_S3> operat-
or^
255     const ap_int_base<_AP_W3, _AP_S3>& a2) {
256         return get() ^ a2;
257     }
258     template <int Hi, int Lo>
259     #if 0
260     template<int Hi, int Lo>
261
262     INLINE ap_int_base<Hi-Lo+1, false> slice() {
263         ap_int_base<_AP_W3, false> bv = get();
264         return bv.slice<Hi,Lo>();
265     }
266 #endif
267
268     INLINE ap_int_base<_AP_W3, false> get() const {
269         ap_int_base<_AP_W3, false> tmpVal(0);
270         int w_ref1 = mbv1.length();
271         int w_ref2 = mbv2.length();
272         ap_int_base<_AP_W2, false> v2(mbv2);
273         ap_int_base<_AP_W1, false> v1(mbv1);
274
275         tmpVal.V = _AP_ROOT_op_set_range(tmpVal.V, 0, w_ref2 - 1,
v2.V);
276         tmpVal.V =
277             _AP_ROOT_op_set_range(tmpVal.V, w_ref2, w_ref1 +
w_ref2 - 1, v1.V);
278         return tmpVal;
279     }
280
281     template <int _AP_W3>
282     INLINE void set(const ap_int_base<_AP_W3, false>& val) {
283         ap_int_base<_AP_W1 + _AP_W2, false> vval(val);
284         int w_ref1 = mbv1.length();
285         int w_ref2 = mbv2.length();
286         ap_int_base<_AP_W1, false> tmpVal1;
287         tmpVal1.V = _AP_ROOT_op_get_range(vval.V, w_ref2, w_ref1
+ w_ref2 - 1);
288         mbv1.set(tmpVal1);
289         ap_int_base<_AP_W2, false> tmpVal2;
290         tmpVal2.V = _AP_ROOT_op_get_range(vval.V, 0, w_ref2 - 1);
291         mbv2.set(tmpVal2);
292     }
293
294     INLINE int length() const { return mbv1.length() +
mbv2.length(); }
295 }; // struct ap_concat_ref
296
297 /* Range (slice) reference.
298
299 */
300 template <int _AP_W, bool _AP_S>
301 struct ap_range_ref {
302     // struct ssdm_int or its sim model.
303     // TODO make it possible to reference to ap_fixed_base/
ap_fixed/ap_ufixed
304     // and then we can retire af_range_ref.
305     typedef ap_int_base<_AP_W, _AP_S> ref_type;
306     ref_type& d_bv;
307     int l_index;
308     int h_index;
309
310 public:
311     INLINE ap_range_ref(const ap_range_ref<_AP_W, _AP_S>& ref)
: d_bv(ref.d_bv), l_index(ref.l_index), h_in-
dex(ref.h_index) {}
312
313     INLINE ap_range_ref(ref_type* bv, int h, int l)
: d_bv(*bv), l_index(l), h_index(h) {}
314
315     INLINE ap_range_ref(const ref_type* bv, int h, int l)
: d_bv(*const_cast<ref_type*>(bv)), l_index(l), h_in-
dex(h) {}
316
317     INLINE operator ap_int_base<_AP_W, false>() const {
318         ap_int_base<_AP_W, false> ret;
319         ret.V = _AP_ROOT_op_get_range(d_bv.V, l_index, h_index);
320         return ret;
321     }
322
323     INLINE operator ap_ulong() const { return to_uint64(); }
324
325     /// @name assign operators
326     /// @{
327
328     // FIXME disabled to work-around lagacy code assigning from
sc_signal<T>,
329     // which dependes on implicit type conversion.
330     //
331     // assign from hls supported C integral types.
332     // template <typename T>
333     // INLINE typename
334     // _ap_type::enable_if<_ap_type::is_integral<T>::value,
335     // _ap_range_ref&>::type
336     // operator=(T val) {
337         // ap_int_base<_AP_W, false> tmp(val);
338         // d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index,
h_index, tmp.V);
339         // return *this;
340     }
341
342     #define AS-
SIGN_WITH_CTYPE(_Tp) \
343     INLINE ap_range_ref& operator=(_Tp val) \
344     { \
345         ap_int_base<_AP_W, false> \
tmp(val); \

```

```

346     d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
tmp.V); \
347     return
*this; \
348 }
349
350 ASSIGN_WITH_CTYPE(bool)
351 ASSIGN_WITH_CTYPE(char)
352 ASSIGN_WITH_CTYPE(signed char)
353 ASSIGN_WITH_CTYPE(unsigned char)
354 ASSIGN_WITH_CTYPE(short)
355 ASSIGN_WITH_CTYPE(unsigned short)
356 ASSIGN_WITH_CTYPE(int)
357 ASSIGN_WITH_CTYPE(unsigned int)
358 ASSIGN_WITH_CTYPE(long)
359 ASSIGN_WITH_CTYPE(unsigned long)
360 ASSIGN_WITH_CTYPE(ap_slong)
361 ASSIGN_WITH_CTYPE(ap_ulong)
362 #if _AP_ENABLE_HALF == 1
363 ASSIGN_WITH_CTYPE(half)
364 #endif
365 ASSIGN_WITH_CTYPE(float)
366 ASSIGN_WITH_CTYPE(double)
367
368 #undef ASSIGN_WITH_CTYPE
369
370 /// assign using string. XXX crucial for cosim.
371 INLINE ap_range_ref& operator=(const char* val) {
372     const ap_int_base<_AP_W, false> tmp(val); // XXX figure
out radix
373     d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
tmp.V);
374     return *this;
375 }
376
377 /// assign from ap_int_base.
378 template <int _AP_W2, bool _AP_S2>
379 INLINE ap_range_ref& operator=(const ap_int_base<_AP_W2,
_AP_S2>& val) {
380     ap_int_base<_AP_W, false> tmp(val);
381     d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
tmp.V);
382     return *this;
383 }
384
385 /// copy assign operator
386 // XXX Be explicit to prevent it from being deleted, as
field d_bv
387 // is of reference type.
388 INLINE ap_range_ref& operator=(const ap_range_ref& val) {
389     return operator=((const ap_int_base<_AP_W, false>)val);
390 }
391
392 /// assign from range reference to ap_int_base.
393 template <int _AP_W2, bool _AP_S2>
394 INLINE ap_range_ref& operator=(const ap_range_ref<_AP_W2,
_AP_S2>& val) {
395     return operator=((const ap_int_base<_AP_W2, false>)val);
396 }
397
398 /// assign from bit reference to ap_int_base.
399 template <int _AP_W2, bool _AP_S2>
400 INLINE ap_range_ref& operator=(const ap_bit_ref<_AP_W2,
_AP_S2>& val) {
401     return operator=((ap_ulong)(bool)(val));
402 }
403
404 /// assign from ap_fixed_base.
405 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
406         ap_o_mode _AP_O2, int _AP_N2>
407 INLINE ap_range_ref& operator=(
408     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&
409     val) {
410     return operator=(val.to_ap_int_base());
411 }
412
413 /// assign from range reference to ap_fixed_base.
414 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
415         ap_o_mode _AP_O2, int _AP_N2>
416 INLINE ap_range_ref& operator=(
417     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
418     return operator=((const ap_int_base<_AP_W2, false>)val);
419 }
420
421 /// assign from bit reference to ap_fixed_base.
422 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
423         ap_o_mode _AP_O2, int _AP_N2>
424 INLINE ap_range_ref& operator=(
425     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
426     return operator=((ap_ulong)(bool)(val));
427 }
428
429 /// assign from compound reference.

```

```

430 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
431     INLINE ap_range_ref& operator=(
432         const ap_concat_ref<_AP_W2, _AP_T3, _AP_W3, _AP_T3>&
val) {
433         return operator=((const ap_int_base<_AP_W2 + _AP_W3,
false>)(val));
434     }
435     // @}
436
437     template <int _AP_W2, bool _AP_S2>
438     INLINE
439         ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2>> >
440         operator,(const ap_range_ref<_AP_W2, _AP_S2> &a2) {
441             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
442                 ap_range_ref<_AP_W2, _AP_S2>> > (
443                 *this, const_cast<ap_range_ref<_AP_W2,
_AP_S2>&>(a2));
444         }
445
446     template <int _AP_W2, bool _AP_S2>
447     INLINE
448         ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2>> >
449         operator,(ap_int_base<_AP_W2, _AP_S2> &a2) {
450             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
451                 ap_int_base<_AP_W2, _AP_S2>> > (*this,
a2);
452         }
453
454     INLINE
455         ap_concat_ref<_AP_W, ap_range_ref, _AP_W,
ap_int_base<_AP_W, _AP_S>> >
456         operator,(ap_int_base<_AP_W, _AP_S>& a2) {
457             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W,
458                 ap_int_base<_AP_W, _AP_S>> > (*this,
a2);
459         }
460
461     template <int _AP_W2, bool _AP_S2>
462     INLINE
463         ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2>> >
464         operator,(volatile ap_int_base<_AP_W2, _AP_S2> &a2) {
465             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
466                 ap_int_base<_AP_W2, _AP_S2>> > (
467                 *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
468         }
469
470     template <int _AP_W2, bool _AP_S2>
471     INLINE
472         ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2>> >
473         operator,(const ap_int_base<_AP_W2, _AP_S2> &a2) {
474             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
475                 ap_int_base<_AP_W2, _AP_S2>> > (
476                 *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
477         }
478
479     template <int _AP_W2, bool _AP_S2>
480     INLINE
481         ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2>> >
482         operator,(const volatile ap_int_base<_AP_W2, _AP_S2>
&a2) {
483             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W2,
484                 ap_int_base<_AP_W2, _AP_S2>> > (
485                 *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
486         }
487
488     template <int _AP_W2, bool _AP_S2>
489     INLINE ap_concat_ref<_AP_W, ap_range_ref, 1,
ap_bit_ref<_AP_W2, _AP_S2>> >
490         operator,(const ap_bit_ref<_AP_W2, _AP_S2> &a2) {
491             return ap_concat_ref<_AP_W, ap_range_ref, 1,
492                 ap_bit_ref<_AP_W2, _AP_S2>> > (
493                 *this, const_cast<ap_bit_ref<_AP_W2, _AP_S2>&>(a2));
494         }
495
496     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
497     INLINE ap_concat_ref<_AP_W, ap_range_ref, _AP_W2 + _AP_W3,
498         ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>> >
499         operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &a2) {
500             return ap_concat_ref<_AP_W, ap_range_ref, _AP_W2 +
501                 ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>> > (
502                 *this, const_cast<ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>&>(a2));
503         }
504
505     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
506         ap_o_mode _AP_O2, int _AP_N2>

```

```

506     INLINE ap_concat_ref<
507         _AP_W, ap_range_ref, _AP_W2,
508         af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >
509     operator,(
510         const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> a2) {
511         return ap_concat_ref<
512             _AP_W, ap_range_ref, _AP_W2,
513             af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(&
514             *this,
515             const_cast<
516                 af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&>(a2));
517         }
518     }
519     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
520             ap_o_mode _AP_O2, int _AP_N2>
521     INLINE
522         ap_concat_ref<_AP_W, ap_range_ref, 1,
523             af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
524         operator,(const af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
525             &a2) {
526             return ap_concat_ref<
527                 _AP_W, ap_range_ref, 1,
528                 af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(&
529                 *this,
530                 const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&>(
531                     a2));
532             }
533     }
534     template <int _AP_W2, bool _AP_S2>
535     INLINE bool operator==(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
536         ap_int_base<_AP_W, false> lop(*this);
537         ap_int_base<_AP_W2, false> hop(op2);
538         return lop == hop;
539     }
540     }
541     template <int _AP_W2, bool _AP_S2>
542     INLINE bool operator!=(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
543         return !(operator==(op2));
544     }
545     }
546     template <int _AP_W2, bool _AP_S2>
547     INLINE bool operator<(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
548         ap_int_base<_AP_W, false> lop(*this);
549         ap_int_base<_AP_W2, false> hop(op2);
550         return lop < hop;
551     }
552     }
553     template <int _AP_W2, bool _AP_S2>
554     INLINE bool operator<=(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
555         ap_int_base<_AP_W, false> lop(*this);
556         ap_int_base<_AP_W2, false> hop(op2);
557         return lop <= hop;
558     }
559     }
560     template <int _AP_W2, bool _AP_S2>
561     INLINE bool operator>(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
562         return !(operator<=(op2));
563     }
564     }
565     template <int _AP_W2, bool _AP_S2>
566     INLINE bool operator>=(const ap_range_ref<_AP_W2, _AP_S2>&
op2) {
567         return !(operator<(op2));
568     }
569     }
570     template <int _AP_W2, bool _AP_S2>
571     INLINE ap_range_ref<_AP_W, _AP_S>& operator|=(
572         const ap_range_ref<_AP_W2, _AP_S2>& op2) {
573         (this->d.bv).V |= (op2.d.bv).V;
574         return *this;
575     };
576     }
577     template <int _AP_W2, bool _AP_S2>
578     INLINE ap_range_ref<_AP_W, _AP_S>& operator|=(
579         const ap_int_base<_AP_W2, _AP_S2>& op2) {
580         (this->d.bv).V |= op2.V;
581         return *this;
582     };
583     }
584     template <int _AP_W2, bool _AP_S2>
585     INLINE ap_range_ref<_AP_W, _AP_S>& operator&=(
586         const ap_range_ref<_AP_W2, _AP_S2>& op2) {
587         (this->d.bv).V &= (op2.d.bv).V;
588         return *this;
589     };

```

```

590
591     template <int _AP_W2, bool _AP_S2>
592     INLINE ap_range_ref<_AP_W, _AP_S>& operator&= (
593         const ap_int_base<_AP_W2, _AP_S2>& op2) {
594         (this->d_bv).V &= op2.V;
595         return *this;
596     };
597
598     template <int _AP_W2, bool _AP_S2>
599     INLINE ap_range_ref<_AP_W, _AP_S>& operator^= (
600         const ap_range_ref<_AP_W2, _AP_S2>& op2) {
601         (this->d_bv).V ^= (op2.d_bv).V;
602         return *this;
603     };
604
605     template <int _AP_W2, bool _AP_S2>
606     INLINE ap_range_ref<_AP_W, _AP_S>& operator^= (
607         const ap_int_base<_AP_W2, _AP_S2>& op2) {
608         (this->d_bv).V ^= op2.V;
609         return *this;
610     };
611
612     INLINE ap_int_base<_AP_W, false> get() const {
613         ap_int_base<_AP_W, false> ret;
614         ret.V = _AP_ROOT_op_get_range(d_bv.V, l_index, h_index);
615         return ret;
616     }
617
618     template <int _AP_W2>
619     INLINE void set(const ap_int_base<_AP_W2, false>& val) {
620         d_bv.V = _AP_ROOT_op_set_range(d_bv.V, l_index, h_index,
val.V);
621     }
622
623     INLINE int length() const {
624         return h_index >= l_index ? h_index - l_index + 1 : l_in-
dex - h_index + 1;
625     }
626
627     INLINE int to_int() const {
628         return (int)(_AP_ROOT_op_get_range(d_bv.V, l_index, h_in-
dex));
629     }
630
631     INLINE unsigned to_uint() const {
632         return (unsigned)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
633     }
634
635     INLINE long to_long() const {
636         return (long)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
637     }
638
639     INLINE unsigned long to_ulong() const {
640         return (unsigned long)(_AP_ROOT_op_get_range(d_bv.V,
l_index, h_index));
641     }
642
643     INLINE ap_slong to_int64() const {
644         return (ap_slong)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
645     }
646
647     INLINE ap_ulong to_uint64() const {
648         return (ap_ulong)(_AP_ROOT_op_get_range(d_bv.V, l_index,
h_index));
649     }
650
651     INLINE bool and_reduce() const {
652         bool ret = true;
653         bool reverse = l_index > h_index;
654         unsigned low = reverse ? h_index : l_index;
655         unsigned high = reverse ? l_index : h_index;
656         for (unsigned i = low; i != high; ++i) {
657             #ifdef __SYNTHESIS__
658             #pragma HLS unroll
659             #endif
660             ret &= _AP_ROOT_op_get_bit(d_bv.V, i);
661         }
662         return ret;
663     }
664
665     INLINE bool or_reduce() const {
666         bool ret = false;
667         bool reverse = l_index > h_index;
668         unsigned low = reverse ? h_index : l_index;
669         unsigned high = reverse ? l_index : h_index;
670         for (unsigned i = low; i != high; ++i) {
671             #ifdef __SYNTHESIS__
672             #pragma HLS unroll
673             #endif
674             ret |= _AP_ROOT_op_get_bit(d_bv.V, i);
675         }
676         return ret;
677     }
678
679     INLINE bool xor_reduce() const {
680         bool ret = false;
681         bool reverse = l_index > h_index;

```



```

682     unsigned low = reverse ? h_index : l_index;
683     unsigned high = reverse ? l_index : h_index;
684     for (unsigned i = low; i != high; ++i) {
685 #ifdef __SYNTHESIS__
686 #pragma HLS unroll
687 #endif
688         ret ^= _AP_ROOT_op_get_bit(d_bv.V, i);
689     }
690     return ret;
691 }
692 #ifndef __SYNTHESIS__
693 std::string to_string(signed char radix = 2) const {
694     ap_int_base<_AP_W, false> ret;
695     ret.V = _AP_ROOT_op_get_range(d_bv.V, l_index, h_index);
696     return ret.to_string(radix);
697 }
698 #else
699 // XXX HLS will delete this in synthesis
700 inline char* to_string(signed char radix = 2) const {
701     return 0;
702 }
703 #endif
704 }; // struct ap_range_ref
705
706 // XXX apcc cannot handle global std::ios_base::Init()
brought in by <iostream>
707 #ifndef AP_AUTOCC
708 #ifndef __SYNTHESIS__
709 template <int _AP_W, bool _AP_S>
710 inline std::ostream& operator<<(std::ostream& os,
711                                const ap_range_ref<_AP_W,
712                                _AP_S>& x) {
713     std::ios_base::fmtflags ff = std::cout.flags();
714     if (ff & std::cout.hex) {
715         os << x.to_string(16); // don't print sign
716     } else if (ff & std::cout.oct) {
717         os << x.to_string(8); // don't print sign
718     } else {
719         os << x.to_string(10);
720     }
721     return os;
722 }
723 #endif // ifndef __SYNTHESIS__
724 #ifndef __SYNTHESIS__
725 template <int _AP_W, bool _AP_S>
726 inline std::istream& operator>>(std::istream& in,
727                                ap_range_ref<_AP_W, _AP_S>&
728 op) {
729     std::string str;
730     in >> str;
731     op = ap_int_base<_AP_W, _AP_S>(str.c_str());
732     return in;
733 }
734 #endif // ifndef __SYNTHESIS__
735 #endif // ifndef AP_AUTOCC
736 /* Bit reference.
737
738 -----
739 */
740 template <int _AP_W, bool _AP_S>
741 struct ap_bit_ref {
742     // struct ssdm_int or its sim model.
743     // TODO make it possible to reference to ap_fixed_base/
744     // and then we can retire af_bit_ref.
745     typedef ap_int_base<_AP_W, _AP_S> ref_type;
746     ref_type& d_bv;
747     int d_index;
748 public:
749     // copy ctor
750     inline ap_bit_ref(const ap_bit_ref<_AP_W, _AP_S>& ref)
751         : d_bv(ref.d_bv), d_index(ref.d_index) {}
752
753     inline ap_bit_ref(ref_type* bv, int index = 0) : d_bv(*bv),
754     d_index(index) {}
755
756     inline ap_bit_ref(const ref_type* bv, int index = 0)
757         : d_bv(*const_cast<ref_type*>(bv)), d_index(index) {}
758
759     inline operator bool() const { return
760     _AP_ROOT_op_get_bit(d_bv.V, d_index); }
761     inline bool to_bool() const { return
762     _AP_ROOT_op_get_bit(d_bv.V, d_index); }
763
764     // assign op from hls supported C integral types.
765     // FIXME disabled to support sc_signal<bool>.
766     // NOTE this used to be unsigned long long.
767     //template <typename T>
768     //inline typename
769     //ap_type::enable_if<ap_type::is_integral<T>::value,
770     //                    ap_bit_ref&::type
771     //operator=(T val) {
772     //    d_bv.V = _AP_ROOT_op_set_bit(d_bv.V, d_index, val);
773     //    return *this;
774     //}
775     #define ASSIGN_WITH_CTYPE(_Tp) \

```

```

772     inline ap_bit_ref& operator=(_Tp val) { \
773         d_bv.V = _AP_ROOT_op_set_bit(d_bv.V, d_index, val); \
774         return *this; \
775     }
776
777     ASSIGN_WITH_CTYPE(bool)
778     ASSIGN_WITH_CTYPE(char)
779     ASSIGN_WITH_CTYPE(signed char)
780     ASSIGN_WITH_CTYPE(unsigned char)
781     ASSIGN_WITH_CTYPE(short)
782     ASSIGN_WITH_CTYPE(unsigned short)
783     ASSIGN_WITH_CTYPE(int)
784     ASSIGN_WITH_CTYPE(unsigned int)
785     ASSIGN_WITH_CTYPE(long)
786     ASSIGN_WITH_CTYPE(unsigned long)
787     ASSIGN_WITH_CTYPE(ap_long)
788     ASSIGN_WITH_CTYPE(ap_ulong)
789
790 #undef ASSIGN_WITH_CTYPE
791
792 #define ASSIGN_WITH_CTYPE_FP(_Tp) \
793     inline ap_bit_ref& operator=(_Tp val) { \
794         bool tmp_val = val; \
795         d_bv.V = _AP_ROOT_op_set_bit(d_bv.V, d_index, tmp_val); \
796         return *this; \
797     }
798
799 #if _AP_ENABLE_HALF == 1
800     ASSIGN_WITH_CTYPE_FP(half)
801 #endif
802     ASSIGN_WITH_CTYPE_FP(float)
803     ASSIGN_WITH_CTYPE_FP(double)
804
805 #undef ASSIGN_WITH_CTYPE_FP
806
807     template <int _AP_W2, bool _AP_S2>
808     inline ap_bit_ref& operator=(const ap_int_base<_AP_W2,
809     _AP_S2>& val) {
810         return operator=((ap_ulong)(val.V != 0));
811     }
812
813     template <int _AP_W2, bool _AP_S2>
814     inline ap_bit_ref& operator=(const ap_range_ref<_AP_W2,
815     _AP_S2>& val) {
816         return operator=((ap_int_base<_AP_W2, false>)val);
817     }
818     // Be explicit to prevent it from being deleted, as field
819     // d_bv
820     // is of reference type.
821     inline ap_bit_ref& operator=(const ap_bit_ref& val) {
822         return operator=((ap_ulong)(bool)val);
823     }
824
825     template <int _AP_W2, bool _AP_S2>
826     inline ap_bit_ref& operator=(const ap_bit_ref<_AP_W2,
827     _AP_S2>& val) {
828         return operator=((ap_ulong)(bool)val);
829     }
830
831     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
832     _AP_Q2,
833     ap_o_mode _AP_O2, int _AP_N2>
834     inline ap_bit_ref& operator=(
835     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
836     _AP_O2, _AP_N2>& val) {
837         return operator=((const ap_int_base<_AP_W2, false>)val);
838     }
839
840     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
841     _AP_Q2,
842     ap_o_mode _AP_O2, int _AP_N2>
843     inline ap_bit_ref& operator=(
844     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
845     _AP_O2, _AP_N2>& val) {
846         return operator=((ap_ulong)(bool)val);
847     }
848
849     template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
850     _AP_T3>
851     inline ap_bit_ref& operator=(
852     const ap_concat_ref<_AP_W2, _AP_T3, _AP_W3, _AP_T3>&
853     val) {
854         return operator=((const ap_int_base<_AP_W2 + _AP_W3,
855     false>)val);
856     }
857
858     template <int _AP_W2, bool _AP_S2>
859     inline ap_concat_ref<1, ap_bit_ref, _AP_W2,
860     ap_int_base<_AP_W2, _AP_S2>>
861     operator,(ap_int_base<_AP_W2, _AP_S2> &a2) {
862         return ap_concat_ref<1, ap_bit_ref, _AP_W2,
863     ap_int_base<_AP_W2, _AP_S2>>(>
864         *this, a2);
865     }
866
867     template <int _AP_W2, bool _AP_S2>
868     inline ap_concat_ref<1, ap_bit_ref, _AP_W2,

```

```

ap_int_base<_AP_W2, _AP_S2> >
858 operator,(volatile ap_int_base<_AP_W2, _AP_S2> &a2) {
859     return ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >(>
860         *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(a2));
861 }
862
863 template <int _AP_W2, bool _AP_S2>
864 INLINE ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
865 operator,(const ap_int_base<_AP_W2, _AP_S2> &a2) {
866     ap_int_base<_AP_W2, _AP_S2> op(a2);
867     return ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >(>
868         *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(op));
869 }
870
871 template <int _AP_W2, bool _AP_S2>
872 INLINE ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >
873 operator,(const volatile ap_int_base<_AP_W2, _AP_S2> &a2) {
874     ap_int_base<_AP_W2, _AP_S2> op(a2);
875     return ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_int_base<_AP_W2, _AP_S2> >(>
876         *this, const_cast<ap_int_base<_AP_W2, _AP_S2>&>(op));
877 }
878
879 template <int _AP_W2, bool _AP_S2>
880 INLINE ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >
881 operator,(const ap_range_ref<_AP_W2, _AP_S2> &a2) {
882     return ap_concat_ref<1, ap_bit_ref, _AP_W2,
ap_range_ref<_AP_W2, _AP_S2> >(>
883         *this, const_cast<ap_range_ref<_AP_W2,
_AP_S2>&>(a2));
884 }
885
886 template <int _AP_W2, bool _AP_S2>
887 INLINE ap_concat_ref<1, ap_bit_ref, 1, ap_bit_ref<_AP_W2,
_AP_S2> > operator,(
888     const ap_bit_ref<_AP_W2, _AP_S2> &a2) {
889
889     return ap_concat_ref<1, ap_bit_ref, 1, ap_bit_ref<_AP_W2,
_AP_S2> >(>
890         *this, const_cast<ap_bit_ref<_AP_W2, _AP_S2>&>(a2));
891 }
892
893 template <int _AP_W2, typename _AP_T2, int _AP_W3, typename
_AP_T3>
894 INLINE ap_concat_ref<1, ap_bit_ref, _AP_W2 + _AP_W3,
895     ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> >
896 operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &a2) {
897     return ap_concat_ref<1, ap_bit_ref, _AP_W2 + _AP_W3,
898     ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >(>
899         *this, const_cast<ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>&>(a2));
900 }
901
902 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
903     ap_o_mode _AP_O2, int _AP_N2>
904 INLINE ap_concat_ref<
905     1, ap_bit_ref, _AP_W2,
906     af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >
907 operator,(
908     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
909     return ap_concat_ref<
910     1, ap_bit_ref, _AP_W2,
911     af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(>
912         *this,
913         const_cast<
914             af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&>(a2));
915 }
916
917 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
918     ap_o_mode _AP_O2, int _AP_N2>
919 INLINE ap_concat_ref<1, ap_bit_ref, 1, af_bit_ref<_AP_W2,
_AP_I2, _AP_S2,
920     _AP_Q2,
_AP_O2, _AP_N2> >
921 operator,(
922     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
923
923     return ap_concat_ref<1, ap_bit_ref, 1, af_bit_ref<_AP_W2,
_AP_I2, _AP_S2,
924     _AP_Q2,
_AP_O2, _AP_N2> >(>
925         *this,

```

```

926     const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&>(
927         a2));
928 }
929
930 template <int _AP_W2, bool _AP_S2>
931 INLINE bool operator==(const ap_bit_ref<_AP_W2, _AP_S2>&
op) {
932     return get() == op.get();
933 }
934
935 template <int _AP_W2, bool _AP_S2>
936 INLINE bool operator!=(const ap_bit_ref<_AP_W2, _AP_S2>&
op) {
937     return get() != op.get();
938 }
939
940 INLINE bool get() const { return
_AP_ROOT_op_get_bit(d_bv.V, d_index); }
941
942 INLINE bool get() { return _AP_ROOT_op_get_bit(d_bv.V,
d_index); }
943
944 template <int _AP_W3>
945 INLINE void set(const ap_int_base<_AP_W3, false>& val) {
946     operator=(val);
947 }
948
949 INLINE bool operator~() const {
950     bool bit = _AP_ROOT_op_get_bit(d_bv.V, d_index);
951     return bit ? false : true;
952 }
953
954 INLINE int length() const { return 1; }
955
956 #ifndef __SYNTHESIS__
957 std::string to_string() const { return get() ? "1" : "0"; }
958 #else
959 // XXX HLS will delete this in synthesis
960 INLINE char* to_string() const { return 0; }
961 #endif
962 }; // struct ap_bit_ref
963
964 /* ap_range_ref with int.
965 *
-----
966 */
967 // equality and relational operators.
968 #define REF_REL_OP_WITH_INT(REL_OP, C_TYPE, _AP_W2,
_AP_S2)
969     template <int _AP_W, bool
970     _AP_S>
971     INLINE bool operator REL_OP(const ap_range_ref<_AP_W,
_AP_S>& op,
972     \
973     C_TYPE op2)
974 {
975     return ap_int_base<_AP_W,
false>(op)
976     \
977     REL_OP ap_int_base<_AP_W2,
_AP_S2>(op2);
978 }
979
980 template <int _AP_W, bool
_AP_S>
981     template <int _AP_W, bool
_AP_S>
982     INLINE bool operator REL_OP(C_TYPE
op2,
983     \
984     const ap_bit_ref<_AP_W, _AP_S>&
op) {
985     return op2 REL_OP
986     \
987     bool(op);
988 }
989
990 template <int _AP_W, typename _AP_T, int _AP_W1, typename
_AP_T1>
991     template <int _AP_W, typename _AP_T, int _AP_W1, typename
_AP_T1>
992     INLINE bool operator
993     REL_OP(
994     \
995     const ap_concat_ref<_AP_W, _AP_T, _AP_W1, _AP_T1>& op,
C_TYPE op2) {
996     \
997     return ap_int_base<_AP_W + _AP_W1,
false>(op)
998     \
999     REL_OP ap_int_base<_AP_W2,
_AP_S2>(op2);
1000 }
1001
1002 // Make the line shorter than 5000 chars

```

```

993 #define REF_REL_WITH_INT_1(C_TYPE, _AP_WI, _AP_SI) \
994   REF_REL_OP_WITH_INT(>, C_TYPE, _AP_WI, _AP_SI) \
995   REF_REL_OP_WITH_INT(<, C_TYPE, _AP_WI, _AP_SI) \
996   REF_REL_OP_WITH_INT(>=, C_TYPE, _AP_WI, _AP_SI) \
997   REF_REL_OP_WITH_INT(<=, C_TYPE, _AP_WI, _AP_SI)
998
999 REF_REL_WITH_INT_1(bool, 1, false)
1000 REF_REL_WITH_INT_1(char, 8, CHAR_IS_SIGNED)
1001 REF_REL_WITH_INT_1(signed char, 8, true)
1002 REF_REL_WITH_INT_1(unsigned char, 8, false)
1003 REF_REL_WITH_INT_1(short, _AP_SIZE_short, true)
1004 REF_REL_WITH_INT_1(unsigned short, _AP_SIZE_short, false)
1005 REF_REL_WITH_INT_1(int, _AP_SIZE_int, true)
1006 REF_REL_WITH_INT_1(unsigned int, _AP_SIZE_int, false)
1007 REF_REL_WITH_INT_1(long, _AP_SIZE_long, true)
1008 REF_REL_WITH_INT_1(unsigned long, _AP_SIZE_long, false)
1009 REF_REL_WITH_INT_1(ap_ulong, _AP_SIZE_ap_ulong, true)
1010 REF_REL_WITH_INT_1(ap_ulong, _AP_SIZE_ap_ulong, false)
1011
1012 // Make the line shorter than 5000 chars
1013 #define REF_REL_WITH_INT_2(C_TYPE, _AP_WI, _AP_SI) \
1014   REF_REL_OP_WITH_INT(==, C_TYPE, _AP_WI, _AP_SI) \
1015   REF_REL_OP_WITH_INT(!=, C_TYPE, _AP_WI, _AP_SI)
1016
1017 REF_REL_WITH_INT_2(bool, 1, false)
1018 REF_REL_WITH_INT_2(char, 8, CHAR_IS_SIGNED)
1019 REF_REL_WITH_INT_2(signed char, 8, true)
1020 REF_REL_WITH_INT_2(unsigned char, 8, false)
1021 REF_REL_WITH_INT_2(short, _AP_SIZE_short, true)
1022 REF_REL_WITH_INT_2(unsigned short, _AP_SIZE_short, false)
1023 REF_REL_WITH_INT_2(int, _AP_SIZE_int, true)
1024 REF_REL_WITH_INT_2(unsigned int, _AP_SIZE_int, false)
1025 REF_REL_WITH_INT_2(long, _AP_SIZE_long, true)
1026 REF_REL_WITH_INT_2(unsigned long, _AP_SIZE_long, false)
1027 REF_REL_WITH_INT_2(ap_ulong, _AP_SIZE_ap_ulong, true)
1028 REF_REL_WITH_INT_2(ap_ulong, _AP_SIZE_ap_ulong, false)
1029
1030 #undef REF_REL_OP_WITH_INT
1031 #undef REF_REL_WITH_INT_1
1032 #undef REF_REL_WITH_INT_2
1033
1034 #define REF_BIN_OP_WITH_INT(BIN_OP, RTYPE, C_TYPE, _AP_W2,
1035   _AP_S2) \
1036   template <int _AP_W, bool \
1037   _AP_S> \
1038   inline typename ap_int_base<_AP_W, false>::template \
1039   RTYPE<_AP_W2, \
1040   _AP_S2>::RTYPE \
1041   operator BIN_OP(const ap_range_ref<_AP_W, _AP_S>& op, \
1042   C_TYPE op2) { \
1043     return ap_int_base<_AP_W, \
1044     false>(op) \
1045     BIN_OP ap_int_base<_AP_W2, \
1046     _AP_S2>(op2); \
1047   }
1048
1049 template <int _AP_W, bool \
1050 _AP_S> \
1051 inline typename ap_int_base<_AP_W2, _AP_S2>::template \
1052 RTYPE<_AP_W, \
1053 _AP_S2>::RTYPE \
1054 operator BIN_OP(C_TYPE op2, const ap_range_ref<_AP_W, \
1055 _AP_S>& op) { \
1056   return ap_int_base<_AP_W2, \
1057   _AP_S2>(op2) \
1058   BIN_OP ap_int_base<_AP_W, \
1059   _AP_S>(op); \
1060 }
1061
1062 // arithmetic operators.
1063 #define REF_BIN_OP_WITH_INT_ARITH(C_TYPE, _AP_W2, _AP_S2) \
1064   REF_BIN_OP_WITH_INT(+, plus, C_TYPE, (_AP_W2), (_AP_S2)) \
1065   REF_BIN_OP_WITH_INT(-, minus, C_TYPE, (_AP_W2), (_AP_S2)) \
1066   REF_BIN_OP_WITH_INT(*, mult, C_TYPE, (_AP_W2), (_AP_S2)) \
1067   REF_BIN_OP_WITH_INT(/, div, C_TYPE, (_AP_W2), (_AP_S2)) \
1068   REF_BIN_OP_WITH_INT(% mod, C_TYPE, (_AP_W2), (_AP_S2))
1069
1070 REF_BIN_OP_WITH_INT_ARITH(bool, 1, false)
1071 REF_BIN_OP_WITH_INT_ARITH(char, 8, CHAR_IS_SIGNED)
1072 REF_BIN_OP_WITH_INT_ARITH(signed char, 8, true)
1073 REF_BIN_OP_WITH_INT_ARITH(unsigned char, 8, false)
1074 REF_BIN_OP_WITH_INT_ARITH(short, _AP_SIZE_short, true)
1075 REF_BIN_OP_WITH_INT_ARITH(unsigned short, _AP_SIZE_short, \
1076 false)
1077 REF_BIN_OP_WITH_INT_ARITH(int, _AP_SIZE_int, true)
1078 REF_BIN_OP_WITH_INT_ARITH(unsigned int, _AP_SIZE_int, false)
1079 REF_BIN_OP_WITH_INT_ARITH(long, _AP_SIZE_long, true)
1080 REF_BIN_OP_WITH_INT_ARITH(unsigned long, _AP_SIZE_long, \
1081 false)
1082 REF_BIN_OP_WITH_INT_ARITH(ap_ulong, _AP_SIZE_ap_ulong, true)
1083 REF_BIN_OP_WITH_INT_ARITH(ap_ulong, _AP_SIZE_ap_ulong, false)
1084
1085 #undef REF_BIN_OP_WITH_INT_ARITH
1086
1087 // bitwise and shift operators
1088 #define REF_BIN_OP_WITH_INT_BITS(C_TYPE, _AP_W2, _AP_S2) \
1089   REF_BIN_OP_WITH_INT(&, logic, C_TYPE, (_AP_W2), (_AP_S2)) \
1090   REF_BIN_OP_WITH_INT(|, logic, C_TYPE, (_AP_W2), (_AP_S2)) \
1091   REF_BIN_OP_WITH_INT(^, logic, C_TYPE, (_AP_W2), (_AP_S2)) \
1092   REF_BIN_OP_WITH_INT(>>, arg1, C_TYPE, (_AP_W2), (_AP_S2)) \
1093   REF_BIN_OP_WITH_INT(<<, arg1, C_TYPE, (_AP_W2), (_AP_S2))
1094
1095 REF_BIN_OP_WITH_INT_BITS(bool, 1, false)
1096 REF_BIN_OP_WITH_INT_BITS(char, 8, CHAR_IS_SIGNED)
1097 REF_BIN_OP_WITH_INT_BITS(signed char, 8, true)
1098 REF_BIN_OP_WITH_INT_BITS(unsigned char, 8, false)
1099 REF_BIN_OP_WITH_INT_BITS(short, _AP_SIZE_short, true)
1100 REF_BIN_OP_WITH_INT_BITS(unsigned short, _AP_SIZE_short, \
1101 false)
1102 REF_BIN_OP_WITH_INT_BITS(int, _AP_SIZE_int, true)
1103 REF_BIN_OP_WITH_INT_BITS(unsigned int, _AP_SIZE_int, false)
1104 REF_BIN_OP_WITH_INT_BITS(long, _AP_SIZE_long, true)
1105 REF_BIN_OP_WITH_INT_BITS(unsigned long, _AP_SIZE_long, false)
1106 REF_BIN_OP_WITH_INT_BITS(ap_ulong, _AP_SIZE_ap_ulong, true)
1107 REF_BIN_OP_WITH_INT_BITS(ap_ulong, _AP_SIZE_ap_ulong, false)
1108
1109 #undef REF_BIN_OP_WITH_INT_BITS
1110
1111 /* ap_range_ref with ap_range_ref
1112 *
1113 -----
1114 */
1115 #define REF_BIN_OP(BIN_OP, \
1116   RTYPE) \
1117   template <int _AP_W, bool _AP_S, int _AP_W2, bool \
1118   _AP_S2> \
1119   inline \
1120   typename ap_int_base<_AP_W, false>::template \
1121   RTYPE<_AP_W2, false>::RTYPE \
1122   operator BIN_OP(const ap_range_ref<_AP_W, _AP_S>& \
1123   lhs, \
1124   const ap_range_ref<_AP_W2, _AP_S2>& \
1125   rhs) { \
1126     return (lhs.operator ap_int_base<_AP_W, \
1127     false>())BIN_OP( \
1128     rhs.operator ap_int_base<_AP_W2, \
1129     false>()); \
1130   }
1131
1132 REF_BIN_OP(+, plus)
1133 REF_BIN_OP(-, minus)
1134 REF_BIN_OP(*, mult)
1135 REF_BIN_OP(/, div)
1136 REF_BIN_OP(% mod)
1137 REF_BIN_OP(&, logic)
1138 REF_BIN_OP(|, logic)
1139 REF_BIN_OP(^, logic)
1140 REF_BIN_OP(>>, arg1)
1141 REF_BIN_OP(<<, arg1)
1142
1143 /* ap_concat_ref with ap_concat_ref.
1144 *
1145 -----
1146 */
1147
1148 //
1149 *****
1150 // Implement
1151 // ap_int_base<M+N> = ap_concat_ref<M> OP ap_con-
1152 cat_ref<N>
1153 // for operators +, -, *, /, %, >>, <<, &, |, ^
1154 // Without these operators the operands are converted to in-
1155 t64 and
1156 // larger results lose informations (higher order bits).
1157 //
1158 //
1159 //
1160 //
1161 //
1162 //
1163 //
1164 //
1165 //
1166 //
1167 //
1168 //
1169 //
1170 //
1171 //
1172 //
1173 //
1174 //
1175 //
1176 //
1177 //
1178 //
1179 //
1180 //
1181 //
1182 //
1183 //
1184 //
1185 //
1186 //
1187 //
1188 //
1189 //
1190 //
1191 //
1192 //
1193 //
1194 //
1195 //
1196 //
1197 //
1198 //
1199 //
1200 //
1201 //
1202 //
1203 //
1204 //
1205 //
1206 //
1207 //
1208 //
1209 //
1210 //
1211 //
1212 //
1213 //
1214 //
1215 //
1216 //
1217 //
1218 //
1219 //
1220 //
1221 //
1222 //
1223 //
1224 //
1225 //
1226 //
1227 //
1228 //
1229 //
1230 //
1231 //
1232 //
1233 //
1234 //
1235 //
1236 //
1237 //
1238 //
1239 //
1240 //
1241 //
1242 //
1243 //
1244 //
1245 //
1246 //
1247 //
1248 //
1249 //
1250 //
1251 //
1252 //
1253 //
1254 //
1255 //
1256 //
1257 //
1258 //
1259 //
1260 //
1261 //
1262 //
1263 //
1264 //
1265 //
1266 //
1267 //
1268 //
1269 //
1270 //
1271 //
1272 //
1273 //
1274 //
1275 //
1276 //
1277 //
1278 //
1279 //
1280 //
1281 //
1282 //
1283 //
1284 //
1285 //
1286 //
1287 //
1288 //
1289 //
1290 //
1291 //
1292 //
1293 //
1294 //
1295 //
1296 //
1297 //
1298 //
1299 //
1300 //
1301 //
1302 //
1303 //
1304 //
1305 //
1306 //
1307 //
1308 //
1309 //
1310 //
1311 //
1312 //
1313 //
1314 //
1315 //
1316 //
1317 //
1318 //
1319 //
1320 //
1321 //
1322 //
1323 //
1324 //
1325 //
1326 //
1327 //
1328 //
1329 //
1330 //
1331 //
1332 //
1333 //
1334 //
1335 //
1336 //
1337 //
1338 //
1339 //
1340 //
1341 //
1342 //
1343 //
1344 //
1345 //
1346 //
1347 //
1348 //
1349 //
1350 //
1351 //
1352 //
1353 //
1354 //
1355 //
1356 //
1357 //
1358 //
1359 //
1360 //
1361 //
1362 //
1363 //
1364 //
1365 //
1366 //
1367 //
1368 //
1369 //
1370 //
1371 //
1372 //
1373 //
1374 //
1375 //
1376 //
1377 //
1378 //
1379 //
1380 //
1381 //
1382 //
1383 //
1384 //
1385 //
1386 //
1387 //
1388 //
1389 //
1390 //
1391 //
1392 //
1393 //
1394 //
1395 //
1396 //
1397 //
1398 //
1399 //
1400 //
1401 //
1402 //
1403 //
1404 //
1405 //
1406 //
1407 //
1408 //
1409 //
1410 //
1411 //
1412 //
1413 //
1414 //
1415 //
1416 //
1417 //
1418 //
1419 //
1420 //
1421 //
1422 //
1423 //
1424 //
1425 //
1426 //
1427 //
1428 //
1429 //
1430 //
1431 //
1432 //
1433 //
1434 //
1435 //
1436 //
1437 //
1438 //
1439 //
1440 //
1441 //
1442 //
1443 //
1444 //
1445 //
1446 //
1447 //
1448 //
1449 //
1450 //
1451 //
1452 //
1453 //
1454 //
1455 //
1456 //
1457 //
1458 //
1459 //
1460 //
1461 //
1462 //
1463 //
1464 //
1465 //
1466 //
1467 //
1468 //
1469 //
1470 //
1471 //
1472 //
1473 //
1474 //
1475 //
1476 //
1477 //
1478 //
1479 //
1480 //
1481 //
1482 //
1483 //
1484 //
1485 //
1486 //
1487 //
1488 //
1489 //
1490 //
1491 //
1492 //
1493 //
1494 //
1495 //
1496 //
1497 //
1498 //
1499 //
1500 //
1501 //
1502 //
1503 //
1504 //
1505 //
1506 //
1507 //
1508 //
1509 //
1510 //
1511 //
1512 //
1513 //
1514 //
1515 //
1516 //
1517 //
1518 //
1519 //
1520 //
1521 //
1522 //
1523 //
1524 //
1525 //
1526 //
1527 //
1528 //
1529 //
1530 //
1531 //
1532 //
1533 //
1534 //
1535 //
1536 //
1537 //
1538 //
1539 //
1540 //
1541 //
1542 //
1543 //
1544 //
1545 //
1546 //
1547 //
1548 //
1549 //
1550 //
1551 //
1552 //
1553 //
1554 //
1555 //
1556 //
1557 //
1558 //
1559 //
1560 //
1561 //
1562 //
1563 //
1564 //
1565 //
1566 //
1567 //
1568 //
1569 //
1570 //
1571 //
1572 //
1573 //
1574 //
1575 //
1576 //
1577 //
1578 //
1579 //
1580 //
1581 //
1582 //
1583 //
1584 //
1585 //
1586 //
1587 //
1588 //
1589 //
1590 //
1591 //
1592 //
1593 //
1594 //
1595 //
1596 //
1597 //
1598 //
1599 //
1600 //
1601 //
1602 //
1603 //
1604 //
1605 //
1606 //
1607 //
1608 //
1609 //
1610 //
1611 //
1612 //
1613 //
1614 //
1615 //
1616 //
1617 //
1618 //
1619 //
1620 //
1621 //
1622 //
1623 //
1624 //
1625 //
1626 //
1627 //
1628 //
1629 //
1630 //
1631 //
1632 //
1633 //
1634 //
1635 //
1636 //
1637 //
1638 //
1639 //
1640 //
1641 //
1642 //
1643 //
1644 //
1645 //
1646 //
1647 //
1648 //
1649 //
1650 //
1651 //
1652 //
1653 //
1654 //
1655 //
1656 //
1657 //
1658 //
1659 //
1660 //
1661 //
1662 //
1663 //
1664 //
1665 //
1666 //
1667 //
1668 //
1669 //
1670 //
1671 //
1672 //
1673 //
1674 //
1675 //
1676 //
1677 //
1678 //
1679 //
1680 //
1681 //
1682 //
1683 //
1684 //
1685 //
1686 //
1687 //
1688 //
1689 //
1690 //
1691 //
1692 //
1693 //
1694 //
1695 //
1696 //
1697 //
1698 //
1699 //
1700 //
1701 //
1702 //
1703 //
1704 //
1705 //
1706 //
1707 //
1708 //
1709 //
1710 //
1711 //
1712 //
1713 //
1714 //
1715 //
1716 //
1717 //
1718 //
1719 //
1720 //
1721 //
1722 //
1723 //
1724 //
1725 //
1726 //
1727 //
1728 //
1729 //
1730 //
1731 //
1732 //
1733 //
1734 //
1735 //
1736 //
1737 //
1738 //
1739 //
1740 //
1741 //
1742 //
1743 //
1744 //
1745 //
1746 //
1747 //
1748 //
1749 //
1750 //
1751 //
1752 //
1753 //
1754 //
1755 //
1756 //
1757 //
1758 //
1759 //
1760 //
1761 //
1762 //
1763 //
1764 //
1765 //
1766 //
1767 //
1768 //
1769 //
1770 //
1771 //
1772 //
1773 //
1774 //
1775 //
1776 //
1777 //
1778 //
1779 //
1780 //
1781 //
1782 //
1783 //
1784 //
1785 //
1786 //
1787 //
1788 //
1789 //
1790 //
1791 //
1792 //
1793 //
1794 //
1795 //
1796 //
1797 //
1798 //
1799 //
1800 //
1801 //
1802 //
1803 //
1804 //
1805 //
1806 //
1807 //
1808 //
1809 //
1810 //
1811 //
1812 //
1813 //
1814 //
1815 //
1816 //
1817 //
1818 //
1819 //
1820 //
1821 //
1822 //
1823 //
1824 //
1825 //
1826 //
1827 //
1828 //
1829 //
1830 //
1831 //
1832 //
1833 //
1834 //
1835 //
1836 //
1837 //
1838 //
1839 //
1840 //
1841 //
1842 //
1843 //
1844 //
1845 //
1846 //
1847 //
1848 //
1849 //
1850 //
1851 //
1852 //
1853 //
1854 //
1855 //
1856 //
1857 //
1858 //
1859 //
1860 //
1861 //
1862 //
1863 //
1864 //
1865 //
1866 //
1867 //
1868 //
1869 //
1870 //
1871 //
1872 //
1873 //
1874 //
1875 //
1876 //
1877 //
1878 //
1879 //
1880 //
1881 //
1882 //
1883 //
1884 //
1885 //
1886 //
1887 //
1888 //
1889 //
1890 //
1891 //
1892 //
1893 //
1894 //
1895 //
1896 //
1897 //
1898 //
1899 //
1900 //
1901 //
1902 //
1903 //
1904 //
1905 //
1906 //
1907 //
1908 //
1909 //
1910 //
1911 //
1912 //
1913 //
1914 //
1915 //
1916 //
1917 //
1918 //
1919 //
1920 //
1921 //
1922 //
1923 //
1924 //
1925 //
1926 //
1927 //
1928 //
1929 //
1930 //
1931 //
1932 //
1933 //
1934 //
1935 //
1936 //
1937 //
1938 //
1939 //
1940 //
1941 //
1942 //
1943 //
1944 //
1945 //
1946 //
1947 //
1948 //
1949 //
1950 //
1951 //
1952 //
1953 //
1954 //
1955 //
1956 //
1957 //
1958 //
1959 //
1960 //
1961 //
1962 //
1963 //
1964 //
1965 //
1966 //
1967 //
1968 //
1969 //
1970 //
1971 //
1972 //
1973 //
1974 //
1975 //
1976 //
1977 //
1978 //
1979 //
1980 //
1981 //
1982 //
1983 //
1984 //
1985 //
1986 //
1987 //
1988 //
1989 //
1990 //
1991 //
1992 //
1993 //
1994 //
1995 //
1996 //
1997 //
1998 //
1999 //
2000 //
2001 //
2002 //
2003 //
2004 //
2005 //
2006 //
2007 //
2008 //
2009 //
2010 //
2011 //
2012 //
2013 //
2014 //
2015 //
2016 //
2017 //
2018 //
2019 //
2020 //
2021 //
2022 //
2023 //
2024 //
2025 //
2026 //
2027 //
2028 //
2029 //
2030 //
2031 //
2032 //
2033 //
2034 //
2035 //
2036 //
2037 //
2038 //
2039 //
2040 //
2041 //
2042 //
2043 //
2044 //
2045 //
2046 //
2047 //
2048 //
2049 //
2050 //
2051 //
2052 //
2053 //
2054 //
2055 //
2056 //
2057 //
2058 //
2059 //
2060 //
2061 //
2062 //
2063 //
2064 //
2065 //
2066 //
2067 //
2068 //
2069 //
2070 //
2071 //
2072 //
2073 //
2074 //
2075 //
2076 //
2077 //
2078 //
2079 //
2080 //
2081 //
2082 //
2083 //
2084 //
2085 //
2086 //
2087 //
2088 //
2089 //
2090 //
2091 //
2092 //
2093 //
2094 //
2095 //
2096 //
2097 //
2098 //
2099 //
2100 //
2101 //
2102 //
2103 //
2104 //
2105 //
2106 //
2107 //
2108 //
2109 //
2110 //
2111 //
2112 //
2113 //
2114 //
2115 //
2116 //
2117 //
2118 //
2119 //
2120 //
2121 //
2122 //
2123 //
2124 //
2125 //
2126 //
2127 //
2128 //
2129 //
2130 //
2131 //
2132 //
2133 //
2134 //
2135 //
2136 //
2137 //
2138 //
2139 //
2140 //
2141 //
2142 //
2143 //
2144 //
2145 //
2146 //
2147 //
2148 //
2149 //
2150 //
2151 //
2152 //
2153 //
2154 //
2155 //
2156 //
2157 //
2158 //
2159 //
2160 //
2161 //
2162 //
2163 //
2164 //
2165 //
2166 //
2167 //
2168 //
2169 //
2170 //
2171 //
2172 //
2173 //
2174 //
2175 //
2176 //
2177 //
2178 //
2179 //
2180 //
2181 //
2182 //
2183 //
2184 //
2185 //
2186 //
2187 //
2188 //
2189 //
2190 //
2191 //
2192 //
2193 //
2194 //
2195 //
2196 //
2197 //
2198 //
2199 //
2200 //
2201 //
2202 //
2203 //
2204 //
2205 //
2206 //
2207 //
2208 //
2209 //
2210 //
2211 //
2212 //
2213 //
2214 //
2215 //
2216 //
2217 //
2218 //
2219 //
2220 //
2221 //
2222 //
2223 //
2224 //
2225 //
2226 //
2227 //
2228 //
2229 //
2230 //
2231 //
2232 //
2233 //
2234 //
2235 //
2236 //
2237 //
2238 //
2239 //
2240 //
2241 //
2242 //
2243 //
2244 //
2245 //
2246 //
2247 //
2248 //
2249 //
2250 //
2251 //
2252 //
2253 //
2254 //
2255 //
2256 //
2257 //
2258 //
2259 //
2260 //
2261 //
2262 //
2263 //
2264 //
2265 //
2266 //
2267 //
2268 //
2269 //
2270 //
2271 //
2272 //
2273 //
2274 //
2275 //
2276 //
2277 //
2278 //
2279 //
2280 //
2281 //
2282 //
2283 //
2284 //
2285 //
2286 //
2287 //
2288 //
2289 //
2290 //
2291 //
2292 //
2293 //
2294 //
2295 //
2296 //
2297 //
2298 //
2299 //
2300 //
2301 //
2302 //
2303 //
2304 //
2305 //
2306 //
2307 //
2308 //
2309 //
2310 //
2311 //
2312 //
2313 //
2314 //
2315 //
2316 //
2317 //
2318 //
2319 //
2320 //
2321 //
2322 //
2323 //
2324 //
2325 //
2326 //
2327 //
2328 //
2329 //
2330 //
2331 //
2332 //
2333 //
2334 //
2335 //
2336 //
2337 //
2338 //
2339 //
2340 //
2341 //
2342 //
2343 //
2344 //
2345 //
2346 //
2347 //
2348 //
2349 //
2350 //
2351 //
2352 //
2353 //
2354 //
2355 //
2356 //
2357 //
2358 //
2359 //
2360 //
2361 //
2362 //
2363 //
2364 //
2365 //
2366 //
2367 //
2368 //
2369 //
2370 //
2371 //
2372 //
2373 //
2374 //
2375 //
2376 //
2377 //
2378 //
2379 //
2380 //
2381 //
2382 //
2383 //
2384 //
2385 //
2386 //
2387 //
2388 //
2389 //
2390 //
2391 //
2392 //
2393 //
2394 //
2395 //
2396 //
2397 //
2398 //
2399 //
2400 //
2401 //
2402 //
2403 //
2404 //
2405 //
2406 //
2407 //
2408 //
2409 //
2410 //
2411 //
2412 //
2413 //
2414 //
2415 //
2416 //
2417 //
2418 //
2419 //
2420 //
2421 //
2422 //
2423 //
2424 //
2425 //
2426 //
2427 //
2428 //
2429 //
2430 //
2431 //
2432 //
2433 //
2434 //
2435 //
2436 //
2437 //
2438 //
2439 //
2440 //
2441 //
2442 //
2443 //
2444 //
2445 //
2446 //
2447 //
2448 //
2449 //
2450 //
2451 //
2452 //
2453 //
2454 //
2455 //
2456 //
2457 //
2458 //
2459 //
2460 //
2461 //
2462 //
2463 //
2464 //
2465 //
2466 //
2467 //
2468 //
2469 //
2470 //
2471 //
2472 //
2473 //
2474 //
2475 //
2476 //
2477 //
2478 //
2479 //
2480 //
2481 //
2482 //
2483 //
2484 //
2485 //
2486 //
2487 //
2488 //
2489 //
2490 //
2491 //
2492 //
2493 //
2494 //
2495 //
2496 //
2497 //
2498 //
2499 //
2500 //
2501 //
2502 //
2503 //
2504 //
2505 //
2506 //
2507 //
2508 //
2509 //
2510 //
2511 //
2512 //
2513 //
2514 //
2515 //
2516 //
2517 //
2518 //
2519 //
2520 //
2521 //
2522 //
2523 //
2524 //
2525 //
2526 //
2527 //
2528 //
2529 //
2530 //
2531 //
2532 //
2533 //
2534 //
2535 //
2536 //
2537 //
2538 //
2539 //
2540 //
2541 //
2542 //
2543 //
2544 //
2545 //
2546 //
2547 //
2548 //
2549 //
2550 //
2551 //
2552 //
2553 //
2554 //
2555 //
2556 //
2557 //
2558 //
2559 //
2560 //
2561 //
2562 //
2563 //
2564 //
2565 //
2566 //
2567 //
2568 //
2569 //
2570 //
2571 //
2572 //
2573 //
2574 //
2575 //
2576 //
2577 //
2578 //
2579 //
2580 //
2581 //
2582 //
2583 //
2584 //
2585 //
2586 //
2587 //
2588 //
2589 //
2590 //
2591 //
2592 //
2593 //
2594 //
2595 //
2596 //
2597 //
2598 //
2599 //
2600 //
2601 //
2602 //
2603 //
2604 //
2605 //
2606 //
2607 //
2608 //
2609 //
2610 //
2611 //
2612 //
2613 //
2614 //
2615 //
2616 //
2617 //
2618 //
2619 //
2620 //
2621 //
2622 //
2623 //
2624 //
2625 //
2626 //
2627 //
2628 //
2629 //
2630 //
2631 //
2632 //
2633 //
2634 //
2635 //
2636 //
2637 //
2638 //
2639 //
2640 //
2641 //
2642 //
2643 //
2644 //
2645 //
2646 //
2647 //
2648 //
2649 //
2650 //
2651 //
2652 //
2653 //
2654 //
2655 //
2656 //
2657 //
2658 //
2659 //
2660 //
2661 //
2662 //
2663 //
2664 //
2665 //
2666 //
2667 //
2668 //
2669 //
2670 //
2671 //
2672 //
2673 //
2674 //
2675 //
2676 //
2677 //
2678 //
2679 //
2680 //
2681 //
2682 //
2683 //
2684 //
2685 //
2686 //
2687 //
2688 //
2689 //
2690 //
2691 //
2692 //
2693 //
2694 //
2695 //
2696 //
2697 //
2698 //
2699 //
2700 //
2701 //
2702 //
2703 //
2704 //
2705 //
2706 //
2707 //
2708 //
2709 //
2710 //
2711 //
2712 //
2713 //
2714 //
2715 //
2716 //
2717 //
2718 //
2719 //
2720 //
2721 //
2722 //
2723 //
2724 //
2725 //
2726 //
2727 //
2728 //
2729 //
2730 //
2731 //
2732 //
2733 //
2734 //
2735 //
2736 //
2737 //
2738 //
2739 //
2740 //
2741 //
2742 //

```



```

1148 template <int _AP_LW1, typename _AP_LT1, int _AP_LW2, type-
name _AP_LT2, \
1149 int _AP_RW1, typename _AP_RT1, int _AP_RW2, type-
name _AP_RT2> \
1150 INLINE typename ap_int_base<_AP_LW1 + _AP_LW2,
false>::RType< \
1151 _AP_RW1 + _AP_RW2,
false>::RType
1152 operator
BIN_OP( \
1153 const ap_concat_ref<_AP_LW1, _AP_LT1, _AP_LW2,
_AP_LT2>& lhs, \
1154 const ap_concat_ref<_AP_RW1, _AP_RT1, _AP_RW2,
_AP_RT2>& rhs) { \
1155 return lhs.get() BIN_OP
rhs.get(); \
1156 }
1157
1158 SYN_CONCAT_REF_BIN_OP(+, plus)
1159 SYN_CONCAT_REF_BIN_OP(-, minus)
1160 SYN_CONCAT_REF_BIN_OP(*, mult)
1161 SYN_CONCAT_REF_BIN_OP(/, div)
1162 SYN_CONCAT_REF_BIN_OP(% , mod)
1163 SYN_CONCAT_REF_BIN_OP(&, logic)
1164 SYN_CONCAT_REF_BIN_OP(|, logic)
1165 SYN_CONCAT_REF_BIN_OP(^, logic)
1166 SYN_CONCAT_REF_BIN_OP(>>, arg1)
1167 SYN_CONCAT_REF_BIN_OP(<<, arg1)
1168
1169 #undef SYN_CONCAT_REF_BIN_OP
1170
1171 #define CONCAT_OP_WITH_INT(C_TYPE, _AP_WI,
_AP_SI) \
1172 template <int _AP_W, bool
_AP_S> \
1173 INLINE ap_int_base<_AP_W + _AP_WI, false> operator,
( \
1174 const ap_int_base<_AP_W, _AP_S> &op1, C_TYPE op2) \
{ \
1175 ap_int_base<_AP_WI + _AP_W, false> \
val(op2); \
1176 ap_int_base<_AP_WI + _AP_W, false> \
ret(op1); \
1177 ret <= \
_AP_WI; \
1178 if (_AP_SI) \
{ \
1179 val <= \
_AP_W; \
1180 val >= \
_AP_W; \
1181 } \
1182 ret |= \
val; \
1183 return \
ret; \
1184 } \
\
1185 template <int _AP_W, bool
_AP_S> \
1186 INLINE ap_int_base<_AP_W + _AP_WI, false> operator,
( \
1187 C_TYPE op1, const ap_int_base<_AP_W, _AP_S> &op2) \
{ \
1188 ap_int_base<_AP_WI + _AP_W, false> \
val(op1); \
1189 ap_int_base<_AP_WI + _AP_W, false> \
ret(op2); \
1190 if (_AP_S) \
{ \
1191 ret <= \
_AP_WI; \
1192 ret >= \
_AP_WI; \
1193 } \
1194 ret |= val << \
_AP_W; \
1195 return \
ret; \
1196 } \
\
1197 template <int _AP_W, bool
_AP_S> \
1198 INLINE ap_int_base<_AP_W + _AP_WI, false> operator,
( \
1199 const ap_range_ref<_AP_W, _AP_S> &op1, C_TYPE op2) \
{ \
1200 ap_int_base<_AP_WI + _AP_W, false> \
val(op2); \
1201 ap_int_base<_AP_WI + _AP_W, false> \
ret(op1); \
1202 ret <= \
_AP_WI; \
1203 if (_AP_SI)

```

```

{ \
1204 val <= \
_AP_W; \
1205 val >= \
_AP_W; \
1206 } \
\
1207 ret |= \
val; \
1208 return \
ret; \
1209 } \
\
1210 template <int _AP_W, bool
_AP_S> \
1211 INLINE ap_int_base<_AP_W + _AP_WI, false> operator,
( \
1212 C_TYPE op1, const ap_range_ref<_AP_W, _AP_S> &op2) \
{ \
1213 ap_int_base<_AP_WI + _AP_W, false> \
val(op1); \
1214 ap_int_base<_AP_WI + _AP_W, false> \
ret(op2); \
1215 int len = \
op2.length(); \
1216 val <= \
len; \
1217 ret |= \
val; \
1218 return \
ret; \
1219 } \
\
1220 template <int _AP_W, bool
_AP_S> \
1221 INLINE ap_int_base<_AP_WI + 1, false> operator,
( \
1222 const ap_bit_ref<_AP_W, _AP_S> &op1, C_TYPE op2) \
{ \
1223 ap_int_base<_AP_WI + 1, false> \
val(op2); \
1224 val[_AP_WI] = \
op1; \
1225 return \
val; \
1226 } \
\
1227 template <int _AP_W, bool
_AP_S> \
1228 INLINE ap_int_base<_AP_WI + 1, false> operator,
( \
1229 C_TYPE op1, const ap_bit_ref<_AP_W, _AP_S> &op2) \
{ \
1230 ap_int_base<_AP_WI + 1, false> \
val(op1); \
1231 val <= \
1; \
1232 val[0] = \
op2; \
1233 return \
val; \
1234 } \
\
1235 template <int _AP_W, typename _AP_T, int _AP_W2, typename
_AP_T2> \
1236 INLINE ap_int_base<_AP_W + _AP_W2 + _AP_WI, false> operat-
or, ( \
1237 const ap_concat_ref<_AP_W, _AP_T, _AP_W2, _AP_T2> &op1,
C_TYPE op2) { \
1238 ap_int_base<_AP_WI + _AP_W + _AP_W2, _AP_SI> \
val(op2); \
1239 ap_int_base<_AP_WI + _AP_W + _AP_W2, _AP_SI> \
ret(op1); \
1240 if (_AP_SI) \
{ \
1241 val <= _AP_W + \
_AP_W2; \
1242 val >= _AP_W + \
_AP_W2; \
1243 } \
1244 ret <= \
_AP_WI; \
1245 ret |= \
val; \
1246 return \
ret; \
1247 } \
\
1248 template <int _AP_W, typename _AP_T, int _AP_W2, typename
_AP_T2> \
1249 INLINE ap_int_base<_AP_W + _AP_W2 + _AP_WI, false> operat-

```

```

or, (
1250     C_TYPE op1, const ap_concat_ref<_AP_W, _AP_T, _AP_W2,
_AP_T2> &op2) { \
1251     ap_int_base<_AP_WI + _AP_W + _AP_W2, _AP_SI>
val(op1);
1252     ap_int_base<_AP_WI + _AP_W + _AP_W2, _AP_SI>
ret(op2);
1253     int len =
op2.length(); \
1254     val <=
len; \
1255     ret |=
val; \
1256     return
ret; \
1257 }

1258 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
1259     ap_o_mode _AP_0, int
_AP_N>
1260     INLINE ap_int_base<_AP_W + _AP_WI, false> operator,
(
1261     const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N> &op1, \
1262     C_TYPE op2) \
{
1263     ap_int_base<_AP_WI + _AP_W, false>
val(op2);
1264     ap_int_base<_AP_WI + _AP_W, false>
ret(op1);
1265     if (_AP_SI) \
{
1266         val <=
_AP_W; \
1267         val >=
_AP_W; \
1268     } \
1269     ret <=
_AP_WI; \
1270     ret |=
val; \
1271     return
ret; \
1272 }

1273 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
1274     ap_o_mode _AP_0, int
_AP_N>
1275     INLINE ap_int_base<_AP_W + _AP_WI, false> operator,
(
1276     C_TYPE
op1,
1277     const af_range_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N> &op2) { \
1278     ap_int_base<_AP_WI + _AP_W, false>
val(op1);
1279     ap_int_base<_AP_WI + _AP_W, false>
ret(op2);
1280     int len =
op2.length(); \
1281     val <=
len; \
1282     ret |=
val; \
1283     return
ret; \
1284 }

1285 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
1286     ap_o_mode _AP_0, int
_AP_N>
1287     INLINE ap_int_base<1 + _AP_WI, false> operator,
(
1288     const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N> &op1, \
1289     C_TYPE op2) \
{
1290     ap_int_base<_AP_WI + 1, _AP_SI>
val(op2);
1291     val[_AP_WI] =
op1; \
1292     return
val; \
1293 }

1294 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
_AP_Q,
1295     ap_o_mode _AP_0, int
_AP_N>
1296     INLINE ap_int_base<1 + _AP_WI, false> operator,
(

```

```

1297     C_TYPE
op1,
1298     const af_bit_ref<_AP_W, _AP_I, _AP_S, _AP_Q, _AP_0,
_AP_N> &op2) { \
1299     ap_int_base<_AP_WI + 1, _AP_SI>
val(op1);
1300     val <=
1; \
1301     val[0] =
op2; \
1302     return
val; \
1303 }
1304
1305 CONCAT_OP_WITH_INT(bool, 1, false)
1306 CONCAT_OP_WITH_INT(char, 8, CHAR_IS_SIGNED)
1307 CONCAT_OP_WITH_INT(signed char, 8, true)
1308 CONCAT_OP_WITH_INT(unsigned char, 8, false)
1309 CONCAT_OP_WITH_INT(short, _AP_SIZE_short, true)
1310 CONCAT_OP_WITH_INT(unsigned short, _AP_SIZE_short, false)
1311 CONCAT_OP_WITH_INT(int, _AP_SIZE_int, true)
1312 CONCAT_OP_WITH_INT(unsigned int, _AP_SIZE_int, false)
1313 CONCAT_OP_WITH_INT(long, _AP_SIZE_long, true)
1314 CONCAT_OP_WITH_INT(unsigned long, _AP_SIZE_long, false)
1315 CONCAT_OP_WITH_INT(ap_slong, _AP_SIZE_ap_slong, true)
1316 CONCAT_OP_WITH_INT(ap_ulong, _AP_SIZE_ap_slong, false)
1317
1318 #undef CONCAT_OP_WITH_INT
1319
1320 #define CONCAT_SHIFT_WITH_INT(C_TYPE,
OP)
1321     template <int _AP_W, typename _AP_T, int _AP_W1, typename
_AP_T1> \
1322     INLINE ap_uint<_AP_W + _AP_W1> operator
OP(
1323     const ap_concat_ref<_AP_W, _AP_T, _AP_W1, _AP_T1> lhs,
C_TYPE rhs) { \
1324     return ap_uint<_AP_W + _AP_W1>(lhs.get()) OP
int(rhs);
1325 }
1326
1327 // FIXME int(rhs) may loose precision.
1328
1329 CONCAT_SHIFT_WITH_INT(int, <<)
1330 CONCAT_SHIFT_WITH_INT(unsigned int, <<)
1331 CONCAT_SHIFT_WITH_INT(long, <<)
1332 CONCAT_SHIFT_WITH_INT(unsigned long, <<)
1333 CONCAT_SHIFT_WITH_INT(ap_slong, <<)
1334 CONCAT_SHIFT_WITH_INT(ap_ulong, <<)
1335
1336 CONCAT_SHIFT_WITH_INT(int, >>)
1337 CONCAT_SHIFT_WITH_INT(unsigned int, >>)
1338 CONCAT_SHIFT_WITH_INT(long, >>)
1339 CONCAT_SHIFT_WITH_INT(unsigned long, >>)
1340 CONCAT_SHIFT_WITH_INT(ap_slong, >>)
1341 CONCAT_SHIFT_WITH_INT(ap_ulong, >>)
1342
1343 #endif // ifndef __cplusplus
1344 #endif // ifndef __AP_INT_REF_H__
1345
1346 // -*- cpp -*-

```

E.11 vendor/ap_int_special.h (223 lines, 6,301 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
5  *  * you may not use this file except in compliance with the
License.
6  *  * You may obtain a copy of the License at
7  *
8  *      http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-
missions and
14 * limitations under the License.
15 */
16
17 #ifndef __AP_INT_SPECIAL_H__
18 #define __AP_INT_SPECIAL_H__
19
20 #ifndef __AP_INT_H__
21 #error "Only ap_fixed.h and ap_int.h can be included directly in
user code."
22 #endif
23
24 #ifndef __SYNTHESIS__
25 #include <cstdio>
26 #include <cstdlib>

```

```

27 #endif
28 // FIXME AP_AUTOCC cannot handle many standard headers, so
declare instead of
29 // include.
30 // #include <complex>
31 namespace std {
32 template<typename _Tp> class complex;
33 }
34
35 /*
36 TODO: Modernize the code using C++11/C++14
37 1. constexpr http://www.open-std.org/jtc1/sc22/wg21/docs/
papers/2016/p0415r0.html
38 2. move constructor
39 */
40
41 namespace std {
42 /*
43 Specialize std::complex<ap_int> to zero initialization
ap_int.
44
45 To reduce the area cost, ap_int is not zero initialized,
just like basic
46 types float or double. However, libstdc++ provides spe-
cialization for float,
47 double and long double, initializing image part to 0 when
not specified.
48
49 This has become a difficulty in switching legacy code from
these C types to
50 ap_int. To ease the tranform of legacy code, we have to
implement
51 specialization of std::complex<> for our type.
52
53 As ap_int is a template, it is impossible to specialize
only the methods
54 that causes default initialization of value type in
std::complex<>. An
55 explicit full specialization of the template class has to
be done, covering
56 all the member functions and operators of std::complex<>
as specified
57 in standard 26.2.4 and 26.2.5.
58 */
59 template <int _AP_W>
60 class complex<ap_int<_AP_W> > {
61 public:
62 typedef ap_int<_AP_W> _Tp;
63 typedef _Tp value_type;
64
65 // 26.2.4/1
66 // Constructor without argument
67
68 // Default initialize, so that in dataflow, the variable is
only written once.
69 complex() : _M_real(_Tp()), _M_imag(_Tp()) {}
70 // Constructor with ap_int.
71 // Zero initialize image part when not specified, so that
`C(1,0)`
72 complex(const _Tp &_r, const _Tp &_i = _Tp(0))
73 : _M_real(_r), _M_imag(_i) {}
74 // Constructor with another complex number
75 template <typename _Up>
76 complex(const complex<_Up> &_z) : _M_real(_z.real()),
_M_imag(_z.imag()) {}
77
78 #if __cplusplus >= 201103L
79 const _Tp& real() const { return _M_real; }
80 const _Tp& imag() const { return _M_imag; }
81 #else
82 _Tp& real() { return _M_real; }
83 const _Tp& real() const { return _M_real; }
84 _Tp& imag() { return _M_imag; }
85 const _Tp& imag() const { return _M_imag; }
86 #endif
87
88 void real(_Tp __val) { _M_real = __val; }
89 void imag(_Tp __val) { _M_imag = __val; }
90
91 // Assign this complex number with ap_int.
92 // Zero initialize image poarrt, so that `C c; c = 1; c ==
C(1,0);`
93
94 complex<_Tp> &operator=(const _Tp __t) {
95     _M_real = __t;
96     _M_imag = _Tp(0);
97     return *this;
98 }
99
100 // 26.2.5/1
101 // Add ap_int to this complex number.
102 complex<_Tp> &operator+=(const _Tp __t) {
103     _M_real += __t;
104     return *this;
105 }
106
107 // 26.2.5/3
108 // Subtract ap_int from this complex number.

```

```

109 complex<_Tp> &operator-=(const _Tp __t) {
110     _M_real -= __t;
111     return *this;
112 }
113
114 // 26.2.5/5
115 // Multiply this complex number by ap_int.
116 complex<_Tp> &operator*=(const _Tp __t) {
117     _M_real *= __t;
118     _M_imag *= __t;
119     return *this;
120 }
121
122 // 26.2.5/7
123 // Divide this complex number by ap_int.
124 complex<_Tp> &operator/=(const _Tp __t) {
125     _M_real /= __t;
126     _M_imag /= __t;
127     return *this;
128 }
129
130 // Assign complex number to this complex number.
131 template <typename _Up>
132 complex<_Tp> &operator=(const complex<_Up> &_z) {
133     _M_real = _z.real();
134     _M_imag = _z.imag();
135     return *this;
136 }
137
138 // 26.2.5/9
139 // Add complex number to this.
140 template <typename _Up>
141 complex<_Tp> &operator+=(const complex<_Up> &_z) {
142     _M_real += _z.real();
143     _M_imag += _z.imag();
144     return *this;
145 }
146
147 // 26.2.5/11
148 // Subtract complex number from this.
149 template <typename _Up>
150 complex<_Tp> &operator-=(const complex<_Up> &_z) {
151     _M_real -= _z.real();
152     _M_imag -= _z.imag();
153     return *this;
154 }
155
156 // 26.2.5/13
157 // Multiply this by complex number.
158 template <typename _Up>
159 complex<_Tp> &operator*=(const complex<_Up> &_z) {
160     const _Tp _r = _M_real * _z.real() - _M_imag *
_z.imag();
161     _M_imag = _M_real * _z.imag() + _M_imag * _z.real();
162     _M_real = _r;
163     return *this;
164 }
165
166 // 26.2.5/15
167 // Divide this by complex number.
168 template <typename _Up>
169 complex<_Tp> &operator/=(const complex<_Up> &_z) {
170     complex<_Tp> cj (_z.real(), -_z.imag());
171     complex<_Tp> a = (*this) * cj;
172     complex<_Tp> b = cj * _z;
173     _M_real = a.real() / b.real();
174     _M_imag = a.imag() / b.real();
175     return *this;
176 }
177
178 private:
179 _Tp _M_real;
180 _Tp _M_imag;
181 }; // class complex<ap_int<_AP_W> >
182
183
184
185 /*
186 Non-member operations
187 These operations are not required by standard in 26.2.6,
but libstdc++
188 defines them for
189 float, double or long double's specialization.
190 */
191 // Compare complex number with ap_int.
192 template <int _AP_W>
193 inline bool operator==(const complex<ap_int<_AP_W> > &_x,
const ap_int<_AP_W> &_y) {
194     return _x.real() == _y &&
195         _x.imag() == 0;
196 }
197
198 // Compare ap_int with complex number.
199 template <int _AP_W>
200 inline bool operator==(const ap_int<_AP_W> &_x, const com-
plex<ap_int<_AP_W> > &_y) {
201     return _x == _y.real() &&
202         0 == _y.imag();
203 }
204

```

```

205 // Compare complex number with ap_int.
206 template <int _AP_W>
207 inline bool operator!=(const complex<ap_int<_AP_W> > &_x,
const ap_int<_AP_W> &_y) {
208     return _x.real() != _y ||
209         _x.imag() != 0;
210 }
211
212 // Compare ap_int with complex number.
213 template <int _AP_W>
214 inline bool operator!=(const ap_int<_AP_W> &_x, const com-
plex<ap_int<_AP_W> > &_y) {
215     return _x != _y.real() ||
216         0 != _y.imag();
217 }
218
219 } // namespace std
220
221 #endif // ifndef __AP_INT_SPECIAL_H__
222
223 // -*- cpp -*-

```

E.12 vendor/etc/ap_decl.h (201 lines, 6,119 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4
5 #ifndef __AP_DECL_H__
6 #define __AP_DECL_H__
7
8 //
-----
9
10 #if !defined(__AP_FIXED_H__) && !defined(__AP_INT_H__) && !
defined(__AUTOPILOT_CBE_H__) && !defined(__HLS_HALF_H__) && !
defined(__AP_FLOAT_H__)
11 #error "Only ap_fixed.h and ap_int.h can be included directly in
user code."
12 #endif
13
14 // Test __SYNTHESIS__ only for mode
15 #if !defined(__SYNTHESIS__) && (defined(AESL_SYN) ||
defined(__HLS_SYN__))
16 // #pragma message "AESL_SYN and __HLS_SYN__ should be re-
placed by __SYNTHESIS__"
17 #define __SYNTHESIS__
18 #endif
19
20 /* for safety*/
21 #if (defined(_AP_N) || defined(_AP_C))
22 #error One or more of the following is defined: _AP_N, _AP_C.
Definition conflicts with their usage as template parameters.
23 #endif
24
25 /* for safety*/
26 #if (defined(_AP_W) || defined(_AP_I) || defined(_AP_S) ||
defined(_AP_Q) || \
27     defined(_AP_0) || defined(_AP_W2) || defined(_AP_I2)
||
28     defined(_AP_S2) || defined(_AP_Q2) || defined(_AP_02)
||
29     defined(_AP_N) || defined(_AP_N2))
30 #error \
31     "One or more of the following is defined: _AP_W, _AP_I,
_AP_S, _AP_Q, _AP_0, _AP_N, _AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_02, _AP_N2. Definition conflicts with their usage as template
parameters."
32 #endif
33
34 /*for safety*/
35 #if (defined(_AP_W3) || defined(_AP_S3) || defined(_AP_W4) ||
defined(_AP_S4))
36 #error \
37     "One or more of the following is defined: _AP_W3, _AP_S3,
_AP_W4, _AP_S4. Definition conflicts with their usage as template
parameters."
38 #endif
39
40 #if (defined(_AP_W1) || defined(_AP_S1) || defined(_AP_T) ||
\
41     defined(_AP_T1) || defined(_AP_T2) || defined(_AP_T3) ||
defined(_AP_T4))
42 #error \
43     "One or more of the following is defined: _AP_W1, _AP_S1,
_AP_T, _AP_T1, _AP_T2, _AP_T3, _AP_T4. Definition conflicts with
their usage as template parameters."
44 #endif
45
46 #ifndef __cplusplus
47 #error "AP data type can only be used in C++"
48 #endif

```

```

49
50 //
-----
51
52 #ifndef __SC_COMPATIBLE__
53 /// ap_fixed quantification mode
54 enum ap_q_mode {
55     AP_RND, //< rounding to plus infinity
56     AP_RND_ZERO, //< rounding to zero
57     AP_RND_MIN_INF, //< rounding to minus infinity
58     AP_RND_INF, //< rounding to infinity
59     AP_RND_CONV, //< convergent rounding
60     AP_TRN, //< truncation
61     AP_TRN_ZERO, //< truncation to zero
62 };
63
64 // FIXME for legacy code
65 #ifndef SYSTEMC_INCLUDED
66 #define SC_RND AP_RND
67 #define SC_RND_ZERO AP_RND_ZERO
68 #define SC_RND_MIN_INF AP_RND_MIN_INF
69 #define SC_RND_INF AP_RND_INF
70 #define SC_RND_CONV AP_RND_CONV
71 #define SC_TRN AP_TRN
72 #define SC_TRN_ZERO AP_TRN_ZERO
73 #endif // !defined(SYSTEMC_INCLUDED)
74
75 /// ap_fixed saturation mode
76 enum ap_o_mode {
77     AP_SAT, //< saturation
78     AP_SAT_ZERO, //< saturation to zero
79     AP_SAT_SYM, //< symmetrical saturation
80     AP_WRAP, //< wrap-around (*)
81     AP_WRAP_SM, //< sign magnitude wrap-around (*)
82 };
83
84 // FIXME for legacy code
85 #ifndef SYSTEMC_INCLUDED
86 #define SC_SAT AP_SAT
87 #define SC_SAT_ZERO AP_SAT_ZERO
88 #define SC_SAT_SYM AP_SAT_SYM
89 #define SC_WRAP AP_WRAP
90 #define SC_WRAP_SM AP_WRAP_SM
91 #endif // !defined(SYSTEMC_INCLUDED)
92
93 #else // defined(__SC_COMPATIBLE__)
94
95 // There will not be sc_fxdefs.h, and the emu should be
defined by ap_fixed.
96
97 /// ap_fixed quantification mode
98 enum ap_q_mode {
99     SC_RND, //< rounding to plus infinity
100     SC_RND_ZERO, //< rounding to zero
101     SC_RND_MIN_INF, //< rounding to minus infinity
102     SC_RND_INF, //< rounding to infinity
103     SC_RND_CONV, //< convergent rounding
104     SC_TRN, //< truncation
105     SC_TRN_ZERO, //< truncation to zero
106 };
107
108 #define AP_RND SC_RND
109 #define AP_RND_ZERO SC_RND_ZERO
110 #define AP_RND_MIN_INF SC_RND_MIN_INF
111 #define AP_RND_INF SC_RND_INF
112 #define AP_RND_CONV SC_RND_CONV
113 #define AP_TRN SC_TRN
114 #define AP_TRN_ZERO SC_TRN_ZERO
115
116 /// ap_fixed saturation mode
117 enum ap_o_mode {
118     SC_SAT, //< saturation
119     SC_SAT_ZERO, //< saturation to zero
120     SC_SAT_SYM, //< symmetrical saturation
121     SC_WRAP, //< wrap-around (*)
122     SC_WRAP_SM, //< sign magnitude wrap-around (*)
123 };
124
125 #define AP_SAT SC_SAT
126 #define AP_SAT_ZERO SC_SAT_ZERO
127 #define AP_SAT_SYM SC_SAT_SYM
128 #define AP_WRAP SC_WRAP
129 #define AP_WRAP_SM SC_WRAP_SM
130
131 #endif // defined(__SC_COMPATIBLE__)
132
133 template <int _AP_W, bool _AP_S>
134 struct ap_int_base;
135
136 template <int _AP_W>
137 struct ap_int;
138
139 template <int _AP_W>
140 struct ap_uint;
141
142 template <int _AP_W, bool _AP_S>
143 struct ap_range_ref;
144
145 template <int _AP_W, bool _AP_S>

```

```

146 struct ap_bit_ref;
147
148 template <int _AP_W1, typename _AP_T1, int _AP_W2, typename
_AP_T2>
149 struct ap_concat_ref;
150
151 template <int _AP_W, int _AP_I, bool _AP_S = true, ap_q_mode
_AP_Q = AP_TRN,
152         ap_o_mode _AP_O = AP_WRAP, int _AP_N = 0>
153 struct ap_fixed_base;
154
155 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q = AP_TRN,
156         ap_o_mode _AP_O = AP_WRAP, int _AP_N = 0>
157 struct ap_fixed;
158
159 template <int _AP_W, int _AP_I, ap_q_mode _AP_Q = AP_TRN,
160         ap_o_mode _AP_O = AP_WRAP, int _AP_N = 0>
161 struct ap_ufixed;
162
163 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
164         int _AP_N>
165 struct af_range_ref;
166
167 template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode _AP_Q,
ap_o_mode _AP_O,
168         int _AP_N>
169 struct af_bit_ref;
170
171 /// string base mode
172
173 enum BaseMode { AP_BIN = 2, AP_OCT = 8, AP_DEC = 10, AP_HEX =
16 };
174
175 #ifndef SYSTEMC_INCLUDED
176 #define SC_BIN 2
177 #define SC_OCT 8
178 #define SC_DEC 10
179 #define SC_HEX 16
180 #endif // !defined(SYSTEMC_INCLUDED)
181
182 // Alias C data types
183 #if defined(_MSC_VER) && _MSC_VER < 1900
184 // Legacy tool before Visual Studio 2015
185 typedef signed __int64 ap_slong;
186 typedef unsigned __int64 ap_ulong;
187 #else
188 typedef signed long long ap_slong;
189 typedef unsigned long long ap_ulong;
190 #endif // _MSC_VER
191
192 enum {
193     _AP_SIZE_char = 8,
194     _AP_SIZE_short = sizeof(short) * 8,
195     _AP_SIZE_int = sizeof(int) * 8,
196     _AP_SIZE_long = sizeof(long) * 8,
197     _AP_SIZE_ap_slong = sizeof(ap_slong) * 8
198 };
199 #endif // !defined(__AP_DECL_H__)
200
201 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689

```

E.13 vendor/etc/ap_private.h (7,199 lines, 261,465 B)

```

1 /*
2  * Copyright 2011-2019 Xilinx, Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
5  * you may not use this file except in compliance with the
License.
6  * You may obtain a copy of the License at
7  *
8  * http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing,
software
11 * distributed under the License is distributed on an "AS IS"
BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 * See the License for the specific language governing per-
missions and
14 * limitations under the License.
15 */
16
17 #ifndef __AP_PRIVATE_H__
18 #define __AP_PRIVATE_H__
19
20 // common macros and type declarations are now defined in
ap_common.h, and
21 // ap_private becomes part of it.
22 #ifndef __AP_COMMON_H__
23 #error "etc/ap_private.h cannot be included directly."
24 #endif
25

```

```

26 // forward declarations
27 //template <int _AP_W, bool _AP_S, bool _AP_C = _AP_W <= 64>
28 //class ap_private; // moved to ap_common.h
29 template <int _AP_W, bool _AP_S>
30 struct _private_range_ref;
31 template <int _AP_W, bool _AP_S>
32 struct _private_bit_ref;
33
34 // TODO clean up this part.
35 #ifndef LLVM_SUPPORT_MATHEXTRAS_H
36 #define LLVM_SUPPORT_MATHEXTRAS_H
37
38 #ifndef _MSC_VER
39 #if _MSC_VER <= 1500
40 typedef __int8 int8_t;
41 typedef unsigned __int8 uint8_t;
42 typedef __int16 int16_t;
43 typedef unsigned __int16 uint16_t;
44 typedef __int32 int32_t;
45 typedef unsigned __int32 uint32_t;
46 typedef __int64 int64_t;
47 typedef unsigned __int64 uint64_t;
48 #else
49 #include <stdint.h>
50 #endif
51 #else
52 #include <stdint.h>
53 #endif
54
55 #ifndef INLINE
56 #define INLINE inline
57 // Enable to debug ap_int/ap_fixed
58 // #define INLINE __attribute__((weak))
59 #endif
60
61 // NOTE: The following support functions use the _32/_64 ex-
tensions instead of
62 // type overloading so that signed and unsigned integers can
be used without
63 // ambiguity.
64 namespace AESL_std {
65 template <class DataType>
66 DataType INLINE min(DataType a, DataType b) {
67     return (a >= b) ? b : a;
68 }
69
70 template <class DataType>
71 DataType INLINE max(DataType a, DataType b) {
72     return (a >= b) ? a : b;
73 }
74 } // namespace AESL_std
75
76 // TODO clean up included headers.
77 #include <math.h>
78 #include <stdio.h>
79 #include <cassert>
80 #include <cstdlib>
81 #include <cstring>
82 #include <iomanip>
83 #include <limits>
84 #include <sstream>
85 #include <string>
86
87 namespace ap_private_ops {
88 // Hi_32 - This function returns the high 32 bits of a 64
bit value.
89 static INLINE uint32_t Hi_32(uint64_t Value) {
90     return static_cast<uint32_t>(Value >> 32);
91 }
92
93 // Lo_32 - This function returns the low 32 bits of a 64 bit
value.
94 static INLINE uint32_t Lo_32(uint64_t Value) {
95     return static_cast<uint32_t>(Value);
96 }
97
98 template <int _AP_W>
99 INLINE bool isNegative(const ap_private<_AP_W, false>& a) {
100     return false;
101 }
102
103 template <int _AP_W>
104 INLINE bool isNegative(const ap_private<_AP_W, true>& a) {
105     enum {
106         APINT_BITS_PER_WORD = 64,
107         _AP_N = (_AP_W + APINT_BITS_PER_WORD - 1) /
APINT_BITS_PER_WORD
108     };
109     static const uint64_t sign_mask = 1ULL << ((_AP_W - 1) %
APINT_BITS_PER_WORD);
110     return (sign_mask & a.get_pVal(_AP_N - 1)) != 0;
111 }
112
113 // CountLeadingZeros_32 - this function performs the plat-
form optimal form of
114 // counting the number of zeros from the most significant
bit to the first one
115 // bit. Ex. CountLeadingZeros_32(0x00F000FF) == 8.
116 // Returns 32 if the word is zero.

```

```

117 static INLINE unsigned CountLeadingZeros_32(uint32_t Value) {
118     unsigned Count; // result
119     #if __GNUC__ >= 4
120     // PowerPC is defined for __builtin_clz(0)
121     #if !defined(__ppc__) && !defined(__ppc64__)
122         if (Value == 0) return 32;
123     #endif
124     Count = __builtin_clz(Value);
125     #else
126     if (Value == 0) return 32;
127     Count = 0;
128     // bisection method for count leading zeros
129     for (unsigned Shift = 32 >> 1; Shift; Shift >= 1) {
130         uint32_t Tmp = (Value) >> (Shift);
131         if (Tmp) {
132             Value = Tmp;
133         } else {
134             Count |= Shift;
135         }
136     }
137 #endif
138     return Count;
139 }
140
141 /// CountLeadingZeros_64 - This function performs the plat-
form optimal form
142 /// of counting the number of zeros from the most significant
bit to the first
143 /// one bit (64 bit edition.)
144 /// Returns 64 if the word is zero.
145 static INLINE unsigned CountLeadingZeros_64(uint64_t Value) {
146     unsigned Count; // result
147     #if __GNUC__ >= 4
148     // PowerPC is defined for __builtin_clzll(0)
149     #if !defined(__ppc__) && !defined(__ppc64__)
150         if (!Value) return 64;
151     #endif
152     Count = __builtin_clzll(Value);
153     #else
154     if (sizeof(long) == sizeof(int64_t)) {
155         if (!Value) return 64;
156         Count = 0;
157         // bisection method for count leading zeros
158         for (unsigned Shift = 64 >> 1; Shift; Shift >= 1) {
159             uint64_t Tmp = (Value) >> (Shift);
160             if (Tmp) {
161                 Value = Tmp;
162             } else {
163                 Count |= Shift;
164             }
165         }
166     } else {
167         // get hi portion
168         uint32_t Hi = Hi_32(Value);
169         // if some bits in hi portion
170         if (Hi) {
171             // leading zeros in hi portion plus all bits in lo por-
tion
172             Count = CountLeadingZeros_32(Hi);
173         } else {
174             // get lo portion
175             uint32_t Lo = Lo_32(Value);
176             // same as 32 bit value
177             Count = CountLeadingZeros_32(Lo) + 32;
178         }
179     }
180 #endif
181     return Count;
182 }
183
184 /// CountTrailingZeros_64 - This function performs the plat-
form optimal form
185 /// of counting the number of zeros from the least signific-
ant bit to the first
186 /// one bit (64 bit edition.)
187 /// Returns 64 if the word is zero.
188 static INLINE unsigned CountTrailingZeros_64(uint64_t Value)
{
189     #if __GNUC__ >= 4
190     return (Value != 0) ? __builtin_ctzll(Value) : 64;
191     #else
192     static const unsigned Mod67Position[] = {
193
194         64, 0, 1, 39, 2, 15, 40, 23, 3, 12, 16, 59, 41, 19,
24, 54, 4,
195         64, 13, 10, 17, 62, 60, 28, 42, 30, 20, 51, 25, 44, 55,
47, 5, 32,
196         65, 38, 14, 22, 11, 58, 18, 53, 63, 9, 61, 27, 29, 50,
43, 46, 31,
197         37, 21, 57, 52, 8, 26, 49, 45, 36, 56, 7, 48, 35, 6,
34, 33, 0};
198     return Mod67Position[(uint64_t)(-(int64_t)Value & (in-
t64_t)Value) % 67];
199     #endif
200 }

201
202 /// CountPopulation_64 - this function counts the number of
set bits in a value,
203 /// (64 bit edition.)
204 static INLINE unsigned CountPopulation_64(uint64_t Value) {
205     #if __GNUC__ >= 4
206     return __builtin_popcountll(Value);
207     #else
208     uint64_t v = Value - (((Value) >> 1) &
0x5555555555555555ULL);
209     v = (v & 0x3333333333333333ULL) + (((v) >> 2) &
0x3333333333333333ULL);
210     v = (v + ((v) >> 4)) & 0x0F0F0F0F0F0F0F0FULL;
211     return unsigned((uint64_t)(v * 0x0101010101010101ULL) >>
56);
212 #endif
213 }
214
215 static INLINE uint32_t countLeadingOnes_64(uint64_t __V,
uint32_t skip) {
216     uint32_t Count = 0;
217     if (skip) (__V) <= (skip);
218     while (__V && (__V & (1ULL << 63))) {
219         Count++;
220         (__V) <= 1;
221     }
222     return Count;
223 }
224
225 static INLINE std::string oct2Bin(char oct) {
226     switch (oct) {
227         case '\0': {
228             return "";
229         }
230         case '.': {
231             return ".";
232         }
233         case '0': {
234             return "000";
235         }
236         case '1': {
237             return "001";
238         }
239         case '2': {
240             return "010";
241         }
242         case '3': {
243             return "011";
244         }
245         case '4': {
246             return "100";
247         }
248         case '5': {
249             return "101";
250         }
251         case '6': {
252             return "110";
253         }
254         case '7': {
255             return "111";
256         }
257     }
258     assert(0 && "Invalid character in digit string");
259     return "";
260 }
261
262 static INLINE std::string hex2Bin(char hex) {
263     switch (hex) {
264         case '\0': {
265             return "";
266         }
267         case '.': {
268             return ".";
269         }
270         case '0': {
271             return "0000";
272         }
273         case '1': {
274             return "0001";
275         }
276         case '2': {
277             return "0010";
278         }
279         case '3': {
280             return "0011";
281         }
282         case '4': {
283             return "0100";
284         }
285         case '5': {
286             return "0101";
287         }
288         case '6': {
289             return "0110";
290         }
291         case '7': {
292             return "0111";
293         }
294         case '8': {
295             return "1000";

```



```

296     }
297     case '9': {
298         return "1001";
299     }
300     case 'A':
301     case 'a': {
302         return "1010";
303     }
304     case 'B':
305     case 'b': {
306         return "1011";
307     }
308     case 'C':
309     case 'c': {
310         return "1100";
311     }
312     case 'D':
313     case 'd': {
314         return "1101";
315     }
316     case 'E':
317     case 'e': {
318         return "1110";
319     }
320     case 'F':
321     case 'f': {
322         return "1111";
323     }
324     }
325     assert(0 && "Invalid character in digit string");
326     return "";
327 }
328
329 static INLINE uint32_t decode_digit(char cdigit, int radix) {
330     uint32_t digit = 0;
331     if (radix == 16) {
332         #define isxdigit-
333         git(c)
334         (((c) >= '0' && (c) <= '9') || ((c) >= 'a' && (c) <= 'f')
335         || \
336         (((c) >= 'A' && (c) <= 'F'))
337         #define isdigit(c) ((c) >= '0' && (c) <= '9')
338         if (!isxdigit(cdigit)) assert(0 && "Invalid hex digit in
339 string");
340         if (isdigit(cdigit))
341             digit = cdigit - '0';
342         else if (cdigit >= 'a')
343             digit = cdigit - 'a' + 10;
344         else if (cdigit >= 'A')
345             digit = cdigit - 'A' + 10;
346         else
347             assert(0 && "huh? we shouldn't get here");
348     } else if (isdigit(cdigit)) {
349         digit = cdigit - '0';
350     } else {
351         assert(0 && "Invalid character in digit string");
352     }
353     #undef isxdigit
354     #undef isdigit
355     return digit;
356 }
357
358 // Determine the radix of "val".
359 static INLINE std::string parseString(const std::string& in-
360 put, unsigned char& radix) {
361     size_t len = input.length();
362     if (len == 0) {
363         if (radix == 0) radix = 10;
364         return input;
365     }
366     size_t startPos = 0;
367     // Trim whitespace
368     while (input[startPos] == ' ' && startPos < len) startPos+
369 +;
370     while (input[len - 1] == ' ' && startPos < len) len--;
371     std::string val = input.substr(startPos, len - startPos);
372     // std::cout << "val = " << val << "\n";
373     len = val.length();
374     startPos = 0;
375     // If the length of the string is less than 2, then radix
376     // is decimal and there is no exponent.
377     if (len < 2) {
378         if (radix == 0) radix = 10;
379         return val;
380     }
381     bool isNegative = false;
382     std::string ans;
383     // First check to see if we start with a sign indicator
384     if (val[0] == '-') {
385         ans = "-";
386         ++startPos;
387         isNegative = true;
388     } else if (val[0] == '+')
389         ++startPos;
390

```

```

391     if (len - startPos < 2) {
392         if (radix == 0) radix = 10;
393         return val;
394     }
395
396     if (val.substr(startPos, 2) == "0x" || val.substr(startPos,
397 2) == "0X") {
398         // If we start with "0x", then the radix is hex.
399         radix = 16;
400         startPos += 2;
401     } else if (val.substr(startPos, 2) == "0b" ||
402 val.substr(startPos, 2) == "0B") {
403         // If we start with "0b", then the radix is binary.
404         radix = 2;
405         startPos += 2;
406     } else if (val.substr(startPos, 2) == "0o" ||
407 val.substr(startPos, 2) == "0O") {
408         // If we start with "0o", then the radix is octal.
409         radix = 8;
410         startPos += 2;
411     } else if (radix == 0) {
412         radix = 10;
413     }
414     int exp = 0;
415     if (radix == 10) {
416         // If radix is decimal, then see if there is an
417         // exponent indicator.
418         size_t expPos = val.find('e');
419         bool has_exponent = true;
420         if (expPos == std::string::npos) expPos = val.find('E');
421         if (expPos == std::string::npos) {
422             // No exponent indicator, so the mantissa goes to the
423             expPos = len;
424             has_exponent = false;
425         }
426         // std::cout << "startPos = " << startPos << " " << exp-
427 Pos << "\n";
428         ans += val.substr(startPos, expPos - startPos);
429         if (has_exponent) {
430             // Parse the exponent.
431             std::istringstream iss(val.substr(expPos + 1, len -
432 expPos - 1));
433             iss >> exp;
434         } else {
435             // Check for a binary exponent indicator.
436             size_t expPos = val.find('p');
437             bool has_exponent = true;
438             if (expPos == std::string::npos) expPos = val.find('P');
439             if (expPos == std::string::npos) {
440                 // No exponent indicator, so the mantissa goes to the
441                 expPos = len;
442                 has_exponent = false;
443             }
444             // std::cout << "startPos = " << startPos << " " << exp-
445 Pos << "\n";
446             assert(startPos <= expPos);
447             // Convert to binary as we go.
448             for (size_t i = startPos; i < expPos; ++i) {
449                 if (radix == 16) {
450                     ans += hex2Bin(val[i]);
451                 } else if (radix == 8) {
452                     ans += oct2Bin(val[i]);
453                 } else { // radix == 2
454                     ans += val[i];
455                 }
456             }
457             // End in binary
458             radix = 2;
459             if (has_exponent) {
460                 // Parse the exponent.
461                 std::istringstream iss(val.substr(expPos + 1, len -
462 expPos - 1));
463                 iss >> exp;
464             }
465             if (exp == 0) return ans;
466             size_t decPos = ans.find('.');
467             if (decPos == std::string::npos) decPos = ans.length();
468             if ((int)decPos + exp >= (int)ans.length()) {
469                 int i = decPos;
470                 for (; i < (int)ans.length() - 1; ++i) ans[i] = ans[i +
471 1];
472                 for (; i < (int)ans.length(); ++i) ans[i] = '0';
473                 for (; i < (int)decPos + exp; ++i) ans += '0';
474                 return ans;
475             } else if ((int)decPos + exp < (int)isNegative) {
476                 std::string dupAns = "0.";
477                 if (ans[0] == '-') dupAns = "-0.";
478                 for (int i = 0; i < isNegative - (int)decPos - exp; ++i)
479                     dupAns += '0';
480                 for (size_t i = isNegative; i < ans.length(); ++i)

```

```

481     if (ans[i] != '.') dupAns += ans[i];
482     return dupAns;
483 }
484
485 if (exp > 0)
486     for (size_t i = decPos; i < decPos + exp; ++i) ans[i] =
ans[i + 1];
487 else {
488     if (decPos == ans.length()) ans += ' ';
489
490     for (int i = decPos; i > (int)decPos + exp; --i) ans[i] =
ans[i - 1];
491     ans[decPos + exp] = '.';
492     return ans;
493 }
494
495 /// sub_1 - This function subtracts a single "digit" (64-bit
word), y, from
496 /// the multi-digit integer array, x[], propagating the bor-
rowed 1 value until
497 /// no further borrowing is needed or it runs out of "di-
gits" in x. The result
498 /// is 1 if "borrowing" exhausted the digits in x, or 0 if x
was not exhausted.
499 /// In other words, if y > x then this function returns 1,
otherwise 0.
500 /// @returns the borrow out of the subtraction
501
502 static INLINE bool sub_1(uint64_t x[], uint32_t len, uint64_t
y) {
503     for (uint32_t i = 0; i < len; ++i) {
504         uint64_t __X = x[i];
505         x[i] -= y;
506         if (y > __X)
507             y = 1; // We have to "borrow 1" from next "digit"
508         else {
509             y = 0; // No need to borrow
510             break; // Remaining digits are unchanged so exit early
511         }
512     }
513     return (y != 0);
514 }
515
516 /// add_1 - This function adds a single "digit" integer, y,
to the multiple
517 /// "digit" integer array, x[]. x[] is modified to reflect
the addition and
518 /// 1 is returned if there is a carry out, otherwise 0 is re-
turned.
519 /// @returns the carry of the addition.
520 static INLINE bool add_1(uint64_t dest[], uint64_t x[],
uint32_t len,
521                          uint64_t y) {
522     for (uint32_t i = 0; i < len; ++i) {
523         dest[i] = y + x[i];
524         if (dest[i] < y)
525             y = 1; // Carry one to next digit.
526         else {
527             y = 0; // No need to carry so exit early
528             break;
529         }
530     }
531     return (y != 0);
532 }
533
534 /// add - This function adds the integer array x to the in-
teger array Y and
535 /// places the result in dest.
536 /// @returns the carry out from the addition
537 /// @brief General addition of 64-bit integer arrays
538 static INLINE bool add(uint64_t* dest, const uint64_t* x,
const uint64_t* y,
539                        uint32_t destlen, uint32_t xlen,
uint32_t ylen,
540                        bool xsigned, bool ysigned) {
541     uint32_t len = AESL_std::min(xlen, ylen);
542     uint32_t i;
543     for (i = 0; i < len && i < destlen; ++i) {
544         uint64_t limit =
545             AESL_std::min(x[i], y[i]); // must come first in case
dest == x
546         dest[i] = x[i] + y[i] + carry;
547         carry = dest[i] < limit || (carry && dest[i] == limit);
548     }
549     if (xlen > ylen) {
550         const uint64_t yext = ysigned && int64_t(y[ylen - 1]) <
0 ? -1 : 0;
551         for (i = ylen; i < xlen && i < destlen; ++i) {
552             uint64_t limit = AESL_std::min(x[i], yext);
553             dest[i] = x[i] + yext + carry;
554             carry = (dest[i] < limit) || (carry && dest[i] == lim-
it);
555         }
556     } else if (ylen > xlen) {
557         const uint64_t xext = xsigned && int64_t(x[xlen - 1]) <
0 ? -1 : 0;
558         for (i = xlen; i < ylen && i < destlen; ++i) {

```

```

559         uint64_t limit = AESL_std::min(xext, y[i]);
560         dest[i] = xext + y[i] + carry;
561         carry = (dest[i] < limit) || (carry && dest[i] == lim-
it);
562     }
563     return carry;
564 }
565
566 /// @returns returns the borrow out.
567 /// @brief Generalized subtraction of 64-bit integer arrays.
568 static INLINE bool sub(uint64_t* dest, const uint64_t* x,
const uint64_t* y,
569                        uint32_t destlen, uint32_t xlen,
uint32_t ylen,
570                        bool xsigned, bool ysigned) {
571     bool borrow = false;
572     uint32_t i;
573     uint32_t len = AESL_std::min(xlen, ylen);
574     for (i = 0; i < len && i < destlen; ++i) {
575         uint64_t x_tmp = borrow ? x[i] - 1 : x[i];
576         borrow = y[i] > x_tmp || (borrow && x[i] == 0);
577         dest[i] = x_tmp - y[i];
578     }
579     if (xlen > ylen) {
580         const uint64_t yext = ysigned && int64_t(y[ylen - 1]) <
0 ? -1 : 0;
581         for (i = ylen; i < xlen && i < destlen; ++i) {
582             uint64_t x_tmp = borrow ? x[i] - 1 : x[i];
583             borrow = yext > x_tmp || (borrow && x[i] == 0);
584             dest[i] = x_tmp - yext;
585         }
586     } else if (ylen > xlen) {
587         const uint64_t xext = xsigned && int64_t(x[xlen - 1]) <
0 ? -1 : 0;
588         for (i = xlen; i < ylen && i < destlen; ++i) {
589             uint64_t x_tmp = borrow ? xext - 1 : xext;
590             borrow = y[i] > x_tmp || (borrow && xext == 0);
591             dest[i] = x_tmp - y[i];
592         }
593     }
594     return borrow;
595 }
596
597 /// Subtracts the RHS ap_private from this ap_private
598 /// @returns this, after subtraction
599 /// @brief Subtraction assignment operator.
600
601 /// Multiplies an integer array, x by a uint64_t integer
and places the result
602 /// into dest.
603 /// @returns the carry out of the multiplication.
604 /// @brief Multiply a multi-digit ap_private by a single di-
git (64-bit) integer.
605 static INLINE uint64_t mul_1(uint64_t dest[], const uint64_t
x[], uint32_t len,
606                              uint64_t y) {
607     // Split y into high 32-bit part (hy) and low 32-bit part
(ly)
608     uint64_t ly = y & 0xffffffffULL, hy = (y) >> 32;
609     uint64_t carry = 0;
610     static const uint64_t two_power_32 = 1ULL << 32;
611     // For each digit of x.
612     for (uint32_t i = 0; i < len; ++i) {
613         // Split x into high and low words
614         uint64_t lx = x[i] & 0xffffffffULL;
615         uint64_t hx = (x[i]) >> 32;
616         // hasCarry - A flag to indicate if there is a carry to
the next digit.
617         // hasCarry == 0, no carry
618         // hasCarry == 1, has carry
619         // hasCarry == 2, no carry and the calculation result ==
0.
620         uint8_t hasCarry = 0;
621         dest[i] = carry + lx * ly;
622         // Determine if the add above introduces carry.
623         hasCarry = (dest[i] < carry) ? 1 : 0;
624         carry = hx * ly + ((dest[i]) >> 32) + (hasCarry ?
two_power_32 : 0);
625         // The upper limit of carry can be (2^32 - 1)(2^32 - 1) +
(2^32 - 1) + 2^32 = 2^64.
626         hasCarry = (!carry && hasCarry) ? 1 : (!carry ? 2 : 0);
627
628         carry += (lx * hy) & 0xffffffffULL;
629         dest[i] = ((carry) < 32) | (dest[i] & 0xffffffffULL);
630         carry = (((!carry && hasCarry != 2) || hasCarry == 1) ?
two_power_32 : 0) +
631             ((carry) >> 32) + ((lx * hy) >> 32) + hx * hy;
632     }
633     return carry;
634 }
635
636 /// Multiplies integer array x by integer array y and stores
the result into
637 /// the integer array dest. Note that dest's size must be >=
xlen + ylen in
638 /// order to
639 /// do a full precision computation. If it is not, then only
the low-order words
640 /// are returned.

```



```

643 /// @brief Generalized multiplication of integer arrays.
644 static INLINE void mul(uint64_t dest[], const uint64_t x[],
uint32_t xlen,
645                      const uint64_t y[], uint32_t ylen,
uint32_t destlen) {
646     assert(xlen > 0);
647     assert(ylen > 0);
648     assert(destlen >= xlen + ylen);
649     if (xlen < destlen) dest[xlen] = mul_1(dest, x, xlen,
y[0]);
650     for (uint32_t i = 1; i < ylen; ++i) {
651         uint64_t ly = y[i] & 0xffffffffFULL, hy = (y[i] >> 32);
652         uint64_t carry = 0, lx = 0, hx = 0;
653         for (uint32_t j = 0; j < xlen; ++j) {
654             lx = x[j] & 0xffffffffFULL;
655             hx = (x[j] >> 32);
656             // hasCarry - A flag to indicate if has carry.
657             // hasCarry == 0, no carry
658             // hasCarry == 1, has carry
659             // hasCarry == 2, no carry and the calculation result
== 0.
660             uint8_t hasCarry = 0;
661             uint64_t res = carry + lx * ly;
662             hasCarry = (res < carry) ? 1 : 0;
663             carry = (hasCarry ? (1ULL << 32) : 0) + hx * ly +
((res >> 32);
664             hasCarry = (!carry && hasCarry) ? 1 : (!carry ? 2 : 0);
665             carry += (lx * hy) & 0xffffffffFULL;
666             res = ((carry) << 32) | (res & 0xffffffffFULL);
667             if (i + j < destlen) dest[i + j] += res;
668             carry =
669                 (((!carry && hasCarry != 2) || hasCarry == 1) ?
(1ULL << 32) : 0) +
670                 ((carry) >> 32) + (dest[i + j] < res ? 1 : 0) +
((lx * hy) >> 32) +
671                 hx * hy;
672         }
673         if (i + xlen < destlen) dest[i + xlen] = carry;
674     }
675 }
676
677 /// Implementation of Knuth's Algorithm D (Division of non-
negative integers)
678 /// from "Art of Computer Programming, Volume 2", section
4.3.1, p. 272. The
679 /// variables here have the same names as in the algorithm.
Comments explain
680 /// the algorithm and any deviation from it.
681 static INLINE void KnuthDiv(uint32_t* u, uint32_t* v,
uint32_t* q, uint32_t* r,
682                      uint32_t m, uint32_t n) {
683     assert(u && "Must provide dividend");
684     assert(v && "Must provide divisor");
685     assert(q && "Must provide quotient");
686     assert(u != v && u != q && v != q && "Must use different
memory");
687     assert(n > 1 && "n must be > 1");
688
689     // Knuth uses the value b as the base of the number system.
In our case b
690     // is 2^31 so we just set it to -1u.
691     uint64_t b = uint64_t(1) << 32;
692
693     // DEBUG(cerr << "KnuthDiv: m=" << m << " n=" << n <<
'\n');
694     // DEBUG(cerr << "KnuthDiv: original:");
695     // DEBUG(for (int i = m+n; i >=0; i--) cerr << " " <<
std::setbase(16) <<
696         // u[i]);
697     // DEBUG(cerr << " by");
698     // DEBUG(for (int i = n; i >0; i--) cerr << " " <<
std::setbase(16) <<
699         // v[i-1]);
700     // DEBUG(cerr << '\n');
701     // D1. [Normalize.] Set d = b / (v[n-1] + 1) and multiply
all the digits of
702     // u and v by d. Note that we have taken Knuth's advice
here to use a power
703     // of 2 value for d such that d * v[n-1] >= b/2 (b is the
base). A power of
704     // 2 allows us to shift instead of multiply and it is easy
to determine the
705     // shift amount from the leading zeros. We are basically
normalizing the u
706     // and v so that its high bits are shifted to the top of
v's range without
707     // overflow. Note that this can require an extra word in u
so that u must
708     // be of length m+n+1.
709     uint32_t shift = CountLeadingZeros_32(v[n - 1]);
710     uint32_t v_carry = 0;
711     uint32_t u_carry = 0;
712     if (shift) {
713         for (uint32_t i = 0; i < m + n; ++i) {
714             uint32_t u_tmp = (u[i] >> (32 - shift));
715             u[i] = ((u[i] << (shift)) | u_carry;
716             u_carry = u_tmp;
717         }
718         for (uint32_t i = 0; i < n; ++i) {
719             uint32_t v_tmp = (v[i] >> (32 - shift));
720             v[i] = ((v[i] << (shift)) | v_carry;
721             v_carry = v_tmp;
722         }
723     }
724     u[m + n] = u_carry;
725     // DEBUG(cerr << "KnuthDiv: normal:");
726     // DEBUG(for (int i = m+n; i >=0; i--) cerr << " " <<
std::setbase(16) <<
727         // u[i]);
728     // DEBUG(cerr << " by");
729     // DEBUG(for (int i = n; i >0; i--) cerr << " " <<
std::setbase(16) <<
730         // v[i-1]);
731     // DEBUG(cerr << '\n');
732
733     // D2. [Initialize j.] Set j to m. This is the loop
counter over the places.
734     int j = m;
735     do {
736         // DEBUG(cerr << "KnuthDiv: quotient digit #" << j <<
'\n');
737         // D3. [Calculate q'.]
738         // Set qp = (u[j+n]*b + u[j+n-1]) / v[n-1].
(qp=qprime=q')
739         // Set rp = (u[j+n]*b + u[j+n-1]) % v[n-1].
(rp=rprime=r')
740         // Now test if qp == b or qp*v[n-2] > b*rp + u[j+n-2]; if
so, decrease
741         // qp by 1, increase rp by v[n-1], and repeat this test if
rp < b. The test
742         // on v[n-2] determines at high speed most of the cases
in which the trial
743         // value qp is one too large, and it eliminates all cases
where qp is two
744         // too large.
745         uint64_t dividend = ((uint64_t(u[j + n]) << 32) + u[j + n -
1]);
746         // DEBUG(cerr << "KnuthDiv: dividend == " << dividend <<
'\n');
747         uint64_t qp = dividend / v[n - 1];
748         uint64_t rp = dividend % v[n - 1];
749         if (qp == b || qp * v[n - 2] > b * rp + u[j + n - 2]) {
750             qp--;
751             rp += v[n - 1];
752             if (rp < b && (qp == b || qp * v[n - 2] > b * rp + u[j
+ n - 2])) qp--;
753         }
754         // DEBUG(cerr << "KnuthDiv: qp == " << qp << ", rp == "
<< rp << '\n');
755         // D4. [Multiply and subtract.] Replace
(u[j+n]u[j+n-1]...u[j]) with
756         // (u[j+n]u[j+n-1]...u[j]) - qp * (v[n-1]...v[1]v[0]).
This computation
757         // consists of a simple multiplication by a one-place
number, combined with
758         // a subtraction.
759         bool isNeg = false;
760         for (uint32_t i = 0; i < n; ++i) {
761             uint64_t u_tmp = uint64_t(u[j + i]) | ((uint64_t(u[j +
i + 1])) << 32);
762             uint64_t subtrahend = uint64_t(qp) * uint64_t(v[i]);
763             bool borrow = subtrahend > u_tmp;
764             //DEBUG(cerr << "KnuthDiv: u_tmp == " << u_tmp
<< ", subtrahend == " << subtrahend
<< ", borrow == " << borrow << '\n');*/
765             uint64_t result = u_tmp - subtrahend;
766             uint32_t k = j + i;
767             u[k++] = (uint32_t)(result & (b - 1)); // subtract low
word
768
769             u[k++] = (uint32_t)((result) >> 32); // subtract high
word
770             while (borrow && k <= m + n) { // deal with
borrow to the left
771                 borrow = u[k] == 0;
772                 u[k]--;
773                 k++;
774             }
775             isNeg |= borrow;
776
777             /*DEBUG(cerr << "KnuthDiv: u[j+i] == " << u[j+i] << ",
u[j+i+1] == " <<
u[j+i+1] << '\n');*/
778         }
779         /*DEBUG(cerr << "KnuthDiv: after subtraction:");
DEBUG(for (int i = m+n; i >=0; i--) cerr << " " <<
u[i]);
780         DEBUG(cerr << '\n');*/
781
782         // The digits (u[j+n]...u[j]) should be kept positive; if
the result of
783
784         // this step is actually negative, (u[j+n]...u[j]) should

```

be left as the

```

787 // true value plus b**(n+1), namely as the b's complement
of
788 // the true value, and a "borrow" to the left should be
remembered.
789 //
790 if (isNeg) {
791     bool carry = true; // true because b's complement is
"complement + 1"
792     for (uint32_t i = 0; i <= m + n; ++i) {
793         u[i] = ~u[i] + carry; // b's complement
794         carry = carry && u[i] == 0;
795     }
796 }
797 /*DEBUG(cerr << "KnuthDiv: after complement:");
798 DEBUG(for (int i = m+n; i >=0; i--) cerr << " " <<
u[i]);
799     DEBUG(cerr << '\n');*/
800
801 // D5. [Test remainder.] Set q[j] = qp. If the result of
step D4 was
802 // negative, go to step D6; otherwise go on to step D7.
803 q[j] = (uint32_t)qp;
804 if (isNeg) {
805     // D6. [Add back]. The probability that this step is
necessary is very
806     // small, on the order of only 2/b. Make sure that test
data accounts for
807     // this possibility. Decrease q[j] by 1
808     q[j]--;
809     // and add (0v[n-1]...v[1]v[0]) to
(u[j+n]u[j+n-1]...u[j+1]u[j]).
810     // A carry will occur to the left of u[j+n], and it
should be ignored
811     // since it cancels with the borrow that occurred in
D4.
812     bool carry = false;
813     for (uint32_t i = 0; i < n; i++) {
814         uint32_t limit = AESL_std::min(u[j + i], v[i]);
815         u[j + i] += v[i] + carry;
816         carry = u[j + i] < limit || (carry && u[j + i] ==
limit);
817     }
818     u[j + n] += carry;
819 }
820 /*DEBUG(cerr << "KnuthDiv: after correction:");
821 DEBUG(for (int i = m+n; i >=0; i--) cerr << " " <<
u[i]);
822     DEBUG(cerr << "\nKnuthDiv: digit result = " << q[j] <<
'\n');*/
823
824 // D7. [Loop on j.] Decrease j by one. Now if j >= 0, go
back to D3.
825 } while (--j >= 0);
826
827 /*DEBUG(cerr << "KnuthDiv: quotient:");
828 DEBUG(for (int i = m; i >=0; i--) cerr << " " << q[i]);
829 DEBUG(cerr << '\n');*/
830
831 // D8. [Unnormalize]. Now q[...] is the desired quotient,
and the desired
832 // remainder may be obtained by dividing u[...] by d. If r
is non-null we
833 // compute the remainder (urem uses this).
834 if (r) {
835     // The value d is expressed by the "shift" value above
since we avoided
836     // multiplication by d by using a shift left. So, all we
have to do is
837     // shift right here. In order to mak
838     if (shift) {
839         uint32_t carry = 0;
840         // DEBUG(cerr << "KnuthDiv: remainder:");
841         for (int i = n - 1; i >= 0; i--) {
842             r[i] = ((u[i] >> (shift)) | carry;
843             carry = (u[i] << (32 - shift));
844             // DEBUG(cerr << " " << r[i]);
845         }
846     } else {
847         for (int i = n - 1; i >= 0; i--) {
848             r[i] = u[i];
849             // DEBUG(cerr << " " << r[i]);
850         }
851     }
852     // DEBUG(cerr << '\n');
853 }
854 // DEBUG(cerr << std::setbase(10) << '\n');
855 }
856
857 template <int _AP_W, bool _AP_S>
858 void divide(const ap_private<_AP_W, _AP_S>& LHS, uint32_t
lhsWords,
859             const ap_private<_AP_W, _AP_S>& RHS, uint32_t
rhsWords,
860             ap_private<_AP_W, _AP_S>* Quotient,
861             ap_private<_AP_W, _AP_S>* Remainder) {
862     assert(lhsWords >= rhsWords && "Fractional result");

```

```

863     enum { APINT_BITS_PER_WORD = 64 };
864     // First, compose the values into an array of 32-bit words
instead of
865     // 64-bit words. This is a necessity of both the "short di-
vision" algorithm
866     // and the the Knuth "classical algorithm" which requires
there to be native
867     // operations for +, -, and * on an m bit value with an m*2
bit result. We
868     // can't use 64-bit operands here because we don't have
native results of
869     // 128-bits. Furthermore, casting the 64-bit values to 32-
bit values won't
870     // work on large-endian machines.
871     uint64_t mask = ~0ull >> (sizeof(uint32_t) * 8);
872     uint32_t n = rhsWords * 2;
873     uint32_t m = (lhsWords * 2) - n;
874
875     // Allocate space for the temporary values we need either
on the stack, if
876     // it will fit, or on the heap if it won't.
877     uint32_t SPACE[128];
878     uint32_t* __U = 0;
879     uint32_t* __V = 0;
880     uint32_t* __Q = 0;
881     uint32_t* __R = 0;
882     if ((Remainder ? 4 : 3) * n + 2 * m + 1 <= 128) {
883         __U = &SPACE[0];
884         __V = &SPACE[m + n + 1];
885         __Q = &SPACE[(m + n + 1) + n];
886         if (Remainder) __R = &SPACE[(m + n + 1) + n + (m + n)];
887     } else {
888         __U = new uint32_t[m + n + 1];
889         __V = new uint32_t[n];
890         __Q = new uint32_t[m + n];
891         if (Remainder) __R = new uint32_t[n];
892     }
893
894     // Initialize the dividend
895     memset(__U, 0, (m + n + 1) * sizeof(uint32_t));
896     for (unsigned i = 0; i < lhsWords; ++i) {
897         uint64_t tmp = LHS.get_pVal(i);
898         __U[i * 2] = (uint32_t)(tmp & mask);
899         __U[i * 2 + 1] = (tmp >> (sizeof(uint32_t) * 8));
900     }
901     __U[m + n] = 0; // this extra word is for "spill" in the
Knuth algorithm.
902
903     // Initialize the divisor
904     memset(__V, 0, (n) * sizeof(uint32_t));
905     for (unsigned i = 0; i < rhsWords; ++i) {
906         uint64_t tmp = RHS.get_pVal(i);
907         __V[i * 2] = (uint32_t)(tmp & mask);
908         __V[i * 2 + 1] = (tmp >> (sizeof(uint32_t) * 8));
909     }
910
911     // initialize the quotient and remainder
912     memset(__Q, 0, (m + n) * sizeof(uint32_t));
913     if (Remainder) memset(__R, 0, n * sizeof(uint32_t));
914
915     // Now, adjust m and n for the Knuth division. n is the
number of words in
916     // the divisor. m is the number of words by which the di-
vidend exceeds the
917     // divisor (i.e. m+n is the length of the dividend). These
sizes must not
918     // contain any zero words or the Knuth algorithm fails.
919     for (unsigned i = n; i > 0 && __V[i - 1] == 0; i--) {
920         n--;
921         m++;
922     }
923     for (unsigned i = m + n; i > 0 && __U[i - 1] == 0; i--)
m--;
924
925     // If we're left with only a single word for the divisor,
Knuth doesn't work
926     // so we implement the short division algorithm here. This
is much simpler
927     // and faster because we are certain that we can divide a
64-bit quantity
928     // by a 32-bit quantity at hardware speed and short divi-
sion is simply a
929     // series of such operations. This is just like doing short
division but we
930     // are using base 2^32 instead of base 10.
931     assert(n != 0 && "Divide by zero?");
932     if (n == 1) {
933         uint32_t divisor = __V[0];
934         uint32_t remainder = 0;
935         for (int i = m + n - 1; i >= 0; i--) {
936             uint64_t partial_dividend = (uint64_t(remainder) << 32 |
__U[i]);
937             if (partial_dividend == 0) {
938                 __Q[i] = 0;
939                 remainder = 0;
940             } else if (partial_dividend < divisor) {
941                 __Q[i] = 0;

```

```

942     remainder = (uint32_t)partial_dividend;
943 } else if (partial_dividend == divisor) {
944     __Q[i] = 1;
945     remainder = 0;
946 } else {
947     __Q[i] = (uint32_t)(partial_dividend / divisor);
948     remainder = (uint32_t)(partial_dividend - (__Q[i] *
divisor));
949 }
950 }
951 if (__R) __R[0] = remainder;
952 } else {
953     // Now we're ready to invoke the Knuth classical divide
algorithm. In this
954     // case n > 1.
955     KnuthDiv(__U, __V, __Q, __R, m, n);
956 }
957
958 // If the caller wants the quotient
959 if (Quotient) {
960     // Set up the Quotient value's memory.
961     if (Quotient->BitWidth != LHS.BitWidth) {
962         if (Quotient->isSingleWord()) Quotient->set_VAL(0);
963     } else
964         Quotient->clear();
965
966     // The quotient is in Q. Reconstitute the quotient into
Quotient's low
967     // order words.
968     if (lhsWords == 1) {
969         uint64_t tmp =
970             uint64_t(__Q[0]) | ((uint64_t(__Q[1])) <<
(APINT_BITS_PER_WORD / 2));
971         Quotient->set_VAL(tmp);
972     } else {
973         assert(!Quotient->isSingleWord() &&
"Quotient ap_private not large enough");
974         for (unsigned i = 0; i < lhsWords; ++i)
975             Quotient->set_pVal(
976                 i, uint64_t(__Q[i * 2]) |
977                 ((uint64_t(__Q[i * 2 + 1])) <<
(APINT_BITS_PER_WORD / 2)));
978     }
979 }
980 Quotient->clearUnusedBits();
981 }
982
983 // If the caller wants the remainder
984 if (Remainder) {
985     // Set up the Remainder value's memory.
986     if (Remainder->BitWidth != RHS.BitWidth) {
987         if (Remainder->isSingleWord()) Remainder->set_VAL(0);
988     } else
989         Remainder->clear();
990
991     // The remainder is in R. Reconstitute the remainder into
Remainder's low
992     // order words.
993     if (rhsWords == 1) {
994         uint64_t tmp =
995             uint64_t(__R[0]) | ((uint64_t(__R[1])) <<
(APINT_BITS_PER_WORD / 2));
996         Remainder->set_VAL(tmp);
997     } else {
998         assert(!Remainder->isSingleWord() &&
"Remainder ap_private not large enough");
999         for (unsigned i = 0; i < rhsWords; ++i)
1000             Remainder->set_pVal(
1001                 i, uint64_t(__R[i * 2]) |
1002                 ((uint64_t(__R[i * 2 + 1])) <<
(APINT_BITS_PER_WORD / 2)));
1003     }
1004 }
1005 Remainder->clearUnusedBits();
1006 }
1007
1008 // Clean up the memory we allocated.
1009 if (__U != &SPACE[0]) {
1010     delete[] __U;
1011     delete[] __V;
1012     delete[] __Q;
1013     delete[] __R;
1014 }
1015 }
1016
1017 template <int _AP_W, bool _AP_S>
1018 void divide(const ap_private<_AP_W, _AP_S> &LHS, uint32_t
lhsWords,
1019             uint64_t RHS, ap_private<_AP_W, _AP_S>* Quotient,
1020             ap_private<_AP_W, _AP_S>* Remainder) {
1021     uint32_t rhsWords = 1;
1022     assert(lhsWords >= rhsWords && "Fractional result");
1023     enum { APINT_BITS_PER_WORD = 64 };
1024     // First, compose the values into an array of 32-bit words
instead of
1025     // 64-bit words. This is a necessity of both the "short di-
vision" algorithm
1026     // and the the Knuth "classical algorithm" which requires
there to be native
1027     // operations for +, -, and * on an m bit value with an m*2

```

```

bit result. We
1028     // can't use 64-bit operands here because we don't have
native results of
1029     // 128-bits. Furthermore, casting the 64-bit values to 32-
bit values won't
1030     // work on large-endian machines.
1031     uint64_t mask = ~0ull >> (sizeof(uint32_t) * 8);
1032     uint32_t n = 2;
1033     uint32_t m = (lhsWords * 2) - n;
1034
1035     // Allocate space for the temporary values we need either
on the stack, if
1036     // it will fit, or on the heap if it won't.
1037     uint32_t SPACE[128];
1038     uint32_t* __U = 0;
1039     uint32_t* __V = 0;
1040     uint32_t* __Q = 0;
1041     uint32_t* __R = 0;
1042     if ((Remainder ? 4 : 3) * n + 2 * m + 1 <= 128) {
1043         __U = &SPACE[0];
1044         __V = &SPACE[m + n + 1];
1045         __Q = &SPACE[(m + n + 1) + n];
1046         if (Remainder) __R = &SPACE[(m + n + 1) + n + (m + n)];
1047     } else {
1048         __U = new uint32_t[m + n + 1];
1049         __V = new uint32_t[n];
1050         __Q = new uint32_t[m + n];
1051         if (Remainder) __R = new uint32_t[n];
1052     }
1053
1054     // Initialize the dividend
1055     memset(__U, 0, (m + n + 1) * sizeof(uint32_t));
1056     for (unsigned i = 0; i < lhsWords; ++i) {
1057         uint64_t tmp = LHS.get_pVal(i);
1058         __U[i * 2] = tmp & mask;
1059         __U[i * 2 + 1] = (tmp >> (sizeof(uint32_t) * 8));
1060     }
1061     __U[m + n] = 0; // this extra word is for "spill" in the
Knuth algorithm.
1062
1063     // Initialize the divisor
1064     memset(__V, 0, (n) * sizeof(uint32_t));
1065     __V[0] = RHS & mask;
1066     __V[1] = (RHS >> (sizeof(uint32_t) * 8));
1067
1068     // initialize the quotient and remainder
1069     memset(__Q, 0, (m + n) * sizeof(uint32_t));
1070     if (Remainder) memset(__R, 0, n * sizeof(uint32_t));
1071
1072     // Now, adjust m and n for the Knuth division. n is the
number of words in
1073     // the divisor. m is the number of words by which the di-
vidend exceeds the
1074     // divisor (i.e. m+n is the length of the dividend). These
sizes must not
1075     // contain any zero words or the Knuth algorithm fails.
1076     for (unsigned i = n; i > 0 && __V[i - 1] == 0; i--) {
1077         n--;
1078         m++;
1079     }
1080     for (unsigned i = m + n; i > 0 && __U[i - 1] == 0; i--)
m--;
1081
1082     // If we're left with only a single word for the divisor,
Knuth doesn't work
1083     // so we implement the short division algorithm here. This
is much simpler
1084     // and faster because we are certain that we can divide a
64-bit quantity
1085     // by a 32-bit quantity at hardware speed and short divi-
sion is simply a
1086     // series of such operations. This is just like doing short
division but we
1087     // are using base 2^32 instead of base 10.
1088     assert(n != 0 && "Divide by zero?");
1089     if (n == 1) {
1090         uint32_t divisor = __V[0];
1091         uint32_t remainder = 0;
1092         for (int i = m + n - 1; i >= 0; i--) {
1093             uint64_t partial_dividend = (uint64_t(remainder)) << 32
| __U[i];
1094             if (partial_dividend == 0) {
1095                 __Q[i] = 0;
1096                 remainder = 0;
1097             } else if (partial_dividend < divisor) {
1098                 __Q[i] = 0;
1099                 remainder = partial_dividend;
1100             } else if (partial_dividend == divisor) {
1101                 __Q[i] = 1;
1102                 remainder = 0;
1103             } else {
1104                 __Q[i] = partial_dividend / divisor;
1105                 remainder = partial_dividend - (__Q[i] * divisor);
1106             }
1107         }
1108         if (__R) __R[0] = remainder;
1109     } else {
1110         // Now we're ready to invoke the Knuth classical divide

```

```

algorithm. In this
1111 // case n > 1.
1112 KnuthDiv(__U, __V, __Q, __R, m, n);
1113 }
1114
1115 // If the caller wants the quotient
1116 if (Quotient) {
1117 // Set up the Quotient value's memory.
1118 if (Quotient->BitWidth != LHS.BitWidth) {
1119 if (Quotient->isSingleWord()) Quotient->set_VAL(0);
1120 } else
1121 Quotient->clear();
1122
1123 // The quotient is in Q. Reconstitute the quotient into
1124 Quotient's low
1125 // order words.
1126 if (lhsWords == 1) {
1127 uint64_t tmp =
1128 (APINT_BITS_PER_WORD / 2);
1129 Quotient->set_VAL(tmp);
1130 } else {
1131 assert(!Quotient->isSingleWord() &&
1132 "Quotient ap_private not large enough");
1133 for (unsigned i = 0; i < lhsWords; ++i)
1134 Quotient->set_pVal(
1135 i, uint64_t(__Q[i * 2]) |
1136 ((uint64_t(__Q[i * 2 + 1])) <<
1137 (APINT_BITS_PER_WORD / 2)));
1138 }
1139 Quotient->clearUnusedBits();
1140 }
1141
1142 // If the caller wants the remainder
1143 if (Remainder) {
1144 // Set up the Remainder value's memory.
1145 if (Remainder->BitWidth != 64 /* RHS.BitWidth */) {
1146 if (Remainder->isSingleWord()) Remainder->set_VAL(0);
1147 } else
1148 Remainder->clear();
1149
1150 // The remainder is in __R. Reconstitute the remainder
1151 into Remainder's low
1152 // order words.
1153 if (rhsWords == 1) {
1154 uint64_t tmp =
1155 (APINT_BITS_PER_WORD / 2);
1156 Remainder->set_VAL(tmp);
1157 } else {
1158 assert(!Remainder->isSingleWord() &&
1159 "Remainder ap_private not large enough");
1160 for (unsigned i = 0; i < rhsWords; ++i)
1161 Remainder->set_pVal(
1162 i, uint64_t(__R[i * 2]) |
1163 ((uint64_t(__R[i * 2 + 1])) <<
1164 (APINT_BITS_PER_WORD / 2)));
1165 }
1166 Remainder->clearUnusedBits();
1167 }
1168
1169 // Clean up the memory we allocated.
1170 if (__U != &SPACE[0]) {
1171 delete[] __U;
1172 delete[] __V;
1173 delete[] __Q;
1174 delete[] __R;
1175 }
1176 }
1177
1178 /// @brief Logical right-shift function.
1179 template <int _AP_W, bool _AP_S, bool _AP_C>
1180 inline ap_private<_AP_W, _AP_S, _AP_C> lshr(
1181 const ap_private<_AP_W, _AP_S, _AP_C> &LHS, uint32_t
1182 shiftAmt) {
1183 return LHS.lshr(shiftAmt);
1184 }
1185
1186 /// Left-shift the ap_private by shiftAmt.
1187 /// @brief Left-shift function.
1188 template <int _AP_W, bool _AP_S, bool _AP_C>
1189 inline ap_private<_AP_W, _AP_S, _AP_C> shl(
1190 const ap_private<_AP_W, _AP_S, _AP_C> &LHS, uint32_t
1191 shiftAmt) {
1192 return LHS.shl(shiftAmt);
1193 }
1194
1195 } // namespace ap_private_ops
1196
1197 #endif // LLVM_SUPPORT_MATHEXTRAS_H
1198
1199 /// This enumeration just provides for internal constants
1200 used in this
1201 translation unit.
1202 enum {
1203 MIN_INT_BITS = 1, ///< Minimum number of bits that can be
1204 specified
1205 ///< Note that this must remain synchronized with Integer-
1206 Type::MIN_INT_BITS
1207 MAX_INT_BITS = (1 << 23) - 1 ///< Maximum number of bits

```

```

that can be specified
1199 ///< Note that this must remain synchronized with Integer-
1200 Type::MAX_INT_BITS
1201 };
1202
1203 //
1204 =====
1205 // ap_private Class
1206 =====
1207
1208 // ap_private - This class represents arbitrary precision
1209 constant integral
1210 values.
1211 It is a functional replacement for common case unsigned
1212 integer type like
1213 "unsigned", "unsigned long" or "uint64_t", but also al-
1214 lows non-byte-width
1215 integer sizes and large integer value types such as 3-
1216 bits, 15-bits, or more
1217 than 64-bits of precision. ap_private provides a variety
1218 of arithmetic
1219 operators
1220 and methods to manipulate integer values of any bit-
1221 width. It supports both
1222 the typical integer arithmetic and comparison operations
1223 as well as bitwise
1224 manipulation.
1225
1226 The class has several invariants worth noting:
1227 * All bit, byte, and word positions are zero-based.
1228 * Once the bit width is set, it doesn't change except
1229 by the Truncate,
1230 SignExtend, or ZeroExtend operations.
1231 * All binary operators must be on ap_private instances
1232 of the same bit
1233 width.
1234 Attempting to use these operators on instances with
1235 different bit
1236 widths will yield an assertion.
1237
1238 * The value is stored canonically as an unsigned value.
1239 For operations
1240 where it makes a difference, there are both signed
1241 and unsigned variants
1242 of the operation. For example, sdiv and udiv. How-
1243 ever, because the bit
1244 widths must be the same, operations such as Mul and
1245 Add produce the same
1246 results regardless of whether the values are inter-
1247 preted as signed or
1248 not.
1249 * In general, the class tries to follow the style of
1250 computation that LLVM
1251 uses in its IR. This simplifies its use for LLVM.
1252
1253 @brief Class for arbitrary precision integers.
1254
1255 #if defined(_MSC_VER)
1256 #if _MSC_VER < 1400 && !defined(for)
1257 #define for if (0); else for
1258 #endif
1259 typedef unsigned __int64 ap_ulong;
1260 typedef signed __int64 ap_slong;
1261 #else
1262 typedef unsigned long long ap_ulong;
1263 typedef signed long long ap_slong;
1264 #endif
1265 template <int _AP_N8, bool _AP_S>
1266 struct valtype;
1267
1268 template <int _AP_N8>
1269 struct valtype<_AP_N8, false> {
1270 typedef uint64_t Type;
1271 };
1272
1273 template <int _AP_N8>
1274 struct valtype<_AP_N8, true> {
1275 typedef int64_t Type;
1276 };
1277
1278 template <>
1279 struct valtype<1, false> {
1280 typedef unsigned char Type;
1281 };
1282
1283 template <>
1284 struct valtype<2, false> {
1285 typedef unsigned short Type;
1286 };
1287
1288 template <>
1289 struct valtype<3, false> {
1290 typedef unsigned int Type;
1291 };
1292
1293 template <>
1294 struct valtype<4, false> {
1295 typedef unsigned int Type;
1296 };
1297
1298 template <>

```

```

1276 struct valtype<1, true> {
1277     typedef signed char Type;
1278 };
1279 template <>
1280 struct valtype<2, true> {
1281     typedef short Type;
1282 };
1283 template <>
1284 struct valtype<3, true> {
1285     typedef int Type;
1286 };
1287 template <>
1288 struct valtype<4, true> {
1289     typedef int Type;
1290 };
1291
1292 template <bool enable>
1293 struct ap_private_enable_if {};
1294 template <>
1295 struct ap_private_enable_if<true> {
1296     static const bool isValid = true;
1297 };
1298
1299 // When bitwidth < 64
1300 template <int _AP_W, bool _AP_S>
1301 class ap_private<_AP_W, _AP_S, true> {
1302     // SFINAE pattern. Only consider this class when _AP_W <=
64
1303     const static bool valid = ap_private_enable_if<_AP_W <=
64>::isValid;
1304
1305     #ifdef _MSC_VER
1306     #pragma warning(disable : 4521 4522)
1307     #endif
1308     public:
1309     typedef typename valtype<(_AP_W + 7) / 8, _AP_S>::Type Val-
Type;
1310     typedef ap_private<_AP_W, _AP_S> Type;
1311     template <int _AP_W2, bool _AP_S2>
1312     struct RType {
1313         enum {
1314             mult_w = _AP_W + _AP_W2,
1315             mult_s = _AP_S || _AP_S2,
1316             plus_w =
1317                 AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 + (_AP_S
&& !_AP_S2)) + 1,
1318             plus_s = _AP_S || _AP_S2,
1319             minus_w =
1320                 AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 + (_AP_S
&& !_AP_S2)) + 1,
1321             minus_s = true,
1322             div_w = _AP_W + _AP_S2,
1323             div_s = _AP_S || _AP_S2,
1324             mod_w = AP_MIN(_AP_W, _AP_W2 + (!_AP_S2 && _AP_S)),
1325             mod_s = _AP_S,
1326             logic_w = AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 +
(_AP_S && !_AP_S2)),
1327             logic_s = _AP_S || _AP_S2
1328         };
1329         typedef ap_private<mult_w, mult_s> mult;
1330         typedef ap_private<plus_w, plus_s> plus;
1331         typedef ap_private<minus_w, minus_s> minus;
1332         typedef ap_private<logic_w, logic_s> logic;
1333         typedef ap_private<div_w, div_s> div;
1334         typedef ap_private<mod_w, mod_s> mod;
1335         typedef ap_private<_AP_W, _AP_S> arg1;
1336         typedef bool reduce;
1337     };
1338     enum { APINT_BITS_PER_WORD = sizeof(uint64_t) * 8 };
1339     enum {
1340         excess_bits = (_AP_W % APINT_BITS_PER_WORD)
1341             ? APINT_BITS_PER_WORD - (_AP_W %
APINT_BITS_PER_WORD)
1342             : 0
1343     };
1344     static const uint64_t mask = ((uint64_t)-0ULL >> (ex-
cess_bits));
1345     static const uint64_t not_mask = ~mask;
1346     static const uint64_t sign_bit_mask = 1ULL <<
(APINT_BITS_PER_WORD - 1);
1347     template <int _AP_W1>
1348     struct sign_ext_mask {
1349         static const uint64_t mask = ~0ULL << _AP_W1;
1350     };
1351     static const int width = _AP_W;
1352
1353     enum {
1354         BitWidth = _AP_W,
1355         _AP_N = 1,
1356     };
1357     ValType VAL; ///< Used to store the <= 64 bits integer
value.
1358     #ifdef AP_CANARY
1359     ValType CANARY;
1360     void check_canary() { assert(CANARY == (Val-
Type)0xDEADBEEFDEADBEEF); }
1361     void set_canary() { CANARY = (ValType)0xDEADBEEFDEADBEEF; }
1362     #else
1363     void check_canary() {}
1364     void set_canary() {}

```

```

1365 #endif
1366
1367     INLINE ValType& get_VAL(void) { return VAL; }
1368     INLINE ValType get_VAL(void) const { return VAL; }
1369     INLINE ValType get_VAL(void) const volatile { return VAL; }
1370     INLINE void set_VAL(uint64_t value) { VAL = (Val-
Type)value; }
1371     INLINE ValType& get_pVal(int i) { return VAL; }
1372     INLINE ValType get_pVal(int i) const { return VAL; }
1373     INLINE const uint64_t* get_pVal() const {
1374         assert(0 && "invalid usage");
1375         return 0;
1376     }
1377     INLINE ValType get_pVal(int i) const volatile { return
VAL; }
1378     INLINE uint64_t* get_pVal() const volatile {
1379         assert(0 && "invalid usage");
1380         return 0;
1381     }
1382     INLINE void set_pVal(int i, uint64_t value) { VAL = (Val-
Type)value; }
1383
1384     INLINE uint32_t getBitWidth() const { return BitWidth; }
1385
1386     template <int _AP_W1, bool _AP_S1>
1387     ap_private<_AP_W, _AP_S>& operator=(const
ap_private<_AP_W1, _AP_S1>& RHS) {
1388         VAL = (ValType)(RHS.get_VAL());
1389         clearUnusedBits();
1390         return *this;
1391     }
1392
1393     template <int _AP_W1, bool _AP_S1>
1394     ap_private<_AP_W, _AP_S>& operator=(
const volatile ap_private<_AP_W1, _AP_S1>& RHS) {
1395         VAL = (ValType)(RHS.get_VAL()); // TODO check here about
ap_private<W,S,false>
1396         clearUnusedBits();
1397         return *this;
1398     }
1399
1400     void operator=(const ap_private& RHS) volatile {
1401         // Don't do anything for X = X
1402         VAL = RHS.get_VAL(); // No need to check because no harm
done by copying.
1403         clearUnusedBits();
1404     }
1405
1406     ap_private& operator=(const ap_private& RHS) {
1407         // Don't do anything for X = X
1408         VAL = RHS.get_VAL(); // No need to check because no harm
done by copying.
1409         clearUnusedBits();
1410         return *this;
1411     }
1412
1413     void operator=(const volatile ap_private& RHS) volatile {
1414         // Don't do anything for X = X
1415         VAL = RHS.get_VAL(); // No need to check because no harm
done by copying.
1416         clearUnusedBits();
1417     }
1418
1419     ap_private& operator=(const volatile ap_private& RHS) {
1420         // Don't do anything for X = X
1421         VAL = RHS.get_VAL(); // No need to check because no harm
done by copying.
1422         clearUnusedBits();
1423         return *this;
1424     }
1425
1426     template <int _AP_W2, bool _AP_S2>
1427     INLINE ap_private& operator=(const
private_range_ref<_AP_W2, _AP_S2>& op2) {
1428         *this = ap_private<_AP_W2, false>(op2);
1429         return *this;
1430     }
1431
1432     #define ASSIGN_OP_FROM_INT(C_TYPE) \
1433     INLINE ap_private& operator=(const C_TYPE v) { \
1434         set_canary(); \
1435         this->VAL = (ValType)v; \
1436         clearUnusedBits(); \
1437         check_canary(); \
1438         return *this; \
1439     }
1440
1441     ASSIGN_OP_FROM_INT(bool)
1442     ASSIGN_OP_FROM_INT(char)
1443     ASSIGN_OP_FROM_INT(signed char)
1444     ASSIGN_OP_FROM_INT(unsigned char)
1445     ASSIGN_OP_FROM_INT(short)
1446     ASSIGN_OP_FROM_INT(unsigned short)
1447     ASSIGN_OP_FROM_INT(int)
1448     ASSIGN_OP_FROM_INT(unsigned int)
1449     ASSIGN_OP_FROM_INT(long)
1450     ASSIGN_OP_FROM_INT(unsigned long)
1451     ASSIGN_OP_FROM_INT(ap_slong)
1452     ASSIGN_OP_FROM_INT(ap_ulong)
1453     #if 0

```



```

1455 ASSIGN_OP_FROM_INT(half)
1456 ASSIGN_OP_FROM_INT(float)
1457 ASSIGN_OP_FROM_INT(double)
1458 #endif
1459 #undef ASSIGN_OP_FROM_INT
1460
1461 // XXX This is a must to prevent pointer being converted to
1462 bool.
1463     INLINE ap_private& operator=(const char* s) {
1464         ap_private tmp(s); // XXX direct-initialization, as ctor
1465         is explicit.
1466         operator=(tmp);
1467         return *this;
1468     }
1469
1470 private:
1471     explicit INLINE ap_private(uint64_t* val) : VAL(val[0]) {
1472         set_canary();
1473         clearUnusedBits();
1474         check_canary();
1475     }
1476
1477     INLINE bool isSingleWord() const { return true; }
1478
1479 public:
1480
1481     INLINE void fromString(const char* strStart, uint32_t slen,
1482     uint8_t radix) {
1483         bool isNeg = strStart[0] == '-';
1484         if (isNeg) {
1485             strStart++;
1486             slen--;
1487         }
1488         if (strStart[0] == '0' && (strStart[1] == 'b' ||
1489         strStart[1] == 'B')) {
1490             //if (radix == 0) radix = 2;
1491             _AP_WARNING(radix != 2, "%s seems to have base %d, but
1492             %d given.", strStart, 2, radix);
1493             strStart += 2;
1494             slen -= 2;
1495         } else if (strStart[0] == '0' && (strStart[1] == 'o' ||
1496         strStart[1] == 'O')) {
1497             //if (radix == 0) radix = 8;
1498             _AP_WARNING(radix != 8, "%s seems to have base %d, but
1499             %d given.", strStart, 8, radix);
1500             strStart += 2;
1501             slen -= 2;
1502         } else if (strStart[0] == '0' && (strStart[1] == 'x' ||
1503         strStart[1] == 'X')) {
1504             //if (radix == 0) radix = 16;
1505             _AP_WARNING(radix != 16, "%s seems to have base %d, but
1506             %d given.", strStart, 16, radix);
1507             strStart += 2;
1508             slen -= 2;
1509         } else if (strStart[0] == '0' && (strStart[1] == 'd' ||
1510         strStart[1] == 'D')) {
1511             //if (radix == 0) radix = 10;
1512             _AP_WARNING(radix != 10, "%s seems to have base %d, but
1513             %d given.", strStart, 10, radix);
1514             strStart += 2;
1515             slen -= 2;
1516         } else if (radix == 0) {
1517             //radix = 2; // XXX default value
1518         }
1519         // Check our assumptions here
1520
1521         assert((radix == 10 || radix == 8 || radix == 16 || radix
1522         == 2) &&
1523         "Radix should be 2, 8, 10, or 16!");
1524         assert(strStart && "String is null?");
1525
1526         // Clear bits.
1527         uint64_t tmpVAL = VAL = 0;
1528
1529         switch (radix) {
1530             case 2:
1531                 //
1532                 sscanf(strStart, "%b", &tmpVAL);
1533                 // tmpVAL = *strStart == '1' ? ~0ULL : 0;
1534                 for (; *strStart; ++strStart) {
1535                     assert((*strStart == '0' || *strStart == '1') &&
1536                     ("Wrong binary number"));
1537                     tmpVAL <= 1;
1538                     tmpVAL |= (*strStart - '0');
1539                 }
1540                 break;
1541             case 8:
1542                 #ifdef _MSC_VER
1543                     sscanf_s(strStart, "%llo", &tmpVAL, slen + 1);
1544                 #else
1545                     if defined(__x86_64__) && !defined(__MINGW32__) && !
1546                     defined(__WIN32__)
1547                         sscanf(strStart, "%lu", &tmpVAL);
1548                     #else
1549                         sscanf(strStart, "%llu", &tmpVAL);
1550                     #endif //__x86_64__
1551                 #endif
1552             case 10:
1553                 #ifdef _MSC_VER
1554                     sscanf_s(strStart, "%lli", &tmpVAL, slen + 1);
1555                 #else
1556                     if defined(__x86_64__) && !defined(__MINGW32__) && !
1557                     defined(__WIN32__)
1558                         sscanf(strStart, "%li", &tmpVAL);
1559                     #else
1560                         sscanf(strStart, "%lli", &tmpVAL);
1561                     #endif //__x86_64__
1562                 #endif
1563                 assert(true && "Unknown radix");
1564                 // error
1565             }
1566         VAL = isNeg ? (ValType)(-tmpVAL) : (ValType)(tmpVAL);
1567         clearUnusedBits();
1568     }
1569
1570 private:
1571     INLINE ap_private(const std::string& val, uint8_t radix =
1572     2) : VAL(0) {
1573         assert(!val.empty() && "String empty?");
1574         set_canary();
1575         fromString(val.c_str(), val.size(), radix);
1576         check_canary();
1577     }
1578
1579     INLINE ap_private(const char strStart[], uint32_t slen,
1580     uint8_t radix) : VAL(0) {
1581         set_canary();
1582         fromString(strStart, slen, radix);
1583         check_canary();
1584     }
1585
1586     INLINE ap_private(uint32_t numWords, const uint64_t bigV-
1587     al[]) : VAL(bigVal[0]) {
1588         set_canary();
1589         clearUnusedBits();
1590         check_canary();
1591     }
1592
1593 public:
1594     INLINE ap_private() {
1595         set_canary();
1596         clearUnusedBits();
1597         check_canary();
1598     }
1599
1600 #define CTOR(TYPE)
1601     INLINE ap_private(TYPE v) : VAL((ValType)v) {
1602         set_canary();
1603         clearUnusedBits();
1604         check_canary();
1605     }
1606
1607     CTOR(bool)
1608     CTOR(char)
1609     CTOR(signed char)
1610     CTOR(unsigned char)
1611     CTOR(short)
1612     CTOR(unsigned short)
1613     CTOR(int)
1614     CTOR(unsigned int)
1615     CTOR(long)
1616     CTOR(unsigned long)
1617     CTOR(ap_slong)
1618     CTOR(ap_ulong)
1619 #if 0
1620     CTOR(half)
1621     CTOR(float)
1622     CTOR(double)
1623 #endif
1624 #undef CTOR
1625
1626 template <int _AP_W1, bool _AP_S1, bool _AP_OPT>
1627     INLINE ap_private(const ap_private<_AP_W1, _AP_S1,
1628     _AP_OPT>& that) : VAL((ValType)that.get_VAL()) {
1629         set_canary();
1630         clearUnusedBits();
1631         check_canary();
1632     }

```

```

1631
1632     template <int _AP_W1, bool _AP_S1, bool _AP_OPT>
1633     INLINE ap_private(const volatile ap_private<_AP_W1, _AP_S1,
1634     _AP_OPT>& that)
1635     : VAL((ValType)that.get_VAL()) {
1636         set_canary();
1637         clearUnusedBits();
1638         check_canary();
1639     }
1640     explicit INLINE ap_private(const char* val) {
1641         set_canary();
1642         unsigned char radix = 10;
1643         std::string str = ap_private_ops::parseString(val,
1644         radix); // will set radix.
1645         std::string::size_type pos = str.find('.');
1646         // trunc all fraction part
1647         if (pos != std::string::npos) str = str.substr(pos);
1648         ap_private<_AP_W, _AP_S> ap_private_val(str, radix);
1649         operator=(ap_private_val);
1650         check_canary();
1651     }
1652
1653     INLINE ap_private(const char* val, signed char rd) {
1654         set_canary();
1655         unsigned char radix = rd;
1656         std::string str = ap_private_ops::parseString(val,
1657         radix); // will set radix.
1658         std::string::size_type pos = str.find('.');
1659         // trunc all fraction part
1660         if (pos != std::string::npos) str = str.substr(pos);
1661         ap_private<_AP_W, _AP_S> ap_private_val(str, radix);
1662         operator=(ap_private_val);
1663         check_canary();
1664     }
1665     INLINE ~ap_private() { check_canary(); }
1666
1667     INLINE bool isNegative() const {
1668         static const uint64_t sign_mask = 1ULL << (_AP_W - 1);
1669         return _AP_S && (sign_mask & VAL);
1670     }
1671
1672     INLINE bool isPositive() const { return !isNegative(); }
1673
1674     INLINE bool isStrictlyPositive() const { return !isNegative()
1675     && VAL != 0; }
1676
1677     INLINE bool isAllOnesValue() const { return (mask & VAL) ==
1678     mask; }
1679
1680     INLINE bool operator==(const ap_private<_AP_W, _AP_S>& RHS)
1681     const {
1682         return VAL == RHS.get_VAL();
1683     }
1684     INLINE bool operator==(const ap_private<_AP_W, !_AP_S>&
1685     RHS) const {
1686         return (uint64_t)VAL == (uint64_t)RHS.get_VAL();
1687     }
1688     INLINE bool operator==(uint64_t Val) const { return
1689     ((uint64_t)VAL == Val); }
1690     INLINE bool operator!=(uint64_t Val) const { return
1691     ((uint64_t)VAL != Val); }
1692
1693     INLINE bool operator!=(const ap_private<_AP_W, _AP_S>& RHS)
1694     const {
1695         return VAL != RHS.get_VAL();
1696     }
1697     INLINE bool operator!=(const ap_private<_AP_W, !_AP_S>&
1698     RHS) const {
1699         return (uint64_t)VAL != (uint64_t)RHS.get_VAL();
1700     }
1701     // postfix increment.
1702     const ap_private operator++(int) {
1703         ap_private orig(*this);
1704         VAL++;
1705         clearUnusedBits();
1706         return orig;
1707     }
1708     // prefix increment.
1709     const ap_private operator++() {
1710         ++VAL;
1711         clearUnusedBits();
1712         return *this;
1713     }
1714     // postfix decrement.
1715     const ap_private operator--(int) {
1716         ap_private orig(*this);
1717         --VAL;
1718         clearUnusedBits();
1719         return orig;

```

```

1716     }
1717     // prefix decrement.
1718     const ap_private operator--() {
1719         --VAL;
1720         clearUnusedBits();
1721         return *this;
1722     }
1723     // one's complement.
1724     INLINE ap_private<_AP_W + !_AP_S, true> operator~() const {
1725         ap_private<_AP_W + !_AP_S, true> Result(*this);
1726         Result.flip();
1727         return Result;
1728     }
1729     // two's complement.
1730     INLINE typename RType<1, false>::minus operator-() const {
1731         return ap_private<1, false>(0) - (*this);
1732     }
1733     // logic negation.
1734     INLINE bool operator!() const { return !VAL; }
1735
1736     INLINE std::string toString(uint8_t radix, bool wantSigned)
1737     const;
1738     INLINE std::string toStringUnsigned(uint8_t radix = 10)
1739     const {
1740         return toString(radix, false);
1741     }
1742     INLINE std::string toStringSigned(uint8_t radix = 10) const
1743     {
1744         return toString(radix, true);
1745     }
1746     INLINE void clear() { VAL = 0; }
1747     INLINE ap_private& clear(uint32_t bitPosition) {
1748         VAL &= ~(1ULL << (bitPosition));
1749         clearUnusedBits();
1750         return *this;
1751     }
1752     INLINE ap_private ashr(uint32_t shiftAmt) const {
1753         if (_AP_S)
1754             return ap_private((shiftAmt == BitWidth) ? 0
1755             : ((int64_t)VAL) >> (shiftAmt));
1756         else
1757             return ap_private((shiftAmt == BitWidth) ? 0
1758             : ((uint64_t)VAL) >> (shiftAmt));
1759     }
1760     INLINE ap_private lshr(uint32_t shiftAmt) const {
1761         return ap_private((shiftAmt == BitWidth)
1762         ? ap_private(0)
1763         : ap_private((VAL & mask) >>
1764         (shiftAmt)));
1765     }
1766     INLINE ap_private shl(uint32_t shiftAmt) const
1767     // just for clang compiler
1768     #if defined(__clang__) && !defined(__CLANG_3_1__)
1769     __attribute__((no_sanitize("undefined")))
1770     #endif
1771     {
1772         if (shiftAmt > BitWidth) {
1773             if (!isNegative())
1774                 return ap_private(0);
1775             else
1776                 return ap_private(-1);
1777         }
1778         if (shiftAmt == BitWidth)
1779             return ap_private(0);
1780         else
1781             return ap_private((VAL) << (shiftAmt));
1782         // return ap_private((shiftAmt == BitWidth) ?
1783         ap_private(0ULL) :
1784         // ap_private(VAL << shiftAmt));
1785     }
1786     INLINE int64_t getSExtValue() const { return VAL; }
1787     // XXX XXX this function is used in CBE
1788     INLINE uint64_t getZExtValue() const { return VAL & mask; }
1789
1790     template <int _AP_W2, bool _AP_S2>
1791     INLINE ap_private(const _private_range_ref<_AP_W2, _AP_S2>&
1792     ref) {
1793         set_canary();
1794         *this = ref.get();
1795         check_canary();
1796     }
1797     template <int _AP_W2, bool _AP_S2>
1798     INLINE ap_private(const _private_bit_ref<_AP_W2, _AP_S2>&
1799     ref) {
1800         set_canary();

```

```

1804     *this = ((uint64_t)(bool)ref);
1805     check_canary();
1806 }
1807
1808 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
1809 // inline ap_private(const ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>& ref) {
1810 //     set_canary();
1811 //     *this = ref.get();
1812 //     check_canary();
1813 // }
1814 //
1815 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
1816 //         ap_o_mode _AP_O2, int _AP_N2>
1817 // inline ap_private(
1818 //     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
1819 //     set_canary();
1820 //     *this = ((val.operator ap_private<_AP_W2, false>()));
1821 //     check_canary();
1822 // }
1823 //
1824 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
1825 //         ap_o_mode _AP_O2, int _AP_N2>
1826 // inline ap_private(
1827 //     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
1828 //     set_canary();
1829 //     *this = ((uint64_t)(bool)val);
1830 //     check_canary();
1831 // }
1832
1833 inline void write(const ap_private<_AP_W, _AP_S>& op2)
volatile {
1834     *this = (op2);
1835 }
1836
1837 // Explicit conversions to C interger types
1838 //-----
1839 inline operator ValType() const { return get_VAL(); }
1840
1841 inline int to_uchar() const { return (unsigned
char)get_VAL(); }
1842
1843 inline int to_char() const { return (signed
char)get_VAL(); }
1844
1845 inline int to_ushort() const { return (unsigned
short)get_VAL(); }
1846
1847 inline int to_short() const { return (short)get_VAL(); }
1848
1849 inline int to_int() const {
1850     // ap_private<64 /* _AP_W */, _AP_S> res(V);
1851     return (int)get_VAL();
1852 }
1853
1854 inline unsigned to_uint() const { return (un-
signed)get_VAL(); }
1855
1856 inline long to_long() const { return (long)get_VAL(); }
1857
1858 inline unsigned long to_ulong() const { return (unsigned
long)get_VAL(); }
1859
1860 inline ap_slong to_int64() const { return
(ap_slong)get_VAL(); }
1861
1862 inline ap_ulong to_uint64() const { return (ap_u-
long)get_VAL(); }
1863
1864 inline double to_double() const {
1865     if (isNegative())
1866         return roundToDouble(true);
1867     else
1868         return roundToDouble(false);
1869 }
1870
1871 inline unsigned length() const { return _AP_W; }
1872
1873 inline bool isMinValue() const { return VAL == 0; }
1874 template <int _AP_W1, bool _AP_S1>
1875 inline ap_private& operator+=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
1876     VAL = (ValType)((((uint64_t)VAL) & RHS.get_VAL());
1877     clearUnusedBits();
1878     return *this;
1879 }
1880
1881 template <int _AP_W1, bool _AP_S1>
1882 inline ap_private& operator|=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
1883     VAL = (ValType)((((uint64_t)VAL) | RHS.get_VAL());
1884     clearUnusedBits();
1885     return *this;
1886 }

```

```

1887
1888 template <int _AP_W1, bool _AP_S1>
1889 inline ap_private& operator^=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
1890     VAL = (ValType)((((uint64_t)VAL) ^ RHS.get_VAL());
1891     clearUnusedBits();
1892     return *this;
1893 }
1894
1895 template <int _AP_W1, bool _AP_S1>
1896 inline ap_private& operator*=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
1897     VAL = (ValType)((((uint64_t)VAL) * RHS.get_VAL());
1898     clearUnusedBits();
1899     return *this;
1900 }
1901
1902 template <int _AP_W1, bool _AP_S1>
1903 inline ap_private& operator+=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
1904     VAL = (ValType)((((uint64_t)VAL) + RHS.get_VAL());
1905     clearUnusedBits();
1906     return *this;
1907 }
1908
1909 template <int _AP_W1, bool _AP_S1>
1910 inline ap_private& operator-=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
1911     VAL = (ValType)((((uint64_t)VAL) - RHS.get_VAL());
1912     clearUnusedBits();
1913     return *this;
1914 }
1915
1916 template <int _AP_W1, bool _AP_S1>
1917 inline typename RType<_AP_W1, _AP_S1>::logic operator&(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
1918     if (RType<_AP_W1, _AP_S1>::logic_w <= 64) {
1919         typename RType<_AP_W1, _AP_S1>::logic
Ret((((uint64_t)VAL) &
1920 RHS.get_VAL());
1921         return Ret;
1922     } else {
1923         typename RType<_AP_W1, _AP_S1>::logic Ret = *this;
1924         return Ret & RHS;
1925     }
1926 }
1927
1928 template <int _AP_W1, bool _AP_S1>
1929 inline typename RType<_AP_W1, _AP_S1>::logic operator^(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
1930     if (RType<_AP_W1, _AP_S1>::logic_w <= 64) {
1931         typename RType<_AP_W1, _AP_S1>::logic
Ret((((uint64_t)VAL) ^
1932 RHS.get_VAL());
1933         return Ret;
1934     } else {
1935         typename RType<_AP_W1, _AP_S1>::logic Ret = *this;
1936         return Ret ^ RHS;
1937     }
1938 }
1939
1940 template <int _AP_W1, bool _AP_S1>
1941 inline typename RType<_AP_W1, _AP_S1>::logic operator|(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
1942     if (RType<_AP_W1, _AP_S1>::logic_w <= 64) {
1943         typename RType<_AP_W1, _AP_S1>::logic
Ret((((uint64_t)VAL) |
1944 RHS.get_VAL());
1945         return Ret;
1946     } else {
1947         typename RType<_AP_W1, _AP_S1>::logic Ret = *this;
1948         return Ret | RHS;
1949     }
1950 }
1951
1952 inline ap_private And(const ap_private& RHS) const {
1953     return ap_private(VAL & RHS.get_VAL());
1954 }
1955
1956 inline ap_private Or(const ap_private& RHS) const {
1957     return ap_private(VAL | RHS.get_VAL());
1958 }
1959
1960 inline ap_private Xor(const ap_private& RHS) const {
1961     return ap_private(VAL ^ RHS.get_VAL());
1962 }
1963
1964 #if 1
1965 template <int _AP_W1, bool _AP_S1>
1966 inline typename RType<_AP_W1, _AP_S1>::mult operator*(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
1967     if (RType<_AP_W1, _AP_S1>::mult_w <= 64) {
1968         typename RType<_AP_W1, _AP_S1>::mult Res-
ult((((uint64_t)VAL) *
1969 RHS.get_VAL());
1970         return Result;
1971     } else {

```



```

1975     typename RType<_AP_W1, _AP_S1>::mult Result(*this);
1976     Result *= RHS;
1977     return Result;
1978 }
1979 }
1980 #endif
1981 inline ap_private Mul(const ap_private& RHS) const {
1982     return ap_private(VAL * RHS.get_VAL());
1983 }
1984
1985 inline ap_private Add(const ap_private& RHS) const {
1986     return ap_private(VAL + RHS.get_VAL());
1987 }
1988
1989 inline ap_private Sub(const ap_private& RHS) const {
1990     return ap_private(VAL - RHS.get_VAL());
1991 }
1992
1993 inline ap_private& operator&=(uint64_t RHS) {
1994     VAL &= (ValType)RHS;
1995     clearUnusedBits();
1996     return *this;
1997 }
1998
1999 inline ap_private& operator|=(uint64_t RHS) {
2000     VAL |= (ValType)RHS;
2001     clearUnusedBits();
2002     return *this;
2003 }
2004
2005 inline ap_private& operator^=(uint64_t RHS) {
2006     VAL ^= (ValType)RHS;
2007     clearUnusedBits();
2008     return *this;
2009 }
2010
2011 inline ap_private& operator*=(uint64_t RHS) {
2012     VAL *= (ValType)RHS;
2013     clearUnusedBits();
2014     return *this;
2015 }
2016
2017 inline ap_private& operator+=(uint64_t RHS) {
2018     VAL += (ValType)RHS;
2019     clearUnusedBits();
2020     return *this;
2021 }
2022
2023 inline ap_private& operator-=(uint64_t RHS) {
2024     VAL -= (ValType)RHS;
2025     clearUnusedBits();
2026     return *this;
2027 }
2028
2029 inline bool isMinSignedValue() const {
2030     static const uint64_t min_mask = ~(-0ULL << (_AP_W - 1));
2031     return BitWidth == 1 ? VAL == 1
2032         :
2033         (ap_private_ops::isNegative<_AP_W>(*this) &&
2034          ((min_mask & VAL) == 0));
2035 }
2036
2037 template <int _AP_W1, bool _AP_S1>
2038 inline typename RType<_AP_W1, _AP_S1>::plus operator+(
2039     const ap_private<_AP_W1, _AP_S1>& RHS) const {
2040     if (RType<_AP_W1, _AP_S1>::plus_w <= 64)
2041         return typename RType<_AP_W1, _AP_S1>::plus(
2042             RType<_AP_W1, _AP_S1>::plus_s
2043             ? int64_t(((uint64_t)VAL) + RHS.get_VAL())
2044             : uint64_t(((uint64_t)VAL) + RHS.get_VAL()));
2045     typename RType<_AP_W1, _AP_S1>::plus Result = RHS;
2046     Result += VAL;
2047     return Result;
2048 }
2049
2050 template <int _AP_W1, bool _AP_S1>
2051 inline typename RType<_AP_W1, _AP_S1>::minus operator-(
2052     const ap_private<_AP_W1, _AP_S1>& RHS) const {
2053     if (RType<_AP_W1, _AP_S1>::minus_w <= 64)
2054         return typename RType<_AP_W1, _AP_S1>::minus(
2055             int64_t(((uint64_t)VAL) - RHS.get_VAL()));
2056     typename RType<_AP_W1, _AP_S1>::minus Result = *this;
2057     Result -= RHS;
2058     return Result;
2059 }
2060
2061 inline uint32_t countPopulation() const {
2062     return ap_private_ops::CountPopulation_64(VAL);
2063 }
2064
2065 inline uint32_t countLeadingZeros() const {
2066     int remainder = BitWidth % 64;
2067     int excessBits = (64 - remainder) % 64;
2068     uint32_t Count =
2069         ap_private_ops::CountLeadingZeros_64(VAL);
2070     if (Count) Count -= excessBits;
2071     return AESL_std::min(Count, (uint32_t)_AP_W);
2072 }
2073
2074 /// HiBits - This function returns the high "numBits" bits
2075 of this ap_private.
2076
2077 inline ap_private<_AP_W, _AP_S> getHiBits(uint32_t numBits)
2078 const {
2079     ap_private<_AP_W, _AP_S> ret(*this);
2080     ret = (ret) >> (BitWidth - numBits);

```

```

2070     return ret;
2071 }
2072
2073 /// LoBits - This function returns the low "numBits" bits
2074 of this ap_private.
2075
2076 inline ap_private<_AP_W, _AP_S> getLoBits(uint32_t numBits)
2077 const {
2078     ap_private<_AP_W, _AP_S> ret(((uint64_t)VAL) << (BitWidth
2079 - numBits));
2080     ret = (ret) >> (BitWidth - numBits);
2081     return ret;
2082     // return ap_private(numBits, (VAL << (BitWidth -
2083 numBits))) >> (BitWidth -
2084 // numBits));
2085 }
2086
2087 inline ap_private<_AP_W, _AP_S> set(uint32_t bitPosition)
2088 {
2089     VAL |= (1ULL << (bitPosition));
2090     clearUnusedBits();
2091     return *this; // clearUnusedBits();
2092 }
2093
2094 inline void set() {
2095     VAL = (ValType)-0ULL;
2096     clearUnusedBits();
2097 }
2098
2099 template <int _AP_W3>
2100 inline void set(const ap_private<_AP_W3, false>& val) {
2101     operator=(ap_private<_AP_W3, _AP_S>(val));
2102 }
2103
2104 inline void set(const ap_private& val) { operator=(val); }
2105
2106 inline void clearUnusedBits(void) volatile
2107 // just for clang compiler
2108 #if defined(__clang__) && !defined(__CLANG_3_1__)
2109 __attribute__((no_sanitize("undefined")))
2110 #endif
2111 {
2112     enum { excess_bits = (_AP_W % 64) ? 64 - _AP_W % 64 :
2113 0 };
2114     VAL = (ValType)(
2115         _AP_S
2116         ? (((int64_t)VAL) << (excess_bits)) >> (ex-
2117 cess_bits))
2118         : (excess_bits ? (((uint64_t)VAL) << (ex-
2119 cess_bits)) >> (excess_bits)
2120           : (uint64_t)VAL));
2121 }
2122
2123 inline void clearUnusedBitsToZero(void) {
2124     enum { excess_bits = (_AP_W % 64) ? 64 - _AP_W % 64 :
2125 0 };
2126     static uint64_t mask = -0ULL >> (excess_bits);
2127     VAL &= mask;
2128 }
2129
2130 inline ap_private udiv(const ap_private& RHS) const {
2131     return ap_private((uint64_t)VAL / RHS.get_VAL());
2132 }
2133
2134 /// Signed divide this ap_private by ap_private RHS.
2135 /// @brief Signed division function for ap_private.
2136 inline ap_private sdiv(const ap_private& RHS) const {
2137     if (isNegative())
2138         if (RHS.isNegative())
2139             return ((uint64_t)(0 - (*this))) / (uint64_t)(0 -
2140 RHS);
2141         else
2142             return 0 - ((uint64_t)(0 - (*this))) / (uint64_t)
2143 (RHS));
2144     else if (RHS.isNegative())
2145         return 0 - (this->udiv((ap_private)(0 - RHS)));
2146     return this->udiv(RHS);
2147 }
2148
2149 template <bool _AP_S2>
2150 inline ap_private urem(const ap_private<_AP_W, _AP_S2>&
2151 RHS) const {
2152     assert(RHS.get_VAL() != 0 && "Divide by 0");
2153     return ap_private(((uint64_t)VAL) %
2154 ((uint64_t)RHS.get_VAL()));
2155 }
2156
2157 /// Signed remainder operation on ap_private.
2158 /// @brief Function for signed remainder operation.
2159 template <bool _AP_S2>
2160 inline ap_private srem(const ap_private<_AP_W, _AP_S2>&
2161 RHS) const {
2162     if (isNegative()) {
2163         ap_private lhs = 0 - (*this);
2164         if (RHS.isNegative()) {
2165             ap_private rhs = 0 - RHS;
2166             return 0 - (lhs.urem(rhs));
2167         } else
2168             return 0 - (lhs.urem(RHS));
2169     }

```

```

2154     } else if (RHS.isNegative()) {
2155         ap_private rhs = 0 - RHS;
2156         return this->urem(rhs);
2157     }
2158     return this->urem(RHS);
2159 }
2160
2161 template <int _AP_W1, bool _AP_S1>
2162
2162 INLINE bool eq(const ap_private<_AP_W1, _AP_S1>& RHS) const
2163 {
2164     return (*this) == RHS;
2165 }
2166
2167 template <int _AP_W1, bool _AP_S1>
2168
2168 INLINE bool ne(const ap_private<_AP_W1, _AP_S1>& RHS) const
2169 {
2170     return !((*this) == RHS);
2171 }
2172
2173 // @brief Signed less than comparison
2174 // @returns true if *this < RHS when both are considered
2175 // signed.
2176 // @brief Unsigned less than comparison
2177 // @returns true if *this < RHS when both are considered
2178 // unsigned.
2179 template <int _AP_W1, bool _AP_S1>
2180
2180 INLINE bool ult(const ap_private<_AP_W1, _AP_S1>& RHS)
2181 const {
2182     if (_AP_W1 <= 64) {
2183         uint64_t lhsZext = ((uint64_t(VAL)) << (64 - _AP_W1)) >>
2184         (64 - _AP_W1);
2185         uint64_t rhsZext =
2186         ((uint64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2187 - _AP_W1);
2188         return lhsZext < rhsZext;
2189     } else
2190         return RHS.uge(*this);
2191 }
2192
2193 // @brief Signed less than comparison
2194 // @returns true if *this < RHS when both are considered
2195 // signed.
2196 // @brief Unsigned less than comparison
2197 // @returns true if *this < RHS when both are considered
2198 // unsigned.
2199 template <int _AP_W1, bool _AP_S1>
2200
2200 INLINE bool slt(const ap_private<_AP_W1, _AP_S1>& RHS)
2201 const
2202 // just for clang compiler
2203 #if defined(__clang__) && !defined(__CLANG_3_1__)
2204 __attribute__((no_sanitize("undefined")))
2205 #endif
2206 {
2207     if (_AP_W1 <= 64) {
2208         int64_t lhsSext = ((int64_t(VAL)) << (64 - _AP_W1)) >>
2209         (64 - _AP_W1);
2210         int64_t rhsSext =
2211         ((int64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2212 - _AP_W1);
2213         return lhsSext < rhsSext;
2214     } else
2215         return RHS.sge(*this);
2216 }
2217
2218 // @brief Signed less or equal comparison
2219 // @returns true if *this <= RHS when both are considered
2220 // signed.
2221 // @brief Unsigned less or equal comparison
2222 // @returns true if *this <= RHS when both are considered
2223 // unsigned.
2224 template <int _AP_W1, bool _AP_S1>
2225
2225 INLINE bool sle(const ap_private<_AP_W1, _AP_S1>& RHS)
2226 const {
2227     return ult(RHS) || eq(RHS);
2228 }
2229
2230 // @brief Signed less or equal comparison
2231 // @returns true if *this <= RHS when both are considered
2232 // signed.
2233 // @brief Unsigned less or equal comparison
2234 // @returns true if *this <= RHS when both are considered
2235 // unsigned.
2236 template <int _AP_W1, bool _AP_S1>
2237
2237 INLINE bool sle(const ap_private<_AP_W1, _AP_S1>& RHS)
2238 const {
2239     return slt(RHS) || eq(RHS);
2240 }
2241
2242 // @brief Signed greater than comparison
2243 // @returns true if *this > RHS when both are considered
2244 // signed.
2245 // @brief Unsigned greater than comparison
2246 // @returns true if *this > RHS when both are considered
2247 // unsigned.
2248 template <int _AP_W1, bool _AP_S1>
2249
2249 INLINE bool ugt(const ap_private<_AP_W1, _AP_S1>& RHS)
2250 const {
2251     if (_AP_W1 <= 64) {
2252         uint64_t lhsZext = ((uint64_t(VAL)) << (64 - _AP_W1)) >>
2253         (64 - _AP_W1);
2254         uint64_t rhsZext =
2255         ((uint64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2256 - _AP_W1);
2257         return lhsZext > rhsZext;
2258     } else
2259         return RHS.ule(*this);
2260 }
2261
2262 // @brief Signed greater than comparison
2263 // @returns true if *this > RHS when both are considered
2264 // signed.
2265 // @brief Unsigned greater than comparison
2266 // @returns true if *this > RHS when both are considered
2267 // unsigned.
2268 template <int _AP_W1, bool _AP_S1>
2269
2269 INLINE bool sgt(const ap_private<_AP_W1, _AP_S1>& RHS)
2270 const {
2271     if (_AP_W1 <= 64) {
2272         int64_t lhsSext = ((int64_t(VAL)) << (64 - _AP_W1)) >>
2273         (64 - _AP_W1);
2274         int64_t rhsSext =
2275         ((int64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2276 - _AP_W1);
2277         return lhsSext > rhsSext;
2278     } else
2279         return RHS.ult(*this);
2280 }
2281
2282 // @brief Signed greater or equal comparison
2283 // @returns true if *this >= RHS when both are considered
2284 // signed.
2285 // @brief Unsigned greater or equal comparison
2286 // @returns true if *this >= RHS when both are considered
2287 // unsigned.
2288 template <int _AP_W1, bool _AP_S1>
2289
2289 INLINE bool sge(const ap_private<_AP_W1, _AP_S1>& RHS)
2290 const {
2291     return sgt(RHS) || eq(RHS);
2292 }
2293
2294 // @brief Signed greater or equal comparison
2295 // @returns true if *this >= RHS when both are considered
2296 // signed.
2297 // @brief Unsigned greater or equal comparison
2298 // @returns true if *this >= RHS when both are considered
2299 // unsigned.
2300 template <int _AP_W1, bool _AP_S1>
2301
2301 INLINE bool uge(const ap_private<_AP_W1, _AP_S1>& RHS)
2302 const {
2303     if (_AP_W1 <= 64) {
2304         uint64_t lhsZext = ((uint64_t(VAL)) << (64 - _AP_W1)) >>
2305         (64 - _AP_W1);
2306         uint64_t rhsZext =
2307         ((uint64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2308 - _AP_W1);
2309         return lhsZext >= rhsZext;
2310     } else
2311         return RHS.ult(*this);
2312 }
2313
2314 // @brief Signed greater or equal comparison
2315 // @returns true if *this >= RHS when both are considered
2316 // signed.
2317 // @brief Unsigned greater or equal comparison
2318 // @returns true if *this >= RHS when both are considered
2319 // unsigned.
2320 template <int _AP_W1, bool _AP_S1>
2321
2321 INLINE bool sge(const ap_private<_AP_W1, _AP_S1>& RHS)
2322 const {
2323     return sgt(RHS) || eq(RHS);
2324 }
2325
2326 // @brief Signed greater or equal comparison
2327 // @returns true if *this >= RHS when both are considered
2328 // signed.
2329 // @brief Unsigned greater or equal comparison
2330 // @returns true if *this >= RHS when both are considered
2331 // unsigned.
2332 template <int _AP_W1, bool _AP_S1>
2333
2333 INLINE bool uge(const ap_private<_AP_W1, _AP_S1>& RHS)
2334 const {
2335     if (_AP_W1 <= 64) {
2336         uint64_t lhsZext = ((uint64_t(VAL)) << (64 - _AP_W1)) >>
2337         (64 - _AP_W1);
2338         uint64_t rhsZext =
2339         ((uint64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2340 - _AP_W1);
2341         return lhsZext >= rhsZext;
2342     } else
2343         return RHS.ult(*this);
2344 }
2345
2346 // @brief Signed greater or equal comparison
2347 // @returns true if *this >= RHS when both are considered
2348 // signed.
2349 // @brief Unsigned greater or equal comparison
2350 // @returns true if *this >= RHS when both are considered
2351 // unsigned.
2352 template <int _AP_W1, bool _AP_S1>
2353
2353 INLINE bool sge(const ap_private<_AP_W1, _AP_S1>& RHS)
2354 const {
2355     return sgt(RHS) || eq(RHS);
2356 }
2357
2358 // @brief Signed greater or equal comparison
2359 // @returns true if *this >= RHS when both are considered
2360 // signed.
2361 // @brief Unsigned greater or equal comparison
2362 // @returns true if *this >= RHS when both are considered
2363 // unsigned.
2364 template <int _AP_W1, bool _AP_S1>
2365
2365 INLINE bool uge(const ap_private<_AP_W1, _AP_S1>& RHS)
2366 const {
2367     if (_AP_W1 <= 64) {
2368         uint64_t lhsZext = ((uint64_t(VAL)) << (64 - _AP_W1)) >>
2369         (64 - _AP_W1);
2370         uint64_t rhsZext =
2371         ((uint64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2372 - _AP_W1);
2373         return lhsZext >= rhsZext;
2374     } else
2375         return RHS.ult(*this);
2376 }
2377
2378 // @brief Signed greater or equal comparison
2379 // @returns true if *this >= RHS when both are considered
2380 // signed.
2381 // @brief Unsigned greater or equal comparison
2382 // @returns true if *this >= RHS when both are considered
2383 // unsigned.
2384 template <int _AP_W1, bool _AP_S1>
2385
2385 INLINE bool sge(const ap_private<_AP_W1, _AP_S1>& RHS)
2386 const {
2387     return sgt(RHS) || eq(RHS);
2388 }
2389
2390 // @brief Signed greater or equal comparison
2391 // @returns true if *this >= RHS when both are considered
2392 // signed.
2393 // @brief Unsigned greater or equal comparison
2394 // @returns true if *this >= RHS when both are considered
2395 // unsigned.
2396 template <int _AP_W1, bool _AP_S1>
2397
2397 INLINE bool uge(const ap_private<_AP_W1, _AP_S1>& RHS)
2398 const {
2399     if (_AP_W1 <= 64) {
2400         uint64_t lhsZext = ((uint64_t(VAL)) << (64 - _AP_W1)) >>
2401         (64 - _AP_W1);
2402         uint64_t rhsZext =
2403         ((uint64_t(RHS.get_VAL())) << (64 - _AP_W1)) >> (64
2404 - _AP_W1);
2405         return lhsZext >= rhsZext;
2406     } else
2407         return RHS.ult(*this);
2408 }
2409
2410 // @brief Signed greater or equal comparison
2411 // @returns true if *this >= RHS when both are considered
2412 // signed.
2413 // @brief Unsigned greater or equal comparison
2414 // @returns true if
```

```

2230     return !ult(RHS) && !eq(RHS);
2231 }
2232
2233 /// Regards both *this and RHS as signed quantities and
compares them for
2234 /// the validity of the greater-than relationship.
2235 /// @returns true if *this > RHS when both are considered
signed.
2236 /// @brief Signed greater than comparison
2237 template <int _AP_W1, bool _AP_S1>
2238 inline bool sgt(const ap_private<_AP_W1, _AP_S1> &RHS)
const {
2239     return !slt(RHS) && !eq(RHS);
2240 }
2241
2242 /// Regards both *this and RHS as unsigned quantities and
compares them for
2243 /// validity of the greater-or-equal relationship.
2244 /// @returns true if *this >= RHS when both are considered
unsigned.
2245 /// @brief Unsigned greater or equal comparison
2246 template <int _AP_W1, bool _AP_S1>
2247 inline bool uge(const ap_private<_AP_W1, _AP_S1> &RHS)
const {
2248     return !ult(RHS);
2249 }
2250
2251 /// Regards both *this and RHS as signed quantities and
compares them for
2252 /// validity of the greater-or-equal relationship.
2253 /// @returns true if *this >= RHS when both are considered
signed.
2254 /// @brief Signed greater or equal comparison
2255 template <int _AP_W1, bool _AP_S1>
2256 inline bool sge(const ap_private<_AP_W1, _AP_S1> &RHS)
const {
2257     return !slt(RHS);
2258 }
2259
2260 inline ap_private abs() const {
2261     if (isNegative()) return -*this;
2262     return *this;
2263 }
2264
2265 inline ap_private<_AP_W, false> get() const {
2266     ap_private<_AP_W, false> ret(*this);
2267     return ret;
2268 }
2269
2270 inline static uint32_t getBitsNeeded(const char* str,
uint32_t slen,
2271                                     uint8_t radix) {
2272     return _AP_W;
2273 }
2274
2275 inline uint32_t getActiveBits() const {
2276     uint32_t bits = _AP_W - countLeadingZeros();
2277     return bits ? bits : 1;
2278 }
2279
2280 inline double roundToDouble(bool isSigned = false) const {
2281     return isSigned ? double((int64_t)VAL) :
double((uint64_t)VAL);
2282 }
2283
2284 /*Reverse the contents of ap_private instance. I.e. LSB be-
comes MSB and vice
2285 * versa*/
2286 inline ap_private& reverse() {
2287     for (int i = 0; i < _AP_W / 2; ++i) {
2288         bool tmp = operator[](i);
2289         if (operator[](_AP_W - 1 - i))
2290             set(i);
2291         else
2292             clear(i);
2293         if (tmp)
2294             set(_AP_W - 1 - i);
2295         else
2296             clear(_AP_W - 1 - i);
2297     }
2298     clearUnusedBits();
2299     return *this;
2300 }
2301
2302 /*Return true if the value of ap_private instance is zero*/
2303 inline bool iszero() const { return isMinValue(); }
2304
2305 inline bool to_bool() const { return !iszero(); }
2306
2307 /* x < 0 */
2308 inline bool sign() const {
2309     if (isNegative()) return true;
2310     return false;
2311 }
2312
2313 /* x[i] = !x[i] */
2314 inline void invert(int i) {
2315     assert(i >= 0 && "Attempting to read bit with negative
index");

```

```

2316     assert(i < _AP_W && "Attempting to read bit beyond MSB");
2317     flip(i);
2318 }
2319
2320 /* x[i] */
2321 INLINE bool test(int i) const {
2322     assert(i >= 0 && "Attempting to read bit with negative
index");
2323     assert(i < _AP_W && "Attempting to read bit beyond MSB");
2324     return operator[](i);
2325 }
2326
2327 // This is used for sc_lv and sc_bv, which is implemented
by sc_uint
2328 // Rotate an ap_private object n places to the left
2329 INLINE void lrotate(int n) {
2330     assert(n >= 0 && "Attempting to shift negative index");
2331     assert(n < _AP_W && "Shift value larger than bit width");
2332     operator=(shl(n) | lshr(_AP_W - n));
2333 }
2334
2335 // This is used for sc_lv and sc_bv, which is implemented
by sc_uint
2336 // Rotate an ap_private object n places to the right
2337 INLINE void rrotate(int n) {
2338     assert(n >= 0 && "Attempting to shift negative index");
2339     assert(n < _AP_W && "Shift value larger than bit width");
2340     operator=(lshr(n) | shl(_AP_W - n));
2341 }
2342
2343 // Set the ith bit into v
2344 INLINE void set(int i, bool v) {
2345     assert(i >= 0 && "Attempting to write bit with negative
index");
2346     assert(i < _AP_W && "Attempting to write bit beyond
MSB");
2347     v ? set(i) : clear(i);
2348 }
2349
2350 // Set the ith bit into v
2351 INLINE void set_bit(int i, bool v) {
2352     assert(i >= 0 && "Attempting to write bit with negative
index");
2353     assert(i < _AP_W && "Attempting to write bit beyond
MSB");
2354     v ? set(i) : clear(i);
2355 }
2356
2357 // Get the value of ith bit
2358 INLINE bool get_bit(int i) const {
2359     assert(i >= 0 && "Attempting to read bit with negative
index");
2360     assert(i < _AP_W && "Attempting to read bit beyond MSB");
2361     return (((1ULL << i) & VAL) != 0);
2362 }
2363
2364 /// Toggle all bits.
2365 INLINE ap_private& flip() {
2366     VAL = (ValType)((~0ULL ^ VAL) & mask);
2367     clearUnusedBits();
2368     return *this;
2369 }
2370
2371 /// Toggles a given bit to its opposite value.
2372 INLINE ap_private& flip(uint32_t bitPosition) {
2373     assert(bitPosition < BitWidth && "Out of the bit-width
range!");
2374     set_bit(bitPosition, !get_bit(bitPosition));
2375     return *this;
2376 }
2377
2378 // complements every bit
2379 INLINE void b_not() { flip(); }
2380
2381 // Binary Arithmetic
2382 //-----
2383 #define OP_BIN_AP(Sym, Rty, Fun) \
2384     template <int _AP_W2, bool _AP_S2> \
2385     INLINE typename RType<_AP_W2, _AP_S2>::Rty operator Sym( \
2386         const ap_private<_AP_W2, _AP_S2>& op) const { \
2387         typename RType<_AP_W2, _AP_S2>::Rty lhs(*this); \
2388         typename RType<_AP_W2, _AP_S2>::Rty rhs(op); \
2389         return lhs.Fun(rhs); \
2390     }
2391
2392 /// Bitwise and, or, xor
2393 // OP_BIN_AP(&, logic, And)
2394 // OP_BIN_AP(|, logic, Or)
2395 // OP_BIN_AP(^, logic, Xor)
2396 #undef OP_BIN_AP
2397
2398 template <int _AP_W2, bool _AP_S2>
2399 INLINE typename RType<_AP_W2, _AP_S2>::div operator/(
2400     const ap_private<_AP_W2, _AP_S2>& op) const {
2401     ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
2402         (_AP_W > _AP_W2 ? _AP_S
2403             : (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
2404         lhs = *this;

```

```

2405     ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
2406         (_AP_W > _AP_W2 ? _AP_S
2407             : (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
2408         rhs = op;
2409     return typename RType<_AP_W2, _AP_S2>::div(
2410         (_AP_S || _AP_S2) ? lhs.sdiv(rhs) : lhs.udiv(rhs));
2411 }
2412
2413 template <int _AP_W2, bool _AP_S2>
2414 INLINE typename RType<_AP_W2, _AP_S2>::mod operator%(
2415     const ap_private<_AP_W2, _AP_S2>& op) const {
2416     ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
2417         (_AP_W > _AP_W2 ? _AP_S
2418             : (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
2419         lhs = *this;
2420     ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
2421         (_AP_W > _AP_W2 ? _AP_S
2422             : (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
2423         rhs = op;
2424     typename RType<_AP_W2, _AP_S2>::mod res =
2425         typename RType<_AP_W2, _AP_S2>::mod(_AP_S ?
lhs.srem(rhs)
2426             :
lhs.urem(rhs));
2427     return res;
2428 }
2429
2430 #define OP_ASSIGN_AP_2(Sym) \
2431     template <int _AP_W2, bool _AP_S2> \
2432     INLINE ap_private<_AP_W, _AP_S>& operator Sym##=( \
2433         const ap_private<_AP_W2, _AP_S2>& op) { \
2434         *this = operator Sym(op); \
2435         return *this; \
2436     }
2437
2438 OP_ASSIGN_AP_2(/)
2439 OP_ASSIGN_AP_2(%)
2440 #undef OP_ASSIGN_AP_2
2441
2442 /// Bitwise assign: and, or, xor
2443 //-----
2444 // OP_ASSIGN_AP(&)
2445 // OP_ASSIGN_AP(^)
2446 // OP_ASSIGN_AP(|)
2447
2448 #define OP_LEFT_SHIFT_CTYPE(TYPE, SIGNED) \
2449     INLINE ap_private operator<<(const TYPE op) const { \
2450         if (op >= _AP_W) return ap_private(0); \
2451         if (SIGNED && op < 0) return *this >> (0 - op); \
2452         return shl(op); \
2453     }
2454
2455 // OP_LEFT_SHIFT_CTYPE(bool, false)
2456 OP_LEFT_SHIFT_CTYPE(char, CHAR_IS_SIGNED)
2457 OP_LEFT_SHIFT_CTYPE(signed char, true)
2458 OP_LEFT_SHIFT_CTYPE(unsigned char, false)
2459 OP_LEFT_SHIFT_CTYPE(short, true)
2460 OP_LEFT_SHIFT_CTYPE(unsigned short, false)
2461 OP_LEFT_SHIFT_CTYPE(int, true)
2462 OP_LEFT_SHIFT_CTYPE(unsigned int, false)
2463 OP_LEFT_SHIFT_CTYPE(long, true)
2464 OP_LEFT_SHIFT_CTYPE(unsigned long, false)
2465 OP_LEFT_SHIFT_CTYPE(long long, true)
2466 OP_LEFT_SHIFT_CTYPE(unsigned long long, false)
2467 #if 0
2468 OP_LEFT_SHIFT_CTYPE(half, false)
2469 OP_LEFT_SHIFT_CTYPE(float, false)
2470 OP_LEFT_SHIFT_CTYPE(double, false)
2471 #endif
2472
2473 #undef OP_LEFT_SHIFT_CTYPE
2474
2475 template <int _AP_W2, bool _AP_S2>
2476 INLINE ap_private operator<<(const ap_private<_AP_W2,
_AP_S2>& op2) const {
2477     if (_AP_S2 == false) {
2478         uint32_t sh = op2.to_uint();
2479         return *this << sh;
2480     } else {
2481         int sh = op2.to_int();
2482         return *this << sh;
2483     }
2484 }
2485
2486 #define OP_RIGHT_SHIFT_CTYPE(TYPE, SIGNED) \
2487     INLINE ap_private operator>>(const TYPE op) const { \
2488         if (op >= _AP_W) { \
2489             if (isNegative()) \
2490                 return ap_private(-1); \
2491             else \
2492                 return ap_private(0); \
2493         } \
2494         if ((SIGNED) && op < 0) return *this << (0 - op); \

```

```

2495     if (_AP_S) \
2496         return ashr(op); \
2497     else \
2498         return lshr(op); \
2499 }
2500
2501 // OP_RIGHT_SHIFT_CTYPE(bool, false)
2502 OP_RIGHT_SHIFT_CTYPE(char, CHAR_IS_SIGNED)
2503 OP_RIGHT_SHIFT_CTYPE(signed char, true)
2504 OP_RIGHT_SHIFT_CTYPE(unsigned char, false)
2505 OP_RIGHT_SHIFT_CTYPE(short, true)
2506 OP_RIGHT_SHIFT_CTYPE(unsigned short, false)
2507 OP_RIGHT_SHIFT_CTYPE(int, true)
2508 OP_RIGHT_SHIFT_CTYPE(unsigned int, false)
2509 OP_RIGHT_SHIFT_CTYPE(long, true)
2510 OP_RIGHT_SHIFT_CTYPE(unsigned long, false)
2511 OP_RIGHT_SHIFT_CTYPE(unsigned long long, false)
2512 OP_RIGHT_SHIFT_CTYPE(long long, true)
2513 #if 0
2514 OP_RIGHT_SHIFT_CTYPE(half, false)
2515 OP_RIGHT_SHIFT_CTYPE(float, false)
2516 OP_RIGHT_SHIFT_CTYPE(double, false)
2517 #endif
2518
2519 #undef OP_RIGHT_SHIFT_CTYPE
2520
2521 template <int _AP_W2, bool _AP_S2>
2522 inline ap_private operator>>(const ap_private<_AP_W2,
2523 _AP_S2>& op2) const {
2524     if (_AP_S2 == false) {
2525         uint32_t sh = op2.to_uint();
2526         return *this >> sh;
2527     } else {
2528         int sh = op2.to_int();
2529         return *this >> sh;
2530     }
2531 }
2532
2533 /// Shift assign
2534 //-----
2535 //inline const ap_private& operator<<=(uint32_t shiftAmt) {
2536 //    VAL <<= shiftAmt;
2537 //    clearUnusedBits();
2538 //    return *this;
2539 //}
2540
2541 #define
OP_ASSIGN_AP(Sym)
\
2542 template <int _AP_W2, bool
2543 _AP_S2>
2544 inline ap_private& operator Sym##=(int op)
2545 {
2546     *this = operator
2547     \
2548     Sym(op);
2549     clearUnused-
2550     \
2551     Bits();
2552     return
2553     \
2554     *this;
2555 }
2556
2557 inline ap_private& operator Sym##=(unsigned int op)
2558 {
2559     *this = operator
2560     \
2561     Sym(op);
2562     clearUnused-
2563     \
2564     Bits();
2565     return
2566     \
2567     *this;
2568 }
2569
2570 template <int _AP_W2, bool
2571 _AP_S2>
2572
2573 inline ap_private& operator Sym##=(const ap_private<_AP_W2,
2574 _AP_S2>& op) { \
2575     *this = operator
2576     \
2577     Sym(op);
2578     clearUnused-
2579     \
2580     Bits();
2581     return
2582     \
2583     *this;
2584 }
2585
2586 // Comparisons
2587 //-----
2588
2589 template <int _AP_W1, bool _AP_S1>
2590 inline bool operator==(const ap_private<_AP_W1, _AP_S1>&
2591 op) const {
2592     enum { _AP_MAX_W = AP_MAX(AP_MAX(_AP_W, _AP_W1), 32) };
2593     ap_private<_AP_MAX_W, false> lhs(*this);
2594
2595     ap_private<_AP_MAX_W, false> rhs(op);
2596     if (_AP_MAX_W <= 64) {
2597         return (uint64_t)lhs.get_VAL() ==
2598         (uint64_t)rhs.get_VAL();
2599     } else
2600         return lhs == rhs;
2601 }
2602
2603 template <int _AP_W2, bool _AP_S2>
2604 inline bool operator!=(const ap_private<_AP_W2, _AP_S2>&
2605 op) const {
2606     return !(*this == op);
2607 }
2608
2609 template <int _AP_W2, bool _AP_S2>
2610 inline bool operator>(const ap_private<_AP_W2, _AP_S2>& op)
2611 const {
2612     enum {
2613         _AP_MAX_W = AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
2614         (_AP_S || _AP_S2))
2615     };
2616     ap_private<_AP_MAX_W, _AP_S> lhs(*this);
2617     ap_private<_AP_MAX_W, _AP_S2> rhs(op);
2618     // this will follow gcc rule for comparison
2619     // between different bitwidth and signness
2620     if (_AP_S == _AP_S2)
2621         return _AP_S ? lhs.sgt(rhs) : lhs.ugt(rhs);
2622     else if (_AP_W < 32 && _AP_W2 < 32)
2623         // different signness but both bitwidth is less than 32
2624         return lhs.sgt(rhs);
2625     else
2626         // different signness but bigger bitwidth
2627         // is greater or equal to 32
2628         if (_AP_S)
2629             if (_AP_W2 >= _AP_W)
2630                 return lhs.ugt(rhs);
2631             else
2632                 return lhs.sgt(rhs);
2633         else if (_AP_W >= _AP_W2)
2634             return lhs.ugt(rhs);
2635         else
2636             return lhs.sgt(rhs);
2637 }
2638
2639 template <int _AP_W2, bool _AP_S2>
2640 inline bool operator<=(const ap_private<_AP_W2, _AP_S2>&
2641 op) const {
2642     return !(*this > op);
2643 }
2644
2645 template <int _AP_W2, bool _AP_S2>
2646 inline bool operator<(const ap_private<_AP_W2, _AP_S2>& op)
2647 const {
2648     enum {
2649         _AP_MAX_W = AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
2650         (_AP_S || _AP_S2))
2651     };
2652     ap_private<_AP_MAX_W, _AP_S> lhs(*this);
2653     ap_private<_AP_MAX_W, _AP_S2> rhs(op);
2654     if (_AP_S == _AP_S2)
2655         return _AP_S ? lhs.slt(rhs) : lhs.ult(rhs);
2656     else if (_AP_W < 32 && _AP_W2 < 32)
2657         return lhs.slt(rhs);
2658     else if (_AP_S)
2659         if (_AP_W2 >= _AP_W)
2660             return lhs.ult(rhs);
2661         else
2662             return lhs.slt(rhs);
2663     else if (_AP_W >= _AP_W2)
2664         return lhs.ult(rhs);
2665     else
2666         return lhs.slt(rhs);
2667 }
2668
2669 template <int _AP_W2, bool _AP_S2>
2670 inline bool operator>=(const ap_private<_AP_W2, _AP_S2>&
2671 op) const {
2672     return !(*this < op);
2673 }
2674
2675 /// Bit and Part Select
2676 //-----
2677
2678 // FIXME now _private_range_ref refs to
2679 _AP_ROOT_TYPE(struct ssdm_int).
2680 inline _private_range_ref<_AP_W, _AP_S> operator()(int Hi,
2681 int Lo) {
2682     return _private_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
2683 }
2684
2685 inline _private_range_ref<_AP_W, _AP_S> operator()(int Hi,
2686 int Lo) const {
2687     return _private_range_ref<_AP_W, _AP_S>{
2688         const_cast<ap_private<_AP_W, _AP_S>*>(this), Hi, Lo);
2689 }
2690
2691 inline _private_range_ref<_AP_W, _AP_S> range(int Hi, int
2692 Lo) const {

```

```

2655     return _private_range_ref<_AP_W, _AP_S>(
2656         (const_cast<ap_private<_AP_W, _AP_S>*>(this)), Hi,
Lo);
2657     }
2658
2659     INLINE _private_range_ref<_AP_W, _AP_S> range(int Hi, int
Lo) {
2660         return _private_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
2661     }
2662
2663     INLINE _private_bit_ref<_AP_W, _AP_S> operator[](int index)
{
2664         return _private_bit_ref<_AP_W, _AP_S>(*this, index);
2665     }
2666
2667     template <int _AP_W2, bool _AP_S2>
2668     INLINE _private_bit_ref<_AP_W, _AP_S> operator[] (
2669         const ap_private<_AP_W2, _AP_S2>& index) {
2670         return _private_bit_ref<_AP_W, _AP_S>(*this, in-
dex.to_int());
2671     }
2672
2673     INLINE const _private_bit_ref<_AP_W, _AP_S> operator[](int
index) const {
2674         return _private_bit_ref<_AP_W, _AP_S>(
2675             const_cast<ap_private<_AP_W, _AP_S>*>(*this), index);
2676     }
2677
2678     template <int _AP_W2, bool _AP_S2>
2679     INLINE const _private_bit_ref<_AP_W, _AP_S> operator[] (
2680         const ap_private<_AP_W2, _AP_S2>& index) const {
2681         return _private_bit_ref<_AP_W, _AP_S>(
2682             const_cast<ap_private<_AP_W, _AP_S>*>(*this), in-
dex.to_int());
2683     }
2684
2685     INLINE _private_bit_ref<_AP_W, _AP_S> bit(int index) {
2686         return _private_bit_ref<_AP_W, _AP_S>(*this, index);
2687     }
2688
2689     template <int _AP_W2, bool _AP_S2>
2690     INLINE _private_bit_ref<_AP_W, _AP_S> bit(const
ap_private<_AP_W2, _AP_S2>& index) {
2691         return _private_bit_ref<_AP_W, _AP_S>(*this, in-
dex.to_int());
2692     }
2693
2694     INLINE const _private_bit_ref<_AP_W, _AP_S> bit(int index)
const {
2695         return _private_bit_ref<_AP_W, _AP_S>(
2696             const_cast<ap_private<_AP_W, _AP_S>*>(*this), index);
2697     }
2698
2699     template <int _AP_W2, bool _AP_S2>
2700     INLINE const _private_bit_ref<_AP_W, _AP_S> bit(
2701         const ap_private<_AP_W2, _AP_S2>& index) const {
2702         return _private_bit_ref<_AP_W, _AP_S>(
2703             const_cast<ap_private<_AP_W, _AP_S>*>(*this), in-
dex.to_int());
2704     }
2705
2706     // template <int _AP_W2, bool _AP_S2>
2707     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2708     //         ap_private<_AP_W2, _AP_S2> >
2709     // concat(const ap_private<_AP_W2, _AP_S2>& a2) const {
2710     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2711     //         ap_private<_AP_W2, _AP_S2> > (
2712     //         const_cast<ap_private<_AP_W, _AP_S>*>(*this),
2713     //         const_cast<ap_private<_AP_W2, _AP_S2>*>(a2));
2714     // }
2715     //
2716     // template <int _AP_W2, bool _AP_S2>
2717     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2718     //         ap_private<_AP_W2, _AP_S2> >
2719     // concat(ap_private<_AP_W2, _AP_S2>& a2) {
2720     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2721     //         ap_private<_AP_W2, _AP_S2> >
2722     // }
2723     //
2724     // template <int _AP_W2, bool _AP_S2>
2725     // INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
2726     // operator,(const ap_private<_AP_W2, _AP_S2> &a2) const {
2727     //     return ap_concat_ref<_AP_W, ap_private, _AP_W2,
2728     //         ap_private<_AP_W2, _AP_S2> > (
2729     //         const_cast<ap_private<_AP_W, _AP_S>*>(*this),
2730     //         const_cast<ap_private<_AP_W2, _AP_S2>*>(a2));
2731     // }
2732     //
2733     // template <int _AP_W2, bool _AP_S2>
2734     // INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
2735     // operator,(const ap_private<_AP_W2, _AP_S2> &a2) {
2736     //     return ap_concat_ref<_AP_W, ap_private, _AP_W2,

```

```

2737     //         ap_private<_AP_W2, _AP_S2> > (
2738     //         *this, const_cast<ap_private<_AP_W2,
_AP_S2>*>(a2));
2739     // }
2740     //
2741     // template <int _AP_W2, bool _AP_S2>
2742     // INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
2743     // operator,(ap_private<_AP_W2, _AP_S2> &a2) const {
2744     //     return ap_concat_ref<_AP_W, ap_private, _AP_W2,
2745     //         ap_private<_AP_W2, _AP_S2> > (
2746     //         const_cast<ap_private<_AP_W, _AP_S>*>(*this), a2);
2747     // }
2748     //
2749     // template <int _AP_W2, bool _AP_S2>
2750     // INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
2751     // operator,(ap_private<_AP_W2, _AP_S2> &a2) {
2752     //     return ap_concat_ref<_AP_W, ap_private, _AP_W2,
2753     //         ap_private<_AP_W2, _AP_S2> >
2754     // }
2755     //
2756     // template <int _AP_W2, bool _AP_S2>
2757     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2758     //         _private_range_ref<_AP_W2, _AP_S2> >
2759     // operator,(const _private_range_ref<_AP_W2, _AP_S2> &a2)
const {
2760     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2761     //         _private_range_ref<_AP_W2, _AP_S2> >
2762     // }
2763     //
2764     // const_cast<ap_private<_AP_W, _AP_S>*>(*this),
2765     // const_cast<_private_range_ref<_AP_W2,
_AP_S2>*>(a2));
2766     // }
2767     //
2768     // template <int _AP_W2, bool _AP_S2>
2769     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2770     //         _private_range_ref<_AP_W2, _AP_S2> >
2771     // operator,(_private_range_ref<_AP_W2, _AP_S2> &a2) {
2772     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
2773     //         _private_range_ref<_AP_W2, _AP_S2> >
2774     // }
2775     //
2776     // template <int _AP_W2, bool _AP_S2>
2777     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>, 1,
2778     //         _private_bit_ref<_AP_W2, _AP_S2> >
2779     // operator,(const _private_bit_ref<_AP_W2, _AP_S2> &a2)
const {
2780     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
1,
2781     //         _private_bit_ref<_AP_W2, _AP_S2> >
2782     // }
2783     //
2784     // template <int _AP_W2, bool _AP_S2>
2785     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>, 1,
2786     //         _private_bit_ref<_AP_W2, _AP_S2> >
2787     // operator,(_private_bit_ref<_AP_W2, _AP_S2> &a2) {
2788     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
1,
2789     //         _private_bit_ref<_AP_W2, _AP_S2> >
2790     // }
2791     //
2792     // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
2793     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,
2794     //         ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >
2795     // operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &a2) const {
2796     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,
2797     //         ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> > (
2798     //         const_cast<ap_private<_AP_W, _AP_S>*>(*this),
2799     //         const_cast<ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>*>(a2));
2800     // }
2801     //
2802     // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
2803     // INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,
2804     //         ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >
2805     // operator,(ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>
&a2) {
2806     //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,

```



```

_AP_W2 + _AP_W3,
2807 //          ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >(*this,
2808 //
2809 //     a2);
2810 // }
2811 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
2812 //          ap_o_mode _AP_O2, int _AP_N2>
2813 //     INLINE ap_concat_ref<
2814 //          _AP_W, ap_private, _AP_W2,
2815 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >
2816 //     operator,(const af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
2817 //          &a2) const {
2818 //     return ap_concat_ref<
2819 //          _AP_W, ap_private, _AP_W2,
2820 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> >(*
2821 //          const_cast<ap_private<_AP_W, _AP_S>&)(*this),
2822 //          const_cast<
2823 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&(&a2));
2824 //     }
2825 //
2826 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
2827 //          ap_o_mode _AP_O2, int _AP_N2>
2828 //     INLINE ap_concat_ref<
2829 //          _AP_W, ap_private, _AP_W2,
2830 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >
2831 //     operator,(af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
2832 //     return ap_concat_ref<
2833 //          _AP_W, ap_private, _AP_W2,
2834 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> >(*this,
2835 //
2836 //     a2);
2837 //     }
2838 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
2839 //          ap_o_mode _AP_O2, int _AP_N2>
2840 //     INLINE
2841 //     ap_concat_ref<_AP_W, ap_private, 1,
2842 //     af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
2843 //     operator,(const af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
2844 //          &a2) const {
2845 //     return ap_concat_ref<
2846 //          _AP_W, ap_private, 1,
2847 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*
2848 //          const_cast<ap_private<_AP_W, _AP_S>&)(*this),
2849 //          const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>&(&
2850 //          a2));
2851 //     }
2852 //
2853 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
2854 //          ap_o_mode _AP_O2, int _AP_N2>
2855 //     INLINE
2856 //     ap_concat_ref<_AP_W, ap_private, 1,
2857 //     af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
2858 //     operator,(
2859 //     af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
2860 //     return ap_concat_ref<
2861 //          _AP_W, ap_private, 1,
2862 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*this, a2);
2863 //     }
2864 //
2865 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
2866 //     INLINE ap_private<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator&
2867 //     (const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
2868 //     return *this & a2.get();
2869 //     }
2870 //
2871 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
2872 //     INLINE ap_private<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator|
2873 //     (const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
2874 //     return *this | a2.get();
2875 //     }
2876 //

```

```

2877 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
2878 //     INLINE ap_private<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator^
2879 //     (const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
2880 //     return *this ^ a2.get();
2881 //     }
2882 //
2883 // Reduce operation
2884 // -----
--
2885 //     INLINE bool and_reduce() const { return (VAL & mask) ==
mask; }
2886 //
2887 //     INLINE bool nand_reduce() const { return (VAL & mask) !=
mask; }
2888 //
2889 //     INLINE bool or_reduce() const { return (bool)VAL; }
2890 //
2891 //     INLINE bool nor_reduce() const { return VAL == 0; }
2892 //
2893 //     INLINE bool xor_reduce() const {
2894 //     unsigned int i = countPopulation();
2895 //     return (i % 2) ? true : false;
2896 //     }
2897 //
2898 //     INLINE bool xnor_reduce() const {
2899 //     unsigned int i = countPopulation();
2900 //     return (i % 2) ? false : true;
2901 //     }
2902 //
2903 //     INLINE std::string to_string(uint8_t radix = 2, bool sign =
false) const {
2904 //     return toString(radix, radix == 10 ? _AP_S : sign);
2905 //     }
2906 // }; // End of class ap_private <_AP_W, _AP_S, true>
2907 //
2908 // template <int _AP_W, bool _AP_S>
2909 //     std::string ap_private<_AP_W, _AP_S, true>::toString(uint8_t
radix,
2910 //     bool
wantSigned) const {
2911 //     assert((radix == 10 || radix == 8 || radix == 16 || radix
== 2) &&
2912 //           "Radix should be 2, 8, 10, or 16!");
2913 //     static const char* digits[] = {"0", "1", "2", "3", "4",
"5", "6", "7",
2914 //           "8", "9", "a", "b", "c",
"d", "e", "f"};
2915 //     std::string result;
2916 //     if (radix != 10) {
2917 //     // For the 2, 8 and 16 bit cases, we can just shift in-
stead of divide
2918 //     // because the number of bits per digit (1,3 and 4 re-
spectively) divides
2919 //     // equally. We just shift until there value is zero.
2920 //
2921 //     // First, check for a zero value and just short circuit
the logic below.
2922 //     if (*this == (uint64_t)(0)) {
2923 //     // Always generate a radix indicator because fixed-
point
2924 //     // formats require it.
2925 //     switch (radix) {
2926 //     case 2:
2927 //     result = "0b0";
2928 //     break;
2929 //     case 8:
2930 //     result = "0o0";
2931 //     break;
2932 //     case 16:
2933 //     result = "0x0";
2934 //     break;
2935 //     default:
2936 //     assert("invalid radix" && 0);
2937 //     }
2938 //     } else {
2939 //     ap_private<_AP_W, false, true> tmp(*this);
2940 //     size_t insert_at = 0;
2941 //     bool leading_zero = true;
2942 //     if (wantSigned && isNegative()) {
2943 //     // They want to print the signed version and it is a
negative value
2944 //     // Flip the bits and add one to turn it into the
equivalent positive
2945 //     // value and put a '-' in the result.
2946 //     tmp.flip();
2947 //     tmp++;
2948 //     result = "-";
2949 //     insert_at = 1;
2950 //     leading_zero = false;
2951 //     }
2952 //     switch (radix) {
2953 //     case 2:
2954 //     result += "0b";
2955 //     break;
2956 //     case 8:
2957 //     result += "0o";

```

```

2958         break;
2959     case 16:
2960         result += "0x";
2961         break;
2962     default:
2963         assert("invalid radix" && 0);
2964     }
2965     insert_at += 2;
2966
2967     // Just shift tmp right for each digit width until it
becomes zero
2968     uint32_t shift = (radix == 16 ? 4 : (radix == 8 ? 3 :
1));
2969     uint64_t mask = radix - 1;
2970     ap_private<_AP_W, false, true> zero(0);
2971     unsigned bits = 0;
2972     bool msb = false;
2973     while (tmp.ne(zero)) {
2974         unsigned digit = (unsigned)(tmp.get_VAL() & mask);
2975         result.insert(insert_at, digits[digit]);
2976         tmp = tmp.lshr(shift);
2977         bits++;
2978         msb = (digit >> (shift - 1)) == 1;
2979     }
2980     bits *= shift;
2981     if (bits < _AP_W && leading_zero && msb)
2982         result.insert(insert_at, digits[0]);
2983 }
2984 return result;
2985 }
2986
2987 ap_private<_AP_W, false, true> tmp(*this);
2988 ap_private<6, false, true> divisor(radix);
2989 ap_private<_AP_W, _AP_S, true> zero(0);
2990 size_t insert_at = 0;
2991 if (wantSigned && isNegative()) {
2992     // They want to print the signed version and it is a neg-
ative value
2993     // Flip the bits and add one to turn it into the equival-
ent positive
2994     // value and put a '-' in the result.
2995     tmp.flip();
2996     tmp++;
2997     result = "-";
2998     insert_at = 1;
2999 }
3000 if (tmp == ap_private<_AP_W, false, true>(0ULL))
3001     result = "0";
3002 else
3003     while (tmp.ne(zero)) {
3004         ap_private<_AP_W, false, true> APdigit = tmp % divisor;
3005         ap_private<_AP_W, false, true> tmp2 = tmp / divisor;
3006         uint32_t digit = (uint32_t)(APdigit.getZExtValue());
3007         assert(digit < radix && "divide failed");
3008         result.insert(insert_at, digits[digit]);
3009         tmp = tmp2;
3010     }
3011 return result;
3012 }
3013 } // End of ap_private<_AP_W, _AP_S, true>::toString()
3014
3015 // bitwidth > 64
3016 template <int _AP_W, bool _AP_S>
3017 class ap_private<_AP_W, _AP_S, false> {
3018     // SFINAE pattern. Only consider this class when _AP_W >
64
3019     const static bool valid = ap_private_enable_if<(_AP_W >
64)>::isValid;
3020
3021 #ifdef _MSC_VER
3022 #pragma warning(disable : 4521 4522)
3023 #endif
3024 public:
3025     enum { BitWidth = _AP_W, _AP_N = (_AP_W + 63) / 64 };
3026     static const int width = _AP_W;
3027
3028 private:
3029     /// This constructor is used only internally for speed of
construction of
3030     /// temporaries. It is unsafe for general use so it is not
public.
3031
3032     /* Constructors */
3033     /// Note that numWords can be smaller or larger than the
corresponding bit
3034     /// width but any extraneous bits will be dropped.
3035     /// @param numWords the number of words in bigVal
3036     /// @param bigVal a sequence of words to form the initial
value of the
3037     /// ap_private
3038     /// @brief Construct an ap_private, initialized as bigV-
al[[]].
3039     INLINE ap_private(uint32_t numWords, const uint64_t bigV-
al[[]]) {
3040         set_canary();
3041         assert(bigVal && "Null pointer detected!");
3042         {
3043             // Get memory, cleared to 0
3044             memset(pVal, 0, _AP_N * sizeof(uint64_t));
3045
3046             // Calculate the number of words to copy
3047             uint32_t words = AESL_std::min<uint32_t>(numWords,
_AP_N);
3048
3049             // Copy the words from bigVal to pVal
3050             memcpy(pVal, bigVal, words * APINT_WORD_SIZE);
3051             if (words >= _AP_W) clearUnusedBits();
3052             // Make sure unused high bits are cleared
3053             check_canary();
3054         }
3055     }
3056     /// This constructor interprets Val as a string in the giv-
en radix. The
3057     /// interpretation stops when the first character that is
not suitable for the
3058     /// radix is encountered. Acceptable radix values are 2, 8,
10 and 16. It is
3059     /// an error for the value implied by the string to require
more bits than
3060     /// numBits.
3061     /// @param val the string to be interpreted
3062     /// @param radix the radix of Val to use for the interpreta-
tion
3063     /// @brief Construct an ap_private from a string represent-
ation.
3064     INLINE ap_private(const std::string& val, uint8_t radix =
2) {
3065         set_canary();
3066         assert(!val.empty() && "The input string is empty.");
3067         const char* c_str = val.c_str();
3068         fromString(c_str, val.size(), radix);
3069         check_canary();
3070     }
3071
3072     /// This constructor interprets the slen characters start-
ing at StrStart as
3073     /// a string in the given radix. The interpretation stops
when the first
3074     /// character that is not suitable for the radix is en-
countered. Acceptable
3075     /// radix values are 2, 8, 10 and 16. It is an error for
the value implied by
3076     /// the string to require more bits than numBits.
3077     /// @param strStart the start of the string to be inter-
preted
3078     /// @param slen the maximum number of characters to inter-
pret
3079     /// @param radix the radix to use for the conversion
3080     /// @brief Construct an ap_private from a string represent-
ation.
3081     /// This method does not consider whether it is negative or
not.
3082     INLINE ap_private(const char strStart[], uint32_t slen,
uint8_t radix) {
3083         set_canary();
3084         fromString(strStart, slen, radix);
3085         check_canary();
3086     }
3087
3088     INLINE void report() {
3089         _AP_ERROR(_AP_W > MAX_MODE(AP_INT_MAX_W) * 1024,
3090             "ap_%sint<td>: Bitwidth exceeds the "
3091             "default max value %d. Please use macro "
3092             "AP_INT_MAX_W to set a larger max value.",
3093             _AP_S ? "" : "u", _AP_W, MAX_MODE(AP_INT_MAX_W)
* 1024);
3094     }
3095     /// This union is used to store the integer value. When the
3096     /// integer bit-width <= 64, it uses VAL, otherwise it uses
pVal.
3097
3098     /// This enum is used to hold the constants we needed for
ap_private.
3099     /// uint64_t VAL; ///< Used to store the <= 64 bits in-
teger value.
3100     uint64_t pVal[_AP_N]; ///< Used to store the >64 bits in-
teger value.
3101 #ifdef AP_CANARY
3102     uint64_t CANARY;
3103     INLINE void check_canary() { assert(CANARY ==
(uint64_t)0xDEADBEEFDEADBEEF); }
3104     INLINE void set_canary() { CANARY =
(uint64_t)0xDEADBEEFDEADBEEF; }
3105 #else
3106     INLINE void check_canary() {}
3107     INLINE void set_canary() {}
3108 #endif
3109
3110 public:
3111     typedef typename valtype<8, _AP_S>::Type ValType;
3112     typedef ap_private<_AP_W, _AP_S> Type;
3113     // FIXME remove friend type?
3114     template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
3115         ap_o_mode _AP_O2, int _AP_N2>

```

```

3116 friend struct ap_fixed_base;
3117 /// return type of variety of operations
3118 //-----
3119 template <int _AP_W2, bool _AP_S2>
3120 struct RType {
3121     enum {
3122         mult_w = _AP_W + _AP_W2,
3123         mult_s = _AP_S || _AP_S2,
3124         plus_w =
3125             AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 + (_AP_S
&& !_AP_S2)) + 1,
3126         plus_s = _AP_S || _AP_S2,
3127         minus_w =
3128             AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 + (_AP_S
&& !_AP_S2)) + 1,
3129         minus_s = true,
3130         div_w = _AP_W + _AP_S2,
3131         div_s = _AP_S || _AP_S2,
3132         mod_w = AP_MIN(_AP_W, _AP_W2 + (!_AP_S2 && _AP_S)),
3133         mod_s = _AP_S,
3134         logic_w = AP_MAX(_AP_W + (_AP_S2 && !_AP_S), _AP_W2 +
(_AP_S && !_AP_S2)),
3135         logic_s = _AP_S || _AP_S2
3136     };
3137     typedef ap_private<mult_w, mult_s> mult;
3138     typedef ap_private<plus_w, plus_s> plus;
3139     typedef ap_private<minus_w, minus_s> minus;
3140     typedef ap_private<logic_w, logic_s> logic;
3141     typedef ap_private<div_w, div_s> div;
3142     typedef ap_private<mod_w, mod_s> mod;
3143     typedef ap_private<_AP_W, _AP_S> arg1;
3144     typedef bool reduce;
3145 };
3146
3147 inline uint64_t& get_VAL(void) { return pVal[0]; }
3148 inline uint64_t get_VAL(void) const { return pVal[0]; }
3149 inline uint64_t& get_VAL(void) const volatile { return
pVal[0]; }
3150 inline void set_VAL(uint64_t value) { pVal[0] = value; }
3151 inline uint64_t& get_pVal(int index) { return pVal[in-
dex]; }
3152 inline uint64_t* get_pVal() { return pVal; }
3153 inline const uint64_t* get_pVal() const { return pVal; }
3154 inline uint64_t get_pVal(int index) const { return pVal[in-
dex]; }
3155 inline uint64_t* get_pVal() const volatile { return pVal; }
3156 inline uint64_t get_pVal(int index) const volatile { return
pVal[index]; }
3157 inline void set_pVal(int i, uint64_t value) { pVal[i] =
value; }
3158
3159 /// This enum is used to hold the constants we needed for
ap_private.
3160 enum {
3161     APINT_BITS_PER_WORD = sizeof(uint64_t) * 8, ///< Bits in
a word
3162     APINT_WORD_SIZE = sizeof(uint64_t)          ///< Byte
size of a word
3163 };
3164
3165 enum {
3166     excess_bits = (_AP_W % APINT_BITS_PER_WORD)
? APINT_BITS_PER_WORD - (_AP_W %
APINT_BITS_PER_WORD)
3167     : 0
3168 };
3169
3170 static const uint64_t mask = ((uint64_t)-0ULL >> (ex-
cess_bits));
3171
3172 public:
3173     // NOTE changed to explicit to be consistent with
ap_private<W,S,true>
3174     explicit inline ap_private(const char* val) {
3175         set_canary();
3176         unsigned char radix = 10;
3177         std::string str = ap_private_ops::parseString(val,
radix); // determine radix.
3178         std::string::size_type pos = str.find('.');
3179         if (pos != std::string::npos) str = str.substr(pos);
3180         ap_private ap_private_val(str, radix);
3181         operator=(ap_private_val);
3182         report();
3183         check_canary();
3184     }
3185
3186     inline ap_private(const char* val, unsigned char rd) {
3187         set_canary();
3188         unsigned char radix = rd;
3189         std::string str = ap_private_ops::parseString(val,
radix); // determine radix.
3190         std::string::size_type pos = str.find('.');
3191         if (pos != std::string::npos) str = str.substr(pos);
3192         ap_private ap_private_val(str, radix);
3193         operator=(ap_private_val);
3194         report();
3195
3196         report();
3197         check_canary();

```

```

3198     }
3199
3200     template <int _AP_W2, bool _AP_S2>
3201     inline ap_private(const _private_range_ref<_AP_W2, _AP_S2>&
ref) {
3202         set_canary();
3203         *this = ref.get();
3204         report();
3205         check_canary();
3206     }
3207
3208     template <int _AP_W2, bool _AP_S2>
3209     inline ap_private(const _private_bit_ref<_AP_W2, _AP_S2>&
ref) {
3210         set_canary();
3211         *this = ((uint64_t)(bool)ref);
3212         report();
3213         check_canary();
3214     }
3215
3216     // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
3217     // inline ap_private(const ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3>& ref) {
3218     //     set_canary();
3219     //     *this = ref.get();
3220     //     report();
3221     //     check_canary();
3222     // }
3223
3224     // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
3225     //     ap_o_mode _AP_O2, int _AP_N2>
3226     // inline ap_private(
3227     //     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
3228     //     set_canary();
3229     //     *this = ((val.operator ap_private<_AP_W2, false>()));
3230     //     report();
3231     //     check_canary();
3232     // }
3233
3234     // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
3235     //     ap_o_mode _AP_O2, int _AP_N2>
3236     // inline ap_private(
3237     //     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>& val) {
3238     //     set_canary();
3239     //     *this = (uint64_t)(bool)val;
3240     //     report();
3241     //     check_canary();
3242     // }
3243
3244     /// Simply makes *this a copy of that.
3245     /// @brief Copy Constructor.
3246     inline ap_private(const ap_private& that) {
3247         set_canary();
3248         memcpy(pVal, that.get_pVal(), _AP_N * APINT_WORD_SIZE);
3249         clearUnusedBits();
3250         check_canary();
3251     }
3252
3253     template <int _AP_W1, bool _AP_S1>
3254     inline ap_private(const ap_private<_AP_W1, _AP_S1, false>&
that) {
3255         set_canary();
3256         operator=(that);
3257         check_canary();
3258     }
3259
3260     template <int _AP_W1, bool _AP_S1>
3261     inline ap_private(const volatile ap_private<_AP_W1, _AP_S1,
false>& that) {
3262         set_canary();
3263         operator=(const_cast<const ap_private<_AP_W1, _AP_S1,
false>&>(that));
3264         check_canary();
3265     }
3266
3267     template <int _AP_W1, bool _AP_S1>
3268     inline ap_private(const ap_private<_AP_W1, _AP_S1, true>&
that) {
3269         set_canary();
3270         static const uint64_t that_sign_ext_mask =
(_AP_W1 == APINT_BITS_PER_WORD)
? 0
: ~0ULL >> (_AP_W1 % APINT_BITS_PER_WORD)
<< (_AP_W1 % APINT_BITS_PER_WORD);
3271         if (that.isNegative()) {
3272             pVal[0] = that.get_VAL() | that_sign_ext_mask;
3273             memset(pVal + 1, -0, sizeof(uint64_t) * (_AP_N - 1));
3274         } else {
3275             pVal[0] = that.get_VAL();
3276             memset(pVal + 1, 0, sizeof(uint64_t) * (_AP_N - 1));
3277         }
3278         clearUnusedBits();
3279         check_canary();
3280     }

```



```

3284 }
3285
3286 template <int _AP_W1, bool _AP_S1>
3287
3288 inline ap_private(const volatile ap_private<_AP_W1, _AP_S1,
true>& that) {
3288     set_canary();
3289     operator=(const_cast<const ap_private<_AP_W1, _AP_S1,
true>&>(that));
3290     check_canary();
3291 }
3292
3293 /// @brief Destructor.
3294 // virtual ~ap_private() {}
3295 inline ~ap_private() { check_canary(); }
3296
3297 /// @name Constructors
3298 /// @{
3299
3300 /// Default constructor that creates an uninitialized
ap_private. This is
3301 /// useful
3302 /// for object deserialization (pair this with the static
method Read).
3303 inline ap_private() {
3304     set_canary();
3305     clearUnusedBits();
3306     check_canary();
3307 }
3308
3309 inline ap_private(uint64_t* val, uint32_t bits = _AP_W)
{ assert(0); }
3310
3311 inline ap_private(const uint64_t* const val, uint32_t bits)
{ assert(0); }
3312
3313 /// If isSigned is true then val is treated as if it were a
signed value
3314 /// (i.e. as an int64_t) and the appropriate sign extension
to the bit width
3315 /// will be done. Otherwise, no sign extension occurs (high
order bits beyond
3316 /// the range of val are zero filled).
3317 /// @param numBits the bit width of the constructed
ap_private
3318 /// @param val the initial value of the ap_private
3319 /// @param isSigned how to treat signedness of val
3320 /// @brief Create a new ap_private of numBits width, initial-
ized as val.
3321 #define CTOR(TYPE, SIGNED) \
3322     inline ap_private(TYPE val, bool isSigned = SIGNED) { \
3323         set_canary(); \
3324         pVal[0] = (ValType)val; \
3325         if (isSigned && int64_t(pVal[0]) < 0) { \
3326             memset(pVal + 1, ~0, sizeof(uint64_t) * (_AP_N - 1)); \
3327         } else { \
3328             memset(pVal + 1, 0, sizeof(uint64_t) * (_AP_N - 1)); \
3329         } \
3330         clearUnusedBits(); \
3331         check_canary(); \
3332     }
3333
3334 CTOR(bool, false)
3335 CTOR(char, CHAR_IS_SIGNED)
3336 CTOR(signed char, true)
3337 CTOR(unsigned char, false)
3338 CTOR(short, true)
3339 CTOR(unsigned short, false)
3340 CTOR(int, true)
3341 CTOR(unsigned int, false)
3342 CTOR(long, true)
3343 CTOR(unsigned long, false)
3344 CTOR(ap_slong, true)
3345 CTOR(ap_ulong, false)
3346 #if 0
3347 CTOR(half, false)
3348 CTOR(float, false)
3349 CTOR(double, false)
3350 #endif
3351 #undef CTOR
3352
3353 /// @returns true if the number of bits <= 64, false other-
wise.
3354 /// @brief Determine if this ap_private just has one word
to store value.
3355 inline bool isSingleWord() const { return false; }
3356
3357 /// @returns the word position for the specified bit posi-
tion.
3358 /// @brief Determine which word a bit is in.
3359 static inline uint32_t whichWord(uint32_t bitPosition) {
3360     // return bitPosition / APINT_BITS_PER_WORD;
3361     return (bitPosition) >> 6;
3362 }
3363
3364 /// @returns the bit position in a word for the specified
bit position
3365 /// in the ap_private.
3366 /// @brief Determine which bit in a word a bit is in.
3367 static inline uint32_t whichBit(uint32_t bitPosition) {

```

```

3367     // return bitPosition % APINT_BITS_PER_WORD;
3368     return bitPosition & 0x3f;
3369 }
3370
3371 /// bit at a specific bit position. This is used to mask
the bit in the
3372 /// corresponding word.
3373 /// @returns a uint64_t with only bit at "which-
Bit(bitPosition)" set
3374 /// @brief Get a single bit mask.
3375 static inline uint64_t maskBit(uint32_t bitPosition) {
3376     return 1ULL << (whichBit(bitPosition));
3377 }
3378
3379 /// @returns the corresponding word for the specified bit
position.
3380 /// @brief Get the word corresponding to a bit position
3381 inline uint64_t getWord(uint32_t bitPosition) const {
3382     return pVal[whichWord(bitPosition)];
3383 }
3384
3385 /// This method is used internally to clear the to "N" bits
in the high order
3386 /// word that are not used by the ap_private. This is
needed after the most
3387 /// significant word is assigned a value to ensure that
those bits are
3388 /// zero'd out.
3389 /// @brief Clear unused high order bits
3390 inline void clearUnusedBits(void) volatile
3391 // just for clang compiler
3392 #if defined(__clang__) && !defined(__CLANG_3_1__)
3393     __attribute__((no_sanitize("undefined")))
3394 #endif
3395 {
3396     pVal[_AP_N - 1] =
3397         _AP_S ? (((int64_t)pVal[_AP_N - 1]) << (ex-
cess_bits)) >> excess_bits
3398             : (excess_bits
3399                 ? ((pVal[_AP_N - 1]) << (excess_bits))
3400                 : pVal[_AP_N - 1]);
3401 }
3402
3403 inline void clearUnusedBitsToZero(void) { pVal[_AP_N - 1]
&= mask; }
3404
3405 inline void clearUnusedBitsToOne(void) { pVal[_AP_N - 1] |=
mask; }
3406
3407 /// This is used by the constructors that take string argu-
ments.
3408 /// @brief Convert a char array into an ap_private
3409 inline void fromString(const char* str, uint32_t slen,
uint8_t radix) {
3410     enum { numbits = _AP_W };
3411     bool isNeg = str[0] == '-';
3412     if (isNeg) {
3413         str++;
3414         slen--;
3415     }
3416
3417     if (str[0] == '0' && (str[1] == 'b' || str[1] == 'B')) {
3418         //if (radix == 0) radix = 2;
3419         _AP_WARNING(radix != 2, "%s seems to have base %d, but
%d given.", str, 2, radix);
3420         str += 2;
3421         slen -= 2;
3422     } else if (str[0] == '0' && (str[1] == 'o' || str[1] ==
'o')) {
3423         //if (radix == 0) radix = 8;
3424         _AP_WARNING(radix != 8, "%s seems to have base %d, but
%d given.", str, 8, radix);
3425         str += 2;
3426         slen -= 2;
3427     } else if (str[0] == '0' && (str[1] == 'x' || str[1] ==
'X')) {
3428         //if (radix == 0) radix = 16;
3429         _AP_WARNING(radix != 16, "%s seems to have base %d, but
%d given.", str, 16, radix);
3430         str += 2;
3431         slen -= 2;
3432     } else if (str[0] == '0' && (str[1] == 'd' || str[1] ==
'D')) {
3433         //if (radix == 0) radix = 10;
3434         _AP_WARNING(radix != 10, "%s seems to have base %d, but
%d given.", str, 10, radix);
3435         str += 2;
3436         slen -= 2;
3437     } else if (radix == 0) {
3438         //radix = 2; // XXX default value
3439     }
3440
3441     // Check our assumptions here
3442     assert((radix == 10 || radix == 8 || radix == 16 || radix

```

```

3443 == 2) &&
3444     "Radix should be 2, 8, 10, or 16!");
3445 assert(str && "String is null?");
3446 // skip any leading zero
3447 while (*str == '0' && *(str + 1) != '\0') {
3448     str++;
3449     slen--;
3450 }
3451 assert((slen <= numbits || radix != 2) && "Insufficient
bit width");
3452 assert(((slen - 1) * 3 <= numbits || radix != 8) &&
3453     "Insufficient bit width");
3454 assert(((slen - 1) * 4 <= numbits || radix != 16) &&
3455     "Insufficient bit width");
3456 assert((((slen - 1) * 64) / 22 <= numbits || radix != 10)
&&
3457     "Insufficient bit width");
3458 // clear bits
3459 memset(pVal, 0, _AP_N * sizeof(uint64_t));
3460 // Figure out if we can shift instead of multiply
3461 uint32_t shift = (radix == 16 ? 4 : radix == 8 ? 3 :
radix == 2 ? 1 : 0);
3462 // Set up an ap_private for the digit to add outside the
loop so we don't
3463 // constantly construct/deconstruct it.
3464 uint64_t bigVal[_AP_N];
3465 memset(bigVal, 0, _AP_N * sizeof(uint64_t));
3466 ap_private<_AP_W, _AP_S> apdigit(getBitWidth(), bigVal);
3467 ap_private<_AP_W, _AP_S> apradix(radix);
3468 // Enter digit traversal loop
3469 for (unsigned i = 0; i < slen; i++) {
3470     // Get a digit
3471     uint32_t digit = 0;
3472     char cdigit = str[i];
3473     if (radix == 16) {
3474         #define isxdi-
3475         git(c)
3476         (((c) >= '0' && (c) <= '9') || ((c) >= 'a' && (c) <= 'f')
|| \
3477         ((c) >= 'A' && (c) <= 'F'))
3478         #define isdigit(c) ((c) >= '0' && (c) <= '9')
3479         if (!isdigit(cdigit)) assert(0 && "Invalid hex digit
in string");
3480         if (isdigit(cdigit))
3481             digit = cdigit - '0';
3482         else if (cdigit >= 'a')
3483             digit = cdigit - 'a' + 10;
3484         else if (cdigit >= 'A')
3485             digit = cdigit - 'A' + 10;
3486         else
3487             assert(0 && "huh? we shouldn't get here");
3488         } else if (isdigit(cdigit)) {
3489             digit = cdigit - '0';
3490         } else if (cdigit != '\0') {
3491             assert(0 && "Invalid character in digit string");
3492         }
3493         #undef isxdi-
3494         #undef isdigit
3495         // Shift or multiply the value by the radix
3496         if (shift)
3497             *this <<= shift;
3498         else
3499             *this *= apradix;
3500         // Add in the digit we just interpreted
3501         apdigit.set_VAL(digit);
3502         *this += apdigit;
3503         // If its negative, put it in two's complement form
3504         if (isNeg) {
3505             (*this)--;
3506             this->flip();
3507         }
3508         clearUnusedBits();
3509     }
3510 }
3511
3512
3513
3514
3515
3516
3517
3518
3519
3520
3521
3522
3523
3524
3525
3526
3527
3528
3529
3530
3531
3532
3533
3534
3535
3536
3537
3538
3539
3540
3541
3542
3543
3544
3545
3546
3547
3548
3549
3550
3551
3552
3553
3554
3555
3556
3557
3558
3559
3560
3561
3562
3563
3564
3565
3566
3567
3568
3569
3570
3571
3572
3573
3574
3575
3576
3577
3578
3579
3580
3581
3582
3583
3584
3585
3586
3587
3588
3589
3590
3591
3592
3593
3594
3595
3596
3597
3598
3599
3600
3601
3602
3603
3604
3605
3606
3607
3608
3609
3610
3611
3612
3613
3614
3615
3616
3617
3618
3619
3620
3621
3622
3623
3624
3625
3626
3627
3628
3629
3630
3631
3632
3633
3634
3635
3636
3637
3638
3639
3640
3641
3642
3643
3644
3645
3646
3647
3648
3649
3650
3651
3652
3653
3654
3655
3656
3657
3658
3659
3660
3661
3662
3663
3664
3665
3666
3667
3668
3669
3670
3671
3672
3673
3674
3675
3676
3677
3678
3679
3680
3681
3682
3683
3684
3685
3686
3687
3688
3689
3690
3691
3692
3693
3694
3695
3696
3697
3698
3699
3700
3701
3702
3703
3704
3705
3706
3707
3708
3709
3710
3711
3712
3713
3714
3715
3716
3717
3718
3719
3720
3721
3722
3723
3724
3725
3726
3727
3728
3729
3730
3731
3732
3733
3734
3735
3736
3737
3738
3739
3740
3741
3742
3743
3744
3745
3746
3747
3748
3749
3750
3751
3752
3753
3754
3755
3756
3757
3758
3759
3760
3761
3762
3763
3764
3765
3766
3767
3768
3769
3770
3771
3772
3773
3774
3775
3776
3777
3778
3779
3780
3781
3782
3783
3784
3785
3786
3787
3788
3789
3790
3791
3792
3793
3794
3795
3796
3797
3798
3799
3800
3801
3802
3803
3804
3805
3806
3807
3808
3809
3810
3811
3812
3813
3814
3815
3816
3817
3818
3819
3820
3821
3822
3823
3824
3825
3826
3827
3828
3829
3830
3831
3832
3833
3834
3835
3836
3837
3838
3839
3840
3841
3842
3843
3844
3845
3846
3847
3848
3849
3850
3851
3852
3853
3854
3855
3856
3857
3858
3859
3860
3861
3862
3863
3864
3865
3866
3867
3868
3869
3870
3871
3872
3873
3874
3875
3876
3877
3878
3879
3880
3881
3882
3883
3884
3885
3886
3887
3888
3889
3890
3891
3892
3893
3894
3895
3896
3897
3898
3899
3900
3901
3902
3903
3904
3905
3906
3907
3908
3909
3910
3911
3912
3913
3914
3915
3916
3917
3918
3919
3920
3921
3922
3923
3924
3925
3926
3927
3928
3929
3930
3931
3932
3933
3934
3935
3936
3937
3938
3939
3940
3941
3942
3943
3944
3945
3946
3947
3948
3949
3950
3951
3952
3953
3954
3955
3956
3957
3958
3959
3960
3961
3962
3963
3964
3965
3966
3967
3968
3969
3970
3971
3972
3973
3974
3975
3976
3977
3978
3979
3980
3981
3982
3983
3984
3985
3986
3987
3988
3989
3990
3991
3992
3993
3994
3995
3996
3997
3998
3999
4000
4001
4002
4003
4004
4005
4006
4007
4008
4009
4010
4011
4012
4013
4014
4015
4016
4017
4018
4019
4020
4021
4022
4023
4024
4025
4026
4027
4028
4029
4030
4031
4032
4033
4034
4035
4036
4037
4038
4039
4040
4041
4042
4043
4044
4045
4046
4047
4048
4049
4050
4051
4052
4053
4054
4055
4056
4057
4058
4059
4060
4061
4062
4063
4064
4065
4066
4067
4068
4069
4070
4071
4072
4073
4074
4075
4076
4077
4078
4079
4080
4081
4082
4083
4084
4085
4086
4087
4088
4089
4090
4091
4092
4093
4094
4095
4096
4097
4098
4099
4100
4101
4102
4103
4104
4105
4106
4107
4108
4109
4110
4111
4112
4113
4114
4115
4116
4117
4118
4119
4120
4121
4122
4123
4124
4125
4126
4127
4128
4129
4130
4131
4132
4133
4134
4135
4136
4137
4138
4139
4140
4141
4142
4143
4144
4145
4146
4147
4148
4149
4150
4151
4152
4153
4154
4155
4156
4157
4158
4159
4160
4161
4162
4163
4164
4165
4166
4167
4168
4169
4170
4171
4172
4173
4174
4175
4176
4177
4178
4179
4180
4181
4182
4183
4184
4185
4186
4187
4188
4189
4190
4191
4192
4193
4194
4195
4196
4197
4198
4199
4200
4201
4202
4203
4204
4205
4206
4207
4208
4209
4210
4211
4212
4213
4214
4215
4216
4217
4218
4219
4220
4221
4222
4223
4224
4225
4226
4227
4228
4229
4230
4231
4232
4233
4234
4235
4236
4237
4238
4239
4240
4241
4242
4243
4244
4245
4246
4247
4248
4249
4250
4251
4252
4253
4254
4255
4256
4257
4258
4259
4260
4261
4262
4263
4264
4265
4266
4267
4268
4269
4270
4271
4272
4273
4274
4275
4276
4277
4278
4279
4280
4281
4282
4283
4284
4285
4286
4287
4288
4289
4290
4291
4292
4293
4294
4295
4296
4297
4298
4299
4300
4301
4302
4303
4304
4305
4306
4307
4308
4309
4310
4311
4312
4313
4314
4315
4316
4317
4318
4319
4320
4321
4322
4323
4324
4325
4326
4327
4328
4329
4330
4331
4332
4333
4334
4335
4336
4337
4338
4339
4340
4341
4342
4343
4344
4345
4346
4347
4348
4349
4350
4351
4352
4353
4354
4355
4356
4357
4358
4359
4360
4361
4362
4363
4364
4365
4366
4367
4368
4369
4370
4371
4372
4373
4374
4375
4376
4377
4378
4379
4380
4381
4382
4383
4384
4385
4386
4387
4388
4389
4390
4391
4392
4393
4394
4395
4396
4397
4398
4399
4400
4401
4402
4403
4404
4405
4406
4407
4408
4409
4410
4411
4412
4413
4414
4415
4416
4417
4418
4419
4420
4421
4422
4423
4424
4425
4426
4427
4428
4429
4430
4431
4432
4433
4434
4435
4436
4437
4438
4439
4440
4441
4442
4443
4444
4445
4446
4447
4448
4449
4450
4451
4452
4453
4454
4455
4456
4457
4458
4459
4460
4461
4462
4463
4464
4465
4466
4467
4468
4469
4470
4471
4472
4473
4474
4475
4476
4477
4478
4479
4480
4481
4482
4483
4484
4485
4486
4487
4488
4489
4490
4491
4492
4493
4494
4495
4496
4497
4498
4499
4500
4501
4502
4503
4504
4505
4506
4507
4508
4509
4510
4511
4512
4513
4514
4515
4516
4517
4518
4519
4520
4521
4522
4523
4524
4525
4526
4527
4528
4529
4530
4531
4532
4533
4534
4535
4536
4537
4538
4539
4540
4541
4542
4543
4544
4545
4546
4547
4548
4549
4550
4551
4552
4553
4554
4555
4556
4557
4558
4559
4560
4561
4562
4563
4564
4565
4566
4567
4568
4569
4570
4571
4572
4573
4574
4575
4576
4577
4578
4579
4580
4581
4582
4583
4584
4585
4586
4587
4588
4589
4590
4591
4592
4593
4594
4595
4596
4597
4598
4599
4600
4601
4602
4603
4604
4605
4606
4607
4608
4609
4610
4611
4612
4613
4614
4615
4616
4617
4618
4619
4620
4621
4622
4623
4624
4625
4626
4627
4628
4629
4630
4631
4632
4633
4634
4635
4636
4637
4638
4639
4640
4641
4642
4643
4644
4645
4646
4647
4648
4649
4650
4651
4652
4653
4654
4655
4656
4657
4658
4659
4660
4661
4662
4663
4664
4665
4666
4667
4668
4669
4670
4671
4672
4673
4674
4675
4676
4677
4678
4679
4680
4681
4682
4683
4684
4685
4686
4687
4688
4689
4690
4691
4692
4693
4694
4695
4696
4697
4698
4699
4700
4701
4702
4703
4704
4705
4706
4707
4708
4709
4710
4711
4712
4713
4714
4715
4716
4717
4718
4719
4720
4721
4722
4723
4724
4725
4726
4727
4728
4729
4730
4731
4732
4733
4734
4735
4736
4737
4738
4739
4740
4741
4742
4743
4744
4745
4746
4747
4748
4749
4750
4751
4752
4753
4754
4755
4756
4757
4758
4759
4760
4761
4762
4763
4764
4765
4766
4767
4768
4769
4770
4771
4772
4773
4774
4775
4776
4777
4778
4779
4780
4781
4782
4783
4784
4785
4786
4787
4788
4789
4790
4791
4792
4793
4794
4795
4796
4797
4798
4799
4800
4801
4802
4803
4804
4805
4806
4807
4808
4809
4810
4811
4812
4813
4814
4815
4816
4817
4818
4819
4820
4821
4822
4823
4824
4825
4826
4827
4828
4829
4830
4831
4832
4833
4834
4835
4836
4837
4838
4839
4840
4841
4842
4843
4844
4845
4846
4847
4848
4849
4850
4851
4852
4853
4854
4855
4856
4857
4858
4859
4860
4861
4862
4863
4864
4865
4866
4867
4868
4869
4870
4871
4872
4873
4874
4875
4876
4877
4878
4879
4880
4881
4882
4883
4884
4885
4886
4887
4888
4889
4890
4891
4892
4893
4894
4895
4896
4897
4898
4899
4900
4901
4902
4903
4904
4905
4906
4907
4908
4909
4910
4911
4912
4913
4914
4915
4916
4917
4918
4919
4920
4921
4922
4923
4924
4925
4926
4927
4928
4929
4930
4931
4932
4933
4934
4935
4936
4937
4938
4939
4940
4941
4942
4943
4944
4945
4946
4947
4948
4949
4950
4951
4952
4953
4954
4955
4956
4957
4958
4959
4960
4961
4962
4963
4964
4965
4966
4967
4968
4969
4970
4971
4972
4973
4974
4975
4976
4977
4978
4979
4980
4981
4982
4983
4984
4985
4986
4987
4988
4989
4990
4991
4992
4993
4994
4995
4996
4997
4998
4999
5000
5001
5002
5003
5004
5005
5006
5007
5008
5009
5010
5011
5012
5013
5014
5015
5016
5017
5018
5019
5020
5021
5022
5023
5024
5025
5026
5027
5028
5029
5030
5031
5032
5033
5034
5035
5036
5037
5038
5039
5040
5041
5042
5043
5044
5045
5046
5047
5048
5049
5050
5051
5052
5053
5054
5055
5056
5057
5058
5059
5060
5061
5062
5063
5064
5065
5066
5067
5068
5069
5070
5071
5072
5073
5074
5075
5076
5077
5078
5079
5080
5081
5082
5083
5084
5085
5086
5087
5088
5089
5090
5091
5092
5093
5094
5095
5096
5097
5098
5099
5100
5101
5102
5103
5104
5105
5106
5107
5108
5109
5110
5111
5112
5113
5114
5115
5116
5117
5118
5119
5120
5121
5122
5123
5124
5125
5126
5127
5128
5129
5130
5131
5132
5133
5134
5135
5136
5137
5138
5139
5140
5141
5142
5143
5144
5145
5146
5147
5148
5149
5150
5151
5152
5153
5154
5155
5156
5157
5158
5159
5160
5161
5162
5163
5164
5165
5166
5167
5168
5169
5170
5171
5172
5173
5174
5175
5176
5177
5178
5179
5180
5181
5182
5183
5184
5185
5186
5187
5188
5189
5190
5191
5192
5193
5194
5195
5196
5197
5198
5199
5200
5201
5202
5203
5204
5205
5206
5207
5208
5209
5210
5211
5212
5213
5214
5215
5216
5217
5218
5219
5220
5221
5222
5223
5224
5225
5226
5227
5228
5229
5230
5231
5232
5233
5234
5235
5236
5237
5238
5239
5240
5241
5242
5243
5244
5245
5246
5247
5248
5249
5250
5251
5252
5253
5254
5255
5256
5257
5258
5259
5260
5261
5262
5263
5264
5265
5266
5267
5268
5269
5270
5271
5272
5273
5274
5275
5276
5277
5278
5279
5280
5281
5282
5283
5284
5285
5286
5287
5288
5289
5290
5291
5292
5293
5294
5295
5296
5297
5298
5299
5300
5301
5302
5303
5304
5305
5306
5307
5308
5309
5310
5311
5312
5313
5314
5315
5316
5317
5318
5319
5320
5321
5322
5323
5324
5325
5326
5327
5328
5329
5330
5331
5332
5333
5334
5335
5336
5337
5338
5339
5340
5341
5342
5343
5344
5345
5346
5347
5348
5349
5350
5351
5352
5353
5354
5355
5356
5357
5358
5359
5360
5361
5362
5363
5364
5365
5366
5367
5368
5369
5370
5371
5372
5373
5374
5375
5376
5377
5378
5379
5380
5381
5382
5383
5384
5385
5386
5387
5388
5389
5390
5391
5392
5393
5394
5395
5396
5397
5398
5399
5400
5401
5402
5403
5404
5405
5406
5407
5408
5409
5410
5411
5412
5413
5414
5415
5416
5417
5418
5419
5420
5421
5422
5423
5424
5425
5426
5427
5428
5429
5430
5431
5432
5433
5434
5435
5436
5437
5438
5439
5440
5441
5442
5443
5444
5445
5446
5447
5448
5449
5450
5451
5452
5453
5454
5455
5456
5457
5458
5459
5460
5461
5462
5463
5464
5465
5466
5467
5468
5469
5470
5471
5472
5473
5474
5475
5476
5477
5478
5479
5480
5481
5482
5483
5484
5485
5486
5487
5488
5489
5490
5491
5492
5493
5494
5495
5496
5497
5498
5499
5500
5501
5502
5503
5504
5505
5506
5507
5508
5509
5510
5511
5512
5513
5514
5515
5516
5517
5518
5519
5520
5521
5522
5523
5524
5525
5526
5527
5528
5529
5530
5531
5532
5533
5534
5535
5536
5537
5538
5539
5540
5541
5542
5543
5544
5545
5546
5547
5548
5549
5550
5551
5552
5553
5554
5555
5556
5557
5558
5559
5560
5561
5562
5563
5564
5565
5566
5567
5568
5569
5570
5571
5572
5573
5574
5575
5576
5577
5578
5579
5580
5581
5582
5583
5584
5585
5586
5587
5588
5589
5590
5591
5592
5593
5594
5595
5596
5597
5598
5599
5600
5601
5602
5603
5604
5605
5606
5607
5608
5609
5610
5611
5612
5613
5614
5615
5616
5617
5618
5619
5620
5621
5622
5623
5624
5625
5626
5627
5628
5629
5630
5631
5632
5633
5634
5635
5636
5637
5638
5639
5640
5641
5642
5643
5644
5645
5646
5647
5648
5649
5650
5651
5652
5653
5654
5655
5656
5657
5658
5659
5660
5661
5662
5663
5664
5665
5666
5667
5668
5669
5670
5671
5672
5673
5674
5675
5676
5677
5678
5679
5680
5681
5682
5683
5684
5685
5686
5687
5688
5689
5690
5691
5692
5693
5694
5695
5696
5697
5698
5699
5700
5701
5702
5703
5704
5705
5706
5707
5708
5709
5710
5711
5712
5713
5714
5715
5716
5717
5718
5719
5720
5721
5722
5723
5724
5725
5726
5727
5728
5729
5730
```

```

3617     clearUnusedBits();
3618 }
3619
3620 // Get the value of ith bit
3621 INLINE bool get(int i) const {
3622     assert(i >= 0 && "Attempting to read bit with negative
index");
3623     assert(i < _AP_W && "Attempting to read bit beyond MSB");
3624     return ((maskBit(i) & (pVal[whichWord(i)])) != 0);
3625 }
3626
3627 // Get the value of ith bit
3628 INLINE bool get_bit(int i) const {
3629     assert(i >= 0 && "Attempting to read bit with negative
index");
3630     assert(i < _AP_W && "Attempting to read bit beyond MSB");
3631     return ((maskBit(i) & (pVal[whichWord(i)])) != 0);
3632 }
3633
3634 // This is used for sc_lv and sc_bv, which is implemented
by sc_uint
3635 // Rotate an ap_private object n places to the left
3636 INLINE void lrotate(int n) {
3637     assert(n >= 0 && "Attempting to shift negative index");
3638     assert(n < _AP_W && "Shift value larger than bit width");
3639     operator=(shl(n) | lshr(_AP_W - n));
3640 }
3641
3642 // This is used for sc_lv and sc_bv, which is implemented
by sc_uint
3643 // Rotate an ap_private object n places to the right
3644 INLINE void rrotate(int n) {
3645     assert(n >= 0 && "Attempting to shift negative index");
3646     assert(n < _AP_W && "Shift value larger than bit width");
3647     operator=(lshr(n) | shl(_AP_W - n));
3648 }
3649
3650 // Set the given bit to 0 whose position is given as "bit-
Position".
3651 // @brief Set a given bit to 0.
3652 INLINE ap_private& clear(uint32_t bitPosition) {
3653     pVal[whichWord(bitPosition)] &= ~maskBit(bitPosition);
3654     clearUnusedBits();
3655     return *this;
3656 }
3657
3658 // @brief Set every bit to 0.
3659 INLINE void clear() { memset(pVal, 0, _AP_N *
APINT_WORD_SIZE); }
3660
3661 // @brief Toggle every bit to its opposite value.
3662 ap_private& flip() {
3663     for (int i = 0; i < _AP_N; ++i) pVal[i] ^= ~0ULL;
3664     clearUnusedBits();
3665     return *this;
3666 }
3667
3668 // @brief Toggles a given bit to its opposite value.
3669 INLINE ap_private& flip(uint32_t bitPosition) {
3670     assert(bitPosition < Bitwidth && "Out of the bit-width
range!");
3671     set_bit(bitPosition, !get_bit(bitPosition));
3672     return *this;
3673 }
3674
3675 // complements every bit
3676 INLINE void b_not() { flip(); }
3677
3678 INLINE ap_private getLoBits(uint32_t numBits) const {
3679     return ap_private_ops::lshr(ap_private_ops::shl(*this,
_AP_W - numBits),
3680     _AP_W - numBits);
3681 }
3682
3683 INLINE ap_private getHiBits(uint32_t numBits) const {
3684     return ap_private_ops::lshr(*this, _AP_W - numBits);
3685 }
3686
3687 // Binary Arithmetic
3688 //-----
3689
3690 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
3691 // INLINE ap_private<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator^(
3692 //     const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
3693 //     return *this ^ a2.get();
3694 // }
3695 //
3696 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
3697 // INLINE ap_private<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator|(
3698 //     const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
3699 //     return *this | a2.get();
3700 // }
3701 //

```

```

3702 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
3703 // INLINE ap_private<AP_MAX(_AP_W2 + _AP_W3, _AP_W), _AP_S>
operator^(
3704 //     const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>&
a2) {
3705 //     return *this ^ a2.get();
3706 // }
3707
3708 /// Arithmetic assign
3709 //-----
3710
3711 #define OP_BIN_LO-
GIC_ASSIGN_AP(Sym)
3712     template <int _AP_W1, bool
3713     _AP_S1>
3714     INLINE ap_private& operator Sym(const ap_private<_AP_W1,
3715     _AP_S1>& RHS) {
3716         const int _AP_N1 = ap_private<_AP_W1,
3717     _AP_S1>::_AP_N;
3718         uint32_t numWords = AESL_std::min((int)_AP_N,
3719     _AP_N1);
3720         uint32_t
3721         i;
3722         if (_AP_W !=
3723     _AP_W1)
3724             fprintf(stderr,
3725     "\nWarning! Bitsize mismatch for ap_[u]int " #Sym
3726     " ap_[u]int.\n");
3727         for (i = 0; i < numWords; ++i) pVal[i] Sym RHS.get_pV-
al(i);
3728         if (_AP_N1 < _AP_N)
3729             uint64_t ext = RHS.isNegative() ? ~0ULL :
3730             0;
3731         for (; i < _AP_N; ++i) pVal[i] Sym
3732         ext;
3733     }
3734
3735     clearUnused-
3736     Bits();
3737     return
3738     *this;
3739 }
3740
3741 OP_BIN_LOGIC_ASSIGN_AP(&=);
3742 OP_BIN_LOGIC_ASSIGN_AP(|=);
3743 OP_BIN_LOGIC_ASSIGN_AP(^=);
3744 #undef OP_BIN_LOGIC_ASSIGN_AP
3745
3746 /// Adds the RHS APint to this ap_private.
3747 /// @returns this, after addition of RHS.
3748 /// @brief Addition assignment operator.
3749 template <int _AP_W1, bool _AP_S1>
3750 INLINE ap_private& operator+=(const ap_private<_AP_W1,
3751     _AP_S1>& RHS) {
3752     const int _AP_N1 = ap_private<_AP_W1, _AP_S1>::_AP_N;
3753     uint64_t RHSpVal[_AP_N1];
3754     for (int i = 0; i < _AP_N1; ++i) RHSpVal[i] = RHS.get_pV-
al(i);
3755     ap_private_ops::add(pVal, pVal, RHSpVal, _AP_N, _AP_N,
3756     _AP_N1, _AP_S,
3757     _AP_S1);
3758     clearUnusedBits();
3759     return *this;
3760 }
3761
3762 template <int _AP_W1, bool _AP_S1>
3763 INLINE ap_private& operator-=(const ap_private<_AP_W1,
3764     _AP_S1>& RHS) {
3765     const int _AP_N1 = ap_private<_AP_W1, _AP_S1>::_AP_N;
3766     uint64_t RHSpVal[_AP_N1];
3767     for (int i = 0; i < _AP_N1; ++i) RHSpVal[i] = RHS.get_pV-
al(i);
3768     ap_private_ops::sub(pVal, pVal, RHSpVal, _AP_N, _AP_N,
3769     _AP_N1, _AP_S,
3770     _AP_S1);
3771     clearUnusedBits();
3772     return *this;
3773 }
3774
3775 template <int _AP_W1, bool _AP_S1>
3776 INLINE ap_private& operator*=(const ap_private<_AP_W1,
3777     _AP_S1>& RHS) {
3778     // Get some bit facts about LHS and check for zero
3779     uint32_t lhsBits = getActiveBits();
3780
3781     uint32_t lhsWords = !lhsBits ? 0 : whichWord(lhsBits - 1)
3782     + 1;
3783     if (!lhsWords) {
3784         // 0 * X ==> 0
3785         return *this;
3786     }
3787
3788     ap_private dupRHS = RHS;

```

```

3770 // Get some bit facts about RHS and check for zero
3771 uint32_t rhsBits = dupRHS.getActiveBits();

3772 uint32_t rhsWords = !rhsBits ? 0 : whichWord(rhsBits - 1)
+ 1;
3773 if (!rhsWords) {
3774 // X * 0 ==> 0
3775 clear();
3776 return *this;
3777 }
3778 // Allocate space for the result
3779 uint32_t destWords = rhsWords + lhsWords;
3780 uint64_t* dest = (uint64_t*)malloc(destWords *
sizeof(uint64_t));
3782 // Perform the long multiply
3783 ap_private_ops::mul(dest, pVal, lhsWords,
dupRHS.getPVal(), rhsWords,
3785 destWords);
3786 // Copy result back into *this
3787 clear();
3788 uint32_t wordsToCopy = destWords >= _AP_N ? _AP_N : dest-
Words;
3790 memcpy(pVal, dest, wordsToCopy * APINT_WORD_SIZE);
3791
3792 uint64_t ext = (isNegative() ^ RHS.isNegative()) ?
~0ULL : 0ULL;
3794 for (int i = wordsToCopy; i < _AP_N; i++) pVal[i] = ext;
3795 clearUnusedBits();
3796 // delete dest array and return
3797 free(dest);
3798 return *this;
3799 }
3800
3801 #define
OP_ASSIGN_AP(Sym)
\
3802 template <int _AP_W2, bool
_AP_S2>
\
3803 inline ap_private& operator Sym##=(const ap_private<_AP_W2,
_AP_S2>& op) { \
3804 *this = operator
Sym(op);
3805 return
\
3806 }
3807
3808 OP_ASSIGN_AP(/)
3809 OP_ASSIGN_AP(%)
3810 #undef OP_ASSIGN_AP
3811
3812 #define OP_BIN_LO-
GIC_AP(Sym)
\
3813 template <int _AP_W1, bool
_AP_S1>
\
3814 inline typename RType<_AP_W1, _AP_S1>::logic operator
Sym(
3815 const ap_private<_AP_W1, _AP_S1>& RHS) const
{
3816 enum
{
\
3817 numWords = (RType<_AP_W1, _AP_S1>::logic_w +
APINT_BITS_PER_WORD - 1) / \
3818 APINT_BITS_PER_WORD
\
3819 };
\
3820 typename RType<_AP_W1, _AP_S1>::logic Res-
ult;
3821 uint32_t
i;
3822 const int _AP_N1 = ap_private<_AP_W1,
_AP_S1>::_AP_N;
3823 uint32_t min_N = std::min((int)_AP_N,
_AP_N1);
3824 uint32_t max_N = std::max((int)_AP_N,
_AP_N1);
3825 for (i = 0; i < min_N; +
i)
3826 Result.set_pVal(i, pVal[i] Sym RHS.getPV-
al(i));
3827 if (numWords > i)
{
3828 uint64_t ext = ((_AP_N < _AP_N1 && isNegative())
||
3829 (_AP_N1 < _AP_N && RHS.isNegat-
ive()))
3830 ?
\
3831 :
\
0;
3832 if (_AP_N >
_AP_N1)
\

```

```

3833 for (; i < max_N; i++) Result.set_pVal(i, pVal[i] Sym
ext);
3834 \
else
\
3835 for (; i < max_N; i++) Result.set_pVal(i, RHS.getPV-
al(i) Sym ext);
3836 if (numWords > i)
{
3837 uint64_t ext2 = ((_AP_N > _AP_N1 && isNegative())
||
3838 (_AP_N1 > _AP_N && RHS.isNegat-
ive()))
3839 ?
\
3840 :
\
0;
3841 Result.set_pVal(i, ext Sym
ext2);
3842 }
\
3843 }
\
3844 Res-
ult.clearUnused-
Bits();
3845 return Res-
ult;
\
3846 }
3847
3848 OP_BIN_LOGIC_AP(|);
3849 OP_BIN_LOGIC_AP(&);
3850 OP_BIN_LOGIC_AP(^);
3851
3852 #undef OP_BIN_LOGIC_AP
3853
3854 template <int _AP_W1, bool _AP_S1>
3855 inline typename RType<_AP_W1, _AP_S1>::plus operator+(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
3856 typename RType<_AP_W1, _AP_S1>::plus Result, lhs(*this),
rhs(RHS);
3858 const int Result_AP_N = (RType<_AP_W1, _AP_S1>::plus_w +
63) / 64;
3859 ap_private_ops::add(Result.getPVal(), lhs.getPVal(),
rhs.getPVal(),
3860 Result_AP_N, Result_AP_N, Res-
ult_AP_N, _AP_S, _AP_S1);
3861 Result.clearUnusedBits();
3862 return Result;
3863 }
3864
3865 template <int _AP_W1, bool _AP_S1>
3866 inline typename RType<_AP_W1, _AP_S1>::minus operator-(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
3867
3868 typename RType<_AP_W1, _AP_S1>::minus Result, lhs(*this),
rhs(RHS);
3869 const int Result_AP_N = (RType<_AP_W1, _AP_S1>::minus_w +
63) / 64;
3870 ap_private_ops::sub(Result.getPVal(), lhs.getPVal(),
rhs.getPVal(),
3871 Result_AP_N, Result_AP_N, Res-
ult_AP_N, _AP_S, _AP_S1);
3872 Result.clearUnusedBits();
3873 return Result;
3874 }
3875
3876 template <int _AP_W1, bool _AP_S1>
3877 inline typename RType<_AP_W1, _AP_S1>::mult operator*(
const ap_private<_AP_W1, _AP_S1>& RHS) const {
3878 typename RType<_AP_W1, _AP_S1>::mult temp = *this;
3879 temp *= RHS;
3880 return temp;
3881 }
3882
3883 template <int _AP_W2, bool _AP_S2>
3884 inline typename RType<_AP_W2, _AP_S2>::div operator/(
const ap_private<_AP_W2, _AP_S2>& op) const {
3885 ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
3886 (_AP_W > _AP_W2 ? _AP_S
: (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
3887 lhs = *this;
3888 ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
3889 (_AP_W > _AP_W2 ? _AP_S
: (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
3890 rhs = op;
3891 return typename RType<_AP_W2, _AP_S2>::div(
(_AP_S || _AP_S2) ? lhs.sdiv(rhs) : lhs.udiv(rhs));
3892 }
3893
3894 template <int _AP_W2, bool _AP_S2>
3895 inline typename RType<_AP_W2, _AP_S2>::mod operator%(
const ap_private<_AP_W2, _AP_S2>& op) const {

```

```

3902     ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
3903         (_AP_W > _AP_W2 ? _AP_S
3904             : (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
3905         lhs = *this;
3906     ap_private<AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S || _AP_S2)),
3907         (_AP_W > _AP_W2 ? _AP_S
3908             : (_AP_W2 > _AP_W ? _AP_S2 :
_AP_S || _AP_S2))>
3909         rhs = op;
3910     typename RType<_AP_W2, _AP_S2>::mod res =
3911         typename RType<_AP_W2, _AP_S2>::mod(_AP_S ?
lhs.srem(rhs)
3912             :
lhs.urem(rhs));
3913     return res;
3914 }
3915
3916 #define OP_LEFT_SHIFT_CTYPE(TYPE, SIGNED) \
3917     INLINE ap_private operator<<(const TYPE op) const { \
3918         if (op >= _AP_W) return ap_private(0); \
3919         if (SIGNED && op < 0) return *this >> (0 - op); \
3920         return shl(op); \
3921     }
3922
3923 OP_LEFT_SHIFT_CTYPE(int, true)
3924 // OP_LEFT_SHIFT_CTYPE(bool, false)
3925 OP_LEFT_SHIFT_CTYPE(signed char, true)
3926 OP_LEFT_SHIFT_CTYPE(unsigned char, false)
3927 OP_LEFT_SHIFT_CTYPE(short, true)
3928 OP_LEFT_SHIFT_CTYPE(unsigned short, false)
3929 OP_LEFT_SHIFT_CTYPE(unsigned int, false)
3930 OP_LEFT_SHIFT_CTYPE(long, true)
3931 OP_LEFT_SHIFT_CTYPE(unsigned long, false)
3932 OP_LEFT_SHIFT_CTYPE(unsigned long long, false)
3933 OP_LEFT_SHIFT_CTYPE(long long, true)
3934 #if 0
3935 OP_LEFT_SHIFT_CTYPE(half, false)
3936 OP_LEFT_SHIFT_CTYPE(float, false)
3937 OP_LEFT_SHIFT_CTYPE(double, false)
3938 #endif
3939 #undef OP_LEFT_SHIFT_CTYPE
3940
3941 template <int _AP_W2, bool _AP_S2>
3942     INLINE ap_private operator<<(const ap_private<_AP_W2,
_AP_S2>& op2) const {
3943     if (_AP_S2 == false) {
3944         uint32_t sh = op2.to_uint();
3945         return *this << sh;
3946     } else {
3947         int sh = op2.to_int();
3948         return *this << sh;
3949     }
3950 }
3951
3952 #define OP_RIGHT_SHIFT_CTYPE(TYPE, SIGNED) \
3953     INLINE ap_private operator>>(const TYPE op) const { \
3954         if (op >= _AP_W) { \
3955             if (isNegative()) \
3956                 return ap_private(-1); \
3957             else \
3958                 return ap_private(0); \
3959         } \
3960         if ((SIGNED) && op < 0) return *this << (0 - op); \
3961         if (_AP_S) \
3962             return ashr(op); \
3963         else \
3964             return lshr(op); \
3965     }
3966
3967 // OP_RIGHT_SHIFT_CTYPE(bool, false)
3968 OP_RIGHT_SHIFT_CTYPE(char, CHAR_IS_SIGNED)
3969 OP_RIGHT_SHIFT_CTYPE(signed char, true)
3970 OP_RIGHT_SHIFT_CTYPE(unsigned char, false)
3971 OP_RIGHT_SHIFT_CTYPE(short, true)
3972 OP_RIGHT_SHIFT_CTYPE(unsigned short, false)
3973 OP_RIGHT_SHIFT_CTYPE(int, true)
3974 OP_RIGHT_SHIFT_CTYPE(unsigned int, false)
3975 OP_RIGHT_SHIFT_CTYPE(long, true)
3976 OP_RIGHT_SHIFT_CTYPE(unsigned long, false)
3977 OP_RIGHT_SHIFT_CTYPE(unsigned long long, false)
3978 OP_RIGHT_SHIFT_CTYPE(long long, true)
3979 #if 0
3980 OP_RIGHT_SHIFT_CTYPE(half, false)
3981 OP_RIGHT_SHIFT_CTYPE(float, false)
3982 OP_RIGHT_SHIFT_CTYPE(double, false)
3983 #endif
3984 #undef OP_RIGHT_SHIFT_CTYPE
3985
3986 template <int _AP_W2, bool _AP_S2>
3987     INLINE ap_private operator>>(const ap_private<_AP_W2,
_AP_S2>& op2) const {
3988     if (_AP_S2 == false) {
3989         uint32_t sh = op2.to_uint();
3990         return *this >> sh;
3991     } else {
3992         int sh = op2.to_int();
3993         return *this >> sh;

```

```

3994     }
3995 }
3996
3997 /// Shift assign
3998 //-----
3999 // TODO call clearUnusedBits ?
4000 #define
OP_ASSIGN_AP(Sym)
4001     template <int _AP_W2, bool
_AP_S2>
4002     INLINE ap_private& operator Sym##=(int op)
4003     {
4004         *this = operator
Sym(op);
4005         return
*this;
4006     }
4007
4008     template <int _AP_W2, bool
_AP_S2>
4009     INLINE ap_private& operator Sym##=(unsigned int op)
4010     {
4011         *this = operator
Sym(op);
4012         return
*this;
4013     }
4014
4015     template <int _AP_W2, bool
_AP_S2>
4016     INLINE ap_private& operator Sym##=(const ap_private<_AP_W2,
_AP_S2>& op) { \
4017         *this = operator
Sym(op);
4018         return
*this;
4019     }
4020 //-----
4021     INLINE bool operator==(const ap_private& RHS) const {
4022         // Get some facts about the number of bits used in the
two operands.
4023         uint32_t n1 = getActiveBits();
4024         uint32_t n2 = RHS.getActiveBits();
4025
4026         // If the number of bits isn't the same, they aren't
equal
4027         if (n1 != n2) return false;
4028
4029         // If the number of bits fits in a word, we only need to
compare the low
4030         // word.
4031         if (n1 <= APINT_BITS_PER_WORD) return pVal[0] ==
RHS.get_pVal(0);
4032
4033         // Otherwise, compare everything
4034         for (int i = whichWord(n1 - 1); i >= 0; --i)
4035             if (pVal[i] != RHS.get_pVal(i)) return false;
4036         return true;
4037     }
4038
4039     template <int _AP_W2, bool _AP_S2>
4040     INLINE bool operator==(const ap_private<_AP_W2, _AP_S2>&
op) const {
4041         enum {
4042             _AP_MAX_W = AP_MAX(_AP_W, _AP_W2),
4043         };
4044         ap_private<_AP_MAX_W, false> lhs(*this);
4045         ap_private<_AP_MAX_W, false> rhs(op);
4046         return lhs == rhs;
4047     }
4048
4049     INLINE bool operator==(uint64_t Val) const {
4050         uint32_t n = getActiveBits();
4051         if (n <= APINT_BITS_PER_WORD)
4052             return pVal[0] == Val;
4053         else
4054             return false;
4055     }
4056
4057     template <int _AP_W2, bool _AP_S2>
4058     INLINE bool operator!=(const ap_private<_AP_W2, _AP_S2>&
op) const {
4059         return !(*this == op);
4060     }
4061
4062     template <bool _AP_S1>
4063     INLINE bool operator!=(const ap_private<_AP_W, _AP_S1>&
RHS) const {
4064         return !(*this == RHS);
4065     }
4066

```



```

4067  inline bool operator!=(uint64_t Val) const { return !
4068  ((*this) == Val); }
4069  template <int _AP_W2, bool _AP_S2>
4070  inline bool operator<=(const ap_private<_AP_W2, _AP_S2>&
op) const {
4071      return !(*this > op);
4072  }
4073
4074  inline bool operator<(const ap_private& op) const {
4075      return _AP_S ? slt(op) : ult(op);
4076  }
4077
4078  template <int _AP_W2, bool _AP_S2>
4079  inline bool operator<(const ap_private<_AP_W2, _AP_S2>& op)
const {
4080      enum {
4081          _AP_MAX_W = AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S ||
4082          _AP_S2));
4083      };
4084      ap_private<_AP_MAX_W, _AP_S> lhs(*this);
4085      ap_private<_AP_MAX_W, _AP_S2> rhs(op);
4086      if (_AP_S == _AP_S2)
4087          return _AP_S ? lhs.slt(rhs) : lhs.ult(rhs);
4088      else if (_AP_S)
4089          if (_AP_W2 >= _AP_W)
4090              return lhs.ult(rhs);
4091          else
4092              return lhs.slt(rhs);
4093      else if (_AP_W >= _AP_W2)
4094          return lhs.ult(rhs);
4095      else
4096          return lhs.slt(rhs);
4097  }
4098  template <int _AP_W2, bool _AP_S2>
4099  inline bool operator>=(const ap_private<_AP_W2, _AP_S2>&
op) const {
4100      return !(*this < op);
4101  }
4102
4103  inline bool operator>(const ap_private& op) const {
4104      return _AP_S ? sgt(op) : ugt(op);
4105  }
4106
4107  template <int _AP_W2, bool _AP_S2>
4108  inline bool operator>(const ap_private<_AP_W2, _AP_S2>& op)
const {
4109      enum {
4110          _AP_MAX_W = AP_MAX(_AP_W + (_AP_S || _AP_S2), _AP_W2 +
(_AP_S ||
4111          _AP_S2));
4112      };
4113      ap_private<_AP_MAX_W, _AP_S> lhs(*this);
4114      ap_private<_AP_MAX_W, _AP_S2> rhs(op);
4115      if (_AP_S == _AP_S2)
4116          return _AP_S ? lhs.sgt(rhs) : lhs.ugt(rhs);
4117      else if (_AP_S)
4118          if (_AP_W2 >= _AP_W)
4119              return lhs.ugt(rhs);
4120          else
4121              return lhs.sgt(rhs);
4122      else if (_AP_W >= _AP_W2)
4123          return lhs.ugt(rhs);
4124      else
4125          return lhs.sgt(rhs);
4126  }
4127  /// Bit and Part Select
4128  //-----
4129  inline _private_range_ref<_AP_W, _AP_S> operator()(int Hi,
int Lo) {
4130      return _private_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
4131  }
4132
4133  inline _private_range_ref<_AP_W, _AP_S> operator()(int Hi,
int Lo) const {
4134      return _private_range_ref<_AP_W, _AP_S>(
const_cast<ap_private<_AP_W, _AP_S>*>(this), Hi, Lo);
4135  }
4136
4137  inline _private_range_ref<_AP_W, _AP_S> range(int Hi, int
Lo) const {
4138      return _private_range_ref<_AP_W, _AP_S>(
const_cast<ap_private<_AP_W, _AP_S>*>(this)), Hi,
Lo);
4139  }
4140
4141  inline _private_range_ref<_AP_W, _AP_S> range(int Hi, int
Lo) {
4142      return _private_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
4143  }
4144
4145  template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
4146  inline _private_range_ref<_AP_W, _AP_S> range(
const ap_private<_AP_W2, _AP_S2>& HiIdx,
4147  const ap_private<_AP_W3, _AP_S3>& LoIdx) {
4148      int Hi = HiIdx.to_int();
4149  }

```

```

4150      int Lo = LoIdx.to_int();
4151      return _private_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
4152  }
4153
4154  template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
4155  inline _private_range_ref<_AP_W, _AP_S> operator()(
const ap_private<_AP_W2, _AP_S2>& HiIdx,
4156  const ap_private<_AP_W3, _AP_S3>& LoIdx) {
4157      int Hi = HiIdx.to_int();
4158      int Lo = LoIdx.to_int();
4159      return _private_range_ref<_AP_W, _AP_S>(this, Hi, Lo);
4160  }
4161
4162  template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
4163  inline _private_range_ref<_AP_W, _AP_S> range(
const ap_private<_AP_W2, _AP_S2>& HiIdx,
4164  const ap_private<_AP_W3, _AP_S3>& LoIdx) const {
4165      int Hi = HiIdx.to_int();
4166      int Lo = LoIdx.to_int();
4167      return _private_range_ref<_AP_W, _AP_S>(const_cast<ap_private*>(this), Hi, Lo);
4168  }
4169
4170  template <int _AP_W2, bool _AP_S2, int _AP_W3, bool _AP_S3>
4171  inline _private_range_ref<_AP_W, _AP_S> operator()(
const ap_private<_AP_W2, _AP_S2>& HiIdx,
4172  const ap_private<_AP_W3, _AP_S3>& LoIdx) const {
4173      int Hi = HiIdx.to_int();
4174      int Lo = LoIdx.to_int();
4175      return this->range(Hi, Lo);
4176  }
4177
4178  inline _private_bit_ref<_AP_W, _AP_S> operator[](int index)
{
4179      return _private_bit_ref<_AP_W, _AP_S>(*this, index);
4180  }
4181
4182  template <int _AP_W2, bool _AP_S2>
4183  inline _private_bit_ref<_AP_W, _AP_S> operator[](
const ap_private<_AP_W2, _AP_S2>& index) {
4184      return _private_bit_ref<_AP_W, _AP_S>(*this, in-
dex.to_int());
4185  }
4186
4187  template <int _AP_W2, bool _AP_S2>
4188  inline _private_bit_ref<_AP_W, _AP_S> operator[](
const ap_private<_AP_W2, _AP_S2>& index) const {
4189      return _private_bit_ref<_AP_W, _AP_S>(*this, in-
dex.to_int());
4190  }
4191
4192  template <int _AP_W2, bool _AP_S2>
4193  inline const _private_bit_ref<_AP_W, _AP_S> operator[](
const ap_private<_AP_W2, _AP_S2>& index) const {
4194      return _private_bit_ref<_AP_W, _AP_S>(
const_cast<ap_private<_AP_W, _AP_S>&>(*this), in-
dex.to_int());
4195  }
4196
4197  inline const _private_bit_ref<_AP_W, _AP_S> operator[](int
index) const {
4198      return _private_bit_ref<_AP_W, _AP_S>(
const_cast<ap_private<_AP_W, _AP_S>&>(*this), index);
4199  }
4200
4201  inline _private_bit_ref<_AP_W, _AP_S> bit(int index) {
4202      return _private_bit_ref<_AP_W, _AP_S>(*this, index);
4203  }
4204
4205  template <int _AP_W2, bool _AP_S2>
4206  inline _private_bit_ref<_AP_W, _AP_S> bit(const
ap_private<_AP_W2, _AP_S2>& index) {
4207      return _private_bit_ref<_AP_W, _AP_S>(*this, in-
dex.to_int());
4208  }
4209
4210  inline const _private_bit_ref<_AP_W, _AP_S> bit(int index)
const {
4211      return _private_bit_ref<_AP_W, _AP_S>(
const_cast<ap_private<_AP_W, _AP_S>&>(*this), index);
4212  }
4213
4214  template <int _AP_W2, bool _AP_S2>
4215  inline const _private_bit_ref<_AP_W, _AP_S> bit(
const ap_private<_AP_W2, _AP_S2>& index) const {
4216      return _private_bit_ref<_AP_W, _AP_S>(
const_cast<ap_private<_AP_W, _AP_S>&>(*this), in-
dex.to_int());
4217  }
4218
4219  // template <int _AP_W2, bool _AP_S2>
4220  // inline ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4221  // ap_private<_AP_W2, _AP_S2> >
4222  // concat(ap_private<_AP_W2, _AP_S2>& a2) {
4223  //     return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4224  // ap_private<_AP_W2, _AP_S2> >
4225  // >(*this, a2);
4226  // }
4227  //
4228  // template <int _AP_W2, bool _AP_S2>
4229  // inline ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4230  // ap_private<_AP_W2, _AP_S2> >
4231  // concat(const ap_private<_AP_W2, _AP_S2>& a2) const {

```

```

4238 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4239 //          ap_private<_AP_W2, _AP_S2> >{
4240 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4241 //          const_cast<ap_private<_AP_W2, _AP_S2>&*>(&a2));
4242 //      }
4243 //
4244 //      template <int _AP_W2, bool _AP_S2>
4245 //      INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
4246 //      operator,(ap_private<_AP_W2, _AP_S2> &a2) {
4247 //      return ap_concat_ref<_AP_W, ap_private, _AP_W2,
4248 //          ap_private<_AP_W2, _AP_S2>
>(&this, a2);
4249 //      }
4250 //
4251 //      template <int _AP_W2, bool _AP_S2>
4252 //      INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
4253 //      operator,(ap_private<_AP_W2, _AP_S2> &a2) const {
4254 //      return ap_concat_ref<_AP_W, ap_private, _AP_W2,
4255 //          ap_private<_AP_W2, _AP_S2> >{
4256 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this), a2);
4257 //      }
4258 //
4259 //      template <int _AP_W2, bool _AP_S2>
4260 //      INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
4261 //      operator,(const ap_private<_AP_W2, _AP_S2> &a2) {
4262 //      return ap_concat_ref<_AP_W, ap_private, _AP_W2,
4263 //          ap_private<_AP_W2, _AP_S2> >{
4264 //          *this, const_cast<ap_private<_AP_W2,
_AP_S2>&*>(&a2));
4265 //      }
4266 //
4267 //      template <int _AP_W2, bool _AP_S2>
4268 //      INLINE ap_concat_ref<_AP_W, ap_private, _AP_W2,
ap_private<_AP_W2, _AP_S2> >
4269 //      operator,(const ap_private<_AP_W2, _AP_S2> &a2) const {
4270 //      return ap_concat_ref<_AP_W, ap_private, _AP_W2,
4271 //          ap_private<_AP_W2, _AP_S2> >{
4272 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4273 //          const_cast<ap_private<_AP_W2, _AP_S2>&*>(&a2));
4274 //      }
4275 //
4276 //      template <int _AP_W2, bool _AP_S2>
4277 //      INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4278 //          _private_range_ref<_AP_W2, _AP_S2> >
4279 //      operator,(const _private_range_ref<_AP_W2, _AP_S2> &a2)
const {
4280 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4281 //          _private_range_ref<_AP_W2, _AP_S2>
>{
4282 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4283 //          const_cast<_private_range_ref<_AP_W2,
_AP_S2>&*>(&a2));
4284 //      }
4285 //
4286 //      template <int _AP_W2, bool _AP_S2>
4287 //      INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4288 //          _private_range_ref<_AP_W2, _AP_S2> >
4289 //      operator,(_private_range_ref<_AP_W2, _AP_S2> &a2) {
4290 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2,
4291 //          _private_range_ref<_AP_W2, _AP_S2>
>(&this, a2);
4292 //      }
4293 //
4294 //      template <int _AP_W2, bool _AP_S2>
4295 //      INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>, 1,
4296 //          _private_bit_ref<_AP_W2, _AP_S2> >
4297 //      operator,(const _private_bit_ref<_AP_W2, _AP_S2> &a2)
const {
4298 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
1,
4299 //          _private_bit_ref<_AP_W2, _AP_S2>
>{
4300 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4301 //          const_cast<_private_bit_ref<_AP_W2, _AP_S2>&*>(&a2));
4302 //      }
4303 //
4304 //      template <int _AP_W2, bool _AP_S2>
4305 //      INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>, 1,
4306 //          _private_bit_ref<_AP_W2, _AP_S2> >
4307 //      operator,(_private_bit_ref<_AP_W2, _AP_S2> &a2) {
4308 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
1,
4309 //          _private_bit_ref<_AP_W2, _AP_S2>
>(&this, a2);
4310 //      }
4311 //
4312 //      template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
4313 //      INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,

```

```

4314 //          ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >
4315 //      operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &a2) const {
4316 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,
4317 //          ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >{
4318 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4319 //          const_cast<ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3>&*>(&a2));
4320 //      }
4321 //
4322 //      template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
4323 //      INLINE ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,
4324 //          ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >
4325 //      operator,(ap_concat_ref<_AP_W2, _AP_T2, _AP_W3, _AP_T3>
&a2) {
4326 //      return ap_concat_ref<_AP_W, ap_private<_AP_W, _AP_S>,
_AP_W2 + _AP_W3,
4327 //          ap_concat_ref<_AP_W2, _AP_T2,
_AP_W3, _AP_T3> >(&this,
4328 //          a2);
4329 //      }
4330 //
4331 //      template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
4332 //          ap_o_mode _AP_O2, int _AP_N2>
4333 //      INLINE ap_concat_ref<
4334 //          _AP_W, ap_private, _AP_W2,
4335 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >
4336 //      operator,(const af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
4337 //          &a2) const {
4338 //      return ap_concat_ref<
4339 //          _AP_W, ap_private, _AP_W2,
4340 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> >{
4341 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4342 //          const_cast<
4343 //              af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2>&*>(&a2));
4344 //      }
4345 //
4346 //      template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
4347 //          ap_o_mode _AP_O2, int _AP_N2>
4348 //      INLINE ap_concat_ref<
4349 //          _AP_W, ap_private, _AP_W2,
4350 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >
4351 //      operator,(af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
4352 //      return ap_concat_ref<
4353 //          _AP_W, ap_private, _AP_W2,
4354 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> >(&this,
4355 //          a2);
4356 //      }
4357 //
4358 //      template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
4359 //          ap_o_mode _AP_O2, int _AP_N2>
4360 //      INLINE
4361 //          ap_concat_ref<_AP_W, ap_private, 1,
4362 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
4363 //      operator,(const af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>
4364 //          &a2) const {
4365 //      return ap_concat_ref<
4366 //          _AP_W, ap_private, 1,
4367 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >{
4368 //          const_cast<ap_private<_AP_W, _AP_S>&*>(&this),
4369 //          const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2>&*>(&a2));
4370 //      }
4371 //
4372 //
4373 //      template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
4374 //          ap_o_mode _AP_O2, int _AP_N2>
4375 //      INLINE
4376 //          ap_concat_ref<_AP_W, ap_private, 1,
4377 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2, _AP_N2> >
4378 //      operator,(
4379 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> &a2) {
4380 //      return ap_concat_ref<
4381 //          _AP_W, ap_private, 1,

```

```

4382 //      af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >(*this, a2);
4383 // }
4384
4385 inline ap_private<_AP_W, false> get() const {
4386     ap_private<_AP_W, false> ret(*this);
4387     return ret;
4388 }
4389
4390 template <int _AP_W3>
4391 inline void set(const ap_private<_AP_W3, false>& val) {
4392     operator=(ap_private<_AP_W3, _AP_S>(val));
4393 }
4394
4395 ///
4396 /// @name Value Tests
4397 ///
4398 /// This tests the high bit of this ap_private to determine
if it is set.
4399 /// @returns true if this ap_private is negative, false
otherwise
4400 /// @brief Determine sign of this ap_private.
4401 inline bool isNegative() const {
4402     // just for get rid of warnings
4403
4404     enum { shift = (_AP_W - APINT_BITS_PER_WORD * (_AP_N - 1)
- 1) };
4405     static const uint64_t mask = 1ULL << (shift);
4406     return _AP_S && (pVal[_AP_N - 1] & mask);
4407 }
4408
4409 /// This tests the high bit of the ap_private to determine
if it is unset.
4410 /// @brief Determine if this ap_private Value is positive
(not negative).
4411 inline bool isPositive() const { return !isNegative(); }
4412
4413 /// This tests if the value of this ap_private is strictly
positive (> 0).
4414 /// @returns true if this ap_private is Positive and not
zero.
4415 /// @brief Determine if this ap_private Value is strictly
positive.
4416 inline bool isStrictlyPositive() const {
4417     return isPositive() && (*this) != 0;
4418 }
4419
4420 /// This checks to see if the value has all bits of the
ap_private are set or
4421 /// not.
4422 /// @brief Determine if all bits are set
4423 inline bool isAllOnesValue() const { return countPopula-
tion() == _AP_W; }
4424
4425 /// This checks to see if the value of this ap_private is
the maximum unsigned
4426 /// value for the ap_private's bit width.
4427 /// @brief Determine if this is the largest unsigned value.
4428 inline bool isMaxValue() const { return countPopulation()
== _AP_W; }
4429
4430 /// This checks to see if the value of this ap_private is
the maximum signed
4431 /// value for the ap_private's bit width.
4432 /// @brief Determine if this is the largest signed value.
4433 inline bool isMaxSignedValue() const {
4434     return !isNegative() && countPopulation() == _AP_W - 1;
4435 }
4436
4437 /// This checks to see if the value of this ap_private is
the minimum unsigned
4438 /// value for the ap_private's bit width.
4439 /// @brief Determine if this is the smallest unsigned
value.
4440 inline bool isMinValue() const { return countPopulation()
== 0; }
4441
4442 /// This checks to see if the value of this ap_private is
the minimum signed
4443 /// value for the ap_private's bit width.
4444 /// @brief Determine if this is the smallest signed value.
4445 inline bool isMinSignedValue() const {
4446     return isNegative() && countPopulation() == 1;
4447 }
4448
4449 /// This function returns a pointer to the internal storage
of the ap_private.
4450
4451 /// This is useful for writing out the ap_private in binary
form without any
4452 /// conversions.
4453 inline const uint64_t* getRawData() const { return
&pVal[0]; }
4454
4455 /// Square Root - this method computes and returns the
square root of "this".
4456 /// Three mechanisms are used for computation. For small
values (<= 5 bits),
4457 // a table lookup is done. This gets some performance for
common cases. For
4458 // values using less than 52 bits, the value is converted
to double and then
4459 // the libc sqrt function is called. The result is rounded
and then converted
4460 // back to a uint64_t which is then used to construct the
result. Finally,
4461 // the Babylonian method for computing square roots is
used.
4462 inline ap_private sqrt() const {
4463     // Determine the magnitude of the value.
4464     uint32_t magnitude = getActiveBits();
4465
4466     // Use a fast table for some small values. This also gets
rid of some
4467     // rounding errors in libc sqrt for small values.
4468     if (magnitude <= 5) {
4469         static const uint8_t results[32] = {
4470             /* 0 */ 0,
4471             /* 1-2 */ 1, 1,
4472             /* 3-6 */ 2, 2, 2, 2,
4473             /* 7-12 */ 3, 3, 3, 3, 3, 3,
4474             /* 13-20 */ 4, 4, 4, 4, 4, 4, 4, 4,
4475             /* 21-30 */ 5, 5, 5, 5, 5, 5, 5, 5,
4476             /* 31 */ 6;
4477         return ap_private<_AP_W, _AP_S>(*BitWidth,*/ res-
ults[magnitude]);
4478     }
4479
4480     // If the magnitude of the value fits in less than 52
bits (the precision of
4481     // an IEEE double precision floating point value), then
we can use the
4482     // libc sqrt function which will probably use a hardware
sqrt computation.
4483     // This should be faster than the algorithm below.
4484     if (magnitude < 52) {
4485         #ifdef _MSC_VER
4486             // Amazingly, VC++ doesn't have round().
4487             return ap_private<_AP_W, _AP_S>(*BitWidth,*/
uint64_t::sqrt(double(get_VAL())) + 0.5);
4488         #else
4489             return ap_private<_AP_W, _AP_S>(*BitWidth,*/
uint64_t(
4490                 ::round(::sqrt(dou-
ble(get_VAL()))));
4491         #endif
4492     }
4493
4494     // Okay, all the short cuts are exhausted. We must com-
pute it. The following
4495     // is a classical Babylonian method for computing the
square root. This code
4496     // was adapted to APInt from a wikipedia article on such
computations.
4497     // See http://www.wikipedia.org/ and go to the page named
4498     // Calculate_an_integer_square_root.
4499     uint32_t nbits = BitWidth, i = 4;
4500     ap_private<_AP_W, _AP_S> testy(16);
4501     ap_private<_AP_W, _AP_S> x_old(*BitWidth,*/ 1);
4502     ap_private<_AP_W, _AP_S> x_new(0);
4503     ap_private<_AP_W, _AP_S> two(*BitWidth,*/ 2);
4504
4505     // Select a good starting value using binary logarithms.
4506     for (; i += 2, testy = testy.shl(2))
4507         if (i >= nbits || this->ule(testy)) {
4508             x_old = x_old.shl(i / 2);
4509             break;
4510         }
4511
4512     // Use the Babylonian method to arrive at the integer
square root:
4513     for (;;) {
4514         x_new = (this->udiv(x_old) + x_old).udiv(two);
4515         if (x_old.ule(x_new)) break;
4516         x_old = x_new;
4517     }
4518
4519     // Make sure we return the closest approximation
4520     // NOTE: The rounding calculation below is correct. It
will produce an
4521     // off-by-one discrepancy with results from pari/gp. That
discrepancy has
4522     // been
4523     // determined to be a rounding issue with pari/gp as it
begins to use a
4524     // floating point representation after 192 bits. There
are no discrepancies
4525     // between this algorithm and pari/gp for bit widths <
192 bits.
4526     ap_private<_AP_W, _AP_S> square(x_old * x_old);
4527     ap_private<_AP_W, _AP_S> nextSquare((x_old + 1) * (x_old
+ 1));
4528     if (this->ult(square))
4529         return x_old;
4530

```



```

4531     else if (this->ule(nextSquare)) {
4532         ap_private<_AP_W, _AP_S> midpoint((nextSquare -
square).udiv(two));
4533         ap_private<_AP_W, _AP_S> offset(*this - square);
4534         if (offset.ult(midpoint))
4535             return x_old;
4536         else
4537             return x_old + 1;
4538     } else
4539         assert(0 && "Error in ap_private<_AP_W, _AP_S>::sqrt
computation");
4540     return x_old + 1;
4541 }
4542
4543 ///
4544 /// @Assignment Operators
4545 ///
4546 /// @returns *this after assignment of RHS.
4547 /// @brief Copy assignment operator.
4548 INLINE ap_private& operator=(const ap_private& RHS) {
4549     if (this != &RHS) memcpy(pVal, RHS.get_pVal(), _AP_N *
APINT_WORD_SIZE);
4550     clearUnusedBits();
4551     return *this;
4552 }
4553 INLINE ap_private& operator=(const volatile ap_private&
RHS) {
4554     if (this != &RHS)
4555         for (int i = 0; i < _AP_N; ++i) pVal[i] = RHS.get_pV-
al(i);
4556     clearUnusedBits();
4557     return *this;
4558 }
4559 INLINE void operator=(const ap_private& RHS) volatile {
4560     if (this != &RHS)
4561         for (int i = 0; i < _AP_N; ++i) pVal[i] = RHS.get_pV-
al(i);
4562     clearUnusedBits();
4563 }
4564 INLINE void operator=(const volatile ap_private& RHS)
volatile {
4565     if (this != &RHS)
4566         for (int i = 0; i < _AP_N; ++i) pVal[i] = RHS.get_pV-
al(i);
4567     clearUnusedBits();
4568 }
4569
4570 template <int _AP_W1, bool _AP_S1>
4571 INLINE ap_private& operator=(const ap_private<_AP_W1,
_AP_S1>& RHS) {
4572     if (_AP_S1)
4573         cpSextOrTrunc(RHS);
4574     else
4575         cpZextOrTrunc(RHS);
4576     clearUnusedBits();
4577     return *this;
4578 }
4579
4580 template <int _AP_W1, bool _AP_S1>
4581 INLINE ap_private& operator=(const volatile
ap_private<_AP_W1, _AP_S1>& RHS) {
4582     if (_AP_S1)
4583         cpSextOrTrunc(RHS);
4584     else
4585         cpZextOrTrunc(RHS);
4586     clearUnusedBits();
4587     return *this;
4588 }
4589
4590 template <int _AP_W2, bool _AP_S2>
4591 INLINE ap_private& operator=(const
_private_range_ref<_AP_W2, _AP_S2>& op2) {
4592     *this = ap_private<_AP_W2, false>(op2);
4593     return *this;
4594 }
4595
4596 #if 0
4597 template<int _AP_W1, bool _AP_S1>
4598 INLINE ap_private& operator=(const ap_private<_AP_W1,
_AP_S1, true>& RHS) {
4599     static const uint64_t that_sign_ext_mask =
(_AP_W1==APINT_BITS_PER_WORD)?
0:-0ULL>>(_AP_W1%APINT_BITS_PER_WORD)<<(_AP_W1%APINT_BITS_PER_WORD)
;
4600     if (RHS.isNegative()) {
4601         pVal[0] = RHS.get_VAL() | that_sign_ext_mask;
4602         memset(pVal+1,-0, APINT_WORD_SIZE*( _AP_N-1));
4603     } else {
4604         pVal[0] = RHS.get_VAL();
4605         memset(pVal+1, 0, APINT_WORD_SIZE*( _AP_N-1));
4606     }
4607     clearUnusedBits();
4608     return *this;
4609 }
4610
4611 template<int _AP_W1, bool _AP_S1>
4612 INLINE ap_private& operator=(const volatile
ap_private<_AP_W1, _AP_S1, true>& RHS) {
4613     static const uint64_t that_sign_ext_mask =
(_AP_W1==APINT_BITS_PER_WORD)?

```

```

0:-0ULL>>(_AP_W1%APINT_BITS_PER_WORD)<<(_AP_W1%APINT_BITS_PER_WORD)
;
4614     if (RHS.isNegative()) {
4615         pVal[0] = RHS.get_VAL() | that_sign_ext_mask;
4616         memset(pVal+1,-0, APINT_WORD_SIZE*( _AP_N-1));
4617     } else {
4618         pVal[0] = RHS.get_VAL();
4619         memset(pVal+1, 0, APINT_WORD_SIZE*( _AP_N-1));
4620     }
4621     clearUnusedBits();
4622     return *this;
4623 }
4624 #endif
4625
4626 /// from all c types.
4627 #define ASSIGN_OP_FROM_INT(C_TYPE, _AP_W2, _AP_S2) \
4628     INLINE ap_private& operator=(const C_TYPE rhs) { \
4629         ap_private<(_AP_W2), (_AP_S2)> tmp = rhs; \
4630         operator=(tmp); \
4631         return *this; \
4632     }
4633
4634 ASSIGN_OP_FROM_INT(bool, 1, false)
4635 ASSIGN_OP_FROM_INT(char, 8, CHAR_IS_SIGNED)
4636 ASSIGN_OP_FROM_INT(signed char, 8, true)
4637 ASSIGN_OP_FROM_INT(unsigned char, 8, false)
4638 ASSIGN_OP_FROM_INT(short, sizeof(short) * 8, true)
4639 ASSIGN_OP_FROM_INT(unsigned short, sizeof(unsigned short) *
8, false)
4640 ASSIGN_OP_FROM_INT(int, sizeof(int) * 8, true)
4641 ASSIGN_OP_FROM_INT(unsigned int, sizeof(unsigned int) * 8,
false)
4642 ASSIGN_OP_FROM_INT(long, sizeof(long) * 8, true)
4643 ASSIGN_OP_FROM_INT(unsigned long, sizeof(unsigned long) *
8, false)
4644 ASSIGN_OP_FROM_INT(ap_slong, sizeof(ap_slong) * 8, true)
4645 ASSIGN_OP_FROM_INT(ap_ulong, sizeof(ap_ulong) * 8, false)
4646 #undef ASSIGN_OP_FROM_INT
4647
4648 /// from c string.
4649 // XXX this is a must, to prevent pointer being converted
to bool.
4650 INLINE ap_private& operator=(const char* s) {
4651     ap_private tmp(s); // XXX direct initialization, as ctor
is explicit.
4652     operator=(tmp);
4653     return *this;
4654 }
4655
4656 ///
4657 /// @name Unary Operators
4658 ///
4659 /// @returns a new ap_private value representing *this in-
cremented by one
4660 /// @brief Postfix increment operator.
4661 INLINE const ap_private operator++(int) {
4662     ap_private API(*this);
4663     ++(*this);
4664     return API;
4665 }
4666
4667 /// @returns *this incremented by one
4668 /// @brief Prefix increment operator.
4669 INLINE ap_private& operator++() {
4670     ap_private_ops::add_1(pVal, pVal, _AP_N, 1);
4671     clearUnusedBits();
4672     return *this;
4673 }
4674
4675 /// @returns a new ap_private representing *this decremen-
ted by one.
4676 /// @brief Postfix decrement operator.
4677 INLINE const ap_private operator--(int) {
4678     ap_private API(*this);
4679     --(*this);
4680     return API;
4681 }
4682
4683 /// @returns *this decremented by one.
4684 /// @brief Prefix decrement operator.
4685 INLINE ap_private& operator--() {
4686     ap_private_ops::sub_1(pVal, _AP_N, 1);
4687     clearUnusedBits();
4688     return *this;
4689 }
4690
4691 /// Performs a bitwise complement operation on this
ap_private.
4692 /// @returns an ap_private that is the bitwise complement
of *this
4693 /// @brief Unary bitwise complement operator.
4694 INLINE ap_private<_AP_W + !_AP_S, true> operator~() const {
4695     ap_private<_AP_W + !_AP_S, true> Result(*this);
4696     Result.flip();
4697     return Result;
4698 }
4699
4700 /// Negates *this using two's complement logic.
4701 /// @returns An ap_private value representing the negation

```

```

of *this.
4702 /// @brief Unary negation operator
4703 INLINE typename RType<1, false>::minus operator-() const {
4704     return ap_private<1, false>(0) - (*this);
4705 }
4706
4707 /// Performs logical negation operation on this ap_private.
4708 /// @returns true if *this is zero, false otherwise.
4709 /// @brief Logical negation operator.
4710 INLINE bool operator!() const {
4711     for (int i = 0; i < _AP_N; ++i)
4712         if (pVal[i]) return false;
4713     return true;
4714 }
4715
4716 template <bool _AP_S1>
4717 INLINE ap_private<_AP_W, _AP_S || _AP_S1> And(
4718     const ap_private<_AP_W, _AP_S1>& RHS) const {
4719     return this->operator&(RHS);
4720 }
4721
4722 template <bool _AP_S1>
4723 INLINE ap_private Or(const ap_private<_AP_W, _AP_S1>& RHS)
4724 const {
4725     return this->operator|(RHS);
4726 }
4727
4728 template <bool _AP_S1>
4729
4730 INLINE ap_private Xor(const ap_private<_AP_W, _AP_S1>& RHS)
4731 const {
4732     return this->operator^(RHS);
4733 }
4734
4735 INLINE ap_private Mul(const ap_private& RHS) const {
4736     ap_private Result(*this);
4737     Result *= RHS;
4738     return Result;
4739 }
4740
4741 INLINE ap_private Add(const ap_private& RHS) const {
4742     ap_private Result(0);
4743     ap_private_ops::add(Result.get_pVal(), pVal, RHS.get_pV-
4744 al(), _AP_N, _AP_N,
4745                        _AP_N, _AP_S, _AP_S);
4746     Result.clearUnusedBits();
4747     return Result;
4748 }
4749
4750 INLINE ap_private Sub(const ap_private& RHS) const {
4751     ap_private Result(0);
4752     ap_private_ops::sub(Result.get_pVal(), pVal, RHS.get_pV-
4753 al(), _AP_N, _AP_N,
4754                        _AP_N, _AP_S, _AP_S);
4755     Result.clearUnusedBits();
4756     return Result;
4757 }
4758
4759 /// Arithmetic right-shift this ap_private by shiftAmt.
4760 /// @brief Arithmetic right-shift function.
4761 INLINE ap_private ashr(uint32_t shiftAmt) const {
4762
4763     assert(shiftAmt <= BitWidth && "Invalid shift amount, too
4764 big");
4765     // Handle a degenerate case
4766     if (shiftAmt == 0) return ap_private(*this);
4767
4768     // If all the bits were shifted out, the result is, tech-
4769 nically, undefined.
4770
4771 // We return -1 if it was negative, 0 otherwise. We check
4772 this early to
4773 // avoid
4774 // issues in the algorithm below.
4775 if (shiftAmt == BitWidth) {
4776     if (isNegative())
4777         return ap_private(-1);
4778     else
4779         return ap_private(0);
4780 }
4781
4782 // Create some space for the result.
4783 ap_private Retval(0);
4784 uint64_t* val = Retval.get_pVal();
4785
4786 // Compute some values needed by the following shift al-
4787 gorithms
4788 uint32_t wordShift =
4789
4790     shiftAmt % APINT_BITS_PER_WORD; // bits
4791 to shift per word
4792
4793 uint32_t offset = shiftAmt / APINT_BITS_PER_WORD; // word
4794 offset for shift
4795
4796 uint32_t breakWord = _AP_N - 1 - offset; // last
4797 word affected
4798 uint32_t bitsInWord = whichBit(BitWidth); // how many
4799 bits in last word?
4800 if (bitsInWord == 0) bitsInWord = APINT_BITS_PER_WORD;
4801
4802 // If we are shifting whole words, just move whole words
4803
4804 if (wordShift == 0) {
4805     // Move the words containing significant bits
4806     for (uint32_t i = 0; i <= breakWord; ++i)
4807         val[i] = pVal[i + offset]; // move whole word
4808
4809     // Adjust the top significant word for sign bit fill,
4810 if negative
4811     if (isNegative())
4812         if (bitsInWord < APINT_BITS_PER_WORD)
4813             val[breakWord] |= ~0ULL << (bitsInWord); // set
4814 high bits
4815 } else {
4816     // Shift the low order words
4817     for (uint32_t i = 0; i < breakWord; ++i) {
4818         // This combines the shifted corresponding word with
4819 the low bits from
4820         // the next word (shifted into this word's high
4821 bits).
4822         val[i] = ((pVal[i + offset]) >> (wordShift));
4823         val[i] |= ((pVal[i + offset + 1]) <<
4824 (APINT_BITS_PER_WORD - wordShift));
4825     }
4826
4827 // Shift the break word. In this case there are no bits
4828 from the next word
4829 // to include in this word.
4830 val[breakWord] = (pVal[breakWord + offset]) >> (word-
4831 Shift);
4832
4833 // Deal with sign extension in the break word, and
4834 possibly the word
4835 // before
4836 // it.
4837 if (isNegative()) {
4838     if (wordShift > bitsInWord) {
4839         if (breakWord > 0)
4840             val[breakWord - 1] |=
4841 ~0ULL << (APINT_BITS_PER_WORD - (wordShift -
4842 bitsInWord));
4843         val[breakWord] |= ~0ULL;
4844     } else
4845         val[breakWord] |= (~0ULL << (bitsInWord - word-
4846 Shift));
4847 }
4848
4849 // Remaining words are 0 or -1, just assign them.
4850 uint64_t fillValue = (isNegative()) ? ~0ULL : 0;
4851 for (int i = breakWord + 1; i < _AP_N; ++i) val[i] =
4852 fillValue;
4853 Retval.clearUnusedBits();
4854 return Retval;
4855 }
4856
4857 // Logical right-shift this ap_private by shiftAmt.
4858 /// @brief Logical right-shift function.
4859 INLINE ap_private lshr(uint32_t shiftAmt) const {
4860     // If all the bits were shifted out, the result is 0.
4861 This avoids issues
4862 // with shifting by the size of the integer type, which
4863 produces undefined
4864
4865 // results. We define these "undefined results" to always
4866 be 0.
4867 if (shiftAmt == BitWidth) return ap_private(0);
4868
4869 // If none of the bits are shifted out, the result is
4870 *this. This avoids
4871 // issues with shifting by the size of the integer type,
4872 which produces
4873 // undefined results in the code below. This is also an
4874 optimization.
4875 if (shiftAmt == 0) return ap_private(*this);
4876
4877 // Create some space for the result.
4878 ap_private Retval(0);
4879 uint64_t* val = Retval.get_pVal();
4880
4881 // If we are shifting less than a word, compute the shift
4882 with a simple
4883 // carry
4884 if (shiftAmt < APINT_BITS_PER_WORD) {
4885     uint64_t carry = 0;
4886     for (int i = _AP_N - 1; i >= 0; --i) {
4887         val[i] = ((pVal[i]) >> (shiftAmt)) | carry;
4888         carry = (pVal[i]) << (APINT_BITS_PER_WORD -
4889 shiftAmt);
4890     }
4891     Retval.clearUnusedBits();
4892     return Retval;
4893 }
4894
4895 // Compute some values needed by the remaining shift al-
4896 gorithms
4897 uint32_t wordShift = shiftAmt % APINT_BITS_PER_WORD;
4898 uint32_t offset = shiftAmt / APINT_BITS_PER_WORD;
4899
4900 // If we are shifting whole words, just move whole words

```

```

4860     if (wordShift == 0) {
4861         for (uint32_t i = 0; i < _AP_N - offset; ++i) val[i] =
pVal[i + offset];
4862         for (uint32_t i = _AP_N - offset; i < _AP_N; i++)
val[i] = 0;
4863         Retval.clearUnusedBits();
4864         return Retval;
4865     }
4866     // Shift the low order words
4867     uint32_t breakWord = _AP_N - offset - 1;
4868     for (uint32_t i = 0; i < breakWord; ++i)
4869         val[i] = ((pVal[i + offset] >> (wordShift)) |
4870                 ((pVal[i + offset + 1] <<
(APINT_BITS_PER_WORD - wordShift)));
4871     // Shift the break word.
4872     val[breakWord] = (pVal[breakWord + offset] >> (word-
Shift));
4873     // Remaining words are 0
4874     for (int i = breakWord + 1; i < _AP_N; ++i) val[i] = 0;
4875     Retval.clearUnusedBits();
4876     return Retval;
4877 }
4878
4879 // Left-shift this ap_private by shiftAmt.
4880 // @brief Left-shift function.
4881
4882
4883
4884
4885
4886
4887
4888
4889
4890
4891
4892
4893
4894
4895
4896
4897
4898
4899
4900
4901
4902
4903
4904
4905
4906
4907
4908
4909
4910
4911
4912
4913
4914
4915
4916
4917
4918
4919
4920
4921
4922
4923
4924
4925
4926
4927
4928
4929
4930
4931
4932
4933
4934
4935
4936
4937
4938
4939
4940
4941
4942
4943
4944
4945
4946
4947
4948
4949
4950
4951
4952
4953
4954
4955
4956
4957
4958
4959
4960
4961
4962
4963
4964
4965
4966
4967
4968
4969
4970
4971
4972
4973
4974
4975
4976
4977
4978
4979
4980
4981
4982
4983
4984
4985
4986
4987
4988
4989
4990
4991
4992
4993
4994
4995
4996
4997
4998
4999
5000
5001
5002
5003
5004
5005
5006
5007
5008
5009
5010
5011
5012
5013
5014
5015
5016
5017
5018
5019

```

```

4940     return hi | lo;
4941 }
4942
4943
4944
4945
4946
4947
4948
4949
4950
4951
4952
4953
4954
4955
4956
4957
4958
4959
4960
4961
4962
4963
4964
4965
4966
4967
4968
4969
4970
4971
4972
4973
4974
4975
4976
4977
4978
4979
4980
4981
4982
4983
4984
4985
4986
4987
4988
4989
4990
4991
4992
4993
4994
4995
4996
4997
4998
4999
5000
5001
5002
5003
5004
5005
5006
5007
5008
5009
5010
5011
5012
5013
5014
5015
5016
5017
5018
5019

```

```

5020     if (lhsWords == 0) {
5021         // 0 % Y ==> 0
5022         return ap_private(0);
5023     } else if (lhsWords < rhsWords || this->ult(RHS)) {
5024         // X % Y ==> X, iff X < Y
5025         return *this;
5026     } else if (*this == RHS) {
5027         // X % X == 0;
5028         return ap_private(0);
5029     } else if (lhsWords == 1) {
5030         // All high words are zero, just use native remainder
5031         return ap_private(pVal[0] % RHS.get_pVal(0));
5032     }
5033
5034     // We have to compute it the hard way. Invoke the Knuth
divide algorithm.
5035     ap_private Remainder(0);
5036     ap_private_ops::divide(*this, lhsWords, RHS, rhsWords,
(ap_private*)(0),
5037                             &Remainder);
5038     return Remainder;
5039 }
5040
5041     INLINE ap_private urem(uint64_t RHS) const {
5042         // Get some facts about the LHS
5043         uint32_t lhsBits = getActiveBits();
5044         uint32_t lhsWords = !lhsBits ? 0 : (whichWord(lhsBits -
1) + 1);
5045         // Get some facts about the RHS
5046         uint32_t rhsWords = 1; //! rhsBits ? 0 :
(ap_private<_AP_W,
5047                                     //! _AP_S::whichWord(rhsBits - 1)
+ 1);
5048         assert(rhsWords && "Performing remainder operation by
zero ???");
5049         // Check the degenerate cases
5050         if (lhsWords == 0) {
5051             // 0 % Y ==> 0
5052             return ap_private(0);
5053         } else if (lhsWords < rhsWords || this->ult(RHS)) {
5054             // X % Y ==> X, iff X < Y
5055             return *this;
5056         } else if (*this == RHS) {
5057             // X % X == 0;
5058             return ap_private(0);
5059         } else if (lhsWords == 1) {
5060             // All high words are zero, just use native remainder
5061             return ap_private(pVal[0] % RHS);
5062         }
5063
5064         // We have to compute it the hard way. Invoke the Knuth
divide algorithm.
5065         ap_private Remainder(0);
5066         divide(*this, lhsWords, RHS, (ap_private*)(0), &Re-
mainder);
5067         return Remainder;
5068     }
5069
5070     /// Signed remainder operation on ap_private.
5071     /// @brief Function for signed remainder operation.
5072     INLINE ap_private srem(const ap_private& RHS) const {
5073         if (isNegative()) {
5074             ap_private lhs = -*this;
5075             if (RHS.isNegative()) {
5076                 ap_private rhs = -RHS;
5077                 return -(lhs.urem(rhs));
5078             } else
5079                 return -(lhs.urem(RHS));
5080         } else if (RHS.isNegative()) {
5081             ap_private rhs = -RHS;
5082             return this->urem(rhs);
5083         }
5084         return this->urem(RHS);
5085     }
5086
5087     /// Signed remainder operation on ap_private.
5088     /// @brief Function for signed remainder operation.
5089     INLINE ap_private srem(int64_t RHS) const {
5090         if (isNegative())
5091             if (RHS < 0)
5092                 return -((-(*this)).urem(-RHS));
5093             else
5094                 return -((-(*this)).urem(RHS));
5095         else if (RHS < 0)
5096             return this->urem(-RHS);
5097         return this->urem(RHS);
5098     }
5099
5100     /// Compares this ap_private with RHS for the validity of
the equality
5101     /// relationship.
5102     /// @returns true if *this == Val
5103     /// @brief Equality comparison.
5104     template <bool _AP_S1>
5105     INLINE bool eq(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5106         return (*this) == RHS;
5107     }
5108

```

```

5109     /// Compares this ap_private with RHS for the validity of
the inequality
5110     /// relationship.
5111     /// @returns true if *this != Val
5112     /// @brief Inequality comparison
5113     template <bool _AP_S1>
5114     INLINE bool ne(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5115         return !((*this) == RHS);
5116     }
5117
5118     /// Regards both *this and RHS as unsigned quantities and
compares them for
5119     /// the validity of the less-than relationship.
5120     /// @returns true if *this < RHS when both are considered
unsigned.
5121     /// @brief Unsigned less than comparison
5122     template <bool _AP_S1>
5123     INLINE bool ult(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5124         // Get active bit length of both operands
5125         uint32_t n1 = getActiveBits();
5126         uint32_t n2 = RHS.getActiveBits();
5127
5128         // If magnitude of LHS is less than RHS, return true.
5129         if (n1 < n2) return true;
5130
5131         // If magnitude of RHS is greater than LHS, return
false.
5132         if (n2 < n1) return false;
5133
5134         // If they bot fit in a word, just compare the low order
word
5135         if (n1 <= APINT_BITS_PER_WORD && n2 <=
APINT_BITS_PER_WORD)
5136             return pVal[0] < RHS.get_pVal(0);
5137
5138         // Otherwise, compare all words
5139         uint32_t topWord = whichWord(AESL_std::max(n1, n2) - 1);
5140         for (int i = topWord; i >= 0; --i) {
5141             if (pVal[i] > RHS.get_pVal(i)) return false;
5142             if (pVal[i] < RHS.get_pVal(i)) return true;
5143         }
5144         return false;
5145     }
5146
5147     INLINE bool ult(uint64_t RHS) const {
5148         // Get active bit length of both operands
5149         uint32_t n1 = getActiveBits();
5150         uint32_t n2 =
5151             64 - ap_private_ops::CountLeadingZeros_64(RHS); //
RHS.getActiveBits();
5152
5153         // If magnitude of LHS is less than RHS, return true.
5154         if (n1 < n2) return true;
5155
5156         // If magnitude of RHS is greater than LHS, return
false.
5157         if (n2 < n1) return false;
5158
5159         // If they bot fit in a word, just compare the low order
word
5160         if (n1 <= APINT_BITS_PER_WORD && n2 <=
APINT_BITS_PER_WORD)
5161             return pVal[0] < RHS;
5162         assert(0);
5163     }
5164
5165     template <bool _AP_S1>
5166     INLINE bool slt(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5167         ap_private lhs(*this);
5168         ap_private<_AP_W, _AP_S1> rhs(RHS);
5169         bool lhsNeg = isNegative();
5170         bool rhsNeg = rhs.isNegative();
5171         if (lhsNeg) {
5172             // Sign bit is set so perform two's complement to make
it positive
5173             lhs.flip();
5174             lhs++;
5175         }
5176         if (rhsNeg) {
5177             // Sign bit is set so perform two's complement to make
it positive
5178             rhs.flip();
5179             rhs++;
5180         }
5181
5182         // Now we have unsigned values to compare so do the com-
parison if necessary
5183         // based on the negativeness of the values.
5184         if (lhsNeg)
5185             if (rhsNeg)
5186                 return lhs.ugt(rhs);
5187             else
5188                 return true;
5189         else if (rhsNeg)
5190             return false;

```

```

5191     else
5192         return lhs.ult(rhs);
5193     }
5194 }
5195 /// Regards both *this and RHS as unsigned quantities and
compares them for
5196 /// validity of the less-or-equal relationship.
5197 /// @returns true if *this <= RHS when both are considered
unsigned.
5198 /// @brief Unsigned less or equal comparison
5199 template <bool _AP_S1>

5200 INLINE bool ule(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5201     return ult(RHS) || eq(RHS);
5202 }
5203 }
5204 /// Regards both *this and RHS as signed quantities and
compares them for
5205 /// validity of the less-or-equal relationship.
5206 /// @returns true if *this <= RHS when both are considered
signed.
5207 /// @brief Signed less or equal comparison
5208 template <bool _AP_S1>

5209 INLINE bool sle(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5210     return slt(RHS) || eq(RHS);
5211 }
5212 }
5213 /// Regards both *this and RHS as unsigned quantities and
compares them for
5214 /// the validity of the greater-than relationship.
5215 /// @returns true if *this > RHS when both are considered
unsigned.
5216 /// @brief Unsigned greater than comparison
5217 template <bool _AP_S1>

5218 INLINE bool ugt(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5219     return !ult(RHS) && !eq(RHS);
5220 }
5221 }
5222 /// Regards both *this and RHS as signed quantities and
compares them for
5223 /// the validity of the greater-than relationship.
5224 /// @returns true if *this > RHS when both are considered
signed.
5225 /// @brief Signed greater than comparison
5226 template <bool _AP_S1>

5227 INLINE bool sgt(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5228     return !slt(RHS) && !eq(RHS);
5229 }
5230 }
5231 /// Regards both *this and RHS as unsigned quantities and
compares them for
5232 /// validity of the greater-or-equal relationship.
5233 /// @returns true if *this >= RHS when both are considered
unsigned.
5234 /// @brief Unsigned greater or equal comparison
5235 template <bool _AP_S1>
5236 INLINE bool uge(const ap_private<_AP_W, _AP_S>& RHS) const
{
5237     return !ult(RHS);
5238 }
5239 }
5240 /// Regards both *this and RHS as signed quantities and
compares them for
5241 /// validity of the greater-or-equal relationship.
5242 /// @returns true if *this >= RHS when both are considered
signed.
5243 /// @brief Signed greater or equal comparison
5244 template <bool _AP_S1>

5245 INLINE bool sge(const ap_private<_AP_W, _AP_S1>& RHS) const
{
5246     return !slt(RHS);
5247 }
5248 }
5249 // Sign extend to a new width.
5250 template <int _AP_W1, bool _AP_S1>
5251 INLINE void cpSext(const ap_private<_AP_W1, _AP_S1>& that)
{
5252     assert(_AP_W1 < BitWidth && "Invalid ap_private SignEx-
tend request");
5253     assert(_AP_W1 <= MAX_INT_BITS && "Too many bits");
5254     // If the sign bit isn't set, this is the same as zext.
5255     if (!that.isNegative()) {
5256         cpZext(that);
5257         return;
5258     }
5259 }

5260 // The sign bit is set. First, get some facts
5261 enum { wordBits = _AP_W1 % APINT_BITS_PER_WORD };
5262 const int _AP_N1 = ap_private<_AP_W1, _AP_S1>::_AP_N;
5263 // Mask the high order word appropriately
5264 if (_AP_N1 == _AP_N) {
5265     enum { newWordBits = _AP_W % APINT_BITS_PER_WORD };

```

```

5266     // The extension is contained to the wordsBefore-1th
word.
5267 static const uint64_t mask = wordBits ? (~0ULL <<
(wordBits)) : 0ULL;
5268 for (int i = 0; i < _AP_N; ++i) pVal[i] = that.get_pV-
al(i);
5269 pVal[_AP_N - 1] |= mask;
5270 return;
5271 }
5272 }
5273 enum { newWordBits = _AP_W % APINT_BITS_PER_WORD };
5274 // The extension is contained to the wordsBefore-1th
word.
5275 static const uint64_t mask = wordBits ? (~0ULL << (word-
Bits)) : 0ULL;
5276 int i;
5277 for (i = 0; i < _AP_N1; ++i) pVal[i] = that.get_pVal(i);
5278 pVal[i - 1] |= mask;
5279 for (; i < _AP_N - 1; i++) pVal[i] = ~0ULL;
5280 pVal[i] = ~0ULL;
5281 clearUnusedBits();
5282 return;
5283 }
5284 }
5285 // Zero extend to a new width.
5286 template <int _AP_W1, bool _AP_S1>
5287 INLINE void cpZext(const ap_private<_AP_W1, _AP_S1>& that)
{
5288     assert(_AP_W1 < BitWidth && "Invalid ap_private ZeroEx-
tend request");
5289     assert(_AP_W1 <= MAX_INT_BITS && "Too many bits");
5290     const int _AP_N1 = ap_private<_AP_W1, _AP_S1>::_AP_N;
5291     int i = 0;
5292     for (; i < _AP_N1; ++i) pVal[i] = that.get_pVal(i);
5293     for (; i < _AP_N; ++i) pVal[i] = 0;
5294     clearUnusedBits();
5295 }
5296 }
5297 template <int _AP_W1, bool _AP_S1>

5298 INLINE void cpZextOrTrunc(const ap_private<_AP_W1, _AP_S1>&
that) {
5299     if (BitWidth > _AP_W1)
5300         cpZext(that);
5301     else {
5302         for (int i = 0; i < _AP_N; ++i) pVal[i] = that.get_pV-
al(i);
5303         clearUnusedBits();
5304     }
5305 }
5306 }
5307 template <int _AP_W1, bool _AP_S1>

5308 INLINE void cpSextOrTrunc(const ap_private<_AP_W1, _AP_S1>&
that) {
5309     if (BitWidth > _AP_W1)
5310         cpSext(that);
5311     else {
5312         for (int i = 0; i < _AP_N; ++i) pVal[i] = that.get_pV-
al(i);
5313         clearUnusedBits();
5314     }
5315 }
5316 }
5317 /// @}
5318 /// @name Value Characterization Functions
5319 /// @{
5320 }
5321 /// @returns the total number of bits.
5322 INLINE uint32_t getBitWidth() const { return BitWidth; }
5323 }
5324 /// Here one word's bitwidth equals to that of uint64_t.
5325 /// @returns the number of words to hold the integer value
of this ap_private.
5326 /// @brief Get the number of words.
5327 INLINE uint32_t getNumWords() const {
5328     return (BitWidth + APINT_BITS_PER_WORD - 1) /
APINT_BITS_PER_WORD;
5329 }
5330 }
5331 /// This function returns the number of active bits which
is defined as the
5332 /// bit width minus the number of leading zeros. This is
used in several
5333 /// computations to see how "wide" the value is.
5334 /// @brief Compute the number of active bits in the value
5335 INLINE uint32_t getActiveBits() const {
5336     uint32_t bits = BitWidth - countLeadingZeros();
5337     return bits ? bits : 1;
5338 }
5339 }
5340 /// This method attempts to return the value of this
ap_private as a zero
5341 /// extended
5342 /// uint64_t. The bitwidth must be <= 64 or the value must
fit within a
5343 /// uint64_t. Otherwise an assertion will result.
5344 /// @brief Get zero extended value
5345 INLINE uint64_t getZExtValue() const {
5346     assert(getActiveBits() <= 64 && "Too many bits for

```



```

uint64_t");
5347     return *pVal;
5348 }
5349
5350 /// This method attempts to return the value of this
ap_private as a sign
5351 /// extended
5352 /// int64_t. The bit width must be <= 64 or the value must
fit within an
5353 /// int64_t. Otherwise an assertion will result.
5354 /// @brief Get sign extended value
5355 INLINE int64_t getSExtValue() const {
5356     assert(getActiveBits() <= 64 && "Too many bits for in-
t64_t");
5357     return int64_t(pVal[0]);
5358 }
5359
5360 /// This method determines how many bits are required to
hold the ap_private
5361 /// equivalent of the string given by \p str of length \p
slen.
5362 /// @brief Get bits required for string value.
5363 INLINE static uint32_t getBitsNeeded(const char* str,
uint32_t slen,
5364                                     uint8_t radix) {
5365     assert(str != 0 && "Invalid value string");
5366     assert(slen > 0 && "Invalid string length");
5367
5368     // Each computation below needs to know if its negative
5369     uint32_t isNegative = str[0] == '-';
5370     if (isNegative) {
5371         slen--;
5372         str++;
5373     }
5374     // For radices of power-of-two values, the bits required
is accurately and
5375     // easily computed
5376     if (radix == 2) return slen + isNegative;
5377     if (radix == 8) return slen * 3 + isNegative;
5378     if (radix == 16) return slen * 4 + isNegative;
5379
5380     // Otherwise it must be radix == 10, the hard case
5381     assert(radix == 10 && "Invalid radix");
5382
5383     // Convert to the actual binary value.
5384     // ap_private<_AP_W, _AP_S> tmp(sufficient, str, slen,
radix);
5385
5386     // Compute how many bits are required.
5387     // return isNegative + tmp.logBase2() + 1;
5388     return isNegative + slen * 4;
5389 }
5390
5391 /// countLeadingZeros - This function is an ap_private ver-
sion of the
5392 /// countLeadingZeros_{32,64} functions in MathExtras.h. It
counts the number
5393 /// of zeros from the most significant bit to the first one
bit.
5394 /// @returns BitWidth if the value is zero.
5395 /// @returns the number of zeros from the most significant
bit to the first
5396 /// one bits.
5397 INLINE uint32_t countLeadingZeros() const {
5398     enum {
5399         msw_bits = (BitWidth % APINT_BITS_PER_WORD)
5400             ? (BitWidth % APINT_BITS_PER_WORD)
5401             : APINT_BITS_PER_WORD,
5402         excessBits = APINT_BITS_PER_WORD - msw_bits
5403     };
5404     uint32_t Count =
ap_private_ops::CountLeadingZeros_64(pVal[_AP_N - 1]);
5405     if (Count >= excessBits) Count -= excessBits;
5406     if (pVal[_AP_N - 1]) {
5407         for (int i = _AP_N - 1; i; --i) {
5408             if (!pVal[i - 1])
5409                 Count += APINT_BITS_PER_WORD;
5410             else {
5411                 Count +=
ap_private_ops::CountLeadingZeros_64(pVal[i - 1]);
5412                 break;
5413             }
5414         }
5415     }
5416     return Count;
5417 }
5418
5419 /// countLeadingOnes - This function counts the number of
contiguous 1 bits
5420 /// in the high order bits. The count stops when the first
0 bit is reached.
5421 /// @returns 0 if the high order bit is not set
5422
5423 /// @returns the number of 1 bits from the most significant
to the least
5424 /// @brief Count the number of leading one bits.
5425 INLINE uint32_t countLeadingOnes() const {
    if (isSingleWord())
5426         return countLeadingOnes_64(get_VAL(),
APINT_BITS_PER_WORD - BitWidth);
5427
5428     uint32_t highWordBits = BitWidth % APINT_BITS_PER_WORD;
5429     uint32_t shift =
    (highWordBits == 0 ? 0 : APINT_BITS_PER_WORD - high-
WordBits);
5430     int i = _AP_N - 1;
5431     uint32_t Count = countLeadingOnes_64(get_pVal(i), shift);
5432     if (Count == highWordBits) {
5433         for (i--; i >= 0; --i) {
5434             if (get_pVal(i) == ~0ULL)
5435                 Count += APINT_BITS_PER_WORD;
5436             else {
5437                 Count += countLeadingOnes_64(get_pVal(i), 0);
5438                 break;
5439             }
5440         }
5441     }
5442     return Count;
5443 }
5444
5445 /// countTrailingZeros - This function is an ap_private
version of the
5446 /// countTrailingZeros_{32,64} functions in MathExtras.h.
It counts
5447 /// the number of zeros from the least significant bit to
the first set bit.
5448 /// @returns BitWidth if the value is zero.
5449
5450 /// @returns the number of zeros from the least significant
bit to the first
5451 /// one bit.
5452 /// @brief Count the number of trailing zero bits.
5453 INLINE uint32_t countTrailingZeros() const {
5454     uint32_t Count = 0;
5455     uint32_t i = 0;
5456     for (; i < _AP_N && get_pVal(i) == 0; ++i) Count +=
APINT_BITS_PER_WORD;
5457     if (i < _AP_N) Count +=
ap_private_ops::CountTrailingZeros_64(get_pVal(i));
5458     return AESL_std::min(Count, BitWidth);
5459 }
5460
5461 /// countPopulation - This function is an ap_private ver-
sion of the
5462 /// countPopulation_{32,64} functions in MathExtras.h. It
counts the number
5463 /// of 1 bits in the ap_private value.
5464 /// @returns 0 if the value is zero.
5465 /// @returns the number of set bits.
5466 /// @brief Count the number of bits set.
5467 INLINE uint32_t countPopulation() const {
5468     uint32_t Count = 0;
5469     for (int i = 0; i < _AP_N - 1; ++i)
5470         Count += ap_private_ops::CountPopulation_64(pVal[i]);
5471     Count += ap_private_ops::CountPopulation_64(pVal[_AP_N -
1] & mask);
5472     return Count;
5473 }
5474
5475 /// @}
5476 /// @name Conversion Functions
5477
5478 /// This is used internally to convert an ap_private to a
string.
5479 /// @brief Converts an ap_private to a std::string
5480
5481
5482 /// Considers the ap_private to be unsigned and converts it
into a string in
5483 /// the
5484 /// radix given. The radix can be 2, 8, 10 or 16.
5485 /// @returns a character interpretation of the ap_private
5486 /// @brief Convert unsigned ap_private to string represent-
ation.
5487 INLINE std::string toStringUnsigned(uint8_t radix = 10)
const {
5488     return toString(radix, false);
5489 }
5490
5491 /// Considers the ap_private to be unsigned and converts it
into a string in
5492 /// the
5493 /// radix given. The radix can be 2, 8, 10 or 16.
5494 /// @returns a character interpretation of the ap_private
5495 /// @brief Convert unsigned ap_private to string represent-
ation.
5496
5497
5498
5499
5500
5501

```

```

5502 // Handle the simple case where the value is contained in
one uint64_t.
5503 if (isSingleWord() || getActiveBits() <=
APINT_BITS_PER_WORD) {
5504     uint64_t val = pVal[0];
5505     if (isSigned) {
5506         int64_t sext = ((int64_t(val)) << (64 - BitWidth)) >>
(64 - BitWidth);
5507         return double(sext);
5508     } else
5509         return double(val);
5510     }
5511
5512 // Determine if the value is negative.
5513 bool isNeg = isSigned ? (*this)[BitWidth - 1] : false;
5514
5515 // Construct the absolute value if we're negative.
5516 ap_private<_AP_W, _AP_S> Tmp(isNeg ? ~(*this) : (*this));
5517
5518 // Figure out how many bits we're using.
5519 uint32_t n = Tmp.getActiveBits();
5520
5521 // The exponent (without bias normalization) is just the
number of bits
5522 // we are using. Note that the sign bit is gone since we
constructed the
5523 // absolute value.
5524 uint64_t exp = n;
5525
5526 // Return infinity for exponent overflow
5527 if (exp > 1023) {
5528     if (!isSigned || !isNeg)
5529         return std::numeric_limits<double>::infinity();
5530     else
5531         return -std::numeric_limits<double>::infinity();
5532 }
5533 exp += 1023; // Increment for 1023 bias
5534
5535 // Number of bits in mantissa is 52. To obtain the man-
tissa value, we must
5536 // extract the high 52 bits from the correct words in
pVal.
5537 uint64_t mantissa;
5538 unsigned hiWord = whichWord(n - 1);
5539 if (hiWord == 0) {
5540     mantissa = Tmp.get_pVal(0);
5541     if (n > 52)
5542         (mantissa) >>= (n - 52); // shift down, we want the
top 52 bits.
5543 } else {
5544     assert(hiWord > 0 && "High word is negative?");
5545     uint64_t hibits = (Tmp.get_pVal(hiWord))
<< (52 - n % APINT_BITS_PER_WORD);
5546     uint64_t lobits =
(Tmp.get_pVal(hiWord - 1)) >> (11 + n %
APINT_BITS_PER_WORD);
5547     mantissa = hibits | lobits;
5548 }
5549
5550 // The leading bit of mantissa is implicit, so get rid of
it.
5551 uint64_t sign = isNeg ? (1ULL << (APINT_BITS_PER_WORD -
1)) : 0;
5552 union {
5553     double __D;
5554     uint64_t __I;
5555 } __T;
5556 __T.__I = sign | ((exp) << 52) | mantissa;
5557 return __T.__D;
5558 }
5559
5560 // @brief Converts this unsigned ap_private to a double
value.
5561 inline double roundToDouble() const { return roundTo-
Double(false); }
5562
5563 // @brief Converts this signed ap_private to a double
value.
5564 inline double signedRoundToDouble() const { return roundTo-
Double(true); }
5565
5566 // The conversion does not do a translation from integer
to double, it just
5567 // re-interprets the bits as a double. Note that it is
valid to do this on
5568 // any bit width. Exactly 64 bits will be translated.
5569 // @brief Converts ap_private bits to a double
5570 inline double bitsToDouble() const {
5571     union {
5572         uint64_t __I;
5573         double __D;
5574     } __T;
5575     __T.__I = pVal[0];
5576     return __T.__D;
5577 }
5578
5579 // The conversion does not do a translation from integer

```

```

to float, it just
5582 // re-interprets the bits as a float. Note that it is val-
id to do this on
5583 // any bit width. Exactly 32 bits will be translated.
5584 // @brief Converts ap_private bits to a double
5585 inline float bitsToFloat() const {
5586     union {
5587         uint32_t __I;
5588         float __F;
5589     } __T;
5590     __T.__I = uint32_t(pVal[0]);
5591     return __T.__F;
5592 }
5593
5594 // The conversion does not do a translation from double to
integer, it just
5595 // re-interprets the bits of the double. Note that it is
valid to do this on
5596 // any bit width but bits from V may get truncated.
5597 // @brief Converts a double to ap_private bits.
5598 inline ap_private& doubleToBits(double __V) {
5599     union {
5600         uint64_t __I;
5601         double __D;
5602     } __T;
5603     __T.__D = __V;
5604     pVal[0] = __T.__I;
5605     return *this;
5606 }
5607
5608 // The conversion does not do a translation from float to
integer, it just
5609 // re-interprets the bits of the float. Note that it is
valid to do this on
5610 // any bit width but bits from V may get truncated.
5611 // @brief Converts a float to ap_private bits.
5612 inline ap_private& floatToBits(float __V) {
5613     union {
5614         uint32_t __I;
5615         float __F;
5616     } __T;
5617     __T.__F = __V;
5618     pVal[0] = __T.__I;
5619 }
5620
5621 // Reduce operation
5622 //-----
5623 inline bool and_reduce() const { return isMaxValue(); }
5624
5625 inline bool nand_reduce() const { return isMinValue(); }
5626
5627 inline bool or_reduce() const { return (bool)countPopula-
tion(); }
5628
5629 inline bool nor_reduce() const { return countPopulation()
== 0; }
5630
5631 inline bool xor_reduce() const {
5632     unsigned int i = countPopulation();
5633     return (i % 2) ? true : false;
5634 }
5635
5636 inline bool xnor_reduce() const {
5637     unsigned int i = countPopulation();
5638     return (i % 2) ? false : true;
5639 }
5640
5641 inline std::string to_string(uint8_t radix = 16, bool sign
= false) const {
5642     return toString(radix, radix == 10 ? _AP_S : sign);
5643 }; // End of class ap_private <_AP_W, _AP_S, false>
5644
5645 namespace ap_private_ops {
5646
5647 enum { APINT_BITS_PER_WORD = 64 };
5648 template <int _AP_W, bool _AP_S>
5649 inline bool operator==(uint64_t V1, const ap_private<_AP_W,
_AP_S>& V2) {
5650     return V2 == V1;
5651 }
5652
5653 template <int _AP_W, bool _AP_S>
5654 inline bool operator!=(uint64_t V1, const ap_private<_AP_W,
_AP_S>& V2) {
5655     return V2 != V1;
5656 }
5657
5658 template <int _AP_W, bool _AP_S, int index>
5659 inline bool get(const ap_private<_AP_W, _AP_S>& a) {
5660     static const uint64_t mask = 1ULL << (index & 0x3f);
5661     return ((mask & a.get_pVal((index) >> 6)) != 0);
5662 }
5663
5664 template <int _AP_W, bool _AP_S, int msb_index, int lsb_in-
dex>
5665 inline void set(ap_private<_AP_W, _AP_S>& a,
5666
5667     const ap_private<AP_MAX(msb_index, 1), true>&

```

```

mark1 = 0,

5667         const ap_private<AP_MAX(lsb_index, 1), true>&
mark2 = 0) {
5668     enum {
5669         APINT_BITS_PER_WORD = 64,
5670         lsb_word = lsb_index / APINT_BITS_PER_WORD,
5671         msb_word = msb_index / APINT_BITS_PER_WORD,
5672         msb = msb_index % APINT_BITS_PER_WORD,
5673         lsb = lsb_index % APINT_BITS_PER_WORD
5674     };
5675     if (msb_word == lsb_word) {
5676         const uint64_t mask = ~0ULL >>

5677         (lsb) << (APINT_BITS_PER_WORD - msb
+ lsb - 1) >>
5678         (APINT_BITS_PER_WORD - msb - 1);
5679         // a.set_pVal(msb_word, a.get_pVal(msb_word) | mask);
5680         a.get_pVal(msb_word) |= mask;
5681     } else {
5682         const uint64_t lsb_mask = ~0ULL >> (lsb) << (lsb);

5683         const uint64_t msb_mask = ~0ULL << (APINT_BITS_PER_WORD -
msb - 1) >>
5684         (APINT_BITS_PER_WORD - msb -
1);
5685         // a.set_pVal(lsb_word, a.get_pVal(lsb_word) | lsb_mask);
5686         a.get_pVal(lsb_word) |= lsb_mask;
5687         for (int i = lsb_word + 1; i < msb_word; i++) {
5688             a.set_pVal(i, ~0ULL);
5689             // a.get_pVal(i)=0;
5690         }
5691         // a.set_pVal(msb_word, a.get_pVal(msb_word) | msb_mask);
5692         a.get_pVal(msb_word) |= msb_mask;
5693     }
5694     a.clearUnusedBits();
5695 }
5696
5697 template<int _AP_W, bool _AP_S, int msb_index, int lsb_in-
dex>
5698 inline void clear(ap_private<_AP_W, _AP_S>& a,
5699                 const ap_private<AP_MAX(msb_index, 1),
true>& mark1 = 0,
5700                 const ap_private<AP_MAX(lsb_index, 1),
true>& mark2 = 0) {
5701     enum {
5702         APINT_BITS_PER_WORD = 64,
5703         lsb_word = lsb_index / APINT_BITS_PER_WORD,
5704         msb_word = msb_index / APINT_BITS_PER_WORD,
5705         msb = msb_index % APINT_BITS_PER_WORD,
5706         lsb = lsb_index % APINT_BITS_PER_WORD
5707     };
5708     if (msb_word == lsb_word) {
5709         const uint64_t mask =
5710             ~0ULL >> (lsb) << (APINT_BITS_PER_WORD - msb + lsb
- 1) >>
5711             (APINT_BITS_PER_WORD - msb - 1));
5712         // a.set_pVal(msb_word, a.get_pVal(msb_word) & mask);
5713         a.get_pVal(msb_word) &= mask;
5714     } else {
5715         const uint64_t lsb_mask = ~0ULL >> (lsb) << (lsb);
5716         const uint64_t msb_mask = ~0ULL << (APINT_BITS_PER_WORD
- msb - 1) >>
5717         (APINT_BITS_PER_WORD - msb -
1);
5718         // a.set_pVal(lsb_word, a.get_pVal(lsb_word) & lsb_mask);
5719         a.get_pVal(lsb_word) &= lsb_mask;
5720         for (int i = lsb_word + 1; i < msb_word; i++) {
5721             // a.set_pVal(i, 0);
5722             a.get_pVal(i) = 0;
5723         }
5724         // a.set_pVal(msb_word, a.get_pVal(msb_word) & msb_mask);
5725         a.get_pVal(msb_word) &= msb_mask;
5726     }
5727     a.clearUnusedBits();
5728 }
5729
5730 template<int _AP_W, bool _AP_S, int index>
5731 inline void set(ap_private<_AP_W, _AP_S>& a,
5732               const ap_private<AP_MAX(index, 1), true>&
mark = 0) {
5733     enum { APINT_BITS_PER_WORD = 64, word = index /
APINT_BITS_PER_WORD };
5734     static const uint64_t mask = 1ULL << (index %
APINT_BITS_PER_WORD);
5735     // a.set_pVal(word, a.get_pVal(word) | mask);
5736     a.get_pVal(word) |= mask;
5737     a.clearUnusedBits();
5738 }
5739
5740 template<int _AP_W, bool _AP_S, int index>
5741 inline void clear(ap_private<_AP_W, _AP_S>& a,
5742                 const ap_private<AP_MAX(index, 1), true>&
mark = 0) {
5743     enum { APINT_BITS_PER_WORD = 64, word = index /
APINT_BITS_PER_WORD };
5744     static const uint64_t mask = ~0ULL << (index %
APINT_BITS_PER_WORD);

```

```

5745     // a.set_pVal(word, a.get_pVal(word) & mask);
5746     a.get_pVal(word) &= mask;
5747     a.clearUnusedBits();
5748 }
5749
5750 } // End of ap_private_ops namespace
5751
5752 template<int _AP_W, bool _AP_S>
5753 inline std::string ap_private<_AP_W, _AP_S, false>::toString(
5754     uint8_t radix, bool wantSigned) const {
5755     assert((radix == 10 || radix == 8 || radix == 16 || radix
== 2) &&
5756         "Radix should be 2, 8, 10, or 16!");
5757     static const char* digits[] = {"0", "1", "2", "3", "4",
"5", "6", "7",
5758         "8", "9", "A", "B", "C",
"D", "E", "F"};
5759     std::string result;
5760     if (radix != 10) {
5761         // For the 2, 8 and 16 bit cases, we can just shift in-
stead of divide
5762         // because the number of bits per digit (1,3 and 4 re-
spectively) divides
5763         // equally. We just shift until there value is zero.
5764         // First, check for a zero value and just short circuit
the logic below.
5765         if (*this == (uint64_t)(0))
5766             result = "0";
5767         else {
5768             ap_private<_AP_W, false> tmp(*this);
5769             size_t insert_at = 0;
5770             bool leading_zero = true;
5771             if (wantSigned && isNegative()) {
5772                 // They want to print the signed version and it is a
negative value
5773                 // Flip the bits and add one to turn it into the
equivalent positive
5774                 // value and put a '-' in the result.
5775                 tmp.flip();
5776                 tmp++;
5777                 tmp.clearUnusedBitsToZero();
5778                 result = "-";
5779                 insert_at = 1;
5780                 leading_zero = false;
5781             }
5782             switch (radix) {
5783                 case 2:
5784                     result += "0b";
5785                     break;
5786                 case 8:
5787                     result += "0o";
5788                     break;
5789                 case 16:
5790                     result += "0x";
5791                     break;
5792                 default:
5793                     assert("invalid radix" && 0);
5794             }
5795             insert_at += 2;
5796             // Just shift tmp right for each digit width until it
becomes zero
5797             uint32_t shift = (radix == 16 ? 4 : (radix == 8 ? 3 :
1));
5798             uint64_t mask = radix - 1;
5799             ap_private<_AP_W, false> zero(0);
5800             unsigned bits = 0;
5801             while (tmp.ne(zero)) {
5802                 uint64_t digit = tmp.get_VAL() & mask;
5803                 result.insert(insert_at, digits[digit]);
5804                 tmp = tmp.lshr(shift);
5805                 ++bits;
5806             }
5807             bits *= shift;
5808             if (bits < _AP_W && leading_zero) res-
ult.insert(insert_at, digits[0]);
5809             return result;
5810         }
5811         ap_private<_AP_W, false> tmp(*this);
5812         ap_private<_AP_W, false> divisor(radix);
5813         ap_private<_AP_W, false> zero(0);
5814         size_t insert_at = 0;
5815         if (wantSigned && isNegative()) {
5816             // They want to print the signed version and it is a neg-
ative value
5817             // Flip the bits and add one to turn it into the equival-
ent positive
5818             // value and put a '-' in the result.
5819             tmp.flip();
5820             tmp++;
5821             tmp.clearUnusedBitsToZero();
5822             result = "-";
5823             insert_at = 1;
5824         }
5825         if (tmp == ap_private<_AP_W, false>(0))
5826             result = "0";
5827         else

```



```

5833     while (tmp.ne(zero)) {
5834         ap_private<_AP_W, false> APdigit(0);
5835         ap_private<_AP_W, false> tmp2(0);
5836         ap_private_ops::divide(tmp, tmp.getNumWords(), divisor,
5837                                divisor.getNumWords(), &tmp2,
&APdigit);
5838         uint64_t digit = APdigit.getZExtValue();
5839         assert(digit < radix && "divide failed");
5840         result.insert(insert_at, digits[digit]);
5841         tmp = tmp2;
5842     }
5843
5844     return result;
5845 } // End of ap_private<_AP_W, _AP_S, false>::toString()
5846
5847 template <int _AP_W, bool _AP_S>
5848 std::ostream &operator<<(std::ostream &os, const
ap_private<_AP_W, _AP_S> &x) {
5849     std::ios_base::fmtflags ff = std::cout.flags();
5850     if (ff & std::cout.hex) {
5851         os << x.toString(16, false); // don't print sign
5852     } else if (ff & std::cout.oct) {
5853         os << x.toString(8, false); // don't print sign
5854     } else {
5855         os << x.toString(10, _AP_S);
5856     }
5857     return os;
5858 }
5859
5860 //
----- //
5861 //             XXX moved here from ap_int_sim.h
XXX //
5862 //
----- //
5863
5864 /// Concatination reference.
5865 /// Proxy class which allows concatenation to be used as
rvalue(for reading) and
5866 lvalue(for writing)
5867 //
-----
5868 // template <int _AP_W1, typename _AP_T1, int _AP_W2, type-
name _AP_T2>
5869 // struct ap_concat_ref {
5870 // #ifdef _MSC_VER
5871 // #pragma warning(disable : 4521 4522)
5872 // #endif
5873 // enum {
5874 //     _AP_WR = _AP_W1 + _AP_W2,
5875 // };
5876 // _AP_T1& mbv1;
5877 // _AP_T2& mbv2;
5878 //
5879 // inline ap_concat_ref(const ap_concat_ref<_AP_W1, _AP_T1,
_AP_W2, _AP_T2>&
5880 // ref)
5881 // : mbv1(ref.mbv1), mbv2(ref.mbv2) {}
5882 //
5883 // inline ap_concat_ref(_AP_T1& bv1, _AP_T2& bv2) :
mbv1(bv1), mbv2(bv2) {}
5884 //
5885 // template <int _AP_W3, bool _AP_S3>
5886 // inline ap_concat_ref& operator=(const ap_private<_AP_W3,
_AP_S3>& val) {
5887 //     ap_private<_AP_W1 + _AP_W2, false> vval(val);
5888 //     int W_ref1 = mbv1.length();
5889 //     int W_ref2 = mbv2.length();
5890 //     ap_private<_AP_W1, false> mask1(-1);
5891 //     mask1 >>= _AP_W1 - W_ref1;
5892 //     ap_private<_AP_W2, false> mask2(-1);
5893 //     mask2 >>= _AP_W2 - W_ref2;
5894 //     mbv1.set(ap_private<_AP_W1, false>((vval >> W_ref2) &
mask1));
5895 //     mbv2.set(ap_private<_AP_W2, false>(vval & mask2));
5896 //     return *this;
5897 // }
5898 //
5899 // inline ap_concat_ref& operator=(unsigned long long val) {
5900 //     ap_private<_AP_W1 + _AP_W2, false> tmpVal(val);
5901 //     return operator=(tmpVal);
5902 // }
5903 //
5904 // template <int _AP_W3, typename _AP_T3, int _AP_W4, type-
name _AP_T4>
5905 // inline ap_concat_ref& operator=(
5906 //     const ap_concat_ref<_AP_W3, _AP_T3, _AP_W4, _AP_T4>&
val) {
5907 //     ap_private<_AP_W1 + _AP_W2, false> tmpVal(val);
5908 //     return operator=(tmpVal);
5909 // }
5910 //
5911 // inline ap_concat_ref& operator=(
5912 //     const ap_concat_ref<_AP_W1, _AP_T1, _AP_W2, _AP_T2>&
val) {
5913 //     ap_private<_AP_W1 + _AP_W2, false> tmpVal(val);
5914 //     return operator=(tmpVal);
5915 // }
5916 //
5917 // template <int _AP_W3, bool _AP_S3>
5918 // inline ap_concat_ref& operator=(const
private_bit_ref<_AP_W3, _AP_S3>&
5919 // val) {
5920 //     ap_private<_AP_W1 + _AP_W2, false> tmpVal(val);
5921 //     return operator=(tmpVal);
5922 // }
5923 //
5924 // template <int _AP_W3, bool _AP_S3>
5925 // inline ap_concat_ref& operator=(const
private_range_ref<_AP_W3, _AP_S3>&
5926 // val) {
5927 //     ap_private<_AP_W1 + _AP_W2, false> tmpVal(val);
5928 //     return operator=(tmpVal);
5929 // }
5930 //
5931 // template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
5932 //     ap_o_mode _AP_O3, int _AP_N3>
5933 // inline ap_concat_ref& operator=(
5934 //     const af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3>& val)
5935 // {
5936 //     return operator=((const ap_private<_AP_W3, false>)
(val));
5937 // }
5938 //
5939 // template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
5940 //     ap_o_mode _AP_O3, int _AP_N3>
5941 // inline ap_concat_ref& operator=(
5942 //     const ap_fixed_base<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3>&
5943 // val) {
5944 //     return operator=(val.to_ap_private());
5945 // }
5946 //
5947 // template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
5948 //     ap_o_mode _AP_O3, int _AP_N3>
5949 // inline ap_concat_ref& operator=(
5950 //     const af_bit_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3>& val) {
5951 //     return operator=((unsigned long long)(bool)(val));
5952 // }
5953 //
5954 // inline operator ap_private<_AP_WR, false>() const { re-
turn get(); }
5955 //
5956 // inline operator unsigned long long() const { return
get().to_uint64(); }
5957 //
5958 // template <int _AP_W3, bool _AP_S3>
5959 // inline ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
private_range_ref<_AP_W3, _AP_S3> >
5960 // operator,(const private_range_ref<_AP_W3, _AP_S3> &a2) {
5961 //     return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
private_range_ref<_AP_W3, _AP_S3> >
5962 //         (*this, a2);
5963 // }
5964 //
5965 // template <int _AP_W3, bool _AP_S3>
5966 // inline
5967 // ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
ap_private<_AP_W3, _AP_S3> >
5968 // operator,(const ap_private<_AP_W3, _AP_S3> &a2) {
5969 //     return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
ap_private<_AP_W3, _AP_S3> >
5970 //         (*this, a2);
5971 // }
5972 //
5973 // template <int _AP_W3, bool _AP_S3>
5974 // inline
5975 // ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
ap_private<_AP_W3, _AP_S3> >
5976 // operator,(const ap_private<_AP_W3, _AP_S3> &a2) {
5977 //     return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3,
ap_private<_AP_W3, _AP_S3> >
5978 //         (*this, const_cast<ap_private<_AP_W3,
_AP_S3>&>(a2));
5979 // }
5980 //
5981 // template <int _AP_W3, bool _AP_S3>
5982 // inline ap_concat_ref<_AP_WR, ap_concat_ref, 1,
private_bit_ref<_AP_W3,
5983 // _AP_S3> >
5984 // operator,(const private_bit_ref<_AP_W3, _AP_S3> &a2) {
5985 //     return ap_concat_ref<_AP_WR, ap_concat_ref, 1,
private_bit_ref<_AP_W3,
5986 // _AP_S3> >
5987 //         (*this, const_cast<private_bit_ref<_AP_W3,
_AP_S3>&>(a2));
5988 // }
5989 //
5990 // template <int _AP_W3, typename _AP_T3, int _AP_W4, type-
name _AP_T4>

```

```

5996 //      INLINE ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3 +
_AP_W4,
5997 //                                ap_concat_ref<_AP_W3, _AP_T3,
_AP_W4, _AP_T4> >
5998 //      operator,(const ap_concat_ref<_AP_W3, _AP_T3, _AP_W4,
_AP_T4> &a2) {
5999 //          return ap_concat_ref<_AP_WR, ap_concat_ref, _AP_W3 +
_AP_W4,
6000 //                                ap_concat_ref<_AP_W3, _AP_T3,
_AP_W4, _AP_T4> >(&
6001 //                  *this, const_cast<ap_concat_ref<_AP_W3, _AP_T3,
_AP_W4,
6002 //                                _AP_T4>&)(a2));
6003 //      }
6004 //  }
6005 //  template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
6006 //              ap_o_mode _AP_O3, int _AP_N3>
6007 //  INLINE ap_concat_ref<
6008 //      _AP_WR, ap_concat_ref, _AP_W3,
6009 //      af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3, _AP_O3,
_AP_N3> >
6010 //  operator,(
6011 //      const af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3> &a2)
6012 //  {
6013 //      return ap_concat_ref<
6014 //          _AP_WR, ap_concat_ref, _AP_W3,
6015 //          af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3, _AP_N3> >(&
6016 //                  *this,
6017 //                  const_cast<
6018 //                      af_range_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3,
_AP_O3,
6019 //                      _AP_N3>&)(a2));
6020 //  }
6021 //  }
6022 //  template <int _AP_W3, int _AP_I3, bool _AP_S3, ap_q_mode
_AP_Q3,
6023 //              ap_o_mode _AP_O3, int _AP_N3>
6024 //  INLINE
6025 //      ap_concat_ref<_AP_WR, ap_concat_ref, 1,
6026 //      af_bit_ref<_AP_W3, _AP_I3, _AP_S3,
_AP_Q3, _AP_O3, _AP_N3>
6027 //      >
6028 //  operator,(const af_bit_ref<_AP_W3, _AP_I3, _AP_S3,
_AP_Q3, _AP_O3,
6029 //      _AP_N3>
6030 //      &a2) {
6031 //      return ap_concat_ref<
6032 //          _AP_WR, ap_concat_ref, 1,
6033 //          af_bit_ref<_AP_W3, _AP_I3, _AP_S3, _AP_Q3, _AP_O3,
_AP_N3> >(&
6034 //          *this,
6035 //          const_cast<af_bit_ref<_AP_W3, _AP_I3, _AP_S3,
_AP_Q3, _AP_O3,
6036 //          _AP_N3>&)(
6037 //              a2));
6038 //  }
6039 //  }
6040 //  template <int _AP_W3, bool _AP_S3>
6041 //  INLINE ap_private<AP_MAX(_AP_WR, _AP_W3), _AP_S3> operat-
or&
6042 //      const ap_private<_AP_W3, _AP_S3>& a2) {
6043 //      return get() & a2;
6044 //  }
6045 //  }
6046 //  template <int _AP_W3, bool _AP_S3>
6047 //  INLINE ap_private<AP_MAX(_AP_WR, _AP_W3), _AP_S3> operat-
or|
6048 //      const ap_private<_AP_W3, _AP_S3>& a2) {
6049 //      return get() | a2;
6050 //  }
6051 //  }
6052 //  template <int _AP_W3, bool _AP_S3>
6053 //  INLINE ap_private<AP_MAX(_AP_WR, _AP_W3), _AP_S3> operat-
or^
6054 //      const ap_private<_AP_W3, _AP_S3>& a2) {
6055 //      return ap_private<AP_MAX(_AP_WR, _AP_W3), _AP_S3>(get()
^ a2);
6056 //  }
6057 //  }
6058 //  INLINE const ap_private<_AP_WR, false> get() const {
6059 //      ap_private<_AP_W1 + _AP_W2, false> tmpVal =
6060 //          ap_private<_AP_W1 + _AP_W2, false>(mbv1.get());
6061 //      ap_private<_AP_W1 + _AP_W2, false> tmpVal2 =
6062 //          ap_private<_AP_W1 + _AP_W2, false>(mbv2.get());
6063 //      int W_ref2 = mbv2.length();
6064 //      tmpVal <= W_ref2;
6065 //      tmpVal |= tmpVal2;
6066 //      return tmpVal;
6067 //  }
6068 //  }
6069 //  INLINE const ap_private<_AP_WR, false> get() {
6070 //      ap_private<_AP_W1 + _AP_W2, false> tmpVal =
6071 //          ap_private<_AP_W1 + _AP_W2, false>(mbv1.get());
6072 //      ap_private<_AP_W1 + _AP_W2, false> tmpVal2 =
6073 //          ap_private<_AP_W1 + _AP_W2, false>(mbv2.get());
6074 //      int W_ref2 = mbv2.length();

```

```

6075 //      tmpVal <= W_ref2;
6076 //      tmpVal |= tmpVal2;
6077 //      return tmpVal;
6078 //  }
6079 //
6080 //  template <int _AP_W3>
6081 //  INLINE void set(const ap_private<_AP_W3, false>& val) {
6082 //      ap_private<_AP_W1 + _AP_W2, false> vval(val);
6083 //      int W_ref1 = mbv1.length();
6084 //      int W_ref2 = mbv2.length();
6085 //      ap_private<_AP_W1, false> mask1(-1);
6086 //      mask1 >>= _AP_W1 - W_ref1;
6087 //      ap_private<_AP_W2, false> mask2(-1);
6088 //      mask2 >>= _AP_W2 - W_ref2;
6089 //      mbv1.set(ap_private<_AP_W1, false>((vval >> W_ref2) &
mask1));
6090 //      mbv2.set(ap_private<_AP_W2, false>(vval & mask2));
6091 //  }
6092 //
6093 //  INLINE int length() const { return mbv1.length() +
mbv2.length(); }
6094 //
6095 //  INLINE std::string to_string(uint8_t radix = 2) const {
6096 //      return get().to_string(radix);
6097 //  }
6098 //}; // struct ap_concat_ref.
6099
6100 /// Range(slice) reference
6101 /// Proxy class, which allows part selection to be used as
rvalue(for reading)
6102 /// and lvalue(for writing)
6103 //-----
6104 template <int _AP_W, bool _AP_S>
6105 struct _private_range_ref {
6106     #ifdef _MSC_VER
6107     #pragma warning(disable : 4521 4522)
6108     #endif
6109     ap_private<_AP_W, _AP_S>& d_bv;
6110     int l_index;
6111     int h_index;
6112
6113     public:
6114     /// copy ctor.
6115     INLINE _private_range_ref(const _private_range_ref<_AP_W,
_AP_S>& ref)
6116         : d_bv(ref.d_bv), l_index(ref.l_index), h_in-
dex(ref.h_index) {}
6117
6118     /// direct ctor.
6119
6120     INLINE _private_range_ref(ap_private<_AP_W, _AP_S>* bv, int
h, int l)
6121         : d_bv(*bv), l_index(l), h_index(h) {
6122             _AP_WARNING(h < 0 || l < 0,
6123                 "Higher bound (%d) and lower bound (%d) can-
not be "
6124                 "negative.",
6125                 h, l);
6126             _AP_WARNING(h >= _AP_W || l >= _AP_W,
6127                 "Higher bound (%d) or lower bound (%d) out of
range (%d).", h, l,
6128                 _AP_W);
6129         }
6130     /// compound or assignment.
6131     template <int _AP_W2, bool _AP_S2>
6132     INLINE _private_range_ref<_AP_W, _AP_S>& operator|=(
6133         const _private_range_ref<_AP_W2, _AP_S2>& ref) {
6134         _AP_WARNING((h_index - l_index) != (ref.h_index -
ref.l_index),
6135             "Bitsize mismatch for ap_private<>.range() &=
"
6136             "ap_private<>.range().");
6137         this->d_bv |= ref.d_bv;
6138         return *this;
6139     }
6140
6141     /// compound or assignment with root type.
6142     template <int _AP_W2, bool _AP_S2>
6143     INLINE _private_range_ref<_AP_W, _AP_S>& operator|=(
6144         const _AP_ROOT_TYPE<_AP_W2, _AP_S2>& ref) {
6145         _AP_WARNING((h_index - l_index + 1) != _AP_W2,
6146             "Bitsize mismatch for ap_private<>.range() |=
_AP_ROOT_TYPE<>.");
6147         this->d_bv |= ref.v;
6148         return *this;
6149     }
6150
6151     /// compound and assignment.
6152     template <int _AP_W2, bool _AP_S2>
6153     INLINE _private_range_ref<_AP_W, _AP_S>& operator&=(
6154         const _private_range_ref<_AP_W2, _AP_S2>& ref) {
6155         _AP_WARNING((h_index - l_index) != (ref.h_index -
ref.l_index),
6156             "Bitsize mismatch for ap_private<>.range() &=
"
6157             "ap_private<>.range().");
6158         this->d_bv &= ref.d_bv;

```

```

6159     return *this;
6160 };
6161
6162 /// compound and assignment with root type.
6163 template <int _AP_W2, bool _AP_S2>
6164 inline _private_range_ref<_AP_W, _AP_S>& operator&=(
6165     const _AP_ROOT_TYPE<_AP_W2, _AP_S2>& ref) {
6166     _AP_WARNING((h_index - l_index + 1) != _AP_W2,
6167         "Bitsize mismatch for ap_private<>.range() &=
6168     _AP_ROOT_TYPE<>.");
6169     this->d_bv &= ref.V;
6170     return *this;
6171 }
6172
6173 /// compound xor assignment.
6174 template <int _AP_W2, bool _AP_S2>
6175 inline _private_range_ref<_AP_W, _AP_S>& operator^=(
6176     const _private_range_ref<_AP_W2, _AP_S2>& ref) {
6177     _AP_WARNING((h_index - l_index) != (ref.h_index -
6178     ref.l_index),
6179         "Bitsize mismatch for ap_private<>.range() ^=
6180     "
6181     "ap_private<>.range().");
6182     this->d_bv ^= ref.d_bv;
6183     return *this;
6184 }
6185
6186 /// compound xor assignment with root type.
6187 template <int _AP_W2, bool _AP_S2>
6188 inline _private_range_ref<_AP_W, _AP_S>& operator^=(
6189     const _AP_ROOT_TYPE<_AP_W2, _AP_S2>& ref) {
6190     _AP_WARNING((h_index - l_index + 1) != _AP_W2,
6191         "Bitsize mismatch for ap_private<>.range() ^=
6192     _AP_ROOT_TYPE<>.");
6193     this->d_bv ^= ref.V;
6194     return *this;
6195 }
6196
6197 /// @name convertors.
6198 // @
6199 inline operator ap_private<_AP_W, false>() const {
6200     ap_private<_AP_W, false> val(0);
6201     if (h_index >= l_index) {
6202         if (_AP_W > 64) {
6203             val = d_bv;
6204             ap_private<_AP_W, false> mask(-1);
6205             mask >>= _AP_W - (h_index - l_index + 1);
6206             val >>= l_index;
6207             val &= mask;
6208         } else {
6209             const static uint64_t mask = (~0ULL >> (64 - _AP_W ?
6210             (64 - _AP_W) : 0));
6211             val = (d_bv >> l_index) & (mask >> (_AP_W - (h_index
6212             - l_index + 1)));
6213         } else {
6214             for (int i = 0, j = l_index; j >= 0 && j >= h_index;
6215             j--, i++)
6216                 if ((d_bv)[j]) val.set(i);
6217         }
6218         return val;
6219     }
6220 }
6221
6222 inline operator unsigned long long() const { return
6223     to_uint64(); }
6224 // @
6225
6226 template <int _AP_W2, bool _AP_S2>
6227 inline _private_range_ref& operator=(const
6228     ap_private<_AP_W2, _AP_S2>& val) {
6229     ap_private<_AP_W, false> vval = ap_private<_AP_W,
6230     false>(val);
6231     if (l_index > h_index) {
6232         for (int i = 0, j = l_index; j >= 0 && j >= h_index;
6233         j--, i++)
6234             (vval)[i] ? d_bv.set(j) : d_bv.clear(j);
6235     } else {
6236         if (_AP_W > 64) {
6237             ap_private<_AP_W, false> mask(-1);
6238             if (l_index > 0) {
6239                 mask <= l_index;
6240                 vval <= l_index;
6241             }
6242             if (h_index < _AP_W - 1) {
6243                 ap_private<_AP_W, false> mask2(-1);
6244                 mask2 >>= _AP_W - h_index - 1;
6245                 mask &= mask2;
6246                 vval &= mask2;
6247             }
6248             mask.flip();
6249             d_bv &= mask;
6250             d_bv |= vval;
6251         } else {
6252             unsigned shift = 64 - _AP_W;
6253             uint64_t mask = ~0ULL >> (shift);
6254             if (l_index > 0) {
6255                 vval = mask & vval << l_index;
6256                 mask = mask & mask << l_index;
6257             }
6258             if (h_index < _AP_W - 1) {

```

```

6248         uint64_t mask2 = mask;
6249         mask2 >>= (_AP_W - h_index - 1);
6250         mask &= mask2;
6251         vval &= mask2;
6252     }
6253     mask = ~mask;
6254     d_bv &= mask;
6255     d_bv |= vval;
6256 }
6257 }
6258 return *this;
6259 } // operator=(const ap_private<>&)
6260
6261 inline _private_range_ref& operator=(unsigned long long
6262     val) {
6263     const ap_private<_AP_W, _AP_S> vval = val;
6264     return operator=(vval);
6265 }
6266
6267 template <int _AP_W2, bool _AP_S2>
6268 inline _private_range_ref& operator=(
6269     const _private_bit_ref<_AP_W2, _AP_S2>& val) {
6270     return operator=((unsigned long long)(bool)val);
6271 }
6272
6273 template <int _AP_W2, bool _AP_S2>
6274 inline _private_range_ref& operator=(
6275     const _private_range_ref<_AP_W2, _AP_S2>& val) {
6276     const ap_private<_AP_W, false> tmpVal(val);
6277     return operator=(tmpVal);
6278 }
6279
6280 // template <int _AP_W3, typename _AP_T3, int _AP_W4, type-
6281 // name _AP_T4>
6282 // inline _private_range_ref& operator=(
6283 //     const ap_concat_ref<_AP_W3, _AP_T3, _AP_W4, _AP_T4>&
6284 //     val) {
6285 //     const ap_private<_AP_W, false> tmpVal(val);
6286 //     return operator=(tmpVal);
6287 // }
6288
6289 // TODO from ap_int_base, ap_bit_ref and ap_range_ref.
6290
6291 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
6292     _AP_Q2,
6293     ap_o_mode _AP_O2, int _AP_N2>
6294 inline _private_range_ref& operator=(
6295     const ap_fixed_base<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
6296     _AP_O2, _AP_N2>& val) {
6297     return operator=(val.to_ap_int_base().V);
6298 }
6299
6300 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
6301     _AP_Q2,
6302     ap_o_mode _AP_O2, int _AP_N2>
6303 inline _private_range_ref& operator=(
6304     const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
6305     _AP_O2, _AP_N2>& val) {
6306     return operator=(val.operator ap_int_base<_AP_W2,
6307     false>().V);
6308 }
6309
6310 template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
6311     _AP_Q2,
6312     ap_o_mode _AP_O2, int _AP_N2>
6313 inline _private_range_ref& operator=(
6314     const af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
6315     _AP_O2, _AP_N2>& val) {
6316     return operator=((unsigned long long)(bool)val);
6317 }
6318
6319 // template <int _AP_W2, bool _AP_S2>
6320 // inline ap_concat_ref<_AP_W, _private_range_ref, _AP_W2,
6321 //     _private_range_ref<_AP_W2, _AP_S2> >
6322 // operator, (const _private_range_ref<_AP_W2, _AP_S2> &a2) {
6323 //     return ap_concat_ref<_AP_W, _private_range_ref, _AP_W2,
6324 //         ap_private<_AP_W2, _AP_S2>
6325 //     >(*this, a2);
6326 // }
6327
6328 // template <int _AP_W2, bool _AP_S2>
6329 // inline ap_concat_ref<_AP_W, _private_range_ref, _AP_W2,
6330 //     ap_private<_AP_W2, _AP_S2> >
6331 // operator, (ap_private<_AP_W2, _AP_S2> &a2) {
6332 //     return ap_concat_ref<_AP_W, _private_range_ref, _AP_W,
6333 //         ap_private<_AP_W, _AP_S> >(*this,
6334 //         a2);
6335 // }

```

```

6332 //
6333 // template <int _AP_W2, bool _AP_S2>
6334 // INLINE ap_concat_ref<_AP_W, _private_range_ref, 1,
6335 // _private_bit_ref<_AP_W2, _AP_S2> >
6336 // operator,(const _private_bit_ref<_AP_W2, _AP_S2> &a2) {
6337 // return ap_concat_ref<_AP_W, _private_range_ref, 1,
6338 // _private_bit_ref<_AP_W2, _AP_S2>
>{
6339 // *this, const_cast<_private_bit_ref<_AP_W2,
6340 // _AP_S2>&(a2));
6341 // }
6342 // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
6343 // INLINE ap_concat_ref<_AP_W, _private_range_ref, _AP_W2 +
6344 // _AP_W3,
6345 // ap_concat_ref<_AP_W2, _AP_T2,
6346 // _AP_W3, _AP_T3> >
6347 // operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
6348 // _AP_T3> &a2) {
6349 // return ap_concat_ref<_AP_W, _private_range_ref, _AP_W2
6350 // + _AP_W3,
6351 // ap_concat_ref<_AP_W2, _AP_T2,
6352 // _AP_W3, _AP_T3> >{
6353 // *this, const_cast<ap_concat_ref<_AP_W2, _AP_T2,
6354 // _AP_W3, _AP_T3>&(a2));
6355 // }
6356 // }
6357 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
6358 // _AP_Q2,
6359 // ap_o_mode _AP_O2, int _AP_N2>
6360 // INLINE ap_concat_ref<
6361 // _AP_W, _private_range_ref, _AP_W2,
6362 // af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
6363 // _AP_N2> >
6364 // operator,(
6365 // const af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
6366 // _AP_O2, _AP_N2> &a2) {
6367 // return ap_concat_ref<
6368 // _AP_W, _private_range_ref, _AP_W2,
6369 // af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
6370 // _AP_O2, _AP_N2> >{
6371 // *this,
6372 // const_cast<
6373 // af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
6374 // _AP_O2, _AP_N2>&(a2));
6375 // }
6376 // }
6377 // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
6378 // _AP_Q2,
6379 // ap_o_mode _AP_O2, int _AP_N2>
6380 // INLINE
6381 // ap_concat_ref<_AP_W, _private_range_ref, 1,
6382 // af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
6383 // _AP_Q2, _AP_O2, _AP_N2> >
6384 // operator,(const af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
6385 // _AP_Q2, _AP_O2, _AP_N2>
6386 // &a2) {
6387 // return ap_concat_ref<
6388 // _AP_W, _private_range_ref, 1,
6389 // af_bit_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
6390 // _AP_N2> >{
6391 // *this,
6392 // const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
6393 // _AP_Q2, _AP_O2, _AP_N2>&(
6394 // a2));
6395 // }
6396 // }
6397 // template <int _AP_W2, bool _AP_S2>
6398 // INLINE bool operator==(const _private_range_ref<_AP_W2,
6399 // _AP_S2>& op2) {
6400 // ap_private<_AP_W, false> lhs = get();
6401 // ap_private<_AP_W2, false> rhs = op2.get();
6402 // return lhs == rhs;
6403 // }
6404 // template <int _AP_W2, bool _AP_S2>
6405 // INLINE bool operator!=(const _private_range_ref<_AP_W2,
6406 // _AP_S2>& op2) {
6407 // ap_private<_AP_W, false> lhs = get();
6408 // ap_private<_AP_W2, false> rhs = op2.get();
6409 // return lhs != rhs;
6410 // }
6411 // template <int _AP_W2, bool _AP_S2>
6412 // INLINE bool operator==(const _private_range_ref<_AP_W2,
6413 // _AP_S2>& op2) {
6414 // ap_private<_AP_W, false> lhs = get();
6415 // ap_private<_AP_W2, false> rhs = op2.get();
6416 // return lhs <= rhs;
6417 // }
6418 // template <int _AP_W2, bool _AP_S2>
6419 // INLINE bool operator<=(const _private_range_ref<_AP_W2,
6420 // _AP_S2>& op2) {
6421 // ap_private<_AP_W, false> lhs = get();
6422 // ap_private<_AP_W2, false> rhs = op2.get();
6423 // return lhs <= rhs;
6424 // }
6425 // template <int _AP_W2>
6426 // INLINE void set(const ap_private<_AP_W2, false>& val) {
6427 // ap_private<_AP_W, _AP_S> vval = val;
6428 // if (l_index > h_index) {
6429 // for (int i = 0, j = l_index; j >= 0 && j >= h_index;
6430 // j--, i++)
6431 // (vval)[i] ? d_bv.set(j) : d_bv.clear(j);
6432 // } else {
6433 // if (_AP_W > 64) {
6434 // ap_private<_AP_W, _AP_S> mask(-1);
6435 // if (l_index > 0) {
6436 // ap_private<_AP_W, false> mask1(-1);
6437 // mask1 >>= _AP_W - l_index;
6438 // mask1.flip();
6439 // mask = mask1;
6440 // vval |= mask1;
6441 // vval <= l_index;
6442 // }
6443 // if (h_index < _AP_W - 1) {
6444 // ap_private<_AP_W, false> mask2(-1);
6445 // mask2 <<= h_index + 1;
6446 // mask2.flip();
6447 // mask &= mask2;
6448 // vval &= mask2;
6449 // }
6450 // mask.flip();
6451 // d_bv &= mask;
6452 // d_bv |= vval;
6453 // } else {
6454 // uint64_t mask = ~0ULL >> (64 - _AP_W);
6455 // if (l_index > 0) {
6456 // uint64_t mask1 = mask;
6457 // mask1 = mask & (mask1 >> (_AP_W - l_index));
6458 // vval = mask & (vval << l_index);
6459 // mask = ~mask1 & mask;
6460 // vval |= mask1;
6461 // }
6462 // if (h_index < _AP_W - 1) {
6463 // uint64_t mask2 = ~0ULL >> (64 - _AP_W);
6464 // mask2 = mask & (mask2 << (h_index + 1));
6465 // mask &= ~mask2;
6466 // vval &= ~mask2;
6467 // }
6468 // d_bv &= (~mask & (~0ULL >> (64 - _AP_W)));
6469 // d_bv |= vval;
6470 // }
6471 // }
6472 // }
6473 // }
6474 // }
6475 // }
6476 // }
6477 // }
6478 // }
6479 // }
6480 // }
6481 // }
6482 // }
6483 // }
6484 // }
6485 // }
6486 // }
6487 // }
6488 // }
6489 // }
6490 // }
6491 // }
6492 // }
6493 // }
6494 // }
6495 // }
6496 // }
6497 // }
6498 // }
6499 // }
6500 // }
6501 // }
6502 // }
6503 // }

```

```

6504     static const uint64_t mask = ~0ULL >> (64 > _AP_W ?
(64 - _AP_W) : 0);
6505     return val &= ((mask) >> (_AP_W - (h_index - l_in-
dex + 1)));
6506     } else {
6507         ap_private<_AP_W, false> mask(-1);
6508         mask >>= _AP_W - (h_index - l_index + 1);
6509         val &= mask;
6510     }
6511 }
6512 }
6513 return val;
6514 }
6515
6516 inline int length() const {
6517     return h_index >= l_index ? h_index - l_index + 1 : l_in-
dex - h_index + 1;
6518 }
6519
6520 inline int to_int() const {
6521     ap_private<_AP_W, false> val = get();
6522     return val.to_int();
6523 }
6524
6525 inline unsigned int to_uint() const {
6526     ap_private<_AP_W, false> val = get();
6527     return val.to_uint();
6528 }
6529
6530 inline long to_long() const {
6531     ap_private<_AP_W, false> val = get();
6532     return val.to_long();
6533 }
6534
6535 inline unsigned long to_ulong() const {
6536     ap_private<_AP_W, false> val = get();
6537     return val.to_ulong();
6538 }
6539
6540 inline ap_slong to_int64() const {
6541     ap_private<_AP_W, false> val = get();
6542     return val.to_int64();
6543 }
6544
6545 inline ap_ulong to_uint64() const {
6546     ap_private<_AP_W, false> val = get();
6547     return val.to_uint64();
6548 }
6549
6550 inline std::string to_string(uint8_t radix = 2) const {
6551     return get().to_string(radix);
6552 }
6553
6554 inline bool and_reduce() {
6555     bool ret = true;
6556     bool reverse = l_index > h_index;
6557     unsigned low = reverse ? h_index : l_index;
6558     unsigned high = reverse ? l_index : h_index;
6559     for (unsigned i = low; i != high; ++i) ret &= d_bv[i];
6560     return ret;
6561 }
6562
6563 inline bool or_reduce() {
6564     bool ret = false;
6565     bool reverse = l_index > h_index;
6566     unsigned low = reverse ? h_index : l_index;
6567     unsigned high = reverse ? l_index : h_index;
6568     for (unsigned i = low; i != high; ++i) ret |= d_bv[i];
6569     return ret;
6570 }
6571
6572 inline bool xor_reduce() {
6573     bool ret = false;
6574     bool reverse = l_index > h_index;
6575     unsigned low = reverse ? h_index : l_index;
6576     unsigned high = reverse ? l_index : h_index;
6577     for (unsigned i = low; i != high; ++i) ret ^= d_bv[i];
6578     return ret;
6579 }
6580 }; // struct _private_range_ref.
6581
6582 /// Bit reference
6583 /// Proxy class, which allows bit selection to be used as
rvalue(for reading)
6584 /// and lvalue(for writing)
6585 //-----
6586 template <int _AP_W, bool _AP_S>
6587 struct _private_bit_ref {
6588     #ifdef _MSC_VER
6589     #pragma warning(disable : 4521 4522)
6590     #endif
6591     ap_private<_AP_W, _AP_S> d_bv;
6592     int d_index;
6593
6594 public:
6595     // copy ctor.
6596     inline _private_bit_ref(const _private_bit_ref<_AP_W,
_AP_S>& ref)

```

```

6597         : d_bv(ref.d_bv), d_index(ref.d_index) {}
6598
6599     // director ctor.
6600     inline _private_bit_ref(ap_private<_AP_W, _AP_S>& bv, int
index = 0)
6601         : d_bv(bv), d_index(index) {
6602         _AP_WARNING(d_index < 0, "Index of bit vector (%d) can-
not be negative.\n",
6603             d_index);
6604         _AP_WARNING(d_index >= _AP_W,
6605             "Index of bit vector (%d) out of range (%d).
\n", d_index, _AP_W);
6606     }
6607
6608     inline operator bool() const { return d_bv.get_bit(d_in-
dex); }
6609
6610     inline bool to_bool() const { return operator bool(); }
6611
6612     template <typename T>
6613     inline _private_bit_ref& operator=(const T& val) {
6614         if (!val)
6615             d_bv.set(d_index);
6616         else
6617             d_bv.clear(d_index);
6618         return *this;
6619     }
6620
6621     // template <int _AP_W2, bool _AP_S2>
6622     // inline ap_concat_ref<1, _private_bit_ref, _AP_W2,
ap_private<_AP_W2,
6623         _AP_S2>>
6624     // operator,(ap_private<_AP_W2, _AP_S2> &a2) const {
6625     //     return ap_concat_ref<1, _private_bit_ref, _AP_W2,
ap_private<_AP_W2,
6626         _AP_S2>> >(
6627         //         const_cast<_private_bit_ref<_AP_W, _AP_S>&(*this),
a2);
6628     // }
6629
6630     // template <int _AP_W2, bool _AP_S2>
6631     // inline ap_concat_ref<1, _private_bit_ref, _AP_W2,
_private_range_ref<_AP_W2,
6632         _AP_S2>>
6633     // operator,(const _private_range_ref<_AP_W2, _AP_S2> &a2)
const {
6634     //     return ap_concat_ref<1, _private_bit_ref, _AP_W2,
_private_range_ref<_AP_W2,
6635         _AP_S2>> >(
6636         //         const_cast<_private_bit_ref<_AP_W, _AP_S>&(*this),
const_cast<_private_range_ref<_AP_W2,
6637             _AP_S2>>(&a2));
6638     // }
6639
6640     // template <int _AP_W2, bool _AP_S2>
6641     // inline ap_concat_ref<1, _private_bit_ref, 1,
_private_bit_ref<_AP_W2,
6642         _AP_S2>> operator,(
6643     //     const _private_bit_ref<_AP_W2, _AP_S2> &a2) const {
6644     //     return ap_concat_ref<1, _private_bit_ref, 1,
_private_bit_ref<_AP_W2, _AP_S2>> >(
6645     //         const_cast<_private_bit_ref<_AP_W, _AP_S>&(*this),
const_cast<_private_bit_ref<_AP_W2, _AP_S2>&(&a2));
6646     // }
6647
6648     // inline ap_concat_ref<1, _private_bit_ref, 1,
_private_bit_ref>
6649     // operator,(
6650     //     const _private_bit_ref &a2) const {
6651     //     return ap_concat_ref<1, _private_bit_ref, 1,
_private_bit_ref> >(
6652     //         const_cast<_private_bit_ref<_AP_W, _AP_S>&(*this),
const_cast<_private_bit_ref<_AP_W2, _AP_S2>&(&a2));
6653     // }
6654
6655     // template <int _AP_W2, typename _AP_T2, int _AP_W3, type-
name _AP_T3>
6656     // inline ap_concat_ref<1, _private_bit_ref, _AP_W2 +
_AP_W3,
6657         _AP_W3, _AP_T3> >
6658     // operator,(const ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
_AP_T3> &a2) const {
6659     //     return ap_concat_ref<1, _private_bit_ref, _AP_W2 +
_AP_W3,
6660         _AP_W3, _AP_T3> >(
6661     //         const_cast<_private_bit_ref<_AP_W, _AP_S>&(*this),
const_cast<ap_concat_ref<_AP_W2, _AP_T2, _AP_W3,
6662             _AP_T3>&(&a2));
6663     // }
6664
6665     // template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
6666         ap_o_mode _AP_O2, int _AP_N2>
6667     // inline ap_concat_ref<
6668     //     1, _private_bit_ref, _AP_W2,
6669     //     af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2, _AP_O2,
_AP_N2> >

```



```

6675 // operator,(const af_range_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2,
6676 // _AP_N2>
6677 //          &a2) const {
6678 //      return ap_concat_ref<
6679 //          1, _private_bit_ref, _AP_W2,
6680 //          af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2, _AP_N2> >(<
6681 //          const_cast<_private_bit_ref<_AP_W, _AP_S>&>(*this),
6682 //          const_cast<
6683 //              af_range_ref<_AP_W2, _AP_I2, _AP_S2, _AP_Q2,
_AP_O2,
6684 //                  _AP_N2>&)(a2));
6685 //  }
6686 //
6687 //  template <int _AP_W2, int _AP_I2, bool _AP_S2, ap_q_mode
_AP_Q2,
6688 //          ap_o_mode _AP_O2, int _AP_N2>
6689 //      INLINE
6690 //          ap_concat_ref<1, _private_bit_ref, 1,
6691 //          af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2,
6692 //              _AP_N2> >
6693 //          operator,(const af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2,
6694 //              _AP_N2>
6695 //                  &a2) const {
6696 //      return ap_concat_ref<1, _private_bit_ref, 1,
af_bit_ref<_AP_W2,
6697 //          _AP_I2, _AP_S2,
6698 //
_AP_Q2, _AP_O2,
6699 //      _AP_N2> >(<
6700 //          const_cast<_private_bit_ref<_AP_W, _AP_S>&>(*this),
6701 //          const_cast<af_bit_ref<_AP_W2, _AP_I2, _AP_S2,
_AP_Q2, _AP_O2,
6702 //              _AP_N2>&)(
6703 //                  a2));
6704 //  }
6705 //
6706 //  template <int _AP_W2, bool _AP_S2>
6707 //      INLINE bool operator==(const _private_bit_ref<_AP_W2,
_AP_S2>& op) const {
6708 //          return get() == op.get();
6709 //      }
6710 //
6711 //  template <int _AP_W2, bool _AP_S2>
6712 //      INLINE bool operator!=(const _private_bit_ref<_AP_W2,
_AP_S2>& op) const {
6713 //          return get() != op.get();
6714 //      }
6715 //
6716 //      INLINE bool get() const { return operator bool(); }
6717 //
6718 //      template <int _AP_W3>
6719 //          INLINE void set(const ap_private<_AP_W3, false>& val) {
6720 //              operator=(val);
6721 //          }
6722 //
6723 //      INLINE bool operator~() const {
6724 //          bool bit = (d_bv)[d_index];
6725 //          return bit ? false : true;
6726 //      }
6727 //
6728 //      INLINE int length() const { return 1; }
6729 //
6730 //      INLINE std::string to_string() const {
6731 //          bool val = get();
6732 //          return val ? "1" : "0";
6733 //      }
6734 //
6735 // }; // struct _private_bit_ref.
6736 //
6737 // char a[100];
6738 // char* ptr = a;
6739 // ap_int<2> n = 3;
6740 // char* ptr2 = ptr + n*2;
6741 // avoid ambiguous errors
6742 #define
OP_BIN_MIX_PTR(BIN_OP)
6743     template <typename PTR_TYPE, int _AP_W, bool
_AP_S>
6744     INLINE PTR_TYPE* operator BIN_OP(PTR_TYPE*
i_op,
6745                                     const ap_private<_AP_W,
_AP_S>& op) { \
6746         typename ap_private<_AP_W, _AP_S>::ValType op2 =
op;
6747         return i_op BIN_OP
op2; \
6748     } \
6749     template <typename PTR_TYPE, int _AP_W, bool
_AP_S>
6750     INLINE PTR_TYPE* operator BIN_OP(const ap_private<_AP_W,
_AP_S>& op, \
6751                                     PTR_TYPE* i_op)
{ \

```

```

6752     typename ap_private<_AP_W, _AP_S>::ValType op2 =
op;
6753     return op2 BIN_OP
i_op;
6754 }
6755
6756 OP_BIN_MIX_PTR(+)
6757 OP_BIN_MIX_PTR(-)
6758 #undef OP_BIN_MIX_PTR
6759
6760 // float OP ap_int
6761 // when ap_int<wa>'s width > 64, then trunc ap_int<w> to
ap_int<64>
6762 #define OP_BIN_MIX_FLOAT(BIN_OP,
C_TYPE)
6763     template <int _AP_W, bool
_AP_S>
6764     INLINE C_TYPE operator BIN_OP(C_TYPE
i_op,
6765                                     const ap_private<_AP_W,
_AP_S>& op) { \
6766         typename ap_private<_AP_W, _AP_S>::ValType op2 =
op;
6767         return i_op BIN_OP
op2;
6768     }
6769     template <int _AP_W, bool
_AP_S>
6770     INLINE C_TYPE operator BIN_OP(const ap_private<_AP_W,
_AP_S>& op, \
6771                                     C_TYPE i_op)
{
6772         typename ap_private<_AP_W, _AP_S>::ValType op2 =
op;
6773         return op2 BIN_OP
i_op;
6774     }
6775
6776 #define OPS_MIX_FLOAT(C_TYPE) \
6777     OP_BIN_MIX_FLOAT(*, C_TYPE) \
6778     OP_BIN_MIX_FLOAT(/, C_TYPE) \
6779     OP_BIN_MIX_FLOAT(+, C_TYPE) \
6780     OP_BIN_MIX_FLOAT(-, C_TYPE)
6781
6782 OPS_MIX_FLOAT(float)
6783 OPS_MIX_FLOAT(double)
6784 #undef OP_BIN_MIX_FLOAT
6785 #undef OPS_MIX_FLOAT
6786
6787 /// Operators mixing Integers with AP_Int
6788 //
-----
6789
6790 // partially specialize template argument _AP_C in order
that:
6791 // for _AP_W > 64, we will explicitly convert operand with
native data type
6792 // into corresponding ap_private
6793 // for _AP_W <= 64, we will implicitly convert operand with
ap_private into
6794 // (unsigned) long long
6795 #define OP_BIN_MIX_INT(BIN_OP, C_TYPE, _AP_WI, _AP_SI,
RTYPE)
6796     template <int _AP_W, bool
_AP_S>
6797     IN-
LINE
\
6798     typename ap_private<_AP_WI, _AP_SI>::template
RTYPE<_AP_W, _AP_S>::RTYPE \
6799     operator BIN_OP(C_TYPE i_op, const ap_private<_AP_W,
_AP_S>& op) { \
6800         return ap_private<_AP_WI, _AP_SI>(i_op).operator
BIN_OP(op);
6801     }
6802
6803     template <int _AP_W, bool
_AP_S>
6804     IN-
LINE
\
6805     typename ap_private<_AP_W, _AP_S>::template
RTYPE<_AP_WI, _AP_SI>::RTYPE \
6806     operator BIN_OP(const ap_private<_AP_W, _AP_S>& op,
C_TYPE i_op) { \
6807         return op.operator BIN_OP(ap_private<_AP_WI,
_AP_SI>(i_op));
6808     }
6809 #define OP_REL_MIX_INT(REL_OP, C_TYPE, _AP_W2,
_AP_S2)
6810     template <int _AP_W, bool
_AP_S>
6811     INLINE bool operator REL_OP(const ap_private<_AP_W, _AP_S>&
op, \
6812                                     C_TYPE op2)

```

```

{
6813     return op.operator REL_OP(ap_private<_AP_W2,
_AP_S2>(op2));
6814 }

6815 template <int _AP_W, bool
_AP_S>
6816     INLINE bool operator REL_OP(C_TYPE
op2,
6817                               const ap_private<_AP_W, _AP_S,
false>& op) { \
6818     return ap_private<_AP_W2, _AP_S2>(op2).operator
REL_OP(op);
6819 }
6820
6821 #define OP_ASSIGN_MIX_INT(ASSIGN_OP, C_TYPE, _AP_W2,
_AP_S2) \
6822     template <int _AP_W, bool
_AP_S>
6823     INLINE ap_private<_AP_W, _AP_S>& operator AS-
SIGN_OP(
6824         ap_private<_AP_W, _AP_S>& op, C_TYPE op2)
{
6825     return op.operator ASSIGN_OP(ap_private<_AP_W2,
_AP_S2>(op2)); \
6826 }
6827
6828 #define OP_BIN_SHIFT_INT(BIN_OP, C_TYPE, _AP_WI, _AP_SI,
RTYPE) \
6829     template <int _AP_W, bool
_AP_S>
6830     C_TYPE operator BIN_OP(C_TYPE
i_op,
6831                             const ap_private<_AP_W, _AP_S,
false>& op) { \
6832     return i_op
BIN_OP(op.get_VAL()); \
6833 }

6834 template <int _AP_W, bool
_AP_S>
6835     IN-
LINE
6836     typename ap_private<_AP_W, _AP_S>::template
RType<_AP_WI, _AP_SI>::RTYPE \
6837     operator BIN_OP(const ap_private<_AP_W, _AP_S>& op,
C_TYPE i_op) { \
6838     return op.operator
BIN_OP(i_op); \
6839 }
6840
6841 #define OP_ASSIGN_RSHIFT_INT(ASSIGN_OP, C_TYPE, _AP_W2,
_AP_S2) \
6842     template <int _AP_W, bool
_AP_S>
6843     INLINE ap_private<_AP_W, _AP_S>& operator AS-
SIGN_OP(
6844         ap_private<_AP_W, _AP_S>& op, C_TYPE op2)
{
6845     op = op.operat-
or>>(op2); \
6846     return
op;
6847 }
6848
6849 #define OP_ASSIGN_LSHIFT_INT(ASSIGN_OP, C_TYPE, _AP_W2,
_AP_S2) \
6850     template <int _AP_W, bool
_AP_S>
6851     INLINE ap_private<_AP_W, _AP_S>& operator AS-
SIGN_OP(
6852         ap_private<_AP_W, _AP_S>& op, C_TYPE op2)
{
6853     op = op.operat-
or<<(op2); \
6854     return
op;
6855 }
6856
6857 #define OPS_MIX_INT(C_TYPE, _AP_W2, _AP_S2) \
6858     OP_BIN_MIX_INT(*, C_TYPE, (_AP_W2), (_AP_S2), mult) \
6859     OP_BIN_MIX_INT(+, C_TYPE, (_AP_W2), (_AP_S2), plus) \
6860     OP_BIN_MIX_INT(-, C_TYPE, (_AP_W2), (_AP_S2), minus) \
6861     OP_BIN_MIX_INT(/, C_TYPE, (_AP_W2), (_AP_S2), div) \
6862     OP_BIN_MIX_INT(% C_TYPE, (_AP_W2), (_AP_S2), mod) \
6863     OP_BIN_MIX_INT(&, C_TYPE, (_AP_W2), (_AP_S2), logic) \
6864     OP_BIN_MIX_INT(|, C_TYPE, (_AP_W2), (_AP_S2), logic) \
6865     OP_BIN_MIX_INT(^, C_TYPE, (_AP_W2), (_AP_S2), logic) \
6866     OP_BIN_SHIFT_INT(>>, C_TYPE, (_AP_W2), (_AP_S2), arg1) \
6867     OP_BIN_SHIFT_INT(<<, C_TYPE, (_AP_W2), (_AP_S2), arg1) \
6868
6869     OP_ASSIGN_MIX_INT(+=, C_TYPE, (_AP_W2), (_AP_S2)) \
6870     OP_ASSIGN_MIX_INT(-=, C_TYPE, (_AP_W2), (_AP_S2)) \
6871     OP_ASSIGN_MIX_INT(*=, C_TYPE, (_AP_W2), (_AP_S2)) \
6872     OP_ASSIGN_MIX_INT(/=, C_TYPE, (_AP_W2), (_AP_S2)) \
6873     OP_ASSIGN_MIX_INT(%=, C_TYPE, (_AP_W2), (_AP_S2)) \
6874     OP_ASSIGN_MIX_INT(&=, C_TYPE, (_AP_W2), (_AP_S2)) \

6875     OP_ASSIGN_MIX_INT(|=, C_TYPE, (_AP_W2), (_AP_S2)) \
6876     OP_ASSIGN_MIX_INT(^=, C_TYPE, (_AP_W2), (_AP_S2)) \
6877     OP_ASSIGN_RSHIFT_INT(>>=, C_TYPE, (_AP_W2), (_AP_S2)) \
6878     OP_ASSIGN_LSHIFT_INT(<<=, C_TYPE, (_AP_W2), (_AP_S2)) \
6879
6880     OP_REL_MIX_INT(>, C_TYPE, (_AP_W2), (_AP_S2)) \
6881     OP_REL_MIX_INT(<, C_TYPE, (_AP_W2), (_AP_S2)) \
6882     OP_REL_MIX_INT(>=, C_TYPE, (_AP_W2), (_AP_S2)) \
6883     OP_REL_MIX_INT(<=, C_TYPE, (_AP_W2), (_AP_S2)) \
6884     OP_REL_MIX_INT(==, C_TYPE, (_AP_W2), (_AP_S2)) \
6885     OP_REL_MIX_INT(!=, C_TYPE, (_AP_W2), (_AP_S2)) \
6886
6887     OPS_MIX_INT(bool, 1, false)
6888     OPS_MIX_INT(char, 8, CHAR_IS_SIGNED)
6889     OPS_MIX_INT(signed char, 8, true)
6890     OPS_MIX_INT(unsigned char, 8, false)
6891     OPS_MIX_INT(short, sizeof(short) * 8, true)
6892     OPS_MIX_INT(unsigned short, sizeof(unsigned short) * 8,
false)
6893     OPS_MIX_INT(int, sizeof(int) * 8, true)
6894     OPS_MIX_INT(unsigned int, sizeof(unsigned int) * 8, false)
6895     OPS_MIX_INT(long, sizeof(long) * 8, true)
6896     OPS_MIX_INT(unsigned long, sizeof(unsigned long) * 8, false)
6897     OPS_MIX_INT(ap_slong, sizeof(ap_slong) * 8, true)
6898     OPS_MIX_INT(ap_ulong, sizeof(ap_ulong) * 8, false)
6899
6900     #undef OP_BIN_MIX_INT
6901     #undef OP_BIN_SHIFT_INT
6902     #undef OP_ASSIGN_MIX_INT
6903     #undef OP_ASSIGN_RSHIFT_INT
6904     #undef OP_ASSIGN_LSHIFT_INT
6905     #undef OP_REL_MIX_INT
6906     #undef OPS_MIX_INT
6907
6908     #define OP_BIN_MIX_RANGE(BIN_OP,
RTYPE) \
6909     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6910     INLINE typename ap_private<_AP_W1, _AP_S1>::template
RType<_AP_W2,
6911
_AP_S2>::RTYPE \
6912     operator BIN_OP(const _private_range_ref<_AP_W1, _AP_S1>&
op1,
6913                     const ap_private<_AP_W2, _AP_S2>& op2)
{
6914     return ap_private<_AP_W1, false>(op1).operator
BIN_OP(op2);
6915 }

6916     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6917     INLINE typename ap_private<_AP_W1, _AP_S1>::template
RType<_AP_W2,
6918
_AP_S2>::RTYPE \
6919     operator BIN_OP(const ap_private<_AP_W1, _AP_S1>&
op1,
6920                     const _private_range_ref<_AP_W2, _AP_S2>&
op2) {
6921     return op1.operator BIN_OP(ap_private<_AP_W2,
false>(op2)); \
6922 }
6923
6924     #define
OP_ASSIGN_MIX_RANGE(ASSIGN_OP) \
6925     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6926     INLINE ap_private<_AP_W1, _AP_S1>& operator AS-
SIGN_OP(
6927         ap_private<_AP_W1, _AP_S1>&
op1,
6928         const _private_range_ref<_AP_W2, _AP_S2>& op2)
{
6929     return op1.operator ASSIGN_OP(ap_private<_AP_W2,
false>(op2)); \
6930 }

6931     template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6932     INLINE _private_range_ref<_AP_W1, _AP_S1>& operator AS-
SIGN_OP(
6933         _private_range_ref<_AP_W1, _AP_S1>&
op1,
6934         ap_private<_AP_W2, _AP_S2>& op2)
{
6935     ap_private<_AP_W1, false>
tmp(op1);
6936     tmp.operator AS-
SIGN_OP(op2);
6937     op1 =
tmp;
6938     return
op1;
6939 }
6940
6941     #define
OP_REL_MIX_RANGE(REL_OP)

```

```

\
6942 template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6943   INLINE bool operator REL_OP(const
_private_range_ref<_AP_W1, _AP_S1>& op1, \
6944   const ap_private<_AP_W2,
_AP_S2>& op2) {
6945   return ap_private<_AP_W1, false>(op1).operator
REL_OP(op2);
6946 }

\
6947 template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6948   INLINE bool operator REL_OP(const ap_private<_AP_W1,
_AP_S1>& op1, \
6949   const
_private_range_ref<_AP_W2, _AP_S2>& op2) { \
6950   return op1.operator REL_OP(op2.operator
ap_private<_AP_W2, false>()); \
6951 }
6952
6953 OP_BIN_MIX_RANGE(+, plus)
6954 OP_BIN_MIX_RANGE(-, minus)
6955 OP_BIN_MIX_RANGE(*, mult)
6956 OP_BIN_MIX_RANGE(/, div)
6957 OP_BIN_MIX_RANGE(% , mod)
6958 OP_BIN_MIX_RANGE(&, logic)
6959 OP_BIN_MIX_RANGE(|, logic)
6960 OP_BIN_MIX_RANGE(^, logic)
6961 OP_BIN_MIX_RANGE(>>, arg1)
6962 OP_BIN_MIX_RANGE(<<, arg1)
6963 #undef OP_BIN_MIX_RANGE
6964
6965 OP_ASSIGN_MIX_RANGE(+=)
6966 OP_ASSIGN_MIX_RANGE(-=)
6967 OP_ASSIGN_MIX_RANGE(*=)
6968 OP_ASSIGN_MIX_RANGE(/=)
6969 OP_ASSIGN_MIX_RANGE(%=)
6970 OP_ASSIGN_MIX_RANGE(&=)
6971 OP_ASSIGN_MIX_RANGE(|=)
6972 OP_ASSIGN_MIX_RANGE(^=)
6973 OP_ASSIGN_MIX_RANGE(>>=)
6974 OP_ASSIGN_MIX_RANGE(<<=)
6975 #undef OP_ASSIGN_MIX_RANGE
6976
6977 OP_REL_MIX_RANGE(>)
6978 OP_REL_MIX_RANGE(<)
6979 OP_REL_MIX_RANGE(>=)
6980 OP_REL_MIX_RANGE(<=)
6981 OP_REL_MIX_RANGE(==)
6982 OP_REL_MIX_RANGE(!=)
6983 #undef OP_REL_MIX_RANGE
6984
6985 #define OP_BIN_MIX_BIT(BIN_OP,
RTYPE)
6986   template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6987   INLINE typename ap_private<1, false>::template
RTYPE<_AP_W2, _AP_S2>::RTYPE \
6988   operator BIN_OP(const _private_bit_ref<_AP_W1, _AP_S1>&
op1, \
6989   const ap_private<_AP_W2, _AP_S2>& op2)
{
6990   return ap_private<1, false>(op1).operator
BIN_OP(op2);
6991 }

\
6992 template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2>
6993   INLINE typename ap_private<_AP_W1, _AP_S1>::template
RTYPE<1, false>::RTYPE \
6994   operator BIN_OP(const ap_private<_AP_W1, _AP_S1>&
op1, \
6995   const _private_bit_ref<_AP_W2, _AP_S2>&
op2) {
6996   return op1.operator BIN_OP(ap_private<1,
false>(op2));
6997 }
6998
6999 #define
OP_ASSIGN_MIX_BIT(ASSIGN_OP) \
7000   template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2> \
7001   INLINE ap_private<_AP_W1, _AP_S1>& operator AS-
SIGN_OP(
\
7002   ap_private<_AP_W1, _AP_S1>&
op1, \
7003   _private_bit_ref<_AP_W2, _AP_S2>& op2)
{
7004   return op1.operator ASSIGN_OP(ap_private<1,
false>(op2)); \
7005 }
\
7006 template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2> \
7007   INLINE _private_bit_ref<_AP_W1, _AP_S1>& operator AS-
SIGN_OP( \

```

```

7008   _private_bit_ref<_AP_W1, _AP_S1>&
op1, \
7009   ap_private<_AP_W2, _AP_S2>& op2)
{
7010   ap_private<1, false>
tmp(op1);
7011   tmp.operator AS-
SIGN_OP(op2);
7012   op1 =
tmp;
7013   return
op1;
7014 }
7015
7016 #define
OP_REL_MIX_BIT(REL_OP) \
7017   template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2> \
7018   INLINE bool operator REL_OP(const _private_bit_ref<_AP_W1,
_AP_S1>& op1, \
7019   const ap_private<_AP_W2,
_AP_S2>& op2) { \
7020   return ap_private<_AP_W1, false>(op1).operator
REL_OP(op2);
7021 }

\
7022 template <int _AP_W1, bool _AP_S1, int _AP_W2, bool
_AP_S2> \
7023   INLINE bool operator REL_OP(const ap_private<_AP_W1,
_AP_S1>& op1, \
7024   const _private_bit_ref<_AP_W2,
_AP_S2>& op2) { \
7025   return op1.operator REL_OP(ap_private<1,
false>(op2)); \
7026 }
7027
7028 OP_ASSIGN_MIX_BIT(+=)
7029 OP_ASSIGN_MIX_BIT(-=)
7030 OP_ASSIGN_MIX_BIT(*=)
7031 OP_ASSIGN_MIX_BIT(/=)
7032 OP_ASSIGN_MIX_BIT(%=)
7033 OP_ASSIGN_MIX_BIT(&=)
7034 OP_ASSIGN_MIX_BIT(|=)
7035 OP_ASSIGN_MIX_BIT(^=)
7036 OP_ASSIGN_MIX_BIT(>>=)
7037 OP_ASSIGN_MIX_BIT(<<=)
7038 #undef OP_ASSIGN_MIX_BIT
7039
7040 OP_BIN_MIX_BIT(+, plus)
7041 OP_BIN_MIX_BIT(-, minus)
7042 OP_BIN_MIX_BIT(*, mult)
7043 OP_BIN_MIX_BIT(/, div)
7044 OP_BIN_MIX_BIT(% , mod)
7045 OP_BIN_MIX_BIT(&, logic)
7046 OP_BIN_MIX_BIT(|, logic)
7047 OP_BIN_MIX_BIT(^, logic)
7048 OP_BIN_MIX_BIT(>>, arg1)
7049 OP_BIN_MIX_BIT(<<, arg1)
7050 #undef OP_BIN_MIX_BIT
7051
7052 OP_REL_MIX_BIT(>)
7053 OP_REL_MIX_BIT(<)
7054 OP_REL_MIX_BIT(<=)
7055 OP_REL_MIX_BIT(>=)
7056 OP_REL_MIX_BIT(==)
7057 OP_REL_MIX_BIT(!=)
7058 #undef OP_REL_MIX_BIT
7059
7060 #define REF_REL_OP_MIX_INT(REL_OP, C_TYPE, _AP_W2,
_AP_S2)
7061   template <int _AP_W, bool
_AP_S> \
7062   INLINE bool operator REL_OP(const _private_range_ref<_AP_W,
_AP_S>& op, \
7063   C_TYPE op2)
{
7064   return (ap_private<_AP_W,
false>(op)) \
7065   .
operator REL_OP(ap_private<_AP_W2,
_AP_S2>(op2));
7066 }
\
7067   template <int _AP_W, bool
_AP_S> \
7068   INLINE bool operator REL_OP(C_TYPE
op2, \
7069   const _private_range_ref<_AP_W,
_AP_S>& op) { \
7070   return ap_private<_AP_W2, _AP_S2>(op2).operator
REL_OP(
\
7071   ap_private<_AP_W,
false>(op)); \

```



```

7073     }
7074     template <int _AP_W, bool
_AP_S>
7075     INLINE bool operator REL_OP(const _private_bit_ref<_AP_W,
_AP_S>& op,
7076                                C_TYPE op2)
{
7077     return (bool(op))REL_OP
op2;
7078 }
7079 template <int _AP_W, bool
_AP_S>
7080     INLINE bool operator REL_OP(C_TYPE
op2,
7081                                const _private_bit_ref<_AP_W,
_AP_S>& op) {
7082     return op2
REL_OP(bool(op));
7083 }
7084
7085 #define REF_REL_MIX_INT(C_TYPE, _AP_W2, _AP_S2) \
7086     REF_REL_OP_MIX_INT(>, C_TYPE, (_AP_W2), (_AP_S2)) \
7087     REF_REL_OP_MIX_INT(<, C_TYPE, (_AP_W2), (_AP_S2)) \
7088     REF_REL_OP_MIX_INT(>=, C_TYPE, (_AP_W2), (_AP_S2)) \
7089     REF_REL_OP_MIX_INT(<=, C_TYPE, (_AP_W2), (_AP_S2)) \
7090     REF_REL_OP_MIX_INT(==, C_TYPE, (_AP_W2), (_AP_S2)) \
7091     REF_REL_OP_MIX_INT(!=, C_TYPE, (_AP_W2), (_AP_S2))
7092
7093 REF_REL_MIX_INT(bool, 1, false)
7094 REF_REL_MIX_INT(char, 8, CHAR_IS_SIGNED)
7095 REF_REL_MIX_INT(signed char, 8, true)
7096 REF_REL_MIX_INT(unsigned char, 8, false)
7097 REF_REL_MIX_INT(short, sizeof(short) * 8, true)
7098 REF_REL_MIX_INT(unsigned short, sizeof(unsigned short) * 8,
false)
7099 REF_REL_MIX_INT(int, sizeof(int) * 8, true)
7100 REF_REL_MIX_INT(unsigned int, sizeof(unsigned int) * 8,
false)
7101 REF_REL_MIX_INT(long, sizeof(long) * 8, true)
7102 REF_REL_MIX_INT(unsigned long, sizeof(unsigned long) * 8,
false)
7103 REF_REL_MIX_INT(ap_slong, sizeof(ap_slong) * 8, true)
7104 REF_REL_MIX_INT(ap_ulong, sizeof(ap_ulong) * 8, false)
7105 #undef REF_REL_OP_MIX_INT
7106 #undef REF_REL_MIX_INT
7107
7108 #define REF_BIN_OP_MIX_INT(BIN_OP, RTYPE, C_TYPE, _AP_W2,
_AP_S2)
7109     template <int _AP_W, bool
_AP_S>
7110     IN-
LINE
7111     typename ap_private<_AP_W, false>::template
RTYPE<_AP_W2, _AP_S2>::RTYPE \
7112     operator BIN_OP(const _private_range_ref<_AP_W, _AP_S>&
op,
7113                     C_TYPE op2)
{
7114     return (ap_private<_AP_W,
false>(op))
7115
7116     operator BIN_OP(ap_private<_AP_W2,
_AP_S2>(op2));
7117 }
7118 template <int _AP_W, bool
_AP_S>
7119     IN-
LINE
7120     typename ap_private<_AP_W2, _AP_S2>::template
RTYPE<_AP_W, false>::RTYPE \
7121     operator BIN_OP(C_TYPE
op2,
7122                     const _private_range_ref<_AP_W, _AP_S>&
op) {
7123     return ap_private<_AP_W2, _AP_S2>(op2).operator
BIN_OP(
7124         ap_private<_AP_W,
false>(op));
7125 }
7126
7127 #define REF_BIN_MIX_INT(C_TYPE, _AP_W2, _AP_S2) \
7128     REF_BIN_OP_MIX_INT(+, plus, C_TYPE, (_AP_W2), (_AP_S2)) \
7129     REF_BIN_OP_MIX_INT(-, minus, C_TYPE, (_AP_W2), (_AP_S2)) \
7130     REF_BIN_OP_MIX_INT(*, mult, C_TYPE, (_AP_W2), (_AP_S2)) \
7131     REF_BIN_OP_MIX_INT(/, div, C_TYPE, (_AP_W2), (_AP_S2)) \
7132     REF_BIN_OP_MIX_INT(% , mod, C_TYPE, (_AP_W2), (_AP_S2)) \
7133     REF_BIN_OP_MIX_INT(&, logic, C_TYPE, (_AP_W2), (_AP_S2)) \
7134     REF_BIN_OP_MIX_INT(|, logic, C_TYPE, (_AP_W2), (_AP_S2))

```

```

7135 REF_BIN_OP_MIX_INT(^, logic, C_TYPE, (_AP_W2), (_AP_S2)) \
7136 REF_BIN_OP_MIX_INT(>, arg1, C_TYPE, (_AP_W2), (_AP_S2)) \
7137 REF_BIN_OP_MIX_INT(<<, arg1, C_TYPE, (_AP_W2), (_AP_S2))
7138
7139 REF_BIN_MIX_INT(bool, 1, false)
7140 REF_BIN_MIX_INT(char, 8, CHAR_IS_SIGNED)
7141 REF_BIN_MIX_INT(signed char, 8, true)
7142 REF_BIN_MIX_INT(unsigned char, 8, false)
7143 REF_BIN_MIX_INT(short, sizeof(short) * 8, true)
7144 REF_BIN_MIX_INT(unsigned short, sizeof(unsigned short) * 8,
false)
7145 REF_BIN_MIX_INT(int, sizeof(int) * 8, true)
7146 REF_BIN_MIX_INT(unsigned int, sizeof(unsigned int) * 8,
false)
7147 REF_BIN_MIX_INT(long, sizeof(long) * 8, true)
7148 REF_BIN_MIX_INT(unsigned long, sizeof(unsigned long) * 8,
false)
7149 REF_BIN_MIX_INT(ap_slong, sizeof(ap_slong) * 8, true)
7150 REF_BIN_MIX_INT(ap_ulong, sizeof(ap_ulong) * 8, false)
7151 #undef REF_BIN_OP_MIX_INT
7152 #undef REF_BIN_MIX_INT
7153
7154 #define REF_BIN_OP(BIN_OP,
RTYPE)
7155     template <int _AP_W, bool _AP_S, int _AP_W2, bool
_AP_S2>
7156     IN-
LINE
7157     \
typename ap_private<_AP_W, false>::template
RTYPE<_AP_W2, false>::RTYPE \
7158     operator BIN_OP(const _private_range_ref<_AP_W, _AP_S>&
lhs,
7159     \
const _private_range_ref<_AP_W2,
_AP_S2>& rhs) {
7160     return ap_private<_AP_W, false>(lhs).operator
BIN_OP(
7161     \
ap_private<_AP_W2,
false>(rhs));
7162     }
7163
7164 REF_BIN_OP(+, plus)
7165 REF_BIN_OP(-, minus)
7166 REF_BIN_OP(*, mult)
7167 REF_BIN_OP(/, div)
7168 REF_BIN_OP(%, mod)
7169 REF_BIN_OP(&, logic)
7170 REF_BIN_OP(|, logic)
7171 REF_BIN_OP(^, logic)
7172 REF_BIN_OP(>>, arg1)
7173 REF_BIN_OP(<<, arg1)
7174 #undef REF_BIN_OP
7175
7176 //
*****
*****
7177 // Implement
7178 // ap_private<M+N> = ap_concat_ref<M> OP ap_con-
cat_ref<N>
7179 // for operators +, -, *, /, %, >>, <<, &, |, ^
7180 // Without these operators the operands are converted to in-
t64 and
7181 // larger results lose informations (higher order bits).
7182 //
7183 //
7184 //
7185 //
7186 //
7187 //
7188 //
7189 //
7190 //
7191 //
7192 //
7193 //
7194 //
7195 //
7196 //
7197 //
7198 //
7199 //
7200 //
7201 //
7202 //
7203 //
7204 //
7205 //
7206 //
7207 //
7208 //
7209 //
7210 //
7211 //
7212 //
7213 //
7214 //
7215 //
7216 //
7217 //
7218 //
7219 //
7220 //
7221 //
7222 //
7223 //
7224 //
7225 //
7226 //
7227 //
7228 //
7229 //
7230 //
7231 //
7232 //
7233 //
7234 //
7235 //
7236 //
7237 //
7238 //
7239 //
7240 //
7241 //
7242 //
7243 //
7244 //
7245 //
7246 //
7247 //
7248 //
7249 //
7250 //
7251 //
7252 //
7253 //
7254 //
7255 //
7256 //
7257 //
7258 //
7259 //
7260 //
7261 //
7262 //
7263 //
7264 //
7265 //
7266 //
7267 //
7268 //
7269 //
7270 //
7271 //
7272 //
7273 //
7274 //
7275 //
7276 //
7277 //
7278 //
7279 //
7280 //
7281 //
7282 //
7283 //
7284 //
7285 //
7286 //
7287 //
7288 //
7289 //
7290 //
7291 //
7292 //
7293 //
7294 //
7295 //
7296 //
7297 //
7298 //
7299 //
7300 //
7301 //
7302 //
7303 //
7304 //
7305 //
7306 //
7307 //
7308 //
7309 //
7310 //
7311 //
7312 //
7313 //
7314 //
7315 //
7316 //
7317 //
7318 //
7319 //
7320 //
7321 //
7322 //
7323 //
7324 //
7325 //
7326 //
7327 //
7328 //
7329 //
7330 //
7331 //
7332 //
7333 //
7334 //
7335 //
7336 //
7337 //
7338 //
7339 //
7340 //
7341 //
7342 //
7343 //
7344 //
7345 //
7346 //
7347 //
7348 //
7349 //
7350 //
7351 //
7352 //
7353 //
7354 //
7355 //
7356 //
7357 //
7358 //
7359 //
7360 //
7361 //
7362 //
7363 //
7364 //
7365 //
7366 //
7367 //
7368 //
7369 //
7370 //
7371 //
7372 //
7373 //
7374 //
7375 //
7376 //
7377 //
7378 //
7379 //
7380 //
7381 //
7382 //
7383 //
7384 //
7385 //
7386 //
7387 //
7388 //
7389 //
7390 //
7391 //
7392 //
7393 //
7394 //
7395 //
7396 //
7397 //
7398 //
7399 //
7400 //
7401 //
7402 //
7403 //
7404 //
7405 //
7406 //
7407 //
7408 //
7409 //
7410 //
7411 //
7412 //
7413 //
7414 //
7415 //
7416 //
7417 //
7418 //
7419 //
7420 //
7421 //
7422 //
7423 //
7424 //
7425 //
7426 //
7427 //
7428 //
7429 //
7430 //
7431 //
7432 //
7433 //
7434 //
7435 //
7436 //
7437 //
7438 //
7439 //
7440 //
7441 //
7442 //
7443 //
7444 //
7445 //
7446 //
7447 //
7448 //
7449 //
7450 //
7451 //
7452 //
7453 //
7454 //
7455 //
7456 //
7457 //
7458 //
7459 //
7460 //
7461 //
7462 //
7463 //
7464 //
7465 //
7466 //
7467 //
7468 //
7469 //
7470 //
7471 //
7472 //
7473 //
7474 //
7475 //
7476 //
7477 //
7478 //
7479 //
7480 //
7481 //
7482 //
7483 //
7484 //
7485 //
7486 //
7487 //
7488 //
7489 //
7490 //
7491 //
7492 //
7493 //
7494 //
7495 //
7496 //
7497 //
7498 //
7499 //
7500 //
7501 //
7502 //
7503 //
7504 //
7505 //
7506 //
7507 //
7508 //
7509 //
7510 //
7511 //
7512 //
7513 //
7514 //
7515 //
7516 //
7517 //
7518 //
7519 //
7520 //
7521 //
7522 //
7523 //
7524 //
7525 //
7526 //
7527 //
7528 //
7529 //
7530 //
7531 //
7532 //
7533 //
7534 //
7535 //
7536 //
7537 //
7538 //
7539 //
7540 //
7541 //
7542 //
7543 //
7544 //
7545 //
7546 //
7547 //
7548 //
7549 //
7550 //
7551 //
7552 //
7553 //
7554 //
7555 //
7556 //
7557 //
7558 //
7559 //
7560 //
7561 //
7562 //
7563 //
7564 //
7565 //
7566 //
7567 //
7568 //
7569 //
7570 //
7571 //
7572 //
7573 //
7574 //
7575 //
7576 //
7577 //
7578 //
7579 //
7580 //
7581 //
7582 //
7583 //
7584 //
7585 //
7586 //
7587 //
7588 //
7589 //
7590 //
7591 //
7592 //
7593 //
7594 //
7595 //
7596 //
7597 //
7598 //
7599 //
7600 //
7601 //
7602 //
7603 //
7604 //
7605 //
7606 //
7607 //
7608 //
7609 //
7610 //
7611 //
7612 //
7613 //
7614 //
7615 //
7616 //
7617 //
7618 //
7619 //
7620 //
7621 //
7622 //
7623 //
7624 //
7625 //
7626 //
7627 //
7628 //
7629 //
7630 //
7631 //
7632 //
7633 //
7634 //
7635 //
7636 //
7637 //
7638 //
7639 //
7640 //
7641 //
7642 //
7643 //
7644 //
7645 //
7646 //
7647 //
7648 //
7649 //
7650 //
7651 //
7652 //
7653 //
7654 //
7655 //
7656 //
7657 //
7658 //
7659 //
7660 //
7661 //
7662 //
7663 //
7664 //
7665 //
7666 //
7667 //
7668 //
7669 //
7670 //
7671 //
7672 //
7673 //
7674 //
7675 //
7676 //
7677 //
7678 //
7679 //
7680 //
7681 //
7682 //
7683 //
7684 //
7685 //
7686 //
7687 //
7688 //
7689 //
7690 //
7691 //
7692 //
7693 //
7694 //
7695 //
7696 //
7697 //
7698 //
7699 //
7700 //
7701 //
7702 //
7703 //
7704 //
7705 //
7706 //
7707 //
7708 //
7709 //
7710 //
7711 //
7712 //
7713 //
7714 //
7715 //
7716 //
7717 //
7718 //
771
```

E.14 vendor/etc/hls all any apfixed.h (102 lines, 1,837 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
  Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da804785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_ALL_ANY_AFFIXED_H
7 #define _X_HLS_ALL_ANY_AFFIXED_H

```

```

8 #include "ap_fixed.h"
9 #include "ap_int.h"
10
11 ///hls_all ap_fixed
12 template <int W_, int I_>
13 bool generic_all(ap_fixed<W_, I_> x)
14 {
15 #pragma HLS pipeline II=1
16 for (int i=0; i<W_;i++)
17     if (x[i]==0)
18         return false;
19     return true;
20 }
21
22 ///hls_all ap_ufixed
23 template <int W_, int I_>
24 bool generic_all(ap_ufixed<W_, I_> x)
25 {
26 #pragma HLS pipeline II=1
27 for (int i=0; i<W_;i++)
28     if (x[i]==0)
29         return false;
30     return true;
31 }
32
33 ///hls_all ap_int
34 template <int I_>
35 bool generic_all(ap_int<I_> x)
36 {
37 #pragma HLS pipeline II=1
38 for (int i=0; i<I_;i++)
39     if (x[i]==0)
40         return false;
41     return true;
42 }
43
44 ///hls_all ap_uint
45 template <int I_>
46 bool generic_all(ap_uint<I_> x)
47 {
48 #pragma HLS pipeline II=1
49 for (int i=0; i<I_;i++)
50     if (x[i]==0)
51         return false;
52     return true;
53 }
54
55 ///hls_any ap_fixed
56 template <int W_, int I_>
57 bool generic_any(ap_fixed<W_, I_> x)
58 {
59 #pragma HLS pipeline II=1
60 for (int i=0; i<W_;i++)
61     if (x[i]==1)
62         return true;
63     return false;
64 }
65
66 ///hls_any ap_ufixed
67 template <int W_, int I_>
68 bool generic_any(ap_ufixed<W_, I_> x)
69 {
70 #pragma HLS pipeline II=1
71 for (int i=0; i<W_;i++)
72     if (x[i]==1)
73         return true;
74     return false;
75 }
76
77 ///hls_any ap_int
78 template <int I_>
79 bool generic_any(ap_int<I_> x)
80 {
81 #pragma HLS pipeline II=1
82 for (int i=0; i<I_;i++)
83     if (x[i]==1)
84         return true;
85     return false;
86 }
87
88 ///hls_any ap_uint
89 template <int I_>
90 bool generic_any(ap_uint<I_> x)
91 {
92 #pragma HLS pipeline II=1
93 for (int i=0; i<I_;i++)
94     if (x[i]==1)
95         return true;
96     return false;
97 }
98
99 #endif
100
101
102

```

E.15 vendor/etc/hls_cbirt_apfixed.h (141 lines, 5,575 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All

```

Rights Reserved.

```

3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 /**
7  * @file hls_cbirt_apfixed.h
8  * @brief Implemenation of cube root for ap_fixed data
type.
9  */
10 #ifndef _X_HLS_CBRT_FIXED_H_
11 #define _X_HLS_CBRT_FIXED_H_
12 #include "ap_fixed.h"
13 #include "etc/hls_round_copysign_apfixed.h"
14 namespace cbirt_internal{
15 template <int W, int I>
16 ap_fixed<W, I> cbirt_fixed(ap_fixed<W, I> x) {
17 #ifndef __SYNTHESIS__
18 #ifdef HLS_DEBUG
19     std::cout<<"W="<<W<<", I="<<I<<std::endl;
20     std::cout<<"x="<<x<<std::endl;
21 #endif
22 #endif
23     const int F = W - I;
24     if(I > 33) return 0;
25     else if(F > 32) return 0;
26
27     ap_fixed<W+1, I+1> xi = x;
28     ap_ufixed<W, I> x_p = fabs_fixed(xi);
29 #ifndef __SYNTHESIS__
30 #ifdef HLS_DEBUG
31     std::cout<<"x_p="<<x_p<<std::endl;
32 #endif
33 #endif
34     ap_ufixed<F+(I+2)/3, (I+2)/3> r = 0;;
35     const static int prcs = (F+1)*3;
36     const static int msbr = (I>0)? (I+2)/3 : 1;//width of
result
37     ap_ufixed<2*msbr+prcs+2, 2*msbr+2> resq = 0;//square of
result;
38     ap_ufixed<msbr+prcs+1, msbr+1> res = 0;//result
39     ap_ufixed<3*msbr+prcs+4, 3*msbr+4> x_rem = x_p;
40     for(int pos = msbr - 1 ; pos >= -F-1; pos--) {
41         ap_ufixed<3*msbr+prcs+2, 3*msbr+2> mul1a = 0;
42         ap_ufixed<3*msbr+prcs+1, 3*msbr+1> mul1b = 0;
43 #ifndef __SYNTHESIS__
44         assert(resq(prcs+2*pos+1,0)==0);
45 #endif
46         //2*resq*bit
47         mul1a(pos+2*msbr+prcs+2, prcs+3*pos+3) =
resq(prcs+2*msbr+1, prcs+2*pos+2);
48         //resq*bit
49         mul1b(pos+2*msbr+prcs+1, prcs+3*pos+2) =
resq(prcs+2*msbr+1, prcs+2*pos+2);
50 #ifndef __SYNTHESIS__
51         assert(res(prcs+pos,0)==0);
52 #endif
53         ap_ufixed<3*msbr+prcs, 3*msbr> mul2a = 0;
54         ap_ufixed<3*msbr+prcs-1, 3*msbr-1> mul2b = 0;
55         //2*res*bit^2;
56         mul2a(pos*2+msbr+prcs+1, pos*3+2+prcs) =
res(msbr+prcs, pos+1+prcs);
57         //res*bit^2
58         mul2b(pos*2+msbr+prcs, pos*3+1+prcs) =
res(msbr+prcs, pos+1+prcs);
59
60         ap_ufixed<3*msbr+prcs+3, 3*msbr+3> mulL = 0;
61         ap_ufixed<3*msbr+prcs+3, 3*msbr+3> mulH = 0;
62         //four adder -> 2 bit carry
63         //only keep(pos*2+msbr+prcs-1, pos*3+1+prcs)
64         mulL(pos*2+msbr+prcs+3, pos*3+1+prcs) =
mul2b(pos*2+msbr+prcs, pos*3+1+prcs) + mul2a(pos*2+msbr+prcs+1,
pos*3+1+prcs) + mul1a(pos*2+msbr+prcs+1, pos*3+1+prcs) +
mul1b(pos*2+msbr+prcs+1, pos*3+1+prcs);
65         //only keep(pos+2*msbr+prcs+2, pos*2+msbr+prcs)
66         mulH(pos+2*msbr+prcs+3, pos*2+msbr+prcs+2) =
mulL(pos*2+msbr+prcs+3, pos*2+msbr+prcs+2) +
mul1a(pos+2*msbr+prcs+2, pos*2+msbr+prcs+2) +
mul1b(pos+2*msbr+prcs+1, pos*2+msbr+prcs+2);
67         mulL[3*pos+prcs] = 1;
68
69 #ifndef __SYNTHESIS__
70 #ifdef HLS_DEBUG
71         std::cout<<"mulH="<<mulH<<std::endl;
72         std::cout<<"mulL="<<mulL<<std::endl;
73 #endif
74 #endif
75         bool cond1 =
(x_rem(pos+2*msbr+prcs+4, pos*2+msbr+prcs+2) >
mulH(pos+2*msbr+prcs+3, pos*2+msbr+prcs+2));
76         bool cond2 =
(x_rem(pos+2*msbr+prcs+4, pos*2+msbr+prcs+2) ==
mulH(pos+2*msbr+prcs+3, pos*2+msbr+prcs+2));
77         bool cond3 = (x_rem(pos*2+msbr+prcs+1, pos*3+prcs)
>= mulL(pos*2+msbr+prcs+1, pos*3+prcs));
78         if(cond1 | (cond2 & cond3)) {
79             ap_ufixed<3*msbr+prcs+3, 3*msbr+3> x_rem_L = 0;
80             x_rem_L(pos*2+msbr+prcs+1, pos*3+prcs) =
x_rem(pos*2+msbr+prcs+1, pos*3+prcs);
81             //carry

```

```

82         x_rem_L[pos*2+msbr+prcs+2] = cond3? 0 : 1;
83         x_rem(pos+2*msbr+prcs+4, pos*2+msbr+prcs+2) =
x_rem(pos+2*msbr+prcs+4, pos*2+msbr+prcs+2) -
mulH(pos+2*msbr+prcs+3, pos*2+msbr+prcs+2) -
x_rem_L(pos*2+msbr+prcs+2, pos*2+msbr+prcs+2);
84         x_rem(pos*2+msbr+prcs+1, pos*3+prcs) =
x_rem_L(pos*2+msbr+prcs+2, pos*3+prcs) -
mulL(pos*2+msbr+prcs+1, pos*3+prcs);
85
86 #ifndef __SYNTHESIS__
87     assert(res(pos+prcs,0)==0);
88     assert(resq(pos*2+1+prcs,0)==0);
89 #endif
90     ap_ufixed<2*msbr+prcs+1, 2*msbr+1> mul1 = 0;
91     mul1(pos+msbr+prcs+1, 2*pos+2+prcs) =
res(msbr+prcs, pos+1+prcs);
92     ap_ufixed<2*msbr+prcs+2, 2*msbr+2> resq_L = 0;
93     resq_L(pos+msbr+prcs+2, 2*pos+2+prcs) =
resq(pos+msbr+prcs+1, 2*pos+2+prcs) +
mul1(pos+msbr+prcs+1, 2*pos+2+prcs);
94     resq(pos+msbr+prcs+1, 2*pos+2+prcs) =
resq_L(pos+msbr+prcs+1, 2*pos+2+prcs);
95     resq(2*msbr+prcs+1, pos+msbr+2+prcs) =
resq_L(2*msbr+prcs+1, pos+msbr+2+prcs) +
resq(2*msbr+prcs+1, pos+msbr+2+prcs);
96     resq_L(pos+msbr+2+prcs, pos+msbr+2+prcs);
97     resq[2*pos+prcs] = 1;
98     res[pos+prcs] = 1;
99 #ifndef __SYNTHESIS__
100     #ifdef HLS_DEBUG
101         std::cout<<"pos="<<pos<<std::endl;
102         std::cout<<"res="<<res.to_string()<<std::endl;
103         std::cout<<"resq="<<resq<<std::endl;
104         std::cout<<"x_rem="<<x_rem<<std::endl;
105     #endif
106 #endif
107 }
108 #ifndef __SYNTHESIS__
109     #ifdef HLS_DEBUG
110         std::cout<<"res="<<res<<std::endl;
111     #endif
112 #endif
113     ap_ufixed<1, -F-1> delta;
114     delta[0] = 1;
115     ap_ufixed<msbr+F+1, msbr> res_s = res;
116     res_s += delta;
117     r = res_s;
118     if(x[W-1]) return -r;
119     else return r;
120 }
121
122 template<int W, int I>
123 ap_ufixed<W, I> cbrt_fixed(ap_ufixed<W, I> x) {
124     ap_fixed<W+1, I+1> xi = x;
125     return cbrt_fixed(xi);
126 }
127 template<int I>
128 ap_int<I> cbrt_fixed(ap_int<I> x) {
129     ap_fixed<I, I> xi = x;
130     return cbrt_fixed(xi);
131 }
132 template<int I>
133 ap_uint<I> cbrt_fixed(ap_uint<I> x) {
134     ap_fixed<I+1, I+1> xi = x;
135     return cbrt_fixed(xi);
136 }
137 }
138 #endif
139
140
141

```

E.16 vendor/etc/hls_comparison_apfixed.h (207 lines, 3,783 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_COMPARISON_APFIXED_H_
7 #define _X_HLS_COMPARISON_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10
11
12 //isgreater(ap_fixed)
13 template<int W, int I>
14 bool isgreater_fixed(ap_fixed<W, I> x1, ap_fixed<W, I> x2){
15     if(x1 > x2) {
16         return true;
17     }
18     else {
19         return false;
20     }
21 }
22 template<int W, int I>
23 bool isgreater_fixed(ap_ufixed<W, I> x1, ap_ufixed<W, I> x2){
24     if(x1 > x2) {

```

```

25         return true;
26     }
27     else {
28         return false;
29     }
30 }
31 template<int I>
32 bool isgreater_fixed(ap_int<I> x1, ap_int<I> x2){
33     if(x1 > x2) {
34         return true;
35     }
36     else {
37         return false;
38     }
39 }
40 template<int I>
41 bool isgreater_fixed(ap_uint<I> x1, ap_uint<I> x2){
42     if(x1 > x2) {
43         return true;
44     }
45     else {
46         return false;
47     }
48 }
49
50 //isgreaterequal(ap_fixed)
51 template<int W, int I>
52 bool isgreaterequal_fixed(ap_fixed<W, I> x1, ap_fixed<W, I>
x2){
53     if(x1 >= x2) {
54         return true;
55     }
56     else {
57         return false;
58     }
59 }
60 template<int W, int I>
61 bool isgreaterequal_fixed(ap_ufixed<W, I> x1, ap_ufixed<W, I>
x2){
62     if(x1 >= x2) {
63         return true;
64     }
65     else {
66         return false;
67     }
68 }
69 template<int I>
70 bool isgreaterequal_fixed(ap_int<I> x1, ap_int<I> x2){
71     if(x1 >= x2) {
72         return true;
73     }
74     else {
75         return false;
76     }
77 }
78 template<int I>
79 bool isgreaterequal_fixed(ap_uint<I> x1, ap_uint<I> x2){
80     if(x1 >= x2) {
81         return true;
82     }
83     else {
84         return false;
85     }
86 }
87
88 //isless(ap_fixed)
89 template<int W, int I>
90 bool isless_fixed(ap_fixed<W, I> x1, ap_fixed<W, I> x2){
91     if(x1 < x2) {
92         return true;
93     }
94     else {
95         return false;
96     }
97 }
98 template<int W, int I>
99 bool isless_fixed(ap_ufixed<W, I> x1, ap_ufixed<W, I> x2){
100     if(x1 < x2) {
101         return true;
102     }
103     else {
104         return false;
105     }
106 }
107 template<int I>
108 bool isless_fixed(ap_int<I> x1, ap_int<I> x2){
109     if(x1 < x2) {
110         return true;
111     }
112     else {
113         return false;
114     }
115 }
116 template<int I>
117 bool isless_fixed(ap_uint<I> x1, ap_uint<I> x2){
118     if(x1 < x2) {
119         return true;
120     }
121     else {
122         return false;

```

```

123     }
124 }
125
126 //islessequal(ap_fixed)
127 template<int W, int I>
128 bool islessequal_fixed(ap_fixed<W,I> x1, ap_fixed<W,I> x2){
129     if(x1 <= x2) {
130         return true;
131     }
132     else {
133         return false;
134     }
135 }
136 template<int W, int I>
137 bool islessequal_fixed(ap_ufixed<W,I> x1, ap_ufixed<W,I> x2)
{
138     if(x1 <= x2) {
139         return true;
140     }
141     else {
142         return false;
143     }
144 }
145 template<int I>
146 bool islessequal_fixed(ap_int<I> x1, ap_int<I> x2){
147     if(x1 <= x2) {
148         return true;
149     }
150     else {
151         return false;
152     }
153 }
154 template<int I>
155 bool islessequal_fixed(ap_uint<I> x1, ap_uint<I> x2){
156     if(x1 <= x2) {
157         return true;
158     }
159     else {
160         return false;
161     }
162 }
163
164 //islessgreater(ap_fixed)
165 template<int W, int I>
166 bool islessgreater_fixed(ap_fixed<W,I> x1, ap_fixed<W,I> x2)
{
167     if((x1 < x2) || (x1 > x2)) {
168         return true;
169     }
170     else {
171         return false;
172     }
173 }
174 template<int W, int I>
175 bool islessgreater_fixed(ap_ufixed<W,I> x1, ap_ufixed<W,I>
x2){
176     if((x1 < x2) || (x1 > x2)) {
177         return true;
178     }
179     else {
180         return false;
181     }
182 }
183 template<int I>
184 bool islessgreater_fixed(ap_int<I> x1, ap_int<I> x2){
185     if((x1 < x2) || (x1 > x2)) {
186         return true;
187     }
188     else {
189         return false;
190     }
191 }
192 template<int I>
193 bool islessgreater_fixed(ap_uint<I> x1, ap_uint<I> x2){
194     if((x1 < x2) || (x1 > x2)) {
195         return true;
196     }
197     else {
198         return false;
199     }
200 }
201
202
203
204
205 #endif
206
207

```

E.17 vendor/etc/hls_cordic_apfixed.h (1,604 lines, 36,667 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6
7 #ifndef _X_HLS_CORDIC_APFIXED_H_

```

```

8 #define _X_HLS_CORDIC_APFIXED_H_
9
10 #if !defined(__SYNTHESIS__) && defined(_MSC_VER)
11 #include <iso646.h>
12 #endif
13
14 #include "ap_fixed.h"
15 #include "ap_int.h"
16 #include "hls_fpo.h"
17 #if !defined(__SYNTHESIS__) && defined(_HLSCLIB_DEBUG_)
18 #include <iostream>
19 #endif
20
21 namespace cordic_apfixed {
22
23     extern const ap_ufixed<128,2> circ_table_arctan_128[128];
24     extern const int hyperb_table_tau_128[128];
25     extern const ap_ufixed<128,4> hy-
perb_table_arctanh_128[128];
26
27     /*
28      * Range reduces input to between 0 and pi/2 by
29      * solving for k and r in x = k*(pi/2) + r
30      */
31     template<int W, int I, int Wo>
32     void circ_range_redux( ap_ufixed<W,I> x,
33                           ap_uint<2> &k,
34                           ap_ufixed<Wo,I> &r) {
35
36         ap_ufixed<Wo+I,0>
inv_pi2("0x0.A2F9836E4E43FC715BBF"); // 2/pi
37         ap_ufixed<Wo+1,1>
pi2("1.5707963267948966192313216916397514420985846996876"); // pi/2
38         ap_ufixed<Wo+I,I> prod = x * inv_pi2;
39         ap_uint<I> kint = prod;
40
41         k = kint;
42         r = x - kint * pi2;
43     };
44
45     /*
46      * Main CORDIC sub-function
47      * MODE 0: 'Rotation': rotate by angle z.  x' <-
K*(x*cos(z) - y*sin(z)), y' <- K*(x*sin(z) + y*cos(z))
48      * MODE 1: 'Vectoring': rotate y to zero.  x' <- sqrt(x^2
+ y^2), z' <- atan(y/x)
49      */
50     template<int W, int I, int MODE>
51     void cordic_circ_apfixed( ap_fixed<W,I> &x,
52                              ap_fixed<W,I> &y,
53                              ap_fixed<W,I> &z) {
54
55         const int Nmax = W;
56
57         ap_fixed<W,I> tx, ty, tz;
58         ap_fixed<W,I> x_s, y_s, z_s;
59         ap_uint<1> d;
60
61         for (int n=0; n<Nmax; n++){
62 #pragma HLS pipeline
63
64             if(MODE == 1) { // Vectoring mode
65                 d = ~y[W-1];
66             }
67             else { // Rotation mode
68                 d = z[W-1];
69             }
70
71             x_s = x >> n;
72             y_s = y >> n;
73             z_s = circ_table_arctan_128[n];
74
75             if(d){ // y[n-1] is positive
76                 tx = x + y_s;
77                 ty = y - x_s;
78                 tz = z + z_s;
79             }
80             else{
81                 tx = x - y_s;
82                 ty = y + x_s;
83                 tz = z - z_s;
84             }
85
86             x = tx;
87             y = ty;
88             z = tz;
89
90 #ifdef _HLSCLIB_DEBUG_
91 #ifndef __SYNTHESIS__
92             std::cout << "    x_s: " << x_s
93                       << "    y_s: " << y_s
94                       << "    z_s: " << z_s
95                       << "    x : " << x
96                       << "    y : " << y
97                       << "    z : " << z << std::endl;
98 #endif
99 #endif
100         }
101     };
102

```

```

103  /*
104  * Main double-CORDIC sub-function
105  * MODE 0: asin
106  * MODE 1: acos
107  */
108  template<int W, int I, int MODE, int INTYPE>
109  void doublecordic_apfixed( ap_fixed<W,I> &x,
110                           ap_fixed<W,I> &y,
111                           ap_fixed<W,I> &z,
112                           ap_fixed<W,I> &t) {
113
114      const int Nmax = W - I + 1;
115      for (int n=0; n<Nmax; n++){
116
117          if(INTYPE==64){
118              #pragma HLS pipeline II=5
119          }
120          else if(INTYPE==32){
121              #pragma HLS pipeline II=3
122          }
123          else if(INTYPE==16){
124              #pragma HLS pipeline II=1
125          }
126          else{
127              #pragma HLS pipeline II=4
128          }
129
130          ap_uint<1> d;
131          if(MODE == 0) { // asin
132              if(y<=t){
133                  d = x[W-1];
134              }
135              else{
136                  d = ~x[W-1];
137              }
138          }
139          else { // acos
140              if(x>=t){
141                  d = y[W-1];
142              }
143              else{
144                  d = ~y[W-1];
145              }
146          }
147
148          ap_fixed<W,I> x_s = x >> (n-1);
149          ap_fixed<W,I> y_s = y >> (n-1);
150          ap_fixed<W,I> x_s2 = x_s >> (n+1);
151          ap_fixed<W,I> y_s2 = y_s >> (n+1);
152
153          x = x - x_s2;
154          y = y - y_s2;
155          if(d){ // d==1, negative
156              x = x + y_s;
157              y = y - x_s;
158          }
159          else{ // d==0, positive
160              x = x - y_s;
161              y = y + x_s;
162          }
163
164          // t
165          ap_fixed<W,I> t_s;
166          int n2 = (n << 1); // 2*n
167          if(n2<W){
168              t_s = (t >> n2);
169              t = t + t_s;
170          }
171
172          // z
173          ap_fixed<W,I> ca = circ_table_arctan_128[n];
174          ap_fixed<W,I> z_s = (ca << 1);
175          ap_fixed<W,I> tz;
176          if(d){ // d==1, negative
177              tz = z - z_s;
178          }
179          else{ // d==0, positive
180              tz = z + z_s;
181          }
182          z = tz;
183
184          #ifdef _HLSCLIB_DEBUG_
185          #ifndef __SYNTHESIS__
186              std::cout << " x : " << x
187                        << " y : " << y
188                        << " z : " << z
189                        << " t : " << t
190                        << std::endl;
191          #endif
192          #endif
193      }
194      #endif
195  }
196  };
197
198  // sin cos
199  template<int W, int I>
200  void generic_sincos(ap_fixed<W,I> in,
201                    ap_fixed<W-I+2,2> & outsin, ap_fixed<W-

```

```

I+2,2> & outcos){
203
204     ap_uint<1> sign0 = (in > 0) ? 1: 0;
205
206     // abs(in)
207     ap_ufixed<W,I> inabs;
208     if(sign0)
209         inabs = in;
210     else
211         inabs = -in;
212
213     // inabs = k * (pi/2) + r
214     const int Fr = (W - I) + I + 1;
215     ap_uint<2> k;
216     ap_ufixed<Fr,1> r;
217     circ_range_redux(inabs, k, r);
218
219     ap_fixed<W+2,3> x, y, z;
220
221     const ap_fixed<W+2,3> K1("0x0.9B74EDA8435E47F277F7");
222     x = K1;
223     y = 0;
224
225     if(k==1 or k==3){
226         ap_ufixed<Fr,1> pi2("0x1.921FB54442D1846");
227         // ap_ufixed<Fr,1>
228         pi2("1.5707963267948966192313216916397514420985846996876");
229         z = pi2 - r;
230     }
231     else{
232         z = r;
233     }
234
235     #ifdef _HLSCLIB_DEBUG_
236     #ifndef __SYNTHESIS__
237         std::cout << "xyz: " << x << " " << y << " " << z <<
238         std::endl;
239     #endif
240     #endif
241
242     cordic_circ_apfixed<W+2,3,0>(x,y,z);
243
244     #ifdef _HLSCLIB_DEBUG_
245     #ifndef __SYNTHESIS__
246         std::cout << "xyz: " << x << " " << y << " " << z <<
247         std::endl;
248     #endif
249     #endif
250
251     if(k==0){
252         outsin = y;
253         outcos = x;
254     }
255     else if(k==1){
256         outsin = y;
257         outcos = -x;
258     }
259     else if(k==2){
260         outsin = -y;
261         outcos = -x;
262     }
263     else{
264         outsin = -y;
265         outcos = x;
266     }
267
268     if(sign0==0)
269         outsin = -outsin;
270
271     };
272
273     // tan
274     template<int W, int I>
275     ap_fixed<W,I> generic_tan(ap_fixed<W,I> x){
276
277         const int Ix = I;
278         const int Fx = (((W-I)>(I-1)) ? (W-I): (I-1)) + I;
279         const int Wx = I + Fx;
280
281         ap_fixed<Wx,Ix> xs = x;
282
283         ap_fixed<Wx-Ix+2,2> outsin;
284         ap_fixed<Wx-Ix+2,2> outcos;
285         generic_sincos(xs, outsin, outcos);
286
287         ap_fixed<W,I> out = outsin / outcos;
288
289         // overflow tan
290         if(I==1){
291             if(out[W-1]==1){
292                 out[W-1] = 0;
293                 for(int i=0;i<W-1;i++){
294                     #pragma HLS UNROLL
295                     out[i] = 1;
296                 }
297             }
298         }
299         return out;
300     };
301 }

```

```

299 // tan(ap_ufixed)
300 template<int W, int I>
301 ap_ufixed<W,I> generic_tan(ap_ufixed<W,I> in) {
302     ap_fixed<W+1,I+1> fin = in;
303     ap_fixed<W+1,I+1> fout = generic_tan(fin);
304     ap_ufixed<W,I> out = 0;
305     if(fout[W]==0){
306         out = fout;
307     }
308     return out;
309 };
310
311 // tan(ap_int)
312 template<int I>
313 ap_int<I> generic_tan(ap_int<I> in) {
314     ap_fixed<I,I> fin = in;
315     ap_fixed<I,I> fout = generic_tan(fin);
316     ap_int<I> out = fout;
317     return out;
318 };
319
320 // tan(ap_uint)
321 template<int I>
322 ap_uint<I> generic_tan(ap_uint<I> in) {
323     ap_fixed<I+1,I+1> fin = in;
324     ap_fixed<I+1,I+1> fout = generic_tan(fin);
325     ap_uint<I> out = 0;
326     if(fout[I]==0){
327         out = fout;
328     }
329     return out;
330 };
331
332 /*
333  * for tanpi
334  */
335
336 /*
337  * Range reduces input to between 0 and pi/2 by
338  * solving for k and r in  $\pi x = k(\pi/2) + r$ 
339  */
340 template<int W, int I, int Wo>
341 void circ_range_redux_pi( ap_ufixed<W,I> x,
342                          ap_uint<2> &k,
343                          ap_ufixed<Wo,1> &r) {
344     //ap_ufixed<(Wo+I),2> v2("2.00000000000000000000");
345     pi2("1.5707963267948966192313216916397514420985846996876");
346     ap_ufixed<W+1,I+1> prod = x << 1; //mul_2;
347     ap_uint<I+1> kint = prod;
348
349     k = kint;
350     r = (prod - kint) * pi2;
351 };
352 // sinpi cospi
353 template<int W, int I>
354 void generic_sincospi(ap_fixed<W,I> in,
355                      ap_fixed<W-I+2,2> &outsin, ap_fixed<W-
356 I+2,2> &outcos){
357     ap_uint<1> sign0 = (in > 0) ? 1: 0;
358
359     // abs(in)
360     ap_ufixed<W,I> inabs;
361     if(sign0)
362         inabs = in;
363     else
364         inabs = -in;
365
366     // inabs = k * (pi/2) + r
367     const int Fr = (W - I) + I + 1;
368     ap_uint<2> k;
369     ap_ufixed<Fr,1> r;
370     circ_range_redux_pi(inabs, k, r);
371
372     ap_fixed<W+2,3> x, y, z;
373
374     const ap_fixed<W+2,3> K1("0x0.9B74EDA8435E47F277F7");
375     x = K1;
376     y = 0;
377
378     if(k==1 or k==3){
379         ap_ufixed<Fr,1> pi2("0x1.921FB54442D1846");
380         z = pi2 - r;
381     }
382     else{
383         z = r;
384     }
385
386 #ifdef _HLSCLIB_DEBUG_
387 #ifndef __SYNTHESIS__
388     std::cout << "xyz: " << x << " " << y << " " << z <<
389 std::endl;
390 #endif
391 #endif
392
393     cordic_circ_apfixed<W+2,3,0>(x,y,z);
394
395 #ifdef _HLSCLIB_DEBUG_
396 #ifndef __SYNTHESIS__

```

```

396     std::cout << "xyz: " << x << " " << y << " " << z <<
397 std::endl;
398 #endif
399
400     if(k==0){
401         outsin = y;
402         outcos = x;
403     }
404     else if(k==1){
405         outsin = y;
406         outcos = -x;
407     }
408     else if(k==2){
409         outsin = -y;
410         outcos = -x;
411     }
412     else{
413         outsin = -y;
414         outcos = x;
415     }
416
417     if(sign0==0)
418         outsin = -outsin;
419 };
420
421 // tanpi
422 template<int W, int I>
423 ap_fixed<W,I> generic_tanpi(ap_fixed<W,I> x){
424
425     const int Ix = I;
426     const int Fx = (((W-I)>(I-1)) ? (W-I): (I-1)) + I;
427     const int Wx = I + Fx;
428
429     ap_fixed<Wx,Ix> xs = x;
430
431     ap_fixed<Wx-Ix+2,2> outsin;
432     ap_fixed<Wx-Ix+2,2> outcos;
433     generic_sincospi(xs, outsin, outcos);
434
435     ap_fixed<W,I> out;
436     ap_int<1> bpoint=0;
437     if(!outcos){
438         bpoint = 1;
439     }else{
440         out = outsin / outcos;
441     }
442     //ap_fixed<W,I,AP_RND,AP_SAT_SYM> out = outsin / outcos;
443
444     // overflow tan
445     // overflow in I ==1 and value< 0
446     if(I==1||bpoint){
447         if( out[W-1]==1 ){//
448             out[W-1] = 0;
449             for(int i=0;i<W-1;i++){
450 #pragma HLS UNROLL
451                 out[i] = 1;
452             }
453         }
454     }
455     // double test_o = out.to_double();
456     return out;
457 };
458
459 // tanpi(ap_ufixed)
460 template<int W, int I>
461 ap_ufixed<W,I> generic_tanpi(ap_ufixed<W,I> in) {
462     ap_fixed<W+1,I+1> fin = in;
463     ap_fixed<W+1,I+1> fout = generic_tanpi(fin);
464     ap_ufixed<W,I> out = 0;
465     if(fout[W]==0){
466         out = fout;
467     }
468     return out;
469 };
470
471 // tanpi(ap_int)
472 template<int I>
473 ap_int<I> generic_tanpi(ap_int<I> in) {
474     _AP_UNUSED_PARAM(in);
475     return ap_int<I>(0);
476 };
477
478 // tanpi(ap_uint)
479 template<int I>
480 ap_uint<I> generic_tanpi(ap_uint<I> in) {
481     _AP_UNUSED_PARAM(in);
482     return ap_uint<I>(0);
483 };
484
485 // atan
486 template<int W, int I>
487 ap_fixed<W,2> generic_atan(ap_fixed<W,I> in){
488
489     ap_uint<1> sign0 = (in > 0) ? 1: 0;
490
491     // abs(in)
492     ap_fixed<W+1,I+1> inabs;
493     if(sign0)
494         inabs = in;

```



```

495     else
496         inabs = -in;
497
498     // sign1
499     ap_uint<1> sign1 = (inabs > 1) ? 1: 0;
500
501     ap_fixed<W+2,3> x, y, z;
502     if(sign1){
503         x = 1;
504         // y = ap_fixed<W+2,3>("0x1.0") / inabs;
505         y = 1.0 / inabs.to_double();
506         z = 0;
507     }
508     else{
509         x = 1;
510         y = inabs;
511         z = 0;
512     }
513     cordic_circ_apfixed<W+2,3,1>(x,y,z);
514
515     ap_fixed<W,2> out;
516     if(sign1){
517         const ap_fixed<W,2> pi2("0x1.921FB54442D1846"); // pi/2
518         out = pi2 - z;
519     }
520     else{
521         out = z;
522     }
523
524     // return
525     if(sign0){
526         return out;
527     }
528     else{
529         return -out;
530     }
531 };
532
533 // atan(ap_ufixed)
534 template<int W, int I>
535 ap_ufixed<W,2> generic_atan(ap_ufixed<W,I> in){
536     ap_fixed<W+1,I+1> fin = in;
537     ap_fixed<W+1,2> fout = generic_atan(fin);
538     ap_ufixed<W,2> out = 0;
539     if(fout[W]==0){
540         out = fout;
541     }
542     return out;
543 };
544
545 // atan(ap_int)
546 template<int I>
547 ap_int<2> generic_atan(ap_int<I> in){
548     if(in<-1)
549         return ap_int<2>(-1);
550     else if(in>1)
551         return ap_int<2>(1);
552     else
553         return ap_int<2>(0);
554 };
555
556 // atan(ap_uint)
557 template<int I>
558 ap_uint<1> generic_atan(ap_uint<I> in){
559     if(in>1)
560         return ap_uint<1>(1);
561     else
562         return ap_uint<1>(0);
563 };
564
565 // atan2
566 template<int W, int I>
567 ap_fixed<W,3> generic_atan2(ap_fixed<W,I> in1,
568                             ap_fixed<W,I> in2){
569
570     const ap_fixed<W+1,3> pi("0x3.243F6A8885A308D3"); // pi
571     const ap_fixed<W+2,3> pi2("0x1.921FB54442D1846"); // pi/
2
572     const ap_fixed<W+1,3> pi4("0x0.C90FDAA22168C23"); // pi/
4
573     const ap_fixed<W+1,3> pi3_("-0x2.5B2F8FE6643A469");// -
pi*(3/4)
574
575     ap_uint<2> signin1 = (in1 > 0) ? 2: (in1==0) ? 1: 0;
576     ap_uint<2> signin2 = (in2 > 0) ? 2: (in2==0) ? 1: 0;
577
578     ap_fixed<W,3> out; // output
579
580     // 1. corner cases
581
582     if(signin1==1 and signin2==2){
583         out = 0;
584         return out;
585     }
586     else if(signin1==1 and signin2==0){
587         out = pi;
588         return out;
589     }
590     else if(signin1==2 and signin2==1){
591         out = pi2;
592
593         return out;
594     }
595     else if(signin1==0 and signin2==1){
596         out = -pi2;
597         return out;
598     }
599     }
600     else if(signin1==2){
601         out = 0;
602         return out;
603     }
604     else if(signin1==1){
605         out = pi3_;
606         return out;
607     }
608     }
609
610     // 2. non-corner cases
611
612     ap_fixed<W+1,I+1> in1abs; // abs(in1)
613     if(signin1==0)
614         in1abs = -in1;
615     else
616         in1abs = in1;
617
618     ap_fixed<W+1,I+1> in2abs; // abs(in2)
619     if(signin2==0)
620         in2abs = -in2;
621     else
622         in2abs = in2;
623
624     ap_fixed<W+1,2> in1abs_sft; // shift
625     ap_fixed<W+1,2> in2abs_sft;
626     for(int i=0;i<W+1;i++){
627         #pragma HLS UNROLL
628         in1abs_sft[i] = in1abs[i];
629         for(int i=0;i<W+1;i++){
630             #pragma HLS UNROLL
631             in2abs_sft[i] = in2abs[i];
632         }
633     }
634
635     ap_fixed<W+7,3> x, y, z;
636     if(in1abs > in2abs){
637         x = in1abs_sft;
638         y = in2abs_sft;
639         z = 0;
640     }
641     else{
642         x = in2abs_sft;
643         y = in1abs_sft;
644         z = 0;
645     }
646
647     cordic_circ_apfixed<W+7,3,1>(x,y,z);
648
649     if(in1abs > in2abs){
650         z = pi2 - z;
651     }
652
653     if(signin2==0 and signin1==2){
654         out = pi - z;
655     }
656     else if(signin2==0 and signin1==0){
657         out = z - pi;
658     }
659     else if(signin2==2 and signin1==0){
660         out = -z;
661     }
662     else{
663         out = z;
664     }
665
666     return out;
667 };
668
669 // atan2(ap_fixed)
670 template<int W, int I>
671 ap_ufixed<W,3> generic_atan2(ap_ufixed<W,I> in1,
672                             ap_ufixed<W,I> in2){
673
674     ap_fixed<W+1,I+1> fin1 = in1;
675     ap_fixed<W+1,I+1> fin2 = in2;
676     ap_fixed<W+1,3> fout = generic_atan2(fin1, fin2);
677     ap_ufixed<W,3> out = 0;
678     if(fout[W]==0){
679         out = fout;
680     }
681     return out;
682 }
683
684 // atan2(ap_int)
685 template<int I>
686 ap_int<3> generic_atan2(ap_int<I> in1,
687                       ap_int<I> in2){
688     ap_fixed<I+1,I+1> fin1 = in1;
689     ap_fixed<I+1,I+1> fin2 = in2;
690     ap_fixed<I+1,3> fout = generic_atan2(fin1, fin2);
691

```

```

692     ap_int<3> out = fout;
693     return out;
694 }
695
696 // atan2(ap_uint)
697 template<int I>
698 ap_uint<2> generic_atan2(ap_uint<I> in1,
699                          ap_uint<I> in2){
700     ap_fixed<I+1,I+1> fin1 = in1;
701     ap_fixed<I+1,I+1> fin2 = in2;
702     ap_fixed<I+1,3> fout = generic_atan2(fin1, fin2);
703     ap_uint<2> out = 0;
704     if(fout[I]==0){
705         out = fout;
706     }
707     return out;
708 }
709
710 /*
711  * Range reduces input to between 0 and ln2 by
712  * solving for k and r in  $x = k \cdot \ln 2 + r$ 
713  */
714 template<int W, int I, int Wo>
715 void hyperb_range_redux( ap_fixed<W,I> x,
716                        ap_uint<I+1> &k,
717                        ap_uint<Wo,0> &r) {
718
719     ap_ufixed<(Wo+I)+1,1> inv_ln2("0x1.71547652b82fe1777d0ff-
da0d23a7d11d6aef551cp0"); // 1.0/log(2)
720     ap_ufixed<Wo,0> ln2("0xb.1721f7d1cf79ab-
c9e3b39803f2f6af40f343267p-4"); // log(2)
721     ap_ufixed<Wo+(I+1),I+1> prod = x * inv_ln2;
722     ap_ufixed<Wo,0> prod_dec = prod;
723     k = prod;
724     r = prod_dec * ln2;
725 };
726
727 /*
728  * Main hyperbolic CORDIC sub-function
729  * MODE 0: 'Rotation': rotate by angle z.  $x' = K \cdot \cosh(z) + y \cdot \sinh(z)$ ,  $y' = x \cdot \sinh(z) + y \cdot \cosh(z)$ 
730  * MODE 1: 'Vectoring': rotate y to zero.  $x' = K \cdot \sqrt{x^2 - y^2}$ ,  $z' = z + \operatorname{atanh}(y/x)$ 
731  */
732 template<int W, int I, int MODE>
733 void cordic_hyperb_apfixed( ap_fixed<W,I> &x,
734                          ap_fixed<W,I> &y,
735                          ap_fixed<W,I> &z) {
736
737     const int Nmax = W + 1;
738
739     ap_fixed<W,I> tx, ty, tz;
740     ap_fixed<W,I> x_s, y_s, z_s;
741     ap_uint<1> d;
742
743     for (int n=0; n<Nmax; n++){
744
745         int k = hyperb_table_tau_128[n];
746
747         if(MODE == 1) { // Vectoring mode
748             d = ~y[W-1];
749         }
750         else { // Rotation mode
751             d = z[W-1];
752         }
753
754         x_s = x >> k;
755         y_s = y >> k;
756         z_s = hyperb_table_arctanh_128[k-1];
757
758         if(d){ // z[n-1] is negative
759             tx = x - y_s;
760             ty = y - x_s;
761             tz = z + z_s;
762         }
763         else{
764             tx = x + y_s;
765             ty = y + x_s;
766             tz = z - z_s;
767         }
768
769         x = tx;
770         y = ty;
771         z = tz;
772
773         // std::cout << " x_s: " << x_s
774         // << " y_s: " << y_s
775         // << " z_s: " << z_s
776         // << " x : " << x
777         // << " y : " << y
778         // << " z : " << z << std::endl;
779     }
780 }
781
782 // Cordic method with range reduce for sinh and cosh
783 template <int W, int I>
784 void sinh_cosh_range_redux_cordic( ap_fixed<W,I> in,
785                                    ap_fixed<W,I> &s_out,

```

```

787                                    ap_fixed<W,I> &c_out){
788
789     ap_uint<1> sign0 = in[W-1];
790
791     ap_ufixed<W,I> inabs;
792     if(sign0 == 0)
793         inabs = in;
794     else
795         inabs = -in;
796
797     const int Fr = (W - I) + I;
798
799     if(in==0){
800         s_out = 0;
801         c_out = 1;
802         return;
803     }
804
805     // inabs = k * ln(2) + r
806     ap_uint<I+1> k;
807     ap_ufixed<Fr,0> r;
808     hyperb_range_redux(inabs, k, r);
809
810     ap_fixed<Fr+2,2> x("1.20749706776307212887772");
811     ap_fixed<Fr+2,2> y(0);
812     ap_fixed<Fr+2,2> z(r);
813
814     // std::cout << x << " " << y << " " << z <<
std::endl;
815
816     // compute x = sinh(r) and y = cosh(r)
817     cordic_hyperb_apfixed<Fr+2,2,0>(x,y,z);
818
819     // std::cout << x << " " << y << " " << z <<
std::endl;
820
821     ap_fixed<Fr+(I+2),I+2> er, e_r;
822     er = x + y; // e^r
823     e_r = x - y; // e^{-r}
824
825     // std::cout << er << " " << e_r << std::endl;
826
827     er <= (k-1); // (e^k * 2^k) / 2
828     e_r >= (k+1); // (e^{-k} * 2^k) / 2
829
830     // std::cout << er << " " << e_r << std::endl;
831
832     ap_fixed<W+1,I+1> s_out_tmp = er - e_r;
833     ap_fixed<W+1,I+1> c_out_tmp = er + e_r;
834
835     // std::cout << "out_tmp: "<< " " << c_out_tmp <<
std::endl;
836
837     // overflow sinh
838     if(s_out_tmp[W-1]==1){
839         s_out_tmp[W-1] = 0;
840         for(int i=0;i<W-1;i++){
841             #pragma HLS UNROLL
842             s_out_tmp[i] = 1;
843         }
844     }
845
846     // return sinh
847     if(sign0 == 0){
848         s_out = s_out_tmp;
849     }
850     else{
851         s_out = -s_out_tmp;
852     }
853
854     // std::cout << c_out << " " << c_out_tmp << std::endl;
855
856     // overflow cosh and return
857     if(c_out_tmp[W-1]==1){
858         c_out[W-1] = 0;
859         for(int i=0;i<W-1;i++){
860             #pragma HLS UNROLL
861             c_out[i] = 1;
862         }
863     }
864     else{
865         c_out = c_out_tmp;
866     }
867 };
868
869     template<int W, int I>
870     void tanh_range_redux_cordic(ap_fixed<W,I> in,
ap_fixed<W,I> &tan_out) {
871         //For detail comment, please take
sinh_range_redux_cordic for example
872         ap_uint<1> sign0 = in[W-1];
873
874         ap_ufixed<W,I> inabs;
875         if(sign0 == 0)
876             inabs = in;
877         else
878             inabs = -in;
879
880         const int Fr = (W - I) + I;
881

```



```

882     if(in==0){
883         tan_out = 0;
884         return;
885     }
886
887     ap_uint<I+1> k;
888     ap_fixed<Fr,0> r;
889     hyperb_range_redux(inabs, k, r);
890
891     ap_fixed<Fr+2,2> x("1.20749706776307212887772");
892     ap_fixed<Fr+2,2> y(0);
893     ap_fixed<Fr+2,2> z(r);
894
895     cordic_hyperb_apfixed<Fr+2,2,0>(x,y,z);
896     ap_fixed<Fr+(I+2),I+2> er, e_r;
897     er = x + y;
898     e_r = x - y;
899     er <= k;
900     e_r >= k;
901     tan_out = (er/(er+e_r))*2 - 1;
902 }
903
904 template<int W, int I>
905 ap_fixed<W,I> generic_tanh(ap_fixed<W,I> in) {
906     ap_fixed<W,I> tan_out;
907     tanh_range_redux_cordic(in, tan_out);
908     return tan_out;
909 }
910
911 template<int W, int I>
912 ap_ufixed<W,I> generic_tanh(ap_ufixed<W,I> in) {
913     ap_fixed<W+1,I+1> fin = in;
914     ap_fixed<W+1,I+1> fout = generic_tanh(fin);
915     ap_ufixed<W,I> tan_out = fout;
916     return tan_out;
917 }
918
919 template<int I>
920 ap_int<I> generic_tanh(ap_int<I> in) {
921     ap_fixed<I,I> fin = in;
922     ap_fixed<I,I> fout = generic_tanh(fin);
923     ap_int<I> tan_out = fout;
924     return tan_out;
925 }
926
927 template<int I>
928 ap_uint<I> generic_tanh(ap_uint<I> in) {
929     ap_fixed<I+1,I+1> fin = in;
930     ap_fixed<I+1,I+1> fout = generic_tanh(fin);
931     ap_uint<I> tan_out = fout;
932     return tan_out;
933 }
934 // sinh(ap_fixed)
935 template<int W, int I>
936 ap_fixed<W,I> generic_sinh(ap_fixed<W,I> in) {
937     ap_fixed<W,I> s_out, c_out;
938     sinh_cosh_range_redux_cordic(in, s_out, c_out);
939     return s_out;
940 }
941
942 // sinh(ap_ufixed)
943 template<int W, int I>
944 ap_ufixed<W,I> generic_sinh(ap_ufixed<W,I> in) {
945     ap_fixed<W+1,I+1> fin = in;
946     ap_fixed<W+1,I+1> fout = generic_sinh(fin);
947     ap_ufixed<W,I> out = 0;
948     if(fout[W]==0){
949         out = fout;
950     }
951     return out;
952 };
953
954 // sinh(ap_int)
955 template<int I>
956 ap_int<I> generic_sinh(ap_int<I> in) {
957     ap_fixed<I,I> fin = in;
958     ap_fixed<I,I> fout = generic_sinh(fin);
959     ap_int<I> out = fout;
960     return out;
961 };
962
963 // sinh(ap_uint)
964 template<int I>
965 ap_uint<I> generic_sinh(ap_uint<I> in) {
966     ap_fixed<I+1,I+1> fin = in;
967     ap_fixed<I+1,I+1> fout = generic_sinh(fin);
968     ap_uint<I> out = 0;
969     if(fout[I]==0){
970         out = fout;
971     }
972     return out;
973 };
974
975 // cosh(ap_fixed)
976 template<int W, int I>
977 ap_fixed<W, I> generic_cosh(ap_fixed<W,I> in) {
978     ap_fixed<W,I> s_out, c_out;
979     sinh_cosh_range_redux_cordic(in, s_out, c_out);
980     return c_out;
981 }
982
983 // cosh(ap_ufixed)
984 template<int W, int I>
985 ap_ufixed<W,I> generic_cosh(ap_ufixed<W,I> in) {
986     ap_fixed<W+1,I+1> fin = in;
987     ap_fixed<W+1,I+1> fout = generic_cosh(fin);
988     ap_ufixed<W,I> out = 0;
989     if(fout[W]==0){
990         out = fout;
991     }
992     return out;
993 };
994
995 // cosh(ap_int)
996 template<int I>
997 ap_int<I> generic_cosh(ap_int<I> in) {
998     ap_fixed<I,I> fin = in;
999     ap_fixed<I,I> fout = generic_cosh(fin);
1000     ap_int<I> out = fout;
1001     return out;
1002 };
1003
1004 // cosh(ap_uint)
1005 template<int I>
1006 ap_uint<I> generic_cosh(ap_uint<I> in) {
1007     ap_fixed<I+1,I+1> fin = in;
1008     ap_fixed<I+1,I+1> fout = generic_cosh(fin);
1009     ap_uint<I> out = 0;
1010     if(fout[I]==0){
1011         out = fout;
1012     }
1013     return out;
1014 };
1015
1016 // asin
1017 template<int W, int I>
1018 ap_fixed<W-I+3,3> generic_asin(ap_fixed<W,I> in){
1019     const int I0 = 3;
1020     const int W0 = W - I + I0 + 6;
1021
1022     const ap_fixed<W+1,I+1> cst1("0x1");
1023     const ap_fixed<W,I> cst0("0x0");
1024     const ap_fixed<W,I> cst_1("-0x1");
1025
1026     // input value should be between -1 and 1
1027 #ifndef __SYNTHESIS__
1028     if(I>=2){
1029         if((in>cst1) or (in<cst_1)){
1030             printf("Error: input value %f (for asin) is not
1031 between -1 and 1.\n", in.to_double());
1032             assert(0);
1033         }
1034     }
1035 #endif
1036
1037     ap_fixed<W-I+3,3> out; // output
1038
1039     // special case
1040     if(in==cst1){
1041         out = ap_fixed<W-I+3,3>("0x1.921FB54442D1846");
1042         return out;
1043     }
1044     else if(in==cst_1){
1045         out = ap_fixed<W-I+3,3>("-0x1.921FB54442D1846");
1046         return out;
1047     }
1048     else if(in==cst0){
1049         out = ap_fixed<W-I+3,3>("0x0");
1050         return out;
1051     }
1052 }
1053
1054 // sign
1055 ap_uint<1> sign0 = (in >= 0) ? 1: 0;
1056
1057 // abs(in)
1058 ap_fixed<W,I> inabs;
1059 if(sign0)
1060     inabs = in;
1061 else
1062     inabs = -in;
1063
1064 ap_fixed<W,I> absout;
1065
1066 // double cordic algorithm
1067 if(inabs[W-I-1]==0){
1068     ap_fixed<W0,I0> x("0x1");
1069     ap_fixed<W0,I0> y("0x0");
1070     ap_fixed<W0,I0> z("0x0");
1071     ap_fixed<W0,I0> t = inabs;
1072     doublecordic_apfixed<W0,I0,0,0>(x,y,z,t);
1073     absout = z;
1074 }
1075 else{
1076     const int F1 = W - I;
1077     const int W1 = F1 + F1 + I;
1078     ap_fixed<W1,I> one("0x1");
1079     ap_fixed<W1,I> onex = one - inabs * inabs;
1080     ap_fixed<W1,I> sqrtonex

```

```

= ::HLS_FP0_SQRT(onex.to_double());
1081 ap_fixed<W1,I> xsqrtonex = inabs / (one + sqrtonex);
1082 absout = generic_atan(xsqrtonex) << 1;
1083 }
1084 // return
1085 if(sign0)
1086 out = absout;
1087 else
1088 out = -absout;
1089 return out;
1090 }
1091 // acos
1092 template<int W, int I>
1093 ap_fixed<W-I+3,3> generic_acos(ap_fixed<W,I> in){
1094
1095     const int I0 = 3;
1096     const int W0 = W - I + I0;
1097
1098     // input value should be between -1 and 1
1099 #ifndef __SYNTHESIS__
1100     if(I>=2){
1101         const ap_fixed<W+1,I+1> cst1("0x1");
1102         const ap_fixed<W,I> cst1("-0x1");
1103
1104         if((in>cst1) or (in<cst1_1)){
1105             printf("Error: input value %f (for acos) is not
1106 between -1 and 1.\n", in.to_double());
1107             assert(0);
1108         }
1109     }
1110 #endif
1111 // acos = pi/2 - asin
1112 const ap_fixed<W0,I0> pi2("0x1.921FB54442D1846"); // pi/2
1113 ap_fixed<W0,I0> out = pi2 - generic_asin(in);
1114
1115 if(out[W0-1]==1){
1116     out = -out;
1117 }
1118 return out;
1119 }
1120
1121 template<int W, int I>
1122 void cordic_semi_recovery_asinh(ap_fixed<W,I> in_top,
1123 ap_fixed<W,I>& out) {
1124
1125     if(in_top == 0) {
1126         out = 0;
1127         return;
1128     }
1129     else {
1130         in = -in_top;
1131     }
1132
1133     int m = in.countLeadingZeros();
1134     int k = I-m;
1135
1136     if(W-I-2 >= k) {
1137         ap_fixed<W,I> boundk = 1.0;
1138         ap_fixed<W,I> bound_k_2 = 1.0;
1139         boundk <<= k;
1140         bound_k_2 >>= k;
1141         bound_k_2 >>= 2;
1142
1143         if(in >= (boundk - bound_k_2)) {
1144             k+=1;
1145         }
1146     }
1147
1148     if(k < 0) {
1149         k = 0;
1150     }
1151
1152     const int Nmax = W - I + 1;
1153     int tau;
1154     ap_fixed<W,I> x_r(1);
1155     ap_fixed<W,I> y_r(0);
1156     ap_fixed<W,I> z_r(0);
1157     ap_fixed<W,I> x1_tau, x2_tau, y1_tau, y2_tau, z_s;
1158     ap_fixed<W,I> xk, x_k, yk, y_k;
1159     ap_fixed<W,I> y_semi_recovery;
1160     ap_fixed<W,I> in_s;
1161
1162     for(int i=0; i<Nmax; i++) {
1163
1164         xk = x_r << k;
1165         x_k = x_r >> k;
1166         yk = y_r << k;
1167         y_k = y_r >> k;
1168
1169         y_semi_recovery = (xk - x_k + yk + y_k) >> 1;
1170
1171         tau = hyperb_table_tau_128[i+1];
1172         z_s = hyperb_table_arctanh_128[tau-1] << 1;
1173
1174         x1_tau = ((x_r >> tau) << 1);
1175         x2_tau = (x_r >> (tau << 1));
1176         y1_tau = ((y_r >> tau) << 1);
1177         y2_tau = (y_r >> (tau << 1));
1178
1179         if(y_semi_recovery <= in) {
1180             x_r += x2_tau + y1_tau;
1181             y_r += y2_tau + x1_tau;
1182             z_r += z_s;
1183         }
1184         else {
1185             x_r += x2_tau - y1_tau;
1186             y_r += y2_tau - x1_tau;
1187             z_r -= z_s;
1188         }
1189
1190         in_s = in >> (tau << 1);
1191         in -= in_s;
1192     }
1193
1194     ap_ufixed<W,0> ln2("0xb.17217f7d1cf79ab-
1195 c9e3b39803f2f6af40f343267p-4");
1196     if(in_top >= 0) {
1197         out = z_r + k*ln2;
1198     }
1199     else {
1200         out = -z_r - k*ln2;
1201     }
1202 }
1203
1204 template<int W, int I>
1205 void cordic_semi_recovery_acosh(ap_fixed<W,I> in,
1206 ap_fixed<W,I>& out) {
1207
1208     #ifndef __SYNTHESIS__
1209     if(in < 1) {
1210         std::cout<<"Input should be equal or greater than
1211 1.0."<<std::endl;
1212     }
1213     #endif
1214
1215     if(in == 1) {
1216         out = 0;
1217         return;
1218     }
1219
1220     int m = in.countLeadingZeros();
1221     int k = I-m;
1222
1223     ap_fixed<W,I> boundk = 1.0;
1224     ap_fixed<W,I> bound_k_1 = 1.0;
1225     boundk <<= k;
1226     boundk >>= 1;
1227     bound_k_1 >>= k;
1228     bound_k_1 >>= 1;
1229
1230     if(in < (boundk - bound_k_1)) {
1231         k-=1;
1232     }
1233
1234     if(k < 0) {
1235         k = 0;
1236     }
1237
1238     const int Nmax = W - I + 1;
1239     int tau;
1240     ap_fixed<W,I> x_r(1);
1241     ap_fixed<W,I> y_r(0);
1242     ap_fixed<W,I> z_r(0);
1243     ap_fixed<W,I> x1_tau, x2_tau, y1_tau, y2_tau, z_s;
1244     ap_fixed<W,I> xk, x_k, yk, y_k;
1245     ap_fixed<W,I> x_semi_recovery;
1246     ap_fixed<W,I> in_s;
1247
1248     for(int i=0; i<Nmax; i++) {
1249
1250         xk = x_r << k;
1251         x_k = x_r >> k;
1252         yk = y_r << k;
1253         y_k = y_r >> k;
1254
1255         x_semi_recovery = (xk + x_k + yk - y_k) >> 1;
1256
1257         tau = hyperb_table_tau_128[i+1];
1258         z_s = hyperb_table_arctanh_128[tau-1] << 1;
1259
1260         x1_tau = ((x_r >> tau) << 1);
1261         x2_tau = (x_r >> (tau << 1));
1262         y1_tau = ((y_r >> tau) << 1);
1263         y2_tau = (y_r >> (tau << 1));
1264
1265         if(x_semi_recovery <= in) {
1266             x_r += x2_tau + y1_tau;
1267             y_r += y2_tau + x1_tau;
1268             z_r += z_s;
1269         }
1270         else {
1271             x_r += x2_tau - y1_tau;
1272             y_r += y2_tau - x1_tau;
1273             z_r -= z_s;
1274         }
1275
1276         in_s = in >> (tau << 1);
1277         in -= in_s;
1278     }

```

```

1275     y_r += y_2tau + x1_tau;
1276     z_r += z_s;
1277 }
1278 else {
1279     x_r += x_2tau - y1_tau;
1280     y_r += y_2tau - x1_tau;
1281     z_r -= z_s;
1282 }
1283
1284     in_s = in >> (tau << 1);
1285     in -= in_s;
1286 }
1287
1288     ap_ufixed<W,0> ln2("0xb.17217f7d1cf79ab-
c9e3b39803f2f6af40f343267p-4");
1289     out = z_r + k*ln2;
1290 }
1291
1292 // asinh(ap_fixed)
1293 template<int W, int I>
1294 ap_fixed<W,I> generic_asinh(ap_fixed<W,I> in) {
1295     ap_fixed<W+1,I+1> input, output; input = in;
1296     cordic_semi_recovery_asinh(input, output);
1297     return output;
1298 }
1299
1300 // asinh(ap_ufixed)
1301 template<int W, int I>
1302 ap_ufixed<W,I> generic_asinh(ap_ufixed<W,I> in) {
1303     ap_fixed<W+1,I+1> input, output;
1304     input = in;
1305     cordic_semi_recovery_asinh(input, output);
1306     return output;
1307 };
1308
1309 // asinh(ap_int)
1310 template<int I>
1311 ap_int<I> generic_asinh(ap_int<I> in) {
1312     ap_fixed<I+1,I+1> input, output;
1313     input = in;
1314     cordic_semi_recovery_asinh(input, output);
1315     return output;
1316 };
1317
1318 // asinh(ap_uint)
1319 template<int I>
1320 ap_uint<I> generic_asinh(ap_uint<I> in) {
1321     ap_fixed<I+1,I+1> input, output;
1322     input = in;
1323     cordic_semi_recovery_asinh(input, output);
1324     return output;
1325 };
1326
1327 // acosh(ap_fixed)
1328 template<int W, int I>
1329 ap_fixed<W,I> generic_acosh(ap_fixed<W,I> in) {
1330     ap_fixed<W+1,I+1> input, output;
1331     input = in;
1332     cordic_semi_recovery_acosh(input, output);
1333     return output;
1334 }
1335
1336 // acosh(ap_ufixed)
1337 template<int W, int I>
1338 ap_ufixed<W,I> generic_acosh(ap_ufixed<W,I> in) {
1339     ap_fixed<W+1,I+1> input, output;
1340     input = in;
1341     cordic_semi_recovery_acosh(input, output);
1342     return output;
1343 };
1344
1345 // acosh(ap_int)
1346 template<int I>
1347 ap_int<I> generic_acosh(ap_int<I> in) {
1348     ap_fixed<I+1,I+1> input, output;
1349     input = in;
1350     cordic_semi_recovery_acosh(input, output);
1351     return output;
1352 };
1353
1354 // acosh(ap_uint)
1355 template<int I>
1356 ap_uint<I> generic_acosh(ap_uint<I> in) {
1357     ap_fixed<I+1,I+1> input, output;
1358     input = in;
1359     cordic_semi_recovery_acosh(input, output);
1360     return output;
1361 };
1362
1363 // asinpi(ap_fixed)
1364 template<int W, int I>
1365 ap_fixed<W,I> generic_asinpi(ap_fixed<W,I> x) {
1366     const static int we = I < 3? 3: I;
1367     ap_fixed<W-I+we, we> xi = x;
1368     ap_fixed<W-I+3,3> asin_x = generic_asin(xi);
1369     #ifndef __SYNTHESIS__
1370     #ifdef _HLSCLIB_DEBUG_
1371     std::cout<<"W="<<W<< ", I="<<I<< ",
x="<<x.to_string()<<std::endl;
1372     std::cout<<"asin_x="<<asin_x.to_string()<<std::endl;
1373
1374     #endif
1375     #endif
1376     static const int F = W - I;
1377     //asin inside [-pi/2, pi/2]
1378     //asinpi inside [-1/2, 1/2]
1379     //to make the precision is 1ulp
1380     //E-R=R*2^(-ulp)/Pi<1/2^F
1381     //ulp=F-1
1382     static const int wf = F < 1? 0: F - 1;
1383     ap_ufixed<2+wf,2> pi = 3.14159265358979323846;
1384     #ifndef __SYNTHESIS__
1385     #ifdef _HLSCLIB_DEBUG_
1386     std::cout<<"pi="<<pi.to_string()<<std::endl;
1387     #endif
1388     #endif
1389     ap_fixed<W,I> r = asin_x/pi;
1390     #ifndef __SYNTHESIS__
1391     #ifdef _HLSCLIB_DEBUG_
1392     std::cout<<"r="<<r.to_string()<<std::endl;
1393     #endif
1394     #endif
1395     return r;
1396 }
1397
1398 template<int W, int I>
1399 ap_ufixed<W,I> generic_asinpi(ap_ufixed<W,I> x) {
1400     ap_fixed<W+1,I+1> xi = x;
1401     return generic_asinpi(xi);
1402 }
1403
1404 template<int I>
1405 ap_int<I> generic_asinpi(ap_int<I> x) {
1406     //asinpi(x) inside [-0.5, 0.5]
1407     if(x==1){
1408         //rounding -0.5 to -1
1409         return -1;
1410     } else if(x==0){
1411         return 0;
1412     } else if(x==1){
1413         //rounding 0.5 to 1
1414         return 1;
1415     } else{
1416         #ifndef __SYNTHESIS__
1417         printf("Error: input value %d (for asinpi) is not
between -1 and 1.\n", x.to_int());
1418         assert(0);
1419         #endif
1420         return 0;
1421     };
1422 }
1423
1424 template<int I>
1425 ap_uint<I> generic_asinpi(ap_uint<I> x) {
1426     //asinpi(x) inside [0, 0.5]
1427     if(x==0){
1428         return 0;
1429     } else if(x==1){
1430         //rounding 0.5 to 1
1431         return 1;
1432     } else{
1433         #ifndef __SYNTHESIS__
1434         printf("Error: input value %d (for asinpi) is not
between -1 and 1.\n", x.to_int());
1435         assert(0);
1436         #endif
1437         return 0;
1438     };
1439 }
1440
1441 // acospi(ap_fixed)
1442 template<int W, int I>
1443 ap_fixed<W,I> generic_acospi(ap_fixed<W,I> x) {
1444     #ifndef __SYNTHESIS__
1445     #ifdef _HLSCLIB_DEBUG_
1446     std::cout<<"W="<<W<< ", I="<<I<< ",
x="<<x.to_string()<<std::endl;
1447     #endif
1448     #endif
1449     const static int we = I < 3? 3: I;
1450     ap_fixed<W-I+we,we> xi = x;
1451     ap_fixed<W-I+3,3> acos_x = generic_acos(xi);
1452     #ifndef __SYNTHESIS__
1453     #ifdef _HLSCLIB_DEBUG_
1454     std::cout<<"acos_x="<<acos_x.to_string()<<std::endl;
1455     #endif
1456     #endif
1457     static const int F = W - I;
1458     //acos inside [0,pi]
1459     //acospi inside [0,1]
1460     //to make the precision is 1ulp
1461     //E-R=R*2^(-ulp)/Pi<1/2^F
1462     //ulp=F
1463     static const int wf = F;
1464     ap_ufixed<2+wf,2> pi = 3.14159265358979323846;
1465     #ifndef __SYNTHESIS__
1466     #ifdef _HLSCLIB_DEBUG_
1467     std::cout<<"pi="<<pi.to_string()<<std::endl;
1468     #endif
1469     #endif
1470     ap_fixed<W,I> r = acos_x/pi;
1471     #ifndef __SYNTHESIS__
1472     #ifdef _HLSCLIB_DEBUG_
1473     std::cout<<"r="<<r.to_string()<<std::endl;
1474     #endif
1475     #endif

```

```

1470     #endif
1471     return r;
1472 }
1473 template<int W, int I>
1474 ap_fixed<W,I> generic_acospi(ap_fixed<W,I> x) {
1475     ap_fixed<W+1,I+1> xi = x;
1476     return generic_acospi(xi);
1477 }
1478 template<int I>
1479 ap_int<I> generic_acospi(ap_int<I> x) {
1480     //acospi(x) inside [0,1]
1481     if(x==1){
1482         return 1;
1483     //rounding 0.5 to 1
1484     } else if(x==0){
1485         return 1;
1486     } else if(x==1){
1487         return 0;
1488     } else{
1489         #ifndef __SYNTHESIS__
1490         printf("Error: input value %d (for acospi) is not
between -1 and 1.\n", x.to_int());
1491         assert(0);
1492         #endif
1493         return 0;
1494     };
1495 }
1496 template<int I>
1497 ap_uint<I> generic_acospi(ap_uint<I> x) {
1498     //acospi(x) inside [0,0.5]
1499     if(x==0){
1500         return 1;
1501     } else if(x==1){
1502         return 0;
1503     } else{
1504         #ifndef __SYNTHESIS__
1505         printf("Error: input value %d (for acospi) is not
between -1 and 1.\n", x.to_int());
1506         assert(0);
1507         #endif
1508         return 0;
1509     };
1510 }
1511 //atanpi(ap_fixed)
1512 template<int W, int I>
1513 ap_fixed<W,I> generic_atanpi(ap_fixed<W,I> x) {
1514     ap_fixed<W-I+3,3> atan_x = generic_atan(x);
1515     #ifndef __SYNTHESIS__
1516     #ifdef _HLSCLIB_DEBUG_
1517     std::cout<<"atan_x="<<atan_x.to_string()<<std::endl;
1518     #endif
1519     #endif
1520     static const int F = W - I;
1521     //atan inside (-pi/2,pi/2)
1522     //atanpi inside (-1/2,1/2)
1523     //to make the precision is 1ulp
1524     //E-R=R*2^(-ulp)/Pi<1/2^F
1525     //ulp=F-1
1526     static const int wf = F < 1? 0: F - 1;
1527     ap_fixed<2+wf,2> pi = 3.14159265358979323846;
1528     ap_fixed<W,I> r = atan_x/pi;
1529     #ifndef __SYNTHESIS__
1530     #ifdef _HLSCLIB_DEBUG_
1531     std::cout<<"r="<<r.to_string()<<std::endl;
1532     #endif
1533     #endif
1534     return r;
1535 }
1536 template<int W, int I>
1537 ap_fixed<W,I> generic_atanpi(ap_fixed<W,I> x) {
1538     ap_fixed<W+1,I+1> xi = x;
1539     return generic_atanpi(xi);
1540 }
1541 template<int I>
1542 ap_int<I> generic_atanpi(ap_int<I> x) {
1543     //atanpi(x) inside (0.5,0.5)
1544     _AP_UNUSED_PARAM(x);
1545     return 0;
1546 }
1547 template<int I>
1548 ap_uint<I> generic_atanpi(ap_uint<I> x) {
1549     //atanpi(x) inside [0,0.5)
1550     _AP_UNUSED_PARAM(x);
1551     return 0;
1552 }
1553 //atan2pi(ap_fixed)
1554 template<int W, int I>
1555 ap_fixed<W,I> generic_atan2pi(ap_fixed<W,I> y, ap_fixed<W,
I> x) {
1556     ap_fixed<W-I+3,3> atan2_x = generic_atan2(y, x);
1557     static const int F = W - I;
1558     //atan2 inside (-pi,pi)
1559     //atan2pi inside (-1,1)
1560     //to make the precision is 1ulp
1561     //E-R=R*2^(-ulp)/Pi<1/2^F
1562     //ulp=F
1563     static const int wf = F;
1564     ap_fixed<2+wf,2> pi = 3.14159265358979323846;
1565     ap_fixed<W,I> r = atan2_x/pi;

```

```

1567     return r;
1568 }
1569 template<int W, int I>
1570 ap_fixed<W,I> generic_atan2pi(ap_fixed<W,I> y,
ap_fixed<W,I> x) {
1571     ap_fixed<W+1,I+1> xi = x;
1572     ap_fixed<W+1,I+1> yi = y;
1573     return generic_atan2pi(yi, xi);
1574 }
1575 template<int I>
1576 ap_int<I> generic_atan2pi(ap_int<I> y, ap_int<I> x) {
1577     //atan2pi inside (-1, 1]
1578     bool x_sig = x[I-1];
1579     bool y_sig = y[I-1];
1580     if(y==0 && x==0) return 0;
1581     //rounding -0.5 to 1
1582     //y<0, x ==0
1583     else if(y_sig && x==0) return -1;
1584     //rounding 0.5 to 1, y >0, x == 0
1585     else if(!y_sig && x==0) return 1;
1586     //y>0 && x < 0
1587     else if(!y_sig && x_sig) return 1;
1588     //y<0, x < 0
1589     else if(y_sig && x_sig) return -1;
1590     else return 0;
1591 }
1592 template<int I>
1593 ap_uint<I> generic_atan2pi(ap_uint<I> y, ap_uint<I> x) {
1594     //atan2pi inside (0, 1]
1595     //rounding 0.5 to 1, y >0, x == 0
1596     if(y!=0 && x==0) return 1;
1597     else return 0;
1598 }
1599 } // namespace
1600 #endif
1601
1602
1603
1604

```

E.18 vendor/etc/hls_diff_apfixed.h (282 lines, 7,036 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_DIFF_APFIXED_H_
7 #define _X_HLS_DIFF_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10
11 //1. fdim_fixed
12 /*
13  * =====
14  * ||      x      y      ||      fdim_fixed(x,y)
15  * ||      (x-y) Normally ||      (x - y)
16  * ||      (x-y) overflow ||      max
17  * ||      x < y          ||      0
18  * =====
19  */
20 template <int W_, int I_>
21 ap_fixed<W_,I_> fdim_fixed(ap_fixed<W_,I_> x, ap_fixed<W_,I_>
y)
22 {
23     #pragma HLS PIPELINE II=1
24     ap_fixed<W+1,I+1> x_overf = 0;
25     ap_fixed<W_,I_> xs = 0;
26     bool overf = 0;
27
28     if(x > y)
29     {
30         x_overf = x - y;
31         overf = x_overf[W_-1];
32         if(overf) {
33             for (int i = 0; i < W_-1; i++){
34                 #pragma HLS UNROLL
35                 xs[i] = 1;
36             }
37             xs[W_-1] = 0;
38         }
39         else{
40             x_overf[W_] = 0;
41             xs = x_overf;
42         }
43     }
44     return xs;
45 }
46 template <int W_, int I_>
47 ap_uint<W_,I_> fdim_fixed(ap_uint<W_,I_> x,
ap_uint<W_,I_> y)
48 {
49     ap_uint<W_,I_> xs = 0;
50     if(x > y){ xs = x - y; }
51     return xs;
52 }
53 template <int I_>

```

```

54 ap_int<I_> fdim_fixed(ap_int<I_> x, ap_int<I_> y)
55 {
56     ap_fixed<I_,I_> xs = x;
57     ap_fixed<I_,I_> ys = y;
58     return fdim_fixed(xs, ys);
59 }
60 template <int I_>
61 ap_uint<I_> fdim_fixed(ap_uint<I_> x, ap_uint<I_> y)
62 {
63     ap_uint<I_> xs = 0;
64     if(x > y) { xs = x - y; }
65     return xs;
66 }
67
68 //2. fmax_fixed
69 /*
70 * =====
71 * || x y || fmax_fixed(x,y)
72 * || x >= y || x
73 * || x < y || y
74 * =====
75 */
76 template <int W_, int I_>
77 ap_fixed<W_,I_> fmax_fixed(ap_fixed<W_,I_> x, ap_fixed<W_,I_>
y )
78 {
79     ap_fixed<W_,I_> xs = 0;
80     if(x >= y) xs = x ;
81     else xs = y ;
82     return xs;
83 }
84 template <int W_, int I_>
85 ap_ufixed<W_,I_> fmax_fixed(ap_ufixed<W_,I_> x,
ap_ufixed<W_,I_> y)
86 {
87     ap_ufixed<W_,I_> xs = 0;
88     if(x >= y) xs = x;
89     else xs = y ;
90     return xs;
91 }
92 template <int I_>
93 ap_int<I_> fmax_fixed(ap_int<I_> x, ap_int<I_> y)
94 {
95     ap_int<I_> xs = 0;
96     if(x >= y) xs = x;
97     else xs = y ;
98     return xs;
99 }
100 template <int I_>
101 ap_uint<I_> fmax_fixed(ap_uint<I_> x, ap_uint<I_> y)
102 {
103     ap_uint<I_> xs = 0;
104     if(x >= y) xs = x;
105     else xs = y ;
106     return xs;
107 }
108
109 //3. fmin_fixed
110 /*
111 * =====
112 * || x y || fmin_fixed(x,y)
113 * || x > y || y
114 * || x <= y || x
115 * =====
116 */
117 template <int W_, int I_>
118 ap_fixed<W_,I_> fmin_fixed(ap_fixed<W_,I_> x, ap_fixed<W_,I_>
y )
119 {
120     ap_fixed<W_,I_> xs = 0;
121     if(x <= y) xs = x ;
122     else xs = y ;
123     return xs;
124 }
125 template <int W_, int I_>
126 ap_ufixed<W_,I_> fmin_fixed(ap_ufixed<W_,I_> x,
ap_ufixed<W_,I_> y)
127 {
128     ap_ufixed<W_,I_> xs = 0;
129     if(x <= y) xs = x;
130     else xs = y ;
131     return xs;
132 }
133 template <int I_>
134 ap_int<I_> fmin_fixed(ap_int<I_> x, ap_int<I_> y)
135 {
136     ap_int<I_> xs = 0;
137     if(x <= y) xs = x;
138     else xs = y ;
139     return xs;
140 }
141 template <int I_>
142 ap_uint<I_> fmin_fixed(ap_uint<I_> x, ap_uint<I_> y)
143 {
144     ap_uint<I_> xs = 0;
145     if(x <= y) xs = x;
146     else xs = y ;
147     return xs;
148 }
149
150 // TODO DEBUG
151 //4. maxmag_fixed
152 /*
153 * =====
154 * || x y || maxmag_fixed(x,y)
155 * || max{|x|,|y|} overflow || max
156 * || |x| >= |y| || x
157 * || |x| < |y| || y
158 * =====
159 */
160 template <int W_, int I_>
161 ap_fixed<W_,I_> maxmag_fixed(ap_fixed<W_,I_> x,
ap_fixed<W_,I_> y )
162 {
163     #pragma HLS PIPELINE II=1
164     ap_fixed<W+1,I+1> xm = -x;
165     ap_fixed<W+1,I+1> ym = -y;
166     ap_fixed<W+1,I+1> x_fabs = ( x[W_-1] ) ? xm :
ap_fixed<W+1,I+1>(x);
167     ap_fixed<W+1,I+1> y_fabs = ( y[W_-1] ) ? ym :
ap_ufixed<W+1,I+1>(y);
168
169     ap_fixed<W+1,I+1> xs_t = 0;
170     ap_fixed<W_,I_> xs = 0;
171     if(x_fabs >= y_fabs)
172     {
173         xs_t = x_fabs ;
174         xs = x;
175     }
176     else{
177         xs_t = y_fabs ;
178         xs = y;
179     }
180     //case overflow
181     if(xs_t[W_-1])
182     {
183         for (int i = 0; i < W_-1; i++){
184             #pragma HLS UNROLL
185             xs[i] = 1;
186         }
187         xs[W_-1] = 0;
188     }
189     return xs;
190 }
191
192 template <int W_, int I_>
193 ap_ufixed<W_,I_> maxmag_fixed(ap_ufixed<W_,I_> x,
ap_ufixed<W_,I_> y)
194 {
195     ap_ufixed<W_,I_> xs = 0;
196     if(x >= y) xs = x;
197     else xs = y ;
198     return xs;
199 }
200 template <int I_>
201 ap_int<I_> maxmag_fixed(ap_int<I_> x, ap_int<I_> y)
202 {
203     ap_fixed<I_,I_> xs = x;
204     ap_fixed<I_,I_> ys = y;
205     return maxmag_fixed(xs, ys);
206 }
207 template <int I_>
208 ap_uint<I_> maxmag_fixed(ap_uint<I_> x, ap_uint<I_> y)
209 {
210     ap_uint<I_> xs = 0;
211     if(x >= y) xs = x;
212     else xs = y ;
213     return xs;
214 }
215
216
217 //5. minmag_fixed
218 /*
219 * =====
220 * || x y || minmag_fixed(x,y)
221 * || min{|x|,|y|} overflow || max
222 * || |x| > |y| || y
223 * || |x| <= |y| || x
224 * =====
225 */
226 template <int W_, int I_>
227 ap_fixed<W_,I_> minmag_fixed(ap_fixed<W_,I_> x,
ap_fixed<W_,I_> y )
228 {
229     #pragma HLS PIPELINE II=1
230     ap_fixed<W+1,I+1> xm = -x;
231     ap_fixed<W+1,I+1> ym = -y;
232     ap_fixed<W+1,I+1> x_fabs = ( x[W_-1] ) ? xm :
ap_fixed<W+1,I+1>(x);
233     ap_fixed<W+1,I+1> y_fabs = ( y[W_-1] ) ? ym :
ap_ufixed<W+1,I+1>(y);
234
235     ap_fixed<W+1,I+1> xs_t = 0;
236     ap_fixed<W_,I_> xs = 0;
237     if(x_fabs <= y_fabs)
238     {
239         xs_t = x_fabs ;
240         xs = x;

```

```

241     }
242     else{
243         xs_t = y_fabs ;
244         xs = y;
245     }
246     if(xs_t[W_-1])
247     {
248         for (int i = 0; i < W_-1; i++){
249             #pragma HLS UNROLL
250             xs[i] = 1;
251         }
252         xs[W_-1] = 0;
253     }
254     return xs;
255 }
256 }
257 template <int W_, int I_>
258 ap_ufixed<W_, I_> minmag_fixed(ap_ufixed<W_, I_> x,
ap_ufixed<W_, I_> y)
259 {
260     ap_ufixed<W_, I_> xs = 0;
261     if(x <= y) xs = x;
262     else xs = y ;
263     return xs;
264 }
265 template <int I_>
266 ap_int<I_> minmag_fixed(ap_int<I_> x, ap_int<I_> y)
267 {
268     ap_fixed<I_, I_> xs = x;
269     ap_fixed<I_, I_> ys = y;
270     return minmag_fixed(xs, ys);
271 }
272 template <int I_>
273 ap_uint<I_> minmag_fixed(ap_uint<I_> x, ap_uint<I_> y)
274 {
275     ap_uint<I_> xs = 0;
276     if(x <= y) xs = x;
277     else xs = y ;
278     return xs;
279 }
280 #endif
281
282

```

E.19 vendor/etc/hls_erf_apfixed.h (660 lines, 23,172 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_ERF_APFIXED_H_
7 #define _X_HLS_ERF_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10 #include "etc/hls_exp_apfixed.h"
11 #include "utils/x_hls_utils.h"
12
13 namespace erf_erfc_fixed {
14
15     namespace {
16         template <typename T> class coeff { };
17
18         template <> class coeff<double> {
19             //class coeff<double> {
20             public:
21                 static const double tiny;
22                 static const double one_over_two;
23                 static const double one;
24                 static const double two;
25                 static const double erx;
26                 static const double efx;
27                 static const double efx8;
28                 static const double pp[5];
29                 static const double qq[6];
30                 static const double pa[7];
31                 static const double qa[7];
32                 static const double ra[8];
33                 static const double sa[9];
34                 static const double rb[7];
35                 static const double sb[8];
36             };
37             const double coeff<double>::tiny = 1e-300;
38             const double coeff<double>::one_over_two=
5.000000000000000000000000e-01; /* 0x3FE00000, 0x00000000 */
39             const double coeff<double>::one =
1.000000000000000000000000e+00; /* 0x3FF00000, 0x00000000 */
40             const double coeff<double>::two =
2.000000000000000000000000e+00; /* 0x40000000, 0x00000000 */
41             /* c = (float)0.84506291151 */
42             const double coeff<double>::erx =
8.45062911510467529297e-01; /* 0x3FEB0AC1, 0x60000000 */
43             /*
44             * Coefficients for approximation to erf on
[0,0.84375]
45             */
46             const double coeff<double>::efx =
1.28379167095512586316e-01; /* 0x3FC06EBA, 0x8214DB69 */

```

```

47     const double coeff<double>::efx8=
1.02703333676410069053e+00; /* 0x3FF06EBA, 0x8214DB69 */
48     const double coeff<double>::pp[] =
49     {1.28379167095512558561e-01, /* 0x3FC06EBA, 0x8214D-
B68 */
50     -3.25042107247001499370e-01, /* 0xBF04CD7D,
0x691CB913 */
51     -2.84817495755985104766e-02, /* 0xBF9D2A51, 0xDB-
D7194F */
52     -5.77027029648944159157e-03, /* 0xBF77A291,
0x236668E4 */
53     -2.37630166566501626084e-05}; /* 0xBEF8EAD6,
0x120016AC */
54     const double coeff<double>::qq[] =
55     {0.0, 3.97917223959155352819e-01, /* 0x3FD97779, 0x-
CDDADC09 */
56     6.50222499887672944485e-02, /* 0x3FB0A54C,
0x5536CEBA */
57     5.08130628187576562776e-03, /* 0x3F7D4022, 0x-
C4D36B0F */
58     1.32494738004321644526e-04, /* 0x3F215DC9,
0x221C1A10 */
59     -3.96022827877536812320e-06}; /* 0xBED09C43,
0x42A26120 */
60     /*
61     * Coefficients for approximation to erf in
[0.84375,1.25]
62     */
63     const double coeff<double>::pa[] =
64     {-2.36211856075265944077e-03, /* 0xBF6359B8, 0xBE-
F77538 */
65     4.14856118683748331666e-01, /* 0x3FDA8D00, 0x-
AD92B34D */
66     -3.72207876035701323847e-01, /* 0xBF07D240, 0xFB-
B8C3F1 */
67     3.18346619901161753674e-01, /* 0x3FD45FCA,
0x805120E4 */
68     -1.10894694282396677476e-01, /* 0xBFBC6398,
0x3D3E28EC */
69     3.54783043256182359371e-02, /* 0x3FA2A2A36,
0x599795EB */
70     -2.16637559486879084300e-03}; /* 0xBF61BF38,
0x0A96073F */
71     const double coeff<double>::qa[] =
72     {0.0, 1.06420880400844228286e-01, /* 0x3FBB3E66,
0x18EEE323 */
73     5.40397917702171048937e-01, /* 0x3FE14AF0, 0x92E-
B6F33 */
74     7.18286544141962662868e-02, /* 0x3FB2635C,
0xD99FE9A7 */
75     1.26171219808761642112e-01, /* 0x3FC02660,
0xE763351F */
76     1.36370839120290507362e-02, /* 0x3F8BEDC2, 0x6B51D-
D1C */
77     1.19844998467991074170e-02}; /* 0x3F888B54,
0x5735151D */
78     /*
79     * Coefficients for approximation to erfc in
[1.25,1/0.35]
80     */
81     const double coeff<double>::ra[] =
82     {-9.86494403484714822705e-03, /* 0xBF843412,
0x600D6435 */
83     -6.93858572707181764372e-01, /* 0xBFE63416,
0xE4BA7360 */
84     -1.05586262253232909814e+01, /* 0xC0251E04,
0x41B0E726 */
85     -6.23753324503260060396e+01, /* 0xC04F300A,
0xE4CBA38D */
86     -1.62396669462573470355e+02, /* 0xC0644CB1,
0xB4282266 */
87     -1.84605092906711035994e+02, /* 0xC067135C, 0xEB-
CCABB2 */
88     -8.12874355063065934246e+01, /* 0xC0545265,
0x57E4D2F2 */
89     -9.81432934416914548592e+00}; /* 0xC023A0EF, 0x-
C69AC25C */
90     const double coeff<double>::sa[] =
91     {0.0,1.96512716674392571292e+01, /* 0x4033A6B9, 0xB-
D707687 */
92     1.37657754143519042600e+02, /* 0x4061350C,
0x526AE721 */
93     4.34565877475229228821e+02, /* 0x407B290D,
0xD58A1A71 */
94     6.45387271733267880336e+02, /* 0x40842B19,
0x21EC2868 */
95     4.29008140027567833386e+02, /* 0x407AD021,
0x57709314 */
96     1.08635005541779435134e+02, /* 0x405B28A3,
0xEE48AE2C */
97     6.57024977031928170135e+00, /* 0x401A47EF,
0xBE484A93 */
98     -6.04244152148580987438e-02}; /* 0xBFAEEFF2,
0xEE749A62 */
99     /*
100     * Coefficients for approximation to erfc in
[1/.35,28]
101     */
102     const double coeff<double>::rb[] =
103     {-9.86494292470009928597e-03, /* 0xBF843412,

```



```

0x39E86F4A */
104 -7.99283237680523006574e-01, /* 0xBFE993BA,
0x70C285DE */
105 -1.77579549177547519889e+01, /* 0xC031C209,
0x555F995A */
106 -1.60636384855821916062e+02, /* 0xC064145D,
0x43C5ED98 */
107 -6.37566443368389627722e+02, /* 0xC083EC88,
0x1395F228 */
108 -1.02509513161107724954e+03, /* 0xC0900461,
0x6A2E5992 */
109 -4.83519191608651397019e+02; /* 0xC07E384E, 0x9BD-
C38E6 */
110 const double coeff<double>::sb[] =
111 {0.0, 3.03380607434824582924e+01, /* 0x403E568B,
0x261D5190 */
112 3.25792512996573918826e+02, /* 0x40745CAE,
0x221B9F0A */
113 1.53672958608443695994e+03, /* 0x409802EB,
0x189D5118 */
114 3.1998582195085953908e+03, /* 0x40A8FFB7,
0x688C246A */
115 2.55305040643316442583e+03, /* 0x40A3F219, 0xCED-
F3BE6 */
116 4.74528541206955367215e+02, /* 0x407DA874,
0xE79FE763 */
117 -2.24409524465858183362e+01; /* 0xC03670E2,
0x42712D62 */
118 #if 0
119 template< > class coeff<float> {
120 public:
121 static const float tiny;
122 static const float one_over_two;
123 static const float one;
124 static const float two;
125 static const float erx;
126 static const float efx;
127 static const float efx8;
128 static const float pp[5];
129 static const float qq[6];
130 static const float pa[7];
131 static const float qa[7];
132 static const float ra[8];
133 static const float sa[9];
134 static const float rb[7];
135 static const float sb[8];
136 };
137 const float coeff<float>::tiny = 1e-30;
138 const float coeff<float>::one_over_two=
5.000000000000e-01; /* 0x3F000000 */
139 const float coeff<float>::one = 1.000000000000e+00; /*
0x3F800000 */
140 const float coeff<float>::two = 2.000000000000e+00; /*
0x40000000 */
141 /* c = (float)0.84506291151 */
142 const float coeff<float>::erx = 8.4506291151e-01; /*
0x3f58560b */
143 /*
144 * Coefficients for approximation to erf on
[0,0.84375]
145 */
146 const float coeff<float>::efx = 1.2837916613e-01; /*
0x3e0375d4 */
147 const float coeff<float>::efx8= 1.0270333290e+00; /*
0x3f8375d4 */
148 const float coeff<float>::pp[] =
149 {1.2837916613e-01, /* 0x3e0375d4 */
150 -3.2504209876e-01, /* 0x3e0375d4 */
151 -2.8481749818e-02, /* 0x3e0375d4 */
152 -5.7702702470e-03, /* 0x3e0375d4 */
153 -2.3763017452e-05; /* 0x3e0375d4 */
154 const float coeff<float>::qq[] =
155 {0.0, 3.9791721106e-01, /* 0x3e0375d4 */
156 6.5022252500e-02, /* 0x3e0375d4 */
157 5.0813062117e-03, /* 0x3e0375d4 */
158 1.3249473704e-04, /* 0x3e0375d4 */
159 -3.9602282413e-06; /* 0x3e0375d4 */
160 /*
161 * Coefficients for approximation to erf in
[0.84375,1.25]
162 */
163 const float coeff<float>::pa[] =
164 {-2.3621185683e-03, /* 0x3e0375d4 */
165 4.1485610604e-01, /* 0x3e0375d4 */
166 -3.7220788002e-01, /* 0x3e0375d4 */
167 3.1834661961e-01, /* 0x3e0375d4 */
168 -1.1089469492e-01, /* 0x3e0375d4 */
169 3.5478305072e-02, /* 0x3e0375d4 */
170 -2.1663755178e-03; /* 0x3e0375d4 */
171 const float coeff<float>::qa[] =
172 {0.0, 1.0642088205e-01, /* 0x3e0375d4 */
173 5.4039794207e-01, /* 0x3e0375d4 */
174 7.1828655899e-02, /* 0x3e0375d4 */
175 1.2617121637e-01, /* 0x3e0375d4 */
176 1.3637083583e-02, /* 0x3e0375d4 */
177 1.1984500103e-02; /* 0x3e0375d4 */
178 /*
179 * Coefficients for approximation to erfc in
[1.25,1/0.35]
180 */

```

```

181 const float coeff<float>::ra[] =
182 {-9.8649440333e-03, /* 0x3e0375d4 */
183 -6.9385856390e-01, /* 0x3e0375d4 */
184 -1.0558626175e+01, /* 0x3e0375d4 */
185 -6.2375331879e+01, /* 0x3e0375d4 */
186 -1.6239666748e+02, /* 0x3e0375d4 */
187 -1.8460508728e+02, /* 0x3e0375d4 */
188 -8.1287437439e+01, /* 0x3e0375d4 */
189 -9.8143291473e+00; /* 0x3e0375d4 */
190 const float coeff<float>::sa[] =
191 {0.0, 1.9651271820e+01, /* 0x419d35ce */
192 1.3765776062e+02, /* 0x4309a863 */
193 4.3456588745e+02, /* 0x43d9486f */
194 6.4538726807e+02, /* 0x442158c9 */
195 4.2900814819e+02, /* 0x43d6810b */
196 1.0863500214e+02, /* 0x42d9451f */
197 6.5702495575e+00, /* 0x40d237fc */
198 -6.0424413532e-02; /* 0x40d237fc */
199 /*
200 * Coefficients for approximation to erfc in
[1/.35,28]
201 */
202 const float coeff<float>::rb[] =
203 {-9.8649431020e-03, /* 0x3e0375d4 */
204 -7.9928326607e-01, /* 0x3e0375d4 */
205 -1.7757955551e+01, /* 0x3e0375d4 */
206 -1.6063638306e+02, /* 0x3e0375d4 */
207 -6.3756646729e+02, /* 0x3e0375d4 */
208 -1.0250950928e+03, /* 0x3e0375d4 */
209 -4.8351919556e+02; /* 0x3e0375d4 */
210 const float coeff<float>::sb[] =
211 {0.0, 3.0338060379e+01, /* 0x41f2b459 */
212 3.2579251099e+02, /* 0x43a2e571 */
213 1.5367296143e+03, /* 0x44c01759 */
214 3.1998581543e+03, /* 0x4547fddb */
215 2.5530502930e+03, /* 0x451f90ce */
216 4.7452853394e+02, /* 0x43ed43a7 */
217 -2.2440952301e+01; /* 0x3bc38712 */
218 #endif
219 template<typename T> class erf_traits{};
220 #if 0
221 template< > class erf_traits<float> {
222 public:
223 static const uint32_t mask;
224 static const uint32_t segment[10];
225 };
226 const uint32_t erf_traits<float>::mask = 0xfffff000;
227 const uint32_t erf_traits<float>::segment[10] =
228 {
229 0x7fffffff,
230 0x7f800000,
231 0x3f580000,
232 0x31800000,
233 0x04000000,
234 0x3fa00000,
235 0x40c00000,
236 0x4036DB6E,
237 31,
238 0
239 };
240 #endif
241 template< > class erf_traits<double> {
242 public:
243 static const uint64_t mask;
244 static const uint32_t segment[10];
245 };
246 const uint64_t erf_traits<double>::mask =
0xffffffff00000000;
247 const uint32_t erf_traits<double>::segment[10] =
248 {
249 0x7fffffff,
250 0x7ff00000,
251 0x3feb0000,
252 0x3e300000,
253 0x00000000,
254 0x3ff40000,
255 0x40180000,
256 0x4006DB6E,
257 63,
258 32
259 };
260 template<typename T> class erfc_traits{};
261 #if 0
262 template< > class erfc_traits<float>{
263 public:
264 static const uint32_t segment[11];
265 };
266 const uint32_t erfc_traits<float>::segment[11] =
267 {
268 0x7fffffff,
269 0x7f800000,
270 0x3f580000,
271 0x23800000,
272 0x3e800000,
273 0x3fa00000,
274 0x41e00000,
275 0x4036DB6D,
276 0x40c00000,
277 31,
278

```

```

279         0
280     };
281 #endif
282     template<> class erfc_traits<double>{
283     public:
284         static const uint32_t segment[11];
285     };
286     const uint32_t erfc_traits<double>::segment[11] =
287     {
288         0x7fffffff,
289         0x7ff00000,
290         0x3feb0000,
291         0x3c700000,
292         0x3fd00000,
293         0x3ff40000,
294         0x403c0000,
295         0x4006DB6D,
296         0x40180000,
297         63,
298         32
299     };
300 } // anonymous namespace
301 //
302 /* erf_fixed(x) or erfc_fixed(x) :Input and Output is same
width */
303 //1. erf_fixed
304 /*
305 *erf(0)      = 0
306 *erf(2)      ~ 1
307 *erf(-2)     ~ 1
308 *erf(+infinity) = 1 but in std and hls =0
309 */
310 /*
311 * =====
312 *          2          x
313 *          |          |
314 *          |          |
315 *   erf(x) = -----  | exp(-t*t)dt
316 *          sqrt(pi)   |
317 *                    0
318 * =====
319 */
320 template <int W_, int I_>
321 ap_fixed<W_, I_> erf(ap_fixed<W_, I_> x_fixed )
322 {
323     int F_ = W_ - I_;
324
325     if (I_>33) return 0;
326     else if (F_>32) return 0;
327
328     double x = x_fixed.to_double();
329     int32_t hx;
330     uint32_t ix;
331     double R,S,P,Q,s,y,z,r;
332     //GET_HIGH_WORD(hx,x);
333     fp_struct<double> fp_abs(x);
334     fp_abs.sign=0;
335     fp_struct<double> fp_x(x);
336     ap_uint<32> tmp = fp_x.data()
337     (erf_traits<double>::segment[8],erf_traits<double>::segment[9]);
338     hx = tmp.to_int();
339     ix = hx & erf_traits<double>::segment[0];
340
341     #if 0
342     ///if(FP_SUBNORMAL == generic_fpclassify(x)) {
343     ///    return 0;
344     ///}
345     #endif
346     if(fp_x.exp == 0x00 && fp_x.sign!= 0x00){
347         return 0;
348     }
349     if(ix>=erf_traits<double>::segment[1]) { /*
erf(nan)=nan */
350         //i = ((uint32_t)hx>31)<1;
351         if(hx>0)
352             return ap_fixed<W_, I_>(((double)1.0
+coeff<double>::one/x); /* erf(+inf)=+1 */
353         else
354             return ap_fixed<W_, I_>(coeff<double>::one/x-
(double)1.0);
355     }
356
357     if(ix < erf_traits<double>::segment[2])
358     {
359         /* |x|<0.84375 */
360         double r1,r2,s1,s2,s3,z2,z4;
361         if(ix < erf_traits<double>::segment[3]) { /* |x|
<2**-28 */
362             if (ix < erf_traits<double>::segment[2])
363                 return
364             ap_fixed<W_, I_>(((double)0.125)*((double)8.0*x+coeff<double>::efx8*
x)); /*avoid underflow */
365             return ap_fixed<W_, I_>(x +
coeff<double>::efx*x);
366         }
367         z = x*x;
368         #ifndef DO_NOT_USE_THIS
369         r = pp0+z*(pp1+z*(pp2+z*(pp3+z*pp4)));
370         s =
coeff<double>::one+z*(qq1+z*(qq2+z*(qq3+z*(qq4+z*qq5)));

```

```

368 #else
369     r1 = coeff<double>::pp[0]+z*coeff<double>::pp[1];
370     z2=z*z;
371     r2 = coeff<double>::pp[2]+z*coeff<double>::pp[3];
372     z4=z2*z2;
373     s1 = coeff<double>::one+z* coeff<double>::qq[1];
374     s2 = coeff<double>::qq[2]+z*coeff<double>::qq[3];
375     s3 = coeff<double>::qq[4]+z*coeff<double>::qq[5];
376     r = r1 + z2*r2 + z4*coeff<double>::pp[4];
377     s = s1 + z2*s2 + z4*s3;
378 #endif
379     y = r/s;
380     return ap_fixed<W_, I_>(x + x*y);
381 }
382 if(ix < erf_traits<double>::segment[5])
383 {
384     /* 0.84375 <= |x| < 1.25 */
385     double s2,s4,s6,P1,P2,P3,P4,Q1,Q2,Q3,Q4;
386     //s = generic_fabs_d(x)-coeff<double>::one;
387     s = fp_abs.to_ieee()-coeff<double>::one;
388     #ifndef DO_NOT_USE_THIS
389     P =
pa0+s*(pa1+s*(pa2+s*(pa3+s*(pa4+s*(pa5+s*pa6)))));
390     Q =
coeff<double>::one+s*(qa1+s*(qa2+s*(qa3+s*(qa4+s*(qa5+s*qa6)))));
391 #else
392     P1 = coeff<double>::pa[0]+s*coeff<double>::pa[1];
393     Q1 = coeff<double>::one+s*coeff<double>::qa[1];
394     s2=s*s;
395     P2 = coeff<double>::pa[2]+s*coeff<double>::pa[3];
396     s4=s2*s2;
397     P3 = coeff<double>::pa[4]+s*coeff<double>::pa[5];
398     Q3 = coeff<double>::qa[4]+s*coeff<double>::qa[5];
399     P4 = coeff<double>::pa[6];
400     Q4 = coeff<double>::qa[6];
401     P = P1 + s2*P2 + s4*P3 + s6*P4;
402     Q = Q1 + s2*Q2 + s4*Q3 + s6*Q4;
403 #endif
404     if(hx>=0) return ap_fixed<W_, I_>(coeff<double>::erx
+ P/Q);
405     else return ap_fixed<W_, I_>(-coeff<double>::erx - P/Q);
406 }
407 if (ix >= erf_traits<double>::segment[6])
408 {
409     /* inf>|x|>=6 */
410     if(hx>=0) return ap_fixed<W_, I_>(coeff<double>::one-
coeff<double>::tiny);
411     else return ap_fixed<W_, I_>(coeff<double>::tiny-
coeff<double>::one);
412 }
413 x = fp_abs.to_ieee();
414 s = coeff<double>::one/(x*x);
415 if(ix< erf_traits<double>::segment[7]) { /* |x| <
1/0.35 */
416     #ifndef DO_NOT_USE_THIS
417     R=ra0+s*(ra1+s*(ra2+s*(ra3+s*(ra4+s*(
ra5+s*(ra6+s*ra7)))));
418     S=coeff<double>::one+s*(sa1+s*(sa2+s*(sa3+s*(sa4+s*(
sa5+s*(sa6+s*(sa7+s*sa8)))));
419 #else
420     double R1,R2,R3,R4,S1,S2,S3,S4,s2,s4,s6,s8;
421     R1 = coeff<double>::ra[0]+s*coeff<double>::ra[1];s2
= s*s;
422     S1 = coeff<double>::sa[0]+s*coeff<double>::sa[1];
423     s4 = s2*s2;
424     R2 = coeff<double>::ra[2]+s*coeff<double>::ra[3];s6
= s4*s2;
425     S2 = coeff<double>::sa[2]+s*coeff<double>::sa[3];s8
= s4*s4;
426     R3 = coeff<double>::ra[4]+s*coeff<double>::ra[5];
427     S3 = coeff<double>::sa[4]+s*coeff<double>::sa[5];
428     R4 = coeff<double>::ra[6]+s*coeff<double>::ra[7];
429     S4 = coeff<double>::sa[6]+s*coeff<double>::sa[7];
430     R = R1 + s2*R2 + s4*R3 + s6*R4;
431     S = S1 + s2*S2 + s4*S3 + s6*S4 +
s8*coeff<double>::sa[8];
432 #endif
433     } else { /* |x| >= 1/0.35 */
434     #ifndef DO_NOT_USE_THIS
435     R=rb0+s*(rb1+s*(rb2+s*(rb3+s*(rb4+s*(
rb5+s*rb6)))));
436     S=coeff<double>::one+s*(sb1+s*(sb2+s*(sb3+s*(sb4+s*(
sb5+s*(sb6+s*sb7)))));
437 #else
438     double R1,R2,R3,S1,S2,S3,S4,s2,s4,s6;
439     R1 = coeff<double>::rb[0]+s*coeff<double>::rb[1];s2
= s*s;
440     S1 = coeff<double>::one+ s*coeff<double>::sb[1];
441     s4 = s2*s2;
442     R2 = coeff<double>::rb[2]+s*coeff<double>::rb[3];s6
= s4*s2;
443     S2 = coeff<double>::sb[2]+s*coeff<double>::sb[3];

```



```

439      R3 = coeff<double>::rb[4]+s*coeff<double>::rb[5];
440      S3 = coeff<double>::sb[4]+s*coeff<double>::sb[5];
441      S4 = coeff<double>::sb[6]+s*coeff<double>::sb[7];
442      R = R1 + s2*R2 + s4*R3 + s6*coeff<double>::rb[6];
443      S = S1 + s2*S2 + s4*S3 + s6*S4;
444  #endif
445  }
446  z = x;
447  //SET_LOW_WORD(z,0);
448  fp_struct<double> fp_z(z);
449  ap_uint<Type_BitWidth<double>::Value> data = fp_z.data()
& erf_traits<double>::mask;
450  fp_struct<double> fp_z_new(data);
451  z = fp_z_new.to_ieee();
452  //r = _ieee754_exp(-z*z-0.5625)*_ieee754_exp((z-
x)*(z+x)+R/S);
453  //r = exp_reduce::exp(-z*z-
(double)0.5625)*exp_reduce::exp((z-x)*(z+x)+R/S);
454  ap_fixed<W_, I> r_fixed = exp_reduce::exp(ap_fixed<W_,
I>(-z*z-(double)0.5625))*exp_reduce::exp(ap_fixed<W_, I>((z-
x)*(z+x)+R/S));
455  r = r_fixed.to_double();
456  //r = exp_reduce::exp(-x*x-(T)0.5625+R/S); // also
work
457  if(hx>=0) return ap_fixed<W_, I>(coeff<double>::one-r/
x);
458  else return ap_fixed<W_, I>(r/x-coeff<double>::one);
459  }
460  }
461  template <int W_, int I>
462  ap_ufixed<W_, I> erf(ap_ufixed<W_, I> x) {
463  ap_fixed<W_+1, I+1> xs = x;
464  return erf(xs);
465  }
466  }
467  template <int I>
468  ap_int<I> erf(ap_int<I> x) {
469  ap_fixed<I, I> xs = x;
470  return erf(xs);
471  }
472  }
473  template <int I>
474  ap_uint<I> erf(ap_uint<I> x) {
475  ap_fixed<I+1, I+1> xs = x;
476  return erf(xs);
477  }
478  }
479  //2. erfc_fixed
480  /*
481  * =====
482  * erfc(x) = 1 - erf(x)
483  * =====
484  */
485  template <int W_, int I>
486  ap_fixed<W_, I> erfc(ap_fixed<W_, I> x_fixed)
487  {
488  #pragma HLS PIPELINE II=1
489  int F_ = W_ - I_;
490  if (I_>33) return 0;
491  else if (F_>32) return 0;
492  double x = x_fixed.to_double();
493  int32_t hx;
494  uint32_t ix;
495  double R,S,P,Q,s,y,z,r;
496  //GET_HIGH_WORD(hx,x);
497  fp_struct<double> fp_abs(x);
498  fp_abs.sign=0;
499  fp_struct<double> fp_x(x);
500  ap_uint<32> tmp = fp_x.data()
(erfc_traits<double>::segment[9],erfc_traits<double>::segment[10]);
501  hx = tmp.to_int();
502  ix = hx & erfc_traits<double>::segment[0];
503  if(ix>=erfc_traits<double>::segment[1])
504  {
505  /* erfc(nan)=nan */
506  if(hx>0)
507  return ap_fixed<W_, I>(coeff<double>::one/x);
508  else
509  return ap_fixed<W_, I>(coeff<double>::two +
coeff<double>::one/x);
510  //return (T)(((uint32_t)hx>>31)<<1)+coeff<T>::one/x;
511  }
512  if(ix < erfc_traits<double>::segment[2])
513  {
514  /* |x|<0.84375 */
515  double r1,r2,s1,s2,s3,z2,z4;
516  if(ix < erfc_traits<double>::segment[3]) /* |x|
<2**-56 */
517  return ap_fixed<W_, I>(coeff<double>::one-x);
518  z = x*x;
519  #ifndef DO_NOT_USE_THIS
520  r = pp0+z*(pp1+z*(pp2+z*(pp3+z*pp4)));
521  s =
coeff<double>::one+z*(qq1+z*(qq2+z*(qq3+z*(qq4+z*qq5)));
522  #else

```

```

523  r1 = coeff<double>::pp[0]+z*coeff<double>::pp[1];
524  z2=z*z;
525  r2 = coeff<double>::pp[2]+z*coeff<double>::pp[3];
526  z4=z2*z2;
527  s1 = coeff<double>::one+z*coeff<double>::qq[1];
528  s2 = coeff<double>::qq[2]+z*coeff<double>::qq[3];
529  s3 = coeff<double>::qq[4]+z*coeff<double>::qq[5];
530  r = r1 + z2*r2 + z4*coeff<double>::pp[4];
531  s = s1 + z2*s2 + z4*s3;
532  #endif
533  y = r/s;
534  if(ix < erfc_traits<double>::segment[4]) { /*
x<1/4 */
535  return ap_fixed<W_, I>(coeff<double>::one-
(x*x*y));
536  } else {
537  r = x*y;
538  r += (x-coeff<double>::one_over_two);
539  return
ap_fixed<W_, I>(coeff<double>::one_over_two - r) ;
540  }
541  }
542  if(ix < erfc_traits<double>::segment[5])
543  {
544  /* 0.84375 <= |x| < 1.25 */
545  double s2,s4,s6,P1,P2,P3,P4,Q1,Q2,Q3,Q4;
546  s =fp_abs.to_ieee()-coeff<double>::one;
547  #ifndef DO_NOT_USE_THIS
548  P =
pa0+s*(pa1+s*(pa2+s*(pa3+s*(pa4+s*(pa5+s*pa6))));
549  Q =
coeff<double>::one+s*(qa1+s*(qa2+s*(qa3+s*(qa4+s*(qa5+s*qa6))));
550  #else
551  P1 = coeff<double>::pa[0]+s*coeff<double>::pa[1];
552  Q1 = coeff<double>::one+s*coeff<double>::qa[1];
553  P2 = coeff<double>::pa[2]+s*coeff<double>::pa[3];
554  Q2 = coeff<double>::qa[2]+s*coeff<double>::qa[3];
555  P3 = coeff<double>::pa[4]+s*coeff<double>::pa[5];
556  Q3 = coeff<double>::qa[4]+s*coeff<double>::qa[5];
557  P4 = coeff<double>::pa[6];
558  Q4 = coeff<double>::qa[6];
559  P = P1 + s2*P2 + s4*P3 + s6*P4;
560  Q = Q1 + s2*Q2 + s4*Q3 + s6*Q4;
561  #endif
562  if(hx>=0) {
563  z = coeff<double>::one-coeff<double>::erx;
564  return ap_fixed<W_, I>(z - P/Q);
565  } else {
566  z = coeff<double>::erx+P/Q; return
ap_fixed<W_, I>(coeff<double>::one+z);
567  }
568  if (ix < erfc_traits<double>::segment[6])
569  {
570  /* |x|<28 */
571  x = fp_abs.to_ieee();
572  s = coeff<double>::one/(x*x);
573  if(ix< erfc_traits<double>::segment[7]) { /* |x| <
1/.35 ~ 2.857143*/
574  #ifndef DO_NOT_USE_THIS
575  R=ra0+s*(ra1+s*(ra2+s*(ra3+s*(ra4+s*(
ra5+s*(ra6+s*ra7))));
576  #else
577  double R1,R2,R3,R4,S1,S2,S3,S4,s2,s4,s6,s8;
578  R1 =
coeff<double>::ra[0]+s*coeff<double>::ra[1];s2 = s*s;
579  S1 = coeff<double>::one+s*coeff<double>::sa[1];
580  s4 = s2*s2;
581  R2 =
coeff<double>::ra[2]+s*coeff<double>::ra[3];s6 = s4*s2;
582  S2 =
coeff<double>::sa[2]+s*coeff<double>::sa[3];s8 = s4*s4;
583  R3 = coeff<double>::ra[4]+s*coeff<double>::ra[5];
584  S3 = coeff<double>::sa[4]+s*coeff<double>::sa[5];
585  R4 = coeff<double>::ra[6]+s*coeff<double>::ra[7];
586  S4 = coeff<double>::sa[6]+s*coeff<double>::sa[7];
587  R = R1 + s2*R2 + s4*R3 + s6*R4;
588  S = S1 + s2*S2 + s4*S3 + s6*S4 +
s8*coeff<double>::sa[8];
589  #endif
590  } else { /* |x| >= 1/.35 ~
2.857143 */
591  double R1,R2,R3,S1,S2,S3,S4,s2,s4,s6;
592  if(hx<0&&ix>=erfc_traits<double>::segment[8])
return ap_fixed<W_, I>(coeff<double>::two-coeff<double>::tiny);/* x
< -6 */
593  #ifndef DO_NOT_USE_THIS
594  R=rb0+s*(rb1+s*(rb2+s*(rb3+s*(rb4+s*(
rb5+s*rb6))));
595  #endif

```

```

595
S=coeff<double>::one+s*(sb1+s*(sb2+s*(sb3+s*(sb4+s*(
596
    sb5+s*(sb6+s*(sb7))))));
597 #else
598     R1 =
599     S1 =
coeff<double>::rb[0]+s*coeff<double>::rb[1]; s2 = s*s;
599
600     R2 =
coeff<double>::one+s*coeff<double>::sb[1]; s4 = s2*s2;
600
601     S2 =
coeff<double>::rb[2]+s*coeff<double>::rb[3]; s6 = s4*s2;
601
602     R3 =
coeff<double>::sb[2]+s*coeff<double>::sb[3];
602
603     S3 =
coeff<double>::rb[4]+s*coeff<double>::rb[5];
603
604     S4 =
coeff<double>::sb[4]+s*coeff<double>::sb[5];
604
605     R = R1 + s2*R2 + s4*R3 +
coeff<double>::sb[6]+s*coeff<double>::sb[7];
605
606     S = S1 + s2*S2 + s4*S3 + s6*S4;
606
607 #endif
608
609     z = x;
610     //SET_LOW_WORD(z,0);
611     fp_struct<double> fp_z(z);
612     ap_uint<Type_BitWidth<double>::Value> data =
fp_z.data() & erf_traits<double>::mask;
613     fp_struct<double> fp_z_new(data);
614     z = fp_z_new.to_ieee();
615     //r = _ieee754_exp(-z*z-0.5625)*
616     // _ieee754_exp((z-x)*(z+x)+R/S);
617     ap_fixed<W_, I_> r_fixed = exp_re-
duce::exp((ap_fixed<W_, I_>)(-z*z-
(double)0.5625))*exp_reduce::exp((ap_fixed<W_, I_>)((z-x)*(z+x)+R/
S));
618     double r = r_fixed.to_double();
619     double r_x = r/x;
620     if(hx>0) {
621         #if 0
622             if(FP_SUBNORMAL == generic_fpclassify(r_x)) {
623                 return 0;
624             }
625         #endif
626         if(fp_x.exp == 0x0 && fp_x.sign!= 0x0){
627             return 0;
628         }
629         return ap_fixed<W_, I_>(r_x);
630     } else
631         return ap_fixed<W_, I_>(coeff<double>::two-
r_x);
632     } else {
633         if(hx>0) return
ap_fixed<W_, I_>(coeff<double>::tiny*coeff<double>::tiny);
634     else return ap_fixed<W_, I_>(coeff<double>::two-
coeff<double>::tiny);
635     }
636 }
637 }
638 template <int W_, int I_>
639 ap_ufixed<W_, I_> erfc(ap_ufixed<W_, I_> x) {
640     ap_fixed<W_+1, I_+1> xs = x;
641     return erfc(xs);
642 }
643 }
644 template <int I_>
645 ap_int<I_> erfc(ap_int<I_> x) {
646     ap_fixed<I_+1, I_+1> xs = x;
647     return erfc(xs);
648 }
649 }
650 template <int I_>
651 ap_uint<I_> erfc(ap_uint<I_> x) {
652     ap_fixed<I_+1, I_+1> xs = x;
653     return erfc(xs);
654 }
655 }
656 //erf_erfc_fixed
657
658 #endif
659
660

```

E.20 vendor/etc/hls_exp_apfixed.h (3,446 lines, 243,307 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_EXP_APFIXED_H_
7 #define _X_HLS_EXP_APFIXED_H_
8 #include "ap_fixed.h"
9
10 namespace exp_reduce {
11 /* exp(x):Input and Output is same width */

```

```

12 template<int W_, int I_>
13 ap_fixed<W_, I_> exp(ap_fixed<W_, I_> x)
14 {
15     #pragma HLS pipeline
16
17     int F_ = W_ - I_;
18     if (I_>33) return 0;
19     else if (F_>32) return 0;
20
21     // I_s_ is the number of integer bits we need to calcu-
late
22     // I_s_ = ceil( ln2 * max ( I_-1 , F_ ) )
23     // I_s_ = [1..6]
24     int I_s_;
25     int M_ = (I_-1>F_) ? (I_-1) : F_;
26     if (M_==1) I_s_ = 1;
27     else if (M_==2) I_s_ = 2;
28     else if (M_<6) I_s_ = 3;
29     else if (M_<12) I_s_ = 4;
30     else if (M_<24) I_s_ = 5;
31     else I_s_ = 6;
32
33     ap_fixed<W_, I_> r;
34     if (I_s_==1) { // I_s_ = 1, M_ = 1, I_ <= 2, F_ <= 1, W
<= 3
35         ap_fixed<3,2> x_l = x;
36         ap_ufixed<2,1> y = 0;
37         if (x_l[2]!=x_l[1]) { // overflow
38             if (!x_l[2]) y = "0x1.8p0";
39         } else {
40             if (!x_l[2]) y[1] = 1;
41             if (x_l[2]||x_l[0]) y[0] = 1;
42         }
43         if (I_<2) { // overflow
44             if (y[1]) {
45                 y[1] = 0;
46                 y[0] = 1;
47             }
48         }
49         r = y;
50     } else if (I_s_==2) { // I_s_ = 2, M_ = 2, I_ <= 3, F_ <= 2,
W <= 5
51         ap_fixed<5,3> x_l = x;
52         ap_ufixed<4,2> y = 0;
53         ap_fixed<2,2> x_l_int;
54         x_l_int(1,0) = x_l(3,2);
55         ap_ufixed<2,0> x_l_fract;
56         x_l_fract(1,0) = x_l(1,0);
57         //if ((x_l[4]!=x_l[3])||(x_l>="0x1.8p0")) { // over-
flow
58             if ((x_l[4]!=x_l[3])||
((x_l_int==1)&&(x_l_fract>=ap_ufixed<2,0>("0x0.8p0")))) { // over-
flow
59                 if (!x_l[4]) y = "0x3.cp0";
60             } else {
61                 if (!x_l[4]&&((x_l[1]&x_l[0])||x_l[2])) y[3] =
1;
62                 if (!x_l[4]&&((!x_l[2]&&!x_l[0])||(!
x_l[1]&&x_l[0]))) y[2] = 1;
63                 if ((!x_l[4]&&x_l[1]&&!x_l[0])||(!
x_l[4]&&x_l[2])||x_l[2]&&x_l[0])||x_l[2]&&x_l[1]) y[1] = 1;
64                 if ((!x_l[2]&&!x_l[1]&&x_l[0])||(!
x_l[2]&&x_l[1]&&!x_l[0])||x_l[2]&&!x_l[1]&&!x_l[0])||
(x_l[2]&&x_l[1]&&x_l[0])||x_l[4]&&!x_l[3])) y[0] = 1;
65             }
66             if (I_<3) { // overflow
67                 bool overf = 0;
68                 for (int j = 3; j >= I_+1; j--) {
69                     #pragma HLS unroll
70                     if (y[j])
71                         overf = 1;
72                 }
73             }
74             if (overf) {
75                 for (int j = 3; j >= I_+1; j--){
76                     #pragma HLS unroll
77                     y[j] = 0;
78                 }
79                 for (int j = I_; j >= 0; j--){
80                     #pragma HLS unroll
81                     y[j] = 1;
82                 }
83             }
84             r = y;
85         } else if (I_s_==3) { // I_s_ = 3, M_ = 3..5, I_ <= 6, F_ <=
5, W <= 11
86             ap_fixed<11,6> x_l = x;
87             ap_ufixed<10,5> y = 0;
88             ap_fixed<3,3> x_l_int;
89             x_l_int(2,0) = x_l(7,5);
90             ap_ufixed<5,0> x_l_fract;
91             x_l_fract(4,0) = x_l(4,0);
92             //if ((x_l[10]!=x_l[9])||x_l[10]!=x_l[8])||x_l[10]!
=x_l[7])||x_l>"0x3.7p0")) { // overflow
93                 if ((x_l[10]!=x_l[9])||x_l[10]!=x_l[8])||x_l[10]!
=x_l[7])||((x_l_int==3)&&(x_l_fract>ap_fixed<5,0>("0x0.7p0"))))
{ // overflow
94                     if (!x_l[10]) y = "0x1f.f8p0";

```

```

95     } else {
96         ap_uint<6> x_msb_ind;
97         x_msb_ind[5] = x_l[10];
98         x_msb_ind(4,0) = x_l(6,2);
99         ap_ufixed<2,-3> x_lsb = x_l;
100
101         ap_uint<4> x_msb_ind;
102         x_msb_ind[3] = x_l[10];
103         x_msb_ind(2,0) = x_l(6,4);
104         ap_uint<4> x_lsb_ind;
105         x_lsb_ind(3,0) = x_l(3,0);
106         // ap_ufixed<4,-1> x_lsb = x_l;
107
108         const ap_ufixed<11,5> exp_x_msb_table[64] = {
109             22.765625, 20.078125,
110             17.71875, 15.640625, 13.796875, 12.1875,
111             10.75, 9.484375, 8.375, 7.390625,
112             6.515625, 5.75, 5.078125, 4.484375,
113             3.953125, 3.484375, 3.078125, 2.71875,
114             2.40625, 2.109375, 1.875, 1.65625,
115             1.453125, 1.28125, 1.140625, 1,
116             0.815625, 0.015625, 0.03125, 0.03125,
117             0.03125, 0.03125, 0.03125, 0.046875,
118             0.046875, 0.0625, 0.0625, 0.078125,
119             0.078125, 0.09375, 0.109375, 0.125,
120             0.140625, 0.15625, 0.171875, 0.203125,
121             0.21875, 0.25, 0.28125, 0.328125,
122             0.375, 0.421875, 0.46875, 0.53125,
123             0.609375, 0.6875, 0.78125, 0.875,
124             1, 1.140625, 1.28125, 1.453125, 1.65625,
125             1.875, 2.109375, 2.40625,
126             2.71875, 3.078125, 3.484375, 3.953125,
127             4.484375, 5.078125, 5.75, 6.515625,
128             7.390625, 8.375, 9.484375, 10.75, 12.1875,
129             13.796875, 15.640625, 17.71875,
130             20.078125, 22.765625, 25.796875, 29.21875,
131             33.109375, 37.53125, 42.515625, 48.1875,
132             0.015625, 0.015625, 0.03125, 0.03125,
133             0.03125, 0.03125, 0.03125, 0.046875,
134             0.046875, 0.0625, 0.0625, 0.078125,
135             0.078125, 0.09375, 0.109375, 0.125,
136             0.140625, 0.15625, 0.171875, 0.203125,
137             0.21875, 0.25, 0.28125, 0.328125,
138             0.375, 0.421875, 0.46875, 0.53125,
139             0.609375, 0.6875, 0.78125, 0.875,
140             };
141
142         const static ap_ufixed<12,0> exp_x_lsb_m_1_t-
143         able[16] = {
144             "0x0.0p0", "0x0.082p0", "0x0.108p0",
145             "0x0.193p0", "0x0.221p0", "0x0.2b5p0", "0x0.34dp0", "0x0.3eap0",
146             "0x0.48bp0", "0x0.532p0", "0x0.5dfp0",
147             "0x0.69p0", "0x0.748p0", "0x0.805p0", "0x0.8c8p0", "0x0.991p0",
148             };
149         ap_ufixed<12, 0> exp_x_lsb_m_1 = exp_x_lsb_m_1_t-
150         able[x_lsb_ind];
151
152         const static ap_ufixed<12,5> exp_x_msb_table[16]
153         = {
154             "0x1.0p0", "0x1.a6p0", "0x2.b8p0",
155             "0x4.7cp0", "0x7.64p0", "0xc.2ep0", "0x14.16p0", "0x21.1ep0",
156             "0x0.04p0", "0x0.08p0", "0x0.0cp0",
157             "0x0.16p0", "0x0.22p0", "0x0.3ap0", "0x0.5ep0", "0x0.9cp0",
158             };
159         ap_ufixed<12,5> exp_x_msb = exp_x_msb_t-
160         able[x_msb_ind];
161
162         ap_ufixed<24,5> y_lo = exp_x_msb * exp_x_lsb_m_1;
163         ap_ufixed<12,5> y_lo_s = y_lo;
164         ap_ufixed<12,5> y_l = y_lo_s + exp_x_msb;
165         y = y_l;
166     }
167     if (I_<6) { // overflow
168         bool overf = 0;
169         for (int j = 9; j >= I_+4; j--) {
170             #pragma HLS unroll
171             if (y[j])
172                 overf = 1;
173         }
174         if (overf) {
175             for (int j = 9; j >= I_+4; j--){
176                 #pragma HLS unroll
177                 y[j] = 0;
178             }
179             for (int j = I_+3; j >= 0; j--){
180                 #pragma HLS unroll
181                 y[j] = 1;
182             }
183         }
184     }
185     r = y;
186 } else if (I_s==4) { // I_s = 4, M_ = 6..11, I_ <= 12,
187 F_ <= 11, W <= 23
188     ap_fixed<23,12> x_l = x;
189     ap_ufixed<22,11> y = 0;
190     ap_fixed<4,4> x_l_int;
191     x_l_int(3,0) = x_l(14,11);
192     ap_ufixed<11,0> x_l_fract;
193     x_l_fract(10,0) = x_l(10,0);
194     bool overf = 0;

```

```

170     for (int j = 14; j < 22; j++){
171         #pragma HLS unroll
172         if (x_l[22]!=x_l[j])
173             overf = 1;
174     }
175     //if (overf||((x_l>"0x7.ap0")) { // overflow
176     if (overf||
177     ((x_l_int==7)&&(x_l_fract>ap_ufixed<11,0>("0x0.ap0")))) { // over-
178     flow
179         if (!x_l[22]) {
180             for (int j = 0; j < 22; j++){
181                 #pragma HLS unroll
182                 y[j] = 1;
183             }
184         }
185     } else {
186         ap_uint<5> x_msb_ind_1;
187         x_msb_ind_1[4] = x_l[22];
188         x_msb_ind_1(3,0) = x_l(13,10);
189         ap_uint<5> x_msb_ind_2;
190         x_msb_ind_2(4,0) = x_l(9,5);
191         // ap_ufixed<5,-1> x_msb_2 = x_l;
192         ap_uint<5> x_lsb_ind;
193         x_lsb_ind(4,0) = x_l(4,0);
194         //ap_ufixed<5,-6> x_lsb = x_l;
195
196         const static ap_ufixed<11,-13> f_x_lsb_table[32]
197         = {
198             "0x0.0p0", "0x0.000002p0", "0x0.000008p0",
199             "0x0.000012p0",
200             "0x0.00002p0", "0x0.000032p0",
201             "0x0.000048p0", "0x0.000062p0", "0x0.0000a2p0",
202             "0x0.0000c9p0", "0x0.0000f3p0",
203             "0x0.000121p0", "0x0.000153p0",
204             "0x0.000189p0", "0x0.0001c3p0",
205             "0x0.000202p0", "0x0.000244p0",
206             "0x0.00028ap0", "0x0.0002d4p0",
207             "0x0.000323p0", "0x0.000375p0", "0x0.0003c-
208             cp0", "0x0.000426p0",
209             "0x0.000485p0", "0x0.0004e7p0",
210             "0x0.00054ep0", "0x0.0005b9p0",
211             "0x0.000627p0", "0x0.00069ap0",
212             "0x0.000711p0", "0x0.00078cp0",
213             };
214         ap_ufixed<11,-13> f_x_lsb = f_x_lsb_t-
215         able[x_lsb_ind];
216         ap_ufixed<18, -6> exp_x_lsb_m_1 = 0;
217         exp_x_lsb_m_1(17,13) = x_lsb_ind(4,0);
218         exp_x_lsb_m_1(10,0) = f_x_lsb(10,0);
219
220         const static ap_ufixed<25,0> exp_x_msb_2_m_1_t-
221         able[32] = {
222             "0x0.0000000p0", "0x0.04080a8p0",
223             "0x0.0820560p0", "0x0.0c49238p0",
224             "0x0.1082b58p0", "0x0.14cd500p0",
225             "0x0.1929370p0", "0x0.1d96b10p0",
226             "0x0.2216048p0", "0x0.26a7790p0",
227             "0x0.2b4b588p0", "0x0.3001ed0p0",
228             "0x0.34cb818p0", "0x0.39a8628p0",
229             "0x0.3e98de8p0", "0x0.439d440p0",
230             "0x0.48b5e40p0", "0x0.4de30f0p0",
231             "0x0.5325180p0", "0x0.587c540p0",
232             "0x0.5de9178p0", "0x0.636bb98p0",
233             "0x0.6904930p0", "0x0.6eb3fc8p0",
234             "0x0.747a510p0", "0x0.7a57ee0p0",
235             "0x0.804d300p0", "0x0.865a778p0",
236             "0x0.8c80248p0", "0x0.92be998p0",
237             "0x0.99163b0p0", "0x0.9f876e8p0",
238             };
239         ap_ufixed<25, 0> exp_x_msb_2_m_1 =
240         exp_x_msb_2_m_1_table[x_msb_ind_2];
241
242         ap_ufixed<43,-6> f_x_msb_2_lsb = exp_x_msb_2_m_1
243         * exp_x_lsb_m_1;
244         ap_ufixed<19,-6> f_x_msb_2_lsb_s = f_x_msb_2_lsb;
245         ap_ufixed<25, 0> exp_x_msb_2_lsb_m_1 =
246         f_x_msb_2_lsb_s + exp_x_lsb_m_1 + exp_x_msb_2_m_1;
247
248         const static ap_ufixed<25,11> exp_x_msb_1_t-
249         able[32] = {
250             "0x1.0000p0", "0x1.a614p0", "0x2.b7e0p0",
251             "0x4.7b50p0",
252             "0x7.6398p0", "0xc.2eb8p0", "0x14.15e4p0",
253             "0x21.1d90p0",
254             "0x36.9920p0", "0x5a.0464p0", "0x94.69c4p0",
255             "0xf4.b124p0",
256             "0x193.6dc4p0", "0x299.2444p0",
257             "0x448.a218p0", "0x710.0adcp0",
258             "0x0.0014p0", "0x0.0024p0", "0x0.003cp0",
259             "0x0.0064p0",
260             "0x0.00a4p0", "0x0.010cp0", "0x0.01b8p0",
261             "0x0.02d8p0",
262             "0x0.04b0p0", "0x0.07bcp0", "0x0.0cc0p0",
263             "0x0.1504p0",
264             "0x0.22a4p0", "0x0.3920p0", "0x0.5e2cp0",
265             "0x0.9b44p0",
266             };
267         ap_ufixed<25,11> exp_x_msb_1 = exp_x_msb_1_t-
268         able[x_msb_ind_1];

```

```

236         ap_ufixed<50,11> y_lo = exp_x_msb_1 *
exp_x_msb_2_lsb_m_1;
237         ap_ufixed<25,11> y_lo_s = y_lo;
238         ap_ufixed<25,11> y_l = y_lo_s + exp_x_msb_1;
239         y = y_l;
240     }
241     if (I_<12) { // overflow
242         bool overf = 0;
243         for (int j = 21; j >= I_+10; j--) {
244             #pragma HLS unroll
245             if (y[j])
246                 overf = 1;
247         }
248         if (overf) {
249             for (int j = 21; j >= I_+10; j--) {
250                 #pragma HLS unroll
251                 y[j] = 0;
252             }
253             for (int j = I_+9; j >= 0; j--) {
254                 #pragma HLS unroll
255                 y[j] = 1;
256             }
257         }
258     }
259     r = y;

260 } else if (I_s==5) { // I_s = 5, M_ = 12..23, I_ <= 24,
F_ <= 23, W <= 47
261     ap_fixed<47,24> x_l = x;
262     ap_ufixed<46,23> y = 0;
263     ap_fixed<5,5> x_l_int;
264     x_l_int(4,0) = x_l(27,23);
265     ap_ufixed<23,0> x_l_fract;
266     x_l_fract(22,0) = x_l(22,0);
267     bool overf = 0;
268     for (int j = 27; j < 46; j++){
269         #pragma HLS unroll
270         if (x_l[46]!=x_l[j])
271             overf = 1;
272     }
273     //if (overf||(x_l=="0xf.f14028p0")) { // overflow
274     if (overf||
((x_l_int==15)&&(x_l_fract==ap_ufixed<23,0>("0x0.f14028p0")))) { //
overflow
275         if (!x_l[46]) {
276             for (int j = 0; j < 46; j++) {
277                 #pragma HLS unroll
278                 y[j] = 1;
279             }
280         }
281     } else {
282         ap_uint<8> x_msb_ind_1;
283         x_msb_ind_1[7] = x_l[46];
284         x_msb_ind_1(6,0) = x_l(26,20);
285         ap_uint<8> x_msb_ind_2;
286         x_msb_ind_2(7,0) = x_l(19,12);
287         ap_uint<5> x_msb_ind_3;
288         x_msb_ind_3(4,0) = x_l(11,7);
289         ap_uint<3> x_msb_ind_4;
290         x_msb_ind_4(2,0) = x_l(6,4);
291         ap_uint<4> x_lsb_ind;
292         x_lsb_ind(3,0) = x_l(3,0);
293
294         ap_ufixed<8, -3> x_msb_2 = x_l;
295         ap_ufixed<5, -11> x_msb_3 = x_l;
296         ap_ufixed<3, -16> x_msb_4 = x_l;
297         ap_ufixed<4, -19> x_lsb = x_l;
298
299         const static ap_ufixed< 6, -33> f_x_msb_4_h_t-
able[8] = {
300             "0x0.000000000000p0", "0x0.0000000002p0", "0x0.0000000008p0",
301             "0x0.0000000012p0",
302             "0x0.0000000020p0", "0x0.0000000032p0", "0x0.0000000048p0",
303             "0x0.0000000062p0",
304             };
305         const static ap_ufixed< 7, -51> f_x_msb_4_l_t-
able[8] = {
306             "0x0.00000000000000p0", "0x0.00000000000000p0",
307             "0x0.00000000000000cp0", "0x0.000000000000024p0",
308             "0x0.00000000000054p0", "0x0.000000000000a8p0",
309             "0x0.00000000000120p0", "0x0.000000000001c8p0",
310             };
311         ap_ufixed< 6, -33> f_x_msb_4_h = f_x_msb_4_h_t-
able[x_msb_ind_4];
312         ap_ufixed< 7, -51> f_x_msb_4_l = f_x_msb_4_l_t-
able[x_msb_ind_4];
313
314         const static ap_ufixed< 8, -39> f_x_lsb_table[16]
= {
315             "0x0.00000000000000p0", "0x0.00000000000002p0",
316             "0x0.00000000000008p0", "0x0.00000000000012p0",
317             "0x0.00000000000020p0", "0x0.00000000000032p0",
318             "0x0.00000000000048p0", "0x0.00000000000062p0",
319             "0x0.00000000000080p0", "0x0.000000000000a2p0",
320             "0x0.000000000000c8p0", "0x0.000000000000f2p0",
321             "0x0.00000000000120p0", "0x0.00000000000152p0",
322             "0x0.00000000000188p0", "0x0.000000000001c2p0",
323             };
324         ap_ufixed< 8, -39> f_x_lsb = f_x_lsb_t-
able[x_lsb_ind];

```

```

317
318         ap_ufixed< 7, -35> f_x_msb_4_lsb = x_msb_4 *
x_lsb;
319
320         ap_ufixed<10, -32> exp_x_msb_4_lsb_m_1_m = 0;
321         exp_x_msb_4_lsb_m_1_m(8,3) = f_x_msb_4_h(5,0);
322         exp_x_msb_4_lsb_m_1_m(2,0) = f_x_lsb(7,5);
323         exp_x_msb_4_lsb_m_1_m = exp_x_msb_4_lsb_m_1_m +
f_x_msb_4_lsb;
324
325         ap_ufixed<42, -16> exp_x_msb_4_lsb_m_1 = 0;
326         exp_x_msb_4_lsb_m_1(41,35) = x_l(6,0);
327         exp_x_msb_4_lsb_m_1(25,16) =
exp_x_msb_4_lsb_m_1_m(9,0);
328         exp_x_msb_4_lsb_m_1(15,11) = f_x_lsb(4,0);
329         exp_x_msb_4_lsb_m_1( 6, 0) = f_x_msb_4_l(6,0);
330
331         const static ap_ufixed<32, -23> f_x_msb_3_t-
able[32] = {
332             "0x0.00000000000000p0", "0x0.0000000000002ap0",
333             "0x0.00000000000156p0", "0x0.00000000000480p0",
334             "0x0.000000000000aaap0", "0x0.0000000000014d6p0",
335             "0x0.000000000001800392cp0",
336             "0x0.000000000005556p0", "0x0.0000000000028807982p0",
337             "0x0.00000000000a6acp0", "0x0.0000000000080dd8p0",
338             "0x0.0000000000048012004p0", "0x0.0000000000054816e30p0",
339             "0x0.00000000000201c95cp0", "0x0.000000000007823288p0",
340             "0x0.000000000002aab6p0", "0x0.000000000008332e2p0",
341             "0x0.000000000003cc12p0", "0x0.000000000004847740p0",
342             "0x0.00000000000c053570p0", "0x0.000000000008607a0p0",
343             "0x0.00000000000f206eed0p0", "0x0.0000000000087ec02p0",
344             "0x0.000000000000036p0", "0x0.000000000001388a2c6ap0",
345             "0x0.000000000001520b71a0p0", "0x0.0000000000016c8cd0d6p0",
346             "0x0.000000000001800e4b0ep0", "0x0.000000000001a48fe148p0",
347             "0x0.000000000001c2119484p0", "0x0.000000000001e9365c0p0",
348             };
349         ap_ufixed<32, -23> f_x_msb_3 = f_x_msb_3_t-
able[x_msb_ind_3];
350         ap_ufixed<44, -11> exp_x_msb_3_m_1 = 0;
351         exp_x_msb_3_m_1(43,39) = x_msb_3(4,0);
352         exp_x_msb_3_m_1(31, 0) = f_x_msb_3(31,0);
353
354         ap_ufixed<86, -27> f_x_msb_3_4_lsb =
exp_x_msb_3_m_1 * exp_x_msb_4_lsb_m_1;
355         ap_ufixed<31, -27> f_x_msb_3_4_lsb_s =
f_x_msb_3_4_lsb;
356         ap_ufixed<48, -10> exp_x_msb_3_4_lsb_m_1 =
f_x_msb_3_4_lsb_s + exp_x_msb_3_m_1 + exp_x_msb_4_lsb_m_1;
357
358         const static ap_ufixed<46, -6> f_x_msb_2_t-
able[256] = {
359             "0x0", "0x0.0000000000015560p0", "0x0.00000000000AAB55p0",
360             "0x0.00000000000204360p0",
361             "0x0.00000000000556001p0", "0x0.00000000000320A6C4B8p0",
362             "0x0.000000000001203608p0", "0x0.00000000000621C9B971p0",
363             "0x0.00000000000802AB5577p0", "0x0.00000000000A23CD119Dp0",
364             "0x0.00000000000C8536F668p0", "0x0.00000000000F26F10D5Dp0",
365             "0x0.000000000001209036103p0", "0x0.00000000000152B75FCE3p0",
366             "0x0.00000000000188E50D86p0", "0x0.000000000001C319C4077p0",
367             "0x0.000000000002015600445p0", "0x0.000000000002439A4487Ep0",
368             "0x0.00000000000289E711DB3p0", "0x0.000000000002D43CE9577p0",
369             "0x0.000000000003229C4C260p0", "0x0.0000000000037505BB805p0",
370             "0x0.000000000003CB79B8B01p0", "0x0.00000000000425F8C50F2p0",
371             "0x0.000000000004848362076p0", "0x0.000000000004E71A11131p0", "0x0.0000000000054DB-
D53BC8p0", "0x0.000000000005B86DAB9E3p0",
372             "0x0.000000000006272B9A630p0", "0x0.00000000000699F7A1C5Dp0",
373             "0x0.00000000000710D24391Ep0", "0x0.0000000000078BB01A29p0",
374             "0x0.0000000000080AB55DE39p0", "0x0.00000000000808BEDA508p0",
375             "0x0.00000000000914D8F8F63p0", "0x0.000000000009A0043BF07p0",
376             "0x0.00000000000A2F41256C2p0", "0x0.00000000000AC29037A63p0",
377             "0x0.00000000000B59F1F4EDBp0", "0x0.00000000000BF566DF9AAp0",
378             "0x0.00000000000C94EF7A206p0", "0x0.00000000000D38C46F4p0", "0x0.00000000000DE03D-
C8B9Bp0", "0x0.00000000000E8C0481FA7p0",
379             "0x0.00000000000F3BE0F56CAp0", "0x0.00000000000FEFD3A5CFAP0", "0x0.0010A7D-
D15F36p0", "0x0.001163FDC8B7Fp0",
380             "0x0.00122436410DDp0", "0x0.0012E88701F5Ep0",
381             "0x0.0013B0F08E817p0", "0x0.00147D7369D22p0", "0x0.0016F-
B98F4A96p0", "0x0.0017D8862BA72p0",
382             "0x0.0018B98F42084p0", "0x0.00199BE48B511p0",
383             "0x0.001A87F71B161p0", "0x0.001B7556E4FC4p0",
384             "0x0.001C6D49C9B3p0", "0x0.001D5C70C612Dp0",
385             "0x0.001E562BE4DFAp0", "0x0.001F54067D067p0",
386             "0x0.00205601127ECp0", "0x0.00215C1C29507p0",
387             "0x0.0022665845940p0", "0x0.00237485EB724p0",
388             "0x0.002487359F24Cp0", "0x0.00259DD7E4F57p0",
389             "0x0.0026B89D413EFp0", "0x0.0027D786386C4p0",
390             "0x0.0028FA934EF90p0", "0x0.002A21C509717p0",
391             "0x0.002B4D1BEC726p0", "0x0.002C7C987CA92p0",
392             "0x0.002D0B3B3ED3Ap0", "0x0.002EE80487C07p0",
393             "0x0.003023F56C4ECp0", "0x0.0031640DE16E3p0",
394             "0x0.0032A84E9C1F5p0", "0x0.0033F6B821730p0",
395             "0x0.00353D4AF68B0p0", "0x0.00368E07A0999p0",
396             "0x0.0037E2EEA4E19p0", "0x0.00393C0088B6Cp0", "0x0.003A993D-
D17D5p0", "0x0.003BFAA704AA3p0",
397             "0x0.003D603CA7C32p0", "0x0.003EC9FF405E6p0", "0x0.004037E-
F54230p0", "0x0.0041AA0D68C8Cp0",

```

```

377 "0x0.0043205A04182p0", "0x0.00449AD5ABEA5p0",
"0x0.00461980E6294p0", "0x0.00479C5C38CF9p0",
378 "0x0.0049236829E8Bp0", "0x0.004AAEA53F90Ep0",
"0x0.004C3E13FFF51p0", "0x0.004DD1B4F152Fp0",
379 "0x0.004F698899F90p0", "0x0.0051058F8046Ap0",
"0x0.0052A5CA2AABFp0", "0x0.00544A391FA9Cp0",
380 "0x0.0055F2DCE5D1Ep0", "0x0.00579FB603C6Ep0",
"0x0.005950C5003C2p0", "0x0.005806A61F5Dp0",
381 "0x0.005CBF86AFC91p0", "0x0.005E7D3A709BEp0",
"0x0.00603F262B650p0", "0x0.0062054A672C2p0",
382 "0x0.0063CFA7AB09Dp0", "0x0.00659E3E7E278p0",
"0x0.0067710F67BFAp0", "0x0.0069481AEF1D5p0",
383 "0x0.006B23619B9CFp0", "0x0.006D02E3F4AB7p0",
"0x0.006EE6A281C6Fp0", "0x0.0070CE9DCA7E6p0",
384 "0x0.0072BAD65671Bp0", "0x0.0074AB4CAD51Cp0",
"0x0.0076A00156E07p0", "0x0.007898F4DAF09p0",
385 "0x0.007A9627C165Fp0", "0x0.007C979A92356p0", "0x0.007E9D4D-
D564Ap0", "0x0.0080A742130A8p0",
386 "0x0.0082B577D34EDp0", "0x0.0084C7EF9E6A7p0",
"0x0.0086DEA9FCA73p0", "0x0.0088FA977601p0",
387 "0x0.008B18E894010p0", "0x0.008D3C6DDE06Fp0", "0x0.008F6437D-
D000p0", "0x0.00919047198B6p0",
388 "0x0.0093C09C1C595p0", "0x0.0095F5376E2B2p0",
"0x0.0098E21997D33p0", "0x0.009A684322352p0",
389 "0x0.009CACA8496458p0", "0x0.009EF26E7D0A2p0",
"0x0.00A13C715F99Ep0", "0x0.00A38ABDC71CEp0",
390 "0x0.00A5DD543CC4p0", "0x0.00A8343549F26p0",
"0x0.00AA8F6177EADp0", "0x0.00ACEED950222p0",
391 "0x0.00AF529D5C165p0", "0x0.00B1BAE25566p0",
"0x0.00B4270C3582Ap0", "0x0.00B69788164C7p0",
392 "0x0.00B90CB25176Ap0", "0x0.00BB85FB70D50p0",
"0x0.00BE0393FE4CCp0", "0x0.00C0857C83D44p0",
393 "0x0.00C30BB58B732p0", "0x0.00C5963F9F424p0",
"0x0.00C8251B496BEp0", "0x0.00CAB849142B5p0",
394 "0x0.00CD4FC989CD6p0", "0x0.00CFEB9D34B00p0", "0x0.00D28B-
C49F428p0", "0x0.00D5304054059p0",
395 "0x0.00D7D10D08B1p0", "0x0.00DA8636C6764p0", "0x0.00D-
D37B2997BCp0", "0x0.00DFEDB84E1618p0",
396 "0x0.00E2A7AE28FECp0", "0x0.00E5662EFB3C3p0",
"0x0.00E82907E313Dp0", "0x0.00EAF0396B910p0",
397 "0x0.00EDBBC41FD08p0", "0x0.00F08BA88B009p0",
"0x0.00F35FE73860Bp0", "0x0.00F63880B341Ep0",
398 "0x0.00F9157587069p0", "0x0.00FBF6C63F228p0", "0x0.00FED-
C37671B0p0", "0x0.0101C67D8A86Cp0",
399 "0x0.0104B4E5350DFp0", "0x0.0107A7AAF26A2p0",
"0x0.010A9ECF4E667p0", "0x0.0109A52D04DF8p0",
400 "0x0.01109A3611C34p0", "0x0.01139E7991116p0", "0x0.0116A71D-
DEDADp0", "0x0.0119B42387423p0",
401 "0x0.011CC58B167B9p0", "0x0.011FDB5518CCAp0",
"0x0.0122F5821A8C7p0", "0x0.01261412A823Cp0",
402 "0x0.012937074E0CDp0", "0x0.012C5E6098D37p0",
"0x0.012F8A1F15151p0", "0x0.0132BA434F80Ap0",
403 "0x0.0135EECD04D6Cp0", "0x0.013927BF31E98p0",
"0x0.013C6517F39CPp0", "0x0.013FA6D8A6E5Dp0",
404 "0x0.0142ED01D8CBp0", "0x0.0146379416673p0", "0x0.0149868FE-
CE26p0", "0x0.014CD9F5E9795p0",
405 "0x0.015031C699799p0", "0x0.01538E028A426p0",
"0x0.0156EEAA4944Bp0", "0x0.015A53BE64033p0",
406 "0x0.015DBD3F68122p0", "0x0.01612B2D2E3178p0",
"0x0.01649D8A62CB1p0", "0x0.0168145574F65p0",
407 "0x0.0168B8F8FA7745p0", "0x0.016F0F3988321p0",
"0x0.01729353A52E4p0", "0x0.01761BDE8C792p0",
408 "0x0.0179A8DAC3509p0", "0x0.017D3A48F295Dp0",
"0x0.0180D0298DE13p0", "0x0.01846A7D2C6ECp0",
409 "0x0.018809445CA7Bp0", "0x0.018BAC7FAD074p0", "0x0.018F542-
FAC1A4p0", "0x0.01930054E87F8p0",
410 "0x0.0196B0EFF0E79p0", "0x0.019A66015414Dp0",
"0x0.019E1F89A0DBAp0", "0x0.01A1D88966221p0",
411 "0x0.01A5A00132E02p0", "0x0.01A966F1961FCp0",
"0x0.01AD325B1EFC9p0", "0x0.01B1023E5CA45p0",
412 "0x0.01B4D69BD569p0", "0x0.01B8AF743364Cp0", "0x0.01BC8CC7E-
B323p0", "0x0.01C06E9795345p0",
413 "0x0.01C454E3C0F24p0", "0x0.01C83FACFE054p0", "0x0.01CC2EF3D-
C187p0", "0x0.01D022B8EAE8Fp0",
414 "0x0.01D41AFCBA45Ep0", "0x0.01D817BFDA104p0", "0x0.01D-
C1902DA3B1p0", "0x0.01E01EC64ACB7p0",
415 "0x0.01E4290ABBD87p0", "0x0.01E837D0BD8B2p0",
"0x0.01EC4B18E01E9p0", "0x0.01F062E3B3DFEp0",
416 "0x0.01F47F31C92E4p0", "0x0.01F8A003B07AFp0", "0x0.01FCC559-
FA492p0", "0x0.0200EF35372E4p0",
417 "0x0.02051D95F7D1Cp0", "0x0.0209507CCCE1Dp0",
"0x0.020D87EA474BEp0", "0x0.0211C3D0EF7C8Dp0",
418 };
419 ap_ufixed<46,-6> f_x_msb_2 = f_x_msb_2_t-
able[x_msb_ind_2];
420
421 ap_ufixed< 5,-6> f_x_msb_2_h;
422 f_x_msb_2_h(4,0) = f_x_msb_2(45,41);
423 ap_ufixed< 9,-2> exp_x_msb_2_m_1_h = f_x_msb_2_h
+ x_msb_2;
424
425 ap_ufixed<50,-2> exp_x_msb_2_m_1;
426 exp_x_msb_2_m_1(49,41) = exp_x_msb_2_m_1_h(8,0);
427 exp_x_msb_2_m_1(40, 0) = f_x_msb_2(40,0);
428
429 ap_ufixed<98,-12> f_x_msb_2_3_4_lsb =
exp_x_msb_2_m_1 * exp_x_msb_3_4_lsb_m_1;
430 ap_ufixed<46,-12> f_x_msb_2_3_4_lsb_s =
f_x_msb_2_3_4_lsb;
431
432 ap_ufixed<50,-2> exp_x_msb_2_3_4_lsb_m_1 =
f_x_msb_2_3_4_lsb_s + exp_x_msb_2_m_1 + exp_x_msb_3_4_lsb_m_1;
433

```

```

431 const static ap_ufixed<50,23> exp_x_msb_1_t-
able[256] = {
432 "0x1.00000000p0", "0x1.2216045p0", "0x1.48B5E3Cp0",
"0x1.747A513p0",
433 "0x1.A61298Ep0", "0x1.DE455DFp0", "0x2.1DF3B68p0",
"0x2.661CB0Fp0",
434 "0x2.B7E1516p0", "0x3.1489176p0", "0x3.7D871DBp0",
"0x3.F47FE87p0",
435 "0x4.7B4F799p0", "0x5.1413452p0", "0x5.C12DA41p0",
"0x6.855466Ep0",
436 "0x7.63992E3p0", "0x8.5F7635Bp0", "0x9.7CDC417p0",
"0xA.C042667p0",
437 "0xC.2EB7EC9p0", "0xD.CDF892Ep0", "0xF.A48385Ep0",
"0x11.B985652p0",
438 "0x14.15E5BF6p0", "0x16.C2887C1p0", "0x19.CA53B78p0", "0x1D.
396AA97p0",
439 "0x21.1D8E427p0", "0x25.865441Cp0", "0x2A.8565A14p0",
"0x30.2EC550Bp0",
440 "0x36.99205C4p0", "0x3D.DE28BF3p0", "0x46.1AFC4DBp0", "0x4F.
7099532p0",
441 "0x5A.0462B78p0", "0x66.00B5BC9p0", "0x73.9593ABBp0",
"0x82.F9621ADp0",
442 "0x94.69C4CB8p0", "0xA8.2C948C1p0", "0xBE.90F6F83p0",
"0xD7.F09B78Cp0",
443 "0xF4.B122790p0", "0x115.45B4704p0", "0x13A.30CF1CCp0",
"0x164.0650296p0",
444 "0x193.6DC5690p0", "0x1C9.250BEDCp0", "0x206.03487A3p0",
"0x24A.FC4533Cp0",
445 "0x299.2442102p0", "0x2F1.B447462p0", "0x356.0F0B0F9p0",
"0x3C7.C67E5BEp0",
446 "0x448.A216ABBp0", "0x4DA.A5EE46Ap0", "0x580.1AD754Fp0",
"0x63B.9782341p0",
447 "0x710.0ADBAC7p0", "0x800.C7CC8E3p0", "0x911.9289C39p0", "0x-
A46.AFAA2ADp0",
448 "0xBA4.F53EA38p0", "0xD31.DE30C52p0", "0xEF3.A035D57p0",
"0x10F1.44ADE60p0",
449 "0x1332.C4D2B7Cp0", "0x15C1.29A744Cp0", "0x18A6.B027DA0p0",
"0x1BEE.F24BB12p0",
450 "0x1FA7.157C470p0", "0x23DD.FF3C8BAp0", "0x28A4.90D2CCEp0",
"0x2E0D.EAE454Bp0",
451 "0x342F.BA11823p0", "0x3B22.8DC5359p0", "0x4302.3A933EEp0",
"0x4BEE.49AFCDCp0",
452 "0x560A.773E541p0", "0x617F.4171BF9p0", "0x6E7A.8ABB4A8p0",
"0x7D30.5191764p0",
453 "0x8DB8.80AF426p0", "0xA0BE.DB0DB10p0", "0xB626.0748BAEp0",
"0xCE66.BE9EADp0",
454 "0xE9E2.2447727p0", "0x10906.4A831F8p0", "0x12C4F.EB75B2Ap0",
"0x1544C.5CB6133p0",
455 "0x1819B.C560F61p0", "0x1B4F3.9F8AF64p0", "0x1E-
F21.8F1B001p0", "0x2310E.996C40Fp0",
456 "0x27BC2.CA9A6F9p0", "0x2D069.571279Ap0",
"0x33055.49F35D9p0", "0x39D06.D2FEF17p0",
457 "0x41831.49596E9p0", "0x4A3C1.FB2AC39p0",
"0x541E7.F56949Dp0", "0x5F51C.72B7532p0",
458 "0x6C02D.645AB25p0", "0x7A648.0CFF534p0", "0x8-
AB06.0A3EE9Fp0", "0x9D27B.AFE4CF1p0",
459 "0xB2148.5EAE56Cp0", "0xC9CA9.07F86EFp0", "0xE4A8D.2881ED-
Bp0", "0x1031AE.8E4996Cp0",
460 "0x1259AC.48BF05Dp0", "0x14CB29.2F2B31Fp0", "0x178FEE.
7792E44p0", "0x1AB312.E89CD0Dp0",
461 "0x1E4127.437732Bp0", "0x224868.979FC2Ep0",
"0x26D8F9.4A204BEp0", "0x2C0521.B4A8E0Bp0",
462 "0x31E199.5F5A550p0", "0x3885D9.FA89D00p0", "0x400C7D.
64D3386p0", "0x4893A8.361032Dp0",
463 "0x523D82.79EDAEPp0", "0x5D30C0.7DAB686p0",
"0x69993D.D4F2D95p0", "0x77A8AD.02A7C71p0",
464 "0x0.000001Ep0", "0x0.0000022p0", "0x0.0000026p0",
"0x0.000002Bp0",
465 "0x0.0000031p0", "0x0.0000038p0", "0x0.000003Fp0",
"0x0.0000048p0",
466 "0x0.0000052p0", "0x0.000005Dp0", "0x0.0000069p0",
"0x0.0000077p0",
467 "0x0.0000087p0", "0x0.0000099p0", "0x0.00000ADp0",
"0x0.00000C4p0",
468 "0x0.00000DFp0", "0x0.00000FCp0", "0x0.000011Ep0",
"0x0.0000144p0",
469 "0x0.0000170p0", "0x0.00001A1p0", "0x0.00001D8p0",
"0x0.0000217p0",
470 "0x0.000025Ep0", "0x0.00002AFp0", "0x0.000030Bp0",
"0x0.0000372p0",
471 "0x0.00003E8p0", "0x0.000046Dp0", "0x0.0000504p0",
"0x0.00005AFp0",
472 "0x0.0000671p0", "0x0.000074Cp0", "0x0.0000845p0",
"0x0.000095Fp0",
473 "0x0.0000A9Fp0", "0x0.0000C09p0", "0x0.0000DA3p0",
"0x0.0000F74p0",
474 "0x0.0001183p0", "0x0.00013D8p0", "0x0.000167Cp0",
"0x0.000197Bp0",
475 "0x0.0001CDFp0", "0x0.00020B7p0", "0x0.0002513p0",
"0x0.0002A02p0",
476 "0x0.0002F9Ap0", "0x0.00035F1p0", "0x0.0003D20p0",
"0x0.0004543p0",
477 "0x0.0004E7Cp0", "0x0.00058F0p0", "0x0.00064C7p0",
"0x0.0007232p0",
478 "0x0.0008167p0", "0x0.00092A2p0", "0x0.000A628p0", "0x0.000B-
C48p0",
479 "0x0.000D55Ap0", "0x0.000F1C2p0", "0x0.00111F3p0",
"0x0.001366Cp0",
480 "0x0.0015FC2p0", "0x0.0018E98p0", "0x0.001C3AAp0",

```



```

"0x0.001FFCEp0",
481 "0x0.00243F3p0", "0x0.002912Bp0", "0x0.002E8ABp0",
"0x0.0034BD2p0",
482 "0x0.003BC2Dp0", "0x0.0043B7Dp0", "0x0.004CBC1p0",
"0x0.0056F3Ap0",
483 "0x0.0062878p0", "0x0.006FA5Fp0", "0x0.007E83Ap0",
"0x0.008F5C0p0",
484 "0x0.00A2728p0", "0x0.00B813Bp0", "0x0.00D0963p0",
"0x0.00EC5C1p0",
485 "0x0.010BD4Ap0", "0x0.012F7DEp0", "0x0.0157E6Bp0",
"0x0.0185B0Fp0",
486 "0x0.01B993Fp0", "0x0.01F45F9p0", "0x0.0236FF5p0",
"0x0.02827E0p0",
487 "0x0.02D80A0p0", "0x0.0338F9Fp0", "0x0.03A6D22p0",
"0x0.04234A7p0",
488 "0x0.04B0556p0", "0x0.0550280p0", "0x0.0605424p0",
"0x0.06D279Ap0",
489 "0x0.07BB040p0", "0x0.08C284Cp0", "0x0.09ED1B4p0",
"0x0.0B3F736p0",
490 "0x0.0CBED86p0", "0x0.0E7149Cp0", "0x0.105D938p0",
"0x0.128B697p0",
491 "0x0.150385Cp0", "0x0.17CFC2p0", "0x0.1AFB718p0",
"0x0.1E9328Bp0",
492 "0x0.22A554p0", "0x0.2742456p0", "0x0.2C7C72Fp0",
"0x0.3268CDBp0",
493 "0x0.391F0EEp0", "0x0.40BA188p0", "0x0.495860Dp0",
"0x0.531C6C9p0",
494 "0x0.5E2D58Dp0", "0x0.6AB7782p0", "0x0.78ED03Ap0",
"0x0.8906E49p0",
495 "0x0.9B4597Ep0", "0x0.AFF230Ap0", "0x0.C75F7CFp0", "0x0.E1E-
B512p0",
496     };
497     ap_ufixed<50,23> exp_x_msb_1 = exp_x_msb_1_t-
able[x_msb_ind_1];
498     ap_ufixed<100,21> y_lo = exp_x_msb_1 *
exp_x_msb_2_3_4_lsb_m_1;
499     ap_ufixed<48,21> y_lo_s = y_lo;
500     ap_ufixed<50,23> y_l = y_lo_s + exp_x_msb_1;
501     y = y_l;
502 }
503 if (I_<24) { // overflow
504     bool overf = 0;
505     for (int j = 45; j >= I_+22; j--) {
506         #pragma HLS unroll
507         if (y[j])
508             overf = 1;
509     }
510     if (overf) {
511         for (int j = 45; j >= I_+22; j--){
512             #pragma HLS unroll
513             y[j] = 0;
514         }
515         for (int j = I_+21; j >= 0; j--) {
516             #pragma HLS unroll
517             y[j] = 1;
518         }
519     }
520 }
521 r = y;
522 } else { // I_s_ = 6, M_ = 24..32, I_ <= 33, F_ <= 32, W_
<= 63
523     //ap_fixed<65,33> x_l = x;
524     ap_fixed<65,33> x_l = 0;
525     for (int j = 32-F_; j < 32+I_; j++){
526         #pragma HLS unroll
527         x_l[j] = x[j-(32-F_)];
528     }
529     for (int j = 32+I_; j <= 64; j++){
530         #pragma HLS unroll
531         x_l[j] = x[W_-1];
532     }
533     ap_ufixed<64,32> y = 0;
534     ap_fixed<6,6> x_l_int;
535     x_l_int(5,0) = x_l(37,32);
536     ap_ufixed<32,0> x_l_fract;
537     x_l_fract(31,0) = x_l(31,0);
538     bool overf = 0;
539     for (int j = 37; j < 64; j++) {
540         #pragma HLS unroll
541         if (x_l[64]!=x_l[j])
542             overf = 1;
543     }
544     //if (overf||(x_l=="0x16.2e42fefap0")) { // overflow
545     if (overf||(x_l_int>22)||
(x_l_int==22)&&(x_l_fract==ap_ufixed<32,0>("0x0.2e42fefap0")))
{ // overflow
546         if (!x_l[64]) {
547             for (int j = 0; j < 64; j++){
548                 #pragma HLS unroll
549                 y[j] = 1;
550             }
551         }
552     } else {
553         ap_uint<8> x_msb_ind_1;
554         x_msb_ind_1[7] = x_l[64];
555         x_msb_ind_1(6,0) = x_l(36,30);
556         ap_uint<8> x_msb_ind_2;
557         x_msb_ind_2(7,0) = x_l(29,22);
558         ap_uint<8> x_msb_ind_3;
559         x_msb_ind_3(7,0) = x_l(21,14);
560         ap_uint<8> x_msb_ind_4;
561         x_msb_ind_4(7,0) = x_l(13,6);
562         ap_uint<6> x_lsb_ind;
563         x_lsb_ind(5,0) = x_l(5,0);
564
565         ap_ufixed<8,-2> x_msb_2 = x_l;
566         ap_ufixed<8,-10> x_msb_3 = x_l;
567         ap_ufixed<8,-18> x_msb_4 = x_l;
568         ap_ufixed<6,-26> x_lsb = x_l;
569
570         const static ap_ufixed<31,-37> f_x_msb_4_t-
able[256] = {
571     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
572     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
573     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
574     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
575     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
576     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
577     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
578     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
579     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
580     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
581     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
582     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
583     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
584     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
585     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
586     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
587     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
588     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
589     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
590     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
591     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
592     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
593     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
594     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
595     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
596     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
597     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
598     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
599     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
600     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
601     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
602     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
603     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
604     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
605     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
606     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
607     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
608     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
609     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
610     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
611     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
612     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
613     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",
614     "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000000p0", "0x0.0000000000000000p0",

```

```
669      exp x msb 4 lsb m 1 m3 = exp x msb 4 lsb m 1 m3 +
```

"0x0.0000038751FC586A18p0", "0x0.00000391FADF81B02Fp0",

```

723 "0x0.0000039CB3C5572F67p0", "0x0.000003A77CADDCE86Ap0",
"0x0.000003B2559916DB4p0", "0x0.000003BD3E87090A84p0",
724 "0x0.000003C83777B774F8p0", "0x0.000003D3406B261BF0p0",
"0x0.000003E5961590011p0", "0x0.000003E9825A54222dp0",
725 "0x0.000003F48B5618B2D7p0", "0x0.00000400904548322CCp0",
"0x0.000004B5D561F02C2p0", "0x0.00000416C65A63236Ep0",
726 "0x0.000004223F61838586p0", "0x0.0000042C8B68429C2p0",
"0x0.0000043961786910DBp0", "0x0.000004450A883638BAp0",
727 "0x0.00000450C39AEFAA8Ap0", "0x0.0000045C8C0995E96p0",
"0x0.000004685C937586Bp0", "0x0.000004744EE4C98C6p0",
728 "0x0.000004804803602067p0", "0x0.0000048C5124F2F00Cp0",
"0x0.000004986A498A0876p0", "0x0.000004A49371296A67p0",
729 "0x0.000004B08CC9B0516A1p0", "0x0.000004BD15C9910DE8p0",
"0x0.000004C96EFA615101p0", "0x0.000004D5D82E49E0B0p0",
730 "0x0.000004E251654EBBDBp0", "0x0.000004EEDA9F73E8F0p0",
"0x0.000004FB73DCB06310p0", "0x0.000005081D1D2F2CE7p0",
731 "0x0.00000514D060CD4740p0", "0x0.000005219FA79BBE27p0",
"0x0.0000052E78F19E70A8p0", "0x0.0000053B623ED98150p0",
732 "0x0.000005485B8F50E5AFp0", "0x0.0000055564E3089E93p0",
"0x0.000005627E3A04ACp0", "0x0.0000056FA794491131p0",
733 "0x0.0000057CE0F1D9C8Ep0", "0x0.0000058A2A52BA0FBAp0",
"0x0.0000059783BF04A88p0", "0x0.000005A4ED1E7E10CFp0",
734 "0x0.000005B2668983064p0", "0x0.000005BFF7B2AB20p0",
"0x0.000005CD89696181DBp0", "0x0.000005DB32DE78B56Fp0",
735 "0x0.000005E8C5E6FC46B6p0", "0x0.000005F6B5D2F0368Cp0",
"0x0.000006048F525885CDp0", "0x0.0000061278D5393558p0",
736 "0x0.00000620725B96460Ap0", "0x0.0000062E7BE57388C3p0",
"0x0.0000063C9572D58E64p0", "0x0.0000064ABF03BFC7Cp0",
737 "0x0.00000658F8983665E5p0", "0x0.0000066742303D698Bp0",
"0x0.0000067598CB08D3A5p0", "0x0.00000684056B0CA519p0",
738 "0x0.000006927F0DDCDECEp0", "0x0.000006A108B44D81AAp0",
"0x0.000006AFA25628E98p0", "0x0.000006BE4C0C20067Fp0",
739 "0x0.000006CD058D89EA4Bp0", "0x0.000006DBCF72A43AE8p0",
"0x0.000006EAA92B72F942p0", "0x0.000006F992E7FA2646p0",
740 "0x0.000007088CA83CD2E4p0", "0x0.00000717966C41D00Ap0",
"0x0.00000726B0340AAEAp0", "0x0.00000735D9F9B3F86p0",
741 "0x0.0000074513CEFA8A1Ep0", "0x0.000007545DA2267C9p0",
"0x0.00000763B77928CA09p0", "0x0.000007732154038F14p0",
742 "0x0.000007829B32BACA82p0", "0x0.000007922515527E1Ap0",
"0x0.000007A1BEFBCAEA04p0", "0x0.000007B16863351AAp0",
743 "0x0.000007C122D447397p0", "0x0.000007DECC6C61195p0",
"0x0.000007E0C6BCFC2A3Ap0", "0x0.000007F0B0B72AC5BEP0",
744
745 ap_ufixed<51, -21> f_x_msb_3 = f_x_msb_3_t-
able[x_msb_ind_3];
746
747 ap_ufixed<62, -10> exp_x_msb_3_m_1 = 0;
748 exp_x_msb_3_m_1(61,54) = x_msb_3(7,0);
749 exp_x_msb_3_m_1(50, 0) = f_x_msb_3(50,0);
750
751 ap_ufixed<50, -10> exp_x_msb_3_m_1_s =
exp_x_msb_3_m_1;
752 ap_ufixed<100, -28> f_x_msb_3_4_lsb =
exp_x_msb_3_m_1_s * exp_x_msb_4_lsb_m_1;
753 ap_ufixed<44, -28> f_x_msb_3_4_lsb_s =
f_x_msb_3_4_lsb;
754 ap_ufixed<63, -9> exp_x_msb_3_4_lsb_m_1 =
f_x_msb_3_4_lsb_s + exp_x_msb_3_m_1 + exp_x_msb_4_lsb_m_1;
755
756 const static ap_ufixed<64, -4> f_x_msb_2_t-
able[256] = {
757 "0x0.0p0", "0x0.00000800AAB555DDEp0",
"0x0.00002005560011127p0", "0x0.00004812036081A9Cp0",
758 "0x0.0000802AB55777D28p0", "0x0.00008536F6684062p0",
"0x0.00012090361037400p0", "0x0.000188E50ED8634A0p0",
759 "0x0.000201560044580C3p0", "0x0.000289E711DB32FD7p0",
"0x0.0003229C4C260197Ep0", "0x0.0003CB798B801FE26p0",
760 "0x0.0004848362976A08DBp0", "0x0.00054DBD53BC80058p0",
"0x0.0006272B9A6306590p0", "0x0.000710D24391E6D7Ap0",
761 "0x0.00080AB55D6E3917ABp0", "0x0.000914D8F8F63D524p0",
"0x0.000A2F41256C297AFp0", "0x0.000B59F1F4EBDE291p0",
762 "0x0.000C94EF7A206DC2Dp0", "0x0.000DE03DC8B9B60B1p0",
"0x0.000F3BE0F56CA15C4p0", "0x0.0010A7DD15F367F40p0",
763 "0x0.00122436410DD14E5p0", "0x0.0013B0F08E817591Ep0",
"0x0.00154E101719FF0C5p0", "0x0.0016FB98F4A96BBEFp0",
764 "0x0.0018B98F42084EFDp0", "0x0.001A87F71B1613137p0",
"0x0.001C66D49CB93B127p0", "0x0.001E562BE4DFA4904p0",
765 "0x0.00205601127EC98E0p0", "0x0.00226658459402659p0",
"0x0.002487359F24C7C99p0", "0x0.0026889D413EF4D5Ep0",
766 "0x0.0028FA934EF909304p0", "0x0.002B4D1BEC726B39Cp0",
"0x0.002DB03B3ED3AA50Ep0", "0x0.003023F56C4EC123Fp0",
767 "0x0.0032A84E9C1F58145p0", "0x0.00353D4AF68B07AA1p0",
"0x0.0037E2EA4E19B185p0", "0x0.003A993DD17D5D2D5p0",
768 "0x0.003D603CA7C32730Fp0", "0x0.004037EF54230B293p0",
"0x0.0043205A04182F12Fp0", "0x0.00461980E62943810p0",
769 "0x0.0049236829E8BC292p0", "0x0.004C3E13FF5512D08p0",
"0x0.004F698899F90A96p0", "0x0.0052A5CA2AABF28D0p0",
770 "0x0.0055F2DCE5D1E966Ep0", "0x0.005905C5003C20723p0",
"0x0.005CBF86AFC91EF28p0", "0x0.00603F262B65057FCp0",
771 "0x0.0063CA7AAB09D1732p0", "0x0.0067710F67BFA0687p0",
"0x0.006B23619B9CF3CDEp0", "0x0.006EE6A281C6F4857p0",
772 "0x0.0072BAD65671B6977p0", "0x0.0076A0015607CF57p0",
"0x0.007A9627165FD4DEp0", "0x0.007E9D4DD564A3F5Ep0",
773 "0x0.0082B577D3AED7D5Bp0", "0x0.0086DEA9FA73E799p0",
"0x0.008B18E894010025p0", "0x0.008F6437DD000BFDp0",
774 "0x0.0093C09C1C595C43Ep0", "0x0.00982E1997D33A9FCp0",
"0x0.009CACB49645847CCp0", "0x0.00A13C715F99EF773p0",
775 "0x0.00A5DD543CC4DDAFp0", "0x0.00AA8F6177EAD3336p0",
"0x0.00AF529D5C1658EBp0", "0x0.00B4270C3582A301Dp0",
776 "0x0.00B90CB25176A4C8Bp0", "0x0.00BE0393FEACC5BD7p0",

```

```

"0x0.00C30BB58B73266CAp0", "0x0.00C8251B496BE5696p0",
777 "0x0.00CD4FC989CD64555p0", "0x0.00D28BC49F428CFA3p0",
"0x0.00D7D910DD8B16743p0", "0x0.00DD3B2997BCA6E6p0",
778 "0x0.00E2A7AE28FEC46FBp0", "0x0.00E69E7F313D5399p0",
"0x0.00EDB8C41FD08C383p0", "0x0.00F35FE73860B9038p0",
779 "0x0.00F91575870692F24p0", "0x0.00FEDC73671B04BDFp0",
"0x0.0104B4E5350DF2386p0", "0x0.010A9ECF4667E12Dp0",
780 "0x0.01109A3611C34FB64p0", "0x0.0116A71DDDEDAD92D8p0",
"0x0.011CC58B167B9D206p0", "0x0.0122F5821A8C74E0Dp0",
781 "0x0.012937074E0CD6893p0", "0x0.012F8A1F15151B2C6p0",
"0x0.0135EECD4D6C5172p0", "0x0.013C6517F39CC6233p0",
782 "0x0.0142ED01D8CB6C1C4p0", "0x0.0149868FEC26935Dp0",
"0x0.015031C6997996937p0", "0x0.0156EEAA4944EE23p0",
783 "0x0.015DBD3F68122303Ep0", "0x0.01649D8A62CB1ACBEp0",
"0x0.0168BF8FA7745BCDBp0", "0x0.01729353A52E403D8p0",
784 "0x0.0179A8DACC350DE1Dp0", "0x0.0180D0298D613D179p0",
"0x0.018809445CA7BF78p0", "0x0.018F542FAC1A492D0p0",
785 "0x0.0196B00EFF0E793D15p0", "0x0.019E1F89A0DBAA128p0",
"0x0.01A5A00132E02CE42p0", "0x0.01AD325B1EFC9B5BCp0",
786 "0x0.01B4D09BDE569A322p0", "0x0.01BC8CC7EB3238553p0",
"0x0.01C454E3C0F2458C8p0", "0x0.01CC2EF3DC187C2F6p0",
787 "0x0.01D41AFBCA45E6ED2p0", "0x0.01DC1902DA3B19A6Dp0",
"0x0.01E4290AB8D87C5BCp0", "0x0.01EC4B18E01E9326Fp0",
788 "0x0.01F47F31C92E464FCp0", "0x0.01FCC559FA492A6B6p0",
"0x0.02051D95F7D1C8917p0", "0x0.020BD7EA474BE6A1Cp0",
789 "0x0.0216045B6F5CC9FCEp0", "0x0.021E92EDF7CB9C0E4p0",
"0x0.02273A669817A8Ap0", "0x0.022FE6894E89F834Cp0",
790 "0x0.0238AB9B321349722p0", "0x0.024182E0A06E9289Bp0",
"0x0.024A6C5E271030733p0", "0x0.02536818549001AC1p0",
791 "0x0.025C7613B8A9AF215p0", "0x0.02659654E43CF52B1p0",
"0x0.026EC8E0694DEC9A9p0", "0x0.02780DBAD80553DA6p0",
792 "0x0.028164E8CDB0D8211p0", "0x0.028ACE6D3C35E85Fp0",
"0x0.02944A518CD54E484p0", "0x0.029DD89587A4D858Dp0",
793 "0x0.02A7793F601642B5Dp0", "0x0.02B12C538A0341090p0",
"0x0.02BAF1D7132FEE788p0", "0x0.02C4C9CE2561B759Fp0",
794 "0x0.02CEB43D844902F7Ap0", "0x0.02DB8129CE8C0D68Fp0",
"0x0.02E2C097A3FBE17C9p0", "0x0.02ECE28BA58D22B56p0",
795 "0x0.02F170A755DF759Ep0", "0x0.03015E18B6BF2966p0",
"0x0.030BB7B80E125E813p0", "0x0.031623F620FD4751Ep0",
796 "0x0.0320A2CE964063DACp0", "0x0.0328344915C941D54p0",
"0x0.0335D86A48AF87F0Cp0", "0x0.03408F36D93566140p0",
797 "0x0.034B58B372C795013p0", "0x0.035634E4C41FDEC0D0p0",
"0x0.036123CF7498B667Cp0", "0x0.036C2578398F157A0p0",
798 "0x0.037739C3C0F32BD30p0", "0x0.03826116BC0E882ACp0",
"0x0.038D9B15DD5460763p0", "0x0.0398E75D864AD0E9p0",
799 "0x0.03A4478B620C73EB5p0", "0x0.03AFBA0B304613EEDp0",
"0x0.03BB3F69FA3990464p0", "0x0.03C6D7AC783CDBBDFp0",
800 "0x0.03D282D763D424DCDp0", "0x0.03DE40FF77B229509p0",
"0x0.03EA11F96FB855C4Fp0", "0x0.03F5F5FA08F76B0BBp0",
801 "0x0.0401ECF601AF700BDp0", "0x0.040DF6F219502AA55p0",
"0x0.041A13F3107962A88p0", "0x0.042643FDA8FB2DDDFp0",
802 "0x0.04328716A5D63C2C4p0", "0x0.043DD42CB3C23B6Dp0",
"0x0.044B4686DE8FAD325p0", "0x0.0457C27FE66520322p0",
803 "0x0.04645269EA828F932p0", "0x0.0470F51273E025F89p0",
"0x0.047DAAE60CA8725B3p0", "0x0.048A73E98038B4ABFp0",
804 "0x0.049750219B212A8A0p0", "0x0.04A43F932B255C0B7p0",
"0x0.04B14242FF3C689A0p0", "0x0.04BE835E79153E1Dp0",
805 "0x0.04CB8170B58352D4Ep0", "0x0.04D8BDF83BA618C04p0",
"0x0.04E60DD14DC22475Ep0", "0x0.04F37100C0D50D88Ep0",
806 "0x0.0500E78B6B1D1D9D6p0", "0x0.050E717623E121CBAP0",
"0x0.051C0EC5C3E1B016Cp0", "0x0.0529BF7F24E87CF6Ap0",
807 "0x0.053783A7220124F51p0", "0x0.05455B42976E2E5EAp0",
"0x0.0553465662A95706Bp0", "0x0.056144F76263E21F4p0",
808 "0x0.056F56FA7686E623Ep0", "0x0.057D7C9480339AD83p0",
"0x0.058BB5BA61C3A75A3p0", "0x0.059A0270FEC97047Ap0",
809 "0x0.05A862BD3C1065F74p0", "0x0.05B6D6A3FF9D52C58p0",
"0x0.05C5E2A30AE974Ep0", "0x0.05D3F95487BCD3A1Bp0",
810 "0x0.05E2A8287E7A8049Ep0", "0x0.05F16AA6F04F267Bp0",
"0x0.060040DF77F44FA13p0", "0x0.060F2ACC843BEF0A0p0",
811 "0x0.061E2876834AA7FA4p0", "0x0.062D39E264FB20F7Ep0",
"0x0.063C5F151A641EB4Cp0", "0x0.064AB981395D8D31Fep0",
812 "0x0.065AE4E2CAE92C8AAp0", "0x0.066A4587AE6224E1Cp0",
"0x0.0679BA07364E10FA3p0", "0x0.0689426659F4E17Bp0",
813 "0x0.0698DEAA11DCBAA37p0", "0x0.06A88ED757C9B3304p0",
"0x0.06B852F326B8B4980p0", "0x0.06C82B027AFD8278Ap0",
814 "0x0.06D8170A520718B46p0", "0x0.06E8170FA09BF867Ep0",
"0x0.06F82B1784BC86C72p0", "0x0.07085326E1A93F207p0",
815 "0x0.07188F42C3E320F0Bp0", "0x0.0728DF702F2BF10B2p0",
"0x0.073943B428868CD5Ap0", "0x0.0749BC13B6373A98Fp0",
816 "0x0.075A4893DFC3F9E4Cp0", "0x0.076AE939ADF4D4083p0",
"0x0.077B9E0A2AD42C9E8p0", "0x0.078C670A61AF122F4p0",
817 "0x0.079D4435FF158EE3Ap0", "0x0.07E435A30DAF94F1p0",
"0x0.07FB3B5BE616454Cp0", "0x0.07D0554D8F2254F0Cp0",
818 "0x0.07E183883D9E498E2p0", "0x0.07F2C611046D4D06p0",
"0x0.08041CECF7B989EA2p0", "0x0.081588212CEFE2D73p0",
819 "0x0.082707B2BAC20DB40p0", "0x0.083B9BA6892B46284p0",
"0x0.084A4402416A1EE6Fp0", "0x0.085C00CA6E045741Fp0",
820 "0x0.086DD2045AC678D2Bp0", "0x0.087FB7524C42936Cp0",
"0x0.0891B1E1EA587BE12p0", "0x0.08A3C08FCB2643FFEp0",
821
822 ap_ufixed<64, -4> f_x_msb_2 = f_x_msb_2_t-
able[x_msb_ind_2];
823
824 ap_ufixed< 6, -4> f_x_msb_2_h;
825 f_x_msb_2_h(5,0) = f_x_msb_2(63,58);
826 ap_ufixed< 9, -1> exp_x_msb_2_m_1_h = f_x_msb_2_h
+ x_msb_2;
827
828 ap_ufixed<67, -1> exp_x_msb_2_m_1;
829 exp_x_msb_2_m_1(66,58) = exp_x_msb_2_m_1_h(8,0);
exp_x_msb_2_m_1(57, 0) = f_x_msb_2(57,0);

```



```

830      ap_ufixed<130,-10> f_x_msb_2_3_4_lsb =
exp_x_msb_2_m_1 * exp_x_msb_3_4_lsb_m_1;
831      ap_ufixed<62,-10> f_x_msb_2_3_4_lsb_s =
f_x_msb_2_3_4_lsb;
832      ap_ufixed<72,0> exp_x_msb_2_3_4_lsb_m_1 =
f_x_msb_2_3_4_lsb_s + exp_x_msb_2_m_1 + exp_x_msb_3_4_lsb_m_1;
833
834      const static ap_ufixed<68,32> exp_x_msb_1_t-
able[256] = {
835      "0x1.000000000p0", "0x1.48B5E3C3Ep0", "0x1.A61298E1Ep0",
"0x2.1DF3B68CFp0",
836      "0x2.B7E151628p0", "0x3.7D871D6B1p0", "0x4.7B4F993Fp0",
"0x5.C12DA416Ep0",
837      "0x7.63992E353p0", "0x9.7CDC417A3p0", "0xC.2EB7EC98Fp0",
"0xF.A48385EDFp0",
838      "0x14.15E5BF6FBp0", "0x19.CA53B7811p0", "0x21.1D8E4272Dp0",
"0x2A.8565A144Cp0",
839      "0x36.99205C4E7p0", "0x46.1AFC4DB59p0", "0x5A.0462B8777p0",
"0x73.9593ABB7Dp0",
840      "0x94.69C4CB819p0", "0xBE.90F6F83E9p0", "0xF4.B122790DDp0",
"0x13A.30CF1CCBBp0",
841      "0x193.6DC5690C0p0", "0x206.03487A3B1p0",
"0x299.2442102D9p0", "0x356.0F0B0F980p0",
842      "0x448.A216ABB76p0", "0x580.1AD754FA3p0",
"0x710.0ADBAC7DAp0", "0x911.9289C3923p0",
843      "0xBA4.F53EA3863p0", "0xEF3.A035D5798p0",
"0x1332.C4D2B7C4Ap0", "0x18A6.B027DA0A7p0",
844      "0x1FA7.157C470F8p0", "0x28A4.90D2CCEFFp0",
"0x342F.BA1823B8p0", "0x4302.3A933EE5Cp0",
845      "0x560A.773E54157p0", "0x6E7A.8ABB4A83Cp0", "0x8DD8.
80AF4269Dp0", "0xB626.0748BAEC7p0",
846      "0xE9E2.2447727BFp0", "0x12C4F.EB75B2AB0p0",
"0x1819B.C560F6113p0", "0x1EF21.8F1B001A4p0",
847      "0x27BC2.CA9A6F934p0", "0x33055.49F35D91Fp0",
"0x41831.49596E996p0", "0x541E7.E56949D58p0",
848      "0x6C02D.645AB2554p0", "0x8AB06.0A3EE9FB1p0",
"0xB2148.5EA56C5Bp0", "0xE4A8D.2881EDBE8p0",
849      "0x1259AC.48BF05D6Ep0", "0x178FE.7792E44B3p0",
"0x1E4127.437732B71p0", "0x26D8F9.4A204BEA5p0",
850      "0x31E199.5F5A550DDp0", "0x400C7D.64D3386CBp0",
"0x523D82.79EDA9F9Cp0", "0x69993D.D4F2D9524p0",
851      "0x87975E.854001024p0", "0xAE1A47.C38A42CD0p0", "0xDF8D5F.6D-
CFE5CEAp0", "0x11F08F8.1E99FDEB6p0",
852      "0x1709348.C0EA4F8CEp0", "0x1D94295.464477ACCp0", "0x25-
FAD90.65C78E342p0", "0x30C4623.616ED2BE2p0",
853      "0x3E9E441.232817A61p0", "0x506744D.3B342FC94p0",
"0x673D70B.C00F1F76Dp0", "0x849012B.C127FC962p0",
854      "0xAA36C7C.F69370B94p0", "0xDA8F202.DD9ACE661p0",
"0x118A2AAE.0AA05236Bp0", "0x16857CFA.1AA8A5555p0",
855      "0x1CEB0888.68E804021p0", "0x2521AEDA.32CD52BE5p0", "0x2-
FAD89E1.79BAB8EADp0", "0x3D3838C6.BE0F13F06p0",
856      "0x4E9B87F6.7BB3F5597p0", "0x64EF201B.01109A4A5p0",
"0x819A1801.9394C0D96p0", "0xA6698403.CF5D9E663p0",
857      "0xD5AD6DCE.21AFFC0DDp0", "0x1125E066B.0B19418CCp0",
"0x160486BFC.05F65FAA9p0", "0x1C45AE0DA.D0C934CDCp0",
858      "0x244D5E676.3FB814AF0p0", "0x2E9CEBF49.9C21839C7p0", "0x3B-
DA2CBD0.9F5AC2146p0", "0x4CDA0FD41.989F004E7p0",
859      "0x62ADFED51.D7EDCCDE6p0", "0x7EB508B20.76F91D6E8p0", "0x-
A2B1FD3D9.A57B6C33Cp0", "0xD0E7A50F4.EF0D7447Fp0",
860      "0x10C3D39209.62C88AAFBp0", "0x1586D0722B.3EEB27301p0",
"0x1BA4068AAC.2B6E3EE34p0", "0x237DCBF1E5.709A9F9C1p0",
861      "0x2D9264D2D4.D6B94153Bp0", "0x3A83F23B18.FE5D0524Ap0",
"0x4B22A1B5C3.27A2F085Ap0", "0x6079C1866C.71A93951Dp0",
862      "0x7BE08BDFD9.CC72249ABp0", "0x9F0FB7309C.B446D3919p0", "0x-
CC3D265776.C3D25779Ep0", "0x1063F7612B0.C0831CA6Cp0",
863      "0x150BBA37963.79A4FB6ADp0", "0x1B05FA9B62E.737D935B8p0",
"0x22B2DC5C41D.AD89559EBp0", "0x2C8DD1AB28B.404414BE7p0",
864      "0x39355C8C638.0AC5631B1p0", "0x49750434675.301B8D0E7p0",
"0x5E521E7A16E.51F9CD3C2p0", "0x791C3B06F8B.EA2DEC48Cp0",
865      "0x9B823857614.764F43E20p0", "0xC7AD559D438.E7C270C30p0",
"0x10063F4E687B.A1A43A35Bp0", "0x149363C905BA.3641D0164p0",
866      "0x1A6B765D8CDF.6CDBF1C63p0", "0x21EC75240E66.E2891CE63p0",
"0x2B8F1073A6AF.BA219DB1p0", "0x37EE3FFC0063.E360F931Fp0",
867      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
868      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
869      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
870      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
871      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
872      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
873      "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
874      "0x0.000000000p0", "0x0.00000001p0", "0x0.00000001p0",
"0x0.00000002p0",
875      "0x0.000000002p0", "0x0.000000003p0", "0x0.000000004p0",
"0x0.000000005p0",
876      "0x0.000000007p0", "0x0.000000009p0", "0x0.00000000Bp0",
"0x0.00000000Ep0",
877      "0x0.000000013p0", "0x0.000000018p0", "0x0.00000001Fp0",
"0x0.000000028p0",
878      "0x0.000000034p0", "0x0.000000042p0", "0x0.000000055p0",
"0x0.00000006Ep0",
879      "0x0.00000008Dp0", "0x0.0000000B5p0", "0x0.0000000E9p0",
"0x0.00000012Bp0",
880      "0x0.000000181p0", "0x0.0000001EEp0", "0x0.00000027Ap0",

```

```

"0x0.00000032Fp0",
881      "0x0.000000416p0", "0x0.00000053Fp0", "0x0.0000006BDp0",
"0x0.0000008A7p0",
882      "0x0.000000B1Cp0", "0x0.000000E44p0", "0x0.000001252p0",
"0x0.000001786p0",
883      "0x0.000001E35p0", "0x0.0000026C9p0", "0x0.0000031CEp0",
"0x0.000003FF3p0",
884      "0x0.00000521Dp0", "0x0.000006970p0", "0x0.000008762p0",
"0x0.00000ADD6p0",
885      "0x0.00000DF36p0", "0x0.000011E9Cp0", "0x0.000017003p0",
"0x0.00001D88Ap0",
886      "0x0.000025EC0p0", "0x0.000030B15p0", "0x0.00003E85Dp0",
"0x0.00005047Ep0",
887      "0x0.000067153p0", "0x0.0000845C6p0", "0x0.0000A9F46p0",
"0x0.0000DA39Ep0",
888      "0x0.000118354p0", "0x0.000167CB5p0", "0x0.0001CDFC2p0",
"0x0.000251334p0",
889      "0x0.0002F9AF3p0", "0x0.0003D205Ap0", "0x0.00047CE2p0",
"0x0.00064C7C6p0",
890      "0x0.000816791p0", "0x0.000A628A3p0", "0x0.000D55A1Fp0",
"0x0.0011F30Fp0",
891      "0x0.0015FC210p0", "0x0.001C3AA91p0", "0x0.00243F374p0",
"0x0.002E8ABFCp0",
892      "0x0.003BC2D73p0", "0x0.004CBC19Ap0", "0x0.006287862p0",
"0x0.007E83A56p0",
893      "0x0.00A2728F8p0", "0x0.00D096336p0", "0x0.010BD4A5Ap0",
"0x0.0157E6BFp0",
894      "0x0.01B993FE0p0", "0x0.0236FF58Fp0", "0x0.02D80A08Dp0",
"0x0.03A6D2222p0",
895      "0x0.04B0556E0p0", "0x0.0605424D5p0", "0x0.07BB04063p0",
"0x0.09ED1B429p0",
896      "0x0.0CBED8666p0", "0x0.105D93892p0", "0x0.150385C09p0",
"0x0.1AFB718E8p0",
897      "0x0.22A555477p0", "0x0.2C7C72FC0p0", "0x0.391F0EE49p0",
"0x0.495860DCAp0",
898      "0x0.5E2D58D8Bp0", "0x0.78ED03AFBp0", "0x0.9B4597E37p0",
"0x0.C75F7CF56p0",
};
899
900      ap_ufixed<68,32> exp_x_msb_1 = exp_x_msb_1_t-
able[x_msb_ind_1];
901      ap_ufixed<140,32> y_lo = exp_x_msb_1 *
exp_x_msb_2_3_4_lsb_m_1;
902      ap_ufixed<68,32> y_lo_s = y_lo;
903      ap_ufixed<68,32> y_l = y_lo_s + exp_x_msb_1;
904      y = y_l;
905  }
906  if (I_<33) { // overflow
907      bool overf = 0;
908      for (int j = 63; j >= I_+31; j--) {
909          #pragma HLS unroll
910          if (y[j])
911              overf = 1;
912      }
913      if (overf) {
914          for (int j = 63; j >= I_+31; j--) {
915              #pragma HLS unroll
916              y[j] = 0;
917          }
918          for (int j = I_+30; j >= 0; j--) {
919              #pragma HLS unroll
920              y[j] = 1;
921          }
922      }
923      }
924      r = y;
925  }
926  return r;
927 }
928 //implement exp() function with Input width larger than out-
put width
929
930 template<int W0_, int I_, int WI_>
931 ap_fixed<W0_,I_> exp_core(ap_fixed<WI_,I_> x)
932 {
933     #pragma HLS pipeline
934
935     const static int F0_ = W0_ - I_;
936     const static int FI_ = WI_ - I_;
937
938     //only support I_ < 33, F0_ < 32 and FI_ < 100
939     if (I_>33) return 0;
940     else if (F0_>32) return 0;
941     else if (FI_>100) return 0;
942
943     // I_s_ is the number of integer bits we need to calcu-
late
944     // I_s_ = ceil( ln2 * max ( I_-1 , F0_ ) )
945     // I_s_ = [1..6]
946     int I_s_;
947     int M_ = (I_-1>F0_) ? (I_-1) : F0_;
948     if (M_==1) I_s_ = 1;
949     else if (M_==2) I_s_ = 2;
950     else if (M_<6) I_s_ = 3;
951     else if (M_<12) I_s_ = 4;
952     else if (M_<24) I_s_ = 5;
953     else I_s_ = 6;
954
955     ap_fixed<W0_,I_> r = 0;
956
957     ap_ufixed<1,-F0_> delta = 0;

```

```

958     delta[0] = 1;
959
960     if (I_s==1) { // I_s = 1, M_ = 1, I_ <= 2, FO_ <= 1, WO
<=3, WI <= 7
961         //00.00000-00.01011 1.0
962         //00.01100-01.11111 1.1
963         //11.11111-11.01010 0.1
964         //11.01001-10.00000 0.0
965         //std::cout<<"x"<<x.to_string()<<std::endl;
966         //w = 7; we = 2
967         const static int w = 7;
968         const static int we = 2;
969         // const static int wf = 5;
970         //truncate if WI>7
971         ap_fixed<w,we> x_l = x;
972         ap_ufixed<3,2> y = 0;
973         ap_ufixed<5,0> x_l_fract=0;
974         x_l_fract(4,0) = x_l(4,0);
975         //std::cout<<"[EXP]x_l="<<x_l.to_string()<<std::endl;
976         //e(01.00000-01.11111)= 1.1
977         if (x_l[6]!=x_l[5]) { // overflow
978             if (!x_l[6]) y = "0x1.8p0";
979         } else {
980             if (!x_l[6]) y[1] = 1;
981             //e(00.01100 - 0.11111) = 1.1
982             if (!x_l[6] && x_l[4] || (x_l[2] && x_l[3]))
y[0] = 1;
983             if (!x_l[6] && x_l[2] && x_l[3]) y[0] = 1; //
00.01100 - 00.01111
984             if (x_l[6] && ((x_l[4] && x_l[3]) || x_l[5]))
y[0] = 1; // 11.01100-11.11111
985             if (x_l[6] && x_l[5] && !x_l[4] && x_l[3] && !
x_l[2] && x_l[1]) y[0] = 1; // 11.01010-11.01011
986         }
987         y += delta;
988         //if (I_<2) { // overflow
989             if (y[1] || y[2]) {
990                 y[2] = 0;
991                 y[1] = 0;
992                 y[0] = 1;
993             }
994         }
995         r = y;
996         //std::cout<<"[EXP]r="<<y.to_string()<<std::endl;
997     } else if (I_s==2) { // I_s = 2, M_ = 2, I_ <= 3, FO_
<= 2/7, WO <= 5, WI <= 10
998         const static int w = 10;
999         const static int we = 3;
1000         const static int wf = 7;
1001         //std::cout<<"[EXP]x="<<x.to_string()<<std::endl;
1002         //truncate if WI>10
1003         ap_fixed<w,we> x_l = x;
1004         ap_ufixed<5,3> y = 0;
1005         ap_fixed<2,2> x_l_int;
1006         x_l_int(1,0) = x_l(wf+1,wf);
1007         ap_ufixed<7,0> x_l_fract;
1008         x_l_fract(6,0) = x_l(6,0);
1009
1010         ap_uint<5> x_msb_ind;
1011         x_msb_ind[3] = x_l[w-1];
1012         x_msb_ind(2,0) = x_l(wf,wf-3);
1013         ap_ufixed<4,-3> x_lsb = x_l;
1014
1015         //001.0101-010 - 011.1111-111 overflow
1016         if ((x_l[w-1]!=x_l[w-2]) ||
((x_l_int==1)&&(x_l_fract>ap_ufixed<wf,0>("0x0.54p0")))) { //
overflow
1017             if (!x_l[w-1]) y = "0x3.cp0";
1018         } else {
1019             ap_ufixed<5,2> exp_x_msb_table[32] = {
1020                 "0x1.00p0", "0x1.48p0", "0x1.A6p0",
"0x2.1Dp0", "0x2.B7p0", "0x3.7Dp0", "0x4.7Bp0", "0x5.C1p0",
1021                 "0x0.22p0", "0x0.2Cp0", "0x0.39p0",
"0x0.49p0", "0x0.5Ep0", "0x0.78p0", "0x0.9Bp0", "0x0.C7p0",
1022             };
1023             ap_ufixed<5,2> exp_x_msb = exp_x_msb_t-
able[x_msb_ind];
1024             //
std::cout<<"exp_x_msb="<<exp_x_msb.to_string()<<std::endl;
1025
1026             ap_ufixed<5,0> y_lo = x_lsb * exp_x_msb;
1027             //
std::cout<<"y_lo="<<y_lo.to_string()<<std::endl;
1028             ap_ufixed<5,2> y_l = y_lo + exp_x_msb;
1029             //std::cout<<"y_l="<<y_l.to_string()<<std::endl;
1030             y = y_l;
1031         }
1032         y += delta;
1033         bool overf = 0;
1034         for (int j = 4; j >= I_+1; j--) {
1035             #pragma HLS unroll
1036             if (y[j])
overf = 1;
1037         }
1038         if (overf) {
1039             for (int j = 4; j >= I_+1; j--) {
1040                 #pragma HLS unroll
1041                 y[j] = 0;
1042             }
1043         }

```

```

1044         for (int j = I_; j >= 0; j--) {
1045             #pragma HLS unroll
1046             y[j] = 1;
1047         }
1048     }
1049     r = y;
1050 } else if (I_s==3) { // I_s = 3, M_ = 3.5, I_ <= 6,
FO_ <= 5/13, WO <=11, WI<=19
1051     //std::cout<<"[EXP]x="<<x.to_string()<<std::endl;
1052     const static int w = 19;
1053     const static int we = 6;
1054     const static int wf = 13;
1055     //truncate x
1056     ap_fixed<w,we> x_l = x;
1057     ap_ufixed<11,6> y = 0;
1058     ap_fixed<3,3> x_l_int;
1059     x_l_int(2,0) = x_l(wf+2,wf);
1060     ap_ufixed<wf,0> x_l_fract;
1061     x_l_fract(wf-1,0) = x_l(wf-1,0);
1062     //
std::cout<<"x_l_fract="<<x_l_fract.to_string()<<std::endl;
1063     //
std::cout<<"x_l_int="<<x_l_int.to_string()<<std::endl;
1064
1065     //3.76FA8 - 1F.FFFF8
1066     if ((x_l[w-1]!=x_l[w-2]) || (x_l[w-1]!=x_l[w-3]) ||
(x_l[w-1]!=x_l[w-4]) || ((x_l_int==3)&&(x_l_fract>ap_ufixed<wf,
0>("0x0.76FAp0")))) { // overflow
1067         if (!x_l[18]) y = "0x1.f8p0";
1068     } else {
1069         const static int g = 2; //2 adder
1070         const static int Maxprecision = FO_+we+g; //
5+6+2=13
1071         ap_uint<4> x_msb_ind_1;
1072         x_msb_ind_1[3] = x_l[w-1];
1073         //xx.x
1074         x_msb_ind_1(2,0) = x_l(wf+1,wf-1);
1075         //00.0xxxxx
1076         ap_uint<6> x_msb_ind_2;
1077         x_msb_ind_2(5,0) = x_l(wf-2,wf-7);
1078         ap_ufixed<6,-1> x_msb_2 = x_l;
1079         //00.000000xxxxx
1080         ap_ufixed<wf-7,-7> x_lsb = x_l;
1081
1082         const static ap_ufixed<Maxprecision,0>
exp_x_msb_2_m_1_table[64] = {
1083             "0x0.0p0", "0x0.0202p0", "0x0.0408p0",
"0x0.0612p0",
1084             "0x0.0820p0", "0x0.0A32p0", "0x0.0C49p0",
"0x0.0E63p0",
1085             "0x0.1082p0", "0x0.12A5p0", "0x0.14CDp0",
"0x0.16F9p0",
1086             "0x0.1929p0", "0x0.1B5Dp0", "0x0.1D96p0",
"0x0.1FD4p0",
1087             "0x0.2216p0", "0x0.245Cp0", "0x0.26A7p0",
"0x0.28F7p0",
1088             "0x0.2B48p0", "0x0.2DA4p0", "0x0.3001p0",
"0x0.3264p0",
1089             "0x0.34CBp0", "0x0.3737p0", "0x0.39A8p0",
"0x0.3C1Ep0",
1090             "0x0.3E98p0", "0x0.4118p0", "0x0.439Dp0",
"0x0.4627p0",
1091             "0x0.48B5p0", "0x0.4B49p0", "0x0.4DE3p0",
"0x0.5081p0",
1092             "0x0.5325p0", "0x0.55CEp0", "0x0.587Cp0",
"0x0.5B2Fp0",
1093             "0x0.5DE9p0", "0x0.60A7p0", "0x0.636Bp0",
"0x0.6635p0",
1094             "0x0.6904p0", "0x0.6BD9p0", "0x0.6EB3p0",
"0x0.7194p0",
1095             "0x0.747Ap0", "0x0.7766p0", "0x0.7A57p0",
"0x0.7D4Fp0",
1096             "0x0.804Dp0", "0x0.8350p0", "0x0.865Ap0",
"0x0.896Ap0",
1097             "0x0.8C80p0", "0x0.8F9Cp0", "0x0.92BEp0",
"0x0.95E7p0",
1098             "0x0.9916p0", "0x0.9C4Bp0", "0x0.9F87p0",
"0x0.A29Cp0",
1099             };
1100         //6*6
1101         ap_ufixed<5,-8> f_x_msb_2_lsb = x_msb_2 *
x_lsb; //Z_5*Z_lsb //1u
1102         //
std::cout<<"f_x_msb_2_lsb="<<f_x_msb_2_lsb.to_string()<<std::endl;
1103
1104         ap_ufixed<7,-6> f_x_msb_2_lsb_m_1_m=0;
1105         f_x_msb_2_lsb_m_1_m(5,0) = x_lsb(5,0);
1106         f_x_msb_2_lsb_m_1_m = f_x_msb_2_lsb_m_1_m +
f_x_msb_2_lsb; //1u
1107         //
std::cout<<"f_x_msb_2_lsb_m_1_m="<<f_x_msb_2_lsb_m_1_m.to_string()<
<std::endl;
1108
1109         ap_ufixed<Maxprecision+1,1> exp_x_msb_2_m_1 =
exp_x_msb_2_m_1_table[x_msb_ind_2];
1110         ap_ufixed<Maxprecision,1> exp_x_msb_2_lsb_m_1 =
f_x_msb_2_lsb_m_1_m + exp_x_msb_2_m_1; //2u
1111         //
std::cout<<"exp_x_msb_2_m_1="<<exp_x_msb_2_m_1.to_string()<<std::e
ndl;

```

```

1112
1113     const static ap_ufixed<Maxprecision,5>
exp_x_msb_1_table[16] = {
1114     "0x1.0000p0", "0x1.A612p0", "0x2.B7E1p0",
"0x4.7B4Fp0",
1115     "0x7.6399p0", "0xC.2EB7p0", "0x14.15E5p0",
"0x21.1D8Ep0",
1116     "0x0.04B0p0", "0x0.07BBp0", "0x0.0CBEp0",
"0x0.1503p0",
1117     "0x0.22A5p0", "0x0.391Fp0", "0x0.5E2Dp0",
"0x0.9B45p0",
1118
1119     };
1120     ap_ufixed<Maxprecision,5> exp_x_msb =
exp_x_msb_1_table[x_msb_ind_1];
1121     //13*13--1 DSP
1122     ap_ufixed<2*Maxprecision+1,6> y_lo = exp_x_msb *
exp_x_msb_2_lsb_m_1;//1u
1123     ap_ufixed<Maxprecision,5> y_lo_s = y_lo;
1124     ap_ufixed<Maxprecision,5> y_l = y_lo_s +
exp_x_msb;//
1125     y = y_l;
1126     //std::cout<<"[EXP]y="<<y.to_string()<<std::endl;
1127 }
1128 y += delta;
1129 //if (I_<6) { // overflow
1130 bool overf = 0;
1131 for (int j = 10; j >= I_+4; j--) {
1132     #pragma HLS unroll
1133     if (y[j])
1134         overf = 1;
1135 }
1136 if (overf) {
1137     for (int j = 10; j >= I_+4; j--) {
1138         #pragma HLS unroll
1139         y[j] = 0;
1140     }
1141     for (int j = I_+3; j >= 0; j--) {
1142         #pragma HLS unroll
1143         y[j] = 1;
1144     }
1145 }
1146 //}
1147 r = y;
1148 } else if (I_s==4) { // I_s_ = 4, M_ = 6..11, I_ <= 12,
FO_ <= 11/25, WO_ <= 23, WI_ <= 37
1149     //std::cout<<"x="<<x.to_string()<<std::endl;
1150     const static int wf = 25;
1151     const static int we = 12;
1152     const static int w = 37;
1153     ap_ufixed<w,we> x_l = x;
1154     ap_ufixed<23,12> y = 0;
1155     ap_ufixed<4,4> x_l_int;
1156     x_l_int(3,0) = x_l(wf+3,wf);
1157     ap_ufixed<wf,0> x_l_fract;
1158     x_l_fract(wf-1,0) = x_l(wf-1,0);
1159     bool overf = 0;
1160     for (int j = wf+3; j < w-1; j++) {
1161         #pragma HLS unroll
1162         if (x_l[w-1]!=x_l[j])
1163             overf = 1;
1164     }
1165     if (overf||((x_l_int==7)&&(x_l_fract>ap_ufixed<wf,
0>("0x0.9fe7038p0")))) { // overflow
1166         if (!x_l[w-1]) {
1167             for (int j = 0; j < 22; j++) {
1168                 #pragma HLS unroll
1169                 y[j] = 1;
1170             }
1171         }
1172     } else {
1173         // const static int g = 2;
1174         // const static int Maxprecision = 25;//
we+FO_+g=11+12+2=25
1175         ap_uint<5> x_msb_ind_1;
1176         x_msb_ind_1[4] = x_l[w-1];
1177         x_msb_ind_1(3,0) = x_l(wf+2,wf-1);
1178
1179         ap_uint<5> x_msb_ind_2;
1180         x_msb_ind_2(4,0) = x_l(wf-2,wf-6);
1181         // ap_ufixed<5,-1> x_msb_2 = x_l;
1182
1183         ap_uint<7> x_msb_ind_3;
1184         x_msb_ind_3(6,0) = x_l(wf-7,wf-13);
1185         ap_ufixed<7,-6> x_msb_3 = x_l;
1186
1187         ap_ufixed<7,-13> x_lsb = x_l;
1188         //to 25-bit
1189         const static ap_ufixed<12,-13> f_x_msb_3_m_1_t-
able[128] = {
1190     "0x0.0p0", "0x0.0000002p0", "0x0.0000008p0", "0x0.0000012p0",
1191     "0x0.0000020p0", "0x0.0000032p0", "0x0.0000048p0",
"0x0.0000062p0",
1192     "0x0.0000080p0", "0x0.00000A2p0", "0x0.00000C8p0",
"0x0.00000F2p0",
1193     "0x0.0000120p0", "0x0.0000152p0", "0x0.0000188p0",
"0x0.00001C2p0",
1194     "0x0.0000200p0", "0x0.0000242p0", "0x0.0000288p0",
"0x0.00002D2p0",
1195     "0x0.0000320p0", "0x0.0000372p0", "0x0.00003C8p0",
"0x0.0000422p0",
1196     "0x0.0000481p0", "0x0.00004E3p0", "0x0.0000549p0",
"0x0.00005B3p0",
1197     "0x0.0000621p0", "0x0.0000693p0", "0x0.000070Ap0",
"0x0.0000784p0",
1198     "0x0.0000802p0", "0x0.0000884p0", "0x0.000090Bp0",
"0x0.0000995p0",
1199     "0x0.0000A23p0", "0x0.0000AB6p0", "0x0.0000B4Cp0",
"0x0.0000BE6p0",
1200     "0x0.0000C85p0", "0x0.0000D27p0", "0x0.0000DCEp0",
"0x0.0000E78p0",
1201     "0x0.0000F26p0", "0x0.0000FD9p0", "0x0.000108Fp0",
"0x0.000114Ap0",
1202     "0x0.0001209p0", "0x0.00012CBp0", "0x0.0001392p0",
"0x0.000145Cp0",
1203     "0x0.000152Bp0", "0x0.00015FEp0", "0x0.00016D4p0",
"0x0.00017AFp0",
1204     "0x0.000188Ep0", "0x0.0001971p0", "0x0.0001A57p0",
"0x0.0001B42p0",
1205     "0x0.0001C31p0", "0x0.0001D24p0", "0x0.0001E1Bp0",
"0x0.0001F16p0",
1206     "0x0.0002015p0", "0x0.0002118p0", "0x0.000221Fp0",
"0x0.000232Ap0",
1207     "0x0.0002439p0", "0x0.000254Cp0", "0x0.0002663p0",
"0x0.000277Fp0",
1208     "0x0.000289Ep0", "0x0.00029C1p0", "0x0.0002AE9p0",
"0x0.0002C14p0",
1209     "0x0.0002D43p0", "0x0.0002E77p0", "0x0.0002FAEp0",
"0x0.00030EAp0",
1210     "0x0.0003229p0", "0x0.000336Dp0", "0x0.00034B4p0",
"0x0.0003600p0",
1211     "0x0.0003750p0", "0x0.00038A4p0", "0x0.00039FBp0",
"0x0.0003B57p0",
1212     "0x0.0003CB7p0", "0x0.0003E1Bp0", "0x0.0003F83p0",
"0x0.00040EFp0",
1213     "0x0.000425Fp0", "0x0.00043D3p0", "0x0.000454Bp0",
"0x0.00046C7p0",
1214     "0x0.0004848p0", "0x0.00049CCp0", "0x0.0004B54p0",
"0x0.0004CE1p0",
1215     "0x0.0004E71p0", "0x0.0005006p0", "0x0.000519Ep0",
"0x0.000533Bp0",
1216     "0x0.00054DBp0", "0x0.0005680p0", "0x0.0005829p0",
"0x0.00059D6p0",
1217     "0x0.0005B86p0", "0x0.0005D3Bp0", "0x0.0005EF4p0",
"0x0.00060B1p0",
1218     "0x0.0006272p0", "0x0.0006437p0", "0x0.0006600p0",
"0x0.00067CEp0",
1219     "0x0.000699Fp0", "0x0.0006B74p0", "0x0.0006D4Ep0",
"0x0.0006F2Bp0",
1220     "0x0.000710Dp0", "0x0.00072F2p0", "0x0.00074DCp0",
"0x0.00076CAp0",
1221     "0x0.00078BBp0", "0x0.0007AB1p0", "0x0.0007CABp0",
"0x0.0007EA9p0",
1222     };
1223     //7*7
1224     ap_ufixed<6,-19> f_x_msb_3_lsb = x_msb_3 *
x_lsb;//1u
1225     //
std::cout<<"f_x_msb_3_lsb="<<f_x_msb_3_lsb.to_string()<<std::endl;
//1u
1226
1227     ap_ufixed<12,-13> f_x_msb_3 = f_x_msb_3_m_1_t-
able[x_msb_ind_3];
1228     //
std::cout<<"f_x_msb_3="<<f_x_msb_3.to_string()<<std::endl;
//1u
1229     ap_ufixed<13,-12> exp_x_msb_3_lsb_m_1_m = 0;
1230     exp_x_msb_3_lsb_m_1_m(11,0) = f_x_msb_3(11,0);
1231     exp_x_msb_3_lsb_m_1_m = exp_x_msb_3_lsb_m_1_m +
f_x_msb_3_lsb;//2u
1232     //
std::cout<<"exp_x_msb_3_lsb_m_1_m="<<exp_x_msb_3_lsb_m_1_m.to_string()<<std::endl;
1233
1234     ap_ufixed<19,-6> x_msb_3_lsb = x_l;
1235     //
std::cout<<"x_msb_3_lsb="<<x_msb_3_lsb.to_string()<<std::endl;
//1u
1236     ap_ufixed<20,-5> exp_x_msb_3_lsb_m_1 = 0;//carry
in
1237     exp_x_msb_3_lsb_m_1(18,0) = x_msb_3_lsb(18,0);
1238     exp_x_msb_3_lsb_m_1 = exp_x_msb_3_lsb_m_1+
exp_x_msb_3_lsb_m_1_m;//2u
1239     //
std::cout<<"exp_x_msb_3_lsb_m_1_m="<<exp_x_msb_3_lsb_m_1.to_string()<<std::endl;
1240
1241     //-2-6
1242     const static ap_ufixed<25,0> exp_x_msb_2_m_1_t-
able[32] = {
1243     "0x0.0000000p0", "0x0.04080a8p0",
"0x0.0820560p0", "0x0.0c49238p0",
1244     "0x0.1082b58p0", "0x0.14cd500p0",
"0x0.1929370p0", "0x0.1d96b10p0",
1245     "0x0.2216048p0", "0x0.26a7790p0",
"0x0.2b4b588p0", "0x0.3001ed0p0",
1246     "0x0.34cb818p0", "0x0.39a8628p0",
"0x0.3e98de8p0", "0x0.439d440p0",
1247     "0x0.48b5e40p0", "0x0.4de30f0p0",
"0x0.5325180p0", "0x0.587c540p0",
1248     "0x0.5de9178p0", "0x0.636bb98p0",
"0x0.6904930p0", "0x0.6eb3fc8p0",
1249     "0x0.747a510p0", "0x0.7a57ee0p0",

```

```

"0x0.804d300p0", "0x0.865a778p0",
1249 "0x0.8c80248p0", "0x0.92be998p0",
"0x0.99163b0p0", "0x0.9f876e8p0",
1250 };
1251 ap_ufixed<25, 0> exp_x_msb_2_m_1 =
exp_x_msb_2_m_1_table[x_msb_ind_2];
1252 //25*25
1253 ap_ufixed<50, -5> f_x_msb_2_3_lsb =
exp_x_msb_2_m_1 * exp_x_msb_3_lsb_m_1;
1254 ap_ufixed<20, -5> f_x_msb_2_3_lsb_s =
f_x_msb_2_3_lsb;
1255 ap_ufixed<25, 0> exp_x_msb_2_3_lsb_m_1 =
f_x_msb_2_3_lsb_s + exp_x_msb_3_lsb_m_1 + exp_x_msb_2_m_1;//3u
1256 //
std::cout<<"exp_x_msb_2_3_lsb_m_1"<<exp_x_msb_2_3_lsb_m_1.to_stri
ng())<<std::endl;
1257
1258 const static ap_ufixed<25,11> exp_x_msb_1_t-
able[32] = {
1259 "0x1.0000p0", "0x1.a614p0", "0x2.b7e0p0",
"0x4.7b50p0",
1260 "0x7.6398p0", "0xc.2eb8p0", "0x14.15e4p0",
"0x21.1d90p0",
1261 "0x36.9920p0", "0x5a.0464p0", "0x94.69c4p0",
"0xf4.b124p0",
1262 "0x193.6dc4p0", "0x299.2444p0",
"0x448.a218p0", "0x710.0adcp0",
1263 "0x0.0014p0", "0x0.0024p0", "0x0.003cp0",
"0x0.0064p0",
1264 "0x0.00a4p0", "0x0.010cp0", "0x0.01b8p0",
"0x0.02d8p0",
1265 "0x0.04b0p0", "0x0.07bcp0", "0x0.0cc0p0",
"0x0.1504p0",
1266 "0x0.22a4p0", "0x0.3920p0", "0x0.5e2cp0",
"0x0.9b44p0",
1267 };
1268 ap_ufixed<25,11> exp_x_msb_1 = exp_x_msb_1_t-
able[x_msb_ind_1];
1269 //
std::cout<<"exp_x_msb_1"<<exp_x_msb_1.to_string())<<std::endl;
1270 //25*25
1271 ap_ufixed<50,11> y_lo = exp_x_msb_1 *
exp_x_msb_2_3_lsb_m_1;
1272 //
std::cout<<"y_lo"<<y_lo.to_string())<<std::endl;
1273 ap_ufixed<25,11> y_lo_s = y_lo;
1274 //
std::cout<<"y_lo_s"<<y_lo_s.to_string())<<std::endl;
1275 ap_ufixed<25,11> y_l = y_lo_s + exp_x_msb_1;
1276 //std::cout<<"y_l"<<y_l.to_string())<<std::endl;
1277 y = y_l;
1278 //std::cout<<"y"<<y.to_string())<<std::endl;
1279 }
1280 y += delta;
1281 //if (I_s<12) { // overflow
1282 overf = 0;
1283 for (int j = 22; j >= I_s+10; j--) {
1284 #pragma HLS unroll
1285 if (y[j])
1286 overf = 1;
1287 }
1288 if (overf) {
1289 for (int j = 22; j >= I_s+10; j--){
1290 #pragma HLS unroll
1291 y[j] = 0;
1292 }
1293 for (int j = I_s+9; j >= 0; j--) {
1294 #pragma HLS unroll
1295 y[j] = 1;
1296 }
1297 }
1298 //}
1299 r = y;

1300 } else if (I_s==5) { // I_s = 5, M_ = 12..23, I_ <= 24,
FO_ <= 23/49, WO <= 47, WI <= 73
1301 //if WI > 73, maybe need to extend w;
1302 if(WI>73) return 0;
1303 const static int we = 24;
1304 const static int wf = 49;
1305 const static int w = 73;
1306 ap_ufixed<w,we> x_l = x;
1307 ap_ufixed<47,24> y = 0;
1308 ap_ufixed<5,5> x_l_int;
1309 x_l_int(4,0) = x_l(wf+4,wf);
1310 ap_ufixed<wf,0> x_l_fract;
1311 x_l_fract(wf-1,0) = x_l(wf-1,0);
1312 bool overf = 0;
1313 for (int j = wf+4; j < w-1; j++){
1314 #pragma HLS unroll
1315 if (x_l[w-1]!=x_l[j])
1316 overf = 1;
1317 }
1318 if (overf||((x_l_int==15)&&(x_l_fract>=ap_ufixed<wf,
0>("0x0.F1402743D99f8p0")))) { // overflow
1319 if (!x_l[w-1]) {
1320 for (int j = 0; j < 46; j++) {
1321 #pragma HLS unroll
1322 y[j] = 1;
1323 }
1324 }
1325 }
1326 }
1327 // const static int Maxprecision = 51;//24+23+4 =
51
1328 ap_uint<8> x_msb_ind_1;
1329 x_msb_ind_1[7] = x_l[w-1];
1330 x_msb_ind_1(6,0) = x_l(wf+3,wf-3);
1331
1332 ap_uint<8> x_msb_ind_2;
1333 x_msb_ind_2(7,0) = x_l(wf-4,wf-11);
1334 ap_uint<5> x_msb_ind_3;
1335 x_msb_ind_3(4,0) = x_l(wf-12,wf-16);
1336 ap_uint<3> x_msb_ind_4;
1337 x_msb_ind_4(2,0) = x_l(wf-17,wf-19);
1338
1339 ap_uint<6> x_msb_ind_5;
1340 x_msb_ind_5(5,0) = x_l(wf-20,wf-25);
1341
1342 ap_ufixed<8, -3> x_msb_2 = x_l;
1343 ap_ufixed<5, -11> x_msb_3 = x_l;
1344 ap_ufixed<3, -16> x_msb_4 = x_l;
1345 ap_ufixed<6, -19> x_msb_5 = x_l;
1346 //truncate
1347 ap_ufixed<7, -25> x_lsb = x_l;
1348
1349 const static ap_ufixed<12, -39> f_x_msb_5_t-
able[64] = {
1350 "0x0.000000000000p0", "0x0.000000000002p0",
"0x0.000000000008p0", "0x0.000000000012p0",
1351 "0x0.000000000020p0", "0x0.000000000032p0",
"0x0.000000000048p0", "0x0.000000000062p0",
1352 "0x0.000000000080p0", "0x0.0000000000A2p0",
"0x0.0000000000C8p0", "0x0.0000000000F2p0",
1353 "0x0.000000000120p0", "0x0.000000000152p0",
"0x0.000000000188p0", "0x0.0000000001C2p0",
1354 "0x0.000000000200p0", "0x0.000000000242p0",
"0x0.000000000288p0", "0x0.0000000002D2p0",
1355 "0x0.000000000320p0", "0x0.000000000372p0",
"0x0.0000000003C8p0", "0x0.000000000422p0",
1356 "0x0.000000000480p0", "0x0.0000000004E2p0",
"0x0.000000000548p0", "0x0.0000000005B2p0",
1357 "0x0.000000000620p0", "0x0.000000000692p0",
"0x0.000000000708p0", "0x0.000000000782p0",
1358 "0x0.000000000800p0", "0x0.000000000882p0",
"0x0.000000000908p0", "0x0.000000000992p0",
1359 "0x0.000000000A20p0", "0x0.000000000AB2p0",
"0x0.000000000B48p0", "0x0.000000000BE2p0",
1360 "0x0.000000000C80p0", "0x0.000000000D22p0",
"0x0.000000000DC8p0", "0x0.000000000E72p0",
1361 "0x0.000000000F20p0", "0x0.000000000FD2p0",
"0x0.000000001088p0", "0x0.000000001142p0",
1362 "0x0.000000001200p0", "0x0.0000000012C2p0",
"0x0.000000001388p0", "0x0.000000001452p0",
1363 "0x0.000000001520p0", "0x0.0000000015F2p0",
"0x0.0000000016C8p0", "0x0.0000000017A2p0",
1364 "0x0.000000001880p0", "0x0.000000001962p0",
"0x0.000000001A48p0", "0x0.000000001B32p0",
1365 "0x0.000000001C20p0", "0x0.000000001D12p0",
"0x0.000000001E08p0", "0x0.000000001F02p0",
1366
1367 };
1368 ap_ufixed<12, -39> f_x_msb_5 = f_x_msb_5_t-
able[x_msb_ind_5];
1369
1370 const static ap_ufixed<7, -33> f_x_msb_4_table[8]
= {
1371 "0x0.0000000000p0", "0x0.0000000002p0", "0x0.0000000008p0",
"0x0.0000000012p0",
1372 "0x0.0000000020p0", "0x0.0000000032p0", "0x0.0000000048p0",
"0x0.0000000062p0",
1373 };
1374 ap_ufixed<7, -33> f_x_msb_4 = f_x_msb_4_t-
able[x_msb_ind_4];
1375 //6*7
1376 ap_ufixed<7, -44> f_x_msb_5_lsb = x_msb_5 *
x_lsb;//Z_lsb*Z_5 //1u
1377
1378 ap_ufixed<16, -19> x_msb_5_lsb = x_l;
1379 //3*16

1380 ap_ufixed<19, -35> f_x_msb_4_5_lsb = x_msb_5_lsb *
x_msb_4_5_lsb;//(Z_5+Z_lsb)*Z_4
1381 ap_ufixed<16, -35> f_x_msb_4_5_lsb_s =
f_x_msb_4_5_lsb;//1u
1382
1383 ap_ufixed<13, -38> exp_x_msb_4_5_lsb_m_1_m3 = 0;
1384 exp_x_msb_4_5_lsb_m_1_m3(11,0)=f_x_msb_5(11,0);
1385 exp_x_msb_4_5_lsb_m_1_m3 =
exp_x_msb_4_5_lsb_m_1_m3+f_x_msb_5_lsb;//3u
1386
1387 ap_ufixed<17, -34> exp_x_msb_4_5_lsb_m_1_m4 = 0;
1388
1389 exp_x_msb_4_5_lsb_m_1_m4(15,0)=f_x_msb_4_5_lsb_s(15,0);
1390 exp_x_msb_4_5_lsb_m_1_m4 =
exp_x_msb_4_5_lsb_m_1_m4 + f_x_msb_4_5_lsb_s;//4u
1391
1392 ap_ufixed<8, -32> exp_x_msb_4_5_lsb_m_1_m5_s = 0;
1393

```



```

exp_x_msb_4_5_lsb_m_1_m5_s(5,0)=exp_x_msb_4_5_lsb_m_1_m4(16,11);
1393 exp_x_msb_4_5_lsb_m_1_m5_s =
f_x_msb_4+exp_x_msb_4_5_lsb_m_1_m5_s; //5u
1394
1395 ap_ufixed<19, -32> exp_x_msb_4_5_lsb_m_1_m5 =
exp_x_msb_4_5_lsb_m_1_m5_s;
1396
exp_x_msb_4_5_lsb_m_1_m5(10,0)=exp_x_msb_4_5_lsb_m_1_m4(10,0);
1397
1398 ap_ufixed<32, -15> exp_x_msb_4_5_lsb_m_1_m6 = 0;
1399 ap_ufixed<wf-16, -16> x_msb_4_5_lsb = x_l;
1400 exp_x_msb_4_5_lsb_m_1_m6(14,0) =
exp_x_msb_4_5_lsb_m_1_m5(18,4);
1401 exp_x_msb_4_5_lsb_m_1_m6 =
exp_x_msb_4_5_lsb_m_1_m6 + x_msb_4_5_lsb;
1402 ap_ufixed<36, -15> exp_x_msb_4_5_lsb_m_1 =
exp_x_msb_4_5_lsb_m_1_m6;
1403 exp_x_msb_4_5_lsb_m_1(3,0) =
exp_x_msb_4_5_lsb_m_1_m5(3,0);
1404
1405
1406 const static ap_ufixed<32, -23> f_x_msb_3_t-
able[32] = {
1407 "0x0.00000000000000p0", "0x0.0000000000002ap0",
"0x0.00000002000156p0", "0x0.00000004800480p0",
1408 "0x0.00000008000aa0p0", "0x0.0000000c8014d6p0",
"0x0.00000012002400p0", "0x0.000000180392cp0",
1409 "0x0.00000020005556p0", "0x0.00000028807982p0",
"0x0.0000003200a6acp0", "0x0.0000003c80dd8dp0",
1410 "0x0.00000048012004p0", "0x0.00000054816e30p0",
"0x0.0000006201c95cp0", "0x0.00000070823288p0",
1411 "0x0.0000008002aa6p0", "0x0.000000908332e2p0",
"0x0.000000a203cc12p0", "0x0.000000b4847740p0",
1412 "0x0.000000c8053570p0", "0x0.000000dc8607a0p0",
"0x0.000000f206eed0p0", "0x0.0000010887ec02p0",
1413 "0x0.00000120090036p0", "0x0.000001388a2c6ap0",
"0x0.000001520b71a0p0", "0x0.0000016c8cd0d6p0",
1414 "0x0.000001880e4b0ep0", "0x0.000001a48fe148p0",
"0x0.000001c2119484p0", "0x0.000001e09365c0p0",
1415 };
1416
1417 ap_ufixed<32, -23> f_x_msb_3 = f_x_msb_3_t-
able[x_msb_ind_3];
1418
1419 ap_ufixed<44, -11> exp_x_msb_3_m_1 = 0;
1420 exp_x_msb_3_m_1(43,39) = x_msb_3(4,0);
1421 exp_x_msb_3_m_1(31, 0) = f_x_msb_3(31,0);
1422
1423 //32*43 = (17+17)*(24+24)
1424 ap_ufixed<80, -26> f_x_msb_3_4_lsb =
exp_x_msb_3_m_1 * exp_x_msb_4_5_lsb_m_1; //1u
1425 ap_ufixed<31, -26> f_x_msb_3_4_lsb_s =
f_x_msb_3_4_lsb;
1426
1427 ap_ufixed<48, -10> exp_x_msb_3_4_lsb_m_1 =
f_x_msb_3_4_lsb_s + exp_x_msb_3_m_1 +
exp_x_msb_4_5_lsb_m_1; //1u+1u+5u=7u
1428
1429 const static ap_ufixed<46, -6> f_x_msb_2_t-
able[256] = {
1430 "0x0.0p0", "0x0.0000020015560p0", "0x0.00000800A855p0",
"0x0.0000120240360p0",
1431 "0x0.0000200556001p0", "0x0.0000320A6C48p0",
"0x0.0000481203608p0", "0x0.0000621C9B971p0",
1432 "0x0.0000802AB5577p0", "0x0.0000A23CD119dp0",
"0x0.0000C8536F668p0", "0x0.0000F26F15D0p0",
1433 "0x0.000109036103p0", "0x0.000152B75FCE3p0",
"0x0.000188E50ED86p0", "0x0.0001C319C4077p0",
1434 "0x0.0002015600445p0", "0x0.0002439A4487Ep0",
"0x0.000289E711DB3p0", "0x0.0002D43CE9577p0",
1435 "0x0.0003229C4C260p0", "0x0.00037505BB805p0",
"0x0.0003CB798B01p0", "0x0.000425F8C50F2p0",
1436 "0x0.0004848362076p0", "0x0.0004E71A11131p0", "0x0.00054DB-
D53BC8p0", "0x0.0005B86DAB9E3p0",
1437 "0x0.0006272B9A630p0", "0x0.000699F7A1C5Dp0",
"0x0.000710D24391Ep0", "0x0.000788BC01A29p0",
1438 "0x0.00080AB55DE39p0", "0x0.00088DBEDA50Bp0",
"0x0.000914D8F8F63p0", "0x0.0009A0043BF07p0",
1439 "0x0.000A2F41256C2p0", "0x0.000AC29037A63p0",
"0x0.000B59F1F4EBDp0", "0x0.000BF566DF9AAp0",
1440 "0x0.000C94EF7A206p0", "0x0.000D388C46FB4p0", "0x0.000DE03D-
C8B9Bp0", "0x0.000E8C0481FA7p0",
1441 "0x0.000F3BE0F450Ap0", "0x0.000FEFD3A5CFAP0", "0x0.0010A7D-
D15F36p0", "0x0.001163FDC8B7Fp0",
1442 "0x0.00122436410DDp0", "0x0.0012E88701F5Ep0",
"0x0.0013B0F08E817p0", "0x0.00147D7369D22p0",
1443 "0x0.00154E101719Fp0", "0x0.001622C7199B7p0", "0x0.0016F-
B98FA96p0", "0x0.0017D8862BA72p0",
1444 "0x0.0018B98F42084p0", "0x0.00199EB4BB511p0",
"0x0.001A87F71B161p0", "0x0.001B7556E4FC4p0",
1445 "0x0.001C6D49C893p0", "0x0.001D5C70C612Dp0",
"0x0.001E562BE4DFAP0", "0x0.001F54067D067p0",
1446 "0x0.00205601127ECp0", "0x0.00215C1C29507p0",
"0x0.0022665845940p0", "0x0.002374B5EB724p0",
1447 "0x0.00248F359F24Cp0", "0x0.00259DD7E4F57p0",
"0x0.0026B89D413FPp0", "0x0.0027D786386C4p0",
1448 "0x0.0028FA934EF90p0", "0x0.002A21C509717p0",
"0x0.002B4D1BEC726p0", "0x0.002C7C987CA92p0",
1449 "0x0.002DB03B3E03Ap0", "0x0.002E804B7C07p0",
"0x0.003023F56C4ECp0", "0x0.0031640DE16E3p0",
"0x0.0032A84E9C1F5p0", "0x0.0033F08821730p0",

```

```

"0x0.00353D4AF68B0p0", "0x0.00368E07A0999p0",
1450 "0x0.0037E2EEA4E19p0", "0x0.00393C008886Cp0", "0x0.003A993D-
D17D5p0", "0x0.003BFAA704AA3p0",
1451 "0x0.003D603CA7C32p0", "0x0.003EC9FF405E6p0", "0x0.004037E-
F5423p0", "0x0.0041AA0D68C8Cp0",
1452 "0x0.0043205A04182p0", "0x0.00449AD5ABEA5p0",
"0x0.00461980E6294p0", "0x0.00479C5C38CF9p0",
1453 "0x0.0049236829E8Bp0", "0x0.004AAEAS3F90Ep0",
"0x0.004C3E13FF51p0", "0x0.004DD1B4F152Fp0",
1454 "0x0.004F698899F90p0", "0x0.0051058F8046Ap0",
"0x0.0052A5CA2AABFp0", "0x0.00544A391FA9Cp0",
1455 "0x0.0055F2DC5E5D1Ep0", "0x0.00579FB603C6Ep0",
"0x0.005950C5003C2p0", "0x0.005B060A61F5Dp0",
1456 "0x0.005CBF86AFC91p0", "0x0.005E7D3A709BEp0",
"0x0.00603F262B650p0", "0x0.0062054A672C2p0",
1457 "0x0.0063CFA7AB09Dp0", "0x0.00659E3E7E278p0",
"0x0.0067710F67BFAp0", "0x0.0069481AEF1D5p0",
1458 "0x0.006B23619B9CFp0", "0x0.006D02E3F4AB7p0",
"0x0.006EE6A281C6Fp0", "0x0.0070CE9DCA7E6p0",
1459 "0x0.0072BAD65671Bp0", "0x0.0074AB4CAD51Cp0",
"0x0.0076A00156E07p0", "0x0.007898F4DAF09p0",
1460 "0x0.007A9627C165Fp0", "0x0.007C979A92356p0", "0x0.007E9D4D-
D564Ap0", "0x0.0080A742130A8p0",
1461 "0x0.0082B577D34EDp0", "0x0.0084C7EF9E6A7p0",
"0x0.0086DEA9FCA73p0", "0x0.0088F9A776601p0",
1462 "0x0.008B18E894010p0", "0x0.008D3C6DDE06Fp0", "0x0.008F6437D-
D000p0", "0x0.00919047198B6p0",
1463 "0x0.0093C09C1C595p0", "0x0.0095F5376E2B2p0",
"0x0.00982E1997D33p0", "0x0.009A6B4322352p0",
1464 "0x0.009CAB496458p0", "0x0.009EF26E7D0A2p0",
"0x0.00A13C715F99Ep0", "0x0.00A38ABDC71CEp0",
1465 "0x0.00A5DD543CC4p0", "0x0.00A8343549F26p0",
"0x0.00AA8F6177EADp0", "0x0.00ACEED950222p0",
1466 "0x0.00AF529D5C165p0", "0x0.00B1BAAE25566p0",
"0x0.00B4270C3582Ap0", "0x0.00B697B8164C7p0",
1467 "0x0.00B90CB25176Ap0", "0x0.00BB85F70D50p0",
"0x0.00BE0393FE4CCp0", "0x0.00C0857C83D44p0",
1468 "0x0.00C30BB58B732p0", "0x0.00C5963F9F424p0",
"0x0.00C8251B496BEp0", "0x0.00CAB849142B5p0",
1469 "0x0.00CD4FC989CD6p0", "0x0.00CFEB9D34B00p0", "0x0.00D28B-
C49F428p0", "0x0.00D5304054059p0",
1470 "0x0.00D7D910DD8B1p0", "0x0.00DA8636C6764p0", "0x0.00D-
D37B2997BCp0", "0x0.00DFED84E1618p0",
1471 "0x0.00E2A7AE28FECp0", "0x0.00E5662EFB3C3p0",
"0x0.00E82907E313Dp0", "0x0.00EAF0396B010p0",
1472 "0x0.00EDB8C41FD08p0", "0x0.00F08BA88B009p0",
"0x0.00F35FE73860Bp0", "0x0.00F63880B341Ep0",
1473 "0x0.00F9157587069p0", "0x0.00FBFC63F228p0", "0x0.00FED-
C73671B0p0", "0x0.0101C67D8A86Cp0",
1474 "0x0.0104B4E5350DFp0", "0x0.0107A7AAF26A2p0",
"0x0.010A9ECF4E667p0", "0x0.010D9A52D4DF8p0",
1475 "0x0.01109A3611C34p0", "0x0.01139F7991116p0", "0x0.0116A71D-
DEDAdp0", "0x0.0119B42387423p0",
1476 "0x0.011CC58B167B9p0", "0x0.011FDB5518CCAp0",
"0x0.0122F5821A8C7p0", "0x0.01261412A823Cp0",
1477 "0x0.012937074E0Dp0", "0x0.012C5E6098D37p0",
"0x0.012F8A1F15151p0", "0x0.0132BA443AF80Ap0",
1478 "0x0.0135CECDD4D6Cp0", "0x0.013927BF31E98p0",
"0x0.013C6517F39CCp0", "0x0.013FA608A6E5Dp0",
1479 "0x0.0142ED01D8CBcp0", "0x0.0146379416673p0", "0x0.0149868FE-
CE26p0", "0x0.014CD9F5E9795p0",
1480 "0x0.015031C699799p0", "0x0.01538E028A426p0",
"0x0.0156EEAA49448p0", "0x0.015A53BE64033p0",
1481 "0x0.015DBD3F68122p0", "0x0.01612B2DE3178p0",
"0x0.01649D8A62CB1p0", "0x0.0168145574F65p0",
1482 "0x0.0168B8F8A7745p0", "0x0.016F0F3988321p0",
"0x0.01729353A5E24p0", "0x0.01761BDE8C792p0",
1483 "0x0.0179A8DACC350p0", "0x0.017D3A48F295Dp0",
"0x0.0180D0298DE13p0", "0x0.01846A7D2C6Cp0",
1484 "0x0.018809445CA7Bp0", "0x0.018BAC7FAD074p0", "0x0.018F542-
FAC1A4p0", "0x0.01930054E87F8p0",
1485 "0x0.0196B0EFF0E79p0", "0x0.019A66015414Dp0",
"0x0.019E1F789A0D6Bp0", "0x0.01A1DB986621p0",
1486 "0x0.01A5A00132E02p0", "0x0.01A966F1961FCp0",
"0x0.01AD325B1EFC9p0", "0x0.01B1023E5CA45p0",
1487 "0x0.01B4D69BDE569p0", "0x0.01B8AF743364Cp0", "0x0.01BC8CC7E-
B323p0", "0x0.01C06E9795345p0",
1488 "0x0.01C454E3C0F24p0", "0x0.01C83FACFE054p0", "0x0.01CC2EF3D-
C187p0", "0x0.01D022B8EAE8Fp0",
1489 "0x0.01D41AFCBA45Ep0", "0x0.01D817BFD104p0", "0x0.01D-
C1902DA3B1p0", "0x0.01E01EC64ACB7p0",
1490 "0x0.01E4290ABDD87p0", "0x0.01E837D0DB8B2p0",
"0x0.01EC4B18E01E9p0", "0x0.01F06E23B3DFEp0",
1491 "0x0.01F47F31C92E4p0", "0x0.01F8A003B07AFp0", "0x0.01FCC559-
FA492p0", "0x0.0200EF35372E4p0",
1492 "0x0.02051D95F7D1Cp0", "0x0.0209507CCCE1Dp0",
"0x0.020D87EA4748Ep0", "0x0.0211C3DEF7C8Dp0",
1493 };
1494 ap_ufixed<46, -6> f_x_msb_2 = f_x_msb_2_t-
able[x_msb_ind_2];
1495
1496 ap_ufixed<5, -6> f_x_msb_2_h;
1497 f_x_msb_2_h(4,0) = f_x_msb_2(45,41);
1498 ap_ufixed<9, -2> exp_x_msb_2_m_1_h = f_x_msb_2_h
+ x_msb_2;
1499
1500 ap_ufixed<50, -2> exp_x_msb_2_m_1;
1501 exp_x_msb_2_m_1(49,41) = exp_x_msb_2_m_1_h(8,0);
1502 exp_x_msb_2_m_1(40, 0) = f_x_msb_2(40,0);
1503 //50*48=(17+17+17)*(24+24)
1504 ap_ufixed<98, -12> f_x_msb_2_3_4_lsb =
exp_x_msb_2_m_1 * exp_x_msb_3_4_lsb_m_1; //1u

```

```

1503         ap_ufixed<46,-12> f_x_msb_2_3_4_lsb_s =
f_x_msb_2_3_4_lsb;
1504         ap_ufixed<50,-2> exp_x_msb_2_3_4_lsb_m_1 =
f_x_msb_2_3_4_lsb_s + exp_x_msb_2_m_1 + exp_x_msb_3_4_lsb_m_1;//9u
1505
1506         const static ap_ufixed<50,23> exp_x_msb_1_t-
able[256] = {
1507     "0x1.0000000p0", "0x1.2216045p0", "0x1.48B5E3Cp0",
"0x1.747A513p0",
1508     "0x1.A61298Ep0", "0x1.DE455DFp0", "0x2.1DF3B68p0",
"0x2.661CB0Fp0",
1509     "0x2.B7E1516p0", "0x3.1489176p0", "0x3.7D871DBp0",
"0x3.F47FE87p0",
1510     "0x4.7B4FF99p0", "0x5.1413452p0", "0x5.C12DA41p0",
"0x6.855466Ep0",
1511     "0x7.63992E3p0", "0x8.5F7635Bp0", "0x9.7CDC417p0",
"0xA.C042667p0",
1512     "0xC.2EB7EC9p0", "0xD.CDF892Ep0", "0xF.A48385Ep0",
"0x11.B9B5652p0",
1513     "0x14.15E5BF6p0", "0x16.C2887C1p0", "0x19.CA53B78p0", "0x1D.
396AA97p0",
1514     "0x21.1D8E427p0", "0x25.865441Cp0", "0x2A.8565A14p0",
"0x30.2EC550Bp0",
1515     "0x36.99205C4p0", "0x3D.DE28BF3p0", "0x46.1AFC4DBp0", "0x4F.
7099532p0",
1516     "0x5A.0462B78p0", "0x66.00B5BC9p0", "0x73.9593ABBp0",
"0x82.F9621ADp0",
1517     "0x94.69C4C88p0", "0xA8.2C948C1p0", "0xBE.90F6F83p0",
"0xD7.F09B78Cp0",
1518     "0xF4.B122790p0", "0x115.45B4704p0", "0x13A.30CF1CCp0",
"0x164.0650296p0",
1519     "0x193.6DC5690p0", "0x1C9.250BEDCp0", "0x206.03487A3p0",
"0x24A.FC4533Cp0",
1520     "0x299.2442102p0", "0x2F1.B447462p0", "0x356.0F0B0F9p0",
"0x3C7.C67E58Ep0",
1521     "0x448.A216AB8p0", "0x4DA.A5EE46Ap0", "0x580.1AD754Fp0",
"0x63B.9782341p0",
1522     "0x710.0ADBAC7p0", "0x800.C7CC8E3p0", "0x911.9289C39p0", "0x-
A46.AFAA2ADp0",
1523     "0xBA4.F53EA38p0", "0xD31.DE30C52p0", "0xEF3.A035D57p0",
"0x10F1.44ADE60p0",
1524     "0x1332.C4D2B7Cp0", "0x15C1.29A744Cp0", "0x18A6.B027DA0p0",
"0x18EE.F24BB12p0",
1525     "0x1FA7.157C470p0", "0x23DD.FF3C8BAp0", "0x28A4.90D2CCEp0",
"0x2E0D.EAE454Bp0",
1526     "0x342F.BA11823p0", "0x3B22.8DC5359p0", "0x4302.3A933EEp0",
"0x4BEE.49AFCDcp0",
1527     "0x560A.773E541p0", "0x617F.4171BF9p0", "0x6E7A.8ABB4A8p0",
"0x7D30.5191764p0",
1528     "0x80DB.80AF426p0", "0xA0BE.DB0DB10p0", "0xB626.0748BAEp0",
"0xC666.BE9EADp0",
1529     "0xE9E2.2447727p0", "0x10906.4A831F8p0", "0x12C4F.EB75B2Ap0",
"0x1544C.5CB6133p0",
1530     "0x1819B.C560F61p0", "0x1B4F3.9F8AF64p0", "0x1E-
F21.8F1B001p0", "0x2310E.996C40Fp0",
1531     "0x27BC2.CA9A6F9p0", "0x2D069.571279Ap0",
"0x33055.49F35D9p0", "0x39D06.D2FEF17p0",
1532     "0x41831.49596E9p0", "0x4A3C1.FB2AC39p0",
"0x541E7.E56949Dp0", "0x5F51C.72B7532p0",
1533     "0x6C02D.645AB25p0", "0x7A648.0CFF534p0", "0x8-
AB06.0A3EE9Fp0", "0x9D27B.AFE4CF1p0",
1534     "0xB2148.5EAE56Cp0", "0xC9CA9.07F86EFp0", "0xE4A8D.2881ED-
Bp0", "0x1031AE.8E4996Cp0",
1535     "0x1259AC.48BF05Dp0", "0x14CB29.2F2B31Fp0", "0x178FEE.
7792E44p0", "0x1AB312.E89C00Dp0",
1536     "0x1E4127.437732Bp0", "0x224868.979FC2Ep0",
"0x26D8F9.4A204BEp0", "0x2C0521.B4A8E0Bp0",
1537     "0x31E199.5F5A550p0", "0x3885D9.FA89D00p0", "0x400C7D.
64D3386p0", "0x4893A8.361032Dp0",
1538     "0x523D82.79EDAEFp0", "0x5D30C0.7DAB686p0",
"0x69993D.D4F2D95p0", "0x77A8AD.02A7C71p0",
1539     "0x0.000001Ep0", "0x0.0000022p0", "0x0.0000026p0",
"0x0.000002Bp0",
1540     "0x0.0000031p0", "0x0.0000038p0", "0x0.000003Fp0",
"0x0.0000048p0",
1541     "0x0.0000052p0", "0x0.000005Dp0", "0x0.0000069p0",
"0x0.0000077p0",
1542     "0x0.0000087p0", "0x0.0000099p0", "0x0.00000ADp0",
"0x0.00000C4p0",
1543     "0x0.00000DFp0", "0x0.00000FCp0", "0x0.000011Ep0",
"0x0.0000144p0",
1544     "0x0.0000170p0", "0x0.00001A1p0", "0x0.00001D8p0",
"0x0.0000217p0",
1545     "0x0.000025Ep0", "0x0.00002AFp0", "0x0.000030Bp0",
"0x0.0000372p0",
1546     "0x0.00003E8p0", "0x0.000046Dp0", "0x0.0000504p0",
"0x0.00005AFp0",
1547     "0x0.0000671p0", "0x0.000074Cp0", "0x0.0000845p0",
"0x0.000095Fp0",
1548     "0x0.0000A9Fp0", "0x0.0000C09p0", "0x0.0000DA3p0",
"0x0.0000F74p0",
1549     "0x0.0001183p0", "0x0.00013D8p0", "0x0.000167Cp0",
"0x0.000197Bp0",
1550     "0x0.0001CDFp0", "0x0.00020B7p0", "0x0.0002513p0",
"0x0.0002A02p0",
1551     "0x0.0002F9Ap0", "0x0.00035F1p0", "0x0.0003D20p0",
"0x0.0004543p0",
1552     "0x0.0004E7Cp0", "0x0.00058F0p0", "0x0.00064C7p0",
"0x0.0007232p0",
1553     "0x0.0008167p0", "0x0.00092A2p0", "0x0.000A628p0", "0x0.000B-
C48p0",
1554     "0x0.000D55Ap0", "0x0.000F1C2p0", "0x0.00111F3p0",
"0x0.001366Cp0",
1555     "0x0.0015FC2p0", "0x0.0018E98p0", "0x0.001C3AAp0",
"0x0.001FFCEp0",
1556     "0x0.00243F3p0", "0x0.002912Bp0", "0x0.002E8ABp0",
"0x0.0034BD2p0",
1557     "0x0.003BC2Dp0", "0x0.0043B7Dp0", "0x0.004CBC1p0",
"0x0.0056F3Ap0",
1558     "0x0.0062878p0", "0x0.006FA5Fp0", "0x0.007E83Ap0",
"0x0.008F5C0p0",
1559     "0x0.00A2728p0", "0x0.00B813Bp0", "0x0.00D0963p0",
"0x0.00EC5C1p0",
1560     "0x0.010BD4Ap0", "0x0.012F7DEp0", "0x0.0157E6Bp0",
"0x0.0185B0Fp0",
1561     "0x0.01B993Fp0", "0x0.01F45F9p0", "0x0.0236FF5p0",
"0x0.02827E0p0",
1562     "0x0.02D80A0p0", "0x0.0338F9Fp0", "0x0.03A6D22p0",
"0x0.04234A7p0",
1563     "0x0.04B0556p0", "0x0.0550280p0", "0x0.0605424p0",
"0x0.06D279Ap0",
1564     "0x0.07BB040p0", "0x0.08C284Cp0", "0x0.09ED1B4p0",
"0x0.0B3F736p0",
1565     "0x0.0CBED86p0", "0x0.0E7149Cp0", "0x0.105D938p0",
"0x0.128B697p0",
1566     "0x0.150385Cp0", "0x0.17CFC22p0", "0x0.1AFB718p0",
"0x0.1E9328Bp0",
1567     "0x0.22A5554p0", "0x0.2742456p0", "0x0.2C7C72Fp0",
"0x0.3268CDBp0",
1568     "0x0.391F0EEp0", "0x0.40BA188p0", "0x0.495860Dp0",
"0x0.531C6C9p0",
1569     "0x0.5E2D58Dp0", "0x0.6AB7782p0", "0x0.78ED03Ap0",
"0x0.8906E49p0",
1570     "0x0.9B4597Ep0", "0x0.AFF230Ap0", "0x0.C75F7CFp0", "0x0.E1E-
B512p0",
1571     };
1572     ap_ufixed<50,23> exp_x_msb_1 = exp_x_msb_1_t-
able[x_msb_ind_1];
1573     //55*50=(17+17+17)*(17+17+17)
1574     ap_ufixed<100,21> y_lo = exp_x_msb_1 *
exp_x_msb_2_3_4_lsb_m_1;
1575     ap_ufixed<48,21> y_lo_s = y_lo;
1576     ap_ufixed<50,23> y_l = y_lo_s + exp_x_msb_1;
1577     y = y_l;
1578 }
1579 y += delta;
1580 //if (I_<24) { // overflow
1581     overf = 0;
1582     for (int j = 46; j >= I_+22; j--) {
1583         #pragma HLS unroll
1584         if (y[j])
1585             overf = 1;
1586     }
1587     if (overf) {
1588         for (int j = 46; j >= I_+22; j--) {
1589             #pragma HLS unroll
1590             y[j] = 0;
1591         }
1592         for (int j = I_+21; j >= 0; j--) {
1593             #pragma HLS unroll
1594             y[j] = 1;
1595         }
1596     }
1597     //
1598     r = y;
1599 } else { // I_s_ = 6, M_ = 24..32, I_ <= 33, FO_ <=
32/27, WO <= 65, WI <= 100
1600     const static int wf = 67;
1601     const static int we = 33;
1602     const static int w = 100;
1603     ap_fixed<w,we> x_l = x;
1604     //std::cout<<"x="<<x.to_string()<<std::endl;
1605     //std::cout<<"x_l="<<x_l.to_string()<<std::endl;
1606     ap_ufixed<65,33> y = 0;
1607
1608     ap_fixed<6,6> x_l_int;
1609     x_l_int(5,0) = x_l(wf+5,wf);
1610     //
1611     std::cout<<"x_l_int="<<x_l_int.to_string()<<std::endl;
1612
1613     ap_ufixed<wf,0> x_l_fract;
1614     x_l_fract(wf-1,0) = x_l(wf-1,0);
1615     //
1616     std::cout<<"x_l_fract="<<x_l_fract.to_string()<<std::endl;
1617     bool overf = 0;
1618     for (int j = wf+5; j < w-1; j++){
1619         #pragma HLS unroll
1620         if (x_l[w-1]!=x_l[j])
1621             overf = 1;
1622     }
1623     if (overf||x_l_int>22){
1624         ((x_l_int==22)&&(x_l_fract>=ap_ufixed<wf,0>("0x0.2E42FEFA39E-
F35783p0")))) { // overflow
1625             if (!x_l[w-1]) {
1626                 for (int j = 0; j < 64; j++) {
1627                     #pragma HLS unroll
1628                     y[j] = 1;
1629                 }
1630             }
1631         }
1632     }

```

```

0x0.00000000029A201FA8p0", "0x0.0000000002A3482050p0",
1689 "0x0.0000000002AC8020FAP0", "0x0.0000000002B5C821A6p0",
"0x0.0000000002BF202255p0", "0x0.0000000002C8882306p0",
1690 "0x0.0000000002D20023B9p0", "0x0.0000000002DB88246Fp0",
"0x0.0000000002E5202527p0", "0x0.0000000002EEC825E1p0",
1691 "0x0.0000000002F880269Ep0", "0x0.00000000030248275Dp0",
"0x0.00000000030C20281Fp0", "0x0.00000000031608281Ep0",
1692 "0x0.0000000003200029AAp0", "0x0.00000000033A082A73p0",
"0x0.00000000034202B3Fp0", "0x0.00000000033E482C0Dp0",
1693 "0x0.000000000348802CDp0", "0x0.000000000352C82DB2p0",
"0x0.00000000035D202E88p0", "0x0.000000000367882F60p0",
1694 "0x0.00000000037200303Cp0", "0x0.00000000037C883119p0",
"0x0.0000000003872031FAp0", "0x0.000000000391C832DDp0",
1695 "0x0.00000000039C80383Cp0", "0x0.0000000003A74834ABp0",
"0x0.0000000003B2203596p0", "0x0.0000000003BD083684p0",
1696 "0x0.0000000003C8003775p0", "0x0.0000000003D0833868p0",
"0x0.0000000003DE20395Ep0", "0x0.0000000003E9483A57p0",
1697 "0x0.0000000003F4803B53p0", "0x0.0000000003FFC83C52p0",
"0x0.00000000040B203D53p0", "0x0.000000000416883E57p0",
1698 "0x0.000000000422003F5Ep0", "0x0.00000000042D884068p0",
"0x0.000000000439204175p0", "0x0.000000000444C84285p0",
1699 "0x0.000000000450804397p0", "0x0.000000000454C84ADAp0",
"0x0.0000000004682045C5p0", "0x0.0000000004740846E1p0",
1700 "0x0.000000000480004800p0", "0x0.00000000048C084921p0",
"0x0.000000000498204A46p0", "0x0.0000000004A4484B6Dp0",
1701 "0x0.0000000004B0804C98p0", "0x0.0000000004BCC84DC5p0",
"0x0.0000000004C9204EF6p0", "0x0.0000000004D588502Ap0",
1702 "0x0.0000000004E2005161p0", "0x0.0000000004EE8529B0p0",
"0x0.0000000004FB2053D8p0", "0x0.000000000507C85518p0",
1703 "0x0.00000000051480565Cp0", "0x0.0000000005214857A3p0",
"0x0.00000000052E058EDp0", "0x0.00000000053B085A3Ap0",
1704 "0x0.0000000005480058BAp0", "0x0.00000000055085CDp0",
"0x0.000000000562205E35p0", "0x0.00000000056F485F8p0",
1705 "0x0.00000000057C806ECp0", "0x0.000000000589C862Dp0",
"0x0.0000000005972063B1p0", "0x0.0000000005A4886519p0",
1706 "0x0.0000000005B2006684p0", "0x0.0000000005BF8867F2p0",
"0x0.0000000005CD206963p0", "0x0.0000000005DC8AD6B8p0",
1707 "0x0.0000000005E8806C51p0", "0x0.0000000005F6486DCDp0",
"0x0.00000000060420664Cp0", "0x0.0000000006120870CFp0",
1708 "0x0.000000000620007255p0", "0x0.00000000062E0873DFp0",
"0x0.00000000063C207562p0", "0x0.00000000064A487F62p0",
1709 "0x0.000000000658807891p0", "0x0.000000000666C87A29p0",
"0x0.000000000675207BC4p0", "0x0.000000000683887064p0",
1710 "0x0.000000000692007F06p0", "0x0.0000000006A08880ADp0",
"0x0.0000000006AF208256p0", "0x0.0000000006BDC8404Bp0",
1711 "0x0.0000000006CC8085B5p0", "0x0.0000000006DE48876Ap0",
"0x0.0000000006EA208923p0", "0x0.0000000006F9088ADFp0",
1712 "0x0.000000000708008CA0p0", "0x0.00000000071088E63p0",
"0x0.00000000072620902Bp0", "0x0.0000000007354891F6p0",
1713 "0x0.0000000007488093C6p0", "0x0.000000000753C89599p0",
"0x0.000000000763209770p0", "0x0.00000000077288994Ap0",
1714 "0x0.000000000782009B29p0", "0x0.000000000791889D0Bp0",
"0x0.0000000007A1209EF2p0", "0x0.0000000007B0C8A0DCp0",
1715 "0x0.0000000007C080A2CAp0", "0x0.0000000007D048A4BCp0",
"0x0.0000000007E020A6B2p0", "0x0.0000000007F008AD08p0",
1716 );
1717 ap_ufixed<32, -37> f_x_msb_4 = f_x_msb_4_t-
able[x_msb_ind_4];
1718
1719 const static ap_ufixed<16, -53> f_x_msb_5_t-
able[256] = {
1720 "0x0.0000000000000000p0", "0x0.0000000000000000p0",
"0x0.0000000000000002p0", "0x0.0000000000000048p0",
1721 "0x0.0000000000000008p0", "0x0.000000000000000Cp0",
"0x0.0000000000000012p0", "0x0.0000000000000018p0",
1722 "0x0.0000000000000000p0", "0x0.0000000000000028p0",
"0x0.0000000000000032p0", "0x0.0000000000000000p0",
1723 "0x0.0000000000000048p0", "0x0.00000000000000548p0",
"0x0.0000000000000062p0", "0x0.00000000000000708p0",
1724 "0x0.0000000000000000p0", "0x0.0000000000000008p0",
"0x0.00000000000000A2p0", "0x0.00000000000000B8p0",
1725 "0x0.0000000000000000C8p0", "0x0.00000000000000DC8p0",
"0x0.00000000000000F2p0", "0x0.0000000000000188p0",
1726 "0x0.00000000000000120p0", "0x0.00000000000001388p0",
"0x0.0000000000000152p0", "0x0.000000000000016C8p0",
1727 "0x0.0000000000000188p0", "0x0.00000000000001A48p0",
"0x0.00000000000001C2p0", "0x0.00000000000001E08p0",
1728 "0x0.0000000000000200p0", "0x0.00000000000002208p0",
"0x0.0000000000000242p0", "0x0.00000000000002648p0",
1729 "0x0.0000000000000288p0", "0x0.00000000000002AC8p0",
"0x0.00000000000002D2p0", "0x0.00000000000002F88p0",
1730 "0x0.000
```



```

1792 ap_ufixed<25,-44> f_x_msb_4_5_lsb = x_msb_5_lsb *
x_msb_4;//(Z_5+Z_lsb)*Z_4 // 1u
1793 //6*8
1794 // ap_ufixed<6,-37> f_x_msb_4_s = f_x_msb_4;
1795 ap_ufixed<6,-63> exp_x_msb_4_5_lsb_m_1_m1 =
x_msb_5 * f_x_msb_4;//Z_5 * f(Z_4) //1u
1796
1797 ap_ufixed<10,-59> exp_x_msb_4_5_lsb_m_1_m2 = 0;
1798 exp_x_msb_4_5_lsb_m_1_m2(8,0) =
f_x_msb_5_lsb(8,0);
1799 exp_x_msb_4_5_lsb_m_1_m2 =
exp_x_msb_4_5_lsb_m_1_m1 + exp_x_msb_4_5_lsb_m_1_m2;//2u
1800
1801 ap_ufixed<17,-52> exp_x_msb_4_5_lsb_m_1_m3 = 0;
1802 exp_x_msb_4_5_lsb_m_1_m3(15,0) = f_x_msb_5(15,0);
1803 exp_x_msb_4_5_lsb_m_1_m3 =
exp_x_msb_4_5_lsb_m_1_m2+exp_x_msb_4_5_lsb_m_1_m3;//3u;
1804
1805 ap_ufixed<26,-43> exp_x_msb_4_5_lsb_m_1_m4 = 0;
1806 exp_x_msb_4_5_lsb_m_1_m4(24,0) =
f_x_msb_4_5_lsb(24,0);
1807 exp_x_msb_4_5_lsb_m_1_m4 =
exp_x_msb_4_5_lsb_m_1_m4 + exp_x_msb_4_5_lsb_m_1_m3;//4u
1808
1809 ap_ufixed<33,-36> exp_x_msb_4_5_lsb_m_1_m5 = 0;
1810 exp_x_msb_4_5_lsb_m_1_m5(31,0) = f_x_msb_4(31,0);
1811 exp_x_msb_4_5_lsb_m_1_m5 =
exp_x_msb_4_5_lsb_m_1_m5 + exp_x_msb_4_5_lsb_m_1_m4;//5u
1812
1813 ap_ufixed<wf-18,-18> x_msb_4_5_lsb_s = x_l;
1814 ap_ufixed<wf-17,-17> exp_x_msb_4_5_lsb_m_1_s = 0;
1815 exp_x_msb_4_5_lsb_m_1_s(30,0) =
exp_x_msb_4_5_lsb_m_1_m5(32,2);
1816 exp_x_msb_4_5_lsb_m_1_s = exp_x_msb_4_5_lsb_m_1_s
+ x_msb_4_5_lsb;
1817
1818 ap_ufixed<52,-17> exp_x_msb_4_5_lsb_m_1 =
exp_x_msb_4_5_lsb_m_1_s;
1819 exp_x_msb_4_5_lsb_m_1(1,0) =
exp_x_msb_4_5_lsb_m_1_m5(1,0);
1820
1821
1822 const static ap_ufixed<51,-21> f_x_msb_3_t-
able[256] = {
1823 "0x0", "0x0.0000000000000000AAAAp0",
"0x0.00000000200005556p0", "0x0.000000004800120003p0",
1824 "0x0.00000000800020002AAB5p0", "0x0.00000000C0053556Fp0",
"0x0.000000012000900036p0", "0x0.000000018000E4A80Ep0",
1825 "0x0.000000020001555600p0", "0x0.000000028001E60111p0",
"0x0.0000000320029AAC4Bp0", "0x0.00000003C8037757B7p0",
1826 "0x0.000000040004800360p0", "0x0.000000054805B8AF50p0",
"0x0.000000062007255B96p0", "0x0.000000070808CA0835p0",
1827 "0x0.00000008000000AAAB55p0", "0x0.000000090800CCB62EDp0",
"0x0.0000000A200F301116p0", "0x0.0000000B4811DCBF00p0",
1828 "0x0.0000000C8014D56F60p0", "0x0.0000000DC8181E1FA7p0",
"0x0.0000000F201BBAD0CBp0", "0x0.00000010881FAF82E1p0",
1829 "0x0.000000120024003600p0", "0x0.000000138828B0EA3Fp0",
"0x0.00000015202DC59F86p0", "0x0.00000016C8342567Fp0",
1830 "0x0.000000188039280EB5p0", "0x0.0000001A483F83C874p0",
"0x0.0000001C2046583D6p0", "0x0.0000001E084D9540Fp0",
1831 "0x0.00000020005556001p0", "0x0.00000022085D9BC106p0",
"0x0.0000002420665B842Cp0", "0x0.00000026486FA84995p0",
1832 "0x0.000000280079811161p0", "0x0.0000002AC883E9DBB6p0",
"0x0.0000002D208EE6A888p0", "0x0.0000002F889A7B78CBp0",
1833 "0x0.0000003200A6AC4B58p0", "0x0.0000003488B37D2145p0",
"0x0.0000003720C0F1FA7Ap0", "0x0.00000039C8CF0ED721p0",
1834 "0x0.0000003C80DD7B765p0", "0x0.0000003F4ED7509B71p0",
"0x0.0000004220FD7D8371p0", "0x0.00000045090E626F9Ap0",
1835 "0x0.000000480120036008p0", "0x0.0000004B09326545FBp0",
"0x0.0000004E2145894E9Fp0", "0x0.000000514959764026p0",
1836 "0x0.00000054816E2F50C1p0", "0x0.00000057C983B859A4p0",
"0x0.0000005B219A156804p0", "0x0.0000005E89B14A7F16p0",
1837 "0x0.00000062019C598611p0", "0x0.0000006589E24CB62Cp0",
"0x0.0000006921FC21DCA0p0", "0x0.0000006CCA16DF09A6p0",
1838 "0x0.000000708232883B78p0", "0x0.0000007444A42127853p0",
"0x0.00000078226ACA873p0", "0x0.0000007C0A8B34004141p0",
1839 "0x0.0000008002AAB5577p0", "0x0.000000840ACB36AEDAp0",
"0x0.0000008822ECBC107Dp0", "0x0.0000008C480F497AA3p0",
1840 "0x0.0000009083322E2D8Ep0", "0x0.00000094C8578C6981p0",
"0x0.00000099237D49ECC0p0", "0x0.0000009D8BA41F7092p0",
1841 "0x0.000000A203C11163Dp0", "0x0.000000A68BF522B908p0",
"0x0.000000AB241F58633p0", "0x0.000000AFC4AB61E22p0",
1842 "0x0.000000B484773FE105p0", "0x0.000000B94CA49FA31p0",
"0x0.000000BE24D3C788F1p0", "0x0.000000C30D0406E94p0",
1843 "0x0.000000C805356F6068p0", "0x0.000000CD068115EBCp0",
"0x0.000000D2259BF769E1p0", "0x0.000000D74DD1258228p0",
1844 "0x0.000000D80B079FA7E4p0", "0x0.000000DE1E03F69D69p0",
"0x0.000000E726788B1D0807p0", "0x0.000000EC8EB2FE6C1p0",
1845 "0x0.000000F026EED0C8DFp0", "0x0.000000F782C0339FAp0",
"0x0.000000FD276A98B71p0", "0x0.00000102FCFAA9844B8p0",
1846 "0x0.0000010887CE0E2231p0", "0x0.00000105E02ED9D903Bp0",
"0x0.000001142873C4F14p0", "0x0.000001A108BF31F3Bdp0",
1847 "0x0.00000120090036013p0", "0x0.000001261148F84FCBp0",
"0x0.0000012C29933FFAF5p0", "0x0.0000013251DF0F13E1p0",
1848 "0x0.000001386A2C6A3FF3p0", "0x0.0000013ED27B557F8Dp0",
"0x0.000001452ACBD4315p0", "0x0.0000014B931DEC3CAFp0",
1849 "0x0.000001520B719F8782p0", "0x0.0000015893C634937p0",
"0x0.0000015F2C1DEAF07Cp0", "0x0.00000165D4768AADA5p0",
1850 "0x0.00000168C0D68133p0", "0x0.00000173552CD26689p0",

```

```

"0x0.0000017A2D8A826D16p0", "0x0.0000018115E9EA8645p0",
1851 "0x0.000001880E480EB785p0", "0x0.0000018F16ADF30147p0",
"0x0.000001962F129B63FAp0", "0x0.0000019D57790BE010p0",
1852 "0x0.000001A48F14875Bp0", "0x0.000001ABD84B552630p0",
"0x0.000001B330B735F122p0", "0x0.000001BA9924EED748p0",
1853 "0x0.000001C2119483D917p0", "0x0.000001C99A05F8F707p0",
"0x0.000001D13279523191p0", "0x0.000001D8DAE93892Ep0",
1854 "0x0.000001E09365C0F5E9p0", "0x0.000001E85BDEDE918Dp0",
"0x0.000001F03459F04347p0", "0x0.000001F81CD6FA1405p0",
1855 "0x0.000002001556000444p0", "0x0.000002081DD7061485p0",
"0x0.00000210365A104547p0", "0x0.000002185EDF22970Dp0",
1856 "0x0.000002209766410A5Ap0", "0x0.00000228DFEF6F9FAFp0",
"0x0.00000231387AB25793p0", "0x0.00000239A1080D328Ap0",
1857 "0x0.00000242199784311Cp0", "0x0.0000024AA2291B53CFp0",
"0x0.000002533ABCD69B2Dp0", "0x0.0000025BE352BA07BEp0",
1858 "0x0.000002649BEAC99A0Dp0", "0x0.0000026D64850952A7p0",
"0x0.000002763D217D3216p0", "0x0.0000027F25C02938EAp0",
1859 "0x0.000002881E1611167B0p0", "0x0.00000291270439BEF8p0",
"0x0.0000029A3FA9A63F53p0", "0x0.000002A368515AE951p0",
1860 "0x0.000002ACA0FB58BB86p0", "0x0.000002B5E9A7AC8B86p0",
"0x0.000002BF425651E6E4p0", "0x0.000002C8AB074F3D36p0",
1861 "0x0.000002D2238AA8C013p0", "0x0.000002DBAC70627012p0",
"0x0.000002E54528804DCDp0", "0x0.000002EEDE30659D0p0",
1862 "0x0.000002F8A69FF894D9p0", "0x0.000003026F5F5AFF61p0",
"0x0.0000030C4821319A10p0", "0x0.0000031630E5806584p0",
1863 "0x0.0000032029AC48625Bp0", "0x0.0000032A3275969134p0",
"0x0.000003344B4165F2B1p0", "0x0.0000033E740FBD8772p0",
1864 "0x0.00000348AC4A1501Bp0", "0x0.00000352F5B4154D4Fp0",
"0x0.0000035D4E8A1D7F82p0", "0x0.00000367B762BDE7Ap0",
1865 "0x0.00000372303FA869Ep0", "0x0.0000037CB91BD75C75p0",
"0x0.0000038751FC586A18p0", "0x0.00000391FADF81B02Fp0",
1866 "0x0.0000039CB3C5572F67p0", "0x0.000003A77CADDCE86Ap0",
"0x0.000003B2559916DBE4p0", "0x0.000003BD3E87090A84p0",
1867 "0x0.000003C83777B774F8p0", "0x0.000003D3406B8261BF0p0",
"0x0.000003DE596159001Cp0", "0x0.000003E9825A54222Dp0",
1868 "0x0.000003F48B561882D7p0", "0x0.0000040004548322CCp0",
"0x0.0000040B50561F02C2p0", "0x0.00000416C65A63236Ep0",
1869 "0x0.000004223F61838586p0", "0x0.0000042DC86B8A29C2p0",
"0x0.0000043961786910DBp0", "0x0.000004450A836388Ap0",
1870 "0x0.00000450C39AEFAA8Ap0", "0x0.0000045C8CB0995E96p0",
"0x0.0000046865C937586Bp0", "0x0.000004744EE4CD98C6p0",
1871 "0x0.0000048048083602067p0", "0x0.0000048C5124F2F0CCp0",
"0x0.000004986A4A98A0876p0", "0x0.000004A49371296A67p0",
1872 "0x0.000004900C9B0516A1p0", "0x0.000004BD15C9910DE8p0",
"0x0.000004C96FEA15101p0", "0x0.000004D5082E49E0B0p0",
1873 "0x0.000004E251654EBDBDp0", "0x0.000004EEDA9F73E8F0p0",
"0x0.000004FB73DCBD6310p0", "0x0.000005081D1D2F2CE7p0",
1874 "0x0.00000514D6060CD740p0", "0x0.000005219FA79B8E7p0",
"0x0.0000052E7BF19E70A8p0", "0x0.0000053B623ED98150p0",
1875 "0x0.0000054858BF50E5AFp0", "0x0.0000055564E3089E93p0",
"0x0.000005627E3A04ACCp0", "0x0.0000056FA794491131p0",
1876 "0x0.0000057CE0F1D9CC8Ep0", "0x0.0000058A2A52BADFBAp0",
"0x0.0000059783B6F04B88p0", "0x0.000005A4ED1E7E10CFp0",
1877 "0x0.000005B26689683064p0", "0x0.000005BFEEF782AB20p0",
"0x0.000005CD89696181DBp0", "0x0.000005DB32DE78B56Fp0",
1878 "0x0.000005E8EC56FC46B6p0", "0x0.000005F685D2F0368Cp0",
"0x0.000006048F525885CDp0", "0x0.0000061278D5393558p0",
1879 "0x0.00000620725B96460Ap0", "0x0.0000062E7BE57388C3p0",
"0x0.0000063C9572D58E64p0", "0x0.0000064ABF03BFC7CEp0",
1880 "0x0.000006587898366E55p0", "0x0.000006674230D698Bp0",
"0x0.000006759BCBD08D3Ap0", "0x0.00000684056B0CA519p0",
1881 "0x0.000006927F0DDCDEEp0", "0x0.000006A108B44D81AAp0",
"0x0.000006AFA25E628EB8p0", "0x0.000006BE4C0C20067Fp0",
1882 "0x0.000006CD058D89EA4Bp0", "0x0.000006DBC72A43AE8p0",
"0x0.000006EAA92B7F9420p0", "0x0.000006F992E7FA2646p0",
1883 "0x0.000007088CA83DC2DE4p0", "0x0.00000717966CA1D00Ap0",
"0x0.00000726B0340A4EAp0", "0x0.00000735D9F9B3FB8p0",
1884 "0x0.0000074513CF8BA1Ep0", "0x0.000007545DA2267CD9p0",
"0x0.00000763B77928CA09p0", "0x0.000007732154038F14p0",
1885 "0x0.000007829B32BAC8A2p0", "0x0.000007922515527E1Ap0",
"0x0.000007A1BEFBCEAA04p0", "0x0.000007B168E63351AAp0",
1886 "0x0.000007C22D4847397p0", "0x0.000007D0ECC6C61195p0",
"0x0.000007E0C6BFC2CAAp0", "0x0.000007F0B0B72AC5BEp0",
1887 };
1888 ap_ufixed<51,-21> f_x_msb_3 = f_x_msb_3_t-
able[x_msb_ind_3];
1889 ap_ufixed<62,-10> exp_x_msb_3_m1 = 0;
1890 exp_x_msb_3_m1(61,54) = x_msb_3(7,0);
1891 exp_x_msb_3_m1(50,0) = f_x_msb_3(50,0);
1892
1893 ap_ufixed<50,-10> exp_x_msb_3_m1_s =
exp_x_msb_3_m1;
1894 //50*51
1895 ap_ufixed<101,-27> f_x_msb_3_4_5_lsb =
exp_x_msb_3_m1_s * exp_x_msb_4_5_lsb_m1;
1896 ap_ufixed<45,-27> f_x_msb_3_4_5_lsb_s =
f_x_msb_3_4_5_lsb;
1897 ap_ufixed<63,-9> exp_x_msb_3_4_5_lsb_m1 =
f_x_msb_3_4_5_lsb_s + exp_x_msb_3_m1 + exp_x_msb_4_5_lsb_m1;
1898
1899 //maybe need to extend to 65
1900 const static ap_ufixed<64,-4> f_x_msb_2_t-
able[256] = {
1901 "0x0.0p0", "0x0.00000800AAB555DDEp0",
"0x0.00002005560011127p0", "0x0.00004812036081A9Cp0",
1902 "0x0.0000802AB55777D28p0", "0x0.0000C8536F668A062p0",
"0x0.0001209036103740Dp0", "0x0.000188E50ED8634A0p0",
1903 "0x0.0002015690445B0C3p0", "0x0.000289E711DB32FD7p0",
"0x0.0003229C4C260197Ep0", "0x0.0003CB798B801FE2p0",
1904 "0x0.0004848362076A08Dp0", "0x0.000540B53BC80058p0",

```

```

"0x0.0006272B9A630659Dp0", "0x0.000710D24391E6D7Ap0",
1905 "0x0.00080AB55DE3917ABp0", "0x0.000914D8F8F63D524p0",
"0x0.000A2F41256C297AFp0", "0x0.000B59F1F4EBDE291p0",
1906 "0x0.000C4EF7A206DC2Dp0", "0x0.000DE0F4C89B86B0B1p0",
"0x0.000F3BE0F56CA15C4p0", "0x0.0010A7DD15F367F40p0",
1907 "0x0.00122436410DD14E5p0", "0x0.0013B0F08E817591Ep0",
"0x0.00154E101719F0C5p0", "0x0.0016F98F4A96BBEFPp0",
1908 "0x0.0018B98F42084EFBDp0", "0x0.001A87F71B1613137p0",
"0x0.001C66D49C893B127p0", "0x0.001E562BE4DFA4904p0",
1909 "0x0.00205601127EC98E0p0", "0x0.00226658459402659p0",
"0x0.002487359F24C7C99p0", "0x0.0026B89D413EF4D5Ep0",
1910 "0x0.0028FA934EF909304p0", "0x0.002B4D1BEC726B39Cp0",
"0x0.002D803B3ED3AA50Ep0", "0x0.00302F5564EC123Fp0",
1911 "0x0.0032A84E9C1F58145p0", "0x0.00353D4AF68B07AA1p0",
"0x0.0037E2EEA4E19B185p0", "0x0.003A993DD17D5D2D5p0",
1912 "0x0.003D603CA7C32730FPp0", "0x0.004037EF54230B293p0",
"0x0.0043205A04182F12Fp0", "0x0.00461980E6294381p0",
1913 "0x0.0049236829E8BC292p0", "0x0.004C3E13F512D8p0",
"0x0.004F69889F90A966p0", "0x0.0052A5CA2AABF28D0p0",
1914 "0x0.0055F2DCE5D1E966Ep0", "0x0.005950C5003C20723p0",
"0x0.005CBF86AFC91EF2Bp0", "0x0.00603F262B65057Cp0",
1915 "0x0.0063CFA7AB09D1732p0", "0x0.006710F67BFA0687p0",
"0x0.006B23619B9CF3CDEp0", "0x0.006E6A281C6F4857p0",
1916 "0x0.0072BAD65671B6977p0", "0x0.0076A00156E07CF57p0",
"0x0.007A9627C165FD4EDp0", "0x0.007E9D4D564A3F5E0p0",
1917 "0x0.0082B577D34ED7D58p0", "0x0.0086DEA9FCA73E799p0",
"0x0.008B18E8940100253p0", "0x0.008F6437D0000BFDp0",
1918 "0x0.0093C09C1C595C43Ep0", "0x0.0098E21997D33A9FCp0",
"0x0.009CACA49645847CCp0", "0x0.00A13C715F99EF773p0",
1919 "0x0.00A5D0543CC4DDAFp0", "0x0.00AA8F6177EAD3336p0",
"0x0.00AF529D5C1658EBCp0", "0x0.00B4270C3582A301Dp0",
1920 "0x0.00B90CB25176A4C8Bp0", "0x0.00B0939FEC4C5BD7p0",
"0x0.00C30BB58B73266CAp0", "0x0.00C82518496BE5696p0",
1921 "0x0.00CD4FC989CD64555p0", "0x0.00D28BC49F428CFA3p0",
"0x0.00D7D910DD8B16743p0", "0x0.00DD37B2997BCA6E6p0",
1922 "0x0.00E2A7AE28FECA6FBp0", "0x0.00E82907E313D5399p0",
"0x0.00EBBC41FD08C383p0", "0x0.00F35FE73860B9038p0",
1923 "0x0.00F91575870692F24p0", "0x0.00F08C273671B04BDFp0",
"0x0.0104B4E5350DF2386p0", "0x0.010A9ECF4E667E12Dp0",
1924 "0x0.01109A3611C34FB64p0", "0x0.0116A71DDAD92D8p0",
"0x0.011CC58B167B9D206p0", "0x0.0122F5821A8C7E0Dp0",
1925 "0x0.012937074E0CD6893p0", "0x0.012F8A1F15151B2C6p0",
"0x0.0135ECCDD4D6C5172p0", "0x0.013C6517F39CC6233p0",
1926 "0x0.0142ED01D8CB6C14p0", "0x0.0149868FEC2E6935Dp0",
"0x0.015031C6997996937p0", "0x0.0156EEAA4944BE23p0",
1927 "0x0.015DBD3F68122303Ep0", "0x0.016498DA62C81ACBp0",
"0x0.01688F8FA7745BCDBp0", "0x0.0172935A52E403D8p0",
1928 "0x0.0179A8DACC350DE1Dp0", "0x0.0180D0298D13D179p0",
"0x0.018809445CA7BFE78p0", "0x0.018F542FAC1A492D0p0",
1929 "0x0.0196B0EFF0E793D15p0", "0x0.019E1F89A0DBAA128p0",
"0x0.01A5A00132E02CE42p0", "0x0.01AD325B1EFC9B58Cp0",
1930 "0x0.01B4D69BDE569A322p0", "0x0.01BC8C27EB323853p0",
"0x0.01C454E3C0F2458C8p0", "0x0.01CC2EF3D187C2F6p0",
1931 "0x0.01D41AFBCA45E6ED3p0", "0x0.01DC1902B8319A6Dp0",
"0x0.01E4290ABBD87C5BCp0", "0x0.01EC4B18E01E9326Fp0",
1932 "0x0.01F47F31C92E464FCp0", "0x0.01FCE559FA492A6B6p0",
"0x0.02051D95F7D1C8917p0", "0x0.020D077EA474E6A1Cp0",
1933 "0x0.0216045B6F5CCF9CEp0", "0x0.021E92EDF7CB9C0E4p0",
"0x0.022733A669817A8Ap0", "0x0.022F6894E89F834Cp0",
1934 "0x0.0238AB9B321349722p0", "0x0.024182E0A06E9289Bp0",
"0x0.024A6C5E271030733p0", "0x0.02536818549001AC1p0",
1935 "0x0.025C7613B8A9AF215p0", "0x0.02695654E43CF52B1p0",
"0x0.026EC8E0694DEC9A9p0", "0x0.02780DBAD63530A6p0",
1936 "0x0.028164E8CDB08211p0", "0x0.028ACE6ED63C5EB5Fp0",
"0x0.02944A518CD54E484p0", "0x0.029D089587A4D858Dp0",
1937 "0x0.02A7793F601642B5Dp0", "0x0.02B12C3803431090p0",
"0x0.02BAF1D7132FEE788p0", "0x0.02C4C9CE2561B759Fp0",
1938 "0x0.02CEB43D844902F7Ap0", "0x0.02CEB8129CE8CCD68Fp0",
"0x0.02E2C097A3FBE17C9p0", "0x0.02ECE28BA58D22B56p0",
1939 "0x0.02F7170A755DF759Ep0", "0x0.03015E1886BBF2966p0",
"0x0.030BB7B80E125E13p0", "0x0.031623F620FD4751Ep0",
1940 "0x0.0320A2CE964063DACp0", "0x0.032B344915C941D54p0",
"0x0.0335D86A48AF87F0Cp0", "0x0.03408F3D93566140p0",
1941 "0x0.034858B372C795013p0", "0x0.035634E4C1FDEC0D0p0",
"0x0.036123CF749B8667Cp0", "0x0.036C2578398F157A0p0",
1942 "0x0.037739C3C0F32BD30p0", "0x0.03826116B0E882ACp0",
"0x0.038D9B15DD5460763p0", "0x0.0398E7E5D864AD0E9p0",
1943 "0x0.03A4478B620C73EB5p0", "0x0.03AFBA0B304613EEDp0",
"0x0.03BB3F69FA3990464p0", "0x0.03C6D7AC783CDBBFPp0",
1944 "0x0.03D282D763D424CDp0", "0x0.03DE40EF77B220509p0",
"0x0.03EA11F96FB855CA4p0", "0x0.03F5F5FA08F76B0BBp0",
1945 "0x0.0401ECF601AF700BDp0", "0x0.040DF6219502AA55p0",
"0x0.041A13F3107962A88p0", "0x0.042643FDABF82DDFp0",
1946 "0x0.04328716A5D63C2C4p0", "0x0.043EDD42CB3C238BDp0",
"0x0.044B4686DE8FAD325p0", "0x0.0457C2E7A66520322p0",
1947 "0x0.04645269EA828F932p0", "0x0.0470F51273E025F89p0",
"0x0.047DAAE60CA8725B3p0", "0x0.048A73E98038BAABFp0",
1948 "0x0.049750219B212A8A0p0", "0x0.04A43F932B25C0B7p0",
"0x0.04B14242FF3C689A0p0", "0x0.04BE5835E79153E1Dp0",
1949 "0x0.04CB8170B58352D4Ep0", "0x0.04D8BDF83BA618C04p0",
"0x0.04E60DD14DC22475Ep0", "0x0.04F37100C0D50D88Ep0",
1950 "0x0.0500E78B6B11D19D6p0", "0x0.050E717623E121CBAp0",
"0x0.051C0EC5C3E18016Cp0", "0x0.0529BF7F24E87CF6Ap0",
1951 "0x0.053783A7220124F51p0", "0x0.0545558A42976E2E5Ap0",
"0x0.0553465662A95706Bp0", "0x0.056144E7263E21F4p0",
1952 "0x0.056F56FA7686E623Ep0", "0x0.057D7C9480339A083p0",
"0x0.058B85BA61C3A75A3p0", "0x0.059A0270FEC97047Ap0",
1953 "0x0.05A862BD3C1065F74p0", "0x0.05B6D6A3FF9D52C58p0",
"0x0.05C5E2A30AE974Ep0", "0x0.05D3F954B7CD3A1Bp0",
1954 "0x0.05E2A8287E7A8049Ep0", "0x0.05F16AA6FD4F267Bp0",

```

```

"0x0.060040DF77F44FA13p0", "0x0.060F2ACC843BEF0A0p0",
1955 "0x0.061E2876834AA7FA4p0", "0x0.062D39E264FB20F7Ep0",
"0x0.063C5F151A641E84Cp0", "0x0.064B981395D8D31FEp0",
1956 "0x0.065AE4E2CA92C8AAP0", "0x0.066A4587AE6224E1Cp0",
"0x0.0679BA07364E10FA3p0", "0x0.0689426659F4EFF1Bp0",
1957 "0x0.0698DEAA11DCBA37p0", "0x0.06A88ED757C9B3304p0",
"0x0.06B852F326BE849B0p0", "0x0.06C82B027AFD8278Ap0",
1958 "0x0.06D8170A520718B46p0", "0x0.06E8170FAA9BF867Ep0",
"0x0.06F82B1784BC86C72p0", "0x0.07085326E1A93F207p0",
1959 "0x0.07188F42C3E320F0Bp0", "0x0.0728DF702F2BF10B2p0",
"0x0.073943B428868CD5Ap0", "0x0.0749BC13B6373A98Fp0",
1960 "0x0.075A4893DFC3F9E4Cp0", "0x0.076AE939ADF4D4083p0",
"0x0.077B9E0A2AD42C9E8p0", "0x0.078C670A61AF122F4p0",
1961 "0x0.079D443F5F158EE3Ap0", "0x0.07AE35AE30DAF94F1p0",
"0x0.07BF3B5BE616454C8p0", "0x0.07D0554D8F2254F0Cp0",
1962 "0x0.07E183883D9E498E2p0", "0x0.07FC611046DD4D06p0",
"0x0.08041CECF7B989EA2p0", "0x0.081588212CE2ED73p0",
1963 "0x0.082707B2BAC20DB40p0", "0x0.08389BA6B92B46284p0",
"0x0.084A4402416A1EE6Fp0", "0x0.085C00CA6E045741Fp0",
1964 "0x0.086DD2045AC678D2Bp0", "0x0.087FB7B524C42936Cp0",
"0x0.0891B1E1EA587BE12p0", "0x0.08A3C08FCB2643FFEp0",
1965 };
1966 ap_ufixed<64,-4> f_x_msb_2 = f_x_msb_2_t-
able[x_msb_ind_2];
1967 ap_ufixed< 6,-4> f_x_msb_2_h;
1968 f_x_msb_2_h(5,0) = f_x_msb_2(63,58);
1969 ap_ufixed< 9,-1> exp_x_msb_2_m_1_h = f_x_msb_2_h
+ x_msb_2;
1970 ap_ufixed<67,-1> exp_x_msb_2_m_1;
1971 exp_x_msb_2_m_1(66,58) = exp_x_msb_2_m_1_h(8,0);
1972 exp_x_msb_2_m_1(57, 0) = f_x_msb_2(57,0);
1973 //67*63
1974 ap_ufixed<130,-10> f_x_msb_2_3_4_5_lsb =
exp_x_msb_2_m_1 * exp_x_msb_3_4_5_lsb_m_1;
1975 ap_ufixed<62,-10> f_x_msb_2_3_4_5_lsb_s =
f_x_msb_2_3_4_5_lsb;
1976 ap_ufixed<72,0> exp_x_msb_2_3_4_5_lsb_m_1 =
f_x_msb_2_3_4_5_lsb_s + exp_x_msb_2_m_1 + exp_x_msb_3_4_5_lsb_m_1;
1977
1978 const static ap_ufixed<68,32> exp_x_msb_1_t-
able[256] = {
1979 "0x1.0000000000p0", "0x1.48B5E3C3Ep0", "0x1.A61298E1Ep0",
"0x2.1DF3B68CFp0",
1980 "0x2.B7E151628p0", "0x3.7D871DB61p0", "0x4.7B4F993Fp0",
"0x5.C12DA416Ep0",
1981 "0x7.63992E353p0", "0x9.7CDC417A3p0", "0xC.2EB7EC98Fp0",
"0xF.A48385EDFp0",
1982 "0x14.15E5BF6FBp0", "0x19.CA53B7811p0", "0x21.1D8E4272Dp0",
"0x2A.8565A144Cp0",
1983 "0x36.99205C4E7p0", "0x46.1AFC4DB59p0", "0x5A.0462B7877p0",
"0x73.9593ABB7Dp0",
1984 "0x94.69C4CB819p0", "0xBE.90F6F83E9p0", "0xF4.B122790DDp0",
"0x13A.30CF1CC8Bp0",
1985 "0x193.6DC5690Cp0", "0x206.03487A3B1p0",
"0x299.2442102D9p0", "0x356.0F0B0F980p0",
1986 "0x448.A216ABB76p0", "0x580.1AD754FA3p0",
"0x710.0ADBAC7DAP0", "0x911.9289C323p0",
1987 "0xBA4.F53EA3863p0", "0xEF3.A035D5798p0",
"0x1332.C4D2B7C4Ap0", "0x18A6.B027DA0A7p0",
1988 "0x1FA7.157C470F8p0", "0x28A4.90D2CCECF1p0",
"0x342F.BA11823B8p0", "0x4302.3A933EE5Cp0",
1989 "0x560A.773E54157p0", "0x6E7A.8ABB4A83Cp0", "0x8DD8.
80AF4269Dp0", "0xB626.0748BAEC7p0",
1990 "0xE9E2.2447727BFp0", "0x12C4F.EB75B2AB0p0",
"0x1819B.C560F6113p0", "0x1EF21.8F1B001A4p0",
1991 "0x27BC2.CA9A6F934p0", "0x3305.49F35D91Fp0",
"0x41831.49596E996p0", "0x541E7.E56949D58p0",
1992 "0x6C02D.645AB2554p0", "0x8AB06.0A3EE9FB1p0",
"0xB2148.5EA56C58p0", "0xE4A8D.2881EDBE8p0",
1993 "0x1259AC.48BF05D6Ep0", "0x178FEE.7792E44B3p0",
"0x1E4127.437732B71p0", "0x26D8F9.4A204BEA5p0",
1994 "0x31E199.5F5A550DDp0", "0x400C7D.64D3386CBp0",
"0x523D82.79EDA9F9Cp0", "0x69993D.D4F2D9524p0",
1995 "0x87975E.854001024p0", "0xAE1A47.C38A42CD0p0", "0xDF8D5F.6D-
CFE5CEAp0", "0x11F0BF8.1E99FDEB6p0",
1996 "0x1709348.C0EA4F8CEp0", "0x1D94295.464477ACCP0", "0x25-
FAD90.65C78E342p0", "0x30C4623.616ED2BE2p0",
1997 "0x3E9E441.232817A61p0", "0x506744D.3B342FC94p0",
"0x673D70B.C00F1F76Dp0", "0x849012B.C127FC962p0",
1998 "0xAA36C7C.F69370B94p0", "0xDA8F202.DD9ACE661p0",
"0x118A2AAE.0AA05236Bp0", "0x16857CFA.1AA8A5555p0",
1999 "0x1CEB088B.68E804021p0", "0x2521AEDA.32CD52BE5p0", "0x2-
FAD89E1.79BAB8EADp0", "0x3D3838C6.BE0F13F06p0",
2000 "0x4E9B87F6.7BB3F5597p0", "0x64EF201B.01109A4A5p0",
"0x819A1801.9394C0D96p0", "0xA6698403.CF5D9E663p0",
2001 "0xD5AD6DCE.21AFFC0DDp0", "0x1125E066B.0B19418CCp0",
"0x160486BCF.05F65FAAp0", "0x1C45AE0DA.D0C934CDCp0",
2002 "0x24D05E676.3F8B14AF0p0", "0x2E9CEBF49.9C21839C7p0", "0x3B-
DA2CBD0.9F5AC2146p0", "0x4CDA0FD41.989F004E7p0",
2003 "0x62ADFED51.D7EDCCDE6p0", "0xE7E50B520.76F91D6E8p0", "0x-
A2B1FD3D9.A57B6C33Cp0", "0xD0E7A50F4.EF0D7447Fp0",
2004 "0x10C3D39209.62C88AAFBp0", "0x1586D0722B.3EEB27301p0",
"0x1BA4068AAC.2B6E3EE34p0", "0x237DCBF1E5.709A9F9C1p0",
2005 "0x2D9264D2D4.D6894153Bp0", "0x3A83F23B18.FE5B0524Ap0",
"0x4B22A1B5C3.27A2F085Ap0", "0x6079C1866C.71A93951Dp0",
2006 "0x7BE08BDF09.CC72249ABp0", "0x9F0F87309C.B446D3919p0", "0x-
CC3D265776.C3D25779Ep0", "0x1063F7612B0.C0831CA6Cp0",
2007 "0x150BBA37963.79A4FB6ADp0", "0x1805FA9B6E2.737D935B8p0",
"0x22B2DC5C41D.AB89559Ep0", "0x2C8DD1AB28B.404414BE7p0",
2008 "0x39355C8C638.0AC5631B1p0", "0x49750434675.301B8D0E7p0",
"0x5E521E7A16E.51F9CD3C2p0", "0x791C3B06F8B.EA2DEC48Cp0",
2009 "0x9B823857614.764F43E20p0", "0xC7AD559D438.E7C270C30p0",
"0x10063F4E687B.A1A43A35Bp0", "0x149363C905BA.3641D0164p0",
2010 "0x1A68765D8CDF.6CDBF1C63p0", "0x21EC75240E66.E2891CE63p0",
"0x2B8F1073A6AF.BA219DB61p0", "0x37EE3FFC0063.E360F931Fp0",
2011 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2012 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2013 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2014 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2015 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2016 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2017 "0x0.000000000p0", "0x0.000000000p0", "0x0.000000000p0",
"0x0.000000000p0",
2018 "0x0.000000000p0", "0x0.000000001p0", "0x0.000000001p0",
"0x0.000000002p0",
2019 "0x0.000000002p0", "0x0.000000003p0", "0x0.000000004p0",
"0x0.000000005p0",
2020 "0x0.000000007p0", "0x0.000000009p0", "0x0.00000000Bp0",
"0x0.00000000Ep0",
2021 "0x0.000000013p0", "0x0.000000018p0", "0x0.00000001Fp0",
"0x0.000000028p0",
2022 "0x0.000000034p0", "0x0.000000042p0", "0x0.000000055p0",
"0x0.00000006Ep0",
2023 "0x0.00000008Dp0", "0x0.0000000B5p0", "0x0.0000000E9p0",
"0x0.00000012Bp0",
2024 "0x0.000000181p0", "0x0.0000001EEp0", "0x0.00000027Ap0",
"0x0.00000032Fp0",
2025 "0x0.000000416p0", "0x0.00000053Fp0", "0x0.0000006BDp0",
"0x0.0000008A7p0",
2026 "0x0.000000B1Cp0", "0x0.000000E44p0", "0x0.000001252p0",
"0x0.000001786p0",
2027 "0x0.000001E35p0", "0x0.0000026C9p0", "0x0.0000031CEp0",
"0x0.000003FF3p0",
2028 "0x0.00000521Dp0", "0x0.000006970p0", "0x0.000008762p0",
"0x0.00000ADD6p0",
2029 "0x0.00000DF36p0", "0x0.000011E9Cp0", "0x0.000017003p0",
"0x0.00001D88Ap0",
2030 "0x0.000025EC0p0", "0x0.000030B15p0", "0x0.00003E85Dp0",
"0x0.00005047Ep0",
2031 "0x0.000067153p0", "0x0.0000845C6p0", "0x0.0000A9F46p0",
"0x0.0000DA39Ep0",
2032 "0x0.000118354p0", "0x0.000167CB5p0", "0x0.0001CDFC2p0",
"0x0.000251334p0",
2033 "0x0.0002F9AF3p0", "0x0.0003D05Ap0", "0x0.00047CE2p0",
"0x0.00064C7C6p0",
2034 "0x0.000816791p0", "0x0.000A628A3p0", "0x0.000D55A1Fp0",
"0x0.0011F30Fp0",
2035 "0x0.0015FC210p0", "0x0.001C3AA91p0", "0x0.00243F374p0",
"0x0.002E8ABFCp0",
2036 "0x0.0038C2D73p0", "0x0.004CBC19Ap0", "0x0.006287862p0",
"0x0.007E839C2p0",
2037 "0x0.00A2728F8p0", "0x0.00D096336p0", "0x0.010BD4A5Ap0",
"0x0.0157E68Fp0",
2038 "0x0.01B993FE0p0", "0x0.0236FF5BFp0", "0x0.02D80A08Dp0",
"0x0.03A6D2222p0",
2039 "0x0.04B055E0p0", "0x0.0605424D5p0", "0x0.07BB04063p0",
"0x0.09ED1B429p0",
2040 "0x0.0CBED8666p0", "0x0.105D93892p0", "0x0.150385C09p0",
"0x0.1AFB718E8p0",
2041 "0x0.22A3055477p0", "0x0.2C7C72FC0p0", "0x0.391F0EE49p0",
"0x0.495860DCAp0",
2042 "0x0.5E2D58D8Bp0", "0x0.78ED03AFBp0", "0x0.9B4597E37p0",
"0x0.C75F7C756p0",
2043 };
2044 ap_ufixed<68,32> exp_x_msb_1 = exp_x_msb_1_t-
able[x_msb_ind_1];
2045 ap_ufixed<140,32> y_lo = exp_x_msb_1 *
exp_x_msb_2_3_4_5_lsb_m_1;
2046 ap_ufixed<68,32> y_lo_s = y_lo;
2047 ap_ufixed<68,32> y_l = y_lo_s + exp_x_msb_1;
2048 y = y_l;
2049 }
2050 y += delta;
2051 //if (I<33) { // overflow
2052 overf = 0;
2053 for (int j = 64; j >= I+31; j--) {
2054 #pragma HLS unroll
2055 if (y[j])
2056 overf = 1;
2057 }
2058 if (overf) {
2059 for (int j = 64; j >= I+31; j--) {
2060 #pragma HLS unroll
2061 y[j] = 0;
2062 }
2063 for (int j = I+30; j >= 0; j--) {
2064 #pragma HLS unroll
2065 y[j] = 1;
2066 }
2067 }
2068 //}
2069 r = y;
2070
2071 return r;

```

```

2072 }
2073
2074 template<int W_, int I_>
2075 ap_fixed<W_, I_> exp(ap_ufixed<W_, I_> x) {
2076     ap_fixed<W+1, I+1> xf = x;
2077     return exp(xf);
2078 }
2079
2080 template<int I_>
2081 ap_int<I_> exp(ap_int<I_> x) {
2082     ap_fixed<I_, I_> xf = x;
2083     return exp(xf);
2084 }
2085
2086 template<int I_>
2087 ap_uint<I_> exp(ap_uint<I_> x) {
2088     ap_fixed<I+1, I+1> xf = x;
2089     return exp(xf);
2090 }
2091
2092 template<int W, int I>
2093 ap_fixed<W, I> expm1(ap_fixed<W, I> x) {
2094     ap_fixed<W, I> result, tmp;
2095     if(I < 1) {
2096         result = 0;
2097     } else {
2098         ap_fixed<W, I> tmp = exp(x);
2099         ap_ufixed<1, I> one = 1;
2100         result = tmp - one;
2101     }
2102     return result;
2103 }
2104
2105 template<int W, int I>
2106 ap_ufixed<W, I> expm1(ap_ufixed<W, I> x) {
2107     ap_fixed<W+1, I+1> xf = x;
2108     return expm1(xf);
2109 }
2110
2111 template<int I>
2112 ap_int<I> expm1(ap_int<I> x) {
2113     ap_fixed<I, I> xf = x;
2114     return expm1(xf);
2115 }
2116
2117 template<int I>
2118 ap_uint<I> expm1(ap_uint<I> x) {
2119     ap_fixed<I+1, I+1> xf = x;
2120     return expm1(xf);
2121 }
2122
2123 template<int W, int I>
2124 ap_fixed<W, I> exp2(ap_fixed<W, I> x) {
2125     int F = W - I;
2126     if(I > 33 || F > 32) {
2127         return 0;
2128     }
2129
2130     int I_s;
2131     int M = ((I - 1) > F) ? (I - 1) : F;
2132
2133     if(M == 1) {
2134         I_s = 1;
2135     } else if(M == 2) {
2136         I_s = 2;
2137     } else if(M == 4) {
2138         I_s = 3;
2139     } else if(M == 8) {
2140         I_s = 4;
2141     } else if(M == 16) {
2142         I_s = 5;
2143     } else {
2144         I_s = 6;
2145     }
2146
2147     ap_fixed<W, I> r;
2148
2149     if(I_s == 1) {
2150
2151         ap_fixed<3, 2> x_l = x;
2152         ap_ufixed<2, 1> y = 0;
2153
2154         if(!x_l[2]) {
2155             y[1] = 1;
2156         }
2157         if(x_l[1]) {
2158             y[0] = 1;
2159         }
2160
2161         if(I < 2) {
2162             if(y[1]) {
2163                 y[1] = 0;
2164                 y[0] = 1;
2165             }
2166         }
2167         r = y;
2168
2169     } else if(I_s == 2) {

```

```

2172     ap_fixed<5, 3> x_l = x;
2173     ap_ufixed<4, 2> y = 0;
2174     ap_fixed<2, 2> x_l_int;
2175     x_l_int(1, 0) = x_l(3, 2);
2176     ap_ufixed<2, 0> x_l_fract;
2177     x_l_fract(1, 0) = x_l(1, 0);
2178
2179     if((x_l[4]&&(!x_l[3])) || ((!x_l[4])&&x_l[3])) {
2180         if(!x_l[4]) {
2181             y = "0x3.cp0";
2182         }
2183     } else {
2184         if((!x_l[3])&&(x_l[2])) y[3] = 1;
2185         if((!x_l[3])&&(x_l[2])) || ((!
x_l[3])&&x_l[2]&&x_l[1]&&x_l[0])) y[2] = 1;
2186         if((x_l[3]&&x_l[2]) || ((!
x_l[3])&&(x_l[2]&&(x_l[1]^x_l[0])) || ((!
x_l[2])&&x_l[1]&&x_l[0])))) y[1] = 1;
2187         if((x_l[3]&&(!x_l[2])) ||
(x_l[1]&&((x_l[0]&&x_l[2]) || ((!x_l[0])&&(!x_l[3]))))) y[0] = 1;
2188     }
2189
2190     if(I < 3) {
2191         bool overf = 0;
2192         for(int j=3; j>=I+1; j--) {
2193             #pragma HLS unroll
2194             if(y[j]) {
2195                 overf = 1;
2196             }
2197         }
2198         if(overf) {
2199             for(int j=3; j>=I+1; j--) {
2200                 #pragma HLS unroll
2201                 y[j] = 0;
2202             }
2203             for(int j=I; j>=0; j--) {
2204                 #pragma HLS unroll
2205                 y[j] = 1;
2206             }
2207         }
2208     }
2209     r = y;
2210
2211 } else if(I_s == 3) {
2212
2213     ap_fixed<9, 5> x_l = x;
2214     ap_ufixed<8, 4> y = 0;
2215     ap_fixed<3, 3> x_l_int;
2216     x_l_int(2, 0) = x_l(6, 4);
2217     ap_ufixed<4, 0> x_l_fract;
2218     x_l_fract(3, 0) = x_l(3, 0);
2219
2220     if((x_l[8]!=x_l[7]) || (x_l[8]!=x_l[6])) {
2221         if(!x_l[8]) {
2222             y = "0xf.fp0";
2223         }
2224     } else {
2225         ap_uint<4> x_msb_ind;
2226         ap_uint<3> x_lsb_ind;
2227         x_msb_ind[3] = x_l[8];
2228         x_msb_ind(2, 0) = x_l(5, 3);
2229         x_lsb_ind(2, 0) = x_l(2, 0);
2230
2231         const static ap_ufixed<10, 4> exp2_x_msb_table[16]
= {
2232             "0x1.00p0", "0x1.6Cp0", "0x2.00p0",
2233             "0x2.04p0",
2234             "0x4.00p0", "0x5.A8p0", "0x8.00p0", "0xB.
50p0",
2235             "0x0.10p0", "0x0.18p0", "0x0.20p0",
2236             "0x0.2Cp0",
2237             "0x0.40p0", "0x0.5Cp0", "0x0.80p0",
2238             "0x0.B4p0"
2239         };
2240         const static ap_ufixed<10, 0> exp2_x_lsb_m_1_t-
able[8] = {
2241             "0x0.0p0", "0x0.0B4p0", "0x0.174p0",
2242             "0x0.238p0",
2243             "0x0.308p0", "0x0.3E0p0", "0x0.4C0p0",
2244             "0x0.5ACp0"
2245         };
2246         ap_ufixed<11, 0> exp2_x_lsb_m_1 =
exp2_x_lsb_m_1_table[x_lsb_ind];
2247         ap_ufixed<10, 4> exp2_x_msb = exp2_x_msb_t-
able[x_msb_ind];
2248         ap_ufixed<21, 4> y_lo = exp2_x_msb *
exp2_x_lsb_m_1;
2249         ap_ufixed<10, 4> y_lo_s = y_lo;
2250         ap_ufixed<10, 4> y_l = y_lo_s + exp2_x_msb;
2251         y = y_l;
2252     }
2253
2254     if(I < 5) {
2255         bool overf = 0;
2256         for(int j=7; j>=3+I; j--) {
2257             #pragma HLS unroll
2258             if(y[j]) {
2259                 overf = 1;
2260             }
2261         }
2262     }

```



```

2256         if(overf) {
2257             for(int j=7; j>=3+I; j--) {
2258                 #pragma HLS unroll
2259                 y[j] = 0;
2260             }
2261             for(int j=I+2; j>=0; j--) {
2262                 #pragma HLS unroll
2263                 y[j] = 1;
2264             }
2265         }
2266     }
2267     r = y;
2268
2269 } else if(I_s == 4) {
2270
2271     ap_fixed<17,9> x_l=x;
2272     ap_ufixed<16,8> y = 0;
2273     ap_fixed<4,4> x_l_int;
2274     x_l_int(3,0) = x_l(11,8);
2275     ap_ufixed<8,0> x_l_fract;
2276     x_l_fract(7,0) = x_l(7,0);
2277
2278     bool overf_1 = 0;
2279     for(int j=11; j<16; j++) {
2280         #pragma HLS unroll
2281         if(x_l[16]!=x_l[j]) {
2282             overf_1 = 1;
2283         }
2284     }
2285     if(overf_1) {
2286         if(!x_l[16]) {
2287             y = "0xff.ffp0";
2288         }
2289     } else {
2290
2291         ap_uint<5> x_msb_ind_1;
2292         x_msb_ind_1(4,0) = x_l(11,7);
2293         ap_uint<4> x_msb_ind_2;
2294         x_msb_ind_2(3,0) = x_l(6,3);
2295         ap_uint<3> x_lsb_ind;
2296         x_lsb_ind = x_l(2,0);
2297
2298         const static ap_ufixed<14,-5> exp2_x_lsb_m_1_t-
2299
2300         able[8] = {
2301             "0x0.0p0", "0x0.00B1Ap0", "0x0.0163Ep0",
2302             "0x0.02168p0",
2303             "0x0.02C9Ap0", "0x0.037D4p0", "0x0.04316p0",
2304             "0x0.04E60p0"
2305         };
2306         ap_ufixed<14,-5> exp2_x_lsb_m_1 =
2307         exp2_x_lsb_m_1_table[x_lsb_ind];
2308
2309         const static ap_ufixed<19,0> exp2_x_msb_2_m_1_t-
2310
2311         able[16] = {
2312             "0x0.0p0", "0x0.059B0p0", "0x0.0B558p0",
2313             "0x0.11302p0",
2314             "0x0.172B8p0", "0x0.1D488p0", "0x0.2387Ap0",
2315             "0x0.29E9Ep0",
2316             "0x0.306FEp0", "0x0.371A8p0", "0x0.3DEA6p0",
2317             "0x0.44E08p0",
2318             "0x0.4BFD4p0", "0x0.5342Cp0", "0x0.5AB08p0",
2319             "0x0.6247Ep0"
2320         };
2321         ap_ufixed<19,0> exp2_x_msb_2_m_1 =
2322         exp2_x_msb_2_m_1_table[x_msb_ind_2];
2323
2324         ap_ufixed<33,-5> f_x_msb_2_lsb = exp2_x_msb_2_m_1
2325         * exp2_x_lsb_m_1;
2326         ap_ufixed<19,0> f_x_msb_2_lsb_s = f_x_msb_2_lsb;
2327         ap_ufixed<19,0> exp2_x_msb_2_lsb_m_1 =
2328         f_x_msb_2_lsb_s + exp2_x_msb_2_m_1 + exp2_x_lsb_m_1;
2329
2330         const static ap_ufixed<19,8> exp2_x_msb_1_t-
2331
2332         able[32] = {
2333             "0x1.000p0", "0x1.6A0p0", "0x2.000p0",
2334             "0x2.D42p0",
2335             "0x4.000p0", "0x5.A82p0", "0x8.000p0", "0xB.
2336             504p0",
2337             "0x10.000p0", "0x16.A0Ap0", "0x20.000p0",
2338             "0x2D.414p0",
2339             "0x40.000p0", "0x5A.828p0", "0x80.000p0",
2340             "0xB5.050p0",
2341             "0x0.010p0", "0x0.016p0", "0x0.020p0",
2342             "0x0.02Ep0",
2343             "0x0.040p0", "0x0.05Ap0", "0x0.080p0",
2344             "0x0.0B6p0",
2345             "0x0.100p0", "0x0.16Ap0", "0x0.200p0",
2346             "0x0.2D4p0",
2347             "0x0.400p0", "0x0.5A8p0", "0x0.800p0",
2348             "0x0.B50p0"
2349         };
2350         ap_ufixed<19,8> exp2_x_msb_1 = exp2_x_msb_1_t-
2351         able[x_msb_ind_1];
2352         ap_ufixed<38,8> y_lo = exp2_x_msb_1 *
2353         exp2_x_msb_2_lsb_m_1;
2354         ap_ufixed<19,8> y_lo_s = y_lo;
2355         ap_ufixed<19,8> y_l = y_lo_s + exp2_x_msb_1;
2356         y = y_l;
2357     }
2358 }

```

```

2332     if(I < 9) {
2333         bool overf_2 = 0;
2334         for(int j=15; j>=7+I; j--) {
2335             #pragma HLS unroll
2336             if(y[j]) {
2337                 overf_2 = 1;
2338             }
2339         }
2340         if(overf_2) {
2341             for(int j=15; j>=7+I; j--) {
2342                 #pragma HLS unroll
2343                 y[j] = 0;
2344             }
2345             for(int j=6+I; j>=0; j--) {
2346                 #pragma HLS unroll
2347                 y[j] = 1;
2348             }
2349         }
2350     }
2351     r = y;
2352
2353 } else if(I_s == 5) {
2354
2355     ap_fixed<33,17> x_l = x;
2356     ap_ufixed<32,16> y = 0;
2357     ap_fixed<5,5> x_l_int;
2358     x_l_int(4,0) = x_l(20,16);
2359     ap_ufixed<16,0> x_l_fract;
2360     x_l_fract(15,0) = x_l(15,0);
2361
2362     bool overf_1 = 0;
2363     for(int j=20; j<32; j++) {
2364         #pragma HLS unroll
2365         if(x_l[32]!=x_l[j]) {
2366             overf_1 = 1;
2367         }
2368     }
2369     if(overf_1) {
2370         if(!x_l[32]) {
2371             y = "0xffff.ffffp0";
2372         }
2373     } else {
2374         ap_uint<6> x_msb_ind_1;
2375         x_msb_ind_1(5,0) = x_l(20,15);
2376         ap_uint<5> x_msb_ind_2;
2377         x_msb_ind_2(4,0) = x_l(14,10);
2378         ap_uint<5> x_msb_ind_3;
2379         x_msb_ind_3(4,0) = x_l(9,5);
2380         ap_uint<5> x_lsb_ind;
2381         x_lsb_ind = x_l(4,0);
2382
2383         const static ap_ufixed<25,-11> exp2_x_lsb_m_1_t-
2384
2385         able[32] = {
2386             "0x0.0p0", "0x0.000B1725p0",
2387             "0x0.000162E52p0", "0x0.000214587p0",
2388             "0x0.0002C5CC3p0", "0x0.000377408p0",
2389             "0x0.0004DA2A7p0",
2390             "0x0.00058BA02p0", "0x0.00063D165p0",
2391             "0x0.0007A0042p0",
2392             "0x0.0008517BBp0", "0x0.000902F3Dp0",
2393             "0x0.0009B46C6p0", "0x0.000A65E75p0",
2394             "0x0.000B175F0p0", "0x0.000BC8D90p0",
2395             "0x0.000C7A538p0", "0x0.000DD4A00p0", "0x0.000E8C5F5p0",
2396             "0x0.000F40425p0", "0x0.000FF1BF4p0",
2397             "0x0.0010A33CAp0", "0x0.001154BA8p0",
2398             "0x0.00120638Dp0", "0x0.0012B7B7Bp0",
2399             "0x0.001369370p0", "0x0.00141AB6Cp0",
2400             "0x0.0014CC371p0", "0x0.00157DB70p0"
2401         };
2402         ap_ufixed<25,-11> exp2_x_lsb_m_1 =
2403         exp2_x_lsb_m_1_table[x_lsb_ind];
2404
2405         const static ap_ufixed<30,-6> exp2_x_msb_3_m_1_t-
2406
2407         able[32] = {
2408             "0x0.0p0", "0x0.00162F390p0",
2409             "0x0.002C605E3p0", "0x0.0042936FBp0",
2410             "0x0.0058C86DAp0", "0x0.006EFF584p0",
2411             "0x0.0085382FBp0", "0x0.009B72F42p0",
2412             "0x0.00B1AFA5Bp0", "0x0.00C7EE449p0",
2413             "0x0.00DE2ED0Fp0", "0x0.00F4714AFp0",
2414             "0x0.010AB5B2Dp0", "0x0.0120FC08Ap0",
2415             "0x0.0137444CAp0", "0x0.014D8E7EFp0",
2416             "0x0.0163DA9FBp0", "0x0.017A28AF2p0",
2417             "0x0.019078AD7p0", "0x0.01A6CA9ABp0",
2418             "0x0.01BD1E771p0", "0x0.01D37442Dp0",
2419             "0x0.01E9CBFE1p0", "0x0.020025A8Fp0",
2420             "0x0.02168143Bp0", "0x0.022CDECE7p0",
2421             "0x0.02433E495p0", "0x0.02599FB48p0",
2422             "0x0.027003104p0", "0x0.028685CAp0",
2423             "0x0.029CCF99Dp0", "0x0.02B338C81p0",
2424             "0x0.030000000p0", "0x0.030000000p0",
2425             "0x0.030000000p0", "0x0.030000000p0",
2426             "0x0.030000000p0", "0x0.030000000p0",
2427             "0x0.030000000p0", "0x0.030000000p0",
2428             "0x0.030000000p0", "0x0.030000000p0",
2429             "0x0.030000000p0", "0x0.030000000p0",
2430             "0x0.030000000p0", "0x0.030000000p0",
2431             "0x0.030000000p0", "0x0.030000000p0",
2432             "0x0.030000000p0", "0x0.030000000p0",
2433             "0x0.030000000p0", "0x0.030000000p0",
2434             "0x0.030000000p0", "0x0.030000000p0",
2435             "0x0.030000000p0", "0x0.030000000p0",
2436             "0x0.030000000p0", "0x0.030000000p0",
2437             "0x0.030000000p0", "0x0.030000000p0",
2438             "0x0.030000000p0", "0x0.030000000p0",
2439             "0x0.030000000p0", "0x0.030000000p0",
2440             "0x0.030000000p0", "0x0.030000000p0",
2441             "0x0.030000000p0", "0x0.030000000p0",
2442             "0x0.030000000p0", "0x0.030000000p0",
2443             "0x0.030000000p0", "0x0.030000000p0",
2444             "0x0.030000000p0", "0x0.030000000p0",
2445             "0x0.030000000p0", "0x0.030000000p0",
2446             "0x0.030000000p0", "0x0.030000000p0",
2447             "0x0.030000000p0", "0x0.030000000p0",
2448             "0x0.030000000p0", "0x0.030000000p0",
2449             "0x0.030000000p0", "0x0.030000000p0",
2450             "0x0.030000000p0", "0x0.030000000p0",
2451             "0x0.030000000p0", "0x0.030000000p0",
2452             "0x0.030000000p0", "0x0.030000000p0",
2453             "0x0.030000000p0", "0x0.030000000p0",
2454             "0x0.030000000p0", "0x0.030000000p0",
2455             "0x0.030000000p0", "0x0.030000000p0",
2456             "0x0.030000000p0", "0x0.030000000p0",
2457             "0x0.030000000p0", "0x0.030000000p0",
2458             "0x0.030000000p0", "0x0.030000000p0",
2459             "0x0.030000000p0", "0x0.030000000p0",
2460             "0x0.030000000p0", "0x0.030000000p0",
2461             "0x0.030000000p0", "0x0.030000000p0",
2462             "0x0.030000000p0", "0x0.030000000p0",
2463             "0x0.030000000p0", "0x0.030000000p0",
2464             "0x0.030000000p0", "0x0.030000000p0",
2465             "0x0.030000000p0", "0x0.030000000p0",
2466             "0x0.030000000p0", "0x0.030000000p0",
2467             "0x0.030000000p0", "0x0.030000000p0",
2468             "0x0.030000000p0", "0x0.030000000p0",
2469             "0x0.030000000p0", "0x0.030000000p0",
2470             "0x0.030000000p0", "0x0.030000000p0",
2471             "0x0.030000000p0", "0x0.030000000p0",
2472             "0x0.030000000p0", "0x0.030000000p0",
2473             "0x0.030000000p0", "0x0.030000000p0",
2474             "0x0.030000000p0", "0x0.030000000p0",
2475             "0x0.030000000p0", "0x0.030000000p0",
2476             "0x0.030000000p0", "0x0.030000000p0",
2477             "0x0.030000000p0", "0x0.030000000p0",
2478             "0x0.030000000p0", "0x0.030000000p0",
2479             "0x0.030000000p0", "0x0.030000000p0",
2480             "0x0.030000000p0", "0x0.030000000p0",
2481             "0x0.030000000p0", "0x0.030000000p0",
2482             "0x0.030000000p0", "0x0.030000000p0",
2483             "0x0.030000000p0", "0x0.030000000p0",
2484             "0x0.030000000p0", "0x0.030000000p0",
2485             "0x0.030000000p0", "0x0.030000000p0",
2486             "0x0.030000000p0", "0x0.030000000p0",
2487             "0x0.030000000p0", "0x0.030000000p0",
2488             "0x0.030000000p0", "0x0.030000000p0",
2489             "0x0.030000000p0", "0x0.030000000p0",
2490             "0x0.030000000p0", "0x0.030000000p0",
2491             "0x0.030000000p0", "0x0.030000000p0",
2492             "0x0.030000000p0", "0x0.030000000p0",
2493             "0x0.030000000p0", "0x0.030000000p0",
2494             "0x0.030000000p0", "0x0.030000000p0",
2495             "0x0.030000000p0", "0x0.030000000p0",
2496             "0x0.030000000p0", "0x0.030000000p0",
2497             "0x0.030000000p0", "0x0.030000000p0",
2498             "0x0.030000000p0", "0x0.030000000p0",
2499             "0x0.030000000p0", "0x0.030000000p0",
2500             "0x0.030000000p0", "0x0.030000000p0",
2501             "0x0.030000000p0", "0x0.030000000p0",
2502             "0x0.030000000p0", "0x0.030000000p0",
2503             "0x0.030000000p0", "0x0.030000000p0",
2504             "0x0.030000000p0", "0x0.030000000p0",
2505             "0x0.030000000p0", "0x0.030000000p0",
2506             "0x0.030000000p0", "0x0.030000000p0",
2507             "0x0.030000000p0", "0x0.030000000p0",
2508             "0x0.030000000p0", "0x0.030000000p0",
2509             "0x0.030000000p0", "0x0.030000000p0",
2510             "0x0.030000000p0", "0x0.030000000p0",
2511             "0x0.030000000p0", "0x0.030000000p0",
2512             "0x0.030000000p0", "0x0.030000000p0",
2513             "0x0.030000000p0", "0x0.030000000p0",
2514             "0x0.030000000p0", "0x0.030000000p0",
2515             "0x0.030000000p0", "0x0.030000000p0",
2516             "0x0.030000000p0", "0x0.030000000p0",
2517             "0x0.030000000p0", "0x0.030000000p0",
2518             "0x0.030000000p0", "0x0.030000000p0",
2519             "0x0.030000000p0", "0x0.030000000p0",
2520             "0x0.030000000p0", "0x0.030000000p0",
2521             "0x0.030000000p0", "0x0.030000000p0",
2522             "0x0.030000000p0", "0x0.030000000p0",
2523             "0x0.030000000p0", "0x0.030000000p0",
2524             "0x0.030000000p0", "0x0.030000000p0",
2525             "0x0.030000000p0", "0x0.030000000p0",
2526             "0x0.030000000p0", "0x0.030000000p0",
2527             "0x0.030000000p0", "0x0.030000000p0",
2528             "0x0.030000000p0", "0x0.030000000p0",
2529             "0x0.030000000p0", "0x0.030000000p0",
2530             "0x0.030000000p0", "0x0.030000000p0",
2531             "0x0.030000000p0", "0x0.030000000p0",
2532             "0x0.030000000p0", "0x0.030000000p0",
2533             "0x0.030000000p0", "0x0.030000000p0",
2534             "0x0.030000000p0", "0x0.030000000p0",
2535             "0x0.030000000p0", "0x0.030000000p0",
2536             "0x0.030000000p0", "0x0.030000000p0",
2537             "0x0.030000000p0", "0x0.030000000p0",
2538             "0x0.030000000p0", "0x0.030000000p0",
2539             "0x0.030000000p0", "0x0.030000000p0",
2540             "0x0.030000000p0", "0x0.030000000p0",
2541             "0x0.030000000p0", "0x0.030000000p0",
2542             "0x0.030000000p0", "0x0.030000000p0",
2543             "0x0.030000000p0", "0x0.030000000p0",
2544             "0x0.030000000p0", "0x0.030000000p0",
2545             "0x0.030000000p0", "0x0.030000000p0",
2546             "0x0.030000000p0", "0x0.030000000p0",
2547             "0x0.030000000p0", "0x0.030000000p0",
2548             "0x0.030000000p0", "0x0.030000000p0",
2549             "0x0.030000000p0", "0x0.030000000p0",
2550             "0x0.030000000p0", "0x0.030000000p0",
2551             "0x0.030000000p0", "0x0.030000000p0",
2552             "0x0.030000000p0", "0x0.030000000p0",
2553             "0x0.030000000p0", "0x0.030000000p0",
2554             "0x0.030000000p0", "0x0.030000000p0",
2555             "0x0.030000000p0", "0x0.030000000p0",
2556             "0x0.030000000p0", "0x0.030000000p0",
2557             "0x0.030000000p0", "0x0.030000000p0",
2558             "0x0.030000000p0", "0x0.030000000p0",
2559             "0x0.030000000p0", "0x0.030000000p0",
2560             "0x0.030000000p0", "0x0.030000000p0",
2561             "0x0.030000000p0", "0x0.030000000p0",
2562             "0x0.030000000p0", "0x0.030000000p0",
2563             "0x0.030000000p0", "0x0.030000000p0",
2564             "0x0.030000000p0", "0x0.030000000p0",
2565             "0x0.030000000p0", "0x0.030000000p0",
2566             "0x0.030000000p0", "0x0.030000000p0",
2567             "0x0.030000000p0", "0x0.030000000p0",
2568             "0x0.030000000p0", "0x0.030000000p0",
2569             "0x0.030000000p0", "0x0.030000000p0",
2570             "0x0.030000000p0", "0x0.030000000p0",
2571             "0x0.030000000p0", "0x0.030000000p0",
2572             "0x0.030000000p0", "0x0.030000000p0",
2573             "0x0.030000000p0", "0x0.030000000p0",
2574             "0x0.030000000p0", "0x0.030000000p0",
2575             "0x0.030000000p0", "0x0.030000000p0",
2576             "0x0.030000000p0", "0x0.030000000p0",
2577             "0x0.030000000p0", "0x0.030000000p0",
2578             "0x0.030000000p0", "0x0.030000000p0",
2579             "0x0.030000000p0", "0x0.030000000p0",
2580             "0x0.030000000p0", "0x0.030000000p0",
2581             "0x0.030000000p0", "0x0.030000000p0",
2582             "0x0.030000000p0", "0x0.030000000p0",
2583             "0x0.030000000p0", "0x0.030000000p0",
2584             "0x0.030000000p0", "0x0.030000000p0",
2585             "0x0.030000000p0", "0x0.030000000p0",
2586             "0x0.030000000p0", "0x0.030000000p0",
2587             "0x0.030000000p0", "0x0.030000000p0",
2588             "0x0.030000000p0", "0x0.030000000p0",
2589             "0x0.030000000p0", "0x0.030000000p0",
2590             "0x0.030000000p0", "0x0.030000000p0",
2591             "0x0.030000000p0", "0x0.030000000p0",
2592             "0x0.030000000p0", "0x0.030000000p0",
2593             "0x0.030000000p0", "0x0.030000000p0",
2594             "0x0.030000000p0", "0x0.030000000p0",
2595             "0x0.030000000p0", "0x0.030000000p0",
2596             "0x0.030000000p0", "0x0.030000000p0",
2597             "0x0.030000000p0", "0x0.030000000p0",
2598             "0x0.030000000p0", "0x0.030000000p0",
2599             "0x0.030000000p0", "0x0.030000000p0",
2600             "0x0.030000000p0", "0x0.030000000p0",
2601             "0x0.030000000p0", "0x0.030000000p0",
2602             "0x0.030000000p0", "0x0.030000000p0",
2603             "0x0.030000000p0", "0x0.030000000p0",
2604             "0x0.030000000p0", "0x0.030000000p0",
2605             "0x0.030000000p0", "0x0.030000000p0",
2606             "0x0.030000000p0", "0x0.030000000p0",
2607             "0x0.030000000p0", "0x0.030000000p0",
2608             "0x0.030000000p0", "0x0.030000000p0",
2609             "0x0.030000000p0", "0x0.030000000p0",
2610             "0x0.030000000p0", "0x0.030000000p0",
2611             "0x0.030000000p0", "0x0.030000000p0",
2612             "0x0.030000000p0", "0x0.030000000p0",
2613             "0x0.030000000p0", "0x0.030000000p0",
2614             "0x0.030000000p0", "0x0.030000000p0",
2615             "0x0.030000000p0", "0x0.030000000p0",
2616             "0x0.030000000p0", "0x0.030000000p0",
2617             "0x0.030000000p0", "0x0.030000000p0",
2618             "0x0.030000000p0", "0x0.030000000p0",
2619             "0x0.030000000p0", "0x0.030000000p0",
2620             "0x0.030000000p0", "0x0.030000000p0",
2621             "0x0.030000000p0", "0x0.030000000p0",
2622             "0x0.030000000p0", "0x0.030000000p0",
2623             "0x0.030000000p0", "0x0.030000000p0",
2624             "0x0.030000000p0", "0x0.030000000p0",
2625             "0x0.030000000p0", "0x0.030000000p0",
2626             "0x0.030000000p0", "0x0.030000000p0",
2627             "0x0.030000000p0", "
```

```

2409
2410     const static ap_ufixed<36,0> exp2_x_msb_2_m1_t-
able[32] = {
2411         "0x0.0p0", "0x0.02C9A3E78p0",
2412         "0x0.059B0D316p0", "0x0.087451876p0",
2413         "0x0.0B5586CFAp0", "0x0.0E3EC32D4p0",
2414         "0x0.11301D012p0", "0x0.1429AAEA9p0",
2415         "0x0.172B83C7Dp0", "0x0.1A35BEB70p0",
2416         "0x0.1D4873169p0", "0x0.2063B8863p0",
2417         "0x0.2387A6E75p0", "0x0.26B4565E2p0",
2418         "0x0.29E9DF520p0", "0x0.2D285A6E4p0",
2419         "0x0.306FE0A32p0", "0x0.33C08B264p0",
2420         "0x0.371A7373Bp0", "0x0.3A7DB34E6p0",
2421         "0x0.3DEA64C12p0", "0x0.4160A21F7p0",
2422         "0x0.44E086062p0", "0x0.486A2B5C1p0",
2423         "0x0.4BFDAD536p0", "0x0.4F9B2769Dp0",
2424         "0x0.5342B569Dp0", "0x0.56F4736B5p0",
2425         "0x0.5AB07DD48p0", "0x0.5E76F15ADp0",
2426         "0x0.6247EB03Ap0", "0x0.662388255p0"
2427     };
2428     ap_ufixed<36,0> exp2_x_msb_2_m1 =
exp2_x_msb_2_m1_table[x_msb_ind_2];
2429
2430     ap_ufixed<66,-6> f_x_msb_2_lsb = exp2_x_msb_2_m1
* exp2_x_msb_3_lsb_m1;
2431     ap_ufixed<30,-6> f_x_msb_2_lsb_s = f_x_msb_2_lsb;
2432     ap_ufixed<36,0> exp2_x_msb_2_3_lsb_m1 =
f_x_msb_2_lsb_s + exp2_x_msb_2_m1 + exp2_x_msb_3_lsb_m1;
2433
2434     const static ap_ufixed<36,16> exp2_x_msb_1_t-
able[64] = {
2435         "0x1.00000p0", "0x1.6A09Ep0", "0x2.00000p0",
2436         "0x2.D413Dp0",
2437         "0x4.00000p0", "0x5.A827Ap0", "0x8.00000p0",
2438         "0xB.504F3p0",
2439         "0x10.00000p0", "0x16.A09E6p0",
2440         "0x20.00000p0", "0x2D.413CDp0",
2441         "0x40.00000p0", "0x5A.8279Ap0",
2442         "0x80.00000p0", "0xB5.04F33p0",
2443         "0x100.00000p0", "0x16A.09E66p0",
2444         "0x200.00000p0", "0x2D4.13CCDp0",
2445         "0x400.00000p0", "0x5A8.2799Ap0",
2446         "0x800.00000p0", "0xB50.4F334p0",
2447         "0x1000.00000p0", "0x16A0.9E668p0",
2448         "0x2000.00000p0", "0x2D41.3CCD0p0",
2449         "0x4000.00000p0", "0x5A82.799A0p0",
2450         "0x8000.00000p0", "0xB504.F3340p0",
2451         "0x0.00010p0", "0x0.00017p0", "0x0.00020p0",
2452         "0x0.0002Dp0",
2453         "0x0.00040p0", "0x0.0005Bp0", "0x0.00080p0",
2454         "0x0.000B5p0",
2455         "0x0.00100p0", "0x0.0016Ap0", "0x0.00200p0",
2456         "0x0.002D4p0",
2457         "0x0.00400p0", "0x0.005A8p0", "0x0.00800p0",
2458         "0x0.00B50p0",
2459         "0x0.01000p0", "0x0.016A1p0", "0x0.02000p0",
2460         "0x0.02D41p0",
2461         "0x0.04000p0", "0x0.05A82p0", "0x0.08000p0",
2462         "0x0.0B505p0",
2463         "0x0.10000p0", "0x0.16A0Ap0", "0x0.20000p0",
2464         "0x0.2D414p0",
2465         "0x0.40000p0", "0x0.5A828p0", "0x0.80000p0",
2466         "0x0.B504Fp0"
2467     };
2468     ap_ufixed<36,16> exp2_x_msb_1 = exp2_x_msb_1_t-
able[x_msb_ind_1];
2469     ap_ufixed<72,16> y_lo = exp2_x_msb_1 *
exp2_x_msb_2_3_lsb_m1;
2470     ap_ufixed<36,16> y_lo_s = y_lo;
2471     ap_ufixed<36,16> y_l = y_lo_s + exp2_x_msb_1;
2472     y = y_l;
2473
2474     if(I<17) {
2475         bool overf_2 = 0;
2476         for(int j=31; j>=I+15; j--) {
2477             #pragma HLS unroll
2478             if(y[j]) {
2479                 overf_2 = 1;
2480             }
2481         }
2482         if(overf_2) {
2483             for(int j=31; j>=I+15; j--) {
2484                 #pragma HLS unroll
2485                 y[j] = 0;
2486             }
2487             for(int j=I+14; j>=0; j--) {
2488                 #pragma HLS unroll
2489                 y[j] = 1;
2490             }
2491         }
2492     }
2493     r = y;
2494 } else {
2495     ap_fixed<65,33> x_l = 0;
2496     for(int j=32-F; j<32+I; j++) {
2497         #pragma HLS unroll
2498         x_l[j] = x[j-(32-F)];
2499     }

```

```

2500     for(int j=32+I; j<=64; j++) {
2501         #pragma HLS unroll
2502         x_l[j] = x[W-1];
2503     }
2504     ap_ufixed<64,32> y = 0;
2505     ap_fixed<6,6> x_l_int;
2506     x_l_int(5,0) = x_l(37,32);
2507     ap_ufixed<32,0> x_l_fract;
2508     x_l_fract(31,0) = x_l(31,0);
2509
2510     bool overf_1 = 0;
2511     for(int j=37; j<64; j++) {
2512         #pragma HLS unroll
2513         if(x_l[j]!=x_l[64]) {
2514             overf_1 = 1;
2515         }
2516     }
2517     if(overf_1) {
2518         if(!x_l[64]) {
2519             for(int j=0; j<64; j++) {
2520                 #pragma HLS unroll
2521                 y[j] = 1;
2522             }
2523         } else {
2524             ap_uint<8> x_msb_ind_1;
2525             x_msb_ind_1(7,0) = x_l(37,30);
2526             ap_uint<8> x_msb_ind_2;
2527             x_msb_ind_2(7,0) = x_l(29,22);
2528             ap_uint<8> x_msb_ind_3;
2529             x_msb_ind_3(7,0) = x_l(21,14);
2530             ap_uint<8> x_msb_ind_4;
2531             x_msb_ind_4(7,0) = x_l(13,6);
2532             ap_uint<6> x_lsb_ind;
2533             x_lsb_ind(5,0) = x_l(5,0);
2534
2535             const static ap_ufixed<42,-26> exp2_x_lsb_m1_t-
able[64] = {
2536                 "0x0.0p0", "0x0.00000000B17217F81p0",
2537                 "0x0.0000000162E42FF0Ap0", "0x0.00000002145647E9Ap0",
2538                 "0x0.00000002C5C85FE32p0",
2539                 "0x0.00000003773A77DD2p0", "0x0.000000042AC8FD79p0",
2540                 "0x0.00000004DA1EA7D28p0",
2541                 "0x0.000000058B90BFCDFp0",
2542                 "0x0.000000063D02D7C9Dp0", "0x0.00000006EE74EFC63p0",
2543                 "0x0.000000079FE707C31p0",
2544                 "0x0.0000000851591FC07p0",
2545                 "0x0.0000000902CB37BE4p0", "0x0.00000009B43D4FBC9p0",
2546                 "0x0.0000000A65AF67BB6p0",
2547                 "0x0.0000000B17217FBAAp0", "0x0.0000000B-
C89397BA6p0", "0x0.0000000C7A05AFBA9p0", "0x0.0000000D2B77C7BB5p0",
2548                 "0x0.0000000DDCE9DFBC8p0", "0x0.0000000E85E8B-
F7BE2p0", "0x0.0000000F3FCE0FC05p0", "0x0.0000000FF14027C2Fp0",
2549                 "0x0.00000010A2B23FC61p0",
2550                 "0x0.00000011542457C9Ap0", "0x0.0000001205966FCDBp0",
2551                 "0x0.00000012B70887D24p0",
2552                 "0x0.00000013687A9FD75p0",
2553                 "0x0.0000001419ECB7DCDp0", "0x0.00000014CB5ECFE2Dp0",
2554                 "0x0.000000157CD0E7E94p0",
2555                 "0x0.000000162E42FFF03p0", "0x0.00000016DF-
B517F7Ap0", "0x0.0000001791272FF9p0", "0x0.0000001842994807Fp0",
2556                 "0x0.00000018F40B6010Dp0",
2557                 "0x0.00000019A57D781A3p0", "0x0.0000001A56EF90240p0",
2558                 "0x0.0000001B0861A82E6p0",
2559                 "0x0.0000001BB9D3C0392p0",
2560                 "0x0.0000001C6B45D8447p0", "0x0.0000001D1CB7F0503p0",
2561                 "0x0.0000001DCE2A085C7p0",
2562                 "0x0.0000001E7F9C20692p0",
2563                 "0x0.0000001F310E38765p0", "0x0.0000001FE2850840p0",
2564                 "0x0.0000002093F268923p0",
2565                 "0x0.00000021456480A0Dp0",
2566                 "0x0.00000021F6D698AFFp0", "0x0.00000022A848B0BF9p0",
2567                 "0x0.0000002359BAC8CFAp0",
2568                 "0x0.000000240B2CE0E03p0", "0x0.00000024BC9E-
F8F14p0", "0x0.000000256E11102Cp0", "0x0.000000261F832914Cp0",
2569                 "0x0.00000026D0F541274p0",
2570                 "0x0.000000278267593A3p0", "0x0.0000002833D9714DBp0",
2571                 "0x0.00000028E548B9619p0",
2572                 "0x0.0000002996BDA1760p0", "0x0.0000002A482F-
B98AEp0", "0x0.0000002AF9A1D1A04p0", "0x0.0000002BAB13E9B61p0"
2573             };
2574             ap_ufixed<42,-26> exp2_x_lsb_m1 =
exp2_x_lsb_m1_table[x_lsb_ind];
2575
2576             const static ap_ufixed<50,-18>
exp2_x_msb_4_m1_table[256] = {
2577                 "0x0.0p0", "0x0.0000002C5C8601CC7p0",
2578                 "0x0.00000058B90C0B48Cp0", "0x0.0000008515921C751p0",
2579                 "0x0.000000B1721835515p0", "0x0.000000D-
DCE9E55DD7p0", "0x0.0000010A2B247E199p0",
2580                 "0x0.0000013687AAAE059p0",
2581                 "0x0.00000162E430E5A19p0",
2582                 "0x0.0000018F40B724ED7p0", "0x0.000001BB9D3D6BE95p0",
2583                 "0x0.000001E7F9C3BA951p0",
2584                 "0x0.00000214564A10F0Dp0",
2585                 "0x0.00000240B2D06EFC7p0", "0x0.0000026D0F56D48B0p0",
2586                 "0x0.000002996BDD42239p0",
2587                 "0x0.000002C5C863B73F0p0",
2588                 "0x0.000002F224EA340A6p0", "0x0.0000031E8170B85Cp0",
2589                 "0x0.0000034ADDF744B10p0",

```

```

2539         "0x0.000003773A7DD88C3p0",
"0x0.000003A3970474175p0", "0x0.000003CFF38B17526p0",
"0x0.000003FC5011C23D0p0",
2540         "0x0.00000428AC9874D86p0",
"0x0.00000455091F2F234p0", "0x0.0000048165A5F11E1p0",
"0x0.000004ADC22C8AC8Dp0",
2541         "0x0.000004DA1EB38C238p0",
"0x0.000005067B3A652E2p0", "0x0.00000532D7C145E8Bp0",
"0x0.0000055F34482E533p0",
2542         "0x0.0000058B90CF1E6D9p0",
"0x0.000005B7ED561637Fp0", "0x0.000005E449DD15B24p0",
"0x0.00000610A6641CDC8p0",
2543         "0x0.0000063D02EB2BB6Bp0",
"0x0.000006695F724240Dp0", "0x0.00000695BBF9607AEp0",
"0x0.000006C218808664Dp0",
2544         "0x0.000006EE7507B3FECp0",
"0x0.0000071AD18EE948Ap0", "0x0.000007472E1626427p0",
"0x0.000007738A9D6AEC2p0",
2545         "0x0.0000079FE724B745Dp0", "0x0.000007C-
C43AC0B4F7p0", "0x0.000007F8A0336708Fp0",
"0x0.00000824FCBACA727p0",
2546         "0x0.000008515942358BEp0", "0x0.0000087D-
B5C9A8553p0", "0x0.000008AA125122CE8p0", "0x0.000008D66ED8A4F7Bp0",
2547         "0x0.00000902CB02ED0Ep0",
"0x0.0000092F27E7C059Fp0", "0x0.0000095B846F59930p0",
"0x0.00000987E0F6FA7C0p0",
2548         "0x0.000009B43D7EA314Ep0",
"0x0.000009E09A06535DCp0", "0x0.00000A0CF68E0B568p0",
"0x0.00000A395315CAFF3p0",
2549         "0x0.00000A65AF9D9257Ep0",
"0x0.00000A920C2561607p0", "0x0.00000ABE68AD38190p0", "0x0.00000AE-
AC53516817p0",
2550         "0x0.00000B1721BCFC99Ep0",
"0x0.00000B437E44EA623p0", "0x0.00000B6FDACCDFA7p0",
"0x0.00000B9C3754D0202p0",
2551         "0x0.00000BC893DCE1DADp0", "0x0.00000B-
F4F064EE62Fp0", "0x0.00000C214CED029AFp0",
"0x0.00000C4DA9751E82Ep0",
2552         "0x0.00000C7A05FD421ADp0",
"0x0.00000CA662856D62Ap0", "0x0.00000CD2BF0DA05A6p0",
"0x0.00000CFF1B95DB022p0",
2553         "0x0.00000D2B781E1D59Cp0",
"0x0.00000D57D4A667615p0", "0x0.00000D84312EB918Dp0", "0x0.00000D-
B08DB712805p0",
2554         "0x0.00000DDCEA3F7397Bp0",
"0x0.00000E0946C7DC5F0p0", "0x0.00000E3A3504CD65p0",
"0x0.00000E61FFD8C4FD8p0",
2555         "0x0.00000E8E5C6144D4Ap0", "0x0.00000EB-
AB8E9CC5BBp0", "0x0.00000EE715725B92Cp0", "0x0.00000F1371-
FAF279Bp0",
2556         "0x0.00000F3FCE8391109p0",
"0x0.00000F6C2B0C37576p0", "0x0.00000F988794E54E3p0",
"0x0.00000FC4E41D9AF4Ep0",
2557         "0x0.00000FF140A6584B8p0",
"0x0.0000101D9D2F1D521p0", "0x0.00001049F9B7EA08Ap0",
"0x0.000010765640BE6F1p0",
2558         "0x0.000010A2B2C99A857p0",
"0x0.000010CF0F527E4BCp0", "0x0.000010FB6BDB69C21p0",
"0x0.00001127C8645CE84p0",
2559         "0x0.0000115424ED57BE6p0",
"0x0.0000118081765A447p0", "0x0.000011ACDDFF647A7p0",
"0x0.000011D93A8876607p0",
2560         "0x0.0000120597118FF65p0", "0x0.00001231F39-
AB13C2p0", "0x0.0000125E023DA31Ep0", "0x0.0000128AACAD0AD7Ap0",
2561         "0x0.000012B70936432D4p0", "0x0.000012E365B-
F8332Dp0", "0x0.0000130FC248CAE85p0", "0x0.0000133C1ED21A4DDp0",
2562         "0x0.000013687B5B71633p0",
"0x0.00001394D7E4D0288p0", "0x0.000013C1346E369DDp0",
"0x0.000013ED90F7A4C30p0",
2563         "0x0.00001419ED811A982p0",
"0x0.000014464A0A981D4p0", "0x0.00001472A6941D524p0",
"0x0.0000149F031DAA373p0",
2564         "0x0.000014CB5FA73ECC1p0", "0x0.000014F7B-
C30DB10Fp0", "0x0.0000152418BA7F05Bp0", "0x0.0000155075442AAA6p0",
2565         "0x0.0000157CD1CDDDF1p0",
"0x0.000015A92E579903Ap0", "0x0.000015D58AE15BB83p0",
"0x0.00001601E76B261CAp0",
2566         "0x0.0000162E43F4F8310p0",
"0x0.0000165AA07ED1F56p0", "0x0.00001686FD08B369Ap0",
"0x0.000016B359929C8DEp0",
2567         "0x0.000016DFB61C8D620p0",
"0x0.0000170C12A685E62p0", "0x0.000017386F30861A2p0",
"0x0.00001764CBBAB8FE1p0",
2568         "0x0.0000179128449D920p0", "0x0.000017BD84CE-
B4D5Dp0", "0x0.000017E9E158D3C9Ap0", "0x0.000018163D2FA6D6p0",
2569         "0x0.000018429A6D28C10p0", "0x0.0000186E-
F6F75EC4Ap0", "0x0.0000189B53819C782p0", "0x0.000018C7B00BE1DBAp0",
2570         "0x0.000018F40C962EEF0p0",
"0x0.00001920692083B26p0", "0x0.0000194CC5AAE025Bp0",
"0x0.0000197922354448Ep0",
2571         "0x0.000019A57EBF01C1p0", "0x0.000019D1D-
B4A239F3p0", "0x0.000019FE37D49ED23p0", "0x0.00001A2A945F21B53p0",
2572         "0x0.00001A56F0E9AC482p0",
"0x0.00001A834D743EB80p0", "0x0.00001AAFA9FED87DCp0", "0x0.00001AD-
C06897A208p0",
2573         "0x0.00001B08631423733p0", "0x0.00001B34B-
F9ED475Dp0", "0x0.00001B611C298D286p0", "0x0.00001B8D78B44D8ADp0",
2574         "0x0.00001BB9D53F159D4p0",
"0x0.00001BE631C9E55FAp0", "0x0.00001C128E54BCD1Fp0",
"0x0.00001C3EEADF9BF43p0",
2575         "0x0.00001C6B476A82C66p0",
"0x0.00001C97A3F571488p0", "0x0.00001CC4080677A9p0",
"0x0.00001CF05D0B655C9p0",
2576         "0x0.00001D1CB9966AEE8p0",
"0x0.00001D49162178306p0", "0x0.00001D7572AC8D223p0",
"0x0.00001DA1CF37A9C40p0",
2577         "0x0.00001DCE2BC2CE15Bp0", "0x0.00001DFA884D-
FA175p0", "0x0.00001E26E4D92DC8Ep0", "0x0.00001E534164692A6p0",
2578         "0x0.00001E7F9DEFAAC3BEp0", "0x0.00001EAB-
FATAF6FD4p0", "0x0.00001ED85706496E9p0", "0x0.00001F04B391A38FEp0",
2579         "0x0.00001F31101D05611p0",
"0x0.00001F5D6CA86EE23p0", "0x0.00001F89C933E0135p0", "0x0.00001F-
B625BF58F45p0",
2580         "0x0.00001FE2824AD9855p0",
"0x0.0000200EDED661C63p0", "0x0.0000203B3B61F1871p0",
"0x0.0000206797ED8957Dp0",
2581         "0x0.00002093F47928A89p0",
"0x0.000020C05104CFA93p0", "0x0.000020ECAD907E59Dp0",
"0x0.000021190A1C34BA6p0",
2582         "0x0.0000214566A7F2CAEp0",
"0x0.00002171C333B88B4p0", "0x0.0000219E1FBF85FBAp0",
"0x0.000021CA7C4B5B1BFp0",
2583         "0x0.000021F6D8D737EC3p0",
"0x0.00002223363631C6C6p0", "0x0.0000224F91EF089C7p0", "0x0.0000227-
BEE7AFC7C8p0",
2584         "0x0.000022A84B06F80C8p0",
"0x0.000022D4A792FB4C7p0", "0x0.00002301041F063C5p0",
"0x0.0000232D60AB18DC3p0",
2585         "0x0.00002359BD37332BFp0",
"0x0.0000238619C3552BAp0", "0x0.000023B27647FEDB4p0",
"0x0.000023DED2DBB03ADp0",
2586         "0x0.0000240B2F67E94A5p0", "0x0.000024378B-
F42A89Dp0", "0x0.00002463E88072793p0", "0x0.00002490450CC2988p0",
2587         "0x0.000024BCA1991A67Dp0",
"0x0.000024E8FE2579E70p0", "0x0.000025155AB1E1163p0",
"0x0.00002541B73E4FF54p0",
2588         "0x0.0000256E13CAC6845p0",
"0x0.0000259A705744C34p0", "0x0.000025C6CE3CAB23p0",
"0x0.000025F3297058511p0",
2589         "0x0.0000261F85FCED9FDp0",
"0x0.0000264BE2898A9E9p0", "0x0.000026783F162F4D4p0",
"0x0.000026A49BA2DBABEp0",
2590         "0x0.000026D0F82F8BA7p0", "0x0.000026FD54B-
C4B78Fp0", "0x0.00002729B1490EE75p0", "0x0.000027560DD5DA058p0",
2591         "0x0.000027826A62ACD41p0", "0x0.000027AEC6E-
F87525p0", "0x0.000027DB237C69808p0", "0x0.000028078009535EAp0",
2592         "0x0.00002833DC9644ECBp0",
"0x0.0000286039233E2ABp0", "0x0.0000288C95B03F18Bp0",
"0x0.000028B8F23D47B69p0",
2593         "0x0.000028E54ECA58046p0", "0x0.00002911-
AB5770023p0", "0x0.0000293E07E48FAFEp0", "0x0.0000296A6471B70D9p0",
2594         "0x0.0000296C0FEE61B2p0",
"0x0.000029C31D8C1CD8Bp0", "0x0.000029EF7A195B463p0",
"0x0.00002A1BD6A6A1639p0",
2595         "0x0.00002A483333EF30Fp0",
"0x0.00002A748FC144AE4p0", "0x0.00002AA0EC4EA1DB8p0",
"0x0.00002ACD48DC06B8Bp0",
2596         "0x0.00002AF9A5697345Dp0",
"0x0.00002B2601F6E782Ep0", "0x0.00002B52E84636FEp0",
"0x0.00002B7EBB11E70CDp0",
2597         "0x0.00002BAB179F7259Bp0", "0x0.00002B-
D7742D05568p0", "0x0.00002C03D0BAA0035p0",
"0x0.00002C3024842600p0",
2598         };
2599         ap_ufixed<50,-18> exp2_x_msb_4_m_1 =
exp2_x_msb_4_m_1_table[x_msb_ind_4];
2600         ap_ufixed<92,-44> f_x_msb_4_lsb = exp2_x_lsb_m_1
* exp2_x_msb_4_m_1;
2601         ap_ufixed<24,-44> f_x_msb_4_lsb_s =
f_x_msb_4_lsb;
2602         ap_ufixed<50,-18> exp2_x_msb_4_lsb_m_1 =
f_x_msb_4_lsb_s + exp2_x_msb_4_m_1 + exp2_x_lsb_m_1;
2603
2604         const static ap_ufixed<58,-10>
exp2_x_msb_3_m_1_table[256] = {
2605             "0x0.0p0", "0x0.00002C5C89D5EC6CAp0",
"0x0.000058B91B5BC9AE3p0", "0x0.00008515B4919919Ap0",
2606             "0x0.0000B17255775C040p0", "0x0.0000DDCE-
FE0D13C26p0", "0x0.00010A2BAE52C1A9Bp0", "0x0.000136886648670F0p0",
2607             "0x0.000162E525EE05475p0",
"0x0.00018F41ED439DA7Bp0", "0x0.0001BB9EBC4931853p0", "0x0.0001E7F-
B92FEC234Bp0",
2608             "0x0.000214587164510B6p0",
"0x0.000240B55779DF5E3p0", "0x0.00026D12453F6E823p0",
"0x0.0002996F3AB4FFCC6p0",
2609             "0x0.0002C5CC37DA9491Dp0",
"0x0.0002F2293CB0E278p0", "0x0.00031E864935CDE29p0",
"0x0.00034AE35D6B7517Ep0",
2610             "0x0.000377407951251C9p0",
"0x0.0003A39D9CE6DF45Bp0", "0x0.0003CFFAC82CA4E84p0",
"0x0.0003FC57FB2277595p0",
2611             "0x0.000428B53C857EDDp0",
"0x0.00045512781E47FAFp0", "0x0.0004816F22448D5Ap0", "0x0.0004AD-
CD13DA5BD2Fp0",
2612             "0x0.0004DA2A6D408247Ep0",
"0x0.00050687CE56BD89Ap0", "0x0.000532E5371D0EED1p0",
"0x0.00055F42A79377C75p0",
2613             "0x0.00058BA01FB9F96D7p0", "0x0.0005B7F-
D9F0995347p0", "0x0.0005E45B27174C716p0",
"0x0.000610B8B64E20794p0",

```



```

2614          "0x0.00063D164D3512A14p0", "0x0.00066973EB-
CC243E4p0", "0x0.000665D1921356A57p0", "0x0.0006C22F400AAB2BCp0",
2615          "0x0.0006EE8CF5B223266p0", "0x0.00071AE-
AB309BFEA4p0", "0x0.00074748781182CC7p0",
"0x0.000773A644C96D221p0",
2616          "0x0.0007A004193180402p0", "0x0.0007C-
C61F549BD7BCp0", "0x0.0007F8BF9122629Ep0", "0x0.0008251DC48ABB9-
FAP0",
2617          "0x0.0008517BB7B37F322p0", "0x0.00087D-
D9B28C72365p0", "0x0.0008AA37B51596015p0", "0x0.0008D695BF4EE-
BE83p0",
2618          "0x0.000902F3D13875400p0",
"0x0.00092F51EAD2335DCp0", "0x0.00095B80C1C2796Ap0",
"0x0.0009880E3516533FAP0",
2619          "0x0.0009B46C65C0B7ADCp0",
"0x0.0009E0CA9E1B56363p0", "0x0.000A0D28DE26302DFp0",
"0x0.000A398725E146EA2p0",
2620          "0x0.000A65E5754C9BBFBp0", "0x0.000A9243C-
C683003Ep0", "0x0.000ABEA22B34050BAp0", "0x0.000AEB0091B01C2C1p0",
2621          "0x0.000B175EFFDC76BA4p0", "0x0.000B43B-
D75B9160B4p0", "0x0.000B701BF345FB742p0",
"0x0.000B9C7A7883284A1p0",
2622          "0x0.000BC8D905709DE20p0", "0x0.000B-
F5379A0E5D912p0", "0x0.000C2196365C68AC6p0", "0x0.000C4D-
F4DA5AC0890p0",
2623          "0x0.000C7A538609667C0p0",
"0x0.000CA6B239685BDAp0", "0x0.000CD310F477A1F96p0",
"0x0.000CFF6FB7373A2DFp0",
2624          "0x0.000D2BCE81A725CD4p0",
"0x0.000D582D53C7662C6p0", "0x0.000D848C2D97FCA05p0", "0x0.000DB0E-
B0F18EA7E4p0",
2625          "0x0.000DD049F84A311B4p0", "0x0.000E09A8E92B-
D1CC6p0", "0x0.000E3607E1BDCDE6Cp0", "0x0.000E6266E20026BF7p0",
2626          "0x0.000E8EC5E9F2DDAB9p0",
"0x0.000EBB24F995F4003p0", "0x0.000EE78410E96B126p0",
"0x0.000F13E32FED44375p0",
2627          "0x0.000F404256A180C40p0",
"0x0.000F6CA18506220D9p0", "0x0.000F9900BB1B29692p0",
"0x0.000FC55FF8E0982BDp0",
2628          "0x0.000FF1BF3E566FAAAp0",
"0x0.00101E1E8B7CB13ACp0", "0x0.00104A7DE0535E314p0", "0x0.001076D-
D3CDA77E34p0",
2629          "0x0.0010A33CA111FFA5Dp0",
"0x0.0010CF9C0CF9F6CE1p0", "0x0.0010FBF880925EB11p0",
"0x0.0011285AF8DB38A40p0",
2630          "0x0.001154BA7ED485FC0p0",
"0x0.0011811A097E480E0p0", "0x0.0011AD799BD8802F5p0",
"0x0.0011D9D935E32F84p0",
2631          "0x0.00120638D79E57F3Fp0", "0x0.001232988109-
FA419p0", "0x0.00125EF8322617F2Dp0", "0x0.00128B57EAF2B25CDp0",
2632          "0x0.0012B7B7AB6FCAD4Bp0",
"0x0.0012E417739D62AFAP0", "0x0.00131077437B7B42Ap0",
"0x0.00133CD71B0A15E2Ep0",
2633          "0x0.00136936FA4933E58p0",
"0x0.00139596E138D69F9p0", "0x0.0013C1F6CFD8FF663p0",
"0x0.0013EE56C629AF8E9p0",
2634          "0x0.00141AB6C42AE86DBp0", "0x0.00144716C9D-
CAB58Dp0", "0x0.00147376D73EF9A50p0", "0x0.00149FD6EC51D4A76p0",
2635          "0x0.0014CC3709153DB51p0",
"0x0.0014F8972D8936234p0", "0x0.001524F759ADBFA46p0",
"0x0.001551578D82DA755p0",
2636          "0x0.00157DB7C90889038p0",
"0x0.0015AA180C3ECC468p0", "0x0.0015D6785725A593Fp0",
"0x0.001602D8A9BD16406p0",
2637          "0x0.00162F3904051FA13p0", "0x0.00165B9965FD-
C30B7p0", "0x0.001687F9CFA701D45p0", "0x0.0016B45A4100DD50Ep0",
2638          "0x0.0016E0BABA08B56D66p0",
"0x0.00170D1B3AC66FB9Ep0", "0x0.0017397BC33229508p0", "0x0.001765D-
C534E84EF7p0",
2639          "0x0.0017923CEB1B83EBCp0",
"0x0.0017BE9D8A9279ABp0", "0x0.0017EAFE31C771515p0",
"0x0.0018175EE0A66264Cp0",
2640          "0x0.001843BF9735FC2A3p0",
"0x0.0018702055763FF6Cp0", "0x0.00189C811B672F1FAp0",
"0x0.0018C8E1E908CAF9p0",
2641          "0x0.0018F542BE5B14DABp0",
"0x0.001921A39B5E0E174p0", "0x0.00194E048011B804Ap0",
"0x0.00197A656C7613F81p0",
2642          "0x0.0019A6C6608B2346Ap0",
"0x0.0019D3275C50E7458p0", "0x0.0019FF885FC76149Dp0",
"0x0.001A2BE96AE92A8Cp0",
2643          "0x0.001A584A7DC67CB77p0", "0x0.001A84-
AB984F20CB1p0", "0x0.001AB10CBA888038Cp0", "0x0.001AD-
D6DE4729C55Ap0",
2644          "0x0.001B09CF160D7676Fp0",
"0x0.001B36304F590FF1Dp0", "0x0.001B629190556A1B5p0", "0x0.001B8E-
F2D9028648Cp0",
2645          "0x0.001BBB54296065CF2p0",
"0x0.001BE7B5816F0A03Cp0", "0x0.001C1416E12E743BBp0",
"0x0.001C4078489EA5CC2p0",
2646          "0x0.001C6CD9B7BFA00A4p0",
"0x0.001C993B2E91644B3p0", "0x0.001CC59CAD13F3E42p0",
"0x0.001CF1FE3347502A0p0",
2647          "0x0.001D1E5FC12B7A72Bp0",
"0x0.001D4AC156C07412Ap0", "0x0.001D7722F4063E5F4p0",
"0x0.001DA38498FCDAADBp0",
2648          "0x0.001DCFE645A44A533p0", "0x0.001D-
FC47F9FC8EA4Dp0", "0x0.001E28A9B0605A8F7Dp0", "0x0.001E550B79B-
F9AA16p0",
2649          "0x0.001E816D452A64F6Ap0", "0x0.001EAD-
CF1846094CCp0", "0x0.001EDA30F31288F8Ep0",
"0x0.001F0692D58FE5505p0",
2650          "0x0.001F32F4BFBE1FA82p0",
"0x0.001F5F56B19D39559p0", "0x0.001F8BB8AB2D33ADCp0", "0x0.001F-
B81AAC6E1005Ep0",
2651          "0x0.001FE47CB55FCFB33p0", "0x0.002010-
DEC602740ACp0", "0x0.00203D40DE55FE61Ep0", "0x0.002069A2FE5A700D-
Ap0",
2652          "0x0.00209605260FCA635p0",
"0x0.0020C26755760EB80p0", "0x0.0020EEC98C8D3E60Fp0",
"0x0.00211B2BCB555AB35p0",
2653          "0x0.0021478E11CE65045p0",
"0x0.002173F05FF85EA93p0", "0x0.0021A052B5D348F70p0", "0x0.0021C-
CB5135F25430p0",
2654          "0x0.0021F917789BF4E26p0",
"0x0.00222579E589B92A6p0", "0x0.002251DC5A2873702p0",
"0x0.00227E3ED6782508Ep0",
2655          "0x0.0022AAA15A78CF49Cp0",
"0x0.0022D703E62A73880p0", "0x0.00230366798D1318Dp0",
"0x0.00232FC914A0AF516p0",
2656          "0x0.00235C2BB7654986Fp0",
"0x0.0023888E61DAE30EAp0", "0x0.0023B4F114017D30Bp0",
"0x0.0023E153CD0919695p0",
2657          "0x0.00240DB68F61B8E6Cp0",
"0x0.00243A19589B5D0B2p0", "0x0.0024667C2986072BAP0", "0x0.002492D-
F0221B89D9p0",
2658          "0x0.0024BF41E26E72B62p0", "0x0.0024E-
BA4CA6C36CA7p0", "0x0.00251807BA1B062Fdp0", "0x0.0025446-
AB17AE23B5p0",
2659          "0x0.002570CDB08BC425p0", "0x0.00259D30B74D-
C599Fp0", "0x0.0025C993C5C0CF977p0", "0x0.0025F5F6DBE4EB900p0",
2660          "0x0.00262259F9BA1AD8Dp0", "0x0.00264EB-
D1F405EC72p0", "0x0.00267B204C77B8803p0",
"0x0.002687B3816029E92p0",
2661          "0x0.0026D3E6BDF9B3C74p0",
"0x0.0027004A0244579FCp0", "0x0.00272CAD4E4016C7Dp0",
"0x0.00275910A1ECF294Bp0",
2662          "0x0.00278573FD4AEC5BAp0",
"0x0.0027B1D7605A0571Cp0", "0x0.0027DE3ACB1A3F2C6p0",
"0x0.00280A9E3D8B9AE0Cp0",
2663          "0x0.00283701B7AE19E40p0",
"0x0.002863653981BD8B6p0", "0x0.00288FC8C306872C3p0", "0x0.0028B-
C2C543C78189p0",
2664          "0x0.0028E88FED2391AECp0", "0x0.002914F38D-
BB053B1p0", "0x0.0029415736054415Ap0", "0x0.00296DBAE5FFDF93Cp0",
2665          "0x0.00299A1E9DABA90A9p0",
"0x0.0029C6825D08A1CF7p0", "0x0.0029F2E62416CB378p0",
"0x0.002A1F49F2D626981p0",
2666          "0x0.002A4BAD9C946B5464p0",
"0x0.002A7811A76878977p0", "0x0.002AA4758D3B71E0Cp0",
"0x0.002AD0D97ABFA2778p0",
2667          "0x0.002AFD3D6FF50BB0Ep0", "0x0.002B29A16CD-
BAEE23p0", "0x0.002B560571738D609p0", "0x0.002B82697DBCAB816p0",
2668          "0x0.002BAEC0D91B70199Cp0", "0x0.002BD-
B31AD6299FF0p0", "0x0.002C0795D0BF73066p0", "0x0.002C33F9FB-
CD8E052p0"
};
2669
2670      ap_ufixed<58,-10> exp2_x_msb_3_m_1 =
exp2_x_msb_3_m_1_table[x_msb_ind_3];
2671      ap_ufixed<108,-28> f_x_msb_3_lsb =
exp2_x_msb_3_m_1 * exp2_x_msb_4_lsb_m_1;
2672      ap_ufixed<40,-28> f_x_msb_3_lsb_s =
f_x_msb_3_lsb;
2673      ap_ufixed<58,-10> exp2_x_msb_3_4_lsb_m_1 =
f_x_msb_3_lsb_s + exp2_x_msb_3_m_1 + exp2_x_msb_4_lsb_m_1;
2674
2675      const static ap_ufixed<68,0> exp2_x_msb_2_m_1_t-
able[256] = {
2676          "0x0.0p0", "0x0.002C605E2E8CEC507p0",
"0x0.0058C86DA1C09EA20p0", "0x0.0085382FAEF831DA9p0",
2677          "0x0.00B1AFA5ABCBE613p0",
"0x0.00DE2ED0EE0F4F5FDp0", "0x0.010AB5B2CBD117074p0",
"0x0.0137444C9B5B4ED49p0",
2678          "0x0.0163DA9FB33356085p0",
"0x0.019078AD6A19EEFF7p0", "0x0.01BD1E77170B415E7p0", "0x0.01E9CB-
FE113EEC7DCp0",
2679          "0x0.02168143B0280DA82p0",
"0x0.02433E494B754B3ADp0", "0x0.02703103B10DEF7Dp0", "0x0.029C-
CF99D720A0593p0",
2680          "0x0.02C9A3E778060EE6Fp0", "0x0.02F67F-
FA765E5C8E5p0", "0x0.032363D42B0277FAAp0", "0x0.03504F75E-
F0716F0Cp0",
2681          "0x0.037D42E11BBCC0AB4p0", "0x0.03AA3E170AAF-
D8396p0", "0x0.03D7411915A8A60DF0p0", "0x0.04044BE896AB66771p0",
2682          "0x0.04315E86E7F84BD74p0",
"0x0.045E78F5640B9135Cp0", "0x0.048BB935659D8090Ep0",
"0x0.04B8C54847A27E186p0",
2683          "0x0.04E5F72F654B12987p0",
"0x0.051330EC1A03F5E6Ap0", "0x0.0540727FC1761950Dp0", "0x0.056DB-
BEBB786B20D9p0",
2684          "0x0.059B0D31585743AE8p0",
"0x0.05C866520045AA948p0", "0x0.05F5C74F0BEC2665Bp0",
"0x0.06233029D082164855p0",
2685          "0x0.0650A0E3C1F88A8D2p0",
"0x0.067E197E26C140C90p0", "0x0.06AB99FA6407BCB43p0",
"0x0.06D92259D794CB783p0",
2686          "0x0.0706B29DDF6DDC6DCp0",
"0x0.07344AC7D9D50B9FCp0", "0x0.0761EAD925492C4F5p0",
"0x0.078F92D32085D37A9p0",
2687          "0x0.07BD42B72A8362646p0",

```

```

"0x0.07EAF86A277111EFp0", "0x0.0818BA42E7D2F916Ep0",
"0x0.084681ED5A461FA1Ap0",
2688 "0x0.0874518759BC808C3p0",
"0x0.08A22912465F18AD3p0", "0x0.08D0088F8093F0774p0", "0x0.08FD-
F00068FE268ECp0",
2689 "0x0.092BDf66607DFA5FEp0",
"0x0.0959D6C2C830D6B7Fp0", "0x0.0987D61701715C5F5p0", "0x0.09B5D-
D646DD76CB61p0",
2690 "0x0.09E3ECAC6F3834522p0",
"0x0.0A1203F067A6359F1p0", "0x0.0A402331B97153804p0",
"0x0.0A6E4A71C726DBF42p0",
2691 "0x0.0A9C79B1F39192BA3p0",
"0x0.0ACAB0F3A1B9BBF9Fp0", "0x0.0AF8F03834E526EC8p0",
"0x0.0B273781109738876p0",
2692 "0x0.0B5586CF9890F6299p0",
"0x0.0B83DE2530D1104A2p0", "0x0.0B823D833D93ED290p0", "0x0.0BE0A4E-
B2353B3818p0",
2693 "0x0.0C0F145E46C8553E6p0", "0x0.0C3D8B-
DE0CE79A306p0", "0x0.0C6C0B6BDAE52AC5Dp0",
"0x0.0C9A930916329AC51p0",
2694 "0x0.0CC922B7247F7407Bp0", "0x0.0CF7BA776B-
B94138Ap0", "0x0.0D26544B520B98932p0", "0x0.0D5502343DE026A4Bp0",
2695 "0x0.0D83B23395DEB90FBp0", "0x0.0D-
B26A4AC0ED4950Dp0", "0x0.0DE12A7B263007860p0",
"0x0.0E0FF2C62D00965371p0",
2696 "0x0.0E3EC32D3D1A20207p0", "0x0.0E6D9B-
B1BE414CFDDp0", "0x0.0E9C7C55189C62628p0",
"0x0.0ECB6518B4874375Bp0",
2697 "0x0.0EFA55DFDA9C4AD89p0",
"0x0.0F294F0653B45570Ap0", "0x0.0F58503328E6CD3F5p0",
"0x0.0F875985E389B439Ep0",
2698 "0x0.0FB66AFFED31AF232p0", "0x0.0FE584A2AF-
B210670p0", "0x0.1014A66F951CE2F7Ep0", "0x0.1043D06807C2F52E1p0",
2699 "0x0.1073028D7233E3A8Ep0",
"0x0.10A23CE13F3E2431Dp0", "0x0.10D17F64D9EF10A19p0",
"0x0.1100CA19AD92F1C6Bp0",
2700 "0x0.11301D0125B50A4ECp0", "0x0.115F781CAE1-
FA1B08p0", "0x0.118EDB6DB2DC0F18Dp0", "0x0.11BE46F5A032C458Fp0",
2701 "0x0.11EDB8A5E2AB58D6Fp0",
"0x0.121D36AFE70C947FBp0", "0x0.124CBAE51A5C7ABB5p0",
"0x0.127C4756E9E055631p0",
2702 "0x0.12ABDC06C31CBFB93p0", "0x0.12D-
B78F613D5B162Dp0", "0x0.130B1E264A0E8963Cp0",
"0x0.133ACB98D40A191C1p0",
2703 "0x0.136A814F204AAF478p0",
"0x0.139A3F4A9D9222F2p0", "0x0.13CA058CBAE1DEBC9p0",
"0x0.13F9D416E77AEB5F6p0",
2704 "0x0.1429AAEA92DDFB341p0", "0x0.14598A092C-
CB74FD3p0", "0x0.1489717425437EFEP0", "0x0.14B9612CEC860A09Ep0",
2705 "0x0.14E95934F312DC8DCp0",
"0x0.1519598DA9A99DA67p0", "0x0.154962388149E0304p0",
"0x0.15797336EB332DDCAp0",
2706 "0x0.15A98C8A58E512481p0",
"0x0.15D9AE343C1F26130p0", "0x0.1609D83606E119FBFp0",
"0x0.163A0A912B6AC1FBBp0",
2707 "0x0.166A45471C3C2063Dp0",
"0x0.169A88594C1570FE6p0", "0x0.16CAD3C92DF734306p0", "0x0.16F-
B279835223A1D7p0",
2708 "0x0.172B83C7D517ADCFp0",
"0x0.175BE85981992056Cp0", "0x0.178C554EAEAB94030p0", "0x0.17B-
CCA8D088877FCp0",
2709 "0x0.17ED48695BB0109Ep0", "0x0.181D-
CE91C506999D8p0", "0x0.184E5D23816C88275p0", "0x0.187E-
F4200632ACB89p0",
2710 "0x0.18AF9388C8DE9BBBFp0",
"0x0.18E03B5F3F36A9D2Ap0", "0x0.1910EBA4DF1F3D2Ap0",
"0x0.19414A5B1F487077Ap0",
2711 "0x0.1972658375D2F52ABp0", "0x0.19A32F1F59-
AB447C4p0", "0x0.19D4013041DC18BF8p0", "0x0.1A04DBB7A5B12F4DCp0",
2712 "0x0.1A35BE6FCB753CB7p0", "0x0.1A66A42FBE-
C6B6ECp0", "0x0.1A979E2363CF8048Fp0", "0x0.1AC89A936440CC90Dp0",
2713 "0x0.1AF99F8138A1C5FEFp0",
"0x0.1B2AACEE59C528D0Dp0", "0x0.1B5BC2DC40BF03807p0",
"0x0.1B8CE14C66E4C2704p0",
2714 "0x0.1BBE084045CD39AB2p0", "0x0.1BE-
F37B95750B2C0p0", "0x0.1C206FB91588F3A67p0", "0x0.1C51B804-
FAD151512p0",
2715 "0x0.1C82F95281C6B4026p0", "0x0.1CB44AE-
F2547A62ECp0", "0x0.1CE5A51860745F298p0", "0x0.1D1707CFAEAE-
CE86Cp0",
2716 "0x0.1D4873168B9AA7806p0",
"0x0.1D796EE731D6C5BD0p0", "0x0.1DAB6358E15E79D2Fp0", "0x0.1D-
DCE85752C7127E2p0",
2717 "0x0.1E0E75EB44026A409p0", "0x0.1E400C1631FD-
B186Ap0", "0x0.1E71AAD999E821A64p0", "0x0.1EA35236F9330670Ep0",
2718 "0x0.1ED5022FCD91CB882p0", "0x0.1F06BAC594-
FA06E38p0", "0x0.1F387BF9CDA384793p0", "0x0.1FA4A5CDF08851B7Cp0",
2719 "0x0.1F9C18438CE4C902Bp0", "0x0.1FCD-
F35C11379D307p0", "0x0.1FFFD7190241E50A9p0", "0x0.2031C37BD-
F8726CFFp0",
2720 "0x0.2063B88628CD63B8Fp0",
"0x0.2095B6395E1D237DAp0", "0x0.20C7BC96FFC717FDEp0",
"0x0.20F9CBA08E48302B7p0",
2721 "0x0.212BE3578A8194D65p0", "0x0.215E03B-
D7580C2FABp0", "0x0.21902CD3D098BF15p0",
"0x0.21C25E9C1D6A9CE17p0",
2722 "0x0.21F49917DDC962553p0", "0x0.2226D-
C4893D63BCFAP0", "0x0.2259282CF1F273650p0", "0x0.228B7C-
CEEAC24D54Fp0",
2723 "0x0.22BDDA27912D13A6Ap0",

```

```

"0x0.22F0403B835D21C70p0", "0x0.2322AF0B63BFF0291p0",
"0x0.2355269997061FE83p0",
2724 "0x0.2387A6E75623866C2p0",
"0x0.23BA2FF6254F390FBp0", "0x0.23ECC1C7890398C8Ap0",
"0x0.241F5C5D05FE5DD26p0",
2725 "0x0.2451FFB82140A35A0p0", "0x0.2484ABDA600E-
F32CEp0", "0x0.24B760C547F15168p0", "0x0.24EA1E7A5EB3482F4p0",
2726 "0x0.251CE4FB2A63F3583p0", "0x0.254F-
B44931560C2A2p0", "0x0.25828C65FA1FF511Fp0", "0x0.25B56D530B9B-
C55D9p0",
2727 "0x0.25E85711ECE754F87p0",
"0x0.261B49A42564482A2p0", "0x0.264E450B3CB81B573p0",
"0x0.26814948BACC2EC3Ap0",
2728 "0x0.26B4565E27CDD257Ap0",
"0x0.26E76C4D0C2E51669p0", "0x0.271A8B16F0A2FE778p0", "0x0.274DB2B-
D5E253F101p0",
2729 "0x0.2780E341DDF29781Ap0",
"0x0.27B41CA5F98CB6B7Ep0", "0x0.27E75EB3AB9820A2p0",
"0x0.281AAA132B83210E1p0",
2730 "0x0.284DFE1F5638096CFp0",
"0x0.28815B11456B0ABADp0", "0x0.28B4C0EA83F35A4Fp0", "0x0.28E82-
FAC9CE9F225p0",
2731 "0x0.291BA7591BB6FDA67p0", "0x0.294F27F18B-
F723AB3p0", "0x0.2982B1777996543D0p0", "0x0.29B643EC70C273890p0",
2732 "0x0.29E9DF51FDEE12C26p0", "0x0.2A1D83A9AD-
D07C0A3p0", "0x0.2A5130F50D65BE58Dp0", "0x0.2A84E735A9EEB96A0p0",
2733 "0x0.2AB8A66D10F129AA3p0",
"0x0.2AEC6E9CD037B426Bp0", "0x0.2B203FC75D1F27F9p0", "0x0.2B5419E-
B90147EDB7p0",
2734 "0x0.2B87FD0DAD98FFDDFp0",
"0x0.2BBBE92E5D3E349F5p0", "0x0.2BEFDE4F2E2800A73p0", "0x0.2C23D-
C71AFBF77E86p0",
2735 "0x0.2C57E39771B2EABFBp0", "0x0.2C8B-
F3C203F5F1F3Fp0", "0x0.2CC00CF2F6C17AB8Dp0", "0x0.2CF42F2B-
DA93D2B35p0",
2736 "0x0.2D285A6E4030B4009p0",
"0x0.2D5CE8BBB8A1513E7p0", "0x0.2D90CC15D53461967p0", "0x0.2D-
C5127E2772CCAFp0",
2737 "0x0.2DF961F641589745Bp0", "0x0.2E2DBA7F-
B4E3E298p0", "0x0.2E621C1C148333651p0", "0x0.2E986CCF2E3A9C88p0",
2738 "0x0.2ECAF493E2F5611CAp0", "0x0.2EFF777277E-
F023C8p0", "0x0.2F33FD6A454D1B313p0", "0x0.2F688C7CDED22B4F1p0",
2739 "0x0.2F9D24ABD886AF562p0", "0x0.2F-
D1C5F8CB69D937p0", "0x0.300670653DDE42058p0",
"0x0.303B23F2D330AA821p0",
2740 };
2741 ap_ufixed<68,0> exp2_x_msb_2_m_1 =
exp2_x_msb_2_m_1_table[x_msb_ind_2];
2742 ap_ufixed<126,-10> f_x_msb_2_lsb =
exp2_x_msb_2_m_1 * exp2_x_msb_3_4_lsb_m_1;
2743 ap_ufixed<58,-10> f_x_msb_2_lsb_s =
f_x_msb_2_lsb;
2744 ap_ufixed<68,0> exp2_x_msb_2_3_4_lsb_m_1 =
f_x_msb_2_lsb_s + exp2_x_msb_2_m_1 + exp2_x_msb_3_4_lsb_m_1;
2745
2746 const static ap_ufixed<68,32> exp2_x_msb_1_t-
able[256] = {
2747 "0x1.0000000000p0", "0x1.306FE0A32p0",
"0x1.6A09E667Fp0", "0x1.AE89F9958p0",
2748 "0x2.0000000000p0", "0x2.60DFC1463p0",
"0x2.D413CCCFEp0", "0x3.5D13F32B6p0",
2749 "0x4.0000000000p0", "0x4.C1BF828C7p0",
"0x5.A827999Fp0", "0x6.BA27E656Bp0",
2750 "0x8.0000000000p0", "0x9.837F0518Ep0", "0xB.
504F333FAP0", "0xD.744FCCAD7p0",
2751 "0x10.0000000000p0", "0x13.06FE0A31Bp0",
"0x16.A09E667F4p0", "0x1A.E89F995ADp0",
2752 "0x20.0000000000p0", "0x26.0DFC14637p0",
"0x2D.413CCCFE7p0", "0x35.D13F32B5Ap0",
2753 "0x40.0000000000p0", "0x4C.1BF828C6Ep0",
"0x5A.827999FCFp0", "0x6B.A27E656B5p0",
2754 "0x80.0000000000p0", "0x98.37F0518DCp0",
"0xB5.04F333F9Ep0", "0xD7.44FCCAD6Ap0",
2755 "0x100.0000000000p0", "0x130.6FE0A31B7p0",
"0x16A.09E667F3Cp0", "0x1AE.89F995AD4p0",
2756 "0x200.0000000000p0", "0x260.DFC14636Ep0",
"0x3D.13CCCFE78p0", "0x35D.13F32B5A7p0",
2757 "0x400.0000000000p0", "0x4C1.BF828C6DCp0",
"0x5A8.27999FCEFP0", "0x6BA.27E656B4Fp0",
2758 "0x800.0000000000p0", "0x983.7F0518DB9p0",
"0xB50.4F333F9DEp0", "0xD74.4FCCAD69Dp0",
2759 "0x1000.0000000000p0", "0x1306.FE0A31B71p0",
"0x16A0.9E667F3BDp0", "0x1AE8.9F995AD3Bp0",
2760 "0x2000.0000000000p0", "0x260D.FC14636E3p0",
"0x2D41.3CCCFE77Ap0", "0x35D1.3F32B5A76p0",
2761 "0x4000.0000000000p0", "0x4C1B.F828C6DC5p0",
"0x5A82.7999FCEFP0", "0x6BA2.7E656B4E8p0",
2762 "0x8000.0000000000p0", "0x9837.F0518DB8Bp0",
"0xB504.F333F9DE6p0", "0xD744.FCCAD69D7p0",
2763 "0x10000.0000000000p0", "0x1306F.FE0A31B715p0",
"0x16A09.E667F3BCDp0", "0x1AE89.F995AD3ADp0",
2764 "0x20000.0000000000p0", "0x260DF.C14636E2Ap0",
"0x2D413.CCCFE7799p0", "0x35D13.F32B5A75Bp0",
2765 "0x40000.0000000000p0", "0x4C1BF.828C6DC55p0",
"0x5A827.999FCEFP0", "0x6BA27.E656B4EB5p0",
2766 "0x80000.0000000000p0", "0x9837F.0518DB8A9p0",

```

```

"0xB504F.333F9DE65p0", "0xD744F.CCAD69D6Bp0",
2767 "0x1000000.00000000p0", "0x1306FE.
0A31B7153p0", "0x16A09E.667F3BCC9p0", "0x1AE89F.995AD3AD6p0",
2768 "0x2000000.00000000p0", "0x260DFC.
14636E2A6p0", "0x2D413C.CCFE77992p0", "0x35D13F.32B5A75ACp0",
2769 "0x4000000.00000000p0", "0x4C1B-
F8.28C6DC54Bp0", "0x5A8279.99FCEf324p0", "0x6BA27E.656B4EB58p0",
2770 "0x8000000.00000000p0",
"0x9837F0.518DB8A97p0", "0xB504F3.33F9DE648p0",
"0xD744FC.CAD69D6AFp0",
2771 "0x1000000.00000000p0",
"0x1306FE0.A31B7152Ep0", "0x16A09E6.67F3BCC91p0",
"0x1AE89F9.95AD3AD5Fp0",
2772 "0x2000000.00000000p0", "0x260D-
FC1.4636E2A5Cp0", "0x2D413CC.CFE779921p0", "0x35D13F3.2B5A75ABDp0",
2773 "0x4000000.00000000p0", "0x4C1B-
F82.8C6DC54B8p0", "0x5A82799.9FCEf3242p0", "0x6BA27E6.56B4EB57Ap0",
2774 "0x8000000.00000000p0",
"0x9837F05.18DB8A96Fp0", "0xB504F33.3F9DE6484p0",
"0xD744FCC.AD69D6AF4p0",
2775 "0x1000000.00000000p0", "0x1306FE0A.
31B7152DFp0", "0x16A09E66.7F3BCC909p0", "0x1AE89F99.5AD3AD5E8p0",
2776 "0x2000000.00000000p0", "0x260D-
FC14.636E2A5BDp0", "0x2D413CCC.FE7799211p0",
"0x35D13F32.B5A75ABD1p0",
2777 "0x4000000.00000000p0", "0x4C1B-
F828.C6DC54B7Ap0", "0x5A827999.FCFE32423p0",
"0x6BA27E65.6B4EB57A2p0",
2778 "0x8000000.00000000p0",
"0x9837F051.8DB8A96F4p0", "0xB504F333.F9DE64846p0",
"0xD744FCCA.D69D6AF44p0",
2779 "0x0.000000010p0", "0x0.000000013p0",
"0x0.000000017p0", "0x0.00000001Bp0",
2780 "0x0.000000020p0", "0x0.000000026p0",
"0x0.00000002Dp0", "0x0.000000036p0",
2781 "0x0.000000040p0", "0x0.00000004Cp0",
"0x0.00000005Bp0", "0x0.00000006Cp0",
2782 "0x0.000000080p0", "0x0.000000098p0",
"0x0.0000000B5p0", "0x0.0000000D7p0",
2783 "0x0.000000100p0", "0x0.000000130p0",
"0x0.00000016Ap0", "0x0.0000001AFp0",
2784 "0x0.000000200p0", "0x0.000000261p0",
"0x0.0000002D4p0", "0x0.00000035Dp0",
2785 "0x0.000000400p0", "0x0.0000004C2p0",
"0x0.0000005A8p0", "0x0.0000006BAP0",
2786 "0x0.000000800p0", "0x0.000000933p0",
"0x0.000000B50p0", "0x0.000000D74p0",
2787 "0x0.000001000p0", "0x0.000001307p0",
"0x0.0000016A1p0", "0x0.000001AE9p0",
2788 "0x0.000002000p0", "0x0.00000260Ep0",
"0x0.000002D41p0", "0x0.0000035D1p0",
2789 "0x0.000004000p0", "0x0.000004C1Cp0",
"0x0.000005A82p0", "0x0.000006BA2p0",
2790 "0x0.000008000p0", "0x0.000009838p0",
"0x0.00000B505p0", "0x0.00000D745p0",
2791 "0x0.000010000p0", "0x0.000013070p0",
"0x0.000016A0Ap0", "0x0.00001AE8Ap0",
2792 "0x0.000020000p0", "0x0.0000260E0p0",
"0x0.00002D414p0", "0x0.000035D14p0",
2793 "0x0.000040000p0", "0x0.00004C1C0p0",
"0x0.00005A828p0", "0x0.00006BA28p0",
2794 "0x0.000080000p0", "0x0.00009837Fp0",
"0x0.0000B504Fp0", "0x0.0000D7450p0",
2795 "0x0.000100000p0", "0x0.0001306FEp0",
"0x0.00016A09Ep0", "0x0.0001AE8A0p0",
2796 "0x0.000200000p0", "0x0.000260DFCp0",
"0x0.0002D413Dp0", "0x0.00035D13Fp0",
2797 "0x0.000400000p0", "0x0.0004C1BF8p0",
"0x0.0005A827Ap0", "0x0.0006BA27Ep0",
2798 "0x0.000800000p0", "0x0.0009837F0p0",
"0x0.000B504F3p0", "0x0.000D744FDp0",
2799 "0x0.001000000p0", "0x0.001306FE1p0",
"0x0.0016A09E6p0", "0x0.001AE89FAP0",
2800 "0x0.002000000p0", "0x0.00260DFC1p0",
"0x0.002D413CDp0", "0x0.0035D13F3p0",
2801 "0x0.004000000p0", "0x0.004C1BF83p0",
"0x0.005A8279Ap0", "0x0.006BA27E6p0",
2802 "0x0.008000000p0", "0x0.009837F05p0",
"0x0.00B504F33p0", "0x0.00D744FCDp0",
2803 "0x0.010000000p0", "0x0.01306FE0Ap0",
"0x0.016A09E66p0", "0x0.01AE89F99p0",
2804 "0x0.020000000p0", "0x0.0260DFC14p0",
"0x0.02D413CCDp0", "0x0.035D13F33p0",
2805 "0x0.040000000p0", "0x0.04C1BF829p0",
"0x0.05A82799Ap0", "0x0.06BA27E65p0",
2806 "0x0.080000000p0", "0x0.09837F052p0",
"0x0.0B504F334p0", "0x0.0D744FCCBp0",
2807 "0x0.100000000p0", "0x0.1306FE0A3p0",
"0x0.16A09E668p0", "0x0.1AE89F996p0",
2808 "0x0.200000000p0", "0x0.260DFC146p0",
"0x0.2D413CCD0p0", "0x0.35D13F32Bp0",
2809 "0x0.400000000p0", "0x0.4C1BF828Cp0",
"0x0.5A82799A0p0", "0x0.6BA27E657p0",
2810 "0x0.800000000p0", "0x0.9837F0519p0",
"0x0.B504F3340p0", "0x0.D744FCCADp0",
2811 };
2812 ap_ufixed<68,32> exp2_x_msb_1 = exp2_x_msb_1_t-
able[x_msb_ind_1];
2813 ap_ufixed<136,32> y_lo = exp2_x_msb_1 *
exp2_x_msb_2_3_4_lsb_m_1;
2814
2815 ap_ufixed<68,32> y_lo_s = y_lo;
2816 ap_ufixed<68,32> y_l = y_lo_s + exp2_x_msb_1;
2817 y = y_l;
2818
2819 if(I<33) {
2820     bool overf_2 = 0;
2821     for(int j=63; j>=I+31; j--) {
2822         #pragma HLS unroll
2823         if(y[j]) {
2824             overf_2 = 1;
2825         }
2826     }
2827     if(overf_2) {
2828         for(int j=63; j>=I+31; j--) {
2829             #pragma HLS unroll
2830             y[j] = 0;
2831         }
2832         for(int j=I+30; j>=0; j--) {
2833             #pragma HLS unroll
2834             y[j] = 1;
2835         }
2836     }
2837     r = y;
2838 }
2839 return r;
2840
2841 template<int W, int I>
2842 ap_ufixed<W,I> exp2(ap_ufixed<W,I> x) {
2843     ap_fixed<W+1,I+1> xf = x;
2844     return exp2(xf);
2845 }
2846
2847 template<int I>
2848 ap_int<I> exp2(ap_int<I> x) {
2849     ap_fixed<I,I> xf = x;
2850     return exp2(xf);
2851 }
2852
2853 template<int I>
2854 ap_uint<I> exp2(ap_uint<I> x) {
2855     ap_fixed<I+1,I+1> xf = x;
2856     return exp2(xf);
2857 }
2858
2859 template<int W_, int I_>
2860 ap_fixed<W_,I_> exp10(ap_fixed<W_,I_> x) {
2861
2862     int F_ = W_ - I_;
2863     if (I_>33) return 0;
2864     else if (F_>32) return 0;
2865
2866     int I_s_;
2867     int M_ = (I_-1>F_) ? (I_-1) : F_;
2868     if (M_==1) I_s_ = 0;
2869     else if (M_<4) I_s_ = 1;
2870     else if (M_<7) I_s_ = 2;
2871     else if (M_<14) I_s_ = 3;
2872     else if (M_<27) I_s_ = 4;
2873     else I_s_ = 5;
2874
2875     ap_fixed<W_,I_> r;
2876     if(I_s_==0) { //I_s_ = 0, M_ = 1, I_ <= 2, F_ <= 1, W
2877         <= 3
2878         ap_fixed<3,2> x_l = x;
2879         ap_ufixed<2,1> y = 0;
2880         if(x_l[2]) {
2881             if(x_l[1]&&x_l[0]) {
2882                 y = ap_ufixed<2,1>{"0x0.8p0"};
2883             }
2884         } else {
2885             if(x_l[1]||x_l[0]) {
2886                 y = ap_ufixed<2,1>{"0x1.8p0"};
2887             } else {
2888                 y = ap_ufixed<2,1>{"0x1.0p0"};
2889             }
2890         }
2891     }
2892     if(I<2) {
2893         // bool overf = 0;
2894         if(y[1]) {
2895             y[1] = 0;
2896             y[0] = 1;
2897         }
2898     }
2899     r = y;
2900 } else if (I_s_==1) { // I_s_ = 1, M_ = 2..3, I_ <= 4,
2901     F_ <= 3, W <= 7
2902     ap_fixed<7,4> x_l = x;
2903     ap_ufixed<6,3> y = 0;
2904     if((x_l[6]!=x_l[5])||(x_l[5]!=x_l[4])||
(x_l[4]!=x_l[3])) {
2905         if(!x_l[6]) {
2906             y = ap_ufixed<6,3>{"0x7.Ep0"};
2907         }
2908     } else {
2909         ap_uint<4> x_ind;
2910         x_ind[3] = x_l[6];

```

```

2911         x_ind(2,0) = x_l(2,0);
2912         const static ap_ufixed<6,3>
exp_x_ind_table[16] = {
2913             "0x1.000p0", "0x1.600p0",
"0x1.C00p0", "0x2.600p0", "0x3.200p0", "0x4.400p0", "0x5.A00p0",
"0x7.800p0",
2914             "0x0.200p0", "0x0.200p0",
"0x0.200p0", "0x0.400p0", "0x0.600p0", "0x0.600p0", "0x0.800p0",
"0x0.C00p0"
2915         };
2916         y = exp_x_ind_table[x_ind];
2917     }
2918     if(I<4){
2919         bool overf = 0;
2920         for(int j = 5; j>=I+2; j--) {
2921             #pragma HLS unroll
2922             if(y[j]) {
2923                 overf = 1;
2924             }
2925         }
2926         if(overf) {
2927             for(int j = 5; j>=I+2; j--) {
2928                 #pragma HLS unroll
2929                 y[j] = 0;
2930             }
2931             for(int j = I+1; j>=0; j--) {
2932                 #pragma HLS unroll
2933                 y[j] = 1;
2934             }
2935         }
2936     }
2937     r = y;
2938 } else if (I_s==2) { // I_s = 2, M_ = 4..6, I_ <= 7,
F_ <= 6, W <= 13
2939     ap_fixed<13,7> x_l = x;
2940     ap_ufixed<12,6> y = 0;
2941     ap_fixed<2,2> x_l_int;
2942     x_l_int(1,0) = x_l(7,6);
2943     ap_ufixed<6,0> x_l_ffrac;
2944     x_l_ffrac(5,0) = x_l(5,0);
2945     bool overf = 0;
2946     for(int j = 11; j>=7; j--) {
2947         #pragma HLS unroll
2948         if(x_l[12]!=x_l[j]) {
2949             overf = 1;
2950         }
2951     }
2952     if(overf||
((x_l_int==1)&&(x_l_ffrac>ap_ufixed<5,0>("0x.D0p0")))) {
2953         if(!x_l[12]) {
2954             y =
ap_ufixed<12,6>("0x3F.FCp0");
2955         }
2956     } else {
2957         ap_uint<4> x_msb_ind;
2958         x_msb_ind(3,0) = x_l(7,4);
2959         ap_uint<4> x_lsb_ind;
2960         x_lsb_ind(3,0) = x_l(3,0);
2961         const static ap_ufixed<14,6>
exp_x_msb_table[16] = {
2962             "0x1.00p0", "0x1.C7p0",
"0x3.2Ap0", "0x5.A0p0", "0xA.00p0", "0x11.C8p0", "0x1F.9Fp0",
"0x3B.3Cp0",
2963             "0x0.03p0", "0x0.05p0",
"0x0.08p0", "0x0.0Ep0", "0x0.1Ap0", "0x0.2Ep0", "0x0.51p0",
"0x0.90p0"
2964         };
2965         const static ap_ufixed<14,0>
exp_x_lsb_m1_table[16] = {
2966             "0x0.0p0", "0x0.0960p0",
"0x0.1318p0", "0x0.1D2Cp0", "0x0.27A0p0", "0x0.3274p0", "0x0.3D-
B0p0", "0x0.4950p0",
2967             "0x0.5560p0", "0x0.61E4p0",
"0x0.6ED8p0", "0x0.7C4Cp0", "0x0.8A38p0", "0x0.98A8p0",
"0x0.A7A4p0", "0x0.B728p0"
2968         };
2969         ap_ufixed<14,6> exp_x_msb =
exp_x_msb_table[x_msb_ind];
2970         ap_ufixed<14,0> exp_x_lsb_m1 =
exp_x_lsb_m1_table[x_lsb_ind];
2971         ap_ufixed<28,6> y_lo = exp_x_msb *
exp_x_lsb_m1;
2972         ap_ufixed<14,6> y_lo_s = y_lo;
2973         ap_ufixed<14,6> y_l = y_lo_s +
exp_x_msb;
2974         y = y_l;
2975     }
2976     if(I<7) {
2977         bool overf = 0;
2978         for(int j = 11; j >= I+5; j--) {
2979             #pragma HLS unroll
2980             if(y[j]) {
2981                 overf = 1;
2982             }
2983         }
2984         if(overf) {
2985             for(int j = 11; j>= I+5; j--)
2986                 #pragma HLS unroll
2987             y[j] = 0;
2988         }
2989         for(int j = I+4; j>=0; j--) {
2990             #pragma HLS unroll
2991             y[j] = 1;
2992         }
2993     }
2994     } else if (I_s==3) { // I_s = 3, M_ = 7..13, I_ <=
14, F_ <= 13, W <= 27
2995         r = y;
2996     } else if (I_s==3) { // I_s = 3, M_ = 7..13, I_ <=
14, F_ <= 13, W <= 27
2997         ap_fixed<27,14> x_l = x;
2998         ap_ufixed<26,13> y = 0;
2999         ap_fixed<3,3> x_l_int;
3000         x_l_int(2,0) = x_l(15,13);
3001         ap_ufixed<13,0> x_l_ffrac;
3002         x_l_ffrac(12,0) = x_l(12,0);
3003         bool overf = 0;
3004         for(int j = 25; j>=15; j--) {
3005             #pragma HLS unroll
3006             if(x_l[26]!=x_l[j]) {
3007                 overf = 1;
3008             }
3009         }
3010         if(overf||
((x_l_int==3)&&(x_l_ffrac>ap_ufixed<12,0>("0x3.E9D0p0")))) {
3011             if(!x_l[26]) {
3012                 y =
ap_ufixed<26,13>("0x1FFF.FFF8p0");
3013             }
3014         } else {
3015             ap_uint<6> x_msb_ind_1;
3016             x_msb_ind_1[5] = x_l[26];
3017             x_msb_ind_1(4,0) = x_l(14,10);
3018             ap_uint<5> x_msb_ind_2;
3019             x_msb_ind_2(4,0) = x_l(9,5);
3020             ap_uint<5> x_lsb_ind;
3021             x_lsb_ind(4,0) = x_l(4,0);
3022             const static ap_ufixed<24,-6>
exp_lsb_ind_m1_table[32] = {
3023                 "0x0p0", "0x0.00126C5Cp0",
"0x0.0024DA0Cp0", "0x0.0037490Cp0", "0x0.0049B964p0",
"0x0.005C2B0Cp0", "0x0.006E9E08p0", "0x0.0081125Cp0",
3024                 "0x0.00938800p0", "0x0.00A5FE-
F8p0", "0x0.00B87748p0", "0x0.00CAF0E8p0", "0x0.00DD6BE0p0",
"0x0.00EFE82Cp0", "0x0.010265C8p0", "0x0.0114E48Cp0",
3025                 "0x0.01276508p0",
"0x0.0139E6A4p0", "0x0.014C6998p0", "0x0.015EEDDCp0",
"0x0.0171737Cp0", "0x0.0183FA6Cp0", "0x0.019682B4p0",
"0x0.01A90C50p0",
3026                 "0x0.01BB9744p0",
"0x0.01CE238Cp0", "0x0.01E0B128p0", "0x0.01F3401Cp0",
"0x0.0205D064p0", "0x0.02186204p0", "0x0.022AF4F8p0",
"0x0.023D8944p0"
3027             };
3028             ap_ufixed<24,-6> exp_lsb_ind_m1 =
exp_lsb_ind_m1_table[x_lsb_ind];
3029             const static ap_ufixed<29,-1>
exp_msb_ind_2_m1_table[32] = {
3030                 "0x0p0", "0x0.02501EE8p0",
"0x0.04A5975Cp0", "0x0.070075C0p0", "0x0.0960C68Cp0", "0x0.0B-
C69660p0", "0x0.0E31F1F0p0", "0x0.10A2E60Cp0",
3031                 "0x0.13197FA8p0", "0x0.1595CB-
D4p0", "0x0.1817D7BCp0", "0x0.1A9FB0ACp0", "0x0.1D2D6410p0",
"0x0.1FC0FF74p0", "0x0.225A9080p0", "0x0.24FA24F8p0",
3032                 "0x0.279FCACCp0",
"0x0.2A4B8FFCp0", "0x0.2CFD82BCp0", "0x0.2FB5B150p0",
"0x0.32742A20p0", "0x0.3538FBC0p0", "0x0.380434D8p0",
"0x0.3AD5E43Cp0",
3033                 "0x0.3DAE18DCp0",
"0x0.408CE1CCp0", "0x0.43724E48p0", "0x0.46560A8p0",
"0x0.49514F68p0", "0x0.4C4B0330p0", "0x0.4F4B98C4p0",
"0x0.5253200Cp0"
3034             };
3035             ap_ufixed<29,-1> exp_msb_ind_2_m1 =
exp_msb_ind_2_m1_table[x_msb_ind_2];
3036             ap_ufixed<53,-7> f_x_msb_2_lsb =
exp_lsb_ind_m1 * exp_msb_ind_2_m1;
3037             ap_ufixed<27,-1> exp_msb_2_lsb_m1 =
f_x_msb_2_lsb + exp_msb_ind_2_m1 + exp_lsb_ind_m1;
3038             const static ap_ufixed<28,13>
exp_msb_ind_1_table[64] = {
3039                 "0x1.0000p0", "0x1.5562p0",
"0x1.C73Ep0", "0x2.5F12p0", "0x3.298Cp0", "0x4.378Cp0",
"0x5.9F98p0", "0x7.7FBAp0",
3040                 "0xA.0000p0", "0xD.55D0p0",
"0x11.C86p0", "0x17.B6B8p0", "0x1F.9F6Ep0", "0x2A.2B6Ep0",
"0x3B.3BF0p0", "0x4A.FD4Ap0",
3041                 "0x64.0000p0", "0x85.5A26p0",
"0xB1.D3F4p0", "0xED.232Ap0", "0x13C.3A4Ep0", "0x1A5.B24Ep0",
"0x232.5762p0", "0x2ED.E4EAp0",
3042                 "0x3E8.0000p0",
"0x535.857Cp0", "0x6F2.4788p0", "0x943.5FACp0", "0xC5A.4714p0",
"0x1078.F70Cp0", "0x15F7.69CAp0", "0x1D4A.F12Ep0",
3043                 "0x0.0000p0", "0x0.0008p0",
"0x0.000Cp0", "0x0.0010p0", "0x0.0014p0", "0x0.001Cp0",
"0x0.0024p0", "0x0.0032p0",
3044                 "0x0.0042p0", "0x0.0058p0",
"0x0.0074p0", "0x0.009Cp0", "0x0.00D0p0", "0x0.0114p0",
"0x0.0170p0", "0x0.01ECp0",

```



```

3045          "0x0.0290p0", "0x0.036Ap0",
"0x0.048Ep0", "0x0.0612p0", "0x0.0818p0", "0x0.0ACp0",
"0x0.0E66p0", "0x0.1332p0",
3046          "0x0.199Ap0", "0x0.2224p0",
"0x0.2D86p0", "0x0.3CB6p0", "0x0.50F4p0", "0x0.6BF4p0",
"0x0.8FF6p0", "0x0.BFFAp0"
3047      };
3048      ap_ufixed<28,13> exp_msb_ind_1 =
exp_msb_ind_1_table[x_msb_ind_1];

3049      ap_ufixed<28,13> f_x_msb_1_msb_2_lsb_l
= exp_msb_ind_1 * exp_msb_2_lsb_m_1;
3050      y = f_x_msb_1_msb_2_lsb_l +
exp_msb_ind_1;
3051      }
3052      if(I_<14) {
3053          bool overf = 0;
3054          for(int j = 25; j>= I_+12; j--) {
3055              #pragma HLS unroll
3056              if(y[j]) {
3057                  overf = 1;
3058              }
3059              if(overf) {
3060                  for(int j = 25; j>=I_+12; j--)
{
3062                      #pragma HLS unroll
3063                      y[j] = 0;
3064                  }
3065                  for(int j = I_+11; j>=0; j--)
{
3066                      #pragma HLS unroll
3067                      y[j] = 1;
3068                  }
3069              }
3070          }
3071          r = y;
3072      } else if (I_s_==4) { // I_s_ = 4, M_ = 14..26, I_ <=
27, F_ <= 26, W <= 53
3073          ap_fixed<53,27> x_l = x;
3074          ap_ufixed<52,26> y = 0;
3075          ap_fixed<4,4> x_l_int;
3076          x_l_int(3,0) = x_l(29,26);
3077          ap_ufixed<26,0> x_l_ffrac;
3078          x_l_ffrac(25,0) = x_l(25,0);
3079          bool overf = 0;
3080          for(int j = 51; j>= 29; j--) {
3081              #pragma HLS unroll
3082              if(x_l[52]!=x_l[j]) {
3083                  overf = 1;
3084              }
3085          }
3086          if(overf)
{
3087              if(!x_l[52]) {
3088                  y =
ap_ufixed<52, 26>("0x3FFFFFF.FFFFFFFp0");
3089              }
3090              } else {
3091                  ap_uint<8> x_msb_ind_1;
3092                  x_msb_ind_1[7] = x_l[52];
3093                  x_msb_ind_1(6,0) = x_l(28,22);
3094                  ap_uint<8> x_msb_ind_2;
3095                  x_msb_ind_2(7,0) = x_l(21,14);
3096                  ap_uint<7> x_msb_ind_3;
3097                  x_msb_ind_3(6,0) = x_l(13,7);
3098                  ap_uint<7> x_lsb_ind;
3099                  x_lsb_ind(6,0) = x_l(6,0);
3100                  const static ap_ufixed<41,-17>
exp_lsb_ind_m_1_table[128] = {
3101                      "0x0.0p0",
"0x0.00000935D8E080p0", "0x0.0000126BB1C650p0", "0x0.000001BA18-
AB16Cp0", "0x0.000024D763A1D4p0", "0x0.00002E0D3C978Cp0",
"0x0.000003743159290p0", "0x0.00004078EE92E0p0",
3102                      "0x0.0000049EAC79880p0",
"0x0.0000052E4A0A368p0", "0x0.000005C1A79B3A4p0",
"0x0.00000655052C928p0", "0x0.000006E862BE3FCp0", "0x0.0000077B-
C05041Cp0", "0x0.0000080F1DE298Cp0", "0x0.000008A27B75448p0",
3103                      "0x0.00000935D908450p0",
"0x0.000009C369B9A4p0", "0x0.00000A5C942F448p0",
"0x0.00000AEFF1C3438p0", "0x0.00000B834F57978p0",
"0x0.00000C16ACEC400p0", "0x0.00000CA0A813D8p0",
"0x0.00000D3D6816900p0",
3104                      "0x0.00000DD0C5AC370p0",
"0x0.00000E642342334p0", "0x0.00000EF780D8840p0",
"0x0.00000F8ADE6F29Cp0", "0x0.0000101E3C06244p0",
"0x0.000010B1999D738p0", "0x0.00001144F73517Cp0",
"0x0.000011D854CD10Cp0",
3105                      "0x0.0000126BB2655E8p0",
"0x0.000012FF0FFE010p0", "0x0.000013926D96F88p0",
"0x0.00001425CB30450p0", "0x0.0000148928C9E60p0",
"0x0.0000154C8663DC0p0", "0x0.0000150FE3FE26Cp0",
"0x0.000016734198C68p0",
3106                      "0x0.000017069F33BB0p0",
"0x0.00001799FCCF044p0", "0x0.0000182D5A6AA24p0",
"0x0.000018C0B806954p0", "0x0.0000195415A2DD0p0",
"0x0.000019E7733F79Cp0", "0x0.00001A7AD0DC6B4p0",
"0x0.00001B0E2E79B18p0",
3107                      "0x0.00001BA18C174C8p0",
"0x0.00001C34E9B53C8p0", "0x0.00001CC84753814p0",
"0x0.00001D5BA4F21B0p0", "0x0.00001DEF0291094p0",
"0x0.00001E8260304C8p0", "0x0.00001F5BDCF4Cp0", "0x0.00001-
FA91B6FD18p0",
3108                      "0x0.0000203C7910134p0",
"0x0.000020CFD6B0AA0p0", "0x0.000021633451954p0",
"0x0.000021F691F2D58p0", "0x0.00002289EF946ACp0",
"0x0.0000231D4D36548p0", "0x0.000023B0AAD8934p0",
"0x0.00002444087B270p0",
3109                      "0x0.000024D7661E0F4p0",
"0x0.0000256AC3C14C8p0", "0x0.000025FE2164DE8p0",
"0x0.000026917F08C58p0", "0x0.00002724DCAD014p0",
"0x0.000027B83A5191Cp0", "0x0.0000284B97F6774p0", "0x0.000028DE-
F59BB14p0",
3110                      "0x0.000029725341408p0",
"0x0.00002A05B0E7244p0", "0x0.00002A990E8D5D0p0",
"0x0.00002B2C6C3E3A8p0", "0x0.00002B8FC9DACD0p0",
"0x0.00002C532782040p0", "0x0.00002CE68529900p0",
"0x0.00002D79E2D1710p0",
3111                      "0x0.00002E0D4079A6Cp0",
"0x0.00002EA09E22314p0", "0x0.00002F3FBCB108p0",
"0x0.00002FC7597444Cp0", "0x0.0000305AB71DCDCp0",
"0x0.000030EE14C7AB8p0", "0x0.000031817271DE4p0",
"0x0.00003214D01C65Cp0",
3112                      "0x0.000032A82DC7420p0",
"0x0.0000333B8B72734p0", "0x0.000033CEE91DF94p0",
"0x0.000034624C9D40p0", "0x0.000034F5A47603Cp0",
"0x0.000035890222884p0", "0x0.0000361C5CF618p0", "0x0.000036AFB-
D7C8FCp0",
3113                      "0x0.000037431B2A12Cp0",
"0x0.000037D678D7EA8p0", "0x0.00003869D686170p0", "0x0.000038F-
D343988p0", "0x0.0000399091E36ECp0", "0x0.00003A23EF929A0p0",
"0x0.00003AB74D421A0p0", "0x0.00003B4AAAF1EECp0",
3114                      "0x0.00003BDE08A2188p0",
"0x0.00003C71665296Cp0", "0x0.00003D04C4036A0p0",
"0x0.00003D9821B4924p0", "0x0.00003E2B7F660F4p0", "0x0.00003EBED-
D17E10p0", "0x0.00003F523ACA078p0", "0x0.00003FE5987C830p0",
3115                      "0x0.00004078F62F534p0",
"0x0.0000410C53E2788p0", "0x0.0000419FB195F24p0",
"0x0.000042330F49C10p0", "0x0.000042C66CFDE4Cp0", "0x0.00004359C-
AB25D0p0", "0x0.000043ED28672A4p0", "0x0.00004480861C4C8p0",
3116                      "0x0.00004513E3D1C34p0",
"0x0.000045A741878F0p0", "0x0.0000463A9F3DAFCp0", "0x0.000046CD-
FCF4250p0", "0x0.000047615AAAEF4p0", "0x0.000047F4B8620E8p0",
"0x0.000048881619824p0", "0x0.0000491B73D14B0p0",
3117                      };
3118          ap_ufixed<41,-17> exp_lsb_ind_m_1 =
exp_lsb_ind_m_1_table[x_lsb_ind];
3119          const static ap_ufixed<48,-10>
exp_msb_ind_3_m_1_table[128] = {
3120              "0x0.0p0",
"0x0.000049AED18968Cp0", "0x0.0000935DB847FC4p0", "0x0.0000D-
D0CB43BC10p0", "0x0.000126B8C564BCCp0", "0x0.0001706AEB2C2F58p0",
"0x0.0001BA1A275671Cp0", "0x0.000203C9781F374p0",
3121              "0x0.00024D78DE1D4C8p0",
"0x0.00029728590B74p0", "0x0.0002E0D7E9B97D8p0",
"0x0.00032A878F57A5Cp0", "0x0.000374374A2B360p0", "0x0.0003B-
DE71A34344p0", "0x0.00040796F72A68p0", "0x0.00045146F9E6930p0",
3122              "0x0.00049AF70990000p0",
"0x0.0004E4A72E6EF34p0", "0x0.0005E576883734p0",
"0x0.00057807B7CD85Cp0", "0x0.0005C1B81C4D314p0",
"0x0.00060B6896027B4p0", "0x0.0006551924ED6A8p0",
"0x0.00069EC9C90E04Cp0",
3123              "0x0.0006E87A8264500p0",
"0x0.0007322B50F052Cp0", "0x0.00077BDC34B2130p0",
"0x0.0007C58D2DA9968p0", "0x0.000807E3BDD6E3Cp0", "0x0.000858E-
F5F3A008p0", "0x0.0008A2A097D2F34p0", "0x0.0008EC51E5A1C1Cp0",
3124              "0x0.0009360348A6724p0",
"0x0.00097FB4C0E10B0p0", "0x0.0009C9664E51920p0",
"0x0.000A1317F0F80D4p0", "0x0.000A5CC9A8D4830p0",
"0x0.000AA67B75E6F94p0", "0x0.000AF02D582F764p0", "0x0.000B39DF4-
FADFFCp0",
3125              "0x0.000B83915C629C4p0",
"0x0.000BCD437E4D51Cp0", "0x0.000C16F5B56E268p0",
"0x0.000C60A801C5200p0", "0x0.000CAA5A6352450p0",
"0x0.000CF40CDA159B8p0", "0x0.000D3DBF660F294p0",
"0x0.000D8772073EF4Cp0",
3126              "0x0.000DD124BDA5040p0",
"0x0.000E1AD789415CCp0", "0x0.000E648A6A1405Cp0",
"0x0.000EAE3D601D048p0", "0x0.000EF7F06B5C5F8p0", "0x0.000F41A38B-
D21CCp0", "0x0.000F8B56C17E424p0", "0x0.000F5D0A0C60D64p0",
3127              "0x0.00101EBD6C79DECp0",
"0x0.00106870E1C9620p0", "0x0.0010B2246C4F660p0", "0x0.0010FB-
D80CB0FC0p0", "0x0.0011458BC0FF880p0", "0x0.00118F3F8B28B34p0",
"0x0.0011D8F36A88F74p0", "0x0.00122A75F1FD8p0",
3128              "0x0.00126C5B8ED630p0",
"0x0.0012B60F87F1974p0", "0x0.0012FFC3BC2C7D0p0",
"0x0.00134978059E1A8p0", "0x0.0013932C644675Cp0", "0x0.0013D-
CE0D82594Cp0", "0x0.00142695613B7E0p0", "0x0.00147049FF88374p0",
3129              "0x0.0014B9FEB30BC70p0",
"0x0.001503B73BC632Cp0", "0x0.00154D6859B7810p0", "0x0.0015971D4CD-
FB00p0", "0x0.0015E0D2553EDD8p0", "0x0.00162A8772D4F7Cp0",
"0x0.0016743CA5A20D0p0", "0x0.00168B71EDA6230p0",
3130              "0x0.001707A74AE1404p0",
"0x0.0017515CBD536ACp0", "0x0.00179B1244FCA88p0", "0x0.0017E4C7E1D-
CFF8p0", "0x0.00182E7D93F4760p0", "0x0.001878335B34124p0",
"0x0.0018C1E937C8DA4p0", "0x0.00190B9F2985D40p0",
3131              "0x0.00195555307A05Cp0",
"0x0.00199F0B4CA5758p0", "0x0.0019E8C17E08294p0",
"0x0.001A3277CA2278p0", "0x0.001A7C2E2073760p0",

```

```

"0x0.001AC5E4917C1ACp0", "0x0.001B0F9B17BC1C4p0",
"0x0.001B5951B333808p0",
3132 "0x0.001BA30863E24D8p0",
"0x0.001BECBF29C8894p0", "0x0.001C367604E63A4p0",
"0x0.001C802CF53B664p0", "0x0.001C9C93FAC8134p0",
"0x0.001D139B158C47Cp0", "0x0.001D5D52458809Cp0", "0x0.001DA7098AB-
B5F4p0",
3133 "0x0.001DF0C0E5264E4p0",
"0x0.001E3A7854C8DD0p0", "0x0.001E842FD9A311Cp0",
"0x0.001ECDE773B4F24p0", "0x0.001F179F22FE850p0",
"0x0.001F6156E77FCFcp0", "0x0.001FAB0EC138D90p0",
"0x0.001FF4C6B029A68p0",
3134 "0x0.00203E7EB4523E8p0",
"0x0.00208836CB2A70p0", "0x0.0020D1EEFC4AE68p0",
"0x0.00211BA7401B028p0", "0x0.0021655F9923018p0",
"0x0.0021AF180762E98p0", "0x0.0021F8008ADAC0Cp0",
"0x0.00224289238A8D4p0",
3135 "0x0.00228C41D172550p0",
"0x0.0022D5FA94921E4p0", "0x0.00231FB36CE9EF0p0",
"0x0.0023696C5A79CD8p0", "0x0.00238325D41BFCp0",
"0x0.0023FCDE7541CC0p0", "0x0.00244697A279F80p0",
"0x0.00249050E4EA4A4p0",
3136 };
3137 ap_ufixed<48,-10> exp_msb_ind_3_m_1 =
exp_msb_ind_3_m_1_table[x_msb_ind_3];
3138 ap_ufixed<89,-27> f_x_msb_ind_3_lsb =
exp_lsb_ind_m_1 * exp_msb_ind_3_m_1;
3139 ap_ufixed<46,-10>
exp_msb_ind_3_lsb_ind_m_1 = f_x_msb_ind_3_lsb + exp_lsb_ind_m_1 +
exp_msb_ind_3_m_1;
3140 const static ap_ufixed<54,-2>
exp_msb_ind_2_m_1_table[256] = {
3141 "0x0.0p0",
"0x0.0024DA0A3C92C9p0", "0x0.0049B96285BC0Ap0", "0x0.006E9E099E-
FA37p0", "0x0.009388004BE7E5p0", "0x0.00B87747503BD4p0", "0x0.00D-
D6BDF6FC8EEp0", "0x0.010265C96E7E4Fp0",
3142 "0x0.01276506106748p0",
"0x0.014C699619A860p0", "0x0.01717374A8E5E9p0",
"0x0.019682B3737048p0", "0x0.01BB97424CCD67p0",
"0x0.01E0B1279F3E51p0", "0x0.0205D0642F77E5p0",
"0x0.022AF4F8C24B54p0",
3143 "0x0.02501EE61CA626p0",
"0x0.02754E2D03923Bp0", "0x0.029A82CE3C35CEp0", "0x0.02BFBCCA8B-
D37Fp0", "0x0.02E4FC22B7CA50p0", "0x0.030A40D78595AFp0",
"0x0.032F8AE9BACD76p0", "0x0.0354DA5A1D25F2p0",
3144 "0x0.037A2F29726F55p0",
"0x0.039F895880988Ap0", "0x0.03C4E8E80DA99Bp0", "0x0.03EA4DD8D-
FC955p0", "0x0.040F882BBD3A78p0", "0x0.043527E16C5C1p0",
"0x0.045A9CFAB3AAB8p0", "0x0.0480177859BE22p0",
3145 "0x0.04A5975B254B8Bp0",
"0x0.04CB1CA3DD2493p0", "0x0.04F0A753483778p0",
"0x0.0516376A2D8F1Ap0", "0x0.053BCC9E5452FFp0",
"0x0.056167D183C75Bp0", "0x0.05870823834D0Ep0",
"0x0.05ACADE01A61AEp0",
3146 "0x0.05D25908109F88p0",
"0x0.05F8099C2DBDA3p0", "0x0.061DBF9D398FC8p0", "0x0.06437B0E-
FC683p0", "0x0.066938E93D2F28p0", "0x0.068F0235C533D7p0",
"0x0.06B4CDF25C5B83p0", "0x0.06DA9F1FCB09EFp0",
3147 "0x0.070075BD98FBBp0",
"0x0.072651D0511A61p0", "0x0.074C3354F9D43Dp0", "0x0.07721A4D9C-
C491p0", "0x0.079806B802DF87p0", "0x0.07BDF89DF53637p0",
"0x0.07E3EFF73CF6ACp0", "0x0.0809ECC7A36BE3p0",
3148 "0x0.082FEF0FF1FD07p0",
"0x0.0855F6D0F23180p0", "0x0.087C040B6DA8D8p0", "0x0.08A216C0E22D-
Fp0", "0x0.08C82EEFFD7BA0p0", "0x0.08EE4C9BA5AC35p0",
"0x0.09146FC3F0CACp0", "0x0.093A896A90AA9p0",
3149 "0x0.0960C68D98BC2Cp0",
"0x0.0986FA308A4CD4p0", "0x0.09AD3353484744p0",
"0x0.09D371F69D5348p0", "0x0.09F9B61B5435D7p0",
"0x0.0A1FFFC237D119p0", "0x0.0A644EEC13246Bp0",
"0x0.0A6CA399B4C61p0",
3150 "0x0.0A92FDCBDD82CEp0",
"0x0.0AB95D83631EC5p0", "0x0.0ADFC2C10D94A0p0",
"0x0.0B062D85A87601p0", "0x0.0B2C9DD1FF71D8p0",
"0x0.0B5313A6DE5467p0", "0x0.0B798F05110749p0", "0x0.0BA00FED-
63916Fp0",
3151 "0x0.0BC69660A2172Dp0",
"0x0.0BED225F98DA36p0", "0x0.0C13B3EB1439A8p0",
"0x0.0C3A4B03E0B20Ap0", "0x0.0C60E7AACADD54p0",
"0x0.0C8789E09F72F1p0", "0x0.0CAE31A62B47C4p0", "0x0.0CD4DE-
FC3B4E2Dp0",
3152 "0x0.0CFB91E39C960Dp0",
"0x0.0D22A5D1C4CC9p0", "0x0.0D49086987BD4Fp0", "0x0.0D6FC-
C09A10C1p0", "0x0.0D96953E578B3Ep0", "0x0.0DBD6408571257p0",
"0x0.0DE4386878A6A5p0", "0x0.0E0B125F8A2704p0",
3153 "0x0.0E31F1EE598F33p0",
"0x0.0E58D715B4F989p0", "0x0.0E7FC1D66AA1C4p0",
"0x0.0EA6B23148D7F6p0", "0x0.0ECD48271E1164p0", "0x0.0EF4A3B88DE-
F8p0", "0x0.0F1BA4E67EF5Dp0", "0x0.0F42ABB27A0FAp0",
3154 "0x0.0F69B81C3E27FEp0",
"0x0.0F90CA25034260p0", "0x0.0FB7E1CD9883E5p0", "0x0.0F-
DEFF16CD3022p0", "0x0.1006220170A885p0", "0x0.102D4A8E526C53p0",
"0x0.105478BE4218B3p0", "0x0.107BAC920F68ACp0",
3155 "0x0.10A2E60A8A352Ep0",
"0x0.10CA2528827515p0", "0x0.10F169ECC83D2Ap0", "0x0.1118B4582B-
C02Dp0", "0x0.1140046B7D4ED5p0", "0x0.11675A278D57D7p0", "0x0.118E-
B58D2C67E8p0", "0x0.11B6169D2B29C2p0",
3156 "0x0.11DD7D585A662Bp0",
"0x0.1204E9BF8B03F4p0", "0x0.122C5BD38E0803p0",
"0x0.1253D395349553p0", "0x0.127B51054FECFBp0",
"0x0.12A2D424B16E31p0", "0x0.12CA5CF42A964Fp0", "0x0.12F1E-
B748D00D7p0",
3157 "0x0.13197FA6AA6777p0",
"0x0.1341198B54A20Dp0", "0x0.1368B9235DA6AEp0",
"0x0.13905E6F9789A8p0", "0x0.13B80970D47D86p0", "0x0.13DF-
BA27E6D314p0", "0x0.14077095A0F967p0", "0x0.142F2CBAD57DDEp0",
3158 "0x0.1456EE98570C27p0",
"0x0.147EB62EF86E43p0", "0x0.14A6837F8C8C8p0", "0x0.14CE568AE66D-
B7p0", "0x0.14F62F51D936DBp0", "0x0.151E0DD5382B74p0",
"0x0.1545F215D6AD66p0", "0x0.156DDC14883D04p0",
3159 "0x0.1595CBD2207913p0",
"0x0.15BDC14F731ECFp0", "0x0.15E5BC8D5409EEp0", "0x0.160DB-
D8C9734A6p0", "0x0.1635C44E10B780p0", "0x0.165DD0D294CA4Ep0",
"0x0.1685E31AF7C24Ep0", "0x0.16ADF8280E140Fp0",
3160 "0x0.16D618FAAC5286p0",
"0x0.16FE3C93A72F41p0", "0x0.172665F3D37A6Cp0",
"0x0.174E951C0622D6p0", "0x0.1776CA0D1435F4p0", "0x0.179F04C7D2D-
FE7p0", "0x0.17C7454D176B81p0", "0x0.17EF8B9DB74247p0",
3161 "0x0.1817D7BA87EC75p0",
"0x0.184029A45F1107p0", "0x0.1868815C127589p0",
"0x0.1890DEE277F60Dp0", "0x0.18B942386504Fp0", "0x0.18E1AB5EB1-
AB9Ap0", "0x0.190A1A563231DCp0", "0x0.19328F1FBDA2DCp0",
3162 "0x0.195B09BC2A7D3Ap0",
"0x0.19838A2C4F5E7Ap0", "0x0.19AC1071030303p0",
"0x0.19D49C8B1C4625p0", "0x0.19FD2E7B72220p0", "0x0.1A25C642DB-
B023p0", "0x0.1A4E63E2302857p0", "0x0.1A77075A46E1DEp0",
3163 "0x0.1A9FB0ABF752DBp0",
"0x0.1AC85FD8191074p0", "0x0.1AF114DF83CED8p0",
"0x0.1B19FC30F6145p0", "0x0.1B429083938A07p0",
"0x0.1B685721E8EA82p0", "0x0.1B94239EE72334p0", "0x0.1BBC5FB66B3B-
Cp0",
3164 "0x0.1BE5CE38400AD8p0",
"0x0.1C0EAC564BB672p0", "0x0.1C37905662639Fp0",
"0x0.1C607A395CDEA5p0", "0x0.1C896A001412FEp0", "0x0.1CB25-
FAB610B61p0", "0x0.1CDB5B3C1CF18Fp0", "0x0.1D045CB3210F50p0",
3165 "0x0.1D2D641146CC91p0",
"0x0.1D56715767B14Cp0", "0x0.1D7F84865D649Ap0", "0x0.1DA89D9F01ACE-
Bp0", "0x0.1DD18CA22E7006p0", "0x0.1DFA190BD8311p0",
"0x0.1E240C6B899A96p0", "0x0.1E4D3D336CA83p0",
3166 "0x0.1E7673E9408633p0",
"0x0.1E9FB08DE07072p0", "0x0.1ECF832226CB7Ep0", "0x0.1E-
F23BA6EE590Fp0", "0x0.1F1B8A1D11FA5Cp0", "0x0.1F44DE856CB01Dp0",
"0x0.1F6E38E0D99A90p0", "0x0.1F97993033F980p0",
3167 "0x0.1FC0FF74572C45p0",
"0x0.1FEA6BAE1EB1CEp0", "0x0.2013DDDE6628A0p0",
"0x0.203D5606094EE1p0", "0x0.2066D425E40256p0",
"0x0.2090583ED2406Ap0", "0x0.20B9E251B02636p0",
"0x0.20E3725F59F080p0",
3168 "0x0.210D0868ABFBC3p0",
"0x0.2136A46E82C433p0", "0x0.21604671BAE5C1p0",
"0x0.2189EE73311C20p0", "0x0.21B39C73C242C8p0", "0x0.21DD50744B54F-
Dp0", "0x0.22070A75A96DD4p0", "0x0.2230CA78B9C835p0",
3169 "0x0.225A907E59BEDFp0",
"0x0.22845C8766CC70p0", "0x0.22AE2E94B8EB69p0", "0x0.22D806A73E-
B62Fp0", "0x0.2301E4BFC52713p0", "0x0.232BC8DF2FD857p0",
"0x0.2355B3065CE42Fp0", "0x0.237FA3362A84CAp0",
3170 "0x0.23A9996F771453p0",
"0x0.23D395B3210CF7p0", "0x0.23FD98020708EAp0",
"0x0.2427A05D07C26Dp0", "0x0.2451AEC50213CEp0", "0x0.247B-
C33AD4F771p0", "0x0.24A5DDBF5F87D3p0", "0x0.24CFE5380FF8F8p0",
3171 "0x0.24FA24F818B962p0",
"0x0.252451AE063030p0", "0x0.254E847628F099p0", "0x0.2578B-
D5160E12Cp0", "0x0.25A2FC408DB20Fp0", "0x0.25CD41448F6D5Fp0",
"0x0.25F78C5E462F0Ap0", "0x0.2621DD8E92333Fp0",
3172 "0x0.264C34D653D674p0",
"0x0.267692366B956Dp0", "0x0.26A0F5AFBA0D3Cp0", "0x0.26CB5F431FF-
B4Cp0", "0x0.26F5CEF17E3D5Fp0", "0x0.272044BBB5D196p0",
"0x0.274AC0A2A7D678p0", "0x0.277542A7358AF1p0",
3173 };
3174 ap_ufixed<54,-2> exp_msb_ind_2_m_1 =
exp_msb_ind_2_m_1_table[x_msb_ind_2];
3175 ap_ufixed<100,-12>
f_x_msb_ind_2_msb_ind3_lsb = exp_msb_ind_3_lsb_ind_m_1 *
exp_msb_ind_2_m_1;
3176 ap_ufixed<52,-2>
exp_msb_ind2_msb_ind3_lsb_ind_m_1 = f_x_msb_ind_2_msb_ind3_lsb +
exp_msb_ind_3_lsb_ind_m_1 + exp_msb_ind_2_m_1;
3177 const static ap_ufixed<54,26>
exp_msb_ind_1_table[256] = {
3178 "0x1.0000000p0", "0x1.279FCD-
p0", "0x1.5561A92p0", "0x1.8A389FFp0", "0x1.C73D51Cp0", "0x2.0D-
B3D0Fp0", "0x2.5F1258Ep0", "0x2.BD0911Bp0",
3179 "0x3.298B076p0",
"0x3.A6D8842p0", "0x4.378B054p0", "0x4.D5A3099p0", "0x5.F9802Dp0",
"0x6.7E6ABF0p0", "0x7.7FBAAB4p0", "0x8.A8DE610p0",
3180 "0xA.0000000p0", "0xB.8C3DE-
BEp0", "0xD.55D09B1p0", "0xF.66363F8p0", "0x11.C86531Bp0",
"0x14.8966295p0", "0x17.B6B778Cp0", "0x1B.625A810p0",
3181 "0x1F.9F6E499p0",
"0x24.8475291p0", "0x2A.2B6E347p0", "0x30.B25E5F8p0", "0x38.3B-
F01BEp0", "0x40.F02B75Fp0", "0x4A.FD4A0B0p0", "0x56.98AFC2p0",
3182 "0x64.0000000p0",
"0x73.7A6B370p0", "0x85.5A260EDp0", "0x99.FE1E7B3p0",
"0xB1.D3F311p0", "0xC.D.5A3D9D0p0", "0xED.232AB7Bp0",
"0x111.D78AE44p0",
3183 "0x13C.3A4EDFBp0", "0x16D.
2C939A9p0", "0x1A5.B24E0C7p0", "0x1E6.F7AFBABp0",
"0x232.5761167p0", "0x289.61B29B6p0", "0x2ED.E4EAE72p0",
"0x361.F6DDE55p0",
3184 "0x3E8.0000000p0",

```

```

"0x482.C830261p0", "0x535.857C941p0", "0x603.ED30CFDp0",
"0x6F2.47876A9p0", "0x805.8668224p0", "0x943.5FAB2CAp0", "0x-
AB2.6B6D26Bp0",
3185 "0xC5A.4714BCAp0",
"0xE43.BDC409Bp0", "0x1078.F70C7CBp0", "0x1305.ACDD4AFp0",
"0x15F7.69CAE07p0", "0x195D.D0FA11Ep0", "0x1D4A.F12D073p0",
"0x21D3.A4AAF51p0",
3186 "0x2710.0000000p0",
"0x2D1B.D1E17C7p0", "0x3417.36DDC85p0", "0x3C27.43E81E5p0",
"0x4576.CB4A29Cp0", "0x5037.401156Cp0", "0x5CA1.BCAFBEB1p0",
"0x6AF8.324382Fp0",
3187 "0x7B86.C6CF5E3p0",
"0x8EA5.69A8609p0", "0xA4B9.A67CDEAp0", "0xBE38.C0A4ED5p0", "0xD-
BAA.21ECC48p0", "0xFDA4.29C4B2Ap0", "0x124ED.6BC2480p0",
"0x15244.6EAD929p0",
3188 "0x186A0.0000000p0",
"0x1C316.32CEDC6p0", "0x208E8.24A9D35p0", "0x25988.A7112F0p0",
"0x2B6A3.F0E5A19p0", "0x32228.80AD63Cp0", "0x39E51.5EDD6CAp0",
"0x42DB1.F6A31D7p0",
3189 "0x4D343.C419ADFp0",
"0x59276.2093C5Ep0", "0x66F40.80E0B21p0", "0x76E37.8671452p0",
"0x894A5.533FACCp0", "0x9E8A5.A1AEFA8p0", "0xB7146.3596CFDp0",
"0xD36AC.52C7B9Bp0",
3190 "0xF4240.0000000p0",
"0x119EDD.FC149BCp0", "0x145911.6EA2417p0", "0x177F56.86ABD5Cp0",
"0x1B2267.68F84F9p0", "0x1F5595.06C5E54p0", "0x242FD2.B4A63E0p0",
"0x29C8F3.A25F263p0",
3191 "0x3040A5.A900CB8p0",
"0x37B89D.45C5BB0p0", "0x405885.08C6F4Ap0", "0x4A4E2B.406CB33p0",
"0x5CE75.407CBF4p0", "0x631678.50D5C93p0", "0x726CBE.17E41E4p0",
"0x8422BB.3BCD40Ap0",
3192 "0x989680.0000000p0",
"0xB034AB.D8CE155p0", "0xCB7AAE.52568E6p0", "0xEAF961.42B6594p0",
"0x10F580A.19B31BCp0", "0x13957D2.43BAF49p0",
"0x169D7C9.0E7E6C2p0", "0x1A1D984.57B77DEp0",
3193 "0x1E28678.9A07F2Fp0",
"0x22D3624.B9B94DFp0", "0x2837532.57C58E8p0", "0x2E70D-
B0.843F002p0", "0x35A1094.84DF78Cp0", "0x3DE0B3.2859DBCp0",
"0x4783F6C.EEE92ECp0", "0x5295B50.5604864p0",
3194 "0x0.0000003p0",
"0x0.0000003p0", "0x0.0000004p0", "0x0.0000004p0", "0x0.0000005p0",
"0x0.0000006p0", "0x0.0000006p0", "0x0.0000007p0",
3195 "0x0.0000008p0",
"0x0.000000Ap0", "0x0.000000Bp0", "0x0.000000Dp0", "0x0.000000Fp0",
"0x0.0000011p0", "0x0.0000014p0", "0x0.0000017p0",
3196 "0x0.0000018p0",
"0x0.000001Fp0", "0x0.0000024p0", "0x0.0000029p0", "0x0.0000030p0",
"0x0.0000037p0", "0x0.0000040p0", "0x0.000004Ap0",
3197 "0x0.0000055p0",
"0x0.0000062p0", "0x0.0000071p0", "0x0.0000083p0", "0x0.0000097p0",
"0x0.00000AEp0", "0x0.00000C9p0", "0x0.00000E8p0",
3198 "0x0.000010Cp0",
"0x0.0000136p0", "0x0.0000166p0", "0x0.000019Dp0", "0x0.00001DDp0",
"0x0.0000227p0", "0x0.000027Dp0", "0x0.00002DFp0",
3199 "0x0.0000351p0",
"0x0.00003D4p0", "0x0.000046Cp0", "0x0.000051Bp0", "0x0.00005E6p0",
"0x0.00006CFp0", "0x0.00007DDp0", "0x0.0000915p0",
3200 "0x0.0000A7Cp0",
"0x0.0000C1Cp0", "0x0.0000DFCp0", "0x0.0001026p0", "0x0.00012A6p0",
"0x0.0001588p0", "0x0.00018DEp0", "0x0.0001CB7p0",
3201 "0x0.0002129p0",
"0x0.000264Bp0", "0x0.0002C38p0", "0x0.0003310p0", "0x0.0003AF7p0",
"0x0.0004418p0", "0x0.0004EA2p0", "0x0.0005ACEp0",
3202 "0x0.000680Cp0",
"0x0.0007916p0", "0x0.00088D4p0", "0x0.000A179p0", "0x0.000BA77p0",
"0x0.000D754p0", "0x0.000F8A8p0", "0x0.001F25p0",
3203 "0x0.0014B97p0",
"0x0.0017EEAp0", "0x0.001BA2Ep0", "0x0.001FE9Fp0", "0x0.0024DA8p0",
"0x0.002A8EDp0", "0x0.0031252p0", "0x0.003C808p0",
3204 "0x0.0041893p0",
"0x0.004BAE0p0", "0x0.005764Cp0", "0x0.0064EBBp0", "0x0.00748A9p0",
"0x0.0086947p0", "0x0.009B691p0", "0x0.00B3771p0",
3205 "0x0.00CF3E3p0",
"0x0.00E-
F521p0", "0x0.01145CFp0", "0x0.013F239p0", "0x0.0170894p0",
"0x0.01A9943p0", "0x0.01EB736p0", "0x0.023784Bp0",
3206 "0x0.028F5C3p0",
"0x0.02F4C-
C4p0", "0x0.0369EFCp0", "0x0.03F134Dp0", "0x0.048D69Cp0",
"0x0.0541CC5p0", "0x0.06121A8p0", "0x0.0702A69p0",
3207 "0x0.08186E2p0",
"0x0.0959348p0", "0x0.0ACBA15p0", "0x0.0C7763Fp0", "0x0.0E655C3p0",
"0x0.109FC99p0", "0x0.133281Bp0", "0x0.162B2F1p0",
3208 "0x0.19999Ap0",
"0x0.FABp0", "0x0.22235DBp0", "0x0.276C100p0", "0x0.2B8621Cp0",
"0x0.3491FB5p0", "0x0.3C8508Ep0", "0x0.461A81Cp0",
3209 "0x0.50F44D9p0",
"0x0.5D7C0D3p0", "0x0.6BF44D5p0", "0x0.7CA9E76p0", "0x0.8FF599Ep0",
"0x0.A63DDFEp0", "0x0.BFF9112p0", "0x0.DDAFD68p0",
3210 };
3211 ap_ufixed<54,26> exp_msb_ind_1 =
exp_msb_ind_1_table[x_msb_ind_1];
3212 ap_ufixed<106,24>
f_x_msb_ind_1_ind_2_ind_1_lsb = exp_msb_ind_1 *
exp_msb_ind_2_msb_ind_3_lsb_ind_m_1;
3213 y = f_x_msb_ind_1_ind_2_ind_1_lsb +
exp_msb_ind_1;
3214 }
3215 if(I_<27) {
3216     bool overf = 0;
3217     for(int j = 51; j >= 25 + I_; j--) {
3218         #pragma HLS unroll

```



```

"0x0.000000ACB1A279F741p0", "0x0.000000AEFF18B2FE0Cp0", "0x0.0000000B14C8EECA025p0", "0x0.0000000B39A052518B8p0", "0x0.0000000B5E77B5E323Fp0", "0x0.0000000B834F1974E40p0",
3276 "0x0.000000BA8267D06FEp0", "0x0.000000BCFCDE099629p0", "0x0.000000BF1D5442C212p0", "0x0.000000C16ACA7BF348p0", "0x0.000000C3B840B529CBp0", "0x0.000000C6056EE659Bp0",
3277 "0x0.000000C8532D27A689p0", "0x0.000000CAA0A360ED24p0", "0x0.000000CCEE199A38DDp0", "0x0.000000CF3B8FD389E2p0", "0x0.000000D189060CE035p0", "0x0.000000D3D67C463BD5p0",
"0x0.000000D623F27F9CC3p0", "0x0.000000D87168B902FEp0", "0x0.000000DABEDF26E86p0", "0x0.000000DD0C552BDF5Bp0",
3278 "0x0.000000DF59CB65557Ep0", "0x0.000000E1A7419ED0EEp0", "0x0.000000E3F4B7D851ABp0", "0x0.000000E6422E11D7B6p0", "0x0.000000E8FA44B630Dp0", "0x0.000000EADD1A84F3B2p0",
"0x0.000000ED2A90BE89A5p0", "0x0.000000EF7806F824E4p0",
3279 "0x0.000000F1C57D31C571p0", "0x0.000000F412F36B8B4Cp0", "0x0.000000F66069A51673p0", "0x0.000000F8ADDFDEC6E8p0", "0x0.000000FAFB56187CAAp0", "0x0.000000FD48CC5237B9p0",
"0x0.000000FF96428BF816p0", "0x0.00000101E3B85BDC0p0",
3280 "0x0.00000104312EFF88B7p0", "0x0.000001067EA53958FCp0", "0x0.00000108CC1B732E8Ep0", "0x0.0000010B1991AD096Dp0", "0x0.0000010D6707E6E999p0", "0x0.0000010FB47E20CF13p0",
"0x0.0000011201F45AB9DAp0", "0x0.000001144F6A94A9EEp0",
3281 "0x0.000001169CE0CE9F4Fp0", "0x0.00000118EA570899FEp0", "0x0.0000011B37CD4299FAp0", "0x0.0000011D85437C9F44p0", "0x0.0000011FD2B9B6A9DAp0", "0x0.00000122202FF0B9BEp0",
"0x0.000001246DA2ACEFFp0",
3282 };
3283 ap_ufixed<49,-23> exp_x_lsb_ind_m_1 =
3284 const static ap_ufixed<56,-16>
exp_x_msb_ind_4_m_1_table[128] = {
3285 "0x0.0p0", "0x0.00000126B-
B1C64FE3Ep0", "0x0.0000024D763A1D4ECAp0", "0x0.00000374315928F1A6p0", "0x0.0000049AEC7987E6D2p0", "0x0.000005C1A79B3A2E51p0",
"0x0.000006F1DE298844Dp0", "0x0.00000935D90844F2CDp0",
3286 "0x0.00000A5C942F4483A6p0", "0x0.00000B834F579766D9p0", "0x0.00000CA0A813D9C67p0", "0x0.00000D00C5AC372453p0", "0x0.00000EF780D883FE9Ep0", "0x0.0000101E3C06242B48p0",
"0x0.00001144F73517AA55p0", "0x0.0000126B2655E7BC5p0",
3287 "0x0.000013926D96F89F9Ap0", "0x0.000014B928C9E615D5p0", "0x0.000015DFE3FE26DE78p0", "0x0.000017069F33BAF985p0", "0x0.0000182D5A6AA266FCp0", "0x0.0000195415A2DD26E0p0",
"0x0.00001A7AD0DC6B3932p0", "0x0.00001BA18C174C9DF4p0",
3288 "0x0.00001CC84753815527p0", "0x0.00001DEF0291095ECCp0", "0x0.00001F15BDCFE4BAE6p0", "0x0.0000203C7910136975p0", "0x0.000021633451956A7Bp0", "0x0.00002289EF946ABDFAp0",
"0x0.00002380AAD89363F3p0", "0x0.000024D7661E0F5C68p0",
3289 "0x0.000025FE2164DEA75Ap0", "0x0.00002724DCAD0144CCp0", "0x0.0000284B97F67734BDp0", "0x0.000029725341407730p0", "0x0.00002A990E8D506C27p0", "0x0.00002BBFC9DACCF3A2p0",
"0x0.00002CE68529902DA4p0", "0x0.00002E0D4079A6BA2Ep0",
3290 "0x0.00002F33FBCB109942p0", "0x0.0000305AB71DCDCAE0p0", "0x0.000031817271DE4F8p0", "0x0.000032A82DC74225C5p0", "0x0.000033CEE91DF94F0Dp0", "0x0.000034F5A47603CAE7p0",
"0x0.0000361C5FCF619954p0", "0x0.000037431B2A12BA54p0",
3291 "0x0.00003869D686172DEBp0", "0x0.0000399091E36EF418p0", "0x0.00003-
AB74D421A0C0Fp0", "0x0.00003BDE08A218783Fp0", "0x0.00003D04C4036A363Cp0", "0x0.00003E2B7F660F46D5p0",
"0x0.00003F523ACA07AA0Ep0", "0x0.00004078F62F535FE7p0",
3292 "0x0.0000419FB195F26862p0", "0x0.000042C66CFDE4C380p0", "0x0.000043ED28672A7144p0", "0x0.00004513E3D1C371ADp0", "0x0.0000463A9F3DAFC48Bp0", "0x0.000047615AAAEF6A7Ap0",
"0x0.0000488816198262E1p0", "0x0.000049AED18968ADF4p0",
3293 "0x0.00004AD58CFAA24B85p0", "0x0.00004BFC486D2F3C26p0", "0x0.00004D2303E10F7F48p0", "0x0.00004E49BF5643151Cp0", "0x0.00004F707ACCC9FDA5p0", "0x0.0000509973644A438E4p0",
"0x0.0000518DF1BD11C6DAp0", "0x0.000052E4AD3852A788p0",
3294 "0x0.0000540B68B426DAF1p0", "0x0.000055322431AE6116p0", "0x0.00005658DFAFC939F8p0", "0x0.0000577F9B2F976599p0", "0x0.000058A656B08B8C3FBp0", "0x0.000059CD12332DB51Ep0",
"0x0.00005AF3CDB6F5D905p0", "0x0.00005C1A893C114FB1p0",
3295 "0x0.00005D4144C2801924p0", "0x0.00005E68004A42355Fp0", "0x0.00005F8EBBD357A463Bp0", "0x0.000060B5775DC06632p0", "0x0.000061DC32E97C7ACEp0", "0x0.00006249AA04EE9C72p0",
"0x0.00006429AA04EE9C72p0", "0x0.000065506594A4A97Dp0",
3296 "0x0.000066772125AE095Ap0", "0x0.0000679DDCB80ABC0p0", "0x0.000068C4984BBAC194p0", "0x0.000069EB53E0BE19F3p0", "0x0.00006B120F7714C52Bp0", "0x0.00006C38CB0EBEC33Ep0",
"0x0.00006D5F86A7BC142CP0", "0x0.00006E8642420CB7F8p0",
3297 "0x0.00006FACFDDDB0AEA2p0", "0x0.000070D3B97AA7F82Dp0",

```

```

"0x0.0043629EAA899713F8p0", "0x0.0043F6232D4E04F00Cp0",
"0x0.004489A804CD8B1B80p0",
3321 "0x0.00451D2D31385A7870p0",
"0x0.0045B0B2B28EA3E90Cp0", "0x0.004644388D0984FA0p0",
"0x0.0046D7BEB3FE688E98p0", "0x0.00476B453418458878p0", "0x0.0047-
FECC091E601FE4p0", "0x0.004892533310E93798p0", "0x0.004925D-
AB1F01B270p0"
3322 };
3323 ap_ufixed<61,-9> exp_x_msb_ind_3_m_1 =
exp_x_msb_ind_3_m_1_table[x_msb_ind_3];
3324 ap_ufixed<115,-25>
f_x_msb_ind_3_msb_ind_4_lsb_ind = exp_x_msb_ind_3_m_1 *
exp_x_msb_ind_4_lsb_ind_m_1;
3325 ap_ufixed<59,-9>
exp_x_msb_ind_3_msb_ind_4_lsb_ind_m_1 =
f_x_msb_ind_3_msb_ind_4_lsb_ind + exp_x_msb_ind_3_m_1 +
exp_x_msb_ind_4_lsb_ind_m_1;
3326 const static ap_ufixed<67,-1>
exp_x_msb_ind_2_m_1_table[256] = {
3327 "0x0.0p0", "0x0.0049B96285B-
C0A736p0", "0x0.009388004BE7E5593p0", "0x0.00DD6BDF6FC8EDEABp0",
"0x0.01276506106747AA4p0", "0x0.0171737A4E8E5E346p0", "0x0.01B-
B97424CCD67360p0", "0x0.0205D0642F77E4885p0",
3328 "0x.02501EE61CA62671Ep0",
"0x0.029A82CE33C5CE0DBp0", "0x0.02E4FC22B7CA4FD73p0",
"0x0.032F8AE9BACD765C6p0", "0x0.032AF29726FE5154p0",
"0x0.03C4E8E80DA99B612p0", "0x0.040FB82BBD3A77A9Ap0", "0x0.045A9C-
FAB3AABAA4p0",
3329 "0x.04A5975B254B8AE40p0",
"0x0.04F0A75348377827Fp0", "0x0.053BCCE95452FF5B2p0",
"0x0.05870823834D0E52Cp0", "0x0.05D25908109F87D7Ap0", "0x0.061DB-
F9D398FC7D9Dp0", "0x0.06693BE93D2F27B7p0", "0x0.06B4CDF25C5B82CCD-
p0",
3330 "0x.070075BED9BFBAAE4p0",
"0x0.074C3354F9D43D40Ap0", "0x0.079806BB02DF8738Ap0",
"0x0.07E3EFF73CF6A8B7Ap0", "0x0.082FEFF0FF1FDD767Bp0",
"0x0.087C040B6DA0D7F50p0", "0x0.08C82EEFFD7B9FB28p0",
"0x0.09146FC3F0CACC34Cp0",
3331 "0x.0960C68D98BC2BF2Ep0",
"0x0.09AD33534847443EAp0", "0x0.09F9B61B5435D762Cp0",
"0x0.0A464EEC13246AE7Ep0", "0x0.0A92FDCBD82CE006p0", "0x0.0AD-
FC2C10D94A01AAp0", "0x0.0B2C9DD1FF71D79A4p0",
"0x0.0B798F0511074887Fp0",
3332 "0x.0BC69660A2172C887p0",
"0x0.0C13BEB143A98A2Ap0", "0x0.0C60E7AACADD54194p0",
"0x0.0CAE31A62B47C3AC7p0", "0x0.0CFB91E39C960CB66p0",
"0x0.0D49086987BD4F4Ep0", "0x0.0D96953E578B3DC88p0",
"0x0.0DE4386878A6A4BEAp0",
3333 "0x.0E31F1EE598F35E7p0",
"0x0.0E7FC1D66AA1C3C87p0", "0x0.0ECD8A271E11639EAp0",
"0x0.0F1BA4E67EF5CBA0p0", "0x0.0F69B81C3E27FE069p0", "0x0.0F-
B7E1CD9883E4871p0", "0x0.1006220170A884803p0",
"0x0.105478BE4218B2CB7p0",
3334 "0x.10A2E60A8A35E513p0",
"0x0.10F169ECC83D29AA6p0", "0x0.1140046B7D4ED4EA6p0", "0x0.118E-
B58D2C67E78FCp0", "0x0.11DD7D585A662A9D7p0", "0x0.122C5BD38E0802D-
B1p0", "0x0.127B51054FEFCB3DBp0", "0x0.12CA5CF42A964F780p0",
3335 "0x.13197FA6AA6776B28p0",
"0x0.1368B9235DA6AE6BAp0", "0x0.13B80970D47D85804p0",
"0x0.14077095A0F9675B6p0", "0x0.1456EE98570C274Eep0",
"0x0.14A6837F8C8C8133p0", "0x0.14F62F51D936DB71Dp0",
"0x0.1545F215D6AD6612Fp0",
3336 "0x.1595CBDD207913796p0",
"0x0.15E5BC8D5409EE22Bp0", "0x0.1635C44E10B7AFD11p0",
"0x0.1685E31AF7C24DFD9p0", "0x0.16D618FAAC528672Ap0",
"0x0.172665F3D37A6C0F3p0", "0x0.1776CA0D1435F3B24p0",
"0x0.17C7454D176B814F4p0",
3337 "0x.1817D7BA87EC725AAp0",
"0x0.1868815C1275B93F8p0", "0x0.18B9423865B04ECDp0",
"0x0.190A1A563231DC114p0", "0x0.195B09BC2A7D3A217p0",
"0x0.19AC1071030302FA1p0", "0x0.19FD2E7B7221FACBp0",
"0x0.1A4E63E2302856BB6p0",
3338 "0x.1A9FB0ABF752DA9BEp0",
"0x0.1AF114DF83CED8647p0", "0x0.1B429083393BA06A18p0",
"0x0.1B94239EE72334542p0", "0x0.1BE5CE38400AD81A5p0",
"0x0.1C37905662639F7FEp0", "0x0.1C896A001412FE793p0", "0x0.1CD-
B5B3C1CF1BF06Bp0",
3339 "0x.1D2D641146CC91022p0",
"0x0.1D7F84865D649A153p0", "0x0.1DD18CA22E7005D96p0",
"0x0.1E240C6B899A96219p0", "0x0.1E7673E94086336D2p0",
"0x0.1EC8F32226CB7D849p0", "0x0.1F1B8A1D11FA5C3FAp0",
3340 "0x.1FC0FF74572C41577p0",
"0x0.2013DDDE6628A071Cp0", "0x0.2066D425E402559C1p0",
"0x0.20B9E251B026360EFp0", "0x0.210D0868ABFC3658p0",
"0x0.21604671BAE5C1485p0", "0x0.21B39C73C242C7830p0",
"0x0.22070A75A96DD4433p0",
3341 "0x.225A907E59BEDE81Cp0",
"0x0.22AE2E94BE8B6896Ap0", "0x0.2301E4BFC52712F67p0",
"0x0.235B3B065CE42F1A7p0", "0x0.23A9996F77145292Cp0", "0x0.23F-
D98020708EA434p0", "0x0.2451AEC50213CDA7p0", "0x0.24A5DD-
F5F87D312Ep0",
3342 "0x.24FA24F818B9620F7p0",
"0x0.254E847628FF08A1Bp0", "0x0.25A2FC408DB20EAB1p0",
"0x0.25F78C5E462F0A48Bp0", "0x0.264C34D653D67439Bp0",
"0x0.26A05FAFBA0D3C70Bp0", "0x0.26F5CEFF17E3D5ECF9p0",
"0x0.274AC0A2A7D6780E4p0",
3343 "0x.279FCACA404E5ACCBp0",
"0x0.27F4ED6F5321A4CF5p0", "0x0.284A2898EDD45466Ap0",
"0x0.289F7C4E1FF25E01Ep0", "0x0.28F4E895FB1041ECEp0",
"0x0.294A6D7792CBA238Bp0", "0x0.29A00AF9FCCBD8FCp0",
"0x0.29F5C12450C28DB50p0",
3344 "0x.2A4B8FFDA86C4D6E8p0",
"0x0.2AA1778D1F911F9B4p0", "0x0.2AF777D9A051D844p0",
"0x0.2B4D90EAE5A909F93p0", "0x0.2BA3C2C7776AE688p0", "0x0.2B-
FA0D76AA468CA2Fp0", "0x0.2C5070FFA746441ACp0",
"0x0.2CA6ED6995835AEE4p0",
3345 "0x.2CFD82BB9F26BD0EAp0",
"0x0.2D5430FCF0698C518p0", "0x0.2DAAF834B795888F5p0",
"0x0.2E01D86A250697ECCp0", "0x0.2E58D1A46B297F504p0",
"0x0.2EAFE3EABE7E5B03Bp0", "0x0.2F070F44559847819p0",
"0x0.2F5E53B8691E2A6E5p0",
3346 "0x.2FB5B14E33CB4BBE3p0",
"0x0.300D280CF26FEF065p0", "0x0.3064B7FBE3F1ECFAEp0", "0x0.30B-
C612249D4D18Bp0", "0x0.311423876594DF7B7p0", "0x0.316BFF327D-
F2D6E06p0", "0x0.31C3F42AD9A962D4Ep0", "0x0.321C0277C2134A11Dp0",
3347 "0x.32742A2082A485035p0",
"0x0.32CC6B2C68EAD87CAp0", "0x0.3324C5A2C48E70995p0",
"0x0.337D398AE7527BCA2p0", "0x0.33D5CEC2515C60F8p0", "0x0.342E6D-
CD03D3545FDp0", "0x0.34872E374BA3003AFp0", "0x0.34E0082FE6BA136AD-
p0",
3348 "0x.3538FBFBF016BE3F00p0",
"0x0.359208EBFA2A701C4p0", "0x0.35EB2FBE3186FA090p0",
"0x0.3644703D0A32A81BDp0", "0x0.369DCA6FE8F1986Fp0",
"0x0.36F73E5E34DF0B37Bp0", "0x0.3750CC0F5E6F0C1Ep0",
"0x0.37AA738ABA4D92580p0",
3349 "0x.380434D7CC6CAA213p0",
"0x0.385E0FFDFCC181BC3p0", "0x0.38B80504BCED8BF00p0",
"0x0.391213F380B716895p0", "0x0.396C3CD1BE09C0665p0", "0x0.39C67-
FA6ECF73FAF1p0", "0x0.3A20DC7A8B7EC35p0",
"0x0.3A7B53540AAB622AFp0",
3350 "0x.3AD5E43AF459209E5p0",
"0x0.3B308F36C57128AF1p0", "0x0.3B8B544F00CC9C88Ap0",
"0x0.3BE6338B2B6E5EB41p0", "0x0.3C412CF2C283B1910p0",
"0x0.3C9C408D06D64D6FC5p0", "0x0.3CF7E62B99580251p0",
"0x0.3D52B679DEC65D8F1p0",
3351 "0x.3DAE18DACCD3DF440p0",
"0x0.3E09958CF5C8B5321p0", "0x0.3E652C97EDDD7FB9Bp0",
"0x0.3EC0DE034B79A0686p0", "0x0.3F1CA9D6A733DAE2Ep0",
"0x0.3F7890199BD2F5FCFp0", "0x0.3FD490D3C64E5D001p0",
"0x0.4030AC0CC5CEC11FFp0",
3352 "0x.408CE1CC3BAEBB1E6p0",
"0x0.40E93219CB7B6D1CCp0", "0x0.41459C2F1AF5249CFp0",
"0x0.41A227DD20FFCB02p0", "0x0.41FEC2AD9AF480553p0",
"0x0.425B7D7622004D04Dp0", "0x0.428B52FD15C6B56D6p0",
"0x0.431543402711645D2p0",
3353 "0x.43724E4708E0FFDB7p0",
"0x0.43CF7419706DCC711p0", "0x0.442CB4BF1528509F4p0",
"0x0.448A103FB0B9F8866p0", "0x0.44E786A2FF0589BB0p0",
"0x0.454517F0BE28B74ACp0", "0x0.45A2C430AE7AE5F03p0",
"0x0.46008B6A928F8075Ep0",
3354 "0x.465E6DA62F369C48Ep0",
"0x0.46BC6AEB4B7BEE3AAp0", "0x0.471AB341B0A94F727p0",
"0x0.4778B6B12A46728E1p0", "0x0.47D705418619B8F25p0", "0x0.48356E-
FA9428D84B1p0", "0x0.4893F3E426B9803ABp0",
"0x0.48F2940612520039Ap0",
3355 "0x.49514F682DB9EDA59p0",
"0x0.49B0261251FACA004p0", "0x0.4A0F180C5A60A95E7p0",
"0x0.4A6E255E247AD906Ap0", "0x0.4ACDAE0F901C863FEp0",
"0x0.4B2C92287F5D65507p0", "0x0.4B8BF1B0D69A58AD0p0", "0x0.4BE-
B6CB07C7618574p0",
3356 "0x.4C4B032F59D0D96D8p0",
"0x0.4CAAB53559F5F5E9Fp0", "0x0.4D0A82CA6A429492Bp0", "0x0.4D6A6B-
F67A8051199p0", "0x0.4DCA70C17C8BE4602p0",
"0x0.4E2A19133653FCD395p0", "0x0.4E8ACD542AB2F8995p0", "0x0.4EE-
B25BC5F86AF99p0",
3357 "0x.4F4B98C2324DE92ACp0",
"0x0.4FAC281F6D2BA1A4Fp0", "0x0.500CD34B7662D5FC3p0",
"0x0.506D9A4E500F84855p0", "0x0.50CE7D2FF9C122BEp0", "0x0.512F7B-
F888C1F4791p0", "0x0.519096AFF78A58CB5p0", "0x0.51F1CD5E564EDB-
F2p0",
3358 "0x.5253200BB2BA1FC90p0",
"0x0.52B48EC01CC882005p0", "0x0.53161983A6C8CA1BBp0",
"0x0.5377C05E655CCE9E1p0", "0x0.53D983586F7A2235Ep0",
"0x0.543B6279DE6ABF4CAp0", "0x0.549D5DCACDCDB3C8Dp0",
"0x0.54FF75535B97CD007p0",
3359 };
3360 ap_ufixed<67,-1> exp_x_msb_ind_2_m_1 =
exp_x_msb_ind_2_m_1_table[x_msb_ind_2];
3361 ap_ufixed<126,-10>
f_x_msb_ind_2_msb_ind_3_msb_ind_4_lsb_ind = exp_x_msb_ind_2_m_1 *
exp_x_msb_ind_3_msb_ind_4_lsb_ind_m_1;
3362 ap_ufixed<65,-1>
exp_x_msb_ind_2_msb_ind_3_msb_ind_4_lsb_ind_m_1 =
f_x_msb_ind_2_msb_ind_3_msb_ind_4_lsb_ind + exp_x_msb_ind_2_m_1 +
exp_x_msb_ind_3_msb_ind_4_lsb_ind_m_1;
3363 const static ap_ufixed<66,32>
exp_x_msb_ind_1_table[256] = {
3364 "0x1.0000000000p0",
"0x1.5561A918Cp0", "0x1.C73D51C54p0", "0x2.5F1258E08p0",
"0x3.298B075B4p0", "0x4.378B053BCp0", "0x5.9F9082C8Cp0", "0x7.7F-
BAAB458p0",
3365 "0xA.0000000000p0", "0xD.
550D9B148p0", "0x11.C86531B4Cp0", "0x17.B6B778C40p0", "0x1F.
9FB6E49910p0", "0x2A.2B6E34724p0", "0x38.3BF01BD84p0", "0x4A.FD4-
A0B864p0",
3366 "0x64.0000000000p0",
"0x85.5A260ECDCp0", "0xB1.D3F310ECp0", "0xEED.232AB7A90p0", "0x13C.

```

```

3A4EDFA98p0", "0x1A5.B24E0C774p0", "0x232.57611671Cp0",
"0x2ED.E4EAE71ECp0",
3367 "0x3E8.000000000p0",
"0x535.857C94088p0", "0x6F2.47876A934p0", "0x943.5FAB2C9B4p0", "0x-
C5A.4714BC9E8p0", "0x1078.F70C7CA94p0", "0x15F7.69CAE0728p0",
"0x104A.F12D0732Cp0",
3368 "0x2710.000000000p0",
"0x3417.36DDC8558p0", "0x4576.CB4A29C18p0", "0x5CA1.BCAFB0F8p0",
"0x7B86.C6CF5E320p0", "0xA4B9.A67CDE9B4p0", "0xDBAA.21ECC4790p0",
"0x124ED.6BC247F88p0",
3369 "0x186A0.000000000p0",
"0x208E8.24A9D3580p0", "0x2B6A3.F0E5A18ECp0",
"0x39E51.5EDD6C9A0p0", "0x4D343.C419ADF30p0",
"0x66F40.80E0B210Cp0", "0x894A5.533FACBA4p0",
"0xB7146.3596CFD3Cp0",
3370 "0xF4240.000000000p0",
"0x145911.6EA2416F4p0", "0x1B2267.68F84F938p0",
"0x242F2D.B4A63E038p0", "0x3040A5.A900C87F0p0",
"0x405885.08C6F4A64p0", "0x55CE75.407CBF46Cp0", "0x726CBE.
17E41E45Cp0",
3371 "0x989680.000000000p0", "0x-
CB7AAE.52568E584p0", "0x10F580A.19B31BC34p0",
"0x169D7C9.0E7E6C220p0", "0x1E28678.9A07F2F6Cp0",
"0x2837532.57C58E7F0p0", "0x35A1094.84DF78C38p0",
"0x4783F6C.EEE92EB90p0",
3372 "0x5F5E100.000000000p0",
"0x7F2CACF.37618F720p0", "0xA997065.00FF159FCp0", "0xE226DDA.
90F039530p0", "0x12D940B6.044F7DA48p0", "0x192293F7.6DB790F54p0",
"0x1284A5CD.30BAB7A30p0", "0x2CB27A41.551BD339Cp0",
3373 "0x3B9ACA00.000000000p0",
"0x4F7BEC18.29CF9A750p0", "0x69FE63F2.09F6D83E4p0",
"0x8D584A89.A9623D3E0p0", "0xB0C7C871C.2B1AE86C4p0", "0xF5B59C7AA.
492BA9948p0", "0x14F2E7A03.E74B2C5F4p0", "0x1BEF8C68D.531640404p0",
3374 "0x2540BE400.000000000p0",
"0x31AD73BF1.A21C08914p0", "0x423EFE774.63A4726F4p0",
"0x58572E960.9DD6646C4p0", "0x75CDD4719.AF0D143A8p0", "0x9D181C-
CA6.DB849FCD0p0", "0xD17D0C427.08EFBB80p0",
"0x1175B7C185.3EDE8282Cp0",
3375 "0x174876E800.000000000p0",
"0x1F0C683970.551855AD0p0", "0x29675F0A8B.E46C78594p0",
"0x37367D1DC6.2A5FEC3A0p0", "0x49A0A4C700.D682CA49Cp0",
"0x622F11FE84.950E3E020p0", "0x82EE27A986.595D55300p0", "0x-
AE992D8F34.74B119180p0",
3376 "0xE8D4A51000.000000000p0",
"0x1C76C123E63.52F358C14p0", "0x19E09B66976.EC3CB37C0p0",
"0x22820E329BD.A7BF3A448p0", "0x2E0466FC608.611BE6E18p0",
"0x3D5D6B3F12D.D28E6C148p0", "0x51D4D8C9F3D.7DA553E00p0", "0x6D1FB-
C7980C.8EEAFB130p0",
3377 "0x9184E72A000.000000000p0",
"0xC20D8B66FE1.3D81778C0p0", "0x102C61201EA5.3A5F02D70p0",
"0x159148DFA168.8D7846AE0p0", "0x1CC2C05DBCC53.CB1704CF4p0",
"0x265A63076BCA.399038CD8p0", "0x3325077E387A.E87546BF0p0",
"0x4433D5CBF07D.952DCEBE8p0",
3378 "0x5AF3107A4000.000000000p0",
"0x794877205ECC.670EAB770p0", "0xA1BB8CB413274.47B61C654p0",
"0xD7ACD88C4E15.86B2C2CC0p0", "0x11F9B83A95B45.EEE6301D8p0",
"0x17F87DE4A35E6.3FA238074p0", "0x1FF724AEE34CD.1494C3758p0",
"0x2AA0659F764E7.D3CA13714p0",
3379 "0x38D7EA4C68000.000000000p0",
"0x48CD4A743B3FC.0692B2A70p0", "0x65155F08BF88A.CD1D1BF4Cp0",
"0x86CC0775B0CD7.42FB9BF90p0", "0xB3C13249D090BB.54FDE1260p0",
"0xEFBAEAE61AFE.7C563047Cp0", "0x13FA76ED4E1002.CDCFA296Cp0",
"0x1AA43F83A9F10E.45E4C26DCp0",
3380 "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0",
3381 "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0",
3382 "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0",
3383 "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0",
3384 "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0",
3385 "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0", "0x0.0p0",
3386 "0x0.0p0", "0x0.0p0", "0x.000000004p0", "0x.000000004p0", "0x.
000000004p0", "0x.000000004p0",
3387 "0x0.000000008p0", "0x.000000008p0", "0x.
000000008p0", "0x.000000008p0", "0x.000000008p0",
3388 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3389 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3390 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3391 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3392 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3393 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3394 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3395 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3396 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3397 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3398 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3399 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3400 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3401 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3402 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3403 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3404 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3405 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3406 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3407 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3408 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3409 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3410 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3411 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3412 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3413 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3414 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3415 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3416 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3417 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3418 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3419 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3420 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3421 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3422 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3423 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3424 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3425 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3426 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3427 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3428 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3429 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3430 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3431 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3432 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3433 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3434 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3435 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3436 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3437 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3438 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3439 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3440 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3441 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3442 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3443 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3444 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3445 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3446 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3447 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3448 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3449 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3450 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3451 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3452 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3453 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3454 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3455 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3456 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3457 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3458 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3459 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3460 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3461 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3462 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3463 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3464 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3465 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3466 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3467 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3468 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3469 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3470 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3471 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3472 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3473 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3474 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3475 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3476 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3477 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3478 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3479 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3480 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3481 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3482 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3483 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3484 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3485 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3486 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3487 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3488 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3489 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3490 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3491 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3492 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3493 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3494 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3495 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3496 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3497 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3498 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3499 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3500 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3501 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3502 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3503 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3504 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3505 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3506 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3507 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3508 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3509 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3510 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3511 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3512 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3513 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3514 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3515 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3516 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3517 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3518 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3519 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3520 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3521 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3522 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3523 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3524 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3525 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3526 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3527 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3528 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3529 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3530 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3531 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3532 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3533 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3534 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3535 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3536 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3537 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3538 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3539 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3540 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3541 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3542 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3543 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3544 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3545 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3546 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3547 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3548 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3549 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3550 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3551 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3552 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3553 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3554 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3555 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3556 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3557 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3558 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3559 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3560 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3561 "0x.000000008p0", "0x.000000008p0", "0x.000000008p0",
3562 "0x.000000008p0", "0x.000000008p0", "0x.000
```

```

26         #else
27             return 0;
28         #endif
29     } else{
30         return(x/y);
31     }
32 }
33 //divide for ap_ufixed numerator and denominator
34 template <int W_, int I>
35 ap_ufixed<W_,I> generic_divide(ap_ufixed<W_,I> x,
ap_ufixed<W_,I> y){
36     ap_fixed<W+1,I+1> x1 = x;
37     ap_fixed<W+1,I+1> y1 = y;
38     return generic_divide(x1,y1);
39 }
40 //divide for ap_int
41 template <int I>
42 ap_int<I> generic_divide(ap_int<I> x, ap_int<I> y){
43     ap_fixed<I,I> x1 = x;
44     ap_fixed<I,I> y1 = y;
45     return generic_divide(x1,y1);
46 }
47 //divide for ap_uint
48 template <int I>
49 ap_uint<I> generic_divide(ap_uint<I> x, ap_uint<I> y){
50     ap_ufixed<I,I> x1 = x;
51     ap_ufixed<I,I> y1 = y;
52     return generic_divide(x1,y1);
53 }
54 //fmod for ap_fixed
55 template <int W_, int I>
56 ap_fixed<W_,I> generic_fmod(ap_fixed<W_,I> x,
ap_fixed<W_,I> y){
57     ap_ufixed<W_,I> x_pos =
fabs_fixed((ap_fixed<W+1,I+1>)x);
58     ap_ufixed<W_,I> y_pos =
fabs_fixed((ap_fixed<W+1,I+1>)y);
59     //std::cout<<"x_pos"<<x_pos<<std::endl;
60     //std::cout<<"y_pos"<<y_pos<<std::endl;
61     if(x_pos==0 || y_pos==0) return 0;
62     else if(x_pos == y_pos) return 0;
63     else if(x_pos < y_pos) return x;
64     else {
65         ap_uint<W> x_int = 0;
66         ap_uint<W> y_int = 0;
67         x_int(W-1,0) = x_pos(W-1,0);
68         y_int(W-1,0) = y_pos(W-1,0);
69         //use less resource
70         //for(int i = 0; i < W; ++i) {
71             // if(x_int > y_int) {
72                 // x_int -= y_int;
73             // }
74         //}
75         ap_uint<W> d = x_int/y_int;
76         //std::cout<<"d"=<d<<std::endl;
77         ap_uint<W> rem = x_int - d*y_int;
78         //std::cout<<"rem"=<rem<<std::endl;
79         ap_fixed<W,I> r = 0;
80         r(W-1,0) = rem(W-1,0);
81         if(x[W-1]) return -r;
82         else return r;
83     }
84 }
85 //fmod for ap_ufixed
86 template <int W_, int I>
87 ap_ufixed<W_,I> generic_fmod(ap_ufixed<W_,I> x,
ap_ufixed<W_,I> y){
88     ap_fixed<W+1,I+1> xi = x;
89     ap_fixed<W+1,I+1> yi = y;
90     return generic_fmod(xi,yi);
91 }
92 //fmod for ap_int
93 template <int I>
94 ap_int<I> generic_fmod(ap_int<I> x, ap_int<I> y){
95     ap_fixed<I,I> xi = x;
96     ap_fixed<I,I> yi = y;
97     return generic_fmod(xi,yi);
98 }
99 //fmod for ap_uint
100 template <int I>
101 ap_uint<I> generic_fmod(ap_uint<I> x, ap_uint<I> y){
102     ap_fixed<I+1,I+1> xi = x;
103     ap_fixed<I+1,I+1> yi = y;
104     return generic_fmod(xi,yi);
105 }
106 //remquo for ap_fixed
107 template <int W_, int I>
108 ap_fixed<W_,I> generic_remquo(ap_fixed<W_,I> x,
ap_fixed<W_,I> y, int* quo){
109     ap_ufixed<W_,I> x_pos =
fabs_fixed((ap_fixed<W+1,I+1>)x);
110     ap_ufixed<W_,I> y_pos =
fabs_fixed((ap_fixed<W+1,I+1>)y);
111     if(x_pos==0 || y_pos==0) {
112         *quo = 0;
113         return 0;
114     } else if(x_pos == y_pos) {
115         if(x[W-1]==y[W-1]) *quo = 1;

```

```

118     else *quo = -1;
119     return 0;
120 }
121 //else if(x_pos < y_pos) return x;
122 else {
123     ap_uint<W> x_int = 0;
124     ap_uint<W> y_int = 0;
125     x_int(W-1,0) = x_pos(W-1,0);
126     y_int(W-1,0) = y_pos(W-1,0);
127     //for(int i = 0; i < W; ++i) {
128         // if(x_int > y_int) {
129             // x_int -= y_int;
130         // }
131     //}
132     ap_uint<W> d = x_int/y_int;
133     ap_uint<W> rem = x_int - d*y_int;
134     //adjust the result
135     ap_uint<W> y_half = 0;
136     y_half(W-2,0) = y_int(W-1,1);
137     ap_uint<I> r_sig = x[W-1];
138     if(rem > y_half) {
139         rem = y_int - rem;
140         d++;
141         if(x[W-1]) r_sig = 0;
142         else r_sig = 1;
143     }
144     //return the results
145     if(x[W-1]==y[W-1]) *quo = d;
146     else *quo = -d;
147     ap_fixed<W,I> r = 0;
148     r(W-1,0) = rem(W-1,0);
149     if(r_sig) return -r;
150     else return r;
151 }
152 //, y, quoremquo for ap_ufixed
153 template <int W_, int I>
154 ap_ufixed<W_,I> generic_remquo(ap_ufixed<W_,I> x,
ap_ufixed<W_,I> y, int* quo){
155     ap_fixed<W+1,I+1> xi = x;
156     ap_fixed<W+1,I+1> yi = y;
157     return generic_remquo(xi,yi,quo);
158 }
159 //remquo for ap_int
160 template <int I>
161 ap_int<I> generic_remquo(ap_int<I> x, ap_int<I> y,
int* quo){
162     ap_fixed<I,I> xi = x;
163     ap_fixed<I,I> yi = y;
164     return generic_remquo(xi,yi,quo);
165 }
166 //remquo for ap_uint
167 template <int I>
168 ap_uint<I> generic_remquo(ap_uint<I> x, ap_uint<I> y,
int* quo){
169     ap_fixed<I+1,I+1> xi = x;
170     ap_fixed<I+1,I+1> yi = y;
171     return generic_remquo(xi,yi,quo);
172 }
173 //remainder for ap_fixed
174 template <int W_, int I>
175 ap_fixed<W_,I> generic_remainder(ap_fixed<W_,I> x,
ap_fixed<W_,I> y){
176     int quo = 0;
177     return generic_remquo(x, y, &quo);
178 }
179 //remainder for ap_ufixed
180 template <int W_, int I>
181 ap_ufixed<W_,I> generic_remainder(ap_ufixed<W_,I> x,
ap_ufixed<W_,I> y){
182     ap_fixed<W+1,I+1> xi = x;
183     ap_fixed<W+1,I+1> yi = y;
184     return generic_remainder(xi,yi);
185 }
186 //remainder for ap_int
187 template <int I>
188 ap_int<I> generic_remainder(ap_int<I> x, ap_int<I> y){
189     ap_fixed<I,I> xi = x;
190     ap_fixed<I,I> yi = y;
191     return generic_remainder(xi,yi);
192 }
193 //remainder for ap_uint
194 template <int I>
195 ap_uint<I> generic_remainder(ap_uint<I> x, ap_uint<I>
y){
196     ap_fixed<I+1,I+1> xi = x;
197     ap_fixed<I+1,I+1> yi = y;
198     return generic_remainder(xi,yi);
199 }
200 }
201 #endif // entire file
202 }
203
204

```

E.22 vendor/etc/hls_fract_apfixed.h (42 lines, 1,028 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3

```



```

4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5 #ifndef _X_HLS_GENERIC_FRACT_H_
6 #define _X_HLS_GENERIC_FRACT_H_
7
8 #include "utils/x_hls_defines.h"
9 #include "utils/x_hls_utils.h"
10 #include "utils/x_hls_traits.h"
11
12 namespace hls_fract {
13 //fract for ap_fixed
14 template <int W_, int I_>
15 ap_fixed<W_, I_> generic_fract(ap_fixed<W_, I_> x){
16     if (W_==I_)return 0;
17     ap_fixed<W_-I_+1,1> x1=x;
18     ap_fixed<W_-I_+1,1> x2=x1+ap_int<2>(1);
19     return x[W_-1]?x2:x1;
20 }
21 //fract for ap_ufixed
22 template <int W_, int I_>
23 ap_ufixed<W_, I_> generic_fract(ap_ufixed<W_, I_> x){
24     ap_fixed<W_+1,I_+1> x1=x;
25     return generic_fract(x1);
26 }
27 //fract for ap_int
28 template <int I_>
29 ap_int<I_> generic_fract(ap_int<I_> x){
30     ap_fixed<I_,I_> x1 = x;
31     return generic_fract(x1);
32 }
33 //fract for ap_uint
34 template <int I_>
35 ap_uint<I_> generic_fract(ap_uint<I_> x){
36     ap_ufixed<I_,I_> x1 = x;
37     return generic_fract(x1);
38 }
39 }
40 #endif
41
42

```

E.23 vendor/etc/hls_frexp_apfixed.h (103 lines, 3,002 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6
7 /**
8  * @file hls_frexp_ap_fixed.h
9  * @brief implement frexp function for ap_fixed data type.
10  */
11 #ifndef _X_HLS_FREXP_APFIXED_H_
12 #define _X_HLS_FREXP_APFIXED_H_
13 #include "ap_fixed.h"
14 #include "etc/hls_round_copysign_apfixed.h"
15 namespace frexp_internal {
16
17 template<int W, int I>
18 ap_fixed<W,I> frexp(ap_fixed<W,I> x,
19                    ap_fixed<W,I>* exp){
20     if(I<0) {
21         *exp = 0;
22         return x;
23     }
24     static const int F = W - I;
25     static const int we = W > 5 ? W/2 : 3; //width of exp,
should be log2(W) + 1;
26
27     ap_int<we> e = 0;
28     ap_ufixed<W, I> r_p = 0;
29
30     ap_fixed<W+1, I+1> xi = x;
31     ap_ufixed<W, I> x_p = fabs_fixed(xi);
32
33     static const ap_uint<3> clz_table_6bit[64] = {7, 6, 5, 5, 4,
4, 4, 4,
34
35     3, 3, 3, 3, 3,
3, 3, 3,
36
37     2, 2, 2, 2, 2,
2, 2, 2,
38
39     2, 2, 2, 2, 2,
2, 2, 2,
40
41     1, 1, 1, 1, 1,
1, 1, 1,
42
43     1, 1, 1, 1, 1,
1, 1, 1,
44
45     1, 1, 1, 1, 1,
1, 1, 1,
46
47     1, 1, 1, 1, 1,
1, 1, 1,
48
49     1, 1, 1, 1, 1,
1, 1, 1,
50
51     1, 1, 1, 1, 1,
1, 1, 1,
52
53     1, 1, 1, 1, 1,
1, 1, 1,
54
55     1, 1, 1, 1, 1,
1, 1, 1,
56
57     1, 1, 1, 1, 1,
1, 1, 1,
58
59     1, 1, 1, 1, 1,
1, 1, 1,
60
61     1, 1, 1, 1, 1,
1, 1, 1,
62
63     1, 1, 1, 1, 1,
1, 1, 1,
64
65     1, 1, 1, 1, 1,
1, 1, 1,
66
67     1, 1, 1, 1, 1,
1, 1, 1,
68
69     1, 1, 1, 1, 1,
1, 1, 1,
70
71     1, 1, 1, 1, 1,
1, 1, 1,
72
73     1, 1, 1, 1, 1,
1, 1, 1,
74
75     1, 1, 1, 1, 1,
1, 1, 1,
76
77     1, 1, 1, 1, 1,
1, 1, 1,
78
79     1, 1, 1, 1, 1,
1, 1, 1,
80
81     1, 1, 1, 1, 1,
1, 1, 1,
82
83     1, 1, 1, 1, 1,
1, 1, 1,
84
85     1, 1, 1, 1, 1,
1, 1, 1,
86
87     1, 1, 1, 1, 1,
1, 1, 1,
88
89     1, 1, 1, 1, 1,
1, 1, 1,
90
91     1, 1, 1, 1, 1,
1, 1, 1,
92
93     1, 1, 1, 1, 1,
1, 1, 1,
94
95     1, 1, 1, 1, 1,
1, 1, 1,
96
97     1, 1, 1, 1, 1,
1, 1, 1,
98
99     1, 1, 1, 1, 1,
1, 1, 1,
100
101     1, 1, 1, 1, 1,
1, 1, 1,
102
103     1, 1, 1, 1, 1,
1, 1, 1,
104
105     1, 1, 1, 1, 1,
1, 1, 1,
106
107     1, 1, 1, 1, 1,
1, 1, 1,
108
109     1, 1, 1, 1, 1,
1, 1, 1,
110
111     1, 1, 1, 1, 1,
1, 1, 1,
112
113     1, 1, 1, 1, 1,
1, 1, 1,
114
115     1, 1, 1, 1, 1,
1, 1, 1,
116
117     1, 1, 1, 1, 1,
1, 1, 1,
118
119     1, 1, 1, 1, 1,
1, 1, 1,
120
121     1, 1, 1, 1, 1,
1, 1, 1,
122
123     1, 1, 1, 1, 1,
1, 1, 1,
124
125     1, 1, 1, 1, 1,
1, 1, 1,
126
127     1, 1, 1, 1, 1,
1, 1, 1,
128
129     1, 1, 1, 1, 1,
1, 1, 1,
130
131     1, 1, 1, 1, 1,
1, 1, 1,
132
133     1, 1, 1, 1, 1,
1, 1, 1,
134
135     1, 1, 1, 1, 1,
1, 1, 1,
136
137     1, 1, 1, 1, 1,
1, 1, 1,
138
139     1, 1, 1, 1, 1,
1, 1, 1,
140
141     1, 1, 1, 1, 1,
1, 1, 1,
142
143     1, 1, 1, 1, 1,
1, 1, 1,
144
145     1, 1, 1, 1, 1,
1, 1, 1,
146
147     1, 1, 1, 1, 1,
1, 1, 1,
148
149     1, 1, 1, 1, 1,
1, 1, 1,
150
151     1, 1, 1, 1, 1,
1, 1, 1,
152
153     1, 1, 1, 1, 1,
1, 1, 1,
154
155     1, 1, 1, 1, 1,
1, 1, 1,
156
157     1, 1, 1, 1, 1,
1, 1, 1,
158
159     1, 1, 1, 1, 1,
1, 1, 1,
160
161     1, 1, 1, 1, 1,
1, 1, 1,
162
163     1, 1, 1, 1, 1,
1, 1, 1,
164
165     1, 1, 1, 1, 1,
1, 1, 1,
166
167     1, 1, 1, 1, 1,
1, 1, 1,
168
169     1, 1, 1, 1, 1,
1, 1, 1,
170
171     1, 1, 1, 1, 1,
1, 1, 1,
172
173     1, 1, 1, 1, 1,
1, 1, 1,
174
175     1, 1, 1, 1, 1,
1, 1, 1,
176
177     1, 1, 1, 1, 1,
1, 1, 1,
178
179     1, 1, 1, 1, 1,
1, 1, 1,
180
181     1, 1, 1, 1, 1,
1, 1, 1,
182
183     1, 1, 1, 1, 1,
1, 1, 1,
184
185     1, 1, 1, 1, 1,
1, 1, 1,
186
187     1, 1, 1, 1, 1,
1, 1, 1,
188
189     1, 1, 1, 1, 1,
1, 1, 1,
189

```

```

40
1, 1, 1;
41
42 int pos = 0; //should be log(mqx(I,F));
43 //use clz_table to reduce the loop_tripcount
44 static const int loop_nm = W > 6 ? (W + 5)/6 : 1;
45 for(int l = loop_nm; l > 0; --l){
46     #pragma HLS pipeline II=1
47     ap_uint<6> t = 0;
48     if(l*6 > W) t = x_p(W - 1, (l - 1) * 6);
49     else t = x_p(l * 6 - 1, (l - 1) * 6);
50     if((t & 0x3f) != 0) {
51         pos = l * 6 - clz_table_6bit[t];
52         break;
53     }
54 }
55 if(pos >= 0) {
56     e = pos + 1 - F;
57     if(F > 0) {
58         ap_uint<we> w_f = (pos + 1) < F ? (pos + 1) : F;
59         r_p(F - 1, F - w_f) = x_p(pos, pos + 1 - w_f);
60     }
61 } else {
62     e = 0;
63     r_p = 0;
64 }
65 ap_fixed<W, I> r = 0;
66 if(xi[W-1]) r = -r_p;
67 else r = r_p;
68 #ifndef __SYNTHESIS__
69 assert(r > -1 && r < 1);
70 #endif
71 *exp = e;
72 return r;
73 }
74 template<int W, int I>
75 ap_ufixed<W,I> frexp(ap_ufixed<W,I> x,
76                    ap_ufixed<W,I>* exp){
77     ap_fixed<W+1, I+1> xe = x;
78     ap_fixed<W+1, I+1> ee = 0;
79     ap_fixed<W+1, I+1> r = frexp(xe,&ee);
80     *exp = ee;
81     return r;
82 }
83 template<int I>
84 ap_uint<I> frexp(ap_uint<I> x,
85                ap_uint<I>* exp) {
86     ap_fixed<I, I> xe = x;
87     ap_fixed<I, I> ee = 0;
88     ap_fixed<I, I> r = frexp(xe,&ee);
89     *exp = ee;
90     return r;
91 }
92 template<int I>
93 ap_int<I> frexp(ap_int<I> x,
94                ap_int<I>* exp) {
95     ap_fixed<I+1, I+1> xe = x;
96     ap_fixed<I+1, I+1> ee = 0;
97     ap_fixed<I+1, I+1> r = frexp(xe,&ee);
98     *exp = ee;
99     return r;
100 }
101 #endif//define _X_HLS_FREXP_APFIXED_H_
102
103

```

E.24 vendor/etc/hls_half_fpo.h (609 lines, 20,888 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef __HLS_HALF_FPO_H_
7 #define __HLS_HALF_FPO_H_
8 // This is just for csim and cosim instead of synthesis.
9
10 // That is to say, __SYNTHESIS__ and HLS_NO_XIL_FPO_LIB are not
defined.
11 #ifndef __cplusplus
12 #error C++ is required to include this header file
13 #endif
14 #include <iostream>
15 #include <limits>
16
17 // XXX work-around old version of GMP with C++11 <csdtdf>
used by fpo.
18 #include <csdtdf>
19 #include "hls_fpo.h"
20
21 // Forward declaration of ap_fixed_base.
22 #include "etc/ap_decl.h"
23
24 // Fiddle an MPFR variable to contain the same information as an
xip_fpo_t variable without allocating memory

```

```

25 // m is the mpfr_t variable (destination), x is the xip_fpo_t
variable (source)
26 // First variant: copy all information
27 #define XIP_FPO_2_MPFPR(m, x) \
28 (m)->mpfr_prec = (x)->xip_fpo_mant_prec; \
29 (m)->mpfr_sign = (x)->xip_fpo_sign; \
30 (m)->mpfr_exp = (x)->xip_fpo_exp; \
31 (m)->mpfr_d = (x)->xip_fpo_d;
32
33 class half {
34 public:
35     typedef unsigned short uint16;
36     template <typename T1, typename T2>
37     struct half_traits;
38
39     /* 16-bit float following IEEE 754
40      16 bits inside          float value
41     0 00000 0000000000 = 0x0000 = 0
42     1 00000 0000000000 = 0x8000 = -0
43
44     0 11111 0000000000 = 0x7c00 = infinity
45     1 11111 0000000000 = 0xfc00 = -infinity
46     0 11110 1111111111 = 0x7bff = max = 65504
47     x 11111 yyyyyyyyyy = nan, in which yyyyyyyy !=0
48
49     */
50     half() : m_data() {}
51     half(float rhs) {
52         xip_fpo_half_set_flt(rhs);
53     }
54
55     /// Conversion constructor from AP types.
56     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
57     _AP_Q, ap_o_mode _AP_O, int _AP_N>
58     half(const ap_fixed_base<_AP_W, _AP_I, _AP_S, _AP_Q,
59     _AP_O, _AP_N>& rhs) {
60         std::cout << "WARNING: explicit method
61         ap_fixed::to_half() should be used to convert ap_fixed to half."
62         << std::endl;
63         *this = rhs.to_half();
64     }
65
66     template <int _AP_W, bool _AP_S>
67     half(const ap_int_base<_AP_W, _AP_S>& rhs) {
68         *this = rhs.to_half();
69     }
70
71     template <int _AP_W, int _AP_I, bool _AP_S, ap_q_mode
72     _AP_Q, ap_o_mode _AP_O, int _AP_N>
73     half& operator=(const ap_fixed_base<_AP_W, _AP_I, _AP_S,
74     _AP_Q, _AP_O, _AP_N>& rhs) {
75         return *this = rhs.to_half();
76     }
77
78     template <int _AP_W, bool _AP_S>
79     half& operator=(const ap_int_base<_AP_W, _AP_S>& rhs) {
80         return *this = rhs.to_half();
81     }
82
83     static bool half_isnan(half hf) { return (hf.m_data &
84     0x7FFF) > 0x7C00; }
85
86     // half to float implicitly or explicitly
87     // for example, static_cast<float>(half)
88     operator float() const { return xip_fpo_half_get_flt(); }
89
90     half& operator=(float rhs) { return *this = half(rhs); }
91
92     //----- unary operation -----/
93     template <typename T>
94     half& operator+=(const T& rhs) {
95         return *this = half_axu_functions::half_plus(*this,
96         rhs);
97     }
98
99     half& operator+=(const half& rhs) { return *this = *this +
100     rhs; }
101
102     template <typename T>
103     half& operator-=(const T& rhs) {
104         return *this = half_axu_functions::half_minus(*this,
105         rhs);
106     }
107
108     half& operator-=(const half& rhs) { return *this = *this
109     - rhs; }
110
111     template <typename T>
112     half& operator*=(const T& rhs) {
113         return *this = half_axu_functions::half_multiplies(*this,
114         rhs);
115     }
116
117     half& operator*=(const half& rhs) { return *this = *this
118     * rhs; }
119
120     template <typename T>
121     half& operator/=(const T& rhs) {
122         return *this = half_axu_functions::half_divides(*this,
123         rhs);
124     }

```

```

109     }
110
111     half& operator/=(const half& rhs) { return *this =
112     *this / rhs; }
113
114     /// Prefix increment.
115     half& operator++() { return *this += 1.0f; }
116
117     /// Prefix decrement.
118     half& operator--() { return *this -= 1.0f; }
119
120     /// Postfix increment.
121     half operator++(int) {
122         half out(*this);
123         ++*this;
124         return out;
125     }
126
127     /// Postfix decrement.
128     half operator--(int) {
129         half out(*this);
130         --*this;
131         return out;
132     }
133
134     //----- unary operation -----/
135
136     //----- native data -----/
137     void set_bits(uint16 d) { m_data = d; }
138
139     uint16 get_bits() { return m_data; }
140
141 private:
142     /// Unsigned integer of (at least) 16 bits width.
143     uint16 m_data;
144     //----- native data -----/
145
146     friend std::ostream& operator<<(std::ostream& out, const
147     half& hf) {
148         float f = static_cast<float>(hf);
149         return out << f;
150     }
151
152     friend std::istream& operator>>(std::istream& in, half&
153     hf) {
154         float f;
155         if (in >> f) hf = static_cast<half>(f);
156         return in;
157     }
158
159 public:
160     //----- arithmetic operations-----/
161
162     // plus T + half
163     template <typename T>
164     friend typename half::half_traits<T, half>::rtvl_t oper-
165     ator+(const T& lhs, const half& rhs) {
166         return half_axu_functions::half_plus(lhs, rhs);
167     }
168
169     // plus half + T
170     template <typename T>
171     typename half::half_traits<half, T>::rtvl_t operator+
172     (const T& rhs) {
173         return half_axu_functions::half_plus(*this, rhs);
174     }
175
176     // plus half + half
177     half operator+(const half& rhs) { return half_axu_func-
178     tions::half_plus(*this, rhs); }
179
180     // sub T - half
181     template <typename T>
182     friend typename half::half_traits<T, half>::rtvl_t oper-
183     ator-(const T& lhs, const half& rhs) {
184         return half_axu_functions::half_minus(lhs, rhs);
185     }
186
187     // sub half - T
188     template <typename T>
189     typename half::half_traits<half, T>::rtvl_t operator-
190     (const T& rhs) {
191         return half_axu_functions::half_minus(*this, rhs);
192     }
193
194     // sub half - half
195     half operator-(const half& rhs) { return half_axu_func-
196     tions::half_minus(*this, rhs); }
197
198     // mul T * half
199     template <typename T>
200     friend typename half::half_traits<T, half>::rtvl_t oper-
201     ator*(const T& lhs, const half& rhs) {
202         return half_axu_functions::half_multiplies(lhs, rhs);
203     }
204
205     // mul half * T
206     template <typename T>
207     typename half::half_traits<half, T>::rtvl_t operat-
208     or*(const T& rhs) {
209         return half_axu_functions::half_multiplies(*this,
210         rhs);
211     }

```

```

197     }
198     // mul half * half
199     half operator*(const half& rhs) { return half_axu_func-
tions::half_multiplies(*this, rhs); }
200
201     // div T / half
202     template <typename T>
203     friend typename half::half_traits<T, half>::rtvl_t oper-
ator/(const T& lhs, const half& rhs) {
204         return half_axu_functions::half_divides(lhs, rhs);
205     }
206
207     // div half / T
208     template <typename T>
209     typename half::half_traits<half, T>::rtvl_t operator/
(const T& rhs) {
210         return half_axu_functions::half_divides(*this, rhs);
211     }
212     // div half/half
213     half operator/(const half& rhs) { return half_axu_func-
tions::half_divides(*this, rhs); }
214
215     //----- comparison-----//
216     /*
217     // It seems to conflict with ap_private.
218     template<typename T>
219     friend bool operator== (const T& lhs, const half& rhs)
{
220         return half_axu_functions::half_isequal(lhs, rhs);
221     }
222
223     template<typename T>
224     friend bool operator!= (const T& lhs, const half& rhs) {
225         return half_axu_functions::half_isnotequal(lhs,
rhs);
226     }
227     template<typename T>
228     friend bool operator> (const T& lhs, const half& rhs) {
229         return half_axu_functions::half_isgreater(lhs, rhs);
230     }
231
232     template<typename T>
233     friend bool operator>= (const T& lhs, const half& rhs) {
234         return half_axu_functions::half_isgreaterequal(lhs,
rhs);
235     }
236
237     template<typename T>
238     friend bool operator< (const T& lhs, const half& rhs) {
239         return half_axu_functions::half_isless(lhs, rhs);
240     }
241
242     template<typename T>
243     friend bool operator<= (const T& lhs, const half& rhs)
{
244         return half_axu_functions::half_islessequal(lhs,
rhs);
245     }
246
247     */
248     template <typename T>
249     bool operator==(const T& rhs) {
250         return half_axu_functions::half_isequal(*this, rhs);
251     }
252     bool operator==(const half& rhs) { return half_axu_func-
tions::half_isequal(*this, rhs); }
253
254     template <typename T>
255     bool operator!=(const T& rhs) {
256         return half_axu_functions::half_isnotequal(*this,
rhs);
257     }
258     bool operator!=(const half& rhs) { return half_axu_func-
tions::half_isnotequal(*this, rhs); }
259
260     template <typename T>
261     bool operator>(const T& rhs) {
262         return half_axu_functions::half_isgreater(*this,
rhs);
263     }
264     bool operator>(const half& rhs) { return half_axu_func-
tions::half_isgreater(*this, rhs); }
265     template <typename T>
266     bool operator>=(const T& rhs) {
267         return half_axu_functions::half_isgreaterequal(*this,
rhs);
268     }
269     return half_axu_functions::half_isgreaterequal(*this,
rhs);
270     }
271     bool operator>=(const half& rhs) { return half_axu_func-
tions::half_isgreaterequal(*this, rhs); }
272
273     template <typename T>
274     bool operator<(const T& rhs) {
275         return half_axu_functions::half_isless(*this, rhs);
276     }
277     bool operator<(const half& rhs) { return half_axu_func-
tions::half_isless(*this, rhs); }
278

```

```

279     template <typename T>
280     bool operator<=(const T& rhs) {
281         return half_axu_functions::half_islessequal(*this,
rhs);
282     }
283     bool operator<=(const half& rhs) { return half_axu_func-
tions::half_islessequal(*this, rhs); }
284     //----- comparison-----//
285
286     //----- xilinx float point -----//
287     // half simulation model with Xilinx Floating Point Oper-
ator IP core
288     // bit-accurate simulation ref PG60, here provides some
functions with one or two arguments which invoke inner
289     // datatype and functions in lib. F is an operation func-
tion, such as xip_fpo_add_flt, xip_fpo_sub.
290     template <typename F>
291     static half math_function_1arg(F f, half x) {
292         half res;
293         xip_fpo_t rop, xop;
294         xip_fpo_inits2(5, 11, rop, xop, (xip_fpo_ptr)0);
295         x.xip_fpo_get_data(xop);
296         f(rop, xop);
297         res.xip_fpo_set_data(rop);
298         xip_fpo_clears(rop, xop, (xip_fpo_ptr)0);
299         return res;
300     }
301
302     template <typename F>
303     static half binary_operator(F f, half x, half y) {
304         half res;
305         xip_fpo_t op, xop, yop;
306         xip_fpo_inits2(5, 11, op, xop, yop, (xip_fpo_ptr)0);
307         x.xip_fpo_get_data(xop);
308         y.xip_fpo_get_data(yop);
309         f(op, xop, yop);
310         res.xip_fpo_set_data(op);
311         xip_fpo_clears(op, xop, yop, (xip_fpo_ptr)0);
312         return res;
313     }
314     // perform comparison
315     template <typename F>
316     static bool binary_operator_compare(F f, half x, half y)
{
317         int res;
318         xip_fpo_t xop, yop;
319         xip_fpo_inits2(5, 11, xop, yop, (xip_fpo_ptr)0);
320         x.xip_fpo_get_data(xop);
321         y.xip_fpo_get_data(yop);
322         f(&res, xop, yop);
323         xip_fpo_clears(xop, yop, (xip_fpo_ptr)0);
324         return res;
325     }
326
327     private:
328     /// Assign value from half data to xip_fpo struct
329     /// \return xip_fpo exceptions
330     xip_fpo_exc_t xip_fpo_get_data(xip_fpo_ptr op) const {
331         int exc = 0;
332         op->xip_fpo_sign = ((m_data & 0x8000) ? -1 : 1);
333         op->xip_fpo_exp = ((m_data & 0x7C00) >> 10) - 14;
334         *(op->xip_fpo_d) = ((mp_limb_t)(m_data & 0x3FF) +
(mp_limb_t)(0x400)) << (8 * sizeof(*op->xip_fpo_d) - 11);
335         if ((m_data & 0x7C00) == 0) { // subnormal
336             exc |= 0x1;
337             xip_fpo_set_zero(op, op->xip_fpo_sign);
338         } else if ((m_data & 0x7FFF) == 0x7C00) { // infinity
339             exc |= 0x2;
340             xip_fpo_set_inf(op, op->xip_fpo_sign);
341         } else if ((m_data & 0x7FFF) > 0x7C00) { // NaN
342             exc |= 0x4;
343             xip_fpo_set_nan(op);
344         }
345         return exc;
346     }
347
348     /// Convert
349     float xip_fpo_half_get_flt() const {
350         xip_fpo_t op;
351         xip_fpo_init2(op, 5, 11);
352         xip_fpo_exc_t exc = xip_fpo_get_data(op);
353         float res;
354         if (exc & 0x1) {
355             res = (op->xip_fpo_sign > 0 ? 0.0f : -0.0f);
356         } else if (exc & 0x2) {
357             res = (op->xip_fpo_sign > 0 ? std::numeri-
c_limits<float>::infinity() : -std::numeri-
c_limits<float>::infinity());
358         } else if (exc & 0x4) {
359             res = std::numeric_limits<float>::quiet_NaN();
360         } else {
361             res = xip_fpo_get_flt(op);
362         }
363         xip_fpo_clear(op);
364         return res;
365     }
366
367     /// Assign value from xip_fpo struct to half data
368     /// \return void
369

```



```

370 void xip_fpo_set_data(xip_fpo_ptr op) {
371     mpfr_t fr;
372     XIP_FPO_2_MPFR(fr, op);
373     m_data = 0;
374     m_data |= (op->xip_fpo_sign == 1 ? 0 : 1) << 15;
375     if (mpfr_zero_p(fr)) {
376         m_data &= 0x8000;
377     } else if (mpfr_inf_p(fr)) {
378         m_data |= 0x7C00;
379     } else if (mpfr_nan_p(fr)) {
380         m_data |= 0x7E00;
381     } else {
382         m_data |= (op->xip_fpo_exp + 14) << 10;
383         m_data |= (*(op->xip_fpo_d) << 1) >> (8 *
sizeof(*(op->xip_fpo_d)) - 10);
384     }
385 }
386
387 /// Assignment
388 void xip_fpo_half_set_flt(float rhs) {
389     xip_fpo_t op;
390     xip_fpo_init2(op, 5, 11);
391     xip_fpo_set_flt(op, rhs);
392     xip_fpo_set_data(op);
393     xip_fpo_clear(op);
394 }
395 // ----- details on arithmetic and comparison opera-
tions ----//
396 // pack them in an embedded struct for convenient manag-
ment.
397 //
398 struct half_axu_functions {
399     // plus +
400     template <typename T1, typename T2>
401     static half half_plus(T1 x, T2 y) {
402         return half::binary_operator(xip_fpo_add, x, y);
403     }
404     static float half_plus(float x, half y) { return
HLS_FPO_ADDF(x, y); }
405     static float half_plus(half x, float y) { return
HLS_FPO_ADDF(x, y); }
406     static double half_plus(double x, half y) { return
HLS_FPO_ADD(x, y); }
407     static double half_plus(half x, double y) { return
HLS_FPO_ADD(x, y); }
408     // sub -
409     template <typename T1, typename T2>
410     static half half_minus(T1 x, T2 y) {
411         return half::binary_operator(xip_fpo_sub, x, y);
412     }
413     static float half_minus(float x, half y) { return
HLS_FPO_SUBF(x, y); }
414     static float half_minus(half x, float y) { return
HLS_FPO_SUBF(x, y); }
415     static double half_minus(double x, half y) { return
HLS_FPO_SUB(x, y); }
416     static double half_minus(half x, double y) { return
HLS_FPO_SUB(x, y); }
417     // mul *
418     template <typename T1, typename T2>
419     static half half_multiplies(T1 x, T2 y) {
420         return half::binary_operator(xip_fpo_mul, x, y);
421     }
422     static float half_multiplies(float x, half y) { re-
turn HLS_FPO_MULF(x, y); }
423     static float half_multiplies(half x, float y) { re-
turn HLS_FPO_MULF(x, y); }
424     static double half_multiplies(double x, half y) { re-
turn HLS_FPO_MUL(x, y); }
425     static double half_multiplies(half x, double y) { re-
turn HLS_FPO_MUL(x, y); }
426     // div /
427     template <typename T1, typename T2>
428     static half half_divides(T1 x, T2 y) {
429         return half::binary_operator(xip_fpo_div, x, y);
430     }
431     static float half_divides(float x, half y) { return
HLS_FPO_DIVF(x, y); }
432     static float half_divides(half x, float y) { return
HLS_FPO_DIVF(x, y); }
433     static double half_divides(double x, half y) { return
HLS_FPO_DIV(x, y); }
434     static double half_divides(half x, double y) { return
HLS_FPO_DIV(x, y); }
435     // equal ==
436     template <typename T1, typename T2>
437     static bool half_isequal(T1 x, T2 y) {
438         return half::bin-
ary_operator_compare(xip_fpo_equal, x, y);
439     }
440     static bool half_isequal(float x, half y) { return
HLS_FPO_EQUALF(x, y); }
441     static bool half_isequal(half x, float y) { return
HLS_FPO_EQUALF(x, y); }
442     static bool half_isequal(double x, half y) { return
HLS_FPO_EQUAL(x, y); }
443     static bool half_isequal(half x, double y) { return
HLS_FPO_EQUAL(x, y); }
444     // not equal !=
445     template <typename T1, typename T2>
446     static bool half_isnotequal(T1 x, T2 y) {
447         return half::bin-
ary_operator_compare(xip_fpo_notequal, x, y);
448     }
449     static bool half_isnotequal(float x, half y) { return
HLS_FPO_NOTEQUALF(x, y); }
450     static bool half_isnotequal(half x, float y) { return
HLS_FPO_NOTEQUALF(x, y); }
451     static bool half_isnotequal(double x, half y) { re-
turn HLS_FPO_NOTEQUAL(x, y); }
452     static bool half_isnotequal(half x, double y) { re-
turn HLS_FPO_NOTEQUAL(x, y); }
453     // greater >
454     template <typename T1, typename T2>
455     static bool half_isgreater(T1 x, T2 y) {
456         return half::bin-
ary_operator_compare(xip_fpo_greater, x, y);
457     }
458     static bool half_isgreater(float x, half y) { return
HLS_FPO_GREATERF(x, y); }
459     static bool half_isgreater(half x, float y) { return
HLS_FPO_GREATERF(x, y); }
460     static bool half_isgreater(double x, half y) { return
HLS_FPO_GREATER(x, y); }
461     static bool half_isgreater(half x, double y) { return
HLS_FPO_GREATER(x, y); }
462     // greater equal >=
463     template <typename T1, typename T2>
464     static bool half_isgreaterequal(T1 x, T2 y) {
465         return half::bin-
ary_operator_compare(xip_fpo_greaterequal, x, y);
466     }
467     static bool half_isgreaterequal(float x, half y)
{ return HLS_FPO_GREATEREQUALF(x, y); }
468     static bool half_isgreaterequal(half x, float y)
{ return HLS_FPO_GREATEREQUALF(x, y); }
469     static bool half_isgreaterequal(double x, half y)
{ return HLS_FPO_GREATEREQUAL(x, y); }
470     static bool half_isgreaterequal(half x, double y)
{ return HLS_FPO_GREATEREQUAL(x, y); }
471     // less <
472     template <typename T1, typename T2>
473     static bool half_isless(T1 x, T2 y) {
474         return half::bin-
ary_operator_compare(xip_fpo_less, x, y);
475     }
476     static bool half_isless(float x, half y) { return
HLS_FPO_LESSF(x, y); }
477     static bool half_isless(half x, float y) { return
HLS_FPO_LESSF(x, y); }
478     static bool half_isless(double x, half y) { return
HLS_FPO_LESS(x, y); }
479     static bool half_isless(half x, double y) { return
HLS_FPO_LESS(x, y); }
480     // less equal <=
481     template <typename T1, typename T2>
482     static bool half_islessequal(T1 x, T2 y) {
483         return half::bin-
ary_operator_compare(xip_fpo_lessequal, x, y);
484     }
485     static bool half_islessequal(float x, half y) { re-
turn HLS_FPO_LESSEQUALF(x, y); }
486     static bool half_islessequal(half x, float y) { re-
turn HLS_FPO_LESSEQUALF(x, y); }
487     static bool half_islessequal(double x, half y) { re-
turn HLS_FPO_LESSEQUAL(x, y); }
488     static bool half_islessequal(half x, double y) { re-
turn HLS_FPO_LESSEQUAL(x, y); }
489
490 }; // struct half_axu_functions
491 //----- xilinx float point -----//
492 }; // class half
493
494 template <>
495 struct half_traits<char, half> {
496     typedef half rtvl_t;
497 };
498
499 template <>
500 struct half_traits<signed char, half> {
501     typedef half rtvl_t;
502 };
503
504 template <>
505 struct half_traits<unsigned char, half> {
506     typedef half rtvl_t;
507 };
508
509 template <>
510 struct half_traits<short, half> {
511     typedef half rtvl_t;
512 };
513
514 template <>

```

```

515 struct half::half_traits<unsigned short, half> {
516     typedef half rtvl_t;
517 };
518
519 template <>
520 struct half::half_traits<int, half> {
521     typedef half rtvl_t;
522 };
523
524 template <>
525 struct half::half_traits<unsigned int, half> {
526     typedef half rtvl_t;
527 };
528
529 template <>
530 struct half::half_traits<float, half> {
531     typedef float rtvl_t;
532 };
533
534 template <>
535 struct half::half_traits<double, half> {
536     typedef double rtvl_t;
537 };
538
539 template <>
540 struct half::half_traits<long, half> {
541     // typedef double rtvl_t;
542     typedef half rtvl_t;
543 };
544
545 template <>
546 struct half::half_traits<unsigned long, half> {
547     typedef half rtvl_t;
548     // typedef double rtvl_t;
549 };
550
551 template <>
552 struct half::half_traits<half, char> {
553     typedef half rtvl_t;
554 };
555
556 template <>
557 struct half::half_traits<half, signed char> {
558     typedef half rtvl_t;
559 };
560
561 template <>
562 struct half::half_traits<half, unsigned char> {
563     typedef half rtvl_t;
564 };
565
566 template <>
567 struct half::half_traits<half, short> {
568     typedef half rtvl_t;
569 };
570
571 template <>
572 struct half::half_traits<half, unsigned short> {
573     typedef half rtvl_t;
574 };
575
576 template <>
577 struct half::half_traits<half, int> {
578     typedef half rtvl_t;
579 };
580
581 template <>
582 struct half::half_traits<half, unsigned int> {
583     typedef half rtvl_t;
584 };
585
586 template <>
587 struct half::half_traits<half, float> {
588     typedef float rtvl_t;
589 };
590
591 template <>
592 struct half::half_traits<half, double> {
593     typedef double rtvl_t;
594 };
595
596 template <>
597 struct half::half_traits<half, long> {
598     // typedef double rtvl_t;
599     typedef half rtvl_t;
600 };
601
602 template <>
603 struct half::half_traits<half, unsigned long> {
604     typedef half rtvl_t;
605     // typedef double rtvl_t;
606 };
607
608
609 #endif // __HLS_HALF_FPO__

```

E.25 vendor/etc/hls_half_x_utils.h (625 lines, 18,339 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All

```

Rights Reserved.

```

3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef __HLS_HALF_UTILS_H_
7 #define __HLS_HALF_UTILS_H_
8
9 #ifndef __cplusplus
10 #ifndef __SYNTHESIS__
11 #error C++ is required to include this header file
12 #endif
13 #endif
14
15 #ifndef __SYNTHESIS__
16 #include <cmath>
17 #include <iostream>
18 #include <limits>
19 #include <climits>
20
21 #if defined(__clang__) // clang
22 #if (defined(__GXX_EXPERIMENTAL_CXX0X__) || __cplusplus >=
201103L) && !defined(HALF_ENABLE_CPP11_LONG_LONG)
23 #define HALF_ENABLE_CPP11_LONG_LONG 1
24 #endif
25 #elif defined(__GNUC__) // gcc
26 #if !defined(HALF_ENABLE_CPP11_LONG_LONG)
27 #define HALF_ENABLE_CPP11_LONG_LONG 1
28 #endif
29 #elif defined(_MSC_VER) // Visual C++
30 #if _MSC_VER >= 1310 && !defined(HALF_ENABLE_CPP11_LONG_LONG)
31 #define HALF_ENABLE_CPP11_LONG_LONG 1
32 #endif
33 #endif
34
35 #define FP_FAST_FMAH 1
36
37 #ifndef FP_ILOGB0
38 #define FP_ILOGB0 INT_MIN
39 #endif
40 #ifndef FP_ILOGBNAN
41 #define FP_ILOGBNAN INT_MAX
42 #endif
43 #ifndef FP_SUBNORMAL
44 #define FP_SUBNORMAL 0
45 #endif
46 #ifndef FP_ZERO
47 #define FP_ZERO 1
48 #endif
49
50 #ifndef FP_NAN
51 #define FP_NAN 2
52 #endif
53 #ifndef FP_INFINITE
54 #define FP_INFINITE 3
55 #endif
56 #ifndef FP_NORMAL
57 #define FP_NORMAL 4
58 #endif
59
60 #ifndef HALF_ROUND_STYLE
61 #define HALF_ROUND_STYLE 1 // = std::round_indeterminate
62 #endif
63 #ifndef HALF_ROUND_TIES_TO_EVEN
64 #define HALF_ROUND_TIES_TO_EVEN 1 // ties away from zero
65 #endif
66 #ifndef _MSC_VER
67 #include <algorithm>
68 #endif
69 class half;
70 namespace detail {
71
72 /* 16-bit float following IEEE 754
73    16 bits inside          float value
74    0 00000 00000000000 = 0x0000 = 0
75    1 00000 00000000000 = 0x8000 = -0
76
77    0 11111 00000000000 = 0x7c00 = infinity
78    1 11111 00000000000 = 0xfc00 = -infinity
79    0 11110 11111111111 = 0x7bff = max = 65504
80    x 11111 yyyyyyyyyy =          = nan, in which yyyyyyyy !=0
81
82 */
83
84 inline bool isfinite(half arg) {
85     return (arg.get_bits() & 0x7C00) != 0x7C00;
86 }
87
88 inline bool isinf(half arg) {
89     return (arg.get_bits() & 0x7FFF) == 0x7C00;
90 }
91
92 inline bool isnan(half arg) {
93     return (arg.get_bits() & 0x7FFF) > 0x7C00;
94 }
95
96 inline bool isnormal(half arg) {
97     return ((arg.get_bits() & 0x7C00) != 0) &&
98         ((arg.get_bits() & 0x7C00) != 0x7C00);
99 }

```

```

99
100 inline bool signbit(half arg) {
101     return (arg.get_bits() & 0x8000) != 0;
102 }
103
104 inline int fpclassify(half arg) {
105     unsigned int abs = arg.get_bits() & 0x7FFF;
106     if (abs == 0x7C00) return FP_INFINITE;
107     if (abs > 0x3FF) return FP_NORMAL;
108     return abs ? FP_SUBNORMAL : FP_ZERO;
109 }
110
111 inline half fmod(half x, half y) {
112     return std::fmod(x, y);
113 }
114
115 inline half sinh(half arg) {
116     return std::sinh(arg);
117 }
118
119 inline half cosh(half arg) {
120     return std::cosh(arg);
121 }
122
123 inline half sin(half arg) {
124     return std::sin(arg);
125 }
126
127 inline half cos(half arg) {
128     return std::cos(arg);
129 }
130
131 inline half frexp(half arg, int* exp) {
132     unsigned int m = arg.get_bits() & 0x7FFF;
133     if (m >= 0x7C00 || !m) return *exp = 0, arg;
134     int e = m >> 10;
135     if (!e)
136         for (m <= 1; m < 0x400; m <= 1, --e)
137             ;
138     *exp = e - 14;
139     half t;
140     t.set_bits(static_cast<half::uint16>((arg.get_bits() &
0x8000) | 0x3800 | (m & 0x3FF)));
141     return t;
142 }
143
144 inline half modf(half arg, half* iptr) {
145     half t;
146     unsigned int e = arg.get_bits() & 0x7C00;
147     if (e > 0x6000) {
148         *iptr = arg;
149         t.set_bits(arg.get_bits() & 0x8000);
150         return (e == 0x7C00 && (arg.get_bits() & 0x3FF)) ?
arg : t;
151     }
152     if (e < 0x3C00) return iptr->set_bits(arg.get_bits() &
0x8000), arg;
153     e >>= 10;
154     unsigned int mask = (1 << (25 - e)) - 1, m =
arg.get_bits() & mask;
155     iptr->set_bits(arg.get_bits() & ~mask);
156     if (!m)
157         t.set_bits(arg.get_bits() & 0x8000);
158     else {
159         for (; m < 0x400; m <= 1, --e)
160             ;
161         t.set_bits(static_cast<half::uint16>((arg.get_bits()
& 0x8000) | (e << 10) | (m & 0x3FF)));
162     }
163     return t;
164 }
165
166 inline int ilogb(half arg) {
167     int exp = arg.get_bits() & 0x7FFF;
168     if (!exp) return FP_ILOGB0;
169     if (exp < 0x7C00) {
170         if (!(exp >= 10))
171             for (unsigned int m = (arg.get_bits() & 0x3FF); m <
0x200; m <= 1, --exp)
172                 ;
173         return exp - 15;
174     }
175     if (exp > 0x7C00) return FP_ILOGBNAN;
176     return INT_MAX;
177 }
178
179 inline float fdim(float x, float y) {
180     return ((x <= y) ? 0.0f : (x - y));
181 }
182
183 // Wrapper for binary half-precision needing specialization
184 // for individual argument types.
185 // tparam T first argument type
186 // tparam U first argument type
187 template <typename T, typename U>
188 struct binary_specialized {
189     static float fmin(float x, float y) {
190         if (isnan(x)) return y;
191         if (isnan(y)) return x;
192         return std::min(x, y);
193     }
194
195     static float fmax(float x, float y) {
196         if (isnan(x)) return y;
197         if (isnan(y)) return x;
198         return std::max(x, y);
199     }
200 };
201
202 template <>
203 struct binary_specialized<half, half> {
204     static half fmin(half x, half y) {
205         if (isnan(x)) return y;
206         if (isnan(y)) return x;
207         return ((signbit(x) ? (static_cast<int>(0x8000) -
x.get_bits()) : static_cast<int>(x.get_bits())) >
208             (signbit(y) ? (static_cast<int>(0x8000) -
y.get_bits()) : static_cast<int>(y.get_bits())))
209             ? y
210             : x;
211     }
212
213     static half fmax(half x, half y) {
214         if (isnan(x)) return y;
215         if (isnan(y)) return x;
216         return ((signbit(x) ? (static_cast<int>(0x8000) -
x.get_bits()) : static_cast<int>(x.get_bits())) <
217             (signbit(y) ? (static_cast<int>(0x8000) -
y.get_bits()) : static_cast<int>(y.get_bits())))
218             ? y
219             : x;
220     }
221 };
222
223 inline half fmax(half x, half y) {
224     return binary_specialized<half, half>::fmax(x, y);
225 }
226
227 inline float fmax(half x, float y) {
228     return binary_specialized<half, float>::fmax(x, y);
229 }
230
231 inline float fmax(float x, half y) {
232     return binary_specialized<float, half>::fmax(x, y);
233 }
234
235 inline float fmax(float x, float y) {
236     return binary_specialized<float, float>::fmax(x, y);
237 }
238
239 inline half fmin(half x, half y) {
240     return binary_specialized<half, half>::fmin(x, y);
241 }
242
243 inline float fmin(half x, float y) {
244     return binary_specialized<half, float>::fmin(x, y);
245 }
246
247 inline float fmin(float x, half y) {
248     return binary_specialized<float, half>::fmin(x, y);
249 }
250
251 inline float fmin(float x, float y) {
252     return binary_specialized<float, float>::fmin(x, y);
253 }
254
255 inline half fabs(half x) {
256     half t;
257     t.set_bits(x.get_bits() & 0x7FFF);
258     return t;
259 }
260
261 inline half abs(half x) {
262     half t;
263     t.set_bits(x.get_bits() & 0x7FFF);
264     return t;
265 }
266
267 inline half scalbln(half arg, long exp) {
268     long e = arg.get_bits() & 0x7C00;
269     if (e == 0x7C00) return arg;
270     unsigned int m = arg.get_bits() & 0x3FF;
271     if (e >= 10)
272         m |= 0x400;
273     else {
274         if (!m) return arg;
275         for (m <= 1; m < 0x400; m <= 1, --e)
276             ;
277     }
278     e += exp;
279     half::uint16 value = arg.get_bits() & 0x8000;
280     if (e > 30) {
281         value |= 0x7C00;
282     } else if (e > 0)
283         value |= (e << 10) | (m & 0x3FF);
284     else if (e > -11) {
285         if (half::round_style == std::round_to_nearest)
286             {
287                 m += 1 << -e;
288             }
289         #if HALF_ROUND_TIES_TO_EVEN
290             m -= (m >> (1 - e)) & 1;
291         #endif
292     }
293     value |= m >> (1 - e);
294     half t;
295     t.set_bits(value);
296     return t;
297 }

```

```

286
287 inline half ldexp(half arg, int exp) {
288     return scalbln(arg, exp);
289 }
290 inline half ldexp(float arg, int exp) {
291     return scalbln(arg, exp);
292 }
293
294 //-----
295
296 /// Convert half-precision floating point to integer.
297 /// \tparam R rounding mode to use,
298 /// \tparam E 'true' for round to even, 'false' for round
299 /// away from zero
300 /// \tparam T type to convert to (builtin integer type with
301 /// at least 16 bits precision, excluding any implicit sign
302 /// bits) \param value binary representation of half-precision value \return integral value
303 template <std::float_round_style R, bool E, typename T>
304 T half2int_impl(half::uint16 value) {
305     unsigned int e = value & 0x7FFF;
306     if (e >= 0x7C00) return (value & 0x8000) ? std::numeric_limits<T>::min() : std::numeric_limits<T>::max();
307     if (e < 0x3800) {
308         if (R == std::round_toward_infinity)
309             return T(-(value >> 15) & (e != 0));
310         else if (R == std::round_toward_neg_infinity)
311             return -T(value > 0x8000);
312         return T();
313     }
314     int m = (value & 0x3FF) | 0x400;
315     e >>= 10;
316     if (e < 25) {
317         if (R == std::round_indeterminate || R == std::round_toward_zero)
318             m >>= 25 - e;
319         else {
320             if (R == std::round_to_nearest)
321                 m = (1 << (24 - e)) - ((m >> (25 - e)) & 1);
322             else if (R == std::round_toward_infinity)
323                 m += ((value >> 15) - 1) & ((1 << (25 - e)) - 1);
324             else if (R == std::round_toward_neg_infinity)
325                 m += -(value >> 15) & ((1 << (25 - e)) - 1);
326             m >>= 25 - e;
327         }
328     } else
329         m <= e - 25;
330     // if(std::numeric_limits<T>::digits < 16)
331     //     return std::min(std::max(m, static_cast<int>(std::numeric_limits<T>::min())), static_cast<int>(std::numeric_limits<T>::max()));
332     return static_cast<T>((value & 0x8000) ? -m : m);
333 }
334
335 /// Convert half-precision floating point to integer.
336 /// \tparam R rounding mode to use,
337 /// \tparam T type to convert to (builtin integer type with
338 /// at least 16 bits precision, excluding any implicit sign
339 /// bits) \param value binary representation of half-precision value \return integral value
340 template <std::float_round_style R, typename T>
341 T half2int(half::uint16 value) {
342     return half2int_impl<R, HALF_ROUND_TIES_TO_EVEN, T>(value);
343 }
344
345 /// Convert half-precision floating point to integer using round-to-nearest-away-from-zero.
346 /// \tparam T type to convert to (builtin integer type with
347 /// at least 16 bits precision, excluding any implicit sign
348 /// bits) \param value binary representation of half-precision value \return integral value
349 template <typename T>
350 T half2int_up(half::uint16 value) {
351     return half2int_impl<std::round_to_nearest, 0, T>(value);
352 }
353
354 /// Round half-precision number to nearest integer value.
355 /// \tparam R rounding mode to use,
356 /// \tparam E 'true' for round to even, 'false' for round
357 /// away from zero
358 /// \param value binary representation of half-precision value
359 template <std::float_round_style R, bool E>
360 half::uint16 round_half_impl(half::uint16 value) {
361     unsigned int e = value & 0x7FFF;
362     half::uint16 result = value;
363     if (e < 0x3C00) {
364         result &= 0x8000;
365         if (R == std::round_to_nearest)
366             result |= 0x3C00U & -(e >= (0x3800 + E));
367     }
368     else if (R == std::round_toward_infinity)
369         result |= 0x3C00U & -(value >> 15) & (e != 0);
370     else if (R == std::round_toward_neg_infinity)
371         result |= 0x3C00U & -(value > 0x8000);
372     } else if (e < 0x6400) {
373         e = 25 - (e >> 10);
374         unsigned int mask = (1 << e) - 1;
375         if (R == std::round_to_nearest)
376             result += (1 << (e - 1)) - ((result >> e) & E);
377         else if (R == std::round_toward_infinity)
378             result += mask & ((value >> 15) - 1);
379         else if (R == std::round_toward_neg_infinity)
380             result += mask & -(value >> 15);
381         result &= ~mask;
382     }
383     return result;
384 }
385
386 template <std::float_round_style R>
387 half::uint16 round_half(half::uint16 value) {
388     return round_half_impl<R, HALF_ROUND_TIES_TO_EVEN>(value);
389 }
390
391 inline half::uint16 round_half_up(half::uint16 value) {
392     return round_half_impl<std::round_to_nearest, 0>(value);
393 }
394
395 inline half floor(half arg) {
396     half t;
397     t.set_bits(round_half<std::round_toward_neg_infinity>(arg.get_bits()));
398     return t;
399 }
400
401 inline half ceil(half arg) {
402     half t;
403     t.set_bits(round_half<std::round_toward_infinity>(arg.get_bits()));
404     return t;
405 }
406
407 inline half trunc(half arg) {
408     half t;
409     t.set_bits(round_half<std::round_toward_zero>(arg.get_bits()));
410     return t;
411 }
412
413 inline half round(half arg) {
414     half t;
415     t.set_bits(round_half_up(arg.get_bits()));
416     return t;
417 }
418
419 inline long lround(half arg) {
420     return half2int_up<long>(arg.get_bits());
421 }
422
423 inline long long llround(half arg) {
424     return half2int_up<long long>(arg.get_bits());
425 }
426
427 inline long long lrint(half arg) {
428     return half2int<half_round_style::round_style, long>(arg.get_bits());
429 }
430
431 inline long long llrint(half arg) {
432     return half2int<half_round_style::round_style, long long>(arg.get_bits());
433 }
434
435 #if HALF_ENABLE_CPP11_LONG_LONG
436 inline long long llround(half arg) {
437     return half2int_up<long long>(arg.get_bits());
438 }
439
440 inline long long llrint(half arg) {
441     return half2int<half_round_style::round_style, long long>(arg.get_bits());
442 }
443 #endif
444
445 #ifndef HLS_NO_XIL_FPO_LIB
446 inline float sqrt(half arg) {
447     return std::sqrt(arg);
448 }
449
450 inline half sqrt(half arg) {
451     return half::math_function_1arg(xip_fpo_sqrt, arg);
452 }
453 #endif
454
455 //-----

```

```

364     else if (R == std::round_toward_infinity)
365         result |= 0x3C00U & -(value >> 15) & (e != 0);
366     else if (R == std::round_toward_neg_infinity)
367         result |= 0x3C00U & -(value > 0x8000);
368     } else if (e < 0x6400) {
369         e = 25 - (e >> 10);
370         unsigned int mask = (1 << e) - 1;
371         if (R == std::round_to_nearest)
372             result += (1 << (e - 1)) - ((result >> e) & E);
373         else if (R == std::round_toward_infinity)
374             result += mask & ((value >> 15) - 1);
375         else if (R == std::round_toward_neg_infinity)
376             result += mask & -(value >> 15);
377         result &= ~mask;
378     }
379     return result;
380 }
381
382 template <std::float_round_style R>
383 half::uint16 round_half(half::uint16 value) {
384     return round_half_impl<R, HALF_ROUND_TIES_TO_EVEN>(value);
385 }
386
387 inline half::uint16 round_half_up(half::uint16 value) {
388     return round_half_impl<std::round_to_nearest, 0>(value);
389 }
390
391 inline half floor(half arg) {
392     half t;
393     t.set_bits(round_half<std::round_toward_neg_infinity>(arg.get_bits()));
394     return t;
395 }
396
397 inline half ceil(half arg) {
398     half t;
399     t.set_bits(round_half<std::round_toward_infinity>(arg.get_bits()));
400     return t;
401 }
402
403 inline half trunc(half arg) {
404     half t;
405     t.set_bits(round_half<std::round_toward_zero>(arg.get_bits()));
406     return t;
407 }
408
409 inline half round(half arg) {
410     half t;
411     t.set_bits(round_half_up(arg.get_bits()));
412     return t;
413 }
414
415 inline long lround(half arg) {
416     return half2int_up<long>(arg.get_bits());
417 }
418
419 inline long long llround(half arg) {
420     return half2int_up<long long>(arg.get_bits());
421 }
422
423 inline long long lrint(half arg) {
424     return half2int<half_round_style::round_style, long>(arg.get_bits());
425 }
426
427 inline long long llrint(half arg) {
428     return half2int<half_round_style::round_style, long long>(arg.get_bits());
429 }
430
431 #if HALF_ENABLE_CPP11_LONG_LONG
432 inline long long llround(half arg) {
433     return half2int_up<long long>(arg.get_bits());
434 }
435
436 inline long long llrint(half arg) {
437     return half2int<half_round_style::round_style, long long>(arg.get_bits());
438 }
439 #endif
440
441 #ifndef HLS_NO_XIL_FPO_LIB
442 inline float sqrt(half arg) {
443     return std::sqrt(arg);
444 }
445
446 inline half sqrt(half arg) {
447     return half::math_function_1arg(xip_fpo_sqrt, arg);
448 }
449 #endif
450
451 //-----

```

```

453 inline bool isunordered(half x, half y) {
454     return isnan(x) || isnan(y);
455 }
456
457 #ifndef HLS_NO_XIL_FPO_LIB
458 inline bool isequal(half x, half y) {
459     return (x.get_bits() == y.get_bits()) || !((x.get_bits() |
y.get_bits()) & 0x7FFF) && !isnan(x);
460 }
461 #else
462 template <typename T1, typename T2>
463 inline bool isequal(T1 x, T2 y) {
464     return half::binary_operator_compare(xip_fpo_equal, x,
y);
465 }
466 inline bool isequal(float x, half y) {
467     return HLS_FPO_EQUALF(x, y);
468 }
469 inline bool isequal(half x, float y) {
470     return HLS_FPO_EQUALF(x, y);
471 }
472 inline bool isequal(double x, half y) {
473     return HLS_FPO_EQUAL(x, y);
474 }
475 inline bool isequal(half x, double y) {
476     return HLS_FPO_EQUAL(x, y);
477 }
478 #endif
479
480 #ifndef HLS_NO_XIL_FPO_LIB
481 inline bool isnotequal(half x, half y) {
482     return (x.get_bits() != y.get_bits()) && ((x.get_bits() |
y.get_bits()) & 0x7FFF) || isnan(x);
483 }
484 #else
485 template <typename T1, typename T2>
486 inline bool isnotequal(T1 x, T2 y) {
487     return half::binary_operator_compare(xip_fpo_notequal, x,
y);
488 }
489 inline bool isnotequal(float x, half y) {
490     return HLS_FPO_NOTEQUALF(x, y);
491 }
492 inline bool isnotequal(half x, float y) {
493     return HLS_FPO_NOTEQUALF(x, y);
494 }
495 inline bool isnotequal(double x, half y) {
496     return HLS_FPO_NOTEQUAL(x, y);
497 }
498 inline bool isnotequal(half x, double y) {
499     return HLS_FPO_NOTEQUAL(x, y);
500 }
501 #endif
502
503 #ifndef HLS_NO_XIL_FPO_LIB
504 inline bool isgreater(half x, half y) {
505     return !isnan(x) && !isnan(y) &&
((signbit(x) ? (static_cast<int>(x.get_bits())) >
x.get_bits() : static_cast<int>(x.get_bits())) >
(signbit(y) ? (static_cast<int>(y.get_bits())) >
y.get_bits() : static_cast<int>(y.get_bits())));
506 }
507 #else
508 template <typename T1, typename T2>
509 inline bool isgreater(T1 x, T2 y) {
510     return half::binary_operator_compare(xip_fpo_greater, x,
y);
511 }
512 inline bool isgreater(float x, half y) {
513     return HLS_FPO_GREATERF(x, y);
514 }
515 inline bool isgreater(half x, float y) {
516     return HLS_FPO_GREATERF(x, y);
517 }
518 inline bool isgreater(double x, half y) {
519     return HLS_FPO_GREATER(x, y);
520 }
521 inline bool isgreater(half x, double y) {
522     return HLS_FPO_GREATER(x, y);
523 }
524 #endif
525
526 #ifndef HLS_NO_XIL_FPO_LIB
527 inline bool isgreaterequal(half x, half y) {
528     return !isnan(x) && !isnan(y) &&
((signbit(x) ? (static_cast<int>(x.get_bits())) >=
x.get_bits() : static_cast<int>(x.get_bits())) >=
(signbit(y) ? (static_cast<int>(y.get_bits())) >=
y.get_bits() : static_cast<int>(y.get_bits())));
529 }
530 #else
531 template <typename T1, typename T2>
532 inline bool isgreaterequal(T1 x, T2 y) {
533     return half::binary_operator_compare(xip_fpo_greaterequal, x,
y);
534 }
535 inline bool isgreaterequal(float x, half y) {
536     return HLS_FPO_GREATEREQUALF(x, y);
537 }
538 inline bool isgreaterequal(half x, float y) {
539     return HLS_FPO_GREATEREQUALF(x, y);
540 }
541 inline bool isgreaterequal(double x, half y) {
542     return HLS_FPO_GREATEREQUAL(x, y);
543 }
544 inline bool isgreaterequal(half x, double y) {
545     return HLS_FPO_GREATEREQUAL(x, y);
546 }
547 #endif
548
549 #ifndef HLS_NO_XIL_FPO_LIB
550 inline bool isless(half x, half y) {
551     return !isnan(x) && !isnan(y) &&
((signbit(x) ? (static_cast<int>(x.get_bits())) <
x.get_bits() : static_cast<int>(x.get_bits())) <
(signbit(y) ? (static_cast<int>(y.get_bits())) <
y.get_bits() : static_cast<int>(y.get_bits())));
552 }
553 #else
554 template <typename T1, typename T2>
555 inline bool isless(T1 x, T2 y) {
556     return half::binary_operator_compare(xip_fpo_less, x, y);
557 }
558 inline bool isless(float x, half y) {
559     return HLS_FPO_LESSF(x, y);
560 }
561 inline bool isless(half x, float y) {
562     return HLS_FPO_LESSF(x, y);
563 }
564 inline bool isless(double x, half y) {
565     return HLS_FPO_LESS(x, y);
566 }
567 inline bool isless(half x, double y) {
568     return HLS_FPO_LESS(x, y);
569 }
570 #endif
571
572 #ifndef HLS_NO_XIL_FPO_LIB
573 inline bool islessequal(half x, half y) {
574     return !isnan(x) && !isnan(y) &&
((signbit(x) ? (static_cast<int>(x.get_bits())) <=
x.get_bits() : static_cast<int>(x.get_bits())) <=
(signbit(y) ? (static_cast<int>(y.get_bits())) <=
y.get_bits() : static_cast<int>(y.get_bits())));
575 }
576 #else
577 template <typename T1, typename T2>
578 inline bool islessequal(T1 x, T2 y) {
579     return half::binary_operator_compare(xip_fpo_lessequal,
x, y);
580 }
581 inline bool islessequal(float x, half y) {
582     return HLS_FPO_LESSEQUALF(x, y);
583 }
584 inline bool islessequal(half x, float y) {
585     return HLS_FPO_LESSEQUALF(x, y);
586 }
587 inline bool islessequal(double x, half y) {
588     return HLS_FPO_LESSEQUAL(x, y);
589 }
590 inline bool islessequal(half x, double y) {
591     return HLS_FPO_LESSEQUAL(x, y);
592 }
593 #endif
594
595 #ifndef HLS_NO_XIL_FPO_LIB
596 inline bool islessgreater(half x, half y) {
597     if (isnan(x) || isnan(y)) return false;
598     return detail::isless(x, y) || detail::isgreater(x, y);
599 }
600 #endif
601
602 // namespace detail
603 using detail::fpclassify;
604 using detail::isfinite;
605 using detail::isgreater;
606 using detail::isgreaterequal;
607 using detail::isinf;
608 using detail::isless;
609 using detail::islessequal;
610 using detail::islessgreater;
611 using detail::isnan;
612 using detail::isnormal;
613 using detail::isunordered;
614 using detail::signbit;
615
616 #endif // __SYNTHESIS__
617
618 #endif // __HLS_HALF_UTILS_H__

```

```

541 }
542 inline bool isgreaterequal(half x, float y) {
543     return HLS_FPO_GREATEREQUAL(x, y);
544 }
545 inline bool isgreaterequal(double x, half y) {
546     return HLS_FPO_GREATEREQUAL(x, y);
547 }
548 inline bool isgreaterequal(half x, double y) {
549     return HLS_FPO_GREATEREQUAL(x, y);
550 }
551 #endif
552
553 #ifndef HLS_NO_XIL_FPO_LIB
554 inline bool isless(half x, half y) {
555     return !isnan(x) && !isnan(y) &&
((signbit(x) ? (static_cast<int>(x.get_bits())) <
x.get_bits() : static_cast<int>(x.get_bits())) <
(signbit(y) ? (static_cast<int>(y.get_bits())) <
y.get_bits() : static_cast<int>(y.get_bits())));
556 }
557 #else
558 template <typename T1, typename T2>
559 inline bool isless(T1 x, T2 y) {
560     return half::binary_operator_compare(xip_fpo_less, x, y);
561 }
562 inline bool isless(float x, half y) {
563     return HLS_FPO_LESSF(x, y);
564 }
565 inline bool isless(half x, float y) {
566     return HLS_FPO_LESSF(x, y);
567 }
568 inline bool isless(double x, half y) {
569     return HLS_FPO_LESS(x, y);
570 }
571 inline bool isless(half x, double y) {
572     return HLS_FPO_LESS(x, y);
573 }
574 #endif
575
576 #ifndef HLS_NO_XIL_FPO_LIB
577 inline bool islessequal(half x, half y) {
578     return !isnan(x) && !isnan(y) &&
((signbit(x) ? (static_cast<int>(x.get_bits())) <=
x.get_bits() : static_cast<int>(x.get_bits())) <=
(signbit(y) ? (static_cast<int>(y.get_bits())) <=
y.get_bits() : static_cast<int>(y.get_bits())));
579 }
580 #else
581 template <typename T1, typename T2>
582 inline bool islessequal(T1 x, T2 y) {
583     return half::binary_operator_compare(xip_fpo_lessequal,
x, y);
584 }
585 inline bool islessequal(float x, half y) {
586     return HLS_FPO_LESSEQUALF(x, y);
587 }
588 inline bool islessequal(half x, float y) {
589     return HLS_FPO_LESSEQUALF(x, y);
590 }
591 inline bool islessequal(double x, half y) {
592     return HLS_FPO_LESSEQUAL(x, y);
593 }
594 inline bool islessequal(half x, double y) {
595     return HLS_FPO_LESSEQUAL(x, y);
596 }
597 #endif
598
599 #ifndef HLS_NO_XIL_FPO_LIB
600 inline bool islessgreater(half x, half y) {
601     if (isnan(x) || isnan(y)) return false;
602     return detail::isless(x, y) || detail::isgreater(x, y);
603 }
604 #endif
605
606 // namespace detail
607 using detail::fpclassify;
608 using detail::isfinite;
609 using detail::isgreater;
610 using detail::isgreaterequal;
611 using detail::isinf;
612 using detail::isless;
613 using detail::islessequal;
614 using detail::islessgreater;
615 using detail::isnan;
616 using detail::isnormal;
617 using detail::isunordered;
618 using detail::signbit;
619
620 #endif // __SYNTHESIS__
621
622 #endif // __HLS_HALF_UTILS_H__

```

E.26 vendor/etc/hls_hotbm_apfixed.h (482 lines, 16,508 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5

```



```

6
7 /**
8  * @file hls_hotbm_apfixed.h
9  * An implementation of sin/cos, based on the HOTBM method
published by
10  * Detrey and deDinechin, "Floating-point Trigonometric Func-
tions for FPGAs"
11  */
12
13 #ifndef _X_HLS_HOTBM_APFIXED_H
14 #define _X_HLS_HOTBM_APFIXED_H
15
16 #include "ap_fixed.h"
17 #include "ap_int.h"
18 #include "utils/x_hls_defines.h"
19 #include "utils/x_hls_utils.h"
20
21 #ifndef M_PI
22 // Some compilers are stingy about giving us all these con-
stants that are usually in the C library.
23 #define M_PI 3.14159265358979323846
24 #endif
25
26 namespace hotbm_apfixed
27 {
28
29     namespace first_order_fixed_16{
30         extern const ap_fixed<19,2> sin_cos_K0[256];
31         extern const ap_fixed<11,2> sin_cos_K1[256];
32     };
33
34     namespace second_order_float{
35         extern const ap_fixed<29,0> cos_K0[128];
36         extern const ap_fixed<23,1> cos_K1[128];
37         extern const ap_fixed<15,0> cos_K2[128];
38         extern const ap_fixed<30,1> sin_K0[128];
39         extern const ap_fixed<21,-1> sin_K1[128];
40         extern const ap_fixed<13,-2> sin_K2[128];
41         extern const ap_ufixed<30,1> sin_cos_K0[256];
42         extern const ap_fixed<23,1> sin_cos_K1[256];
43         extern const ap_fixed<15,0> sin_cos_K2[256];
44     };
45
46     namespace fourth_order_double{
47         extern const ap_fixed<59,0> cos_K0[128];
48         extern const ap_fixed<53,1> cos_K1[128];
49         extern const ap_fixed<45,0> cos_K2[128];
50         extern const ap_fixed<35,-3> cos_K3[128];
51         extern const ap_fixed<27,-4> cos_K4[128];
52         extern const ap_fixed<60,1> sin_K0[128];
53         extern const ap_fixed<51,-1> sin_K1[128];
54         extern const ap_fixed<43,-2> sin_K2[128];
55         extern const ap_fixed<33,-5> sin_K3[128];
56         extern const ap_fixed<24,-7> sin_K4[128];
57         extern const ap_fixed<59,1> sin_cos_K0[256];
58         extern const ap_fixed<52,1> sin_cos_K1[256];
59         extern const ap_fixed<44,0> sin_cos_K2[256];
60         extern const ap_fixed<34,-3> sin_cos_K3[256];
61         extern const ap_fixed<26,-4> sin_cos_K4[256];
62     };
63
64     template<class T>
65     struct value_list{
66     };
67
68     template<>
69     struct value_list<double>{
70         typedef double T;
71         static const int g = 10; // guard bits during core
approx
72
73         static const int p1 = 61; // padding bits during range
reduction
74         static const int alpha = 7;
75         // The number of bits of B that are not used in the K1
computation
76         static const int beta1 = 0;
77         // The number of bits of B that are not used in the K2
computation
78         static const int beta2 = alpha;
79
80         static const int W_dout = fp_struct<T>::SIG_BITS+1+g;
81         static const int W_Mx = fp_struct<T>::SIG_BITS+1+g;
82
83         template <int Wx, int Wr>
84         static ap_ufixed<Wx, 1> sin_or_cos_approximation(bool
do_cos,
85
86         ap_uint<3> &k,
87
88         ap_ufixed<Wx, 0> &x,
89
90         ap_ufixed<Wr, 0> &Mx,
91
92         ap_int<fp_struct<double>::EXP_BITS> &Ex){
88

```

```

89         const bool swap_table[8] = {0,1,1,0,0,1,1,0};
90 #pragma HLS array_partition variable=swap_table complete
91 #pragma HLS BIND_STORAGE vari-
able=fourth_order_double::sin_cos_K0 type=ROM_1P impl=LUTRAM
92 #pragma HLS BIND_STORAGE vari-
able=fourth_order_double::sin_cos_K1 type=ROM_1P impl=LUTRAM
93 #pragma HLS BIND_STORAGE vari-
able=fourth_order_double::sin_cos_K2 type=ROM_1P impl=LUTRAM
94 #pragma HLS BIND_STORAGE vari-
able=fourth_order_double::sin_cos_K3 type=ROM_1P impl=LUTRAM
95 #pragma HLS BIND_STORAGE vari-
able=fourth_order_double::sin_cos_K4 type=ROM_1P impl=LUTRAM
96
97     _AP_UNUSED_PARAM(Ex);
98     ap_uint<alpha+1> A = x(x.wl()-x.iwl()-1,x.wl()-
x.iwl()-alpha);
99     ap_ufixed<Wx-alpha-beta1, -alpha> B = x;
100     ap_ufixed<Wx-alpha-beta2, -alpha> B_trunc = B;
101     ap_ufixed<Wx-2*alpha, -2*alpha> B_squared =
B_trunc*B_trunc;
102     ap_ufixed<Wx-3*alpha, -3*alpha> B_third_power =
B_trunc*B_squared;
103     ap_ufixed<Wx-4*alpha, -4*alpha> B_fourth_power =
B_squared*B_squared;
104
105     bool sin_basis = do_cos? swap_table[k]:!swap_t-
able[k];
106     A[alpha] = sin_basis; // cos and sin are concat-
entated.
107
108     ap_fixed<Wx, 1> t1 =
fourth_order_double::sin_cos_K0[A];
109     ap_fixed<Wx, 1> t2 =
B*fourth_order_double::sin_cos_K1[A];
110     ap_fixed<Wx, 1> t3 =
B_squared*fourth_order_double::sin_cos_K2[A];
111     ap_fixed<Wx, 1> t4 =
B_third_power*fourth_order_double::sin_cos_K3[A];
112     ap_fixed<Wx, 1> t5 =
B_fourth_power*fourth_order_double::sin_cos_K4[A];
113     ap_ufixed<Wx, 1> result = Mx*(t1+t2+t3+t4+t5);
114     return result;
115 }
116
117     template <int Wx, int Wr>
118     static void sincos_approximation(ap_uint<3> &k,
119
120     ap_ufixed<Wx, 0> &x,
121
122     ap_ufixed<Wr, 0> &Mx,
123
124     ap_int<fp_struct<double>::EXP_BITS> Ex,
125
126     ap_ufixed<Wr, 1> &sin_result,
127
128     ap_ufixed<Wr, 1> &cos_result
129 ) {
130
131 #pragma HLS BIND_STORAGE variable=fourth_order_double::cos_K0
type=ROM_1P impl=LUTRAM
132
133 #pragma HLS BIND_STORAGE variable=fourth_order_double::cos_K1
type=ROM_1P impl=LUTRAM
134
135 #pragma HLS BIND_STORAGE variable=fourth_order_double::cos_K2
type=ROM_1P impl=LUTRAM
136
137 #pragma HLS BIND_STORAGE variable=fourth_order_double::cos_K3
type=ROM_1P impl=LUTRAM
138
139 #pragma HLS BIND_STORAGE variable=fourth_order_double::cos_K4
type=ROM_1P impl=LUTRAM
140
141 #pragma HLS BIND_STORAGE variable=fourth_order_double::sin_K0
type=ROM_1P impl=LUTRAM
142
143 #pragma HLS BIND_STORAGE variable=fourth_order_double::sin_K1
type=ROM_1P impl=LUTRAM
144
145 #pragma HLS BIND_STORAGE variable=fourth_order_double::sin_K2
type=ROM_1P impl=LUTRAM
146
147 #pragma HLS BIND_STORAGE variable=fourth_order_double::sin_K3
type=ROM_1P impl=LUTRAM
148
149 #pragma HLS BIND_STORAGE variable=fourth_order_double::sin_K4
type=ROM_1P impl=LUTRAM
150
151     _AP_UNUSED_PARAM(k);
152     _AP_UNUSED_PARAM(Ex);
153     ap_uint<alpha> A = x(x.wl()-x.iwl()-1,
x.wl()-
x.iwl()-alpha);

```

```

140     ap_ufixed<Wx-alpha-beta1, -alpha> B = x;
141     ap_ufixed<Wx-alpha-beta2, -alpha> B_trunc = B;
142     ap_ufixed<Wx-2*alpha, -2*alpha> B_squared =
B_trunc*B_trunc;
143     ap_ufixed<Wx-3*alpha, -3*alpha> B_third_power =
B_trunc*B_squared;
144     ap_ufixed<Wx-4*alpha, -4*alpha> B_fourth_power =
B_squared*B_squared;
145
146     ap_fixed<Wx, 1> cos_t1 =
fourth_order_double::cos_K0[A];
147     ap_fixed<Wx, 1> cos_t2 =
B*fourth_order_double::cos_K1[A];
148     ap_fixed<Wx, 1> cos_t3 =
B_squared*fourth_order_double::cos_K2[A];
149     ap_fixed<Wx, 1> cos_t4 =
B_third_power*fourth_order_double::cos_K3[A];
150     ap_fixed<Wx, 1> cos_t5 =
B_fourth_power*fourth_order_double::cos_K4[A];
151     cos_result = 1 - (cos_t1 + cos_t2 + cos_t3 + cos_t4
+ cos_t5);
152
153     ap_fixed<Wx, 1> sin_t1 =
fourth_order_double::sin_K0[A];
154     ap_fixed<Wx, 1> sin_t2 =
B*fourth_order_double::sin_K1[A];
155     ap_fixed<Wx, 1> sin_t3 =
B_squared*fourth_order_double::sin_K2[A];
156     ap_fixed<Wx, 1> sin_t4 =
B_third_power*fourth_order_double::sin_K3[A];
157     ap_fixed<Wx, 1> sin_t5 =
B_fourth_power*fourth_order_double::sin_K4[A];
158
159     sin_result = Mx * (sin_t1 + sin_t2 + sin_t3 + sin_t4 +
sin_t5);
160 }
161 };
162
163 template<>
164 struct value_list<float> {
165     typedef float T;
166     static const int g = 5;
167     static const int p1 = 29;
168     static const int alpha = 7;
169     static const int beta1 = 0;
170     static const int beta2 = 7;
171
172     static const int W_dout = fp_struct<T>::SIG_BITS+1+g;
173     static const int W_Mx = fp_struct<T>::SIG_BITS+1+g;
174
175     template<int Wx, int Wr>
176     static ap_ufixed<Wx, 1> sin_or_cos_approximation(bool
do_cos,
177
178     ap_uint<3> &k,
179
180     ap_ufixed<Wx, 0> &x,
181
182     ap_ufixed<Wr, 0> &Mx,
183
184     ap_int<fp_struct<float>::EXP_BITS> &Ex){
185
186     const bool swap_table[8] = {0,1,1,0,0,1,1,0};
187
188     #pragma HLS array_partition variable=swap_table complete
189     #pragma HLS BIND_STORAGE variable=second_order_float::sin_cos_K0 type=ROM_1P impl=LUTRAM
190     #pragma HLS BIND_STORAGE variable=second_order_float::sin_cos_K1 type=ROM_1P impl=LUTRAM
191     #pragma HLS BIND_STORAGE variable=second_order_float::sin_cos_K2 type=ROM_1P impl=LUTRAM
192
193     _AP_UNUSED_PARAM(Ex);
194     ap_uint<alpha+1> A = x(x.wl()-x.iwl()-1,x.wl()-
x.iwl()-alpha);
195     ap_ufixed<Wx-alpha-beta1, -alpha> B = x;
196     ap_ufixed<Wx-alpha-beta2, -alpha> B_trunc = B;
197     ap_ufixed<Wx-2*alpha, -2*alpha> B_squared =
B_trunc*B_trunc;
198
199     bool sin_basis = do_cos? swap_table[k]:!swap_t-
able[k];
200     A[alpha] = sin_basis; // cos and sin are concat-
entated.
201
202     ap_fixed<Wx, 1> t1 =
second_order_float::sin_cos_K0[A];
203     ap_fixed<Wx, 1> t2 =
B*second_order_float::sin_cos_K1[A];
204     ap_fixed<Wx, 1> t3 =
B_squared*second_order_float::sin_cos_K2[A];
205     ap_ufixed<Wx, 1> result = (t1+t2+t3) * Mx;
206
207     return result;
208 }
209 }

```

```

205     template<int Wx, int Wr>
206     static void sincos_approximation(ap_uint<3> &k,
207
208     ap_ufixed<Wx, 0> &x,
209
210     ap_int<fp_struct<float>::EXP_BITS> &Ex,
211
212     ap_ufixed<Wr, 1> &sin_result,
213
214     ap_ufixed<Wr, 1> &cos_result
215 ) {
216     #pragma HLS BIND_STORAGE variable=second_order_float::cos_K0
type=ROM_1P impl=LUTRAM
217     #pragma HLS BIND_STORAGE variable=second_order_float::cos_K1
type=ROM_1P impl=LUTRAM
218     #pragma HLS BIND_STORAGE variable=second_order_float::cos_K2
type=ROM_1P impl=LUTRAM
219     #pragma HLS BIND_STORAGE variable=second_order_float::sin_K0
type=ROM_1P impl=LUTRAM
220     #pragma HLS BIND_STORAGE variable=second_order_float::sin_K1
type=ROM_1P impl=LUTRAM
221     #pragma HLS BIND_STORAGE variable=second_order_float::sin_K2
type=ROM_1P impl=LUTRAM
222
223     _AP_UNUSED_PARAM(k);
224     _AP_UNUSED_PARAM(Ex);
225     ap_uint<alpha> A = x(x.wl()-x.iwl()-1,
x.wl()-
x.iwl()-alpha);
226     ap_ufixed<Wx-alpha-beta1, -alpha> B = x;
227     ap_ufixed<Wx-alpha-beta2, -alpha> B_trunc = B;
228     ap_ufixed<Wx-2*alpha, -2*alpha> B_squared =
B_trunc*B_trunc;
229
230     // TABLES tables;
231     ap_ufixed<Wx, 1> cos_t1 =
second_order_float::cos_K0[A];
232     ap_ufixed<Wx, 1> cos_t2 =
B*second_order_float::cos_K1[A];
233     ap_ufixed<Wx, 1> cos_t3 =
B_squared*second_order_float::cos_K2[A];
234     cos_result = 1 - (cos_t1 + cos_t2 + cos_t3);
235
236     ap_fixed<Wx, 1> sin_t1 =
second_order_float::sin_K0[A];
237     ap_fixed<Wx, 1> sin_t2 =
B*second_order_float::sin_K1[A];
238     ap_fixed<Wx, 1> sin_t3 =
B_squared*second_order_float::sin_K2[A];
239     sin_result = Mx * (sin_t1 + sin_t2 + sin_t3);
240
241 }
242
243 struct sin_cos_fixed_16 {
244     typedef half T;
245     static const int g = 3;
246     static const int p1 = 12;
247     static const int alpha = 8;
248     static const int beta1 = 4;
249
250     template<int W_dout>
251     static ap_ufixed<W_dout, 1> sin_or_cos_approxima-
tion(bool do_cos,
252
253     ap_uint<2> &k,
254
255     ap_ufixed<W_dout, 0> &x){
256
257     #pragma HLS BIND_STORAGE variable=first_order_fixed_16::sin_cos_K0 type=ROM_1P impl=LUTRAM
258     #pragma HLS BIND_STORAGE variable=first_order_fixed_16::sin_cos_K1 type=ROM_1P impl=LUTRAM
259     _AP_UNUSED_PARAM(do_cos);
260     _AP_UNUSED_PARAM(k);
261     ap_uint<alpha> A = x(x.wl()-x.iwl()-1,x.wl()-
x.iwl()-alpha);
262
263     const int BW = (W_dout-alpha-beta1)>1 ? W_dout-al-
pha-beta1 : 2;
264     ap_ufixed<BW, -alpha> B = x;
265
266     ap_fixed<W_dout, 1> t1 =
first_order_fixed_16::sin_cos_K0[A];
267     ap_fixed<W_dout, 1> t2 =
B*first_order_fixed_16::sin_cos_K1[A];
268     ap_ufixed<W_dout, 1> result = (t1+t2);
269

```



```

268         return result;
269     };
270 };
271
272 template<int W, int I>
273 ap_fixed<W,2>
274 sinf_or_cosf(ap_fixed<W,I> t_in, bool do_cos, bool do_pi,
275
276             typename hls::enable_if<(W > 16) &&
277 (W <= 32), bool>::type dummy = true)
278 {
279     const int W0 = W;
280     const int g = 3;
281     const bool swap_table[8] = {0,1,1,0,0,1,1,0};
282     const bool neg_sin_table[16] = {0,0,1,0,1,1,0,1,
1,0,1,1,0,1,0,0};
283     const bool neg_cos_table[16] = {0,0,0,1,1,1,1,0,
0,1,1,1,0,0,0};
284 #pragma HLS array_partition variable=swap_table complete
285 #pragma HLS array_partition variable=neg_sin_table complete
286 #pragma HLS array_partition variable=neg_cos_table complete
287     _AP_UNUSED_PARAM(dummy);
288     ap_uint<3> k;
289     ap_ufixed<W0+1+g, 0> x;
290     ap_ufixed<W0+1+g, 0> Mx;
291     ap_int<fp_struct<float>::EXP_BITS> Ex = 0;
292     int exactly_pi_over_4 = 0;
293     bool is_negative = t_in[t_in.wl()-1];
294     ap_ufixed<W, I> din = is_negative ? ap_fixed<W, I>(-
t_in) : t_in;
295     if(do_pi) {
296         k = din*4;
297         x = din*4;
298         ap_ufixed<W, 1> Mx_bits_extended = din*4;
299 #if !defined(__SYNTHESIS__) && defined(_HLSCLIB_DEBUG_)
300         std::cout << "sin_cos: Mx_bits_extended = " <<
Mx_bits_extended.to_string(2) << "\n";
301 #endif
302         exactly_pi_over_4 = (Mx_bits_extended == 1);
303     } else {
304         ap_ufixed<W0+1+g+3, 3> Wx = din * ap_ufixed<W+1+g,
2>(4/M_PI);
305         k = Wx;
306         x = Wx;
307     }
308     #if !defined(__SYNTHESIS__) && defined(_HLSCLIB_DEBUG_)
309     std::cout << "sin_cos: k = " << k.to_string(2) <<
"\n";
310 #endif
311     // If we're in an odd quadrant, then the base func-
312     tions need to be flipped.
313     if(k[0] == 1) {
314         x = 1-x;
315     }
316     Mx = x;
317     bool cos_basis = do_cos ? !swap_table[k] : swap_table[k];
318     if(cos_basis) {
319         // In this case, we are using cos() as the basis, so
320         we don't
321         // multiply by Mx*2^Ex.
322         Mx = ap_ufixed<value_list<float>::W_dout, 0, AP_RND,
AP_SAT>(1);
323         Ex = 0;
324     }
325     ap_ufixed<W0+1+g, 1> result =
value_list<float>::sin_or_cos_approximation(do_cos,k,x,Mx,Ex);
326     if(exactly_pi_over_4) {
327         result = 0.7071067811865475244008443;
328     }
329     ap_uint<4> index = k;
330     index[3] = is_negative;
331     if(cos_basis ? neg_cos_table[index] : neg_sin_t-
able[index]) {
332         return -result;
333     } else {
334         return result;
335     }
336 }
337
338 template<int W, int I>
339 ap_fixed<W,2>
340 sinf_or_cosf(ap_fixed<W,I> t_in, bool do_cos, bool do_pi,
341
342             typename hls::enable_if<(W > 32) &&
343 (W <= 64), bool>::type dummy = true)
344 {
345     const int W0 = W;

```

```

351     const int g = 3;
352
353     const bool swap_table[8] = {0,1,1,0,0,1,1,0};
354     const bool neg_sin_table[16] = {0,0,1,0,1,1,0,1,
1,0,1,1,0,1,0,0};
355     const bool neg_cos_table[16] = {0,0,0,1,1,1,1,0,
0,1,1,1,0,0,0};
356 #pragma HLS array_partition variable=swap_table complete
357 #pragma HLS array_partition variable=neg_sin_table complete
358 #pragma HLS array_partition variable=neg_cos_table complete
359     _AP_UNUSED_PARAM(dummy);
360     ap_uint<3> k;
361     ap_ufixed<W0+1+g, 0> x;
362     ap_ufixed<W0+1+g, 0> Mx;
363     ap_int<fp_struct<double>::EXP_BITS> Ex = 0;
364     int exactly_pi_over_4 = 0;
365     bool is_negative = t_in[t_in.wl()-1];
366     ap_ufixed<W, I> din = is_negative ? ap_fixed<W, I>(-
t_in) : t_in;
367     if(do_pi) {
368         k = din*4;
369         x = din*4;
370         ap_ufixed<W, 1> Mx_bits_extended = din*4;
371 #if !defined(__SYNTHESIS__) && defined(_HLSCLIB_DEBUG_)
372         std::cout << "sin_cos: Mx_bits_extended = " <<
Mx_bits_extended.to_string(2) << "\n";
373 #endif
374         exactly_pi_over_4 = (Mx_bits_extended == 1);
375     } else {
376         ap_ufixed<W0+1+g+3, 3> Wx = din * ap_ufixed<W+1+g,
2>(4/M_PI);
377         k = Wx;
378         x = Wx;
379     }
380     #if !defined(__SYNTHESIS__) && defined(_HLSCLIB_DEBUG_)
381     std::cout << "sin_cos: k = " << k.to_string(2) <<
"\n";
382 #endif
383     // If we're in an odd quadrant, then the base func-
384     tions need to be flipped.
385     if(k[0] == 1) {
386         x = 1-x;
387     }
388     Mx = x;
389     bool cos_basis = do_cos ? !swap_table[k] : swap_table[k];
390     if(cos_basis) {
391         // In this case, we are using cos() as the basis, so
392         we don't
393         // multiply by Mx*2^Ex.
394         Mx = ap_ufixed<value_list<double>::W_dout, 0,
AP_RND, AP_SAT>(1);
395         Ex = 0;
396     }
397     ap_ufixed<W0+1+g, 1> result =
value_list<double>::sin_or_cos_approximation(do_cos,k,x,Mx,Ex);
398     if(exactly_pi_over_4) {
399         result = 0.7071067811865475244008443;
400     }
401     ap_uint<4> index = k;
402     index[3] = is_negative;
403     if(cos_basis ? neg_cos_table[index] : neg_sin_t-
able[index]) {
404         return -result;
405     } else {
406         return result;
407     }
408 }
409
410 template<int W, int I>
411 ap_fixed<W,2> // Result has same number of fractional bits
412 as the input.
413 sinf_or_cosf(ap_fixed<W,I> t_in, bool do_cos, bool do_pi,
414
415             typename hls::enable_if<W <= 16,
bool>::type dummy = true)
416 {
417     const int W0 = W;
418     const int g = 4;
419     const bool neg_sin_table[8] = {0,0,1,1, 1,1,0,0};
420     const bool neg_cos_table[8] = {0,1,1,0, 0,1,1,0};
421 #pragma HLS array_partition variable=neg_sin_table complete
422 #pragma HLS array_partition variable=neg_cos_table complete
423     _AP_UNUSED_PARAM(dummy);
424     ap_uint<2> k;
425     ap_ufixed<W0+1+g, 0> x;
426     bool is_negative = t_in[t_in.wl()-1];
427     ap_ufixed<W, I> din = is_negative ? ap_fixed<W, I>(-
t_in) : t_in;

```

```

436     if(do_pi) {
437         k = din*2;
438         x = din*2;
439         ap_ufixed<W, 1> Mx_bits_extended = din*2;
440     }
441     else {
442         ap_ufixed<W0+1+g+2, 2> Wx = din * ap_ufixed<W+1+g,
2>(2/M_PI);
444         k = Wx;
445         x = Wx;
446     }
447     bool is_special_case = false;
448     if((k[0] == 1) ^ !do_cos) {
449         ap_ufixed<W0+1+g+1, 1> Wx = 1-x;
450         x = Wx;
451         is_special_case = (Wx.wl()-1) == 1;
452     }
453     ap_ufixed<W0+1+g, 1> result =
sin_cos_fixed_i6::sin_or_cos_approximation(do_cos,k,x);
457     if(is_special_case) {
458         ap_uint<3> index = k;
459         index[2] = do_cos;
460         const bool special_case_table[8] =
{0,1,0,1,1,0,1,0};
462         result(result.wl()-2,0) = 0;
463         result[result.wl()-1] = special_case_table[index];
464     }
465     {
466         ap_uint<3> index = k;
467         index[2] = is_negative;
468         if(do_cos ? neg_cos_table[index] : neg_sin_t-
able[index]) {
469             return -result;
470         } else {
471             return result;
472         }
473     }
474     };
475     };
476     };
477     };
478     };
479     };
480 #endif
481
482

```

E.27 vendor/etc/hls_hypot_apfixed.h (206 lines, 6,908 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 /**
7  * @file hls_hypot_apfixed.h
8  * @brief Implementation of hypot for ap_fixed data type.
9  */
10 #ifndef _X_HLS_HYPOT_FIXED_H_
11 #define _X_HLS_HYPOT_FIXED_H_
12 #include "ap_fixed.h"
13 #include "ap_int.h"
14 #include "etc/hls_round_copysign_apfixed.h"
15 namespace hypot_internal {
16 template<int W, int I>
17 ap_fixed<W, I> hypot_fixed(ap_fixed<W, I> x, ap_fixed<W, I>
y) {
18     const static int F = W - I;
19     if(I > 33) return 0;
20     else if(F > 32) return 0;
21
22     #ifndef __SYNTHESIS__
23     #ifdef HLS_DEBUG
24         std::cout<<"W="<<W<<"", I="<<I<<std::endl;
25         std::cout<<"(x)="<<x.to_string()<<std::endl;
26         std::cout<<"(y)="<<y.to_string()<<std::endl;
27     #endif
28     #endif
29     if(x==0) return fabs_fixed(y);
30     if(y==0) return fabs_fixed(x);
31
32     ap_ufixed<2*W, I*2> x_sq = x * x; //square of x
33     ap_ufixed<2*W, I*2> y_sq = y * y; // square of y
34     ap_ufixed<2*W, I*2> xy_sq = x_sq + y_sq;
35     #ifndef __SYNTHESIS__
36     #ifdef HLS_DEBUG
37         std::cout<<"square(x)="<<x_sq.to_string()<<std::endl;
38         std::cout<<"square(y)="<<y_sq.to_string()<<std::endl;
39         std::cout<<"square(x) + square(y)
="<<xy_sq.to_string()<<std::endl;
40     #endif
41     #endif
42     //the following is optimized sqrt function.

```

```

43     const static int prcs = (F+1)*2;
44     const static int msbr = I > 0? I : 1;
45     ap_ufixed<msbr, msbr> res_I = 0;
46     ap_ufixed<2*msbr+1,2*msbr+1> x_l_I = 0;
47     if(I>0) {
48         x_l_I = xy_sq(2*W-1, 2*W-2*I);
49         for(int pos = msbr - 1; pos >= 0; pos--) {
50             ap_ufixed<2*msbr,2*msbr> mul_I = 0;
51             #ifndef __SYNTHESIS__
52             assert(res_I(pos, 0)==0);
53             #endif
54             mul_I(msbr+pos, 2*pos+1) = res_I(msbr-1, pos);
55             mul_I[2*pos] = 1;
56             if(x_l_I(msbr+pos+1, 2*pos) >= mul_I(msbr+pos,
2*pos)){
57             #ifndef __SYNTHESIS__
58                 assert(x_l_I >= mul_I);
59             #endif
60             ap_ufixed<2*msbr+1, 2*msbr+1> x_l_I_ = x_l_I;
61             x_l_I(msbr+pos+1, 2*pos) =
x_l_I(msbr+pos+1,2*pos) - mul_I(msbr+pos, 2*pos);
62             #ifndef __SYNTHESIS__
63             assert(x_l_I == x_l_I_ - mul_I);
64             #endif
65             res_I[pos] = 1;
66         }
67     }
68     ap_ufixed<msbr+1, msbr+1> x_l_I_s = x_l_I;
69     #ifndef __SYNTHESIS__
70     assert(x_l_I_s == x_l_I);
71     #endif
72     #ifdef HLS_DEBUG
73         std::cout<<"res_I="<<res_I.to_string()<<std::endl;
74         std::cout<<"x_l_I ="<<x_l_I.to_string()<<std::endl;
75         std::cout<<"x_l_I_s="<<x_l_I_s.to_string()<<std::endl;
76     #endif
77     #endif
78     ap_ufixed<prcs/2,0> res_F = 0;
79     ap_ufixed<prcs/2,0> x_l_FH = xy_sq;
80     ap_ufixed<prcs/2, -prcs/2> x_l_FL = xy_sq;
81     for(int pos = -1; pos >= -F-1; pos--) {
82         ap_ufixed<msbr+prcs, msbr> mul = 0;
83         #ifndef __SYNTHESIS__
84         assert(res_F(pos+prcs/2,0)==0);
85         #endif
86         mul(msbr+pos+prcs, pos+1+prcs) = res_I(msbr-1,
0);
87         mul(pos+prcs, 2*pos+1+prcs) = res_F(prcs/
2-1,pos+prcs/2);
88         mul[2*pos+prcs] = 1;
89
90         ap_ufixed<msbr, msbr> mul_I = 0;
91         mul_I = mul;
92         ap_ufixed<prcs/2, 0> mul_FH = mul;
93         ap_ufixed<prcs/2, -prcs/2> mul_FL = mul;
94
95         #ifndef __SYNTHESIS__
96         #ifdef HLS_DEBUG
97             std::cout<<"res_I="<<res_I.to_string()<<std::endl;
98             std::cout<<"res_F="<<res_F.to_string()<<std::endl;
99             std::cout<<"mul="<<mul.to_string()<<std::endl;
100             std::cout<<"pos="<<pos<<std::endl;
101         #endif
102         std::cout<<"x_l_I_s="<<x_l_I_s.to_string()<<std::endl;
103         std::cout<<"x_l_FH="<<x_l_FH.to_string()<<std::endl;
104         std::cout<<"x_l_FL="<<x_l_FL.to_string()<<std::endl;
105         std::cout<<"mul_I="<<mul_I.to_string()<<std::endl;
106         std::cout<<"mul_FH="<<mul_FH.to_string()<<std::endl;
107         std::cout<<"mul_FL="<<mul_FL.to_string()<<std::endl;
108     #endif
109     #endif
110     ap_ufixed<msbr+prcs+1, msbr+1> x_l_1 = 0;
111     x_l_1(prcs+msbr, prcs) = x_l_I_s (msbr,
0);
112     x_l_1(prcs-1, prcs/2) = x_l_FH(prcs/2-1,
0);
113     x_l_1(prcs/2-1, 0) = x_l_FL(prcs/2-1,
0);
114     #ifndef __SYNTHESIS__
115     #ifdef HLS_DEBUG
116         std::cout<<"x_l_1="<<x_l_1.to_string()<<std::endl;
117         std::cout<<"mul="<<mul.to_string()<<std::endl;
118     #endif
119     #endif
120     assert(x_l_1 >= mul);
121     ap_ufixed<prcs/2+1, -prcs/2+1> x_l_FL_1 = x_l_FL;
122     //carry bit
123     if(x_l_FL < mul_FL) x_l_FL_1[prcs/2] =
1;
124     x_l_FL_1 -= mul_FL;
125     #ifndef __SYNTHESIS__
126     assert(!x_l_FL_1[prcs/2]);
127     #endif

```

```

128     ap_ufixed<prcs/2+1,1> x_l_FH_1 = x_l_FH;
129     //carry bit
130     if((x_l_FH < mul_FH) || ((x_l_FH == mul_FH) &&
(x_l_FL < mul_FL))) x_l_FH_1[prcs/2] = 1;
131     ap_ufixed<1, -prcs/2+1> delta = 0;
132     if(x_l_FL < mul_FL) delta[0] = 1;
133     //subtract carry bit
134     x_l_FH_1 -= delta;
135     x_l_FH_1 -= mul_FH;
136     //subtract carry bit
137     if((x_l_FH < mul_FH) || ((x_l_FH == mul_FH) &&
(x_l_FL < mul_FL))) x_l_I_s--;
138     x_l_I_s -= mul_I;
139     x_l_FH = x_l_FH_1;
140     x_l_FL = x_l_FL_1;
141 #ifndef __SYNTHESIS__
142     ap_ufixed<msbr+prcs+1, msbr+1> x_l_2 = 0;
143     x_l_2[msbr+prcs, prcs] = x_l_I_s[msbr, 0];
144     x_l_2[prcs-1, prcs/2] = x_l_FH[prcs/2-1, 0];
145     x_l_2[prcs/2-1, 0] = x_l_FL[prcs/2-1, 0];
146     assert(x_l_2 == x_l_1 - mul);
147 #endif
148     res_F[pos+prcs/2] = 1;
149 }
150 }
151 ap_ufixed<prcs/2+1, 1> res_F_1 = res_F;
152 //rounding the result
153 ap_ufixed<1, -prcs/2+1> delta = 0;
154 delta[0] = 1;
155 res_F_1 += delta;
156 if(res_F_1[prcs/2]) res_I++;
157 ap_ufixed<msbr+prcs/2, msbr> res = 0;
158 res[msbr+prcs/2-1, prcs/2] = res_I[msbr-1,0];
159 res[prcs/2-1, 0] = res_F[prcs/2-1, 0];
160 #ifndef __SYNTHESIS__
161 #ifdef HLS_DEBUG
162     std::cout<<"res="<<res.to_string()<<std::endl;
163 #endif
164 #endif
165 //overflow
166 ap_fixed<W,I> r = 0;
167 if(res[msbr+prcs/2-1]) {
168     //return max
169     for(int i = 0; i < W - 1; ++i) {
170         #pragma HLS unroll
171         r[i] = 1;
172     }
173 } else {
174     r = res;
175 }
176 #ifndef __SYNTHESIS__
177 #ifdef HLS_DEBUG
178     std::cout<<"r="<<r.to_string()<<std::endl;
179 #endif
180 #endif
181 return r;
182 }
183 template<int W, int I>
184 ap_ufixed<W, I> hypot_fixed(ap_ufixed<W, I> x, ap_fixed<W, I>
y) {
185     ap_fixed<W+1, I+1> xi = x;
186     ap_fixed<W+1, I+1> yi = y;
187     return hypot_fixed(xi, yi);
188 }
189 template<int I>
190 ap_int<I> hypot_fixed(ap_int<I> x, ap_int<I> y) {
191     ap_fixed<I, I> xi = x;
192     ap_fixed<I, I> yi = y;
193     return hypot_fixed(xi, yi);
194 }
195 template<int I>
196 ap_uint<I> hypot_fixed(ap_uint<I> x, ap_uint<I> y) {
197     ap_fixed<I+1, I+1> xi = x;
198     ap_fixed<I+1, I+1> yi = y;
199     return hypot_fixed(xi, yi);
200 }
201 }
202 #endif
203
204
205
206

```

E.28 vendor/etc/hls_isequal_isnotequal_apfixed.h (145 lines, 2,915 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_isequal_isnotequal_APFIXED_H_
7 #define _X_HLS_isequal_isnotequal_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "stdint.h"
10 // #include "hls_fpo.h"

```

```

11 // #include "hls_half.h"
12
13 ///isequal apfixed a & apfixed b
14 template<int W_, int I_>
15 bool generic_isequal(ap_fixed<W_, I_> x, ap_fixed<W_, I_> y)
16 {
17     #pragma HLS pipeline
18     return (x==y);
19 }
20
21 ///isnotequal apfixed a & apfixed b
22 template<int W_, int I_>
23 bool generic_isnotequal(ap_fixed<W_, I_> x, ap_fixed<W_, I_> y)
24 {
25     #pragma HLS pipeline
26     return (!(x==y));
27 }
28
29 ///isequal ap_ufixed a & ap_fixed b
30 template<int W_, int I_>
31 bool generic_isequal(ap_ufixed<W_, I_> x, ap_fixed<W_, I_> y)
32 {
33     #pragma HLS pipeline
34     return (x==y);
35 }
36
37 ///isnotequal ap_ufixed a & ap_fixed b
38 template<int W_, int I_>
39 bool generic_isnotequal(ap_ufixed<W_, I_> x, ap_fixed<W_, I_> y)
40 {
41     #pragma HLS pipeline
42     return (!(x==y));
43 }
44
45 ///isequal ap_fixed a & ap_ufixed b
46 template<int W_, int I_>
47 bool generic_isequal(ap_fixed<W_, I_> x, ap_ufixed<W_, I_> y)
48 {
49     #pragma HLS pipeline
50     return (x==y);
51 }
52
53 ///isnotequal ap_fixed a & ap_ufixed b
54 template<int W_, int I_>
55 bool generic_isnotequal(ap_fixed<W_, I_> x, ap_ufixed<W_, I_> y)
56 {
57     #pragma HLS pipeline
58     return (!(x==y));
59 }
60
61 ///isequal ap_ufixed a & ap_ufixed b
62 template<int W_, int I_>
63 bool generic_isequal(ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y)
64 {
65     #pragma HLS pipeline
66     return (x==y);
67 }
68
69 ///isnotequal ap_ufixed a & ap_ufixed b
70 template<int W_, int I_>
71 bool generic_isnotequal(ap_ufixed<W_, I_> x, ap_ufixed<W_, I_>
y)
72 {
73     #pragma HLS pipeline
74     return (!(x==y));
75 }
76
77
78 ///isequal apint a & apint b
79 template<int I_>
80 bool generic_isequal(ap_int<I_> x, ap_int<I_> y)
81 {
82     #pragma HLS pipeline
83     return (x==y);
84 }
85
86 ///isnotequal apint a & apint b
87 template<int I_>
88 bool generic_isnotequal(ap_int<I_> x, ap_int<I_> y)
89 {
90     #pragma HLS pipeline
91     return (!(x==y));
92 }
93
94 ///isequal ap_uint a & ap_int b
95 template<int I_>
96 bool generic_isequal(ap_uint<I_> x, ap_int<I_> y)
97 {
98     #pragma HLS pipeline
99     return (x==y);
100 }
101
102 ///isnotequal ap_uint a & ap_int b
103 template<int I_>
104 bool generic_isnotequal(ap_uint<I_> x, ap_int<I_> y)
105 {
106     #pragma HLS pipeline
107     return (!(x==y));
108 }
109

```

```

110 ///isequal ap_int a & ap_uint b
111 template <int I>
112 bool generic_isequal(ap_int<I> x, ap_uint<I> y)
113 {
114 #pragma HLS pipeline
115     return (x==y);
116 }
117
118 ///isnotequal ap_int a & ap_uint b
119 template <int I>
120 bool generic_isnotequal(ap_int<I> x, ap_uint<I> y)
121 {
122 #pragma HLS pipeline
123     return (!(x==y));
124 }
125
126 ///isequal ap_uint a & ap_uint b
127 template <int I>
128 bool generic_isequal(ap_uint<I> x, ap_uint<I> y)
129 {
130 #pragma HLS pipeline
131     return (x==y);
132 }
133
134 ///isnotequal ap_uint a & ap_uint b
135 template <int I>
136 bool generic_isnotequal(ap_uint<I> x, ap_uint<I> y)
137 {
138 #pragma HLS pipeline
139     return (!(x==y));
140 }
141
142 #endif
143
144
145

```

E.29 vendor/etc/hls_ldexp_apfixed.h (101 lines, 2,061 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3

```

```

4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_LDEXP_APFIXED_H_
7 #define _X_HLS_LDEXP_APFIXED_H_
8 #include "ap_fixed.h"
9
10 namespace hls_ldexp {
11
12 template<int W, int I>
13 ap_fixed<W,I> ldexp(ap_fixed<W,I> x, int exp) {
14     ap_fixed<W,I> result;
15     if(exp >= W || exp <= -W) {
16         result = 0;
17     } else {
18         result = x << exp;
19     }
20     return result;
21 }
22
23 template<int W, int I>
24 ap_ufixed<W,I> ldexp(ap_ufixed<W,I> x, int exp) {
25     ap_fixed<W+1,I+1> xf = x;
26     return ldexp(xf, exp);
27 }
28
29 template<int I>
30 ap_int<I> ldexp(ap_int<I> x, int exp) {
31     ap_fixed<I,I> xf = x;
32     return ldexp(xf, exp);
33 }
34
35 template<int I>
36 ap_uint<I> ldexp(ap_uint<I> x, int exp) {
37     ap_fixed<I+1,I+1> xf = x;
38     return ldexp(xf, exp);
39 }
40
41 template<int W, int I>
42 ap_fixed<W,I> scalbn(ap_fixed<W,I> x, int n) {
43     ap_fixed<W,I> result;
44     if(n >= W || n <= -W) {
45         result = 0;
46     } else {
47         result = x << n;
48     }
49     return result;
50 }
51
52 template<int W, int I>
53 ap_ufixed<W,I> scalbn(ap_ufixed<W,I> x, int n) {
54     ap_fixed<W+1,I+1> xf = x;
55     return scalbn(xf, n);
56 }
57
58 template<int I>
59 ap_int<I> scalbn(ap_int<I> x, int n) {

```

```

60     ap_fixed<I,I> xf = x;
61     return scalbn(xf, n);
62 }
63
64 template<int I>
65 ap_uint<I> scalbn(ap_uint<I> x, int n) {
66     ap_fixed<I+1,I+1> xf = x;
67     return scalbn(xf, n);
68 }
69
70 template<int W, int I>
71 ap_fixed<W,I> scalbn(ap_fixed<W,I> x, long int n) {
72     ap_fixed<W,I> result;
73     if(n >= W || n <= -W) {
74         result = 0;
75     } else {
76         result = x << n;
77     }
78     return result;
79 }
80
81 template<int W, int I>
82 ap_ufixed<W,I> scalbn(ap_ufixed<W,I> x, long int n) {
83     ap_fixed<W+1,I+1> xf = x;
84     return scalbn(xf, n);
85 }
86
87 template<int I>
88 ap_int<I> scalbn(ap_int<I> x, long int n) {
89     ap_fixed<I,I> xf = x;
90     return scalbn(xf, n);
91 }
92
93 template<int I>
94 ap_uint<I> scalbn(ap_uint<I> x, long int n) {
95     ap_fixed<I+1,I+1> xf = x;
96     return scalbn(xf, n);
97 }
98
99 }
100
101 #endif

```

E.30 vendor/etc/hls_lgamma_apfixed.h (66 lines, 1,878 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3

```

```

4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_LGAMMA_APFIXED_H_
7 #define _X_HLS_LGAMMA_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10 #include "utils/x_hls_defines.h"
11 #include "utils/x_hls_traits.h"
12 #include "utils/x_hls_utils.h"
13
14 namespace lgamma_fixed {
15
16 template <int W, int I> ap_fixed<W, I>
lgamma_generic(ap_fixed<W, I> x_fixed) {
17 #pragma HLS pipeline
18
19     double x = x_fixed.to_double();
20     double r = hls::lgamma(x);
21     fp_struct<double> out(r);
22     ap_ufixed<W+1, I+1> ret = 1;
23     int F = W - I;
24     if (F > 52) F = 52;
25     if (W > I) ret(F - 1, 0) = out.sig(52 - 1, 52 - F);
26     ret = (ret << (out.exp - fp_struct<double>::EXP_BIAS));
27     return ap_fixed<W, I>(ret);
28 }
29
30 template <int W, int I> ap_ufixed<W, I>
lgamma_generic(ap_ufixed<W, I> x) {
31     ap_fixed<W + 1, I + 1> xf = x;
32     return lgamma_generic(xf);
33 }
34
35 template <int I> ap_int<I> lgamma_generic(ap_int<I> x) {
36     ap_fixed<I, I> xf = x;
37     return lgamma_generic(xf);
38 }
39
40 template <int I> ap_uint<I> lgamma_generic(ap_uint<I> x) {
41     ap_fixed<I_ + 1, I_ + 1> xf = x;
42     return lgamma_generic(xf);
43 }
44
45 //lgamma_r
46 template <int W, int I> ap_fixed<W, I>
lgamma_r_generic(ap_fixed<W, I> x, int *signgamp) {
47     *signgamp = 1;
48     return lgamma_generic(x);
49 }
50
51 template <int W, int I> ap_ufixed<W, I>
lgamma_r_generic(ap_ufixed<W, I> x, int *signgamp) {

```

```

50     ap_fixed<W + 1, I + 1> xf = x;
51     return lgamma_r_generic(xf, signgamp);
52 }
53
54 template <int I> ap_int<I> lgamma_r_generic(ap_int<I> x,
int *signgamp) {
55     ap_fixed<I_, I_> xf = x;
56     return lgamma_r_generic(xf, signgamp);
57 }
58
59 template <int I> ap_uint<I> lgamma_r_generic(ap_uint<I> x,
int *signgamp) {
60     ap_fixed<I_ + 1, I_ + 1> xf = x;
61     return lgamma_r_generic(xf, signgamp);
62 }
63
64 } // lgamma_fixed
65
66 #endif

```

E.31 vendor/etc/hls_log_apfixed.h (1,448 lines, 70,493 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
  Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_LOG_AFFIXED_H_
7 #define _X_HLS_LOG_AFFIXED_H_
8 #include "ap_fixed.h"
9 #ifndef __SYNTHESIS__
10 #include <assert.h>
11 #endif
12
13 namespace log_apfixed_reduce {
14
15 // #include "hls_log_apfixed_tables.h"
16 template <typename T, int p, int alpha, int size> class
log_lut_table { public:
17   log_lut_table<T,p,alpha,size>();
18   static const T array [size];};
19
20 template <> class log_lut_table<ap_fixed<29, 7, (ap_q_mode)5,
(ap_o_mode)3, 0>,4,4,16> { public:
21   log_lut_table<ap_fixed<29, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,
4,4,16>();
22   static const ap_fixed<29, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
23
24 template <> class log_lut_table<ap_fixed<44, 7, (ap_q_mode)5,
(ap_o_mode)3, 0>,4,4,16> { public:
25   log_lut_table<ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,
4,4,16>();
26   static const ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
27
28 template <> class log_lut_table<ap_fixed<59, 7, (ap_q_mode)5,
(ap_o_mode)3, 0>,4,4,16> { public:
29   log_lut_table<ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,
4,4,16>();
30   static const ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
31
32 template <> class log_lut_table<ap_fixed<74, 7, (ap_q_mode)5,
(ap_o_mode)3, 0>,4,4,16> { public:
33   log_lut_table<ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,
4,4,16>();
34   static const ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
35
36 template <> class log_lut_table<ap_fixed<89, 7, (ap_q_mode)5,
(ap_o_mode)3, 0>,4,4,16> { public:
37   log_lut_table<ap_fixed<89, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,
4,4,16>();
38   static const ap_fixed<89, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
39
40 template <> class log_lut_table<ap_fixed<104, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16> { public:
41   log_lut_table<ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,4,4,16>();
42   static const ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
43
44 template <> class log_lut_table<ap_fixed<119, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,4,4,16> { public:
45   log_lut_table<ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,4,4,16>();
46   static const ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [16];};
47
48 template <> class log_lut_table<ap_fixed<144, 7, (ap_q_mode)5,
(ap_o_mode)3, 0>,7,6,64> { public:
49   log_lut_table<ap_fixed<144, 7, (ap_q_mode)5, (ap_o_mode)3, 0>,
7,6,64>();
50   static const ap_fixed<144, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};

```

[illegible]


```

91 template <> class log_lut_table<ap_fixed<104, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,22,6,64> { public:
92 log_lut_table<ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,22,6,64>();
93 static const ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
94 template <> class log_lut_table<ap_fixed<119, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,22,6,64> { public:
95 log_lut_table<ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,22,6,64>();
96 static const ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
97
98 template <> class log_lut_table<ap_fixed<104, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,27,6,64> { public:
99 log_lut_table<ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,27,6,64>();
100 static const ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
101 template <> class log_lut_table<ap_fixed<119, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,27,6,64> { public:
102 log_lut_table<ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,27,6,64>();
103 static const ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
104
105 template <> class log_lut_table<ap_fixed<119, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,32,6,64> { public:
106 log_lut_table<ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,32,6,64>();
107 static const ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
108
109 template <typename T, int p, int alpha, int size> class
log0_lut_table { public:
110 log0_lut_table<T,p,alpha,size>();
111 static const T array[size];};
112 template <> class log0_lut_table<ap_fixed<20, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
113 log0_lut_table<ap_fixed<20, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
114
115 static const ap_fixed<20, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
116 template <> class log0_lut_table<ap_fixed<29, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
117 log0_lut_table<ap_fixed<29, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
118 static const ap_fixed<29, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
119 template <> class log0_lut_table<ap_fixed<44, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
120 log0_lut_table<ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
121 static const ap_fixed<44, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
122 template <> class log0_lut_table<ap_fixed<59, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
123 log0_lut_table<ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
124 static const ap_fixed<59, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
125 template <> class log0_lut_table<ap_fixed<74, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
126 log0_lut_table<ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
127 static const ap_fixed<74, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
128 template <> class log0_lut_table<ap_fixed<89, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
129 log0_lut_table<ap_fixed<89, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
130 static const ap_fixed<89, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
131 template <> class log0_lut_table<ap_fixed<104, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
132 log0_lut_table<ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
133 static const ap_fixed<104, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
134 template <> class log0_lut_table<ap_fixed<119, 7,
(ap_q_mode)5, (ap_o_mode)3, 0>,0,5,64> { public:
135 log0_lut_table<ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3,
0>,0,5,64>();
136 static const ap_fixed<119, 7, (ap_q_mode)5, (ap_o_mode)3, 0>
array [64];};
137
138 class log_inverse_lut_table { public:
139 log_inverse_lut_table();
140 static const ap_ufixed<6, 1, (ap_q_mode)5, (ap_o_mode)3, 0>
array[64];};
141
142
143 template <typename LOG_TYPE, int p, int alpha, int size, int
sizeout>
144 void range_reduce(ap_ufixed<size, -p> zN, LOG_TYPE &log,
ap_ufixed<sizeout, -(p+alpha-1)> &zN1)
145 {
146     // #pragma HLS inline off
147     const int Tsize = 1 << alpha; // The size of the first

```

```

table
148 #define log_lut log_lut_table<LOG_TYPE, p,alpha,
Tsize>::array
149
150 // #pragma HLS RESOURCE variable=log_lut core=ROM_1P_LUT-
RAM
151
152 ap_ufixed<alpha, -p> a = zN; // Extract alpha
MSBs from z1.
153 ap_ufixed<size-alpha, -p-alpha> b = zN; // Extract the
LSBs from z1.
154 ap_ufixed<size+p+2*p+1, 1> zNext = 1+zN; // Extend
z1 for shift
155 int e_shift = (p!=4 || a[a.wl()-1]==1) ? 2*p : 2*p+1; //
Stage dependent?
156 ap_ufixed<size+p+1, 1-2*p> eZ = zNext >> e_shift;
157
158 zN1 = ((b + eZ) - a*zN);
159 ap_uint<alpha> index = a(alpha-1,0);
160 log = log_lut[index];
161
162 #undef log_lut
163 }
164
165 // i 0 1 2 3 4 5 6 7 8
166 // =====
167 // pi 0 4 7 12 17 22 27 32 37
168 const int p0 = 0; // We start with zero zeroed bits

169 const int alpha0 = 5; // 1+alpha0 = number of bits considered
by stage 0.
170 const int p1 = (alpha0 == 5) ? 4 : (alpha0-2); // The number
of bits zeroed in y1.
171 const int alpha1 = 4;
172 const int p2 = p1 + alpha1 - 1;
173 const int alpha2 = 6;
174 const int p3 = p2 + alpha2 - 1;
175 const int alpha3 = 6;
176 const int p4 = p3 + alpha3 - 1;
177 const int alpha4 = 6;
178 const int p5 = p4 + alpha4 - 1;
179 const int alpha5 = 6;
180 const int p6 = p5 + alpha5 - 1;
181 const int alpha6 = 6;
182 const int p7 = p6 + alpha6 - 1;
183 const int alpha7 = 6;
184 const int p8 = p7 + alpha7 - 1;
185
186 template <int W_s> class log_traits{};
187
188 template <>
189 class log_traits<1>{
190 public:
191     const static int we = 6; // ceil(log2(ln(2^(W-1))))), <=
ceil(log2(ln(2^32)))
192     const static int wf = 11; // round to F_
193     const static int org_wf = wf;
194
195     const static int gbits = 1;
196     const static int p_generic = p1;
197     const static int MaxPrecision = wf+gbits+1; // 11+1=12;
198     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <19,7>
199
200     template <int size>
201     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
202         _AP_UNUSED_PARAM(log_sum);
203         return z1;
204     };
205 };
206
207 template <>
208 class log_traits<2>{
209 public:
210     const static int we = 6; // ceil(log2(ln(2^(W-1))))), <=
ceil(log2(ln(2^32)))
211     const static int wf = 19; // round to F_
212     const static int org_wf = wf;
213
214     const static int gbits = 2;
215     const static int p_generic = p2;
216     const static int MaxPrecision = wf+gbits+1; // 19+2=21
217     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <28,7>
218
219     const static int w1 = wf+1;
220     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
221
222     template <int size>
223     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
224         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2>
z2;
225         LOG_TYPE logn;
226
227         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
228         log_sum += logn;
229         return z2;

```

```

230     }
231 };
232
233 template <>
234 class log_traits<3>{
235 public:
236     const static int we = 6; // ceil(log2(ln(2^(W_-1))))), <=
ceil(log2(ln(2^32)))
237     const static int wf = 33; // round to F_
238     const static int org_wf = wf;
239
240     const static int gbits = 3;
241     const static int p_generic = p3;
242     const static int MaxPrecision = wf+gbits+1;//33+3=36
243     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <43,7>
244
245     const static int w1 = wf+1;
246     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
247     const static int w3 = w2+2*p2+1-(alpha2-1);
248
249     template <int size>
250     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
251         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2> z2;
252         ap_ufixed<(w3 < MaxPrecision-p3) ? w3 : (MaxPreci-
sion-p3), -p3> z3;
253         LOG_TYPE logn;
254
255         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
256         log_sum += logn;
257         range_reduce<LOG_TYPE, p2, alpha2> (z2, logn, z3);
258         log_sum += logn;
259         return z3;
260     }
261 };
262
263 template <>
264 class log_traits<4>{
265 public:
266     const static int we = 6; // ceil(log2(ln(2^(W_-1))))), <=
ceil(log2(ln(2^32)))
267     const static int wf = 48; // round to F_
268     const static int org_wf = wf;
269
270     const static int gbits = 3;
271     const static int p_generic = p4;
272     const static int MaxPrecision = wf+gbits+1;//3+48=51
273     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <58,7>
274
275     const static int w1 = wf+1;
276     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
277     const static int w3 = w2+2*p2+1-(alpha2-1);
278     const static int w4 = w3+2*p3+1-(alpha3-1);
279
280     template <int size>
281     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
282         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2> z2;
283         ap_ufixed<(w3 < MaxPrecision-p3) ? w3 : (MaxPreci-
sion-p3), -p3> z3;
284         ap_ufixed<(w4 < MaxPrecision-p4) ? w4 : (MaxPreci-
sion-p4), -p4> z4;
285         LOG_TYPE logn;
286
287         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
288         log_sum += logn;
289         range_reduce<LOG_TYPE, p2, alpha2> (z2, logn, z3);
290         log_sum += logn;
291         range_reduce<LOG_TYPE, p3, alpha3> (z3, logn, z4);
292         log_sum += logn;
293         return z4;
294     }
295 };
296
297 template <>
298 class log_traits<5>{
299 public:
300     const static int we = 6; // ceil(log2(ln(2^(W_-1))))), <=
ceil(log2(ln(2^32)))
301     const static int wf = 63; // round to (W_-1)
302     const static int org_wf = wf;
303
304     const static int gbits = 3;
305     const static int p_generic = p5;
306     const static int MaxPrecision = wf+gbits+1;//3+63=66;
307     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <73,7>
308
309     const static int w1 = wf+1;
310     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
311     const static int w3 = w2+2*p2+1-(alpha2-1);
312     const static int w4 = w3+2*p3+1-(alpha3-1);
313     const static int w5 = w4+2*p4+1-(alpha4-1);

```

```

314     template <int size>
315     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
316         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2> z2;
317         ap_ufixed<(w3 < MaxPrecision-p3) ? w3 : (MaxPreci-
sion-p3), -p3> z3;
318         ap_ufixed<(w4 < MaxPrecision-p4) ? w4 : (MaxPreci-
sion-p4), -p4> z4;
319         ap_ufixed<(w5 < MaxPrecision-p5) ? w5 : (MaxPreci-
sion-p5), -p5> z5;
320         LOG_TYPE logn;
321
322         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
323         log_sum += logn;
324         range_reduce<LOG_TYPE, p2, alpha2> (z2, logn, z3);
325         log_sum += logn;
326         range_reduce<LOG_TYPE, p3, alpha3> (z3, logn, z4);
327         log_sum += logn;
328         range_reduce<LOG_TYPE, p4, alpha4> (z4, logn, z5);
329         log_sum += logn;
330         return z5;
331     }
332 };
333
334 template <>
335 class log_traits<6>{
336 public:
337     const static int we = 6; // ceil(log2(ln(2^(W_-1))))), <=
ceil(log2(ln(2^32)))
338     const static int wf = 76; // round to F_
339     const static int org_wf = wf;
340
341     const static int gbits = 4;
342     const static int p_generic = p6;
343     const static int MaxPrecision = wf+gbits+2;//76+4=80
344     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <87,7>
345
346     const static int w1 = wf+1;
347     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
348     const static int w3 = w2+2*p2+1-(alpha2-1);
349     const static int w4 = w3+2*p3+1-(alpha3-1);
350     const static int w5 = w4+2*p4+1-(alpha4-1);
351     const static int w6 = w5+2*p5+1-(alpha5-1);
352
353     template <int size>
354     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
355         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2> z2;
356         ap_ufixed<(w3 < MaxPrecision-p3) ? w3 : (MaxPreci-
sion-p3), -p3> z3;
357         ap_ufixed<(w4 < MaxPrecision-p4) ? w4 : (MaxPreci-
sion-p4), -p4> z4;
358         ap_ufixed<(w5 < MaxPrecision-p5) ? w5 : (MaxPreci-
sion-p5), -p5> z5;
359         ap_ufixed<(w6 < MaxPrecision-p6) ? w6 : (MaxPreci-
sion-p6), -p6> z6;
360         LOG_TYPE logn;
361
362         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
363         log_sum += logn;
364         range_reduce<LOG_TYPE, p2, alpha2> (z2, logn, z3);
365         log_sum += logn;
366         range_reduce<LOG_TYPE, p3, alpha3> (z3, logn, z4);
367         log_sum += logn;
368         range_reduce<LOG_TYPE, p4, alpha4> (z4, logn, z5);
369         log_sum += logn;
370         range_reduce<LOG_TYPE, p5, alpha5> (z5, logn, z6);
371         log_sum += logn;
372         return z6;
373     }
374 };
375
376 template <>
377 class log_traits<7>{
378 public:
379     const static int we = 6; // ceil(log2(ln(2^(W_-1))))), <=
ceil(log2(ln(2^32)))
380     const static int wf = 91; // round to (W_-1)
381     const static int org_wf = wf;
382
383     const static int gbits = 4;
384     const static int p_generic = p7;
385     const static int MaxPrecision = wf+gbits+2;//91+4=95
386     typedef ap_fixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <102,7>
387
388     const static int w1 = wf+1;
389     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
390     const static int w3 = w2+2*p2+1-(alpha2-1);
391     const static int w4 = w3+2*p3+1-(alpha3-1);
392     const static int w5 = w4+2*p4+1-(alpha4-1);
393     const static int w6 = w5+2*p5+1-(alpha5-1);
394     const static int w7 = w6+2*p6+1-(alpha6-1);

```



```

397     template <int size>
398     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
399         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2> z2;
400         ap_ufixed<(w3 < MaxPrecision-p3) ? w3 : (MaxPreci-
sion-p3), -p3> z3;
401         ap_ufixed<(w4 < MaxPrecision-p4) ? w4 : (MaxPreci-
sion-p4), -p4> z4;
402         ap_ufixed<(w5 < MaxPrecision-p5) ? w5 : (MaxPreci-
sion-p5), -p5> z5;
403         ap_ufixed<(w6 < MaxPrecision-p6) ? w6 : (MaxPreci-
sion-p6), -p6> z6;
404         ap_ufixed<(w7 < MaxPrecision-p7) ? w7 : (MaxPreci-
sion-p7), -p7> z7;
405         LOG_TYPE logn;
406
407         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
408         log_sum += logn;
409         range_reduce<LOG_TYPE, p2, alpha2> (z2, logn, z3);
410         log_sum += logn;
411         range_reduce<LOG_TYPE, p3, alpha3> (z3, logn, z4);
412         log_sum += logn;
413         range_reduce<LOG_TYPE, p4, alpha4> (z4, logn, z5);
414         log_sum += logn;
415         range_reduce<LOG_TYPE, p5, alpha5> (z5, logn, z6);
416         log_sum += logn;
417         range_reduce<LOG_TYPE, p6, alpha6> (z6, logn, z7);
418         log_sum += logn;
419         return z7;
420     }
421 };
422
423 template <>
424 class log_traits<8>{
425 public:
426     const static int we = 6; // ceil(log2(ln(2^(W-1))))), <=
ceil(log2(ln(2^32)))
427     const static int wf = 106; // round to F_
428     const static int org_wf = wf;
429
430     const static int gbits = 4;
431     const static int p_generic = p8; // 37
432     const static int MaxPrecision = wf+gbits+2; // 106+4=110
433     typedef ap_ufixed<1 + we + MaxPrecision, 1 + we >
LOG_TYPE; // <117, 7>
434
435     const static int w1 = wf+1;
436     const static int w2 = w1+2*p1+1-(alpha1-1); // MaxWidth
of z2, needs to be truncated to MaxPrecision
437     const static int w3 = w2+2*p2+1-(alpha2-1);
438     const static int w4 = w3+2*p3+1-(alpha3-1);
439     const static int w5 = w4+2*p4+1-(alpha4-1);
440     const static int w6 = w5+2*p5+1-(alpha5-1);
441     const static int w7 = w6+2*p6+1-(alpha6-1);
442     const static int w8 = w7+2*p7+1-(alpha7-1);
443
444     template <int size>
445     static ap_ufixed<MaxPrecision-p_generic, -p_generic>
range_reduction(ap_ufixed<size, -p1> z1, LOG_TYPE &log_sum) {
446         ap_ufixed<(w2 < MaxPrecision-p2) ? w2 : (MaxPreci-
sion-p2), -p2> z2;
447         ap_ufixed<(w3 < MaxPrecision-p3) ? w3 : (MaxPreci-
sion-p3), -p3> z3;
448         ap_ufixed<(w4 < MaxPrecision-p4) ? w4 : (MaxPreci-
sion-p4), -p4> z4;
449         ap_ufixed<(w5 < MaxPrecision-p5) ? w5 : (MaxPreci-
sion-p5), -p5> z5;
450         ap_ufixed<(w6 < MaxPrecision-p6) ? w6 : (MaxPreci-
sion-p6), -p6> z6;
451         ap_ufixed<(w7 < MaxPrecision-p7) ? w7 : (MaxPreci-
sion-p7), -p7> z7;
452         ap_ufixed<(w8 < MaxPrecision-p8) ? w8 : (MaxPreci-
sion-p8), -p8> z8;
453         LOG_TYPE logn;
454
455         range_reduce<LOG_TYPE, p1, alpha1> (z1, logn, z2);
456         log_sum += logn;
457         range_reduce<LOG_TYPE, p2, alpha2> (z2, logn, z3);
458         log_sum += logn;
459         range_reduce<LOG_TYPE, p3, alpha3> (z3, logn, z4);
460         log_sum += logn;
461         range_reduce<LOG_TYPE, p4, alpha4> (z4, logn, z5);
462         log_sum += logn;
463         range_reduce<LOG_TYPE, p5, alpha5> (z5, logn, z6);
464         log_sum += logn;
465         range_reduce<LOG_TYPE, p6, alpha6> (z6, logn, z7);
466         log_sum += logn;
467         range_reduce<LOG_TYPE, p7, alpha7> (z7, logn, z8);
468         log_sum += logn;
469         return z8;
470     }
471 };
472 template<int W_, int I_>
473 ap_fixed<W_, I_> log(ap_fixed<W_, I_> x)
474 {
475     #pragma HLS pipeline
476
477     // bitwidth that currently not supported
478     const static int F_ = W_ - I_;

```

```

479     if (I_>34) return 0;
480     else if (F_>100) return 0;
481
482     // return ap_fixed value
483     ap_fixed<W_, I_> r = 0;
484     ap_ufixed<W_-1, I_-1> x_s = x;
485
486     if (F_==0) {
487         if (I_<=7) {
488             if (x<=0) r[W_-1] = 1;
489
490             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x1p0")) r =
0;
491
492             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x4p0")) r =
1;
493
494             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xcp0")) r =
2;
495
496             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x21p0")) r =
3;
497
498             else r = 4;
499         } else if (I_<=10) {
500             if (x<=0) r[W_-1] = 1;
501
502             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x1p0")) r =
0;
503
504             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x4p0")) r =
1;
505
506             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xcp0")) r =
2;
507
508             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x21p0")) r =
3;
509
510             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x5ap0")) r =
4;
511
512             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xf4p0")) r =
5;
513
514             else r = 6;
515         } else if (I_<=13) {
516             if (x<=0) r[W_-1] = 1;
517             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x1p0")) r =
0;
518             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x4p0")) r =
1;
519             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xcp0")) r =
2;
520             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x21p0")) r =
3;
521             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x5ap0")) r =
4;
522             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xf4p0")) r =
5;
523             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x299p0")) r =
6;
524             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x710p0")) r =
7;
525             else r = 8;
526         } else if (I_<=17) {
527             if (x<=0) r[W_-1] = 1;
528             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x1p0")) r =
0;
529             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x4p0")) r =
1;
530             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xcp0")) r =
2;
531             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x21p0")) r =
3;
532             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x5ap0")) r =
4;
533             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xf4p0")) r =
5;
534             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x299p0")) r =
6;
535             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x710p0")) r =
7;
536             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x1332p0")) r =
8;
537             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x342fp0")) r =
9;
538             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x8ddbp0")) r =
10;
539             else r = 11;
540         } else if (I_<=19) {
541             if (x<=0) r[W_-1] = 1;
542             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x1p0")) r =
0;
543             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x4p0")) r =
1;
544             else if (x_s<=ap_ufixed<W_-1, I_-1>("0xcp0")) r =
2;
545             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x21p0")) r =
3;
546             else if (x_s<=ap_ufixed<W_-1, I_-1>("0x5ap0")) r =
4;

```

```

535         else if (x_s<=ap_ufixed<W-1,I-1>("0xf4p0"))
r = 5;
536         else if (x_s<=ap_ufixed<W-1,I-1>("0x299p0"))
r = 6;
537         else if (x_s<=ap_ufixed<W-1,I-1>("0x710p0"))
r = 7;
538         else if (x_s<=ap_ufixed<W-1,I-1>("0x1332p0"))
r = 8;
539         else if (x_s<=ap_ufixed<W-1,I-1>("0x342fp0"))
r = 9;
540         else if (x_s<=ap_ufixed<W-1,I-1>("0x8ddb0p0"))
r = 10;
541         else if (x_s<=ap_ufixed<W-1,I-1>("0x1819bp0"))
r = 11;
542         else
r = 12;
543     } else if (I<=26) {
544         if (x<=0)
r[W-1] = 1;

545         else if (x_s<=ap_ufixed<W-1,I-1>("0x1p0"))
r
= 0;

546         else if (x_s<=ap_ufixed<W-1,I-1>("0x4p0"))
r
= 1;

547         else if (x_s<=ap_ufixed<W-1,I-1>("0xcp0"))
r
= 2;

548         else if (x_s<=ap_ufixed<W-1,I-1>("0x21p0"))
r
= 3;

549         else if (x_s<=ap_ufixed<W-1,I-1>("0x5ap0"))
r
= 4;

550         else if (x_s<=ap_ufixed<W-1,I-1>("0xf4p0"))
r
= 5;

551         else if (x_s<=ap_ufixed<W-1,I-1>("0x299p0"))
r
= 6;

552         else if (x_s<=ap_ufixed<W-1,I-1>("0x710p0"))
r
= 7;

553         else if (x_s<=ap_ufixed<W-1,I-1>("0x1332p0"))
r
= 8;

554         else if (x_s<=ap_ufixed<W-1,I-1>("0x342fp0"))
r
= 9;

555         else if (x_s<=ap_ufixed<W-1,I-1>("0x8ddb0p0"))
r
= 10;

556         else if (x_s<=ap_ufixed<W-1,I-1>("0x1819bp0"))
r
= 11;

557         else if (x_s<=ap_ufixed<W-1,I-1>("0x41831p0"))
r
= 12;

558         else if (x_s<=ap_ufixed<W-1,I-1>("0xb2148p0"))
r
= 13;

559         else if (x_s<=ap_ufixed<W-1,I-1>("0x1e4127p0"))
r
= 14;

560         else if (x_s<=ap_ufixed<W-1,I-1>("0x523d82p0"))
r
= 15;

561         else if (x_s<=ap_ufixed<W-1,I-1>("0xdf8d5fp0"))
r
= 16;
562         else
r = 17;
563     } else if (I<=32) {
564         if (x<=0)
r[W-1] = 1;
565         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1p0"))
r = 0;
566         else if
(x_s<=ap_ufixed<W-1,I-1>("0x4p0"))
r = 1;
567         else if
(x_s<=ap_ufixed<W-1,I-1>("0xcp0"))
r = 2;
568         else if
(x_s<=ap_ufixed<W-1,I-1>("0x21p0"))
r = 3;
569         else if
(x_s<=ap_ufixed<W-1,I-1>("0x5ap0"))
r = 4;
570         else if
(x_s<=ap_ufixed<W-1,I-1>("0xf4p0"))
r = 5;
571         else if
(x_s<=ap_ufixed<W-1,I-1>("0x299p0"))
r = 6;
572         else if
(x_s<=ap_ufixed<W-1,I-1>("0x710p0"))
r = 7;
573         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1332p0"))
r = 8;
574         else if
(x_s<=ap_ufixed<W-1,I-1>("0x342fp0"))
r = 9;
575         else if
(x_s<=ap_ufixed<W-1,I-1>("0x8ddb0p0"))
r = 10;
576         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1819bp0"))
r = 11;
577         else if
(x_s<=ap_ufixed<W-1,I-1>("0x41831p0"))
r = 12;
578         else if
(x_s<=ap_ufixed<W-1,I-1>("0xb2148p0"))
r = 13;
579         else if

```

```

(x_s<=ap_ufixed<W-1,I-1>("0x1e4127p0"))
r = 14;
580         else if
(x_s<=ap_ufixed<W-1,I-1>("0x523d82p0"))
r = 15;
581         else if
(x_s<=ap_ufixed<W-1,I-1>("0xdf8d5fp0"))
r = 16;
582         else if
(x_s<=ap_ufixed<W-1,I-1>("0x25fad90p0"))
r = 17;
583         else if
(x_s<=ap_ufixed<W-1,I-1>("0x673d70bp0"))
r = 18;
584         else if
(x_s<=ap_ufixed<W-1,I-1>("0x118a2aap0"))
r = 19;
585         else if
(x_s<=ap_ufixed<W-1,I-1>("0x2fad89e1p0"))
r = 20;
586         else
r = 21;
587     } else {
588         if (x<=0)
r[W-1] = 1;
589         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1p0"))
r = 0;
590         else if
(x_s<=ap_ufixed<W-1,I-1>("0x4p0"))
r = 1;
591         else if
(x_s<=ap_ufixed<W-1,I-1>("0xcp0"))
r = 2;
592         else if
(x_s<=ap_ufixed<W-1,I-1>("0x21p0"))
r = 3;
593         else if
(x_s<=ap_ufixed<W-1,I-1>("0x5ap0"))
r = 4;
594         else if
(x_s<=ap_ufixed<W-1,I-1>("0xf4p0"))
r = 5;
595         else if
(x_s<=ap_ufixed<W-1,I-1>("0x299p0"))
r = 6;
596         else if
(x_s<=ap_ufixed<W-1,I-1>("0x710p0"))
r = 7;
597         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1332p0"))
r = 8;
598         else if
(x_s<=ap_ufixed<W-1,I-1>("0x342fp0"))
r = 9;
599         else if
(x_s<=ap_ufixed<W-1,I-1>("0x8ddb0p0"))
r = 10;
600         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1819bp0"))
r = 11;
601         else if
(x_s<=ap_ufixed<W-1,I-1>("0x41831p0"))
r = 12;
602         else if
(x_s<=ap_ufixed<W-1,I-1>("0xb2148p0"))
r = 13;
603         else if
(x_s<=ap_ufixed<W-1,I-1>("0x1e4127p0"))
r = 14;
604         else if
(x_s<=ap_ufixed<W-1,I-1>("0x523d82p0"))
r = 15;
605         else if
(x_s<=ap_ufixed<W-1,I-1>("0xdf8d5fp0"))
r = 16;
606         else if
(x_s<=ap_ufixed<W-1,I-1>("0x25fad90p0"))
r = 17;
607         else if
(x_s<=ap_ufixed<W-1,I-1>("0x673d70bp0"))
r = 18;
608         else if
(x_s<=ap_ufixed<W-1,I-1>("0x118a2aap0"))
r = 19;
609         else if
(x_s<=ap_ufixed<W-1,I-1>("0x2fad89e1p0"))
r = 20;
610         else if
(x_s<=ap_ufixed<W-1,I-1>("0x819a1801p0"))
r = 21;
611         else
r = 22;
612     }
613     } else if (F<=4 && I<=5) {
614         if (x<=0)
r[W-1] = 1;
615         else if (x_s==ap_ufixed<W-1,I-1>("0x0.1p0"))
r
= "-0x2.cp0";
616         else if (x_s==ap_ufixed<W-1,I-1>("0x0.2p0"))
r
= "-0x2.1p0";
617         else if (x_s==ap_ufixed<W-1,I-1>("0x0.3p0"))
r
= "-0x1.bp0";
618         else if (x_s==ap_ufixed<W-1,I-1>("0x0.4p0"))
r
= "-0x1.6p0";
619         else if (x_s==ap_ufixed<W-1,I-1>("0x0.5p0"))
r
= "-0x1.3p0";
620         else if (x_s==ap_ufixed<W-1,I-1>("0x0.6p0"))
r
= "-0x1p0";
621         else if (x_s==ap_ufixed<W-1,I-1>("0x0.7p0"))
r
= "-0x0.dp0";
622         else if (x_s==ap_ufixed<W-1,I-1>("0x0.8p0"))
r
= "-0x0.bp0";
623         else if (x_s==ap_ufixed<W-1,I-1>("0x0.9p0"))
r
= "-0x0.9p0";
624         else if (x_s==ap_ufixed<W-1,I-1>("0x0.ap0"))
r
= "-0x0.8p0";
625         else if (x_s==ap_ufixed<W-1,I-1>("0x0.bp0"))
r
= "-0x0.6p0";
626         else if (x_s==ap_ufixed<W-1,I-1>("0x0.cp0"))
r
= "-0x0.5p0";
627         else if (x_s==ap_ufixed<W-1,I-1>("0x0.dp0"))
r
= "-0x0.3p0";
628         else if (x_s==ap_ufixed<W-1,I-1>("0x0.ep0"))
r
= "-0x0.2p0";
629         else if (x_s==ap_ufixed<W-1,I-1>("0x0.fp0"))
r
= "-0x0.1p0";
630         else if (x_s==ap_ufixed<W-1,I-1>("0x1p0"))
r
= "0x0p0";
631         else if (x_s==ap_ufixed<W-1,I-1>("0x1.1p0"))
r
= "0x0.1p0";
632         else if (x_s==ap_ufixed<W-1,I-1>("0x1.2p0"))
r
= "0x0.2p0";

```

```

633     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.3p0")) r
634 = "0x0.3p0";
635     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.5p0")) r
636 = "0x0.4p0";
637     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.6p0")) r
638 = "0x0.5p0";
639     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.8p0")) r
640 = "0x0.6p0";
641     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.9p0")) r
642 = "0x0.7p0";
643     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.bp0")) r
644 = "0x0.8p0";
645     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.cp0")) r
646 = "0x0.9p0";
647     else if (x_s==ap_ufixed<W_-1,I_-1>("0x1.ep0")) r
648 = "0x0.ap0";
649     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2p0")) r
650 = "0x0.bp0";
651     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2.2p0")) r
652 = "0x0.cp0";
653     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2.5p0")) r
654 = "0x0.dp0";
655     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2.7p0")) r
656 = "0x0.ep0";
657     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2.ap0")) r
658 = "0x0.fp0";
659     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2.cp0")) r
660 = "0x1p0";
661     else if (x_s==ap_ufixed<W_-1,I_-1>("0x2.fp0")) r
662 = "0x1.1p0";
663     else if (x_s==ap_ufixed<W_-1,I_-1>("0x3.2p0")) r
664 = "0x1.2p0";
665     else if (x_s==ap_ufixed<W_-1,I_-1>("0x3.6p0")) r
666 = "0x1.3p0";
667     else if (x_s==ap_ufixed<W_-1,I_-1>("0x3.9p0")) r
668 = "0x1.4p0";
669     else if (x_s==ap_ufixed<W_-1,I_-1>("0x3.dp0")) r
670 = "0x1.5p0";
671     else if (x_s==ap_ufixed<W_-1,I_-1>("0x4.1p0")) r
672 = "0x1.6p0";
673     else if (x_s==ap_ufixed<W_-1,I_-1>("0x4.5p0")) r
674 = "0x1.7p0";
675     else if (x_s==ap_ufixed<W_-1,I_-1>("0x4.9p0")) r
676 = "0x1.8p0";
677     else if (x_s==ap_ufixed<W_-1,I_-1>("0x4.ep0")) r
678 = "0x1.9p0";
679     else if (x_s==ap_ufixed<W_-1,I_-1>("0x5.3p0")) r
680 = "0x1.ap0";
681     else if (x_s==ap_ufixed<W_-1,I_-1>("0x5.9p0")) r
682 = "0x1.bp0";
683     else if (x_s==ap_ufixed<W_-1,I_-1>("0x5.ep0")) r
684 = "0x1.cp0";
685     else if (x_s==ap_ufixed<W_-1,I_-1>("0x6.5p0")) r
686 = "0x1.dp0";
687     else if (x_s==ap_ufixed<W_-1,I_-1>("0x6.bp0")) r
688 = "0x1.ep0";
689     else if (x_s==ap_ufixed<W_-1,I_-1>("0x7.2p0")) r
690 = "0x1.fp0";
691     else if (x_s==ap_ufixed<W_-1,I_-1>("0x7.9p0")) r
692 = "0x2p0";
693     else if (x_s==ap_ufixed<W_-1,I_-1>("0x8.1p0")) r
694 = "0x2.1p0";
695     else if (x_s==ap_ufixed<W_-1,I_-1>("0x8.ap0")) r
696 = "0x2.2p0";
697     else if (x_s==ap_ufixed<W_-1,I_-1>("0x9.3p0")) r
698 = "0x2.3p0";
699     else if (x_s==ap_ufixed<W_-1,I_-1>("0x9.cp0")) r
700 = "0x2.4p0";
701     else if (x_s==ap_ufixed<W_-1,I_-1>("0xa.6p0")) r
702 = "0x2.5p0";
703     else if (x_s==ap_ufixed<W_-1,I_-1>("0xb.1p0")) r
704 = "0x2.6p0";
705     else if (x_s==ap_ufixed<W_-1,I_-1>("0xb.cp0")) r
706 = "0x2.7p0";
707     else if (x_s==ap_ufixed<W_-1,I_-1>("0xc.9p0")) r
708 = "0x2.8p0";
709     else if (x_s==ap_ufixed<W_-1,I_-1>("0xd.6p0")) r
710 = "0x2.9p0";
711     else if (x_s==ap_ufixed<W_-1,I_-1>("0xe.3p0")) r
712 = "0x2.ap0";
713     else if (x_s==ap_ufixed<W_-1,I_-1>("0xf.2p0")) r
714 = "0x2.bp0";
715     else
716         r = "0x2.cp0";
717 } else {
718     if (x<=0) {r[W_-1] = 1;}
719     else {
720         //std::cout<<"[LOG]x="<<x.to_string()<<std::endl;
721         const static int W_s_ = (F_+F_/77*2+12)/15-
(100+F_-)/133+(99+F_-)/133-(100+F_-)/118+(98+F_-)/118+1;
722         //if (W_<=11) W_s_ = 1;
723         //else if (W_<=19) W_s_ = 2;
724         //else if (W_<=33) W_s_ = 3;
725         //else if (W_<=48) W_s_ = 4;
726         //else if (W_<=63) W_s_ = 5;
727         //else if (W_<=76) W_s_ = 6;
728         //else if (W_<=91) W_s_ = 7;
729         //else W_s_ = 8;
730     }
731 }

```

```

691     const static int wf =
log_traits<W_s_>::wf;
692     const static int I_s= I_>0? I_-:I_;
693     const static int org_wf =
log_traits<W_s_>::org_wf+I_s;
694     const static int MaxPrecision =
log_traits<W_s_>::MaxPrecision;
695     // x = b_frac * (2 ^ b_exp)
696     // .75 <= b_frac < 1.5
697     // -F_ <= b_exp <= I_-1
698     ap_int<7> b_exp;
699     ap_ufixed<1 + org_wf,1> b_frac=0;
700     b_frac[org_wf] = 0;
701     b_frac(org_wf-1,org_wf-W_-1) = x(W_-2,0);
702     for (b_exp = I_-1; b_exp >= -F_; b_exp--) {
703         #pragma HLS unroll
704         if (!
b_frac[org_wf]&b_frac[org_wf-1]&b_frac[org_wf-2])|(b_frac[org_wf]&
b_frac[org_wf-1]))
705             break;
706             b_frac <= 1;
707         }
708         //std::cout<<"[LOG]b_exp="<<b_exp<<std::endl;
709         //
710         std::cout<<"[LOG]b_frac="<<b_frac.to_string()<<std::endl;
711         const ap_ufixed<MaxPrecision,0> LOG2 =
"0x0.817217F7D1CF79ABC9E3B39803p0";//
0.69314718055989033018704503774643;
712         ap_fixed<MaxPrecision,6> Elog2 = LOG2 * b_exp;
713         //maybe need to extend ???
714         typedef typename log_traits<W_s_>::LOG_TYPE
LOG_TYPE;
715         const static int bypass_threshold =
log_traits<W_s_>::p_generic;
716         //
717         std::cout<<"[LOG]b_frac="<<b_frac.to_string()<<std::endl;
718         ap_uint<1+alpha0> index0; // ap_uint<6>
719         if (b_frac[org_wf]) index0 =
b_frac(org_wf-1,org_wf-6);
720         else index0 =
b_frac(org_wf-2,org_wf-7);
721         const int T0size = 1 << (1+alpha0); // The size
of the first table
722         #define inverse_lut log_inverse_lut_table::array
723         #define log_lut log_lut_t-
able<LOG_TYPE,p0,alpha0,T0size>::array
724         ap_ufixed<1 + alpha0, 1> b_frac_tilde_inverse =
inverse_lut[index0];
725         //
726         std::cout<<"[LOG]b_frac_tilde_inverse="<<b_frac_tilde_inverse.to_s
tring()<<std::endl;
727         LOG_TYPE log_sum = log_lut[index0];
728         ap_ufixed<1 + org_wf+1+(1+alpha0), 1> b_frac1 =
b_frac * b_frac_tilde_inverse;
729         //
730         std::cout<<"[LOG]b_frac1="<<b_frac1.to_string()<<std::endl;
731         #ifndef __SYNTHESIS__
732         assert(b_frac1( b_frac1.wl()-b_frac1.iwl()-1,
b_frac1.wl()-b_frac1.iwl()-p1) == 0);
733         #endif
734         const int z1_width = 1+wf+1+(1+alpha0)-p1+1;
735         ap_ufixed<z1_width, -p1> z1 =
b_frac1; // Extract a1+b1 from y1.
736         //
737         std::cout<<"[LOG]z1="<<z1.to_string()<<std::endl;
738         ap_fixed<1-bypass_threshold+1+MaxPrecision, -by-
pass_threshold+1> zk;
739         zk =
log_traits<W_s_>::range_reduction(z1, log_sum);
740         //
741         std::cout<<"[LOG]zk="<<zk.to_string()<<std::endl;
742         ap_fixed<1-2*bypass_threshold+1+MaxPrecision, -
bypass_threshold+1> zk_trunc = zk;
743         LOG_TYPE sum = zk-(zk_trunc*zk_trunc/2);
744         LOG_TYPE log_base;
745         log_base = Elog2 + log_sum + sum;
746         #undef log_lut
747         #undef inverse_lut
748         ap_fixed<2 + F_, 1> delta = 0;
749         delta[delta.wl()-1] = log_base[log_base.wl()-1];
750         delta[delta.wl()-delta.iwl()-1] = 1;
751         log_base = log_base + ( delta >> F_ );
752         r = log_base;
753     }

```

```

764     }
765 }
766
767     return r;
768 }
769
770 template<int W_, int I_>
771 ap_ufixed<W_, I_> log(ap_ufixed<W_, I_> x) {
772     ap_fixed<W_+1, I_+1> xf = x;
773     return log(xf);
774 }
775
776 template<int I_>
777 ap_int<I_> log(ap_int<I_> x) {
778     ap_fixed<I_, I_> xf = x;
779     return log(xf);
780 }
781
782 template<int I_>
783 ap_uint<I_> log(ap_uint<I_> x) {
784     ap_fixed<I_+1, I_+1> xf = x;
785     return log(xf);
786 }
787
788
789 template<int W_, int I_>
790 ap_fixed<W_, I_> log10(ap_fixed<W_, I_> x)
791 {
792     #pragma HLS pipeline
793
794     // bitwidth that currently not supported
795     const static int F_ = W_ - I_;
796     if (I_ > 33) return 0;
797     else if (F_ > 32) return 0;
798
799     // return ap_fixed value
800     ap_fixed<W_, I_> r = 0;
801     ap_ufixed<W_-1, I_-1> x_s = x;
802
803     if (F_ == 0) {
804         if (I_ <= 9) {
805             if (x <= 0) {r[W_-1] = 1;}
806             else {
807                 ap_ufixed<8, 8> x_s_l = x_s;
808                 if (x_s_l(7, 2) == 0) r = 0;
809                 else if (x_s_l(7, 5) == 0) r = 1;
810                 else r = 2;
811             }
812         } else if (I_ <= 15) {
813             if (x <= 0) r[W_-1] = 1;
814             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x3p0")) r
= 0;
815             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1fp0")) r
= 1;
816             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x13cp0")) r
= 2;
817             else if (x_s <= ap_ufixed<W_-1, I_-1>("0xc5ap0")) r
= 3;
818             else r = 4;
819         } else if (I_ <= 19) {
820             if (x <= 0) r[W_-1] = 1;
821             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x3p0")) r =
0;
822             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1fp0")) r =
1;
823             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x13cp0")) r =
2;
824             else if (x_s <= ap_ufixed<W_-1, I_-1>("0xc5ap0")) r =
3;
825             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x7b86p0")) r =
4;
826             else r = 5;
827         } else if (I_ <= 25) {
828             if (x <= 0) r[W_-1] = 1;
829             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x3p0")) r
= 0;
830             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1fp0")) r
= 1;
831             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x13cp0")) r
= 2;
832             else if (x_s <= ap_ufixed<W_-1, I_-1>("0xc5ap0")) r
= 3;
833             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x7b86p0")) r
= 4;
834             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x4d343p0")) r
= 5;
835             else if (x_s <= ap_ufixed<W_-1, I_-1>("0x3040a5p0")) r
= 6;

```

```

836         else r = 7;
837     } else if (I_ <= 32) {
838         if (x <= 0) r[W_-1] = 1;
839         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x3p0")) r = 0;
840         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x1fp0")) r = 1;
841         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x13cp0")) r = 2;
842         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0xc5ap0")) r = 3;
843         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x7b86p0")) r = 4;
844         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x4d343p0")) r = 5;
845         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x3040a5p0")) r = 6;
846         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x1e28678p0")) r = 7;
847         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x12d940b6p0")) r = 8;
848         else r = 9;
849     } else {
850         if (x <= 0) r[W_-1] = 1;
851         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x3p0")) r = 0;
852         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x1fp0")) r = 1;
853         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x13cp0")) r = 2;
854         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0xc5ap0")) r = 3;
855         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x7b86p0")) r = 4;
856         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x4d343p0")) r = 5;
857         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x3040a5p0")) r = 6;
858         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x1e28678p0")) r = 7;
859         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x12d940b6p0")) r = 8;
860         else if
(x_s <= ap_ufixed<W_-1, I_-1>("0x12d940b6p0")) r = 8;
861         else r = 10;
862     }
863     } else if (F_ <= 4 && I_ <= 5) {
864         if (x <= 0) r[W_-1] = 1;
865         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.1p0")) r
= "-0x1.3p0";
866         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.2p0")) r
= "-0x0.ep0";
867         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.3p0")) r
= "-0x0.cp0";
868         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.4p0")) r
= "-0x0.ap0";
869         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.5p0")) r
= "-0x0.8p0";
870         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.6p0")) r
= "-0x0.7p0";
871         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.7p0")) r
= "-0x0.6p0";
872         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.8p0")) r
= "-0x0.5p0";
873         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.9p0")) r
= "-0x0.4p0";
874         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.bp0")) r
= "-0x0.3p0";
875         else if (x_s == ap_ufixed<W_-1, I_-1>("0x0.cp0")) r
= "-0x0.2p0";
876         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x0.ep0")) r
= "-0x0.1p0";
877         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1.1p0")) r
= "0x0p0";
878         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1.3p0")) r
= "0x0.1p0";
879         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1.6p0")) r
= "0x0.2p0";
880         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1.ap0")) r
= "0x0.3p0";
881         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x1.ep0")) r
= "0x0.4p0";
882         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x2.3p0")) r
= "0x0.5p0";
883         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x2.8p0")) r
= "0x0.6p0";
884         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x2.fp0")) r
= "0x0.7p0";
885         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x3.6p0")) r
= "0x0.8p0";
886         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x3.ep0")) r
= "0x0.9p0";
887         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x4.8p0")) r
= "0x0.ap0";
888         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x5.3p0")) r
= "0x0.bp0";
889         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x6p0")) r
= "0x0.cp0";
890         else if (x_s <= ap_ufixed<W_-1, I_-1>("0x6.fp0")) r
= "0x0.dp0";

```

```

891         else if (x_s<=ap_ufixed<W-1,I-1>("0x8p0")) r
= "0x0.ep0";
892         else if (x_s<=ap_ufixed<W-1,I-1>("0x9.4p0")) r
= "0x0.fp0";
893         else if (x_s<=ap_ufixed<W-1,I-1>("0xa.bp0")) r
= "0x1p0";
894         else if (x_s<=ap_ufixed<W-1,I-1>("0xc.6p0")) r
= "0x1.1p0";
895         else if (x_s<=ap_ufixed<W-1,I-1>("0xe.5p0")) r
= "0x1.2p0";
896         else
897             r = "0x1.3p0";
898     } else { // I was here!
899         if (x<=0) {r[W-1] = 1;}
900         else {
901             const static int W_s = (F_+F_/77*2+12)/15-
(100+F_-)/133+(99+F_-)/133-(100+F_-)/118+(98+F_-)/118+1;
902             //if (W_<=11) W_s_ = 1;
903             //else if (W_<=19) W_s_ = 2;
904             //else if (W_<=33) W_s_ = 3;
905             //else if (W_<=48) W_s_ = 4;
906             //else if (W_<=63) W_s_ = 5;
907             //else if (W_<=76) W_s_ = 6;
908             const static int wf =
log_traits<W_s>::wf;
909             const static int I_s = I_>0? I_-:~I_;
910             const static int org_wf =
log_traits<W_s>::org_wf+I_s;
911             const static int MaxPrecision =
log_traits<W_s>::MaxPrecision;
912             // x = b_frac * (2 ^ b_exp)
913             // .75 <= b_frac < 1.5
914             // -F_ <= b_exp <= I_-1
915             ap_int<7> b_exp=0;
916             ap_ufixed<1 + org_wf,1> b_frac=0;
917             b_frac[org_wf] = 0;
918             b_frac(org_wf-1,org_wf-W_+1) = x(W_-2,0);
919             for (b_exp = I_-1; b_exp >= -F_; b_exp--) {
920                 #pragma HLS unroll
921                 if (!
b_frac[org_wf]&b_frac[org_wf-1]&b_frac[org_wf-2])|(b_frac[org_wf]&I
b_frac[org_wf-1]))
922                     break;
923                 b_frac <<= 1;
924             }
925             const ap_ufixed<25,0> LOG1_35_s =
0.3010300099849700927734375;//0.693147182464599609375;
926             const ap_ufixed<43,0> LOG1_35_l =
0.30102999566395283181918784976006;//0.6931471805598903301870450377
4643;
927             ap_fixed<30,5> Elog2_s = LOG1_35_s * b_exp;
928             ap_fixed<48,5> Elog2_l = LOG1_35_l * b_exp;
929             typedef typename log_traits<W_s>::LOG_TYPE
LOG_TYPE;
930             const static int bypass_threshold =
log_traits<W_s>::p_generic;
931             ap_uint<1+alpha0> index0; // ap_uint<6>
932             if (b_frac[org_wf]) index0 =
b_frac(org_wf-1,org_wf-6);
933             else
934                 index0 =
b_frac(org_wf-2,org_wf-7);
935             const int T0size = 1 << (1+alpha0); // The size
of the first table
936             #define inverse_lut_log_inverse_lut_table::array
937             #define log_lut_log0_lut_t-
able<LOG_TYPE,p0,alpha0,T0size>::array
938             ap_ufixed<1 + alpha0, 1> b_frac_tilde_inverse =
inverse_lut[index0];
939             LOG_TYPE log_sum = log_lut[index0];
940             ap_ufixed<1 + org_wf+1+(1+alpha0), 1> b_frac1 =
b_frac * b_frac_tilde_inverse;
941             #ifndef __SYNTHESIS__
942             assert(b_frac1( b_frac1.wl()-b_frac1.iwl()-1,
b_frac1.wl()-b_frac1.iwl()-p1) == 0);
943             #endif
944             const int z1_width = 1+wf+1+(1+alpha0)-p1+1;
945             ap_ufixed<z1_width, -p1> z1 =
b_frac1; // Extract a1+b1 from y1.
946             ap_fixed<1-bypass_threshold+1+MaxPrecision, -by-
pass_threshold+1> zk;
947             zk =
log_traits<W_s>::range_reduction(z1, log_sum);
948             ap_fixed<1-2*bypass_threshold+1+MaxPrecision, -
bypass_threshold+1> zk_trunc = zk;
949             LOG_TYPE sum = zk-(zk_trunc*zk_trunc/2);
950         }

```

```

951         LOG_TYPE log_base = log_sum + sum;
952         const ap_ufixed<25,0> LOG1_54_s =
0.4342944920063018798828125;//0.693147182464599609375;
953         const ap_ufixed<43,0> LOG1_54_l =
0.43429448190329367207596078515053;//0.6931471805598903301870450377
4643;
954         if (F_<=16) log_base = Elog2_s + log_base *
LOG1_54_s;
955         else
956             log_base = Elog2_l + log_base *
LOG1_54_l;
957         #undef log_lut
958         #undef inverse_lut
959         ap_fixed<1 + 33, 1> delta = 0;
960         delta[delta.wl()-1] =
log_base[log_base.wl()-1];
961         delta[delta.wl()-delta.iwl()-1] = 1;
962         log_base = log_base + ( delta >> F_ );
963         r = log_base;
964     }
965     return r;
966 }
967 template<int W_, int I_>
log10(ap_ufixed<W_,I_> x) {
968     ap_fixed<W_+1,I_+1> xf = x;
969     return log10(xf);
970 }
971 template<int I_>
log10(ap_int<I_> x) {
972     ap_fixed<I_,I_> xf = x;
973     return log10(xf);
974 }
975 template<int I_>
log10(ap_uint<I_> x) {
976     ap_fixed<I_+1,I_+1> xf = x;
977     return log10(xf);
978 }
979 template<int W, int I>
log1p(ap_ufixed<W,I> x) {
980     ap_ufixed<1,1> inc = 1;
981     ap_fixed<W+1,I+1> xp1 = x + inc;
982     return log(xp1);
983 }
984 template<int W, int I>
log1p(ap_ufixed<W,I> x) {
985     ap_fixed<W+1,I+1> xf = x;
986     return log1p(xf);
987 }
988 template<int I>
log1p(ap_int<I> x) {
989     ap_fixed<I,I> xf = x;
990     return log1p(xf);
991 }
992 template<int I>
log1p(ap_uint<I> x) {
993     ap_fixed<I+1,I+1> xf = x;
994     return log1p(xf);
995 }
996 template<int W,int I>
ilogb(ap_ufixed<W,I> x) {
997     int F = W - I;
998     if(F>32 || I>33) {
999         return 0;
1000     }
1001     ap_fixed<W+1,I+1> xf;
1002     if(x>0) {
1003         xf = x;
1004     }
1005     else {
1006         xf = -x;
1007     }
1008     return I + 1 - xf.countLeadingZeros() - 1;
1009 }
1010 template<int W,int I>
ilogb(ap_ufixed<W,I> x) {
1011     ap_fixed<W+1,I+1> xf = x;
1012     return ilogb(xf);
1013 }
1014 template<int I>
ilogb(ap_int<I> x) {
1015     ap_fixed<I,I> xf = x;
1016     return ilogb(xf);
1017 }
1018 }

```



```

1058 template<int I>
1059 ap_uint<I> ilogb(ap_uint<I> x) {
1060     ap_fixed<I+1,I+1> xf = x;
1061     return ilogb(xf);
1062 }
1063
1064
1065 template<int W_, int I_>
1066 ap_fixed<W_,I_> log2(ap_fixed<W_,I_> x)
1067 {
1068     #pragma HLS pipeline
1069
1070     // bandwidth that currently not supported
1071     const static int F_ = W_ - I_;
1072     if (I_>33) return 0;
1073     else if (F_>32) return 0;
1074
1075     // return ap_fixed value
1076     ap_fixed<W_,I_> r = 0;
1077     ap_ufixed<W_-1,I_-1> x_s = x;
1078
1079     if (F_==0) {
1080         if (I_<=5) {
1081             if (x<=0) r[W_-1] = 1;
1082             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1083             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1084             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1085             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1086             else r = 4;
1087             } else if (I_<=9) {
1088                 if (x<=0) r[W_-1] = 1;
1089                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1090                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1091                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1092                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1093                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1094                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1095                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1096                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1097                 else r = 8;
1098                 } else if (I_<=13) {
1099                     if (x<=0) r[W_-1] = 1;
1100                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1101                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1102                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1103                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1104                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1105                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1106                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1107                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1108                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16Ap0")) r = 8;
1109                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D4p0")) r = 9;
1110                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8p0")) r = 10;
1111                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB50p0")) r = 11;
1112                     else r = 12;
1113                     } else if (I_<=17) {
1114                         if (x<=0) r[W_-1] = 1;
1115                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1116                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1117                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1118                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1119                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1120                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1121                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1122                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1123                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16Ap0")) r = 8;
1124                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D4p0")) r = 9;
1125                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8p0")) r = 10;
1126                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB50p0")) r = 11;
1127                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A0p0")) r = 12;
1128                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D41p0")) r = 13;
1129                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82p0")) r = 14;
1130                         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504p0")) r = 15;
1131                         else r = 16;
1132                         } else if (I_<=21) {
1133                             if (x<=0) r[W_-1] = 1;
1134                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1135                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1136                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1137                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1138                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1139                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1140                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1141                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1142                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16Ap0")) r = 8;
1143                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D4p0")) r = 9;
1144                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8p0")) r = 10;
1145                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB50p0")) r = 11;
1146                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A0p0")) r = 12;
1147                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D41p0")) r = 13;
1148                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82p0")) r = 14;
1149                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504p0")) r = 15;
1150                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09p0")) r = 16;
1151                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413p0")) r = 17;
1152                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A827p0")) r = 18;
1153                             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504Fp0")) r = 19;
1154                             else r = 20;
1155                             } else if (I_<=25) {
1156                                 if (x<=0) r[W_-1] = 1;
1157                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1158                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1159                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1160                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1161                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1162                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1163                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1164                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1165                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16Ap0")) r = 8;
1166                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D4p0")) r = 9;
1167                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8p0")) r = 10;
1168                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB50p0")) r = 11;
1169                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A0p0")) r = 12;
1170                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D41p0")) r = 13;
1171                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82p0")) r = 14;
1172                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504p0")) r = 15;
1173                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09p0")) r = 16;
1174                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413p0")) r = 17;
1175                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A827p0")) r = 18;
1176                                 else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504Fp0")) r = 19;
1177                                 else if

```

```

(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09Ep0")) r = 20;
1178     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413Cp0")) r = 21;
1179     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8279p0")) r = 22;
1180     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504F3p0")) r = 23;
1181     else
1182         } else if (I_<=29) {
1183             if (x<=0) r[W_-1] = 1;
1184             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1185             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1186             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1187             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1188             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1189             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1190             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1191             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1192             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16Ap0")) r = 8;
1193             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D4p0")) r = 9;
1194             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8p0")) r = 10;
1195             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB50p0")) r = 11;
1196             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A0p0")) r = 12;
1197             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D41p0")) r = 13;
1198             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82p0")) r = 14;
1199             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504p0")) r = 15;
1200             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09p0")) r = 16;
1201             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413p0")) r = 17;
1202             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A827p0")) r = 18;
1203             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504Fp0")) r = 19;
1204             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09Ep0")) r = 20;
1205             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413Cp0")) r = 21;
1206             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8279p0")) r = 22;
1207             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504F3p0")) r = 23;
1208             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09Ep0")) r = 24;
1209             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413CCp0")) r = 25;
1210             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82799p0")) r = 26;
1211             else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504F33p0")) r = 27;
1212             else
1213                 } else {
1214                     if (x<=0) r[W_-1] = 1;
1215                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1p0")) r = 0;
1216                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2p0")) r = 1;
1217                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5p0")) r = 2;
1218                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xBp0")) r = 3;
1219                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16p0")) r = 4;
1220                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2Dp0")) r = 5;
1221                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5Ap0")) r = 6;
1222                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB5p0")) r = 7;
1223                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16Ap0")) r = 8;
1224                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D4p0")) r = 9;
1225                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8p0")) r = 10;
1226                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB50p0")) r = 11;
1227                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A0p0")) r = 12;
1228                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D41p0")) r = 13;
1229                     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82p0")) r = 14;
1230                     else if

```

```

(x_s<=ap_ufixed<W_-1,I_-1>("0xB504p0")) r = 15;
1231     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09p0")) r = 16;
1232     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413p0")) r = 17;
1233     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A827p0")) r = 18;
1234     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504Fp0")) r = 19;
1235     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09Ep0")) r = 20;
1236     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413Cp0")) r = 21;
1237     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A8279p0")) r = 22;
1238     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504F3p0")) r = 23;
1239     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09Ep0")) r = 24;
1240     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413CCp0")) r = 25;
1241     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A82799p0")) r = 26;
1242     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504F33p0")) r = 27;
1243     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x16A09Ep0")) r = 28;
1244     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2D413CCp0")) r = 29;
1245     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5A827999p0")) r = 30;
1246     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB504F333p0")) r = 31;
1247     else
1248         }
1249     } else if (F_<=4 && I_<=5) {
1250         if (x<=0) r[W_-1] = 1;
1251         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.1p0")) r
= "0x4.0p0";
1252         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.2p0")) r
= "0x3.0p0";
1253         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.3p0")) r
= "0x2.7p0";
1254         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.4p0")) r
= "0x2.0p0";
1255         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.5p0")) r
= "0x1.8p0";
1256         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.6p0")) r
= "0x1.7p0";
1257         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.7p0")) r
= "0x1.3p0";
1258         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.8p0")) r
= "0x1.0p0";
1259         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.9p0")) r
= "0x0.0p0";
1260         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.Ap0")) r
= "0x0.8p0";
1261         else if (x_s==ap_ufixed<W_-1,I_-1>("0x0.Bp0")) r
= "0x0.9p0";
1262         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x0.Cp0")) r = "0x0.7p0";
1263         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x0.Dp0")) r = "0x0.5p0";
1264         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x0.Ep0")) r = "0x0.3p0";
1265         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x0.Fp0")) r = "0x0.1p0";
1266         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.0p0")) r = "0x0.0p0";
1267         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.1p0")) r = "0x0.1p0";
1268         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.2p0")) r = "0x0.3p0";
1269         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.3p0")) r = "0x0.4p0";
1270         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.4p0")) r = "0x0.5p0";
1271         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.5p0")) r = "0x0.6p0";
1272         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.6p0")) r = "0x0.7p0";
1273         else if
(x_s==ap_ufixed<W_-1,I_-1>("0x1.7p0")) r = "0x0.8p0";
1274         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.8p0")) r = "0x0.9p0";
1275         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.9p0")) r = "0x0.Ap0";
1276         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.Ap0")) r = "0x0.Bp0";
1277         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.Bp0")) r = "0x0.Cp0";
1278         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.Cp0")) r = "0x0.Dp0";
1279         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.Dp0")) r = "0x0.Ep0";
1280         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x1.Fp0")) r = "0x0.Fp0";
1281         else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.0p0")) r = "0x1.0p0";
1282         else if

```



```

(x_s<=ap_ufixed<W_-1,I_-1>("0x2.2p0")) r = "0x1.1p0";
1283     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.3p0")) r = "0x1.2p0";
1284     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.5p0")) r = "0x1.3p0";
1285     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.6p0")) r = "0x1.4p0";
1286     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.8p0")) r = "0x1.5p0";
1287     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.Ap0")) r = "0x1.6p0";
1288     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.Cp0")) r = "0x1.7p0";
1289     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x2.Ep0")) r = "0x1.8p0";
1290     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.0p0")) r = "0x1.9p0";
1291     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.2p0")) r = "0x1.Ap0";
1292     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.4p0")) r = "0x1.Bp0";
1293     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.6p0")) r = "0x1.Cp0";
1294     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.9p0")) r = "0x1.Dp0";
1295     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.Bp0")) r = "0x1.Ep0";
1296     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x3.Ep0")) r = "0x1.Fp0";
1297     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x4.1p0")) r = "0x2.0p0";
1298     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x4.4p0")) r = "0x2.1p0";
1299     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x4.7p0")) r = "0x2.2p0";
1300     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x4.Ap0")) r = "0x2.3p0";
1301     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x4.Dp0")) r = "0x2.4p0";
1302     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5.1p0")) r = "0x2.5p0";
1303     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5.4p0")) r = "0x2.6p0";
1304     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5.8p0")) r = "0x2.7p0";
1305     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x5.Cp0")) r = "0x2.8p0";
1306     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x6.0p0")) r = "0x2.9p0";
1307     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x6.4p0")) r = "0x2.Ap0";
1308     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x6.9p0")) r = "0x2.Bp0";
1309     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x6.Dp0")) r = "0x2.Cp0";
1310     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x7.2p0")) r = "0x2.Dp0";
1311     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x7.7p0")) r = "0x2.Ep0";
1312     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x7.Dp0")) r = "0x2.Fp0";
1313     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x8.2p0")) r = "0x3.0p0";
1314     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x8.8p0")) r = "0x3.1p0";
1315     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x8.Ep0")) r = "0x3.2p0";
1316     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x9.4p0")) r = "0x3.3p0";
1317     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0x9.Bp0")) r = "0x3.4p0";
1318     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xA.2p0")) r = "0x3.5p0";
1319     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xA.9p0")) r = "0x3.6p0";
1320     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB.1p0")) r = "0x3.7p0";
1321     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xB.8p0")) r = "0x3.8p0";
1322     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xC.1p0")) r = "0x3.9p0";
1323     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xC.9p0")) r = "0x3.Ap0";
1324     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xD.2p0")) r = "0x3.Bp0";
1325     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xD.8p0")) r = "0x3.Cp0";
1326     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xE.5p0")) r = "0x3.Dp0";
1327     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xE.Fp0")) r = "0x3.Ep0";
1328     else if
(x_s<=ap_ufixed<W_-1,I_-1>("0xF.Ap0")) r = "0x3.Fp0";
1329     else
        r = "0x4.0p0";
1330 } else {
1331     if (x==0) {r[W_-1] = 1;}
1332     else {
1333         const static int W_s_ = (F_+F_-/77*2+12)/15-
(100+F_-)/133+(99+F_-)/133-(100+F_-)/118+(98+F_-)/118+1;

```

```

1335
1336     const static int      wf =
log_traits<W_s_>::wf;
1337     const static int      I_s = I_>0? I_-:I_;
1338     const static int      org_wf =
log_traits<W_s_>::org_wf+I_s;
1339     const static int MaxPrecision =
log_traits<W_s_>::MaxPrecision;
1340
1341     ap_int<7> b_exp=0;
1342     ap_ufixed<1 + org_wf,1> b_frac=0;
1343     b_frac[org_wf] = 0;
1344     b_frac(org_wf-1,org_wf-W_+1) = x(W_-2,0);
1345     for (b_exp = I_-1; b_exp >= -F_; b_exp--) {
1346         #pragma HLS unroll
1347         if (!
b_frac[org_wf]&b_frac[org_wf-1]&b_frac[org_wf-2])|(b_frac[org_wf]&
b_frac[org_wf-1]))
1348             break;
1349         b_frac <= 1;
1350     }
1351
1352     ap_fixed<30,5> Elog2_s = b_exp;
1353     ap_fixed<48,5> Elog2_l = b_exp;
1354
1355
1356     typedef typename log_traits<W_s_>::LOG_TYPE
LOG_TYPE;
1357     const static int bypass_threshold =
log_traits<W_s_>::p_generic;
1358
1359     ap_uint<1+alpha0> index0; // ap_uint<6>
1360     if (b_frac[org_wf]) index0 =
b_frac(org_wf-1,org_wf-6);
1361     else
        index0 =
b_frac(org_wf-2,org_wf-7);
1362
1363     const int T0size = 1 << (1+alpha0);
1364
1365     #define inverse_lut log_inverse_lut_table::array
1366     #define log_lut log_lut_t-
able<LOG_TYPE,p0,alpha0,T0size>::array
1367
1368     ap_ufixed<1 + alpha0, 1> b_frac_tilde_inverse =
inverse_lut[index0];
1369
1370     LOG_TYPE log_sum = log_lut[index0];
1371
1372     ap_ufixed<1 + org_wf+1+(1+alpha0), 1> b_frac1 =
b_frac * b_frac_tilde_inverse;
1373
1374     #ifndef __SYNTHESIS__
1375         assert(b_frac1(b_frac1.wl()-b_frac1.iwl()-1,
b_frac1.wl()-b_frac1.iwl()-p1) == 0);
1376     #endif
1377     const int z1_width = 1+wf+1+(1+alpha0)-p1+1;
1378     ap_ufixed<z1_width, -p1> z1 =
b_frac1;
1379
1380     ap_fixed<1-bypass_threshold+1+MaxPrecision, -by-
pass_threshold+1> zk;
1381     zk =
log_traits<W_s_>::range_reduction(z1,log_sum);
1382
1383     ap_fixed<1-2*bypass_threshold+1+MaxPrecision, -
bypass_threshold+1> zk_trunc = zk;
1384     LOG_TYPE sum = zk-(zk_trunc*zk_trunc/2);
1385
1386     LOG_TYPE log_base = log_sum + sum;
1387
1388     const ap_ufixed<26,1> LOG1_54_s =
"0x1.7154765p0";
1389     const ap_ufixed<44,1> LOG1_54_l =
"0x1.71547652B82Fp0";
1390
1391     if (F_<=16) log_base = Elog2_s + log_base *
LOG1_54_s;
1392     else
        log_base = Elog2_l + log_base *
LOG1_54_l;
1393     #undef log_lut
1394     #undef inverse_lut
1395
1396     r = log_base;
1397 }
1398 }
1399 }
1400
1401     return r;
1402 }
1403
1404 template<int W_, int I_>
1405 ap_ufixed<W_,I_> log2(ap_ufixed<W_,I_> x) {
1406     ap_fixed<W_+1,I_+1> xf = x;
1407     return log2(xf);
1408 }
1409
1410 template<int I_>
1411 ap_int<I_> log2(ap_int<I_> x) {
1412     ap_fixed<I_,I_> xf = x;
1413     return log2(xf);

```

```

1414 }
1415
1416 template<int I>
1417 ap_uint<I> log2(ap_uint<I> x) {
1418     ap_fixed<I+1,I+1> xf = x;
1419     return log2(xf);
1420 }
1421
1422 template<int W_, int I>
1423 ap_fixed<W_,I> logb(ap_fixed<W_,I> x) {
1424     return log2(x);
1425 }
1426
1427 template<int W_, int I>
1428 ap_ufixed<W_,I> logb(ap_ufixed<W_,I> x) {
1429     ap_fixed<W+1,I+1> xf = x;
1430     return logb(xf);
1431 }
1432
1433 template<int I>
1434 ap_int<I> logb(ap_int<I> x) {
1435     ap_fixed<I,I> xf = x;
1436     return logb(xf);
1437 }
1438
1439 template<int I>
1440 ap_uint<I> logb(ap_uint<I> x) {
1441     ap_fixed<I+1,I+1> xf = x;
1442     return logb(xf);
1443 }
1444
1445 }
1446 #endif
1447
1448

```

E.32 vendor/etc/hls_mad_apfixed.h (64 lines, 1,944 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5 #ifndef _X_HLS_GENERIC_MAD_H_
6 #define _X_HLS_GENERIC_MAD_H_
7 #include "ap_fixed.h"
8 namespace hls_mad {
9     //mad for ap_fixed
10     template<int W_, int I>
11     ap_fixed<W_,I> generic_mad(ap_fixed<W_,I> x,
ap_fixed<W_,I> y, ap_fixed<W_,I> z){
12         const int F_ = W_ - I_;
13         ap_fixed<2*W_, 2*I> mul = x * y;
14         //rounding
15         ap_fixed<2, -F_> delta = 0;
16         delta[0] = 1;
17         delta[1] = mul[2*W_-1];
18         //add delta or subtract delta
19         mul += delta;
20         //if overflow
21         ap_fixed<W+1, I+1> mul_s = mul;
22         ap_fixed<W+1, I+1> sum = mul_s + z;
23         ap_fixed<W,I> r = 0;
24         //overflow
25         if(sum[W-1]!=sum[W_]) {
26             for(int i = 0; i < W_-1; ++i) {
27                 #pragma HLS unroll
28                 r[i] = !sum[W_];
29             }
30             r[W_-1] = sum[W_];
31         } else {
32             r = sum;
33         }
34         return r;
35     }
36     //mad for ap_ufixed
37     template<int W_, int I>
38     ap_ufixed<W_,I> generic_mad(ap_ufixed<W_,I> x,
ap_ufixed<W_,I> y, ap_ufixed<W_,I> z){
39         ap_fixed<W+1, I+1> xi = x;
40         ap_fixed<W+1, I+1> yi = y;
41         ap_fixed<W+1, I+1> zi = z;
42         return generic_mad(xi, yi, zi);
43     }
44     //mad for ap_int
45     template<int I>
46     ap_int<I> generic_mad(ap_int<I> x, ap_int<I> y,
ap_int<I> z){
47         ap_fixed<I, I> xi = x;
48         ap_fixed<I, I> yi = y;
49         ap_fixed<I, I> zi = z;
50         return generic_mad(xi, yi, zi);
51     }
52     //mad for ap_uint
53     template<int I>
54     ap_uint<I> generic_mad(ap_uint<I> x, ap_uint<I> y,
ap_uint<I> z){
55         ap_fixed<I+1,I+1> xi = x;
56         ap_fixed<I+1,I+1> yi = y;

```

```

57     ap_fixed<I+1,I+1> zi = z;
58     return generic_mad(xi,yi,zi);
59 }
60 }
61
62 #endif
63
64

```

E.33 vendor/etc/hls_modf_apfixed.h (59 lines, 1,533 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6
7 /**
8  * @file hls_modf_ap_fixed.h
9  * @brief implement modf function for ap_fixed data type.
10  */
11 #ifndef _X_HLS_MODF_APFIXED_H_
12 #define _X_HLS_MODF_APFIXED_H_
13 #include "etc/hls_round_copysign_apfixed.h"
14 namespace modf_internal {
15     template<int W, int I>
16     ap_fixed<W,I> modf(ap_fixed<W, I> x,
ap_fixed<W, I>* int_part){
17         ap_fixed<W, I> frac_part = 0;
18         if(W == I) {
19             *int_part = x;
20             frac_part = 0;
21         } else if(I > 0) {
22             ap_fixed<W+1, I+1> xi = x;
23             ap_ufixed<W, I> x_p = fabs_fixed(xi);
24             frac_part(W - I - 1,0) = x_p(W - I - 1,0);
25             if(xi[W-1]) frac_part = - frac_part;
26             *int_part = x_p(W-1, W-I);
27             if(xi[W-1]) *int_part = -*int_part;
28         } else {
29             *int_part = 0;
30             frac_part = x;
31         }
32     }
33     return frac_part;
34 }
35 template<int W, int I>
36 ap_ufixed<W,I> modf(ap_ufixed<W, I> x,
ap_ufixed<W, I>* int_part){
37     ap_fixed<W+1, I+1> xi = x;
38     ap_fixed<W+1, I+1> ii = 0;
39     ap_fixed<W+1, I+1> r = modf(xi, &ii);
40     *int_part = ii;
41     return r;
42 }
43
44 template<int I>
45 ap_int<I> modf(ap_int<I> x,
ap_int<I>* int_part){
46     *int_part = x;
47     return 0;
48 }
49
50 template<int I>
51 ap_uint<I> modf(ap_uint<I> x,
ap_uint<I>* int_part){
52     *int_part = x;
53     return 0;
54 }
55
56 }
57 #endif//ifndef _X_HLS_MODF_APFIXED_H_
58
59

```

E.34 vendor/etc/hls_pow_apfixed.h (543 lines, 18,358 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4
5 #ifndef _X_HLS_POW_APFIXED_H_
6 #define _X_HLS_POW_APFIXED_H_
7 #include "ap_fixed.h"
8 #include "etc/hls_log_apfixed.h"
9 #include "etc/hls_exp_apfixed.h"
10 #include "etc/hls_round_copysign_apfixed.h"
11 namespace pow_apfixed_reduce{
12     /*
13      * Normally, pow(x,y) = exp(ylog(|x|))*sig(x)^y.
14      * To make sure the result has 1ulp relative error, the rel-
ative error of log(|x|) is 2^(-W-I-2) and the relative error of
ylog(|x|) is 2^(-W-2).
15      * Exception handling. (Note: if 1 > Max, then return Max)
16      * =====
17      * || x || y || pow(x,y)
18      * || 0 || 0 || 1
19      * || 0 || <0 || Max
20      * || 0 || >0 || 0
21      * || 1 || Random || 1

```

```

22 * || Random || 0 || 1
23 * || <0 ||Not Integer|| 0
24 * =====
25 */
26 template<int W_,int I_>
27 ap_fixed<W_,I_> pow(ap_fixed<W_,I_> x, ap_fixed<W_,I_> y) {
28     const static int F_ = W_ - I_;
29     //only support ap_fixed with I < 34 and F < 33
30     if (I_>34) return 0;
31     else if (F_>33) return 0;
32     ap_fixed<W_,I_> r = 0;
33
34     bool r_is_neg = 0;
35     bool y_is_frac = 0;
36     //y is fraction or not
37     if (F_>0&&y(F_-1,0)>0)
38         y_is_frac = 1;
39     if (x==0) {
40         //x==0, y==0, return 0
41         if (y==0&&I_>1) {
42             return 1;
43         }
44         //x==0, y==0, return 0
45         } else if (y>0) {
46             return 0;
47         } else {
48             //x==0, y<0, return the Max
49             for (int j = 0; j < W_-1; j++){
50                 #pragma HLS unroll
51                 r[j] = 1;
52             }
53             r[W_-1] = 0;
54             return r;
55         }
56     } else if (x<0) {
57         //x<0 and y is fractoin, return 0(NaN)
58         if (y_is_frac) return 0;
59         //x<0 and y is integer, if y is odd, r is positive
60         //otherwise r is negative.
61         else {
62             if (y==0||((I_>1&&y(F_]==0))
63                 r_is_neg = 0;
64             else
65                 r_is_neg = 1;
66         }
67     }
68     //|x|
69     ap_fixed<W_+1,I_+1> x_e_1 = x;
70     //avoid overflow after abs
71     ap_fixed<W_+1,I_+1> x_p = 0;
72     x_p = fabs_fixed(x_e_1);
73     //ln(|x|)
74     //extend width of fraction
75     const static int E_l = I_+2;
76     const static int F_l = W_+E_l;
77     //The output of log shoule be less then 2^5
78     const static int I_l = I_+1>6?I_+1:6;
79     const static int W_l = F_l + I_l;
80     //std::cout<<"[POW]x_p="<<x_p.to_string()<<std::endl;
81     ap_fixed<W_l,I_l> x_l = x_p;
82     //-----
83     //std::cout<<"[POW]x_l="<<x_l.to_string()<<std::endl;
84     ap_fixed<W_l,I_l> ln_x = log_apfixed_reduce::log(x_l);
85     //std::cout<<"[POW]ln_x="<<ln_x.to_string()<<std::endl;
86     //range of log is less 2^5
87     ap_fixed<F_l+6,6> ln_x_s = ln_x;
88     //-----
89
90     //y*ln(|x|)--Maxprecision=2^(-W-2)
91     const static int FI_m = W_+2;
92     //The input of exp should be less then 2^5. If it > 2^5,
93     //i will definitely overflow.
94     const static int I_m = I_>6 ? I_ : 6; //add 1 due to min
95     //value is -2^I;
96     const static int WI_m = FI_m + I_m;
97     const static int WO_m = F_ + I_m;
98     ap_fixed<F_l+W_+6,6+I_> mul_y_ln = ln_x_s * y;
99     //
100     std::cout<<"[POW]mul_y_ln="<<mul_y_ln.to_string()<<std::endl;
101     //Rounding
102     //ap_fixed<1+, 1> delta = 0;
103     //delta[delta.wl()-1] = mul_e[mul_e.wl()-1];
104     //delta[delta.wl()-delta.iwl()-1] = 1;
105     //mul_e = mul_e + ( delta >> wf_m );
106     //std::cout<<"mul_e="<<mul_e.to_string()<<std::endl;
107
108     //check overflow before truncate
109     ap_fixed<WO_m,I_m> exp_r = 0;
110     ap_fixed<W_,I_> r_1 = 0;
111     bool m_overf = 0;
112     bool e_overf = 0;
113     bool overf = 0;
114     //check overflow or underflow after multiply
115     for (int j = F_l+F_+I_m-1; j < F_l+W_+5; j++){
116         #pragma HLS unroll
117         if (mul_y_ln[F_l+W_+5]!=mul_y_ln[j])
118             m_overf = 1;
119     }
120
121     if (!m_overf) {
122         //truncate multiply result
123
124         ap_fixed<WI_m,I_m> mul_y_ln_s = mul_y_ln;
125         //
126         std::cout<<"[POW]mul_y_ln_s="<<mul_y_ln_s.to_string()<<std::endl;
127         exp_r = exp_reduce::exp_core<WO_m,I_m,WI_m>(mul_y_ln_s); //
128         //
129         std::cout<<"[POW]exp_r="<<exp_r.to_string()<<std::endl;
130         //check overflow after exp
131         if (I_<I_m) {
132             for (int j = WO_m-1; j >= W_-1; j--) {
133                 #pragma HLS unroll
134                 if (exp_r[j])
135                     e_overf=1;
136             }
137         }
138         r_1 = exp_r;
139     }
140
141     //overflow when exp reults overflow or multiply result is
142     //positive and ovelow
143     if (e_overf||((m_overf&&!mul_y_ln[F_l+W_+5])) {
144         overf = 1;
145     }
146     //sig(x)^y = -1
147     if (r_is_neg) {
148         //overflow, return MIN
149         if (overf) {
150             r=0;
151             r[W_-1]=1;
152             //return -exp(y*log(|x|))
153             } else {
154                 if (r_1!=0) {
155                     r = -r_1;
156                     r[W_-1] = 1;
157                 }
158             }
159         }
160     }
161     //sig(x)^y = 1
162     } else {
163         //return MAX
164         if (overf) {
165             r[W_-1] = 0;
166             for (int j = W_-2; j >= 0; j--){
167                 #pragma HLS unroll
168                 r[j] = 1;
169             }
170             //return exp(y*log(|x|))
171             } else {
172                 r = r_1;
173             }
174         }
175     }
176     //std::cout<<"[OWW]r="<<r.to_double()<<std::endl;
177     return r;
178 }
179
180 /*
181 * Normally, pown(x,n) = exp(log(|x|)*n)*sig(x)^n.
182 * To make sure the result has iulp relative error,
183 * the relative error of log(|x|) is 2^(-F-I/2+1) and the re-
184 * lative error of log(|x|)/n is 2^(-F-I/2).
185 * Exception handling. (Note: if output > Max, then return
186 * Max)
187 */
188
189 * =====
190 * || x || n || pown(x,n)
191 * || 0 || 0 || 1
192 * || 0 || any || 0
193 * || 1 || any || 1
194 * || -1 || any || +/-1
195 * || any || 0 || 1
196 * || any || 1 || x
197 * || any || -1 || 1/x
198 * || x!=0 || n!=0,1,-1 || exp(log|x|*n)*sig(x)^n
199 * =====
200
201 * optimization: implement the exp for input <W,I> I<5,
202 * W<32+5 and output<W,I> W<65, I<32
203 */
204
205 template<int W_, int I_>
206 ap_fixed<W_,I_> pown(ap_fixed<W_,I_> x, int n) {
207     #ifndef __SYNTHESIS__
208     std::cout<<"[POWN]x="<<x.to_string()<<std::endl;
209     std::cout<<"[POWN]n="<<n<<std::endl;
210     #endif
211     #ifndef HLS_DEBUG
212     const static int F_ = W_ - I_;
213     //only support ap_fixed with I < 34 and F < 33
214     if (I_>34) return 0;
215     else if (F_>33) return 0;
216
217     ap_fixed<W_,I_> r = 0;
218     ap_fixed<32,32> n_fix = n;
219
220     bool x_sig = x[W_-1];
221     bool n_sig = n_fix[31];
222     bool n_is_odd = n_fix[0];
223     bool x_gt_0 = 0;
224
225     if (x == 0 && n_fix != 0) r = 0;
226     else if (n_fix == 0) r = 1;
227     else if (n_fix == 1) r = x;
228     //else if (n_fix == -1) r = 1/x;

```

```

209     else if(x == 1)      r = 1;
210     else if(x == -1) {
211         if(n_is_odd) r = -1;
212         else        r = 1;
213     } else {
214         //if x in [2, Max], and (0, 0.5]
215         //-----then n must be less than 32 to avoid overflow
216         //-----calculat the width of log
217         //-----the extra width is 5 for 1ulp precision after
mul;
218         //-----the extra extern width is I+1 for 1ulp precision
after exp;
219         //x in [] for n >= 2, it will overflow;
220         //x in (0.5, 2) it will be difficult;
221         //
=====
222         //|| I || F || x || precision of log ||
max(n)      || Note ||
223         //|| any || 0 || || 5 + I ||
<32         //|| 0 || overflow || || 5 + F ||
>0          //|| overflow ||
225         //|| any || <27 || || F + I + log2(max(n)) || lln2/
ln(1+1/2^F)||
226         //|| any || >=27 || || 31 + F + I ||
2^31         //||
227         //
=====
228         //|| >18 i || <27 || || 5 + F + F + I || lln2/
ln(1+1/2^F)||
229         //|| >9, <=18 || <27 || || 4 + F + F + I || lln2/
ln(1+1/2^F)||
230         //|| >4, <=9 || <27 || || 3 + F + F + I || lln2/
ln(1+1/2^F)||
231         //|| >2, <=4 || <27 || || 2 + F + F + I || lln2/
ln(1+1/2^F)||
232         //|| 2 || <27 || || 1 + F + F + I || lln2/
ln(1+1/2^F)|| maybe optimize further
233         //|| 1 || <27 || || 0 + F + F + I || lln2/
ln(1+1/2^F)||
234         //worse scenario
235         //x = 1 + 1/2^F, n = max
236         //x = 1 - 1/2^F, n = max
237         //x = 1 + 1/2^F, n = min
238         //x = 1 - 1/2^F, n = min
239
240         const static int we_n = F_ > 26? 32 : 6 + F_;//F_+
(I_+32)/(20+32) + (I_+32)/(11+32) + (I_+32)/(6+32) + (I_+32)/
(4+32)+ (I_+32)/(3+32) + (I_+32)/(1+32);
241         //const static ap_ufixed<32,32> max_n = we_n > 31?
2147483648:(1 << (we_n-1)) - 1;
242         const static int wf_log = I_ == F_>26 ? (31+F_+I_) :
(we_n-1+F_+I_);
243         const static int I_e = I_>6 ? I_ : 6;//add 1 due to
min value is -2^I;
244         const static int WI_e = I_e + wf_log - we_n;
245         const static int W0_e = F_ + I_e;
246         ap_fixed<33,33> max_n = 0;
247         max_n[we_n-1] = 1;
248         // ap_ufixed<32,32> n_pos =
fabs_fixed(ap_fixed<33,33>(n));
249         ap_ufixed<W0_e,I_e> exp_r=0;
250         bool ovf = 0;
251         ap_ufixed<W_,I_> x_pos =
fabs_fixed((ap_fixed<W_+1,I_+1>)x);
252         if(x_pos>1) x_gt_0 = 1;
253         ap_fixed<32,32> min_n = -max_n;
254         if(n_fix != -1 && ((n >= max_n) || (n < min_n))){
255             //overflow or underflow
256             ovf = 1;
257         } else {
258             ap_fixed<we_n, we_n> n_s = n;
259             #ifndef __SYNTHESIS__
260             assert(n_s == n);
261             #endif
262
263             //extend the width of x to MaxPrecision
264             const static int I_l = I_+1 < 6? 6 : I_+1;
265             ap_fixed<wf_log+I_l,I_l> x_e = x_pos;
266             const static int we_log = 6;//I_+1>6? I_+1:6;
267             ap_fixed<wf_log+we_log,we_log> x_log = log_ap-
fixed_reduce::log(x_e);
268             #ifndef __SYNTHESIS__
269             #ifdef HLS_DEBUG
270
std::cout<<"[POWN]x_log="<<x_log.to_string()<<std::endl;
271             #endif
272             #endif
273             //the absolute of mul will enlgrae
274
ap_fixed<wf_log+we_log, we_log+we_n> x_log_mul_n =
n_s * x_log;
275
276             #ifndef __SYNTHESIS__
277             #ifdef HLS_DEBUG
std::cout<<"[POWN]x_log_mul_n="<<x_log_mul_n.to_string()<<std::endl;
278             #endif
279             #endif
;

```

```

278         #endif
279         #endif
280
281         for(int i = wf_log+we_log-2; i > WI_e - 2; --i) {
282             #pragma HLS unroll
283             if(x_log_mul_n[i] !=
x_log_mul_n[wf_log+we_log-1]){
284                 //underflow or overflow
285                 ovf = 1;
286             }
287         }
288         ap_fixed<WI_e, I_e> x_log_mul_n_1 = x_log_mul_n ;
289         #ifndef __SYNTHESIS__
290         #ifdef HLS_DEBUG
291
std::cout<<"x_log_mul_n_1="<<x_log_mul_n.to_string()<<std::endl;
292         #endif
293         #endif
294         exp_r = exp_re-
duce::exp_core<W0_e,I_e,WI_e>(x_log_mul_n_1);
295         #ifndef __SYNTHESIS__
296         #ifdef HLS_DEBUG
297
std::cout<<"[POWN]exp_r="<<exp_r.to_string()<<std::endl;
298         #endif
299         #endif
300         if (I_ < I_e) {
301             for (int j = W0_e-1; j >= W_-1; j--) {
302                 #pragma HLS unroll
303                 //overflow
304                 if (exp_r[j])
305                     ovf=1;
306             }
307         }
308         if(ovf) {
309             if(x_gt_0 ^ n_sig) {
310                 //overflow
311                 if(x_sig && !n_is_odd) {
312                     r[W_-1] = 1;
313                 } else {
314                     for(int i = 0; i < W_-1; ++i) {
315                         #pragma HLS unroll
316                         r[W_-1] = 1;
317                     }
318                 }
319             }
320             } else {
321                 if(x_sig && n_is_odd) r = -exp_r;
322                 else
323                     r = exp_r;
324             }
325         }
326         return r;
327     }
328
329     /*
330     * Normally, rootn(x,n) = exp(log(|x|)/n)*sig(x)^(1/n).
331     * To make sure the result has 1ulp relative error,
332     * the relative error of log(|x|) is 2^(-F-I/2+1) and the re-
lative error of log(|x|)/n is 2^(-F-I/2).
333     * Exception handling. (Note: if output > Max, then return
Max)
334     * =====
335     * || x || n || rootn(x,n)
336     * || any || 0 || 0
337     * || any || 1 || x
338     * || any || -1 || 1/x
339     * || x<0 || n is even || 0
340     * || x<0 || n is odd || -exp(log(|x|)/n)
341     * || 0 || any || 0
342     * || 1 || !=0 || 1
343     * || -1 || n is odd || -1
344     * || x>0 || !=0 || exp(log(|x|)/n)
345     * =====
346     * optimization: implement the exp for input <W,I> W<65+32/2
and output<W,I> W<65, I<32
347     */
348     //worst scenario
349     //x = 2^I, n = 2, r ~ 2^(I/2)
350     //x = 2^I, n = -2, r ~ 1/2^(I/2)
351     //x = 1/2^F, n = -2, r ~ 2^(F/2)
352     //x = 1/2^F, n = 2, r ~ 1/2^(F/2)
353     //extend width = min(I, max(F/2,I/2))
354     template<int W_,int I_>
355     ap_fixed<W_,I_> rootn(ap_fixed<W_,I_> x, int n) {
356         #ifndef __SYNTHESIS__
357         #ifdef HLS_DEBUG
358             std::cout<<"x="<<x.to_string()<<std::endl;
359             std::cout<<"n="<<n<<std::endl;
360             #endif
361             #endif
362         const static int F_ = W_ - I_;
363         //only support ap_fixed with I < 34 and F < 33
364         if (I_ > 34) return 0;
365         else if (F_ > 33) return 0;
366
367         ap_fixed<W_,I_> r = 0;
368         ap_fixed<32,32> n_fix = n;
369         bool n_is_odd = 0;
370         if(n_fix[0]) n_is_odd = 1;

```

```

371
372     if (x == 0)         r = 0;
373     else if(n_fix == 0) r = 0;
374     else if(n_fix == 1) r = x;
375     else if(n_fix == -1) r = ap_fixed<W_, I_>(1)/x;
376     else if(x<0 && !n_is_odd) r = 0;
377     else if(x==1)       r = 1;
378     else if(x== -1 && n_is_odd) r = -1;
379     else {
380         //|x|
381         ap_ufixed<W_, I_> x_p =
fabs_fixed((ap_fixed<W_+1, I_+1>)x);
382         //ln(|x|)
383         //extend width of fraction
384         const static int m_we = (F_+1)/2 > (I_+1)/2? (F_+1)/
2:(I_+1)/2;
385         const static int we = I_ > m_we ? m_we : I_;
386         const static int E_l = we - 1;
387         const static int F_l = F_ + E_l;
388         //The output of log should be less than 2^5
389         const static int I_l = I_ + 1 > 6 ? I_ + 1 : 6;
390         const static int W_l = F_l + I_l;
391         //std::cout<<"[POW]x_p="<<x_p.to_string()<<std::endl;
392         ap_fixed<W_l, I_l> x_l = x_p;
393         //-----
394         //std::cout<<"[POW]x_l="<<x_l.to_string()<<std::endl;
395         ap_fixed<W_l, I_l> ln_x = log_ap-
fixed_reduce::log(x_l);
396         #ifndef __SYNTHESIS__
397         #ifdef HLS_DEBUG
398
std::cout<<"[ROOTN]ln_x="<<ln_x.to_string()<<std::endl;
399         #endif
400         #endif
401         //range of log is less 2^5
402         ap_fixed<F_l+6, 6> ln_x_s = ln_x;
403         //-----
404         //ln(|x|)/n--Maxprecision=2^(-F-I/2)
405         const static int FI_d = F_l + 1;
406         //The input of exp should be less than 2^5. If it >
2^5, i will definitely overflow.
407         const static int I_d = we+1 > 5? we+1 : 5;
408         const static int WI_d = FI_d + I_d;
409         const static int WO_d = F_ + I_d;
410         ap_fixed<F_l+6, 5> divd_n_ln = ln_x_s / n_fix;
411         #ifndef __SYNTHESIS__
412         #ifdef HLS_DEBUG
413
std::cout<<"[ROOTN]divd_yn_ln="<<divd_n_ln.to_string()<<std::endl;
414         #endif
415         #endif
416
//truncate divide result
417         ap_fixed<WI_d, I_d> divd_n_ln_s = divd_n_ln;
418         #ifndef __SYNTHESIS__
419         #ifdef HLS_DEBUG
420
std::cout<<"[ROOTN]divd_n_ln_s="<<divd_n_ln_s.to_string()<<std::en
dl;
421         #endif
422         #endif
423         #endif
424         ap_ufixed<WO_d, I_d> exp_r = exp_re-
duce::exp_core<WO_d, I_d, WI_d>(divd_n_ln_s);
425         #ifndef __SYNTHESIS__
426         #ifdef HLS_DEBUG
427
std::cout<<"[ROOTN]exp_r="<<exp_r.to_string()<<std::endl;
428         #endif
429         #endif
430         bool ovf = 0;
431         //check overflow after exp
432         if(I_ < I_d) {
433             for (int j = WO_d-1; j >= W_-1; j--) {
434                 #pragma HLS unroll
435                 //overflow
436                 if (exp_r[j])
437                     ovf=1;
438             }
439         }
440         if(!ovf) {
441             if(x[W_-1]&&n_is_odd) r = -exp_r;
442             else r = exp_r;
443         }else {
444             if(x[W_-1]&&n_is_odd) {
445                 r[W_-1] = 1;
446             } else {
447                 for(int i = 0; i < W_-1; ++i) {
448                     #pragma HLS unroll
449                     r[i] = 1;
450                 }
451             }
452         }
453     }
454     return r;
455 }
456
457
458 template<int W_, int I_>

```

```

459 ap_ufixed<W_, I_> pow(ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y)
{
460     ap_fixed<W_+1, I_+1> xf = x;
461     ap_fixed<W_+1, I_+1> yf = y;
462     return pow(xf, yf);
463 }
464
465 template<int I_>
466 ap_int<I_> pow(ap_int<I_> x, ap_int<I_> y) {
467     ap_fixed<I_, I_> xf = x;
468     ap_fixed<I_, I_> yf = y;
469     return pow(xf, yf);
470 }
471
472 template<int I_>
473 ap_uint<I_> pow(ap_uint<I_> x, ap_uint<I_> y) {
474     ap_fixed<I_+1, I_+1> xf = x;
475     ap_fixed<I_+1, I_+1> yf = y;
476     return pow(xf, yf);
477 }
478 //powr is same with pown except for NaN, infinite.
479 //for ap_fixed, no need consider the NaN, infinite.
480 template<int W_, int I_>
481 ap_fixed<W_, I_> powr(ap_fixed<W_, I_> x, ap_fixed<W_, I_> y) {
482     return pow(x, y);
483 }
484
485 template<int W_, int I_>
486 ap_ufixed<W_, I_> powr(ap_ufixed<W_, I_> x, ap_ufixed<W_, I_> y) {
487     ap_fixed<W_+1, I_+1> xf = x;
488     ap_fixed<W_+1, I_+1> yf = y;
489     return pow(xf, yf);
490 }
491
492 template<int I_>
493 ap_int<I_> powr(ap_int<I_> x, ap_int<I_> y) {
494     ap_fixed<I_, I_> xf = x;
495     ap_fixed<I_, I_> yf = y;
496     return pow(xf, yf);
497 }
498
499 template<int I_>
500 ap_uint<I_> powr(ap_uint<I_> x, ap_uint<I_> y) {
501     ap_fixed<I_+1, I_+1> xf = x;
502     ap_fixed<I_+1, I_+1> yf = y;
503     return pow(xf, yf);
504 }
505
506 template<int W_, int I_>
507 ap_ufixed<W_, I_> pown(ap_ufixed<W_, I_> x, int n) {
508     ap_fixed<W_+1, I_+1> xi = x;
509     return pown(xi, n);
510 }
511
512 template<int I_>
513 ap_int<I_> pown(ap_int<I_> x, int n) {
514     ap_fixed<I_, I_> xi = x;
515     return pown(xi, n);
516 }
517
518 template<int I_>
519 ap_uint<I_> pown(ap_uint<I_> x, int n) {
520     ap_fixed<I_+1, I_+1> xi = x;
521     return pown(xi, n);
522 }
523
524 template<int W_, int I_>
525 ap_ufixed<W_, I_> rootn(ap_ufixed<W_, I_> x, int n) {
526     ap_fixed<W_+1, I_+1> xi = x;
527     return rootn(xi, n);
528 }
529
530 template<int I_>
531 ap_int<I_> rootn(ap_int<I_> x, int n) {
532     ap_fixed<I_, I_> xi = x;
533     return rootn(xi, n);
534 }
535
536 template<int I_>
537 ap_uint<I_> rootn(ap_uint<I_> x, int n) {
538     ap_fixed<I_+1, I_+1> xi = x;
539     return rootn(xi, n);
540 }
541 #endif
542
543 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689

```

E.35 vendor/etc/hls_round_copysign_apfixed.h (540 lines, 11,796 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5

```

```

6 #ifndef _X_HLS_ROUND_COPYSIGN_AFFIXED_H_
7 #define _X_HLS_ROUND_COPYSIGN_AFFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10
11 template <int W_, int I_>
12 ap_fixed<W_, I_> ceil_fixed(ap_fixed<W_, I_> x)
13 {
14     #pragma HLS pipeline
15     if(W_ == I_) {
16         return x;
17     } else {
18         if(I_ < 0) return 0;
19         else {
20             if ( x(W_-I_-1,0) != 0 ) {
21                 x(W_-I_-1,0) = 0;
22                 x += 1;
23             }
24             return x;
25         }
26     }
27 }
28 template <int W_, int I_>
29 ap_ufixed<W_, I_> ceil_fixed(ap_ufixed<W_, I_> x)
30 {
31     #pragma HLS pipeline
32     if(W_ == I_) {
33         return x;
34     }
35     else {
36         if(I_ < 0) return 0;
37         else {
38             if ( x(W_-I_-1,0) != 0 ) {
39                 x(W_-I_-1,0) = 0;
40                 x += 1;
41             }
42             return x;
43         }
44     }
45 }
46 template <int I_>
47 ap_int<I_> ceil_fixed(ap_int<I_> x)
48 {
49     return x;
50 }
51 template <int I_>
52 ap_uint<I_> ceil_fixed(ap_uint<I_> x)
53 {
54     return x;
55 }
56
57 template <int W_, int I_>
58 ap_fixed<W_, I_> floor_fixed(ap_fixed<W_, I_> x)
59 {
60     if(W_ == I_) {
61         return x;
62     }
63     else {
64         if(I_ < 0) return 0;
65         else {
66             if ( x(W_-I_-1,0) != 0 ) x(W_-I_-1,0) = 0;
67             return x;
68         }
69     }
70 }
71 template <int W_, int I_>
72 ap_ufixed<W_, I_> floor_fixed(ap_ufixed<W_, I_> x)
73 {
74     ap_fixed<W_+1, I_+1> xi = x;
75     return floor_fixed(xi);
76 }
77 template <int I_>
78 ap_int<I_> floor_fixed(ap_int<I_> x)
79 {
80     return x;
81 }
82 template <int I_>
83 ap_uint<I_> floor_fixed(ap_uint<I_> x)
84 {
85     return x;
86 }
87
88 template <int W_, int I_>
89 ap_fixed<W_, I_> trunc_fixed(ap_fixed<W_, I_> x)
90 {
91     #pragma HLS pipeline
92     if(W_ == I_) {
93         return x;
94     }
95     else {
96         if(I_ < 0) return 0;
97         else {
98             if ( x(W_-I_-1,0) != 0 ) {
99                 x(W_-I_-1,0) = 0;
100                 if ( x[W_-1] )
101                     x += 1;
102             }
103             return x;
104         }
105     }

```

```

106 }
107 template <int W_, int I_>
108 ap_ufixed<W_, I_> trunc_fixed(ap_ufixed<W_, I_> x)
109 {
110     ap_fixed<W_+1, I_+1> xi = x;
111     return trunc_fixed(xi);
112 }
113 template <int I_>
114 ap_int<I_> trunc_fixed(ap_int<I_> x)
115 {
116     return x;
117 }
118 template <int I_>
119 ap_uint<I_> trunc_fixed(ap_uint<I_> x)
120 {
121     return x;
122 }
123
124 template <int W_, int I_>
125 ap_fixed<W_, I_> copysign_fixed(ap_fixed<W_, I_> x,
126 ap_fixed<W_, I_> y)
127 {
128     x[W_-1] = y[W_-1];
129     return x;
130 }
131 template <int W_, int I_>
132 ap_ufixed<W_, I_> copysign_fixed(ap_ufixed<W_, I_> x,
133 ap_ufixed<W_, I_> y)
134 {
135     _AP_UNUSED_PARAM(y);
136     return x;
137 }
138 template <int I_>
139 ap_int<I_> copysign_fixed(ap_int<I_> x, ap_int<I_> y)
140 {
141     x[I_-1] = y[I_-1];
142     return x;
143 }
144 template <int I_>
145 ap_uint<I_> copysign_fixed(ap_uint<I_> x, ap_uint<I_> y)
146 {
147     _AP_UNUSED_PARAM(y);
148     return x;
149 }
150 template <int W_, int I_>
151 ap_fixed<W_, I_> fabs_fixed(ap_fixed<W_, I_> x)
152 {
153     ap_fixed<W_, I_> xs = -x;
154     xs[W_-1] = 0;
155     return ( ( x[W_-1] ) ? xs : x );
156 }
157 template <int W_, int I_>
158 ap_ufixed<W_, I_> fabs_fixed(ap_ufixed<W_, I_> x)
159 {
160     return x;
161 }
162 template <int I_>
163 ap_int<I_> fabs_fixed(ap_int<I_> x)
164 {
165     ap_int<I_> xs = -x;
166     xs[I_-1] = 0;
167     return ( ( x[I_-1] ) ? xs : x );
168 }
169 template <int I_>
170 ap_uint<I_> fabs_fixed(ap_uint<I_> x)
171 {
172     return x;
173 }
174 template <int W_, int I_>
175 ap_fixed<W_, I_> round_fixed(ap_fixed<W_, I_> x)
176 {
177     #pragma HLS pipeline
178     if(W_ == I_) {
179         return x;
180     }
181     else {
182         if(I_ < 0) return 0;
183         else {
184             ap_ufixed<1, 0> half_val = 0.5;
185             ap_ufixed<W_, I_> x_pos = fabs_fixed(x);
186             ap_ufixed<W_+1, I_+1> r = x_pos + half_val;
187             r(W_-I_-1,0) = 0;
188             if(x[W_-1]) return -r;
189             else return r;
190         }
191     }
192 }
193 template <int W_, int I_>
194 ap_ufixed<W_, I_> round_fixed(ap_ufixed<W_, I_> x)
195 {
196     ap_fixed<W_+1, I_+1> xi = x;
197     return round_fixed(xi);
198 }
199 template <int I_>
200 ap_int<I_> round_fixed(ap_int<I_> x)
201 {
202     return x;
203 }

```



```

204 template <int I_>
205 ap_uint<I_> round_fixed(ap_uint<I_> x)
206 {
207     return x;
208 }
209
210 template <int W_, int I_>
211 ap_fixed<W_, I_> rint_fixed(ap_fixed<W_, I_> x)
212 {
213     #pragma HLS pipeline
214     if(W_ == I_) {
215         return x;
216     }
217     else {
218         if(I_ < 0) return 0;
219         else {
220             ap_ufixed<W_, I_> x_p =
221             fabs_fixed((ap_fixed<W_+1, I_+1>)x);
222             ap_ufixed<1, 0> half_val = 0.5;
223             ap_ufixed<W_+1, I_+1> xUp = x_p + half_val;
224             if(xUp(W_-I_-1, 0) == 0) {
225                 //round to even
226                 xUp[W_-I_] = 0;
227             } else {
228                 xUp(W_-I_-1, 0) = 0;
229             }
230             if(x[W_-1]) return -xUp;
231             else return xUp;
232         }
233         //if ( x(W_-I_-1, 0) != 0 ) {
234         //    ap_fixed<W_+2, I_+2> xs = 0;
235         //    ap_fixed<W_, I_> tmp=0;
236         //    ap_fixed<2, 1> half_val = 0.5;
237         //    if(x[W_-1]) {
238         //        xs = -x;
239         //        xs = xs + half_val;
240         //        tmp= xs;
241         //        if (tmp(W_-I_-1, 0) != 0) {
242         //            xs(W_-I_-1, 0) = 0;
243         //        }
244         //        else {
245         //            xs[W_-I_] = 0;
246         //        }
247         //        x = -xs;
248         //    }
249         //    else {
250         //        xs = x + half_val;
251         //        tmp= xs;
252         //        if (tmp(W_-I_-1, 0) != 0) {
253         //            xs(W_-I_-1, 0) = 0;
254         //        }
255         //        else {
256         //            xs[W_-I_] = 0;
257         //        }
258         //        x = xs;
259         //    }
260         //    //}
261         //    //return x;
262     }
263 }
264
265 template <int W_, int I_>
266 ap_ufixed<W_, I_> rint_fixed(ap_ufixed<W_, I_> x)
267 {
268     ap_fixed<W_+1, I_+1> xi = x;
269     return rint_fixed(xi);
270 }
271
272 template <int I_>
273 ap_int<I_> rint_fixed(ap_int<I_> x)
274 {
275     return x;
276 }
277
278 template <int I_>
279 ap_uint<I_> rint_fixed(ap_uint<I_> x)
280 {
281     return x;
282 }
283
284 template <int W_, int I_>
285 ap_fixed<W_, I_> nearbyint_fixed(ap_fixed<W_, I_> x)
286 {
287     //for ap_fixed, there is no difference between rint and
288     nearbyint.
289     return rint_fixed(x);
290 }
291
292 template <int W_, int I_>
293 ap_ufixed<W_, I_> nearbyint_fixed(ap_ufixed<W_, I_> x)
294 {
295     ap_fixed<W_+1, I_+1> xi = x;
296     return nearbyint_fixed(xi);
297 }
298
299 template <int I_>
300 ap_int<I_> nearbyint_fixed(ap_int<I_> x)
301 {
302     return x;
303 }
304
305 template <int I_>
306 ap_uint<I_> nearbyint_fixed(ap_uint<I_> x)
307 {
308     return x;
309 }

```

```

302     return x;
303 }
304
305 template <int W_, int I_>
306 bool signbit_fixed(ap_fixed<W_, I_> x){
307     if(x[W_-1]) return true;
308     else return false;
309 }
310
311 template <int W_, int I_>
312 bool signbit_fixed(ap_ufixed<W_, I_> x){
313     _AP_UNUSED_PARAM(x);
314     return false;
315 }
316
317 template <int I_>
318 bool signbit_fixed(ap_int<I_> x){
319     if(x[I_-1]) return true;
320     else return false;
321 }
322
323 template <int I_>
324 bool signbit_fixed(ap_uint<I_> x){
325     _AP_UNUSED_PARAM(x);
326     return false;
327 }
328
329 namespace fp_internal {
330     template<int W, int I>
331     ap_fixed<W, I> nextafter(ap_fixed<W, I> x, ap_fixed<W, I>
332     y){
333         const static int F = W - I;
334         ap_ufixed<1, -F+1> ulp = 0;
335         ulp[0] = 1;
336         ap_fixed<W, I> r = 0;
337         if(x == y) r = y;
338         else if(x < y) r = x + ulp;
339         else r = x - ulp;
340         return r;
341     }
342
343     template<int W, int I>
344     ap_ufixed<W, I> nextafter(ap_ufixed<W, I> x, ap_ufixed<W,
345     I> y) {
346         ap_fixed<W+1, I+1> xi = x;
347         ap_fixed<W+1, I+1> yi = y;
348         return nextafter(xi, yi);
349     }
350
351     template<int I>
352     ap_int<I> nextafter(ap_int<I> x, ap_int<I> y) {
353         ap_fixed<I, I> xi = x;
354         ap_fixed<I, I> yi = y;
355         return nextafter(xi, yi);
356     }
357
358     template<int I>
359     ap_uint<I> nextafter(ap_uint<I> x, ap_uint<I> y) {
360         ap_fixed<I+1, I+1> xi = x;
361         ap_fixed<I+1, I+1> yi = y;
362         return nextafter(xi, yi);
363     }
364
365     template<int W, int I>
366     ap_fixed<W, I> nexttoward(ap_fixed<W, I> x, ap_fixed<W, I>
367     y){
368         return nextafter(x, y);
369     }
370
371     template<int W, int I>
372     ap_ufixed<W, I> nexttoward(ap_ufixed<W, I> x, ap_ufixed<W,
373     I> y){
374         return nextafter(x, y);
375     }
376
377     template<int I>
378     ap_int<I> nexttoward(ap_int<I> x, ap_int<I> y){
379         return nextafter(x, y);
380     }
381
382     template<int I>
383     ap_uint<I> nexttoward(ap_uint<I> x, ap_uint<I> y){
384         return nextafter(x, y);
385     }
386 }
387
388 //llround(ap_fixed)
389 template <int W, int I>
390 long long int llround_fixed(ap_fixed<W, I> x){
391     return round_fixed<W+1, I+1>(ap_fixed<W+1, I+1> (x));
392 }
393
394 //llround(ap_ufixed)
395 template <int W, int I>
396 long long int llround_fixed(ap_ufixed<W, I> x){
397     return round_fixed<W+1, I+1>(ap_ufixed<W+1, I+1> (x));
398 }
399
400 //llround(ap_int)
401 template <int I>
402 long long int llround_fixed(ap_int<I> x){
403     return x;
404 }
405
406 //llround(ap_uint)
407 template <int I>
408 long long int llround_fixed(ap_uint<I> x){
409     return x;
410 }

```



```

398 }
399
400 // lround(ap_fixed)
401 template <int W, int I>
402 long int lround_fixed(ap_fixed<W,I> x){
403     ap_int<8*sizeof(long int)> minval = 0;
404     minval[8*sizeof(long int)-1] = 1;
405     ap_int<8*sizeof(long int)> maxval = -1;
406     maxval[8*sizeof(long int)-1] = 0;
407     if((x>maxval) || (x<minval)) {
408         return (x[W-1])? minval : maxval;
409     }
410     else {
411         return round_fixed<W+1,I+1>(ap_fixed<W+1,I+1> (x));
412     }
413 }
414
415 // lround(ap_ufixed)
416 template <int W, int I>
417 long int lround_fixed(ap_ufixed<W,I> x){
418     ap_int<8*sizeof(long int)> maxval = -1;
419     maxval[8*sizeof(long int)-1] = 0;
420     if((x>maxval) ) {
421         return maxval;
422     }
423     else {
424         return round_fixed<W+1,I+1>(ap_ufixed<W+1,I+1> (x));
425     }
426 }
427
428 // lround(ap_int)
429 template <int I>
430 long int lround_fixed(ap_int<I> x){
431     ap_int<8*sizeof(long int)> minval = 0;
432     minval[8*sizeof(long int)-1] = 1;
433     ap_int<8*sizeof(long int)> maxval = -1;
434     maxval[8*sizeof(long int)-1] = 0;
435     if((x>maxval) || (x<minval)) {
436         return (x[I-1])? minval : maxval;
437     }
438     else {
439         return x;
440     }
441 }
442
443 // lround(ap_uint)
444 template <int I>
445 long int lround_fixed(ap_uint<I> x){
446     ap_int<8*sizeof(long int)> maxval = -1;
447     maxval[8*sizeof(long int)-1] = 0;
448     if((x>maxval) ) {
449         return maxval;
450     }
451     else {
452         return x;
453     }
454 }
455
456 // llrint_fixed(ap_fixed)
457 template <int W, int I>
458 long long int llrint_fixed(ap_fixed<W,I> x){
459     return rint_fixed<W+1,I+1>(ap_fixed<W+1,I+1> (x));
460 }
461
462 // llrint_fixed(ap_ufixed)
463 template <int W, int I>
464 long long int llrint_fixed(ap_ufixed<W,I> x){
465     return rint_fixed<W+1,I+1>(ap_ufixed<W+1,I+1> (x));
466 }
467
468 // llrint_fixed(ap_int)
469 template <int I>
470 long long int llrint_fixed(ap_int<I> x){
471     return x;
472 }
473
474 // llrint_fixed(ap_uint)
475 template <int I>
476 long long int llrint_fixed(ap_uint<I> x){
477     return x;
478 }
479
480 // lrint_fixed(ap_fixed)
481 template <int W, int I>
482 long int lrint_fixed(ap_fixed<W,I> x){
483     ap_int<8*sizeof(long int)> minval = 0;
484     minval[8*sizeof(long int)-1] = 1;
485     ap_int<8*sizeof(long int)> maxval = -1;
486     maxval[8*sizeof(long int)-1] = 0;
487     if((x>maxval) || (x<minval)) {
488         return (x[W-1])? minval : maxval;
489     }
490     else {
491         return rint_fixed<W+1, I+1>(ap_fixed<W+1,I+1> (x));
492     }
493 }
494
495 // lrint_fixed(ap_ufixed)
496 template <int W, int I>

```

```

498 long int lrint_fixed(ap_ufixed<W,I> x){
499     ap_int<8*sizeof(long int)> maxval = -1;
500     maxval[8*sizeof(long int)-1] = 0;
501     if(x>maxval){
502         return maxval;
503     }
504     else {
505         return rint_fixed<W+1, I+1>(ap_ufixed<W+1,I+1> (x));
506     }
507 }
508
509 // lrint_fixed(ap_int)
510 template <int I>
511 long int lrint_fixed(ap_int<I> x){
512     ap_int<8*sizeof(long int)> minval = 0;
513     minval[8*sizeof(long int)-1] = 1;
514     ap_int<8*sizeof(long int)> maxval = -1;
515     maxval[8*sizeof(long int)-1] = 0;
516     if((x>maxval) || (x<minval)) {
517         return (x[I-1])? minval : maxval;
518     }
519     else {
520         return x;
521     }
522 }
523
524 // lrint_fixed(ap_uint)
525 template <int I>
526 long int lrint_fixed(ap_uint<I> x){
527     ap_int<8*sizeof(long int)> maxval = -1;
528     maxval[8*sizeof(long int)-1] = 0;
529     if(x>maxval) {
530         return maxval;
531     }
532     else {
533         return x;
534     }
535 }
536
537
538 #endif
539
540

```

E.36 vendor/etc/hls_select_bitselect_apfixed.h (79 lines, 2,142 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_SELECT_BITSELECT_APFIXED_H_
7 #define _X_HLS_SELECT_BITSELECT_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10 // #include <stdio.h>
11
12 /*
13 //select ap_fixed
14 template <int W_, int I_>
15 ap_fixed<W_,I_> generic_select(ap_fixed<W_,I_>
x,ap_fixed<W_,I_> y, ap_fixed<W_,I_> z)
16 {
17     #pragma HLS pipeline
18     ap_fixed<W_,I_> result;
19     //for (int i=0; i<J;i++)
20     result=(z & y)|(~z & x); //meaning(z?y:x) or
(x^(z&(y^x)))
21     return result;
22 }
23
24 //select ap_ufixed
25 template <int W_, int I_>
26 ap_ufixed<W_,I_> generic_select(ap_ufixed<W_,I_>
x,ap_ufixed<W_,I_> y, ap_ufixed<W_,I_> z)
27 {
28     #pragma HLS pipeline
29     ap_ufixed<W_,I_> result;
30     //for (int i=0; i<J;i++)
31     result=(z & y)|(~z & x); //meaning(z?y:x) or
(x^(z&(y^x)))
32     return result;
33 }
34 */
35 //bitselect ap_fixed
36 template <int W_, int I_>
37 ap_fixed<W_,I_> generic_bitselect(ap_fixed<W_,I_>
x,ap_fixed<W_,I_> y, ap_fixed<W_,I_> z)
38 {
39     #pragma HLS pipeline
40     return (z & y)|(~z & x); //meaning(z?y:x) or (x^(z&(y^x)))
41 }
42
43 //bitselect ap_ufixed
44 template <int W_, int I_>
45 ap_ufixed<W_,I_> generic_bitselect(ap_ufixed<W_,I_>

```

```

x, ap_ufixed<W_, I_> y, ap_ufixed<W_, I_> z)
46 {
47 #pragma HLS pipeline
48     return (z & y) | (~z & x); //meaning(z?y:x) or (x^(z&(y^x)))
49 }
50
51 //bitselect ap_int
52 template <int I_>
53 ap_int<I_> generic_bitselect(ap_int<I_> x, ap_int<I_> y,
ap_int<I_> z)
54 {
55 #pragma HLS pipeline
56     return (z & y) | (~z & x); //meaning(z?y:x) or (x^(z&(y^x)))
57 }
58 //bitselect ap_uint
59 template <int I_>
60 ap_uint<I_> generic_bitselect(ap_uint<I_> x, ap_uint<I_> y,
ap_uint<I_> z)
61 {
62 #pragma HLS pipeline
63     return (z & y) | (~z & x); //meaning(z?y:x) or (x^(z&(y^x)))
64 }
65
66 /*
67 ///select ap_ufixed
68 template <int W_, int I_, int J_>
69 void hls_select(ap_ufixed<W_, I_> x[J_], ap_ufixed<W_, I_>
y[J_], ap_ufixed<W_, I_> z[J_], ap_ufixed<W_, I_> result[J_])
70 {
71     //ap_ufixed<W_, I_> result[J_];
72     for (int i=0; i<J_; i++)
73         result[i] = (z[i] & y[i]) | (~z[i] & x[i]); //meaning(z?
y:x) or (x^(z&(y^x)))
74     return;
75 }
76 */
77 #endif
78
79

```

E.37 vendor/etc/hls_sqrt_apfixed.h (440 lines, 20,674 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_SQRT_APFIXED_H_
7 #define _X_HLS_SQRT_APFIXED_H_
8 #include "ap_int.h"
9 #include "ap_fixed.h"
10
11 #ifndef __SYNTHESIS__
12 #include "assert.h"
13 #endif
14
15 #pragma clang diagnostic push
16 #pragma clang diagnostic ignored "-Wmisleading-indentation"
17
18 template <int W_, int I_>
19 ap_fixed<W_, I_> sqrt_fixed(ap_fixed<W_, I_> x)
20 {
21 #pragma HLS pipeline
22
23     // bitwidth that currently not supported
24     const int F_ = W_ - I_;
25     if (I_ > 34) return 0;
26     else if (F_ > 32) return 0;
27
28     // return ap_fixed value
29     ap_ufixed<F_+(I_+1)/2, (I_+1)/2> r;
30     ap_ufixed<W_-1, I_-1> x_s = x;
31
32     if ((F_==0)&&(I_==2)) {
33         r = x_s;
34     } else if ((F_==0)&&(I_<=13)) {
35         ap_ufixed<W_, I_> x_s_l = x_s + 1;
36         ap_ufixed<W_-1, I_-1> x_s_1;
37         x_s_1(W_-2, 0) = x_s_l(W_-1, 1);
38         if (I_<=8) {
39             if (x_s_1==ap_ufixed<W_-1, I_-1>("0x0p0")) r =
0;
40             else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x1p0")) r =
1;
41             else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x3p0")) r =
2;
42             else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x6p0")) r =
3;
43             else if (x_s_1==ap_ufixed<W_-1, I_-1>("0xap0")) r =
4;
44             else if (x_s_1==ap_ufixed<W_-1, I_-1>("0xfp0")) r =
5;

```

```

45         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x15p0")) r =
6;
46         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x1cp0")) r =
7;
47         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x24p0")) r =
8;
48         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x2dp0")) r =
9;
49         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x37p0")) r =
10;
50         else
51             r = 11;
52     } else if (I_<=9) {
53         if (x_s_1==ap_ufixed<W_-1, I_-1>("0x0p0")) r =
0;
54         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x1p0")) r =
1;
55         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x3p0")) r =
2;
56         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x6p0")) r =
3;
57         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0xap0")) r =
4;
58         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0xfp0")) r =
5;
59         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x15p0")) r =
6;
60         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x1cp0")) r =
7;
61         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x24p0")) r =
8;
62         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x2dp0")) r =
9;
63         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x37p0")) r =
10;
64         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x42p0")) r =
11;
65         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x4ep0")) r =
12;
66         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x5bp0")) r =
13;
67         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x69p0")) r =
14;
68         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x78p0")) r =
15;
69         else
70             r = 16;
71     } else {
72         if (x_s_1==ap_ufixed<W_-1, I_-1>("0x0p0"))
73             r = 0;
74         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x1p0"))
75             r = 1;
76         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x3p0"))
77             r = 2;
78         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x6p0"))
79             r = 3;
80         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0xap0"))
81             r = 4;
82         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0xfp0"))
83             r = 5;
84         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x15p0"))
85             r = 6;
86         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x1cp0"))
87             r = 7;
88         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x24p0"))
89             r = 8;
90         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x2dp0"))
91             r = 9;
92         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x37p0"))
93             r = 10;
94         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x42p0"))
95             r = 11;
96         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x4ep0"))
97             r = 12;
98         else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x5bp0"))
99             r = 13;
100        else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x69p0"))
101            r = 14;
102        else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x78p0"))
103            r = 15;
104        else if (x_s_1==ap_ufixed<W_-1, I_-1>("0x88p0"))
105            r = 16;

```

```

r = 16;
87
r = 17;
88
r = 18;
89
r = 19;
90
r = 20;
91
r = 21;
92
r = 22;
93
r = 23;
94
r = 24;
95
r = 25;
96
r = 26;
97
r = 27;
98
r = 28;
99
r = 29;
100
r = 30;
101
r = 31;
102
r = 32;
103
r = 33;
104
r = 34;
105
r = 35;
106
r = 36;
107
r = 37;
108
r = 38;
109
r = 39;
110
r = 40;
111
r = 41;
112
r = 42;
113
r = 43;
114
r = 44;
115
r = 45;
116
r = 46;
117
r = 47;
118
r = 48;
119
r = 49;
120
r = 50;
121
r = 51;
122
r = 52;
123
r = 53;
124
r = 54;
125
r = 55;
126
r = 56;
127
r = 57;
128
r = 58;
129
r = 59;
130
r = 60;
131
r = 61;
132
r = 62;
133
r = 63;
134
135
136
137
r = "0x0.0p0";

else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x99p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0xabp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0xbep0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0xd2p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0xe7p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0xfdp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x114p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x12cp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x145p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x15fp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x17ap0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x196p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x1b3p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x1d1p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x1fp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x210p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x231p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x253p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x276p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x29ap0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x2bfp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x2e5p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x30cp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x334p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x35dp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x387p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x3b2p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x3dep0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x40bp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x439p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x468p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x498p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x4c9p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x4fbp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x52ep0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x562p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x597p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x5cdp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x604p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x63cp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x675p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x6afp0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x6eap0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x726p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x763p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x7a1p0"))
else if (x_s_1<=ap_ufixed<W-1,I_-1>("0x7e0p0"))
else
r = 64;

} else if (F_<=4 && I_<=5) {
if (x_s==ap_ufixed<W-1,I_-1>("0x0.0p0"))

```

```

138
r = "0x0.4p0";
139
r = "0x0.6p0";
140
r = "0x0.7p0";
141
r = "0x0.8p0";
142
r = "0x0.9p0";
143
r = "0x0.ap0";
144
r = "0x0.bp0";
145
r = "0x0.cp0";
146
r = "0x0.dp0";
147
r = "0x0.ep0";
148
r = "0x0.fp0";
149
r = "0x1.0p0";
150
r = "0x1.1p0";
151
r = "0x1.2p0";
152
r = "0x1.3p0";
153
r = "0x1.4p0";
154
r = "0x1.5p0";
155
r = "0x1.6p0";
156
r = "0x1.7p0";
157
r = "0x1.8p0";
158
r = "0x1.9p0";
159
r = "0x1.ap0";
160
r = "0x1.bp0";
161
r = "0x1.cp0";
162
r = "0x1.dp0";
163
r = "0x1.ep0";
164
r = "0x1.fp0";
165
r = "0x2.0p0";
166
r = "0x2.1p0";
167
r = "0x2.2p0";
168
r = "0x2.3p0";
169
r = "0x2.4p0";
170
r = "0x2.5p0";
171
r = "0x2.6p0";
172
r = "0x2.7p0";
173
r = "0x2.8p0";
174
r = "0x2.9p0";
175
r = "0x2.ap0";
176
r = "0x2.bp0";
177
r = "0x2.cp0";
178
r = "0x2.dp0";
179
r = "0x2.ep0";
180
r = "0x2.fp0";
181
r = "0x3.0p0";
182
r = "0x3.1p0";
183
r = "0x3.2p0";
184
r = "0x3.3p0";
185
r = "0x3.4p0";
186
r = "0x3.5p0";
187
r = "0x3.6p0";

else if (x_s==ap_ufixed<W-1,I_-1>("0x0.1p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.2p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.3p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.4p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.5p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.6p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.8p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.9p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.bp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.dp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.ep0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x0.fp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.1p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.3p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.5p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.7p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.ap0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.cp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x1.fp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x2.2p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x2.5p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x2.8p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x2.bp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x2.fp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x3.2p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x3.ap0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x3.ep0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x4.2p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x4.6p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x4.ap0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x4.ep0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x5.3p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x5.7p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x5.cp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x6.1p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x6.6p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x6.bp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x7.0p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x7.6p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x7.bp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x8.1p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x8.7p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x8.dp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x9.3p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x9.9p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0x9.fp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0xa.5p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0xa.cp0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0xb.2p0"))
else if (x_s==ap_ufixed<W-1,I_-1>("0xb.9p0"))

```

```

188         else if (x_s<=ap_ufixed<W-1,I-1>("0xc.0p0"))
r = "0x3.7p0";
189         else if (x_s<=ap_ufixed<W-1,I-1>("0xc.7p0"))
r = "0x3.8p0";
190         else if (x_s<=ap_ufixed<W-1,I-1>("0xc.ep0"))
r = "0x3.9p0";
191         else if (x_s<=ap_ufixed<W-1,I-1>("0xd.5p0"))
r = "0x3.ap0";
192         else if (x_s<=ap_ufixed<W-1,I-1>("0xd.dp0"))
r = "0x3.bp0";
193         else if (x_s<=ap_ufixed<W-1,I-1>("0xe.4p0"))
r = "0x3.cp0";
194         else if (x_s<=ap_ufixed<W-1,I-1>("0xe.cp0"))
r = "0x3.dp0";
195         else if (x_s<=ap_ufixed<W-1,I-1>("0xf.4p0"))
r = "0x3.ep0";
196         else if (x_s<=ap_ufixed<W-1,I-1>("0xf.cp0"))
r = "0x3.fp0";
197         else
r = "0x4.0p0";
198     } else {
199         if (x[W-1]) return 0;
200         if (I_<=0) {
201
202             const static int lsbx = (-I+2<=W) ? (W+I-2) :
0;
203             if ( x(W-1,lsbx) != 0 ) return 0;
204         }
205
206         const static int prcs = (F+1)*2; // max precision
207
208         const static int msbr = (I>0) ? (I+1)/2 : 1; // msb
of result
209         const static int msbx = (I>0) ? I+3 : 4;
210         const static int msbm = (I>0) ? I+1 : 2;
211
212         // ap_ufixed<I+3 + prcs , I+3> x_l = x;
213         ap_ufixed<msbx , msbx> x_l_I = x;
214         ap_ufixed< prcs/2, 0> x_l_FH = x;
215         ap_ufixed< prcs/2, -prcs/2> x_l_FL = 0;
216         ap_ufixed<msbr + prcs , msbr> res = 0; // res-
ult to return
217         ap_ufixed<msbr , msbr> res_I = 0;
218         ap_ufixed< prcs/2, 0> res_FH = 0;
219         //ap_ufixed< prcs/2, -prcs/2> res_FL = 0;
220
221         // pragma HLS unroll
222         // each iteration calculate 1 bit of result, start-
ing from MSB
223         // bit is used to store the current bit under cal-
culation
224         // if accepted bit will be added to result
225         for ( int pos = msbr-1; pos >= -F-1; pos-- ) {
226             ap_ufixed<I+1 + prcs, I+1> mul = 0; // 2 *
res * bit + bit^2
227             assert ( res(pos+prcs,0)==0 );
228             mul ( msbr+pos + prcs , pos*2+1 + prcs ) = res
( msbr-1 + prcs , pos + prcs );
229             assert ( !mul[I-1+prcs] );
230             mul [ pos*2 + prcs ] = 1;
231
232             assert ( x_l(I+2+prcs,msbr+pos+2+prcs)==0 );
233             if ( x_l ( msbr+pos + prcs , pos*2 + prcs ) >=
mul ( msbr+pos + prcs , pos*2 + prcs ) ) {
234                 assert ( x_l >= mul );
235                 ap_ufixed<I_ + prcs, I_> x_l_ = x_l; //
just for assert
236                 x_l ( msbr+pos + prcs , pos*2 + prcs ) =
x_l ( msbr+pos + prcs , pos*2 + prcs ) - mul ( msbr+pos + prcs ,
pos*2 + prcs );
237                 assert ( x_l == x_l_ - mul );
238
239                 res [ pos + prcs ] = 1;
240             }
241
242             if (I_>0)
243                 for ( int pos = msbr-1; pos >= 0; pos-- ) {
244                     #pragma HLS unroll
245                     ap_ufixed<msbm , msbm> mul_I = 0;
246                     #ifndef __SYNTHESIS__
247                         assert ( res_I(pos,0)==0 );
248                     #endif
249                     mul_I ( msbr+pos , pos*2+1 ) = res_I ( msbr-1 ,
pos );
250                     #ifndef __SYNTHESIS__
251                         assert ( !mul_I[I-1] );
252                     #endif
253                     mul_I [ pos*2 ] = 1;
254
255                     #ifndef __SYNTHESIS__
256                         assert ( x_l_I(I+2,msbr+pos+2)==0 );
257                     #endif
258                     if ( x_l_I ( msbr+pos+1 , pos*2 ) >= mul_I
( msbr+pos , pos*2 ) ) {
259                         #ifndef __SYNTHESIS__
260                             assert ( x_l_I >= mul_I );
261                         #endif
262                         ap_ufixed<msbx,msbx> x_l_I_ = x_l_I;
263                         x_l_I ( msbr+pos+1 , pos*2 ) = x_l_I_

```

```

( msbr+pos+1 , pos*2 ) - mul_I ( msbr+pos , pos*2 );
264 #ifndef __SYNTHESIS__
265     assert ( x_l_I == x_l_I_ - mul_I );
266 #endif
267
268     res_I [ pos ] = 1;
269 }
270
271 for ( int pos = -1; pos >= -F-1; pos-- ) {
272     #pragma HLS unroll
273     ap_ufixed<msbm + prcs , msbm> mul = 0;
274     #ifndef __SYNTHESIS__
275         assert ( res_FH(pos+prcs/2,0)==0 );
276     #endif
277     //assert ( res_FL ==0 );
278
279     mul ( msbr+pos + prcs , pos +1 + prcs ) = res_I
( msbr-1 , 0 );
280
281     mul ( pos + prcs , pos*2+1 + prcs ) = res_FH
( -1+prcs/2 , pos+prcs/2 );
282     #ifndef __SYNTHESIS__
283         assert ( !mul[I-1+prcs] );
284     #endif
285     mul [ pos*2 + prcs ] = 1;
286     ap_ufixed<msbm , msbm> mul_I = mul;
287     ap_ufixed< prcs/2, 0> mul_FH = mul;
288     ap_ufixed< prcs/2, -prcs/2> mul_FL = mul;
289
290     ap_ufixed<msbx + prcs , msbx> x_l;
291     x_l ( -1+msbx + prcs , prcs ) = x_l_I
( -1+msbx , 0 );
292     x_l ( -1 + prcs , prcs/2 ) = x_l_FH
( -1+prcs/2 , 0 );
293     x_l ( -1 + prcs/2 , 0 ) = x_l_FL
( -1+prcs/2 , 0 );
294     #ifndef __SYNTHESIS__
295         assert ( x_l(I+2+prcs,msbr+pos+2+prcs)==0 );
296     #endif
297     if ( (x_l_I>mul_I) ||
((x_l_I==mul_I)&&(x_l_FH>mul_FH)) ||
((x_l_I==mul_I)&&(x_l_FH==mul_FH)&&(x_l_FL>mul_FL)) ) {
298         #ifndef __SYNTHESIS__
299             assert ( x_l >= mul );
300         #endif
301         ap_ufixed< prcs/2+1 , -prcs/2+1 >
x_l_FL_l = x_l_FL;
302         if ( x_l_FL < mul_FL )
x_l_FL_l[prcs/2] = 1;
303         x_l_FL_l -= mul_FL;
304         #ifndef __SYNTHESIS__
305             assert ( !x_l_FL_l[prcs/2] );
306         #endif
307         ap_ufixed< prcs/2+1 , 1 >
x_l_FH_l = x_l_FH;
308         if ( (x_l_FH<mul_FH) ||
((x_l_FH==mul_FH)&&(x_l_FL<mul_FL)) ) x_l_FH_l[prcs/2] = 1;
309         ap_ufixed< 1 , -prcs/2+1 >
delta = 0;
310         if ( x_l_FL <
mul_FL ) delta[0] =
1;
311         x_l_FH_l -= delta;
312         x_l_FH_l -= mul_FH;
313         #ifndef __SYNTHESIS__
314             assert ( !x_l_FH_l[prcs/2] );
315         #endif
316         if ( (x_l_FH<mul_FH) ||
((x_l_FH==mul_FH)&&(x_l_FL<mul_FL)) ) x_l_I -=;
317         x_l_I -= mul_I;
318
319         x_l_FH = x_l_FH_l;
320         x_l_FL = x_l_FL_l;
321
322         ap_ufixed<msbx + prcs , msbx> x_l_ = x_l;
323         x_l ( -1+msbx + prcs , prcs ) = x_l_I
( -1+msbx , 0 );
324         x_l ( -1 + prcs , prcs/2 ) = x_l_FH
( -1+prcs/2 , 0 );
325         x_l ( -1 + prcs/2 , 0 ) = x_l_FL
( -1+prcs/2 , 0 );
326         #ifndef __SYNTHESIS__
327             assert ( x_l == x_l_ - mul );
328         #endif
329         res_FH [ pos+prcs/2 ] = 1;
330     }
331 }

```

```

332 //      r = res + ( delta >> F_ );
333 //
334 //      ap_ufixed< prcs/2+1 , 1 > res_FH_l      =
res_FH;
335 //      ap_ufixed< prcs/2 , 0 > delta;
336 //      delta[delta.wl()-1] = 1;
337 //      res_FH_l      +=
( delta >> F_ );
338 //      res_FH      =
res_FH_l;
339 //      if ( res_FH_l[prcs/2] )      res_I      ++;
340 //
341 //
342 //      res ( msbr-1 + prcs , prcs ) = res_I ( msbr-1 ,
0 );
343 //      res ( -1 + prcs , prcs/2 ) = res_FH ( -1+prcs/2 ,
0 );
344 //
345 //      r = res;
346 //
347 //
348 //      return r;
349 //
350 template<int W_, int I_>
351 ap_ufixed<W_, I_> sqrt_fixed(ap_ufixed<W_, I_> x) {
352     ap_fixed<W+1, I+1> xf = x;
353     return sqrt_fixed(xf);
354 }
355 template<int I_>
356 ap_int<I_> sqrt_fixed(ap_int<I_> x) {
357     ap_fixed<I_, I_> xf = x;
358     return sqrt_fixed(xf);
359 }
360 template<int I_>
361 ap_uint<I_> sqrt_fixed(ap_uint<I_> x) {
362     ap_fixed<I+1, I+1> xf = x;
363     return sqrt_fixed(xf);
364 }
365 //
366 //
367 template <int W_, int I_>
368 ap_fixed<W_, I_> rsqrt_fixed(ap_fixed<W_, I_> x)
369 {
370     #pragma HLS pipeline
371     ap_ufixed<W-1, I-1> xs = x;
372     const int Ix = ( I_ > 1 )? I_-1 : 1;
373     ap_ufixed<Ix, Ix> xs_I = xs;
374     ap_ufixed<W-1, I-1> r;
375     const int I = ( I_-1 > W_-I+1 ) ? I_-1 : W_-I+1;
376     ap_ufixed<I+W_-I, I> y1;
377     ap_ufixed<I+W_-I, I> y2;
378     if ( xs == 0 ) {
379         return 0;
380     } else if ( xs_I != 0 ) {
381         y1 = xs;
382     } else {
383         y1 = 1;
384         y1 = y1/xs;
385     }
386     y2 = sqrt_fixed(y1);
387     if ( y2 == 0 ) {
388         return 0;
389     } else if ( xs_I != 0 ) {
390         r = 1;
391         r = r/y2;
392     } else {
393         r = y2;
394     }
395     return r;
396 }
397 template<int W_, int I_>
398 ap_ufixed<W_, I_> rsqrt_fixed(ap_ufixed<W_, I_> x) {
399     ap_fixed<W+1, I+1> xf = x;
400     return rsqrt_fixed(xf);
401 }
402 template<int I_>
403 ap_int<I_> rsqrt_fixed(ap_int<I_> x) {
404     ap_fixed<I_, I_> xf = x;
405     return rsqrt_fixed(xf);
406 }
407 template<int I_>
408 ap_uint<I_> rsqrt_fixed(ap_uint<I_> x) {
409     ap_fixed<I+1, I+1> xf = x;
410     return rsqrt_fixed(xf);
411 }
412 //
413 //
414 template<int W_, int I_>
415 ap_fixed<W_, I_> recip_fixed(ap_fixed<W_, I_> x)
416 {
417     #pragma HLS pipeline
418     if ( x == 0 ) return 0;
419     ap_fixed<W_, I_> r = 1;
420     return r/x;
421 }
422 template<int W_, int I_>
423 ap_ufixed<W_, I_> recip_fixed(ap_ufixed<W_, I_> x) {
424     ap_fixed<W+1, I+1> xf = x;

```

```

425     return recip_fixed(xf);
426 }
427 template<int I_>
428 ap_int<I_> recip_fixed(ap_int<I_> x) {
429     ap_fixed<I_, I_> xf = x;
430     return recip_fixed(xf);
431 }
432 template<int I_>
433 ap_uint<I_> recip_fixed(ap_uint<I_> x) {
434     ap_fixed<I+1, I+1> xf = x;
435     return recip_fixed(xf);
436 }
437 //
438 #pragma clang diagnostic pop
439 //
440 #endif

```

E.38 vendor/etc/hls_tgamma_apfixed.h (46 lines, 1,286 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef _X_HLS_TGAMMA_APFIXED_H_
7 #define _X_HLS_TGAMMA_APFIXED_H_
8 #include "ap_fixed.h"
9 #include "ap_int.h"
10 // #include "hls_tgamma_generic.h"
11 #include "utils/x_hls_defines.h"
12 #include "utils/x_hls_traits.h"
13 #include "utils/x_hls_utils.h"
14
15 namespace tgamma_fixed {
16
17 template <int W, int I> ap_fixed<W, I>
tgamma_generic(ap_fixed<W, I> x_fixed) {
18     #pragma HLS pipeline
19
20     double x = x_fixed.to_double();
21     double r = hls::tgamma(x);
22     fp_struct<double> out(r);
23     ap_ufixed<W+1, I+1> ret = 1;
24     int F = W - I;
25     if (F > 52) F = 52;
26     if (W > I) ret(F - 1, 0) = out.sig(52 - 1, 52 - F);
27     ret = (ret << (out.exp - fp_struct<double>::EXP_BIAS));
28     return ap_fixed<W, I>(ret);
29 }
30
31 template <int W, int I> ap_ufixed<W, I>
tgamma_generic(ap_ufixed<W, I> x) {
32     ap_fixed<W + 1, I + 1> xf = x;
33     return tgamma_generic(xf);
34 }
35 template <int I_> ap_int<I_> tgamma_generic(ap_int<I_> x) {
36     ap_fixed<I_, I_> xf = x;
37     return tgamma_generic(xf);
38 }
39 template <int I_> ap_uint<I_> tgamma_generic(ap_uint<I_> x) {
40     ap_fixed<I_ + 1, I_ + 1> xf = x;
41     return tgamma_generic(xf);
42 }
43 //
44 // tgamma_fixed
45 //
46 #endif

```

E.39 vendor/floating_point_v7_1_bitacc_cmodel.h (403 lines, 23,001 B)

```

1 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
2 //-----
3 //
4 // / \ / \ / \
5 // / \ / \ / \      Vendor: Xilinx
6 // \ / \ / \ / \      Version: 6.0
7 // \ / \ / \ / \      Filename: $RCSfile: float-
ing_point_v7_1_bitacc_cmodel.h,v $
8 // / \ / \      Date Last Modified: $Date: 2011/06/15
13:06:43 $
9 // / \ / \      Date Created: 2011
10 // \ / \ / \
11 // \ / \ / \
12 //
13 // Device : All
14 // Library : floating_point_v7_1
15 // Purpose : Header file for bit accurate model of Floating
Point Operator
16 // Revision: $Revision: 1.6.6.2 $
17 //
18 //-----
19 // (c) Copyright 2011-2012 Xilinx, Inc. All rights reserved.
20 // Copyright 2022-2023 Advanced Micro Devices, Inc. All

```



```

Rights Reserved.
21 //
22 // This file contains confidential and proprietary informa-
tion
23 // of Xilinx, Inc. and is protected under U.S. and
24 // international copyright and other intellectual property
25 // laws.
26 //
27 // DISCLAIMER
28 // This disclaimer is not a license and does not grant any
29 // rights to the materials distributed herewith. Except as
30 // otherwise provided in a valid license issued to you by
31 // Xilinx, and to the maximum extent permitted by applicable
32 // law: (1) THESE MATERIALS ARE MADE AVAILABLE "AS IS" AND
33 // WITH ALL FAULTS, AND XILINX HEREBY DISCLAIMS ALL WAR-
RANTIES
34 // AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING
35 // BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-
36 // INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and
37 // (2) Xilinx shall not be liable (whether in contract or
tort,
38 // including negligence, or under any other theory of
39 // liability) for any loss or damage of any kind or nature
40 // related to, arising under or in connection with these
41 // materials, including for any direct, or any indirect,
42 // special, incidental, or consequential loss or damage
43 // (including loss of data, profits, goodwill, or any type
of
44 // loss or damage suffered as a result of any action brought
45 // by a third party) even if such damage or loss was
46 // reasonably foreseeable or Xilinx had been advised of the
47 // possibility of the same.
48 //
49 // CRITICAL APPLICATIONS
50 // Xilinx products are not designed or intended to be fail-
51 // safe, or for use in any application requiring fail-safe
52 // performance, such as life-support or safety devices or
53 // systems, Class III medical devices, nuclear facilities,
54 // applications related to the deployment of airbags, or any
55 // other applications that could lead to death, personal
56 // injury, or severe property or environmental damage
57 // (individually and collectively, "Critical
58 // Applications"). Customer assumes the sole risk and
59 // liability of any use of Xilinx products in Critical
60 // Applications, subject only to applicable laws and
61 // regulations governing limitations on product liability.
62 //
63 // THIS COPYRIGHT NOTICE AND DISCLAIMER MUST BE RETAINED AS
64 // PART OF THIS FILE AT ALL TIMES.
65 //-----
66
67 #ifndef __xip_fpo_bitacc_cmodel_h
68 #define __xip_fpo_bitacc_cmodel_h
69
70 #ifndef NT
71 #define __XIP_FPO_DLLIMPORT __declspec(dllimport)
72 #define __XIP_FPO_DLLEXPORT __declspec(dllexport)
73 #else
74 #define __XIP_FPO_DLLIMPORT
75 #define __XIP_FPO_DLLEXPORT
76 #endif
77
78 // Only define __XIP_FPO_BUILD_DLL when building the C model
79 // DLL; do not define it when using the C model
80 #ifndef __XIP_FPO_BUILD_DLL
81 #define __XIP_FPO_DLL __XIP_FPO_DLLEXPORT
82 #else
83 #define __XIP_FPO_DLL __XIP_FPO_DLLIMPORT
84 #endif
85
86 /* Extra define for functions with variable numbers of argu-
ments */
87 #define __XIP_FPO_SENTINEL_ATTR
88 #if defined (__GNUC__)
89 # if __GNUC__ >= 4
90 #  undef __XIP_FPO_SENTINEL_ATTR
91 #  define __XIP_FPO_SENTINEL_ATTR __attribute__((sentinel))
92 # endif
93 #endif
94
95 /* Define Floating Point Operator core version number */
96 #define XIP_FPO_VERSION_MAJOR 7
97 #define XIP_FPO_VERSION_MINOR 1
98 #define XIP_FPO_REVISION 0
99 /* Version string does not include revision if revision is 0
(revision >0 reserved for future use) */
100 #define XIP_FPO_VERSION_STRING "7.1"
101
102 /* Use C99 exact width integer types for 64-bit integers and
*_uj and *_sj functions */
103 // For Windows platforms, stdint.h and inttypes.h are not
present in Visual Studio 2005/2008
104 // Therefore we define the required types ourselves
105 // For Linux platforms, we need to continue using stdint.h
(and not re-define intmax_t, uintmax_t)
106 // because SysGen already uses this header.
107 #ifndef NT
108 typedef signed char xint8;
109 typedef signed short xint16;
110 typedef signed int xint32;
111 typedef signed long long xint64;
112 typedef unsigned char xuint8;
113 typedef unsigned short xuint16;
114 typedef unsigned int xuint32;
115 typedef unsigned long long xuint64;
116 typedef xint64 intmax_t;
117 typedef xuint64 uintmax_t;
118 #else
119 #include <stdint.h>
120 typedef int8_t xint8;
121 typedef int16_t xint16;
122 typedef int32_t xint32;
123 typedef int64_t xint64;
124 typedef uint8_t xuint8;
125 typedef uint16_t xuint16;
126 typedef uint32_t xuint32;
127 typedef uint64_t xuint64;
128 #endif
129
130 #ifndef NT
131 // Allow C99 compilation using GCC
132 #include <stdbool.h>
133 #endif
134
135 // REVISIT: included before mpfr.h to permit definition of
prototypes for mpfr_printf
136 #include <stdio.h>
137
138 // Force MPFR to use intmax_t and uintmax_t types (the com-
piled libraries have the functions that use these)
139 #define MPFR_USE_INTMAX_T
140
141 // Tell MPIR on Windows platforms that it is compiled into a
DLL
142 #ifndef NT
143 #define __GMP_LIBGMP_DLL 1
144 #endif
145
146 /* Check if GMP is included, and try to include it (Works
with local GMP)
147 Note that where MPIR is provided as a compatible alternat-
ive to GMP,
148 it also provides a gmp.h header file to allow MPIR and GMP
to be easily interchanged.
149 Select the correct gmp.h by setting the -I option (in-
cludes path) for your compiler. */
150 #ifndef __GMP_H__
151 #include <gmp.h>
152 #endif
153
154 /* Check if MPFR is included, and try to include it (Works
with local MPFR) */
155 #ifndef __MPFR_H__
156 #include <mpfr.h>
157 #endif
158
159 /* Precision of mantissa or exponent (bits) */
160 typedef long xip_fpo_prec_t;
161
162 /* Definition of sign */
163 typedef int xip_fpo_sign_t;
164
165 /* Definition of exponent */
166 typedef long xip_fpo_exp_t;
167
168 /* The main floating point number structure */
169 typedef struct {
170 xip_fpo_prec_t xip_fpo_exp_prec;
171 xip_fpo_prec_t xip_fpo_mant_prec;
172 xip_fpo_sign_t xip_fpo_sign;
173 xip_fpo_exp_t xip_fpo_exp;
174 mp_limb_t xip_fpo_d;
175 } __xip_fpo_struct;
176
177 /* The main fixed point number structure */
178 typedef struct {
179 xip_fpo_prec_t xip_fpo_i_prec;
180 xip_fpo_prec_t xip_fpo_frac_prec;
181 xint64 xip_fpo_i;
182 xint64 xip_fpo_frac;
183 } __xip_fpo_fix_struct;
184
185 /* User-visible types for floating point and fixed point num-
bers */
186 typedef __xip_fpo_struct xip_fpo_t[1];
187 typedef __xip_fpo_fix_struct xip_fpo_fix_t[1];
188
189 /* Pointers to floating point and fixed point numbers, for
function prototypes */
190 typedef __xip_fpo_struct *xip_fpo_ptr;
191 typedef const __xip_fpo_struct *xip_fpo_srcptr;
192 typedef __xip_fpo_fix_struct *xip_fpo_fix_ptr;
193 typedef const __xip_fpo_fix_struct *xip_fpo_fix_srcptr;
194
195 /* Definition of exception flags - return type of most func-
tions
196 Flags are as follows:
197 bit 0 : underflow

```

```

197     bit 1 : overflow
198     bit 2 : invalid operation
199     bit 3 : divide by zero
200     bit 4 : operation not supported
201     bit 5 : Accumulator Input Overflow
202     bit 6 : Accumulator Overflow
203     */
204     typedef int xip_fpo_exc_t;
205
206     #ifdef __cplusplus
207     extern "C" {
208     #endif
209
210     typedef struct xil_fpo_accum_state xil_fpo_accum_state;
211
212     /* Information functions */
213     __XIP_FPO_DLL const char * xip_fpo_get_version (void);
214
215     /* Initialization functions */
216     __XIP_FPO_DLL void xip_fpo_init2 (xip_fpo_ptr,
xip_fpo_prec_t, xip_fpo_prec_t);
217     __XIP_FPO_DLL void xip_fpo_fix_init2 (xip_fpo_fix_ptr,
xip_fpo_prec_t, xip_fpo_prec_t);
218     __XIP_FPO_DLL void xip_fpo_inits2 (xip_fpo_prec_t,
xip_fpo_prec_t, xip_fpo_ptr, ...) __XIP_FPO_SENTINEL_ATTR;
219     __XIP_FPO_DLL void xip_fpo_fix_inits2 (xip_fpo_prec_t,
xip_fpo_prec_t, xip_fpo_fix_ptr, ...) __XIP_FPO_SENTINEL_ATTR;
220     __XIP_FPO_DLL void xip_fpo_clear (xip_fpo_ptr);
221     __XIP_FPO_DLL void xip_fpo_fix_clear (xip_fpo_fix_ptr);
222     __XIP_FPO_DLL void xip_fpo_clears (xip_fpo_ptr, ...)
__XIP_FPO_SENTINEL_ATTR;
223     __XIP_FPO_DLL void xip_fpo_fix_clears
(xip_fpo_fix_ptr, ...) __XIP_FPO_SENTINEL_ATTR;
224     __XIP_FPO_DLL void xip_fpo_set_prec (xip_fpo_ptr,
xip_fpo_prec_t, xip_fpo_prec_t);
225     __XIP_FPO_DLL void xip_fpo_fix_set_prec (xip_fpo_fix_ptr,
xip_fpo_prec_t, xip_fpo_prec_t);
226     __XIP_FPO_DLL xip_fpo_prec_t xip_fpo_get_prec_mant
(xip_fpo_ptr);
227     __XIP_FPO_DLL xip_fpo_prec_t xip_fpo_get_prec_exp
(xip_fpo_ptr);
228     __XIP_FPO_DLL xip_fpo_prec_t xip_fpo_fix_get_prec_frac
(xip_fpo_fix_ptr);
229     __XIP_FPO_DLL xip_fpo_prec_t xip_fpo_fix_get_prec_int
(xip_fpo_fix_ptr);
230
231     /* Assignment functions */
232     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set (xip_fpo_ptr,
xip_fpo_srcptr);
233     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set
(xip_fpo_fix_ptr, xip_fpo_fix_srcptr);
234     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_ui (xip_fpo_ptr,
unsigned long);
235     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_ui
(xip_fpo_fix_ptr, unsigned long);
236     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_si (xip_fpo_ptr,
long);
237     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_si
(xip_fpo_fix_ptr, long);
238     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_uj (xip_fpo_ptr,
uintmax_t);
239     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_uj
(xip_fpo_fix_ptr, uintmax_t);
240     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_sj (xip_fpo_ptr,
intmax_t);
241     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_sj
(xip_fpo_fix_ptr, intmax_t);
242     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_setflt (xip_fpo_ptr,
float);
243     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_setflt
(xip_fpo_fix_ptr, float);
244     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_d (xip_fpo_ptr,
double);
245     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_d
(xip_fpo_fix_ptr, double);
246     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_z (xip_fpo_ptr,
mpz_srcptr);
247     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_z
(xip_fpo_fix_ptr, mpz_srcptr);
248     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_q (xip_fpo_ptr,
mpq_srcptr);
249     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_q
(xip_fpo_fix_ptr, mpq_srcptr);
250     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_f (xip_fpo_ptr,
mpf_srcptr);
251     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_f
(xip_fpo_fix_ptr, mpf_srcptr);
252     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_fr (xip_fpo_ptr,
mpfr_srcptr);
253     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_fr
(xip_fpo_fix_ptr, mpfr_srcptr);
254     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_ui_2exp (xip_fpo_ptr,
unsigned long, xip_fpo_exp_t);
255     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_si_2exp (xip_fpo_ptr,
long, xip_fpo_exp_t);
256     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_uj_2exp (xip_fpo_ptr,
uintmax_t, intmax_t);
257     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_sj_2exp (xip_fpo_ptr,
intmax_t, intmax_t);
258     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_set_str (xip_fpo_ptr,
const char *, int);
259     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_set_str
(xip_fpo_fix_ptr, const char *, int);
260     __XIP_FPO_DLL void xip_fpo_set_nan
(xip_fpo_ptr);
261     __XIP_FPO_DLL void xip_fpo_set_inf (xip_fpo_ptr,
int);
262     __XIP_FPO_DLL void xip_fpo_set_zero (xip_fpo_ptr,
int);
263
264     /* Conversion functions */
265     __XIP_FPO_DLL unsigned long xip_fpo_get_ui
(xip_fpo_srcptr);
266     __XIP_FPO_DLL unsigned long xip_fpo_fix_get_ui
(xip_fpo_fix_srcptr);
267     __XIP_FPO_DLL long xip_fpo_get_si
(xip_fpo_srcptr);
268     __XIP_FPO_DLL long xip_fpo_fix_get_si
(xip_fpo_fix_srcptr);
269     __XIP_FPO_DLL uintmax_t xip_fpo_get_uj
(xip_fpo_srcptr);
270     __XIP_FPO_DLL uintmax_t xip_fpo_fix_get_uj
(xip_fpo_fix_srcptr);
271     __XIP_FPO_DLL intmax_t xip_fpo_get_sj
(xip_fpo_srcptr);
272     __XIP_FPO_DLL intmax_t xip_fpo_fix_get_sj
(xip_fpo_fix_srcptr);
273     __XIP_FPO_DLL float xip_fpo_getflt
(xip_fpo_srcptr);
274     __XIP_FPO_DLL float xip_fpo_fix_getflt
(xip_fpo_fix_srcptr);
275     __XIP_FPO_DLL double xip_fpo_get_d
(xip_fpo_srcptr);
276     __XIP_FPO_DLL double xip_fpo_fix_get_d
(xip_fpo_fix_srcptr);
277     __XIP_FPO_DLL double xip_fpo_get_d_2exp (long *,
xip_fpo_srcptr);
278     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_get_z (mpz_ptr,
xip_fpo_srcptr);
279     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_get_z (mpz_ptr,
xip_fpo_fix_srcptr);
280     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_get_f (mpf_ptr,
xip_fpo_srcptr);
281     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_get_f (mpf_ptr,
xip_fpo_fix_srcptr);
282     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_get_fr (mpfr_ptr,
xip_fpo_srcptr);
283     __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix_get_fr (mpfr_ptr,
xip_fpo_fix_srcptr);
284     __XIP_FPO_DLL char * xip_fpo_get_str (char *,
xip_fpo_exp_t *, int, int, xip_fpo_srcptr);
285     __XIP_FPO_DLL char * xip_fpo_fix_get_str (char *,
int, xip_fpo_fix_srcptr);
286     __XIP_FPO_DLL void xip_fpo_free_str (char *);
287     __XIP_FPO_DLL void xip_fpo_fix_free_str (char *);
288     __XIP_FPO_DLL int xip_fpo_sizeinbase
(xip_fpo_srcptr, int);
289     __XIP_FPO_DLL int xip_fpo_fix_sizeinbase
(xip_fpo_fix_srcptr, int);
290
291     /* Operation functions */
292     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_add (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_srcptr);
293     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_addflt (float *, float, float);
294     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_add_d (double *, double, double);
295     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_sub (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_srcptr);
296     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_subflt (float *, float, float);
297     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_sub_d (double *, double, double);
298     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_mul (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_srcptr);
299     __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_mulf (float *, float, float);

```



```

300 __XIP_FPO_DLL xip_fpo_exc_t (double *, double, double);
xip_fpo_mul_d
301 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_fma
xip_fpo_srcptr, xip_fpo_srcptr);
302 __XIP_FPO_DLL xip_fpo_exc_t (float *, float, float, float);
xip_fpo_fma_flt
303 __XIP_FPO_DLL xip_fpo_exc_t (double *, double, double,
xip_fpo_fma_d
double);
304 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_fms
xip_fpo_srcptr, xip_fpo_srcptr);
305 __XIP_FPO_DLL xip_fpo_exc_t (float *, float, float, float);
xip_fpo_fms_flt
306 __XIP_FPO_DLL xip_fpo_exc_t (double *, double, double,
xip_fpo_fms_d
double);
307 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_div
xip_fpo_srcptr);
308 __XIP_FPO_DLL xip_fpo_exc_t (float *, float, float);
xip_fpo_div_flt
309 __XIP_FPO_DLL xip_fpo_exc_t (double *, double, double);
xip_fpo_div_d
310 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr);
xip_fpo_rec
311 __XIP_FPO_DLL xip_fpo_exc_t (float *, float);
xip_fpo_rec_flt
312 __XIP_FPO_DLL xip_fpo_exc_t (double *, double);
xip_fpo_rec_d
313 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr);
xip_fpo_abs
314 __XIP_FPO_DLL xip_fpo_exc_t (float *, float);
xip_fpo_abs_flt
315 __XIP_FPO_DLL xip_fpo_exc_t (double *, double);
xip_fpo_abs_d
316 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr);
xip_fpo_log
317 __XIP_FPO_DLL xip_fpo_exc_t (float *, float);
xip_fpo_log_flt
318 __XIP_FPO_DLL xip_fpo_exc_t (double *, double);
xip_fpo_log_d
319 __XIP_FPO_DLL int xip_fpo_exp_ar-
ray (xip_fpo_t *, xip_fpo_t *, xip_fpo_exc_t *,
unsigned long long);
320 __XIP_FPO_DLL void xip_fpo_exp_flt_ar-
ray (float *, float *, xip_fpo_exc_t *, un-
signed long long);
321 __XIP_FPO_DLL void xip_fpo_exp_d_ar-
ray (double *, double *, xip_fpo_exc_t *,
unsigned long long);
322 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_sr-
cptr
);
323 __XIP_FPO_DLL xip_fpo_exc_t (float *,
xip_fpo_exp_flt
float
);
324 __XIP_FPO_DLL xip_fpo_exc_t (double *,
xip_fpo_exp_d
double
);
325 __XIP_FPO_DLL struct xil_fpo_accum_state * xip_fpo_ac-
cum_create_state(int, int, int, int, int);
326 __XIP_FPO_DLL void xip_fpo_ac-
cum_reset_state(struct xil_fpo_accum_state *);
327 __XIP_FPO_DLL void xip_fpo_ac-
cum_destroy_state(struct xil_fpo_accum_state *);
328 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ac-
cum_sample(xip_fpo_t, xip_fpo_t, bool, struct
xil_fpo_accum_state *);
329 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ac-
cum_sample_flt(float *, float, bool, struct
xil_fpo_accum_state *);
330 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ac-
cum_sample_d(double *, double, bool, struct
xil_fpo_accum_state *);
331 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr);
xip_fpo_sqrt
332 __XIP_FPO_DLL xip_fpo_exc_t (float *, float);
xip_fpo_sqrt_flt
333 __XIP_FPO_DLL xip_fpo_exc_t (double *, double);
xip_fpo_sqrt_d
334 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_rec-
sqrt(xip_fpo_ptr, xip_fpo_srcptr);
335 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_rec-
sqrt_flt(float *, float);
336 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_rec-
sqrt_d(double *, double);
337 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_un-
ordered(int *, xip_fpo_srcptr, xip_fpo_srcptr);
338 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_un-
ordered_flt(int *, float, float);
339 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_un-
ordered_d(int *, double, double);
340 __XIP_FPO_DLL xip_fpo_exc_t (int *, xip_fpo_srcptr,
xip_fpo_equal
xip_fpo_srcptr);
341 __XIP_FPO_DLL xip_fpo_exc_t (int *, float, float);
xip_fpo_equal_flt
342 __XIP_FPO_DLL xip_fpo_exc_t
xip_fpo_equal_d (int *, double, double);
343 __XIP_FPO_DLL xip_fpo_exc_t (int *, xip_fpo_srcptr,
xip_fpo_less
xip_fpo_srcptr);
344 __XIP_FPO_DLL xip_fpo_exc_t (int *, float, float);
xip_fpo_less_flt
345 __XIP_FPO_DLL xip_fpo_exc_t (int *, double, double);
xip_fpo_less_d
346 __XIP_FPO_DLL xip_fpo_exc_t (int *, xip_fpo_srcptr,
xip_fpo_lessequal
xip_fpo_srcptr);
347 __XIP_FPO_DLL xip_fpo_exc_t (int *, float, float);
xip_fpo_lessequal_flt
348 __XIP_FPO_DLL xip_fpo_exc_t (int *, double, double);
xip_fpo_lessequal_d
349 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_great-
er(int *, xip_fpo_srcptr, xip_fpo_srcptr);
350 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_great-
er_flt(int *, float, float);
351 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_great-
er_d(int *, double, double);
352 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_greate-
requal(int *, xip_fpo_srcptr, xip_fpo_srcptr);
353 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_greate-
requal_flt(int *, float, float);
354 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_greate-
requal_d(int *, double, double);
355 __XIP_FPO_DLL xip_fpo_exc_t (int *, xip_fpo_srcptr,
xip_fpo_notequal
xip_fpo_srcptr);
356 __XIP_FPO_DLL xip_fpo_exc_t (int *, float, float);
xip_fpo_notequal_flt
357 __XIP_FPO_DLL xip_fpo_exc_t (int *, double, double);
xip_fpo_notequal_d
358 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_cond-
code(int *, xip_fpo_srcptr, xip_fpo_srcptr);
359 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_cond-
code_flt(int *, float, float);
360 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_cond-
code_d(int *, double, double);
361 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
tofix(xip_fpo_fix_ptr, xip_fpo_srcptr);
362 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
tofix_int_flt(int *, float);
363 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
tofix_int_d(int *, double);
364 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt(xip_fpo_ptr, xip_fpo_fix_srcptr);
365 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_flt_int(float *, int);
366 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_d_int(double *, int);
367 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_flt_int64(float *, long long);
368 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_d_int64(double *, long long);
369 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_flt_uint(float *, unsigned int);
370 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_d_uint(double *, unsigned int);
371 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_flt_uint64(float *, unsigned long long);
372 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_d_uint64(double *, unsigned long long);
373 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt(xip_fpo_ptr, xip_fpo_srcptr);
374 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_flt_flt(float *, float);
375 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_flt_d(float *, double);
376 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_d_flt(double *, float);
377 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_fix-
toflt_d_d(double *, double);
378
379 /* versal additions */
380 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_uhma
xip_fpo_srcptr, xip_fpo_srcptr);
381 __XIP_FPO_DLL xip_fpo_exc_t (float *, float, float, float);
xip_fpo_uhma_flt
382 __XIP_FPO_DLL xip_fpo_exc_t (xip_fpo_ptr, xip_fpo_srcptr,
xip_fpo_uhms
xip_fpo_srcptr, xip_fpo_srcptr);
383 __XIP_FPO_DLL xip_fpo_exc_t (float *, float, float, float);
xip_fpo_uhms_flt
384
385 __XIP_FPO_DLL float * xip_fpo_ac-
cum_prim_create_state(int, int, int, int, int);
386 __XIP_FPO_DLL void xip_fpo_ac-
cum_prim_reset_state(float *);
387 __XIP_FPO_DLL void xip_fpo_ac-
cum_prim_destroy_state(float *);
388 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ac-
cum_prim_sample(xip_fpo_t, xip_fpo_t, bool, float *);
389 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ac-
cum_prim_sample_flt(float *, float, bool, float *);
390
391 __XIP_FPO_DLL float * xip_fpo_ufmacc_create_state
(int, int, int, int, int);

```

```

392 __XIP_FPO_DLL void xip_fpo_ufmacc_reset_state
(float *);
393 __XIP_FPO_DLL void xip_fpo_ufmacc_destroy_state
(float *);

394 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ufmacc_a_sample
(xip_fpo_t, xip_fpo_t, float *);

395 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ufmacc_s_sample
(xip_fpo_t, xip_fpo_t, float *);

396 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ufmacc_a_sampleflt
(float , float , float *);

397 __XIP_FPO_DLL xip_fpo_exc_t xip_fpo_ufmacc_s_sampleflt
(float , float , float *);
398
399 #ifdef __cplusplus
400 } /* End of "C" linkage block */
401 #endif
402
403 #endif // __xip_fpo_bitacc_cmodel_h

```

E.40 vendor/gmp.h (2,391 lines, 91,885 B)

```

1 /* Definitions for GNU multiple precision functions.  -*-
mode: c -*-
2
3 Copyright 1991, 1993, 1994, 1995, 1996, 1997, 1999, 2000,
2001, 2002, 2003,
4 2004, 2005, 2006, 2007, 2008, 2009 Free Software Foundation,
Inc.
5
6 Copyright 2008 William Hart, Gonzalo Tornaria
7
8 This file is part of the MPIR Library.
9
10 The MPIR Library is free software; you can redistribute it
and/or modify
11 it under the terms of the GNU Lesser General Public License
as published by
12 the Free Software Foundation; either version 3 of the Li-
cense, or (at your
13 option) any later version.
14
15 The MPIR Library is distributed in the hope that it will be
useful, but
16 WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY
17 or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser Gen-
eral Public
18 License for more details.
19
20 You should have received a copy of the GNU Lesser General
Public License
21 along with the GNU MP Library. If not, see http://
www.gnu.org/licenses/. */
22
23 #ifndef __GMP_H__
24
25 #if defined (__cplusplus)
26 #include <iostream> /* for std::istream, std::ostream,
std::string */
27 #include <cstdio>
28 #endif
29
30
31 /* Instantiated by configure. */
32 #if ! defined (__GMP_WITHIN_CONFIGURE)
33 #define __GMP_BITS_PER_MP_LIMB 64
34 #define __GMP_HAVE_HOST_CPU_FAMILY_power 0
35 #define __GMP_HAVE_HOST_CPU_FAMILY_powerpc 0
36 #define __GMP_LIMB_BITS 64
37 #define __GMP_NAIL_BITS 0
38 #endif
39 #define GMP_NUMB_BITS (GMP_LIMB_BITS - GMP_NAIL_BITS)
40 #define GMP_NUMB_MASK ((~ __GMP_CAST (mp_limb_t, 0)) >>
GMP_NAIL_BITS)
41 #define GMP_NUMB_MAX GMP_NUMB_MASK
42 #define GMP_NAIL_MASK (~ GMP_NUMB_MASK)
43
44
45 /* The following (everything under ifndef __GNU_MP__) must be
identical in
46 mpir.h and mp.h to allow both to be included in an applic-
ation or during
47 the library build. */
48 #ifndef __GNU_MP__
49 #define __GNU_MP__ 4
50
51 #define __need_size_t /* tell gcc stddef.h we only want
size_t */
52 #if defined (__cplusplus)
53 #include <stddef> /* for size_t */
54 #else
55 #include <stddef.h> /* for size_t */
56 #endif
57 #undef __need_size_t
58

```

```

59 /* Instantiated by configure. */
60 #if ! defined (__GMP_WITHIN_CONFIGURE)
61 /* #undef __LONG_LONG_LIMB */
62 #define __GMP_LIBGMP_DLL 0
63 #endif
64
65 /* #if defined (__GMP_WITHIN_CONFIGURE) && defined (__WIN64)
*/
66 #ifdef __WIN64
67 #define __LONG_LONG_LIMB1
68 #endif
69
70
71 /* __STDC__ - some ANSI compilers define this only to 0,
hence the use of
72 "defined" and not "__STDC__-0". In particular Sun
workshop C 5.0
73 sets __STDC__ to 0, but requires "##" for token past-
ing.
74
75
76
77
78 __DECC - current versions of DEC C (5.9 for instance) for
alpha are ANSI,
79 but don't define __STDC__ in their default mode.
Don't know if old
80 versions might have been K&R, but let's not worry
about that unless
81 someone is still using one.
82
83 __mips - gnu ansidecl.h says the RISC/OS MIPS compiler is
ANSI in SVR4
84 mode, but doesn't define __STDC__.
85
86 __MSC_VER - Microsoft C is ANSI, but __STDC__ is undefined
unless the /Za
87 option is given (in which case it's 1).
88
89 __WIN32 - tested for by gnu ansidecl.h, no doubt on the as-
sumption that
90 all w32 compilers are ansi.
91
92 Note: This same set of tests is used by gen-psqr.c and
demos/expr/expr-impl.h, so if anything needs adding, then
be sure to
93 update those too. */
94
95
96 #if defined (__STDC__) \
97 || defined (__cplusplus) \
98 || defined (__AIX) \
99 || defined (__DECC) \
100 || (defined (__mips) && defined (__SYSTYPE_SVR4)) \
101 || defined (__MSC_VER) \
102 || defined (__WIN32) \
103 #define __GMP_HAVE_CONST 1
104 #define __GMP_HAVE_PROTOTYPES 1
105 #define __GMP_HAVE_TOKEN_PASTE 1
106 #else
107 #define __GMP_HAVE_CONST 0
108 #define __GMP_HAVE_PROTOTYPES 0
109 #define __GMP_HAVE_TOKEN_PASTE 0
110 #endif
111
112
113 #if __GMP_HAVE_CONST
114 #define __gmp_const const
115 #define __gmp_signed signed
116 #else
117 #define __gmp_const
118 #define __gmp_signed
119 #endif
120
121
122 /* __GMP_DECLSPEC supports Windows DLL versions of libmpir,
and is empty in
123 all other circumstances.
124
125 When compiling objects for libmpir, __GMP_DECLSPEC is an
export directive,
126 or when compiling for an application it's an import dir-
ective. The two
127 cases are differentiated by __GMP_WITHIN_GMP defined by
the GMP Makefiles
128 (and not defined from an application).
129
130 __GMP_DECLSPEC_XX is similarly used for libmpirxx. __G-
MP_WITHIN_GMPXX
131 indicates when building libmpirxx, and in that case libm-
pirxx functions are
132 exports, but libmpir functions which might get called are
imports.
133
134 libmp.la uses __GMP_DECLSPEC, just as if it were libm-
pir.la. libmpir and
135 libmp don't call each other, so there's no conflict or
confusion.
136

```

```

137 Libtool DLL_EXPORT define is not used.
138
139 There's no attempt to support GMP built both static and
DLL. Doing so
140 would mean applications would have to tell us which of the
two is going
141 to be used when linking, and that seems very tedious and
error prone if
142 using GMP by hand, and equally tedious from a package
since autoconf and
143 automake don't give much help.
144
145 __GMP_DECLSPEC is required on all documented global func-
tions and
146 variables, the various internals in gmp-impl.h etc can be
left unadorned.
147
148 But internals used by the test programs or speed measuring
programs
149 should have __GMP_DECLSPEC, and certainly constants or
variables must
150 have it or the wrong address will be resolved.
151
152 In gcc __declspec can go at either the start or end of a
prototype.
153
154 In Microsoft C __declspec must go at the start, or after
the type like
155 void __declspec(...) *foo()). There's no __dllexport or
anything to
156 guard against someone foolish #defining dllexport. _ex-
port used to be
157 available, but no longer.
158
159 In Borland C _export still exists, but needs to go after
the type, like
160 "void _export foo()". Would have to change the __G-
MP_DECLSPEC syntax to
161
162 make use of that. Probably more trouble than it's worth.
*/
161
162 #if defined (__GNUC__)
163 #define __GMP_DECLSPEC_EXPORT __declspec(dllexport)
164 #define __GMP_DECLSPEC_IMPORT __declspec(dllimport)
165 #endif
166 #if defined (_MSC_VER) || defined (__BORLANDC__)
167 #define __GMP_DECLSPEC_EXPORT __declspec(dllexport)
168 #define __GMP_DECLSPEC_IMPORT __declspec(dllimport)
169 #endif
170 #ifdef __WATCOMC__
171 #define __GMP_DECLSPEC_EXPORT __export
172 #define __GMP_DECLSPEC_IMPORT __import
173 #endif
174 #ifdef __IBMC__
175 #define __GMP_DECLSPEC_EXPORT _Export
176 #define __GMP_DECLSPEC_IMPORT _Import
177 #endif
178
179 #if __GMP_LIBGMP_DLL
180 #if __GMP_WITHIN_GMP
181 /* compiling to go into a DLL libmpir */
182 #define __GMP_DECLSPEC __GMP_DECLSPEC_EXPORT
183 #else
184 /* compiling to go into an application which will link to a
DLL libmpir */
185 #define __GMP_DECLSPEC __GMP_DECLSPEC_IMPORT
186 #endif
187 #else
188 /* all other cases */
189 #define __GMP_DECLSPEC
190 #endif
191
192
193 #ifdef __GMP_SHORT_LIMB
194 typedef unsigned int mp_limb_t;
195 typedef int mp_limb_signed_t;
196 #else
197 #ifdef __LONG_LONG_LIMB
198 typedef unsigned long long int mp_limb_t;
199 typedef long long int mp_limb_signed_t;
200 #else
201 typedef unsigned long int mp_limb_t;
202 typedef long int mp_limb_signed_t;
203 #endif
204 #endif
205
206 #ifdef __WIN64
207 typedef unsigned long long int mp_bitcnt_t;
208 #else
209 typedef unsigned long int mp_bitcnt_t;
210 #endif
211
212 /* For reference, note that the name __mpz_struct gets into
C++ mangled
213 function names, which means although the "__" suggests an
internal, we
214 must leave this name for binary compatibility. */
215 typedef struct

```

```

216 {
217 int _mp_alloc; /* Number of *limbs* allocated
and pointed
218 to by the _mp_d field. */
219 int _mp_size; /* abs(_mp_size) is the number
of limbs the
220 last field points to. If
_mp_size is
221 negative this is a negative
number. */
222 mp_limb_t *_mp_d; /* Pointer to the limbs. */
223 } __mpz_struct;
224
225 #ifndef __GNU_MP__
226
227 typedef __mpz_struct mpz_t[1];
228
229 typedef mp_limb_t * mp_ptr;
230 typedef __gmp_const mp_limb_t *mp_srcptr;
231 #if defined (__WIN64)
232 #define __GMP_MP_SIZE_T_INT 0
233 typedef long long int mp_size_t;
234 typedef long int mp_exp_t;
235 #else
236 #define __GMP_MP_SIZE_T_INT 0
237 typedef long int mp_size_t;
238 typedef long int mp_exp_t;
239 #endif
240
241 typedef struct
242 {
243 __mpz_struct _mp_num;
244 __mpz_struct _mp_den;
245 } __mpq_struct;
246
247 typedef __mpq_struct mpq_t[1];
248
249 typedef struct
250 {
251 int _mp_prec; /* Max precision, in number of
`mp_limb_t's.
252 Set by mpf_init and modi-
fied by
253 mpf_set_prec. The area
pointed to by the
254 _mp_d field contains `prec' +
1 limbs. */
255 int _mp_size; /* abs(_mp_size) is the number
of limbs the
256 last field points to. If
_mp_size is
257 negative this is a negative
number. */
258 mp_exp_t _mp_exp; /* Exponent, in the base of
`mp_limb_t's. */
259 mp_limb_t *_mp_d; /* Pointer to the limbs. */
260 } __mpf_struct;
261
262 typedef __mpf_struct mpf_t[1];
263
264 /* Available random number generation algorithms. */
265 typedef enum
266 {
267 GMP_RAND_ALG_DEFAULT = 0,
268 GMP_RAND_ALG_LC = GMP_RAND_ALG_DEFAULT /* Linear congruen-
tial. */
269 } gmp_randalg_t;
270
271 /* Random state struct. */
272 typedef struct
273 {
274 mpf_t _mp_seed; /* _mp_d member points to state of
the generator. */
275 gmp_randalg_t _mp_alg; /* Currently unused. */
276 union {
277 void *_mp_lc; /* Pointer to function pointers
structure. */
278 } _mp_algdata;
279 } __gmp_randstate_struct;
280 typedef __gmp_randstate_struct gmp_randstate_t[1];
281
282 /* Types for function declarations in gmp files. */
283 /* ??? Should not pollute user name space with these ??? */
284 typedef __gmp_const __mpz_struct *mpz_srcptr;
285 typedef __mpz_struct *mpz_ptr;
286 typedef __gmp_const __mpf_struct *mpf_srcptr;
287 typedef __mpf_struct *mpf_ptr;
288 typedef __gmp_const __mpq_struct *mpq_srcptr;
289 typedef __mpq_struct *mpq_ptr;
290
291
292 /* This is not wanted in mp.h, so put it outside the
__GNU_MP__ common

```

```

293     section. */
294 #if __GMP_LIBGMP_DLL
295 #if __GMP_WITHIN_GMPXX
296 /* compiling to go into a DLL libmpirxx */
297 #define __GMP_DECLSPEC_XX __GMP_DECLSPEC_EXPORT
298 #else
299 /* compiling to go into a application which will link to a
DLL libmpirxx */
300 #define __GMP_DECLSPEC_XX __GMP_DECLSPEC_IMPORT
301 #endif
302 #else
303 /* all other cases */
304 #define __GMP_DECLSPEC_XX
305 #endif
306
307
308 #if __GMP_HAVE_PROTOTYPES
309 #define __GMP_PROTO(x) x
310 #else
311 #define __GMP_PROTO(x) ()
312 #endif
313
314 #ifndef __MPN
315 #if __GMP_HAVE_TOKEN_PASTE
316 #define __MPN(x) __gmpn_##x
317 #else
318 #define __MPN(x) __gmpn_/**/x
319 #endif
320 #endif
321
322 /* For reference, "defined(EOF)" cannot be used here. In g++
2.95.4,
323 <iostream> defines EOF but not FILE. */
324 #if defined
(FILE)
325 || defined (H_ST-
DIO)
326 || defined (__STDIO) \
*/
327 || defined (__STDIO_H) /* glibc, Sun, SCO
*/
328 || defined (__STDIO_H_) /* BSD, OSF
*/
329 || defined (__STDIO_H) /* Borland
*/
330 || defined (__STDIO_H_) /* IRIX
*/
331 || defined (__STDIO_INCLUDED) /* HPUX
*/
332 || defined (__FILE_DEFINED) /* Microsoft
*/
333 || defined (__STDIO__) /* Apple MPW MrC
*/
334 || defined (__MSL_STDIO_H) /* Metrowerks
*/
335 || defined (__STDIO_H_INCLUDED) /* QNX4 */
\
336 || defined (__ISO_STDIO_ISO_H) /* Sun C++ */
337 #define __GMP_H_HAVE_FILE 1
338 #endif
339
340 /* In ISO C, if a prototype involving "struct obstack *" is
given without
341 that structure defined, then the struct is scoped down to
just the
342 prototype, causing a conflict if it's subsequently defined
for real. So
343 only give prototypes if we've got obstack.h. */
344 #if defined (__OBSTACK_H) /* glibc <obstack.h> */
345 #define __GMP_H_HAVE_OBSTACK 1
346 #endif
347
348 /* The prototypes for gmp_vprintf etc are provided only if
va_list is
349 available, via an application having included <stdarg.h>
or <varargs.h>.
350 Usually va_list is a typedef so can't be tested directly,
but C99
351 specifies that va_start is a macro (and it was normally a
macro on past
352 systems too), so look for that.
353
354 <stdio.h> will define some sort of va_list for vprintf and
vfprintf, but
355 let's not bother trying to use that since it's not stand-
ard and since
356 application uses for gmp_vprintf etc will almost certainly
require the
357 whole <stdarg.h> or <varargs.h> anyway. */
358
359 #ifdef va_start
360 #define __GMP_H_HAVE_VA_LIST 1
361 #endif
362
363 /* Test for gcc >= maj.min, as per __GNUC_PREREQ in glibc */
364 #if defined (__GNUC__) && defined (__GNUC_MINOR__)
365 #define __GMP_GNUC_PREREQ(maj, min) \
366 ((__GNUC__ <= 16) + __GNUC_MINOR__ >= ((maj) <= 16) +
(min))
367 #else
368 #define __GMP_GNUC_PREREQ(maj, min) 0
369 #endif
370
371 /* "pure" is in gcc 2.96 and up, see "(gcc)Function Attrib-
utes". Basically
372 it means a function does nothing but examine its arguments
and memory
373 (global or via arguments) to generate a return value, but
changes nothing
374 and has no side-effects. __GMP_NO_ATTRIBUTE_CONST_PURE
lets
375 tune/common.c etc turn this off when trying to write tim-
ing loops. */
376 #if __GMP_GNUC_PREREQ (2,96) && ! defined (__G-
MP_NO_ATTRIBUTE_CONST_PURE)
377 #define __GMP_ATTRIBUTE_PURE __attribute__ ((__pure__))
378 #else
379 #define __GMP_ATTRIBUTE_PURE
380 #endif
381
382
383 /* __GMP_CAST allows us to use static_cast in C++, so our
macros are clean
384 to "g++ -Wold-style-cast".
385
386 Casts in "extern inline" code within an extern "C" block
don't induce
387 these warnings, so __GMP_CAST only needs to be used on
documented
388 macros. */
389
390 #ifndef __cplusplus
391 #define __GMP_CAST(type, expr) (static_cast<type> (expr))
392 #else
393 #define __GMP_CAST(type, expr) ((type) (expr))
394 #endif
395
396 /* An empty "throw ()" means the function doesn't throw any
C++ exceptions,
397 this can save some stack frame info in applications.
398
399 Currently it's given only on functions which never divide-
by-zero etc,
400 don't allocate memory, and are expected to never need to
allocate
401 memory.
402 This leaves open the possibility of a C++ throw from a fu-
ture GMP
403 exceptions scheme.
404
405 mpz_set_ui etc are omitted to leave open the lazy alloca-
tion scheme
406 described in doc/tasks.html. mpz_get_d etc are omitted to
leave open
407 exceptions for float overflows.
408
409 Note that __GMP_NOTHROW must be given on any inlines the
same as on their
410 prototypes (for g++ at least, where they're used togeth-
er). Note also
411 that g++ 3.0 demands that __GMP_NOTHROW is before other
attributes like
412 __GMP_ATTRIBUTE_PURE. */
413
414 #if defined (__cplusplus)
415 #define __GMP_NOTHROW throw ()
416 #else
417 #define __GMP_NOTHROW
418 #endif
419
420 /* PORTME: What other compilers have a useful "extern in-
line"? "static
421 inline" would be an acceptable substitute if the compiler
(or linker)
422 discards unused statics. */
423
424 /* gcc has __inline__ in all modes, including strict ansi.
Give a prototype
425 for an inline too, so as to correctly specify "dllimport"
on windows, in
426 case the function is called rather than inlined. */
427
428 #ifndef __GNUC__
429 #if defined (__APPLE_CC__) && (__APPLE_CC__ != 1) /* FSF GCC
sets this flag to 1 on Apple machines */
430
431 #if ! (__APPLE_CC__ >= 5465 && __STDC_VERSION__ >= 199901L)
432 #define __GMP_EXTERN_INLINE extern __inline__
433 #define __GMP_INLINE_PROTOTYPES 1
434 #endif
435
436 #else /* GNU CC */
437

```

```

438 #if defined(__GNUC_STDC_INLINE__) || defined (__G-
NUC_GNU_INLINE__)
439 #define __GMP_EXTERN_INLINE extern __inline__ __attrib-
ute__((gnu_inline__))
440 #else
441 #define __GMP_EXTERN_INLINE extern __inline__
442 #endif
443 #define __GMP_INLINE_PROTOTYPES 1
444
445 #endif
446 #endif
447
448 /* DEC C (eg. version 5.9) supports "static __inline foo()",
even in -std1
449 strict ANSI mode. Inlining is done even when not optimiz-
ing (ie. -O0

450 mode, which is the default), but an unnecessary local copy
of foo is

451 emitted unless -O is used. "extern __inline" is accepted,
but the
452 "extern" appears to be ignored, ie. it becomes a plain
global function
453 but which is inlined within its file. Don't know if all
old versions of
454 DEC C supported __inline, but as a start let's do the
right thing for
455 current versions. */
456 #ifdef __DECC
457 #define __GMP_EXTERN_INLINE static __inline
458 #endif
459
460 /* SCO OpenUNIX 8 cc supports "static inline foo()" but not
in -Xc strict
461 ANSI mode (__STDC__ is 1 in that mode). Inlining only ac-
tually takes
462 place under -O. Without -O "foo" seems to be emitted
whether it's used
463 or not, which is wasteful. "extern inline foo()" isn't
useful, the
464 "extern" is apparently ignored, so foo is inlined if pos-
sible but also
465 emitted as a global, which causes multiple definition er-
rors when
466 building a shared libmpir. */
467 #ifdef __SCO_VERSION__
468 #if __SCO_VERSION__ > 400000000 && __STDC__ != 1 \
&& ! defined (__GMP_EXTERN_INLINE)
469 #define __GMP_EXTERN_INLINE static inline
470 #endif
471 #endif
472 #endif
473
474 #if defined __MSC_VER
475 #define __GMP_EXTERN_INLINE static __inline
476 #endif
477
478 /* C++ always has "inline" and since it's a normal feature
the linker should
479 discard duplicate non-inlined copies, or if it doesn't
then that's a
480 problem for everyone, not just GMP. */
481 #if defined (__cplusplus) && ! defined (__GMP_EXTERN_INLINE)
482 #define __GMP_EXTERN_INLINE inline
483 #endif
484
485 /* Don't do any inlining within a configure run, since if the
compiler ends

486 up emitting copies of the code into the object file it can
end up

487 demanding the various support routines (like mpn_popcount)
for linking,
488 making the "alloca" test and perhaps others fail. And on
hppa ia64 a
489 pre-release gcc 3.2 was seen not respecting the "extern"
in "extern
490 __inline__", triggering this problem too. */
491 #if defined (__GMP_WITHIN_CONFIGURE) && ! __G-
MP_WITHIN_CONFIGURE_INLINE
492 #undef __GMP_EXTERN_INLINE
493 #endif
494
495 /* By default, don't give a prototype when there's going to
be an inline
496 version. Note in particular that Cray C++ objects to the
combination of
497 prototype and inline. */
498 #ifdef __GMP_EXTERN_INLINE
499 #ifndef __GMP_INLINE_PROTOTYPES
500 #define __GMP_INLINE_PROTOTYPES 0
501 #endif
502 #else
503 #define __GMP_INLINE_PROTOTYPES 1
504 #endif
505
506
507 #define __GMP_ABS(x) ((x) >= 0 ? (x) : -(x))

```

```

508 #define __GMP_MAX(h,i) ((h) > (i) ? (h) : (i))
509
510 /* __GMP_USHRT_MAX is not "~ (unsigned short) 0" because
short is promoted
511 to int by "-". */
512 #define __GMP_UINT_MAX (~ (unsigned) 0)
513 #define __GMP_ULONG_MAX (~ (unsigned long) 0)
514 #define __GMP_USHRT_MAX ((unsigned short) ~0)
515
516
517 /* __builtin_expect is in gcc 3.0, and not in 2.95. */
518 #if __GMP_GNUC_PREREQ (3,0)
519 #define __GMP_LIKELY(cond) __builtin_expect ((cond) != 0,
1)
520 #define __GMP_UNLIKELY(cond) __builtin_expect ((cond) != 0,
0)
521 #else
522 #define __GMP_LIKELY(cond) (cond)
523 #define __GMP_UNLIKELY(cond) (cond)
524 #endif
525
526 /* Allow direct user access to numerator and denominator of a
mpq_t object. */
527 #define mpq_numref(Q) (&((Q)->_mp_num))
528 #define mpq_denref(Q) (&((Q)->_mp_den))
529
530
531 #if defined (__cplusplus)
532 extern "C" {
533 using std::FILE;
534 #endif
535
536 #define mp_set_memory_functions __gmp_set_memory_functions
537 __GMP_DECLSPEC void mp_set_memory_functions __GMP_PROTO
((void (*)(size_t),
538 void (*)(void *,
size_t, size_t),
539 void (*)(void *,
size_t))) __GMP_NOTHROW;
540
541 #define mp_get_memory_functions __gmp_get_memory_functions
542 __GMP_DECLSPEC void mp_get_memory_functions __GMP_PROTO
((void (*)(size_t),
543 void (*)(void *,
size_t, size_t),
544 void (*)(void *,
size_t))) __GMP_NOTHROW;
545
546 #define mp_bits_per_limb __gmp_bits_per_limb
547 __GMP_DECLSPEC extern __gmp_const int mp_bits_per_limb;
548
549 #define gmp_errno __gmp_errno
550 __GMP_DECLSPEC extern int gmp_errno;
551
552 #define gmp_version __gmp_version
553 __GMP_DECLSPEC extern __gmp_const char * __gmp_const gmp_ver-
sion;
554
555 #define mpir_version __mpir_version
556 __GMP_DECLSPEC extern __gmp_const char * __gmp_const
mpir_version;
557
558 /***** Random number routines. *****/
559
560 #define gmp_randinit_default __gmp_randinit_default
561 __GMP_DECLSPEC void gmp_randinit_default __GMP_PROTO
((gmp_randstate_t));
562
563 #define gmp_randinit_lc_2exp __gmp_randinit_lc_2exp
564 __GMP_DECLSPEC void gmp_randinit_lc_2exp __GMP_PROTO
((gmp_randstate_t,
565 mpz_sr-
cptr, unsigned long int,
566 mp_bitcnt_t));
567
568 #define gmp_randinit_lc_2exp_size __gmp_randinit_lc_2exp_size
569 __GMP_DECLSPEC int gmp_randinit_lc_2exp_size __GMP_PROTO
((gmp_randstate_t, mp_bitcnt_t));
570
571 #define gmp_randinit_mt __gmp_randinit_mt
572 __GMP_DECLSPEC void gmp_randinit_mt __GMP_PROTO ((gmp_rand-
state_t));
573
574 #define gmp_randinit_set __gmp_randinit_set
575 __GMP_DECLSPEC void gmp_randinit_set __GMP_PROTO ((gmp_rand-
state_t, __gmp_const __gmp_randstate_struct *));
576
577 #define gmp_randseed __gmp_randseed
578 __GMP_DECLSPEC void gmp_randseed __GMP_PROTO ((gmp_rand-
state_t, mpz_srcptr));
579
580 #define gmp_randseed_ui __gmp_randseed_ui
581 __GMP_DECLSPEC void gmp_randseed_ui __GMP_PROTO ((gmp_rand-
state_t, unsigned long int));
582
583 #define gmp_randclear __gmp_randclear
584 __GMP_DECLSPEC void gmp_randclear __GMP_PROTO ((gmp_rand-

```



```

state_t));
586
587 #define gmp_urandomb_ui __gmp_urandomb_ui
588 __GMP_DECLSPEC unsigned long gmp_urandomb_ui __GMP_PROTO
((gmp_randstate_t, unsigned long));
589
590 #define gmp_urandomm_ui __gmp_urandomm_ui
591 __GMP_DECLSPEC unsigned long gmp_urandomm_ui __GMP_PROTO
((gmp_randstate_t, unsigned long));
592
593
594 /***** Formatted output routines.
595 *****/
596
597 #define gmp_asprintf __gmp_asprintf
598 __GMP_DECLSPEC int gmp_asprintf __GMP_PROTO ((char **, __g-
mp_const char *, ...));
599
600 #define gmp_fprintf __gmp_fprintf
601 #ifdef _GMP_H_HAVE_FILE
602 __GMP_DECLSPEC int gmp_fprintf __GMP_PROTO ((FILE *, __g-
mp_const char *, ...));
603 #endif
604
605 #define gmp_obstack_printf __gmp_obstack_printf
606 #if defined (_GMP_H_HAVE_OBSTACK)
607 __GMP_DECLSPEC int gmp_obstack_printf __GMP_PROTO ((struct
obstack *, __gmp_const char *, ...));
608 #endif
609
610 #define gmp_obstack_vprintf __gmp_obstack_vprintf
611 #if defined (_GMP_H_HAVE_OBSTACK) && defined
(_GMP_H_HAVE_VA_LIST)
612 __GMP_DECLSPEC int gmp_obstack_vprintf __GMP_PROTO ((struct
obstack *, __gmp_const char *, va_list));
613 #endif
614
615 #define gmp_printf __gmp_printf
616 __GMP_DECLSPEC int gmp_printf __GMP_PROTO ((__gmp_const char
*, ...));
617
618 #define gmp_snprintf __gmp_snprintf
619
620 __GMP_DECLSPEC int gmp_snprintf __GMP_PROTO ((char *, size_t,
__gmp_const char *, ...));
621
622 #define gmp_sprintf __gmp_sprintf
623 __GMP_DECLSPEC int gmp_sprintf __GMP_PROTO ((char *, __g-
mp_const char *, ...));
624
625 #define gmp_vasprintf __gmp_vasprintf
626 #if defined (_GMP_H_HAVE_VA_LIST)
627 __GMP_DECLSPEC int gmp_vasprintf __GMP_PROTO ((char **, __g-
mp_const char *, va_list));
628 #endif
629
630 #define gmp_vfprintf __gmp_vfprintf
631 #if defined (_GMP_H_HAVE_FILE) && defined
(_GMP_H_HAVE_VA_LIST)
632 __GMP_DECLSPEC int gmp_vfprintf __GMP_PROTO ((FILE *, __g-
mp_const char *, va_list));
633 #endif
634
635 #define gmp_vprintf __gmp_vprintf
636 #if defined (_GMP_H_HAVE_VA_LIST)
637 __GMP_DECLSPEC int gmp_vprintf __GMP_PROTO ((__gmp_const char
*, va_list));
638 #endif
639
640 #define gmp_vsnprintf __gmp_vsnprintf
641 #if defined (_GMP_H_HAVE_VA_LIST)
642 __GMP_DECLSPEC int gmp_vsnprintf __GMP_PROTO ((char *,
size_t, __gmp_const char *, va_list));
643 #endif
644
645 #define gmp_vsprintf __gmp_vsprintf
646 #if defined (_GMP_H_HAVE_VA_LIST)
647 __GMP_DECLSPEC int gmp_vsprintf __GMP_PROTO ((char *, __g-
mp_const char *, va_list));
648 #endif
649
650 /***** Formatted input routines.
651 *****/
652
653 #define gmp_fscanf __gmp_fscanf
654 #ifdef _GMP_H_HAVE_FILE
655 __GMP_DECLSPEC int gmp_fscanf __GMP_PROTO ((FILE *, __g-
mp_const char *, ...));
656 #endif
657
658 #define gmp_scanf __gmp_scanf
659 __GMP_DECLSPEC int gmp_scanf __GMP_PROTO ((__gmp_const char
*, ...));
660
661 #define gmp_sscanf __gmp_sscanf
662 __GMP_DECLSPEC int gmp_sscanf __GMP_PROTO ((__gmp_const char
*, __gmp_const char *, ...));

```

```

663 #define gmp_vfscanf __gmp_vfscanf
664 #if defined (_GMP_H_HAVE_FILE) && defined
(_GMP_H_HAVE_VA_LIST)
665 __GMP_DECLSPEC int gmp_vfscanf __GMP_PROTO ((FILE *, __g-
mp_const char *, va_list));
666 #endif
667
668 #define gmp_vscanf __gmp_vscanf
669 #if defined (_GMP_H_HAVE_VA_LIST)
670 __GMP_DECLSPEC int gmp_vscanf __GMP_PROTO ((__gmp_const char
*, va_list));
671 #endif
672
673 #define gmp_vsscanf __gmp_vsscanf
674 #if defined (_GMP_H_HAVE_VA_LIST)
675 __GMP_DECLSPEC int gmp_vsscanf __GMP_PROTO ((__gmp_const char
*, __gmp_const char *, va_list));
676 #endif
677
678 /***** Integer (i.e. Z) routines.
679 *****/
680
681 #define __GMP_BITS_PER_ULONGLONG (8*sizeof(unsigned long))
682
683 #define __GMP_REALLOC __gmpz_realloc
684 #define __GMP_REALLOC __gmpz_realloc
685 __GMP_DECLSPEC void *__GMP_REALLOC __GMP_PROTO ((mpz_ptr,
mp_size_t));
686
687 #define __GMP_ABS __gmpz_abs
688 #define __GMP_ABS_MIN_ALLOC(x,y) (__GMP_ABS(y->mp_size))
689 #define __GMP_INLINE_PROTOTYPES || defined (_GMP_FORCE_mps_abs)
690 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr));
691 #endif
692
693 #define __GMP_MPZ_ADD_MIN_ALLOC(x,y,z) (__G-
MP_MAX(__GMP_ABS(y->mp_size), __GMP_ABS(z->mp_size))+1)
694 #define __GMP_MPZ_ADD __gmpz_add
695 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
696
697 #define __GMP_MPZ_ADD_UI_MIN_ALLOC(x,y,z) (__G-
MP_MAX(__GMP_ABS(y->mp_size), 1+(__GMP_BITS_PER_ULONGLONG-1)/
GMP_NUMB_BITS)+1)
698 #define __GMP_MPZ_ADD_UI __gmpz_add_ui
699 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long int));
700
701 #define __GMP_MPZ_ADDMUL __gmpz_addmul
702 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
703
704 #define __GMP_MPZ_ADDMUL_UI __gmpz_addmul_ui
705 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr,
mpz_srcptr, unsigned long int));
706
707 #define __GMP_MPZ_AND __gmpz_and
708 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
709
710 #define __GMP_MPZ_ARRAY_INIT __gmpz_array_init
711 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr,
mp_size_t, mp_size_t));
712
713 #define __GMP_MPZ_BIN_UI __gmpz_bin_ui
714 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long int));
715
716 #define __GMP_MPZ_BIN_UIUI __gmpz_bin_uiui
717 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, un-
signed long int, unsigned long int));
718
719 #define __GMP_MPZ_CDIV_Q __gmpz_cdiv_q
720 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
721
722 #define __GMP_MPZ_CDIV_Q_2EXP __gmpz_cdiv_q_2exp
723 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr,
mpz_srcptr, unsigned long int));
724
725 #define __GMP_MPZ_CDIV_Q_UI __gmpz_cdiv_q_ui
726 __GMP_DECLSPEC unsigned long int __GMP_REALLOC __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
727
728 #define __GMP_MPZ_CDIV_QR __gmpz_cdiv_qr
729 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr,
mpz_ptr, mpz_srcptr, mpz_srcptr));
730
731 #define __GMP_MPZ_CDIV_QR_UI __gmpz_cdiv_qr_ui
732 __GMP_DECLSPEC unsigned long int __GMP_REALLOC __GMP_PROTO
((mpz_ptr, mpz_ptr, mpz_srcptr, unsigned long int));
733
734 #define __GMP_MPZ_CDIV_R __gmpz_cdiv_r
735 __GMP_DECLSPEC void __GMP_REALLOC __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
736
737 #define __GMP_MPZ_CDIV_R_2EXP __gmpz_cdiv_r_2exp

```

```

737 __GMP_DECLSPEC void mpz_cdiv_r_2exp __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mp_bitcnt_t));
738
739 #define mpz_cdiv_r_ui __gmpz_cdiv_r_ui
740 __GMP_DECLSPEC unsigned long int mpz_cdiv_r_ui __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
741
742 #define mpz_cdiv_ui __gmpz_cdiv_ui
743 __GMP_DECLSPEC unsigned long int mpz_cdiv_ui __GMP_PROTO
((mpz_srcptr, unsigned long int)) __GMP_ATTRIBUTE_PURE;
744
745 #define mpz_clear __gmpz_clear
746 __GMP_DECLSPEC void mpz_clear __GMP_PROTO ((mpz_ptr));
747
748 #define mpz_clears __gmpz_clears
749 __GMP_DECLSPEC void mpz_clears __GMP_PROTO ((mpz_ptr, ...));
750
751 #define mpz_clrbit __gmpz_clrbit
752 __GMP_DECLSPEC void mpz_clrbit __GMP_PROTO ((mpz_ptr,
mp_bitcnt_t));
753
754 #define mpz_cmp __gmpz_cmp
755 __GMP_DECLSPEC int mpz_cmp __GMP_PROTO ((mpz_srcptr, mpz_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
756
757 #define mpz_cmp_d __gmpz_cmp_d
758 __GMP_DECLSPEC int mpz_cmp_d __GMP_PROTO ((mpz_srcptr,
double)) __GMP_ATTRIBUTE_PURE;
759
760 #define mpz_cmp_si __gmpz_cmp_si
761 __GMP_DECLSPEC int mpz_cmp_si __GMP_PROTO ((mpz_srcptr,
signed long int)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
762
763 #define mpz_cmp_ui __gmpz_cmp_ui
764 __GMP_DECLSPEC int mpz_cmp_ui __GMP_PROTO ((mpz_srcptr, un-
signed long int)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
765
766 #define mpz_cmpabs __gmpz_cmpabs
767 __GMP_DECLSPEC int mpz_cmpabs __GMP_PROTO ((mpz_srcptr,
mpz_srcptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
768
769 #define mpz_cmpabs_d __gmpz_cmpabs_d
770 __GMP_DECLSPEC int mpz_cmpabs_d __GMP_PROTO ((mpz_srcptr,
double)) __GMP_ATTRIBUTE_PURE;
771
772 #define mpz_cmpabs_ui __gmpz_cmpabs_ui
773 __GMP_DECLSPEC int mpz_cmpabs_ui __GMP_PROTO ((mpz_srcptr,
unsigned long int)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
774
775 #define mpz_com __gmpz_com
776 __GMP_DECLSPEC void mpz_com __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr));
777
778 #define mpz_combit __gmpz_combit
779 __GMP_DECLSPEC void mpz_combit __GMP_PROTO ((mpz_ptr,
mp_bitcnt_t));
780
781 #define mpz_congruent_p __gmpz_congruent_p
782 __GMP_DECLSPEC int mpz_congruent_p __GMP_PROTO ((mpz_srcptr,
mpz_srcptr, mpz_srcptr)) __GMP_ATTRIBUTE_PURE;
783
784 #define mpz_congruent_2exp_p __gmpz_congruent_2exp_p
785 __GMP_DECLSPEC int mpz_congruent_2exp_p __GMP_PROTO ((mpz_sr-
cptr, mpz_srcptr, mp_bitcnt_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
786
787 #define mpz_congruent_ui_p __gmpz_congruent_ui_p
788 __GMP_DECLSPEC int mpz_congruent_ui_p __GMP_PROTO ((mpz_sr-
cptr, unsigned long, unsigned long)) __GMP_ATTRIBUTE_PURE;
789
790 #define mpz_divexact __gmpz_divexact
791 __GMP_DECLSPEC void mpz_divexact __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mpz_srcptr));
792
793 #define mpz_divexact_ui __gmpz_divexact_ui
794 __GMP_DECLSPEC void mpz_divexact_ui __GMP_PROTO ((mpz_ptr,
mpz_srcptr, unsigned long));
795
796 #define mpz_divisible_p __gmpz_divisible_p
797 __GMP_DECLSPEC int mpz_divisible_p __GMP_PROTO ((mpz_srcptr,
mpz_srcptr)) __GMP_ATTRIBUTE_PURE;
798
799 #define mpz_divisible_ui_p __gmpz_divisible_ui_p
800 __GMP_DECLSPEC int mpz_divisible_ui_p __GMP_PROTO ((mpz_sr-
cptr, unsigned long)) __GMP_ATTRIBUTE_PURE;
801
802 #define mpz_divisible_2exp_p __gmpz_divisible_2exp_p
803 __GMP_DECLSPEC int mpz_divisible_2exp_p __GMP_PROTO ((mpz_sr-
cptr, mp_bitcnt_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
804
805 #define mpz_dump __gmpz_dump
806 __GMP_DECLSPEC void mpz_dump __GMP_PROTO ((mpz_srcptr));
807
808 #define mpz_export __gmpz_export
809 __GMP_DECLSPEC void *mpz_export __GMP_PROTO ((void *, size_t
*, int, size_t, int, size_t, mpz_srcptr));
810
811 #define mpz_fac_ui __gmpz_fac_ui
812 __GMP_DECLSPEC void mpz_fac_ui __GMP_PROTO ((mpz_ptr, un-
signed long int));
813

```

```

814 #define mpz_fdiv_q __gmpz_fdiv_q
815 __GMP_DECLSPEC void mpz_fdiv_q __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
816
817 #define mpz_fdiv_q_2exp __gmpz_fdiv_q_2exp
818 __GMP_DECLSPEC void mpz_fdiv_q_2exp __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mp_bitcnt_t));
819
820 #define mpz_fdiv_q_ui __gmpz_fdiv_q_ui
821 __GMP_DECLSPEC unsigned long int mpz_fdiv_q_ui __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
822
823 #define mpz_fdiv_qr __gmpz_fdiv_qr
824 __GMP_DECLSPEC void mpz_fdiv_qr __GMP_PROTO ((mpz_ptr,
mpz_ptr, mpz_srcptr, mpz_srcptr));
825
826 #define mpz_fdiv_qr_ui __gmpz_fdiv_qr_ui
827 __GMP_DECLSPEC unsigned long int mpz_fdiv_qr_ui __GMP_PROTO
((mpz_ptr, mpz_ptr, mpz_srcptr, unsigned long int));
828
829 #define mpz_fdiv_r __gmpz_fdiv_r
830 __GMP_DECLSPEC void mpz_fdiv_r __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
831
832 #define mpz_fdiv_r_2exp __gmpz_fdiv_r_2exp
833 __GMP_DECLSPEC void mpz_fdiv_r_2exp __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mp_bitcnt_t));
834
835 #define mpz_fdiv_r_ui __gmpz_fdiv_r_ui
836 __GMP_DECLSPEC unsigned long int mpz_fdiv_r_ui __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
837
838 #define mpz_fdiv_ui __gmpz_fdiv_ui
839 __GMP_DECLSPEC unsigned long int mpz_fdiv_ui __GMP_PROTO
((mpz_srcptr, unsigned long int)) __GMP_ATTRIBUTE_PURE;
840
841 #define mpz_fib_ui __gmpz_fib_ui
842 __GMP_DECLSPEC void mpz_fib_ui __GMP_PROTO ((mpz_ptr, un-
signed long int));
843
844 #define mpz_fib2_ui __gmpz_fib2_ui
845 __GMP_DECLSPEC void mpz_fib2_ui __GMP_PROTO ((mpz_ptr,
unsigned long int));
846
847 #define mpz_fits_sint_p __gmpz_fits_sint_p
848 __GMP_DECLSPEC int mpz_fits_sint_p __GMP_PROTO ((mpz_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
849
850 #define mpz_fits_slong_p __gmpz_fits_slong_p
851 __GMP_DECLSPEC int mpz_fits_slong_p __GMP_PROTO ((mpz_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
852
853 #define mpz_fits_ushort_p __gmpz_fits_ushort_p
854 __GMP_DECLSPEC int mpz_fits_ushort_p __GMP_PROTO ((mpz_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
855
856 #define mpz_fits_uint_p __gmpz_fits_uint_p
857 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mpf_fits_uint_p)
858 __GMP_DECLSPEC int mpz_fits_uint_p __GMP_PROTO ((mpz_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
859 #endif
860
861 #define mpz_fits_ulong_p __gmpz_fits_ulong_p
862 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mpf_fits_ulong_p)
863 __GMP_DECLSPEC int mpz_fits_ulong_p __GMP_PROTO ((mpz_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
864 #endif
865
866 #define mpz_fits_ushort_p __gmpz_fits_ushort_p
867 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mpf_fits_ushort_p)
868 __GMP_DECLSPEC int mpz_fits_ushort_p __GMP_PROTO ((mpz_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
869 #endif
870
871 #define mpz_gcd __gmpz_gcd
872 __GMP_DECLSPEC void mpz_gcd __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
873
874 #define mpz_gcd_ui __gmpz_gcd_ui
875 __GMP_DECLSPEC unsigned long int mpz_gcd_ui __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
876
877 #define mpz_gcdext __gmpz_gcdext
878 __GMP_DECLSPEC void mpz_gcdext __GMP_PROTO ((mpz_ptr,
mpz_ptr, mpz_ptr, mpz_srcptr, mpz_srcptr));
879
880 #define mpz_get_d __gmpz_get_d
881 __GMP_DECLSPEC double mpz_get_d __GMP_PROTO ((mpz_srcptr))
__GMP_ATTRIBUTE_PURE;
882
883 #define mpz_get_d_2exp __gmpz_get_d_2exp
884 __GMP_DECLSPEC double mpz_get_d_2exp __GMP_PROTO ((signed
long int *, mpz_srcptr));
885
886 #define mpz_get_si __gmpz_get_si

```



```

887 __GMP_DECLSPEC /* signed */ long int mpz_get_si __GMP_PROTO
((mpz_srcptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
888
889 #define mpz_get_str __gmpz_get_str
890 __GMP_DECLSPEC char *mpz_get_str __GMP_PROTO ((char *, int,
mpz_srcptr));
891
892 #define mpz_get_ui __gmpz_get_ui
893 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mmpz_get_ui)
894 __GMP_DECLSPEC unsigned long int mpz_get_ui __GMP_PROTO
((mpz_srcptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
895 #endif
896
897 #define mpz_getlimbn __gmpz_getlimbn
898 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mmpz_getlimbn)
899 __GMP_DECLSPEC mp_limb_t mpz_getlimbn __GMP_PROTO ((mpz_sr-
cptr, mp_size_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
900 #endif
901
902 #define mpz_hamdist __gmpz_hamdist
903 __GMP_DECLSPEC mp_bitcnt_t mpz_hamdist __GMP_PROTO ((mpz_sr-
cptr, mpz_srcptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
904
905 #define mpz_import __gmpz_import
906 __GMP_DECLSPEC void mpz_import __GMP_PROTO ((mpz_ptr, size_t,
int, size_t, int, size_t, __gmp_const void *));
907
908 #define mpz_init __gmpz_init
909 __GMP_DECLSPEC void mpz_init __GMP_PROTO ((mpz_ptr));
910
911 #define mpz_init2 __gmpz_init2
912 __GMP_DECLSPEC void mpz_init2 __GMP_PROTO ((mpz_ptr,
mp_bitcnt_t));
913
914 #define mpz_inits __gmpz_inits
915 __GMP_DECLSPEC void mpz_inits __GMP_PROTO ((mpz_ptr, ...));
916
917 #define mpz_init_set __gmpz_init_set
918 __GMP_DECLSPEC void mpz_init_set __GMP_PROTO ((mpz_ptr,
mpz_srcptr));
919
920 #define mpz_init_set_d __gmpz_init_set_d
921 __GMP_DECLSPEC void mpz_init_set_d __GMP_PROTO ((mpz_ptr,
double));
922
923 #define mpz_init_set_si __gmpz_init_set_si
924 __GMP_DECLSPEC void mpz_init_set_si __GMP_PROTO ((mpz_ptr,
signed long int));
925
926 #define mpz_init_set_str __gmpz_init_set_str
927 __GMP_DECLSPEC int mpz_init_set_str __GMP_PROTO ((mpz_ptr,
__gmp_const char *, int));
928
929 #define mpz_init_set_ui __gmpz_init_set_ui
930 __GMP_DECLSPEC void mpz_init_set_ui __GMP_PROTO ((mpz_ptr,
unsigned long int));
931
932 #define mpz_inp_raw __gmpz_inp_raw
933 #ifdef _GMP_H_HAVE_FILE
934 __GMP_DECLSPEC size_t mpz_inp_raw __GMP_PROTO ((mpz_ptr, FILE
*));
935 #endif
936
937 #define mpz_inp_str __gmpz_inp_str
938 #ifdef _GMP_H_HAVE_FILE
939 __GMP_DECLSPEC size_t mpz_inp_str __GMP_PROTO ((mpz_ptr, FILE
*, int));
940 #endif
941
942 #define mpz_invert __gmpz_invert
943 __GMP_DECLSPEC int mpz_invert __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
944
945 #define mpz_ior __gmpz_ior
946 __GMP_DECLSPEC void mpz_ior __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
947
948 #define mpz_jacobi __gmpz_jacobi
949 __GMP_DECLSPEC int mpz_jacobi __GMP_PROTO ((mpz_srcptr,
mpz_srcptr)) __GMP_ATTRIBUTE_PURE;
950
951 #define mpz_kronecker mpz_jacobi /* alias */
952
953 #define mpz_kronecker_si __gmpz_kronecker_si
954 __GMP_DECLSPEC int mpz_kronecker_si __GMP_PROTO ((mpz_srcptr,
long)) __GMP_ATTRIBUTE_PURE;
955
956 #define mpz_kronecker_ui __gmpz_kronecker_ui
957 __GMP_DECLSPEC int mpz_kronecker_ui __GMP_PROTO ((mpz_srcptr,
unsigned long)) __GMP_ATTRIBUTE_PURE;
958
959 #define mpz_si_kronecker __gmpz_si_kronecker
960 __GMP_DECLSPEC int mpz_si_kronecker __GMP_PROTO ((long,

```

```

mpz_srcptr)) __GMP_ATTRIBUTE_PURE;
961
962 #define mpz_ui_kronecker __gmpz_ui_kronecker
963 __GMP_DECLSPEC int mpz_ui_kronecker __GMP_PROTO ((unsigned
long, mpz_srcptr)) __GMP_ATTRIBUTE_PURE;
964
965 #define mpz_lcm __gmpz_lcm
966 __GMP_DECLSPEC void mpz_lcm __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
967
968 #define mpz_lcm_ui __gmpz_lcm_ui
969 __GMP_DECLSPEC void mpz_lcm_ui __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long));
970
971 #define mpz_legendre mpz_jacobi /* alias */
972
973 #define mpz_lucnum_ui __gmpz_lucnum_ui
974 __GMP_DECLSPEC void mpz_lucnum_ui __GMP_PROTO ((mpz_ptr, un-
signed long int));
975
976 #define mpz_lucnum2_ui __gmpz_lucnum2_ui
977 __GMP_DECLSPEC void mpz_lucnum2_ui __GMP_PROTO ((mpz_ptr,
mpz_ptr, unsigned long int));
978
979 #define mpz_millerrabin __gmpz_millerrabin
980 __GMP_DECLSPEC int mpz_millerrabin __GMP_PROTO ((mpz_srcptr,
int)) __GMP_ATTRIBUTE_PURE;
981
982 #define mpz_mod __gmpz_mod
983 __GMP_DECLSPEC void mpz_mod __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
984
985 #define mpz_mod_ui mpz_fdiv_r_ui /* same as fdiv_r because
divisor unsigned */
986
987 #define __GMP_MPZ_MUL_MIN_ALLOC(x,y,z) (__GMP_ABS(y->_mp_s-
ize)+__GMP_ABS(z->_mp_size)+1)
988 #define mpz_mul __gmpz_mul
989 __GMP_DECLSPEC void mpz_mul __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
990
991 #define mpz_mul_2exp __gmpz_mul_2exp
992 __GMP_DECLSPEC void mpz_mul_2exp __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mp_bitcnt_t));
993
994 #define __GMP_MPZ_MUL_SI_MIN_ALLOC(x,y,z) (__GMP_ABS(y->
_mp_size)+(__GMP_BITS_PER_ULONG-1)/GMP_NUMB_BITS+1)
995 #define mpz_mul_si __gmpz_mul_si
996 __GMP_DECLSPEC void mpz_mul_si __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, long int));
997
998 #define __GMP_MPZ_MUL_UI_MIN_ALLOC(x,y,z) (__GMP_ABS(y->
_mp_size)+(__GMP_BITS_PER_ULONG-1)/GMP_NUMB_BITS+1)
999 #define mpz_mul_ui __gmpz_mul_ui
1000 __GMP_DECLSPEC void mpz_mul_ui __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long int));
1001
1002 #define mpz_neg __gmpz_neg
1003 #define __GMP_MPZ_NEG_MIN_ALLOC(x,y) (__GMP_ABS(y->_mp_size))
1004 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mmpz_neg)
1005 __GMP_DECLSPEC void mpz_neg __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr));
1006 #endif
1007
1008 #define mpz_nextprime __gmpz_nextprime
1009 __GMP_DECLSPEC void mpz_nextprime __GMP_PROTO ((mpz_ptr,
mpz_srcptr));
1010
1011 #define mpz_next_likely_prime __gmpz_next_likely_prime
1012 __GMP_DECLSPEC void mpz_next_likely_prime __GMP_PROTO
((mpz_ptr, mpz_srcptr, gmp_randstate_t));
1013
1014 #define mpz_out_raw __gmpz_out_raw
1015 #ifdef _GMP_H_HAVE_FILE
1016 __GMP_DECLSPEC size_t mpz_out_raw __GMP_PROTO ((FILE *,
mpz_srcptr));
1017 #endif
1018
1019 #define mpz_out_str __gmpz_out_str
1020 #ifdef _GMP_H_HAVE_FILE
1021 __GMP_DECLSPEC size_t mpz_out_str __GMP_PROTO ((FILE *, int,
mpz_srcptr));
1022 #endif
1023
1024 #define mpz_perfect_power_p __gmpz_perfect_power_p
1025 __GMP_DECLSPEC int mpz_perfect_power_p __GMP_PROTO ((mpz_sr-
cptr)) __GMP_ATTRIBUTE_PURE;
1026
1027 #define mpz_perfect_square_p __gmpz_perfect_square_p
1028 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mmpz_perfect_square_p)
1029 __GMP_DECLSPEC int mpz_perfect_square_p __GMP_PROTO ((mpz_sr-
cptr)) __GMP_ATTRIBUTE_PURE;
1030 #endif
1031
1032 #define mpz_popcount __gmpz_popcount
1033 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mmpz_popcount)
1034 __GMP_DECLSPEC mp_bitcnt_t mpz_popcount __GMP_PROTO ((mpz_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;

```

```

1035 #endif
1036
1037 #define mpz_pow_ui __gmpz_pow_ui
1038 __GMP_DECLSPEC void mpz_pow_ui __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long int));
1039
1040 #define mpz_powm __gmpz_powm
1041 __GMP_DECLSPEC void mpz_powm __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr, mpz_srcptr));
1042
1043 #define mpz_powm_ui __gmpz_powm_ui
1044 __GMP_DECLSPEC void mpz_powm_ui __GMP_PROTO ((mpz_ptr, mpz_s-
rcptr, unsigned long int, mpz_srcptr));
1045
1046 #define mpz_probab_prime_p __gmpz_probab_prime_p
1047 __GMP_DECLSPEC int mpz_probab_prime_p __GMP_PROTO ((mpz_sr-
cptr, int)) __GMP_ATTRIBUTE_PURE;
1048
1049 #define mpz_probable_prime_p __gmpz_probable_prime_p
1050 __GMP_DECLSPEC int mpz_probable_prime_p __GMP_PROTO ((mpz_sr-
cptr, mpz_randstate_t, int, unsigned long));
1051
1052 #define mpz_likely_prime_p __gmpz_likely_prime_p
1053 __GMP_DECLSPEC int mpz_likely_prime_p __GMP_PROTO ((mpz_sr-
cptr, mpz_randstate_t, unsigned long));
1054
1055 #define mpz_realloc2 __gmpz_realloc2
1056 __GMP_DECLSPEC void mpz_realloc2 __GMP_PROTO ((mpz_ptr,
mp_bitcnt_t));
1057
1058 #define mpz_remove __gmpz_remove
1059 __GMP_DECLSPEC unsigned long int mpz_remove __GMP_PROTO
((mpz_ptr, mpz_srcptr, mpz_srcptr));
1060
1061 #define mpz_root __gmpz_root
1062 __GMP_DECLSPEC int mpz_root __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long int));
1063
1064 #define mpz_nthroot __gmpz_nthroot
1065 __GMP_DECLSPEC void mpz_nthroot __GMP_PROTO ((mpz_ptr, mpz_s-
rcptr, unsigned long int));
1066
1067 #define mpz_rootrem __gmpz_rootrem
1068 __GMP_DECLSPEC void mpz_rootrem __GMP_PROTO
((mpz_ptr, mpz_ptr, mpz_srcptr, unsigned long int));
1069
1070 #define mpz_rrandomb __gmpz_rrandomb
1071 __GMP_DECLSPEC void mpz_rrandomb __GMP_PROTO ((mpz_ptr,
gmp_randstate_t, mp_bitcnt_t));
1072
1073 #define mpz_scan0 __gmpz_scan0
1074 __GMP_DECLSPEC mp_bitcnt_t mpz_scan0 __GMP_PROTO ((mpz_sr-
cptr, mp_bitcnt_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1075
1076 #define mpz_scan1 __gmpz_scan1
1077 __GMP_DECLSPEC mp_bitcnt_t mpz_scan1 __GMP_PROTO ((mpz_sr-
cptr, mp_bitcnt_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1078
1079 #define __GMP_MPZ_SET_MIN_ALLOC(x,y) __GMP_ABS(y->mp_size)
1080 #define mpz_set __gmpz_set
1081 __GMP_DECLSPEC void mpz_set __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr));
1082
1083 #define mpz_set_d __gmpz_set_d
1084 __GMP_DECLSPEC void mpz_set_d __GMP_PROTO ((mpz_ptr,
double));
1085
1086 #define mpz_set_f __gmpz_set_f
1087 __GMP_DECLSPEC void mpz_set_f __GMP_PROTO ((mpz_ptr, mpf_sr-
cptr));
1088
1089 #define mpz_set_q __gmpz_set_q
1090 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mpz_set_q)
1091 __GMP_DECLSPEC void mpz_set_q __GMP_PROTO ((mpz_ptr, mpq_sr-
cptr));
1092 #endif
1093
1094 #define __GMP_MPZ_SET_SI_MIN_ALLOC(x,y) (1+(__G-
MP_BITS_PER_ULONGLONG-1)/GMP_NUMB_BITS)
1095 #define mpz_set_si __gmpz_set_si
1096 __GMP_DECLSPEC void mpz_set_si __GMP_PROTO ((mpz_ptr, signed
long int));
1097
1098 #define mpz_set_str __gmpz_set_str
1099 __GMP_DECLSPEC int mpz_set_str __GMP_PROTO ((mpz_ptr, __g-
mp_const char *, int));
1100
1101 #define __GMP_MPZ_SET_UI_MIN_ALLOC(x,y) (1+(__G-
MP_BITS_PER_ULONGLONG-1)/GMP_NUMB_BITS)
1102 #define mpz_set_ui __gmpz_set_ui
1103 __GMP_DECLSPEC void mpz_set_ui __GMP_PROTO ((mpz_ptr, un-
signed long int));
1104
1105 #define mpz_setbit __gmpz_setbit
1106 __GMP_DECLSPEC void mpz_setbit __GMP_PROTO ((mpz_ptr,
mp_bitcnt_t));
1107
1108 #define mpz_size __gmpz_size
1109 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mpz_size)

```

```

1110 __GMP_DECLSPEC size_t mpz_size __GMP_PROTO ((mpz_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1111 #endif
1112
1113 #define mpz_sizeinbase __gmpz_sizeinbase
1114 __GMP_DECLSPEC size_t mpz_sizeinbase __GMP_PROTO ((mpz_sr-
cptr, int)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1115
1116 #define mpz_sqrt __gmpz_sqrt
1117 __GMP_DECLSPEC void mpz_sqrt __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr));
1118
1119 #define mpz_sqrtrem __gmpz_sqrtrem
1120 __GMP_DECLSPEC void mpz_sqrtrem __GMP_PROTO ((mpz_ptr,
mpz_ptr, mpz_srcptr));
1121
1122 #define __GMP_MPZ_SUB_MIN_ALLOC(x,y,z) (__G-
MP_MAX(__GMP_ABS(y->mp_size), __GMP_ABS(z->mp_size))+1)
1123 #define mpz_sub __gmpz_sub
1124 __GMP_DECLSPEC void mpz_sub __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
1125
1126 #define __GMP_MPZ_SUB_UI_MIN_ALLOC(x,y,z) (__G-
MP_MAX(__GMP_ABS(y->mp_size), 1+(__GMP_BITS_PER_ULONGLONG-1)/
GMP_NUMB_BITS)+1)
1127 #define mpz_sub_ui __gmpz_sub_ui
1128 __GMP_DECLSPEC void mpz_sub_ui __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, unsigned long int));
1129
1130 #define __GMP_MPZ_UI_SUB_MIN_ALLOC(x,y,z) (__G-
MP_MAX(__GMP_ABS(z->mp_size), 1+(__GMP_BITS_PER_ULONGLONG-1)/
GMP_NUMB_BITS)+1)
1131 #define mpz_ui_sub __gmpz_ui_sub
1132 __GMP_DECLSPEC void mpz_ui_sub __GMP_PROTO ((mpz_ptr, un-
signed long int, mpz_srcptr));
1133
1134 #define mpz_submul __gmpz_submul
1135 __GMP_DECLSPEC void mpz_submul __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
1136
1137 #define mpz_submul_ui __gmpz_submul_ui
1138 __GMP_DECLSPEC void mpz_submul_ui __GMP_PROTO ((mpz_ptr,
mpz_srcptr, unsigned long int));
1139
1140 #define mpz_swap __gmpz_swap
1141
1142 __GMP_DECLSPEC void mpz_swap __GMP_PROTO ((mpz_ptr, mpz_ptr))
__GMP_NOTHROW;
1143
1144 #define mpz_tdiv_ui __gmpz_tdiv_ui
1145 __GMP_DECLSPEC unsigned long int mpz_tdiv_ui __GMP_PROTO
((mpz_srcptr, unsigned long int)) __GMP_ATTRIBUTE_PURE;
1146
1147 #define mpz_tdiv_q __gmpz_tdiv_q
1148 __GMP_DECLSPEC void mpz_tdiv_q __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
1149
1150 #define mpz_tdiv_q_2exp __gmpz_tdiv_q_2exp
1151 __GMP_DECLSPEC void mpz_tdiv_q_2exp __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mp_bitcnt_t));
1152
1153 #define mpz_tdiv_q_ui __gmpz_tdiv_q_ui
1154 __GMP_DECLSPEC unsigned long int mpz_tdiv_q_ui __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
1155
1156 #define mpz_tdiv_qr __gmpz_tdiv_qr
1157 __GMP_DECLSPEC void mpz_tdiv_qr __GMP_PROTO ((mpz_ptr,
mpz_ptr, mpz_srcptr, mpz_srcptr));
1158
1159 #define mpz_tdiv_qr_ui __gmpz_tdiv_qr_ui
1160 __GMP_DECLSPEC unsigned long int mpz_tdiv_qr_ui __GMP_PROTO
((mpz_ptr, mpz_ptr, mpz_srcptr, unsigned long int));
1161
1162 #define mpz_tdiv_r __gmpz_tdiv_r
1163 __GMP_DECLSPEC void mpz_tdiv_r __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
1164
1165 #define mpz_tdiv_r_2exp __gmpz_tdiv_r_2exp
1166 __GMP_DECLSPEC void mpz_tdiv_r_2exp __GMP_PROTO ((mpz_ptr,
mpz_srcptr, mp_bitcnt_t));
1167
1168 #define mpz_tdiv_r_ui __gmpz_tdiv_r_ui
1169 __GMP_DECLSPEC unsigned long int mpz_tdiv_r_ui __GMP_PROTO
((mpz_ptr, mpz_srcptr, unsigned long int));
1170
1171 #define mpz_tstbit __gmpz_tstbit
1172 __GMP_DECLSPEC int mpz_tstbit __GMP_PROTO ((mpz_srcptr,
mp_bitcnt_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1173
1174 #define mpz_ui_pow_ui __gmpz_ui_pow_ui
1175 __GMP_DECLSPEC void mpz_ui_pow_ui __GMP_PROTO ((mpz_ptr, un-
signed long int, unsigned long int));
1176
1177 #define mpz_urandomb __gmpz_urandomb
1178 __GMP_DECLSPEC void mpz_urandomb __GMP_PROTO ((mpz_ptr,
gmp_randstate_t, mp_bitcnt_t));
1179
1180 #define mpz_urandomm __gmpz_urandomm
1181 __GMP_DECLSPEC void mpz_urandomm __GMP_PROTO ((mpz_ptr,
gmp_randstate_t, mpz_srcptr));

```

```

1181
1182 #define mpz_xor __gmpq_xor
1183 #define mpz_eor __gmpq_xor
1184 __GMP_DECLSPEC void mpz_xor __GMP_PROTO ((mpz_ptr, mpz_sr-
cptr, mpz_srcptr));
1185
1186
1187 /***** Rational (i.e. Q) routines. *****/
1188
1189 #define mpq_abs __gmpq_abs
1190 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mpq_abs)
1191 __GMP_DECLSPEC void mpq_abs __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr));
1192 #endif
1193
1194 #define mpq_add __gmpq_add
1195 __GMP_DECLSPEC void mpq_add __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr, mpq_srcptr));
1196
1197 #define mpq_canonicalize __gmpq_canonicalize
1198 __GMP_DECLSPEC void mpq_canonicalize __GMP_PROTO ((mpq_ptr));
1199
1200 #define mpq_clear __gmpq_clear
1201 __GMP_DECLSPEC void mpq_clear __GMP_PROTO ((mpq_ptr));
1202
1203 #define mpq_clears __gmpq_clears
1204 __GMP_DECLSPEC void mpq_clears __GMP_PROTO ((mpq_ptr, ...));
1205
1206 #define mpq_cmp __gmpq_cmp
1207 __GMP_DECLSPEC int mpq_cmp __GMP_PROTO ((mpq_srcptr, mpq_sr-
cptr)) __GMP_ATTRIBUTE_PURE;
1208
1209 #define mpq_cmp_si __gmpq_cmp_si
1210 __GMP_DECLSPEC int mpq_cmp_si __GMP_PROTO ((mpq_srcptr,
long, unsigned long)) __GMP_ATTRIBUTE_PURE;
1211
1212 #define mpq_cmp_ui __gmpq_cmp_ui
1213 __GMP_DECLSPEC int mpq_cmp_ui __GMP_PROTO ((mpq_srcptr, un-
signed long int, unsigned long int)) __GMP_ATTRIBUTE_PURE;
1214
1215 #define mpq_div __gmpq_div
1216 __GMP_DECLSPEC void mpq_div __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr, mpq_srcptr));
1217
1218 #define mpq_div_2exp __gmpq_div_2exp
1219 __GMP_DECLSPEC void mpq_div_2exp __GMP_PROTO ((mpq_ptr,
mpq_srcptr, mp_bitcnt_t));
1220
1221 #define mpq_equal __gmpq_equal
1222 __GMP_DECLSPEC int mpq_equal __GMP_PROTO ((mpq_srcptr, mpq_s-
rcptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1223
1224 #define mpq_get_num __gmpq_get_num
1225 __GMP_DECLSPEC void mpq_get_num __GMP_PROTO ((mpz_ptr, mpq_s-
rcptr));
1226
1227 #define mpq_get_den __gmpq_get_den
1228 __GMP_DECLSPEC void mpq_get_den __GMP_PROTO ((mpz_ptr, mpq_s-
rcptr));
1229
1230 #define mpq_get_d __gmpq_get_d
1231 __GMP_DECLSPEC double mpq_get_d __GMP_PROTO ((mpq_srcptr))
__GMP_ATTRIBUTE_PURE;
1232
1233 #define mpq_get_str __gmpq_get_str
1234 __GMP_DECLSPEC char *mpq_get_str __GMP_PROTO ((char *, int,
mpq_srcptr));
1235
1236 #define mpq_init __gmpq_init
1237 __GMP_DECLSPEC void mpq_init __GMP_PROTO ((mpq_ptr));
1238
1239 #define mpq_inits __gmpq_inits
1240 __GMP_DECLSPEC void mpq_inits __GMP_PROTO ((mpq_ptr, ...));
1241
1242 #define mpq_inp_str __gmpq_inp_str
1243 #ifndef __GMP_H_HAVE_FILE
1244 __GMP_DECLSPEC size_t mpq_inp_str __GMP_PROTO ((mpq_ptr, FILE
*, int));
1245 #endif
1246
1247 #define mpq_inv __gmpq_inv
1248 __GMP_DECLSPEC void mpq_inv __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr));
1249
1250 #define mpq_mul __gmpq_mul
1251 __GMP_DECLSPEC void mpq_mul __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr, mpq_srcptr));
1252
1253 #define mpq_mul_2exp __gmpq_mul_2exp
1254 __GMP_DECLSPEC void mpq_mul_2exp __GMP_PROTO ((mpq_ptr,
mpq_srcptr, mp_bitcnt_t));
1255
1256 #define mpq_neg __gmpq_neg
1257 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mpq_neg)
1258 __GMP_DECLSPEC void mpq_neg __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr));
1259 #endif
1260

```

```

1261 #define mpq_out_str __gmpq_out_str
1262 #ifndef __GMP_H_HAVE_FILE
1263 __GMP_DECLSPEC size_t mpq_out_str __GMP_PROTO ((FILE *, int,
mpq_srcptr));
1264 #endif
1265
1266 #define mpq_set __gmpq_set
1267 __GMP_DECLSPEC void mpq_set __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr));
1268
1269 #define mpq_set_d __gmpq_set_d
1270 __GMP_DECLSPEC void mpq_set_d __GMP_PROTO ((mpq_ptr,
double));
1271
1272 #define mpq_set_den __gmpq_set_den
1273 __GMP_DECLSPEC void mpq_set_den __GMP_PROTO ((mpq_ptr, mpz_s-
rcptr));
1274
1275 #define mpq_set_f __gmpq_set_f
1276 __GMP_DECLSPEC void mpq_set_f __GMP_PROTO ((mpq_ptr, mpf_sr-
cptr));
1277
1278 #define mpq_set_num __gmpq_set_num
1279 __GMP_DECLSPEC void mpq_set_num __GMP_PROTO ((mpq_ptr, mpz_s-
rcptr));
1280
1281 #define mpq_set_si __gmpq_set_si
1282 __GMP_DECLSPEC void mpq_set_si __GMP_PROTO ((mpq_ptr, signed
long int, unsigned long int));
1283
1284 #define mpq_set_str __gmpq_set_str
1285 __GMP_DECLSPEC int mpq_set_str __GMP_PROTO ((mpq_ptr, __g-
mp_const char *, int));
1286
1287 #define mpq_set_ui __gmpq_set_ui
1288 __GMP_DECLSPEC void mpq_set_ui __GMP_PROTO ((mpq_ptr, un-
signed long int, unsigned long int));
1289
1290 #define mpq_set_z __gmpq_set_z
1291 __GMP_DECLSPEC void mpq_set_z __GMP_PROTO ((mpq_ptr, mpz_sr-
cptr));
1292
1293 #define mpq_sub __gmpq_sub
1294 __GMP_DECLSPEC void mpq_sub __GMP_PROTO ((mpq_ptr, mpq_sr-
cptr, mpq_srcptr));
1295
1296 #define mpq_swap __gmpq_swap
1297 __GMP_DECLSPEC void mpq_swap __GMP_PROTO ((mpq_ptr, mpq_ptr))
__GMP_NOTHROW;
1298
1299
1300 /***** Float (i.e. F) routines. *****/
1301
1302 #define mpf_abs __gmpf_abs
1303 __GMP_DECLSPEC void mpf_abs __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1304
1305 #define mpf_add __gmpf_add
1306 __GMP_DECLSPEC void mpf_add __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, mpf_srcptr));
1307
1308 #define mpf_add_ui __gmpf_add_ui
1309 __GMP_DECLSPEC void mpf_add_ui __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, unsigned long int));
1310
1311 #define mpf_ceil __gmpf_ceil
1312 __GMP_DECLSPEC void mpf_ceil __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1313
1314 #define mpf_clear __gmpf_clear
1315 __GMP_DECLSPEC void mpf_clear __GMP_PROTO ((mpf_ptr));
1316
1317 #define mpf_clears __gmpf_clears
1318 __GMP_DECLSPEC void mpf_clears __GMP_PROTO ((mpf_ptr, ...));
1319
1320 #define mpf_cmp __gmpf_cmp
1321 __GMP_DECLSPEC int mpf_cmp __GMP_PROTO ((mpf_srcptr, mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1322
1323 #define mpf_cmp_d __gmpf_cmp_d
1324 __GMP_DECLSPEC int mpf_cmp_d __GMP_PROTO ((mpf_srcptr,
double)) __GMP_ATTRIBUTE_PURE;
1325
1326 #define mpf_cmp_si __gmpf_cmp_si
1327 __GMP_DECLSPEC int mpf_cmp_si __GMP_PROTO ((mpf_srcptr,
signed long int)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1328
1329 #define mpf_cmp_ui __gmpf_cmp_ui
1330 __GMP_DECLSPEC int mpf_cmp_ui __GMP_PROTO ((mpf_srcptr, un-
signed long int)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1331
1332 #define mpf_div __gmpf_div
1333 __GMP_DECLSPEC void mpf_div __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, mpf_srcptr));
1334
1335 #define mpf_div_2exp __gmpf_div_2exp
1336 __GMP_DECLSPEC void mpf_div_2exp __GMP_PROTO ((mpf_ptr,
mpf_srcptr, mp_bitcnt_t));
1337
1338 #define mpf_div_ui __gmpf_div_ui

```

```

1338 __GMP_DECLSPEC void mpf_div_ui __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, unsigned long int));
1339
1340 #define mpf_dump __gmpf_dump
1341 __GMP_DECLSPEC void mpf_dump __GMP_PROTO ((mpf_srcptr));
1342
1343 #define mpf_eq __gmpf_eq
1344 __GMP_DECLSPEC int mpf_eq __GMP_PROTO ((mpf_srcptr, mpf_sr-
cptr, unsigned long int)) __GMP_ATTRIBUTE_PURE;
1345
1346 #define mpf_fits_sint_p __gmpf_fits_sint_p
1347 __GMP_DECLSPEC int mpf_fits_sint_p __GMP_PROTO ((mpf_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1348
1349 #define mpf_fits_slong_p __gmpf_fits_slong_p
1350 __GMP_DECLSPEC int mpf_fits_slong_p __GMP_PROTO ((mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1351
1352 #define mpf_fits_ushort_p __gmpf_fits_ushort_p
1353 __GMP_DECLSPEC int mpf_fits_ushort_p __GMP_PROTO ((mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1354
1355 #define mpf_fits_uint_p __gmpf_fits_uint_p
1356 __GMP_DECLSPEC int mpf_fits_uint_p __GMP_PROTO ((mpf_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1357
1358 #define mpf_fits_ulong_p __gmpf_fits_ulong_p
1359 __GMP_DECLSPEC int mpf_fits_ulong_p __GMP_PROTO ((mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1360
1361 #define mpf_fits_ushort_p __gmpf_fits_ushort_p
1362 __GMP_DECLSPEC int mpf_fits_ushort_p __GMP_PROTO ((mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1363
1364 #define mpf_floor __gmpf_floor
1365 __GMP_DECLSPEC void mpf_floor __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1366
1367 #define mpf_get_d __gmpf_get_d
1368 __GMP_DECLSPEC double mpf_get_d __GMP_PROTO ((mpf_srcptr))
__GMP_ATTRIBUTE_PURE;
1369
1370 #define mpf_get_d_2exp __gmpf_get_d_2exp
1371 __GMP_DECLSPEC double mpf_get_d_2exp __GMP_PROTO ((signed
long int *, mpf_srcptr));
1372
1373 #define mpf_get_default_prec __gmpf_get_default_prec
1374 __GMP_DECLSPEC mp_bitcnt_t mpf_get_default_prec __GMP_PROTO
((void)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1375
1376 #define mpf_get_prec __gmpf_get_prec
1377 __GMP_DECLSPEC mp_bitcnt_t mpf_get_prec __GMP_PROTO ((mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1378
1379 #define mpf_get_si __gmpf_get_si
1380 __GMP_DECLSPEC long mpf_get_si __GMP_PROTO ((mpf_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1381
1382 #define mpf_get_str __gmpf_get_str
1383 __GMP_DECLSPEC char *mpf_get_str __GMP_PROTO ((char *,
mp_exp_t *, int, size_t, mpf_srcptr));
1384
1385 #define mpf_get_ui __gmpf_get_ui
1386 __GMP_DECLSPEC unsigned long mpf_get_ui __GMP_PROTO ((mpf_sr-
cptr)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1387
1388 #define mpf_init __gmpf_init
1389 __GMP_DECLSPEC void mpf_init __GMP_PROTO ((mpf_ptr));
1390
1391 #define mpf_init2 __gmpf_init2
1392 __GMP_DECLSPEC void mpf_init2 __GMP_PROTO ((mpf_ptr,
mp_bitcnt_t));
1393
1394 #define mpf_inits __gmpf_inits
1395 __GMP_DECLSPEC void mpf_inits __GMP_PROTO ((mpf_ptr, ...));
1396
1397 #define mpf_init_set __gmpf_init_set
1398 __GMP_DECLSPEC void mpf_init_set __GMP_PROTO ((mpf_ptr,
mpf_srcptr));
1399
1400 #define mpf_init_set_d __gmpf_init_set_d
1401 __GMP_DECLSPEC void mpf_init_set_d __GMP_PROTO ((mpf_ptr,
double));
1402
1403 #define mpf_init_set_si __gmpf_init_set_si
1404 __GMP_DECLSPEC void mpf_init_set_si __GMP_PROTO ((mpf_ptr,
signed long int));
1405
1406 #define mpf_init_set_str __gmpf_init_set_str
1407 __GMP_DECLSPEC int mpf_init_set_str __GMP_PROTO ((mpf_ptr,
__gmp_const char *, int));
1408
1409 #define mpf_init_set_ui __gmpf_init_set_ui
1410 __GMP_DECLSPEC void mpf_init_set_ui __GMP_PROTO ((mpf_ptr,
unsigned long int));
1411
1412 #define mpf_inp_str __gmpf_inp_str
1413 #ifndef __GMP_H_HAVE_FILE

```

```

1414 __GMP_DECLSPEC size_t mpf_inp_str __GMP_PROTO ((mpf_ptr, FILE
*, int));
1415 #endif
1416
1417 #define mpf_integer_p __gmpf_integer_p
1418 __GMP_DECLSPEC int mpf_integer_p __GMP_PROTO ((mpf_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1419
1420 #define mpf_mul __gmpf_mul
1421 __GMP_DECLSPEC void mpf_mul __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, mpf_srcptr));
1422
1423 #define mpf_mul_2exp __gmpf_mul_2exp
1424 __GMP_DECLSPEC void mpf_mul_2exp __GMP_PROTO ((mpf_ptr,
mpf_srcptr, mp_bitcnt_t));
1425
1426 #define mpf_mul_ui __gmpf_mul_ui
1427 __GMP_DECLSPEC void mpf_mul_ui __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, unsigned long int));
1428
1429 #define mpf_neg __gmpf_neg
1430 __GMP_DECLSPEC void mpf_neg __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1431
1432 #define mpf_out_str __gmpf_out_str
1433 #ifndef __GMP_H_HAVE_FILE
1434 __GMP_DECLSPEC size_t mpf_out_str __GMP_PROTO ((FILE *, int,
size_t, mpf_srcptr));
1435 #endif
1436
1437 #define mpf_pow_ui __gmpf_pow_ui
1438 __GMP_DECLSPEC void mpf_pow_ui __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, unsigned long int));
1439
1440 #define mpf_random2 __gmpf_random2
1441 __GMP_DECLSPEC void mpf_random2 __GMP_PROTO ((mpf_ptr, mp_s-
ize_t, mp_exp_t));
1442
1443 #define mpf_rrandomb __gmpf_rrandomb
1444 __GMP_DECLSPEC void mpf_rrandomb __GMP_PROTO ((mpf_ptr,
gmp_randstate_t, mp_size_t, mp_exp_t));
1445
1446 #define mpf_reldiff __gmpf_reldiff
1447 __GMP_DECLSPEC void mpf_reldiff __GMP_PROTO ((mpf_ptr, mpf_s-
rcptr, mpf_srcptr));
1448
1449 #define mpf_set __gmpf_set
1450 __GMP_DECLSPEC void mpf_set __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1451
1452 #define mpf_set_d __gmpf_set_d
1453 __GMP_DECLSPEC void mpf_set_d __GMP_PROTO ((mpf_ptr,
double));
1454
1455 #define mpf_set_default_prec __gmpf_set_default_prec
1456 __GMP_DECLSPEC void mpf_set_default_prec __GMP_PROTO
((mp_bitcnt_t)) __GMP_NOTHROW;
1457
1458 #define mpf_set_prec __gmpf_set_prec
1459 __GMP_DECLSPEC void mpf_set_prec __GMP_PROTO ((mpf_ptr,
mp_bitcnt_t));
1460
1461 #define mpf_set_prec_raw __gmpf_set_prec_raw
1462 __GMP_DECLSPEC void mpf_set_prec_raw __GMP_PROTO ((mpf_ptr,
mp_bitcnt_t)) __GMP_NOTHROW;
1463
1464 #define mpf_set_q __gmpf_set_q
1465 __GMP_DECLSPEC void mpf_set_q __GMP_PROTO ((mpf_ptr, mpq_sr-
cptr));
1466
1467 #define mpf_set_si __gmpf_set_si
1468 __GMP_DECLSPEC void mpf_set_si __GMP_PROTO ((mpf_ptr, signed
long int));
1469
1470 #define mpf_set_str __gmpf_set_str
1471 __GMP_DECLSPEC int mpf_set_str __GMP_PROTO ((mpf_ptr, __g-
mp_const char *, int));
1472
1473 #define mpf_set_ui __gmpf_set_ui
1474 __GMP_DECLSPEC void mpf_set_ui __GMP_PROTO ((mpf_ptr, un-
signed long int));
1475
1476 #define mpf_set_z __gmpf_set_z
1477 __GMP_DECLSPEC void mpf_set_z __GMP_PROTO ((mpf_ptr, mpz_sr-
cptr));
1478
1479 #define mpf_size __gmpf_size
1480 __GMP_DECLSPEC size_t mpf_size __GMP_PROTO ((mpf_srcptr))
__GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1481
1482 #define mpf_sqrt __gmpf_sqrt
1483 __GMP_DECLSPEC void mpf_sqrt __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1484
1485 #define mpf_sqrt_ui __gmpf_sqrt_ui
1486 __GMP_DECLSPEC void mpf_sqrt_ui __GMP_PROTO ((mpf_ptr, un-
signed long int));
1487
1488 #define mpf_sub __gmpf_sub

```



```

1489 __GMP_DECLSPEC void mpf_sub __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, mpf_srcptr));
1490
1491 #define mpf_sub_ui __gmpf_sub_ui
1492 __GMP_DECLSPEC void mpf_sub_ui __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr, unsigned long int));
1493
1494 #define mpf_swap __gmpf_swap
1495
1496 __GMP_DECLSPEC void mpf_swap __GMP_PROTO ((mpf_ptr, mpf_ptr))
__GMP_NOTHROW;
1497
1498 #define mpf_trunc __gmpf_trunc
1499 __GMP_DECLSPEC void mpf_trunc __GMP_PROTO ((mpf_ptr, mpf_sr-
cptr));
1500
1501 #define mpf_ui_div __gmpf_ui_div
1502 __GMP_DECLSPEC void mpf_ui_div __GMP_PROTO ((mpf_ptr, un-
signed long int, mpf_srcptr));
1503
1504 #define mpf_ui_sub __gmpf_ui_sub
1505 __GMP_DECLSPEC void mpf_ui_sub __GMP_PROTO ((mpf_ptr, un-
signed long int, mpf_srcptr));
1506
1507 #define mpf_urandomb __gmpf_urandomb
1508 __GMP_DECLSPEC void mpf_urandomb __GMP_PROTO ((mpf_t,
gmp_randstate_t, mp_bitcnt_t));
1509
1510 /****** Low level positive-integer (i.e. N) routines.
******/
1511
1512 /* This is ugly, but we need to make user calls reach the
prefixed function. */
1513
1514 #define mpn_add __MPN(add)
1515 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mpn_add)
1516 __GMP_DECLSPEC mp_limb_t mpn_add __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_size_t, mp_srcptr, mp_size_t));
1517 #endif
1518
1519 #define mpn_add_1 __MPN(add_1)
1520 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mpn_add_1)
1521 __GMP_DECLSPEC mp_limb_t mpn_add_1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t)) __GMP_NOTHROW;
1522 #endif
1523
1524 #define mpn_add_n __MPN(add_n)
1525 __GMP_DECLSPEC mp_limb_t mpn_add_n __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_srcptr, mp_size_t));
1526
1527 #define mpn_addmul_1 __MPN(addmul_1)
1528 __GMP_DECLSPEC mp_limb_t mpn_addmul_1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t));
1529
1530 #define mpn_bdivmod __MPN(bdivmod)
1531 __GMP_DECLSPEC mp_limb_t mpn_bdivmod __GMP_PROTO ((mp_ptr,
mp_ptr, mp_size_t, mp_srcptr, mp_size_t, unsigned long int));
1532
1533 #define mpn_divrem __MPN(divrem)
1534 __GMP_DECLSPEC mp_limb_t mpn_divrem __GMP_PROTO ((mp_ptr,
mp_size_t, mp_ptr, mp_size_t, mp_srcptr, mp_size_t));
1535
1536 #define mpn_mulmod_2expp1 __MPN(mulmod_2expp1)
1537 __GMP_DECLSPEC int mpn_mulmod_2expp1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_srcptr, int, unsigned long, mp_ptr));
1538
1539 #define mpn_mulmod_2expm1 __MPN(mulmod_2expm1)
1540 __GMP_DECLSPEC void mpn_mulmod_2expm1 __GMP_PROTO ((mp_ptr,
mp_ptr, mp_ptr, unsigned long, mp_ptr));
1541
1542 #define mpn_cmp __MPN(cmp)
1543 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mpn_cmp)
1544 __GMP_DECLSPEC int mpn_cmp __GMP_PROTO ((mp_srcptr, mp_sr-
cptr, mp_size_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1545 #endif
1546
1547 #define mpn_divexact_by3(dst, src, size) \
1548   mpn_divexact_by3c (dst, src, size, __GMP_CAST (mp_limb_t,
0))
1549
1550 #define mpn_divexact_by3c __MPN(divexact_by3c)
1551 __GMP_DECLSPEC void mpn_divexact_by3c __GMP_PROTO
((mp_ptr, mp_srcptr, mp_size_t, mp_limb_t));
1552
1553 #define mpn_divmod_1(qp, np, nsize, dlimb) \
1554   mpn_divrem_1 (qp, __GMP_CAST (mp_size_t, 0), np, nsize,
dlimb)
1555
1556 #define mpn_divrem_1 __MPN(divrem_1)
1557 __GMP_DECLSPEC mp_limb_t mpn_divrem_1 __GMP_PROTO ((mp_ptr,
mp_size_t, mp_srcptr, mp_size_t, mp_limb_t));
1558
1559 #define mpn_divrem_2 __MPN(divrem_2)
1560 __GMP_DECLSPEC mp_limb_t mpn_divrem_2 __GMP_PROTO ((mp_ptr,
mp_size_t, mp_ptr, mp_size_t, mp_srcptr));
1561
1562 #define mpn_invert __MPN(invert)
1563 __GMP_DECLSPEC void mpn_invert __GMP_PROTO ((mp_ptr xp, mp_s-

```

```

rcptr ap, mp_size_t n));
1564
1565 #define mpn_sb_divappr_q __MPN(sb_divappr_q)
1566 __GMP_DECLSPEC mp_limb_t mpn_sb_divappr_q __GMP_PROTO
((mp_ptr qp, mp_ptr np, mp_size_t nn,
1567   mp_srcptr dp, mp_size_t dn, mp_limb_t
dip));
1568
1569 #define mpn_dc_divappr_q_n __MPN(dc_divappr_q_n)
1570 __GMP_DECLSPEC mp_limb_t mpn_dc_divappr_q_n __GMP_PROTO
((mp_ptr qp, mp_ptr np, mp_srcptr dp, mp_size_t n,
1571   mp_limb_t dip, mp_ptr tp));
1572
1573 #define mpn_dc_bdiv_q_n __MPN(dc_bdiv_q_n)
1574 __GMP_DECLSPEC void mpn_dc_bdiv_q_n __GMP_PROTO ((mp_ptr qp,
mp_ptr wp, mp_ptr np, mp_srcptr dp, mp_size_t n,
1575   mp_limb_t dinv, mp_ptr scratch));
1576
1577 #define mpn_inv_divappr_q_n __MPN(inv_divappr_q_n)
1578 __GMP_DECLSPEC mp_limb_t mpn_inv_divappr_q_n __GMP_PROTO
((mp_ptr qp, mp_ptr np, mp_srcptr dp, mp_size_t n,
1579   mp_srcptr dip));
1580
1581 #define mpn_dc_divappr_q __MPN(dc_divappr_q)
1582 __GMP_DECLSPEC mp_limb_t mpn_dc_divappr_q __GMP_PROTO
((mp_ptr qp, mp_ptr np, mp_size_t nn, mp_srcptr dp, mp_size_t n,
1583   mp_limb_t dinv));
1584
1585 #define mpn_dc_div_q __MPN(dc_div_q)
1586 __GMP_DECLSPEC mp_limb_t mpn_dc_div_q __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1587   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1588
1589 #define mpn_inv_divappr_q __MPN(inv_divappr_q)
1590 __GMP_DECLSPEC mp_limb_t mpn_inv_divappr_q __GMP_PROTO
((mp_ptr qp, mp_ptr np, mp_size_t nn, mp_srcptr dp, mp_size_t n,
1591   mp_srcptr dinv));
1592
1593 #define mpn_inv_div_q __MPN(inv_div_q)
1594 __GMP_DECLSPEC mp_limb_t mpn_inv_div_q __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1595   mp_srcptr dp, mp_size_t dn, mp_srcptr dinv));
1596
1597 #define mpn_inv_div_qr __MPN(inv_div_qr)
1598 __GMP_DECLSPEC mp_limb_t mpn_inv_div_qr __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1599   mp_srcptr dp, mp_size_t dn, mp_srcptr dinv));
1600
1601 #define mpn_inv_div_qr_n __MPN(inv_div_qr_n)
1602 __GMP_DECLSPEC mp_limb_t mpn_inv_div_qr_n __GMP_PROTO
((mp_ptr qp, mp_ptr np,
1603   mp_srcptr dp, mp_size_t dn, mp_srcptr dinv));
1604
1605 #define mpn_dc_div_qr __MPN(dc_div_qr)
1606 __GMP_DECLSPEC mp_limb_t mpn_dc_div_qr __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1607   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1608
1609 #define mpn_dc_div_qr_n __MPN(dc_div_qr_n)
1610 __GMP_DECLSPEC mp_limb_t mpn_dc_div_qr_n __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_srcptr dp, mp_size_t n,
1611   mp_limb_t dinv, mp_ptr tp));
1612
1613 #define mpn_sb_div_q __MPN(sb_div_q)
1614 __GMP_DECLSPEC mp_limb_t mpn_sb_div_q __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1615   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1616
1617 #define mpn_sb_bdiv_q __MPN(sb_bdiv_q)
1618 __GMP_DECLSPEC void mpn_sb_bdiv_q __GMP_PROTO ((mp_ptr qp,
mp_ptr wp, mp_ptr np, mp_size_t nn,
1619   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1620
1621 #define mpn_dc_bdiv_q __MPN(dc_bdiv_q)
1622 __GMP_DECLSPEC void mpn_dc_bdiv_q __GMP_PROTO ((mp_ptr qp,
mp_ptr np, mp_size_t nn,
1623   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1624
1625 #define mpn_dc_bdiv_qr __MPN(dc_bdiv_qr)
1626 __GMP_DECLSPEC mp_limb_t mpn_dc_bdiv_qr __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1627   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1628
1629 #define mpn_dc_bdiv_qr_n __MPN(dc_bdiv_qr_n)
1630 __GMP_DECLSPEC mp_limb_t mpn_dc_bdiv_qr_n __GMP_PROTO
((mp_ptr qp, mp_ptr np,
1631   mp_srcptr dp, mp_size_t n, mp_limb_t dinv, mp_ptr
tp));
1632
1633 #define mpn_sb_div_qr __MPN(sb_div_qr)
1634 __GMP_DECLSPEC mp_limb_t mpn_sb_div_qr __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1635   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1636
1637 #define mpn_sb_bdiv_qr __MPN(sb_bdiv_qr)
1638 __GMP_DECLSPEC mp_limb_t mpn_sb_bdiv_qr __GMP_PROTO ((mp_ptr
qp, mp_ptr np, mp_size_t nn,
1639   mp_srcptr dp, mp_size_t dn, mp_limb_t dinv));
1640

```

```

1641 #define mpn_tdiv_q __MPN(tdiv_q)
1642 __GMP_DECLSPEC void mpn_tdiv_q __GMP_PROTO ((mp_ptr qp, mp_s-
rcptr np, mp_size_t nn,
1643                                     mp_srcptr dp, mp_size_t
dn));
1644
1645 #define mpn_divexact __MPN(divexact)
1646 __GMP_DECLSPEC void mpn_divexact __GMP_PROTO ((mp_ptr qp,
1647                                     mp_srcptr np, mp_size_t nn, mp_srcptr dp, mp_s-
ize_t dn));
1648
1649 #define mpn_redc_1 __MPN(redc_1)
1650 __GMP_DECLSPEC void mpn_redc_1 __GMP_PROTO ((mp_ptr, mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t));
1651
1652 #define mpn_gcd __MPN(gcd)
1653 __GMP_DECLSPEC mp_size_t mpn_gcd __GMP_PROTO ((mp_ptr,
mp_ptr, mp_size_t, mp_ptr, mp_size_t));
1654
1655 #define mpn_gcd_1 __MPN(gcd_1)
1656 __GMP_DECLSPEC mp_limb_t mpn_gcd_1 __GMP_PROTO ((mp_srcptr,
mp_size_t, mp_limb_t)) __GMP_ATTRIBUTE_PURE;
1657
1658 #define mpn_gcdext __MPN(gcdext)
1659 __GMP_DECLSPEC mp_size_t mpn_gcdext __GMP_PROTO ((mp_ptr,
mp_ptr, mp_size_t *, mp_ptr, mp_size_t, mp_ptr, mp_size_t));
1660
1661 #define mpn_get_str __MPN(get_str)
1662 __GMP_DECLSPEC size_t mpn_get_str __GMP_PROTO ((unsigned char
*, int, mp_ptr, mp_size_t));
1663
1664 #define mpn_hamdist __MPN(hamdist)
1665 __GMP_DECLSPEC mp_bitcnt_t mpn_hamdist __GMP_PROTO ((mp_sr-
cptr, mp_srcptr, mp_size_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1666
1667 #define mpn_lshift __MPN(lshift)
1668 __GMP_DECLSPEC mp_limb_t mpn_lshift __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, unsigned int));
1669
1670 #define mpn_mod_1 __MPN(mod_1)
1671 __GMP_DECLSPEC mp_limb_t mpn_mod_1 __GMP_PROTO ((mp_srcptr,
mp_size_t, mp_limb_t)) __GMP_ATTRIBUTE_PURE;
1672
1673 #define mpn_mul __MPN(mul)
1674 __GMP_DECLSPEC mp_limb_t mpn_mul __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_size_t, mp_srcptr, mp_size_t));
1675
1676 #define mpn_mul_1 __MPN(mul_1)
1677 __GMP_DECLSPEC mp_limb_t mpn_mul_1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t));
1678
1679 #define mpn_mul_n __MPN(mul_n)
1680 __GMP_DECLSPEC void mpn_mul_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1681
1682 #define mpn_sqr __MPN(sqr)
1683 __GMP_DECLSPEC void mpn_sqr __GMP_PROTO ((mp_ptr, mp_srcptr,
mp_size_t));
1684
1685 #define mpn_neg_n __MPN(neg_n)
1686 #define mpn_neg __MPN(neg_n)
1687 __GMP_DECLSPEC mp_limb_t mpn_neg_n __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t));
1688
1689 #define mpn_com_n __MPN(com_n)
1690 #define mpn_com __MPN(com_n)
1691 __GMP_DECLSPEC void mpn_com_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_size_t));
1692
1693 #define mpn_perfect_square_p __MPN(perfect_square_p)
1694 __GMP_DECLSPEC int mpn_perfect_square_p __GMP_PROTO ((mp_sr-
cptr, mp_size_t)) __GMP_ATTRIBUTE_PURE;
1695
1696 #define mpn_popcount __MPN(popcount)
1697 __GMP_DECLSPEC mp_bitcnt_t mpn_popcount __GMP_PROTO ((mp_sr-
cptr, mp_size_t)) __GMP_NOTHROW __GMP_ATTRIBUTE_PURE;
1698
1699 #define mpn_pow_1 __MPN(pow_1)
1700 __GMP_DECLSPEC mp_size_t mpn_pow_1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t, mp_ptr));
1701
1702 /* undocumented now, but retained here for upward compatibil-
ity */
1703 #define mpn_preinv_mod_1 __MPN(preinv_mod_1)
1704 __GMP_DECLSPEC mp_limb_t mpn_preinv_mod_1 __GMP_PROTO ((mp_s-
rcptr, mp_size_t, mp_limb_t, mp_limb_t)) __GMP_ATTRIBUTE_PURE;
1705
1706 #define mpn_random __MPN(random)
1707 __GMP_DECLSPEC void mpn_random __GMP_PROTO ((mp_ptr, mp_s-
ize_t));
1708
1709 #define mpn_random2 __MPN(random2)
1710 __GMP_DECLSPEC void mpn_random2 __GMP_PROTO ((mp_ptr, mp_s-
ize_t));
1711
1712 #define mpn_urandomb __MPN(urandomb)
1713 __GMP_DECLSPEC void mpn_urandomb __GMP_PROTO ((mp_ptr,
gmp_randstate_t, unsigned long));
1714

```

```

1715 #define mpn_urandomm __MPN(urandomm)
1716 __GMP_DECLSPEC void mpn_urandomm __GMP_PROTO ((mp_ptr,
gmp_randstate_t, mp_srcptr, mp_size_t));
1717
1718 #define mpn_randomb __MPN(randomb)
1719 __GMP_DECLSPEC void mpn_randomb __GMP_PROTO ((mp_ptr,
gmp_randstate_t, mp_size_t));
1720
1721 #define mpn_rrandom __MPN(rrandom)
1722 __GMP_DECLSPEC void mpn_rrandom __GMP_PROTO ((mp_ptr,
gmp_randstate_t, mp_size_t));
1723
1724 #define mpn_rshift __MPN(rshift)
1725 __GMP_DECLSPEC mp_limb_t mpn_rshift __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, unsigned int));
1726
1727 #define mpn_scan0 __MPN(scan0)
1728 __GMP_DECLSPEC mp_bitcnt_t mpn_scan0 __GMP_PROTO ((mp_srcptr,
mp_bitcnt_t)) __GMP_ATTRIBUTE_PURE;
1729
1730 #define mpn_scan1 __MPN(scan1)
1731 __GMP_DECLSPEC mp_bitcnt_t mpn_scan1 __GMP_PROTO ((mp_srcptr,
mp_bitcnt_t)) __GMP_ATTRIBUTE_PURE;
1732
1733 #define mpn_set_str __MPN(set_str)
1734 __GMP_DECLSPEC mp_size_t mpn_set_str __GMP_PROTO ((mp_ptr,
__gmp_const unsigned char *, size_t, int));
1735
1736 #define mpn_sqrtrem __MPN(sqrtrem)
1737 __GMP_DECLSPEC mp_size_t mpn_sqrtrem __GMP_PROTO ((mp_ptr,
mp_ptr, mp_srcptr, mp_size_t));
1738
1739 #define mpn_sub __MPN(sub)
1740 #if __GMP_INLINE_PROTOTYPES || defined (__GMP_FORCE_mpn_sub)
1741 __GMP_DECLSPEC mp_limb_t mpn_sub __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_size_t, mp_srcptr, mp_size_t));
1742 #endif
1743
1744 #define mpn_sub_1 __MPN(sub_1)
1745 #if __GMP_INLINE_PROTOTYPES || defined (__G-
MP_FORCE_mpn_sub_1)
1746 __GMP_DECLSPEC mp_limb_t mpn_sub_1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t)) __GMP_NOTHROW;
1747 #endif
1748
1749 #define mpn_sub_n __MPN(sub_n)
1750 __GMP_DECLSPEC mp_limb_t mpn_sub_n __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_srcptr, mp_size_t));
1751
1752 #define mpn_submul_1 __MPN(submul_1)
1753 __GMP_DECLSPEC mp_limb_t mpn_submul_1 __GMP_PROTO ((mp_ptr,
mp_srcptr, mp_size_t, mp_limb_t));
1754
1755 #define mpn_tdiv_qr __MPN(tdiv_qr)
1756 __GMP_DECLSPEC void mpn_tdiv_qr __GMP_PROTO ((mp_ptr, mp_ptr,
mp_size_t, mp_srcptr, mp_size_t, mp_srcptr, mp_size_t));
1757
1758 #define mpn_and_n __MPN(and_n)
1759 __GMP_DECLSPEC void mpn_and_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1760 #define mpn_andn_n __MPN(andn_n)
1761 __GMP_DECLSPEC void mpn_andn_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1762 #define mpn_nand_n __MPN(nand_n)
1763 __GMP_DECLSPEC void mpn_nand_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1764 #define mpn_iorn_n __MPN(iorn_n)
1765 __GMP_DECLSPEC void mpn_iorn_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1766 #define mpn_iorn_n __MPN(iorn_n)
1767 __GMP_DECLSPEC void mpn_iorn_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1768 #define mpn_nior_n __MPN(nior_n)
1769 __GMP_DECLSPEC void mpn_nior_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1770 #define mpn_xor_n __MPN(xor_n)
1771 __GMP_DECLSPEC void mpn_xor_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1772 #define mpn_xnor_n __MPN(xnor_n)
1773 __GMP_DECLSPEC void mpn_xnor_n __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_srcptr, mp_size_t));
1774
1775 #define mpn_copyi __MPN(copyi)
1776 __GMP_DECLSPEC void mpn_copyi __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_size_t));
1777 #define mpn_copyd __MPN(copyd)
1778 __GMP_DECLSPEC void mpn_copyd __GMP_PROTO ((mp_ptr, mp_sr-
cptr, mp_size_t));
1779 #define mpn_zero __MPN(zero)
1780 __GMP_DECLSPEC void mpn_zero __GMP_PROTO ((mp_ptr, mp_s-
ize_t));
1781
1782 /***** mpz inlines *****/
1783
1784 /* The following are provided as inlines where possible, but
always exist as
1785 library functions too, for binary compatibility.

```

```

1786
1787 Within gmp itself this inlining generally isn't relied on,
since it
1788 doesn't get done for all compilers, whereas if something
is worth
1789 inlining then it's worth arranging always.
1790
1791 There are two styles of inlining here. When the same bit
of code is
1792 wanted for the inline as for the library version, then
__GMP_FORCE_foo
1793 arranges for that code to be emitted and the __G-
MP_EXTERN_INLINE
1794 directive suppressed, eg. mpz_fits_uint_p. When a differ-
ent bit of code
1795 is wanted for the inline than for the library version,
then
1796 __GMP_FORCE_foo arranges the inline to be suppressed, eg.
mpz_abs. */
1797
1798 #if defined (__GMP_EXTERN_INLINE) && ! defined (__G-
MP_FORCE_mpz_abs)
1799 __GMP_EXTERN_INLINE void
1800 mpz_abs (mpz_ptr __gmp_w, mpz_srcptr __gmp_u)
1801 {
1802     if (__gmp_w != __gmp_u)
1803         mpz_set (__gmp_w, __gmp_u);
1804     __gmp_w->_mp_size = __GMP_ABS (__gmp_w->_mp_size);
1805 }
1806 #endif
1807
1808 #if GMP_NAIL_BITS == 0
1809 #define __GMPZ_FITS_UTYPE_P(z,maxval)
1810
1811     mp_size_t __gmp_n = z->_mp_size;
1812     \
1813
1814     mp_ptr __gmp_p = z->_mp_d;
1815     \
1816     return (__gmp_n == 0 || (__gmp_n == 1 && __gmp_p[0] <=
maxval));
1817 #else
1818
1819 #define __GMPZ_FITS_UTYPE_P(z,maxval)
1820
1821     mp_size_t __gmp_n = z->_mp_size;
1822     \
1823
1824     mp_ptr __gmp_p = z->_mp_d;
1825     \
1826     return (__gmp_n == 0 || (__gmp_n == 1 && __gmp_p[0] <=
maxval) \
1827         || (__gmp_n == 2 && __gmp_p[1] <= ((mp_limb_t) maxv-
al >> GMP_NUMB_BITS)));
1828 #endif
1829
1830 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_fits_uint_p)
1831 #if ! defined (__GMP_FORCE_mpz_fits_uint_p)
1832 __GMP_EXTERN_INLINE
1833 #endif
1834 int
1835 mpz_fits_uint_p (mpz_srcptr __gmp_z) __GMP_NOTHROW
1836 {
1837     __GMPZ_FITS_UTYPE_P (__gmp_z, __GMP_UINT_MAX);
1838 }
1839 #endif
1840
1841 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_fits_ulong_p)
1842 #if ! defined (__GMP_FORCE_mpz_fits_ulong_p)
1843 __GMP_EXTERN_INLINE
1844 #endif
1845 int
1846 mpz_fits_ulong_p (mpz_srcptr __gmp_z) __GMP_NOTHROW
1847 {
1848     __GMPZ_FITS_UTYPE_P (__gmp_z, __GMP_ULONG_MAX);
1849 }
1850 #endif
1851
1852 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_fits_ushort_p)
1853 #if ! defined (__GMP_FORCE_mpz_fits_ushort_p)
1854 __GMP_EXTERN_INLINE
1855 #endif
1856 int
1857 mpz_fits_ushort_p (mpz_srcptr __gmp_z) __GMP_NOTHROW
1858 {
1859     __GMPZ_FITS_UTYPE_P (__gmp_z, __GMP_USHRT_MAX);
1860 }
1861 #endif
1862
1863 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_get_ui)
1864 #if ! defined (__GMP_FORCE_mpz_get_ui)
1865 __GMP_EXTERN_INLINE

```

```

1857 #endif
1858 unsigned long
1859 mpz_get_ui (mpz_srcptr __gmp_z) __GMP_NOTHROW
1860 {
1861     mp_ptr __gmp_p = __gmp_z->_mp_d;
1862     mp_size_t __gmp_n = __gmp_z->_mp_size;
1863     mp_limb_t __gmp_l = __gmp_p[0];
1864     /* This is a "if" rather than a plain "if" so as to avoid
gcc warnings
1865
1866     about "<< GMP_NUMB_BITS" exceeding the type size, and to
avoid Borland
1867     C++ 6.0 warnings about condition always true for some-
thing like
1868     " __GMP_ULONG_MAX < GMP_NUMB_MASK". */
1869     #if GMP_NAIL_BITS == 0 || defined (__LONG_LONG_LIMB)
1870     /* limb==long and no nails, or limb==longlong, one limb is
enough */
1871     return (unsigned long)(__gmp_n != 0 ? __gmp_l : 0);
1872     #else
1873     /* limb==long and nails, need two limbs when available */
1874     __gmp_n = __GMP_ABS (__gmp_n);
1875     if (__gmp_n <= 1)
1876         return (__gmp_n != 0 ? __gmp_l : 0);
1877     else
1878         return __gmp_l + (__gmp_p[1] << GMP_NUMB_BITS);
1879     #endif
1880 #endif
1881
1882 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_getlimbn)
1883 #if ! defined (__GMP_FORCE_mpz_getlimbn)
1884 __GMP_EXTERN_INLINE
1885 #endif
1886 mp_limb_t
1887 mpz_getlimbn (mpz_srcptr __gmp_z, mp_size_t __gmp_n) __G-
MP_NOTHROW
1888 {
1889     mp_limb_t __gmp_result = 0;
1890     if (__GMP_LIKELY (__gmp_n >= 0 && __gmp_n < __GMP_ABS (__g-
mp_z->_mp_size)))
1891         __gmp_result = __gmp_z->_mp_d[__gmp_n];
1892     return __gmp_result;
1893 }
1894 #endif
1895
1896 #if defined (__GMP_EXTERN_INLINE) && ! defined (__G-
MP_FORCE_mpz_neg)
1897 __GMP_EXTERN_INLINE void
1898 mpz_neg (mpz_ptr __gmp_w, mpz_srcptr __gmp_u)
1899 {
1900     if (__gmp_w != __gmp_u)
1901         mpz_set (__gmp_w, __gmp_u);
1902     __gmp_w->_mp_size = - __gmp_w->_mp_size;
1903 }
1904 #endif
1905
1906 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_perfect_square_p)
1907 #if ! defined (__GMP_FORCE_mpz_perfect_square_p)
1908 __GMP_EXTERN_INLINE
1909 #endif
1910 int
1911 mpz_perfect_square_p (mpz_srcptr __gmp_a)
1912 {
1913     mp_size_t __gmp_asize;
1914     int __gmp_result;
1915
1916     __gmp_asize = __gmp_a->_mp_size;
1917     __gmp_result = (__gmp_asize >= 0); /* zero is a square,
negatives are not */
1918     if (__GMP_LIKELY (__gmp_asize > 0))
1919         __gmp_result = mpn_perfect_square_p (__gmp_a->_mp_d, __g-
mp_asize);
1920     return __gmp_result;
1921 }
1922 #endif
1923
1924 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_popcount)
1925 #if ! defined (__GMP_FORCE_mpz_popcount)
1926 __GMP_EXTERN_INLINE
1927 #endif
1928 mp_bitcnt_t
1929 mpz_popcount (mpz_srcptr __gmp_u) __GMP_NOTHROW
1930 {
1931     mp_size_t __gmp_usize;
1932     mp_bitcnt_t __gmp_result;
1933
1934     __gmp_usize = __gmp_u->_mp_size;
1935     __gmp_result = (__gmp_usize < 0 ? __GMP_ULONG_MAX : 0);
1936     if (__GMP_LIKELY (__gmp_usize > 0))
1937         __gmp_result = mpn_popcount (__gmp_u->_mp_d, __g-
mp_usize);
1938     return __gmp_result;
1939 }
1940 #endif
1941
1942 #if defined (__GMP_EXTERN_INLINE) || defined (__G-

```



```

MP_FORCE_mpz_set_q)
1943 #if ! defined (__GMP_FORCE_mpz_set_q)
1944 __GMP_EXTERN_INLINE
1945 #endif
1946 void
1947 mpz_set_q (mpz_ptr __gmp_w, mpq_srcptr __gmp_u)
1948 {
1949     mpz_tdiv_q (__gmp_w, mpq_numref (__gmp_u), mpq_denref (__g-
mp_u));
1950 }
1951 #endif
1952
1953 #if defined (__GMP_EXTERN_INLINE) || defined (__G-
MP_FORCE_mpz_size)
1954 #if ! defined (__GMP_FORCE_mpz_size)
1955 __GMP_EXTERN_INLINE
1956 #endif
1957 size_t
1958 mpz_size (mpz_srcptr __gmp_z) __GMP_NOTHROW
1959 {
1960     return __GMP_ABS (__gmp_z->_mp_size);
1961 }
1962 #endif
1963
1964 /***** mpq inlines *****/
1965
1966 #if defined (__GMP_EXTERN_INLINE) && ! defined (__G-
MP_FORCE_mpq_abs)
1967 __GMP_EXTERN_INLINE void
1968 mpq_abs (mpq_ptr __gmp_w, mpq_srcptr __gmp_u)
1969 {
1970     if (__gmp_w != __gmp_u)
1971         mpq_set (__gmp_w, __gmp_u);
1972     __gmp_w->_mp_num._mp_size = __GMP_ABS (__gmp_w-
>_mp_num._mp_size);
1973 }
1974 #endif
1975
1976 #if defined (__GMP_EXTERN_INLINE) && ! defined (__G-
MP_FORCE_mpq_neg)
1977 __GMP_EXTERN_INLINE void
1978 mpq_neg (mpq_ptr __gmp_w, mpq_srcptr __gmp_u)
1979 {
1980     if (__gmp_w != __gmp_u)
1981         mpq_set (__gmp_w, __gmp_u);
1982     __gmp_w->_mp_num._mp_size = - __gmp_w->_mp_num._mp_size;
1983 }
1984 #endif
1985
1986 /***** mpn inlines *****/
1987
1988 /* The comments with __GMPN_ADD_1 below apply here too.
1989
1990
1991
1992 The test for FUNCTION returning 0 should predict well. If
it's assumed
1993 {yp,ysize} will usually have a random number of bits then
the high limb
1994 won't be full and a carry out will occur a good deal less
than 50% of the
1995 time.
1996 ysize==0 isn't a documented feature, but is used intern-
ally in a few
1997 places.
1998 Producing cout last stops it using up a register during
the main part of
1999
2000 the calculation, though gcc (as of 3.0) on an "if (mpn_add
(...))"
2001 doesn't seem able to move the true and false legs of the
conditional up
2002 to the two places cout is generated. */
2003
2004
2005 #define __GMPN_AORS(cout, wp, xp, xsize, yp, ysize, FUNCTION,
TEST) \
2006 do \
2007 { \
2008     mp_size_t  _g- \
mp_i; \
2009     mp_limb_t  _g- \
mp_x; \
2010 \
2011     /* ASSERT ((ysize) >= 0); \
2012     /* ASSERT ((xsize) >= (ysize)); \
2013     /* ASSERT (MPN_SAME_OR_SEPARATE2_P (wp, xsize, xp, \
xsize)); \
2014     /* ASSERT (MPN_SAME_OR_SEPARATE2_P (wp, xsize, yp, ys- \
ize)); \

```

```

2015     __gmp_i = (ys- \
ize); \
2016     if (__gmp_i != \
0) \
2017 { \
2018         if (FUNCTION (wp, xp, yp, _g- \
mp_i)) \
2019 { \
2020         do \
2021 { \
2022             if (__gmp_i >= \
(xsize)) \
2023 { \
2024                 (cout) = \
1; \
2025                 goto _g- \
mp_done; \
2026             } \
2027             __gmp_x = (xp)[_g- \
mp_i]; \
2028         } \
2029         while \
(TEST); \
2030     } \
2031 } \
2032 if ((wp) != \
(xp)) \
2033     __GMPN_COPY_REST (wp, xp, xsize, _g- \
mp_i); \
2034     (cout) = \
0; \
2035     _g- \
mp_done: \
2036 ; \
2037 } while (0)
2038
2039 #define __GMPN_ADD(cout, wp, xp, xsize, yp, ys- \
ize) \
2040 __GMPN_AORS (cout, wp, xp, xsize, yp, ysize, \
mpn_add_n, \
2041 (((wp)[_gmp_i++] = (__gmp_x + 1) & \
GMP_NUMB_MASK) == 0)) \
2042 #define __GMPN_SUB(cout, wp, xp, xsize, yp, ys- \
ize) \
2043 __GMPN_AORS (cout, wp, xp, xsize, yp, ysize, \
mpn_sub_n, \
2044 (((wp)[_gmp_i++] = (__gmp_x - 1) & \
GMP_NUMB_MASK), __gmp_x == 0))
2045
2046 /* The use of __gmp_i indexing is designed to ensure a com- \
pile time src==dst \
2047 remains nice and clear to the compiler, so that __G- \
MPN_COPY_REST can \
2048 disappear, and the load/add/store gets a chance to become \
a \
2049 read-modify-write on CISC CPUs.
2050
2051 Alternatives:
2052
2053 Using a pair of pointers instead of indexing would be pos- \
sible, but gcc \
2054 isn't able to recognise compile-time src==dst in that \
case, even when the \
2055 pointers are incremented more or less together. Other \
compilers would \
2056 very likely have similar difficulty.
2057
2058 gcc could use "if (__builtin_constant_p(src==dst) && \
src==dst)" or \
2059 similar to detect a compile-time src==dst. This works \
nicely on gcc \
2060 2.95.x, it's not good on gcc 3.0 where __built- \
in_constant_p(p==p) seems \
2061
2062 to be always false, for a pointer p. But the current code \
form seems \
2063 good enough for src==dst anyway.
2064
2065 gcc on x86 as usual doesn't give particularly good flags \
handling for the \
2066 carry/borrow detection. It's tempting to want some multi \
instruction asm

```

```

2067     blocks to help it, and this was tried, but in truth
there's only a few
2068     instructions to save and any gain is all too easily lost
by register
2069     juggling setting up for the asm.  */
2070
2071 #if GMP_NAIL_BITS == 0

2072 #define __GMPN_AORS_1(cout, dst, src, n, v, OP, CB)
\

2073 do {
\

2074     mp_size_t __gmp_i;
\
2075     mp_limb_t __gmp_x, __g-
mp_r;
\
2076
2077     /* ASSERT ((n) >= 1); */
\
2078     /* ASSERT (MPN_SAME_OR_SEPARATE_P (dst, src, n)); */
\
2079
2080     __gmp_x = (src)[0];
\
2081     __gmp_r = __gmp_x OP
(v);
\
2082     (dst)[0] = __gmp_r;
\
2083     if (CB (__gmp_r, __gmp_x,
(v)))
\
2084     {
\
2085         (cout) = 1;
\
2086         for (__gmp_i = 1; __gmp_i <
(n);)
\
2087         {
\
2088             __gmp_x = (src)[__g-
mp_i];
\
2089             __gmp_r = __gmp_x OP
1;
\
2090             (dst)[__gmp_i] = __g-
mp_r;
\
2091             ++__gmp_i;
\
2092             if (!CB (__gmp_r, __gmp_x,
1))
\
2093             {
\
2094                 if ((src) != (dst))
\
2095                 __GMPN_COPY_REST (dst, src, n, __g-
mp_i);
\
2096                 (cout) = 0;
\
2097                 break;
\
2098             }
\
2099         }
\
2100     }
\
2101     else
\
2102     {
\
2103         if ((src) != (dst))
\
2104         __GMPN_COPY_REST (dst, src, n, 1);
\
2105         (cout) = 0;
\

```

```

2106     }
\
2107     } while (0)
2108 #endif
2109
2110 #if GMP_NAIL_BITS >= 1

2111 #define __GMPN_AORS_1(cout, dst, src, n, v, OP, CB)
\

2112 do {
\

2113     mp_size_t __gmp_i;
\
2114     mp_limb_t __gmp_x, __gmp_r;
\
2115
2116     /* ASSERT ((n) >= 1); */
\
2117     /* ASSERT (MPN_SAME_OR_SEPARATE_P (dst, src, n)); */
\
2118
2119     __gmp_x = (src)[0];
\
2120     __gmp_r = __gmp_x OP (v);
\
2121     (dst)[0] = __gmp_r & GMP_NUMB_MASK;
\
2122     if (__gmp_r >> GMP_NUMB_BITS != 0)
\
2123     {
\
2124         (cout) = 1;
\
2125         for (__gmp_i = 1; __gmp_i < (n);)
\
2126         {
\
2127             __gmp_x = (src)[__gmp_i];
\
2128             __gmp_r = __gmp_x OP 1;
\
2129             (dst)[__gmp_i] = __gmp_r & GMP_NUMB_MASK;
\
2130             ++__gmp_i;
\
2131             if (__gmp_r >> GMP_NUMB_BITS == 0)
\
2132             {
\
2133                 if ((src) != (dst))
\
2134                 __GMPN_COPY_REST (dst, src, n, __gmp_i);
\
2135                 (cout) = 0;
\
2136                 break;
\
2137             }
\
2138         }
\
2139     }
\
2140     else
\
2141     {
\

```

```

2142     if ((src) != (dst))
2143     {
2144         __GMPN_COPY_REST(dst, src, n, 1);
2145     }
2146     (cout) = 0;
2147 } while (0)
2148 #endif
2149 #define __GMPN_ADDCB(r,x,y) ((r) < (y))
2150 #define __GMPN_SUBCB(r,x,y) ((x) < (y))
2151
2152 #define __GMPN_ADD_1(cout, dst, src, n, v) \
2153     __GMPN_AORS_1(cout, dst, src, n, v, +, __GMPN_ADDCB) \
2154 #define __GMPN_SUB_1(cout, dst, src, n, v) \
2155     __GMPN_AORS_1(cout, dst, src, n, v, -, __GMPN_SUBCB)
2156
2157 /* Compare {xp,size} and {yp,size}, setting "result" to positive, zero or negative. size==0 is allowed. On random data usually only one limb will need to be examined to get a result, so it's worth having it inline. */
2158 #define __GMPN_CMP(result, xp, yp, size) \
2159 { \
2160     mp_size_t __g- \
2161     mp_i; \
2162     mp_limb_t __gmp_x, __g- \
2163     mp_y; \
2164     \
2165     /* ASSERT ((size) >= 0); \
2166     */ \
2167     \
2168     (result) = \
2169     0; \
2170     __gmp_i = \
2171     (size); \
2172     while (--__gmp_i >= \
2173     0) \
2174     { \
2175         __gmp_x = (xp)[__g- \
2176         mp_i]; \
2177         __gmp_y = (yp)[__g- \
2178         mp_i]; \
2179         if (__gmp_x != __g- \
2180         mp_y) \
2181         { \
2182             /* Cannot use __gmp_x - __gmp_y, may overflow an \
2183             "int" */ \
2184             (result) = (__gmp_x > __gmp_y ? 1 : \
2185             -1); \
2186             break; \
2187         } \
2188     } \
2189     } while (0)
2190
2191 #if defined (__GMPN_COPY) && ! defined (__GMPN_COPY_REST)
2192 #define __GMPN_COPY_REST(dst, src, size, \
2193 start) \
2194 { \
2195     do \
2196     { \
2197         /* ASSERT ((start) >= 0); \
2198         */ \
2199         /* ASSERT ((start) <= (size)); \
2200         */ \
2201         __GMPN_COPY ((dst)+(start), (src)+(start), (size)- \
2202         (start)); \
2203     } while (0) \
2204     #endif
2205
2206 /* Copy {src,size} to {dst,size}, starting at "start". This is designed to keep the indexing dst[j] and src[j] nice and simple for __GMPN_ADD_1, __GMPN_ADD, etc. */
2207 #if ! defined (__GMPN_COPY_REST)
2208 #define __GMPN_COPY_REST(dst, src, size, \
2209 start) \

```

```

2198     do \
2199     { \
2200         mp_size_t __g- \
2201         mp_j; \
2202         /* ASSERT ((size) >= 0); \
2203         */ \
2204         /* ASSERT ((start) >= 0); \
2205         */ \
2206         /* ASSERT ((start) <= (size)); \
2207         */ \
2208         /* ASSERT (MPN_SAME_OR_SEPARATE_P (dst, src, size)); \
2209         */ \
2210         for (__gmp_j = (start); __gmp_j < (size); __gmp_j+ \
2211         +) \
2212             (dst)[__gmp_j] = (src)[__g- \
2213             mp_j]; \
2214     } while (0) \
2215     #endif
2216
2217 /* Enhancement: Use some of the smarter code from gmp-impl.h. Maybe use mpn_copyi if there's a native version, and if we don't mind demanding binary compatibility for it (on targets which use it). */
2218 #if ! defined (__GMPN_COPY)
2219 #define __GMPN_COPY(dst, src, size) __GMPN_COPY_REST(dst, \
2220 src, size, 0)
2221 #endif
2222
2223 #if defined (__GMP_EXTERN_INLINE) || defined (__G- \
2224 MP_FORCE_mpn_add)
2225 #if ! defined (__GMP_FORCE_mpn_add)
2226 __GMP_EXTERN_INLINE
2227 #endif
2228 mp_limb_t \
2229 mpn_add (mp_ptr __gmp_wp, mp_srcptr __gmp_xp, mp_size_t __g- \
2230 mp_xsize, mp_srcptr __gmp_yp, mp_size_t __gmp_ysize) \
2231 { \
2232     mp_limb_t __gmp_c; \
2233     __GMPN_ADD (__gmp_c, __gmp_wp, __gmp_xp, __gmp_xsize, __g- \
2234     mp_yp, __gmp_ysize); \
2235     return __gmp_c; \
2236 } \
2237 #endif
2238
2239 #if defined (__GMP_EXTERN_INLINE) || defined (__G- \
2240 MP_FORCE_mpn_add_1)
2241 #if ! defined (__GMP_FORCE_mpn_add_1)
2242 __GMP_EXTERN_INLINE
2243 #endif
2244 mp_limb_t \
2245 mpn_add_1 (mp_ptr __gmp_dst, mp_srcptr __gmp_src, mp_size_t \
2246 __gmp_size, mp_limb_t __gmp_n) __GMP_NOTHROW \
2247 { \
2248     mp_limb_t __gmp_c; \
2249     __GMPN_ADD_1 (__gmp_c, __gmp_dst, __gmp_src, __gmp_size, \
2250     __gmp_n); \
2251     return __gmp_c; \
2252 } \
2253 #endif
2254
2255 #if defined (__GMP_EXTERN_INLINE) || defined (__G- \
2256 MP_FORCE_mpn_cmp)
2257 #if ! defined (__GMP_FORCE_mpn_cmp)
2258 __GMP_EXTERN_INLINE
2259 #endif
2260 int \
2261 mpn_cmp (mp_srcptr __gmp_xp, mp_srcptr __gmp_yp, mp_size_t \
2262 __gmp_size) __GMP_NOTHROW \
2263 { \
2264     int __gmp_result; \
2265     __GMPN_CMP (__gmp_result, __gmp_xp, __gmp_yp, __gmp_size); \
2266     return __gmp_result; \
2267 } \
2268 #endif
2269
2270 #if defined (__GMP_EXTERN_INLINE) || defined (__G- \
2271 MP_FORCE_mpn_sub)
2272 #if ! defined (__GMP_FORCE_mpn_sub)
2273 __GMP_EXTERN_INLINE
2274 #endif
2275 mp_limb_t \
2276 mpn_sub (mp_ptr __gmp_wp, mp_srcptr __gmp_xp, mp_size_t __g- \
2277 mp_xsize, mp_srcptr __gmp_yp, mp_size_t __gmp_ysize) \
2278 { \
2279     mp_limb_t __gmp_c; \
2280     __GMPN_SUB (__gmp_c, __gmp_wp, __gmp_xp, __gmp_xsize, __g- \
2281     mp_yp, __gmp_ysize); \
2282     return __gmp_c; \
2283 } \
2284 #endif
2285
2286 #if defined (__GMP_EXTERN_INLINE) || defined (__G- \
2287 MP_FORCE_mpn_sub_1)
2288 #if ! defined (__GMP_FORCE_mpn_sub_1)
2289 __GMP_EXTERN_INLINE
2290 #endif
2291 mp_limb_t \

```

```

2275 mpn_sub_1 (mp_ptr __gmp_dst, mp_srcptr __gmp_src, mp_size_t
__gmp_size, mp_limb_t __gmp_n) __GMP_NOTHROW
2276 {
2277     mp_limb_t __gmp_c;
2278     __GMPN_SUB_1 (__gmp_c, __gmp_dst, __gmp_src, __gmp_size,
__gmp_n);
2279     return __gmp_c;
2280 }
2281 #endif
2282
2283 #if defined (__cplusplus)
2284 }
2285 #endif
2286
2287 /* Allow faster testing for negative, zero, and positive. */
2288 #define mpz_sgn(Z) ((Z)->_mp_size < 0 ? -1 : (Z)->_mp_size >
0)
2290 #define mpf_sgn(F) ((F)->_mp_size < 0 ? -1 : (F)->_mp_size >
0)
2291 #define mpq_sgn(Q) ((Q)->_mp_num._mp_size < 0 ? -1 : (Q)-
>_mp_num._mp_size > 0)
2292
2293 /* When using GCC, optimize certain common comparisons. */
2294 #if defined (__GNUC__)
2295 #define mpz_cmp_ui(Z,UI) \
2296     (__builtin_constant_p (UI) && (UI) == 0
\
2297      ? mpz_sgn (Z) : _mpz_cmp_ui (Z,UI))
2298 #define mpz_cmp_si(Z,SI) \
2299     (__builtin_constant_p (SI) && (SI) == 0 ? mpz_sgn (Z)
\
2300      : _builtin_constant_p (SI) && (SI) > 0
\
2301      ? _mpz_cmp_ui (Z, __GMP_CAST (unsigned long int, SI))
\
2302      : _mpz_cmp_si (Z,SI))
2303 #define mpq_cmp_ui(Q,NUI,DUI) \
2304     (__builtin_constant_p (NUI) && (NUI) == 0
\
2305      ? mpq_sgn (Q) : _mpq_cmp_ui (Q,NUI,DUI))
2306 #define mpq_cmp_si(q,n,d) \
2307     (__builtin_constant_p ((n) >= 0) && (n) >= 0 \
2308      ? mpq_cmp_ui (q, __GMP_CAST (unsigned long, n), d) \
2309      : _mpq_cmp_si (q, n, d))
2310 #else
2311 #define mpz_cmp_ui(Z,UI) _mpz_cmp_ui (Z,UI)
2312 #define mpz_cmp_si(Z,UI) _mpz_cmp_si (Z,UI)
2313 #define mpq_cmp_ui(Q,NUI,DUI) _mpq_cmp_ui (Q,NUI,DUI)
2314 #define mpq_cmp_si(q,n,d) _mpq_cmp_si(q,n,d)
2315 #endif
2316
2317 /* Using "&" rather than "&&" means these can come out
branch-free. Every
2318
2319 mpz_t has at least one limb allocated, so fetching the low
limb is always
2320 allowed. */
2321 #define mpz_odd_p(z) (((z)->_mp_size != 0) & __GMP_CAST
(int, (z)->_mp_d[0]))
2322 #define mpz_even_p(z) (! mpz_odd_p (z))
2323
2324
2325 /****** C++ routines *****/
2326
2327 #ifdef __cplusplus
2328 __GMP_DECLSPEC_XX std::ostream& operator<< (std::ostream &,
mpz_srcptr);
2329 __GMP_DECLSPEC_XX std::ostream& operator<< (std::ostream &,
mpq_srcptr);
2330 __GMP_DECLSPEC_XX std::ostream& operator<< (std::ostream &,
mpf_srcptr);
2331 __GMP_DECLSPEC_XX std::istream& operator>> (std::istream &,
mpz_ptr);
2332 __GMP_DECLSPEC_XX std::istream& operator>> (std::istream &,
mpq_ptr);
2333 __GMP_DECLSPEC_XX std::istream& operator>> (std::istream &,
mpf_ptr);
2334 #endif
2335
2336 /* Source-level compatibility with GMP 1. */
2337 #define mpz_mdiv mpz_fdiv_q
2338 #define mpz_mdivmod mpz_fdiv_qr
2339 #define mpz_mmod mpz_fdiv_r
2340 #define mpz_mdiv_ui mpz_fdiv_q_ui
2341 #define mpz_mdivmod_ui(q,r,n,d) \
2342     (((r) == 0) ? mpz_fdiv_q_ui (q,n,d) : mpz_fdiv_qr_ui
(q,r,n,d))
2343 #define mpz_mmod_ui(r,n,d) \
2344     (((r) == 0) ? mpz_fdiv_ui (n,d) : mpz_fdiv_r_ui (r,n,d))
2345
2346 #define gmp_randinit(x,y,z) gmp_randinit_lc_2exp_size(x,z)
2347

```

```

2348 typedef __mpz_struct MP_INT; /* gmp 1 source compatibility
*/
2349 typedef __mpq_struct MP_RAT; /* gmp 1 source compatibility
*/
2350
2351 #define mpz_div mpz_fdiv_q
2352 #define mpz_divmod mpz_fdiv_qr
2353 #define mpz_div_ui mpz_fdiv_q_ui
2354 #define mpz_divmod_ui mpz_fdiv_qr_ui
2355 #define mpz_div_2exp mpz_fdiv_q_2exp
2356 #define mpz_mod_2exp mpz_fdiv_r_2exp
2357
2358 enum
2359 {
2360     GMP_ERROR_NONE = 0,
2361     GMP_ERROR_UNSUPPORTED_ARGUMENT = 1,
2362     GMP_ERROR_DIVISION_BY_ZERO = 2,
2363     GMP_ERROR_SQRT_OF_NEGATIVE = 4,
2364     GMP_ERROR_INVALID_ARGUMENT = 8
2365 };
2366
2367 /* Major version number is the value of __GNU_MP__ too, above
and in mp.h. */
2368 #define __GNU_MP_VERSION 5
2369 #define __GNU_MP_VERSION_MINOR 0
2370 #define __GNU_MP_VERSION_PATCHLEVEL 1
2371 #define GMP_VERSION "5.0.1"
2372
2373 #define __MPIR_VERSION 2
2374 #define __MPIR_VERSION_MINOR 2
2375 #define __MPIR_VERSION_PATCHLEVEL 1
2376 #if defined( _MSC_VER )
2377 #define __MSC_MPIR_VERSION "2.2.1"
2378 #endif
2379
2380 /* These are for programs like MPFR to use the same CC and
CFLAGS as MPIR */
2381
2382 #if ! defined (__GMP_WITHIN_CONFIGURE)
2383 #define __GMP_CC "gcc"
2384 #define __GMP_CFLAGS "-O3"
2385 #define __MPIR_CC "gcc -std=gnu99"
2386 #define __MPIR_CFLAGS "-O3"
2387 #endif
2388
2389 #define __GMP_H__
2390 #endif /* __GMP_H__ */
2391

```

E.41 vendor/hls_compat.h (9 lines, 372 B)

```

1 /* Vitis HLS 내부 매크로 보충.
2  * 공개 저장소 헤더에는 없고 Vitis HLS 설치본의 전처리기 정의로만 들어온
다.
3  * 의미는 "이 파라미터는 합성에서 안 쓰인다"이므로 (void) 캐스트하면 충분
하다. */
4 #ifndef HLS_COMPAT_H_
5 #define HLS_COMPAT_H_
6 #ifndef _AP_UNUSED_PARAM
7 #define _AP_UNUSED_PARAM(x) ((void)(x))
8 #endif
9 #endif

```

E.42 vendor/hls_fpo.h (619 lines, 18,568 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6
7 #ifndef __AESL_FPO_H_
8 #define __AESL_FPO_H_
9
10 #ifndef __SYNTHESIS__
11 #include <math.h>
12 #endif
13
14 #include <assert.h>
15
16 #if defined __arm__ && !(defined HLS_NO_XIL_FPO_LIB)
17 #warning "Xilinx Floating Point Operator IP core does not
provide simulation models for ARM architecture. Automatically de-
fining HLS_NO_XIL_FPO_LIB in order to avoid this library depend-
ency, although bit-accurate simulation of some functions is no
longer possible. You can make this warning go away by adding this
define yourself before including any other files."
18 #define HLS_NO_XIL_FPO_LIB
19 #endif
20
21 #ifdef __SYNTHESIS__
22 #define HLS_FPO_SQRTF(a) __builtin_sqrtf(a)
23 #define HLS_FPO_SQRT(a) __builtin_sqrt(a)
24 #define HLS_FPO_RECSQRTF(a) __builtin_rsqrtf(a)
25 #define HLS_FPO_RECSQRT(a) __builtin_rsqrt(a)

```

```

26 #define HLS_FPO_ABSF(a)      __builtin_fabsf(a)
27 #define HLS_FPO_ABS(a)      __builtin_fabs(a)
28 #define HLS_FPO_LOGF(a)     __builtin_logf(a)
29 #define HLS_FPO_LOG(a)      __builtin_log(a)
30 #define HLS_FPO_EXPF(a)     __builtin_expf(a)
31 #define HLS_FPO_EXP(a)      __builtin_exp(a)
32 #define HLS_FPO_RECIPF(a)   __builtin_recipf(a)
33 #define HLS_FPO_RECIP(a)    __builtin_recip(a)
34 #else// csim
35 #ifndef HLS_NO_XIL_FPO_LIB
36 #define HLS_FPO_SQRTF(a)    sqrtf(a)
37 #define HLS_FPO_SQRT(a)    sqrt(a)
38 #define HLS_FPO_RECSQRTF(a) (1.0f/sqrtf(a))
39 #define HLS_FPO_RECSQRT(a) (1.0/sqrt(a))
40 #define HLS_FPO_ABSF(a)    fabsf(a)
41 #define HLS_FPO_ABS(a)    fabs(a)
42 #define HLS_FPO_LOGF(a)    logf(a)
43 #define HLS_FPO_LOG(a)    log(a)
44 #define HLS_FPO_EXPF(a)    expf(a)
45 #define HLS_FPO_EXP(a)    exp(a)
46 #define HLS_FPO_RECIPF(a)  (1.0f/a)
47 #define HLS_FPO_RECIP(a)  (1.0/a)
48 #else
49 #define HLS_FPO_SQRTF(a)    xil_fpo_sqrt_flt(a)
50 #define HLS_FPO_SQRT(a)    xil_fpo_sqrt_d(a)
51 #define HLS_FPO_RECSQRTF(a) xil_fpo_rec-
52                               sqrt_flt(a)
53 #define HLS_FPO_RECSQRT(a)  xil_fpo_recsqrt_d(a)
54 #define HLS_FPO_ABSF(a)    xil_fpo_abs_flt(a)
55 #define HLS_FPO_ABS(a)    xil_fpo_abs_d(a)
56 #define HLS_FPO_LOGF(a)    xil_fpo_log_flt(a)
57 #define HLS_FPO_LOG(a)    xil_fpo_log_d(a)
58 #define HLS_FPO_EXPF(a)    xil_fpo_exp_flt(a)
59 #define HLS_FPO_EXP(a)    xil_fpo_exp_d(a)
60 #define HLS_FPO_RECIPF(a)  xil_fpo_rec_flt(a)
61 #define HLS_FPO_RECIP(a)  xil_fpo_rec_d(a)
62 #endif //HLS_NO_XIL_FPO_LIB
63 #endif //__SYNTHESIS__
64 #if (defined __SYNTHESIS__ || defined HLS_NO_XIL_FPO_LIB)
65 #define HLS_FPO_ADDF(a,b)    ((a) + (b))
66 #define HLS_FPO_ADD(a,b)    ((a) + (b))
67 #define HLS_FPO_SUBF(a,b)    ((a) - (b))
68 #define HLS_FPO_SUB(a,b)    ((a) - (b))
69 #define HLS_FPO_MULF(a,b)    ((a) * (b))
70 #define HLS_FPO_MUL(a,b)    ((a) * (b))
71 #define HLS_FPO_DIVF(a,b)    ((a)/(b))
72 #define HLS_FPO_DIV(a,b)    ((a)/(b))
73 #define HLS_FPO_RECIPF(a)    HLS_FPO_RECIPF(a)
74 #define HLS_FPO_REC(a)      HLS_FPO_RECIP(a)
75 #define HLS_FPO_RSQRTF(a)    HLS_FPO_RECSQRTF(a)
76 #define HLS_FPO_RSQRT(a)    HLS_FPO_RECSQRT(a)
77 // #define HLS_FPO_UNORDEREDF(a,b)
78 // #define HLS_FPO_UNORDERED(a,b)
79 #define HLS_FPO_EQUALF(a,b)  ((a) == (b))
80 #define HLS_FPO_EQUAL(a,b)  ((a) == (b))
81 #define HLS_FPO_LESSF(a,b)  ((a) < (b))
82 #define HLS_FPO_LESS(a,b)  ((a) < (b))
83 #define HLS_FPO_LESSEQUALF(a,b) ((a) <= (b))
84 #define HLS_FPO_LESSEQUAL(a,b) ((a) <= (b))
85 #define HLS_FPO_GREATERF(a,b) ((a) > (b))
86 #define HLS_FPO_GREATER(a,b) ((a) > (b))
87 #define HLS_FPO_GREATEREQUALF(a,b) ((a) >= (b))
88 #define HLS_FPO_GREATEREQUAL(a,b) ((a) >= (b))
89 #define HLS_FPO_NOTEQUALF(a,b) ((a) != (b))
90 #define HLS_FPO_NOTEQUAL(a,b) ((a) != (b))
91 // #define HLS_FPO_CONDCODEF(a,b)
92 // #define HLS_FPO_CONDCODE(a,b)
93 #define HLS_FPO_FTOI(a)      ((int)(a))
94 #define HLS_FPO_DTOI(a)      ((int)(a))
95 #define HLS_FPO_ITOF(a)      ((float)(a))
96 #define HLS_FPO_ITOD(a)      ((double)(a))
97 #define HLS_FPO_FTOF(a)      ((float)(a))
98 #define HLS_FPO_DTOF(a)      ((float)(a))
99 #define HLS_FPO_FTOD(a)      ((double)(a))
100 #define HLS_FPO_DTOD(a)      ((double)(a))
101 #else
102 #define HLS_FPO_ADDF(a,b)    xil_fpo_add_flt(a,b)
103 #define HLS_FPO_ADD(a,b)    xil_fpo_add_d(a,b)
104 #define HLS_FPO_SUBF(a,b)    xil_fpo_sub_flt(a,b)
105 #define HLS_FPO_SUB(a,b)    xil_fpo_sub_d(a,b)
106 #define HLS_FPO_MULF(a,b)    xil_fpo_mul_flt(a,b)
107 #define HLS_FPO_MUL(a,b)    xil_fpo_mul_d(a,b)
108 #define HLS_FPO_DIVF(a,b)    xil_fpo_div_flt(a,b)
109 #define HLS_FPO_DIV(a,b)    xil_fpo_div_d(a,b)
110 #define HLS_FPO_RECIPF(a)    HLS_FPO_RECIPF(a)
111 #define HLS_FPO_REC(a)      HLS_FPO_RECIP(a)
112 #define HLS_FPO_RSQRTF(a)    HLS_FPO_RECSQRTF(a)
113 #define HLS_FPO_RSQRT(a)    HLS_FPO_RECSQRT(a)
114 #define HLS_FPO_UNORDEREDF(a,b) xil_fpo_un-
115                               ordered_flt(a,b)
116 #define HLS_FPO_UNORDERED(a,b) xil_fpo_un-
117                               ordered_d(a,b)
118 #define HLS_FPO_EQUALF(a,b)  xil_fpo_equal_flt(a,b)
119 #define HLS_FPO_EQUAL(a,b)  xil_fpo_equal_d(a,b)
120 #define HLS_FPO_LESSF(a,b)  xil_fpo_less_flt(a,b)
121 #define HLS_FPO_LESS(a,b)  xil_fpo_less_d(a,b)
122 #define HLS_FPO_LESSEQUALF(a,b) xil_fpo_lessequal_flt(a,b)
123 #define HLS_FPO_LESSEQUAL(a,b) xil_fpo_lessequal_d(a,b)
124 #define HLS_FPO_GREATERF(a,b) xil_fpo_greater_flt(a,b)
125 #define HLS_FPO_GREATER(a,b) xil_fpo_greater_d(a,b)
126 #define HLS_FPO_GREATEREQUALF(a,b) xil_fpo_greate-
127                               requal_flt(a,b)
128 #define HLS_FPO_GREATEREQUAL(a,b) xil_fpo_greate-
129                               requal_d(a,b)
130 #define HLS_FPO_NOTEQUALF(a,b) xil_fpo_notequal_flt(a,b)
131 #define HLS_FPO_NOTEQUAL(a,b) xil_fpo_notequal_d(a,b)
132 #define HLS_FPO_FTOI(a)      xil_fpo_ftoi_flt(a)
133 #define HLS_FPO_DTOI(a)      xil_fpo_ftoi_d(a)
134 #define HLS_FPO_ITOF(a)      xil_fpo_itof_flt(a)
135 #define HLS_FPO_ITOD(a)      xil_fpo_itof_d(a)
136 #define HLS_FPO_FTOF(a)      xil_fpo_ftof_flt(a)
137 #define HLS_FPO_FTOD(a)      xil_fpo_ftof_d(a)
138 #define HLS_FPO_DTOD(a)      xil_fpo_dtod_flt(a)
139 #define HLS_FPO_DTOD(a)      xil_fpo_dtod_d(a)
140 #include "floating_point_v7_1_bitacc_cmodel.h" // Must in-
141 clude before GMP and MPFR
142 #include "gmp.h"
143 #include "mpfr.h"
144 // Operation functions: add
145 // Operation functions: subtract
146 // Operation functions: multiply
147 // Operation functions: divide
148 inline float xil_fpo_add_flt(float a, float b)
149 {
150     float res_flt = 0.0f;
151     // xil_fpo_add_flt
152     xil_fpo_add_flt(&res_flt, a, b); // normal operation
153     return res_flt;
154 }
155 inline double xil_fpo_add_d(double a, double b)
156 {
157     double res_d = 0.0;
158     // xil_fpo_add_d
159     xil_fpo_add_d(&res_d, a, b); // normal operation
160     return res_d;
161 }
162 // Operation functions: subtract
163 // Operation functions: multiply
164 // Operation functions: divide
165 inline float xil_fpo_sub_flt(float a, float b)
166 {
167     float res_flt = 0.0f;
168     // xil_fpo_sub_flt
169     xil_fpo_sub_flt(&res_flt, a, b); // normal operation
170     return res_flt;
171 }
172 inline double xil_fpo_sub_d(double a, double b)
173 {
174     double res_d = 0.0;
175     // xil_fpo_sub_d
176     xil_fpo_sub_d(&res_d, a, b); // normal operation
177     return res_d;
178 }
179 // Operation functions: multiply
180 // Operation functions: divide
181 inline float xil_fpo_mul_flt(float a, float b)
182 {
183     float res_flt = 0.0f;
184     // xil_fpo_mul_flt
185     xil_fpo_mul_flt(&res_flt, a, b); // normal operation
186     return res_flt;
187 }
188 inline double xil_fpo_mul_d(double a, double b)
189 {
190     double res_d = 0.0;
191     // xil_fpo_mul_d
192     xil_fpo_mul_d(&res_d, a, b); // normal operation
193     return res_d;
194 }
195 // Operation functions: divide
196 // Operation functions: divide
197 inline float xil_fpo_div_flt(float a, float b)
198 {
199     float res_flt = 0.0f;
200     // xil_fpo_div_flt
201     xil_fpo_div_flt(&res_flt, a, b); // normal operation
202     return res_flt;
203 }
204 inline double xil_fpo_div_d(double a, double b)
205 {
206     double res_d = 0.0;
207     // xil_fpo_div_d
208     xil_fpo_div_d(&res_d, a, b); // normal operation
209     return res_d;
210 }

```

```

197 xip_fpo_mul_flt(&res_flt, a, b); // normal operation
198 return res_flt;
199 }
200
201 inline double xil_fpo_mul_d(double a, double b)
202 {
203     double res_d = 0.0;
204
205     // xip_fpo_mul_d
206     xip_fpo_mul_d(&res_d, a, b); // normal operation
207     return res_d;
208 }
209
210 ///////////////////////////////////////////////////////////////////
211 // Operation functions: divide
212 ///////////////////////////////////////////////////////////////////
213
214 inline float xil_fpo_div_flt(float a, float b)
215 {
216     float res_flt = 0.0f;
217
218     // xip_fpo_div_flt
219     xip_fpo_div_flt(&res_flt, a, b); // normal operation
220     return res_flt;
221 }
222
223 inline double xil_fpo_div_d(double a, double b)
224 {
225     double res_d = 0.0;
226
227     // xip_fpo_div_d
228     xip_fpo_div_d(&res_d, a, b); // normal operation
229     return res_d;
230 }
231
232 ///////////////////////////////////////////////////////////////////
233 // Operation functions: reciprocal
234 ///////////////////////////////////////////////////////////////////
235
236 inline float xil_fpo_rec_flt(float a)
237 {
238     float res_flt = 0.0f;
239
240     // xip_fpo_rec_flt
241     xip_fpo_rec_flt(&res_flt, a); // normal operation
242     return res_flt;
243 }
244
245 inline double xil_fpo_rec_d(double a)
246 {
247     double res_d = 0.0;
248
249     // xip_fpo_rec_d
250     xip_fpo_rec_d(&res_d, a); // normal operation
251     return res_d;
252 }
253
254 ///////////////////////////////////////////////////////////////////
255 // Operation functions: square root
256 ///////////////////////////////////////////////////////////////////
257
258 inline float xil_fpo_sqrt_flt(float a)
259 {
260     // printf("Testing operation functions: square root\n");
261     float res_flt = 0.0f;
262
263     // xip_fpo_sqrt_flt
264     xip_fpo_sqrt_flt(&res_flt, a); // normal operation
265     // printf("float = sqrt(a), and got res_flt=%f\n",
res_flt);
266     return res_flt;
267 }
268
269 inline double xil_fpo_sqrt_d(double a)
270 {
271     double res_d = 0.0;
272
273     // xip_fpo_sqrt_d
274     xip_fpo_sqrt_d(&res_d, a); // normal operation
275     // printf("double = sqrt(a), and got res_d=%f\n", res_d);
276     return res_d;
277 }
278
279 ///////////////////////////////////////////////////////////////////
280 // Operation functions: reciprocal square root
281 ///////////////////////////////////////////////////////////////////
282
283 inline float xil_fpo_recsqrt_flt(float a)
284 {
285     float res_flt = 0.0f;
286
287     // xip_fpo_recsqrt_flt

```

```

288 xip_fpo_recsqrt_flt(&res_flt, a); // normal operation
289 return res_flt;
290 }
291
292 inline double xil_fpo_recsqrt_d(double a)
293 {
294     double res_d = 0.0;
295
296     // xip_fpo_recsqrt_d
297     xip_fpo_recsqrt_d(&res_d, a); // normal operation
298     return res_d;
299 }
300
301 ///////////////////////////////////////////////////////////////////
302 // Operation functions: absolute value
303 ///////////////////////////////////////////////////////////////////
304
305 inline float xil_fpo_abs_flt(float a)
306 {
307     float res_flt = 0.0f;
308
309     xip_fpo_abs_flt(&res_flt, a);
310     return res_flt;
311 }
312
313 inline double xil_fpo_abs_d(double a)
314 {
315     double res_d = 0.0;
316
317     xip_fpo_abs_d(&res_d, a);
318     return res_d;
319 }
320
321 ///////////////////////////////////////////////////////////////////
322 // Operation functions: logarithm
323 ///////////////////////////////////////////////////////////////////
324
325 inline float xil_fpo_log_flt(float a)
326 {
327     float res_flt = 0.0f;
328
329     // xip_fpo_log_flt
330     xip_fpo_log_flt(&res_flt, a); // normal operation
331     return res_flt;
332 }
333
334 inline double xil_fpo_log_d(double a)
335 {
336     double res_d = 0.0;
337
338     // xip_fpo_log_d
339     xip_fpo_log_d(&res_d, a); // normal operation
340     return res_d;
341 }
342
343 ///////////////////////////////////////////////////////////////////
344 // Operation functions: Exponential
345 ///////////////////////////////////////////////////////////////////
346
347 inline float xil_fpo_exp_flt(float a)
348 {
349     float res_flt = 0.0f;
350
351     // xip_fpo_exp_flt
352     xip_fpo_exp_flt(&res_flt, a); // normal operation
353     return res_flt;
354 }
355
356 inline double xil_fpo_exp_d(double a)
357 {
358     double res_d = 0.0;
359
360     // xip_fpo_exp_d
361     xip_fpo_exp_d(&res_d, a); // normal operation
362     return res_d;
363 }
364
365 ///////////////////////////////////////////////////////////////////
366 // Operation functions: compare unordered
367 ///////////////////////////////////////////////////////////////////
368
369 inline int xil_fpo_unordered_flt(float a, float b)
370 {
371     int res_int = 0;
372
373     // xip_fpo_unordered_flt
374     xip_fpo_unordered_flt(&res_int, a, b); // normal operation
375     return res_int;
376 }
377
378 inline int xil_fpo_unordered_d(double a, double b)
379 {

```



```

380  int res_int = 0;
381
382  // xip_fpo_unordered_d
383  xip_fpo_unordered_d(&res_int, a, b); // normal operation
384  return res_int;
385 }
386
387 ///////////////////////////////////////////////////////////////////
388 // Operation functions: compare equal
389 ///////////////////////////////////////////////////////////////////
390
391 inline int xil_fpo_equal_flt(float a, float b)
392 {
393     int res_int = 0;
394
395     // xip_fpo_equal_flt
396     xip_fpo_equal_flt(&res_int, a, b); // normal operation
397     return res_int;
398 }
399
400 inline int xil_fpo_equal_d(double a, double b)
401 {
402     int res_int = 0;
403
404     // xip_fpo_equal_d
405     xip_fpo_equal_d(&res_int, a, b); // normal operation
406     return res_int;
407 }
408
409 ///////////////////////////////////////////////////////////////////
410 // Operation functions: compare less than
411 ///////////////////////////////////////////////////////////////////
412
413 inline int xil_fpo_less_flt(float a, float b)
414 {
415     int res_int = 0;
416
417     // xip_fpo_less_flt
418     xip_fpo_less_flt(&res_int, a, b); // normal operation
419     return res_int;
420 }
421
422 inline int xil_fpo_less_d(double a, double b)
423 {
424     int res_int = 0;
425
426     // xip_fpo_less_d
427     xip_fpo_less_d(&res_int, a, b); // normal operation
428     return res_int;
429 }
430
431 ///////////////////////////////////////////////////////////////////
432 // Operation functions: compare less than or equal
433 ///////////////////////////////////////////////////////////////////
434
435 inline int xil_fpo_lessequal_flt(float a, float b)
436 {
437     int res_int = 0;
438
439     // xip_fpo_lessequal_flt
440     xip_fpo_lessequal_flt(&res_int, a, b); // normal operation
441     return res_int;
442 }
443
444 inline int xil_fpo_lessequal_d(double a, double b)
445 {
446     int res_int = 0;
447
448     // xip_fpo_lessequal_d
449     xip_fpo_lessequal_d(&res_int, a, b); // normal operation
450     return res_int;
451 }
452
453 ///////////////////////////////////////////////////////////////////
454 // Operation functions: compare greater than
455 ///////////////////////////////////////////////////////////////////
456
457 inline int xil_fpo_greater_flt(float a, float b)
458 {
459     int res_int = 0;
460
461     // xip_fpo_greater_flt
462     xip_fpo_greater_flt(&res_int, a, b); // normal operation
463     return res_int;
464 }
465
466 inline int xil_fpo_greater_d(double a, double b)
467 {
468     int res_int = 0;
469
470     // xip_fpo_greater_d

```

```

472  xip_fpo_greater_d(&res_int, a, b); // normal operation
473  return res_int;
474 }
475
476 ///////////////////////////////////////////////////////////////////
477 // Operation functions: compare greater than or equal
478 ///////////////////////////////////////////////////////////////////
479
480 inline int xil_fpo_greaterequal_flt(float a, float b)
481 {
482     int res_int = 0;
483
484     // xip_fpo_greaterequal_flt
485     xip_fpo_greaterequal_flt(&res_int, a, b); // normal opera-
486     return res_int;
487 }
488
489 inline int xil_fpo_greaterequal_d(double a, double b)
490 {
491     int res_int = 0;
492
493     // xip_fpo_greaterequal_d
494     xip_fpo_greaterequal_d(&res_int, a, b); // normal opera-
495     return res_int;
496 }
497
498 ///////////////////////////////////////////////////////////////////
499 // Operation functions: compare not equal
500 ///////////////////////////////////////////////////////////////////
501
502 inline int xil_fpo_notequal_flt(float a, float b)
503 {
504     int res_int = 0;
505
506     // xip_fpo_notequal_flt
507     xip_fpo_notequal_flt(&res_int, a, b); // normal operation
508     return res_int;
509 }
510
511 inline int xil_fpo_notequal_d(double a, double b)
512 {
513     int res_int = 0;
514
515     xip_fpo_notequal_d(&res_int, a, b); // normal operation
516     return res_int;
517 }
518
519 ///////////////////////////////////////////////////////////////////
520 // Operation functions: compare condition code
521 ///////////////////////////////////////////////////////////////////
522
523 inline int xil_fpo_condcode_flt(float a, float b)
524 {
525     int res_int = 0;
526
527     // xip_fpo_condcode_flt
528     xip_fpo_condcode_flt(&res_int, a, b); // normal operation
529     return res_int;
530 }
531
532 inline int xil_fpo_condcode_d(double a, double b)
533 {
534     int res_int = 0;
535
536     // xip_fpo_condcode_d
537     xip_fpo_condcode_d(&res_int, a, b); // normal operation
538     return res_int;
539 }
540
541 ///////////////////////////////////////////////////////////////////
542 // Conversion functions: conversion code
543 ///////////////////////////////////////////////////////////////////
544
545 inline int xil_fpo_flttofix_int_flt(float a)
546 {
547     int res_int = 0;
548
549     // xip_fpo_flttofix_int_flt
550     xip_fpo_flttofix_int_flt(&res_int, a); // normal operation
551     return res_int;
552 }
553
554 inline int xil_fpo_flttofix_int_d(double a)
555 {
556     int res_int = 0;
557
558     // xip_fpo_flttofix_int_d
559     xip_fpo_flttofix_int_d(&res_int, a); // normal operation
560     return res_int;
561 }

```



```

562 inline float xil_fpo_fixtoflt_flt_int(int a)
563 {
564     float res_flt = 0.0f;
565
566     // xil_fpo_fixtoflt_flt_int
567     xil_fpo_fixtoflt_flt_int(&res_flt, a); // normal operation
568     return res_flt;
569 }
570
571 inline double xil_fpo_fixtoflt_d_int(int a)
572 {
573     double res_d = 0.0;
574
575     // xil_fpo_fixtoflt_d_int
576     xil_fpo_fixtoflt_d_int(&res_d, a); // normal operation
577     return res_d;
578 }
579
580 inline float xil_fpo_flttoflt_flt_flt(float a)
581 {
582     float res_flt = 0.0f;
583
584     // xil_fpo_flttoflt_flt_flt
585     xil_fpo_flttoflt_flt_flt(&res_flt, a); // normal operation
586     return res_flt;
587 }
588
589 inline float xil_fpo_flttoflt_flt_d(double a)
590 {
591     float res_flt = 0.0f;
592
593     // xil_fpo_flttoflt_flt_d
594     xil_fpo_flttoflt_flt_d(&res_flt, a); // normal operation
595     return res_flt;
596 }
597
598 inline double xil_fpo_flttoflt_d_flt(float a)
599 {
600     double res_d = 0.0;
601
602     // xil_fpo_flttoflt_d_flt
603     xil_fpo_flttoflt_d_flt(&res_d, a); // normal operation
604     return res_d;
605 }
606
607 inline double xil_fpo_flttoflt_d_d(double a)
608 {
609     double res_d = 0.0;
610
611     // xil_fpo_flttoflt_d_d
612     xil_fpo_flttoflt_d_d(&res_d, a); // normal operation
613     return res_d;
614 }
615
616 #endif
617 #endif /* #ifndef __AESL_FPO_H__ */
618
619

```

E.43 vendor/hls_half.h (155 lines, 3,956 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef __HLS_HALF_H__
7 #define __HLS_HALF_H__
8
9 #ifdef __SYNTHESIS__
10
11 #ifndef __HLS_HALF_DEFINED__
12 typedef __fp16 half;
13 #endif
14 #else // AESL_SYN
15
16 #ifndef __cplusplus
17 #error C++ is required to include this header file
18 #endif
19
20 #if defined __arm__ && !(defined HLS_NO_XIL_FPO_LIB)
21 #warning "Xilinx Floating Point Operator IP core does not
provide simulation models for ARM architecture. Automatically de-
fining HLS_NO_XIL_FPO_LIB in order to avoid this library depend-
ency, although bit-accurate simulation of some functions is no
longer possible. You can make this warning go away by adding this
define yourself before including any other files."
22 #define HLS_NO_XIL_FPO_LIB
23 #endif
24
25 #ifndef HLS_NO_XIL_FPO_LIB
26 #include "etc/hls_half_fpo.h"
27 #else
28 #include "etc/hls_half_sim.h"
29 #endif
30
31 #include <limits>
32 #include <limits>

```

```

33 /// Extensions to the C++ standard library.
34 /// It is used in hlsmath lib.
35 namespace std {
36 /// Numeric limits for half-precision floats.
37
38 /// Because of the underlying single-precision implementation of
many operations, it inherits some properties from
38 /// 'std::numeric_limits<float>'.
39 template <>
40 struct numeric_limits<half> : public numeric_limits<float> {
41     public:
42         /// Supports signed values.
43         static const bool is_signed = true;
44
45         /// Is not exact.
46         static const bool is_exact = false;
47
48         /// Doesn't provide modulo arithmetic.
49         static const bool is_modulo = false;
50
51         /// IEEE conformant.
52         static const bool is_iec559 = true;
53
54         /// Supports infinity.
55         static const bool has_infinity = true;
56
57         /// Supports quiet NaNs.
58         static const bool has_quiet_NaN = true;
59
60         /// Supports subnormal values.
61         static const float_denorm_style has_denorm = de-
norm_present;
62
63         /// Significant digits.
64         static const int digits = 11;
65
66         /// Significant decimal digits.
67         static const int digits10 = 3;
68
69         /// Required decimal digits to represent all possible
values.
70         static const int max_digits10 = 5;
71
72         /// Number base.
73         static const int radix = 2;
74
75         /// One more than smallest exponent.
76         static const int min_exponent = -13;
77
78         /// Smallest normalized representable power of 10.
79         static const int min_exponent10 = -4;
80
81         /// One more than largest exponent
82         static const int max_exponent = 16;
83
84         /// Largest finitely representable power of 10.
85         static const int max_exponent10 = 4;
86
87         /// Smallest positive normal value.
88         static half min() throw() {
89             half t;
90             t.set_bits(0x0400);
91             return t;
92         }
93
94         /// Smallest finite value.
95         static half lowest() throw() {
96             half t;
97             t.set_bits(0xFBFF);
98             return t;
99         }
100
101         /// Largest finite value.
102         static half max() throw() {
103             half t;
104             t.set_bits(0x7BFF);
105             return t;
106         }
107
108         /// Difference between one and next representable value.
109         static half epsilon() throw() {
110             half t;
111             t.set_bits(0x1400);
112             return t;
113         }
114
115         /// Maximum rounding error.
116         static half round_error() throw() {
117             half t;
118             t.set_bits(0x3800);
119             return t;
120         }
121
122         /// Positive infinity.
123         static half infinity() throw() {
124             half t;
125             t.set_bits(0x7C00);
126             return t;
127         }
128
129         /// Quiet NaN.

```

```

129     static half quiet_NaN() throw() {
130         half t;
131         t.set_bits(0x7FFF);
132         return t;
133     }
134
135     /// Signalling NaN.
136     static half signaling_NaN() throw() {
137         half t;
138         t.set_bits(0x7DFF);
139         return t;
140     }
141
142     /// Smallest positive subnormal value.
143     static half denorm_min() throw() {
144         half t;
145         t.set_bits(0x0001);
146         return t;
147     }
148 }; // numeric_limits<half>
149 } // namespace std
150
151 #include "etc/hls_half_x_utils.h"
152
153 #endif // __SYNTHESIS__
154
155 #endif // __HLS_HALF__

```

E.44 vendor/hls_math.h (3,209 lines, 78,904 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6
7 #ifndef _X_HLSTMP_MATH_
8 #define _X_HLSTMP_MATH_
9
10 #ifndef __cplusplus
11 #error "hls_math.h requires C++ and is not support in C com-
pilation."
12 #endif
13
14 #if __cplusplus >= 201402L
15 # define DEPRECATED_MSG(msg) [[deprecated(msg)]]
16 #else
17 #if defined(__GNUC__) || defined(__clang__)
18 # define DEPRECATED_MSG(msg) __attrib-
ute__((__deprecated__(msg)))
19 #elif defined(_MSC_VER)
20 # define DEPRECATED_MSG(msg) __declspec(deprecated(msg))
21 #else
22 # define DEPRECATED_MSG(msg)
23 #endif
24 #endif
25
26 #include <cmath>
27 #include "hls_half.h"
28 #include "hls_fpo.h"
29 #include <stdint.h>
30
31 namespace hls {
32
33     // signbit
34     int signbit(double);
35     int signbit(float);
36     int signbit(half);
37     int signbitf(float);
38     int half_signbit(half);
39
40     // nan
41     double nan(const char*);
42     float nanf(const char*);
43     half half_nan(const char *);
44
45     // isfinite
46     int isfinite(double);
47     int isfinite(float);
48     int isfinite(half);
49     int isfinitef(float);
50     int half_isfinite(half);
51
52     // __isfinite
53     int __isfinite(double);
54     int __isfinite(float);
55     int __isfinite(half);
56     int __isfinitef(float);
57
58     // isinf
59     int isinf(double);
60     int isinf(float);
61     int isinf(half);
62     int isnanf(float);
63     int half_isinf(half);
64
65     // __isinf
66     int __isinf(double);

```

```

67     int __isinf(float);
68     int __isinf(half);
69     int __isninf(float);
70
71     // isnan
72     int isnan(double);
73     int isnan(float);
74     int isnan(half);
75     int isnanf(float);
76     int half_isnan(half);
77
78     // __isnan
79     int __isnan(double);
80     int __isnan(float);
81     int __isnan(half);
82     int __isnanf(float);
83
84     // isnormal
85     int isnormal(double);
86     int isnormal(float);
87     int isnormal(half);
88     int isnormalf(float);
89     int half_isnormal(half);
90
91     // __isnormal
92     int __isnormal(double);
93     int __isnormal(float);
94     int __isnormal(half);
95     int __isnormalf(float);
96
97     // fpclassify
98     int fpclassify(double);
99     int fpclassify(float);
100     int fpclassify(half);
101     int fpclassifyf(float);
102     int half_fpclassify(half);
103
104     // __fpclassify
105     int __fpclassify(double);
106     int __fpclassify(float);
107     int __fpclassify(half);
108     int __fpclassifyf(float);
109
110     // copysign
111     double copysign(double, double);
112     float copysign(float, float);
113     half copysign(half, half);
114     float copysignf(float, float);
115     half half_copysign(half, half);
116     int8_t copysign(int8_t, int8_t);
117     uint8_t copysign(uint8_t, uint8_t);
118     int16_t copysign(int16_t, int16_t);
119     uint16_t copysign(uint16_t, uint16_t);
120     int32_t copysign(int32_t, int32_t);
121     uint32_t copysign(uint32_t, uint32_t);
122
123     // fabs
124     double fabs(double);
125     float fabs(float);
126     half fabs(half);
127     float fabsf(float);
128     half half_fabs(half);
129     int8_t fabs(int8_t);
130     uint8_t fabs(uint8_t);
131     int16_t fabs(int16_t);
132     uint16_t fabs(uint16_t);
133     int32_t fabs(int32_t);
134     uint32_t fabs(uint32_t);
135
136     // abs
137     double abs(double);
138     float abs(float);
139     half abs(half);
140     float absf(float);
141     half half_abs(half);
142     int8_t abs(int8_t);
143     uint8_t abs(uint8_t);
144     int16_t abs(int16_t);
145     uint16_t abs(uint16_t);
146     int32_t abs(int32_t);
147     uint32_t abs(uint32_t);
148
149     // fma
150     double fma(double, double, double);
151     float fma(float, float, float);
152     half fma(half, half, half);
153     float fmaf(float, float, float);
154     half half_fma(half, half, half);
155
156     // mad
157     double mad(double, double, double);
158     float mad(float, float, float);
159     half mad(half, half, half);
160     float madf(float, float, float);
161     half half_mad(half, half, half);
162
163     // isequal
164     int isequal(double, double);
165     int isequal(float, float);
166     int isequal(half, half);

```

```

167 int isequalf(float, float);
168 int half_isequal(half, half);
169
170 // isgreater
171 int isgreater(double, double);
172 int isgreater(float, float);
173 int isgreater(half, half);
174 int isgreaterf(float, float);
175 int half_isgreater(half, half);
176
177 // isgreaterequal
178 int isgreaterequal(double, double);
179 int isgreaterequal(float, float);
180 int isgreaterequal(half, half);
181 int isgreaterequalf(float, float);
182 int half_isgreaterequal(half, half);
183
184 // isless
185 int isless(double, double);
186 int isless(float, float);
187 int isless(half, half);
188 int islessf(float, float);
189 int half_isless(half, half);
190
191 // islessequal
192 int islessequal(double, double);
193 int islessequal(float, float);
194 int islessequal(half, half);
195 int islessequalf(float, float);
196 int half_islessequal(half, half);
197
198 // islessgreater
199 int islessgreater(double, double);
200 int islessgreater(float, float);
201 int islessgreater(half, half);
202 int islessgreaterf(float, float);
203 int half_islessgreater(half, half);
204
205 // isnotequal
206 int isnotequal(double, double);
207 int isnotequal(float, float);
208 int isnotequal(half, half);
209 int isnotequalf(float, float);
210 int half_isnotequal(half, half);
211
212 // fmax
213 double fmax(double, double);
214 float fmax(float, float);
215 half fmax(half, half);
216 float fmaxf(float, float);
217 half fmaxf(half, half);
218 int8_t fmax(int8_t, int8_t);
219 uint8_t fmax(uint8_t, uint8_t);
220 int16_t fmax(int16_t, int16_t);
221 uint16_t fmax(uint16_t, uint16_t);
222 int32_t fmax(int32_t, int32_t);
223 uint32_t fmax(uint32_t, uint32_t);
224
225 // fmin
226 double fmin(double, double);
227 float fmin(float, float);
228 half fmin(half, half);
229 float fminf(float, float);
230 half fminf(half, half);
231 int8_t fmin(int8_t, int8_t);
232 uint8_t fmin(uint8_t, uint8_t);
233 int16_t fmin(int16_t, int16_t);
234 uint16_t fmin(uint16_t, uint16_t);
235 int32_t fmin(int32_t, int32_t);
236 uint32_t fmin(uint32_t, uint32_t);
237
238 // max
239 template <class T>
240 T max(T x, T y){
241     return (x > y) ? x : y;
242 }
243
244 // min
245 template <class T>
246 T min(T x, T y){
247     return (x < y) ? x : y;
248 }
249
250 // fdim
251 double fdim(double, double);
252 float fdim(float, float);
253 half fdim(half, half);
254 float fdimf(float, float);
255 half fdimf(half, half);
256 int8_t fdim(int8_t, int8_t);
257 uint8_t fdim(uint8_t, uint8_t);
258 int16_t fdim(int16_t, int16_t);
259 uint16_t fdim(uint16_t, uint16_t);
260 int32_t fdim(int32_t, int32_t);
261 uint32_t fdim(uint32_t, uint32_t);
262
263 // maxmag
264 double maxmag(double, double);
265 float maxmag(float, float);
266 half maxmag(half, half);
267
268 float maxmagf(float, float);
269 half maxmag(half, half);
270 int8_t maxmag(int8_t, int8_t);
271 uint8_t maxmag(uint8_t, uint8_t);
272 int16_t maxmag(int16_t, int16_t);
273 uint16_t maxmag(uint16_t, uint16_t);
274 int32_t maxmag(int32_t, int32_t);
275 uint32_t maxmag(uint32_t, uint32_t);
276
277 // minmag
278 double minmag(double, double);
279 float minmag(float, float);
280 half minmag(half, half);
281 float minmagf(float, float);
282 half minmagf(half, half);
283 int8_t minmag(int8_t, int8_t);
284 uint8_t minmag(uint8_t, uint8_t);
285 int16_t minmag(int16_t, int16_t);
286 uint16_t minmag(uint16_t, uint16_t);
287 int32_t minmag(int32_t, int32_t);
288 uint32_t minmag(uint32_t, uint32_t);
289
290 // frexp
291 double frexp(double, int*);
292 float frexp(float, int*);
293 half frexp(half, int*);
294 float frexpf(float, int*);
295 half half_frexp(half, int*);
296
297 // ldexp
298 double ldexp(double, int);
299 float ldexp(float, int);
300 half ldexp(half, int);
301 float ldexpf(float, int);
302 half half_ldexp(half, int);
303
304 // ilogb
305 int ilogb(double);
306 int ilogb(float);
307 int ilogb(half);
308 int ilogbf(float);
309 int half_ilogb(half);
310
311 // scalbn
312 double scalbn(double, int);
313 float scalbn(float, int);
314 half scalbn(half, int);
315 float scalbnf(float, int);
316 half half_scalbn(half, int);
317
318 // scalbln
319 double scalbln(double, long int);
320 float scalbln(float, long int);
321 half scalbln(half, long int);
322 float scalblnf(float, long int);
323 half half_scalbln(half, long int);
324
325 // ceil
326 double ceil(double);
327 float ceil(float);
328 half ceil(half);
329 float ceilf(float);
330 half half_ceil(half);
331
332 // floor
333 double floor(double);
334 float floor(float);
335 half floor(half);
336 float floorf(float);
337 half half_floor(half);
338
339 // trunc
340 double trunc(double);
341 float trunc(float);
342 half trunc(half);
343 float truncf(float);
344 half half_trunc(half);
345
346 // round
347 double round(double);
348 float round(float);
349 half round(half);
350 float roundf(float);
351 half half_round(half);
352
353 // rint
354 double rint(double);
355 float rint(float);
356 half rint(half);
357 float rintf(float);
358 half half_rint(half);
359
360 // lround
361 long int lround(double);
362 long int lround(float);
363 long int lround(half);
364 long int lroundf(float);
365 long int half_lround(half);
366

```

```

367 // llround
368 long long int llround(double);
369 long long int llround(float);
370 long long int llround(half);
371 long long int llroundf(float);
372 long long int half_llround(half);
373
374 // lrint
375 long int lrint(double);
376 long int lrint(float);
377 long int lrint(half);
378 long int lrintf(float);
379 long int half_lrint(half);
380
381 // llrint
382 long long int llrint(double);
383 long long int llrint(float);
384 long long int llrint(half);
385 long long int llrintf(float);
386 long long int half_llrint(half);
387
388 // cast_IEEE754
389 template<class T>
390 T cast_IEEE754(double, bool);
391 template<class T>
392 T cast_IEEE754(float, bool);
393 template<class T>
394 T cast_IEEE754(half, bool);
395
396 template<class T>
397 T cast_IEEE754(double);
398 template<class T>
399 T cast_IEEE754(float);
400 template<class T>
401 T cast_IEEE754(half);
402
403
404 // cast_IEEE754<char>
405 template<>
406 char cast_IEEE754<char>(double, bool);
407 template<>
408 char cast_IEEE754<char>(float, bool);
409 template<>
410 char cast_IEEE754<char>(half, bool);
411 template<>
412 char cast_IEEE754<char>(double);
413 template<>
414 char cast_IEEE754<char>(float);
415 template<>
416 char cast_IEEE754<char>(half);
417
418 // cast_IEEE754<int8_t>
419 template<>
420 int8_t cast_IEEE754<int8_t>(double, bool);
421 template<>
422 int8_t cast_IEEE754<int8_t>(float, bool);
423 template<>
424 int8_t cast_IEEE754<int8_t>(half, bool);
425 template<>
426 int8_t cast_IEEE754<int8_t>(double);
427 template<>
428 int8_t cast_IEEE754<int8_t>(float);
429 template<>
430 int8_t cast_IEEE754<int8_t>(half);
431
432 // cast_IEEE754<uint8_t>
433 template<>
434 uint8_t cast_IEEE754<uint8_t>(double, bool);
435 template<>
436 uint8_t cast_IEEE754<uint8_t>(float, bool);
437 template<>
438 uint8_t cast_IEEE754<uint8_t>(half, bool);
439 template<>
440 uint8_t cast_IEEE754<uint8_t>(double);
441 template<>
442 uint8_t cast_IEEE754<uint8_t>(float);
443 template<>
444 uint8_t cast_IEEE754<uint8_t>(half);
445
446 // cast_IEEE754<int16_t>
447 template<>
448 int16_t cast_IEEE754<int16_t>(double, bool);
449 template<>
450 int16_t cast_IEEE754<int16_t>(float, bool);
451 template<>
452 int16_t cast_IEEE754<int16_t>(half, bool);
453 template<>
454 int16_t cast_IEEE754<int16_t>(double);
455 template<>
456 int16_t cast_IEEE754<int16_t>(float);
457 template<>
458 int16_t cast_IEEE754<int16_t>(half);
459
460 // cast_IEEE754<uint16_t>
461 template<>
462 uint16_t cast_IEEE754<uint16_t>(double, bool);
463 template<>
464 uint16_t cast_IEEE754<uint16_t>(float, bool);
465 template<>
466 uint16_t cast_IEEE754<uint16_t>(half, bool);
467
468 template<>
469 uint16_t cast_IEEE754<uint16_t>(double);
470 template<>
471 uint16_t cast_IEEE754<uint16_t>(float);
472 template<>
473 uint16_t cast_IEEE754<uint16_t>(half);
474
475 // cast_IEEE754<int32_t>
476 template<>
477 int32_t cast_IEEE754<int32_t>(double, bool);
478 template<>
479 int32_t cast_IEEE754<int32_t>(float, bool);
480 template<>
481 int32_t cast_IEEE754<int32_t>(half, bool);
482 template<>
483 int32_t cast_IEEE754<int32_t>(double);
484 template<>
485 int32_t cast_IEEE754<int32_t>(float);
486 template<>
487 int32_t cast_IEEE754<int32_t>(half);
488
489 // cast_IEEE754<uint32_t>
490 template<>
491 uint32_t cast_IEEE754<uint32_t>(double, bool);
492 template<>
493 uint32_t cast_IEEE754<uint32_t>(float, bool);
494 template<>
495 uint32_t cast_IEEE754<uint32_t>(half, bool);
496 template<>
497 uint32_t cast_IEEE754<uint32_t>(double);
498 template<>
499 uint32_t cast_IEEE754<uint32_t>(float);
500 template<>
501 uint32_t cast_IEEE754<uint32_t>(half);
502
503 // cast_IEEE754<int64_t>
504 template<>
505 int64_t cast_IEEE754<int64_t>(double, bool);
506 template<>
507 int64_t cast_IEEE754<int64_t>(float, bool);
508 template<>
509 int64_t cast_IEEE754<int64_t>(half, bool);
510 template<>
511 int64_t cast_IEEE754<int64_t>(double);
512 template<>
513 int64_t cast_IEEE754<int64_t>(float);
514 template<>
515 int64_t cast_IEEE754<int64_t>(half);
516
517 // cast_IEEE754<uint64_t>
518 template<>
519 uint64_t cast_IEEE754<uint64_t>(double, bool);
520 template<>
521 uint64_t cast_IEEE754<uint64_t>(float, bool);
522 template<>
523 uint64_t cast_IEEE754<uint64_t>(half, bool);
524 template<>
525 uint64_t cast_IEEE754<uint64_t>(double);
526 template<>
527 uint64_t cast_IEEE754<uint64_t>(float);
528 template<>
529 uint64_t cast_IEEE754<uint64_t>(half);
530
531 // nearbyint
532 double nearbyint(double);
533 float nearbyint(float);
534 half nearbyint(half);
535 float nearbyintf(float);
536 half half_nearbyint(half);
537
538 // modf
539 double modf(double, double*);
540 float modf(float, float*);
541 half modf(half, half*);
542 float modff(float, float*);
543 half half_modf(half, half*);
544
545 // fract
546 double fract(double, double*);
547 float fract(float, float*);
548 half fract(half, half*);
549 float fractf(float, float*);
550 half half_fract(half, half*);
551
552 // isordered
553 int isordered(double, double);
554 int isordered(float, float);
555 int isordered(half, half);
556 int isorderedf(float, float);
557 int half_isordered(half, half);
558
559 // isunordered
560 int isunordered(double, double);
561 int isunordered(float, float);
562 int isunordered(half, half);
563 int isunorderedf(float, float);
564 int half_isunordered(half, half);
565
566 // bitselect

```

```

567 double bitselect(double, double, double);
568 float bitselect(float, float, float);
569 int bitselect(int, int, int);
570 uint32_t bitselect(uint32_t, uint32_t, uint32_t);
571 half bitselect(half, half, half);
572 float bitselectf(float, float, float);
573 half half_bitselect(half, half, half);
574
575 // sin cos implemented with hotbm method
576 namespace hotbm {
577     double sin(double);
578     float sin(float);
579     half sin(half);
580     float sinf(float);
581     half half_sin(half);
582
583     double cos(double);
584     float cos(float);
585     half cos(half);
586     float cosf(float);
587     half half_cos(half);
588
589     void sincos(double, double*, double*);
590     void sincos(float, float*, float*);
591     void sincos(half, half*, half*);
592     void sincosf(float, float*, float*);
593     void half_sincos(half, half*, half*);
594 }
595
596 // sin cos implemented with cordic method
597 namespace cordic {
598     double sin(double);
599     float sin(float);
600     half sin(half);
601     float sinf(float);
602     half half_sin(half);
603
604     double cos(double);
605     float cos(float);
606     half cos(half);
607     float cosf(float);
608     half half_cos(half);
609
610     void sincos(double, double*, double*);
611     void sincos(float, float*, float*);
612     void sincos(half, half*, half*);
613     void sincosf(float, float*, float*);
614     void half_sincos(half, half*, half*);
615 }
616
617 // sin cos implemented with hotbm method
618 double sin(double);
619 float sin(float);
620 half sin(half);
621 float sinf(float);
622 half half_sin(half);
623
624 double cos(double);
625 float cos(float);
626 half cos(half);
627 float cosf(float);
628 half half_cos(half);
629
630 // sincos
631 void sincos(double, double*, double*);
632 void sincos(float, float*, float*);
633 void sincos(half, half*, half*);
634 void sincosf(float, float*, float*);
635 void half_sincos(half, half*, half*);
636
637 // sinpi
638 double sinpi(double);
639 float sinpi(float);
640 half sinpi(half);
641 float sinpif(float);
642 half half_sinpi(half);
643
644 // cospi
645 double cospi(double);
646 float cospi(float);
647 half cospi(half);
648 float cospif(float);
649 half half_cospi(half);
650
651 // tanpi
652 double tanpi(double);
653 float tanpi(float);
654 half tanpi(half);
655 float tanpif(float);
656 half half_tanpi(half);
657
658 // atan
659 double atan(double);
660 float atan(float);
661 half atan(half);
662 float atanf(float);
663 half half_atan(half);
664
665 // atan2
666 double atan2(double, double);
667 float atan2(float, float);
668 half atan2(half, half);
669 float atan2f(float, float);
670 half half_atan2(half, half);
671
672 // tan
673 double tan(double);
674 float tan(float);
675 half tan(half);
676 float tanf(float);
677 half half_tan(half);
678
679 // sinh
680 double sinh(double);
681 float sinh(float);
682 half sinh(half);
683 float sinhf(float);
684 half half_sinh(half);
685
686 // cosh
687 double cosh(double);
688 float cosh(float);
689 half cosh(half);
690 float coshf(float);
691 half half_cosh(half);
692
693 // exp
694 double exp(double);
695 float exp(float);
696 half exp(half);
697 float expf(float);
698 half half_exp(half);
699 int8_t exp(int8_t);
700 uint8_t exp(uint8_t);
701 int16_t exp(int16_t);
702 uint16_t exp(uint16_t);
703 int32_t exp(int32_t);
704 uint32_t exp(uint32_t);
705
706 // exp2
707 double exp2(double);
708 float exp2(float);
709 half exp2(half);
710 float exp2f(float);
711 half half_exp2(half);
712
713 // exp10
714 double exp10(double);
715 float exp10(float);
716 half exp10(half);
717 float exp10f(float);
718 half half_exp10(half);
719
720 // expm1
721 double expm1(double);
722 float expm1(float);
723 half expm1(half);
724 float expm1f(float);
725 half half_expm1(half);
726
727 // log
728 double log(double);
729 float log(float);
730 half log(half);
731 float logf(float);
732 half half_log(half);
733 int8_t log(int8_t);
734 uint8_t log(uint8_t);
735 int16_t log(int16_t);
736 uint16_t log(uint16_t);
737 int32_t log(int32_t);
738 uint32_t log(uint32_t);
739
740 // log2
741 double log2(double);
742 float log2(float);
743 half log2(half);
744 float log2f(float);
745 half half_log2(half);
746
747 // log10
748 double log10(double);
749 float log10(float);
750 half log10(half);
751 float log10f(float);
752 half half_log10(half);
753 int8_t log10(int8_t);
754 uint8_t log10(uint8_t);
755 int16_t log10(int16_t);
756 uint16_t log10(uint16_t);
757 int32_t log10(int32_t);
758 uint32_t log10(uint32_t);
759
760 // logb
761 double logb(double);
762 float logb(float);
763 half logb(half);
764 float logbf(float);
765 half half_logb(half);
766 // log1p

```

```

767 double log1p(double);
768 float log1p(float);
769 half log1p(half);
770 float log1pf(float);
771 half half_log1p(half);
772
773 // pow
774 half pow(half, half);
775 float pow(float, float);
776 double pow(double, double);
777 float powf(float, float);
778 half half_pow(half, half);
779 int8_t pow(int8_t, int8_t);
780 uint8_t pow(uint8_t, uint8_t);
781 int16_t pow(int16_t, int16_t);
782 uint16_t pow(uint16_t, uint16_t);
783 int32_t pow(int32_t, int32_t);
784 uint32_t pow(uint32_t, uint32_t);
785 int8_t powr(int8_t, int8_t);
786 uint8_t powr(uint8_t, uint8_t);
787 int16_t powr(int16_t, int16_t);
788 uint16_t powr(uint16_t, uint16_t);
789 int32_t powr(int32_t, int32_t);
790 uint32_t powr(uint32_t, uint32_t);
791
792 // powr
793 half powr(half, half);
794 float powr(float, float);
795 double powr(double, double);
796 float powrf(float, float);
797 half half_powr(half, half);
798
799 // pown
800 half pown(half, int);
801 float pown(float, int);
802 double pown(double, int);
803 float pownf(float, int);
804 half half_pown(half, int);
805
806 // rootn
807 half rootn(half, int);
808 float rootn(float, int);
809 double rootn(double, int);
810 float rootnf(float, int);
811 half half_rootn(half, int);
812
813 // sqrt
814 double sqrt(double);
815 float sqrt(float);
816 half sqrt(half);
817 float sqrtf(float);
818 half half_sqrt(half);
819 int8_t sqrt(int8_t);
820 uint8_t sqrt(uint8_t);
821 int16_t sqrt(int16_t);
822 uint16_t sqrt(uint16_t);
823 int32_t sqrt(int32_t);
824 uint32_t sqrt(uint32_t);
825
826 // template<int W, int I>
827 // ap_fixed<W,I> sqrt(ap_fixed<W,I> x){
828 // return sqrt_fixed(x);
829 // };
830 // template<int W, int I>
831 // ap_ufixed<W,I> sqrt(ap_ufixed<W,I> x){
832 // return sqrt_fixed(x);
833 // };
834 // template<int I>
835 // ap_int<I> sqrt(ap_int<I> x){
836 // return sqrt_fixed(x);
837 // };
838 // template<int I>
839 // ap_uint<I> sqrt(ap_uint<I> x){
840 // return sqrt_fixed(x);
841 // };
842
843 // recip
844 double recip(double);
845 float recip(float);
846 half recip(half);
847 float recipf(float);
848 half half_recip(half);
849 int8_t recip(int8_t);
850 uint8_t recip(uint8_t);
851 int16_t recip(int16_t);
852 uint16_t recip(uint16_t);
853 int32_t recip(int32_t);
854 uint32_t recip(uint32_t);
855
856 // rsqrt
857 double rsqrt(double);
858 float rsqrt(float);
859 half rsqrt(half);
860 float rsqrtf(float);
861 half half_rsqrt(half);
862 int8_t rsqrt(int8_t);
863 uint8_t rsqrt(uint8_t);
864 int16_t rsqrt(int16_t);
865 uint16_t rsqrt(uint16_t);
866 int32_t rsqrt(int32_t);

```

```

867 uint32_t rsqrt(uint32_t);
868
869 // cbrt
870 double cbrt(double);
871 float cbrt(float);
872 half cbrt(half);
873 float cbrtf(float);
874 half half_cbrt(half);
875
876 // hypot
877 double hypot(double, double);
878 float hypot(float, float);
879 half hypot(half, half);
880 float hypotf(float, float);
881 half half_hypot(half, half);
882
883 // erf
884 double erf(double);
885 float erf(float);
886 half erf(half);
887 float erff(float);
888 half half_erf(half);
889 int8_t erf(int8_t);
890 uint8_t erf(uint8_t);
891 int16_t erf(int16_t);
892 uint16_t erf(uint16_t);
893 int32_t erf(int32_t);
894 uint32_t erf(uint32_t);
895
896 // erfc
897 double erfc(double);
898 float erfc(float);
899 half erfc(half);
900 float erfcf(float);
901 half half_erfc(half);
902 int8_t erfc(int8_t);
903 uint8_t erfc(uint8_t);
904 int16_t erfc(int16_t);
905 uint16_t erfc(uint16_t);
906 int32_t erfc(int32_t);
907 uint32_t erfc(uint32_t);
908
909 // lgamma
910 double lgamma(double);
911 float lgamma(float);
912 half lgamma(half);
913 float lgammaf(float);
914 half half_lgamma(half);
915 int8_t lgamma(int8_t);
916 uint8_t lgamma(uint8_t);
917 int16_t lgamma(int16_t);
918 uint16_t lgamma(uint16_t);
919 int32_t lgamma(int32_t);
920 uint32_t lgamma(uint32_t);
921
922 // lgamma_r
923 double lgamma_r(double, int*);
924 float lgamma_r(float, int*);
925 half lgamma_r(half, int*);
926 float lgamma_rf(float, int*);
927 half half_lgamma_r(half, int*);
928 int8_t lgamma_r(int8_t);
929 uint8_t lgamma_r(uint8_t);
930 int16_t lgamma_r(int16_t);
931 uint16_t lgamma_r(uint16_t);
932 int32_t lgamma_r(int32_t);
933 uint32_t lgamma_r(uint32_t);
934
935 // cbrt
936 double tgamma(double);
937 float tgamma(float);
938 half tgamma(half);
939 float tgammaf(float);
940 half half_tgamma(half);
941 int8_t tgamma(int8_t);
942 uint8_t tgamma(uint8_t);
943 int16_t tgamma(int16_t);
944 uint16_t tgamma(uint16_t);
945 int32_t tgamma(int32_t);
946 uint32_t tgamma(uint32_t);
947
948 double tgamma_p_reduce(double);
949 float tgamma_p_reduce(float);
950 DEPRECATED_MSG("tgamma_p_reduce will be removed in future
release. Please use tgamma_p_reducef instead.")
951 float tgamma_p_reducef(float);
952 float tgamma_p_reducef(float);
953
954 // fmod
955 double fmod(double, double);
956 float fmod(float, float);
957 half fmod(half, half);
958 float fmodf(float, float);
959 half half_fmod(half, half);
960
961 // remainder
962 double remainder(double, double);
963 float remainder(float, float);
964 half remainder(half, half);
965 float remainderf(float, float);

```



```

966 half half_remainder(half, half);
967
968 // remquo
969 double remquo(double, double, int*);
970 float remquo(float, float, int*);
971 half remquo(half, half, int*);
972 float remquoof(float, float, int*);
973 half half_remquo(half, half, int*);
974
975 // divide
976 double divide(double, double);
977 float divide(float, float);
978 half divide(half, half);
979 float dividef(float, float);
980 half half_divide(half, half);
981
982 // nextafter
983 double nextafter(double, double);
984 float nextafter(float, float);
985 half nextafter(half, half);
986 float nextafterf(float, float);
987 half half_nextafter(half, half);
988
989 // tanh
990 double tanh(double);
991 float tanh(float);
992 half tanh(half);
993 float tanhf(float);
994 half half_tanh(half);
995
996 // atanh
997 double atanh(double);
998 float atanh(float);
999 half atanh(half);
1000 float atanhf(float);
1001 half half_atanh(half);
1002
1003 // asinh
1004 double asinh(double);
1005 float asinh(float);
1006 half asinh(half);
1007 float asinhf(float);
1008 half half_asinh(half);
1009
1010 // acosh
1011 double acosh(double);
1012 float acosh(float);
1013 half acosh(half);
1014 float acoshf(float);
1015 half half_acosh(half);
1016
1017 // asin
1018 double asin(double);
1019 float asin(float);
1020 half asin(half);
1021 float asinf(float);
1022 half half_asin(half);
1023
1024 // acos
1025 double acos(double);
1026 float acos(float);
1027 half acos(half);
1028 float acosf(float);
1029 half half_acos(half);
1030
1031 // asinpi
1032 double asinpi(double);
1033 float asinpi(float);
1034 half asinpi(half);
1035 float asinpihf(float);
1036 half half_asinpi(half);
1037
1038 // acospi
1039 double acospi(double);
1040 float acospi(float);
1041 half acospi(half);
1042 float acospihf(float);
1043 half half_acospi(half);
1044
1045 // atanpi
1046 double atanpi(double);
1047 float atanpi(float);
1048 half atanpi(half);
1049 float atanpihf(float);
1050 half half_atanpi(half);
1051 }
1052
1053 #include "etc/hls_hotbm_apfixed.h"
1054 #include "etc/hls_cordic_apfixed.h"
1055 #include "etc/hls_exp_apfixed.h"
1056 #include "etc/hls_log_apfixed.h"
1057 #include "etc/hls_sqrt_apfixed.h"
1058 #include "etc/hls_round_copysign_apfixed.h"
1059 #include "etc/hls_pow_apfixed.h"
1060 #include "etc/hls_diff_apfixed.h"
1061 #include "etc/hls_comparison_apfixed.h"
1062 #include "etc/hls_frexp_apfixed.h"
1063 #include "etc/hls_modf_apfixed.h"
1064 #include "etc/hls_isequal_isnotequal_apfixed.h"
1065 #include "etc/hls_all_any_apfixed.h"

```

```

1066 #include "etc/hls_select_bitselect_apfixed.h"
1067 #include "etc/hls_erf_apfixed.h"
1068 #include "etc/hls_fmod_rem_quo_remainder_divide_apfixed.h"
1069 #include "etc/hls_lgamma_apfixed.h"
1070 #include "etc/hls_tgamma_apfixed.h"
1071 #include "etc/hls_fract_apfixed.h"
1072 #include "etc/hls_mad_apfixed.h"
1073 #include "etc/hls_cbrt_apfixed.h"
1074 #include "etc/hls_hypot_apfixed.h"
1075 #include "etc/hls_ldexp_apfixed.h"
1076
1077 namespace hls {
1078
1079 // // sin(ap_fixed)
1080 // template<int W, int I>
1081 // ap_fixed<W, 2> sin(ap_fixed<W, I> x){
1082 //     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 0, 0);
1083 // }
1084 // template<int W, int I>
1085 // ap_fixed<W, 2> sinf(ap_fixed<W, I> x){
1086 //     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 0, 0);
1087 // }
1088
1089 // // cos(ap_fixed)
1090 // template<int W, int I>
1091 // ap_fixed<W, 2> cos(ap_fixed<W, I> x){
1092 //     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 1, 0);
1093 // }
1094 // template<int W, int I>
1095 // ap_fixed<W, 2> cosf(ap_fixed<W, I> x){
1096 //     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 1, 0);
1097 // }
1098
1099 // sinpi(ap_fixed), cospi(ap_fixed)
1100 template<int W, int I>
1101 ap_fixed<W, 2> sinpi(ap_fixed<W, I> x){
1102     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 0, 1);
1103 }
1104 /* template<int W, int I>
1105 ap_fixed<W, 2> sinpif(ap_fixed<W, I> x){
1106     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 0, 1);
1107 }*/
1108 template<int W, int I>
1109 ap_ufixed<W, 2> sinpi(ap_ufixed<W, I> x){
1110     ap_fixed<W+1, I+1> xin = x;
1111     ap_fixed<W+1, 3> xout = hls::sinpi<W+1, I+1>(x);
1112     ap_ufixed<W, 2> out = 0;
1113     if(xout[W]==0){
1114         out = xout;
1115     }
1116     return out;
1117 }
1118
1119 template<int W, int I>
1120 ap_fixed<W, 2> cospi(ap_fixed<W, I> x){
1121     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 1, 1);
1122 }
1123 template<int W, int I>
1124 ap_ufixed<W, 2> cospi(ap_ufixed<W, I> x){
1125     ap_fixed<W+1, I+1> xin = x;
1126     ap_fixed<W+1, 3> xout = hls::cospi<W+1, I+1>(x);
1127     ap_ufixed<W, 2> out = 0;
1128     if(xout[W]==0){
1129         out = xout;
1130     }
1131     return out;
1132 }
1133 /* template<int W, int I>
1134 ap_fixed<W, 2> cospif(ap_fixed<W, I> x){
1135     return hotbm_apfixed::sinf_or_cosf<W, I>(x, 1, 1);
1136 }*/
1137 //ap_int sinpi
1138 template<int I>
1139 ap_int<I> sinpi(ap_int<I> in){
1140     _AP_UNUSED_PARAM(in);
1141     return ap_int<I>(0);
1142 }
1143 //ap_int cospi
1144 template<int I>
1145 ap_int<I> cospi(ap_int<I> in){
1146     _AP_UNUSED_PARAM(in);
1147     return ap_int<I>(0);
1148 }
1149 //ap_uint sinpi
1150 template<int I>
1151 ap_uint<I> sinpi(ap_uint<I> in){
1152     _AP_UNUSED_PARAM(in);
1153     return ap_uint<I>(0);
1154 }
1155 //ap_uint cospi
1156 template<int I>
1157 ap_uint<I> cospi(ap_uint<I> in){
1158     _AP_UNUSED_PARAM(in);
1159     return ap_uint<I>(0);
1160 }
1161 int8_t sinpi(int8_t);
1162 uint8_t sinpi(uint8_t);
1163 int16_t sinpi(int16_t);
1164 uint16_t sinpi(uint16_t);
1165 int32_t sinpi(int32_t);

```



```

1166 uint32_t sinpi(uint32_t);
1167
1168 int8_t cospi(int8_t);
1169 uint8_t cospi(uint8_t);
1170 int16_t cospi(int16_t);
1171 uint16_t cospi(uint16_t);
1172 int32_t cospi(int32_t);
1173 uint32_t cospi(uint32_t);
1174
1175 // exp(ap_fixed)
1176 template<int W, int I>
1177 ap_fixed<W,I> exp(ap_fixed<W,I> x){
1178     return exp_reduce::exp(x);
1179 }
1180 template<int W, int I>
1181 ap_ufixed<W,I> exp(ap_ufixed<W,I> x){
1182     return exp_reduce::exp(x);
1183 }
1184 template<int I>
1185 ap_int<I> exp(ap_int<I> x){
1186     return exp_reduce::exp(x);
1187 }
1188 template<int I>
1189 ap_uint<I> exp(ap_uint<I> x){
1190     return exp_reduce::exp(x);
1191 }
1192
1193 // log(ap_fixed)
1194 template<int W, int I>
1195 ap_fixed<W,I> log(ap_fixed<W,I> x){
1196     return log_apfixed_reduce::log(x);
1197 }
1198 template<int W, int I>
1199 ap_ufixed<W,I> log(ap_ufixed<W,I> x){
1200     return log_apfixed_reduce::log(x);
1201 }
1202 template<int I>
1203 ap_int<I> log(ap_int<I> x){
1204     return log_apfixed_reduce::log(x);
1205 }
1206 template<int I>
1207 ap_uint<I> log(ap_uint<I> x){
1208     return log_apfixed_reduce::log(x);
1209 }
1210
1211 // log10(ap_fixed)
1212 template<int W, int I>
1213 ap_fixed<W,I> log10(ap_fixed<W,I> x){
1214     return log_apfixed_reduce::log10(x);
1215 }
1216 template<int W, int I>
1217 ap_ufixed<W,I> log10(ap_ufixed<W,I> x){
1218     return log_apfixed_reduce::log10(x);
1219 }
1220 template<int I>
1221 ap_int<I> log10(ap_int<I> x){
1222     return log_apfixed_reduce::log10(x);
1223 }
1224 template<int I>
1225 ap_uint<I> log10(ap_uint<I> x){
1226     return log_apfixed_reduce::log10(x);
1227 }
1228
1229 // log2(ap_fixed)
1230 template<int W, int I>
1231 ap_fixed<W,I> log2(ap_fixed<W,I> x){
1232     return log_apfixed_reduce::log2(x);
1233 }
1234 template<int W, int I>
1235 ap_ufixed<W,I> log2(ap_ufixed<W,I> x){
1236     return log_apfixed_reduce::log2(x);
1237 }
1238 template<int I>
1239 ap_int<I> log2(ap_int<I> x){
1240     return log_apfixed_reduce::log2(x);
1241 }
1242 template<int I>
1243 ap_uint<I> log2(ap_uint<I> x){
1244     return log_apfixed_reduce::log2(x);
1245 }
1246
1247 // logb(ap_fixed)
1248 template<int W, int I>
1249 ap_fixed<W,I> logb(ap_fixed<W,I> x){
1250     return log_apfixed_reduce::logb(x);
1251 }
1252 template<int W, int I>
1253 ap_ufixed<W,I> logb(ap_ufixed<W,I> x){
1254     return log_apfixed_reduce::logb(x);
1255 }
1256 template<int I>
1257 ap_int<I> logb(ap_int<I> x){
1258     return log_apfixed_reduce::logb(x);
1259 }
1260 template<int I>
1261 ap_uint<I> logb(ap_uint<I> x){
1262     return log_apfixed_reduce::logb(x);
1263 }
1264
1265 // sqrt(ap_fixed)

```

```

1266 template<int W, int I>
1267 ap_fixed<W,I> sqrt(ap_fixed<W,I> x){
1268     return sqrt_fixed(x);
1269 }
1270 template<int W, int I>
1271 ap_ufixed<W,I> sqrt(ap_ufixed<W,I> x){
1272     return sqrt_fixed(x);
1273 }
1274 template<int I>
1275 ap_int<I> sqrt(ap_int<I> x){
1276     return sqrt_fixed(x);
1277 }
1278 template<int I>
1279 ap_uint<I> sqrt(ap_uint<I> x){
1280     return sqrt_fixed(x);
1281 }
1282
1283 // rsqrt(ap_fixed)
1284 template<int W, int I>
1285 ap_fixed<W,I> rsqrt(ap_fixed<W,I> x){
1286     return rsqrt_fixed(x);
1287 }
1288 template<int W, int I>
1289 ap_ufixed<W,I> rsqrt(ap_ufixed<W,I> x){
1290     return rsqrt_fixed(x);
1291 }
1292 template<int I>
1293 ap_int<I> rsqrt(ap_int<I> x){
1294     return rsqrt_fixed(x);
1295 }
1296 template<int I>
1297 ap_uint<I> rsqrt(ap_uint<I> x){
1298     return rsqrt_fixed(x);
1299 }
1300
1301 // recip(ap_fixed)
1302 template<int W, int I>
1303 ap_fixed<W,I> recip(ap_fixed<W,I> x){
1304     return recip_fixed(x);
1305 }
1306 template<int W, int I>
1307 ap_ufixed<W,I> recip(ap_ufixed<W,I> x){
1308     return recip_fixed(x);
1309 }
1310 template<int I>
1311 ap_int<I> recip(ap_int<I> x){
1312     return recip_fixed(x);
1313 }
1314 template<int I>
1315 ap_uint<I> recip(ap_uint<I> x){
1316     return recip_fixed(x);
1317 }
1318
1319 // ceil(ap_fixed)
1320 template<int W, int I>
1321 ap_fixed<W,I> ceil(ap_fixed<W,I> x){
1322     return ceil_fixed(x);
1323 }
1324 template<int W, int I>
1325 ap_ufixed<W,I> ceil(ap_ufixed<W,I> x){
1326     return ceil_fixed(x);
1327 }
1328 template<int I>
1329 ap_int<I> ceil(ap_int<I> x){
1330     return ceil_fixed(x);
1331 }
1332 template<int I>
1333 ap_uint<I> ceil(ap_uint<I> x){
1334     return ceil_fixed(x);
1335 }
1336
1337 // floor(ap_fixed)
1338 template<int W, int I>
1339 ap_fixed<W,I> floor(ap_fixed<W,I> x){
1340     return floor_fixed(x);
1341 }
1342 template<int W, int I>
1343 ap_ufixed<W,I> floor(ap_ufixed<W,I> x){
1344     return floor_fixed(x);
1345 }
1346 template<int I>
1347 ap_int<I> floor(ap_int<I> x){
1348     return floor_fixed(x);
1349 }
1350 template<int I>
1351 ap_uint<I> floor(ap_uint<I> x){
1352     return floor_fixed(x);
1353 }
1354
1355 template<int W, int I>
1356 ap_fixed<W,I> trunc(ap_fixed<W,I> x){
1357     return trunc_fixed(x);
1358 }
1359 template<int W, int I>
1360 ap_ufixed<W,I> trunc(ap_ufixed<W,I> x){
1361     return trunc_fixed(x);
1362 }
1363 template<int I>
1364 ap_int<I> trunc(ap_int<I> x){
1365     return trunc_fixed(x);

```

```

1366 }
1367 template<int I>
1368 ap_uint<I> trunc(ap_uint<I> x){
1369     return trunc_fixed(x);
1370 }
1371 // copysign(ap_fixed)
1372 template<int W, int I>
1373 ap_fixed<W,I> copysign(ap_fixed<W,I> x, ap_fixed<W,I> y){
1374     return copysign_fixed(x,y);
1375 }
1376 template<int W, int I>
1377 ap_ufixed<W,I> copysign(ap_ufixed<W,I> x, ap_ufixed<W,I> y)
1378 {
1379     return copysign_fixed(x,y);
1380 }
1381 template<int I>
1382 ap_int<I> copysign(ap_int<I> x, ap_int<I> y){
1383     return copysign_fixed(x,y);
1384 }
1385 template<int I>
1386 ap_uint<I> copysign(ap_uint<I> x, ap_uint<I> y){
1387     return copysign_fixed(x,y);
1388 }
1389 // fabs(ap_fixed)
1390 template<int W, int I>
1391 ap_fixed<W,I> fabs(ap_fixed<W,I> x){
1392     return fabs_fixed(x);
1393 }
1394 template<int W, int I>
1395 ap_ufixed<W,I> fabs(ap_ufixed<W,I> x){
1396     return fabs_fixed(x);
1397 }
1398 template<int I>
1399 ap_int<I> fabs(ap_int<I> x){
1400     return fabs_fixed(x);
1401 }
1402 template<int I>
1403 ap_uint<I> fabs(ap_uint<I> x){
1404     return fabs_fixed(x);
1405 }
1406 // abs(ap_fixed)
1407 template<int W, int I>
1408 ap_fixed<W,I> abs(ap_fixed<W,I> x){
1409     return fabs_fixed(x);
1410 }
1411 template<int W, int I>
1412 ap_ufixed<W,I> abs(ap_ufixed<W,I> x){
1413     return fabs_fixed(x);
1414 }
1415 template<int I>
1416 ap_int<I> abs(ap_int<I> x){
1417     return fabs_fixed(x);
1418 }
1419 template<int I>
1420 ap_uint<I> abs(ap_uint<I> x){
1421     return fabs_fixed(x);
1422 }
1423 // fdim(ap_fixed)
1424 template<int W, int I>
1425 ap_fixed<W,I> fdim(ap_fixed<W,I> x, ap_fixed<W,I> y){
1426     return fdim_fixed(x,y);
1427 }
1428 template<int W, int I>
1429 ap_ufixed<W,I> fdim(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1430     return fdim_fixed(x,y);
1431 }
1432 template<int I>
1433 ap_int<I> fdim(ap_int<I> x, ap_int<I> y){
1434     return fdim_fixed(x,y);
1435 }
1436 template<int I>
1437 ap_uint<I> fdim(ap_uint<I> x, ap_uint<I> y){
1438     return fdim_fixed(x,y);
1439 }
1440 // fmax(ap_fixed)
1441 template<int W, int I>
1442 ap_fixed<W,I> fmax(ap_fixed<W,I> x, ap_fixed<W,I> y){
1443     return fmax_fixed(x,y);
1444 }
1445 template<int W, int I>
1446 ap_ufixed<W,I> fmax(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1447     return fmax_fixed(x,y);
1448 }
1449 template<int I>
1450 ap_int<I> fmax(ap_int<I> x, ap_int<I> y){
1451     return fmax_fixed(x,y);
1452 }
1453 template<int I>
1454 ap_uint<I> fmax(ap_uint<I> x, ap_uint<I> y){
1455     return fmax_fixed(x,y);
1456 }
1457 // fmin(ap_fixed)
1458 template<int W, int I>
1459 ap_fixed<W,I> fmin(ap_fixed<W,I> x, ap_fixed<W,I> y){
1460     return fmin_fixed(x,y);
1461 }
1462 template<int W, int I>

```

```

1465 ap_ufixed<W,I> fmin(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1466     return fmin_fixed(x,y);
1467 }
1468 template<int I>
1469 ap_int<I> fmin(ap_int<I> x, ap_int<I> y){
1470     return fmin_fixed(x,y);
1471 }
1472 template<int I>
1473 ap_uint<I> fmin(ap_uint<I> x, ap_uint<I> y){
1474     return fmin_fixed(x,y);
1475 }
1476 // maxmag(ap_fixed)
1477 template<int W, int I>
1478 ap_fixed<W,I> maxmag(ap_fixed<W,I> x, ap_fixed<W,I> y){
1479     return maxmag_fixed(x,y);
1480 }
1481 template<int W, int I>
1482 ap_ufixed<W,I> maxmag(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1483     return maxmag_fixed(x,y);
1484 }
1485 template<int I>
1486 ap_int<I> maxmag(ap_int<I> x, ap_int<I> y){
1487     return maxmag_fixed(x,y);
1488 }
1489 template<int I>
1490 ap_uint<I> maxmag(ap_uint<I> x, ap_uint<I> y){
1491     return maxmag_fixed(x,y);
1492 }
1493 // minmag(ap_fixed)
1494 template<int W, int I>
1495 ap_fixed<W,I> minmag(ap_fixed<W,I> x, ap_fixed<W,I> y){
1496     return minmag_fixed(x,y);
1497 }
1498 template<int W, int I>
1499 ap_ufixed<W,I> minmag(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1500     return minmag_fixed(x,y);
1501 }
1502 template<int I>
1503 ap_int<I> minmag(ap_int<I> x, ap_int<I> y){
1504     return minmag_fixed(x,y);
1505 }
1506 template<int I>
1507 ap_uint<I> minmag(ap_uint<I> x, ap_uint<I> y){
1508     return minmag_fixed(x,y);
1509 }
1510 // sinh(ap_fixed)
1511 template<int W, int I>
1512 ap_fixed<W,I> sinh(ap_fixed<W,I> x){
1513     return cordic_apfixed::generic_sinh(x);
1514 }
1515 // sinh(ap_ufixed)
1516 template<int W, int I>
1517 ap_ufixed<W,I> sinh(ap_ufixed<W,I> x){
1518     return cordic_apfixed::generic_sinh(x);
1519 }
1520 // sinh(ap_int)
1521 template<int I>
1522 ap_int<I> sinh(ap_int<I> x){
1523     return cordic_apfixed::generic_sinh(x);
1524 }
1525 // sinh(ap_uint)
1526 template<int I>
1527 ap_uint<I> sinh(ap_uint<I> x){
1528     return cordic_apfixed::generic_sinh(x);
1529 }
1530 // cosh(ap_fixed)
1531 template<int W, int I>
1532 ap_fixed<W,I> cosh(ap_fixed<W,I> x){
1533     return cordic_apfixed::generic_cosh(x);
1534 }
1535 // cosh(ap_ufixed)
1536 template<int W, int I>
1537 ap_ufixed<W,I> cosh(ap_ufixed<W,I> x){
1538     return cordic_apfixed::generic_cosh(x);
1539 }
1540 // cosh(ap_int)
1541 template<int I>
1542 ap_int<I> cosh(ap_int<I> x){
1543     return cordic_apfixed::generic_cosh(x);
1544 }
1545 // cosh(ap_uint)
1546 template<int I>
1547 ap_uint<I> cosh(ap_uint<I> x){
1548     return cordic_apfixed::generic_cosh(x);
1549 }
1550 // tanh(ap_fixed)
1551 template<int W, int I>
1552 ap_fixed<W,I> tanh(ap_fixed<W,I> x){
1553     return cordic_apfixed::generic_tanh(x);
1554 }

```

```

1565 // tanh(ap_ufixed)
1566 template<int W, int I>
1567 ap_ufixed<W,I> tanh(ap_ufixed<W,I> x){
1568     return cordic_apfixed::generic_tanh(x);
1569 }
1570
1571 // tanh(ap_int)
1572 template<int I>
1573 ap_int<I> tanh(ap_int<I> x){
1574     return cordic_apfixed::generic_tanh(x);
1575 }
1576
1577 // tanh(ap_uint)
1578 template<int I>
1579 ap_uint<I> tanh(ap_uint<I> x){
1580     return cordic_apfixed::generic_tanh(x);
1581 }
1582
1583 // atanh(ap_fixed)
1584 template<int W, int I>
1585 ap_fixed<W,I> atanh(ap_fixed<W,I> x){
1586     ap_fixed<W,I> one_plus_x = 1 + x;
1587     ap_fixed<W,I> one_minus_x = 1 - x;
1588     ap_fixed<W,I> input = one_plus_x / one_minus_x;
1589     ap_fixed<W,I> result = log_apfixed_reduce::log(input);
1590     result >>= I;
1591     return result;
1592 }
1593
1594 // atanh(ap_ufixed)
1595 template<int W, int I>
1596 ap_ufixed<W,I> atanh(ap_ufixed<W,I> x){
1597     ap_fixed<W+1,I+1> input = x;
1598     ap_fixed<W+1,I+1> result = hls::atanh(input);
1599     return result;
1600 }
1601
1602 // atanh(ap_int)
1603 template<int I>
1604 ap_int<I> atanh(ap_int<I> x){
1605     ap_fixed<I,I> input = x;
1606     ap_fixed<I,I> result = hls::atanh(input);
1607     return result;
1608 }
1609
1610 // atanh(ap_uint)
1611 template<int I>
1612 ap_uint<I> atanh(ap_uint<I> x){
1613     ap_fixed<I+1,I+1> input = x;
1614     ap_fixed<I+1,I+1> result = hls::atanh(input);
1615     return result;
1616 }
1617
1618 // asinh(ap_fixed)
1619 template<int W, int I>
1620 ap_fixed<W,I> asinh(ap_fixed<W,I> x){
1621     return cordic_apfixed::generic_asinh(x);
1622 }
1623
1624 // asinh(ap_ufixed)
1625 template<int W, int I>
1626 ap_ufixed<W,I> asinh(ap_ufixed<W,I> x){
1627     return cordic_apfixed::generic_asinh(x);
1628 }
1629
1630 // asinh(ap_int)
1631 template<int I>
1632 ap_int<I> asinh(ap_int<I> x){
1633     return cordic_apfixed::generic_asinh(x);
1634 }
1635
1636 // asinh(ap_uint)
1637 template<int I>
1638 ap_uint<I> asinh(ap_uint<I> x){
1639     return cordic_apfixed::generic_asinh(x);
1640 }
1641
1642 // acosh(ap_fixed)
1643 template<int W, int I>
1644 ap_fixed<W,I> acosh(ap_fixed<W,I> x){
1645     return cordic_apfixed::generic_acosh(x);
1646 }
1647
1648 // acosh(ap_ufixed)
1649 template<int W, int I>
1650 ap_ufixed<W,I> acosh(ap_ufixed<W,I> x){
1651     return cordic_apfixed::generic_acosh(x);
1652 }
1653
1654 // acosh(ap_int)
1655 template<int I>
1656 ap_int<I> acosh(ap_int<I> x){
1657     return cordic_apfixed::generic_acosh(x);
1658 }
1659
1660 // acosh(ap_uint)
1661 template<int I>
1662 ap_uint<I> acosh(ap_uint<I> x){
1663     return cordic_apfixed::generic_acosh(x);
1664 }
1665 }
1666
1667 int8_t sinh(int8_t);
1668 uint8_t sinh(uint8_t);
1669 int16_t sinh(int16_t);
1670 uint16_t sinh(uint16_t);
1671 int32_t sinh(int32_t);
1672 uint32_t sinh(uint32_t);
1673
1674 int8_t cosh(int8_t);
1675 uint8_t cosh(uint8_t);
1676 int16_t cosh(int16_t);
1677 uint16_t cosh(uint16_t);
1678 int32_t cosh(int32_t);
1679 uint32_t cosh(uint32_t);
1680
1681 int8_t tanh(int8_t);
1682 uint8_t tanh(uint8_t);
1683 int16_t tanh(int16_t);
1684 uint16_t tanh(uint16_t);
1685 int32_t tanh(int32_t);
1686 uint32_t tanh(uint32_t);
1687
1688 int8_t atanh(int8_t);
1689 uint8_t atanh(uint8_t);
1690 int16_t atanh(int16_t);
1691 uint16_t atanh(uint16_t);
1692 int32_t atanh(int32_t);
1693 uint32_t atanh(uint32_t);
1694
1695 int8_t asinh(int8_t);
1696 uint8_t asinh(uint8_t);
1697 int16_t asinh(int16_t);
1698 uint16_t asinh(uint16_t);
1699 int32_t asinh(int32_t);
1700 uint32_t asinh(uint32_t);
1701
1702 int8_t acosh(int8_t);
1703 uint8_t acosh(uint8_t);
1704 int16_t acosh(int16_t);
1705 uint16_t acosh(uint16_t);
1706 int32_t acosh(int32_t);
1707 uint32_t acosh(uint32_t);
1708
1709 // asin(ap_fixed)
1710 template<int W, int I>
1711 ap_fixed<W-I+3,3> asin(ap_fixed<W,I> x){
1712     return cordic_apfixed::generic_asin(x);
1713 }
1714
1715 // acos(ap_fixed)
1716 template<int W, int I>
1717 ap_fixed<W-I+3,3> acos(ap_fixed<W,I> x){
1718     return cordic_apfixed::generic_acos(x);
1719 }
1720
1721 // atan(ap_fixed)
1722 template<int W, int I>
1723 ap_fixed<W,2> atan(ap_fixed<W,I> x){
1724     return cordic_apfixed::generic_atan(x);
1725 }
1726
1727 // atan(ap_ufixed)
1728 template<int W, int I>
1729 ap_ufixed<W,2> atan(ap_ufixed<W,I> x){
1730     return cordic_apfixed::generic_atan(x);
1731 }
1732
1733 // atan(ap_int)
1734 template<int I>
1735 ap_int<2> atan(ap_int<I> x){
1736     return cordic_apfixed::generic_atan(x);
1737 }
1738
1739 // atan(ap_uint)
1740 template<int I>
1741 ap_uint<1> atan(ap_uint<I> x){
1742     return cordic_apfixed::generic_atan(x);
1743 }
1744
1745 int8_t asin(int8_t);
1746 uint8_t asin(uint8_t);
1747 int16_t asin(int16_t);
1748 uint16_t asin(uint16_t);
1749 int32_t asin(int32_t);
1750 uint32_t asin(uint32_t);
1751
1752 int8_t acos(int8_t);
1753 uint8_t acos(uint8_t);
1754 int16_t acos(int16_t);
1755 uint16_t acos(uint16_t);
1756 int32_t acos(int32_t);
1757 uint32_t acos(uint32_t);
1758
1759 int8_t atan(int8_t);
1760 uint8_t atan(uint8_t);
1761 int16_t atan(int16_t);
1762 uint16_t atan(uint16_t);
1763 int32_t atan(int32_t);
1764 uint32_t atan(uint32_t);

```

```

1765 // asinpi(ap_fixed)
1766 template<int W, int I>
1767 ap_fixed<W,I> asinpi(ap_fixed<W,I> x){
1768     return cordic_apfixed::generic_asinpi(x);
1769 }
1770 // asinpi(ap_fixed)
1771 template<int W, int I>
1772 ap_ufixed<W,I> asinpi(ap_ufixed<W,I> x){
1773     return cordic_apfixed::generic_asinpi(x);
1774 }
1775 // asinpi(ap_fixed)
1776 template<int I>
1777 ap_int<I> asinpi(ap_int<I> x){
1778     return cordic_apfixed::generic_asinpi(x);
1779 }
1780 // asinpi(ap_fixed)
1781 template<int I>
1782 ap_uint<I> asinpi(ap_uint<I> x){
1783     return cordic_apfixed::generic_asinpi(x);
1784 }
1785 }
1786
1787 int8_t asinpi(int8_t);
1788 uint8_t asinpi(uint8_t);
1789 int16_t asinpi(int16_t);
1790 uint16_t asinpi(uint16_t);
1791 int32_t asinpi(int32_t);
1792 uint32_t asinpi(uint32_t);
1793
1794 // acospi(ap_fixed)
1795 template<int W, int I>
1796 ap_fixed<W,I> acospi(ap_fixed<W,I> x){
1797     return cordic_apfixed::generic_acospi(x);
1798 }
1799 // acospi(ap_fixed)
1800 template<int W, int I>
1801 ap_ufixed<W,I> acospi(ap_ufixed<W,I> x){
1802     return cordic_apfixed::generic_acospi(x);
1803 }
1804 // acospi(ap_fixed)
1805 template<int I>
1806 ap_int<I> acospi(ap_int<I> x){
1807     return cordic_apfixed::generic_acospi(x);
1808 }
1809 // acospi(ap_fixed)
1810 template<int I>
1811 ap_uint<I> acospi(ap_uint<I> x){
1812     return cordic_apfixed::generic_acospi(x);
1813 }
1814
1815 int8_t acospi(int8_t);
1816 uint8_t acospi(uint8_t);
1817 int16_t acospi(int16_t);
1818 uint16_t acospi(uint16_t);
1819 int32_t acospi(int32_t);
1820 uint32_t acospi(uint32_t);
1821
1822 // atanpi(ap_fixed)
1823 template<int W, int I>
1824 ap_fixed<W,I> atanpi(ap_fixed<W,I> x){
1825     return cordic_apfixed::generic_atanpi(x);
1826 }
1827 // atanpi(ap_ufixed)
1828 template<int W, int I>
1829 ap_ufixed<W,I> atanpi(ap_ufixed<W,I> x){
1830     return cordic_apfixed::generic_atanpi(x);
1831 }
1832 // atanpi(ap_int)
1833 template<int I>
1834 ap_int<I> atanpi(ap_int<I> x){
1835     return cordic_apfixed::generic_atanpi(x);
1836 }
1837 // atanpi(ap_uint)
1838 template<int I>
1839 ap_uint<I> atanpi(ap_uint<I> x){
1840     return cordic_apfixed::generic_atanpi(x);
1841 }
1842
1843 int8_t atanpi(int8_t);
1844 uint8_t atanpi(uint8_t);
1845 int16_t atanpi(int16_t);
1846 uint16_t atanpi(uint16_t);
1847 int32_t atanpi(int32_t);
1848 uint32_t atanpi(uint32_t);
1849
1850 // atan2(ap_fixed)
1851 template<int W, int I>
1852 ap_fixed<W,3> atan2(ap_fixed<W,I> x, ap_fixed<W,I> y){
1853     return cordic_apfixed::generic_atan2(x,y);
1854 }
1855 // atan2(ap_ufixed)
1856 template<int W, int I>
1857 ap_ufixed<W,3> atan2(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1858     return cordic_apfixed::generic_atan2(x,y);
1859 }
1860 // atan2(ap_int)
1861 template<int I>
1862 ap_int<I> atan2(ap_int<I> x, ap_int<I> y){
1863     return cordic_apfixed::generic_atan2(x,y);
1864 }
1865 // atan2(ap_uint)
1866 template<int I>
1867 ap_uint<I> atan2(ap_uint<I> x, ap_uint<I> y){
1868     return cordic_apfixed::generic_atan2(x,y);
1869 }
1870
1871 int8_t atan2(int8_t, int8_t);
1872 uint8_t atan2(uint8_t, uint8_t);
1873 int16_t atan2(int16_t, int16_t);
1874 uint16_t atan2(uint16_t, uint16_t);
1875 int32_t atan2(int32_t, int32_t);
1876 uint32_t atan2(uint32_t, uint32_t);
1877
1878 // atan2pi(ap_fixed)
1879 template<int W, int I>
1880 ap_fixed<W,I> atan2pi(ap_fixed<W,I> x, ap_fixed<W,I> y){
1881     return cordic_apfixed::generic_atan2pi(x,y);
1882 }
1883 // atan2pi(ap_ufixed)
1884 template<int W, int I>
1885 ap_ufixed<W,I> atan2pi(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
1886     return cordic_apfixed::generic_atan2pi(x,y);
1887 }
1888 // atan2pi(ap_int)
1889 template<int I>
1890 ap_int<I> atan2pi(ap_int<I> x, ap_int<I> y){
1891     return cordic_apfixed::generic_atan2pi(x,y);
1892 }
1893 // atan2pi(ap_uint)
1894 template<int I>
1895 ap_uint<I> atan2pi(ap_uint<I> x, ap_uint<I> y){
1896     return cordic_apfixed::generic_atan2pi(x,y);
1897 }
1898
1899 int8_t atan2pi(int8_t, int8_t);
1900 uint8_t atan2pi(uint8_t, uint8_t);
1901 int16_t atan2pi(int16_t, int16_t);
1902 uint16_t atan2pi(uint16_t, uint16_t);
1903 int32_t atan2pi(int32_t, int32_t);
1904 uint32_t atan2pi(uint32_t, uint32_t);
1905
1906 template<int W, int I>
1907 void sincos(ap_fixed<W,I> in,
1908             ap_fixed<W-I+2,2>* outsin,
1909             ap_fixed<W-I+2,2>* outcos){
1910     cordic_apfixed::generic_sincos(in, *outsin, *outcos);
1911 }
1912
1913 template<int W, int I>
1914 ap_fixed<W-I+2,2> sin(ap_fixed<W,I> x){
1915     ap_fixed<W-I+2,2> outsin;
1916     ap_fixed<W-I+2,2> outcos;
1917     cordic_apfixed::generic_sincos(x, outsin, outcos);
1918     return outsin;
1919 }
1920
1921 template<int W, int I>
1922 ap_ufixed<W-I+2,2> sin(ap_ufixed<W,I> x){
1923     ap_fixed<W+1, I+1> xin = x;
1924     ap_fixed<W-I+3, 3> xout = hls::sin(x);
1925     ap_fixed<W-I+2, 2> out = 0;
1926     if(xout[W-I+2]==0){
1927         out=xout;
1928     }
1929     return out;
1930 }
1931
1932 template<int W, int I>
1933 ap_fixed<W-I+2,2> cos(ap_fixed<W,I> x){
1934     ap_fixed<W-I+2,2> outsin;
1935     ap_fixed<W-I+2,2> outcos;
1936     cordic_apfixed::generic_sincos(x, outsin, outcos);
1937     return outcos;
1938 }
1939
1940 template<int W, int I>
1941 ap_ufixed<W-I+2,2> cos(ap_ufixed<W,I> x){
1942     ap_fixed<W+1, I+1> xin = x;
1943     ap_fixed<W-I+3, 3> xout = hls::cos(x);
1944     ap_fixed<W-I+2, 2> out = 0;
1945     if(xout[W-I+2]==0){
1946         out=xout;
1947     }
1948     return out;
1949 }
1950
1951 template<int W, int I>
1952 ap_ufixed<W-I+2,2> cos(ap_ufixed<W,I> x){
1953     ap_fixed<W+1, I+1> xin = x;
1954     ap_fixed<W-I+3, 3> xout = hls::cos(x);
1955     ap_fixed<W-I+2, 2> out = 0;
1956     if(xout[W-I+2]==0){
1957         out=xout;
1958     }
1959     return out;
1960 }
1961
1962 /* template<int W, int I>
1963 ap_fixed<W-I+2,2> sinf(ap_fixed<W,I> x){
1964     ap_fixed<W-I+2,2> outsin;
1965     ap_fixed<W-I+2,2> outcos;
1966     cordic_apfixed::generic_sincos(x, outsin, outcos);
1967 }
1968 */

```

```

1964     return outsin;
1965 }
1966
1967 template<int W, int I>
1968 ap_fixed<W-I+2,2> cosf(ap_fixed<W,I> x){
1969     ap_fixed<W-I+2,2> outsin;
1970     ap_fixed<W-I+2,2> outcos;
1971     cordic_apfixed::generic_sincos(x, outsin, outcos);
1972     return outcos;
1973 }
1974 */
1975 // sin cos(ap_int)
1976 template<int I>
1977 ap_int<I> sin(ap_int<I> in) {
1978     _AP_UNUSED_PARAM(in);
1979     return ap_int<I>(0);
1980 };
1981 template<int I>
1982 ap_int<I> cos(ap_int<I> in) {
1983     _AP_UNUSED_PARAM(in);
1984     return ap_int<I>(0);
1985 };
1986 // sin cos(ap_uint)
1987 template<int I>
1988 ap_uint<I> sin(ap_uint<I> in) {
1989     _AP_UNUSED_PARAM(in);
1990     return ap_uint<I>(0);
1991 };
1992 template<int I>
1993 ap_uint<I> cos(ap_uint<I> in) {
1994     _AP_UNUSED_PARAM(in);
1995     return ap_uint<I>(0);
1996 };
1997
1998 void sincos(int8_t, int8_t*, int8_t*);
1999 void sincos(uint8_t, uint8_t*, uint8_t*);
2000 void sincos(int16_t, int16_t*, int16_t*);
2001 void sincos(uint16_t, uint16_t*, uint16_t*);
2002 void sincos(int32_t, int32_t*, int32_t*);
2003 void sincos(uint32_t, uint32_t*, uint32_t*);
2004
2005 int8_t sin(int8_t);
2006 uint8_t sin(uint8_t);
2007 int16_t sin(int16_t);
2008 uint16_t sin(uint16_t);
2009 int32_t sin(int32_t);
2010 uint32_t sin(uint32_t);
2011
2012 int8_t cos(int8_t);
2013 uint8_t cos(uint8_t);
2014 int16_t cos(int16_t);
2015 uint16_t cos(uint16_t);
2016 int32_t cos(int32_t);
2017 uint32_t cos(uint32_t);
2018
2019 // tan
2020 template<int W, int I>
2021 ap_fixed<W,I> tan(ap_fixed<W,I> x){
2022     return cordic_apfixed::generic_tan(x);
2023 }
2024 template<int W, int I>
2025 ap_ufixed<W,I> tan(ap_ufixed<W,I> x){
2026     return cordic_apfixed::generic_tan(x);
2027 }
2028 template<int I>
2029 ap_int<I> tan(ap_int<I> x){
2030     return cordic_apfixed::generic_tan(x);
2031 }
2032 template<int I>
2033 ap_uint<I> tan(ap_uint<I> x){
2034     return cordic_apfixed::generic_tan(x);
2035 }
2036
2037 int8_t tan(int8_t);
2038 uint8_t tan(uint8_t);
2039 int16_t tan(int16_t);
2040 uint16_t tan(uint16_t);
2041 int32_t tan(int32_t);
2042 uint32_t tan(uint32_t);
2043
2044 // tanpi
2045 template<int W, int I>
2046 ap_fixed<W,I> tanpi(ap_fixed<W,I> x){
2047     return cordic_apfixed::generic_tanpi(x);
2048 }
2049 template<int W, int I>
2050 ap_ufixed<W,I> tanpi(ap_ufixed<W,I> x){
2051     return cordic_apfixed::generic_tanpi(x);
2052 }
2053 template<int I>
2054 ap_int<I> tanpi(ap_int<I> x){
2055     return cordic_apfixed::generic_tanpi(x);
2056 }
2057 template<int I>
2058 ap_uint<I> tanpi(ap_uint<I> x){
2059     return cordic_apfixed::generic_tanpi(x);
2060 }
2061
2062 int8_t tanpi(int8_t);

```

```

2064 uint8_t tanpi(uint8_t);
2065 int16_t tanpi(int16_t);
2066 uint16_t tanpi(uint16_t);
2067 int32_t tanpi(int32_t);
2068 uint32_t tanpi(uint32_t);
2069
2070 //pow(ap_fixed)
2071 template<int W, int I>
2072 ap_fixed<W,I> pow(ap_fixed<W,I> x, ap_fixed<W,I> y) {
2073     return pow_apfixed_reduce::pow(x,y);
2074 }
2075
2076 template<int W, int I>
2077 ap_ufixed<W,I> pow(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2078     return pow_apfixed_reduce::pow(x,y);
2079 }
2080 template<int I>
2081 ap_int<I> pow(ap_int<I> x, ap_int<I> y){
2082     return pow_apfixed_reduce::pow(x,y);
2083 }
2084 template<int I>
2085 ap_uint<I> pow(ap_uint<I> x, ap_uint<I> y){
2086     return pow_apfixed_reduce::pow(x,y);
2087 }
2088 //pown(ap_fixed)
2089 template<int W, int I>
2090 ap_fixed<W,I> pown(ap_fixed<W,I> x, int n) {
2091     return pow_apfixed_reduce::pown(x,n);
2092 }
2093
2094 template<int W, int I>
2095 ap_ufixed<W,I> pown(ap_ufixed<W,I> x, int n){
2096     return pow_apfixed_reduce::pown(x,n);
2097 }
2098 template<int I>
2099 ap_int<I> pown(ap_int<I> x, int n){
2100     return pow_apfixed_reduce::pown(x,n);
2101 }
2102 template<int I>
2103 ap_uint<I> pown(ap_uint<I> x, int n){
2104     return pow_apfixed_reduce::pown(x,n);
2105 }
2106 int8_t pown(int8_t, int n);
2107 uint8_t pown(uint8_t, int n);
2108 int16_t pown(int16_t, int n);
2109 uint16_t pown(uint16_t, int n);
2110 int32_t pown(int32_t, int n);
2111 uint32_t pown(uint32_t, int n);
2112 //rootn(ap_fixed)
2113 template<int W, int I>
2114 ap_fixed<W,I> rootn(ap_fixed<W,I> x, int n) {
2115     return pow_apfixed_reduce::rootn(x,n);
2116 }
2117
2118 template<int W, int I>
2119 ap_ufixed<W,I> rootn(ap_ufixed<W,I> x, int n){
2120     return pow_apfixed_reduce::rootn(x,n);
2121 }
2122 template<int I>
2123 ap_int<I> rootn(ap_int<I> x, int n){
2124     return pow_apfixed_reduce::rootn(x,n);
2125 }
2126 template<int I>
2127 ap_uint<I> rootn(ap_uint<I> x, int n){
2128     return pow_apfixed_reduce::rootn(x,n);
2129 }
2130 int8_t rootn(int8_t, int n);
2131 uint8_t rootn(uint8_t, int n);
2132 int16_t rootn(int16_t, int n);
2133 uint16_t rootn(uint16_t, int n);
2134 int32_t rootn(int32_t, int n);
2135 uint32_t rootn(uint32_t, int n);
2136 //powr(ap_fixed)
2137 template<int W, int I>
2138 ap_fixed<W,I> powr(ap_fixed<W,I> x, ap_fixed<W,I> y) {
2139     return pow_apfixed_reduce::powr(x,y);
2140 }
2141
2142 template<int W, int I>
2143 ap_ufixed<W,I> powr(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2144     return pow_apfixed_reduce::powr(x,y);
2145 }
2146 template<int I>
2147 ap_int<I> powr(ap_int<I> x, ap_int<I> y){
2148     return pow_apfixed_reduce::powr(x,y);
2149 }
2150 template<int I>
2151 ap_uint<I> powr(ap_uint<I> x, ap_uint<I> y){
2152     return pow_apfixed_reduce::powr(x,y);
2153 }
2154
2155 // round(ap_fixed)
2156 template<int W, int I>
2157 ap_fixed<W,I> round(ap_fixed<W,I> x){
2158     return round_fixed(x);
2159 }
2160 template<int W, int I>
2161 ap_ufixed<W,I> round(ap_ufixed<W,I> x){
2162     return round_fixed(x);
2163 }

```



```

2164 }
2165 template<int I>
2166 ap_int<I> round(ap_int<I> x){
2167     return round_fixed(x);
2168 }
2169 template<int I>
2170 ap_uint<I> round(ap_uint<I> x){
2171     return round_fixed(x);
2172 }
2173 int8_t round(int8_t);
2174 uint8_t round(uint8_t);
2175 int16_t round(int16_t);
2176 uint16_t round(uint16_t);
2177 int32_t round(int32_t);
2178 uint32_t round(uint32_t);
2179
2180 // rint(ap_fixed)
2181 template<int W, int I>
2182 ap_fixed<W,I> rint(ap_fixed<W,I> x){
2183     return rint_fixed(x);
2184 }
2185 template<int W, int I>
2186 ap_ufixed<W,I> rint(ap_ufixed<W,I> x){
2187     return rint_fixed(x);
2188 }
2189 template<int I>
2190 ap_int<I> rint(ap_int<I> x){
2191     return rint_fixed(x);
2192 }
2193 template<int I>
2194 ap_uint<I> rint(ap_uint<I> x){
2195     return rint_fixed(x);
2196 }
2197 int8_t rint(int8_t);
2198 uint8_t rint(uint8_t);
2199 int16_t rint(int16_t);
2200 uint16_t rint(uint16_t);
2201 int32_t rint(int32_t);
2202 uint32_t rint(uint32_t);
2203
2204 // nearbyint(ap_fixed)
2205 template<int W, int I>
2206 ap_fixed<W,I> nearbyint(ap_fixed<W,I> x){
2207     return nearbyint_fixed(x);
2208 }
2209 template<int W, int I>
2210 ap_ufixed<W,I> nearbyint(ap_ufixed<W,I> x){
2211     return nearbyint_fixed(x);
2212 }
2213 template<int I>
2214 ap_int<I> nearbyint(ap_int<I> x){
2215     return nearbyint_fixed(x);
2216 }
2217 template<int I>
2218 ap_uint<I> nearbyint(ap_uint<I> x){
2219     return nearbyint_fixed(x);
2220 }
2221 int8_t nearbyint(int8_t);
2222 uint8_t nearbyint(uint8_t);
2223 int16_t nearbyint(int16_t);
2224 uint16_t nearbyint(uint16_t);
2225 int32_t nearbyint(int32_t);
2226 uint32_t nearbyint(uint32_t);
2227
2228 // llround(ap_fixed)
2229 template<int W, int I>
2230 long long int llround(ap_fixed<W,I> x){
2231     return llround_fixed(x);
2232 }
2233 template<int W, int I>
2234 long long int llround(ap_ufixed<W,I> x){
2235     return llround_fixed(x);
2236 }
2237 template<int I>
2238 long long int llround(ap_int<I> x){
2239     return llround_fixed(x);
2240 }
2241 template<int I>
2242 long long int llround(ap_uint<I> x){
2243     return llround_fixed(x);
2244 }
2245
2246 // llround(int)
2247 long long int llround(int8_t);
2248 long long int llround(uint8_t);
2249 long long int llround(int16_t);
2250 long long int llround(uint16_t);
2251 long long int llround(int32_t);
2252 long long int llround(uint32_t);
2253
2254 // lround(ap_fixed)
2255 template<int W, int I>
2256 long int lround(ap_fixed<W,I> x){
2257     return lround_fixed(x);
2258 }
2259 template<int W, int I>
2260 long int lround(ap_ufixed<W,I> x){
2261     return lround_fixed(x);
2262 }
2263 template<int I>

```

```

2264 long int lround(ap_int<I> x){
2265     return lround_fixed(x);
2266 }
2267 template<int I>
2268 long int lround(ap_uint<I> x){
2269     return lround_fixed(x);
2270 }
2271
2272 //lround(int)
2273 long int lround(int8_t);
2274 long int lround(uint8_t);
2275 long int lround(int16_t);
2276 long int lround(uint16_t);
2277 long int lround(int32_t);
2278 long int lround(uint32_t);
2279
2280 // llrint(ap_fixed)
2281 template<int W, int I>
2282 long long int llrint(ap_fixed<W,I> x){
2283     return llrint_fixed(x);
2284 }
2285 template<int W, int I>
2286 long long int llrint(ap_ufixed<W,I> x){
2287     return llrint_fixed(x);
2288 }
2289 template<int I>
2290 long long int llrint(ap_int<I> x){
2291     return llrint_fixed(x);
2292 }
2293 template<int I>
2294 long long int llrint(ap_uint<I> x){
2295     return llrint_fixed(x);
2296 }
2297
2298 //llrint(int)
2299 long long int llrint(int8_t);
2300 long long int llrint(uint8_t);
2301 long long int llrint(int16_t);
2302 long long int llrint(uint16_t);
2303 long long int llrint(int32_t);
2304 long long int llrint(uint32_t);
2305
2306 // lrint(ap_fixed)
2307 template<int W, int I>
2308 long int lrint(ap_fixed<W,I> x){
2309     return lrint_fixed(x);
2310 }
2311 template<int W, int I>
2312 long int lrint(ap_ufixed<W,I> x){
2313     return lrint_fixed(x);
2314 }
2315 template<int I>
2316 long int lrint(ap_int<I> x){
2317     return lrint_fixed(x);
2318 }
2319 template<int I>
2320 long int lrint(ap_uint<I> x){
2321     return lrint_fixed(x);
2322 }
2323
2324 //lrint(int)
2325 long int lrint(int8_t);
2326 long int lrint(uint8_t);
2327 long int lrint(int16_t);
2328 long int lrint(uint16_t);
2329 long int lrint(int32_t);
2330 long int lrint(uint32_t);
2331
2332 //isgreater(ap_fixed)
2333 template<int W, int I>
2334 bool isgreater(ap_fixed<W,I> x1, ap_fixed<W,I> x2){
2335     return isgreater_fixed(x1, x2);
2336 }
2337 template<int W, int I>
2338 bool isgreater(ap_ufixed<W,I> x1, ap_ufixed<W,I> x2){
2339     return isgreater_fixed(x1, x2);
2340 }
2341 template<int I>
2342 bool isgreater(ap_int<I> x1, ap_int<I> x2){
2343     return isgreater_fixed(x1, x2);
2344 }
2345 template<int I>
2346 bool isgreater(ap_uint<I> x1, ap_uint<I> x2){
2347     return isgreater_fixed(x1, x2);
2348 }
2349 bool isgreater(int8_t, int8_t);
2350 bool isgreater(uint8_t, uint8_t);
2351 bool isgreater(int16_t, int16_t);
2352 bool isgreater(uint16_t, uint16_t);
2353 bool isgreater(int32_t, int32_t);
2354 bool isgreater(uint32_t, uint32_t);
2355
2356 //isgreaterequal(ap_fixed)
2357 template<int W, int I>
2358 bool isgreaterequal(ap_fixed<W,I> x1, ap_fixed<W,I> x2){
2359     return isgreaterequal_fixed(x1, x2);
2360 }
2361 template<int W, int I>
2362 bool isgreaterequal(ap_ufixed<W,I> x1, ap_ufixed<W,I> x2){
2363     return isgreaterequal_fixed(x1, x2);

```

```

2364 }
2365 template<int I>
2366 bool isgreaterequal(ap_int<I> x1, ap_int<I> x2){
2367     return isgreaterequal_int(x1, x2);
2368 }
2369 template<int I>
2370 bool isgreaterequal(ap_uint<I> x1, ap_uint<I> x2){
2371     return isgreaterequal_fixed(x1, x2);
2372 }
2373 bool isgreaterequal(int8_t, int8_t);
2374 bool isgreaterequal(uint8_t, uint8_t);
2375 bool isgreaterequal(int16_t, int16_t);
2376 bool isgreaterequal(uint16_t, uint16_t);
2377 bool isgreaterequal(int32_t, int32_t);
2378 bool isgreaterequal(uint32_t, uint32_t);
2379
2380 //isless(ap_fixed)
2381 template<int W, int I>
2382 bool isless(ap_fixed<W,I> x1, ap_fixed<W,I> x2){
2383     return isless_fixed(x1, x2);
2384 }
2385 template<int W, int I>
2386 bool isless(ap_ufixed<W,I> x1, ap_ufixed<W,I> x2){
2387     return isless_fixed(x1, x2);
2388 }
2389 template<int I>
2390 bool isless(ap_int<I> x1, ap_int<I> x2){
2391     return isless_fixed(x1, x2);
2392 }
2393 template<int I>
2394 bool isless(ap_uint<I> x1, ap_uint<I> x2){
2395     return isless_fixed(x1, x2);
2396 }
2397 bool isless(int8_t, int8_t);
2398 bool isless(uint8_t, uint8_t);
2399 bool isless(int16_t, int16_t);
2400 bool isless(uint16_t, uint16_t);
2401 bool isless(int32_t, int32_t);
2402 bool isless(uint32_t, uint32_t);
2403
2404 //islessequal(ap_fixed)
2405 template<int W, int I>
2406 bool islessequal(ap_fixed<W,I> x1, ap_fixed<W,I> x2){
2407     return islessequal_fixed(x1, x2);
2408 }
2409 template<int W, int I>
2410 bool islessequal(ap_ufixed<W,I> x1, ap_ufixed<W,I> x2){
2411     return islessequal_fixed(x1, x2);
2412 }
2413 template<int I>
2414 bool islessequal(ap_int<I> x1, ap_int<I> x2){
2415     return islessequal_fixed(x1, x2);
2416 }
2417 template<int I>
2418 bool islessequal(ap_uint<I> x1, ap_uint<I> x2){
2419     return islessequal_fixed(x1, x2);
2420 }
2421 bool islessequal(int8_t, int8_t);
2422 bool islessequal(uint8_t, uint8_t);
2423 bool islessequal(int16_t, int16_t);
2424 bool islessequal(uint16_t, uint16_t);
2425 bool islessequal(int32_t, int32_t);
2426 bool islessequal(uint32_t, uint32_t);
2427
2428 //islessgreater(ap_fixed)
2429 template<int W, int I>
2430 bool islessgreater(ap_fixed<W,I> x1, ap_fixed<W,I> x2){
2431     return islessgreater_fixed(x1, x2);
2432 }
2433 template<int W, int I>
2434 bool islessgreater(ap_ufixed<W,I> x1, ap_ufixed<W,I> x2){
2435     return islessgreater_fixed(x1, x2);
2436 }
2437 template<int I>
2438 bool islessgreater(ap_int<I> x1, ap_int<I> x2){
2439     return islessgreater_fixed(x1, x2);
2440 }
2441 template<int I>
2442 bool islessgreater(ap_uint<I> x1, ap_uint<I> x2){
2443     return islessgreater_fixed(x1, x2);
2444 }
2445
2446 bool islessgreater(int8_t, int8_t);
2447 bool islessgreater(uint8_t, uint8_t);
2448 bool islessgreater(int16_t, int16_t);
2449 bool islessgreater(uint16_t, uint16_t);
2450 bool islessgreater(int32_t, int32_t);
2451 bool islessgreater(uint32_t, uint32_t);
2452
2453 int8_t frexp(int8_t, int8_t*);
2454 uint8_t frexp(uint8_t, uint8_t*);
2455 int16_t frexp(int16_t, int16_t*);
2456 uint16_t frexp(uint16_t, uint16_t*);
2457 int32_t frexp(int32_t, int32_t*);
2458 uint32_t frexp(uint32_t, uint32_t*);
2459 //frexp(ap_fixed)
2460 template<int W, int I>
2461 ap_fixed<W,I> frexp(ap_fixed<W,I> x, ap_fixed<W,I>* exp) {
2462     return frexp_internal::frexp(x,exp);
2463 }

```

```

2464 template<int W, int I>
2465 ap_ufixed<W,I> frexp(ap_ufixed<W,I> x, ap_ufixed<W,I>* exp)
2466 {
2467     return frexp_internal::frexp(x,exp);
2468 }
2469 template<int I>
2470 ap_int<I> frexp(ap_int<I> x, ap_int<I>* exp){
2471     return frexp_internal::frexp(x,exp);
2472 }
2473 template<int I>
2474 ap_uint<I> frexp(ap_uint<I> x, ap_uint<I>* exp){
2475     return frexp_internal::frexp(x,exp);
2476 }
2477
2478 int8_t modf(int8_t, int8_t*);
2479 uint8_t modf(uint8_t, uint8_t*);
2480 int16_t modf(int16_t, int16_t*);
2481 uint16_t modf(uint16_t, uint16_t*);
2482 int32_t modf(int32_t, int32_t*);
2483 uint32_t modf(uint32_t, uint32_t*);
2484 //modf(ap_fixed)
2485 template<int W, int I>
2486 ap_fixed<W,I> modf(ap_fixed<W,I> x, ap_fixed<W,I>* intpart)
2487 {
2488     return modf_internal::modf(x,intpart);
2489 }
2490 template<int W, int I>
2491 ap_ufixed<W,I> modf(ap_ufixed<W,I> x, ap_ufixed<W,I>* int-
2492 part){
2493     return modf_internal::modf(x,intpart);
2494 }
2495 template<int I>
2496 ap_int<I> modf(ap_int<I> x, ap_int<I>* intpart){
2497     return modf_internal::modf(x,intpart);
2498 }
2499 template<int I>
2500 ap_uint<I> modf(ap_uint<I> x, ap_uint<I>* intpart){
2501     return modf_internal::modf(x,intpart);
2502 }
2503 // isequal(int)
2504 bool isequal(int8_t, int8_t);
2505 bool isequal(uint8_t, uint8_t);
2506 bool isequal(int16_t, int16_t);
2507 bool isequal(uint16_t, uint16_t);
2508 bool isequal(int32_t, int32_t);
2509 bool isequal(uint32_t, uint32_t);
2510 // isnotequal(int)
2511 bool isnotequal(int8_t, int8_t);
2512 bool isnotequal(uint8_t, uint8_t);
2513 bool isnotequal(int16_t, int16_t);
2514 bool isnotequal(uint16_t, uint16_t);
2515 bool isnotequal(int32_t, int32_t);
2516 bool isnotequal(uint32_t, uint32_t);
2517 // isequal(ap_fixed)
2518 template<int W, int I>
2519 bool isequal(ap_fixed<W,I> x, ap_fixed<W,I> y){
2520     return generic_isequal(x,y);
2521 }
2522 template<int W, int I>
2523 bool isequal(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2524     return generic_isequal(x,y);
2525 }
2526 template<int I>
2527 bool isequal(ap_int<I> x, ap_int<I> y){
2528     return generic_isequal(x,y);
2529 }
2530 template<int I>
2531 bool isequal(ap_uint<I> x, ap_uint<I> y){
2532     return generic_isequal(x,y);
2533 }
2534 // isnotequal(ap_fixed)
2535 template<int W, int I>
2536 bool isnotequal(ap_fixed<W,I> x, ap_fixed<W,I> y){
2537     return generic_isnotequal(x,y);
2538 }
2539 template<int W, int I>
2540 bool isnotequal(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2541     return generic_isnotequal(x,y);
2542 }
2543 template<int I>
2544 bool isnotequal(ap_int<I> x, ap_int<I> y){
2545     return generic_isnotequal(x,y);
2546 }
2547 template<int I>
2548 bool isnotequal(ap_uint<I> x, ap_uint<I> y){
2549     return generic_isnotequal(x,y);
2550 }
2551 // any(ap_fixed)
2552 template<int W, int I>
2553 bool any(ap_fixed<W,I> x){
2554     return generic_any(x);
2555 }
2556 template<int W, int I>
2557 bool any(ap_ufixed<W,I> x){
2558     return generic_any(x);
2559 }

```



```

2560 template<int I>
2561 bool any(ap_int<I> x){
2562     return generic_any(x);
2563 };
2564 template<int I>
2565 bool any(ap_uint<I> x){
2566     return generic_any(x);
2567 };
2568 // all(ap_fixed)
2569 template<int W,int I>
2570 bool all(ap_fixed<W,I> x){
2571     return generic_all(x);
2572 };
2573 template<int W,int I>
2574 bool all(ap_ufixed<W,I> x){
2575     return generic_all(x);
2576 };
2577 template<int I>
2578 bool all(ap_int<I> x){
2579     return generic_all(x);
2580 };
2581 template<int I>
2582 bool all(ap_uint<I> x){
2583     return generic_all(x);
2584 };
2585 //bitselect(ap_fixed)
2586 template<int W,int I>
2587 ap_fixed<W,I> bitselect(ap_fixed<W,I> x,ap_fixed<W,I> y,
2588 ap_fixed<W,I> z){
2589     return generic_bitselect(x,y,z);
2590 };
2591 template<int W,int I>
2592 ap_ufixed<W,I> bitselect(ap_ufixed<W,I> x,ap_ufixed<W,I>
2593 y,ap_ufixed<W,I> z){
2594     return generic_bitselect(x,y,z);
2595 };
2596 template<int I>
2597 ap_int<I> bitselect(ap_int<I> x,ap_int<I> y,ap_int<I> z){
2598     return generic_bitselect(x,y,z);
2599 };
2600 template<int I>
2601 ap_uint<I> bitselect(ap_uint<I> x,ap_uint<I> y,ap_uint<I>
2602 z){
2603     return generic_bitselect(x,y,z);
2604 };
2605 //erf(ap_fixed)
2606 template<int W, int I>
2607 ap_fixed<W,I> erf(ap_fixed<W,I> x){
2608     return erf_erfc_fixed::erf(x);
2609 };
2610 template<int W, int I>
2611 ap_ufixed<W,I> erf(ap_ufixed<W,I> x){
2612     return erf_erfc_fixed::erf(x);
2613 };
2614 template<int I>
2615 ap_int<I> erf(ap_int<I> x){
2616     return erf_erfc_fixed::erf(x);
2617 };
2618 template<int I>
2619 ap_uint<I> erf(ap_uint<I> x){
2620     return erf_erfc_fixed::erf(x);
2621 };
2622 //erfc(ap_fixed)
2623 template<int W, int I>
2624 ap_fixed<W,I> erfc(ap_fixed<W,I> x){
2625     return erf_erfc_fixed::erfc(x);
2626 };
2627 template<int W, int I>
2628 ap_ufixed<W,I> erfc(ap_ufixed<W,I> x){
2629     return erf_erfc_fixed::erfc(x);
2630 };
2631 template<int I>
2632 ap_int<I> erfc(ap_int<I> x){
2633     return erf_erfc_fixed::erfc(x);
2634 };
2635 template<int I>
2636 ap_uint<I> erfc(ap_uint<I> x){
2637     return erf_erfc_fixed::erfc(x);
2638 };
2639 // divide (int)
2640 int8_t divide(int8_t,int8_t);
2641 int16_t divide(int16_t,int16_t);
2642 int32_t divide(int32_t,int32_t);
2643 uint8_t divide(uint8_t,uint8_t);
2644 uint16_t divide(uint16_t,uint16_t);
2645 uint32_t divide(uint32_t,uint32_t);
2646 //divide(ap_fixed)
2647 template<int W, int I>
2648 ap_fixed<W,I> divide(ap_fixed<W,I> x, ap_fixed<W,I> y){
2649     return hls_internal::generic_divide(x, y);
2650 };
2651 template<int W, int I>
2652 ap_ufixed<W,I> divide(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2653     return hls_internal::generic_divide(x, y);
2654 };
2655 template<int I>
2656 ap_int<I> divide(ap_int<I> x, ap_int<I> y){
2657     return hls_internal::generic_divide(x, y);
2658 };
2659 template<int I>
2660 ap_uint<I> divide(ap_uint<I> x, ap_uint<I> y){
2661     return hls_internal::generic_divide(x, y);
2662 };
2663 //fmod(int)
2664 int8_t fmod(int8_t,int8_t);
2665 int16_t fmod(int16_t,int16_t);
2666 int32_t fmod(int32_t,int32_t);
2667 uint8_t fmod(uint8_t,uint8_t);
2668 uint16_t fmod(uint16_t,uint16_t);
2669 uint32_t fmod(uint32_t,uint32_t);
2670 //fmod(ap_fixed)
2671 template<int W, int I>
2672 ap_fixed<W,I> fmod(ap_fixed<W,I> x, ap_fixed<W,I> y){
2673     return hls_internal::generic_fmod(x, y);
2674 };
2675 template<int W, int I>
2676 ap_ufixed<W,I> fmod(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2677     return hls_internal::generic_fmod(x, y);
2678 };
2679 template<int I>
2680 ap_int<I> fmod(ap_int<I> x, ap_int<I> y){
2681     return hls_internal::generic_fmod(x, y);
2682 };
2683 template<int I>
2684 ap_uint<I> fmod(ap_uint<I> x, ap_uint<I> y){
2685     return hls_internal::generic_fmod(x, y);
2686 };
2687 //remainder (int)
2688 int8_t remainder(int8_t,int8_t);
2689 int16_t remainder(int16_t,int16_t);
2690 int32_t remainder(int32_t,int32_t);
2691 uint8_t remainder(uint8_t,uint8_t);
2692 uint16_t remainder(uint16_t,uint16_t);
2693 uint32_t remainder(uint32_t,uint32_t);
2694 //remainder(ap_fixed)
2695 template<int W, int I>
2696 ap_fixed<W,I> remainder(ap_fixed<W,I> x, ap_fixed<W,I> y){
2697     return hls_internal::generic_remainder(x, y);
2698 };
2699 template<int W, int I>
2700 ap_ufixed<W,I> remainder(ap_ufixed<W,I> x, ap_ufixed<W,I>
2701 y){
2702     return hls_internal::generic_remainder(x, y);
2703 };
2704 template<int I>
2705 ap_int<I> remainder(ap_int<I> x, ap_int<I> y){
2706     return hls_internal::generic_remainder(x, y);
2707 };
2708 template<int I>
2709 ap_uint<I> remainder(ap_uint<I> x, ap_uint<I> y){
2710     return hls_internal::generic_remainder(x, y);
2711 };
2712 //remquo (int)
2713 int8_t remquo(int8_t,int8_t,int*);
2714 int16_t remquo(int16_t,int16_t,int*);
2715 int32_t remquo(int32_t,int32_t,int*);
2716 uint8_t remquo(uint8_t,uint8_t,int*);
2717 uint16_t remquo(uint16_t,uint16_t,int*);
2718 uint32_t remquo(uint32_t,uint32_t,int*);
2719 //remquo(ap_fixed)
2720 template<int W, int I>
2721 ap_fixed<W,I> remquo(ap_fixed<W,I> x, ap_fixed<W,I> y, int*
2722 quo){
2723     return hls_internal::generic_remquo(x, y, quo);
2724 };
2725 template<int W, int I>
2726 ap_ufixed<W,I> remquo(ap_ufixed<W,I> x, ap_ufixed<W,I> y,
2727 int* quo){
2728     return hls_internal::generic_remquo(x, y, quo);
2729 };
2730 template<int I>
2731 ap_int<I> remquo(ap_int<I> x, ap_int<I> y, int* quo){
2732     return hls_internal::generic_remquo(x, y, quo);
2733 };
2734 template<int I>
2735 ap_uint<I> remquo(ap_uint<I> x, ap_uint<I> y, int* quo){
2736     return hls_internal::generic_remquo(x, y, quo);
2737 };
2738 //signbit(ap_fixed)
2739 template<int W, int I>
2740 bool signbit(ap_fixed<W,I> x) {
2741     return signbit_fixed(x);
2742 };
2743 template<int W, int I>
2744 bool signbit(ap_ufixed<W,I> x) {
2745     return signbit_fixed(x);
2746 };
2747 template<int I>
2748 bool signbit(ap_int<I> x) {
2749     return signbit_fixed(x);
2750 };
2751 template<int I>
2752 bool signbit(ap_uint<I> x) {
2753     return signbit_fixed(x);
2754 };

```

```

2657 ap_uint<I> divide(ap_uint<I> x, ap_uint<I> y){
2658     return hls_internal::generic_divide(x, y);
2659 };
2660 //fmod(int)
2661 int8_t fmod(int8_t,int8_t);
2662 int16_t fmod(int16_t,int16_t);
2663 int32_t fmod(int32_t,int32_t);
2664 uint8_t fmod(uint8_t,uint8_t);
2665 uint16_t fmod(uint16_t,uint16_t);
2666 uint32_t fmod(uint32_t,uint32_t);
2667 //fmod(ap_fixed)
2668 template<int W, int I>
2669 ap_fixed<W,I> fmod(ap_fixed<W,I> x, ap_fixed<W,I> y){
2670     return hls_internal::generic_fmod(x, y);
2671 };
2672 template<int W, int I>
2673 ap_ufixed<W,I> fmod(ap_ufixed<W,I> x, ap_ufixed<W,I> y){
2674     return hls_internal::generic_fmod(x, y);
2675 };
2676 template<int I>
2677 ap_int<I> fmod(ap_int<I> x, ap_int<I> y){
2678     return hls_internal::generic_fmod(x, y);
2679 };
2680 template<int I>
2681 ap_uint<I> fmod(ap_uint<I> x, ap_uint<I> y){
2682     return hls_internal::generic_fmod(x, y);
2683 };
2684 //remainder (int)
2685 int8_t remainder(int8_t,int8_t);
2686 int16_t remainder(int16_t,int16_t);
2687 int32_t remainder(int32_t,int32_t);
2688 uint8_t remainder(uint8_t,uint8_t);
2689 uint16_t remainder(uint16_t,uint16_t);
2690 uint32_t remainder(uint32_t,uint32_t);
2691 //remainder(ap_fixed)
2692 template<int W, int I>
2693 ap_fixed<W,I> remainder(ap_fixed<W,I> x, ap_fixed<W,I> y){
2694     return hls_internal::generic_remainder(x, y);
2695 };
2696 template<int W, int I>
2697 ap_ufixed<W,I> remainder(ap_ufixed<W,I> x, ap_ufixed<W,I>
2698 y){
2699     return hls_internal::generic_remainder(x, y);
2700 };
2701 template<int I>
2702 ap_int<I> remainder(ap_int<I> x, ap_int<I> y){
2703     return hls_internal::generic_remainder(x, y);
2704 };
2705 template<int I>
2706 ap_uint<I> remainder(ap_uint<I> x, ap_uint<I> y){
2707     return hls_internal::generic_remainder(x, y);
2708 };
2709 //remquo (int)
2710 int8_t remquo(int8_t,int8_t,int*);
2711 int16_t remquo(int16_t,int16_t,int*);
2712 int32_t remquo(int32_t,int32_t,int*);
2713 uint8_t remquo(uint8_t,uint8_t,int*);
2714 uint16_t remquo(uint16_t,uint16_t,int*);
2715 uint32_t remquo(uint32_t,uint32_t,int*);
2716 //remquo(ap_fixed)
2717 template<int W, int I>
2718 ap_fixed<W,I> remquo(ap_fixed<W,I> x, ap_fixed<W,I> y, int*
2719 quo){
2720     return hls_internal::generic_remquo(x, y, quo);
2721 };
2722 template<int W, int I>
2723 ap_ufixed<W,I> remquo(ap_ufixed<W,I> x, ap_ufixed<W,I> y,
2724 int* quo){
2725     return hls_internal::generic_remquo(x, y, quo);
2726 };
2727 template<int I>
2728 ap_int<I> remquo(ap_int<I> x, ap_int<I> y, int* quo){
2729     return hls_internal::generic_remquo(x, y, quo);
2730 };
2731 template<int I>
2732 ap_uint<I> remquo(ap_uint<I> x, ap_uint<I> y, int* quo){
2733     return hls_internal::generic_remquo(x, y, quo);
2734 };
2735 //signbit(ap_fixed)
2736 template<int W, int I>
2737 bool signbit(ap_fixed<W,I> x) {
2738     return signbit_fixed(x);
2739 };
2740 template<int W, int I>
2741 bool signbit(ap_ufixed<W,I> x) {
2742     return signbit_fixed(x);
2743 };
2744 template<int I>
2745 bool signbit(ap_int<I> x) {
2746     return signbit_fixed(x);
2747 };
2748 template<int I>
2749 bool signbit(ap_uint<I> x) {
2750     return signbit_fixed(x);
2751 };
2752 bool signbit(int8_t);
2753 bool signbit(int16_t);
2754 bool signbit(int32_t);
2755 bool signbit(uint8_t);
2756 bool signbit(uint16_t);
2757 bool signbit(uint32_t);

```

```

2753 bool signbit(int32_t);
2754 bool signbit(uint32_t);
2755
2756 //lgamma_r(ap_fixed)
2757 template<int W, int I>
2758 ap_fixed<W,I> lgamma_r(ap_fixed<W,I> x, int *signgamp){
2759     return lgamma_fixed::lgamma_r_generic(x, signgamp);
2760 }
2761 template<int W, int I>
2762 ap_ufixed<W,I> lgamma_r(ap_ufixed<W,I> x, int *signgamp){
2763     return lgamma_fixed::lgamma_r_generic(x, signgamp);
2764 }
2765 template<int I>
2766 ap_int<I> lgamma_r(ap_int<I> x, int *signgamp){
2767     return lgamma_fixed::lgamma_r_generic(x, signgamp);
2768 }
2769 template<int I>
2770 ap_uint<I> lgamma_r(ap_uint<I> x, int *signgamp){
2771     return lgamma_fixed::lgamma_r_generic(x, signgamp);
2772 }
2773 //lgamma(ap_fixed)
2774 template<int W, int I>
2775 ap_fixed<W,I> lgamma(ap_fixed<W,I> x){
2776     return lgamma_fixed::lgamma_generic(x);
2777 }
2778 template<int W, int I>
2779 ap_ufixed<W,I> lgamma(ap_ufixed<W,I> x){
2780     return lgamma_fixed::lgamma_generic(x);
2781 }
2782 template<int I>
2783 ap_int<I> lgamma(ap_int<I> x){
2784     return lgamma_fixed::lgamma_generic(x);
2785 }
2786 template<int I>
2787 ap_uint<I> lgamma(ap_uint<I> x){
2788     return lgamma_fixed::lgamma_generic(x);
2789 }
2790 //mad (int)
2791 int8_t mad(int8_t,int8_t,int8_t);
2792 int16_t mad(int16_t,int16_t,int16_t);
2793 int32_t mad(int32_t,int32_t,int32_t);
2794 uint8_t mad(uint8_t,uint8_t,uint8_t);
2795 uint16_t mad(uint16_t,uint16_t,uint16_t);
2796 uint32_t mad(uint32_t,uint32_t,uint32_t);
2797 //mad (ap_fixed)
2798 template<int W, int I>
2799 ap_fixed<W,I> mad(ap_fixed<W,I> x,ap_fixed<W,I>
y,ap_fixed<W,I> z){
2800     return hls_mad::generic_mad(x,y,z);
2801 }
2802 template<int W, int I>
2803 ap_ufixed<W,I> mad(ap_ufixed<W,I> x,ap_ufixed<W,I>
y,ap_ufixed<W,I> z){
2804     return hls_mad::generic_mad(x,y,z);
2805 }
2806 template<int I>
2807 ap_int<I> mad(ap_int<I> x,ap_int<I> y,ap_int<I> z){
2808     return hls_mad::generic_mad(x,y,z);
2809 }
2810 template<int I>
2811 ap_uint<I> mad(ap_uint<I> x,ap_uint<I> y,ap_uint<I> z){
2812     return hls_mad::generic_mad(x,y,z);
2813 }
2814 //fma (int)
2815 int8_t fma(int8_t,int8_t,int8_t);
2816 int16_t fma(int16_t,int16_t,int16_t);
2817 int32_t fma(int32_t,int32_t,int32_t);
2818 uint8_t fma(uint8_t,uint8_t,uint8_t);
2819 uint16_t fma(uint16_t,uint16_t,uint16_t);
2820 uint32_t fma(uint32_t,uint32_t,uint32_t);
2821 //fma (ap_fixed)
2822 template<int W, int I>
2823 ap_fixed<W,I> fma(ap_fixed<W,I> x,ap_fixed<W,I>
y,ap_fixed<W,I> z){
2824     return hls_mad::generic_mad(x,y,z);
2825 }
2826 template<int W, int I>
2827 ap_ufixed<W,I> fma(ap_ufixed<W,I> x,ap_ufixed<W,I>
y,ap_ufixed<W,I> z){
2828     return hls_mad::generic_mad(x,y,z);
2829 }
2830 template<int I>
2831 ap_int<I> fma(ap_int<I> x,ap_int<I> y,ap_int<I> z){
2832     return hls_mad::generic_mad(x,y,z);
2833 }
2834 template<int I>
2835 ap_uint<I> fma(ap_uint<I> x,ap_uint<I> y,ap_uint<I> z){
2836     return hls_mad::generic_mad(x,y,z);
2837 }
2838 //fract (int)
2839 int8_t fract(int8_t);
2840 int16_t fract(int16_t);
2841 int32_t fract(int32_t);
2842 uint8_t fract(uint8_t);
2843 uint16_t fract(uint16_t);
2844 uint32_t fract(uint32_t);
2845 //fract(ap_fixed)
2846 template<int W, int I>
2847 ap_fixed<W,I> fract(ap_fixed<W,I> x){
2848     return hls_fract::generic_fract(x);

```

```

2849 }
2850 template<int W, int I>
2851 ap_ufixed<W,I> fract(ap_ufixed<W,I> x){
2852     return hls_fract::generic_fract(x);
2853 }
2854 template<int I>
2855 ap_int<I> fract(ap_int<I> x){
2856     return hls_fract::generic_fract(x);
2857 }
2858 template<int I>
2859 ap_uint<I> fract(ap_uint<I> x){
2860     return hls_fract::generic_fract(x);
2861 }
2862 //cbrt for fixed data type
2863 template<int W, int I>
2864 ap_fixed<W,I> cbrt(ap_fixed<W,I> x) {
2865     return cbrt_internal::cbrt_fixed(x);
2866 }
2867 template<int W, int I>
2868 ap_ufixed<W,I> cbrt(ap_ufixed<W,I> x) {
2869     return cbrt_internal::cbrt_fixed(x);
2870 }
2871 template<int I>
2872 ap_int<I> cbrt(ap_int<I> x) {
2873     return cbrt_internal::cbrt_fixed(x);
2874 }
2875 template<int I>
2876 ap_uint<I> cbrt(ap_uint<I> x) {
2877     return cbrt_internal::cbrt_fixed(x);
2878 }
2879 int8_t cbrt(int8_t);
2880 uint8_t cbrt(uint8_t);
2881 int16_t cbrt(int16_t);
2882 uint16_t cbrt(uint16_t);
2883 int32_t cbrt(int32_t);
2884 uint32_t cbrt(uint32_t);
2885
2886 //tgamma(ap_fixed)
2887 template<int W, int I>
2888 ap_fixed<W,I> tgamma(ap_fixed<W,I> x){
2889     return tgamma_fixed::tgamma_generic(x);
2890 }
2891 template<int W, int I>
2892 ap_ufixed<W,I> tgamma(ap_ufixed<W,I> x){
2893     return tgamma_fixed::tgamma_generic(x);
2894 }
2895 template<int I>
2896 ap_int<I> tgamma(ap_int<I> x){
2897     return tgamma_fixed::tgamma_generic(x);
2898 }
2899 template<int I>
2900 ap_uint<I> tgamma(ap_uint<I> x){
2901     return tgamma_fixed::tgamma_generic(x);
2902 }
2903
2904 //nextafter(ap_fixed)
2905 template<int W, int I>
2906 ap_fixed<W,I> nextafter(ap_fixed<W,I> x, ap_fixed<W,I> y)
{
2907     return fp_internal::nextafter(x, y);
2908 }
2909 template<int W, int I>
2910 ap_ufixed<W,I> nextafter(ap_ufixed<W,I> x, ap_ufixed<W,I>
y) {
2911     return fp_internal::nextafter(x, y);
2912 }
2913 template<int I>
2914 ap_int<I> nextafter(ap_int<I> x, ap_int<I> y) {
2915     return fp_internal::nextafter(x, y);
2916 }
2917 template<int I>
2918 ap_uint<I> nextafter(ap_uint<I> x, ap_uint<I> y) {
2919     return fp_internal::nextafter(x, y);
2920 }
2921 int8_t nextafter(int8_t, int8_t);
2922 uint8_t nextafter(uint8_t, uint8_t);
2923 int16_t nextafter(int16_t, int16_t);
2924 uint16_t nextafter(uint16_t, uint16_t);
2925 int32_t nextafter(int32_t, int32_t);
2926 uint32_t nextafter(uint32_t, uint32_t);
2927
2928 //nexttoward(ap_fixed)
2929 template<int W, int I>
2930 ap_fixed<W,I> nexttoward(ap_fixed<W,I> x, ap_fixed<W,I>
y) {
2931     return fp_internal::nexttoward(x, y);
2932 }
2933 template<int W, int I>
2934 ap_ufixed<W,I> nexttoward(ap_ufixed<W,I> x, ap_ufixed<W,I>
I> y) {
2935     return fp_internal::nexttoward(x, y);
2936 }
2937 template<int I>
2938 ap_int<I> nexttoward(ap_int<I> x, ap_int<I> y) {
2939     return fp_internal::nexttoward(x, y);
2940 }
2941 template<int I>
2942 ap_uint<I> nexttoward(ap_uint<I> x, ap_uint<I> y) {

```

```

2943     return fp_internal::nexttoward(x, y);
2944 }
2945 int8_t nexttoward(int8_t, int8_t);
2946 uint8_t nexttoward(uint8_t, uint8_t);
2947 int16_t nexttoward(int16_t, int16_t);
2948 uint16_t nexttoward(uint16_t, uint16_t);
2949 int32_t nexttoward(int32_t, int32_t);
2950 uint32_t nexttoward(uint32_t, uint32_t);
2951
2952 //hypot for fixed data type
2953 template<int W, int I>
2954 ap_fixed<W,I> hypot(ap_fixed<W,I> x, ap_fixed<W,I> y) {
2955     return hypot_internal::hypot_fixed(x,y);
2956 }
2957 template<int W, int I>
2958 ap_ufixed<W,I> hypot(ap_ufixed<W,I> x, ap_ufixed<W,I> y) {
2959     return hypot_internal::hypot_fixed(x,y);
2960 }
2961 template<int I>
2962 ap_int<I> hypot(ap_int<I> x, ap_int<I> y) {
2963     return hypot_internal::hypot_fixed(x,y);
2964 }
2965 template<int I>
2966 ap_uint<I> hypot(ap_uint<I> x, ap_uint<I> y) {
2967     return hypot_internal::hypot_fixed(x,y);
2968 }
2969 int8_t hypot(int8_t, int8_t);
2970 uint8_t hypot(uint8_t, uint8_t);
2971 int16_t hypot(int16_t, int16_t);
2972 uint16_t hypot(uint16_t, uint16_t);
2973 int32_t hypot(int32_t, int32_t);
2974 uint32_t hypot(uint32_t, uint32_t);
2975
2976 template<int W, int I>
2977 ap_fixed<W,I> exp2(ap_fixed<W,I> x) {
2978     return exp_reduce::exp2(x);
2979 }
2980
2981 template<int W, int I>
2982 ap_ufixed<W,I> exp2(ap_ufixed<W,I> x) {
2983     return exp_reduce::exp2(x);
2984 }
2985
2986 template<int I>
2987 ap_int<I> exp2(ap_int<I> x) {
2988     return exp_reduce::exp2(x);
2989 }
2990
2991 template<int I>
2992 ap_uint<I> exp2(ap_uint<I> x) {
2993     return exp_reduce::exp2(x);
2994 }
2995
2996 template<int W, int I>
2997 ap_fixed<W,I> exp10(ap_fixed<W,I> x) {
2998     return exp_reduce::exp10(x);
2999 }
3000
3001 template<int W, int I>
3002 ap_ufixed<W,I> exp10(ap_ufixed<W,I> x) {
3003     return exp_reduce::exp10(x);
3004 }
3005
3006 template<int I>
3007 ap_int<I> exp10(ap_int<I> x) {
3008     return exp_reduce::exp10(x);
3009 }
3010
3011 template<int I>
3012 ap_uint<I> exp10(ap_uint<I> x) {
3013     return exp_reduce::exp10(x);
3014 }
3015
3016 template<int W, int I>
3017 ap_fixed<W,I> expm1(ap_fixed<W,I> x) {
3018     return exp_reduce::expm1(x);
3019 }
3020
3021 template<int W, int I>
3022 ap_ufixed<W,I> expm1(ap_ufixed<W,I> x) {
3023     return exp_reduce::expm1(x);
3024 }
3025
3026 template<int I>
3027 ap_int<I> expm1(ap_int<I> x) {
3028     return exp_reduce::expm1(x);
3029 }
3030
3031 template<int I>
3032 ap_uint<I> expm1(ap_uint<I> x) {
3033     return exp_reduce::expm1(x);
3034 }
3035
3036 template<int W, int I>
3037 ap_fixed<W,I> log1p(ap_fixed<W,I> x) {
3038     return log_apfixed_reduce::log1p(x);
3039 }
3040
3041 template<int W, int I>
3042 ap_ufixed<W,I> log1p(ap_ufixed<W,I> x) {
3043     return log_apfixed_reduce::log1p(x);
3044 }
3045
3046 template<int I>
3047 ap_int<I> log1p(ap_int<I> x) {
3048     return log_apfixed_reduce::log1p(x);
3049 }
3050
3051 template<int I>
3052 ap_uint<I> log1p(ap_uint<I> x) {
3053     return log_apfixed_reduce::log1p(x);
3054 }
3055
3056 template<int W, int I>
3057 ap_fixed<W,I> ilogb(ap_fixed<W,I> x) {
3058     return log_apfixed_reduce::ilogb(x);
3059 }
3060
3061 template<int W, int I>
3062 ap_ufixed<W,I> ilogb(ap_ufixed<W,I> x) {
3063     return log_apfixed_reduce::ilogb(x);
3064 }
3065
3066 template<int I>
3067 ap_int<I> ilogb(ap_int<I> x) {
3068     return log_apfixed_reduce::ilogb(x);
3069 }
3070
3071 template<int I>
3072 ap_uint<I> ilogb(ap_uint<I> x) {
3073     return log_apfixed_reduce::ilogb(x);
3074 }
3075
3076 template<int W, int I>
3077 ap_fixed<W,I> ldexp(ap_fixed<W,I> x, int exp) {
3078     return hls_ldexp::ldexp(x, exp);
3079 }
3080
3081 template<int W, int I>
3082 ap_ufixed<W,I> ldexp(ap_ufixed<W,I> x, int exp) {
3083     return hls_ldexp::ldexp(x, exp);
3084 }
3085
3086 template<int I>
3087 ap_int<I> ldexp(ap_int<I> x, int exp) {
3088     return hls_ldexp::ldexp(x, exp);
3089 }
3090
3091 template<int I>
3092 ap_uint<I> ldexp(ap_uint<I> x, int exp) {
3093     return hls_ldexp::ldexp(x, exp);
3094 }
3095
3096 template<int W, int I>
3097 ap_fixed<W,I> scalbn(ap_fixed<W,I> x, int n) {
3098     return hls_ldexp::scalbn(x, n);
3099 }
3100
3101 template<int W, int I>
3102 ap_ufixed<W,I> scalbn(ap_ufixed<W,I> x, int n) {
3103     return hls_ldexp::scalbn(x, n);
3104 }
3105
3106 template<int I>
3107 ap_int<I> scalbn(ap_int<I> x, int n) {
3108     return hls_ldexp::scalbn(x, n);
3109 }
3110
3111 template<int I>
3112 ap_uint<I> scalbn(ap_uint<I> x, int n) {
3113     return hls_ldexp::scalbn(x, n);
3114 }
3115
3116 template<int W, int I>
3117 ap_fixed<W,I> scalbln(ap_fixed<W,I> x, long int n) {
3118     return hls_ldexp::scalbln(x, n);
3119 }
3120
3121 template<int W, int I>
3122 ap_ufixed<W,I> scalbln(ap_ufixed<W,I> x, long int n) {
3123     return hls_ldexp::scalbln(x, n);
3124 }
3125
3126 template<int I>
3127 ap_int<I> scalbln(ap_int<I> x, long int n) {
3128     return hls_ldexp::scalbln(x, n);
3129 }
3130
3131 template<int I>
3132 ap_uint<I> scalbln(ap_uint<I> x, long int n) {
3133     return hls_ldexp::scalbln(x, n);
3134 }
3135
3136 int8_t ldexp(int8_t x, int exp);
3137 int16_t ldexp(int16_t x, int exp);
3138 int32_t ldexp(int32_t x, int exp);
3139 uint8_t ldexp(uint8_t x, int exp);
3140 uint16_t ldexp(uint16_t x, int exp);
3141 uint32_t ldexp(uint32_t x, int exp);

```

```

3143 int8_t scalbn(int8_t x, int n);
3144 int16_t scalbn(int16_t x, int n);
3145 int32_t scalbn(int32_t x, int n);
3146 uint8_t scalbn(uint8_t x, int n);
3147 uint16_t scalbn(uint16_t x, int n);
3148 uint32_t scalbn(uint32_t x, int n);
3149
3150 int8_t scalbln(int8_t x, int n);
3151 int16_t scalbln(int16_t x, int n);
3152 int32_t scalbln(int32_t x, int n);
3153 uint8_t scalbln(uint8_t x, int n);
3154 uint16_t scalbln(uint16_t x, int n);
3155 uint32_t scalbln(uint32_t x, int n);
3156
3157 int8_t exp2(int8_t x);
3158 int16_t exp2(int16_t x);
3159 int32_t exp2(int32_t x);
3160 uint8_t exp2(uint8_t x);
3161 uint16_t exp2(uint16_t x);
3162 uint32_t exp2(uint32_t x);
3163
3164 int8_t exp10(int8_t x);
3165 int16_t exp10(int16_t x);
3166 int32_t exp10(int32_t x);
3167 uint8_t exp10(uint8_t x);
3168 uint16_t exp10(uint16_t x);
3169 uint32_t exp10(uint32_t x);
3170
3171 int8_t expm1(int8_t x);
3172 int16_t expm1(int16_t x);
3173 int32_t expm1(int32_t x);
3174 uint8_t expm1(uint8_t x);
3175 uint16_t expm1(uint16_t x);
3176 uint32_t expm1(uint32_t x);
3177
3178 int8_t ilogb(int8_t x);
3179 int16_t ilogb(int16_t x);
3180 int32_t ilogb(int32_t x);
3181 uint8_t ilogb(uint8_t x);
3182 uint16_t ilogb(uint16_t x);
3183 uint32_t ilogb(uint32_t x);
3184
3185 int8_t log1p(int8_t x);
3186 int16_t log1p(int16_t x);
3187 int32_t log1p(int32_t x);
3188 uint8_t log1p(uint8_t x);
3189 uint16_t log1p(uint16_t x);
3190 uint32_t log1p(uint32_t x);
3191
3192 int8_t log2(int8_t x);
3193 uint8_t log2(uint8_t x);
3194 int16_t log2(int16_t x);
3195 uint16_t log2(uint16_t x);
3196 int32_t log2(int32_t x);
3197 uint32_t log2(uint32_t x);
3198
3199 int8_t logb(int8_t x);
3200 uint8_t logb(uint8_t x);
3201 int16_t logb(int16_t x);
3202 uint16_t logb(uint16_t x);
3203 int32_t logb(int32_t x);
3204 uint32_t logb(uint32_t x);
3205
3206 };
3207 #endif
3208
3209

```

E.45 vendor/hls_stream.h (530 lines, 13,032 B)

```

1 // (c) Copyright 2011-2022 Xilinx, Inc.
2 // All Rights Reserved.
3 //
4 // Licensed to the Apache Software Foundation (ASF) under one
5 // or more contributor license agreements. See the NOTICE
file
6 // distributed with this work for additional information
7 // regarding copyright ownership. The ASF licenses this file
8 // to you under the Apache License, Version 2.0 (the
9 // "License"); you may not use this file except in compliance
10 // with the License. You may obtain a copy of the License at
11 //
12 // http://www.apache.org/licenses/LICENSE-2.0
13 //
14 // Unless required by applicable law or agreed to in writing,
15 // software distributed under the License is distributed on
an
16 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 // KIND, either express or implied. See the License for the
18 // specific language governing permissions and limitations
19 // under the License.
20
21 #ifndef X_HLS_STREAM_SIM_H
22 #define X_HLS_STREAM_SIM_H
23
24 /*
25 * This file contains a C++ model of hls::stream.
26 * It defines C simulation model.
27 */
28 #ifndef __cplusplus

```

```

29
30 #error C++ is required to include this header file
31
32 #else
33
34 // C level simulation models for hls::stream
35 // C level simulation models for hls::stream
36 // C level simulation models for hls::stream
37 #include <queue>
38 #include <iostream>
39 #include <typeinfo>
40 #include <string>
41 #include <sstream>
42 #include <unordered_map>
43 #include <cstring>
44 #include <array>
45 #include <limits>
46 #include <thread>
47 #include <unistd.h>
48
49 #include <mutex>
50 #include <atomic>
51 #include <condition_variable>
52
53 #ifndef _MSC_VER
54 #include <cxxabi.h>
55 #include <stdlib.h>
56 #endif
57
58 namespace hls {
59 #if !defined(__HLS_COSIM__) && defined(__VITIS_HLS__)
60 // We are in bcsim mode, where reads must be non-blocking
61 #define ALLOW_EMPTY_HLS_STREAM_READS
62 #ifdef X_HLS_TASK_H
63 #error "bcsim is not supported with hls::tasks"
64 #endif
65 #endif
66
67 template<size_t SIZE>
68 class stream_delegate {
69 public:
70     virtual bool read(void *elem) = 0;
71     virtual void write(const void *elem) = 0;
72     virtual bool read_nb(void *elem) = 0;
73     virtual size_t size() = 0;
74 };
75
76 class stream_globals {
77 public:
78     static void print_max_size() {
79         std::cout << "INFO [HLS SIM]: The maximum depth reached
by any hls::stream() instance in the design is " << get_max_size()
<< std::endl;
80     }
81
82     static void incr_blocked_counter() {
83         get_blocked_counter()++;
84     }
85
86     static void decr_blocked_counter() {
87         get_blocked_counter()--;
88     }
89
90     static void incr_task_counter() {
91         get_task_counter()++;
92     }
93
94     static void decr_task_counter() {
95         get_task_counter()--;
96     }
97
98     static void start_threads() {
99         // These initializations must be in ONE static function
that is called elsewhere
100 #if defined(__HLS_COSIM__)
101     static std::thread t(deadlock_thread);
102 #endif
103     static std::atomic_flag init_done = ATOMIC_FLAG_INIT;
104     if (!init_done.test_and_set()) {
105         // Perform global initialization actions once
106         // Register function executed at exit
107         std::atexit(print_max_size);
108     }
109 #if defined(__HLS_COSIM__)
110     // Detach the thread to avoid error at end with un-
waited thread
111     t.detach();
112 #endif
113 }
114
115 static std::atomic<int> &get_max_size() {
116     static std::atomic<int> max_size(0);
117
118     return max_size;
119 }
120
121 private:
122     static bool check_deadlock() {
123

```

```

124 // Check that it is larger than, because the testbench main
thread is not counted.
125     return get_blocked_counter() > get_task_counter();
126 }
127
128 #ifdef HLS_STREAM_THREAD_SAFE
129     static std::mutex &get_mutex() {
130         static std::mutex mutex;
131
132         return mutex;
133     }
134 #endif
135
136     static void deadlock_thread() {
137         while (1) {
138             sleep(1);
139             if (stream_globals::check_deadlock()) {
140                 if (get_task_counter()) {
141                     std::cout << "ERROR [HLS SIM]: deadlock detected
when simulating hls::tasks."
142                     << std::endl;
143                     std::cout << "Execute C simulation in debug mode
in the GUI and examine the"
144                     << " source code location of all the
blocked hls::stream::read()"
145                     << " calls to debug." << std::endl;
146                 } else {
147                     std::cout << "ERROR [HLS SIM]: an hls::stream is
read while empty,"
148                     << " which may result in RTL simulation
hanging." << std::endl;
149                     std::cout << "If this is not expected, execute C
simulation in debug mode in"
150                     << " the GUI and examine the source code
location of the blocked"
151                     << " hls::stream::read() call to debug."
<< std::endl;
152                     std::cout << "If this is expected, add -DAL-
LOW_EMPTY_HLS_STREAM_READS"
153                     << " to -cflags to turn this error into a
warning and allow empty"
154                     << " hls::stream reads to return the de-
fault value for the data type."
155                     << std::endl;
156                 }
157                 abort();
158             }
159         }
160     }
161
162     static std::atomic<int> &get_task_counter() {
163         static std::atomic<int> task_counter(0);
164
165         return task_counter;
166     }
167
168     static std::atomic<int> &get_blocked_counter() {
169         static std::atomic<int> blocked_counter(0);
170
171         return blocked_counter;
172     }
173 };
174
175 template<size_t SIZE>
176 class stream_entity {
177 public:
178     #ifdef HLS_STREAM_THREAD_SAFE
179     stream_entity() : d(0), invalid(false) {}
180     ~stream_entity() {
181         std::unique_lock<std::mutex> ul(mutex);
182         invalid = true;
183         condition_var.notify_all();
184     }
185     #else
186     stream_entity() : d(0) {}
187     #endif
188
189     bool read(void *elem) {
190         if (d)
191             return d->read(elem);
192
193     #ifdef HLS_STREAM_THREAD_SAFE
194         std::unique_lock<std::mutex> ul(mutex);
195     #endif
196     // needed to start the deadlock detector and size report-
er
197     stream_globals::start_threads();
198
199     if (data.empty()) {
200     #ifdef ALLOW_EMPTY_HLS_STREAM_READS
201         std::cout << "WARNING [HLS SIM]: hls::stream '"
202         << name
203         << "' is read while empty,"
204         << " which may result in RTL simulation
hanging."
205         << std::endl;
206         return false;
207     #else
208         stream_globals::incr_blocked_counter();

```

```

209         while (data.empty()) {
210     #ifdef HLS_STREAM_THREAD_SAFE
211             while (invalid) { sleep(1); }
212             condition_var.wait(ul);
213     #endif
214         }
215         stream_globals::decr_blocked_counter();
216     #endif
217     }
218     std::array<char, SIZE> &elem_data = data.front();
219     memcpy(elem, elem_data.data(), SIZE);
220     data.pop_front();
221     return true;
222 }
223
224 void write(const void *elem) {
225     if (d) {
226         d->write(elem);
227         return;
228     }
229
230     std::array<char, SIZE> elem_data;
231     memcpy(elem_data.data(), elem, SIZE);
232
233     #ifdef HLS_STREAM_THREAD_SAFE
234     std::unique_lock<std::mutex> ul(mutex);
235     #endif
236     data.push_back(elem_data);
237
238     // needed to start the deadlock detector and size report-
er
239     stream_globals::start_threads();
240
241     if (stream_globals::get_max_size() < data.size())
242         stream_globals::get_max_size() = data.size();
243     #ifdef HLS_STREAM_THREAD_SAFE
244     condition_var.notify_one();
245     #endif
246 }
247
248 /// Nonblocking read
249 bool read_nb(void *elem) {
250     if (d)
251         return d->read_nb(elem);
252
253     #ifdef HLS_STREAM_THREAD_SAFE
254     std::lock_guard<std::mutex> lg(mutex);
255     #endif
256     bool is_empty = data.empty();
257     if (!is_empty) {
258         std::array<char, SIZE> &elem_data = data.front();
259         memcpy(elem, elem_data.data(), SIZE);
260         data.pop_front();
261     }
262     return !is_empty;
263 }
264
265 /// Fifo size
266 size_t size() {
267     if (d)
268         return d->size();
269
270     #ifdef HLS_STREAM_THREAD_SAFE
271     std::lock_guard<std::mutex> lg(mutex);
272     #endif
273     return data.size();
274 }
275
276 /// Set name for c-sim debugging.
277 void set_name(const char *n) {
278     #ifdef HLS_STREAM_THREAD_SAFE
279     std::lock_guard<std::mutex> lg(mutex);
280     #endif
281     name = n;
282 }
283
284 stream_delegate<SIZE> *d;
285 std::string name;
286 std::deque<std::array<char, SIZE> > data;
287 #ifdef HLS_STREAM_THREAD_SAFE
288 std::mutex mutex;
289 std::condition_variable condition_var;
290 bool invalid;
291 #endif
292 };
293
294 template<size_t SIZE>
295 class stream_map {
296 public:
297     static size_t count(void *p) {
298     #ifdef HLS_STREAM_THREAD_SAFE
299         std::lock_guard<std::mutex> lg(get_mutex());
300     #endif
301         return get_map().count(p);
302     }
303
304     static void insert(void *p) {
305     #ifdef HLS_STREAM_THREAD_SAFE
306         std::lock_guard<std::mutex> lg(get_mutex());
307     #endif

```



```

308     auto &map = get_map();
309     map.erase(p);
310     map[p];
311 }
312
313 static stream_entity<SIZE> &get_entity(void *p) {
314 #ifdef HLS_STREAM_THREAD_SAFE
315     std::lock_guard<std::mutex> lg(get_mutex());
316 #endif
317     return get_map()[p];
318 }
319
320 private:
321 #ifdef HLS_STREAM_THREAD_SAFE
322     static std::mutex &get_mutex() {
323         static std::mutex *mutex = new std::mutex();
324         return *mutex;
325     }
326 #endif
327     static std::unordered_map<void*, stream_entity<SIZE> >
&get_map() {
328         static std::unordered_map<void*, stream_entity<SIZE> >
*map =
329             new std::unordered_map<void*, stream_entity<SIZE>
>();
330         return *map;
331     }
332 };
333
334 template<typename __STREAM_T__, int DEPTH=0>
335 class stream;
336 template<typename __STREAM_T__>
337 class stream<__STREAM_T__, 0>
338 {
339 public:
340     using value_type = __STREAM_T__;
341
342 private:
343     typedef stream_map<sizeof(__STREAM_T__)> map_t;
344
345 protected:
346 #if defined(__VITIS_HLS__)
347     __STREAM_T__ _data;
348 #else
349     stream_entity<sizeof(__STREAM_T__)> _data;
350 #endif
351
352 protected:
353 public:
354     /// Constructors
355     // Keep consistent with the synthesis model's construct-
ors
356     stream() {
357         std::stringstream ss;
358 #ifndef _MSC_VER
359         char* _demangle_name =
abi::__cxa_demangle(typeid(*this).name(), 0, 0, 0);
360         if (_demangle_name) {
361             ss << _demangle_name;
362             free(_demangle_name);
363         }
364         else {
365             ss << "hls_stream";
366         }
367     #else
368         ss << typeid(*this).name();
369     #endif
370
371 #ifdef HLS_STREAM_THREAD_SAFE
372         static std::atomic<unsigned> counter(0);
373     #else
374         static unsigned counter = 0;
375     #endif
376
377 #if defined(__VITIS_HLS__)
378         map_t::insert(&_data);
379     #endif
380         ss << counter++;
381         get_entity().set_name(ss.str().c_str());
382     }
383
384     stream(const char *name) {
385         // default constructor,
386         // capacity set to predefined maximum
387     #if defined(__VITIS_HLS__)
388         map_t::insert(&_data);
389     #endif
390         get_entity().set_name(name);
391     }
392
393     /// Make copy constructor and assignment operator private
394     /// They should not be called.
395 private:
396     stream(const stream<__STREAM_T__ >& chn):
397         _data(chn._data) {}
398
399     stream& operator = (const stream<__STREAM_T__ >& chn) {
400         return *this;
401     }
402 }

```

```

403     stream_entity<sizeof(__STREAM_T__)> &get_entity() {
404 #if defined(__VITIS_HLS__)
405         return map_t::get_entity(&_data);
406 #else
407         return _data;
408 #endif
409 }
410
411 public:
412     /// Overload >> and << operators to implement read() and
write()
413     void operator >> (__STREAM_T__ &rdata) {
414         read(rdata);
415     }
416
417     void operator << (const __STREAM_T__ &wdata) {
418         write(wdata);
419     }
420
421 public:
422     /// Destructor
423     /// Check status of the queue
424     ~stream() {
425         if (!empty())
426         {
427             std::cout << "WARNING [HLS SIM]: hls::stream '"
428                 << get_entity().name
429                 << "' contains leftover data,"
430                 << " which may result in RTL simulation
431                 << std::endl;
432             hanging."
433         }
434     }
435
436 #if defined(__VITIS_HLS__)
437     bool exist() {
438         return map_t::count(&_data);
439     }
440 #endif
441
442     /// Status of the queue
443     bool empty() {
444         return size() == 0;
445     }
446
447     bool full() const { return false; }
448
449     /// Blocking read
450     void read(__STREAM_T__ &head) {
451         head = read();
452     }
453
454     /// Blocking read
455     bool read_dep(__STREAM_T__ &head, volatile bool flag) {
456         head = read();
457         return flag;
458     }
459
460     __STREAM_T__ read() {
461         __STREAM_T__ elem;
462         auto &entity = get_entity();
463         if (!entity.read(&elem))
464             elem = __STREAM_T__();
465         return elem;
466     }
467
468     /// Blocking write
469     void write(const __STREAM_T__ &tail) {
470         get_entity().write(&tail);
471     }
472
473     /// Blocking write
474     bool write_dep(const __STREAM_T__ &tail, volatile bool
flag) {
475         write(tail);
476         return flag;
477     }
478
479     /// Nonblocking read
480     bool read_nb(__STREAM_T__ &head) {
481         __STREAM_T__ elem;
482         auto &entity = get_entity();
483         bool not_empty = entity.read_nb(&elem);
484         if (not_empty)
485             head = elem;
486         return not_empty;
487     }
488
489     /// Nonblocking write
490     bool write_nb(const __STREAM_T__ &tail) {
491         bool is_full = full();
492         write(tail);
493         return !is_full;
494     }
495
496     /// Fifo size
497     size_t size() {
498         return get_entity().size();
499     }

```

```

500     }
501
502     /// Fifo capacity
503     size_t capacity() {
504         // actually no limit on simulation model
505         return std::numeric_limits<std::size_t>::max();
506     }
507
508     /// Set name for c-sim debugging.
509     void set_name(const char *name) {
510         get_entity().set_name(name);
511     }
512
513     void set_delegate(stream_delegate<sizeof(__STREAM_T__)>
514 *d) {
515         get_entity().d = d;
516     }
517
518 template<typename __STREAM_T__, int DEPTH>
519 class stream : public stream<__STREAM_T__, 0> {
520 public:
521     stream() {}
522     stream(const char* name) : stream<__STREAM_T__, 0>(name) {}
523 };
524
525 } // namespace hls
526
527 #endif // __cplusplus
528 #endif // X_HLS_STREAM_H
529
530

```

E.46 vendor/hls_x_complex.h (374 lines, 9,874 B)

```

1 // Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
2 // Copyright 2022-2025 Advanced Micro Devices, Inc. All
Rights Reserved.
3
4 // 67d7842db-
be25473c3c32b93c0da8047785f30d78e8a024de1b57352245f9689
5
6 #ifndef HLS_X_COMPLEX_H
7 #define HLS_X_COMPLEX_H
8
9 #ifndef __SYNTHESIS__
10 #include <iostream>
11 #endif
12
13 #include "ap_fixed.h"
14
15 namespace std {
16 template <typename _Tp> class complex;
17 }
18
19 namespace hls {
20
21 /*
22
23 *****
24 * Complex class
25 *
26 * This is used over std::complex for the following reasons.
27 * 1. <complex> auto initializes all variables which forces an
28 * auto-initialization loop to be inserted for every declara-
29 * tion of an array of
30 * complex data
31 * 2. Using std::complex as base class leads to compilation
32 * errors.
33 *****
34 */
35
36 template <typename T> class x_complex {
37 private:
38     T re, im;
39
40 public:
41     // Multiply return type
42     typedef x_complex<T> MULT_RT;
43     // default constructors
44     inline x_complex() = default;
45     inline ~x_complex() = default;
46     inline x_complex(x_complex<T> &&) = default;
47     inline x_complex(const x_complex<T> &z) = default;
48     inline x_complex<T> &operator=(x_complex<T> &&) = default;
49     inline x_complex<T> &operator=(const x_complex<T> &) = de-
fault;
50
51     // other constructors
52     inline x_complex(const T &r, const T &i) : re(r), im(i){};
53
54     inline x_complex(const T &r) : re(r), im(0){};
55
56     inline x_complex(const std::complex<T> &z) {
57         x_complex<T> tmp(z.real(), z.imag());
58         *this = tmp;
59     }
60

```

```

56 template <typename T2> inline x_complex(const x_complex<T2>
&z) {
57     x_complex<T2> tmp(z);
58     x_complex<T> tmp2(tmp.real(), tmp.imag());
59     *this = tmp2;
60 }
61
62 // Data access
63 inline T real() const { return re; };
64 inline T &real() { return re; };
65
66 inline void real(const T &r) { re = r; };
67
68 inline T imag() const { return im; };
69 inline T &imag() { return im; };
70
71 inline void imag(const T &i) { im = i; };
72
73 // Operator =
74 inline x_complex<T> &operator=(const T &rhs) {
75     x_complex<T> tmp(rhs, 0);
76     *this = tmp;
77     return *this;
78 };
79
80 template <typename T2>
81 inline x_complex<T> &operator=(const x_complex<T2> &rhs) {
82     x_complex<T> tmp(rhs);
83     *this = tmp;
84     return *this;
85 };
86
87 // Operator *=, *
88 inline x_complex<T> &operator*=(const T &rhs) {
89     x_complex<T> tmp(*this);
90     tmp.re *= rhs;
91     tmp.im *= rhs;
92     *this = tmp;
93     return *this;
94 };
95
96 inline x_complex<T> &operator*=(const x_complex<T> &rhs) {
97     x_complex<T> tmp_rhs(rhs);
98     x_complex<T> tmp_this(*this);
99     T tmp1 = tmp_this.re * tmp_rhs.real();
100     T tmp2 = tmp_this.im * tmp_rhs.imag();
101     T tmp3 = tmp_this.re * tmp_rhs.imag();
102     T tmp4 = tmp_this.im * tmp_rhs.real();
103     x_complex<T> tmp5(tmp1 - tmp2, tmp3 + tmp4);
104     *this = tmp5;
105     return *this;
106 };
107
108 template <typename T2>
109 inline x_complex<T> &operator*=(const x_complex<T2> &rhs) {
110     x_complex<T2> tmp(rhs);
111     x_complex<T> tmp2(tmp.real(), tmp.imag());
112     *this *= tmp2;
113     return *this;
114 };
115
116 inline MULT_RT operator*(const T &rhs) {
117     x_complex<T> tmp(*this);
118     tmp *= rhs;
119     return tmp;
120 };
121
122 inline MULT_RT operator*(const x_complex<T> &rhs) {
123     x_complex<T> tmp(*this);
124     tmp *= rhs;
125     return tmp;
126 };
127
128 template <typename T2> inline MULT_RT operator*(const
x_complex<T2> &rhs) {
129     x_complex<T2> tmp(rhs);
130     x_complex<T> tmp2(tmp.real(), tmp.imag());
131     MULT_RT res = *this * tmp2;
132     return res;
133 };
134
135 // Operator /=, /
136 inline x_complex<T> &operator/=(const T &rhs) {
137     x_complex<T> tmp(*this);
138     tmp.re /= rhs;
139     tmp.im /= rhs;
140     *this = tmp;
141     return *this;
142 };
143
144 inline x_complex<T> &operator/=(const x_complex<T> &rhs) {
145     x_complex<T> tmp(rhs);
146     x_complex<T> conj(tmp.real(), -tmp.imag());
147     x_complex<T> a = (*this) * conj;
148     x_complex<T> b = conj * rhs;
149     x_complex<T> tmp2(a.real() / b.real(), a.imag() /
b.imag());
150     *this = tmp2;
151     return *this;
152 };
153
154 template <typename T2>
155 inline x_complex<T> &operator/=(const x_complex<T2> &rhs) {
156     x_complex<T2> tmp2(rhs);
157     x_complex<T> tmp(tmp2.real(), tmp2.imag());
158     *this /= tmp;
159 }

```



```

153     return *this;
154 };
155
156 inline x_complex<T> operator/(const T &rhs) {
157     x_complex<T> tmp(*this);
158     tmp /= rhs;
159     return tmp;
160 };
161 inline x_complex<T> operator/(const x_complex<T> &rhs) {
162     x_complex<T> tmp(*this);
163     tmp /= rhs;
164     return tmp;
165 };
166 template <typename T2>
167 inline x_complex<T> operator/(const x_complex<T2> &rhs) {
168     x_complex<T> tmp2(*this);
169     tmp /= rhs;
170     return tmp;
171 };
172
173 // Operator +=, +
174 inline x_complex<T> &operator+=(const T &rhs) {
175     re += rhs;
176     return *this;
177 };
178 inline x_complex<T> operator+=(const x_complex<T> &rhs) {
179     x_complex<T> tmp(rhs);
180     x_complex<T> tmp2(*this);
181     tmp2.re += tmp.real();
182     tmp2.im += tmp.imag();
183     *this = tmp2;
184     return *this;
185 };
186 template <typename T2>
187 inline x_complex<T> operator+=(const x_complex<T2> &rhs) {
188     x_complex<T2> tmp(rhs);
189     x_complex<T> tmp2(tmp.real(), tmp.imag());
190     *this += tmp2;
191     return *this;
192 };
193
194 inline x_complex<T> operator+(const T &rhs) {
195     x_complex<T> tmp;
196     tmp = *this;
197     tmp += rhs;
198     return tmp;
199 };
200 inline x_complex<T> operator+(const x_complex<T> &rhs) {
201     x_complex<T> tmp;
202     tmp = *this;
203     tmp += rhs;
204     return tmp;
205 };
206 template <typename T2>
207 inline x_complex<T> operator+(const x_complex<T2> &rhs) {
208     x_complex<T> tmp;
209     tmp = *this;
210     tmp += rhs;
211     return tmp;
212 };
213
214 // Operator -=, -
215 inline x_complex<T> &operator-=(const T &rhs) {
216     re -= rhs;
217     return *this;
218 };
219 inline x_complex<T> &operator-=(const x_complex<T> &rhs) {
220     x_complex<T> tmp(rhs);
221     x_complex<T> tmp2(*this);
222     tmp2.re -= tmp.real();
223     tmp2.im -= tmp.imag();
224     *this = tmp2;
225     return *this;
226 };
227 template <typename T2>
228 inline x_complex<T> &operator-=(const x_complex<T2> &rhs) {
229     x_complex<T2> tmp(rhs);
230     x_complex<T> tmp2(*this);
231     tmp2.re -= tmp.real();
232     tmp2.im -= tmp.imag();
233     *this = tmp2;
234     return *this;
235 };
236
237 inline x_complex<T> operator-(const T &rhs) {
238     x_complex<T> tmp;
239     tmp = *this;
240     tmp -= rhs;
241     return tmp;
242 };
243 inline x_complex<T> operator-(const x_complex<T> &rhs) {
244     x_complex<T> tmp;
245     tmp = *this;
246     tmp -= rhs;
247     return tmp;
248 };
249 template <typename T2>
250 inline x_complex<T> operator-(const x_complex<T2> &rhs) {
251     x_complex<T> tmp;
252     tmp = *this;
253
254     tmp -= rhs;
255     return tmp;
256 };
257 // neg
258 inline x_complex<T> operator-() {
259     x_complex<T> tmp(*this);
260     tmp.real(-tmp.real());
261     tmp.imag(-tmp.imag());
262     return tmp;
263 };
264 // std::x_complex --> hls::x_complex
265 x_complex<T> &operator=(const std::complex<T> &t) {
266     std::complex<T> tmp(t);
267     x_complex<T> tmp2(tmp.real(), tmp.imag());
268     *this = tmp2;
269     return *this;
270 };
271 // hls::x_complex --> std::x_complex
272 operator std::complex<T>() const {
273     x_complex<T> tmp(this);
274     return std::complex<T>(tmp.re, tmp.im);
275 }
276
277 std::complex<T> to_std_complex() {
278     x_complex<T> tmp(this);
279     return std::complex<T>(tmp.re, tmp.im);
280 }
281
282 #ifndef __SYNTHESIS__
283 // Operator <<
284 friend std::ostream &operator<<(std::ostream &out, const
x_complex<T> &c) {
285     bool neg_imag = c.imag() <= -0 ? true : false;
286     T imag = neg_imag ? (T)-c.imag() : (T)c.imag();
287
288     out << c.real() << (neg_imag ? " - j*" : " + j") <<
imag;
289     return out;
290 }
291 #endif
292
293 }; // x_complex
294
295 // Non-member Operator ==, != for x_complex
296 template <typename T>
297 inline bool operator==(const T &lhs, const x_complex<T> &rhs) {
298     x_complex<T> tmp(rhs);
299     return (lhs == tmp.real()) && (0 == tmp.imag());
300 }
301 template <typename T>
302 inline bool operator==(const x_complex<T> &lhs, const T &rhs) {
303     x_complex<T> tmp(lhs);
304     return (tmp.real() == rhs) && (tmp.imag() == 0);
305 }
306 template <typename T>
307 inline bool operator==(const x_complex<T> &lhs, const x_complex<T> &rhs) {
308     x_complex<T> tmp1(lhs);
309     x_complex<T> tmp2(rhs);
310     return (tmp1.real() == tmp2.real()) && (tmp1.imag() ==
tmp2.imag());
311 }
312
313 template <typename T>
314 inline bool operator!=(const T &lhs, const x_complex<T> &rhs) {
315     x_complex<T> tmp(rhs);
316     return (lhs != tmp.real()) || (0 != tmp.imag());
317 }
318 template <typename T>
319 inline bool operator!=(const x_complex<T> &lhs, const T &rhs) {
320     x_complex<T> tmp(lhs);
321     return (tmp.real() != rhs) || (tmp.imag() != 0);
322 }
323 template <typename T>
324 inline bool operator!=(const x_complex<T> &lhs, const x_complex<T> &rhs) {
325     x_complex<T> tmp1(lhs);
326     x_complex<T> tmp2(rhs);
327     return (tmp1.real() != tmp2.real()) || (tmp1.imag() !=
tmp2.imag());
328 }
329
330 template <typename T> x_complex<T> x_neg(x_complex<T> &din) {
331     x_complex<T> tmp(din);
332     tmp.real(-tmp.real());
333     tmp.imag(-tmp.imag());
334     return (tmp);
335 }
336
337 template <typename T> x_complex<T> x_conj_sq(x_complex<T>
&din) {
338     x_complex<T> tmp(din);
339     return ((tmp.real() * tmp.real()) + (tmp.imag() *
tmp.imag()));
340 }

```

```

341
342 template <typename T> x_complex<T> x_conj(const x_complex<T>
&din) {
343     x_complex<T> tmp(din);
344     tmp.imag(-tmp.imag());
345     return tmp;
346 }
347
348 // This function is called when a non-const x_complex is con-
jugated. It calls
349 // the version of x_conj() which takes a constant parameter
350 template <typename T> x_complex<T> x_conj(x_complex<T> &din)
{
351     const x_complex<T> tmp(din);
352     return x_conj(tmp);
353 }
354
355 template <int W, int I> void set_to_one(x_complex<ap_fixed<W,
I>> &a) {
356     ap_int<W> tmp_sat = ((ap_int<W> - I + 1)>1 << (W - I)) - 1;
357     ap_fixed<W, I> tmp;
358     tmp.range() = tmp_sat;
359     a = tmp;
360 }
361
362 template <typename T> T x_real(const x_complex<T> &din) { re-
turn din.real(); }
363
364 template <typename T> T x_imag(const x_complex<T> &din) { re-
turn din.imag(); }
365
366 template <typename T1>
367 typename x_complex<T1>::MULT_RT operator*(const T1 &lhs,
x_complex<T1> &rhs) {
368     return rhs * lhs; // Flip direction and re-use complex
class defined operator
369 }
370
371 } // end namespace hls
372
373 #endif
374

```

E.47 vendor/mpfr.h (945 lines, 47,985 B)

```

1 /* mpfr.h -- Include file for mpfr.
2
3 Copyright 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006,
2007, 2008, 2009, 2010, 2011 Free Software Foundation, Inc.
4 Contributed by the Aenaire and Cacao projects, INRIA.
5
6 This file is part of the GNU MPFR Library.
7
8 The GNU MPFR Library is free software; you can redistribute
it and/or modify
9 it under the terms of the GNU Lesser General Public License
as published by
10 the Free Software Foundation; either version 3 of the Li-
cense, or (at your
11 option) any later version.
12
13 The GNU MPFR Library is distributed in the hope that it will
be useful, but
14 WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY
15 or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser Gen-
eral Public
16 License for more details.
17
18 You should have received a copy of the GNU Lesser General
Public License
19 along with the GNU MPFR Library; see the file COPY-
ING.LESSER. If not, see
20 http://www.gnu.org/licenses/ or write to the Free Software
Foundation, Inc.,
21 51 Franklin St, Fifth Floor, Boston, MA 02110-1301, USA. */
22
23 #ifndef __MPFR_H
24 #define __MPFR_H
25
26 /* Define MPFR version number */
27 #define MPFR_VERSION_MAJOR 3
28 #define MPFR_VERSION_MINOR 0
29 #define MPFR_VERSION_PATCHLEVEL 1
30 #define MPFR_VERSION_STRING "3.0.1-p4"
31
32 /* Macros dealing with MPFR VERSION */
33 #define MPFR_VERSION_NUM(a,b,c) (((a) << 16L) | ((b) << 8) |
(c))
34 #define MPFR_VERSION \
35 MPFR_VER-
SION_NUM(MPFR_VER-
SION_MAJOR,MPFR_VERSION_MINOR,MPFR_VERSION_PATCHLEVEL)
36
37 /* Check if GMP is included, and try to include it (Works
with local GMP) */
38 #ifndef __GMP_H__
39 #include <gmp.h>
40 #endif

```

```

41
42 /* Check if stdio.h is included or if the user wants FILE */
43 #if defined (__GMP_H_HAVE_FILE) || defined (MPFR_USE_FILE)
44 #define _MPFR_H_HAVE_FILE 1
45 #endif
46
47 #if defined (__GMP_H_HAVE_VALIST)
48 #define _MPFR_H_HAVE_VALIST 1
49 #endif
50
51 /* Check if <stdint.h> / <inttypes.h> is included or if the
user
52 explicitly wants intmax_t. Automatical detection is done
by
53 checking:
54 - INTMAX_C and UINTMAX_C, but not if the compiler is a
C++ one
55 (as suggested by Patrick Pelissier) because the test
does not
56 work well in this case. See:
57 http://websympa.loria.fr/wwsympa/arc/mpfr/2010-02/
msg00025.html
58 We do not check INTMAX_MAX and UINTMAX_MAX because un-
der Solaris,
59 these macros are always defined by <limits.h> (i.e.
even when
60 <stdint.h> and <inttypes.h> are not included).
61 - _STDINT_H (defined by the glibc) and _STDINT_H_
(defined under
62 Mac OS X), but this test may not work with all imple-
mentations.
63 Portable software should not rely on these tests.
64 */
65 #if (defined (INTMAX_C) && defined (UINTMAX_C) && !
defined(__cplusplus)) || \
66 defined (MPFR_USE_INTMAX_T) || defined (_STDINT_H) ||
defined (_STDINT_H_)
67 #define _MPFR_H_HAVE_INTMAX_T 1
68 #endif
69
70 /* Avoid some problems with macro expansion if the user
defines macros
71 with the same name as keywords. By convention, identifiers
and macro
72 names starting with mpfr_ are reserved by MPFR. */
73 typedef void mpfr_void;
74 typedef int mpfr_int;
75 typedef unsigned int mpfr_uint;
76 typedef long mpfr_long;
77 typedef unsigned long mpfr_ulong;
78 typedef size_t mpfr_size_t;
79
80 /* Definition of rounding modes (DON'T USE MPFR_RNDNA!).
81
82 Warning! Changing the contents of this enum should be seen as
an
83 interface change since the old and the new types are not
compatible
84 (the integer type compatible with the enumerated type can
even change,
85 see ISO C99, 6.7.2.2#4), and in Makefile.am, AGE should be
set to 0.
86
87 MPFR_RNDU must appear just before MPFR_RNDD (see
MPFR_IS_RNDUTEST_OR_RNDDNOTTEST in mpfr-impl.h).
88
89 MPFR_RNDF has been added, though not implemented yet, in
order to avoid
90 to break the ABI once faithful rounding gets implemented.
91
92 If you change the order of the rounding modes, please up-
date the routines
93 in texexceptions.c which assume 0=RNDN, 1=RNDZ, 2=RNDU,
3=RNDD, 4=RNDA.
94 */
95 typedef enum {
96     MPFR_RNDN=0, /* round to nearest, with ties to even */
97     MPFR_RNDZ, /* round toward zero */
98     MPFR_RNDU, /* round toward +Inf */
99     MPFR_RNDD, /* round toward -Inf */
100     MPFR_RNDA, /* round away from zero */
101     MPFR_RNDF, /* faithful rounding (not implemented yet) */
102
103     MPFR_RNDNA=-1 /* round to nearest, with ties away from zero
(mpfr_round) */
104 } mpfr_rnd_t;
105
106 /* kept for compatibility with MPFR 2.4.x and before */
107 #define GMP_RNDN MPFR_RNDN
108 #define GMP_RNDZ MPFR_RNDZ
109 #define GMP_RNDU MPFR_RNDU
110 #define GMP_RNDD MPFR_RNDD
111
112 /* Define precision : 1 (short), 2 (int) or 3 (long) (DON'T
USE IT!)/
113 #ifndef __MPFR_PREC_FORMAT
114 #if __GMP_MP_SIZE_T_INT == 1
115 #define _MPFR_PREC_FORMAT 2

```

```

115 # else
116 # define _MPFR_PREC_FORMAT 3
117 # endif
118 #endif
119
120 /* Let's make mpfr_prec_t signed in order to avoid problems
due to the
121 usual arithmetic conversions when mixing mpfr_prec_t and
mpfr_exp_t
122 in an expression (for error analysis) if casts are forgot-
ten. */
123 #if _MPFR_PREC_FORMAT == 1
124 typedef short mpfr_prec_t;
125 typedef unsigned short mpfr_uprec_t;
126 #elif _MPFR_PREC_FORMAT == 2
127 typedef int mpfr_prec_t;
128 typedef unsigned int mpfr_uprec_t;
129 #elif _MPFR_PREC_FORMAT == 3
130 typedef long mpfr_prec_t;
131 typedef unsigned long mpfr_uprec_t;
132 #else
133 # error "Invalid MPFR Prec format"
134 #endif
135
136 /* Definition of precision limits without needing <limits.h>
*/
137 /* Note: the casts allows the expression to yield the wanted
behavior
138 for _MPFR_PREC_FORMAT == 1 (due to integer promotion
rules). */
139 #define MPFR_PREC_MIN 2
140 #define MPFR_PREC_MAX ((mpfr_prec_t)((mpfr_uprec_t)
(-(mpfr_uprec_t)0)>>1))
141
142 /* Definition of sign */
143 typedef int mpfr_sign_t;
144
145 /* Definition of the exponent: same as in GMP. */
146 typedef mp_exp_t mpfr_exp_t;
147
148 /* Definition of the standard exponent limits */
149 #define MPFR_EMAX_DEFAULT ((mpfr_exp_t) (((mpfr_ulong) 1 <<
30) - 1))
150 #define MPFR_EMIN_DEFAULT (~(MPFR_EMAX_DEFAULT))
151
152 /* Definition of the main structure */
153 typedef struct {
154     mpfr_prec_t mpfr_prec;
155     mpfr_sign_t mpfr_sign;
156     mpfr_exp_t mpfr_exp;
157     mp_limb_t *mpfr_d;
158 } __mpfr_struct;
159
160 /* Compatibility with previous types of MPFR */
161 #ifndef mp_rnd_t
162 # define mp_rnd_t mpfr_rnd_t
163 #endif
164 #ifndef mp_prec_t
165 # define mp_prec_t mpfr_prec_t
166 #endif
167
168 /*
169 The represented number is
170 _sign*(_d[k-1]/B+_d[k-2]/B^2+...+_d[0]/B^k)*2^_exp
171 where k=ceil(_mp_prec/GMP_NUMB_BITS) and
B=2^GMP_NUMB_BITS.
172
173 For the msb (most significant bit) normalized representa-
tion, we must have
174 _d[k-1]>=B/2, unless the number is singular.
175
176 We must also have the last k*GMP_NUMB_BITS-_prec bits set
to zero.
177 */
178
179 typedef __mpfr_struct mpfr_t[1];
180 typedef __mpfr_struct *mpfr_ptr;
181 typedef __gmp_const __mpfr_struct *mpfr_srcptr;
182
183 /* For those who need a direct and fast access to the sign
field.
184 However it is not in the API, thus use it at your own
risk: it might
185 not be supported, or change name, in further versions!
186 Unfortunately, it must be defined here (instead of MPFR's
internal
187 header file mpfr-impl.h) because it is used by some macros
below.
188 */
189 #define MPFR_SIGN(x) ((x)->_mpfr_sign)
190
191 /* Stack interface */
192 typedef enum {
193     MPFR_NAN_KIND = 0,
194     MPFR_INF_KIND = 1, MPFR_ZERO_KIND = 2, MPFR_REGULAR_KIND =
3
195 } mpfr_kind_t;
196
197 /* GMP defines:

```

```

198 + size_t: Standard size_t
199 + __GMP_ATTRIBUTE_PURE Attribute for math functions.
200 + __GMP_NOTHROW For C++: can't throw .
201 + __GMP_EXTERN_INLINE Attribute for inline function.
202
203 * __gmp_const const (Supports for K&R compiler
only for mpfr.h).
204 + __GMP_DECLSPEC_EXPORT compiling to go into a DLL
205 + __GMP_DECLSPEC_IMPORT compiling to go into a applica-
tion
206
207 /* Extra MPFR defines */
208 #define __MPFR_SENTINEL_ATTR
209 #if defined (__GNUC__)
210 # if __GNUC__ >= 4
211 # undef __MPFR_SENTINEL_ATTR
212 # define __MPFR_SENTINEL_ATTR __attribute__((sentinel))
213 # endif
214 #endif
215
216 /* Prototypes: Support of K&R compiler */
217 #if defined (__GMP_PROTO)
218 # define _MPFR_PROTO __GMP_PROTO
219 #elif defined (__STDC__) || defined (__cplusplus)
220 # define _MPFR_PROTO(x) x
221 #else
222 # define _MPFR_PROTO(x) ()
223 #endif
224
225 /* Support for WINDOWS DLL:
226 Check if we are inside a MPFR build, and if so export the
functions.
227 Otherwise does the same thing as GMP */
228 #if defined(__MPFR_WITHIN_MPFR) && __GMP_LIBGMP_DLL
229 # define __MPFR_DECLSPEC __GMP_DECLSPEC_EXPORT
230 #else
231 # define __MPFR_DECLSPEC __GMP_DECLSPEC
232 #endif
233
234 /* Note: In order to be declared, some functions need a spe-
cific
235 system header to be included *before* "mpfr.h". If the
user
236 forgets to include the header, the MPFR function prototype
in
237 the user object file is not correct. To avoid wrong res-
ults,
238 we raise a linker error in that case by changing their in-
ternal
239 name in the library (prefixed by __gmpfr instead of mpfr).
See
240 the lines of the form "#define mpfr_xxx __gmpfr_xxx" be-
low. */
241
242 #if defined (__cplusplus)
243 extern "C" {
244 #endif
245
246 __MPFR_DECLSPEC __gmp_const char * mpfr_get_version _MP-
FR_PROTO ((void));
247 __MPFR_DECLSPEC __gmp_const char * mpfr_get_patches _MP-
FR_PROTO ((void));
248 __MPFR_DECLSPEC int mpfr_buildopt_tls_p _MPFR_PROTO
((void));
249 __MPFR_DECLSPEC int mpfr_buildopt_decimal_p _MPFR_PROTO
((void));
250 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_emin _MPFR_PROTO
((void));
251 __MPFR_DECLSPEC int mpfr_set_emin _MPFR_PROTO
((mpfr_exp_t));
252 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_emin_min _MPFR_PROTO
((void));
253 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_emin_max _MPFR_PROTO
((void));
254 __MPFR_DECLSPEC int mpfr_set_emin_max _MPFR_PROTO
((mpfr_exp_t));
255 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_emin_min _MPFR_PROTO
((void));
256 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_emin_max _MPFR_PROTO
((void));
257
258 __MPFR_DECLSPEC void mpfr_set_default_rounding_mode _MP-
FR_PROTO((mpfr_rnd_t));
259 __MPFR_DECLSPEC mpfr_rnd_t mpfr_get_default_rounding_mode _M-
PFR_PROTO((void));
260 __MPFR_DECLSPEC __gmp_const char *
261 mpfr_print_rnd_mode _MPFR_PROTO((mpfr_rnd_t));
262
263 __MPFR_DECLSPEC void mpfr_clear_flags _MPFR_PROTO ((void));
264 __MPFR_DECLSPEC void mpfr_clear_underflow _MPFR_PROTO
((void));
265 __MPFR_DECLSPEC void mpfr_clear_overflow _MPFR_PROTO
((void));
266 __MPFR_DECLSPEC void mpfr_clear_nanflag _MPFR_PROTO ((void));
267 __MPFR_DECLSPEC void mpfr_clear_inexflag _MPFR_PROTO
((void));

```

```

268 __MPFR_DECLSPEC void mpfr_clear_erangeflag __MPFR_PROTO
((void));
269
270 __MPFR_DECLSPEC void mpfr_set_underflow __MPFR_PROTO ((void));
271 __MPFR_DECLSPEC void mpfr_set_overflow __MPFR_PROTO ((void));
272 __MPFR_DECLSPEC void mpfr_set_nanflag __MPFR_PROTO ((void));
273 __MPFR_DECLSPEC void mpfr_set_inexflag __MPFR_PROTO ((void));
274 __MPFR_DECLSPEC void mpfr_set_erangeflag __MPFR_PROTO
((void));
275
276 __MPFR_DECLSPEC int mpfr_underflow_p __MPFR_PROTO ((void));
277 __MPFR_DECLSPEC int mpfr_overflow_p __MPFR_PROTO ((void));
278 __MPFR_DECLSPEC int mpfr_nanflag_p __MPFR_PROTO ((void));
279 __MPFR_DECLSPEC int mpfr_inexflag_p __MPFR_PROTO ((void));
280 __MPFR_DECLSPEC int mpfr_erangeflag_p __MPFR_PROTO ((void));
281
282 __MPFR_DECLSPEC int
283 mpfr_check_range __MPFR_PROTO ((mpfr_ptr, int, mpfr_rnd_t));
284
285 __MPFR_DECLSPEC void mpfr_init2 __MPFR_PROTO ((mpfr_ptr,
mpfr_prec_t));
286 __MPFR_DECLSPEC void mpfr_init __MPFR_PROTO ((mpfr_ptr));
287 __MPFR_DECLSPEC void mpfr_clear __MPFR_PROTO ((mpfr_ptr));
288
289 __MPFR_DECLSPEC void
290 mpfr_inits2 __MPFR_PROTO ((mpfr_prec_t, mpfr_ptr, ...) __M-
FR_SENTINEL_ATTR;
291 __MPFR_DECLSPEC void
292 mpfr_inits __MPFR_PROTO ((mpfr_ptr, ...) __MP-
FR_SENTINEL_ATTR;
293 __MPFR_DECLSPEC void
294 mpfr_clears __MPFR_PROTO ((mpfr_ptr, ...) __MP-
FR_SENTINEL_ATTR;
295
296 __MPFR_DECLSPEC int
297 mpfr_prec_round __MPFR_PROTO ((mpfr_ptr, mpfr_prec_t,
mpfr_rnd_t));
298 __MPFR_DECLSPEC int
299 mpfr_can_round __MPFR_PROTO ((mpfr_srcptr, mpfr_exp_t,
mpfr_rnd_t, mpfr_rnd_t,
mpfr_prec_t));
300 __MPFR_DECLSPEC mpfr_prec_t mpfr_min_prec __MPFR_PROTO
((mpfr_srcptr));
301 __MPFR_DECLSPEC mpfr_prec_t mpfr_min_prec __MPFR_PROTO
((mpfr_srcptr));
302
303 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_exp __MPFR_PROTO ((mpfr_s-
rcptr));
304 __MPFR_DECLSPEC int mpfr_set_exp __MPFR_PROTO ((mpfr_ptr,
mpfr_exp_t));
305 __MPFR_DECLSPEC mpfr_prec_t mpfr_get_prec __MP-
FR_PROTO((mpfr_srcptr));
306 __MPFR_DECLSPEC void mpfr_set_prec __MPFR_PROTO((mpfr_ptr,
mpfr_prec_t));
307 __MPFR_DECLSPEC void mpfr_set_prec_raw __MPFR_PROTO((mpfr_ptr,
mpfr_prec_t));
308 __MPFR_DECLSPEC void mpfr_set_default_prec __MP-
FR_PROTO((mpfr_prec_t));
309 __MPFR_DECLSPEC mpfr_prec_t mpfr_get_default_prec __MP-
FR_PROTO((void));
310
311 __MPFR_DECLSPEC int mpfr_set_d __MPFR_PROTO ((mpfr_ptr,
double, mpfr_rnd_t));
312 __MPFR_DECLSPEC int mpfr_setflt __MPFR_PROTO ((mpfr_ptr,
float, mpfr_rnd_t));
313 #ifdef MPFR_WANT_DECIMAL_FLOATS
314 __MPFR_DECLSPEC int mpfr_set_decimal64 __MPFR_PROTO
((mpfr_ptr, _Decimal64,
mpfr_rnd_t));
315 #endif
316
317 __MPFR_DECLSPEC int
318 mpfr_set_ld __MPFR_PROTO ((mpfr_ptr, long double,
mpfr_rnd_t));
319 __MPFR_DECLSPEC int
320 mpfr_set_z __MPFR_PROTO ((mpfr_ptr, mpz_srcptr,
mpfr_rnd_t));
321 __MPFR_DECLSPEC int
322 mpfr_set_z_2exp __MPFR_PROTO ((mpfr_ptr, mpz_srcptr,
mpfr_exp_t, mpfr_rnd_t));
323 __MPFR_DECLSPEC void mpfr_set_nan __MPFR_PROTO ((mpfr_ptr));
324 __MPFR_DECLSPEC void mpfr_set_inf __MPFR_PROTO ((mpfr_ptr,
int));
325 __MPFR_DECLSPEC void mpfr_set_zero __MPFR_PROTO ((mpfr_ptr,
int));
326 __MPFR_DECLSPEC int
327 mpfr_set_f __MPFR_PROTO ((mpfr_ptr, mpf_srcptr,
mpfr_rnd_t));
328 __MPFR_DECLSPEC int
329 mpfr_get_f __MPFR_PROTO ((mpfr_ptr, mpf_srcptr,
mpfr_rnd_t));
330 __MPFR_DECLSPEC int mpfr_set_si __MPFR_PROTO ((mpfr_ptr, long,
mpfr_rnd_t));
331 __MPFR_DECLSPEC int
332 mpfr_set_ui __MPFR_PROTO ((mpfr_ptr, unsigned long,
mpfr_rnd_t));
333 __MPFR_DECLSPEC int
334 mpfr_set_si_2exp __MPFR_PROTO ((mpfr_ptr, long, mpfr_exp_t,
mpfr_rnd_t));
335 __MPFR_DECLSPEC int
336 mpfr_set_ui_2exp __MPFR_PROTO ((mpfr_ptr, unsigned
long, mpfr_exp_t, mpfr_rnd_t));
337 __MPFR_DECLSPEC int
338 mpfr_set_q __MPFR_PROTO ((mpfr_ptr, mpq_srcptr,
mpfr_rnd_t));
339 __MPFR_DECLSPEC int
340 mpfr_set_str __MPFR_PROTO ((mpfr_ptr, __gmp_const char *,
int, mpfr_rnd_t));
341 __MPFR_DECLSPEC int
342 mpfr_init_set_str __MPFR_PROTO ((mpfr_ptr, __gmp_const char
*, int,
mpfr_rnd_t));
343 __MPFR_DECLSPEC int
344 mpfr_set4 __MPFR_PROTO ((mpfr_ptr, mpfr_srcptr, mpfr_rnd_t,
int));
345 __MPFR_DECLSPEC int
346 mpfr_abs __MPFR_PROTO ((mpfr_ptr, mpfr_srcptr, mpfr_rnd_t));
347 __MPFR_DECLSPEC int
348 mpfr_set __MPFR_PROTO ((mpfr_ptr, mpfr_srcptr, mpfr_rnd_t));
349 __MPFR_DECLSPEC int mpfr_neg __MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
350 __MPFR_DECLSPEC int mpfr_signbit __MPFR_PROTO ((mpfr_srcptr));
351 __MPFR_DECLSPEC int
352 mpfr_setsign __MPFR_PROTO ((mpfr_ptr, mpfr_srcptr, int,
mpfr_rnd_t));
353 __MPFR_DECLSPEC int
354 mpfr_copysign __MPFR_PROTO ((mpfr_ptr, mpfr_srcptr, mpfr_sr-
cptr, mpfr_rnd_t));
355 #ifdef MPFR_H_HAVE_INTMAX_T
356 #define mpfr_set_sj __gmpfr_set_sj
357 #define mpfr_set_sj_2exp __gmpfr_set_sj_2exp
358 #define mpfr_set_uj __gmpfr_set_uj
359 #define mpfr_set_uj_2exp __gmpfr_set_uj_2exp
360 #define mpfr_get_sj __gmpfr_get_sj
361 #define mpfr_get_uj __gmpfr_get_uj
362 #define mpfr_set_z_2exp __gmpfr_set_z_2exp
363 #define mpfr_get_z_2exp __gmpfr_get_z_2exp
364 __MPFR_DECLSPEC int mpfr_set_sj __MPFR_PROTO ((mpfr_t, int-
max_t, mpfr_rnd_t));
365 __MPFR_DECLSPEC int
366 mpfr_set_sj_2exp __MPFR_PROTO ((mpfr_t, intmax_t, intmax_t,
mpfr_rnd_t));
367 __MPFR_DECLSPEC int mpfr_set_uj __MPFR_PROTO ((mpfr_t,
uintmax_t, mpfr_rnd_t));
368 __MPFR_DECLSPEC int
369 mpfr_set_uj_2exp __MPFR_PROTO ((mpfr_t, uintmax_t, intmax_t,
mpfr_rnd_t));
370 __MPFR_DECLSPEC intmax_t mpfr_get_sj __MPFR_PROTO ((mpfr_sr-
cptr, mpfr_rnd_t));
371 __MPFR_DECLSPEC uintmax_t mpfr_get_uj __MPFR_PROTO ((mpfr_sr-
cptr, mpfr_rnd_t));
372 #endif
373
374 __MPFR_DECLSPEC mpfr_exp_t mpfr_get_z_2exp __MPFR_PROTO
((mpfr_ptr, mpfr_srcptr));
375 __MPFR_DECLSPEC float mpfr_getflt __MPFR_PROTO ((mpfr_srcptr,
mpfr_rnd_t));
376 __MPFR_DECLSPEC double mpfr_get_d __MPFR_PROTO ((mpfr_srcptr,
mpfr_rnd_t));
377 #ifdef MPFR_WANT_DECIMAL_FLOATS
378 __MPFR_DECLSPEC _Decimal64 mpfr_get_decimal64 __MPFR_PROTO
((mpfr_srcptr,
mpfr_rnd_t));
379 #endif
380
381 __MPFR_DECLSPEC long double mpfr_get_ld __MPFR_PROTO ((mpfr_s-
rcptr,
mpfr_rnd_t));
382 __MPFR_DECLSPEC double mpfr_get_d1 __MPFR_PROTO ((mpfr_sr-
cptr));
383 __MPFR_DECLSPEC double mpfr_get_d_2exp __MPFR_PROTO ((long*,
mpfr_srcptr,
mpfr_rnd_t));
384 __MPFR_DECLSPEC long double mpfr_get_ld_2exp __MPFR_PROTO
((long*, mpfr_srcptr,
mpfr_rnd_t));
385 __MPFR_DECLSPEC long double mpfr_get_ld_2exp __MPFR_PROTO
((long*, mpfr_srcptr,
mpfr_rnd_t));
386 __MPFR_DECLSPEC long mpfr_get_si __MPFR_PROTO ((mpfr_srcptr,
mpfr_rnd_t));
387 __MPFR_DECLSPEC unsigned long mpfr_get_ui __MPFR_PROTO
((mpfr_srcptr,
mpfr_rnd_t));
388 __MPFR_DECLSPEC char*mpfr_get_str __MPFR_PROTO ((char*,
mpfr_exp_t*, int, size_t,
mpfr_srcptr,
mpfr_rnd_t));
389 __MPFR_DECLSPEC int mpfr_get_z __MPFR_PROTO ((mpfr_ptr z,
mpfr_srcptr f,
mpfr_rnd_t));
390 __MPFR_DECLSPEC void mpfr_free_str __MPFR_PROTO ((char *));
391 __MPFR_DECLSPEC int mpfr_urandom __MPFR_PROTO ((mpfr_ptr,
gmp_randstate_t,
mpfr_rnd_t));
392 __MPFR_DECLSPEC int mpfr_urandomb __MPFR_PROTO ((mpfr_ptr,

```

```

gmp_randstate_t));
401
402 __MPFR_DECLSPEC void mpfr_nextabove _MPFR_PROTO ((mpfr_ptr));
403 __MPFR_DECLSPEC void mpfr_nextbelow _MPFR_PROTO ((mpfr_ptr));
404 __MPFR_DECLSPEC void mpfr_nexttoward _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr));
405
406 #ifdef _MPFR_H_HAVE_FILE
407 #define mpfr_inp_str __gmpfr_inp_str
408 #define mpfr_out_str __gmpfr_out_str
409 __MPFR_DECLSPEC size_t mpfr_inp_str _MPFR_PROTO ((mpfr_ptr,
FILE*, int,
410 mpfr_rnd_t));
411 __MPFR_DECLSPEC size_t mpfr_out_str _MPFR_PROTO ((FILE*, int,
size_t,
412 mpfr_sr-
cptr, mpfr_rnd_t));
413 #define mpfr_fprintf __gmpfr_fprintf
414 __MPFR_DECLSPEC int mpfr_fprintf _MPFR_PROTO ((FILE*, __g-
mp_const char*,
415 ...));
416 #endif
417 __MPFR_DECLSPEC int mpfr_printf _MPFR_PROTO ((__gmp_const
char*, ...));
418 __MPFR_DECLSPEC int mpfr_asprintf _MPFR_PROTO ((char**, __g-
mp_const char*,
419 ...));
420 __MPFR_DECLSPEC int mpfr_sprintf _MPFR_PROTO ((char*, __g-
mp_const char*,
421 ...));
422 __MPFR_DECLSPEC int mpfr_snprintf _MPFR_PROTO ((char*,
size_t,
423 __gmp_const
char*, ...));
424
425 #ifdef _MPFR_H_HAVE_VALIST
426 #define mpfr_vfprintf __gmpfr_vfprintf
427 #define mpfr_vfprintf _MPFR_PROTO ((FILE*, __g-
mp_const char*,
428 va_list));
429 #endif /* _MPFR_H_HAVE_FILE */
430 #define mpfr_vprintf __gmpfr_vprintf
431 #define mpfr_vasprintf __gmpfr_vasprintf
432 #define mpfr_vsnprintf __gmpfr_vsnprintf
433 #define mpfr_vprintf _MPFR_PROTO ((__gmp_const
char*, va_list));
434 __MPFR_DECLSPEC int mpfr_vasprintf _MPFR_PROTO ((char**, __g-
mp_const char*,
435 va_list));
436 __MPFR_DECLSPEC int mpfr_vsnprintf _MPFR_PROTO ((char*, __g-
mp_const char*,
437 va_list));
438 __MPFR_DECLSPEC int mpfr_vsnprintf _MPFR_PROTO ((char*, __g-
mp_const char*,
439 va_list));
440 __MPFR_DECLSPEC int mpfr_vsnprintf _MPFR_PROTO ((char*,
size_t,
441 __gmp_const
char*, va_list));
442 #endif /* _MPFR_H_HAVE_VALIST */
443
444 __MPFR_DECLSPEC int mpfr_pow _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,
445 mpfr_rnd_t));
446 __MPFR_DECLSPEC int mpfr_pow_si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
447 long int,
mpfr_rnd_t));
448 __MPFR_DECLSPEC int mpfr_pow_ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
449 unsigned long
int, mpfr_rnd_t));
450 __MPFR_DECLSPEC int mpfr_ui_pow_ui _MPFR_PROTO ((mpfr_ptr,
unsigned long int,
451 unsigned long
int, mpfr_rnd_t));
452 __MPFR_DECLSPEC int mpfr_ui_pow _MPFR_PROTO ((mpfr_ptr, un-
signed long int,
453 mpfr_srcptr,
mpfr_rnd_t));
454 __MPFR_DECLSPEC int mpfr_pow_z _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
455 mpz_srcptr,
mpfr_rnd_t));
456
457 __MPFR_DECLSPEC int mpfr_sqrt _MPFR_PROTO ((mpfr_ptr, mpfr_s-
rcptr,
458 mpfr_rnd_t));
459 __MPFR_DECLSPEC int mpfr_sqrt_ui _MPFR_PROTO ((mpfr_ptr, un-
signed long,
460 mpfr_rnd_t));
461 __MPFR_DECLSPEC int mpfr_rec_sqrt _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
462 mpfr_rnd_t));
463
464 __MPFR_DECLSPEC int mpfr_add _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,
465 mpfr_srcptr,

```

```

mpfr_rnd_t));
466 __MPFR_DECLSPEC int mpfr_sub _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,
467 mpfr_srcptr,
mpfr_rnd_t));
468 __MPFR_DECLSPEC int mpfr_mul _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,
469 mpfr_srcptr,
mpfr_rnd_t));
470 __MPFR_DECLSPEC int mpfr_div _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,
471 mpfr_srcptr,
mpfr_rnd_t));
472
473 __MPFR_DECLSPEC int mpfr_add_ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
474 unsigned long,
mpfr_rnd_t));
475 __MPFR_DECLSPEC int mpfr_sub_ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
476 unsigned long,
mpfr_rnd_t));
477 __MPFR_DECLSPEC int mpfr_ui_sub _MPFR_PROTO ((mpfr_ptr, un-
signed long,
478 mpfr_srcptr,
mpfr_rnd_t));
479 __MPFR_DECLSPEC int mpfr_mul_ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
480 unsigned long,
mpfr_rnd_t));
481 __MPFR_DECLSPEC int mpfr_div_ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
482 unsigned long,
mpfr_rnd_t));
483 __MPFR_DECLSPEC int mpfr_ui_div _MPFR_PROTO ((mpfr_ptr, un-
signed long,
484 mpfr_srcptr,
mpfr_rnd_t));
485
486 __MPFR_DECLSPEC int mpfr_add_si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
487 long int,
mpfr_rnd_t));
488 __MPFR_DECLSPEC int mpfr_sub_si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
489 long int,
mpfr_rnd_t));
490 __MPFR_DECLSPEC int mpfr_si_sub _MPFR_PROTO ((mpfr_ptr, long
int,
491 mpfr_srcptr,
mpfr_rnd_t));
492 __MPFR_DECLSPEC int mpfr_mul_si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
493 long int,
mpfr_rnd_t));
494 __MPFR_DECLSPEC int mpfr_div_si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
495 long int,
mpfr_rnd_t));
496 __MPFR_DECLSPEC int mpfr_si_div _MPFR_PROTO ((mpfr_ptr, long
int,
497 mpfr_srcptr,
mpfr_rnd_t));
498
499 __MPFR_DECLSPEC int mpfr_add_d _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
500 double,
mpfr_rnd_t));
501 __MPFR_DECLSPEC int mpfr_sub_d _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
502 double,
mpfr_rnd_t));
503 __MPFR_DECLSPEC int mpfr_d_sub _MPFR_PROTO ((mpfr_ptr,
double,
504 mpfr_srcptr,
mpfr_rnd_t));
505 __MPFR_DECLSPEC int mpfr_mul_d _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
506 double,
mpfr_rnd_t));
507 __MPFR_DECLSPEC int mpfr_div_d _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
508 double,
mpfr_rnd_t));
509 __MPFR_DECLSPEC int mpfr_d_div _MPFR_PROTO ((mpfr_ptr,
double,
510 mpfr_srcptr,
mpfr_rnd_t));
511
512 __MPFR_DECLSPEC int mpfr_sqr _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
513
514 __MPFR_DECLSPEC int mpfr_const_pi _MPFR_PROTO ((mpfr_ptr,
mpfr_rnd_t));
515 __MPFR_DECLSPEC int mpfr_const_log2 _MPFR_PROTO ((mpfr_ptr,
mpfr_rnd_t));
516 __MPFR_DECLSPEC int mpfr_const_euler _MPFR_PROTO ((mpfr_ptr,
mpfr_rnd_t));
517 __MPFR_DECLSPEC int mpfr_const_catalan _MPFR_PROTO
((mpfr_ptr, mpfr_rnd_t));

```



```

518
519 __MPFR_DECLSPEC int mpfr_agm _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_srcptr,
520                                     mpfr_rnd_t));
521
522 __MPFR_DECLSPEC int mpfr_log _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
523 __MPFR_DECLSPEC int mpfr_log2 _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
524 __MPFR_DECLSPEC int mpfr_log10 _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
525                                     mpfr_rnd_t));
526 __MPFR_DECLSPEC int mpfr_log1p _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
527                                     mpfr_rnd_t));
528
529 __MPFR_DECLSPEC int mpfr_exp _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
530 __MPFR_DECLSPEC int mpfr_exp2 _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
531 __MPFR_DECLSPEC int mpfr_exp10 _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
532                                     mpfr_rnd_t));
533 __MPFR_DECLSPEC int mpfr_expn1 _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
534                                     mpfr_rnd_t));
535 __MPFR_DECLSPEC int mpfr_eint _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
536 __MPFR_DECLSPEC int mpfr_li2 _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
537
538 __MPFR_DECLSPEC int mpfr_cmp _MPFR_PROTO ((mpfr_srcptr,
mpfr_srcptr));
539 __MPFR_DECLSPEC int mpfr_cmp3 _MPFR_PROTO ((mpfr_srcptr,
mpfr_srcptr, int));
540 __MPFR_DECLSPEC int mpfr_cmp_d _MPFR_PROTO ((mpfr_srcptr,
double));
541 __MPFR_DECLSPEC int mpfr_cmp_ld _MPFR_PROTO ((mpfr_srcptr,
long double));
542 __MPFR_DECLSPEC int mpfr_cmpabs _MPFR_PROTO ((mpfr_srcptr,
mpfr_srcptr));
543 __MPFR_DECLSPEC int mpfr_cmp_ui _MPFR_PROTO ((mpfr_srcptr,
unsigned long));
544 __MPFR_DECLSPEC int mpfr_cmp_si _MPFR_PROTO ((mpfr_srcptr,
long));
545 __MPFR_DECLSPEC int mpfr_cmp_ui_2exp _MPFR_PROTO ((mpfr_sr-
cptr, unsigned long,
546 mpfr_exp_t));
547 __MPFR_DECLSPEC int mpfr_cmp_si_2exp _MPFR_PROTO ((mpfr_sr-
cptr, long,
548 mpfr_exp_t));
549 __MPFR_DECLSPEC void mpfr_reldiff _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
550                                     mpfr_srcptr,
mpfr_rnd_t));
551 __MPFR_DECLSPEC int mpfr_eq _MPFR_PROTO((mpfr_srcptr, mpfr_s-
rcptr,
552                                     unsigned long));
553 __MPFR_DECLSPEC int mpfr_sgn _MPFR_PROTO ((mpfr_srcptr));
554
555 __MPFR_DECLSPEC int mpfr_mul_2exp _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
556                                     unsigned
long, mpfr_rnd_t));
557 __MPFR_DECLSPEC int mpfr_div_2exp _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
558                                     unsigned
long, mpfr_rnd_t));
559 __MPFR_DECLSPEC int mpfr_mul_2ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
560                                     unsigned long,
mpfr_rnd_t));
561 __MPFR_DECLSPEC int mpfr_div_2ui _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
562                                     unsigned long,
mpfr_rnd_t));
563 __MPFR_DECLSPEC int mpfr_mul_2si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
564                                     long,
mpfr_rnd_t));
565 __MPFR_DECLSPEC int mpfr_div_2si _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
566                                     long,
mpfr_rnd_t));
567
568 __MPFR_DECLSPEC int mpfr_rint _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr, mpfr_rnd_t));
569 __MPFR_DECLSPEC int mpfr_round _MPFR_PROTO((mpfr_ptr, mpfr_s-
rcptr));
570 __MPFR_DECLSPEC int mpfr_trunc _MPFR_PROTO((mpfr_ptr, mpfr_s-
rcptr));
571 __MPFR_DECLSPEC int mpfr_ceil _MPFR_PROTO((mpfr_ptr, mpfr_sr-
cptr));
572 __MPFR_DECLSPEC int mpfr_floor _MPFR_PROTO((mpfr_ptr, mpfr_s-
rcptr));
573 __MPFR_DECLSPEC int mpfr_rint_round _MPFR_PROTO ((mpfr_ptr,

```

```

mpfr_srcptr,
574 mpfr_rnd_t));
575 __MPFR_DECLSPEC int mpfr_rint_trunc _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
576 mpfr_rnd_t));
577 __MPFR_DECLSPEC int mpfr_rint_ceil _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
578 mpfr_rnd_t));
579 __MPFR_DECLSPEC int mpfr_rint_floor _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
580 mpfr_rnd_t));
581 __MPFR_DECLSPEC int mpfr_frac _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
582 __MPFR_DECLSPEC int mpfr_modf _MPFR_PROTO ((mpfr_ptr,
mpfr_ptr, mpfr_srcptr,
583 mpfr_rnd_t));
584 __MPFR_DECLSPEC int mpfr_remquo _MPFR_PROTO ((mpfr_ptr,
long*, mpfr_srcptr,
585                                     mpfr_srcptr,
mpfr_rnd_t));
586 __MPFR_DECLSPEC int mpfr_remainder _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
587                                     mpfr_srcptr,
mpfr_rnd_t));
588 __MPFR_DECLSPEC int mpfr_fmod _MPFR_PROTO ((mpfr_ptr, mpfr_s-
rcptr,
589                                     mpfr_srcptr,
mpfr_rnd_t));
590
591 __MPFR_DECLSPEC int mpfr_fits_ulong_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
592 __MPFR_DECLSPEC int mpfr_fits_slong_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
593 __MPFR_DECLSPEC int mpfr_fits_uint_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
594 __MPFR_DECLSPEC int mpfr_fits_sint_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
595 __MPFR_DECLSPEC int mpfr_fits_ushort_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
596 __MPFR_DECLSPEC int mpfr_fits_sshort_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
597 __MPFR_DECLSPEC int mpfr_fits_uintmax_p _MP-
FR_PROTO((mpfr_srcptr,mpfr_rnd_t));
598 __MPFR_DECLSPEC int mpfr_fits_intmax_p _MP-
FR_PROTO((mpfr_srcptr, mpfr_rnd_t));
599
600 __MPFR_DECLSPEC void mpfr_extract _MPFR_PROTO ((mpz_ptr,
mpfr_srcptr,
601                                     unsigned
int));
602 __MPFR_DECLSPEC void mpfr_swap _MPFR_PROTO ((mpfr_ptr,
mpfr_ptr));
603 __MPFR_DECLSPEC void mpfr_dump _MPFR_PROTO ((mpfr_srcptr));
604
605 __MPFR_DECLSPEC int mpfr_nan_p _MPFR_PROTO((mpfr_srcptr));
606 __MPFR_DECLSPEC int mpfr_inf_p _MPFR_PROTO((mpfr_srcptr));
607 __MPFR_DECLSPEC int mpfr_number_p _MPFR_PROTO((mpfr_srcptr));
608 __MPFR_DECLSPEC int mpfr_integer_p _MPFR_PROTO ((mpfr_sr-
cptr));
609 __MPFR_DECLSPEC int mpfr_zero_p _MPFR_PROTO ((mpfr_srcptr));
610 __MPFR_DECLSPEC int mpfr_regular_p _MPFR_PROTO ((mpfr_sr-
cptr));
611
612 __MPFR_DECLSPEC int mpfr_greater_p _MPFR_PROTO ((mpfr_srcptr,
mpfr_srcptr));
613 __MPFR_DECLSPEC int mpfr_greaterequal_p _MPFR_PROTO ((mpfr_s-
rcptr,
614                                     mpfr_s-
rcptr));
615 __MPFR_DECLSPEC int mpfr_less_p _MPFR_PROTO ((mpfr_srcptr,
mpfr_srcptr));
616 __MPFR_DECLSPEC int mpfr_lessequal_p _MPFR_PROTO ((mpfr_sr-
cptr, mpfr_srcptr));
617 __MPFR_DECLSPEC int mpfr_lessgreater_p _MP-
FR_PROTO((mpfr_srcptr,mpfr_srcptr));
618 __MPFR_DECLSPEC int mpfr_equal_p _MPFR_PROTO ((mpfr_srcptr,
mpfr_srcptr));
619 __MPFR_DECLSPEC int mpfr_unordered_p _MPFR_PROTO ((mpfr_sr-
cptr, mpfr_srcptr));
620
621 __MPFR_DECLSPEC int mpfr_atanh _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr,mpfr_rnd_t));
622 __MPFR_DECLSPEC int mpfr_acosh _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr,mpfr_rnd_t));
623 __MPFR_DECLSPEC int mpfr_asinh _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr,mpfr_rnd_t));
624 __MPFR_DECLSPEC int mpfr_cosh _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr, mpfr_rnd_t));
625 __MPFR_DECLSPEC int mpfr_sinh _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr, mpfr_rnd_t));
626 __MPFR_DECLSPEC int mpfr_tanh _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr, mpfr_rnd_t));

```

```

627 __MPFR_DECLSPEC int mpfr_sinh_cosh _MPFR_PROTO ((mpfr_ptr,
mpfr_ptr,
628 mpfr_srcptr,
mpfr_rnd_t));
629
630 __MPFR_DECLSPEC int mpfr_sech _MPFR_PROTO ((mpfr_ptr, mpfr_s-
rcptr,mpfr_rnd_t));
631 __MPFR_DECLSPEC int mpfr_csch _MPFR_PROTO ((mpfr_ptr, mpfr_s-
rcptr,mpfr_rnd_t));
632 __MPFR_DECLSPEC int mpfr_coth _MPFR_PROTO ((mpfr_ptr, mpfr_s-
rcptr,mpfr_rnd_t));
633
634 __MPFR_DECLSPEC int mpfr_acos _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
635 __MPFR_DECLSPEC int mpfr_asin _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
636 __MPFR_DECLSPEC int mpfr_atan _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
637 __MPFR_DECLSPEC int mpfr_sin _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
638 __MPFR_DECLSPEC int mpfr_sin_cos _MPFR_PROTO ((mpfr_ptr,
mpfr_ptr,
639 mpfr_srcptr,
mpfr_rnd_t));
640 __MPFR_DECLSPEC int mpfr_cos _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
641 __MPFR_DECLSPEC int mpfr_tan _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
642 __MPFR_DECLSPEC int mpfr_atan2 _MPFR_PROTO ((mpfr_ptr,mpfr_s-
rcptr,mpfr_srcptr,
643 mpfr_rnd_t));
644 __MPFR_DECLSPEC int mpfr_sec _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
645 __MPFR_DECLSPEC int mpfr_csc _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
646 __MPFR_DECLSPEC int mpfr_cot _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
647
648 __MPFR_DECLSPEC int mpfr_hypot _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
649 mpfr_srcptr,
mpfr_rnd_t));
650 __MPFR_DECLSPEC int mpfr_erf _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr,mpfr_rnd_t));
651 __MPFR_DECLSPEC int mpfr_erfc _MPFR_PROTO ((mpfr_ptr, mpfr_s-
rcptr,mpfr_rnd_t));
652 __MPFR_DECLSPEC int mpfr_cbrt _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
653 __MPFR_DECLSPEC int mpfr_root _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,unsigned long,mpfr_rnd_t));
654 __MPFR_DECLSPEC int mpfr_gamma _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr,mpfr_rnd_t));
655 __MPFR_DECLSPEC int mpfr_lgamma _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr,mpfr_rnd_t));
656 __MPFR_DECLSPEC int mpfr_lgamma _MP-
FR_PROTO((mpfr_ptr,int*,mpfr_srcptr,mpfr_rnd_t));
657 __MPFR_DECLSPEC int mpfr_digamma _MP-
FR_PROTO((mpfr_ptr,mpfr_srcptr,mpfr_rnd_t));
658 __MPFR_DECLSPEC int mpfr_zeta _MPFR_PROTO ((mpfr_ptr,mpfr_sr-
cptr,mpfr_rnd_t));
659 __MPFR_DECLSPEC int mpfr_zeta_ui _MPFR_PROTO ((mpfr_ptr,un-
signed long,mpfr_rnd_t));
660 __MPFR_DECLSPEC int mpfr_fac_ui _MPFR_PROTO ((mpfr_ptr, un-
signed long int,
661 mpfr_rnd_t));
662 __MPFR_DECLSPEC int mpfr_j0 _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
663 __MPFR_DECLSPEC int mpfr_j1 _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
664 __MPFR_DECLSPEC int mpfr_jn _MPFR_PROTO ((mpfr_ptr, long,
mpfr_srcptr,
665 mpfr_rnd_t));
666 __MPFR_DECLSPEC int mpfr_y0 _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
667 __MPFR_DECLSPEC int mpfr_y1 _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
668 __MPFR_DECLSPEC int mpfr_yn _MPFR_PROTO ((mpfr_ptr, long,
mpfr_srcptr,
669 mpfr_rnd_t));
670
671 __MPFR_DECLSPEC int mpfr_ai _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_rnd_t));
672
673 __MPFR_DECLSPEC int mpfr_min _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_srcptr,
674 mpfr_rnd_t));
675 __MPFR_DECLSPEC int mpfr_max _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_srcptr,
676 mpfr_rnd_t));
677 __MPFR_DECLSPEC int mpfr_dim _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_srcptr,
678 mpfr_rnd_t));
679
680 __MPFR_DECLSPEC int mpfr_mul_z _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
681 mpz_srcptr,
mpfr_rnd_t));
682 __MPFR_DECLSPEC int mpfr_div_z _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
683 mpz_srcptr,

```

```

mpfr_rnd_t));
684 __MPFR_DECLSPEC int mpfr_add_z _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
685 mpz_srcptr,
mpfr_rnd_t));
686 __MPFR_DECLSPEC int mpfr_sub_z _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
687 mpz_srcptr,
mpfr_rnd_t));
688 __MPFR_DECLSPEC int mpfr_cmp_z _MPFR_PROTO ((mpfr_srcptr,
mpz_srcptr));
689
690 __MPFR_DECLSPEC int mpfr_mul_q _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
691 mpq_srcptr,
mpfr_rnd_t));
692 __MPFR_DECLSPEC int mpfr_div_q _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
693 mpq_srcptr,
mpfr_rnd_t));
694 __MPFR_DECLSPEC int mpfr_add_q _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
695 mpq_srcptr,
mpfr_rnd_t));
696 __MPFR_DECLSPEC int mpfr_sub_q _MPFR_PROTO ((mpfr_ptr,
mpfr_srcptr,
697 mpq_srcptr,
mpfr_rnd_t));
698 __MPFR_DECLSPEC int mpfr_cmp_q _MPFR_PROTO ((mpfr_srcptr,
mpq_srcptr));
699
700 __MPFR_DECLSPEC int mpfr_cmp_f _MPFR_PROTO ((mpfr_srcptr,
mpf_srcptr));
701
702 __MPFR_DECLSPEC int mpfr_fma _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_srcptr,
703 mpfr_srcptr,
mpfr_rnd_t));
704 __MPFR_DECLSPEC int mpfr_fms _MPFR_PROTO ((mpfr_ptr, mpfr_sr-
cptr, mpfr_srcptr,
705 mpfr_srcptr,
mpfr_rnd_t));
706 __MPFR_DECLSPEC int mpfr_sum _MPFR_PROTO ((mpfr_ptr, mpfr_ptr
*_gmp_const,
707 unsigned long,
mpfr_rnd_t));
708
709 __MPFR_DECLSPEC void mpfr_free_cache _MPFR_PROTO ((void));
710
711 __MPFR_DECLSPEC int mpfr_subnormalize _MPFR_PROTO
((mpfr_ptr, int,
712 mpfr_rnd_t));
713
714 __MPFR_DECLSPEC int mpfr_strtofr _MPFR_PROTO ((mpfr_ptr,
_gmp_const char *,
715 char **, int,
mpfr_rnd_t));
716
717 __MPFR_DECLSPEC size_t mpfr_custom_get_size _MPFR_PROTO
((mpfr_prec_t));
718 __MPFR_DECLSPEC void mpfr_custom_init _MPFR_PROTO ((void
*, mpfr_prec_t));
719 __MPFR_DECLSPEC void * mpfr_custom_get_significand _MP-
FR_PROTO ((mpfr_srcptr));
720 __MPFR_DECLSPEC mpfr_exp_t mpfr_custom_get_exp _MPFR_PROTO
((mpfr_srcptr));
721 __MPFR_DECLSPEC void mpfr_custom_move _MPFR_PROTO
((mpfr_ptr, void *));
722 __MPFR_DECLSPEC void mpfr_custom_init_set _MPFR_PROTO
((mpfr_ptr, int,
723 mpfr_exp_t,
mpfr_prec_t, void *));
724 __MPFR_DECLSPEC int mpfr_custom_get_kind _MPFR_PROTO
((mpfr_srcptr));
725
726 #if defined (__cplusplus)
727 }
728 #endif
729
730 /* DON'T USE THIS! (For MPFR-public macros only, see below.)
731 The mpfr_sgn macro uses the fact that __MPFR_EXP_NAN and
__MPFR_EXP_ZERO
732 are the smallest values.
733 FIXME: In the following macros, the cast of an unsigned
type with MSB set
734 to the signed type mpfr_exp_t yields an integer overflow,
which can give
735 unexpected results with future compilers and aggressive
optimisations.
736 Why not working only with signed types, using INT_MIN and
LONG_MIN? */
737 #if __GMP_MP_SIZE_T_INT
738 #define __MPFR_EXP_NAN ((mpfr_exp_t)((~((mpfr_uint_t)0)>>1))
+2))
739 #define __MPFR_EXP_ZERO ((mpfr_exp_t)((~((mpfr_uint_t)0)>>1))
+1))

```



```

740 #define __MPFR_EXP_INF ((mpfr_exp_t)((~((~(mpfr_uint)0)>>1))
+3))
741 #else
742 #define __MPFR_EXP_NAN ((mpfr_exp_t)((~((~(mpfr_u-
long)0)>>1))+2))
743 #define __MPFR_EXP_ZERO ((mpfr_exp_t)((~((~(mpfr_u-
long)0)>>1))+1))
744 #define __MPFR_EXP_INF ((mpfr_exp_t)((~((~(mpfr_u-
long)0)>>1))+3))
745 #endif
746
747 /* Define MPFR_USE_EXTENSION to avoid "gcc -pedantic" warn-
ings. */
748 #ifndef MPFR_EXTENSION
749 # if defined(MPFR_USE_EXTENSION)
750 #  define MPFR_EXTENSION __extension__
751 # else
752 #  define MPFR_EXTENSION
753 # endif
754 #endif
755
756 /* Warning! This macro doesn't work with K&R C (e.g., compare
the "gcc -E"
757 output with and without -traditional) and shouldn't be
used internally.
758 For public use only, but see the MPFR manual. */
759 #define MPFR_DECL_INIT(x, \
p) \
760 MPFR_EXTENSION mp_limb_t __gmpfr_local_tab_##x[((p)-1)/
GMP_NUMB_BITS+1]; \
761 MPFR_EXTENSION mpfr_t _x = {{(p),1,__MP-
FR_EXP_NAN,__gmpfr_local_tab_##x}}
762
763 /* Fast access macros to replace function interface.
764 If the USER don't want to use the macro interface, let him
make happy
765 even if it produces faster and smaller code. */
766 #ifndef MPFR_USE_NO_MACRO
767
768 /* Inlining these functions is both faster and smaller */
769 #define mpfr_nan_p(x) ((x)->_mpfr_exp == __MP-
FR_EXP_NAN)
770 #define mpfr_inf_p(x) ((x)->_mpfr_exp == __MP-
FR_EXP_INF)
771 #define mpfr_zero_p(x) ((x)->_mpfr_exp == __MP-
FR_EXP_ZERO)
772 #define mpfr_regular_p(x) ((x)->_mpfr_exp > __MP-
FR_EXP_INF)
773 #define \
mpfr_sgn(x) \
774 ((x)->_mpfr_exp < __MP-
FR_EXP_INF ? \
775 (mpfr_nan_p(x) ? mpfr_set_erangeflag() : (mpfr_void)
0), 0 : \
776 MPFR_SIGN(x))
777
778 /* Prevent them from using as lvalues */
779 #define MPFR_VALUE_OF(x) (0 ? (x) : (x))
780 #define mpfr_get_prec(x) MPFR_VALUE_OF((x)->_mpfr_prec)
781 #define mpfr_get_exp(x) MPFR_VALUE_OF((x)->_mpfr_exp)
782 /* Note: if need be, the MPFR_VALUE_OF can be used for other
expressions
783 (of any type). Thanks to Wojtek Lerch and Tim Rentsch for
the idea. */
784
785 #define mpfr_round(a,b) mpfr_rint((a), (b), MPFR_RNDNA)
786 #define mpfr_trunc(a,b) mpfr_rint((a), (b), MPFR_RNDZ)
787 #define mpfr_ceil(a,b) mpfr_rint((a), (b), MPFR_RNDU)
788 #define mpfr_floor(a,b) mpfr_rint((a), (b), MPFR_RNDD)
789
790 #define mpfr_cmp_ui(b,i) mpfr_cmp_ui_2exp((b),(i),0)
791 #define mpfr_cmp_si(b,i) mpfr_cmp_si_2exp((b),(i),0)
792 #define mpfr_set(a,b,r) mpfr_set4(a,b,r,MPFR_SIGN(b))
793 #define mpfr_abs(a,b,r) mpfr_set4(a,b,r,1)
794 #define mpfr_copysign(a,b,c,r) mpfr_set4(a,b,r,MPFR_SIGN(c))
795 #define mpfr_setsign(a,b,s,r) mpfr_set4(a,b,r,(s) ? -1 : 1)
796 #define mpfr_signbit(x) (MPFR_SIGN(x) < 0)
797 #define mpfr_cmp(b, c) mpfr_cmp3(b, c, 1)
798 #define mpfr_mul_2exp(y,x,n,r) mpfr_mul_2ui((y),(x),(n),(r))
799 #define mpfr_div_2exp(y,x,n,r) mpfr_div_2ui((y),(x),(n),(r))
800
801
802 /* When using GCC, optimize certain common comparisons and
affectations.
803 + Remove ICC since it defines __GNUC__ but produces a
huge number of warnings if you use this code.
804 VL: I couldn't reproduce a single warning when enabling
these macros
805 with icc 10.1 20080212 on Itanium. But with this ver-
sion, __ICC isn't
806 defined (__INTEL_COMPILER is, though), so that these
macros are enabled
807 anyway. Checking with other ICC versions is needed. Pos-
sibly detect
808 whether warnings are produced or not with a configure
test.
809 + Remove C++ too, since it complains too much. */
810
811 /* Added casts to improve robustness in case of undefined be-

```

```

havior and
812 compiler extensions based on UB (in particular -fwrapv).
MPFR doesn't
813 use such extensions, but these macros will be used by 3rd-
party code,
814 where such extensions may be required.
815 Moreover casts to unsigned long have been added to avoid
warnings in
816 programs that use MPFR and are compiled with -Wconversion;
such casts
817 are OK since if X is a constant expression, then (unsigned
long) X is
818 also a constant expression, so that the optimizations
still work. The
819 warnings are probably related to the following two bugs:
820 http://gcc.gnu.org/bugzilla/show\_bug.cgi?id=4210
821 http://gcc.gnu.org/bugzilla/show\_bug.cgi?id=38470 (pos-
sibly a variant)
822 and the casts could be removed once these bugs are fixed.
823 Casts shouldn't be used on the generic calls (to
the ..._2exp functions),
824 where implicit conversions are performed. Indeed, having
at least one
825 implicit conversion in the macro allows the compiler to
emit diagnostics
826 when normally expected, for instance in the following
call:
827 mpfr_set_ui(x, "foo", MPFR_RNDN);
828 If this is not possible (for future macros), one of the
tricks described
829 on http://groups.google.com/group/comp.std.c/msg/e92ab-
d24bf9eaf7b could
830 be used. */
831 #if defined(__GNUC__) && !defined(__ICC) && !
defined(__cplusplus)
832 #if (__GNUC__ >= 2)
833 #undef mpfr_cmp_ui
834 /* We use the fact that mpfr_sgn on NaN sets the erange flag
and returns 0.
835 But warning! mpfr_sgn is specified as a macro in the API,
thus the macro
836 mustn't be used if side effects are possible, like here.
*/
837 #define \
mpfr_cmp_ui(f,u) \
838 (__builtin_constant_p(u) && (mpfr_ulong)(u) ==
0 ? \
839 (mpfr_sgn) \
840 (f) : \
841 mpfr_cmp_ui_2exp((f),(u),0)) \
842 #define \
mpfr_cmp_si(f,s) \
843 (__builtin_constant_p(s) && (mpfr_ulong)(s) >=
0 ? \
844 (mpfr_cmp_ui((f),(mpfr_ulong)(mpfr_long)
(s))) : \
845 mpfr_cmp_si_2exp((f),(s),0))
846 #if __GNUC__ > 2 || __GNUC_MINOR__ >= 95
847 #undef mpfr_set_ui
848 #define \
mpfr_set_ui(f,u,r) \
849 (__builtin_constant_p(u) && (mpfr_ulong)(u) ==
0 ? \
850 __extension__ \
851 { \
852 mpfr_ptr_p = \
853 (f); \
854 _p->_mpfr_sign = \
855 1; \
856 _p->_mpfr_exp = __MP-
FR_EXP_ZERO; \
857 (mpfr_void)(r); \
858 0; } : \
859 mpfr_set_ui_2exp((f),(u),0,(r)))
860 #endif
861 #define \
mpfr_set_si(f,s,r) \
862 (__builtin_constant_p(s) && (mpfr_ulong)(s) >=
0 ? \
863 (mpfr_set_ui((f),(mpfr_ulong)(mpfr_long)
(s)), \
864 (r)) : \
865 mpfr_set_si_2exp((f),(s),0,(r)))
866 #endif
867 #endif
868 /* Macro version of mpfr_stack interface for fast access */
869 #define mpfr_custom_get_size(p) ((mpfr_s-
ize_t) \
870 (((p)+GMP_NUMB_BITS-1)/GMP_NUMB_BITS*sizeof
(mp_limb_t)))
871 #define mpfr_custom_init(m,p) do { } while (0)
872 #define mpfr_custom_get_significand(x) ((mpfr_void*)((x)->_m-
pfr_d))
873 #define mpfr_custom_get_exp(x) ((x)->_mpfr_exp)
874 #define mpfr_custom_move(x,m) do { ((x)->_mpfr_d =
(mp_limb_t*)(m)); } while (0)

```

```

872 #define mpfr_custom_init_set(x,k,e,p,m) do
{
873     mpfr_ptr _x =
874     mpfr_exp_t
_e;
875     mpfr_kind_t
_t;
876     mpfr_int _s,
_k;
877     _k =
(k);
878     if (_k >= 0)
{
879         _t = (mpfr_kind_t)
_k;
880         _s =
1;
881     } else
{
882         _t = (mpfr_kind_t) -
k;
883         _s =
-1;
884     }
\
885     _e = _t == MPFR_REGULAR_KIND ?
(e) :
886     _t == MPFR_NAN_KIND ? __MP-
FR_EXP_NAN :
887     _t == MPFR_INF_KIND ? __MPFR_EXP_INF : __MP-
FR_EXP_ZERO;
888     _x->_mpfr_prec =
(p);
889     _x->_mpfr_sign =
_s;
890     _x->_mpfr_exp =
_e;
891     _x->_mpfr_d = (mp_limb_t*)
(m);
892     } while (0)
893 #define mpfr_cus-
tom_get_kind(x)
894     ( (x)->_mpfr_exp > __MP-
FR_EXP_INF ?
895     (mpfr_int) MPFR_REGULAR_KIND * MPFR_SIGN
(x)
896     : (x)->_mpfr_exp == __MP-
FR_EXP_INF ?
897     (mpfr_int) MPFR_INF_KIND * MPFR_SIGN
(x)
898     : (x)->_mpfr_exp == __MPFR_EXP_NAN ? (mpfr_int)
MPFR_NAN_KIND
899     : (mpfr_int) MPFR_ZERO_KIND * MPFR_SIGN (x) )
900
901
902 #endif /* MPFR_USE_NO_MACRO */
903
904 /* These are defined to be macros */
905 #define mpfr_init_set_si(x, i, rnd) \
906 ( mpfr_init(x), mpfr_set_si((x), (i), (rnd)) )
907 #define mpfr_init_set_ui(x, i, rnd) \
908 ( mpfr_init(x), mpfr_set_ui((x), (i), (rnd)) )
909 #define mpfr_init_set_d(x, d, rnd) \
910 ( mpfr_init(x), mpfr_set_d((x), (d), (rnd)) )
911 #define mpfr_init_set_ld(x, d, rnd) \
912 ( mpfr_init(x), mpfr_set_ld((x), (d), (rnd)) )
913 #define mpfr_init_set_z(x, i, rnd) \
914 ( mpfr_init(x), mpfr_set_z((x), (i), (rnd)) )
915 #define mpfr_init_set_q(x, i, rnd) \
916 ( mpfr_init(x), mpfr_set_q((x), (i), (rnd)) )
917 #define mpfr_init_set(x, y, rnd) \
918 ( mpfr_init(x), mpfr_set((x), (y), (rnd)) )
919 #define mpfr_init_set_f(x, y, rnd) \
920 ( mpfr_init(x), mpfr_set_f((x), (y), (rnd)) )
921
922 /* Compatibility layer -- obsolete functions and macros */
923 #define mpfr_cmp_abs mpfr_cmpabs
924 #define mpfr_round_prec(x,r,p) mpfr_prec_round(x,p,r)
925 #define __gmp_default_rounding_mode (mpfr_get_de-
fault_rounding_mode())
926 #define __mpfr_emin (mpfr_get_emin())
927 #define __mpfr_emax (mpfr_get_emax())
928 #define __mpfr_default_fp_bit_precision (mpfr_get_de-
fault_fp_bit_precision())
929 #define MPFR_EMIN_MIN mpfr_get_emin_min()
930 #define MPFR_EMIN_MAX mpfr_get_emin_max()
931 #define MPFR_EMAX_MIN mpfr_get_emax_min()
932 #define MPFR_EMAX_MAX mpfr_get_emax_max()
933 #define mpfr_version (mpfr_get_version())
934 #ifndef mpz_set_fr
935 # define mpz_set_fr mpfr_get_z
936 #endif
937 #define mpfr_add_one_ulp(x,r) \
938 (mpfr_sgn(x) > 0 ? mpfr_nextabove(x) : mpfr_nextbelow(x))
939 #define mpfr_sub_one_ulp(x,r) \
940 (mpfr_sgn(x) > 0 ? mpfr_nextbelow(x) : mpfr_nextabove(x))
941 #define mpfr_get_z_exp mpfr_get_z_2exp
942 #define mpfr_custom_get_mantissa mpfr_custom_get_significand

```

```

943
944 #endif /* __MPFR_H*/
945

```

E.48 vendor/utils/std_complex_utils.h (151 lines, 5,699 B)

```

1 // (C) Copyright 2016-2021 Xilinx, Inc.
2 // All Rights Reserved.
3 //
4 // Licensed to the Apache Software Foundation (ASF) under one
5 // or more contributor license agreements. See the NOTICE
file
6 // distributed with this work for additional information
7 // regarding copyright ownership. The ASF licenses this file
8 // to you under the Apache License, Version 2.0 (the
9 // "License"); you may not use this file except in compliance
10 // with the License. You may obtain a copy of the License at
11 //
12 // http://www.apache.org/licenses/LICENSE-2.0
13 //
14 // Unless required by applicable law or agreed to in writing,
15 // software distributed under the License is distributed on
an
16 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 // KIND, either express or implied. See the License for the
18 // specific language governing permissions and limitations
19 // under the License.
20
21 // (C) Copyright 2016-2021 Xilinx, Inc.
22 // All Rights Reserved.
23 //
24 // Licensed to the Apache Software Foundation (ASF) under one
25 // or more contributor license agreements. See the NOTICE
file
26 // distributed with this work for additional information
27 // regarding copyright ownership. The ASF licenses this file
28 // to you under the Apache License, Version 2.0 (the
29 // "License"); you may not use this file except in compliance
30 // with the License. You may obtain a copy of the License at
31 //
32 // http://www.apache.org/licenses/LICENSE-2.0
33 //
34 // Unless required by applicable law or agreed to in writing,
35 // software distributed under the License is distributed on
an
36 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
37 // KIND, either express or implied. See the License for the
38 // specific language governing permissions and limitations
39 // under the License.
40
41
42 #ifndef STD_HLS_COMPLEX_UTILS_H
43 #define STD_HLS_COMPLEX_UTILS_H
44
45 #include <complex>
46 #include "ap_fixed.h"
47 #include "utils/x_hls_defines.h"
48 #include "utils/x_hls_traits.h"
49 #ifndef __SYNTHESIS__
50 #include <iostream>
51 #endif
52
53 namespace hls {
54 /*
55
*****
56 * Vivado HLS complex data type
57 * -----
58 *
59 * std::complex
60 * x_traits for std::complex
61 * set_to_one
62 *
63
*****
64 */
65
66
67 template<typename T1, typename T2>
68 struct x_traits<std::complex<T1>, std::complex<T2> > {
69     typedef std::complex<T1> ADD_T;
70     typedef std::complex<T1> SAT_T;
71     typedef std::complex<T1> MULT_T;
72     typedef std::complex<T1> MADD_T;
73     typedef std::complex<int> INT_T;
74     typedef std::complex<float_struct<24,8> >
FLOAT_STRUCT_T;
75 };
76
77 #define X_COMPLEX_MAX(a, b) ((a) > (b) ? (a) : (b))
78
79 template<int W1, int I1, int W2, int I2>
80 struct x_traits<std::complex<ap_fixed<W1,I1> >, std::com-
plex<ap_fixed<W2,I2> > > {
81     typedef std::com-
plex<ap_fixed<X_COMPLEX_MAX(I1,I2)+1+X_COMPLEX_MAX(W1-I1,W2-
I2),X_COMPLEX_MAX(I1,I2)+1> > ADD_T;
82     typedef std::complex<ap_fixed<W1,I1,AP_RND,AP_SAT> >

```

```

SAT_T;
83     typedef std::complex<ap_fixed<W1+W2+1,I1+I2+1> >
MULT_T;
84     typedef std::complex<ap_fixed<W1+W2+2,I1+I2+2> >
MADD_T;
85     typedef std::complex<ap_fixed<(2*W1)+1,(2*I1)+1> >
CONJ_SQ_T;
86     typedef std::complex<int>
INT_T;
87     typedef std::complex<float_struct<24,8> >
FLOAT_STRUCT_T;
88     };
89
90     template<int W1, int W2>
91     struct x_traits<std::complex<ap_int<W1> >,std::com-
plex<ap_int<W2> > > {
92     typedef std::complex<ap_int<X_COMPLEX_MAX(W1,W2)+1> >
ADD_T;
93     typedef std::complex<ap_int<W1> >
SAT_T;
94     typedef std::complex<ap_int<W1+W2+1> >
MULT_T;
95     typedef std::complex<ap_int<W1+W2+2> >
MADD_T;
96     typedef std::complex<ap_int<(2*W1)+1> >
CONJ_SQ_T;
97     typedef std::complex<int>
INT_T;
98     typedef std::complex<float_struct<24,8> >
FLOAT_STRUCT_T;
99     };
100
101 #undef X_COMPLEX_MAX
102
103     template<int DIM, typename T>
104     struct x_traits_d <std::complex<T>,DIM> {
105     typedef std::complex<T> ACCUM_T;
106     typedef std::complex<T> ACCUM2_T;
107     typedef std::complex<ap_int<25+DIM> > ADD_TREE_T;
108     };
109
110     template<int DIM, int W, int I>
111     struct x_traits<std::complex<ap_fixed<W,I> >,DIM> {
112     typedef std::com-
plex<ap_fixed<W+BitWidth<DIM>::Value,I+BitWidth<DIM>::Value>
> ACCUM_T;
113     typedef std::complex<ap_fixed<(2*W)
+BitWidth<DIM>::Value,(2*I)+BitWidth<DIM>::Value> > ACCUM2_T;
114     };
115
116     template<typename T> T x_neg(T &din);
117
118     static float x_neg(float din);
119
120     static double x_neg(double din);
121
122     template<typename T> std::complex<T> x_neg(std::complex<T>
&din);
123
124     template<typename T> typename x_traits<T,T>::MULT_T
x_conj_sq(T &din);
125
126     template<typename T> typename x_traits<T,T>::MADD_T
x_conj_sq(std::complex<T> &din);
127
128     template<typename T> T x_conj(T &din);
129
130     template<typename T> std::complex<T> x_conj(const std::com-
plex<T> &din);
131
132
133 // This function is called when a non-const std::complex is
conjugated. It calls the
134 // version of x_conj() which takes a constant parameter
135
136 template<typename T> std::complex<T> x_conj(std::complex<T>
&din);
137
138 template<int W, int I> void set_to_one(std::com-
plex<ap_fixed<W,I> > &a);
139
140 // Functions to extract real and imaginary components
141 template<typename T> T x_real(const T &din);
142 template<typename T> T x_imag(const T &din);
143 template<typename T> T x_real(const std::complex<T> &din);
144 template<typename T> T x_imag(const std::complex<T> &din);
145 } // end namespace hls
146
147 #endif
148
149
150
151

```

E.49 vendor/utis/x_hls_defines.h (137 lines, 4,143 B)

```

1 // (C) Copyright 2016-2021 Xilinx, Inc.
2 // All Rights Reserved.

```

```

3 //
4 // Licensed to the Apache Software Foundation (ASF) under one
5 // or more contributor license agreements. See the NOTICE
file
6 // distributed with this work for additional information
7 // regarding copyright ownership. The ASF licenses this file
8 // to you under the Apache License, Version 2.0 (the
9 // "License"); you may not use this file except in compliance
10 // with the License. You may obtain a copy of the License at
11 //
12 // http://www.apache.org/licenses/LICENSE-2.0
13 //
14 // Unless required by applicable law or agreed to in writing,
15 // software distributed under the License is distributed on
an
16 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 // KIND, either express or implied. See the License for the
18 // specific language governing permissions and limitations
19 // under the License.
20
21 // (C) Copyright 2016-2021 Xilinx, Inc.
22 // All Rights Reserved.
23 //
24 // Licensed to the Apache Software Foundation (ASF) under one
25 // or more contributor license agreements. See the NOTICE
file
26 // distributed with this work for additional information
27 // regarding copyright ownership. The ASF licenses this file
28 // to you under the Apache License, Version 2.0 (the
29 // "License"); you may not use this file except in compliance
30 // with the License. You may obtain a copy of the License at
31 //
32 // http://www.apache.org/licenses/LICENSE-2.0
33 //
34 // Unless required by applicable law or agreed to in writing,
35 // software distributed under the License is distributed on
an
36 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
37 // KIND, either express or implied. See the License for the
38 // specific language governing permissions and limitations
39 // under the License.
40
41
42 /**
43  * @file x_hls_defines.h
44  */
45
46 #ifndef X_HLS_DEFINES_H
47 #define X_HLS_DEFINES_H
48
49 #include "ap_int.h"
50
51 /*
52
*****
53 *
54 * Contains global defines, macros, and structs.
55 *
56
*****
57 */
58
59 #define PRAGMA(x) _Pragma(#x)
60 /*#define X_PIPELINE(ii) {\
61     PRAGMA(AP PIPELINE II=ii)\
62 }
63 */
64 // #define NO_FORCING
65
66 // #define X_AP_Q_MODE AP_TRN
67 // #define X_AP_O_MODE AP_WRAP
68
69 /*
70 * May be used to specify all intantiated ap_fixed to have
different options.
71 * Currently unused.
72
73
74
75 #define COMPARE_APPROX_SGEMM_BASE_FLOAT 0.01
76 #define COMPARE_APPROX_CGEMM_BASE_FLOAT 0.01
77 #define COMPARE_APPROX_SGEQRF_BASE_FLOAT 0.01
78 #define COMPARE_APPROX_CGEQRF_BASE_FLOAT 0.01
79 #define COMPARE_APPROX_SBSUT_BASE_FLOAT 0.01
80 #define COMPARE_APPROX_CBSUT_BASE_FLOAT 0.01
81 #define COMPARE_APPROX_SGETRI_BASE_FLOAT 10.0
82 #define COMPARE_APPROX_CGETRI_BASE_FLOAT 10.0
83 #define COMPARE_APPROX_SPBTRF_BASE_FLOAT 0.01
84 #define COMPARE_APPROX_CPBTRF_BASE_FLOAT 0.01
85 #define COMPARE_APPROX_CPBTRI_BASE_FLOAT 100.0
86 #define COMPARE_APPROX_SGESVD_BASE_FLOAT 0.01
87 #define COMPARE_APPROX_CGESVD_BASE_FLOAT 1.00
88 #define COMPARE_APPROX_CPOSV_BASE_FLOAT 200.00
89
90 #define COMPARE_APPROX_SGEMM_BASE_FIXED 0.01
91 #define COMPARE_APPROX_CGEMM_BASE_FIXED 0.01
92 #define COMPARE_APPROX_SGEQRF_BASE_FIXED 0.01
93 #define COMPARE_APPROX_CGEQRF_BASE_FIXED 0.01

```

```

94 #define COMPARE_APPROX_SBSUT_BASE_FIXED 0.01
95 #define COMPARE_APPROX_CBSUT_BASE_FIXED 0.01
96 #define COMPARE_APPROX_SGETRI_BASE_FIXED 10.0
97 #define COMPARE_APPROX_CGETRI_BASE_FIXED 10.0
98 #define COMPARE_APPROX_SPBTRF_BASE_FIXED 0.01
99 #define COMPARE_APPROX_CPBTRF_BASE_FIXED 100.00
100 #define COMPARE_APPROX_SGESVD_BASE_FIXED 0.01
101 #define COMPARE_APPROX_CGESVD_BASE_FIXED 1.00
102
103 #define COMPARE_APPROX_ERROR_FLOAT 0.10 // in percentage
104
105 #define COMPARE_APPROX_ERROR_FIXED 40.00
106 */
107
108 /*
109 * Macros
110 */
111 // #define __HLS_MAX(x,y) ((x) > (y) ? (x) : (y))
112 // #define __HLS_MIN(x,y) ((x) < (y) ? (x) : (y))
113
114 /*
115 * Floating point data structure (with unsigned mantissa)
116 */
117 template<int M, int E>
118 struct float_struct {
119     ap_uint<M> mant; // 23
120     ap_uint<E> exp; // 8
121     ap_uint<1> sign;
122 };
123
124 /*
125 * Floating point data structure (with signed mantissa)
126 */
127 template<int M, int E>
128 struct float_struct2 {
129     ap_int<M> mant; // 24
130     ap_uint<E> exp; // 8
131 };
132
133 #endif
134
135
136
137

```

E.50 vendor/utils/x_hls_traits.h (1,132 lines, 38,531 B)

```

1 // (C) Copyright 2016-2021 Xilinx, Inc.
2 // All Rights Reserved.
3 //
4 // Licensed to the Apache Software Foundation (ASF) under one
5 // or more contributor license agreements. See the NOTICE
file
6 // distributed with this work for additional information
7 // regarding copyright ownership. The ASF licenses this file
8 // to you under the Apache License, Version 2.0 (the
9 // "License"); you may not use this file except in compliance
10 // with the License. You may obtain a copy of the License at
11 //
12 // http://www.apache.org/licenses/LICENSE-2.0
13 //
14 // Unless required by applicable law or agreed to in writing,
15 // software distributed under the License is distributed on
an
16 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 // KIND, either express or implied. See the License for the
18 // specific language governing permissions and limitations
19 // under the License.
20
21 // (C) Copyright 2016-2021 Xilinx, Inc.
22 // All Rights Reserved.
23 //
24 // Licensed to the Apache Software Foundation (ASF) under one
25 // or more contributor license agreements. See the NOTICE
file
26 // distributed with this work for additional information
27 // regarding copyright ownership. The ASF licenses this file
28 // to you under the Apache License, Version 2.0 (the
29 // "License"); you may not use this file except in compliance
30 // with the License. You may obtain a copy of the License at
31 //
32 // http://www.apache.org/licenses/LICENSE-2.0
33 //
34 // Unless required by applicable law or agreed to in writing,
35 // software distributed under the License is distributed on
an
36 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
37 // KIND, either express or implied. See the License for the
38 // specific language governing permissions and limitations
39 // under the License.
40
41
42 #ifndef X_HLS_TRAITS_H
43 #define X_HLS_TRAITS_H
44
45 #include "ap_fixed.h"
46 #include "ap_int.h"
47 // #include "hls_half.h"
48 #include "x_hls_utils.h"
49 #include "x_hls_defines.h"

```

```

50 #include <stdint.h>
51
52 class half;
53 namespace hls
54 {
55
56 template<typename T>
57 struct x_traits_default {
58     typedef T ADD_T;
59     typedef T SUB_T;
60     typedef T SAT_T;
61     typedef T MULT_T;
62     typedef T MADD_T;
63 };
64
65 template<typename T1, typename T2>
66 struct x_traits {
67     // typedef T1 ADD_T;
68     // typedef T1 SAT_T;
69     // typedef T1 MULT_T;
70     // typedef T1 MADD_T;
71     // typedef T1 CMADD_T;
72     // typedef T1 CONJ_SQ_T;
73     // typedef T1 CONJ_SQ_2_T;
74     // typedef int INT_T;
75     // typedef T1 INV_T;
76     // typedef float_struct<24,8> FLOAT_STRUCT_T;
77 };
78
79 template<typename T2>
80 struct x_traits<double, T2> : public x_traits_default<double>
81 { };
82
83 template<typename T1>
84 struct x_traits<T1, double> : public x_traits_default<double>
85 { };
86
87 template<typename T2>
88 struct x_traits<float, T2> : public x_traits_default<float>
89 { };
90
91 template<typename T1>
92 struct x_traits<T1, float> : public x_traits_default<float>
93 { };
94
95 template<typename T2>
96 struct x_traits<half, T2> : public x_traits_default<float>
97 { };
98
99 template<typename T1>
100 struct x_traits<T1, half> : public x_traits_default<float>
101 { };
102
103 //-----//
104 /* enumeration c data types:
105 * char, signed char, unsigned char,
106 * short, unsigned short,
107 * int, unsigned int,
108 * long, unsigned long,
109 * ap_long, ap_ulong,
110 * half,
111 * float,
112 * double
113 */
114
115 template<>
116 struct x_traits<char, char> : public x_traits_default<
char > { };
117
118 template<>
119 struct x_traits<char, signed char> : public
x_traits_default<signed char> { };
120
121 template<>
122 struct x_traits<char, unsigned char> : public
x_traits_default<unsigned char> { };
123
124 template<>
125 struct x_traits<char, short> : public x_traits_default<
short > { };
126
127 template<>
128 struct x_traits<char, unsigned short> : public
x_traits_default<unsigned short> { };
129
130 template<>
131 struct x_traits<char, int> : public x_traits_default<int
> { };
132
133 template<>
134 struct x_traits<char, long> : public x_traits_default<

```

```

long > {};
134
135 template<
136 struct x_traits< char , unsigned long > : public
x_traits_default< unsigned long > {};
137
138 template<
139 struct x_traits< char , ap_slong > : public x_traits_default<
ap_slong > {};
140
141 template<
142 struct x_traits< char , ap_ulong > : public x_traits_default<
ap_ulong > {};
143
144 template<
145 struct x_traits< char , half > : public x_traits_default<
float > {};
146
147 template<
148 struct x_traits< char , float > : public x_traits_default<
float > {};
149
150 template<
151 struct x_traits< char , double > : public x_traits_default<
double > {};
152
153 template<
154 struct x_traits< signed char , char > : public
x_traits_default< signed char > {};
155
156 template<
157 struct x_traits< signed char , signed char > : public
x_traits_default< signed char > {};
158
159 template<
160 struct x_traits< signed char , unsigned char > : public
x_traits_default< unsigned char > {};
161
162 template<
163 struct x_traits< signed char , short > : public
x_traits_default< short > {};
164
165 template<
166 struct x_traits< signed char , unsigned short > : public
x_traits_default< unsigned short > {};
167
168 template<
169 struct x_traits< signed char , int > : public
x_traits_default< int > {};
170
171 template<
172 struct x_traits< signed char , unsigned int > : public
x_traits_default< unsigned int > {};
173
174 template<
175 struct x_traits< signed char , long > : public
x_traits_default< long > {};
176
177 template<
178 struct x_traits< signed char , unsigned long > : public
x_traits_default< unsigned long > {};
179
180 template<
181 struct x_traits< signed char , ap_slong > : public
x_traits_default< ap_slong > {};
182
183 template<
184 struct x_traits< signed char , ap_ulong > : public
x_traits_default< ap_ulong > {};
185
186 template<
187 struct x_traits< signed char , half > : public
x_traits_default< float > {};
188
189 template<
190 struct x_traits< signed char , float > : public
x_traits_default< float > {};
191
192 template<
193 struct x_traits< signed char , double > : public
x_traits_default< double > {};
194
195 template<
196 struct x_traits< unsigned char , char > : public
x_traits_default< unsigned char > {};
197
198 template<
199 struct x_traits< unsigned char , signed char > : public
x_traits_default< unsigned char > {};
200
201 template<
202 struct x_traits< unsigned char , unsigned char > : public
x_traits_default< unsigned char > {};
203
204 template<
205 struct x_traits< unsigned char , short > : public
x_traits_default< short > {};
206

```

```

207 template<
208 struct x_traits< unsigned char , unsigned short > : public
x_traits_default< unsigned short > {};
209
210 template<
211 struct x_traits< unsigned char , int > : public
x_traits_default< int > {};
212
213 template<
214 struct x_traits< unsigned char , unsigned int > : public
x_traits_default< unsigned int > {};
215
216 template<
217 struct x_traits< unsigned char , long > : public
x_traits_default< long > {};
218
219 template<
220 struct x_traits< unsigned char , unsigned long > : public
x_traits_default< unsigned long > {};
221
222 template<
223 struct x_traits< unsigned char , ap_slong > : public
x_traits_default< ap_slong > {};
224
225 template<
226 struct x_traits< unsigned char , ap_ulong > : public
x_traits_default< ap_ulong > {};
227
228 template<
229 struct x_traits< unsigned char , half > : public
x_traits_default< float > {};
230
231 template<
232 struct x_traits< unsigned char , float > : public
x_traits_default< float > {};
233
234 template<
235 struct x_traits< unsigned char , double > : public
x_traits_default< double > {};
236
237 template<
238 struct x_traits< short , char > : public x_traits_default<
short > {};
239
240 template<
241 struct x_traits< short , signed char > : public
x_traits_default< short > {};
242
243 template<
244 struct x_traits< short , unsigned char > : public
x_traits_default< short > {};
245
246 template<
247 struct x_traits< short , short > : public x_traits_default<
short > {};
248
249 template<
250 struct x_traits< short , unsigned short > : public
x_traits_default< unsigned short > {};
251
252 template<
253 struct x_traits< short , int > : public x_traits_default< int >
{};
254
255 template<
256 struct x_traits< short , unsigned int > : public
x_traits_default< unsigned int > {};
257
258 template<
259 struct x_traits< short , long > : public x_traits_default<
long > {};
260
261 template<
262 struct x_traits< short , unsigned long > : public
x_traits_default< unsigned long > {};
263
264 template<
265 struct x_traits< short , ap_slong > : public
x_traits_default< ap_slong > {};
266
267 template<
268 struct x_traits< short , ap_ulong > : public
x_traits_default< ap_ulong > {};
269
270 template<
271 struct x_traits< short , half > : public x_traits_default<
float > {};
272
273 template<
274 struct x_traits< short , float > : public x_traits_default<
float > {};
275
276 template<
277 struct x_traits< short , double > : public x_traits_default<
double > {};
278
279 template<
280 struct x_traits< unsigned short , char > : public
x_traits_default< unsigned short > {};

```



```

281
282 template<
283 struct x_traits< unsigned short , signed char > : public
x_traits_default< unsigned short > {};
284
285 template<
286 struct x_traits< unsigned short , unsigned char > : public
x_traits_default< unsigned short > {};
287
288 template<
289 struct x_traits< unsigned short , short > : public
x_traits_default< unsigned short > {};
290
291 template<
292 struct x_traits< unsigned short , unsigned short > : public
x_traits_default< unsigned short > {};
293
294 template<
295 struct x_traits< unsigned short , int > : public
x_traits_default< int > {};
296
297 template<
298 struct x_traits< unsigned short , unsigned int > : public
x_traits_default< unsigned int > {};
299
300 template<
301 struct x_traits< unsigned short , long > : public
x_traits_default< long > {};
302
303 template<
304 struct x_traits< unsigned short , unsigned long > : public
x_traits_default< unsigned long > {};
305
306 template<
307 struct x_traits< unsigned short , ap_slong > : public
x_traits_default< ap_slong > {};
308
309 template<
310 struct x_traits< unsigned short , ap_ulong > : public
x_traits_default< ap_ulong > {};
311
312 template<
313 struct x_traits< unsigned short , half > : public
x_traits_default< float > {};
314
315 template<
316 struct x_traits< unsigned short , float > : public
x_traits_default< float > {};
317
318 template<
319 struct x_traits< unsigned short , double > : public
x_traits_default< double > {};
320
321 template<
322 struct x_traits< int , char > : public x_traits_default< int
> {};
323
324 template<
325 struct x_traits< int , signed char > : public
x_traits_default< int > {};
326
327 template<
328 struct x_traits< int , unsigned char > : public
x_traits_default< int > {};
329
330 template<
331 struct x_traits< int , short > : public x_traits_default< int >
{};
332
333 template<
334 struct x_traits< int , unsigned short > : public
x_traits_default< int > {};
335
336 template<
337 struct x_traits< int , int > : public x_traits_default< int >
{};
338
339 template<
340 struct x_traits< int , unsigned int > : public
x_traits_default< unsigned int > {};
341
342 template<
343 struct x_traits< int , long > : public x_traits_default< long >
{};
344
345 template<
346 struct x_traits< int , unsigned long > : public
x_traits_default< unsigned long > {};
347
348 template<
349 struct x_traits< int , ap_slong > : public x_traits_default<
ap_slong > {};
350
351 template<
352 struct x_traits< int , ap_ulong > : public x_traits_default<
ap_ulong > {};
353

```

```

354 template<
355 struct x_traits< int , half > : public x_traits_default<
float > {};
356
357 template<
358 struct x_traits< int , float > : public x_traits_default<
float > {};
359
360 template<
361 struct x_traits< int , double > : public x_traits_default<
double > {};
362
363 template<
364 struct x_traits< unsigned int , char > : public
x_traits_default< unsigned int > {};
365
366 template<
367 struct x_traits< unsigned int , signed char > : public
x_traits_default< unsigned int > {};
368
369 template<
370 struct x_traits< unsigned int , unsigned char > : public
x_traits_default< unsigned int > {};
371
372 template<
373 struct x_traits< unsigned int , short > : public
x_traits_default< unsigned int > {};
374
375 template<
376 struct x_traits< unsigned int , unsigned short > : public
x_traits_default< unsigned int > {};
377
378 template<
379 struct x_traits< unsigned int , int > : public
x_traits_default< unsigned int > {};
380
381 template<
382 struct x_traits< unsigned int , unsigned int > : public
x_traits_default< unsigned int > {};
383
384 template<
385 struct x_traits< unsigned int , long > : public
x_traits_default< long > {};
386
387 template<
388 struct x_traits< unsigned int , unsigned long > : public
x_traits_default< unsigned long > {};
389
390 template<
391 struct x_traits< unsigned int , ap_slong > : public
x_traits_default< ap_slong > {};
392
393 template<
394 struct x_traits< unsigned int , ap_ulong > : public
x_traits_default< ap_ulong > {};
395
396 template<
397 struct x_traits< unsigned int , half > : public
x_traits_default< float > {};
398
399 template<
400 struct x_traits< unsigned int , float > : public
x_traits_default< float > {};
401
402 template<
403 struct x_traits< unsigned int , double > : public
x_traits_default< double > {};
404
405 template<
406 struct x_traits< long , char > : public x_traits_default<
long > {};
407
408 template<
409 struct x_traits< long , signed char > : public
x_traits_default< long > {};
410
411 template<
412 struct x_traits< long , unsigned char > : public
x_traits_default< long > {};
413
414 template<
415 struct x_traits< long , short > : public x_traits_default<
long > {};
416
417 template<
418 struct x_traits< long , unsigned short > : public
x_traits_default< long > {};
419
420 template<
421 struct x_traits< long , int > : public x_traits_default< long >
{};
422
423 template<
424 struct x_traits< long , unsigned int > : public
x_traits_default< long > {};
425
426 template<
427 struct x_traits< long , long > : public x_traits_default<
long > {};

```

```

428
429 template<
430 struct x_traits< long , unsigned long > : public
x_traits_default< unsigned long > {};
431
432 template<
433 struct x_traits< long , ap_slong > : public x_traits_default<
ap_slong > {};
434
435 template<
436 struct x_traits< long , ap_ulong > : public x_traits_default<
ap_ulong > {};
437
438 template<
439 struct x_traits< long , half > : public x_traits_default<
float > {};
440
441 template<
442 struct x_traits< long , float > : public x_traits_default<
float > {};
443
444 template<
445 struct x_traits< long , double > : public x_traits_default<
double > {};
446
447 template<
448 struct x_traits< unsigned long , char > : public
x_traits_default< unsigned long > {};
449
450 template<
451 struct x_traits< unsigned long , signed char > : public
x_traits_default< unsigned long > {};
452
453 template<
454 struct x_traits< unsigned long , unsigned char > : public
x_traits_default< unsigned long > {};
455
456 template<
457 struct x_traits< unsigned long , short > : public
x_traits_default< unsigned long > {};
458
459 template<
460 struct x_traits< unsigned long , unsigned short > : public
x_traits_default< unsigned long > {};
461
462 template<
463 struct x_traits< unsigned long , int > : public
x_traits_default< unsigned long > {};
464
465 template<
466 struct x_traits< unsigned long , unsigned int > : public
x_traits_default< unsigned long > {};
467
468 template<
469 struct x_traits< unsigned long , long > : public
x_traits_default< unsigned long > {};
470
471 template<
472 struct x_traits< unsigned long , unsigned long > : public
x_traits_default< unsigned long > {};
473
474 template<
475 struct x_traits< unsigned long , ap_slong > : public
x_traits_default< ap_slong > {};
476
477 template<
478 struct x_traits< unsigned long , ap_ulong > : public
x_traits_default< ap_ulong > {};
479
480 template<
481 struct x_traits< unsigned long , half > : public
x_traits_default< float > {};
482
483 template<
484 struct x_traits< unsigned long , float > : public
x_traits_default< float > {};
485
486 template<
487 struct x_traits< unsigned long , double > : public
x_traits_default< double > {};
488
489 template<
490 struct x_traits< ap_slong , char > : public x_traits_default<
ap_slong > {};
491
492 template<
493 struct x_traits< ap_slong , signed char > : public
x_traits_default< ap_slong > {};
494
495 template<
496 struct x_traits< ap_slong , unsigned char > : public
x_traits_default< ap_slong > {};
497
498 template<
499 struct x_traits< ap_slong , short > : public
x_traits_default< ap_slong > {};
500

```

```

501 template<
502 struct x_traits< ap_slong , unsigned short > : public
x_traits_default< ap_slong > {};
503
504 template<
505 struct x_traits< ap_slong , int > : public x_traits_default<
ap_slong > {};
506
507 template<
508 struct x_traits< ap_slong , unsigned int > : public
x_traits_default< ap_slong > {};
509
510 template<
511 struct x_traits< ap_slong , long > : public x_traits_default<
ap_slong > {};
512
513 template<
514 struct x_traits< ap_slong , unsigned long > : public
x_traits_default< ap_slong > {};
515
516 template<
517 struct x_traits< ap_slong , ap_slong > : public
x_traits_default< ap_slong > {};
518
519 template<
520 struct x_traits< ap_slong , ap_ulong > : public
x_traits_default< ap_ulong > {};
521
522 template<
523 struct x_traits< ap_slong , half > : public x_traits_default<
float > {};
524
525 template<
526 struct x_traits< ap_slong , float > : public
x_traits_default< float > {};
527
528 template<
529 struct x_traits< ap_slong , double > : public
x_traits_default< double > {};
530
531 template<
532 struct x_traits< ap_ulong , char > : public x_traits_default<
ap_ulong > {};
533
534 template<
535 struct x_traits< ap_ulong , signed char > : public
x_traits_default< ap_ulong > {};
536
537 template<
538 struct x_traits< ap_ulong , unsigned char > : public
x_traits_default< ap_ulong > {};
539
540 template<
541 struct x_traits< ap_ulong , short > : public
x_traits_default< ap_ulong > {};
542
543 template<
544 struct x_traits< ap_ulong , unsigned short > : public
x_traits_default< ap_ulong > {};
545
546 template<
547 struct x_traits< ap_ulong , int > : public x_traits_default<
ap_ulong > {};
548
549 template<
550 struct x_traits< ap_ulong , unsigned int > : public
x_traits_default< ap_ulong > {};
551
552 template<
553 struct x_traits< ap_ulong , long > : public x_traits_default<
ap_ulong > {};
554
555 template<
556 struct x_traits< ap_ulong , unsigned long > : public
x_traits_default< ap_ulong > {};
557
558 template<
559 struct x_traits< ap_ulong , ap_slong > : public
x_traits_default< ap_ulong > {};
560
561 template<
562 struct x_traits< ap_ulong , ap_ulong > : public
x_traits_default< ap_ulong > {};
563
564 template<
565 struct x_traits< ap_ulong , half > : public x_traits_default<
float > {};
566
567 template<
568 struct x_traits< ap_ulong , float > : public
x_traits_default< float > {};
569
570 template<
571 struct x_traits< ap_ulong , double > : public
x_traits_default< double > {};

```



```

572 template<
573 struct x_traits< half , char > : public x_traits_default<
float > {};
575 template<
576 struct x_traits< half , signed char > : public
x_traits_default< float > {};
578 template<
579 struct x_traits< half , unsigned char > : public
x_traits_default< float > {};
581 template<
582 struct x_traits< half , short > : public x_traits_default<
float > {};
584 template<
585 struct x_traits< half , unsigned short > : public
x_traits_default< float > {};
587 template<
588 struct x_traits< half , int > : public x_traits_default<
float > {};
590 template<
591 struct x_traits< half , unsigned int > : public
x_traits_default< float > {};
593 template<
594 struct x_traits< half , long > : public x_traits_default<
float > {};
596 template<
597 struct x_traits< half , unsigned long > : public
x_traits_default< float > {};
599 template<
600 struct x_traits< half , ap_slong > : public x_traits_default<
float > {};
602 template<
603 struct x_traits< half , ap_ulong > : public x_traits_default<
float > {};
605 template<
606 struct x_traits< half , half > : public x_traits_default<
half > {};
608 template<
609 struct x_traits< half , float > : public x_traits_default<
float > {};
611 template<
612 struct x_traits< half , double > : public x_traits_default<
double > {};
614 template<
615 struct x_traits< float , char > : public x_traits_default<
float > {};
617 template<
618 struct x_traits< float , signed char > : public
x_traits_default< float > {};
620 template<
621 struct x_traits< float , unsigned char > : public
x_traits_default< float > {};
623 template<
624 struct x_traits< float , short > : public x_traits_default<
float > {};
626 template<
627 struct x_traits< float , unsigned short > : public
x_traits_default< float > {};
629 template<
630 struct x_traits< float , int > : public x_traits_default<
float > {};
632 template<
633 struct x_traits< float , unsigned int > : public
x_traits_default< float > {};
635 template<
636 struct x_traits< float , long > : public x_traits_default<
float > {};
638 template<
639 struct x_traits< float , unsigned long > : public
x_traits_default< float > {};
641 template<
642 struct x_traits< float , ap_slong > : public
x_traits_default< float > {};
644 template<
645 struct x_traits< float , ap_ulong > : public
x_traits_default< float > {};

```

```

646 struct x_traits< float , ap_ulong > : public
x_traits_default< float > {};
647 template<
648 struct x_traits< float , half > : public x_traits_default<
float > {};
650 template<
651 struct x_traits< float , float > : public x_traits_default<
float > {};
653 template<
654 struct x_traits< float , double > : public x_traits_default<
double > {};
656 template<
657 struct x_traits< double , char > : public x_traits_default<
double > {};
659 template<
660 struct x_traits< double , signed char > : public
x_traits_default< double > {};
662 template<
663 struct x_traits< double , unsigned char > : public
x_traits_default< double > {};
665 template<
666 struct x_traits< double , short > : public x_traits_default<
double > {};
668 template<
669 struct x_traits< double , unsigned short > : public
x_traits_default< double > {};
671 template<
672 struct x_traits< double , int > : public x_traits_default<
double > {};
674 template<
675 struct x_traits< double , unsigned int > : public
x_traits_default< double > {};
677 template<
678 struct x_traits< double , long > : public x_traits_default<
double > {};
680 template<
681 struct x_traits< double , unsigned long > : public
x_traits_default< double > {};
683 template<
684 struct x_traits< double , ap_slong > : public
x_traits_default< double > {};
686 template<
687 struct x_traits< double , ap_ulong > : public
x_traits_default< double > {};
689 template<
690 struct x_traits< double , half > : public x_traits_default<
double > {};
692 template<
693 struct x_traits< double , float > : public x_traits_default<
double > {};
695 template<
696 struct x_traits< double , double > : public x_traits_default<
double > {};
698
699
700
701 //-----//
702
703 template<int A,int B> class MAX{
704 public:
705     const static int value = (A > B) ? A : B;
706 };
707
708 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
709 struct x_traits<ap_fixed<W1,I1,Q1,O1>,ap_fixed<W2,I2,Q2,O2> > {
710     typedef typename ap_fixed<W1,I1>::template
RTYPE<W2,I2,true> RTYPE;
711     typedef ap_fixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
712     typedef ap_fixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
713     typedef ap_fixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
714     typedef ap_fixed<W1,I1,AP_RND,AP_SAT> SAT_T;
715     typedef ap_fixed<W1+W2+1,I1+I2+1>
MADD_T;
716     typedef ap_fixed<W1+W2+2,I1+I2+2>
CMADD_T;
717     typedef ap_fixed<(2*W1)+1,(2*I1)>
CONJ_SQ_T;
718     typedef ap_fixed<(2*W1)+2,(2*I1)+1> CONJ_SQ_2_T;
719     typedef ap_fixed<2*(MAX<I1,W1-I1>::value+1),(MAX<I1,W1-

```

```

I1>::value+1)> INV_T;
720     typedef int INT_T;
721     typedef float_struct<24,8> FLOAT_STRUCT_T;
722 };
723
724 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
725 struct x_traits<ap_fixed<W1,I1,Q1,O1>,ap_ufixed<W2,I2,Q2,O2>
> {
726     typedef typename ap_fixed<W1,I1>::template
RType<W2,I2,false> RTYPE;
727     typedef ap_fixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
728     typedef ap_fixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
729     typedef ap_fixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
730 };
731
732 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
733 struct x_traits<ap_ufixed<W1,I1,Q1,O1>,ap_fixed<W2,I2,Q2,O2>
> {
734     typedef typename ap_ufixed<W1,I1>::template
RType<W2,I2,true> RTYPE;
735     typedef ap_fixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
736     typedef ap_fixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
737     typedef ap_fixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
738 };
739
740 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
741 struct x_traits<ap_ufixed<W1,I1,Q1,O1>,ap_ufixed<W2,I2,Q2,O2>
> {
742     typedef typename ap_ufixed<W1,I1>::template
RType<W2,I2,false> RTYPE;
743     typedef ap_ufixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
744     typedef ap_ufixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
745     typedef ap_ufixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
746     typedef ap_ufixed<W1,I1,AP_RND,AP_SAT> SAT_T;
747     typedef ap_ufixed<W1+W2+1,I1+I2+1>
MADD_T;
748     typedef ap_ufixed<W1+W2+2,I1+I2+2>
CMADD_T;
749     typedef ap_ufixed<(2*W1)+1,(2*I1)>
CONJ_SQ_2_T;
750     typedef ap_ufixed<(2*W1)+2,(2*I1)+1> CONJ_SQ_2_T;
751     typedef ap_ufixed<2*(MAX<I1,W1-I1>::value+1),(MAX<I1,W1-
I1>::value+1)> INV_T;
752     typedef int INT_T;
753     typedef float_struct<24,8> FLOAT_STRUCT_T;
754 };
755
756 template<int W1, int W2>
757 struct x_traits<ap_int<W1>,ap_int<W2> > {
758     typedef ap_int<MAX<W1,W2>::value + 1>
ADD_T;
759     typedef ap_int<MAX<W1,W2>::value + 1>
SUB_T;
760     typedef ap_int<W1+W2> MULT_T;
761     typedef ap_int<(W1 > W2)? W1:W2> MADD_T;
762 };
763
764 template<int W1, int W2>
765 struct x_traits<ap_int<W1>,ap_uint<W2> > {
766     typedef ap_int<MAX<W1,W2+1>::value + 1>
ADD_T;
767     typedef ap_int<MAX<W1,W2+1>::value + 1>
SUB_T;
768     typedef ap_int<W1+W2> MULT_T;
769 };
770
771 template<int W1, int W2>
772 struct x_traits<ap_uint<W1>,ap_int<W2> > {
773     typedef ap_int<MAX<W1+1,W2>::value + 1>
ADD_T;
774     typedef ap_int<MAX<W1+1,W2>::value + 1>
SUB_T;
775     typedef ap_int<W1+W2> MULT_T;
776 };
777
778 template<int W1, int W2>
779 struct x_traits<ap_uint<W1>,ap_uint<W2> > {
780     typedef ap_uint<MAX<W1,W2>::value + 1>
ADD_T;
781     typedef ap_int<MAX<W1,W2>::value + 1>
SUB_T;
782     typedef ap_uint<W1+W2> MULT_T;
783 };
784
785 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
786 struct x_traits<ap_fixed<W1,I1,Q1,O1>,ap_int<W2> > : public
x_traits<ap_fixed<W1,I1>, ap_fixed<W2,W2> > {};
787

```

```

788 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
789 struct x_traits<ap_fixed<W1,I1,Q1,O1>,ap_uint<W2> > : public
x_traits<ap_fixed<W1,I1>, ap_ufixed<W2,W2> > {};
790
791 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
792 struct x_traits<ap_ufixed<W1,I1,Q1,O1>,ap_int<W2> > : public
x_traits<ap_ufixed<W1,I1>, ap_fixed<W2,W2> > {};
793
794 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
795 struct x_traits<ap_ufixed<W1,I1,Q1,O1>,ap_uint<W2> > : public
x_traits<ap_ufixed<W1,I1>, ap_ufixed<W2,W2> > {};
796
797 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
798 struct x_traits<ap_int<W1>, ap_fixed<W2,I2,Q2,O2> > : public
x_traits<ap_fixed<W1,W1>, ap_fixed<W2,I2> > {};
799
800 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
801 struct x_traits<ap_int<W1>, ap_ufixed<W2,I2,Q2,O2> > : public
x_traits<ap_fixed<W1,W1>, ap_ufixed<W2,I2> > {};
802
803 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
804 struct x_traits<ap_uint<W1>, ap_fixed<W2,I2,Q2,O2> > : public
x_traits<ap_ufixed<W1,W1>, ap_fixed<W2,I2> > {};
805
806 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
807 struct x_traits<ap_uint<W1>, ap_ufixed<W2,I2,Q2,O2> > : pub-
lic x_traits<ap_ufixed<W1,W1>, ap_ufixed<W2,I2> > {};
808
809 // fixed_type<X>::T is the smallest ap_fixed/ap_ufixed type
that can losslessly contain X
810 // e.g. X x; fixed_type<X>::T y(x); is sensible;
811
812 template<typename T> struct fixed_type { };
813 template<> struct fixed_type<char> { typedef
ap_fixed<8,8> T; };
814 template<> struct fixed_type<unsigned char> { typedef
ap_ufixed<8,8> T; };
815 template<> struct fixed_type<short> { typedef
ap_fixed<16,16> T; };
816 template<> struct fixed_type<unsigned short> { typedef
ap_ufixed<16,16> T; };
817 template<> struct fixed_type<int> { typedef
ap_fixed<32,32> T; };
818 template<> struct fixed_type<unsigned int> { typedef
ap_ufixed<32,32> T; };
819 template<> struct fixed_type<float> { typedef
ap_ufixed<64,32> T; };
820 template<int W> struct fixed_type<ap_int<W> > { typedef
ap_fixed<W,W> T; };
821 template<int W> struct fixed_type<ap_uint<W> > { typedef
ap_ufixed<W,W> T; };
822 template<int W, int I, ap_q_mode _AP_Q, ap_o_mode _AP_O>
struct fixed_type<ap_fixed<W,I, _AP_Q, _AP_O> > { typedef
ap_fixed<W,I> T; };
823 template<int W, int I, ap_q_mode _AP_Q, ap_o_mode _AP_O>
struct fixed_type<ap_ufixed<W,I, _AP_Q, _AP_O> > { typedef
ap_ufixed<W,I> T; };
824
825 template<typename T1, int DIM>
826 struct x_traits_d_default {
827     typedef T1 ACCUM_T;
828     typedef T1 ACCUM2_T;
829     typedef T1 ADD_TREE_T;
830 };
831
832 template<typename T1, int DIM>
833 struct x_traits_d {};
834
835 template<int DIM>
836 struct x_traits_d <int,DIM>:x_traits_d_default<int, DIM> {};
837
838 template<int DIM>
839 struct x_traits_d <float,DIM>:x_traits_d_default<float, DIM>
{};
840
841 template<int DIM>
842 struct x_traits_d <double,DIM>:x_traits_d_default<double,
DIM> {};
843
844 template<int DIM>
845 struct x_traits_d <half,DIM>:x_traits_d_default<half, DIM>
{};
846
847 template<int W, int DIM>
848 struct x_traits_d <ap_int<W>,DIM> {
849     typedef ap_int<W> ACCUM_T;
850     typedef ap_int<2*W> ACCUM2_T;
851     typedef ap_int<25+W> ADD_TREE_T;
852 };
853
854 template<int W, int DIM>
855 struct x_traits_d <ap_uint<W>,DIM> {
856     typedef ap_uint<W> ACCUM_T;
857     typedef ap_uint<2*W> ACCUM2_T;
858     typedef ap_uint<25+W> ADD_TREE_T;
859 };
860

```

```

861 template<int W, int I, ap_q_mode Q, ap_o_mode O, int DIM>
862 struct x_traits_d <ap_fixed<W,I,Q,O>,DIM> {
863     typedef
ap_fixed<W+BitWidth<DIM>::Value,I+BitWidth<DIM>::Value,Q,O>
ACCUM_T;
864     typedef ap_fixed<(2*W)+BitWidth<DIM>::Value,(2*I)
+BitWidth<DIM>::Value,Q,O> ACCUM2_T;
865     typedef ap_fixed<25+W+BitWidth<DIM>::Value,
25+I+BitWidth<DIM>::Value,Q,O> ADD_TREE_T; // TODO placeholder
866 };
867
868 template<int W, int I, ap_q_mode Q, ap_o_mode O, int DIM>
869 struct x_traits_d <ap_ufixed<W,I,Q,O>,DIM> {
870     typedef
ap_ufixed<W+BitWidth<DIM>::Value,I+BitWidth<DIM>::Value,Q,O>
ACCUM_T;
871     typedef ap_ufixed<(2*W)+BitWidth<DIM>::Value,(2*I)
+BitWidth<DIM>::Value,Q,O> ACCUM2_T;
872     typedef ap_ufixed<25+W+BitWidth<DIM>::Value,
25+I+BitWidth<DIM>::Value,Q,O> ADD_TREE_T; // TODO placeholder
873 };
874
875 };
876
877
878
879 namespace hlstmp
880 {
881
882 template<typename T>
883 struct x_traits_default {
884     typedef T ADD_T;
885     typedef T SUB_T;
886     typedef T SAT_T;
887     typedef T MULT_T;
888     typedef T MADD_T;
889 };
890
891 template<typename T1, typename T2>
892 struct x_traits {
893     // typedef T1 ADD_T;
894     // typedef T1 SAT_T;
895     // typedef T1 MULT_T;
896     // typedef T1 MADD_T;
897     // typedef T1 CMADD_T;
898     // typedef T1 CONJ_SQ_T;
899     // typedef T1 CONJ_SQ_2_T;
900     // typedef int INT_T;
901     // typedef T1 INV_T;
902     // typedef float_struct<24,8> FLOAT_STRUCT_T;
903
904 };
905
906 template<>
907 struct x_traits<double, double> : public
x_traits_default<double> { };
908
909 template<typename T2>
910 struct x_traits<double, T2> : public x_traits_default<double>
{ };
911
912 template<typename T1>
913 struct x_traits<T1, double> : public x_traits_default<double>
{ };
914
915 template<>
916 struct x_traits<float, float> : public
x_traits_default<float> { };
917
918 template<typename T2>
919 struct x_traits<float, T2> : public x_traits_default<float>
{ };
920
921 template<typename T1>
922 struct x_traits<T1, float> : public x_traits_default<float>
{ };
923
924 template<>
925 struct x_traits<half, half> : public x_traits_default<half>
{ };
926
927 template<typename T2>
928 struct x_traits<half, T2> : public x_traits_default<float>
{ };
929
930 template<typename T1>
931 struct x_traits<T1, half> : public x_traits_default<float>
{ };
932
933 template<>
934 struct x_traits<double, float> : public
x_traits_default<double> { };
935
936 template<>
937 struct x_traits<float, double> : public
x_traits_default<double> { };
938
939 template<>

```

```

940 struct x_traits<double, half> : public
x_traits_default<double> { };
941
942 template<>
943 struct x_traits<half, double> : public
x_traits_default<double> { };
944
945 template<>
946 struct x_traits<int, int> : public x_traits_default<int> { };
947
948 template<>
949 struct x_traits<unsigned int, unsigned int> : public
x_traits_default<unsigned int> { };
950
951 template<>
952 struct x_traits<int64_t, int64_t> : public
x_traits_default<int64_t> { };
953
954 template<int A, int B> class MAX{
955     public:
956         const static int value = (A > B) ? A : B;
957 };
958
959 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
960 struct x_traits<ap_fixed<W1, I1, Q1, O1>, ap_fixed<W2, I2, Q2, O2> > {
961     typedef typename ap_fixed<W1, I1>::template
RTYPE<W2, I2, true> RTYPE;
962     typedef ap_fixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
963     typedef ap_fixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
964     typedef ap_fixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
965     typedef ap_fixed<W1, I1, AP_RND, AP_SAT> SAT_T;
966     typedef ap_fixed<W1+W2+1, I1+I2+1>
MADD_T;
967     typedef ap_fixed<W1+W2+2, I1+I2+2>
CMADD_T;
968     typedef ap_fixed<(2*W1)+1, (2*I1)>
CONJ_SQ_T;
969     typedef ap_fixed<(2*W1)+2, (2*I1)+1> CONJ_SQ_2_T;
970     typedef ap_fixed<2*(MAX<I1, W1-I1>::value+1), (MAX<I1, W1-
I1>::value+1)> INV_T;
971     typedef int INT_T;
972     typedef float_struct<24, 8> FLOAT_STRUCT_T;
973 };
974
975 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
976 struct x_traits<ap_fixed<W1, I1, Q1, O1>, ap_ufixed<W2, I2, Q2, O2>
> {
977     typedef typename ap_fixed<W1, I1>::template
RTYPE<W2, I2, false> RTYPE;
978     typedef ap_fixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
979     typedef ap_fixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
980     typedef ap_fixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
981 };
982
983 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
984 struct x_traits<ap_ufixed<W1, I1, Q1, O1>, ap_fixed<W2, I2, Q2, O2>
> {
985     typedef typename ap_ufixed<W1, I1>::template
RTYPE<W2, I2, true> RTYPE;
986     typedef ap_fixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
987     typedef ap_fixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
988     typedef ap_fixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
989 };
990
991 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2,
int I2, ap_q_mode Q2, ap_o_mode O2>
992 struct x_traits<ap_ufixed<W1, I1, Q1, O1>, ap_ufixed<W2, I2, Q2, O2> >
{
993     typedef typename ap_ufixed<W1, I1>::template
RTYPE<W2, I2, false> RTYPE;
994     typedef ap_ufixed<RTYPE::plus_w, RTYPE::plus_i> ADD_T;
995     typedef ap_ufixed<RTYPE::minus_w, RTYPE::minus_i> SUB_T;
996     typedef ap_ufixed<RTYPE::mult_w, RTYPE::mult_i> MULT_T;
997     typedef ap_ufixed<W1, I1, AP_RND, AP_SAT> SAT_T;
998     typedef ap_ufixed<W1+W2+1, I1+I2+1>
MADD_T;
999     typedef ap_ufixed<W1+W2+2, I1+I2+2>
CMADD_T;
1000     typedef ap_ufixed<(2*W1)+1, (2*I1)>
CONJ_SQ_T;
1001     typedef ap_ufixed<(2*W1)+2, (2*I1)+1> CONJ_SQ_2_T;
1002     typedef ap_ufixed<2*(MAX<I1, W1-I1>::value+1), (MAX<I1, W1-
I1>::value+1)> INV_T;
1003     typedef int INT_T;
1004     typedef float_struct<24, 8> FLOAT_STRUCT_T;
1005 };
1006
1007 template<int W1, int W2>
1008 struct x_traits<ap_int<W1>, ap_int<W2> > {
1009     typedef ap_int<MAX<W1, W2>::value + 1>
ADD_T;
1010     typedef ap_int<MAX<W1, W2>::value + 1>
SUB_T;

```

```

1011     typedef ap_int<W1+W2>          MULT_T;
1012     typedef ap_int<(W1 > W2)? W1:W2> MADD_T;
1013 };
1014
1015 template<int W1, int W2>
1016 struct x_traits<ap_int<W1>, ap_uint<W2> > {
1017
1018     typedef ap_int<MAX<W1,W2+1>::value + 1>
ADD_T;
1019
1020     typedef ap_int<MAX<W1,W2+1>::value + 1>
SUB_T;
1021
1022     typedef ap_int<W1+W2>          MULT_T;
1023 };
1024
1025 template<int W1, int W2>
1026 struct x_traits<ap_uint<W1>, ap_int<W2> > {
1027
1028     typedef ap_int<MAX<W1+1,W2>::value + 1>
ADD_T;
1029
1030     typedef ap_int<MAX<W1+1,W2>::value + 1>
SUB_T;
1031
1032     typedef ap_uint<W1+W2>          MULT_T;
1033 };
1034
1035 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
1036 struct x_traits<ap_fixed<W1,I1,Q1,O1>, ap_int<W2> > : public
x_traits<ap_fixed<W1,I1>, ap_fixed<W2,W2> > {};
1037
1038 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
1039 struct x_traits<ap_fixed<W1,I1,Q1,O1>, ap_uint<W2> > : public
x_traits<ap_fixed<W1,I1>, ap_ufixed<W2,W2> > {};
1040
1041 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
1042 struct x_traits<ap_ufixed<W1,I1,Q1,O1>, ap_int<W2> > : public
x_traits<ap_ufixed<W1,I1>, ap_fixed<W2,W2> > {};
1043
1044 template<int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int W2>
1045 struct x_traits<ap_ufixed<W1,I1,Q1,O1>, ap_uint<W2> > : public
x_traits<ap_ufixed<W1,I1>, ap_ufixed<W2,W2> > {};
1046
1047 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
1048 struct x_traits<ap_int<W1>, ap_fixed<W2,I2,Q2,O2> > : public
x_traits<ap_fixed<W1,W1>, ap_fixed<W2,I2> > {};
1049
1050 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
1051 struct x_traits<ap_int<W1>, ap_ufixed<W2,I2,Q2,O2> > : public
x_traits<ap_fixed<W1,W1>, ap_ufixed<W2,I2> > {};
1052
1053 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
1054 struct x_traits<ap_uint<W1>, ap_fixed<W2,I2,Q2,O2> > : public
x_traits<ap_ufixed<W1,W1>, ap_fixed<W2,I2> > {};
1055
1056 template<int W1, int W2, int I2, ap_q_mode Q2, ap_o_mode O2>
1057 struct x_traits<ap_uint<W1>, ap_ufixed<W2,I2,Q2,O2> > : public
x_traits<ap_ufixed<W1,W1>, ap_ufixed<W2,I2> > {};
1058
1059 // fixed_type<X>::T is the smallest ap_fixed/ap_ufixed type
that can losslessly contain X
1060 // e.g. X x; fixed_type<X>::T y(x); is sensible;
1061
1062 template<typename T> struct fixed_type {};
1063
1064 template<> struct fixed_type<char> { typedef
ap_fixed<8,8> T; };
1065
1066 template<> struct fixed_type<unsigned char> { typedef
ap_ufixed<8,8> T; };
1067
1068 template<> struct fixed_type<short> { typedef
ap_fixed<16,16> T; };
1069
1070 template<> struct fixed_type<unsigned short> { typedef
ap_ufixed<16,16> T; };
1071
1072 template<> struct fixed_type<int> { typedef
ap_fixed<32,32> T; };
1073
1074 template<> struct fixed_type<unsigned int> { typedef
ap_ufixed<32,32> T; };
1075
1076 template<> struct fixed_type<float> { typedef
ap_ufixed<64,32> T; };
1077
1078 template<int W> struct fixed_type<ap_int<W> > { typedef
ap_fixed<W,W> T; };
1079
1080 template<int W> struct fixed_type<ap_uint<W> > { typedef
ap_ufixed<W,W> T; };
1081
1082 template<int W, int I, ap_q_mode _AP_Q, ap_o_mode _AP_O>
1083 struct fixed_type<ap_fixed<W,I,_AP_Q,_AP_O> > { typedef
ap_fixed<W,I> T; };
1084
1085 template<int W, int I, ap_q_mode _AP_Q, ap_o_mode _AP_O>
1086 struct fixed_type<ap_ufixed<W,I,_AP_Q,_AP_O> > { typedef

```

```

ap_ufixed<W,I> T; };
1075
1076 template<typename T1, int DIM>
1077 struct x_traits_d_default {
1078     typedef T1 ACCUM_T;
1079     typedef T1 ACCUM2_T;
1080     typedef T1 ADD_TREE_T;
1081 };
1082
1083 template<typename T1, int DIM>
1084 struct x_traits_d {};
1085
1086 template<int DIM>
1087 struct x_traits_d <int,DIM>:x_traits_d_default<int, DIM> {};
1088
1089 template<int DIM>
1090 struct x_traits_d <float,DIM>:x_traits_d_default<float, DIM>
{};
1091
1092 template<int DIM>
1093 struct x_traits_d <double,DIM>:x_traits_d_default<double,
DIM> {};
1094
1095 template<int DIM>
1096 struct x_traits_d <half,DIM>:x_traits_d_default<half, DIM>
{};
1097
1098 template<int W, int DIM>
1099 struct x_traits_d <ap_int<W>,DIM> {
1100     typedef ap_int<W> ACCUM_T;
1101     typedef ap_int<2*W> ACCUM2_T;
1102     typedef ap_int<25+W> ADD_TREE_T;
1103 };
1104
1105 template<int W, int DIM>
1106 struct x_traits_d <ap_uint<W>,DIM> {
1107     typedef ap_uint<W> ACCUM_T;
1108     typedef ap_uint<2*W> ACCUM2_T;
1109     typedef ap_uint<25+W> ADD_TREE_T;
1110 };
1111
1112 template<int W, int I, ap_q_mode Q, ap_o_mode O, int DIM>
1113 struct x_traits_d <ap_fixed<W,I,Q,O>,DIM> {
1114     typedef
ap_fixed<W+BitWidth<DIM>::Value,I+BitWidth<DIM>::Value,Q,O>
ACCUM_T;
1115     typedef ap_fixed<(2*W)+BitWidth<DIM>::Value,(2*I)
+BitWidth<DIM>::Value,Q,O> ACCUM2_T;
1116     typedef ap_fixed<25+W+BitWidth<DIM>::Value,
25+I+BitWidth<DIM>::Value,Q,O> ADD_TREE_T; // TODO placeholder
1117 };
1118
1119 template<int W, int I, ap_q_mode Q, ap_o_mode O, int DIM>
1120 struct x_traits_d <ap_ufixed<W,I,Q,O>,DIM> {
1121     typedef
ap_ufixed<W+BitWidth<DIM>::Value,I+BitWidth<DIM>::Value,Q,O>
ACCUM_T;
1122     typedef ap_ufixed<(2*W)+BitWidth<DIM>::Value,(2*I)
+BitWidth<DIM>::Value,Q,O> ACCUM2_T;
1123     typedef ap_ufixed<25+W+BitWidth<DIM>::Value,
25+I+BitWidth<DIM>::Value,Q,O> ADD_TREE_T; // TODO placeholder
1124 };
1125
1126 }
1127
1128 #endif
1129
1130
1131
1132

```

E.51 vendor/utils/x_hls_utils.h (684 lines, 17,745 B)

```

1 // (C) Copyright 2016-2021 Xilinx, Inc.
2 // All Rights Reserved.
3 //
4 // Licensed to the Apache Software Foundation (ASF) under one
5 // or more contributor license agreements. See the NOTICE
file
6 // distributed with this work for additional information
7 // regarding copyright ownership. The ASF licenses this file
8 // to you under the Apache License, Version 2.0 (the
9 // "License"); you may not use this file except in compliance
10 // with the License. You may obtain a copy of the License at
11 //
12 // http://www.apache.org/licenses/LICENSE-2.0
13 //
14 // Unless required by applicable law or agreed to in writing,
15 // software distributed under the License is distributed on
an
16 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
17 // KIND, either express or implied. See the License for the
18 // specific language governing permissions and limitations
19 // under the License.
20
21 // (C) Copyright 2016-2021 Xilinx, Inc.
22 // All Rights Reserved.
23 //
24 // Licensed to the Apache Software Foundation (ASF) under one
25 // or more contributor license agreements. See the NOTICE

```

```

file
26 // distributed with this work for additional information
27 // regarding copyright ownership. The ASF licenses this file
28 // to you under the Apache License, Version 2.0 (the
29 // "License"); you may not use this file except in compliance
30 // with the License. You may obtain a copy of the License at
31 //
32 // http://www.apache.org/licenses/LICENSE-2.0
33 //
34 // Unless required by applicable law or agreed to in writing,
35 // software distributed under the License is distributed on
an
36 // "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
37 // KIND, either express or implied. See the License for the
38 // specific language governing permissions and limitations
39 // under the License.
40
41
42 #ifndef X_HLS_UTILS_H
43 #define X_HLS_UTILS_H
44 #include <stdint.h>
45 #include "ap_fixed.h"
46 // #include "hls_half.h"
47 #include <limits>
48 #ifndef __SYNTHESIS__
49 #include <iostream>
50 #include <cmath>
51 #endif
52 class half;
53 /*
54
*****
55 * HLS utilities
56 * -----
57 *
58 * General utilities usable by all functions. Mainly contains
classes to
59 * generate static values from recursive class declarations.
60 *
61 * pow2
62 * sqrtf - C declaration
63 * to_float
64 *
65 * fp_struct<float>
66 * fp_struct<double>
67 * fp_struct<half>
68 * dumpDouble
69 *
70 * Power - B^N
71 * BitWidth = calculate minimum # of bits to store given in-
teger value
72 *
73 * Table
74 *
75
*****
76 */
77
78 namespace hls {
79
80 template<typename T>
81 class numeric_limits {
82 public:
83     static T max();
84     static T min();
85     static T epsilon();
86 };
87
88 template<int W, int I, ap_q_mode Q, ap_o_mode O>
89 class numeric_limits<ap_fixed<W,I,Q,O>> {
90 public:
91     static ap_fixed<W,I,Q,O> max();
92     static ap_fixed<W,I,Q,O> min();
93     static ap_fixed<W,I,Q,O> epsilon();
94 };
95
96 template<int W, int I, ap_q_mode Q, ap_o_mode O>
97 class numeric_limits<ap_ufixed<W,I,Q,O>> {
98 public:
99     static ap_ufixed<W,I,Q,O> max();
100     static ap_ufixed<W,I,Q,O> min();
101     static ap_ufixed<W,I,Q,O> epsilon();
102 };
103
104 template<int W>
105 class numeric_limits<ap_int<W>> {
106 public:
107     static ap_int<W> max();
108     static ap_int<W> min();
109     static ap_int<W> epsilon();
110 };
111
112 template<int W>
113 class numeric_limits<ap_uint<W>> {
114 public:
115     static ap_uint<W> max();
116     static ap_uint<W> min();
117     static ap_uint<W> epsilon();
118 };
119 }
120
121 namespace hlstm {
122
123 template<typename T>
124 class numeric_limits {
125 public:
126     static T max();
127     static T min();
128     static T epsilon();
129 };
130
131 template<int W, int I, ap_q_mode Q, ap_o_mode O>
132 class numeric_limits<ap_fixed<W,I,Q,O>> {
133 public:
134     static ap_fixed<W,I,Q,O> max();
135     static ap_fixed<W,I,Q,O> min();
136     static ap_fixed<W,I,Q,O> epsilon();
137 };
138
139 template<int W, int I, ap_q_mode Q, ap_o_mode O>
140 class numeric_limits<ap_ufixed<W,I,Q,O>> {
141 public:
142     static ap_ufixed<W,I,Q,O> max();
143     static ap_ufixed<W,I,Q,O> min();
144     static ap_ufixed<W,I,Q,O> epsilon();
145 };
146
147 template<int W>
148 class numeric_limits<ap_int<W>> {
149 public:
150     static ap_int<W> max();
151     static ap_int<W> min();
152     static ap_int<W> epsilon();
153 };
154
155 template<int W>
156 class numeric_limits<ap_uint<W>> {
157 public:
158     static ap_uint<W> max();
159     static ap_uint<W> min();
160     static ap_uint<W> epsilon();
161 };
162
163 }
164
165 static inline
166 const
167 uint32_t pow2(uint32_t e)
168 {
169     ;
170 }
171
172 template<class T>
173 T reg(T in)
174 {
175     /*
176
-----
177 * References C square root function for single precision
square root.
178 * Maps to FPU sqrt core
179
-----
180 */
181 // extern "C" float sqrtf(float);
182
183 /*
184
-----
185 * Converts data to float type
186
-----
187 */
188 static inline
189 float to_float(float v)
190 {
191     return v;
192 }
193
194 template<int _W, int _I>
195 float to_float(ap_fixed<_W, _I> v)
196 {
197     return v.to_float();
198 }
199
200
201 /*
202
-----
203 * Single-precision floating point union to convert between
32-bit int and float
204
-----

```



```

-----
205 */
206
207 template <typename T>
208 class fp_struct
209 {
210 };
211
212 union single_cast {
213     float f;
214     uint32_t i;
215 };
216
217 template <>
218 class fp_struct<float>
219 {
220 public:
221     const static int EXP_INFNaN = 255;
222     const static int EXP_BIAS = 127;
223     const static int EXP_BITS = 8;
224     const static int SIG_BITS = 23;
225     const static int BITS = 32;
226
227     fp_struct() {
228     }
229     fp_struct(float f);
230     fp_struct(ap_uint<32> data);
231     fp_struct(uint32_t i);
232     inline ap_uint<32> data() const;
233     inline int expv() const;
234     inline int32_t to_int() const;
235     inline float to_float() const;
236     inline void set_mantissa(ap_ufixed<1+SIG_BITS,1> man-
tissa);
237     inline ap_ufixed<1+SIG_BITS,1> mantissa() const;
238     inline float to_ieee() const;
239     inline int __signbit() const;
240
241     static float infinity();
242
243     static float minus_infinity();
244
245     typedef uint32_t inttype;
246     typedef ap_uint<32> data_type;
247     ap_uint<1> sign;
248     ap_uint<EXP_BITS> exp;
249     ap_uint<SIG_BITS> sig;
250 };
251
252 #ifndef __SYNTHESIS__
253 static inline
254 std::ostream &operator << (std::ostream &os, const
fp_struct<float> &s)
255 ;
256 #endif
257
258 static inline
259 void
260 castSingle(
261     float din,
262     fp_struct<float> &dout)
263 ;
264
265 static inline
266 float
267 castSingle(
268     fp_struct<float> din)
269 ;
270
271 static inline
272 void
273 dumpSingle(
274     float da,
275     fp_struct<float> ds)
276 ;
277
278
279 /*
280
-----
281 * Double-precision floating point union to convert between
64-bit int and double
282
-----
283 */
284 union double_cast {
285     double d;
286     uint64_t i;
287 };
288
289 template <>
290 class fp_struct<double>
291 {
292 public:
293     const static int EXP_INFNaN = 2047;
294     const static int EXP_BIAS = 1023;
295     const static int EXP_BITS = 11;
296     const static int SIG_BITS = 52;

```

```

297     const static int BITS = 64;
298
299     fp_struct() {
300     }
301     fp_struct(double f);
302     fp_struct(ap_uint<64> data);
303     fp_struct(uint64_t i);
304     inline ap_uint<64> data() const;
305     inline int64_t to_int() const;
306     inline int expv() const;
307     inline ap_uint<20> sig_msb() const;
308     inline ap_uint<32> sig_lsb() const;
309     inline double to_double() const;
310     inline void set_mantissa(ap_ufixed<1+SIG_BITS,1> man-
tissa);
311     inline ap_ufixed<1+SIG_BITS,1> mantissa() const;
312     inline double to_ieee() const;
313     inline int __signbit() const;
314
315     static double infinity();
316
317     static double minus_infinity();
318
319     typedef uint64_t inttype;
320     typedef ap_uint<64> data_type;
321     ap_uint<1> sign;
322     ap_uint<EXP_BITS> exp;
323     ap_uint<SIG_BITS> sig;
324 };
325
326 #ifndef __SYNTHESIS__
327 static inline
328 std::ostream &operator << (std::ostream &os, const
fp_struct<double> &s)
329 ;
330 #endif
331
332 static inline
333 void
334 castDouble(
335     double din,
336     fp_struct<double> &dout)
337 ;
338
339 static inline
340 double
341 castDouble(
342     fp_struct<double> din)
343 ;
344
345 static inline
346 void
347 dumpDouble(
348     double da,
349     fp_struct<double> ds)
350 ;
351
352 /*
353
-----
354 * Half-precision floating point union to convert between 16-
bit int and half
355
-----
356 */
357 #define HAS_NO_HALF_TYPE
358 #ifndef HAS_NO_HALF_TYPE
359 #ifndef __SYNTHESIS__
360 union half_cast {
361     half d;
362     uint16_t i;
363 };
364 #endif
365
366 template <>
367 class fp_struct<half>
368 {
369 public:
370     const static int EXP_INFNaN = 31;
371     const static int EXP_BIAS = 15;
372     const static int EXP_BITS = 5;
373     const static int SIG_BITS = 10;
374     const static int BITS = 16;
375
376     fp_struct();
377     fp_struct(half f);
378     fp_struct(ap_uint<16> data);
379     fp_struct(uint16_t i);
380     inline ap_uint<16> data() const;
381     inline int expv() const;
382     inline int16_t to_int() const;
383     inline half to_half() const;
384     inline void set_mantissa(ap_ufixed<1+SIG_BITS,1> man-
tissa);
385     inline ap_ufixed<1+SIG_BITS,1> mantissa() const;
386     inline half to_ieee() const;
387     inline int __signbit() const;
388

```

```

389     static half infinity();
390
391     static half minus_infinity();
392
393     typedef uint16_t inttype;
394     typedef ap_uint<16> data_type;
395     ap_uint<1> sign;
396     ap_uint<EXP_BITS> exp;
397     ap_uint<SIG_BITS> sig;
398 };
399
400 #ifndef __SYNTHESIS__
401 static inline
402 std::ostream &operator << (std::ostream &os, const
fp_struct<half> &s)
403 ;
404 #endif
405
406 static inline
407 void
408 castHalf(
409     half din,
410     fp_struct<half> &dout)
411 ;
412
413 static inline
414 half
415 castHalf(
416     fp_struct<half> din)
417 ;
418
419 static inline
420 void
421 dumpHalf(
422     half da,
423     fp_struct<half> ds)
424 ;
425 #endif
426
427 /*
428
429
430 * -----
431 * Raise a value to a certain power (B^N).
432 * This class is necessary in template arguments due to its
static const nature
433 * and the fact that it performs the power operation as part
of the recursive
434 * class declaration. A standard recursive power function
would not return
435 * a static const value.
436 *
437 * -----
438 */
439 template < unsigned int _Base, unsigned int _Num >
440 class Power
441 {
442 public:
443     static const unsigned int Value = _Base * Power< _Base,
_Num - 1 >::Value;
444 };
445
446 template < unsigned int _Base >
447 class Power< _Base, 0 >
448 {
449 public:
450     static const unsigned int Value = 1;
451 };
452 /*
453
454 * -----
455 * Calculate the minimum # of bits to store the given integer
value.
456 * UnsignedBitWidth<2>::Value == 3
457 * UnsignedBitWidth<3>::Value == 3
458 * UnsignedBitWidth<4>::Value == 4
459
460 * -----
461 */
462 template < unsigned int _Num, unsigned int _I=_Num/2>
463 class BitWidth
464 {
465 public:
466     static const unsigned int Value = 1 + BitWidth<_Num,_I/
2>::Value;
467 };
468
469 template < unsigned int _Num>
470 class BitWidth<_Num, 0>
471 {
472 public:
473     static const unsigned int Value = 1;
474 };

```

```

474
475 /*
476
477 * -----
478 * Calculate the minimum # of bits to store the given unsigned
integer value.
479 * UnsignedBitWidth<2>::Value == 2
480 * UnsignedBitWidth<3>::Value == 2
481 * UnsignedBitWidth<4>::Value == 3
482
483 * -----
484 */
485 template < unsigned int _Num, unsigned int _I=_Num/2>
486 class UnsignedBitWidth
487 {
488 public:
489     static const unsigned int Value = 1 + Unsigned-
BitWidth<_Num,_I/2>::Value;
490 };
491
492 template < unsigned int _Num>
493 class UnsignedBitWidth<_Num, 0>
494 {
495 public:
496     static const unsigned int Value = 0;
497 };
498 /*
499
500 * -----
501 * return num of bits of a specific type
502 * This class returns the bitwidth result of a specific type
503
504 * -----
505 */
506 template < typename T >
507 class Type_BitWidth
508 {
509 public:
510     static const unsigned int Value = 8*sizeof(T);
511 };
512
513 template < int W >
514 class Type_BitWidth< ap_uint<W> >
515 {
516 public:
517     static const unsigned int Value = W;
518 };
519
520 template < int W >
521 class Type_BitWidth< ap_int<W> >
522 {
523 public:
524     static const unsigned int Value = W;
525 };
526
527 template < int W, int I >
528 class Type_BitWidth< ap_ufixed<W, I> >
529 {
530 public:
531     static const unsigned int Value = W;
532 };
533
534 template < int W, int I >
535 class Type_BitWidth< ap_fixed<W, I> >
536 {
537 public:
538     static const unsigned int Value = W;
539 };
540 /*
541
542 * -----
543 * Table class
544 * T: table data type
545 * N: Depth of table entries
546 * public:
547 * dummy Table data type
548 * array[N] Table array
549
550 * -----
551 */
552 template < typename _T, int _Num, int _I=_Num-1>
553 class Table : public Table<_T, _Num, _I-1>
554 {
555 public:
556     typedef typename _T::TableType TableType;
557     static const typename _T::TableType dummy;
558     static const int size = _Num;

```



```

559
560 };
561
562 template <typename _T, int _Num>
563 class Table<_T, _Num, 0>
564 {
565 public:
566     static const typename _T::TableType dummy;
567     static typename _T::TableType array[_Num];
568 };
569
570 /*
571 * Declares the templatized and static Table dummy variable
572 *
573 * Assign the global static const Table::dummy variable to the
574 * associated Table entry by invoking T::apply, assigning it
575 * to array[I]
576 */
577
578 * and adding smaller templates of Table::dummy to it, thereby
579 recursively
580 * assigning the smaller templates for Tables..
581 */
582
583 template <typename _T, int _Num, int _I>
584 const typename _T::TableType Table<_T, _Num, _I>::dummy
585 = Table<_T, _Num, 0>::array[_I] = _T::apply(_I, _Num) +
586 0*Table<_T, _Num, _I-1>::dummy;
587
588 template <typename _T, int _Num>
589 const typename _T::TableType Table<_T, _Num, 0>::dummy
590 = Table<_T, _Num, 0>::array[0] = _T::apply(0, _Num);
591
592 /*
593 * Declares the templatized and static Table array
594 */
595
596 template <typename _T, int _Num>
597 typename _T::TableType Table<_T, _Num, 0>::array[_Num];
598
599 // Some standard junk... this is in the new standard and in
600 boost,
601 // but for a few definitions it's not worth the dependency
602
603 template <class T>
604 struct is_fptype { static const bool value = false; };
605
606 template <> struct is_fptype<float> { static const bool value =
607 true; };
608
609 template <> struct is_fptype<double> { static const bool
610 value = true; };
611
612 template <> struct is_fptype<half> { static const bool value
613 = true; };
614
615 template <class T>
616 struct is_integralttype { static const bool value = false; };
617
618 template <> struct is_integralttype<int> { static const bool
619 value = true; };
620
621 template <> struct is_integralttype<unsigned int> { static
622 const bool value = true; };
623
624 template <> struct is_integralttype<char> { static const bool
625 value = true; };
626
627 template <> struct is_integralttype<signed char> { static
628 const bool value = true; };
629
630 template <> struct is_integralttype<unsigned char> { static
631 const bool value = true; };
632
633 template <> struct is_integralttype<short> { static const bool
634 value = true; };
635
636 template <> struct is_integralttype<unsigned short> { static
637 const bool value = true; };
638
639 template <> struct is_integralttype<long> { static const bool
640 value = true; };
641
642 template <> struct is_integralttype<unsigned long> { static
643 const bool value = true; };
644
645 template <> struct is_integralttype<long long> { static const
646 bool value = true; };
647
648 template <> struct is_integralttype<unsigned long long> {
649 static const bool value = true; };
650
651 template <int W> struct is_integralttype<ap_int<W>> { static
652 const bool value = true; };

```

```

615 template <int W> struct is_integralttype<ap_uint<W>> { static
616 const bool value = true; };
617
618 template <class T>
619 struct is_fixedtype { static const bool value = false; };
620
621 template <int W, int I, ap_q_mode Q, ap_o_mode O> struct
622 is_fixedtype<ap_fixed<W,I,Q,O>> { static const bool value =
623 true; };
624
625 template <int W, int I, ap_q_mode Q, ap_o_mode O> struct
626 is_fixedtype<ap_ufixed<W,I,Q,O>> { static const bool value =
627 true; };
628
629 namespace hls {
630
631 template<bool B, class T = void>
632 struct enable_if {};
633
634 template<class T>
635 struct enable_if<true, T> { typedef T type; };
636
637 template<typename T, T _v>
638 struct integral_constant
639 {
640     static const T value = _v;
641     typedef T value_type;
642     typedef integral_constant<T, _v> type;
643     operator value_type() { return value; }
644 };
645
646 typedef integral_constant<bool, true> true_type;
647 typedef integral_constant<bool, false> false_type;
648
649 template<typename T1, typename T2>
650 struct is_same;
651
652 template<typename T1, typename T2>
653 struct is_same : public false_type { };
654
655 template<typename T1>
656 struct is_same<T1, T1> : public true_type { };
657
658 template<typename T>
659 struct is_arithmetic : public integral_constant<bool,
660 (is_integralttype<T>::value || is_fptype<T>::value)> { };
661
662 template<typename T1, typename T2>
663 struct enable_or { static const bool value = T1::value ||
664 T2::value; };
665
666 template<typename T1, typename T2>
667 struct enable_and { static const bool value = T1::value &&
668 T2::value; };
669
670 template<typename T, bool = is_integralttype<T>::value>
671 struct __promote { typedef double type; };
672
673 template<typename T>
674 struct __promote<T, false> { };
675
676 template<>
677 struct __promote<double> { typedef double type; };
678
679 template<>
680 struct __promote<float> { typedef float type; };
681
682 template<>
683 struct __promote<half> { typedef half type; };
684
685 //End standard junk
686
687 #endif

```